

Kinect and Natural User Interface

[See Also](#)



Natural User Interface (NUI) is the API for the Kinect Sensor for Xbox 360.

In This Section

[NUI Overview](#)

Overview material about the NUI API used for Kinect.

[NUI Reference](#)

Contains reference information for Natural User Interface (NUI), which is used for Kinect.

[NUI Samples](#)

Demonstrate using the Natural User Interface (NUI) APIs, for Kinect titles.

[NUI Tools](#)

Tools for developing Kinect titles.

[NUI White Papers](#)

Contains white papers related to developing with Natural User Interface (NUI) software and hardware.

See Also

[Input Technologies and Devices](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface Overviews



Overview material about Natural User Interface (NUI), which is the API for the Kinect Sensor for Xbox 360.

In This Section

[Natural User Interface: Getting Started](#)

Preliminary documentation for Natural User Interface development.

[Natural User Interface: Programming Guide](#)

Using the Kinect hardware and software to develop games for the Xbox 360 console.

Related Topics

[XDK Unlocked - Videos](#)

[NUI Reference](#)

[NUI Samples](#)

[NUI Tools](#)

[NUI White Papers](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface: Getting Started



This section covers preliminary information for Natural User Interface (NUI) programming for the Kinect Sensor.

In This Section

[Specifications of the Kinect Sensor](#)

Specifications for the Kinect Sensor.

[Starting Up the Kinect Sensor](#)

Connecting, calibrating, and testing the Kinect Sensor.

[Troubleshooting the Kinect Sensor](#)

Troubleshooting tips for setting up and operating your Kinect Sensor.

[Quick-Start Tutorial for Kinect Programming](#)

Adds Kinect functionality to the Dolphin sample, to control swimming speed and direction through arm elevation.

[Adding NUI Header and Library Files](#)

Adding NUI support to an existing Xbox 360 project, including header files, library files, and database files.

[Kinect Database Files to Include on the Game Disc](#)

Kinect database files to include on the game disc.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Specifications of the Kinect Sensor



Figure 1. Kinect Sensor



The infrared laser and the CMOS depth sensor work in conjunction to produce the *depth image stream*.
The color sensor produces the *color image stream*.
The depth image stream and color image stream are used to produce the *skeleton tracking pipeline*.
The microphone array produces the *audio stream*, which is used for speech recognition and to detect the horizontal location of the player who is speaking.
The Kinect Sensor has the following specifications.

Item	Description
Dynamic range (distance from sensor)	2.6 ft to 13.12 ft (0.8 m to 4 m)
Color and depth stream	4 ft to 11.5 ft (1.2 m to 3.5 m) playable range (a software-based specification)
Skeleton tracking	4 ft to 11.5 ft (1.2 m to 3.5 m) playable range (a software-based specification)
Viewing angle	43-degree vertical by 57-degree horizontal field of view
Mechanized tilt range (vertical)	Plus or minus 28 degrees
Frame rate (depth and color stream)	30 fps
Resolution, depth stream	QVGA (320 × 240)
Resolution, color stream	VGA (640 × 480)
Audio input characteristics	A 4-microphone array with 24-bit analog-to-digital conversion (ADC), and Kinect-resident signal processing such as echo cancellation and noise suppression
Audio format (accessed with XHV2 , NuiSpeech , and NuiAudio)	16 kHz 16-bit mono PCM

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Starting Up the Kinect Sensor

[See Also](#)



Connect, calibrate, and test the Kinect Sensor, as follows.

Sections

- [Powering Up the Kinect Sensor](#)
- [Calibrating the Microphone Array](#)
- [Testing the Kinect Sensor](#)

Powering Up the Kinect Sensor

To start the Kinect Sensor

1. Connect the USB cable from the Kinect Sensor to your development console.
2. Connect the power supply for the Kinect Sensor.
3. Turn on the Xbox 360 development console.

Calibrating the Microphone Array

You must profile your acoustic environment before using any audio stream data from the Kinect Sensor. Profiling ensures that the microphone array is properly calibrated and providing the best audio data possible. Use the [Kinect Tuner for Audio](#) for initial calibration and any subsequent calibrations. After calibration, speech recognition and other audio features can be used by the title.

When run, test tones play in the speakers in sequence, profiling the acoustic environment.

You will need to re-profile the acoustic environment under the following circumstances:

- Console is cold rebooted
- Kinect Sensor loses power or is disconnected from the console
- Speakers that are used for console audio are moved
- Kinect Sensor is moved
- Large objects (such as a desk) are moved, changing the acoustics of the room

Note

Changing the tilt of the Kinect Sensor does not require re-profiling.

Testing the Kinect Sensor

Once the Kinect Sensor has been connected and turned on, you can run sample applications from the XDK to verify that the sensors in the array are working correctly.

To verify that the depth and color sensors are working, run the [Simple Skeleton Tracking](#) sample.

To test the depth and color sensors by using Simple Skeleton Tracking

1. From the launcher of the Xbox 360 development console, select **Simple Skeleton Tracking**, and then press the **A** button.
2. Stand within the Kinect Sensor's view, and use the application to recognize and render your skeletal frame from your image.
3. Use the application's controls to switch between the various depth- and color-sensor views.

To verify that the microphone array is working, run the [Simple Speech Recognition](#) sample.

To test the microphone by using Simple Speech Recognition

1. From the launcher of the Xbox 360 development console, select **Simple Speech Recognition**.
2. Stand within the Kinect Sensor's view and speak.

If the microphone is working, the sample will provide some sort of output when you speak.

Depending on language settings, calibration, and whether the tester is a native speaker of the current language, the output may indicate failures or low confidence values.

If the microphone is *not* working properly, the sample will not show any output in response to speech.

See Also

[Troubleshooting the Kinect Sensor](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Troubleshooting the Kinect Sensor



The following are tips for troubleshooting the Kinect Sensor.

Sections

- [Tips](#)
- [Troubleshooting](#)

Tips

Make Sure the Kinect Sensor is Properly Connected

Verify that the Xbox 360 Kinect sensor array is connected to a USB port on the Xbox 360 development console.

Do not connect the sensor array through a USB hub. Having multiple devices on the same USB bus as the Kinect Sensor can cause data streams from the sensor array to be corrupted.

Troubleshooting

Simple Skeleton Tracking sample does not run

This may be because the sensor array is not turned on or is not connected. Check the USB connection and verify that the red light on the sensor array is on.

Camera streams do not function properly after rebooting the console

This can occur when the camera is streaming data while the power to the console is manually turned off or a reboot is requested through the development computer. On restart, the camera reconnects to the console, but it may be left in an indeterminate state. Detach and reattach the USB cable to the development console to force the device to reset.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Quick-Start Tutorial for Kinect Programming



These procedures add Kinect functionality to the [Dolphin](#) sample.

This sample initially shows a dolphin swimming around in a circle at a constant speed. After you Kinect-enable the sample, the dolphin swims forward faster as you lower your arm, and swims backwards faster as you raise your arm. This Kinect-enabled sample displays the Kinect camera images and skeleton at the top of the screen. After doing this tutorial, you'll have an idea of how to add other Kinect functionality to other samples.

The Dolphin sample uses the [ATG Framework](#), which provides C++ libraries that are used by most of the XDK samples.

Note

The following sequence is recommended:

1. Set up your Xbox 360 development environment (see [Setting Up Your Xbox 360 Development Kit](#)) and connect your Kinect Sensor (see [Starting Up the Kinect Sensor](#)).
2. Watch the [Quick-Start Kinect Tutorial](#) video.
3. Work through this tutorial.

[Part 1. Installing and Running the Dolphin Sample](#)

[Part 2. Deploying the NUI Database Files](#)

[Part 3. Initializing the NUI System](#)

[Part 4. Opening the Color Image Stream and Getting Color Image Data](#)

[Part 5. Opening the Depth Image Stream and Getting Depth Image Data](#)

[Part 6. Enabling Skeleton Tracking and Getting Skeleton Data](#)

[Part 7. Adding and Compiling Shaders for Rendering Kinect Data](#)

[Part 8. Displaying the Image Streams and Skeleton Data in PiP Windows](#)

[Part 9. Using Skeleton Tracking to Control the Application](#)

[Files Involved in This Tutorial](#)

Part 1. Installing and Running the Dolphin Sample

Each of the following steps in the exact sequence described will result in adding Kinect functionality to your Dolphin sample. Before adding Kinect capability to the Dolphin sample, make sure the Dolphin sample compiles and runs using your development environment, as follows.

Installing the Dolphin sample

1. Click the Windows **Start** menu, then click **All Programs: Microsoft Xbox 360 SDK: Sample Browser**.

The **Microsoft Xbox 360 Sample Browser** appears.

2. In the **show** drop-down list in the upper left, click **Graphics**. Scroll to the **Dolphin** sample, and click **Install Project**.

The **Install 'Dolphin'** dialog box appears. The **Location** text box shows the default location:

C:\Users\[username]\Documents\Visual Studio 2010\Projects\Xbox360Samples\Dolphin

3. Click the **Install** button.

If there is already a Dolphin directory there, the **Directory Already Exists** dialog box appears. Click **OK** to close it, then in **Windows Explorer**, rename that existing directory, such

as `Dolphin_orig`. Then click **Install** again. (You can install to a different directory name.)

The **Installation Successful** dialog box appears.

- Click **Yes** to view the new Dolphin project folder.
Windows Explorer appears, showing the default location.

Compiling and Running the Dolphin Sample

- To use Visual Studio 2010, double-click `Dolphin2010.sln`. If you want to use a different environment, you can do this tutorial in Visual Studio and then again in your environment, using the equivalent steps.
The Dolphin solution opens in Visual Studio.
- In **Solution Explorer**, expand the **Dolphin** project, then double-click `Dolphin.cpp`.
The `Dolphin.cpp` code listing appears.
- Compile the solution. (**Build** menu: **Build Solution**, or **F7**)
The ATG and Dolphin project files are compiled.
- Turn on your Xbox console's video monitor.
- Run the solution. (**Debug** menu: **Start Debugging**, or **F5**)
The unmodified sample is deployed to the development console. The dolphin swims forward in circles, and is not responding to the Kinect Sensor yet.
- Stop running the solution. (**Debug** menu: **Stop Debugging**, or **Shift+F5**)

The Dolphin sample is now ready to modify with Kinect support.

Part 2. Deploying the NUI Database Files

Natural User Interface (NUI) is the API that supports the Kinect Sensor. All NUI-enabled projects require the database file `Database.xmlpr` for skeleton tracking, and the Identity database file `NuiIdentity.bin.be` to keep track of specific players. A Kinect title that uses speech recognition would also be required to include the speech exemplar database for each supported language, such as English.

To deploy these files, you add the XDK NUI Databases directory to the Deployment Files property of the project. This causes the `Database.xmlpr` and `NuiIdentity.bin.be` database files to be deployed from the XDK NUI Databases directory to the Xbox console when the sample is run.

- In **Solution Explorer**, right-click the **Dolphin** project, then select **Properties**.
The **Dolphin Property Pages** dialog box appears.
- On the left, click **Configuration Properties** and expand **Console Deployment**. Select the **Copy To Hard Drive** property page.
- Click in the column to the right of **Deployment Files**, click the drop-down arrow, then click **<Edit...>**.
The **Deployment Files** dialog box appears.
- In the **Source Files (From PC)** column, double-click an empty row, and enter the following path:
\$(XEDK)\Redist\xbox\NaturalInput\Databases
`$(XEDK)` will resolve to the XDK installation path on the development machine.
Do not click **OK** at this point. Do click in the white space to confirm.
- In the same row, double-click in the **Target Media Path** column, enter the following:
\$(RemoteRoot)
Then press the **Tab** key or click outside the cell to accept the value.
- Click **OK** to close the dialog boxes.
- Save the solution. (**File** menu: **Save All**, or **Ctrl+Shift+S**)
- Compile the solution. (**F7**)
- Run the solution. (**F5**)
The dolphin swims forward in circles. The NUI database files for skeleton tracking and Identity are now deployed to the console when the sample is run.
- Stop running the solution. (**Shift+F5**)

Part 3. Initializing the NUI System

To initialize NUI, first add `#include` directives for the ATG Framework and the main NUI header, and then add NUI API and Skeleton Tracking libraries as linker dependencies, as follows.

Adding #include Directives for the ATG Framework and the Main NUI Header

- In `Dolphin.cpp`, below the existing `#include` directives, add the following, to include the ATG utility header and the main NUI header:

```
C++

#include <AtgNuiCommon.h>
#include <NuiApi.h>
```

`AtgNuiCommon.h` is an ATG Framework header file that provides a few NUI utility functions and constants, including `NUI_CAMERA_TIMEOUT_DEFAULT`, which is used in this example.
`NuiApi.h` is the main Xbox 360 library header for NUI support.

Adding NUI API and Skeleton Tracking Libraries as Linker Dependencies

- In **Solution Explorer**, right-click the **Dolphin** project, then click **Properties**.
The **Property Pages** dialog box appears.
- On the left, expand **Configuration Properties: Linker**, then click the **Input** property page.
- Click in the column to the right of **Additional Dependencies**, (for Visual Studio 2010) click the drop-down arrow, and then click **<Edit...>**.
The **Additional Dependencies** dialog box appears.
- Scroll to the bottom of the list, then enter the following:
`nuiapid.lib`
`Std.lib`
These are the library files for NUI API (debug version) and skeleton tracking (debug version).
- Click **OK** to close the dialog boxes.

Initializing the NUI System

- In `Dolphin.cpp`, go to the top of the **Initialize** method (in the **Sample** class). Below the statement `HRESULT hr;` add the following code, which initializes the NUI system.

```
C++

// Initialize NUI system
hr = NuiInitialize( NUI_INITIALIZE_FLAG_USES_SKELETON |
                  NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
```



```

        NUI_INITIALIZE_FLAG_USES_COLOR,
        NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD );

if( FAILED( hr ) )
{
    ATG_PrintError( "Couldn't initialize NUI\n" );
    return hr;
}

```

Tip

To select a block of code in a Help topic, triple-click the first line of code.

Tip

After you paste code, correct the indentation by selecting lines and then pressing **Tab**.

[NuiInitialize](#) initializes Kinect to use the specified hardware thread and subsystems. [NUI_INITIALIZE flags](#) are combined and passed as the first argument. These flags enable the specified subsystems, which consume resources.

- [NUI_INITIALIZE_FLAG_USES_SKELETON](#) starts the skeleton tracking engine.
- [NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX](#) tells the sensor array to use the depth sensor, which provides depth information and player segmentation (masking the player from the background).
- [NUI_INITIALIZE_FLAG_USES_COLOR](#) tells the sensor array to use the color camera, which provides color information.

Other subsystem flags such as for speech recognition are not specified, saving resources.

ATG_PrintError is declared in [AtgUtil.h](#), which is included by [AtgNuiCommon.h](#).

2. Save the solution. (**Ctrl+Shift+S**)
3. Position your Kinect Sensor so that it can tilt down and up.
4. Compile the solution. (**F7**)
5. Run the solution. (**F5**)

The dolphin swims forward in a circle. The Kinect Sensor may tilt down and up, to detect the floor plane. Your code now initializes the NUI system to use the color camera, depth sensor, and skeleton tracking engine. The data from the Kinect Sensor isn't used yet.

If an error dialog reports that the connection between the PC and the console has been lost, run the solution again, or close and reopen Visual Studio.

6. Stop running the solution. (**Shift+F5**)

Part 4. Opening the Color Image Stream and Getting Color Image Data

These steps use the NUI color image stream. The NUI depth stream and the skeleton tracking pipeline are used later. The NUI audio stream is not used in this tutorial.

Opening the Color Image Stream

1. In [Dolphin.cpp](#), go to the [Global variables and definitions](#) section near the top. Below the existing global variables, declare the handle for the color image stream, as follows:

C++

```
HANDLE m_hColorImageStream;
```

2. Go to the **Initialize** method. Below the existing NUI code you added, enter the following code, which opens the color image stream:

C++

```

// Open NUI color image stream
hr = NuiImageStreamOpen( NUI_IMAGE_TYPE_COLOR,
                        NUI_IMAGE_RESOLUTION_640x480,
                        0,
                        1,
                        NULL,
                        &m_hColorImageStream );

if( FAILED( hr ) )
{
    ATG_PrintError( "Couldn't initialize NUI color stream\n" );
    return hr;
}

```

[NuiImageStreamOpen](#) opens a color or depth image stream — in this case, a color image stream.

The [NUI_IMAGE_TYPE_COLOR](#) member of the [NUI_IMAGE_TYPE](#) enumeration indicates that the image stream uses RGB images from the color camera.

The [NUI_IMAGE_RESOLUTION_640x480](#) member of the [NUI_IMAGE_RESOLUTION](#) enumeration selects 640x480 resolution of images from the color camera.

Getting Color Image Data from the Color Image Stream

1. Go to the **Update** method. Above the line [// Animation attributes for the dolphin](#), add the following code, which gets color image data from the color image stream:

C++

```

// Get next NUI color image frame
HRESULT hr;
const NUI_IMAGE_FRAME* pColorImageFrame;

hr = NuiImageStreamGetNextFrame( m_hColorImageStream,
                                NUI_CAMERA_TIMEOUT_DEFAULT,
                                &pColorImageFrame );

if( SUCCEEDED( hr ) )
{
    // Process data for current frame
    // ...

    NuiImageStreamReleaseFrame( m_hColorImageStream, pColorImageFrame );
}

```

The [NUI_IMAGE_FRAME](#) structure stores information about a color image frame (or depth image frame), including frame number, image type, resolution, and view area.

[NuiImageStreamGetNextFrame](#) gets the next frame from an image stream from the Kinect Sensor — in this case, the color image stream.

[NUI_CAMERA_TIMEOUT_DEFAULT](#) is defined in [AtgNuiCommon.h](#), which is part of the ATG Framework.

[NuiImageStreamReleaseFrame](#) is required, to release the image frame buffer of the color image stream so the buffer can be reused, to allow another color image frame to be returned later.

2. Save the solution. (**Ctrl+Shift+S**)
3. Compile the solution. (**F7**)
4. Run the solution. (**F5**)

Your code now initializes the NUI color image stream and retrieves the latest color image frame. The data from the Kinect Sensor isn't displayed yet.

5. Stop running the solution. (**Shift+F5**)

Part 5. Opening the Depth Image Stream and Getting Depth Image Data

This depth image code is similar to the previous color image code.

Open the depth image stream and then get the depth image data from the color image stream, as follows.

Opening the Depth Image Stream

1. In `Dolphin.cpp`, go to the `Global variables and definitions` section near the top. Below the existing global variables, declare the handle for the depth image stream, as follows:

C++

```
HANDLE m_hDepthImageStream;
```

2. Go to the **Initialize** method. Below the existing NUI code you added, add the following code, which opens the depth image stream:

C++

```
// Open NUI depth image stream
hr = NuiImageStreamOpen( NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX,
                        NUI_IMAGE_RESOLUTION_320x240,
                        0,
                        1,
                        NULL,
                        &m_hDepthImageStream );

if( FAILED( hr ) )
{
    ATG_PrintError( "Couldn't initialize NUI depth stream\n" );
    return hr;
}
```

[NuiImageStreamOpen](#) opens a color or depth image stream — in this case, a depth image stream.

The `NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX` member of the `NUI_IMAGE_TYPE` enumeration indicates that the image stream uses images from the depth camera, combined with image segmentation data generated by the skeleton tracking system that labels pixels in the depth map as belonging to specific players.

The `NUI_IMAGE_RESOLUTION_320x240` member of the `NUI_IMAGE_RESOLUTION` enumeration selects 320x240 resolution of images from the color camera.

Getting Depth Image Data from the Depth Image Stream

1. Go to the **Update** method. Below the NUI code you added, add the following code, which gets depth image data from the depth image stream:

C++

```
// Get next NUI depth image frame
const NUI_IMAGE_FRAME* pDepthImageFrame;

hr = NuiImageStreamGetNextFrame( m_hDepthImageStream,
                                NUI_CAMERA_TIMEOUT_DEFAULT,
                                &pDepthImageFrame );

if( SUCCEEDED( hr ) )
{
    // Process data for current frame
    // ...
    NuiImageStreamReleaseFrame( m_hDepthImageStream, pDepthImageFrame );
}
```

The `NUI_IMAGE_FRAME` structure stores information about a depth image frame (or color image frame), including frame number, image type, resolution, and view area.

[NuiImageStreamGetNextFrame](#) gets the next frame from an image stream from the Kinect Sensor — in this case, the depth image stream.

[NuiImageStreamReleaseFrame](#) is required, to release the image frame buffer of the depth image stream so the buffer can be reused, to allow another depth image frame to be returned later.

2. Save the solution. (**Ctrl+Shift+S**)
3. Compile the solution. (**F7**)
4. Run the solution. (**F5**)

Your code now initializes the NUI depth stream and retrieves the latest depth image frame from the depth image stream. The data from the Kinect Sensor isn't displayed yet.

5. Stop running the solution. (**Shift+F5**)

Part 6. Enabling Skeleton Tracking and Getting Skeleton Data

The skeleton tracking pipeline provides a series of `NUI_SKELETON_FRAME` structures that store a frame of skeleton tracking data. Each frame of skeleton data contains an array of `NUI_SKELETON_DATA` structures, one for each tracked player. Each `NUI_SKELETON_DATA` structure contains an array of skeleton joint positions, such as the positions of the right hand and right shoulder. Each skeleton joint position is an `XMVECTOR`, with X, Y, and Z coordinate values.

This example procedure will compare the Y coordinate of the right hand joint with the Y coordinate of the right shoulder joint to control the speed and direction of the dolphin.

Enabling Skeleton Tracking

1. In `Dolphin.cpp`, go to the `Global variables and definitions` section near the top. Below the existing global variables, declare variables for a frame of skeleton data and to select a skeleton in a frame of skeleton data, as follows:

C++

```
NUI_SKELETON_FRAME m_SkeletonFrame;
int iCurrentSkeletonIndex;
```

2. Go to the **Initialize** method. Below the existing NUI code you added, enter the following code, which enables and initializes the skeleton tracking system:

C++

```
// Initialize NUI skeleton tracking
hr = NuiSkeletonTrackingEnable( NULL, 0 );
iCurrentSkeletonIndex = 0;

if( FAILED( hr ) )
{
    ATG_PrintError( "Couldn't initialize NUI skeletal tracking stream\n" );
}
```

```
        return hr;
    }
}
```

[NuiSkeletonTrackingEnable](#) enables skeleton tracking.

Getting Skeleton Data

- 1. Go to the **Update** method. Below the NUI code you added, add the following code, which gets the next frame of skeleton data:

C++

```
// Get next NUI skeleton frame
bool bSkeletonFound = FALSE;

hr = NuiSkeletonGetNextFrame( NUI_CAMERA_TIMEOUT_DEFAULT, &m_SkeletonFrame );

if( SUCCEEDED( hr ) )
{
    UINT i = 0;

    hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );

    // Process the skeleton data
    // ...
}
```

[NuiSkeletonGetNextFrame](#) gets the next available frame of skeleton data, including data for each skeleton.

[NuiTransformSmooth](#) filters skeleton positions to reduce jitter between frames.

- 2. Save the solution. (**Ctrl+Shift+S**)
 - 3. Compile the solution. (**F7**)
 - 4. Run the solution. (**F5**)
- The dolphin swims forward in a circle. Your code now enables skeleton tracking and gets skeleton tracking data. The data from the Kinect Sensor isn't displayed yet.
- 5. Stop running the solution. (**Shift+F5**)

Part 7. Adding and Compiling Shaders for Rendering Kinect Data

Next, add the shaders file to the project and define a custom build step to compile the shader, as follows. The shaders will be used to render skeleton data.

Adding SimpleShaders.fx to the Project

Next, we'll create an **Effects** directory, copy a **.fx** file from the **XDK Source** directory into the new directory, create a filter (folder) in **Solution Explorer**, and add the sample's copy of the **.fx** file as an existing item into that folder.

- 1. In **Windows Explorer**, go to the **Media** directory of your Dolphin sample. For example:
C:\Users\[username]\Documents\Visual Studio 2010\Projects\Xbox360Samples\Dolphin\Media
- 2. Create an **Effects** subdirectory.
- 3. In **Windows Explorer**, copy the file **SimpleShaders.fx** from the **XDK Media\Effects** directory, such as:
C:\Program Files (x86)\Microsoft Xbox 360 SDK\Source\Samples\Media\Effects
to the **Effects** directory you created, such as:
C:\Users\[username]\Documents\Visual Studio 2010\Projects\Xbox360Samples\Dolphin\Media\Effects
- 4. In Visual Studio, in **Solution Explorer**, right-click the **Dolphin** project, then click **Add: New Filter**.
A new folder appears in **Solution Explorer**, ready to rename.
- 5. Name the new folder **FXLite Effects**.
- 6. Right-click the **FXLite Effects** folder, click **Add**, then click **Existing Item**.
The **Add Existing Item** dialog box appears.
- 7. Go to the **Effects** directory you created, such as:
C:\Users\[username]\Documents\Visual Studio 2010\Projects\Xbox360Samples\Dolphin\Media\Effects
- 8. Select the **SimpleShaders.fx** file, then click **Add**.
- 9. If the **Matching Custom Build Rule Not Found** dialog box appears, click **No**. We'll create a custom build step instead that asks whether to create a build rule, rather than a custom build rule.
SimpleShaders.fx appears in the **FXLite Effects** folder in **Solution Explorer**.

Defining a Custom Build Step to Compile the Shader File

- 1. In **Solution Explorer**, right-click the **SimpleShaders.fx** file you added, then click **Properties**.
The **SimpleShaders.fx Property Pages** dialog box appears.
- 2. Select the **General** property page for defining a custom build step, as follows.
For Visual Studio 2010:
 - a. In the tree on the left, expand **Configuration Properties**, and then click the **General** property page node.
 - b. Set the **Item Type** property to **Custom Build Tool**, then click the **Apply** button.
The **Custom Build Tool** property pages group appears in the tree on the left.
 - c. Expand **Custom Build Tool**, then click the **General** property page node.
- 3. In the **Command Line** box:
(For Visual Studio 2010) Enter the following:
fxc /nologo /XZi /XFd"% (OutDir) FXL_% (Filename).updb" /Tfxl_3_0 /Fo"% (OutDir) Media\Effects%\% (Filename).fxobj" "% (FullPath)"
This build command has the following parts.

Command Fragment	Meaning
fxc	The Effects Command-line Compiler , which compiles a D3DX9 shader for Xbox 360.
/nologo	Suppresses the logo display.
/XZi	Generates a UPDB file containing debugging information.
/XFd	Names the UPDB debugging information file.
/Tfxl_3_0	Defines the target profile as fxl 3.0.
/Fo	Names the object output file.

- | | |
|-------------------------|--------------------------------------|
| (final path in command) | Specifies the input file to compile. |
|-------------------------|--------------------------------------|
- In the **Description** box:
(For Visual Studio 2010) Enter the following:
`Build FXLite Effect %(FullPath)`
 - In the **Outputs** box:
(For Visual Studio 2010) Enter the following:
`$(OutDir)Media\Effects\%(Filename).fxobj`
 - Click **OK** to close the dialog box.
 - Save the solution. (**Ctrl+Shift+S**)
 - Compile the solution. (**F7**)
 - Run the solution. (**F5**)
The dolphin swims forward in a circle. The custom build tool step you defined compiles the D3DX9 shader for Xbox 360 and places the resulting `.fxobj` file in the output directory. Your code now initializes shaders that will be used to render skeleton data.
 - Stop running the solution. (**Shift+F5**)

Part 8. Displaying the Image Streams and Skeleton Data in PiP Windows

The ATG Framework project supports Picture-in-Picture (PiP) display. Next, you add a PiP window on the left to display the color image stream overlaid with the skeleton tracking pipeline data, and a PiP window on the right to display the depth image stream.

Initialize the Shaders and PiP Windows

1. At the top of `Dolphin.cpp`, below the existing `#include` directives, add the following, to include the ATG PiP and shaders headers:

C++

```
#include <AtgNuiVisualization.h>
#include <AtgSimpleShaders.h>
```

`AtgNuiVisualization.h` is the ATG Framework header file for creating NUI Picture-in-Picture windows. The **ATG::NuiVisualization** class includes an **Initialize** method and methods to render the current frames of color, depth, and skeleton data.

`AtgSimpleShaders.h` is the ATG Framework header file for using simple shaders. The **ATG::SimpleShaders** class includes an **Initialize** method. This class uses the compiled file `SimpleShaders.fxobj`.

2. In `Dolphin.cpp`, go to the `Global variables and definitions` section near the top. Below the existing global variables, declare the PiP window variable, as follows:

C++

```
ATG::NuiVisualization m_Visualizer;
```

m_Visualizer is a PiP window, of type **ATG::NuiVisualization**. That class displays one or more PiP windows that display color image, depth image, and skeleton tracking data.

3. Go to the **Initialize** method. Below the existing NUI code you added, enter the following code, which initializes the shaders and PiP window:

C++

```
// Initialize simple shaders
ATG::SimpleShaders::Initialize( NULL, NULL );

// Initialize PiP window
hr = m_Visualizer.Initialize( m_pd3dDevice,
    NUI_INITIALIZE_FLAG_USES_COLOR |
    NUI_INITIALIZE_FLAG_USES_SKELETON |
    NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX,
    NUI_IMAGE_RESOLUTION_640x480 );
```

The PiP window is initialized by calling **ATG::NuiVisualization::Initialize**.

Update the PiP Window to Point to the Latest Color, Depth, and Skeleton Frames

1. Go to the **Update** method. Replace the code block `// Get next NUI color image frame` by the following final version:

C++

```
// Get next NUI color image frame
HRESULT hr;
const NUI_IMAGE_FRAME* pColorImageFrame;

hr = NuiImageStreamGetNextFrame( m_hColorImageStream,
    NUI_CAMERA_TIMEOUT_DEFAULT,
    &pColorImageFrame );

if( SUCCEEDED( hr ) )
{
    // Process data for current frame
    m_Visualizer.SetColorTexture( pColorImageFrame->pFrameTexture );

    NuiImageStreamReleaseFrame( m_hColorImageStream, pColorImageFrame );
}
```

That change added the call to **ATG::NuiVisualization::SetColorTexture**, which updates the PiP window to point to the latest color image frame, but doesn't render the frame. This method takes a texture as an **IDirect3DTexture9** data member of a color image frame (which is a `NUI_IMAGE_FRAME` structure).

2. Replace the code block `// Get next NUI depth image frame` by the following final version:

C++

```
// Get next NUI depth image frame
const NUI_IMAGE_FRAME* pDepthImageFrame;

hr = NuiImageStreamGetNextFrame( m_hDepthImageStream,
    NUI_CAMERA_TIMEOUT_DEFAULT,
    &pDepthImageFrame );

if( SUCCEEDED( hr ) )
{
    // Process data for current frame
    m_Visualizer.SetDepthTexture( pDepthImageFrame->pFrameTexture );
}
```

```

        NuiImageStreamReleaseFrame( m_hDepthImageStream, pDepthImageFrame );
    }

```

That change added the call to **ATG::NuiVisualization::SetDepthTexture**, which updates the PiP window to point to the latest depth image frame, but doesn't render the frame.

3. Replace the code block `// Get next NUI skeleton frame` by the following expanded version:

```

C++

// Get next NUI skeleton frame
bool bSkeletonFound = FALSE;

hr = NuiSkeletonGetNextFrame( NUI_CAMERA_TIMEOUT_DEFAULT, &m_SkeletonFrame );

if( SUCCEEDED( hr ) )
{
    UINT i = 0;

    hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );

    // Process the skeleton data
    while ( ( i < NUI_SKELETON_COUNT ) && bSkeletonFound != TRUE )
    {
        if(m_SkeletonFrame.SkeletonData[i].eTrackingState !=
            NUI_SKELETON_NOT_TRACKED )
        {
            m_Visualizer.SetSkeletons( &m_SkeletonFrame );
            iCurrentSkeletonIndex=i;
            bSkeletonFound= TRUE;
        }
        i++;
    }
}

```

The change added the code in `// Process the skeleton data`, including a call to **ATG::NuiVisualization::SetSkeletons**, which updates the PiP window to point to the latest frame of skeleton tracking data, but doesn't render the frame.

Before setting the skeleton tracking frame to the latest frame, **m_SkeletonFrame.SkeletonData[i].eTrackingState** is checked to make sure the skeleton tracking frame contains valid skeleton data. A [NUI_SKELETON_FRAME](#) structure contains an array of [NUI_SKELETON_DATA](#) structures, one for each tracked player. **eTrackingState** is an [NUI_SKELETON_TRACKING_STATE](#) enumeration constant that indicates how much detail the NUI_SKELETON_DATA structure contains, such as whether the array of skeleton joint positions contains valid data.

That change also added the **bSkeletonFound** and **i** declarations.

Render the Image Streams and Skeleton Data

Next, you render the color image stream, skeleton data, and depth image stream using methods of the **ATG::NuiVisualization** class.

1. Go to the **Render** method. Near the end of the method, above the line `// Present the scene`, add:

```

C++

// Render the PiP windows for the NUI data
const FLOAT WIDTH = 300.0f;
const FLOAT HEIGHT = WIDTH * 3.0f / 4.0f;
const FLOAT COLOR_X = 300.0f;
const FLOAT COLOR_Y = 50.0f;

m_Visualizer.BeginRender();
m_Visualizer.RenderColorStream( COLOR_X, COLOR_Y, WIDTH, HEIGHT );
if( m_SkeletonFrame.SkeletonData[iCurrentSkeletonIndex].eTrackingState !=
    NUI_SKELETON_NOT_TRACKED )
{
    m_Visualizer.RenderSingleSkeleton( iCurrentSkeletonIndex, COLOR_X, COLOR_Y,
                                      WIDTH, HEIGHT, FALSE, TRUE );
}

m_Visualizer.RenderDepthStream( COLOR_X + WIDTH + 10, COLOR_Y, WIDTH, HEIGHT );
m_Visualizer.EndRender();

```

RenderColorStream renders the color image frames at X coordinate **COLOR_X** (the left PiP window).

RenderSingleSkeleton renders the skeleton tracking data frames at X coordinate **COLOR_X** (the left PiP window).

RenderDepthStream renders the depth image frames at X coordinate **COLOR_X + WIDTH + 10** (the right PiP window).

2. Save the solution. (**Ctrl+Shift+S**)
3. Position your Kinect Sensor, and turn on your Xbox console's video monitor.
4. Compile the solution. (**F7**)
5. Run the solution. (**F5**)

Move far enough away from the sensor, and rise up enough out of your chair, so that your skeleton appears in the color image PiP window, which is on the left.

Your color image stream and skeleton appear in the PiP window on the left, and your depth image stream appears in the PiP window on the right. This Kinect data is not yet hooked up to control the dolphin's motion.

6. Stop running the solution. (**Shift+F5**)

Part 9. Using Skeleton Tracking to Control the Application

To hook up skeleton tracking data to the dolphin's motion, initialize the speed variables and then update the speed based on skeleton joint positions, as follows.

Initialize the Speed Variables

1. In `Dolphin.cpp`, go to the `Global variables and definitions` section near the top. Below the existing global variables, declare variables for dolphin's target speed and current speed, as follows:

```

C++

FLOAT reqSpeed;
FLOAT speed;

```

2. Go to the **Initialize** method. Above the existing NUI initialization code (`// Initialize NUI system`), add the following:

```

C++

reqSpeed = 0.0f;
speed = 0.0f;

```

Update the Speed Based On Skeleton Joint Positions

1. Go to the **Update** method. Above `// Animation attributes for the dolphin`, enter the following code, which updates the dolphin's speed, based on skeleton data:

C++

```
// Set speed based on elevation of hand joint relative to shoulder joint
FLOAT SpeedRatio;
FLOAT fElapsedTime = ( FLOAT )m_Timer.GetElapsedTime();
static ATG::Timer SkelTimer;
static FLOAT fSkelTime = ( FLOAT )SkelTimer.GetElapsedTime();

reqSpeed = speed;
static FLOAT backward = 0.0f;
static FLOAT forward = 0.0f;

if( bSkeletonFound )
{
    // Find the right hand position

    // If the right hand is currently not tracked, pause the dolphin animation
    if( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
        .eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_HAND_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED ||
        m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
        .eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED )
    {
        reqSpeed = 0.0f;
    }
    else
    {
        // Vertical distance from hand to shoulder
        FLOAT a = fabs( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].x -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].x );

        // Horizontal distance from hand to shoulder
        FLOAT b = fabs( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y );

        // Distance from hand to shoulder
        FLOAT radius = sqrt( a * a + b * b );

        backward = radius / 2.0f;
        forward = ( radius * -1.0f ) / 2.0f;
        reqSpeed = m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
            .SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;
    }

    // Determine the current swim direction
    if( reqSpeed > backward )
        speed = backward;
    else if( reqSpeed < forward )
        speed = forward;
    else
        speed = reqSpeed;

    // Determine the current swim speed
    if( forward == 0.0f )
        SpeedRatio = 1.0;
    else
        SpeedRatio = speed * 2 / forward;
}
```

The following is the main statement to adjust the dolphin's speed based on the height of your hand:

```
reqSpeed = m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
    .SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y -
    m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
    .SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;
```

2. In the section `// Animation attributes for the dolphin`, replace the line

```
FLOAT fKickFreq = 2 * fTime;
```

by the following:

C++

```
static FLOAT fKickFreq = 0.0f;
fKickFreq += 2.0f * SpeedRatio * fElapsedTime;
```

3. Replace the line

```
FLOAT fPhase = fTime / 3;
```

by the following:

C++

```
static FLOAT fPhase = 0.0f;
fPhase += fElapsedTime / ( 3 / SpeedRatio );
```

4. Save the solution. (**Ctrl+Shift+S**)
5. Position the Kinect Sensor so it has a clear view. Turn on the monitor for your development kit.
6. Compile the solution. (**F7**)
7. Run the solution. (**F5**)

The dolphin swims forward in a circle. The Kinect Sensor may tilt down and up, to detect the floor plane. The color image stream appears in the PIP window in the top left, and the depth image stream appears in the PIP window in the top right.

If an error dialog reports that the connection between the PC and the console has been lost, run the solution again, or close and reopen Visual Studio.

8. Move far enough away from the sensor, and rise up enough out of your chair, so that your skeleton appears in the color image PIP window, which is on the left.

- As you lower your right hand below your right shoulder, the dolphin swims forward faster. As you raise your right hand above your right shoulder, the dolphin swims backwards faster. When skeleton tracking of your right hand and shoulder is lost, the dolphin pauses.
- For the most accurate control, stand where your skeleton is clearly detected throughout the range of your arm motion.
- Your code now uses skeleton tracking to control the modified Dolphin sample, by using NUI classes and methods in the ATG Framework, API items in the Xbox 360 NUI libraries, and various supporting files.
9. Stop running the solution. (**Shift+F5**)

Files Involved in This Tutorial

File Added	Description
NuiApi.h	The main Xbox 360 library header for NUI support. This file declares NuiInitialize and the constant NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD , and includes most of the other NUI header files. NuiInitialize here uses the NUI subsystems initialization flags which enable the color image stream, depth image stream, and skeleton tracking.
NuiImageCamera.h	Included by NuiApi.h . Declares the NUI_IMAGE_FRAME structure and the functions NuiImageStreamOpen , NuiImageStreamGetNextFrame , and NuiImageStreamReleaseFrame .
NuiSkeleton.h	Included by NuiApi.h . Declares the functions NuiSkeletonTrackingEnable , NuiSkeletonGetNextFrame , and NuiTransformSmooth ; the NUI_SKELETON_FRAME and NUI_SKELETON_DATA structures; the NUI_SKELETON_POSITION_INDEX enumeration members NUI_SKELETON_POSITION_SHOULDER_RIGHT and NUI_SKELETON_POSITION_HAND_RIGHT ; and the NUI_SKELETON_POSITION_TRACKING_STATE enumeration.
AtgNuiCommon.h	An ATG Framework header file that provides a few NUI utility functions and constants, including NUI_CAMERA_TIMEOUT_DEFAULT . This header includes AtgUtil.h , which declares ATG_PrintError .
AtgNuiVisualization.h	An ATG Framework header file for NUI Picture-in-Picture windows. This file declares the ATG::NuiVisualization class, including the following methods: Initialize , SetColorTexture , SetDepthTexture , SetSkeletons , BeginRender , RenderColorStream , RenderSingleSkeleton , RenderDepthStream , and EndRender .
AtgSimpleShaders.h	An ATG Framework header file for simple shaders. The ATG::SimpleShaders class includes the Initialize method.
SimpleShaders.fx	The simple shaders effects file.
fxc.exe	A command-line utility that compiles a shader. See Effects Command-line Compiler .
nuiapid.lib	The debug version of the static library for most of the NUI feature areas. See Adding NUI Header and Library Files .
STd.lib	The debug version of the static library for NUI skeleton tracking. See Adding NUI Header and Library Files .
Database.xmlplr	The NUI skeleton tracking database. See Kinect Database Files to Include on the Game Disc .
NuiIdentity.bin.be	The NUI Identity database, to keep track of specific players. See Kinect Database Files to Include on the Game Disc .

What to do next

- After you complete the tutorial, we recommend the following activities.
- Watch the videos on GDN, in the [XDK Unlocked: A Video Series](#).
 - Install, compile, and run some of the XDK samples. See [Using the XDK samples](#), which directs you to all categories of samples and explains how to install, compile, and run. For a list of samples that are specifically about NUI, see [Natural User Interface Samples](#).
 - Find samples related to your title, and modify them to start prototyping features for your title. As a starting point for prototyping, you can use the XDK samples, which use the ATG Framework. If you want to prototype without relying on the ATG Framework, see [Adding NUI Header and Library Files](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Adding NUI Header and Library Files

[See Also](#)



Adding Kinect support to an Xbox 360 application requires the addition of NUI header files and NUI library files to the project. You will also deploy the NUI database files to the console. For more information, see [Kinect Database Files to Include on the Game Disc](#).

Sections

- [Adding NUI Header Files to a Project](#)
- [Adding NUI Library Files to a Project](#)
- [Headers and Library Files for PC Executables](#)

Adding NUI Header Files to a Project

- Open an existing Xbox 360 solution in Visual Studio.
- Add the NUI header files the top of the main header file for the existing project, such as:

```
#include <NuiApi.h>
```
- Include any other essential NUI header files that aren't already included by NuiApi.h.

NuiApi.h

NuiApi.h is the main NUI header file, which adds support for most of the NUI components to any implementation file that includes this header file. NuiApi.h includes most of the NUI header files.

Header File	Description	Library File
NuiApi.h	Initializes Kinect with the subsystems you specify, and includes most of the NUI headers.	NuiApi.lib and ST.lib

Headers Included by NuiApi.h

Do not directly include the following NUI header files, which NuiApi.h already includes.

Header File	Description	Library File
NuiDiagnostics.h	Displays the Kinect Troubleshooter in the Xbox Guide.	NuiApi.lib
NuiFitnessApi.h	Tracks a player's physical fitness data.	NuiApi.lib
NuiGesture.h	Performs run-time gesture detection by using the data generated with the Visual Gesture Builder tool.	NuiApi.lib
NuiHeadOrientation.h	Provides lightweight head orientation tracking without requiring skeleton tracking.	NuiApi.lib and ST.lib

NuiHeadPosition.h	Provides lightweight head position tracking without requiring skeleton tracking.	NuiApi.lib and ST.lib
NuiIdentity.h	Identifies players and enrolls them in the list of tracked players.	NuiApi.lib and ST.lib
NuiImageCamera.h	Processes color and depth image data from the Kinect Sensor.	NuiApi.lib and ST.lib
NuiSkeleton.h	Tracks skeleton joint positions of players based on color and depth images from the Kinect Sensor.	NuiApi.lib and ST.lib
NuiSpeech.h	Performs speech recognition using the Kinect microphone array.	NuiSpeech.lib and NuiAudio.lib
NuiWave.h	Detects the wave gesture, which is also used in the Xbox Dashboard and Guide.	NuiApi.lib

Headers Not Included by NuiApi.h

If your title needs any of the following functionality, include the appropriate headers.

For example, if you use the NUI Handles library, also include NuiHandles.h, which isn't included by NuiApi.h. The NUI handles code is in a separate library so that if a title doesn't use the NUI Handles library, the title can omit this library.

Header File	Description	Library File
NuiAudio.h	Performs multichannel echo cancellation for the microphone array in the Kinect Sensor, to support speech recognition.	NuiAudio.lib
NuiHandles.h	Enables hand movements to manipulate UI controls such as buttons, scroll bars, and menus.	NuiHandles.lib

See also [Natural User Interface Reference](#).

NUI Headers in the ATG Framework

There are additional classes and methods defined by the ATG Framework that are not defined by NuiApi.h or the other main NUI header files. If these additional API items are used, you will need to specify additional `#include` statements in your project.

For a list of the specialized NUI header files that are provided in the ATG Framework, see [Natural User Interface](#) in *ATG Framework for Samples*. For an example of modifying an XDK sample that uses the ATG framework, see [Quick-Start Tutorial for Kinect Programming](#).

Adding NUI Library Files to a Project

- In Solution Explorer, right-click the default project, then select **Properties**.
The Property Pages dialog box appears.
- On the left, expand **Configuration Properties: Linker**, then select the **Input** property page.
- (Visual Studio 2010) Click to the right of **Additional Dependencies**, click the drop-down arrow, and then select: **<Edit...>**
The Additional Dependencies dialog box appears.
- Scroll to the bottom of the list and add the NUI library files for the subsystems that your project uses.
For example, to add the library files for NUI API (debug version) and skeleton tracking (debug version), add:

```
NuiApid.lib
Std.lib
```
- Click **OK** to close the dialog boxes.

List of NUI Library Files

The file with the d suffix is the Debug version.

Library File	Description
NuiApi.lib, NuiApid.lib	Main library file. Covers: - NUI color image camera and depth image camera. - NUI system initialization. - Wave gesture recognition. - Most Fitness tracking API items. - Tracks a player's head orientation.
ST.lib, STd.lib	The Skeleton Tracking library.
NuiFitnessApi.lib	Functions that return fitness tracking data. Other Fitness API items are in NuiApi.lib.
NuiHandles.lib, NuiHandlesd.lib	Graphical UI controls for NUI.
NuiSpeech.lib, NuiSpeechd.lib	NUI speech recognition and audio.
NuiAudio.lib	Multichannel echo cancellation for the Kinect microphone array, to support speech recognition.

Headers and Library Files for PC Executables

These files are for use in tools on the development computer, not for titles on the Xbox 360 console.

Headers

Header File	Description	Library File
NuiTools.h	For use in Win32-based tools that manipulate previously captured NUI data.	NuiApi.lib and ST.lib
XedFile.h	Operates on the Xbox event data (XED) files created by the Xbox Studio tool.	XedFile.lib
XStudio.h	Records and plays back the XED files used by the Xbox Studio tool.	XStudio.lib

Library Files

Library File	Description
XedFile.lib	Operates on the Xbox event data (XED) files created by the Xbox Studio tool.
XStudio.lib	Records and plays back the XED files used by the Xbox Studio tool.

See Also

[Natural User Interface: Programming Guide](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Kinect Database Files to Include on the Game Disc



Include the appropriate Kinect database files on your game disc.

Sections

- [Required Database Files](#)
- [Including the Correct Version](#)
- [Speech Databases](#)
- [List of All Database Files](#)
- [Deploying NUI Database Files During Development](#)

Required Database Files

All Kinect titles require the skeleton-tracking database file `Database.xmlr` or the head position tracking database `HeadPosition.xmlr`, or both. `Database.xmlr` is for skeleton tracking only. `HeadPosition.xmlr` is for head-position tracking only. If your title uses skeleton tracking and head-position tracking, include both files on the game disc.

All Kinect titles require the Identity database file `NuiIdentity.bin.be`, to keep track of specific players.

Including the Correct Version

Kinect relies on database files to translate information from the Kinect sensor array into data understood by a game. Because these database files are tightly coupled to a particular Kinect software release, you must include the version of the database files on each game disc that matches the version of the Kinect link libraries included in the game title.

You can continue to rely on the database files installed on your Xbox 360 Development Console's hard drive, but we recommend you test titles from game disk images that contain the required versions of the database files. This ensures that the tested disk image is the same as the disk image that will be submitted for certification. Add the necessary database files to your game disc using the [Game Disc Layout Editor](#) tool. Click the **Tools** menu, and then **Kinect Options**.

Speech Databases

Kinect titles that use speech recognition need to include the speech exemplar database for each supported language, such as English.

Each speech database stores an acoustic model for speech recognition, and corresponds to a [NUI_SPEECH_LANGUAGE](#) enumeration member. Since each speech database is specific to a single language, you need only include the database files for the languages supported by your title. For example, if your title only supports English and French, you don't need to include the Spanish database on the game disc.

List of All Database Files

Feature	File name	Size
Skeletal tracking database	database.xmlr	12.1 MB
Head position tracking database	HeadPosition.xmlr	828 KB
Head orientation tracking database	NuiHeadOrientation.bin.be	2.60 MB
Identity database	NuiIdentity.bin.be	3.12 MB

Speech Database Files

Language Code (Language - Country)	File name	Size
de-AT (German - Austria)	nuisp3079	880 KB
de-CH (German - Switzerland)	nuisp2055	880 KB
de-DE (German - Germany)	nuisp1031	860 KB
en-AU (English - Australia), also covers en-nz (English - New Zealand)	nuisp3081	1.46 MB
en-GB (English - United Kingdom), also covers en-ie (English - Ireland)	nuisp2057	1.41 MB
en-US (English - United States), also covers en-ca (English - Canada)	nuisp1033	996 KB
es-ES (Spanish - Spain)	nuisp3082	1.18 MB
es-MX (Spanish - Mexico)	nuisp2058	1.11 MB
fr-CA (French - Canada)	nuisp3084	1.48 MB
fr-CH (French - Switzerland)	nuisp4108	1.42 MB
fr-FR (French - France)	nuisp1036	1.44 MB
it-IT (Italian - Italy)	nuisp1040	1.28 MB
ja-JP (Japanese - Japan)	nuisp1041	1.33 MB
pt-BR (Portugese - Brazil)	nuisp1046	860 KB

See also [Local Language Support with NUI Speech](#).

For confidence thresholds for each language-country combination, see [Developing NUI Speech Grammars](#).

For the list of language-country combinations in the API, see [NUI_SPEECH_LANGUAGE](#).

Deploying NUI Database Files During Development

To deploy these files to the console during development, you can add the XDK NUI Databases directory to the Deployment Files property of the project. This causes the `Database.xmlr` and `NuiIdentity.bin.be` database files to be deployed from the XDK NUI Databases directory to the Xbox console when the project is run. This approach is used in the [Quick-Start Tutorial for Kinect Programming](#).

1. In Solution Explorer, right-click the default project, then select **Properties**.
The Property Pages dialog box appears.
2. On the left, expand **Configuration Properties: Console Deployment**, then in Visual Studio 2010, select the **Copy To Hard Drive** property page.
3. Click to the right of **Deployment Files**, click the drop-down arrow, then click **<Edit...>**.
The Deployment Files dialog box appears.
4. In the **Source Files (From PC)** column, double-click an empty row, and type: `$(XEDK)\Redist\xbox\NaturalInput\Databases`
`$(XEDK)` will resolve to the XDK installation path on the development machine.

- In the same row, double-click in the **Target Media Path** column, and type: **\$(RemoteRoot)**
Then press the **Tab** key or click outside the cell to accept the value.
- Click **OK** to close the dialog boxes.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface: Programming Guide

[See Also](#)



This section describes how to use Natural User Interface (NUI) software to develop games for the Xbox 360 console based on Kinect .

In This Section

[NUI Architecture](#)

Hardware, subsystem libraries, and tools for developing Kinect titles.

[NUI Resource Usage](#)

Initialization flags select which NUI subsystems to include, to save resources.

[NUI Fitness Tracking](#)

Tracks the amount of energy expended by a player, based on the movement of a tracked skeleton.

[NUI Gesture Detection](#)

Enables a data-driven approach to gesture detection at runtime.

[NUI Handles](#)

Enables hand movements to manipulate UI controls such as buttons, scroll bars, and menus.

[NUI Head Tracking](#)

Indicates the location and direction of the player's head.

[NUI Identity](#)

The NUI Identity system enables player identification and enrollment in the list of tracked players, to differentiate between detected skeletons and to associate the skeletons with gamer profiles, while enabling casual jump-in play.

[NUI Image Camera](#)

Enables processing color and depth image data from the Kinect Sensor, and modifying the sensor settings.

[NUI Skeleton Tracking](#)

Enables skeleton tracking of player joint positions based on color and depth images from the Kinect Sensor.

[NUI Speech](#)

Enables speech recognition with the Kinect microphone array.

[NUI Wave Gesture Detection](#)

Enables detecting the wave gesture, which is also used in the Xbox Dashboard and Guide.

[NUI Coordinate System Transformations](#)

Coordinate transformations for image frames and skeleton tracking.

[NUI Development Tools](#)

Enables the creation of development tools.

Related Topics

In [Xbox 360 Technical Certification Requirements](#), see the section *Natural User Interface (NUI)*.

In *Notification Identifiers*, in the [System Notifications](#) section, see the XN_SYS_NUI notification IDs.

+ See Also

[Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Architecture

[See Also](#)

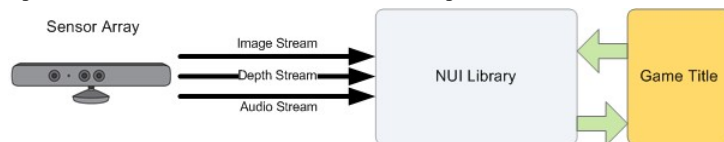


Natural User Interface (NUI) is a revolutionary step forward for interaction between people and computers that changes the way that players experience video games. By monitoring the human voice and body for speech, movement, and gestures, video games become more immersive, more real, and more exciting. Through this new technology, developers can revitalize existing genres of games and create new ones.

NUI combines hardware, software libraries, and tools to create this rich form of natural input, that senses real-world events in a way that was not possible before in a video games.

The NUI hardware and software library interact with your game as shown in Figure 1.

Figure 1. NUI hardware and software interaction with a game.



✓ Note

The Xbox 360 Development Kit (XDK) is currently the best environment for creating Xbox 360 and Kinect titles. Although the Microsoft Research group (MSR) has developed a noncommercial software development kit (SDK) to enable academic researchers and hobbyists to develop titles for Microsoft Windows that use the Xbox 360 Kinect Sensor, that SDK should not be used to develop prototype titles with the intent of porting those titles to the Xbox 360 console. There are numerous architectural, performance, and behavioral differences between Xbox 360 and Windows. These differences affect how the Kinect technology is implemented for Windows 7. As a result, design decisions and performance characteristics are often very different between the two platforms. Microsoft recommends using the XDK to create Xbox 360 titles. The descriptions in this documentation refer only to the XDK.

NUI Header Files

Games that use the NUI APIs must include the NuiApi.h header file in game code. That file includes most other NUI header files automatically. The header files that define some of the NUI component APIs, such as NuiIdentity.h, should not be included directly in game code, and in fact code will not compile if it includes those files explicitly.

To add NUI header files to a project, see [Adding NUI Header and Library Files](#). For a list of header files, see [Natural User Interface Reference](#).

NUI Library Components

The NUI library consists of the following components:

- [Audio Acquisition with Echo Cancellation \(NuiAudio\)](#)
- [Image Retrieval and Camera Control \(NuiImageCamera\)](#)
- [Skeleton Tracking \(NuiSkeleton\)](#)
- [Player Identity \(NuiIdentity\)](#)
- [Speech Input and Control \(NuiSpeech\)](#)
- [User Interface \(NuiHandles\)](#)

These components are described in the following sections.

Audio Acquisition with Echo Cancellation (NuiAudio)

Game code calls the [NuiAudio](#) API to acquire audio from the Kinect Sensor. NuiAudio fetches data from the Kinect Sensor and provides echo cancellation and processing to remove console audio from the audio stream.

Because the processing for NuiAudio is performed by the NUI library, it does not depend on system updates that may have been applied to a console, providing consistent audio performance that is important for titles that perform sensitive audio processing.

NuiAudio only works with the Kinect Sensor. For more information about using NuiAudio, see [Multichannel Echo Cancellation with NuiAudio](#).

Image Retrieval and Camera Control (NuiImageCamera)

Game code calls the [NUI Image and Camera API](#) to retrieve data from the color image and depth sensors in the Kinect sensor array. The data streams available from the sensor include color image data, depth data, and player segmentation data, which isolates the player's image from the background. Game code can control frame rate and exposure to control image quality directly, or activate a mode that automatically maintains exposure and white balance. Game code can also adjust the tilt of the sensor array using the motorized tilt mechanism in the sensor array's base to ensure that players are fully visible to the sensor.

For information about using this API, see [NUI Image Camera](#).

Skeleton Tracking (NuiSkeleton)

Game code calls the [NUI Skeleton API](#) to retrieve information about players standing in the field of view of the NUI sensor array. The API provides basic information about the location of up to six players, and details of the position and orientation of skeleton joints for up to two of those players. The skeletal information is represented as a defined set of *skeleton positions* (for example, wrists, elbows, hips, and head), with each skeleton position having a three-dimensional coordinate in space.

This skeleton data represents the player's current position and pose. Game code can use this information as input to control the player's in-game character (as when avatar retargeting with the [IXAvatarNuiMapper Interface](#)) or directly as input for the game's user interface.

For information about using this API, see [NUI Skeleton Tracking](#).

Coordinate Transformations

The NUI API provides easy methods for converting coordinate values between the world coordinates used by game code and the coordinate systems used for data coming from the sensor array.

For more information, see [NUI Coordinate System Transformations](#). The transform functions themselves are in [NuiSkeleton.h](#).

Player Identity (NuiIdentity)

Game code calls the [NUI Identity API](#) to retrieve information that identifies players by their skeleton data. The NUI Identity subsystem remembers players even when they are not visible to the sensor array. This enables players to take turns playing without having to identify themselves every time they return to the play area. NUI Identity also facilitates signing recognized players in to the Xbox LIVE service.

For information about using this API, see [NUI Identity](#).

Speech Input and Control (NuiSpeech)

Game code calls the [NUI Speech API](#) for speaker-independent recognition of words spoken by players, which enables the game to accept voice commands. The list of recognized words is determined by rules that can be changed dynamically as game state changes, so that voice command inputs can be adjusted for the current context of the game.

For information about using this API, see [NUI Speech](#).

User Interface (NuiHandles)

Game code calls the [NUI Handles API](#) to implement user interface elements that players manipulate with their hands. Handles can be configured to respond to hand motions only when they are made in certain directions, or when the hand stays stationary for a specific amount of time. Handles can be combined to implement more sophisticated user interfaces.

For information about using this API, see [NUI Handles](#).

See Also

- [Specifications of the Kinect Sensor](#)
- [Natural User Interface Tools](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Resource Usage

[See Also](#)



The Natural User Interface (NUI) libraries consume resources when you use them in your game, such as CPU time, GPU time, and memory. The resource cost depends upon what specific NUI features your game code uses.

The NUI libraries use the standard [XMemAlloc](#) function to allocate memory, using the *dwAllocatorId* value [eXALLOCALlocatorId](#). [NuiApi](#).

Note

The Xbox 360 development team is continually working to minimize resource consumption, and the figures on this page may not reflect the latest, optimized resource costs. You should use the figures on this page for project planning purposes, but then test resource consumption in your title to determine actual costs.

Sections

- [Color Image Stream](#)
- [Depth Image Stream](#)
- [Color Image Stream, Depth Image Stream, and Skeleton Tracking](#)
- [Depth Image Stream Plus Skeleton Tracking](#)
- [Lightweight Head Position Tracking](#)
- [Head Orientation Tracking](#)
- [NUI Identity](#)
- [NUI Speech](#)
- [NUI Handles](#)

- [XStudio](#)
- [Related Topics](#)

Color Image Stream

These figures are for "Low-level Data Only" Access Mode.

RGB:

Resource Type	Amount
CPU	0 ms
GPU	At most 0.5ms
Memory	1MB + 3.6MB for output textures (3 buffers 640x480xRGBA)

Registered RGB:

Resource Type	Amount
CPU	0 ms
GPU	At most 0.7ms
Memory	1MB + 3.6MB for output textures (3 buffers 640x480xRGBA)

Depth Image Stream

These figures are for "Low-level Data Only" Access Mode

Depth:

Resource Type	Amount
CPU	0 ms
GPU	At most 0.5ms
Memory	2MB + 550KB for output textures (3 buffers 384x240xD16)

Registered Depth:

Resource Type	Amount
CPU	0 ms
GPU	At most 0.5ms
Memory	550KB for output textures (3 buffers 384x240xD16)

Color Image Stream, Depth Image Stream, and Skeleton Tracking

Resource Type	Amount
CPU	CPU and GPU usage does not increase when color images are used. The base resource consumption (CPU/GPU) includes time to process color image stream data.
GPU	See the preceding row.
Memory	The following figures include memory allocations for the depth camera and skeleton tracking. Using one or more color image streams cause additional system memory allocations. Calls to NuiImageStreamOpen include a value specifying the number of buffers to allocate for each image stream (depth or color), and also specify flags indicating which image streams will be used. The implementation of NuiImageStreamOpen makes two memory allocations for each image stream in use: • The first memory allocation is a single block large enough to hold the requested number of image buffers for the image stream (depth or color). This allocation is about the size of one frame buffer multiplied by the number of buffers requested. • The second memory allocation holds a small implementation-specific block per buffer used to manage the image buffers. This allocation is less than 1 KB per requested buffer, multiplied by the number of buffers requested.

Depth Image Stream Plus Skeleton Tracking

This section lists the total resources used if you use both the depth image stream and skeleton tracking.

Resource Type	Amount
CPU	No more than 10 ms per 33 ms frame on one hardware thread for depth image stream and skeleton data processing. Game code specifies which hardware thread is used. Depth image stream processing and skeleton tracking should use a different hardware thread than for NUI Speech.
GPU	No more than 3.3 ms per 33 ms frame, for depth image data and skeleton data.
Memory	No more than 48 MB of RAM for basic depth image and skeleton tracking data. See also NUI_INITIALIZE Flags .

Lightweight Head Position Tracking

[Lightweight head position tracking](#) requires the following resources. CPU and GPU costs assume a title running at 30fps. The lightweight head position tracking detector runs at 30fps at all times.

Resource Type	Amount
CPU (single thread)	0.5 ms
GPU	0.176 ms (average), 0.100ms (minimum), 0.224ms (maximum)
Memory	950 KB

If you use skeleton tracking for head tracking, see the resources used for skeleton tracking.

Head Orientation Tracking

[Head orientation tracking](#) requires the following resources. CPU and GPU costs assume a title running at 30 fps. Orientation is assumed to be running at 15 fps.

Resource Type	Amount
CPU (single thread)	9 ms per player. The CPU cost is ammortized over multiple frames. For example, if the title is running at 30 fps and the color feed is running at 15 fps, the 9 ms cost will be split over two title frames.

GPU	0 ms
Memory	4.7 MB

NUI Identity

Resource Type	Amount
CPU	Calling the NUI Identity functions NuiIdentityIdentify or NuiIdentityEnroll causes the NUI Identity system to use some CPU time. The time required averages 2 ms per frame for the duration of the operation. Also, these functions may steal frames from the color image stream while performing the requested operation. The time required to complete the operation depends on several factors, including camera exposure settings, room lighting, and the player's location and pose.
GPU	0 ms
Memory	The memory used for Identity comes out of the 48MB skeletal tracking budget. There is no additional memory cost to the title.

NUI Speech

The resources used by the NUI Speech engine vary depending on which language (acoustic model) being recognized, the number of words in loaded grammars, and how many of those words are active. The figures listed here represent maximum usage.

CPU

A 100-word grammar consumes no more than 5.5 ms of CPU per 33 ms frame on one hardware thread. Using fewer active words reduces the CPU used, though the exact reduction depends on the specific words involved. Ten active words would typically cost about 5.2–5.3 ms per frame.

Game code specifies which hardware thread is used. NUI Speech should use a different hardware thread than for depth image processing and skeleton tracking.

Memory

The NUI Speech engine allocates RAM in these instances:

- Memory when you call [NuiSpeechEnable](#).
- More memory when you load a grammar.
- More memory when you call [NuiSpeechStartRecognition](#).

The total of all of these allocations does not exceed 6 MB for a 100-word grammar. Your actual consumption will depend on the number of words in your grammar and the acoustic model in use (that is, which language you are recognizing).

Disc Space Requirements

Acoustic models and other required files require disc space. See [Kinect Database Files to Include on the Game Disc](#).

Recommendations

To reduce the resources required when using NUI Speech, follow these guidelines.

- Limit grammars to about 10 active words or phrases at any given time. Using more rules increases the memory footprint and CPU requirements of the engine.
- Use static grammars. Dynamically created grammars incur the memory hit of having the entire acoustic model loaded all the time, and also incur the performance hit of having to prepare the grammar at run time.
- Run speech processing on a separate hardware thread from the one the main game thread runs on. While speech recognition doesn't consume much CPU time, some NUISpeech calls can block for unpredictable times, and might cause occasional frame-out if they are made on the main game thread. If you move all the speech code onto a separate hardware thread that communicates with the main game through message queues, you can avoid such problems. To specify the hardware thread used by NUI Speech, set the *dwHardwareThread* argument to [NuiSpeechEnable](#) appropriately.

For more information about performance and memory implications of grammar size and number of active rules, see [Silver Tonque: Best Practices for Speech Development with Kinect for Xbox 360](#).

Memory Allocation for Each Grammar at Each Stage

The following table shows the following, for each grammar, and for the different run modes (.cfgp versus .cfg):

- The file size on disc.
- The memory allocation used for the various stages of speech recognition setup. The 3 function name columns represent how much additional memory each step costs. The **Total Max** column shows the highest total memory for speech measured during testing.

[NuiSpeechEnable](#) uses 3.5 MB without anything being active. This is because when you call **NuiSpeechEnable**, an engine thread is started and parts of the model for the language are loaded. So a best practice is to call [NuiSpeechDisable](#) after you've unloaded your grammars, rather than just calling [NuiSpeechStopRecognition](#) when not engaged.

Some grammars or languages are much larger than others for load grammar. For example, 1.6 MB for Australia versus 768 KB US for essentially the same grammar. When talking about grammars it is only useful to compare .cfg to .cfg and .cfgp to .cfgp. The main difference in sizes is due to the language model lexicon.

Grammar	File Size	Enable	Load Grammar	Start Recog.	Total Max
medieval_en_us.cfgp	4300 bytes	1488 KB	0 KB	3240 KB	4728 KB
medieval_fr_ca.cfgp	4548 bytes	3540 KB	64 KB	1124 KB	4728 KB
medieval_en_gb.cfgp	3576 bytes	3540 KB	64 KB	1060 KB	4664 KB
medieval_es_mx.cfgp	4096 bytes	3476 KB	64 KB	1060 KB	4600 KB
medieval_ja_jp.cfgp	3612 bytes	3412 KB	0 KB	996 KB	4408 KB
medieval_en_au.cfgp	3532 bytes	3540 KB	64 KB	1124 KB	4728 KB
medieval_fr_fr.cfgp	4548 bytes	3476 KB	64 KB	1188 KB	4728 KB
medieval_de_de.cfgp	3512 bytes	3604 KB	64 KB	1188 KB	4856 KB
medieval_it_it.cfgp	4404 bytes	3540 KB	64 KB	1124 KB	4728 KB
medieval_es_es.cfgp	4096 bytes	3476 KB	64 KB	1060 KB	4600 KB
medieval_en_us.cfg	640 bytes	3540 KB	768 KB	1188 KB	5496 KB
medieval_fr_ca.cfg	1288 bytes	3540 KB	1472 KB	1124 KB	6136 KB
medieval_en_gb.cfg	1200 bytes	3540 KB	1536 KB	996 KB	6072 KB
medieval_es_mx.cfg	1304 bytes	3476 KB	1024 KB	1124 KB	5624 KB
medieval_ja_jp.cfg	2316 bytes	3412 KB	0 KB	996 KB	4408 KB
medieval_en_au.cfg	1116 bytes	3540 KB	1600 KB	996 KB	6136 KB
medieval_fr_fr.cfg	1288 bytes	3476 KB	1408 KB	1188 KB	6072 KB

medieval_de_de.cfg	1132 bytes	3604 KB	704 KB	1124 KB	5432 KB
medieval_it_it.cfg	1296 bytes	3540 KB	1408 KB	1124 KB	6072 KB
medieval_es_es.cfg	1236 bytes	3476 KB	1216 KB	996 KB	5688 KB

NUI Handles

Resource Type	Amount
CPU	The NuiHandlesArmsUpdate function takes 50 to 100 microseconds per call, tracking both arms of one player.
GPU	0 ms
Memory	Game code allocates all memory for the NUI Handles API.

XStudio

Resource Type	Amount
CPU	0 ms
GPU	0 ms
Memory	When you run your game on an Xbox 360 Development Kit (XDK), the system software allocates memory buffers to store the data reported to XStudio. The exact amount of memory allocated depends on what data you are recording at the time, and can be enough to impact the amount of RAM available to your title.

Related Topics

These requirements apply in addition to the standard Xbox 360 system resource reservation. For more information about the system reservation, see the white paper [Xbox 360 System Resources Reservation](#).

See Also

[NUI_INITIALIZE Flags](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Fitness Tracking

[See Also](#)



The NuiFitness API tracks energy expended by a player, based on the movement of the tracked skeleton. The amount of energy expended is calculated using metabolic equivalent of task (MET) values, basal metabolic rate (BMR), and duration of activity.

In automatic fitness tracking mode, NuiFitness calculates the amount of energy expended based on the movement of tracked skeletons along with internal equations in the NuiFitness library.

In manual fitness tracking mode, NuiFitness uses values supplied by the title to calculate the amount of energy expended. This can be MET or calorie values. Using manual mode, a title can provide data tailored to activities that may be difficult to calculate solely from skeletal movement, such as yoga or activities using weights.

Sections:

- [Initializing NuiFitness](#)
- [Starting and Stopping Fitness Tracking](#)
- [Pausing and Resuming Fitness Tracking](#)
- [Recording Values When Using Manual Fitness Tracking](#)
- [Retrieving Fitness Tracking Data](#)
- [Setting and Retrieving Player Body Profile Data](#)
- [Privacy Settings for Fitness Data](#)

Initializing NuiFitness

To enable the NuiFitness API, call the [NuiInitialize](#) function with the NUI_INITIALIZE_FLAG_USES_FITNESS flag.

```
hr = NuiInitialize(NUI_INITIALIZE_FLAG_USES_SKELETON |
    NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
    NUI_INITIALIZE_FLAG_USES_COLOR |
    NUI_INITIALIZE_FLAG_USES_FITNESS,
    NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD );
```

Starting and Stopping Fitness Tracking

To start fitness tracking, separately call [NuiFitnessStartTracking](#) for each skeleton that needs to be tracked. The *TrackingMode* argument controls whether automatic fitness tracking (specified with NUI_FITNESS_TRACKING_AUTO) or manual fitness tracking (specified with NUI_FITNESS_TRACKING_MANUAL) will be used.

```
hr = NuiFitnessStartTracking( trackingMode, trackingID, userIndex );
```

To stop fitness tracking, call [NuiFitnessStopTracking](#) for each tracked player.

```
NuiFitnessStopTracking( trackingID );
```

Pausing and Resuming Fitness Tracking

To pause and resume automatic fitness tracking, call [NuiFitnessPauseTracking](#) and [NuiFitnessResumeTracking](#). Once [NuiFitnessPauseTracking](#) has been called, no additional fitness tracking data will be accumulated for the specified skeleton until [NuiFitnessResumeTracking](#) is called.

To pause manual fitness tracking, simply avoid calling the **Record...Value** functions while the game is paused. Resume calling the **Record...Value** functions whenever activity has resumed. [NuiFitnessPauseTracking](#) and [NuiFitnessResumeTracking](#) are not used for manual fitness tracking and should not be called for skeletons that are being manually tracked.

Recording Values When Using Manual Fitness Tracking

To use manual fitness tracking, periodically call a **Record...Value** function to record the value for player activity during a preceding interval. This value can be the calories or metabolic equivalent of task (MET) expended.

To record calorie values, call [NuiFitnessRecordCalorieValue](#). To record MET values, call [NuiFitnessRecordMETValue](#).

The *DurationInMS* argument specifies the duration of the activity, in milliseconds. The recording interval should always be the most recent *DurationInMS* before the time the function is called.

Retrieving Fitness Tracking Data

The fitness tracking data includes the energy expended, the average metabolic equivalent of task (MET), and the duration of the activity.

To get a player's accumulated fitness tracking data for the current session of using the current title, call [NuiFitnessGetCurrentFitnessData](#). For example, a title could implement a real-time fitness display by periodically calling this function and using the results returned in the *pFitnessData* argument to update displayed values.

```
NuiFitnessGetCurrentFitnessData( trackingID, pFitnessData );
```

To get the fitness tracking data for the specified player across all the player's sessions of using the current title, call [NuiFitnessGetCurrentTitleSummary](#). The total duration of fitness tracking for the player in the title is returned, along with the time stamp of when the player last used fitness tracking in the title.

To get the fitness tracking data for the specified player across all the player's sessions of using the current title, optionally broken out by day, month, or year, call [XFitnessQueryCreateEnumerator](#), passing an [XFITNESS_QUERY_PARAMETERS](#) structure.

To get the fitness tracking data for the specified player across all the player's sessions of using all titles, call [NuiFitnessGetOverallSummary](#). The total duration of fitness tracking for the player across all sessions in all titles is returned, along with the time stamp of when the player last used fitness tracking in any title.

Setting and Retrieving Player Body Profile Data

The Height, Weight, Age and Gender (HWAG) data for a player can be retrieved and set by calling [NuiFitnessGetBodyProfileRecord](#) and [NuiFitnessSetBodyProfileRecord](#).

To invoke the standard NUI-based UI for setting and updating a player's HWAG data, call [XShowNuiFitnessBodyProfileUI](#).

To invoke the standard controller-based UI for setting and updating a player's HWAG data, call [XShowFitnessBodyProfileUI](#).

Privacy Settings for Fitness Data

The player can set how widely to reveal their tracking fitness data, such as only locally on the Xbox console, only to friends on Xbox LIVE, or to anyone on Xbox LIVE. Privacy levels are listed in the [NUI_FITNESS_PRIVACY_SETTINGS](#) enumeration.

To display a controller-based UI for the player to change their privacy settings, call [XShowFitnessPrivacySettingsUI](#). To display a Kinect-based UI for the player to change their privacy settings, call [XShowNuiFitnessPrivacySettingsUI](#). To get the current privacy settings for fitness data for a player, call [NuiFitnessGetPrivacySettings](#).

See Also

[NuiFitness Reference](#)
[Running Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

NUI Gesture Detection

[See Also](#)



The Visual Gesture Builder (VGB) tool provides an Interactive Development Environment (IDE) for gesture tagging. The tagging information is used to generate a database that you can integrate in your title to perform gesture detection at runtime. By using a data-driven model, the VGB tool shifts the emphasis from code writing to content producing. This simplifies the developer's task for gesture detection.

The following are some of the results of using the VGB tool:

- Mitigating the current code-intensive approach.
- Reusing gesture data in different versions of the same title.
- Sharing gesture data among titles.
- Creating a library of gesture data.
- Improving predictability and reliability of gesture data.
- Testing framework for gesture detection.
- Analyzing and visualizing gesture-detection results.
- Managing a large quantity of data in a user friendly IDE.

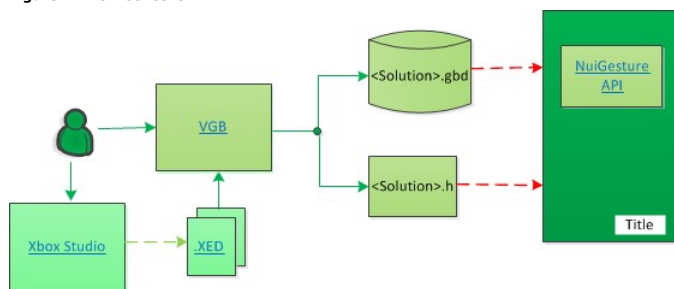
The VGB tool produces a gesture database and a related header file. By using the [NuiGesture Reference](#) API and the auto generated header file, you can integrate the database in your title.

Note

Before using the tool for gesture tagging, you must record a set of Xbox Studio (.xed) files that contain at least depth and skeleton information of several users performing the gesture.

The following illustration shows the context for the VGB tool.

Figure 1. VGB Context



Using the Gesture Database

The following steps show how to use the gesture database in your title.

1. Load the database in memory and initialize the gesture detector. At the time you initialize your gesture detection, you must make a call similar to the one shown next.

```
HRESULT result = NuiGestureInitialize("C:\\MyMedia\\Gestures\\Golf.gbd", Golf::DWORD DIGEST , 0, NULL );
```

The parameter *Golf::DIGEST* is defined in the auto generated *Golf.h* file. For more information, see [NuiGestureInitialize](#).

2. Enable the gestures that the title must handle. The following is an example of how to do it.

```
HRESULT result = NuiGestureEnableGesture(Golf::GESTURE_GOLFSWING, TRUE);
```

The parameter *Golf::GESTURE_GOLFSWING* is defined in the auto generated *Golf.h* file. For more information, see [NuiGestureEnableGesture](#).

3. Initialize the gesture frame structure. The following is an example of how to do it.

```
Golf::FRAME m_GolfGestureFrame;  
NUI_GESTURE_FRAME m_GestureFrame;  
Golf::INITIALIZE_NUI_GESTURE_FRAME(m_GolfGestureFrame, m_GestureFrame);
```

The structure `Golf::FRAME` and the function `Golf::INITIALIZE_NUI_GESTURE_FRAME` are defined in the auto generated *Golf.h* file. `NUI_GESTURE_FRAME` is defined in *NuiApi.h*. For more information, see [NUI_GESTURE_FRAME](#).

- Use the gesture data in real-time. The following is an example of how to get the next frame from the gesture detector stream.

```
HRESULT result = NuiGestureGetNextFrame( 0, m_GestureFrame);
```

For more information, see [NuiGestureGetNextFrame](#).

Migrating to New SDK Version

You can convert the .XED files to a new version of the SDK by using the [Xed Skeleton Updater](#) tool.

See Also

[Visual Gesture Builder](#)
[NuiGesture Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

NUI Handles

New forms of input provided by the Kinect sensor array, such as voice commands and gestures, enable new control schemes for Xbox 360 designers and developers. The NuiHandles API provides support for implementing user interface controls like buttons, scroll bars, and menus, using hand movement to manipulate simple graphical elements referred to as handles. Games can implement a custom user interface system that combines graphical elements rendered by the game with tracking and control logic from the NuiHandles API, to interpret simple player hand movements as manipulations of the graphical elements.

Navigating game menus and settings using data from the Kinect sensor array is a new challenge for Xbox 360 developers. With access to the player skeleton, a game has a rich source of real-time movement data that can be used for control. Using skeleton data and the NuiHandles API, game developers can provide a custom user interface that combines graphical elements rendered by the game with tracking and control logic from the NuiHandles API, to interpret simple player hand movements as manipulations of the graphical elements.

To use the [NuiHandles API](#), include NuiHandles.h in your game code.

Basic Usage of Handles

In the context of NuiHandles, a handle is the visual representation of a means to manipulate game state in some way. The programming abstraction implementing a handle is known as a *grip*. Game code manipulates grips and their components when responding to a player's manipulation of a handle onscreen.

Grips have one or more constraints that define what hand motions will be interpreted as input for the handle. A grip's motion constraints can be as simple as a single linear rail—move left to go left, move right to go right. On the other hand, a grip can combine multiple constraints to provide more complex input, such as scrolling menus.

When a user is actively manipulating a handle, the game determines progress along the handle's rails and updates the screen appropriately, drawing handle elements and possibly playing sounds or changing other aspects of the game as feedback for the user's movements.

To use a handle, the user places his or her hand in a way that is recognized as demonstrating the intent to manipulate the handle. In a typical gaming setup, where the Kinect sensor array is directly above or below the display screen, the player points toward the part of the screen where the handle is shown. The player is now said to have *attached* his or her hand to the handle.

While the player's hand is attached to the handle, the game can retrieve progress values indicating how the player's hand has progressed along the motion constraints of the handle. The game uses the progress value to update the handle's appearance as well as interpreting that progress as an indication of user choice. For example, moving all the way to the left or right ends of a linear constraint might be interpreted as "scroll left" or "scroll right" commands for a list of items drawn on screen.

A player remains attached to a handle as long as the player's hand remains within a threshold distance of any of the handle's manipulation constraints. The player disengages from the handle (and cancels the user interface interaction) by moving the attached hand away from the handle.

Terminology

The following terms are often used when discussing the Natural User Interface (NUI) Handles component and handle-based user interfaces.

Handle

The visual representation of a means to manipulate game controls using a player's hand movements.

Grip

The programming abstraction implementing a handle. Game code manipulates grips, and their components, when responding to players' manipulation of handles onscreen. A grip has one or more manipulation constraints. A grip can be configured to respond solely to either the left or right hand, or to respond equally to either hand.

Manipulation constraint (rail)

A manipulation constraint defines which player motions are considered valid for purposes of manipulating a given handle. In the API, manipulation constraints are called *rails*.

Circular constraint (analog joystick rail)

A manipulation constraint on a handle that defines an angle of deviation from the handle's grip. Motion away from the grip can be in any direction within the constraint. A circular constraint is implemented by adding an analog joystick rail to the grip using [NuiHandlesGripAddRailAnalogStick](#).

Gearshift constraint (gearshift rail)

A manipulation constraint that allows a handle to move along a branching track that resembles the diagram on the shift lever of a manual automobile transmission. A gearshift constraint is implemented by adding a gearshift rail to the grip using [NuiHandlesGripAddRailGearShift](#).

Line constraint (arc rail)

A manipulation constraint that allows a handle to move between two points along a straight line. A line constraint is implemented by adding an arc rail to the grip using [NuiHandlesGripAddRailArc](#).

Hover constraint (hover timer)

A manipulation constraint that triggers an action by placing a hand over a handle for a specified length of time. A hover constraint is implemented by adding a hover timer to a grip using [NuiHandlesGripAddRailHoverTimer](#).

Attach/attached

A player attaches to a handle by pointing his or her hand directly at the handle's grip. The player remains attached to the grip as long as the player's hand remains within the threshold of any of that handle's manipulation constraints. A player's hand can be attached to only one handle at a time.

Detach/detached

A player detaches from a handle by moving his or her hand beyond the threshold of the handle's manipulation constraints. This action cancels the user interface interaction.

Detachable handle

A handle that allows a player to re-attach after detaching. When the player re-attaches to this kind of handle, the handle's progress value is set to be the same as it was when the player detached.

Dead zone

An area around a manipulation constraint where small changes in hand position are ignored. The shape and size of the dead zone vary with the geometry of the constraint.

Committed

A handle is committed to a particular manipulation constraint when the handle has begun moving within that constraint—that is, the handle is no longer in its initial resting position. A player can be attached to a handle without being committed to a constraint.

Snap point

A point in a manipulation constraint that behaves like a small magnetic field. When a user moves a handle near a snap point, the handle aligns itself ("snaps to") the snap point.

In This Section

[Coordinate Spaces for NUI Handles](#)
[The Physical Interaction Zone \(PHIZ\)](#)
[Manipulation Constraints \(Rails\)](#)

[Implementing a Simple Handle](#)
[Fine-tuning Grip Parameters](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Coordinate Spaces for NUI Handles

[See Also](#)



The NUI Handles API uses several coordinate spaces.

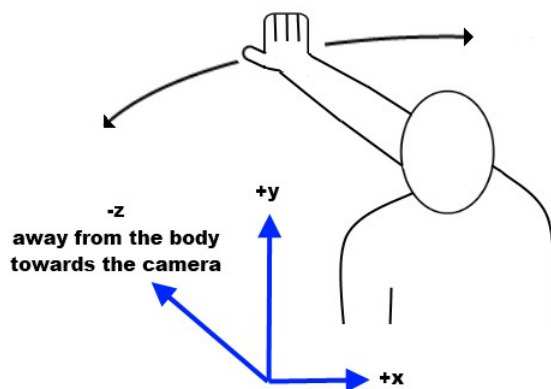
Sections

- [Body Space](#)
- [Grip Space](#)
- [Screen Space](#)

Body Space

Body space is a coordinate system used by the NuiHandles API that is relative to a player's body.

Figure 1. The body space coordinate system



The vertical plane (x- and y-axis) is formed from the following vectors:

- Shoulder vector = right shoulder position – left shoulder position
- Body vector = head position – hip center position

In order to minimize body-induced jitter in the coordinate system, the shoulder and body vectors are a running, weighted average over the last 100 frames. The z-axis is the vector cross-product of the averaged shoulder and body vectors.

The origin of the coordinate system changes is based on the handedness of the arm. If the arm is left-handed (the arm was initialized or set to `NUI_HANDLES_ARMS_HANDEDNESS_LEFT_ARM`), the origin is the left shoulder. If the arm is right-handed (the arm was initialized or set to `NUI_HANDLES_ARMS_HANDEDNESS_RIGHT_ARM`), the origin is the right shoulder.

A position ray is the vector between the origin in body space and some object. The arm's position ray (as returned by [NuiHandlesGripGetPositionRay](#)) is measured from the NuiHandles-refined position of the wrist relative to its corresponding shoulder. The wrist position is refined by the NuiHandles API so as to remove jitter and does not correspond exactly to the wrist joint returned by the skeletal pipeline. Rather, it approximates the position of the palm of the player's hand, so that handle tracking appears to track hand motion.

Grip Space

Grip space is a two-dimensional space that is parallel to the xy plane in body space. Grip space is used to describe the motion of a grip as it would move if the grip space were translated into screen space. The grip space origin is determined by a position ray from the attached arm's shoulder through the attached hand when the hand is first recognized as having attached to a handle. The y-coordinate increases as the user's hand rises vertically, and the x-coordinate increases as the user's hand moves to the right.

Screen Space

Screen space is a two-dimensional coordinate space describing a point on or near the screen. This has its origin at the center of the screen (0.0, 0.0) with X increasing to the right and Y increasing downward. Screen space coordinates are normalized so that the X and Y values of pixels on the screen are in the range [-1..1]. It should be noted that it is possible for some of the functions that deal in screen space coordinates to return values outside the range [-1..1]. Game code should ensure that screen space values outside [-1..1] are handled appropriately.

Screen space coordinates are usually represented as an `XMVECTOR` with a value of 0.0 for the Z and W elements.

See Also

[The Physical Interaction Zone \(PHIZ\)](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

The Physical Interaction Zone (PHIZ)

[See Also](#)



The physical interaction zone (PHIZ) is a defined region of space in front of the player where the player's hand motions are mapped to cursor control and movement on the screen. The NuiHandles API allows you to customize the PHIZ in order to change the shape and size of the space where user movements are considered as input to the handle. For example, you might move the lower edge of the PHIZ up if you anticipate that players will be playing while sitting down, and wouldn't want to reach way down between their knees to manipulate a handle at the bottom of the screen. Or you might make the PHIZ bigger in order that large player movements don't move the handle quickly offscreen.

The NUI Handles library allows games to customize the PHIZ using [NuiHandlesGlobalSetTuningParams](#).

Sections

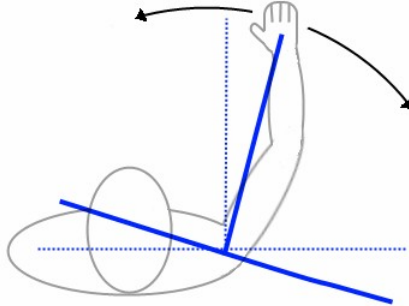
- [Horizontal Offset](#)

- [Width](#)
- [Vertical Offset](#)
- [Height](#)
- [Recommended and Default Values](#)

Horizontal Offset

The horizontal offset allows games to specify a horizontal angle that helps to define the neutral position of the arm. Horizontal movements in the PHIZ are then measured from this neutral position. In the figure, the clockwise rotation from the neutral, dashed-line position to the solid-line position is a positive horizontal offset.

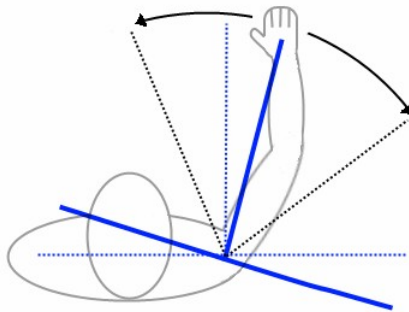
Figure 1. Horizontal offset of the PHIZ



Width

The width of the PHIZ defines the valid horizontal range through which the arm's position ray is allowed to move. When the arm is outside of this range, the Handles library will ignore the arm. In the figure, the width of the PHIZ is represented by the angle between the dashed lines.

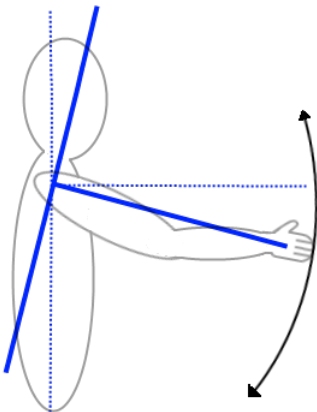
Figure 2. Width of the PHIZ



Vertical Offset

The vertical offset allows games to specify a vertical angle that helps to define the neutral position of the arm. Vertical movements in the PHIZ are then measured from this neutral position. In the figure, the clockwise rotation from the neutral, dashed-line position to the solid-line position is a positive vertical offset.

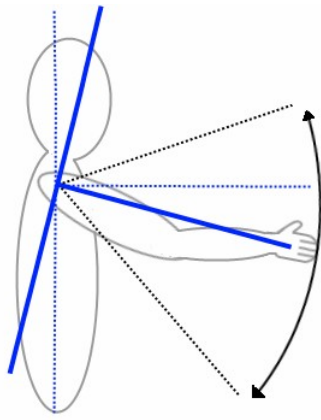
Figure 3. Vertical offset of the PHIZ



Height

The height of the PHIZ defines the valid vertical range through which the arm's position ray is allowed to move. When the arm is outside of this range, the Handles library will ignore the arm. In the figure, the height of the PHIZ is represented by the angle between the dashed lines.

Figure 4. Height of the PHIZ



Recommended and Default Values

Games can retrieve the current PHIZ settings by calling [NuiHandlesGlobalGetTuningParams](#), and they can retrieve the default PHIZ settings by calling [NuiHandlesGlobalGetDefaultTuningParams](#). One potential use for these functions is to retrieve either the current or the standard values into the appropriate structure, change the members of that structure whose values you want to tune, and then pass the structure to [NuiHandlesGlobalSetTuningParams](#).

The default PHIZ is optimized for [seated skeleton tracking](#). The NuiHandles library was designed especially to support menu systems for seated skeleton tracking.

Named constants are provided, to serve as the default and recommended values for the PHIZ size and location, for the seated and standing pipelines. See [NUIHANDLES_SEATED_GLOBAL_PARAMETERS](#) and [NUIHANDLES_STANDING_GLOBAL_PARAMETERS](#).

See Also

[Coordinate Spaces for NUI Handles](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Manipulation Constraints (Rails)



A manipulation constraint defines player motions that are considered valid for purposes of manipulating a given handle. In the API, manipulation constraints are called *rails*.

Sections

- [Linear Constraints with the Arc Rail](#)
- [Circular Constraints with the Analog Joystick Rail](#)
- [Gearshift Constraints with the Gearshift Rail](#)
- [Time-based Constraints with the Hover Timer](#)

Linear Constraints with the Arc Rail

An arc rail implements a manipulation constraint that allows a handle to move between two points. This motion is along a straight line on the screen, or an arc on the surface of a sphere centered at the user's shoulder and a radius equal to the length of the user's arm. A linear constraint is implemented by adding an arc rail to the grip using [NuiHandlesGripAddRailArc](#).

An arc rail is defined by:

- A direction vector indicating the path of the rail.
- Minimum and maximum angles (in radians) that represent the end points of the arc. Angles are in body space, with angles increasing as the user's hand moves from the root of the direction vector to the tip.
- The dead zone for an arc rail is also defined as an angle using the same conventions as the endpoint angles.

When tracking progress in a game, the tracked value is the amount of progress from the minimum angle to the maximum angle, normalized to a value from 0.0 at minimum angle to 1.0 at maximum angle. The unnormalized progress in radians is also reported.

Circular Constraints with the Analog Joystick Rail

An analog joystick rail is a manipulation constraint on a handle that defines an angle of deviation from the handle's grip. Motion away from the grip can be in any direction within the constraint radius.

A circular constraint is implemented by adding an analog joystick rail to the grip using [NuiHandlesGripAddRailAnalogStick](#).

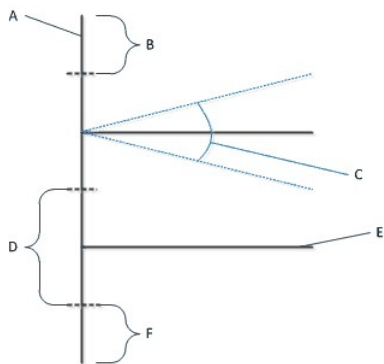
When tracking progress in a game, the current hand position is reported in polar coordinates, relative to an origin at the rail's center point.

Gearshift Constraints with the Gearshift Rail

A gearshift rail is a manipulation constraint that allows a handle to move along a branching track that resembles the diagram on the shift lever of a manual automobile transmission. The central track is referred to as the "spine" and the branches as "teeth." This constraint is useful for implementing menus, where each menu item is a separate tooth, and scrolling through the menu is performed by moving along the spine.

A gearshift constraint is implemented by adding a gearshift rail to the grip using [NuiHandlesGripAddRailGearShift](#).

Figure 1. Features of a gearshift rail



- Elements of a gearshift rail:
- A: The spine.
 - B: Upper scroll area.
 - C: Tooth tolerance cone. If the user's hand stays within this cone, it will be considered to be tracking along the tooth.
 - D: Tooth 1 zone. If the user's hand is within this zone, it is considered to be tracking along tooth 1.
 - E: Tooth 1.
 - F: Lower scroll area.

Progress along a gearshift rail is reported using three values. The progress along the spine is always reported, with the full length of the spine normalized into the range [0..1]. If a tooth has been committed, the index of the committed tooth is reported, and progress along the tooth is reported, normalized to the range [0..1].

The gearshift rail supports creation of scroll zones, or areas along the spine before the first tooth and after the last tooth. User progress into a scroll zone can be used to scroll the list of items being displayed alongside the teeth, to give a scrolling menu of more items than can fit on the screen at one time. Games can use the amount of progress into the scroll zone to govern the speed of scrolling.

Time-based Constraints with the Hover Timer

The hover timer is not a constraint on hand movement. Instead, it is a way to tell how long a player's hand has been attached to a grip. The progress value of a hover timer gives the amount of time that the hand has been attached, after the hand initially hovered over the grip for the specified time-to-commit duration. The hover timer can be used to implement fading-in a grip's visible user interface, or to take other actions where the progress of time, rather than movement in space, is significant to the user interface.

A timer constraint is implemented by adding a hover timer rail to the grip using [NuiHandlesGripAddRailHoverTimer](#). When you create a hover timer, you set the time-to-commit and the total hover time at which progress is considered complete. Progress reporting for a hover timer gives the amount of time the player's hand has been attached to the grip, normalized between 0 and 1.

The time-to-commit value specifies how long a user's hand must hover over the grip before the hand is considered to have attached to the hover timer. Once a user has committed to a hover rail, the user's progress time and percentage starts increasing from 0. *dwHoverTimeToCommitInMilliseconds* must be less than or equal to *dwTotalHoverTimeInMilliseconds*.

For example, if a hover timer is created with a *dwHoverTimeToCommitInMilliseconds* of 500, and *dwTotalHoverTimeInMilliseconds* of 2000, then a user would have to hold a hand on the grip for half a second to reach the Attached state and have the progress start increasing from 0, and would have to hold a hand on the grip for a total of 2 seconds to reach a progress value of 1.0.

The key API items for these durations are detailed below.

API Item	Description
<i>dwTotalHoverTimeInMilliseconds</i> parameter of NuiHandlesGripAddRailHoverTimer	You set this value. This is the duration starting from when the player's hand first starts to hover over the grip, to the time when progress is complete (that is, when fPercentageComplete reaches 1.0).
<i>dwHoverTimeToCommitInMilliseconds</i> parameter of NuiHandlesGripAddRailHoverTimer	You set this value. This is the duration starting from when the player's hand first starts to hover over the grip, to the time when the hand is considered to be attached (committed) to the grip.
dwTimePassedInMilliseconds member of NUI_HANDLES_RAILSTATE_HOVERTIMER	The system reports this value. This is the duration starting from when the player's hand is attached (committed) to the grip, to the current time. Before the <i>dwHoverTimeToCommitInMilliseconds</i> is reached, this value is 0. After this value reaches <i>dwTotalHoverTimeInMilliseconds</i> , this value no longer increases.
fPercentageComplete member of NUI_HANDLES_RAILSTATE_HOVERTIMER	The system reports this value. This indicates how long the hand has been attached (committed) to the grip, compared to (the <i>dwTotalHoverTimeInMilliseconds</i> minus <i>dwHoverTimeToCommitInMilliseconds</i>). • A value of 0 means that the commit time hasn't been reached, or has just been reached. • A value between 0 and 1 means that the commit time has been reached and exceeded, but that the specified total hover time hasn't been reached. • A value of 1 means that the specified <i>dwTotalHoverTimeInMilliseconds</i> value has been reached or exceeded.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Implementing a Simple Handle

This section discusses the actions necessary to use a simple arc rail in a program:

- Initializing a handle, grip and rail
- Updating the grip with fresh skeleton data
- Getting progress and updating the display to reflect progress

For a complete working example of using handles, see [Menu Using Handles Sample](#).

Sections

- [Prerequisites](#)
- [Creating the Handle](#)
- [Updating the Handle](#)
- [Getting and Using Progress Data](#)

Prerequisites

In order to use **NuiHandles**, you must ensure that all of the necessary data is being retrieved from the Kinect sensor array.

- You must initialize Natural User Interface (NUI) by calling [NuiInitialize](#) and passing it the flags that indicate you'll be using the skeleton-tracking stream, the high-resolution depth stream, and the low-resolution depth stream.

- You must open the skeleton-tracking stream and both depth-tracking streams by calling [NuiImageStreamOpen](#) once for each stream.
- You must allocate a **NUI_HANDLES_ARMS** structure and initialize it using [NuiHandlesArmsInit](#).

Creating the Handle

Every handle has one grip. You must create the grip and initialize it with one or more rails that determine how the user can interact with the handle.

- Start by allocating space for a **NUI_HANDLES_GRIP** structure and calling [NuiHandlesGripInit](#) to initialize it.
- Attach a rail to your grip with [NuiHandlesGripAddRailArc](#) or one of the other **NuiHandlesGripAdd** functions.
- Set the location of the activation area of your grip on the screen using [NuiHandlesGripSetScreenSpaceLocation](#).
- Enable your grip with [NuiHandlesGripSetEnabled](#).

Updating the Handle

You update the handle whenever you have new skeleton data and the player is facing the camera. For an example of how to do those things, see the Menu Using Handles sample.

You must record the time interval between the previously-used skeleton data and the new skeleton data. Note that skeleton data may not be updating as frequently as you are drawing frames. The time interval to use is the time between skeleton frames, not the time between game display frames.

- Update the **NuiHandles** library's internal tracking state by calling [NuiHandlesArmsUpdate](#) with the current **NUI_HANDLES_ARMS** struct, skeleton data, and depth data.
- For each grip, call [NuiHandlesGripUpdate](#) to update that grip's state based on current tracking state.
- Call [NuiHandlesArmsUpdateRailProgress](#) to update rail progress data.

Getting and Using Progress Data

For each currently-active grip:

- Determine if the grip is attached by calling [NuiHandlesGripGetIsAttached](#).
 - If the grip is attached:
 - Find out which rail is active by calling [NuiHandlesGripGetCommittedRailId](#).
 - Get the progress value along the committed rail by calling [NuiHandlesGripGetRailState](#). The returned **NUI_HANDLES_RAILSTATE** gives progress values for the rail. The proper interpretation of the progress structure depends on the type of the attached handle.
 - Update the handle's visual representation to reflect progress. What you do to indicate progress depends on your game's visual design. The [sample](#) shows one example of how to combine sound with drawing the parts of the handle, to show attachment, rail progress, and selection.
- If the progress value indicates that the user has selected something, and selection is going to cause you to leave the current screen and its grips, call [NuiHandlesGripForceDetach](#) on the current grip before abandoning the scene, to avoid resource leaks associated with the abandoned grip.
- If no grips are currently attached, then draw a cursor on the screen to provide visual feedback to help the user point to a handle. For each arm, get the screen space location where the user's hand is pointing by calling [NuiHandlesArmGetScreenSpaceLocation](#) for that arm, and transform the returned vector from screen space coordinates to pixel coordinates on the screen. Draw a cursor image (based on your game's visual design) at the indicated screen location.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Fine-tuning Grip Parameters

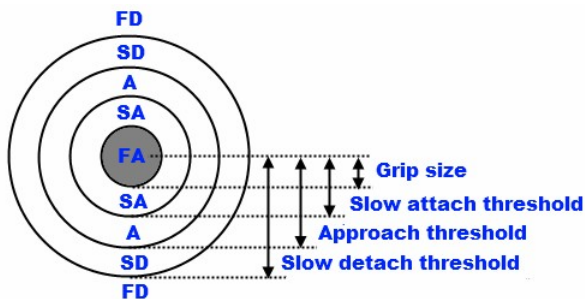


Part of making best use of **NuiHandles** is to tune your grips for the particular way they are used in your game. You can tune two different aspects of the grip:

- The shape characteristics of a grip determine the sizes of the regions around a grip where certain behaviors operate. To change shape characteristics, set new values in a **NUI_HANDLES_GRIPSHAPE** structure, and pass it to [NuiHandlesGripSetShape](#).
- The temporal characteristics of a grip determine how quickly or slowly those behaviors operate. To change temporal characteristics, set new values in a **NUI_HANDLES_TUNING_PARAMS_GRIP** structure, and pass it to [NuiHandlesGripSetTuningParams](#).

Key tunable areas are shown in the following figure.

Figure 1. Spatial tuning parameters



The attach zones are shown above.

- FA = fast attach zone
- SA = slow attach zone
- A = approach zone
- SD = slow detach zone
- FD = fast detach zone

Games can get the default values for tunable parameters using the [NuiHandlesGripGetDefaultTuningParams](#) and [NuiHandlesArmsGetDefaultTuningParams](#) functions. Games can also get the current values for tunable parameters using the [NuiHandlesGripGetTuningParams](#) and [NuiHandlesArmsGetTuningParams](#) functions. One potential use for these functions is to retrieve either the current or the standard values into an appropriate structure, change the members of that structure whose values you want to tune, and then pass the structure to the appropriate setting function.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Head Tracking



Head tracking indicates the location and direction of the player's head. Head tracking can be used in hands-free Kinect titles or in hybrid titles where the player's hands are already occupied by a controller.

Head tracking has two components: head position tracking and head orientation tracking. You can use one without the other, to save resources. For example, you can use head position tracking without head orientation tracking, to reduce resource costs and avoid the RGB color stream dependency.

[Head position tracking](#) detects the location of the center of mass of the player's head. This is useful for detecting lean. There are two approaches to head position tracking: the lightweight head position tracking API, and the more resource-intensive approach of using skeleton tracking and monitoring the head joint position.

[Head orientation tracking](#) detects the direction the player's head is facing. This is useful for aiming and camera control mechanisms. Head orientation tracking is 3-axis tracking, including roll, pitch, and yaw. Head orientation tracking can be used with or without skeleton tracking, and indicates the direction of the head, which skeleton tracking doesn't provide.

In This Section

[Head Position Tracking](#)

Indicates the location of the player's head.

[Head Orientation Tracking](#)

Indicates the direction of the player's head.

Limitations of Head Tracking

If a title needs to associate controllers with heads, the title must implement this association.

When skeleton tracking is not running, Identity is not supported.

XStudio Support

XStudio supports recording and playback of head tracking data. A lightweight mode where the data itself is recorded and played back (not derived from depth and color) is available, so that you do not have to record the color feed as part of your clips. Head tracking swim lanes and visualization are not supported.

Samples

- [Simple Head Tracking Sample](#)
- [Head Tracker Sample](#)
- [Stereoscopic 3D Head Tracking Sample](#)
- [Head Orientation Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Head Position Tracking

[See Also](#)



Head position tracking detects the location of the center of mass of the player's head. This is useful for detecting lean. Head tracking can be used in hands-free Kinect titles or in hybrid titles where the player's hands are already occupied by a controller.

Head position tracking uses the depth sensor.

For more general information about head tracking, including head orientation tracking, see [NUI Head Tracking](#).

Sections

- [Approaches to Head Position Tracking](#)
- [Game Scenarios Supported by Head Position Tracking](#)
- [Rank and Choosing a Head](#)
- [False Positives](#)
- [Quality Flags](#)

Approaches to Head Position Tracking

There are two approaches to head position tracking: the lightweight head position tracking API, and the more resource-intensive approach of using skeleton tracking and monitoring the head joint position. For the resources used by each approach, see [NUI Resource Usage](#). You can initialize either subsystem, but not both.

Lightweight Head Position Tracking

When you use lightweight head tracking, you use the head position tracking API. This approach doesn't use the skeleton tracking subsystem and its resources.

To use lightweight head position tracking, first call [NuiInitialize](#) passing in the flag [NUI_INITIALIZE_FLAG_USES_HEAD_POSITION](#). Call [NuiHeadPositionEnable](#), then [NuiHeadPositionGetNextFrame](#). In the returned [NUI_HEAD_POSITION_FRAME](#) structure, access each [NUI_HEAD_POSITION_DATA](#) structure, then access the **Position** member.

A given head proposal (tracking ID) occupies the same index in the array in every frame until the proposal is lost and deleted, the head position is disabled, or NUI is shut down.

Head Position Tracking Using Skeleton Tracking

When you use head position tracking that is based on skeleton tracking, you use skeleton tracking and query the head joint of the returned skeletons; you don't use the head position tracking API.

To use head position tracking that is based on skeleton tracking, first call [NuiInitialize](#) passing in the flag [NUI_INITIALIZE_FLAG_USES_SKELETON](#). Call [NuiSkeletonTrackingEnable](#), then [NuiSkeletonGetNextFrame](#). In the returned [NUI_SKELETON_FRAME](#) structure, access each [NUI_SKELETON_DATA](#) structure, then access **SkeletonPositions** [[NUI_SKELETON_POSITION_HEAD](#)], which is the position of the associated skeleton's head joint. The tracking ID corresponds with the associated skeleton.

The rest of this Help topic covers lightweight head position tracking, not the skeleton tracking (head joint) approach.

Resources Used

For the resources used by either approach, see [NUI Resource Usage](#).

Game Scenarios Supported by Head Position Tracking

- Perspective shift; for example, leaning to look around a corner in a controller-based shooter.
- Headbutting an enemy when they get up close.

Rank and Choosing a Head

To use lightweight head tracking, you must identify which head goes with which controller (that is, with which player), by using the Rank-based approach or the Choosing approach.

In the Rank approach, you use the rank number that the system provides. The system assigns rank to possible heads based on looking for the appropriate size expected for a head, normalized for distance from the sensor. You can simply use the highest rank of all the possible heads, or use the highest rank among the heads that are within a region such as the left, right, or center of the field of view. The **Rank** member of the [NUI_HEAD_POSITION_DATA](#) structure is the relative confidence of the current proposals.

In the Choosing approach, you have the player choose which head to use before gameplay begins, or have the player wave their hand or shake their head while holding down a button on the controller. The head that is closest to the most depth movement would then be associated with the player. The title would need to implement this matching of heads with controllers (players), similar to the [SimpleHeadTracking sample](#).

Variations of the 'Rank' and 'Choosing' Approaches

The title has to decide which of the up to 16 found heads to use. The following are various approaches, some of which use Rank data, and some of which instead provide a way for user to choose which head to use.

For a single-player game, you could use any of the following approaches:

- The "Rank" approach: Automatically use the top-ranked head.
- The "Rank" approach, within a region: Automatically use the top-ranked head within a zone such as the central region.
- The "Choosing" approach: Provide a way for the player to indicate which head to use.

For a multiplayer game, you could use any of the following approaches. For a multiplayer game, more than one player has a controller and a tracked head.

- The "Rank" approach: Use the top-ranked head in each region, such as the left and right side of the field of view.
- The "Choosing" approach: Provide a way for the player to indicate which head to use.

False Positives

The false-positives problem means that there might be more heads found than players. Rank can be used to counter this by picking the top-ranked head in each region you define, such as the left and right regions of the field of view.

Quality Flags

During lightweight head position tracking, the system detects failure cases and reports those to the title.

The **Quality Flags** member of the [NUI_HEAD_POSITION_DATA](#) structure indicates whether a matching *head proposal* was found. When a tracked head proposal is not found for 30 frames, the head proposal is tagged with the [NUI_HEAD_POSITION_QUALITY_LOST](#) flag. After 30 more frames of not being found, the head proposal is deleted. The number of frames in each case is subject to change.

To check quality flags for lightweight head position tracking, call [NuiHeadPositionGetNextFrame](#), then in a returned [NUI_HEAD_POSITION_DATA](#) structure, examine the **QualityFlags** member, which might contain the flag [NUI_HEAD_POSITION_QUALITY_LOST](#).

See Also

[NUI Head Tracking](#)
[NuiHeadPosition Reference](#)
[Simple Head Tracking Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Head Orientation Tracking

[See Also](#)



Head orientation tracking detects the direction the player's head is facing. This is useful for aiming and camera control mechanisms. Head orientation tracking is 3-axis tracking, including pitch, roll, and yaw.

Head orientation tracking can be used with or without skeleton tracking, and indicates the direction of the head, which skeleton tracking doesn't provide. Head tracking can be used in hands-free Kinect titles or in hybrid titles where the player's hands are already occupied by a controller.

Head orientation tracking uses the RGB color stream, not the depth sensor.

For more general information about head tracking, including head position tracking, see [NUI Head Tracking](#).

Sections

- [Game Scenarios Supported by Head Orientation Tracking](#)
- [Function Call Sequence](#)
- [Maximum/Minimum Framerate](#)
- [Lighting Requirements](#)
- [Face-Based Autoexposure, Pan/Zoom, and NUI Identity](#)
- [Quality Flags for Head Orientation Tracking](#)
- [Resources Used](#)

Game Scenarios Supported by Head Orientation Tracking

Head orientation tracking is an additional input you can take advantage of to provide greater immersion and interactivity in Kinect titles, such as the following:

- Turning your head to look into a corner in a racing game.
- Aiming a ball with your head as you throw it.
- Gesturing to the system by shaking your head yes or no.
- Avateering feels more immersive because it matches the avatar's head orientation with the player's head orientation.

Function Call Sequence

To use head orientation tracking:

1. Call [NuiInitialize](#) passing in the flag [NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION](#).
2. Call [NuiHeadOrientationEnable](#).
3. Call [NuiHeadOrientationSetTrackedHeads](#), selecting which players to track.

The head detector tracks positions of all the possible heads in the room, using tracking IDs. The title picks one or two heads of interest, and set them to be tracked for orientation and detecting the face rectangle.

4. Call [NuiHeadOrientationGetNextFrame](#).
A frame of head orientation data is returned.
5. Call [NuiHeadOrientationSmooth](#) to even out the head orientation over several frames.
6. In the returned [NUI_HEAD_ORIENTATION_FRAME](#) structure, access each [NUI_HEAD_ORIENTATION_DATA](#) structure, then access the **HeadOrientation** member.

See also [Head Orientation Sample](#), which calls these functions.

Maximum/Minimum Framerate

The face and orientation detectors rely on the RGB color feed. Head orientation tracking can operate at 10 or 15 frames per second. These rates are fast enough to avoid motion blur and slow enough to work in low lighting and to reduce CPU usage.

The system amortizes the cost over the appropriate amount of game frames. For example, if a game is running at 30fps, and head orientation tracking is running at 15fps, the system spreads the work out as evenly as possible over two frames. Coarse amortization is used; that is, the face detector is used on one frame, and head orientation tracking is used on the next frame.

Lighting Requirements

Head orientation tracking requires a 10 lux, evenly-lit face, like Identity requires. Scenarios where a player is strongly side-lit, especially by sunlight, are not supported.

Face-Based Autoexposure, Pan/Zoom, and NUI Identity

Head orientation tracking uses Face-Based Autoexposure and Pan/Zoom from the NUI Identity system. Head orientation tracking is not as dependent on hue and saturation accuracy as NUI Identity is.

The title has access to the color stream while head orientation tracking is running, but the title needs to take into account that head orientation tracking can change the exposure and zoom settings at any time while head orientation tracking is turned on. If a title is using head orientation tracking and the title issues an Identification operation (such as Identify or Enroll), the title will lose the color stream, because titles don't have access to the color stream while Identity runs.

Quality Flags for Head Orientation Tracking

When head orientation tracking is being used, the system detects failure cases and reports those to the title. The flags for head orientation tracking can indicate conditions such as the following:

- Face detection failure
- Hands occluding face
- Low light

To check quality flags for head orientation tracking, call [NuiHeadOrientationGetNextFrame](#), then in a returned [NUI_HEAD_ORIENTATION_DATA](#) structure, examine the **QualityFlags** member, which might contain any of the [NUI_HEAD_ORIENTATION_QUALITY](#) flags.

Resources Used

For the resources used by head orientation tracking, see [NUI Resource Usage](#).

See Also

[NUI Head Tracking](#)
[NuiHeadOrientation Reference](#)
[Head Orientation Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

NUI Identity

[See Also](#)



The NUI Identity system enables player identification and enrollment in the list of tracked players, to differentiate between detected skeletons and to associate the skeletons with gamer profiles, while enabling casual jump-in play.

In This Section

[Overview of NUI Identity](#)

Uniquely recognizes skeletons that become visible, and tracks their scores as they leave and reenter the field of view, without requiring creating gamer profiles or signing in.

[Identifying, Enrolling, and Signing-in Players](#)

The sequence for identifying a skeleton as a particular player, enrolling the skeleton in the session identity store, and signing the player in to a gamer profile and Xbox LIVE.

[Supporting Biometric Sign-In](#)

Required behaviors for recognizing and differentiating unique skeletons and signing them into biometric profiles and Xbox LIVE.

[Enabling Players to Correct Their Identity](#)

The correction process used when a player has been mistakenly identified.

[Pausing Identity to Use the Color Stream](#)

Pausing identity so the title can access the color image stream, such as to take a snapshot or use NUI head orientation tracking.

[Checking Identification Status Using GetMessage](#)

Checking the status of Identity operations, including type of operation (Identify or Enroll) and quality flags.

[Levels of Player Identity](#)

The degrees to which NUI recognizes the identity of a tracked skeleton.

[Forcing Enrollment Due to Quality Issues](#)

Forcing enrollment when the sensor can't see a new player well enough.

[Maximum Number of Skeletons Tracked](#)

The number of enrolled, center-tracked, joint-tracked, and signed-in players.

[Players Can Come and Go at Any Time](#)

Acknowledging the change in player line-up and clearly indicating to the players what changes are taking place and how the changes will impact the game in progress.

[Carry-Over Players](#)

Retaining enrollment for signed-in players who are already enrolled, during title initialization.

[Automatic Identity Handling](#)

Identifies skeletons when they become visible and enrolls and signs-in skeletons that have matching biometric profiles, automatically handling the details of identification behaviors to meet the minimum TCR requirements.

[Glossary for NUI Identity](#)

Terms used in NUI Identity, for distinguishing between different skeletons as specific players based on biometric data.

Samples for NUI Identity

Run and interact with these samples to see the flow of Identity behavior that meets the TCR requirements. Look at the Identify API calls in the code to see how that behavior is implemented. To fully exercise these samples, create a biometric-enabled gamer profile and a gamer profile that's not biometric-enabled.

- [Simple Identity Sample](#) - Manages checking if a skeleton is recognized as an enrolled player, signing in to a gamer profile, biometric sign-in, and responding to intent to play. To see the use of the Identity API, see this sample first, because it uses automatic identity handling.
- [Simple Identity Using Callbacks Sample](#) - Same behavior as the Simple Identity sample, but implemented without using automatic identity handling.
- [Identity Sample](#) - Uniquely recognizes up to eight enrolled players, with up to two active players at a time. Simulates a party game in which many players take part in short activities, competing against each other. Players join for a few rounds and then switch places with other participants.

Tools for NUI Identity

- [Viewing Identity data in Xbox Studio](#)

White Papers about NUI Identity

- [NUI Identity Basics: Implementing Essential Skeleton Identity Processing](#) - Basic behavior design and API for NUI Identity, using automatic identity handling. Read first.
- [Who Is Playing My Kinect Game Right Now?](#) - Behavior design for NUI Identity. Written before automatic identity handling was available.
- [Using Nuiidentity Effectively](#) - Using the NUI Identity API. Written before automatic identity handling was available.

TCRs for NUI Identity

- [TCR 165: NUI Biometric Sign-In](#) (described in [Supporting Biometric Sign-In](#))
- [TCR 158: NUI Identity Player Feedback](#) (described in [Enabling Players to Correct Their Identity](#))
- [TCR 166: NUI Identity Maximum Delay](#)

See Also

[NUI Identity Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Overview of NUI Identity

[See Also](#)

The Natural User Interface (NUI) Identity system supports casual jump-in play with a collection of visually recognized signed-in players and guest players. This way of uniquely recognizing skeletons even when they leave and reenter the field of view provides continuity of recognizing players within a title session, and keeps their scores for the duration of the title session, without requiring players to sign-in and configure gamer profiles with biometric data.

Sections

- [Capabilities Provided by NUI Identity](#)
- [Benefits for Various Levels of Identity](#)
- [How Skeleton Identity Works](#)

Capabilities Provided by NUI Identity

The NUI Identity API provides the following capabilities:

- Sign-in without a controller—Players can sign in using only skeleton identification, avoiding the need for signing in with a controller.
- Enables an intuitive and easy method for entering play—An observer (a player not previously known to NUI) can enter the field of view and start playing with minimal interruption. A title can create a record of that player (that is, an enrollment as a recognized skeleton) and maintain scores over the course of the gaming session. A simple example of using this information is to greet the player when the player returns from outside the field of view, and to display player-related information such as the player's avatar and high score.
- Continuity of skeletal tracking—If a player becomes occluded or steps out of the field of view during gameplay, the game can use the NUI Identity API to determine if the skeleton currently entering the field of view belongs to a new player or is a previously recognized player that stepped out of view, and then returned.
- Automatic association of the player with various gameplay events—The game can maintain an association with the player throughout gameplay. This association can then be used to ensure that achievements, save-game data, and Xbox LIVE interactions are attributed to the correct player.

Benefits for Various Levels of Identity

The Identity system makes it easy to play as a recognized player, for several categories of players. These scenarios assume the player indicates intent to play, and enrolls in the identity system.

- A player that has a Kinect ID-enabled gamer profile is visually recognized and automatically signed-in to their gamer profile, without having to do anything to sign-in. The player's scores are saved across sessions of using the title. When the player's skeleton disappears and reappears, the skeleton is recognized as belonging to the signed-in, biometrically enabled gamer profile.
- A player that has a gamer profile without Kinect ID is visually recognized as a particular guest player. The player can play the game without having to sign-in to their gamer profile, and the player's scores are maintained throughout the current session of using the title. When the player's skeleton disappears and reappears, the skeleton is recognized as belonging to the guest player. If the player chooses to manually sign-in to their gamer profile, the player's scores are saved across sessions of using the title, and when the player's skeleton disappears and reappears, the skeleton is recognized as belonging to the signed-in gamer profile.
- A player without a gamer profile is visually recognized as a particular guest player. The player can play the game without having to create a gamer profile. The player's scores are maintained throughout the current session of using the title. When the player's skeleton disappears and reappears, the skeleton is recognized as belonging to the enrolled guest player.

For more information about the categories of players based on whether the player is enrolled in the identity system and signed-in to a gamer profile, see [Levels of Player Identity](#).

How Skeleton Identity Works

To match a skeleton that has become visible with a known guest identity or gamer profile, the Identity system forms a *biometric print* using face recognition from the color camera of the Kinect Sensor, together with skeleton dimensions from the skeleton tracking pipeline. This incoming biometric print is compared to the biometric data in the session identity store and the available biometric-enabled gamer profiles on the console.

The player is enrolled into an enrollment slot with a particular enrollment index, within the session identity store. The enrollment slot is bound to a user index. This is equivalent to how an Xbox title that uses controllers associates a specific controller with a user index.

Information related to NUI Identity is stored in the skeleton-frame data received from the Kinect Sensor. Each frame of skeleton data is provided as a [NUI_SKELETON_FRAME](#) structure, which contains multiple [NUI_SKELETON_DATA](#) structures, one for each player. Each NUI_SKELETON_DATA structure contains an enrollment index and a user index. Games can use this information to determine which currently detected skeletons are enrolled, and which skeleton is assigned to a specific user index.

See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Identifying, Enrolling, and Signing-in Players

[See Also](#)

The basic flow of the NUI identity process consists of the following steps. This assumes that the title has started but gameplay has not started. If gameplay is underway, a subset of the following steps occurs, depending on the title.

1. The NUI identity system automatically attempts to silently identify any skeleton that enters the field of view, without displaying feedback about resolving image quality issues, and without representing the person on the screen. Most of the remaining steps are done by the title.
2. Detect that the skeleton is indicating intent to play, as defined by the title, and report that intent to the NUI identity system.
3. If the attempt to silently identify the skeleton was unsuccessful, prompt the player to resolve any image quality issues, while the NUI identity system re-tries identifying the skeleton.
4. If the player is not already enrolled and the skeleton matches a biometric-enabled gamer profile, the NUI identity system enrolls the player in the session identity store and signs-in the player to the gamer profile. If the skeleton doesn't match a biometric-enabled gamer profile, optionally enroll the skeleton as a guest player, who is not signed-in to a gamer profile.
5. Display which gamer profile or guest identity will be used for the player.
6. Allow the player to correct which gamer profile or guest identity is used, and allow the player to manually sign-in to a gamer profile.
7. Start or resume gameplay. During gameplay, support NUI Identity actions such as pausing identity processing to use the color camera to take a snapshot, and handling additional players who show intent to play.

These steps are described below, in order.

Sections

- [Attempting to Silently Identify the Skeleton](#)
- [Detecting and Reporting Intent to Play](#)
- [Identifying the Skeleton While Prompting About Image Quality Issues](#)
- [Enrolling and Signing-in Biometric Players, and Enrolling Guest Players](#)
- [Indicating Which Gamer Profile or Guest Identity Will Be Used](#)
- [Allowing Identity Correction and Manual Sign-in](#)
- [Supporting Identity Actions During Gameplay](#)

Attempting to Silently Identify the Skeleton

Each time a skeleton becomes visible to the Kinect Sensor, the NUI identity system performs identification processing on the skeleton to try to recognize the face and skeleton as belonging to a particular player who is enrolled in the session identity store. This processing is automatically done during gameplay, as well, for any skeleton that enters the field of view.

To *identify* a skeleton means to perform face recognition to determine whether the skeleton that has just become visible is recognized as a player who is enrolled in the session identity store, or if not enrolled, whether the face and skeleton matches a biometric-enabled gamer profile. If a match is found, the NUI identity system links the skeleton to the matched enrollment slot, through the slot's enrollment ID.

During title startup, pass the flag `NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY` in the call to [NuiInitialize](#). This turns on automatic skeleton recognition processing.

Image quality issues, such as insufficient lighting, might prevent this silent recognition processing from completing successfully. But you must not prompt the detected person to resolve these issues yet, because the person hasn't indicated intent to play, and might only be a passerby or observer rather than a player.

When a skeleton becomes visible to the Kinect Sensor, the NUI identity system calls [NuiIdentityIdentify](#) to determine whether the skeleton is an enrolled player.

Detecting and Reporting Intent to Play

Next, at the introduction screen, detect that a skeleton has indicated intent to play. Then call [NuiIdentityDetectedIntentToPlay](#) to report to the NUI identity system that this skeleton has indicated intent to play, and thus might need to be enrolled and signed-in.

It is a good idea to always call **NuiIdentityDetectedIntentToPlay** when a player engages with the title, even if the number of profiles supported simultaneously by the title has been reached. This allows the NUI identity system to enroll a player who has a matching biometric profile, if an enrollment slot is available beyond the number of simultaneous profiles supported.

The title defines what action on the part of the player indicates *intent to play* the game. A title can define various ways for players to indicate intent to play, and these can vary by context. For example:

- Gesture - The gesture should fit the theme of the title, but a title can choose to use the wave gesture used by the Xbox Dashboard. You can use the [NuiWave](#) API to detect when a player is waving at the Kinect Sensor.
- Voice command
- Correcting Identity - Accessing a **Correct My Identity** menu may be considered indicating intent to play.
- Sign-in - A title could make the sign-in option available immediately, before the player is identified or enrolled, since the Kinect Guide will take care of identification and enrollment as needed when the player signs in.
- Resume button - If a person selects the **Resume** option from the title's **Pause** menu, that constitutes intent to play.

If more than one player is attempting to show intent to play, but only one profile is required, a good, simple approach is to show gesture feedback for one player only. That situation occurs in a single-player game, or at the begin screen when determining the primary profile, for a multiplayer game. For example, usually it works well to show gesture feedback only for the player who is the closest to completing indicating intent to play. Alternatively, you could show gesture feedback for the player who is closest to the Kinect Sensor, or closest to the center of the play area.

Identifying the Skeleton While Prompting About Image Quality Issues

If the attempt to silently identify the skeleton was unsuccessful, prompt the player to resolve any image quality issues, while the NUI identity system re-tries identifying the skeleton. At the point in time when you call **NuiIdentityDetectedIntentToPlay**, identification processing for the specified skeleton is in one of the following states:

- Silent identification processing completed successfully; the NUI identity system has determined either that the player is not enrolled, or is enrolled as a particular player. In this case, when you call **NuiIdentityDetectedIntentToPlay**, the NUI identity system proceeds to the next step, which is conditionally enrolling and signing-in the player.
- Silent identification processing completed unsuccessfully; image quality issues prevented determining whether the skeleton is an already-enrolled player. In this case, when you call **NuiIdentityDetectedIntentToPlay**, the NUI identity system retries identification processing. During that processing, retrieve image quality feedback from the NUI identity system by calling [NuiIdentityGetNextMessage](#). That function returns the [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure, which contains a flag for each quality issue, such as `NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK`. Use this quality issue feedback to prompt the player to adjust their posture or environment so that skeleton recognition processing can succeed, such as "Face the sensor" or "Make sure your room is well lit."
- Identification processing is still in progress. Provide feedback to the player about any image quality issues, to guide the player and enable identification processing to succeed.

If identification processing hasn't completed after 15 seconds since the player indicated intent to play, treat the player as a new guest player.

In addition to **NuiIdentityGetNextMessage**, the NUI identity system also provides feedback to you about the identity status of each skeleton at all times, through the `NUI_SKELETON_DATA` structure. The skeleton continues to be tracked with a stable skeleton tracking ID, until the skeleton disappears.

After running **NuiIdentityIdentify**, the NUI identity system has determined whether the skeleton that became visible represents an enrolled player, or if not, matches a biometric-enabled gamer profile.

- If a skeleton is enrolled, **dwEnrollmentFlags** in the [NUI_ENROLLMENT_INFORMATION](#) structure is `NUI_IDENTITY_ENROLL_SKELETON`, and **dwEnrollmentIndex** in the [NUI_SKELETON_DATA](#) and [NUI_IDENTITY_MESSAGE_COMPLETE](#) structures is 0-7.
- If a skeleton is not enrolled, **dwEnrollmentIndex** in the [NUI_SKELETON_DATA](#) and [NUI_IDENTITY_MESSAGE_COMPLETE](#) structures is [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#).

To determine whether a matching biometric-enabled gamer profile was found for the detected skeleton, examine the **bProfileMatched** member in the [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure that's returned by [NuiIdentityGetNextMessage](#).

Enrolling and Signing-in Biometric Players, and Enrolling Guest Players

Enrolling a skeleton enables the NUI identity system to recognize a skeleton as a particular player each time the skeleton becomes visible, and consistently associate that skeleton with a score and avatar during a session of using the title.

When you call **NuiIdentityDetectedIntentToPlay**, the NUI identity system enrolls and signs-in any non-enrolled skeleton that has a matching biometric-enabled gamer profile.

Enrolling a skeleton adds the skeleton's biometric print to the session identity store, in an enrollment slot with an enrollment ID, 0-7. Enrolling a skeleton links the player's face and skeleton dimensions from the Kinect Sensor with the player's guest identity or gamer profile. Whenever the skeleton disappears and reappears, the skeleton is quickly linked to the same session identity, without interrupting gameplay to confirm the identity of the skeleton.

Enrolling is optional, to enter gameplay; you can allow a non-enrolled guest player to play. For a skeleton that doesn't match a biometric-enabled gamer profile, you can optionally enroll the skeleton as a guest player, by calling the low-level function [NuiIdentityEnroll](#).

A player might already be enrolled and signed-in before starting the title, or through the dashboard after the title started, such as by accessing the dashboard through a **Pause** menu.

Indicating Which Gamer Profile or Guest Identity Will Be Used

When a skeleton enters the field of view and indicates intent to play, before entering gameplay, you must display which identity will be used.

If the player is signed in, you can retrieve in-game assets such as the player's avatar, gamer-tag, and a color and use them to represent the player during gameplay. For a player that is not signed-in, you can display "Guest" or a term that is adapted to the context of the game.

Allowing Identity Correction and Manual Sign-in

Allow the player to correct which gamer profile or guest identity is used. The player must be able to select an already-enrolled player identity, or manually sign-in to a gamer profile. If the player selects an option that advances toward gameplay, that action is considered as the player confirming the indicated gamer profile or guest identity.

Do not force a player to sign-in to a gamer profile. Your title must be ready to proceed with gameplay using a guest identity rather than a gamer profile, even if it means some functionality is unavailable. Typically, a title doesn't require a player to sign-in to a gamer profile in order to play. A signed-in gamer profile may offer a more complete experience, such as access to Xbox LIVE saved games, rich presence, and avatar assets.

To encourage a guest player to sign-in to a gamer profile, a good strategy is to present a "Correct My Identity" button that calls [XShowNuiSigninUI](#), which directly brings up the Sign In screen of the Kinect Guide.

If your title doesn't enroll guest players, the Sign In screen of the Kinect Guide is all you need to support profile correction. If your title enrolls guest players, you will have to provide your own profile correction UI. For an in-depth discussion of providing a UI to let players correct their identity, see the white paper "Who Is Playing My Kinect Game Right Now?"

Until the player has confirmed the gamer profile or guest identity that the identity system selected, you can only read from the profile; you must not write to it yet.

The screen that allows the player to correct a misidentification or sign in to a gamer profile can also enable the player to select a game asset such as avatar or color.

Supporting Identity Actions During Gameplay

Start or resume gameplay, supporting identity actions during gameplay, such as pausing identity processing, and handling additional players who show intent to play.

The player can use the Kinect Guide to manually sign in or sign out at any time. When a player signs out of a gamer profile, the player is automatically unenrolled by the NUI identity system.

Pausing Identity Processing During Gameplay

To temporarily access the color image stream from the color camera, and any other system resources that are used by the NUI identity system, pause identity processing by calling **NuiIdentityPause**. To re-enable identity processing, call **NuiIdentityResume**.

Although you are allowed to pause skeleton identity processing, it is preferable to let identity processing run in the background so players keep being identified whenever tracking is lost and regained.

The NUI identity system uses the color image stream to capture a unique biometric print from a player. While capturing the biometric print, the NUI identity system will change the camera settings, such as exposure and zoom, to get a clear view of the player's face, and therefore the color image stream won't be available to you. When you call **NuiIdentityPause**, if there is no identification operation in progress, the color image stream will immediately become available to you. If there is an identification operation in progress, the NUI identity system will suspend the operation and return the color image stream settings to the last settings you selected.

Returning the color image stream to your settings takes up to a few frames, so ideally you should call **NuiIdentityPause** in advance, so the color image stream can be restored to the application.

If your title is using the color image stream intensively, consider pausing the NUI identity system for the duration of the gameplay session. However, while paused, NUI identity processing will not be able to support your title and help you identify the players as they move around and enter or leave the play area.

Consider pausing the NUI identity system during gameplay if your title is using every bit of CPU power available and cannot afford the occasional spike caused when identity tries to identify a newly tracked skeleton. However, this is a rare, extreme case.

Changing Profiles During Gameplay

Once the player has accepted the gamer profile or guest identity, it is OK to use that identity until the player explicitly expresses the need to change profiles. The player could change profiles by returning to the title's lobby, by signing out from the dashboard, or by accessing the Sign In screen of the Kinect Guide through the title's **Pause** menu.

For example, a player who is playing as a non-enrolled guest player, or as a player who is enrolled but not signed-in, can perform the system gesture to call up the Kinect Guide at any time, and then select the Sign In screen and manually sign-in to a gamer profile.

Additional Players Showing Intent to Play During Gameplay

Even when skeleton identity recognition isn't essential to your title at certain times during gameplay, allow automatic identity processing to run on all skeletons that enter the field of view. If a person has a matching biometric-enabled gamer profile, they will be automatically enrolled and signed-in. Refrain from providing feedback about player identification, to not unduly disturb the players.

During gameplay, when a skeleton enters the field of view, don't interact with that person unless the skeleton indicates intent to play. This prevents interruptions in the gameplay when non-players enter the play space.

Even if you won't be using the profile yet, register the intent-to-play with the NUI identity system by calling [NuiIdentityDetectedIntentToPlay](#), so that any related biometric profile is loaded.

During gameplay, there are many ways a player can show intent to play, and it is up to you to decide which ways are available at a specific time. For example, if the game is paused, entering the play area and selecting the **Resume** option should be considered indicating intent to play, because the player has indicated the intention to continue where the previous player left off.

If you enroll a player during gameplay, that interrupts gameplay, because enrolling requires the player to resolve any image quality issues and confirm or correct their identity.

Signout During Gameplay

The player can sign out of your title at any time, such as by accessing the dashboard through the title's **Pause** menu, or by removing the Memory Unit (MU) on which the current profile is stored.

✦ See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Supporting Biometric Sign-In

[See Also](#)



When a player with a biometric-enabled gamer profile indicates intent to play, the title enrolls the player and signs-in the player into the player's gamer profile. A biometric-enabled gamer profile is also called a *biometric profile* or a *Kinect ID-enabled gamer profile*. Biometric sign-in is required by [TCR 165: NUI Biometric Sign-In](#), to provide players with automatic sign-in when the player indicates intent to play. This automatic sign-in upon signalling intent to play occurs with all Kinect ID-enabled gamer profiles across all Xbox 360 experiences.

Sections

- [Overview of Biometric Sign-in](#)
- [Ways to Attempt Enrollment for Biometric Sign-in](#)
- [Deciding How Much Customization to Use](#)
- [Rules for Identifying and Enrolling, with Biometric Profiles](#)

Overview of Biometric Sign-in

Biometric sign-in relies on the title's use of the NUI identity system. Biometric sign-in associates a player with a gamer profile based on Kinect Sensor data, using face recognition and skeleton dimensions, which together form a biometric print.

The incoming biometric print from the Kinect Sensor is matched against any biometric prints of enrolled players in the session identity store. If the player is not found enrolled there, the incoming biometric print is matched against any biometric profiles available on the console. If a matching biometric profile is found, ([NuiIdentityIdentify](#) sets a flag to indicate this.

The title waits for the player to indicate intent to play. When a player with a biometric profile indicates intent to play (as defined by each title), the title immediately enrolls and signs-in the player, if there are available enrollment slots, without first providing feedback and waiting for confirmation of which gamer profile will be used.

Ways to Attempt Enrollment for Biometric Sign-in

Titles typically attempt biometric sign-in of a player as soon as the title detects intent to play from a skeleton that matches a biometric profile. For many titles, intent to play will be indicated in the UI before gameplay starts.

The following two procedures describe how to use the NuiIdentity API in general, with an emphasis on biometric sign-in.

Using Automatic Identity Handling

Automatic identity handling is recommended for all titles, and provides baseline compliance with [TCR 165: NUI Biometric Sign-In](#). The XDK documentation assumes you are using automatic identity handling. To use automatic identity handling:

1. When calling [NuiInitialize](#), pass the flag [NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY](#).
2. Detect intent to play by any skeleton. When a skeleton indicates intent to play, as defined by your title, call [NuiIdentityDetectedIntentToPlay](#) for that skeleton.

Directly Calling Identify and Enroll

To directly use the lower-level Identity API, use this low-level, non-automatic approach. Parts of this approach can supplement automatic identity handling.

1. Monitor a skeleton for the flag [CALL_IDENTIFY](#) in the **dwEnrollmentIndex** member of the [skeleton's data structure](#). This flag indicates that this skeleton is newly visible and therefore needs to be checked to see which enrolled player, if any, the skeleton represents.
2. Call [NuiIdentityIdentify](#) when that flag is set.
NuiIdentityIdentify sets flags indicating whether the skeleton is enrolled, and whether a biometric profile match was found.
3. To determine whether the skeleton was found to be enrolled, examine the **dwEnrollmentIndex** member of the [skeleton's data structure](#), checking for the value [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#), which means the skeleton is not enrolled.
4. To determine whether a biometric profile match is available for the detected skeleton, examine the **bProfileMatched** member in the [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure that's returned by [NuiIdentityGetNextMessage](#) or the optional callback function.
5. When the skeleton indicates intent to play (as defined by each title), if the skeleton is not enrolled but has a biometric profile match, call [NuiIdentityEnroll](#).

The NUI identity system then enrolls the player, and because there is a biometric profile match, also signs the player in to the matched biometric profile.

Deciding How Much Customization to Use

- If your title only requires baseline functionality per the NUI Identity TCRs, use automatic identity handling without additional calls to the NUI Identity API. This works well for typical single-player games.
- To fulfill the NUI Identity TCRs and provide additional custom behavior, use automatic identity handling with added calls to the NUI Identity API. You must ensure that the title's customizations are compliant with the NUI Identity TCRs, including the rules below, which cover the Biometric Sign-in TCR.
- To use the NUI Identity API entirely at the lower level, call **NuiIdentityIdentify** and **NuiIdentityEnroll** directly. You must ensure that the title's behavior is compliant with the NUI Identity TCRs, including the rules below, which cover the Biometric Sign-in TCR.

Rules for Identifying and Enrolling, with Biometric Profiles

The NUI identity system provides automatic identity handling, which fulfills the requirements of [TCR 165: NUI Biometric Sign-In](#). This TCR requires the following actions and behavior in all Kinect titles.

No feedback required before enrolling a biometric profile

If a non-enrolled skeleton has a biometric profile match, when the player shows intent to play, the title can immediately enroll the skeleton, without having to first provide feedback about which gamer profile the player will be signed-in to. This policy is to support the feature *automatic sign-in using Kinect ID*.

Provide feedback before enrolling as guest

If a non-enrolled skeleton does not have a biometric profile match, enrolling the player is optional. Before enrolling the player, the title must first provide feedback showing what guest player will be associated with the skeleton, and provide a correction opportunity to the player (see [Enabling Players to Correct Their Identity](#)). After receiving confirmation from the player, the title can enroll that skeleton as that guest player.

Don't force-enroll enrolled player unless player corrects identity

If the player is already enrolled, the title must not force enrollment of the player, except as the direct result of the player correcting identity (see [Enabling Players to Correct Their Identity](#)). This rule helps respect the player's selected identity.

Forcing enrollment due to quality issues is optional

If an identity operation cannot be completed due to quality issues, no further action is required for that skeleton. It is not required that you force enrollment of the player if there are quality issues preventing recognizing the skeleton as a particular player.

For a biometric profile match, attempt enrolling at least once

When a player shows intent to play, if there is a biometric profile match, the title must enroll and sign-in the player, if there is an available enrollment slot. The title must make at least one attempt to enroll the player's skeleton.

Attempt biometric sign-in for as many profiles as supported

The minimum requirement is for the title to attempt biometric sign-in for as many gamer profiles as your title can support simultaneously. A title is said to support a player profile if the title performs profile-related actions to a signed-in profile, such as awarding achievements or saving game progress. The session identity store enables simultaneously enrolling four players signed-in to gamer profiles and four guest players, who are by definition not signed-in. An enrolled, signed-in player might or might not have a biometric profile. A title is not required to unenroll a guest player to make room for a later player who has a biometric profile.

No biometric sign-in is required for a new skeleton if it's already known which gamer profile to use

In some cases, when a skeleton becomes visible, the title already knows which profile to use without having to call [NuiIdentityIdentify](#) to check whether the skeleton is enrolled and therefore known. This is typically the case if a specific player's turn, side of the play space, or some other context is strongly associated with a profile. In this situation, the title does not have to attempt biometric sign-in for a newly visible skeleton.

No biometric sign-in is required for controller sign-in

For titles that support both controller and Kinect input, if a player initially signs in with a controller, biometric sign-in of that player is not required during skeletal tracking portions of the title. If the skeletal tracking portions support additional players joining and signing in, biometric sign-in must be supported for those additional players.

See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Enabling Players to Correct Their Identity

[See Also](#)



Identification is the process through which the NUI identity system recognizes a skeleton and face as a particular player. The identity system uses re-enrollment to correct mistaken identity. [TCR 158: NUI Identity Player Feedback](#) requires that titles inform players of the identity detected, to allow the player to confirm or correct which guest identity or gamer profile the player is associated with.

Sections

- [Action Sequence](#)
- [Example Scenario](#)
- [When Player Recognition Is Attempted](#)
- [Misidentification Cases to Cover](#)
- [Allow Correction Once per Identification](#)
- [If the Skeleton Is Reported as Non-Enrolled](#)
- [Providing Correction Opportunity Before Affecting a Gamer Profile](#)
- [UI for Indicating and Correcting Identity](#)
- [Re-enrolling a Player Who Corrects Their Identity](#)
- [Correction Opportunity Not Required When Identity Is Known by Context](#)

Action Sequence

The following are the main steps for attempting recognition, indicating player identity, and correcting misidentification.

1. Detect a skeleton.
2. Attempt to recognize the skeleton as an enrolled player, or if not, as matching a biometric-enabled gamer profile. ([NuiIdentityIdentify](#))
3. Indicate that player as a particular guest player or gamer profile.
4. Provide an opportunity for the player to correct the identification.
5. If the player corrects their identity, re-enroll the player. ([NuiIdentityEnroll](#))
6. Start gameplay, or perform other actions that affect a gamer profile.

Example Scenario

A player who was identified previously as enrollment index 7 disappears and re-appears, and is mistakenly identified as player 2. In this scenario, Player 7 could bring up the dialog box by using the [system gesture](#) and the in-game Pause screen, and specify that the player is player 7 rather than player 2. The title re-enrolls the player, specifying enrollment index 2.

The title must provide a visual way to correct this error, such as a re-enrollment dialog box; correcting identity through the Kinect Guide must not be the only way available.

When Player Recognition Is Attempted

Whenever a skeleton becomes visible, the NUI identity system automatically calls [NuiIdentityIdentify](#) to attempt to recognize the skeleton as an enrolled player.

(To enable automatic identity handling, pass the `NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY` flag when calling [NuiInitialize](#).)

If a title is not using automatic identity handling, the title might directly call **NuiIdentityIdentify** after a skeleton has indicated intent to play, if the function hasn't been run yet for that skeleton.

Misidentification Cases to Cover

The exact user experience for allowing a player to correct identity is up to the title, but TCR #158 requires that the following cases be covered:

- An existing player is misidentified as a new.
- An existing player is misidentified as a different player.
- A new player is misidentified as an existing player.

Allow Correction Once per Identification

Providing identity feedback and the opportunity for correction are only required once per identification operation. The exact timing of this is up to the title. Feedback and an opportunity for correction do not need to happen right away, but they must occur before the title takes a profile-related action. Once completed, the title is not required to re-confirm again before additional profile-related operations.

If the Skeleton Is Reported as Non-Enrolled

If the skeleton is identified as a non-enrolled player, the title may respond appropriately based on the context. For example, if the time is appropriate for the title to take on a new player, the title could welcome the player as a new, guest player and give the player a chance to correct identity, in case the player is actually already enrolled.

Providing Correction Opportunity Before Affecting a Gamer Profile

The title must provide feedback and a correction opportunity before the title takes any action that would affect a gamer profile, such as granting points to a gamer profile, or not granting points to a gamer profile, when the player wants that to happen.

The title must provide feedback and a correction opportunity if a misidentification would affect a player's gamer profile data or the association of a player with the player's gamer profile.

Examples of affected profile-related actions include:

- Enrolling a skeleton as a guest player when a matching biometric-enabled gamer profile isn't found (that is, when **bProfileMatched** is FALSE). This action might be related to a gamer profile; the player might have a non-biometric enabled gamer profile and want points to be granted to that gamer profile.
- Awarding an achievement.
- Saving game progress.
- Updating Rich Presence.
- Writing to Xbox LIVE storage.
- Writing to a leaderboard.

UI for Indicating and Correcting Identity

Communication of the player's identity can take many forms, and it is up to you to choose the form most appropriate for your title. Titles will typically use an asset that was associated with the player when the player was first enrolled, such as the player's gamertag or avatar.

Allowing the player to launch the sign-in dialog with [XShowNuiSignInUI](#) is an acceptable way to correct identity, if the player who is correcting identity is expected to be signed in. The title must provide a visual way to correct misidentification, such as a re-enrollment dialog box; correcting identity through the Kinect Guide must not be the only way available.

Enabling Correction as an Unobtrusive Option

We recommend that a title does not block progress or force a player to confirm the player's identity. Providing an option to correct identity, by use of a menu in a lobby, a pause menu, or in other contextual UI, is preferred.

Re-enrolling a Player Who Corrects Their Identity

If a player corrects identity after the title indicates which identity will be used, the title must re-enroll the player as the player-selected identity.

To re-enroll a player, call [NuiIdentityEnroll](#), passing the correct enrollment index in the *dwFlags* argument, and passing the [NUI_IDENTITY_FORCE_ENROLL](#) flag.

Correction Opportunity Not Required When Identity Is Known by Context

In many contexts, the active gamer profile and identity is already known, and the title has no need to call [NuiIdentityIdentify](#) after gameplay starts. In these situations, the title isn't required to explicitly indicate player identity and enable the player to correct their identity.

For example, when the current turn is already associated with a particular gamer profile, a title may allow any skeleton to step up and play without being identified, and award any achievements to the gamer profile or guest identity that is associated with the current turn. This lack of confirming identity is acceptable, if the title indicates the player's identity and enables correcting it, in the pre-game UI.

See Also

[TCR 158: NUI Identity Player Feedback Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Pausing Identity to Use the Color Stream

[See Also](#)



Pause the NUI identity system when accessing the RGB color image stream, such as to take a snapshot or use NUI head orientation tracking. Pausing identity pauses and queues the aspects of identity processing that use the color stream.

Sections

- [When to Pause the Identity System](#)
- [How to Pause Identity Processing](#)
- [What Happens When Identity Is Paused](#)
- [How to Resume Identity Processing](#)
- [If Not Using Automatic Identity Handling](#)

When to Pause the Identity System

The NUI identity system uses the color stream, optimized for face recognition, whenever a skeleton becomes visible, which could happen at any time, and when enrolling a player in the session identity store. If the title tries to use the color stream to take a snapshot, or to track head orientation, the title must pause the NUI identity system, to make the color stream available for these other uses.

During the identification and enrollment phases, the color stream will be redirected to the identity system, meaning that for a length of time, your title won't be able to access the color stream. The color stream will be restored when the system is done with the NUI identity operation.

If a title requires access to the color stream while a NUI identity operation is in process, the title can do one of the following:

- Pause the NUI identity system, letting the NUI identity operation be queued and resumed later.
- Cancel the NUI identity operation, by calling [NuiIdentityAbort](#). This aborts all NUI identity operations except those initiated by the Kinect Guide, and releases the color stream to enable it to be used by the title. Call the NUI identity operation again later, when the color stream is available.

How to Pause Identity Processing

To pause the NUI identity system, call [NuiIdentityPause](#).

What Happens When Identity Is Paused

While the NUI identity system is paused:

- Any NUI identity operations that were in progress also pause.
- The NUI identity system doesn't use the RGB stream for any of its operations.
- Any calls to [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) return E_PENDING, because no immediate processing can be done on those skeletons. Those operation requests are queued up to be processed after the NUI identity system is resumed. The **dwEnrollmentIndex** on the skeletons will be set to [NUI_IDENTITY_ENROLLMENT_INDEX_BUSY](#) to reflect this pending operation.
- When a skeleton becomes visible, the skeleton will have a *dwEnrollmentIndex* value of **NUI_IDENTITY_ENROLLMENT_INDEX_BUSY**, to reflect the pending **NuiIdentityIdentify** operation.
- If a skeleton with a pending NUI identity operation disappears while the NUI identity system is paused, that pending operation will immediately return with a completion code of **E_NUI_IDENTITY_LOST_TRACK**.

How to Resume Identity Processing

When the title no longer requires the RGB stream, call [NuiIdentityResume](#). The identity system then processes any pending NUI identity operations.

If Not Using Automatic Identity Handling

If the title doesn't use automatic identity handling, pause NUI identity handling while using the color stream or NUI head orientation tracking, by not calling **NuiIdentityIdentify** or **NuiIdentityEnroll**.

If a title makes heavy use of the color stream, it may have to display a special UI during NUI identity operations. This would require that the title have control over when the NUI identity operations occur, and thus only applies when not using automatic identity handling.

See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Checking Identification Status Using GetMessage

[See Also](#)

Provide feedback to players about identification progress and issues. A title can get the status of ongoing identity operations by calling [NuiIdentityGetNextMessage](#).

Each call to **NuiIdentityGetNextMessage** gets the next message from the identity message queue, and removes the message from the queue. The message content contains information about whether identity operations are ongoing or completed.

NuiIdentityGetNextMessage returns the message as a [NUI_IDENTITY_MESSAGE](#) structure. That structure's **OperationID** member indicates whether the operation was the [NuiIdentityIdentify](#) or the [NuiIdentityEnroll](#) operation.

When **MessageId** in the structure is [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#), the **Data** member of the structure contains a [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure that contains any [NUI_IDENTITY_QUALITY_FLAGS](#) that describe problems encountered while processing a single frame. Any issues with identity are reported by using the quality flags. The title can use these flags to determine when to display visual feedback to the player regarding identity operation status and to resolve issues with identification.

Note

We recommend that you design your title to dequeue all messages, keeping the title up-to-date on operation status. The identity message queue only stores one message per skeleton tracking ID, and discards old messages for the same skeleton tracking ID.

See Also

[Forcing Enrollment Due to Quality Issues](#)
[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Levels of Player Identity

[See Also](#)

NUI recognizes the identity of a tracked skeleton to a greater or lesser extent. Any detected skeleton that indicates intent to play can play as a non-enrolled guest player, as an enrolled guest player, or as an enrolled, signed-in player.

Sections

- [Summary of Levels of Player Identity](#)
- [How to Determine Whether a Skeleton Is Enrolled](#)
- [How to Determine Whether a Skeleton Is Signed-in](#)
- [Unidentified Skeleton](#)
- [Non-Enrolled Guest Player](#)
- [Enrolled Guest Player](#)
- [Enrolled, Signed-in Player](#)
- [Borrowing a Gamer Profile](#)

Summary of Levels of Player Identity

Non-enrolled guest player

A generic, non-unique skeleton that is not enrolled in the session identity store. The system is able to track the skeleton, but doesn't recognize the skeleton as belonging to anyone in particular. If the skeleton leaves the field of view and returns into the field of view, the title doesn't know that the newly visible skeleton is the same player who just left the field of view. A biometric print is not stored or used for such a skeleton.

Enrolled guest player

A skeleton that is enrolled in the session identity store, but isn't linked to a signed-in gamer profile. The system can continually distinguish and remember one enrolled skeleton versus another as they come in and out of view, such as "Player 4" versus "Player 5", but doesn't know these as signed-in players with particular gamer profiles. A biometric print is stored and used for such a skeleton.

Enrolled, signed-in player

A skeleton that is enrolled in the session identity store and is linked to a signed-in gamer profile. The skeleton's identity is fully recognized as a specific player with a specific gamer profile, such as MyTestUser. There are two sub-cases:

- Manually signed-in to a gamer profile that's not biometric-enabled.
- Automatically signed-in to a biometric-enabled gamer profile.

In the following table, the three columns on the left define scenarios. The two columns on the right give the resulting behavior for those scenarios.

Type of player	Enrolled?	Signed-in?	Recognized when reappear?	Scores maintained?
Non-enrolled guest player	Not enrolled	Not signed in	No	Until skeleton disappears
Enrolled guest player	Manually enrolled	Not signed in	During title session	During title session
Manually signed-in player	Manually enrolled	Manually signed in	Across title sessions and console reboots	Across title sessions and console reboots
Biometrically signed-in player	Automatically enrolled	Automatically signed in	Across title sessions and console reboots	Across title sessions and console reboots

How to Determine Whether a Skeleton Is Enrolled

Check the following values after **NuiIdentityIdentify** has completed.

If a skeleton is enrolled:

- **dwEnrollmentFlags** in the [NUI_ENROLLMENT_INFORMATION](#) structure is [NUI_IDENTITY_ENROLL_SKELETON](#).
- **dwEnrollmentIndex** in the [NUI_SKELETON_DATA](#) and [NUI_IDENTITY_MESSAGE_COMPLETE](#) structures is 0-7.

If a skeleton is not enrolled, **dwEnrollmentIndex** in the [NUI_SKELETON_DATA](#) and [NUI_IDENTITY_MESSAGE_COMPLETE](#) structures is [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#).

How to Determine Whether a Skeleton Is Signed-in

Check the following values after **NuiIdentityIdentify** has completed.

If a skeleton is signed-in, **dwUserIndex** in the [NUI_ENROLLMENT_INFORMATION](#) structure is 0-3.

If a skeleton is not signed-in, **dwUserIndex** in the [NUI_ENROLLMENT_INFORMATION](#) structure is [XUSER_INDEX_NONE](#).

Unidentified Skeleton

In the context of the NUI identity system, *unidentified* specifically means that **NuiIdentityIdentify** has been not been run for the skeleton, to perform face and skeleton recognition. The *dwEnrollmentIndex* for such a skeleton is [NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY](#), in the [NUI_SKELETON_DATA](#) structure.

Non-Enrolled Guest Player

A *non-enrolled guest player* has no identity record with the title. When a skeleton becomes visible and is being tracked, but **NuiIdentityIdentify** has not been called, if the skeleton indicates intent to play, this is a non-enrolled guest player.

A non-enrolled guest player can interact with a title, including menu screens, and gameplay if the title allows that, but is not recognized uniquely when the skeleton disappears and then reappears. Thus a different guest name might be used for the player after reappearing, and the title might not maintain a score for such a player. This player type can be used during gameplay periods where player identity does not affect game play, such as UI navigation or an interactive screensaver.

If a non-enrolled guest player indicates intent to play, the title can optionally enroll the player, and should provide the opportunity for the player to correct their identity and sign-in to a gamer profile.

The category of *non-enrolled, signed-in player* isn't directly supported. If a player is already manually signed-in to a gamer profile before a title starts, but the player has not been enrolled in the session identity store, the player could play as a non-enrolled guest player.

Enrolled Guest Player

An *enrolled guest player* is uniquely identified within a game session. When a player is enrolled, the player can leave the field of view, and return, and be identified correctly by the game as the same person that had previously left. This allows continuity of game play despite any temporary absence of an enrolled player.

Enrollment is the process of adding a player to the NUI identity system. Information such as height or a facial descriptor is saved locally for later use in recognizing the player when the player disappears and reappears. When the skeleton is enrolled, the NUI identity system stores the detected face and skeleton dimensions in the session identity store using a particular enrollment index, 0-7. The NUI identity system can maintain up to eight enrolled players at a time.

Typically, a game enrolls a skeleton if the skeleton indicates an intent to play. This intent to play can be expressed as a simple gesture, such as a hand wave, or it can be as complex as the game requires.

After detecting an intent to play, the title should associate a game asset or identification with the player. For example, it could be an avatar provided by the game or a game title such as "the red player." From this point on, the game can accumulate data about the player, including the player's game performance and choices during gameplay. When the player returns for another turn, the game can perform any number of unique features such as welcoming the player back by name, automatically placing the associated avatar properly in the game space, and informing the player of the player's current standing.

Enrolled, Signed-in Player

An *enrolled, signed-in player* has an enrollment index, a user index, a quadrant of the Ring of Light, and a signed-in gamer profile. Up to four players from the currently enrolled player pool can be signed in. These players do not have to be in any type of order or position within the enrolled array.

An enrolled, signed-in player remains enrolled and signed-in when the title exits and the next title starts. This means that an enrolled, signed-in player can start a new game and not have to enroll again. In contrast, enrolled players who were not signed-in must be enrolled again by the new game.

An enrolled, signed-in player has the benefits of a gamer profile, including a player-defined avatar, the ability to win achievements, access to Xbox LIVE, and a method of saving game data. A title should encourage sign-in of the first player, because this provides a place to save related game data.

Borrowing a Gamer Profile

There are some scenarios where a player signs in to another player's gamer profile. This discussion is not directly related to the TCR about biometric sign-in.

Signing In Manually to Someone Else's Biometric-Enabled Profile

Anyone can sign-in manually to any Kinect ID-enabled profile, which is a biometric-enabled gamer profile. If a person other than the true profile owner signs-in manually to a Kinect ID-enabled profile, if the true owner subsequently appears, biometric sign-in won't occur for the owner.

Family Profiles

A family profile may be associated with multiple players who share a single profile. You should be aware of this possibility, but family profiles normally won't affect your title, because profiles are managed by the Xbox 360 system.

See Also

Glossary for NUI Identity

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Forcing Enrollment Due to Quality Issues

See Also



When [NuiIdentityIdentify](#) is called, it may fail due to quality issues. Quality issues are problems with the clarity of the color stream that prevent checking whether a skeleton that has become visible is recognizable as an enrolled player or matches a biometric-enabled gamer profile. The possible quality issues are listed in [NUI_IDENTITY_QUALITY Flags](#).

The sensor requires a certain positioning and an adequate amount of light to identify a player. If a player is too far away or too close, or if the sensor is aimed too high or too low, data is incomplete and detection will fail. Identity detection will also fail if light is inadequate for detecting appropriate levels of shape and color.

It is sometimes necessary for a title to force enrollment of a player. If the identity systems fail to enroll the player in the normal way, for example, a title will have little choice but to forcibly enroll the player. Doing so will allow the title to treat the player in much the same way it would treat a player who has enrolled normally (whether the player is bound to a profile or not). However, forced enrollment won't allow automatic re-identification of the player when tracking is lost, because the system won't have a biometric print of the player to match against.

If tracking of a player who was forcibly enrolled is lost (either accidentally or intentionally on the part of the player), the player must go through normal identification and enrollment, or the player will likely be forcibly enrolled again when the player becomes visible again. Furthermore, if the player was bound to a gamer profile that isn't biometric-enabled, the player will have to again select the gamer profile to use.

You must determine how important it is for your title to identify the player in any given the situation. For example, if tracking is lost and re-acquired in the middle of a player's turn, the title is welcome to let the player continue to play as the player whose turn it is, without re-identifying or force-enrolling the player.

Force-enrollment causes the identity system to always allocate an enrollment index, even if current conditions would prevent it from successfully generating the data it needs to properly identify the player when the player returns, such as [NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK](#) or [NUI_IDENTITY_QUALITY_USER_OCCLUDED_FACE](#). If you force the enrollment, the system attempts to generate the required data. If the enrollment must be forced, and the player exits the field of view and returns into the field of view, the system is not able to identify that player, and you must re-enroll the player so that the player is associated with the correct enrollment index.

During force-enrollment, the existing enrollment data for the given enrollment index is destroyed and replaced with fresh enrollment data for the re-enrolling player. The player who was formerly enrolled at that enrollment index is identified as a new player the next time that player appears. That player will have to go through the re-enrollment process as well, to be assigned to the proper enrollment index.

Per [TCR 165: NUI Biometric Sign-In](#), if an identity operation cannot be completed due to quality issues, no further action is required for that skeleton. It is not required that you force-enroll the player if there are quality issues preventing checking whether the skeleton is enrolled.

For more information on force-enrollment, see [NuiIdentityEnroll](#).

See Also

Glossary for NUI Identity

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Maximum Number of Skeletons Tracked

See Also



NUI supports multiple enrolled, center-tracked, joint-tracked, and signed-in players.

Type of player tracking	Description	Maximum number supported
Enrolled players	The number of skeletons that NUI can readily, immediately distinguish from each other during a title session.	8
Simultaneously tracked players	The number of skeletons that NUI can track at least the center position of. A passively tracked skeleton has only its center position tracked. Passively tracked skeletons are flagged with the NUI_SKELETON_POSITION_ONLY enumeration constant. This total of 6 includes up to 2 skeletons that may be actively tracked; that is, 4 passively tracked (center-tracked) skeletons and 2 actively tracked (joint-tracked) skeletons.	6
Simultaneously signed-in players	The number of players with gamer profiles who are signed-in.	4
Actively tracked skeletons	The number of skeletons that NUI can return all joint positions for. An actively tracked skeleton has all its joint positions tracked. Actively tracked skeletons are flagged with the NUI_SKELETON_TRACKED enumeration constant.	2

There is a direct link between the data in the depth mask and the skeleton data structures. The mask uses values from 1 to 7 to distinguish the player pixels from the rest of the environment, which uses 0. Subtracting 1 allows you to directly index one of the skeleton data structures.

A fully tracked skeleton (up to 2 of the total 6) can be linked to an identity store in memory. It can also be linked to a profile if that identity is bound to one.

Unenrolling a Player

A title can unenroll a player, as long as the player isn't signed-in to a profile. This can be used to allow new players to enroll when the limit of 8 enrolled players has been reached. Call [NuiIdentityUnenroll](#).

How the User Index Is Assigned

Like controllers, NUI-identified users are assigned the lowest vacant user index. For example, if a player powers up the console, plugs in a controller, and then signs in using the NUI identity system, the controller and the skeleton are bound to user index 0.

To associate a NUI-identified player with a controller, the player doesn't need to be signed-in. The title can manage the association of a NUI enrollment index to a user index of a controller.

✚ See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Players Can Come and Go at Any Time

[See Also](#)



In a Kinect-enabled title, a player can disappear and reappear at any time, and a title should always be ready to react to this situation. Though the exact reaction will vary depending on the type of game, the title should always acknowledge the change in player line-up and clearly indicate to the players what changes are taking place and how it will impact the game in progress. For some games, this will be an expected and normal behavior, while for others it will be exceptional and may have a profound impact, possibly even halting the game in progress. In all cases, the title must steer the player through the process.

There are many reasons for a player to disappear temporarily. For example:

- The Kinect Sensor's view of the player is blocked by an object or by another player.
- The player fell to the floor.
- The player stepped out of the play space.
- Someone turned the lights off.

In these cases, when the player becomes visible again, the player will be immediately recognized (*identified*) again. Most of the time everything will happen so quickly the player won't notice, but there is no guarantee, so the title must be ready to guide players at all times.

Pausing the Game After Tracking Is Lost

If tracking is lost, depending on the style of gameplay, the title could decide to pause the game until tracking is reacquired. Your title should not pause immediately; the player may step right back in.

Your title dictates how long a player should disappear before the game is paused. As a rule of thumb, if tracking is lost for more than a few seconds, the game should enter its pause mode until the lost player or players are re-acquired.

Guiding Through Enrollment if Auto-enrollment Fails

Sometimes auto-enrollment will fail after the identity system attempts to re-identify a player. This usually happens because the identity system cannot single out the corresponding enrollment data among all the enrolled players. When this happens, the title must guide the player through the enrollment process and possibly ask the player to select a profile again.

Various factors may cause auto-enrollment to fail:

- Two or more player may be too similar.
- The lighting conditions may have changed since the original enrollment.
- Players may have changed clothing.

Signing in Through the Kinect Guide

In addition to signing in within the title, players can also sign in or sign out through the Kinect Guide. The Kinect Guide automatically enrolls players before signing them in, and it will automatically un-enroll a player who signs out. The Kinect Guide also provides the following:

- Lets the player play as someone else.
- Always forces enrollment, so there can be profiles signed-in without skeletons attached.
- Uses a new enrollment index if a signed-in player signs into a different profile.
- Lets a player complete the sign-in operation even if the player is not the person who opened the Kinect Guide.

✚ See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Carry-Over Players

[See Also](#)



If a player was already enrolled in the session identity store and signed-in to a gamer profile, the player remains enrolled when the title starts. An enrolled, signed-in player can start a new Kinect title and not have to enroll again. In contrast, an enrolled player who was not signed-in must be enrolled again by the new game. The title should not attempt to actively engage with a carry-over player until the player shows intent to play.

During a title's initialization phase, the NUI identity system looks for players who are already enrolled and signed-in. The title can use that information to speed up the enrollment process, allowing an enrolled, signed-in player to get into the new game faster.

A title should honor existing enrollments at startup. Such an enrollment represents a player who is already signed in, and there's no reason to make the player select their gamer profile again. The title should not enroll an already enrolled, signed-in player as a guest player, because that would lose the connection to the player's gamer profile.

When starting, game code can poll for persisted enrolled, signed-in players by calling [NuiIdentityGetEnrollmentInformation](#). If the returned **NUI_ENROLLMENT_INFORMATION.dwEnrollmentFlags** value for a given enrollment index is zero, then that enrollment index is not in use. Otherwise, the enrollment index refers to an enrolled, signed-in player, and the returned **NUI_ENROLLMENT_INFORMATION** structure contains information about that player.

✚ See Also

[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Automatic Identity Handling

[See Also](#)



Automatic identity handling identifies skeletons when they become visible and enrolls and signs-in skeletons that have matching biometric profiles, automatically handling the details of identification behaviors to meet the minimum TCR requirements.

Automatic identity handling is recommended for all titles. Automatic identity handling provides the baseline NUI identity features that are required, with minimum effort. There is no need to implement NUI identity handling from the ground up. To add custom behavior, make additional calls to the NUI identity API.

Sections

- [The Minimum Required Behaviors](#)
- [The Minimum Required Coding](#)
- [TCRs Addressed by Automatic Identity Handling](#)
- [Deciding How Much Customization to Use](#)
- [Difference Between Automatic and Manual Identity Handling](#)
- [Enabling Automatic Identity Handling \(NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY\)](#)
- [Enrolling Players Who Show Intent to Play \(NuiIdentityDetectedIntentToPlay\)](#)
- [Flags](#)

The Minimum Required Behaviors

Automatic identity handling helps your title to pass the NUI identity TCRs, by providing the following required behaviors.

Identify all new players

Whenever a skeleton becomes visible, the NUI identity system attempts (one time only) to run [NuiIdentityIdentify](#) on the skeleton to determine whether the skeleton is enrolled, and whether the skeleton has a matching biometric-enabled gamer profile.

Sign-in players that have a biometric match

If a skeleton that becomes visible is not enrolled, has a matching biometric gamer profile, and the title reports that the skeleton indicated intent to play, the NUI identity system enrolls that skeleton in the session identity store, and signs in the skeleton to the matched biometric-enabled gamer profile.

Don't enroll guest players

If the player doesn't have a biometric profile match, the NUI identity system doesn't enroll the player when the title indicates that the player has indicated intent to play. Optionally, to enroll the player, the title can call [NuiIdentityEnroll](#) directly.

The Minimum Required Coding

Handling identity in a Kinect title means keeping track of detected skeletons as unique players. Do the following to get NUI identity running at a baseline level and pass the NUI identity TCRs. This is often sufficient for a single-player game.

- Pass the flag [NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY](#) to [NuiInitialize](#).
This flag causes the NUI identity system to automatically identify any newly detected skeletons.
- Call [NuiIdentityDetectedIntentToPlay](#) when a user indicates intent to play. You call this once for each player who shows intent to play.
Your title defines what action by the player indicates intent to play. **NuiIdentityDetectedIntentToPlay** causes the player to be enrolled if and only if the skeleton's incoming biometric print matches any of the biometric-enabled gamer profiles on the console.

In addition to the baseline functionality provided by the NUI identity system, you can make additional calls to the NUI identity API as required for your title. For example:

- If your title requires temporarily accessing the RGB color stream (to take a snapshot, for example), call [NuiIdentityPause](#) to suspend automatically calling **NuiIdentityIdentify**. (While **NuiIdentityIdentify** is running, it uses the RGB stream and sets custom camera settings to optimize the biometric recognition of players.)
- If your title needs to do some additional identity processing, call [NuiIdentityGetNextMessage](#). For example, if identity quality flags indicate a problem, you can optionally display a user guidance UI to help identify the player.

TCRs Addressed by Automatic Identity Handling

Subject	TCR	Explanatory topic
When a player indicates intent to play, if the player has a matching biometric-enabled gamer profile, the player is automatically signed-in.	TCR 165: NUI Biometric Sign-In	Supporting Biometric Sign-In
The player must be shown their identity and given an opportunity to correct their identity, before the title can do anything that would affect a gamer profile, such as granting points.	TCR 158: NUI Identity Player Feedback	Enabling Players to Correct Their Identity

Deciding How Much Customization to Use

The following alternatives provide a choice of ways to design and implement your title's identity process.

- If your title only requires baseline functionality per the NUI identity TCRs, you can simply use automatic identity handling, with no additional use of the NUI identity API. This works well for typical single-player games.
- To meet the baseline functionality per the NUI identity TCRs, but provide greater control and additional behavior, use automatic identity handling, with added calls to the NUI identity API.
- To use the NUI identity API entirely at the lower level, call **NuiIdentityIdentify** and **NuiIdentityEnroll** directly. This approach provides your title with greatest control of when **NuiIdentityIdentify** is run and thus of when the RGB color image stream is directly available to the title, and available for head orientation tracking. However, this approach is the most difficult to implement, and provides the least assistance in complying with the NUI identity TCRs. Most titles do not require this level of control.

For expanded discussion of the two main NUI identity TCRs, see [Supporting Biometric Sign-In](#) and [Enabling Players to Correct Their Identity](#).

Difference Between Automatic and Manual Identity Handling

Many titles don't require custom implementation of NUI identity features, such as directly calling **NuiIdentityIdentify** and **NuiIdentityEnroll** for every detected skeleton. A title can solely rely on automatic identity handling if the title only requires the minimum implementation per the NUI identity TCRs.

The NUI identity system automatically runs **NuiIdentityIdentify** on all skeletons that become visible, and when a title reports that a player has indicated intent to play, the NUI identity system enrolls and signs-in that player, only if the player has a matching biometric-enabled gamer profile. If the player doesn't have a matching biometric-enabled gamer profile, the player can play as a non-enrolled guest player, or your title can directly call **NuiIdentityEnroll** to enroll the player.

A title can manually handle identity (defining the timing of uniquely recognizing a skeleton). The behavior must fulfill the NUI identity TCRs.

Mode	Advantage	Disadvantage
Automatic Identity Handling	• Provides a quick way to implement basic identity behaviors. • Automatically does what is required by the NUI identity TCRs. • It can easily be augmented by calling the NUI identity API. • There's no added resource cost for using automatic instead of manual identity mode. • To monitor quality issues, call NuiIdentityGetNextMessage.	You can't precisely control when NUI identity operations occur and take over the RGB color image stream, particularly NuiIdentityIdentify .
Manual Identity Handling	• You can control when NUI identity operations start. This allows you to delay the operation for a few frames. • You can control when NUI identity operations end. This allows you to stop an operation early. You must remember to re-launch the operation later. • Useful if the title needs intensive usage of the color stream. • Useful if the title cannot afford the CPU hit during a particular section. • You can optimize performance. • You can precisely control when players are identified or enrolled. • You can precisely control when individual operations time out. • To monitor quality issues, pass a callback function to NuiIdentityIdentify or	You must ensure that you follow all NUI identity TCRs; conformant behavior isn't automatically provided.

[NuiIdentityEnroll](#).

Enabling Automatic Identity Handling (NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY)

To use the NUI identity API in your code, `#include NuiApi.h`, which includes `NuiIdentity.h`.

To enable automatic identity handling, when calling [NuiInitialize](#), pass the flag `NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY`.

Regardless of whether automatic identity handling is enabled, the NUI identity system also requires that you pass the following flags to **NuiInitialize**:

- `NUI_INITIALIZE_FLAG_USES_SKELETON`
- `NUI_INITIALIZE_FLAG_USES_COLOR`
- `NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX` or `NUI_INITIALIZE_FLAG_USES_DEPTH`

Enrolling Players Who Show Intent to Play (NuiIdentityDetectedIntentToPlay)

The NUI identity system runs [NuiIdentityIdentify](#) on all skeletons that become visible. Biometric sign-in (including enrolling a player in the session identity store) is only required for players who show intent to play, not for passive observers. The title informs the NUI identity system that a skeleton indicated intent to play (as defined by the title), by calling [NuiIdentityDetectedIntentToPlay](#). If the skeleton matches a biometric-enabled gamer profile, the NUI identity system enrolls the specified skeleton in the session identity store, and signs the skeleton into the gamer profile.

When the title calls **NuiIdentityDetectedIntentToPlay**, if the **NuiIdentityIdentify** operation has not already completed on the specified skeleton, the NUI identity system will wait to complete the **NuiIdentityIdentify** operation, before enrolling and signing-in the player.

If **NuiIdentityIdentify** failed earlier due to quality issues preventing recognizing the player before **NuiIdentityDetectedIntentToPlay** was called, the NUI identity system will call **NuiIdentityIdentify** again, to try to recognize (*identify*) the player. After **NuiIdentityIdentify** has been successfully run, the NUI identity system will then do one of the following:

- If the skeleton is already enrolled, the NUI identity system does nothing, and the title can start gameplay.
- If the skeleton is not enrolled and has a matching biometric-enabled gamer profile, the NUI identity system runs [NuiIdentityEnroll](#) for the skeleton, and signs in the skeleton to the matching gamer profile.
- If the skeleton is not enrolled and doesn't have a matching biometric-enabled gamer profile, the NUI identity system does nothing. The title can then optionally allow the player to sign-in to a gamer profile manually, optionally call [NuiIdentityEnroll](#), and then start gameplay.

Timing Out when Identifying a Skeleton

With automatic identity handling, if an identify operation on a skeleton times out, the NUI identity system will not try to identify that skeleton again until the title calls [NuiIdentityDetectedIntentToPlay](#). With automatic identity handling, the timeout value is about 60 seconds, for [NuiIdentityIdentify](#) and for other identity operations.

Per [TCR 166: NUI Identity Maximum Delay](#), the perceived wait time for a NUI identity operation must be no more than 15 seconds.

Flags

Automatic mode runs [NuiIdentityIdentify](#) automatically on each new skeleton that appears in the Kinect Sensor's field of view. As a result, when a new skeleton enters the field of view, its [dwEnrollmentIndex](#) will be `NUI_IDENTITY_ENROLLMENT_INDEX_BUSY`, not `NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTITY`.

- `NUI_IDENTITY_ENROLLMENT_INDEX_BUSY` means that the skeleton is involved in a [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) operation.
- `NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTITY` means that the NUI identity system hasn't tried to identify the skeleton yet and therefore the title needs to call [NuiIdentityIdentify](#). With automatic identity handling, this situation could happen if the automatic call to **NuiIdentityIdentify** failed after the timeout period.

+ See Also

[NuiIdentity Reference](#)
[Simple Identity Sample](#)
[Nui Samples: Identity](#)
[NUI White Papers: Identity](#)
[Glossary for NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Glossary for NUI Identity



The following terms are used in NUI Identity, for distinguishing between different skeletons as specific players based on biometric data.

actively tracked skeleton

A skeleton that NUI can return individual joint positions for. Also called *fully tracked*. Compare *passively tracked (position-only tracked)*.

associating a player with a controller

When using a controller rather than the Kinect Sensor, a player is associated with a controller, which is identified by a user index. When using the Kinect Sensor and the NUI identity system, a detected skeleton has a skeleton tracking ID and is associated with an enrollment index and user index, through a biometric print.

automatic identity handling

Implements the minimum required identity behavior. Recommended for most titles, to provide the mandatory baseline identity functionality. When a new skeleton is detected by the Kinect Sensor, the skeleton is marked as needing to be identified. The NUI identity system monitors that flag and calls [NuiIdentityIdentify](#) when the flag is set. The function call is attempted one time, meeting the minimum requirements.

When the title indicates that a particular player has indicated intent to play (as defined by the title), instead of having to conditionally call [NuiIdentityEnroll](#) based on whether the player has a biometric profile, the title simply calls [NuiIdentityDetectedIntentToPlay](#) regardless of the kind of player. The NUI identity system handles determining whether to call **NuiIdentityEnroll**, and calls that function if there's no matching biometric profile.

automatic sign-in

See *biometric sign-in*.

avatar

An onscreen depiction of a player, for a guest player or a signed-in profile.

bind

The act of associating a gamer profile and a biometric print. The NUI identity system manages the binding process; it is transparent to the title.

biometric-enabled gamer profile

See *biometric profile*.

biometric identity data

See *biometric print*.

biometric print

The biometric data that uniquely identifies a player and enables the NUI identity system to recognize the player by matching the incoming biometric print to the biometric prints in the session identity store and the biometric prints saved with biometric profiles. This data includes biometric information such as facial attributes and skeleton dimensions.

biometric profile

A gamer profile that has biometric data (a biometric print) stored with it on the console drive. This is a regular gamer profile, but includes additional, biometric data. In contrast, an enrolled guest player has only a *session biometric print*, in the session identity store, which is non-permanent. Many biometric profiles may be stored on a console. A biometric profile enables biometric sign-in.

In the UI, a biometric profile is presented to the player as a *Kinect ID*, which is a *Kinect ID-enabled profile*. Also called a *biometric-enabled profile*.

biometric profile match

There's a biometric profile match when the incoming biometric print from the Kinect Sensor matches any available biometric profile stored on the Xbox console or available to the console. Equivalent to *matching biometric profile*.

biometric sign-in

The [NuiIdentityIdentify](#) function may indicate that a particular detected skeleton is not enrolled and has a biometric profile match. If a player who has a biometric profile shows intent to play, the title must enroll the player. When a title enrolls a player with a biometric profile, the NUI identity system also automatically signs-in the player. From the player's perspective, this is *automatic sign-in* using *Kinect ID*. See also *sign-in*.

carry-over player

A player who is already enrolled in the session identity store and signed-in to a gamer profile when a title is launched. The player may have signed-in using the Xbox Dashboard or another title. All signed-in players are retained when a Kinect title starts. For such players, the session identity store can be used across sessions of different titles. A guest player is not a carry-over player; neither a guest player's identity or scores are retained when the game session ends. See also *game session*.

correcting identity

The player might see that the NUI identity system has misidentified them as a different player, so that the incorrect gamer profile is shown. In that case, the player can manually select the correct gamer profile or create a new gamer profile to be associated with their biometric print. If the player selects an identity that is already enrolled in the session identity store, this is called *forcing enrollment due to misidentification*.

enroll

Assign an enrollment index to a skeleton detected by the Kinect Sensor. A player is identified, then enrolled, then signed-in. To *enroll a player* is to associate an incoming biometric print from the Kinect Sensor with a particular enrollment slot in the session identity store, which stores the biometric print in the enrollment slot. A title must get consent from a player before enrolling the player as a guest player.

enrolled player

A tracked skeleton that is associated with an enrollment index in the session identity store. An enrolled player is enrolled as a known skeleton, through a biometric print.

- A skeleton that is enrolled and signed-in with a biometric profile has a biometric profile linked to the enrollment slot.
- A skeleton that is enrolled and signed-in without a biometric profile has a gamer profile linked to the enrollment slot.
- A skeleton that is enrolled and not signed-in is a guest player. There's no gamer profile linked to the enrollment slot.
- A skeleton that is force-enrolled due to quality issues and is signed-in has a gamer profile linked to the enrollment slot. Such a skeleton has no session biometric print in the session identity store, so if the player disappears and reappears, the player will need to be force-enrolled again, to resume using the same gamer profile and scores.
- A skeleton that is force-enrolled due to quality issues and is not signed-in is a guest player, and does not have a gamer profile linked to the enrollment slot. Such a skeleton has no session biometric print in the session identity store, so if the player disappears and reappears, the player will need to be force-enrolled again, to resume using the same guest player identity and scores.
- A skeleton that is not enrolled and is not signed in is an unknown guest player. Such a skeleton can't be consistently, uniquely identified if the player disappears and reappears; the title then doesn't know that the newly visible skeleton is the same player who just left the field of view, though in some types of gameplay, the title may assume the newly visible skeleton is the same player.

See [Levels of Player Identity](#).

enrolled player database

See *session identity store*.

enrollment

The process through which the incoming biometric print that identifies a particular skeleton as a particular player is added to the session identity store.

enrollment index

The index of an enrollment slot, 0-7, used to keep track of player session identities in the identity session store. The enrollment index corresponds with 8 players whose scores can be maintained and updated during a game session, of which 2 players can be fully tracked simultaneously.

In contrast, a user index, 0-3, corresponds with the concept of 4 game controllers, 4 controller ports, and the 4 quadrants of the Ring of Light. When a skeleton become actively tracked (fully tracked), one of the user indexes is re-mapped to that skeleton. The user index in a skeleton data structure should not be trusted unless the enrollment index in that structure is valid, such as 0-7.

Compare *skeleton tracking ID* and *user index*.

enrollment slot

The session identity store has 8 enrollment slots, each with an enrollment index 0-7. The enrollment slot for an enrolled player contains a biometric print, unless the player has been force-enrolled due to quality issues. An enrollment slot may contain an enrolled guest player or a signed-in player.

entering play

Beginning to play the actual game, not just using the initial lobby or sign-in portions of a title. A title can allow a player to enter play whether or not the player is enrolled. If the title is in the lobby, the title is not in gameplay. To play the game, a player must show intent to play, but doesn't need to be enrolled.

feedback

The main kinds of feedback to the player during identification operations are the following:

- Feedback about quality issues, such as too dark to form the incoming biometric print. The title can tell the player to adjust these conditions.
- Feedback about who the NUI identity system thinks the player is, such as a particular guest or signed-in gamer profile.

first player

After title bootup, the first player to be detected by the Kinect Sensor and identified by the NUI identity system.

forcing enrollment due to misidentification

Forcing enrollment due to misidentification, when player A has been incorrectly identified as player B: To do this, you call **NuiIdentityEnroll**, passing the enrollment index to use. This forces the incoming biometric print to be linked with the enrollment slot indicated by the player. Also called *re-enrolling*; see *re-enroll*.

Equivalent usages: *force-enrolled due to misidentification*, *force-enroll to correct misidentification*.

forcing enrollment due to quality issues

If there is a quality issue during the [NuiIdentityIdentify](#) or **NuiIdentityEnroll** operation, such as "too dark", no incoming biometric print can be formed, and therefore the NUI identity system couldn't attempt to determine whether the player is an enrolled (known) player. To enable the player to play the game, the title can treat the detected player as a non-enrolled guest player, or can force-enroll the player as a guest player, which consumes an enrollment slot and typically means pausing the game and showing an enroll screen.

Forcing enrollment due to quality issues won't allow automatic re-identification of the player when tracking is lost, because the system won't have a biometric print of the player to match. In this forced enrollment, like with a non-enrolled player, no biometric print is saved in the session identity store. If a skeleton with no biometric print saved in the session identity store disappears and then reappears, the NUI identity system can't determine that the newly visible skeleton is the same player who disappeared. The title would then force-enroll the skeleton again, which typically means pausing the game and showing an enroll screen again.

Equivalent usage: *force-enrolled due to quality issues*. See also *quality issues*.

fully tracked

See *actively tracked*.

game asset

Items that can be associated with a gamer profile, such as an avatar. These can be displayed as part of feedback showing which player the NUI identity system thinks the detected skeleton is.

game session

Playing a title during a particular unique period of time. The game session begins when the title starts, rather than when actually playing the game starts. The game session ends when a different title is booted or the console is powered off. See also *carry-over player*.

gamer profile

Information about a unique player. Saved on the console drive and used in Xbox LIVE. A gamer profile that includes a biometric print is a *biometric profile*. Also called *Xbox profile*.

guest player

A guest player is a player that is tracked with skeleton tracking but is not signed-in to a gamer profile. A guest player may be enrolled or not enrolled. A title must get consent from a player before enrolling the player as a guest player. If an enrolled guest player disappears and reappears, that player can be consistently identified, so that player's scores can be updated throughout the session.

An enrolled guest player is unenrolled when the game session ends; a guest player is not a *carry-over player*. If an unenrolled guest player disappears and reappears, that player cannot be consistently identified, so that player's scores cannot be reliably updated throughout the session. You can use the term 'Guest' or 'Guest Player' in the UI text of your game, but use other text if appropriate for the theme of your game. See also *force-enrolled due to quality issues*.

identification

The process through which the NUI identity system attempts to recognize a player.

When a skeleton is marked as newly detected, the NUI identity system compares the incoming biometric print against the biometric prints for the currently enrolled guest players and signed-in players, in the session identity store. If a match is found in the session identity store, the newly detected skeleton is recognized as a particular enrolled player, and bound to that player's session identity (stored in an enrollment slot). If a match is found in a biometric profile that isn't enrolled or signed-in, the NUI identity system indicates this, and the title must directly or indirectly call [NuiIdentityEnroll](#) for that skeleton.

With automatic identity handling, a title doesn't need to initiate an identification operation; the NUI identity system calls [NuiIdentityIdentify](#) when a skeleton becomes visible.

To directly initiate an identification operation, call **NuiIdentityIdentify**.

identified as unknown

See *unknown player*.

identify

To identify a player is to determine whether the player is enrolled in the session identity store, and at which enrollment index, by comparing the incoming biometric print to the biometric prints in the session identity store. If the player isn't enrolled, the incoming print is compared to each available biometric-enabled gamer profile to see whether or not there is a matching profile.

The result of the [NuiIdentityIdentify](#) call is stored in the skeleton data structure. There are 3 main possible outcomes of the **NuiIdentityIdentify** call:

- There was a quality issue such as low light, so an incoming biometric print couldn't be created; the NUI identity system couldn't attempt to find a match.
- The player is enrolled in the session identity store, at an enrollment index, 0-7.
- The player is not enrolled in the identity session store. If a match was found in the available biometric profiles, a flag indicates this.

identity failed due to quality issues

There is a quality issue such as "too dark", so an incoming biometric print couldn't be formed, and therefore the NUI identity system couldn't attempt to find a biometric match in the session identity store or among the biometric profiles.

identity failed to find a match

The NUI identity system didn't find a match between the incoming biometric print and the biometric prints in the session identity store. That is, the player is not enrolled, and is thus considered "unknown" to the session identity store and in the tracked skeleton's data structure.

identity failed due to misidentification

The system misidentified player A as player B. The title must provide feedback of who it thinks the player is, so that the player may correct their identity.

identity operation

Typically means the **NuiIdentityIdentify** or **NuiIdentityEnroll** function.

identity system

See *NUI identity system*.

intent to play

The way players indicate that they want to engage with the title. The player indicates by a defined gesture or button press that they want to play the game or in some way engage with the title. The title defines what action by the player constitutes indicating intent to play. See also *entering play*.

Kinect ID, Kinect ID-enabled profile

See *biometric profile*.

LIVE profile

An Xbox LIVE-enabled profile, which allows players to connect to LIVE. A LIVE profile might or might not be a biometric profile.

lobby

Displayed by the title prior to gameplay. Enables indicating intent to play. Provides a place to show a player what gamer profile is associated with their biometric print and skeleton and allow the player to correct their identity (that is, associate their tracked skeleton with a different gamer profile).

matching biometric profile

See *biometric profile match*.

misidentify, misidentification

When a skeleton becomes visible, and **NuiIdentityIdentify** is called, and the incoming biometric print seems to match an enrolled session identity or biometric profile match of someone other than the player. As part of required feedback, a matched guest identity or a gamer profile must be displayed to the player, to allow the player to correct any misidentification, by picking the correct gamer profile, or creating a new gamer profile. If the player selects an enrolled session identity (an already enrolled player), the title then uses *force-enroll* to correct misidentification.

new player

A skeleton that has become visible, has indicated intent to play, and is not enrolled in the session identity store.

In contrast, an *existing player* is already enrolled in the session identity store as an enrolled guest player, a manually signed-in player, or a biometrically signed-in player. An existing player may also be a player who has been playing as a non-enrolled guest player.

NUI identity system, NuiIdentity

The NUI identity system enables a player who is detected by the Kinect Sensor to be recognized and associated with a particular gamer profile or guest player, and enables differentiating the skeleton from other skeletons as a specific player, particularly when the skeleton disappears and reappears. NUI Identity enables recognizing a skeleton as belonging to a particular player, providing continuity of scores and identities within a session of using a title, while also supporting casual jump-in play with a collection of guest and signed-in players, without requiring a player to create a gamer profile and sign-in to it.

To differentiate detected skeletons as particular players, a Kinect title uses gamer profiles and user indexes, but instead of controllers and ports that are differentiated, a Kinect title uses the NUI identity system, including enrollment slots with enrollment indexes in the session identity store, and optional biometric-enabled gamer profiles.

passively tracked, position-only tracked

A skeleton for which only the center position is tracked, rather than having all joints tracked. Compare *actively tracked* (*fully tracked*).

player, gamer

Any person who is detected by the Kinect Sensor, particularly if the player indicates intent to play. Each time the player becomes visible to the Kinect Sensor, the player is uniquely recognized by the player's incoming biometric print, which is formed from face recognition (using the color stream) and skeleton dimensions.

In each frame of skeleton tracking data from the skeleton tracking pipeline, the player is represented by a skeleton data structure that has a skeleton ID to distinguish it from the other skeletons in the frame. The skeleton is enrolled with an enrollment index (0-7), which is associated with a user index (0-3). These 3 values are stored in the skeleton data structure.

print

See *biometric print*.

profile

An Xbox gamer profile, stored on the console or a memory card. A game profile that includes biometric identity data is a *biometric profile*.

quality issues

Problems with the clarity of the color stream that prevent checking whether a skeleton that has become visible is recognizable as an enrolled player or matches a biometric-enabled gamer profile. To identify a detected skeleton as a particular known player or as an unknown person, the player must be in the field of view, facing the Kinect Sensor, with adequate lighting.

re-enroll

Saving a biometric print into an existing enrollment slot, overwriting the biometric print that was previously in that enrollment slot. Re-enroll saves the player's biometric print to an enrollment slot that is already used rather than occupying a new enrollment slot. Re-enrollment is typically used to correct the enrollment data for a player in the case where an already enrolled player is mis-identified as a new player or as a different enrolled player. Re-enrolling can also be used to enroll a different person into a specific enrollment slot. Also called *forcing enrollment* due to misidentification.

re-identify

Sense 1: When a skeleton disappears and returns into view, the NUI identity system runs the [NuiIdentityIdentify](#) routine on the newly detected skeleton. (See *identify*.) If the skeleton is identified as a player who was recently visible, the NUI identity system has re-identified the player.

Sense 2: If a skeleton is being tracked but the player couldn't be identified, the player is a non-enrolled guest player. For example, when **NuiIdentityIdentify** was called, the lighting was too dark, preventing deriving a useable biometric print. The title can attempt again to identify the player by calling **NuiIdentityIdentify**; that is, re-identify the player.

Ring of Light

The Ring of Light consists of four LEDs on the front of the Xbox console, each of which illuminates a single quadrant. Each quadrant corresponds with a user index. The quadrants light up for the user indexes that are mapped to the active (that is, fully tracked) skeletons, for enrolled, signed-in players. See also *user index*.

session biometric print

See *session identity print*.

session identity

The unique name and scores of a player during a session of using a title. Enrolling a skeleton creates a session identity for the player. A guest player's session identity lasts for the entire session of using the title. The session ends when the title is exited; then the guest player's identity and scores are lost. A signed-in player's session identity is carried over into the next Kinect title that is played. A session identity is lost if the console is rebooted. A session identity provides a way to bind together a detected skeleton, a biometric print, and a guest identity (in the case of a player who isn't signed in), or a gamer profile (in the case of a signed-in player). A session identity is stored in a particular enrollment slot in the session identity store.

session identity match

The incoming biometric print is first matched against the session identity prints, which are stored in the session identity store. If the incoming biometric print matches a session identity print, that constitutes a *session identity match* (not a biometric profile match).

If no match is found in the session identity store, the incoming biometric print is matched against any biometric profiles on the console. This constitutes a *biometric profile match*.

session identity print

The biometric print that is stored in an enrollment slot in the session identity store for the duration of the session of using the title. The session identity store can contain up to 8 session identity prints, in memory. Session identity prints only persist while the console is turned on. Compare *biometric profile*, which saves the biometric print to the console drive and is thus not volatile. Also called a *session biometric print*.

session identity store

The list of 8 enrollment slots. Each enrollment slot is identified by an enrollment index, 0-7. Up to 4 enrollment slots may contain a signed-in player. The remaining slots may contain guest players. Each enrollment slot can contain a session biometric print, as part of that player's session identity.

The session identity store only lasts for the duration of a title's session; the identity session data is retained until the player goes to the dashboard or another title. Signed-in players are carry-over players; they remain signed-in across sessions of different titles.

When a skeleton becomes visible, a flag is set, and the title automatically or manually calls **NuiIdentityIdentify**. **NuiIdentityIdentify** compares the incoming biometric print from the Kinect Sensor against the biometric prints that are saved in the session identity store, and then if a match isn't found there, compares the incoming biometric print against the biometric prints that are permanently saved with biometric-enabled profiles.

Sometimes called the *enrolled player database*.

sign-in

The act of signing in a player. A guest player manually signs-in to a specific gamer profile. Or biometric sign-in automatically signs-in a skeleton to the matching biometric profile when the title determines and indicates that the player has indicated intent to play (as defined by the title).

The title can explicitly offer the opportunity to sign in, but the sign-in operation itself is initiated automatically through biometric sign-in, or is initiated by the player. The actual sign-in operation is performed by the system, and the title has no control over it. The player can use the Kinect Guide to sign in or sign out at any time.

When a player signs out, the NUI identity system automatically unenrolls the player. See also *biometric sign-in*.

signed-in player

A player who is enrolled in the session identity store and is signed-in to a gamer profile. The skeleton's identity is recognized as a specific player with a specific gamer profile saved on the console drive. See [Levels of Player Identity](#).

skeleton, skeleton data structure

A skeleton data structure contains the following identifiers to distinguish the skeleton as a specific player: skeleton ID, enrollment index, and user index. A skeleton data structure contains tracking status, joint positions, enrollment status, and sign-in status. A skeleton is tracked as a skeleton data structure that holds the tracking data for a single player in the play space.

A skeleton data structure resides with other skeleton data structures in a frame of skeleton tracking data from the skeleton tracking pipeline. The system can track up to 6 skeletons: 2 actively tracked (fully tracked, with all joint positions) and 4 passively tracked (only the center position is tracked).

skeleton frame

Each skeleton frame is a structure that contains multiple skeleton data structures. Each skeleton data structure contains tracking information about one tracked skeleton. A skeleton frame contains every skeleton that is currently tracked (detected), including actively tracked (fully tracked) skeletons and passively tracked (position-only tracked) skeletons.

skeleton tracking ID

The ID of one of the skeletons in a frame of skeleton tracking data. The skeleton tracking ID is subject to change, and is accessed through the enrollment index, which changes less. The skeleton tracking ID and enrollment index are stored in a NUI_SKELETON_DATA structure in a frame of skeleton tracking data. Compare *enrollment index*, *user index*.

skeleton tracking pipeline

A derived data feed from the Kinect Sensor, providing a series of skeleton frames. Each skeleton frame is a structure that contains multiple skeleton data structures. Each skeleton data structure contains tracking information about one tracked skeleton. A skeleton frame contains every skeleton that is currently tracked (detected), including actively tracked (fully tracked) skeletons and passively tracked (position-only tracked) skeletons. The skeleton tracking pipeline is based on the depth image stream. The depth image stream is produced from the infrared laser and the CMOS depth sensor.

title session

See *game session*.

tracked skeleton

A skeleton detected by the Kinect Sensor. For a passively tracked skeleton, only the center position of the skeleton is reported. For an actively tracked skeleton, all joints, or all upper-body joints, are tracked, thus joint position data is available for each joint. See [NUI Skeleton Tracking](#).

unenroll, unenrollment

Unenrolling frees up an enrollment slot that was used by a guest player, and removes the guest player's biometric print from the identity session store. Only a guest player can be unenrolled. A title cannot unenroll a skeleton that is signed-in; that is, a biometric print that is bound to a gamer profile.

unknown player, unknown skeleton

In most Identity discussion, "Unknown" refers to a skeleton's flag and means "not enrolled". If the player is unknown (that is, not enrolled), there might or might not be a biometric profile match.

When a skeleton becomes visible, it is initially flagged as unidentified, indicating that **NuiIdentityIdentify** needs to be called. When **NuiIdentityIdentify** is called for that skeleton, the NUI identity system attempts to find a match between the incoming biometric print and the session identities, which are enrolled in the session identity store. If "unknown" is the result (as the skeleton's enrollment index), that means that that skeleton is not enrolled.

If the skeleton is not enrolled, **NuiIdentityIdentify** then matches the incoming biometric print against any biometric profiles on the console, and sets a flag if it finds a biometric profile match. If a biometric profile match is found, when the player indicates intent to play, the title must enroll that player. The NUI identity system will then handle enrolling and signing-in the player as the matching biometric profile. The **NuiIdentityIdentify** function tells the title whether the incoming print matches a biometric profile, but doesn't tell the title which biometric profile matched, because the NUI identity system handles enrolling the particular biometric profile, based on the skeleton tracking ID. Contrast *identification failed*.

If you enroll an unknown guest player who doesn't sign in manually or automatically through a biometric profile, the player becomes an enrolled guest player.

An unknown guest player is a completely generic skeleton, not associated with an enrollment index. An unknown guest player can't be consistently, uniquely identified if the player disappears and reappears; the title then doesn't know that the newly visible skeleton is the same player who just left the field of view, though in some types of gameplay, the title may assume the newly visible skeleton is the same player.

Compare *guest player*. See [Levels of Player Identity](#).

user index

The Xbox user index, 0-3. The user indexes correspond to 4 game controllers, 4 controller ports, and the 4 quadrants of the Ring of Light. In contrast, the enrollment index corresponds with 8 players whose scores can be maintained and updated during a game session, of which 2 players can be fully tracked simultaneously. When a skeleton become actively tracked (fully tracked), and is enrolled and signed-in, one of the user indexes is re-mapped to that skeleton.

The title frequently checks the user index in a skeleton data structure to detect a change in player sign-ins. The user index in a skeleton data structure should not be trusted unless the enrollment index in that structure is valid, such as 0-7. Compare *skeleton tracking ID* and *enrollment index*.

Like controllers, NUI-identified users are assigned the lowest vacant user index. For example, if a player powers up the console, plugs in a controller, and then signs in using the NUI identity system, the controller and the skeleton are bound to user index 0.

Xbox profile

See *gamer profile*.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Image Camera

[See Also](#)



The Natural User Interface (NUI) Image and Camera API provides the means to modify settings for the Kinect sensor array, and access to image data from the sensor array. To use the API, program code should include NuiApi.h, which automatically provides all necessary NUI header files. To view the definitions of the Image and Camera API, see the [NuiImageCamera.h](#) header file.

Getting and Using Image Data

A game has access to three kinds of image data from the sensor array. Color image data shows the players, furniture, and other items in the playspace. Depth image data gives the distance, in millimeters, from the sensor array to the nearest thing in the field of view. Player segmentation data tells which player, if any, is visible at a given pixel location in the color and depth images.

Each kind of data is accessible by way of a stream of the appropriate type. Your game can open more than one stream at a time, and combine the information from multiple streams to

interpret player actions as game controls. Image data can be zoomed, creating a window on the full field of view. The zoomed window can be moved within the current field of view to give the effect of pan and tilt.

For a more detailed discussion of getting and using image data, see [NUI Image Camera Overview](#).

Using Sensor Properties

Your game can adjust the properties of the Kinect sensors to yield the best image quality for the way your game is using the images. The color image sensor exposes controls for frame rate and exposure, including a mode that automatically maintains exposure and white balance.

For a more detailed discussion of setting sensor properties, see [Setting Camera Properties and Tilting the Sensor](#).

In This Section

[NUI Image Camera Overview](#)

Describes the basic components and methods for retrieving image data from the Kinect sensor array and changing the sensor settings.

[Initializing NUI Image Camera](#)

How to initialize the [NUI Image API](#) to read data stream input.

[Face-Based Autoexposure](#)

Adjusts image brightness based on face exposure.

[Reading NUI Image Streams](#)

Reading the data stream input with the NUI Image and Camera API.

[Setting Camera Properties and Tilting the Sensor](#)

How to set properties to configure the Kinect cameras and tilt the Kinect Sensor.

See Also

[Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Image Camera Overview



The Natural User Interface (NUI) Image and Camera API provide the means to modify settings for the Kinect sensor array, and it enables you to access image data from the sensor array. The API is defined in the [NuiImageCamera.h](#) header file.

Sections

- [Image Data Streams](#)
- [Opening and Reading Data from an Image Stream](#)
- [Panning and Zooming Images](#)
- [Memory and Thread Management](#)

Image Data Streams

A title has access to three kinds of image data from the sensor array:

- Color data
- Depth data
- Player segmentation data

The image data is delivered as a succession of still-image frames.

Color Data

Color image data is available at two quality levels, and in two different formats. Quality level determines how quickly data is transferred from the Kinect sensor array to the Xbox 360 console. The available color formats determine whether the image data returned to title code is encoded as RGB or YUV. All four combinations of quality level and format may be used by titles, though only one quality level may be used at any given time.

The sensor array uses a USB connection to pass data to the Xbox 360 console, and that connection provides a given amount of bandwidth. Your choice of image data quality allows you to tune how that bandwidth is used. High quality images send more data per frame and update less frequently, while regular quality images update more frequently, with some loss in image quality due to compression.

- At regular quality, color image data is compressed in the sensor before being transmitted to the console. The console then decompresses the data before passing it to your title. The use of compression makes it possible to return color data at frame rates as high as 30 frames per second (fps), but the algorithm used leads to some loss of image fidelity.
- At high quality, color image data is not compressed in the sensor—it is transmitted to the console as originally captured by the sensor. Because the data is not compressed, more data must be transmitted per frame, and the maximum frame rate is no more than 15 fps. Also, the uncompressed data requires that the NUI system allocate larger buffers when you pass the high quality flag to [NuiInitialize](#).

You need to evaluate what kind of color data is most appropriate in your title: High quality data in larger buffers at lower frame rates, or data at higher frame rates with some loss of fidelity and a smaller memory footprint.

Color data is available in two formats: RGB or YUV.

- RGB color provides 32-bit, linear X8R8G8B8-formatted color bitmaps, in sRGB color space. Because the data is in sRGB color space, to correctly sample from the color image, you should use a gamma-correct D3D format. For an example of how to create such a format, see the **ATG::GetAs16SRGBFormat(D3DFMT_LIN_X8R8G8B8)** function in the ATG Sample Framework.
- YUV color provides 16-bit, gamma-corrected linear UYVY-formatted color bitmaps, where the gamma correction in YUV space is equivalent to sRGB gamma in RGB space. The Direct3D format of this data is D3DFMT_LE_LIN_UYVY. Because the YUV stream uses 16 bits per pixel, this format uses less memory to hold bitmap data, and allocates less buffer memory when [NuiImageStreamOpen](#) is called.

Both color formats are computed from the same camera data, so that the YUV data and RGB data represent the same image. Choose the data format that is most convenient given your title's implementation.

Depth Data

The depth data stream provides frames of D3DFMT_LIN_D16-formatted bitmaps, where the high 13 bits of each pixel give the distance, in millimeters, to the nearest object at that particular x and y coordinate in the depth sensor's field of view. Two depth data streams are available, one with frame size of 320×240 and another with frame size of 80×60 pixels. Titles can process data from either of the depth streams to support various custom features, such as tracking players' motions or identifying background objects to ignore during gameplay.

Player Segmentation Data

The Kinect system processes sensor data to identify up to six human figures in front of the sensor array, and creates the Player Segmentation map. This map is a bitmap where the pixel values correspond to the player index of the person in the field of view who is closest to the camera, at that pixel position.

Though the player segmentation data is a separate logical stream, in practice the depth data and player segmentation data are merged into a single frame. The 13 high-order bits of each pixel represent the distance from the depth sensor to the closest object, in millimeters. The three low-order bits of each pixel represent the player index of the tracked player who is visible at the pixel's x and y coordinates. These three bits are treated as an integer value. The bits are not used as flags in a bit field. A player index value of zero indicates that no player was found at that location. Values of one to six identify players. The value seven is not used.

A common use of player segmentation data is as a mask to isolate specific players or regions of interest from the raw color and depth images.

Registered Streams

In addition to color and depth image data directly as it comes from the sensor, it is also possible to get data of one stream type that has been "registered" in the coordinate space of another stream type. Registration compensates for parallax errors caused by the fact that the sensors do not occupy the same physical space, but are separated horizontally. There are two registered streams: Depth in color space and color in depth space. You can get registered depth data with or without player index data included. The pixel formats of registered streams are exactly the same as their unregistered counterparts. Registered color data is only available in the default color format. There is no registered version of high quality color data.

When using "depth in color space," you are assured that the depth value at given coordinates is the distance to the object in the color image at the same coordinates. You can use this to produce so-called "green screen" effects by copying pixels that correspond to a player index from the color stream into another scene, but not copying color stream pixels that do not correspond to a player index.

The [Segmentation Sample](#) demonstrates the basic "green screen" technique by drawing color pixels from the color stream only when the player segmentation data in the registered depth stream indicates that a player is present at that pixel. It draws a black pixel where no player is found. The [Motion-Based Background Removal Sample](#) demonstrates the inverse of a green screen effect—hiding a moving player by rendering saved background pixels in place of the player's silhouette.

Opening and Reading Data from an Image Stream

To use a particular stream in your title, first pass the appropriate flag to [NuiInitialize](#), indicating that you will use that stream type. Next, open the stream by calling [NuiImageStreamOpen](#), passing the appropriate parameters. **NuiImageStreamOpen** returns a stream handle. Use this handle to read from, and interact with, the specific stream. If you are going to use more than one stream, pass the bitwise-or combination of stream type flags to **NuiInitialize**, and open each stream with a separate call to **NuiImageStreamOpen**.

For each type of enabled stream, you can request the latest frame of data with a call to [NuiImageStreamGetNextFrame](#). The frame data is returned in one of a limited number of buffers that are allocated internally by the Kinect system. **NuiImageStreamGetNextFrame** always gets the most recent frame of data. Frames are not queued and failure to poll in a timely fashion will result in lost frames. The *dwFrameLimit* parameter that is passed to [NuiImageStreamOpen](#) functions as a check out limit. Calling **NuiImageStreamGetNextFrame** checks out the most recent frame of data, and calling [NuiImageStreamReleaseFrame](#) checks the frame back in. There is no limit on how long a given frame may be held, and frames may be released in any order. Once *dwFrameLimit* has been reached, **NuiImageStreamGetNextFrame** will fail with **E_NUI_FRAME_LIMIT_EXCEEDED** until one or more frames are released.

The NUI Image and Camera API offers two usage models for retrieving new frames: A polling model and an event model.

Polling for New Image Data

The *polling model* is the simplest option for reading image frames. titles may call [NuiImageStreamGetNextFrame](#) on the render thread or on another thread. By default, the API assumes polling for the render thread. In this mode, each time your title calls [IDirect3DDevice::Present\(\)](#), or [IDirect3DDevice::Swap\(\)](#), a single frame would be released. If data was received in time for processing to begin during the frame swap, a valid frame will be generated. Data is generally available a few milliseconds after **Present()** returns. **NuiImageStreamGetNextFrame** may be called with a timeout between 0 and infinite. If image processing for that frame completes before the timeout expires, the API will return either **S_OK** and a new frame, or **E_NUI_NO_FRAME_DATA**, which indicates that no camera data was available at the beginning of this frame processing. When the API times out, **E_PENDING** may be returned. Other errors that may be returned are **E_NUI_DEVICE_NOT_CONNECTED** and **E_NUI_DEVICE_NOT_READY**.

If polling on a thread other than the render thread, the flag **NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA** may be passed to [NuiImageStreamOpen](#). In this case, **E_NUI_NO_FRAME_DATA** will not be returned. Rather, the API will continue to wait for the next frame, unless the timeout expires first, in which case **E_PENDING** will be returned. In this case, beware of polling for color and depth on the same thread. The color stream may not return a frame every time that the depth stream does. For example, the identity system can sometimes intercept the color stream, or the color stream can be set to allow a lower frame rate.

Using Events for New Image Data

The *event model* uses events to signal your title code when frame data is available. There are two ways to use the event model. One model signals an event when a new frame is available for a given stream. Under this model, if you are reading from more than one stream, you are using separate events for each stream, and each event is signaled when that stream has a frame available. The second model signals a single event when new frames from multiple streams are all available. This model is used to synchronize multiple streams so that the frame data for all streams was sampled at the same time. The model that uses one event for all streams is recommended for titles that are pumping multiple streams with a single thread. Code that tries to pump multiple streams on one thread, using one event per stream, can encounter issues with concurrency control and order of execution that may affect the title's frame rate.

If you are working with a single stream, you pass an event handle to [NuiImageStreamOpen](#) when you initially open an image data stream. When a new frame of data is ready, the event is signaled, and any waiting title thread wakes and gets the image frame with a call to [NuiImageStreamGetNextFrame](#). During this time, the event is reset by the NUI Image and Camera API.

If you are working with more than one stream, and you want to get frames from multiple streams that were all taken at the same moment in time, call [NuiSetFrameEndEvent](#) to designate the event that will be signaled when frames from all of the currently open streams are available. At that time, call [NuiImageStreamGetNextFrame](#) on each of the streams of interest. If you've enabled skeleton tracking, you can call [NuiSkeletonGetNextFrame](#) to get the corresponding skeleton data. The returned frames will all have been taken during the same period. This event is not reset by the NUI Image and Camera API. Your code must reset the event once it has finished using the stream data and is ready to be notified of the completion of the next frame. This method is recommended for titles that are pumping multiple streams with a single thread, as opposed to using separate per-stream events.

Note

Because the event handles that are used are initialized and configured by the title code, manual reset events are preferable to automatic reset events. The NUI API has no mechanism for determining which type of reset event the title code provided, automatic or manual.

Panning and Zooming Images

You can zoom in on a portion of the color image, getting double magnification, by calling [NuiCameraSetColorImageViewArea](#). This function causes future images from the color camera stream to be centered at your chosen point within the camera's field of view, and zoomed by the appropriate factor. Image quality is not degraded by zooming. Rather, at maximum zoom, the color image sensor is shown at its native resolution, while non-zoomed images are constructed by applying a hardware binning algorithm over multiple sensor pixels.

To return to non-zoomed images, call [NuiCameraSetColorImageViewArea](#) with the field of view centered at (0, 0) and the zoom factor set to **NUI_IMAGE_DIGITAL_ZOOM_1X**.

The [Simple Skeleton Tracking Sample](#) uses pan and zoom functionality to implement head tracking, where it zooms the camera and then pans to keep the tracked skeleton's head in the frame.

Memory and Thread Management

All NUI image memory is allocated and managed by the system and not by the title code, though title code can override [XMemAlloc](#) to gather run-time details about the NUI API's use of memory. NUI image memory is allocated as physical memory because the NUI Image and Camera API uses the GPU. For more information about physical memory, see [Physical Memory](#) and [XPhysicalAlloc](#).

NUI APIs are designed to be thread-safe and can be called from any thread. Once title code retrieves an image frame, it can freely read and write from the buffer containing the frame data. However, the title may not delete that buffer. Rather, the title should return the buffer for use by the NUI system by calling [NuiImageStreamReleaseFrame](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Initializing NUI Image Camera



Any title that uses data from the various image streams provided by the Kinect sensor array must initialize the depth and image cameras, as well as any required streams. Once initialized, data can then be read and interpreted by the application. During cleanup, performed by the title upon exit, the camera should be uninitialized and the Natural User Interface (NUI) system shut down.

You must initialize the D3D device before calling [NuiInitialize](#). You must call [NuiShutdown](#) and clean up all NUI API resources before you shut down the D3D device. Failure to initialize and shut down in the proper order will cause the title code to assert or throw an exception.

Sections

- [Initializing the Cameras](#)
- [Initializing NUI](#)
- [Opening Streams and Retrieving Stream Data](#)
- [Shutting Down](#)

Initializing the Cameras

1. Initialize any high-level NUI features used by the application by calling [NuiInitialize](#).


```
HRESULT hr = NuiInitialize(
    NUI_INITIALIZE_FLAG_USES_SKELETON | NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX | NUI_INITIALIZE_FLAG_USES_COLOR,
    NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD );

if ( FAILED(hr) )
{
    // Perform error notification/handling.
    return hr;
}
```

2. Enable a specific data stream (or streams) by calling [NuiImageStreamOpen](#).

```
hr = NuiImageStreamOpen(
    NUI_IMAGE_TYPE_COLOR,
    NUI_IMAGE_RESOLUTION_640x480,
    0,
    1,
    NULL,
    &g_ColorStreamHandle);

if ( FAILED( hr ) )
{
    // Perform error notification/handling.
    return hr;
}
```

Initializing NUI

When **NuiInitialize** is called, it allocates resources for the NUI Image subsystem to operate, and establishes connections to the title's D3D device, which should be initialized before the call to **NuiInitialize**.

The title indicates which data streams are available by passing appropriate *dwFlags* values to **NuiInitialize**. Each data stream that is enabled requires system resources (cycles and memory), so it is important to only enable streams that the title will use.

The *dwHardwareThreadSkeleton* argument designates the hardware thread on which the skeleton subsystem is to run. If you have no preference, use the constant [NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD](#), which assigns a default thread.

NuiInitialize is not meant to be called multiple times in a row, but rather must always be paired with a call to **NuiShutdown**. If you need to re-initialize the camera subsystem, for example, to change what information streams are enabled, you must call **NuiShutdown** first to terminate the NUI system, and can then call **NuiInitialize** with new arguments.

Opening Streams and Retrieving Stream Data

A title opens a given data stream by calling **NuiImageStreamOpen** with the appropriate *eImageType* value. To retrieve a frame from an open stream, call **NuiImageStreamGetNextFrame** on the appropriate stream.

There are a limited number of buffers available to return frame data when **NuiImageStreamGetNextFrame** is called. Titles must call **NuiImageStreamReleaseFrame** to return a buffer to the pool of usable buffers, after the title has finished using that frame's data. Failing to release buffers to the system in this way will eventually cause **NuiImageStreamGetNextFrame** calls to fail for lack of a buffer to use to return the frame data.

Shutting Down

When your title is shutting down, terminate the NUI system, and release all resources by calling [NuiShutdown](#).

To ensure an orderly shutdown of the NUI system, **NuiShutdown** should be called at the termination of the title, before the D3D device is shut down.

The Xbox 360 system services will ensure that the NUI system terminates and releases its resources if a title aborts without notification, such as by an exit to the Xbox Dashboard or by ejection of the game disc.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Face-Based Autoexposure

[Example](#)

Face-based autoexposure adjusts image brightness based on face exposure when using the Kinect Sensor. In most lighting conditions, faces are not overexposed or underexposed.

When you use face-based autoexposure, the system identifies a bounding box around each tracked player's eyes, nose, and mouth, and optimizes the camera's exposure settings to make the player's face clearly visible. The processing required to adjust exposure in this way imposes a small CPU requirement, but if your title is using the image streams, it's the best way to ensure that the players are clearly visible in the images that you're using. For more information, see [Standard Autoexposure versus Face-Based Autoexposure](#).

The following API items used for face-based autoexposure:

- In the enumeration [NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES](#), the enumeration constant **NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED** optimizes exposure for the faces of players whose skeletons are being tracked.
- The enumeration [NUI_CAMERA_PROPERTY](#) defines the following properties:
 - **NUI_CAMERA_PROPERTY_AE_FACEBASED_TRACKING_ID** explicitly specifies which player to apply face-based autoexposure to.
 - **NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS** lets the title know when face-based autoexposure has reached a steady state.
 - Face-based autoexposure is also affected by the optional property **NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN**.
- In the enumeration [NUI_CAMERA_PROPERTYF](#), the enumeration constant **NUI_CAMERA_PROPERTYF_AE_FACEBASED_MAX_GAIN** sets the maximum gain value that face-based autoexposure is allowed to set.
- The enumeration [NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES](#) lets titles know when face-based autoexposure has reached a steady state. The property [NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS](#) returns one of these enumeration constants.

To use face-based autoexposure, skeleton tracking must be enabled. If skeleton tracking isn't enabled, face detection won't work and [NuiCameraSetProperty](#) will return **E_NUI_FEATURE_NOT_INITIALIZED**. Call **NuiCameraSetProperty** as shown in the following example.

Example

```
NuiCameraSetProperty(NUI_CAMERA_TYPE_COLOR,
    NUI_CAMERA_PROPERTY_AE_AWB_MODE,
    NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED);
```

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Reading NUI Image Streams

To read image streams from the Kinect Sensor, use the [Natural User Interface \(NUI\) Image and Camera API](#). Once a data stream is enabled ([NuiImageStreamOpen](#)), the title receives frame data in a buffer provided by the system. The system allocates a limited number of buffers, and requires that your title code release buffers for reuse when they are no longer being used by

your title.

To read a single frame from the data stream

- Call the function [NuiImageStreamGetNextFrame](#) to capture the next frame of data stream. The system buffer used to return the frame data is marked by the system as being in use by your title code.
- When title code is finished using the frame data, unlock the frame buffer with a call to [NuiImageStreamReleaseFrame](#). This enables the buffer to be used to return frames in future calls to [NuiImageStreamGetNextFrame](#).

Usage

Calls to the function **NuiImageStreamGetNextFrame** identify a stream from which to read the next frame, and provide a pointer that will be set to point to the buffer containing the returned frame data. The specified stream must have been previously enabled with a call to **NuiImageStreamOpen**. When the operation is complete, the buffer contains a single frame of the camera's data stream. This buffer must be released back to the system, or the system will eventually run out of buffers, and calls to **NuiImageStreamGetNextFrame** will fail.

The NUI system does not independently queue any stream data. **NuiImageStreamGetNextFrame** only fills the system-allocated buffer with the stream data of the next available frame. If title code does not call **NuiImageStreamGetNextFrame** often enough, then the unretrieved frames are lost.

Title code can use one of two methods to ensure that it does not miss any frames: polling for new frame data, or using an event to be notified when a new frame is available. For more information on these methods, see [NUI Image Camera Overview](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Setting Camera Properties and Tilting the Sensor



The Natural User Interface (NUI) Image and Camera API enables modification of the Kinect Sensor settings. This API controls the quality of the images retrieved from the sensor, and ensures that players are visible in the frame. To use the API, program code should include NuiApi.h, which automatically provides all necessary NUI header files. To view the definitions of the Image and Camera API, see the [NuiImageCamera.h](#) header file.

Sections

- [Setting Camera Properties](#)
- [Tilting the Kinect Sensor](#)

Setting Camera Properties

The NUI Image and Camera API allow game code to query and set configuration properties of the color and depth cameras. Some of the properties, such as the physical field of view of the lens, are read-only. Others, such as exposure time, can be configured to affect how the device behaves and the data it produces.

Most camera properties are specific to the color camera of the sensor array and are similar to those exposed for the Xbox LIVE Vision camera. Color camera settings, once applied, subsequently affect the color image stream. A subset of these properties applies to the depth camera.

Integer and Float Properties

Properties are divided into two groups: those that involve integer values, and those that involve floating-point values.

- Integer-valued properties (and Boolean-valued ones) are listed in the [NUI_CAMERA_PROPERTY](#) enumeration, retrieved using [NuiCameraGetProperty](#), and set using [NuiCameraSetProperty](#).
- Float-valued properties are listed in the [NUI_CAMERA_PROPERTYF](#) enumeration, retrieved using [NuiCameraGetPropertyF](#), and set using [NuiCameraSetPropertyF](#).

The descriptions of the properties on the [NUI_CAMERA_PROPERTY](#) and [NUI_CAMERA_PROPERTYF](#) pages provide detailed descriptions of accepted values for each property, and how a given property value effects the images returned from the sensor. Some properties are mutually exclusive, and that some settings render other settings invalid. For example, a setting for exposure time is ignored if an autoexposure mode is active.

Standard Autoexposure versus Face-Based Autoexposure

You can enable two different autoexposure settings. Standard autoexposure optimizes exposure settings for the current autoexposure region, or for the entire frame if no region of interest has been set. Face-based autoexposure optimizes exposure settings for the faces of the players whose skeletons are being actively tracked.

- When standard autoexposure is enabled, you can set a rectangular region in the field of view as the region of interest for setting the exposure, using [NuiCameraSetExposureRegionOfInterest](#). You might use this feature to set a region of interest around the currently-active player in a turn-based multiplayer game, based on skeleton tracking data, to ensure that the active player is adequately exposed in color camera images.

The region of interest is only used when standard autoexposure is active. When autoexposure is turned off or set to face-based mode, the region of interest is not used nor changed.

- To use [face-based autoexposure](#), you must enable skeleton tracking. The system software identifies a bounding box around each tracked player's eyes, nose, and mouth, and optimizes the camera's exposure settings to make the player's face clearly visible. The processing required to adjust exposure in this way imposes a small CPU requirement, but if your title is using the image streams, it's the best way to ensure that the players are clearly visible in the images that you're using. Prefer face-based autoexposure unless you find that its processing requirement reduces game performance.

Changing Multiple Settings Together

When changing more than one property at a time, it is usually more efficient to group all of the changes into a single transaction. To do this, call [NuiCameraSynchronizePropertyChangesBegin](#), call [NuiCameraSetProperty](#) and/or [NuiCameraSetPropertyF](#) as desired, and finally call [NuiCameraSynchronizePropertyChangesEnd](#) to commit all of the changes in a single transaction.

Tilting the Kinect Sensor

The Kinect sensor has a motorized tilt mechanism in its base that can be used to tilt the camera to ensure that players are visible in the frame. The tilt mechanism is not controlled explicitly as a sensor property, but rather by game code calling [NuiCameraAdjustTilt](#) to specify the game's requirements for how to frame players. The sensor array then adjusts the tilt angle to accommodate the game's requirements, given the geometry of the play space.

Some Xbox 360 owners have large, open areas in which to play. These players are able stand sufficiently far from the Kinect sensor that their entire bodies are in the sensor's field of view, which enables skeleton tracking to identify entire skeletons. *Far space* is the part of the play space where the player's entire body is visible to the sensor. The exact dimensions of far space depend on the placement of the sensor, the player's height, and how close the player is to the sensor. Players standing more than 1.8 meters from the sensor are usually in far space, assuming average-height players and reasonable sensor placement. The [NuiCameraAdjustTilt](#) function can be used to tilt the camera consistently to capture the entire play space within the field of view.

Near space is the part of the play space where the player's entire body is not visible to the sensor. It is important to account for near space in your game's design. This is particularly important for players who do not have large rooms in which to play. These players are unable to move far enough from the sensor, due to walls or furniture, to make their entire bodies visible to the sensor. The [NuiCameraAdjustTilt](#) function can be used to tilt the camera appropriately to ensure that the most relevant part of the player's body is in the field of view. Games can specify whether a near-field player's face should be toward the top of the frame, so that more of the body is visible, or if the player's face should be toward the center or bottom of the frame, so that arm movements above shoulder level are still in the field of view. The choice of near-field adjustment depends on which movements are most significant in your game's design.

The tilt mechanism can take several seconds to adjust the tilt angle. Titles should encourage players to remain still and in position when calling **NuiCameraAdjustTilt**.

The tilt mechanism in the sensor array is not rated for frequent use. Your code should not call **NuiCameraAdjustTilt** more than 15 times in any given 2-minute window. Changing the tilt too often will result in an error message from the function.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Skeleton Tracking

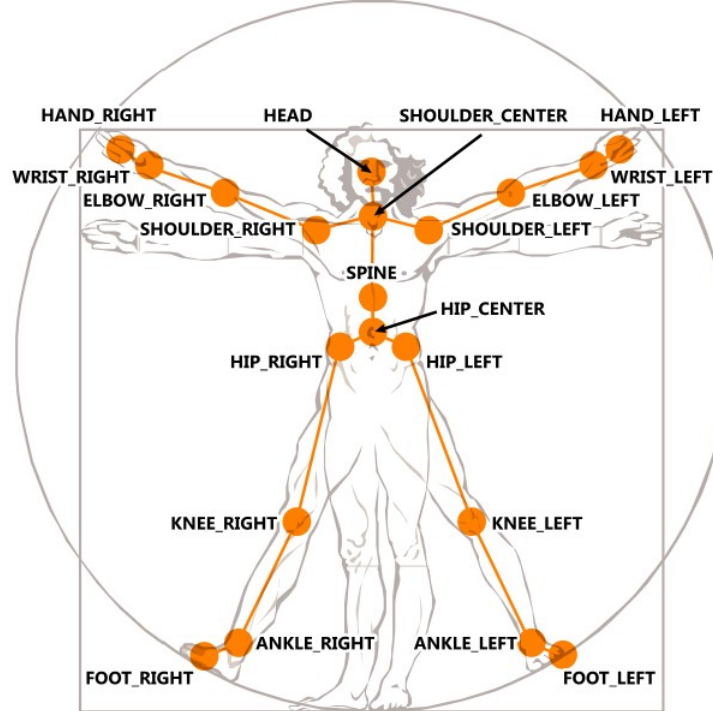
[See Also](#)



The NUI Skeleton API provides information about the location of up to six players found in front of the Kinect Sensor, with detailed position and orientation information for up to two of those players. Skeleton tracking has two modes, or processing pipelines, for tracking player skeletons. One mode is optimized for standing players, and the other is optimized for seated players. For more information about skeleton tracking of seated players, see [Seated Skeleton Tracking](#).

Tracked skeleton data is provided to game code as a set of points, called *skeleton positions*, that compose a skeleton. This skeleton represents the player's current position and pose. Titles can use skeleton position information to control an in-game character directly, or they can interpret skeleton position as input for gameplay or navigation of the title's user interface. To use the NUI Skeleton API, include NuiApi.h in your code.

Figure 1. Skeleton joint positions relative to the human body



In the API, the joints are listed in [NUI_SKELETON_POSITION_INDEX](#), such as NUI_SKELETON_POSITION_HIP_CENTER.

For basic tasks associated with using skeleton tracking in your title, see [Skeleton Tracking Tasks](#).

Sections

- [Retrieving Skeleton Information](#)
- [Alternatives to Full-Skeleton Tracking](#)
- [Active and Passive Skeleton Tracking](#)
- [Setting Which Skeleton Is Active](#)
- [Data for Each Skeleton](#)
- [The Skeleton Tracking ID](#)

Retrieving Skeleton Information

Game code gets the latest frame of skeleton data by passing a buffer to [NuiSkeletonGetNextFrame](#). If the latest frame of data is ready, it is copied into the buffer. If your code requests frames of skeleton data faster than new frames are available, you can choose whether to wait for the next frame to be ready, or to return immediately and try again later. The NUI Skeleton API never provides the same frame of data more than once.

The NUI Skeleton API offers two usage models: a polling model and an event model.

The *polling* model is the simplest option for reading data frames. First, the game code calls [NuiSkeletonGetNextFrame](#), specifying how long to wait for the next frame of data (between 0 and an infinite number of milliseconds). [NuiSkeletonGetNextFrame](#) returns when a new frame of data is ready or when the wait time expires, whichever comes first. Specifying an infinite wait causes the call for frame data to block and to wait as long as is necessary for the next frame. When [NuiSkeletonGetNextFrame](#) returns successfully, the new frame of skeleton data is ready for processing. If the timeout value is set to zero, the game code can poll for completion of a new frame while doing other work on the same thread.

The *event* model supports the ability to integrate retrieval of a skeleton frame into a game engine with more flexibility and more accuracy. In this method, game code passes an event handle to [NuiSkeletonTrackingEnable](#). When a new frame of skeleton data is ready, the event is signaled. Any waiting thread wakes, and gets the frame of skeleton data by calling [NuiSkeletonGetNextFrame](#). During this time, the event is reset by the NUI Skeleton API.

If you are using the event model and want to get skeleton data in synchronization with image frame data, you can call [NuiSetFrameEndEvent](#) to designate an event that will be signaled when frames from all of the currently-open streams are available. When the designated event is signaled, you can call [NuiSkeletonGetNextFrame](#) to get the skeleton data that corresponds to the image data you're retrieving with [NuiImageStreamGetNextFrame](#).

Alternatives to Full-Skeleton Tracking

For some purposes, standing or seated skeleton tracking may not be the best use of resources. Other libraries provide more limited ways to base game behavior on player movement, while requiring fewer resources.

If you want to track only the head joint position, you can use [lightweight head position tracking](#) instead of using the more resource-intensive approach of using skeleton tracking and monitoring the position of the head joint.

If you want to track the orientation (direction) of the player's head, you can use [head orientation tracking](#), which is a separate subsystem than skeleton tracking.

Active and Passive Skeleton Tracking

A skeleton is actively tracked or passively tracked.

When a player's skeleton is being *actively* tracked, the data that is returned by [NuiSkeletonGetNextFrame](#) for that skeleton includes the position of every joint of the player's skeleton. The NUI skeleton tracking component provides full skeleton tracking (that is, *active skeleton tracking*) for one or two players in the Kinect Sensor's field of view. By default, the first two skeletons found (the soonest) by the skeleton tracking system are actively tracked.

When a player's skeleton is being *passively* tracked, the data that is returned by [NuiSkeletonGetNextFrame](#) for that skeleton includes only the position of the center of the player's skeleton, not the position of every joint. Passive tracking is provided automatically for up to four additional players in the Kinect Sensor's field of view.

Tracking status	Joint positions tracked	Flag	Players supported	Notes
Passively tracked skeleton	Center of mass only	NUI_SKELETON_POSITION_ONLY	6	Up to 2 skeletons can be <i>actively</i> tracked; when this is the case, up to 4 skeletons can be <i>passively</i> tracked.
Actively tracked skeleton	All joints	NUI_SKELETON_TRACKED	2	Occluded joints may need to be inferred, per Bowling .
Non-tracked skeleton	None	NUI_SKELETON_NOT_TRACKED	n/a	The NUI_SKELETON_DATA structure is invalid.

Setting Which Skeleton Is Active

Game code can choose which players are actively tracked. To do this, pass the flag [NUI_SKELETON_TRACKING_FLAG_TITLE_SETS_TRACKED_SKELETONS](#) to [NuiSkeletonTrackingEnable](#). Then call [NuiSkeletonSetTrackedSkeletons](#) to specify which skeletons are tracked. Your title design dictates the proper tracking policy to use for determining which players to track. Different titles might track one player or two, and might choose to track players closest to the sensor array, players closest to the center of the field of view, the shortest player visible, or according to some other criterion.

When you call [NuiSkeletonSetTrackedSkeletons](#), specify the tracking IDs for the active players in the *TrackingIDs* parameter. Specifying a tracking ID value of 0 causes NUI skeleton tracking to begin actively searching for a player to track within the playing field. For example, if a valid tracking ID and 0 are specified as the two active tracking IDs, full tracking is enabled for the player with the specified tracking ID, and a search begins for a new player in the playing field.

The [NuiView tool](#) visually demonstrates this functionality with a specialized mode. The following screenshot was taken while running a tracking policy of *Single active player closest to center of playing field*. Notice that the segmentation data for each player is visible, but only one player has skeleton data.

Figure 2. Active Tracking for a single player



Data for Each Skeleton

When game code calls [NuiSkeletonGetNextFrame](#), the returned [NUI_SKELETON_FRAME](#) structure contains an array of [NUI_SKELETON_DATA](#) structures, one for each skeleton tracked by the NUI skeleton tracking system. The data about the tracked skeleton is provided in the [NUI_SKELETON_DATA](#) structure, described in the following table.

Skeleton data field	Description
Tracking state	Indicates that the skeleton with the specified skeleton tracking ID is being fully tracked (actively tracked), is being tracked by its center position only (passively tracked), or is not being tracked. An active skeleton has a value of NUI_SKELETON_TRACKED . A passive skeleton has a value of NUI_SKELETON_POSITION_ONLY . To select which skeleton is actively tracked, call NuiSkeletonSetTrackedSkeletons .
Skeleton tracking ID	Uniquely refers to skeleton that's currently tracked, until the player disappears from view.
Enrollment index	Specifies which enrollment slot the skeleton is enrolled in, if any (0-7), in the session identity store. This is part of the NUI Identity system, to recognize a skeleton as a specific player based on face recognition and skeleton dimensions. If the skeleton has become visible and hasn't been checked yet to see if it can be recognized as one of the enrolled players, a flag, "identify", is set here, telling the title that the Identify operation needs to be run. If the skeleton has been checked and is found to be not enrolled, a flag, "unknown", is set here.
User Index	Indicates which player's skeleton data is contained in this skeleton data structure, 0-3. If the skeleton is not bound to a user index, this is a flag, "none".
Position	The location of the center of mass of the player. This value is the only available positional value for a passively tracked skeleton.
Skeleton joint positions	The position of each joint of an actively tracked (fully tracked) skeleton.
Joint position tracking state	For each joint in an actively tracked (fully tracked) skeleton, reports whether the position is currently tracked, inferred (hidden), or not tracked.
Quality flags	Indicate whether the player is partially out of the field of view.

The Skeleton Tracking ID

In the sequence of frames of skeleton tracking data, a skeleton ID is consistently applied to the same player for as long as the player remains visible. A given tracking ID remains at the same index in the [NUI_SKELETON_DATA](#) array for as long as the tracking ID is in use. This tracking ID is the value passed to [NuiSkeletonSetTrackedSkeletons](#) to change which players are actively tracked.

If the tracking ID of the skeleton at a particular index in the **SkeletonData** array in the [NUI_SKELETON_FRAME](#) changes, one of two things has happened:

- The tracked player left the field of view, and tracking started on another player in the field of view.
- The tracked player left the field of view and then returned, and the player is now being tracked again.

Call [NuiIdentityIdentify](#) with the new tracking ID to begin receiving identity information about the player with the new tracking ID.

In This Section

- [Skeleton Tracking Tasks](#)
Describes using the NUI Skeleton API to initialize NUI skeleton tracking and retrieve skeleton-tracking data.
- [Seated Skeleton Tracking](#)
Skeleton tracking for seated players.

See Also

- [NuiSkeleton Reference](#)
- [Simple Skeleton Tracking Sample](#)
- [Active/Passive Skeletons Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Skeleton Tracking Tasks

Use the [NuiSkeleton API](#) to initialize the Natural User Interface (NUI) Skeleton Tracking system and retrieve skeleton-tracking data. For more information on the NUI Skeleton Tracking system, see [NUI Skeleton Tracking](#).

Sections

- [Initializing NUI Skeleton Tracking](#)
- [Retrieving the Next Frame of Skeleton Tracking Data](#)
- [Parsing Skeleton Tracking Data](#)
- [Disabling Active Skeleton Tracking](#)

Initializing NUI Skeleton Tracking

First, make skeleton tracking available to the title by calling [NuiInitialize](#) with the flag **NUI_INITIALIZE_FLAG_USES_SKELETON**.

```
HRESULT hr = NuiInitialize(
    NUI_INITIALIZE_FLAG_USES_SKELETON |
    NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
    NUI_INITIALIZE_FLAG_USES_COLOR,
    NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD );

if ( FAILED(hr) )
{
    // Perform error notification/handling.
    return hr;
}
```

Then, enable active skeleton tracking by calling [NuiSkeletonTrackingEnable](#).

```
hr = NuiSkeletonTrackingEnable( NULL, 0 );

if ( FAILED (hr) )
{
    // Perform any necessary error handling
}
```

Retrieving the Next Frame of Skeleton Tracking Data

After initializing the NUI system and enabling active skeleton tracking, call [NuiSkeletonGetNextFrame](#), which processes depth camera data from the sensor array to create skeleton tracking data.

```
// If there is no new skeleton data, program execution continues without frame
// processing.
hr = NuiSkeletonGetNextFrame( 10, &m_SkeletonFrame );

if ( SUCCEEDED( hr ) )
{
    // Perform any necessary processing
}

if ( FAILED( hr ) )
{
    // Perform any necessary error handling
}
```

Parsing Skeleton Tracking Data

NuiSkeletonGetNextFrame returns a pointer to a title-allocated [NUI_SKELETON_FRAME](#) structure that is filled with the latest frame data.

This structure contains an array of [NUI_SKELETON_DATA](#) structures, each of which describes the world-space position of a part of the detected human frame. This array contains a fixed number of elements equal to the number of skeleton joint positions currently supported by NUI.

The position of each entry within the array determines which part of the human frame it describes, such as the head, right hand, or left knee. The [NUI_SKELETON_POSITION_INDEX](#) enumeration values identify the skeleton positions stored in the array. In this way, the position of the player's left elbow can be found at [SkeletonPositions](#) [NUI_SKELETON_POSITION_ELBOW_LEFT]. Each skeleton joint position is stored as an XMVECTOR that has X and Y coordinates as members.

The following example sets the speed to the difference in height, in skeleton space, from the right hand joint to the right shoulder joint.

```
reqSpeed = m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
    .SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y -
    m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ]
    .SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;
```

You can then call [NuiTransformSmooth](#) to reduce the jitter between skeleton frames:

```
hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );
```

Disabling Active Skeleton Tracking

To disable skeleton tracking, call the shutdown function, [NuiSkeletonTrackingDisable](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Seated Skeleton Tracking

[See Also](#)



Skeleton tracking has two modes, or processing pipelines, for tracking player skeletons. One mode is optimized for standing players, and the other is optimized for seated players. Only one of the two modes can run at any time. A title cannot have one seated player using the seated pipeline while a second standing player is using the full-body pipeline.

Differences Between the Pipelines

You use the same API for seated tracking as for full-body tracking, and both pipelines support the same number of tracked and passive players. However, there are differences in behavior.

- With seated tracking, players in seated poses are recognized and tracked.
- Seated tracking does not track any joints for the lower body, including spine, hips, knees, ankles, and feet. Seated tracking only returns joint positions for head, neck, shoulders, elbows, wrists, and hands.
- Even when the seated tracking pipeline is active, players must sit forward on their seats to be tracked properly. Players leaning backwards on very soft couches are not tracked properly.
- The seated tracking pipeline provides a segmentation mask that is different in several ways.
 - Continuity of the segmentation mask is not guaranteed outside of the arms, head, and shoulder areas.
 - The seated segmentation mask doesn't correspond exactly to the player outline like the standing (full-body) mask does.
 - The seated pipeline environment has less data, with more noise and variability than the standing environment. This is because of the inherently more challenging environment of seated tracking.
 - Players that don't have skeletons are not guaranteed to have player masks from the seated pipeline.
- Seated tracking doesn't support avatar re-targeting.
- A player must move in order for seated tracking to recognize that they are present and begin to track them. Skeleton tracking can't tell a stationary seated player from a sofa.

Alternatives to Full-Skeleton Tracking

For some purposes, even seated skeleton tracking may not be the best use of resources. Other libraries provide ways to base game behavior on player movement.

If you want to track only the head joint position, you can use [lightweight head position tracking](#) instead of using the more resource-intensive approach of using skeleton tracking and monitoring the position of the head joint.

If you want to track the orientation (direction) of the player's head, you can use [head orientation tracking](#), which is a separate subsystem than skeleton tracking.

Switching Between Skeleton Tracking Pipelines

At run-time, the skeleton tracking library supports switching between the seated skeleton tracking pipeline and the regular full body standing skeleton tracking pipeline. To switch between the seated and full body pipelines at run-time, you must do the following:

1. Call [NuiSkeletonTrackingDisable](#) to disable the current skeleton tracking pipeline.
2. Call [NuiSkeletonTrackingEnable](#) with the appropriate *dwFlags* value to enable the new pipeline. To enable seated skeleton tracking, pass the [NUI_SKELETON_TRACKING_FLAG_ENABLE_SEATED_SUPPORT](#) flag. If seated skeleton tracking is not explicitly requested, then standing skeleton tracking is enabled.

There is no provision for automatically recognizing that players are seated and switching to seated tracking. Titles must explicitly choose a skeleton tracking mode.

For basic tasks associated with using skeleton tracking, see [Skeleton Tracking Tasks](#). For an overview of standing skeleton tracking in your game, see [NUI Skeleton Tracking](#).

See Also

[NUI Skeleton Tracking](#)
[NuiSkeletonReference](#)
[Simple Seated Skeleton Tracking Sample](#)
[Simple Skeleton Tracking Sample](#)
[Active/Passive Skeletons Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

NUI Speech

[See Also](#)



In This Section

[Introduction to NUI Speech](#)

Enables speech recognition with the Kinect microphone array.

[Using the NUI Speech API](#)

API call sequence for speech recognition with the Kinect microphone array.

[Audio Input with NUI Speech](#)

Contains basic information about audio input management in the Natural User Interface (NUI) Speech API.

[Multichannel Echo Cancellation with NuiAudio](#)

Performs multichannel echo cancellation for the microphone array in the Kinect Sensor, to support speech recognition. This processing is independent of the installed version of system software, unlike XHV.

[NUI Speech Grammars](#)

Developing speech grammars for speech recognition in Kinect titles.

[Local Language Support with NUI Speech](#)

Contains basic information about using different local languages with the Natural User Interface (NUI) Speech API.

[Glossary of Speech Recognition Terms](#)

Defines terms used when discussing speech recognition and grammar construction, including speech recognition in Natural User Interface (NUI).

TCRs for NUI Speech

- [TCR 155: NUI Reserved Grammar](#)
- [TCR 156: NUI Speech Recognition Language Support](#)

See Also

[NUI Speech Reference](#)
[NUI Samples: Audio and Speech](#)
[NUI Speech Tools](#)
[NUI Whitenpapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.

Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Introduction to NUI Speech

[See Also](#)



NUI Speech enables your title to accept voice commands from players. With a recognition grammar you provide, the NUI Speech engine can match what players say to a list of vocal commands supported in your title, in English or other supported languages. You can change active grammar rules at any given time. This enables the recognition engine to work with a short list of commands appropriate to the current game context.

Sections

- [Underlying Technologies](#)
- [Getting Started](#)
- [Audio Input Management](#)
- [NuiAudio for Multichannel Echo Cancellation](#)
- [Grammars for Speech Recognition](#)
- [Language Support](#)

Underlying Technologies

The Kinect Sensor array includes an array of 4 microphones used to gather audio data from the play space. The audio data is filtered using multichannel echo cancellation (MEC) so that game audio doesn't feed back into the input audio stream. Sound data from the microphones is fed into the speech recognition engine. This makes it possible to recognize game- and context-specific voice commands, and to associate a voice command with a particular area in the play space.

The NUI Speech component is an Xbox-specific version of the Microsoft Speech Engine that has already been shipped in a variety of products, ranging from server to device platforms. NUI Speech enables phrase recognition in a context-free grammar, and rules in the grammar can be changed by the title depending on game context. The NUI speech engine is a direct sibling of the Microsoft Speech Engine (SAPI). The SAPI documentation on MSDN provides relevant details about the behavior of the speech engine that NUI uses.

Getting Started

To use NUI Speech, you must first create a grammar specifying the set of phrases your title will recognize. The grammar is specified using the W3C Speech Grammar schema. For more information about grammars, see the [Simple Speech Recognition](#) sample and the white paper [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Compile your grammar with the [Grammar Compiler](#), and include the compiled grammar in your title's disk image.

To initialize NUI Speech, in game code, populate a [NUI_SPEECH_INIT_PROPERTIES](#) structure, specifying the [NUI_SPEECH_KINECT](#) microphone type, and then pass that structure to [NuiSpeechEnable](#).

Note

Use of NuiSpeech functionality requires adding both NuiSpeech.lib and NuiAudio.lib to your project.

To use speech recognition, load a grammar with [NuiSpeechLoadGrammar](#), and then call [NuiSpeechStartRecognition](#). Specify speech event types of interest with a call to [NuiSpeechSetEventInterest](#). In the game loop, call [NuiSpeechGetEvents](#) regularly, and process the returned events.

To stop recognition without unloading the speech engine, call [NuiSpeechStopRecognition](#). When changing grammars or enabling or disabling individual grammar rules, do the following: Stop recognition, change the grammar, and then start recognition again. Recognition can also be stopped during gameplay where speech commands are not in use. However, stopping recognition does not return resources for use by game code.

Note

You must include certain Speech support files on your title's disk, as described in [Kinect Database Files to Include on the Game Disc](#).

To stop using NUI Speech and return its resources for use by game code, call [NuiSpeechDisable](#).

For a more detailed look at using the NUI Speech API, see [Using the NUI Speech API](#).

For a list of terms about speech recognition and grammar construction, see [Glossary of Speech Recognition Terms](#).

Audio Input Management

NUI Speech is responsible for managing the audio stream from the microphone array. The microphone array is calibrated based on characteristics of the room. Developers should use the [Kinect Tuner for Audio](#) whenever the sensor array is moved or the room characteristics change.

The audio stream can be captured and the input stream can be fed from captured audio rather than from the sensor array. This facilitates grammar development and testing with repeatable inputs.

For more information about audio management, calibrating and using microphones, and redirecting audio into or out of the NUI Speech engine, see [Audio Input with NUI Speech](#).

NuiAudio for Multichannel Echo Cancellation

The NUI Audio subsystem performs multichannel echo cancellation (MEC) for the microphone array in the Kinect Sensor, to support speech recognition. See [Multichannel Echo Cancellation with NuiAudio](#).

Grammars for Speech Recognition

The NUI Speech engine is an Xbox-specific version of the Microsoft Speech Engine, which has already been shipped in a variety of products, ranging from server to device platforms. NUI Speech performs phrase recognition in a context-free grammar (CFG). The grammar, specified in XML, consists of rules that specify which phrases are to be recognized. Phrase recognition results are returned as events, and return both the text of the recognized phrase and a grammar-specific identifier that can be used for easy command parsing.

For more information about using, defining, and tuning grammars for speech recognition in the NUI Speech engine, see [NUI Speech Grammars](#).

Additional resources:

- For a sample using the NUI Speech component, see [Simple Speech Recognition](#).
- You can use the [Speech Lab \(SpeechLab.exe\)](#) tool to load grammars and test them. This tool is a good resource for testing and refining grammars.
- You can find information about the W3C Speech Grammar specification on the W3C's website at <http://www.w3.org/TR/speech-grammar/>.
- The NUI speech engine is a direct sibling of the Microsoft Speech Engine (SAPI). The SAPI documentation on MSDN provides relevant details about the behavior of the speech engine that NUI uses.

Language Support

The NUI speech engine supports recognition of several languages. Only one language can be active for speech recognition at a time. For a list of supported languages, see the [NUI_SPEECH_LANGUAGE](#) enumeration.

Titles that support multiplayer and speech recognition must require that all players use the same language for commands. When initializing the speech engine, titles that have developed grammars for more than one language can and should give the player an option to select from the supported languages. When supporting multiple languages, a title must include grammars for each language, but also include the appropriate acoustic models for all supported languages in the title's disk layout. For more information about supported languages, and how to include the required acoustic models in your title's disk layout, see [Kinect Database Files to Include on the Game Disc](#).

For more information about language support, including details about specifying grammars for non-English languages, see [Local Language Support with NUI Speech](#).

See Also

[Simple Speech Recognition sample](#)

Using the NUI Speech API

[See Also](#)



The NUI Speech API enables speech recognition with the Kinect microphone array.

Sections

- [Basic Application Flow](#)
- [Processing Speech Events](#)
- [Confidence Values for Speech Events](#)

Basic Application Flow

Implementing speech support for a title follows a general procedure.

First, you must initialize the NUI Speech component, and load any grammars that will be used during game play.

Second, in the game loop, check for speech events and take the appropriate action, for example, controlling an avatar in the game or accessing the main menu. This second phase is similar to other input processing, and should be done at the same place in the game loop where the title is checking other control inputs.

The third and final phase occurs during shutdown of the game. Actions in this phase include stopping speech recognition by the title, unloading grammars, and terminating the NUI Speech component.

Note

Use of NuiSpeech functionality requires adding both NuiSpeech.lib and NuiAudio.lib to your project.

The following sequence provides more details about the speech-related phases that occur during the execution of a game.

Initialization Phase

1. Initialize the NUI speech component with a call to [NuiSpeechEnable](#).
Calling this function allocates the resources required for the component. Use the [NUI_SPEECH_INIT_PROPERTIES](#) structure to specify the properties of the component.
Note
The microphone type for all supported microphones, including the microphone array in the Kinect Sensor, is [NUI_SPEECH_KINECT](#).
2. Call the [NuiSpeechSetProperty](#) function to set individual speech properties such as defaults for confidence thresholds and the number of alternatives returned in recognition results.
For more information about confidence thresholds, see [Confidence Values for Speech Events](#), on this page. For default speech property values, see [NUI_SPEECH_PROPERTY](#).
3. Specify the list of speech events that the game code will listen for with a call to [NuiSpeechSetEventInterest](#).
4. Register for multiple events by combining event identifiers.
5. Specify grammars at development time, in XML that conforms to the [W3C Speech Recognition Grammar Specification](#). Then compile the XML specification, using the [xbpg.exe](#) grammar compiler, into a .CFG or .CFGP file.
During the initialization phase at run time, load the compiled grammar, or grammars, used by the title by calling [NuiSpeechLoadGrammar](#).
Grammars can be built in memory and loaded from memory, but that usage is beyond the scope of this topic.

Game Play Phase

1. Start speech recognition with a call to [NuiSpeechStartRecognition](#).
Starting speech recognition can only begin when at least one top-level rule is activated. In addition, starting and stopping recognition can be done at any time.
2. In the game loop, while processing other inputs, retrieve speech events that were raised during the recognition phase by calling [NuiSpeechGetEvents](#). Process the events and update the game's state accordingly.
See [Processing Speech Events](#) for details.
It is possible for fewer events to be returned than specified by the call to [NuiSpeechGetEvents](#).
Use the returned **puiFetched* value to determine how many events to process. If there are more events waiting for processing than requested, multiple calls to [NuiSpeechGetEvents](#) will retrieve all waiting events eventually.
3. After you process an event, destroy the event with a call to [NuiSpeechDestroyEvent](#).
This frees any resources that are associated with the destroyed event.
4. During gameplay, events in the game might determine the need to change which grammars or rules are active.
Retrieve the activation state of a given loaded grammar by calling [NuiSpeechGetGrammarState](#). Set a grammar's activation state by calling [NuiSpeechSetGrammarState](#).
It is possible to modify a loaded grammar and activate or deactivate grammars while speech recognition is active. However, it is recommended that the game code stop recognition before modifying a grammar, and start recognition again after modification is complete.

Termination Phase

1. Stop the speech recognition engine by calling [NuiSpeechStopRecognition](#).
2. Unload each previously used grammar by calling [NuiSpeechUnloadGrammar](#) on the loaded grammar.
To avoid resource leaks, you must unload every loaded grammar.
3. Terminate the NUI Speech component by calling [NuiSpeechDisable](#).
This completely frees all system resources used by the NUI Speech component.

Processing Speech Events

[NuiSpeechGetEvents](#) fills in an array of event objects, and returns a count of the number of event objects provided. Different events can be recognized by the value of the **eventId** member, and the rest of the event structure interpreted accordingly.

For recognition events, the event object contains a pointer to a recognition-result object. A recognition result consists of the recognized phrase, followed by a potential list of alternative phrases. Each phrase contains the information about the rule that was satisfied, the words that made up that phrase, and any semantic property information associated with the rule in the grammar. In addition, the confidence values, audio stream offset information, a language identifier, and the grammar identifier are also specified. Since a rule may contain other rules, the results are returned as a tree of rules and elements that the game code can parse.

Confidence Values for Speech Events

The biggest difference between handling speech recognition and handling controller input is that the results of speech recognition have a confidence value attached. The recognizer returns a list of alternatives, in decreasing order of confidence. It is up to the developer to determine the behavior and user-interaction model for various confidence values and whether alternatives are even considered.

For example, your title could vary the level of prompting based on confidence values. The following table describes this approach. *The threshold values listed here are only examples.* You must determine what threshold values work best for your title, given the specifics of how and when your title uses speech, the details of the grammar your title uses, and so on. This is best accomplished by testing grammars and settings with a number of different speakers.

Confidence Value	Action Taken
------------------	--------------

Less than 20%	Ignores current speech input.
Equal to 20% or greater and less than 50%	Explicitly prompts the user. For example, "Did you choose a pet?"
Equal to 50% or greater and less than 80%	Implicitly prompts the user. For example, "I think you said anteater. Say 'No' if that is incorrect."
Greater than 80%	Performs the requested action with no prompting.

✚ See Also

- [Speech Design for Game Designers: An Overview](#)
- [Talk To Me: Microphone and Speech Technology in XDK Kinect Hardware](#)
- [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#)
- [Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Audio Input with NUI Speech

[See Also](#)



The Kinect Sensor includes an array of microphones that are used to gather audio data from the play space. The Natural User Interface (NUI) Speech API can use a live or recorded audio data stream from the Kinect microphone array. Once the audio data is gathered, it is filtered using multichannel echo cancellation (MEC) so that game audio is not fed back into the NUI audio input stream, and the direction to the source of the sound is identified. Sound data from the microphones is fed into the speech recognition engine, making it possible to recognize game- and context-specific voice commands and to associate a voice command with a particular area in the play space.

The microphone array is calibrated based on the characteristics of the room. The [Kinect Tuner for Audio](#) should be used to recalibrate whenever the sensor array is moved or the room characteristics change.

The audio stream can be captured and the input stream can be fed from captured audio rather than from the sensor array, to facilitate grammar development and testing with repeatable inputs.

Titles can also use the NuiAudio API to access the audio stream from the microphone array. This method is useful if the title is performing its own processing on the audio data, such as emotional analysis or pitch detection.

Sections

- [Hardware Setup Guidelines](#)
- [Audio Input for Testing](#)

Hardware Setup Guidelines

Microphone placement, microphone calibration, and audio system configuration all impact the quality of speech recognition by the NUI Speech engine. Recognition is most accurate when the speech input is adequately loud and free of interference. The microphone array in the Kinect Sensor array must be calibrated for its current position in the play space in order for MEC to work effectively.

Follow these guidelines to get the best possible performance from the microphone array:

- Before testing speech recognition, run the [Kinect Tuner for Audio](#), which is available through the Guide.
- Speech recognition works best when the player is 1 to 2 meters from the microphone.
- Any audio being played by the development console over main speakers should measure no more than 70 dB SPL loudness at 1 meter from the microphone.
- If using an audio receiver, any additional signal processing (loudness, surround, and so on) should be turned off.
- Any background noise not generated by the development console should be kept to a minimum.
- Optimal sensor array placement is centered between the left and right speakers, but at some distance from the center speaker if one is present.
- Try to avoid indirect vibrations being transmitted to the sensor array.

Audio Input for Testing

You can use previously-recorded audio to automate some kinds of speech-related testing, such as grammar testing. Titles should not rely exclusively on the following method, though. It is vital that titles be tested extensively using the actual Kinect microphone array, preferably in a variety of room conditions.

A test program or a special testing build of your game can be written to read previously-recorded sound samples from test case files into memory, and then to use [NuiSpeechProcessWaveInput](#) or [NuiSpeechProcessWaveInputFromMemory](#) to provide known, repeatable audio inputs to the program. For example, this method could be used to measure how changes to a grammar correspond to changes in the number or quality of recognition results.

A word of warning: It might be possible to gather voice input from a headset microphone into a memory buffer, and then to pass the buffer to the speech engine using [NuiSpeechProcessWaveInputFromMemory](#). This use of the NUI Speech API is not a supported gameplay scenario, and should not be done in game code. Headset microphone input might be appropriate for some limited testing scenarios, but must never be done in shipping game code. The NUI Speech pipeline is optimized for input from the Kinect microphone array, which is acoustically different from headset microphone input. Voice recognition in games should use only the Kinect microphone array for input.



Note

Use of NuiSpeech functionality requires adding both NuiSpeech.lib and NuiAudio.lib to your project.

✚ See Also

- [Talk To Me: Microphone and Speech Technology in XDK Kinect Hardware](#)
- [Natural Environs: Understanding and Developing for the Kinect Play Space](#)
- [Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Multichannel Echo Cancellation with NuiAudio

[See Also](#)



The NUI Audio subsystem performs multichannel echo cancellation for the microphone array in the Kinect Sensor, to support speech recognition. The NuiAudio interface enables a title to consume echo-canceled audio from the Kinect Sensor. This processing is independent of the installed version of system software, unlike XHV.

This library fetches data from Kinect and processes the data to remove console audio from the data stream that is returned to the title.

The [Simple Speech Recognition](#) sample demonstrates NuiAudio Multichannel Echo Cancellation.

Sections

- [NuiAudio versus XHV and XMic](#)
- [Chat versus Speech Pipelines](#)
- [Privacy Protection with Automatic Use of the Audio Stream](#)

- [Processing Requirements](#)
- [No Automatic Gain Control on Speech Pipeline](#)
- [Parameter Validation with the Debug Build](#)
- [Avoid Thread Blocking](#)
- [Calling Functions from Callbacks](#)
- [Terminology](#)

 Note

Use of NuiAudio functionality requires adding NuiAudio.lib to your project. (NuiSpeech.lib isn't required for this.) To do this, see [Adding NUI Header and Library Files](#).

NuiAudio versus XHV and XMic

If a title is dependent on exactly repeatable audio performance, then NuiAudio is the suitable interface to use, rather than Xbox High Level Voice (XHV), provided that you use the speech pipeline rather than the chat pipeline. NuiAudio with the speech pipeline provides constancy of audio processing regardless of whether system updates have been applied to the console, because the audio processing is in the NUI library rather than in the system software. This constancy of audio processing can be useful for titles that do sensitive audio processing of their own for features such as the following:

- Voiceprint identification
- Emotion detection
- Speech processing

Feature	NuiAudio Chat Pipeline	NuiAudio Speech Pipeline	XHV	XMic
Kinect audio	Yes	Yes	Yes	No
Headset audio	No	No	Yes	No
USB or wireless microphone audio	No	No	No	Yes
Echo cancelation for Kinect	Yes	Yes	Yes	No
Audio processing consumes title CPU	No	Yes	No*	No*
Audio stream muted in response to privacy and user protection settings	No**	No**	Yes	No
Automatic gain control on audio stream	Yes	No	Yes	No
Audio processing may change with system update	Yes	No	Yes	No

*Titles using XHV or XMic will incur some overhead for data handling.

**NuiAudio always returns voice data regardless of whether the user mutes the Kinect microphone array.

Chat versus Speech Pipelines

Two pipelines are provided: the chat pipeline and the speech pipeline. Both versions of the pipeline can be active at the same time.

Aspect	Chat Pipeline	Speech Pipeline
Flag	NUIAUDIO_CHAT_PIPELINE	NUIAUDIO_SPEECH_PIPELINE
Location of Processing	Returns audio data from the Kinect device with a minimal CPU and memory impact.	Returns audio data from a console version of the Kinect audio pipeline.
Performance Costs	Reduced. Memory usage: <100 KB. CPU usage: <2% of CPU.	Memory usage: 1,975 KB. CPU usage: 10% of a hardware thread.
Beam Steering	Provides automatic beam steering. Doesn't support manual beam steering.	Supports manual beam steering, through NuiAudioSetMicArrayBeamAngle , as well as automatic beam steering.
Automatic Gain Control	The audio data has AGC applied.	AGC is not applied to audio data returned from this pipeline.
Release Ramifications	The audio can change over time with flash updates to the console. Chat behaviour is part of the flash and may change with flash updates, possibly breaking algorithms that do analysis on the stream.	Guarantees the audio will not change with future flash updates to the console. Speech processing is part of the XDK and is compiled into the title, so is guaranteed to be consistent across flash changes.

The [Sound Location Tracking sample](#) uses the NUI Audio chat pipeline.

Privacy Protection with Automatic Use of the Audio Stream

NuiAudio only works with the Kinect Sensor; no other audio input device is supported. If the Kinect Sensor is present, the audio stream will be available regardless of console privacy settings or child safety settings. To comply with technical certification requirements, titles must account for this if using NuiAudio for chatting or otherwise transmitting audio data.

Processing Requirements

Because audio processing is library-based, echo cancelation happens in title space, and the processing requirements should be accounted for in the title's CPU budget. This processing is shared with speech recognition, so if the title uses NuiSpeech, there is no additional penalty to consume NuiAudio.

No Automatic Gain Control on Speech Pipeline

Automatic gain control (AGC) is not applied to NuiAudio data from the speech pipeline, because AGC can interfere with speech processing. Because there is no AGC with the speech pipeline, volume levels in the NuiAudio stream may drop off significantly for quiet talkers or talkers at a distance from the Kinect Sensor.

Parameter Validation with the Debug Build

Use the debug build to enable parameter validation by way of asserts.

Avoid Thread Blocking

To minimize glitches, the realtime audio processing thread must not block. Methods that are called by the realtime thread are non-blocking and therefore should not use blocking synchronization, allocate memory, or access the disk.

Calling Functions from Callbacks

[NuiAudioSetMicArrayBeamAngle](#) can be called from [NuiAudio callbacks](#), because it doesn't block. The other [NuiAudio functions](#) cannot be called from NuiAudio callbacks, because they block.

Terminology

Frame

A block of samples, one per channel, to be played simultaneously. For example, a mono stream has one sample per frame.

 See Also

- [Sound Location Tracking Sample](#)
- [Simple Speech Recognition Sample](#)

[NuiAudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Speech Grammars

[See Also](#)



Speech grammars support speech recognition in Kinect titles.

In This Section

[Using NUI Speech Grammars](#)

A speech grammar for NUI defines what spoken phrases are recognized by a Kinect title.

[Workflow for NUI Speech Grammar Development](#)

The workflow for developing, testing, and tuning a speech recognition grammar and confidence thresholds for a Kinect title.

[Developing NUI Speech Grammars](#)

Developing, tuning, and testing a speech grammar for speech recognition in Kinect titles.

See Also

[Speech Tools for NUI](#)

[NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Using NUI Speech Grammars

[See Also](#)



Games that use the Natural User Interface (NUI) Speech API specify one or more lists of phrases to be recognized in the game, and then activate one particular list of phrases appropriately for the current game state. This topic contains basic information about how grammars are used in game code and specified in XML to describe those phrases.

Sections

- [Using a Compiled Grammar File in a Title](#)
- [How grammars return speech recognition results](#)
- [Creating and Refining a Grammar and Using It in Title Code](#)
- [Adding or Updating Phrases During Gameplay](#)
- [Japanese Grammar Format](#)

Using a Compiled Grammar File in a Title

To use a compiled grammar file in a title

1. Create a .grxml grammar file, compile it to a .cfg compiled grammar file, and refine these grammar files. See [Workflow for NUI Speech Grammar Development](#) and [Developing NUI Speech Grammars](#).
2. Include the compiled grammar file (.cfg) on your game disc image.
3. Include the required NUI Speech files on the game disc image, such as nuisp1033 for English-U.S. (en-us). See [Kinect Database Files to Include on the Game Disc](#).
4. In your title, initialize the NUI Speech component, and call [NuiSpeechLoadGrammar](#) to load any grammars (.cfg or .cfgp files) that will be used during game play.
5. In the game loop, check for speech events and take the appropriate action, for example, controlling an avatar in the game or accessing the main menu.
6. Stop speech recognition, unload grammars, and terminate the NUI Speech component.

How grammars return speech recognition results

A game can have multiple grammars loaded and active at the same time. Grammars are identified by a unique identifier that is specified by the game when the grammar is loaded. This identifier is passed back to the game in the recognition event. By default, grammars are activated when they are loaded.

The speech recognizer evaluates an utterance against all active top level rules in all active grammars, and returns a recognition result that lists all rules that were matched. The recognition result reports a confidence value for each rule match, indicating how well the utterance matches the rule.

When more than one rule is matched, it is up to game code to decide what action to take. Usually, it is best to use the result with highest confidence. To reduce the chance of ambiguity in recognition, keep the number of active rules low (around a dozen phrases,) and design grammars so that similar phrases are not used for different purposes.

Once a grammar is loaded, it can be activated or deactivated by a call to [NuiSpeechSetGrammarState](#). Internally, this call activates or deactivates the top-level rule in that grammar. The game can also specify a relative weighting to the top-level rule by a call to [NuiSpeechSetRuleWeight](#).

To free resources associated with a grammar, call [NuiSpeechUnloadGrammar](#).

Creating and Refining a Grammar and Using It in Title Code

Grammars are created in GRXML format, compiled with a special tool, refined using speech grammar tools, and included on the game disc. See [Workflow for NUI Speech Grammar Development](#).

Adding or Updating Phrases During Gameplay

You load a compiled grammar by calling [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#)). When you load a compiled grammar, you can specify whether to load it as a *dynamic grammar* or as a *static grammar*. Dynamic grammars are useful if you anticipate changes or updates to the words or phrases recognized at run time. For example, if your game allows the user to specify a word to recognize, you must update the grammar at run time with the new word.

To add a rule to a dynamic grammar, begin by calling [NuiSpeechCreateRule](#) to create a rule with default state. You can access the initial state of this new rule directly from the rule object. Next, add the list of words or phrases that should be recognized by calling [NuiSpeechAddWordTransition](#). This method takes two parameters: origin state and destination state. A null value for the destination state indicates a termination state. When you reach this state, a complete phrase has been recognized. After you add all the words or phrases to the rule, you must call [NuiSpeechCommitGrammar](#) to recompile the new grammar.

Intermediate States

It is also possible to create intermediate states within a rule and attach words to these intermediate states. Create a new state by calling [NuiSpeechCreateState](#). The new state can then be used in the call to [NuiSpeechAddWordTransition](#) discussed earlier.

Exporting Sub-rules for Reuse in Other Grammars

Sub-rules that are marked as exportable can be reused in other grammars. Sub-rules are created in the same way as top-level rules (by calling [NuiSpeechCreateRule](#)), but they should not be marked as top-level. Rules can then be attached to the transition between states by calling [NuiSpeechAddRuleTransition](#).

Japanese Grammar Format

For a discussion of the special format required for specifying grammars for the Japanese language, see [Local Language Support with NUI Speech](#).

See Also

[NUI Speech Reference](#)
[NUI Samples: Audio and Speech](#)
[NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Workflow for NUI Speech Grammar Development

Example



The workflow for developing, testing, and tuning a grammar and confidence thresholds follows this sequence.

Generally, the output of a given tool is used as the input for the next tool in this sequence.

Workflow

1. Author a .grxml grammar file.

Tool: [Authoring Grammar Files in Visual Studio](#)

In Visual Studio, create a .grxml grammar file that specifies the set of phrases your title will recognize. See also [Using NUI Speech Grammars](#), [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#), and [Simple Speech Recognition Sample](#).

2. Compile the .grxml file into a .cfg compiled grammar file.

Tool: [Grammar Compiler \(xbpg.exe\)](#)

3. Do ad-hoc testing of the grammar on the Xbox console.

Tool: [Speech Lab \(SpeechLab.exe\)](#)

Do ad-hoc testing of the grammar on the Xbox console, by saying words or phrases in the displayed list of grammar elements.

Input: A .cfg compiled grammar file, and live voice input into the Kinect microphone array.

Output: A log file that is created on the Xbox console's hard drive, listing the recognition confidence and number of times the word or phrase was recognized.

Perform ad-hoc testing using participants that match your intended audience. Leverage your co-workers and their families to help with the initial testing, either in Speech Lab (SpeechLab.exe on the Xbox console) or in-game. Iterate on your grammars in your native language to support your desired scenarios. Replace poorly performing phrases with alternate phrases.

4. Record echo-cancelled samples of users saying in-grammar and out-of-grammar phrases.

Tool: [NUI Voice Collection \(NuiVoiceCollection.exe\)](#) (on the Xbox console)

Gather data from at least 10 males, 10 females, and 10 children per locale. Skew the participants to the target audience; for example, if you are developing a title for children, use more child participants during data collection.

Input: Input file with the phrases that users will read, and live voice input into the Kinect microphone array.

Output: VoiceCollection.ini, and multiple Utterance####.wav files.

5. Generate an EMMA-formatted collection of the recorded voice samples.

Tool: [Speech Prep \(xbSpeechPrep.exe\)](#)

Input: Recorded voice samples from the Xbox console.

Output: An EMMA-formatted collection of the recorded voice input samples.

6. Perform speech recognition using the recorded voice samples, logging the results.

Tool: [Simulator \(Simulator.exe\)](#) (on your computer, with the Xbox console connected)

Perform speech recognition using the speech recognition engine on an Xbox console, using the recorded speech samples.

Input: Recorded speech samples, transcript text for input, and grammars.

Output: An EMMA-formatted file that indicates which phrase in a grammar was matched by the input. This recognition results file is a language-specific file, **RecoConfig_language_locale.xml**.

For Japanese speech, or when you use custom pronunciations in your grammar for any language, you must manually modify the EMMA-format input file to replace the transcript text with DLP (`</display></lexical></pronunciation>;`) information, to obtain accurate results.

7. Generate analyzed results of speech recognition on recorded samples.

Tool: [Simulator Results Analyzer \(SimulatorResultsAnalyzer.exe\)](#)

Generate analyzed results of automated testing of speech recognition.

Input: A language-specific EMMA-formatted recognition results file that indicates which phrase in a grammar was matched by the input.

Output: An XML file containing information on correct matches and incorrect matches, with confidence scores and data that will be used to generate the CAFA and FAFR charts used to evaluate grammar performance. (CAFA = Correct Accept/False Accept. FAFR = False Accept/False Reject.)

8. Generate a report of the speech recognition results, to select confidence values.

Tool: [Speech Lab Analysis Tool \(xbSpeechLab.exe\)](#)

Generate a report showing the results of the speech recognition simulation run, to help select the best confidence value for your scenarios.

Input: The log files generated by Speech Lab (`devkit:\speechLab\speechlab.xml`) and Simulator Results Analyzer (`SimulatorResultsAnalyzerOutput.xml`).

Output: A report in HTML format, including:

- Highlighted reports of weakest performing phrases
- Guidelines for initial confidence values
- The confidence distribution
- Attempted phrase occurrences
- The phrases with the worst error rates.

This report helps you select the best confidence value for your scenarios: those that maximize correct accepts while minimizing false accepts and false rejects.

Once the grammars are finalized in your native language, localize into the languages supported by your title. Repeat the data collection and tuning steps to select the best confidence values for those languages.

Use translators who are familiar with dialogue to provide the optimal translations for spoken language.

See also [The User Says What? Testing and Tuning Guidelines for Speech Recognition with Kinect for Xbox 360](#).

9. View graphical analysis of the speech recognition results, to select confidence values.

Tool: [Analyzer Results Viewer \(AnalyzerResultsViewer.exe\)](#)

View graphical analysis of the speech recognition simulation run, to help select the best confidence value for your scenarios.

Input: The log files generated by Speech Lab and Simulator Results Analyzer.

Output: Graphical results displays.

See the above step for more information about using this displayed information.

See also [The User Says What? Testing and Tuning Guidelines for Speech Recognition with Kinect for Xbox 360](#).

Example

The following batch file chains together the various tools that are used to generate the final report of the speech recognition results.

Batch File

```
xbspeechprep /data:C:\NuiSpeech\NuiVoiceCollectionRecordings\ /cfg:C:\NuiSpeech\Grammars\medieval.cfg /out:C:\NuiSpeech\Temp\EmmaFile.xml /model:er
Simulator.exe -in C:\NuiSpeech\Temp\EmmaFile.xml -out C:\NuiSpeech\Temp\Simulate.xml -Recoconfig C:\NuiSpeech\ConfigFiles\RecoConfig_en_us.xml
SimulatorResultsAnalyzer.exe -in C:\NuiSpeech\Temp\Simulate.xml -out C:\NuiSpeech\Temp\Results.xml
xbspeechlab /in:C:\NuiSpeech\Temp\Results.xml /out:C:\NuiSpeech\Reports\Report.htm
```

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Developing NUI Speech Grammars

[See Also](#)

For speech recognition to work well in a Kinect title, tune your use of speech to your particular grammar and your target audience. The speech grammar testing tools address the two most important elements for successful speech:

- A grammar or grammars that maximize the chance of a successful match, and arrive at the match with acceptable performance.
- A confidence threshold that maximizes correct accepts while minimizing false accepts and false rejects.

Sections

- [Concepts for Tuning Speech Grammars and Confidence Thresholds](#)
- [Confidence Thresholds](#)
- [Sample Confidence Thresholds](#)
- [Testing Custom Pronunciations](#)
- [Testing Dynamic Grammars](#)

Concepts for Tuning Speech Grammars and Confidence Thresholds

Use the speech tools to gather speech samples, feed those samples into your grammar, and classify the results to generate correct accept/false accept (CAFA) and false accept/false reject (FAFR) charts. These charts, based on test data you design for you specific application, enable you to select the confidence threshold that optimizes the speech experience for your scenarios.

- Designing well-performing grammars that maximize the chance of a successful match.
A successful match will be indicated by NuiSpeech returning the correct phrase—a correct accept (CA) with a high corresponding confidence value.
- Defining for the title's grammars a confidence threshold that maximizes CAs while minimizing FAs and FRs.

The confidence threshold is how the title determines, through a data set of user utterances collected by the title, what confidence value for a returned spoken phrase delineates between acting on the phrase or rejecting it.

Gameplay design is an important consideration for defining acceptable rates of FA and FR. For destructive actions, like deletion of a saved game, a higher confidence threshold and/or explicit user confirmation might be desirable. For lesser actions, or when there is little penalty for an FA, the confidence threshold may be allowed to be significantly lower. Additionally, consider the engagement model: when is NuiSpeech actively listening for commands? Always? In context-sensitive gameplay scenarios? Only in menus? Also consider how this information is conveyed to the player.

These guidelines for setting default confidence thresholds were developed while designing the NUI Hub and other Dashboard experiences. The NUI Hub and Dashboard grammars further defend against FAs by using a gating keyword ("Xbox") that protects from sidetalk, user conversation not intended to be directed at the console. To derive a confidence threshold from actual speech-focused testing, use a confidence threshold of 0.0 to ensure all recognitions are being triggered by the engine, and present debug information that displays which commands were accepted or rejected.

Confidence Thresholds

The confidence threshold is an important value to your scenario and design. This value determines the level of confidence that the recognizer must have before accepting an input from a user as matching an action in the grammar.

- If the confidence threshold is set too high, there is a risk of false rejects (a player speaks the phrase, but it is ignored because because the confidence value is not high enough.)
- If the confidence threshold is set too low, the player may experience false accepts (a player says something not intended as a command, but it is close to a command and recognized as such.)

Either situation is frustrating for players. Selecting the best confidence threshold will depend on a variety of factors including the game audio conditions, player configurations, engagement model, and your overall game design.

You should consider higher confidence thresholds for commands that have potentially serious consequences, and consider asking for explicit confirmations for the most critical commands. But you should avoid cluttering your design with burdensome confirmations for non-critical commands. It is important to collect audio data closely mirroring game conditions to enable testing and optimizing the confidence threshold for the experience your title offers.

When you are first creating the grammars for your title, you should begin testing using a confidence threshold of 20% (property confidence of "0.2"). The 20% value is a balanced starting point for evaluating the performance of your grammar(s), and is recommended based on the experience of designing the NUI Hub and other Dashboard experiences with US English and a gating keyword of "Xbox".

You should expect to set different confidence levels for different scenarios in your game. For example, you might offer speech control of a menu at the beginning of the game, while also offering speech control of a menu used during gameplay. The two scenarios represent different background noise levels, and probably different sets of spoken commands. You should test each of the scenarios separately, to determine the appropriate confidence thresholds for each scenario.

Sample Confidence Thresholds

The following table lists the confidence thresholds being used per acoustic model for the Xbox 360 Media Playback grammar, for two different listening modes. The appropriate threshold varies depending on the language, the grammar, and how listening modes determine which rules are active at a given time. (These are the thresholds in use at publication time; current values may be different due to further tuning efforts.)

Grammar	Media Playback (active listening)	Media Playback (normal listening)
de-AT	10	10
de-CH	15	45
de-DE	10	10

en-AU (& en-NZ)	10	10
en-GB (& en-IE)	25	20
en-US (& en-CA)	25	25
es-ES	20	25
es-MX	25	30
fr-CA	10	15
fr-CH	25	25
fr-FR	20	25
it-IT	10	15
ja-JP	10	20
pt-BR	20	35

active listening

This state is when the end-user says the keyword, "Xbox," and waits for the user interface to display the available phrases. For example, the user says, "Xbox",, the user interface displays options, then the user says, "Play." The effect of active listening is that fewer rules are active at a given time (rules are activated and deactivated as the UI changes), and so confidence thresholds can be relaxed while still maintaining low error rates.

normal listening

This state is when the end-user says the full command and the user interface is not displayed. For example, the user says, "Xbox, Play." This mode has more active rules at a given time, and so confidence thresholds must be higher to prevent errors in recognition.

Auxiliary Language-Country Combinations

Some language-country combinations are not explicitly used in the API. Those language-country combinations are instead covered by other language-country combinations. For example, "en-us" is a language code representing the English-United States locale and the English-United States acoustic model database. The en-us language code and acoustic model database is also used to cover the en-ca (English-Canada) locale. The "en-ca" language code is not explicitly used in the NUI Speech recognition API.

For the list of acoustic model databases for language-country combinations, see [Kinect Database Files to Include on the Game Disc](#).

For the list of language-country combinations in the API, see [NUI_SPEECH_LANGUAGE](#).

Testing Custom Pronunciations

Custom pronunciations are unsupported by the tools for automated speech testing and tuning. That is, using <token> tags and UPS pronunciations are unsupported. In order to retain automated test support, titles may phonetically hint at pronunciations if the default pronunciation for a spelled word demonstrates poor returned confidence. For Japanese support, see [this note](#).

For this scenario, ad-hoc and in-app testing are generally the best methods to collect data; the Retain Audio mode of NuiSpeech (demonstrated by the [Speech Retain Audio](#) sample) can be used for this purpose, logging and recording utterances for offline analysis.

Testing Dynamic Grammars

Dynamic grammars are unsupported by the tools for automated speech testing and tuning. In particular, programmatically injecting new words or phrases into an active grammar is unsupported. For testing purposes, use static compiled grammar equivalents.

For this scenario, ad-hoc and in-app testing are generally the best methods to collect data; the Retain Audio mode of NuiSpeech (demonstrated by the [Speech Retain Audio](#) sample) can be used for this purpose, logging and recording utterances for offline analysis.

- See Also
- [Workflow for NUI Speech Grammar Development](#)
[Speech Tools for NUI](#)
[NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Local Language Support with NUI Speech

The Natural User Interface (NUI) Speech API enables titles to support the use of some local languages. This topic discusses how to add support for local languages, and some specific issues in specifying grammars for the Japanese language, in the following sections:

- [Language Support](#)
- [List of Languages and Acoustic Model Files](#)
- [Display/Lexical/Pronunciation \(D/L/P\) Grammar Format](#)
- [sapi:alphabet Attribute for Japanese](#)

Language Support

The NUI speech engine supports recognition of several languages, in addition to English. However, only one language can be active for speech recognition at a time. Titles that support both multiplayer and speech recognition must require that all players use the same language for voice commands.

Titles that have grammars for more than one language can and should give players an option to select from the supported languages before initializing the speech engine. To determine which language to use as a default, titles should use a process like the following:

- Call [XGetLocale](#) to derive the console country.
For some countries, this may be sufficient to glean the spoken language, since only one language is supported for that country.
- Call [XGetLanguage](#).
Based on the language-country combination, compare the result against the list of grammars and acoustic models included on the title's disc.
- From the list of languages the title supports, choose the most appropriate language for the console settings.

For example, if the console is set to XC_LOCALE_MEXICO:

- The only language supported by the console in that locale is *Spanish - Mexico*.
- If the title supports Spanish - Mexico, that language is specified to **NuiSpeechEnable**.
- If the title does not support Spanish - Mexico, the title shows a message to that effect, and lists the supported languages. The player selects a supported language, and the title passes that language to **NuiSpeechEnable**.

The speech engine loads only one language-specific model during initialization. For the title to switch to another language after initialization, the engine must first be uninitialized (by calling [NuiSpeechDisable](#)), and then reinitialized for the new language (by calling [NuiSpeechEnable](#)).

List of Languages and Acoustic Model Files

Each acoustic model covers a language-country combination. For each language a title supports, the title must include grammars and the appropriate acoustic model in the title's disc layout.

- For the list of acoustic models, and the disc space requirements for languages, see [Kinect Database Files to Include on the Game Disc](#).
- The [NUI_SPEECH_LANGUAGE](#) enumeration shows the API names of these acoustic models. If you have any questions about support for other languages, consult your account manager.

Display/Lexical/Pronunciation (D/L/P) Grammar Format

There are some applications where the display text for a phrase and the pronunciation of the phrase need to be described separately. One such application would be a game localized to Japanese, where the display text could be in Kanji, Hiragana, or Katakana. Another example is a fantasy or science fiction game where character names are pronounced in the historical or made-up language of the game setting. The grammar format Display/Lexical/Pronunciation is used to specify the exact display format, pronunciation, and reported string for a grammar rule.

D/L/P in Japanese

If your title uses spoken Japanese as speech input, the grammar must specify additional information to describe the display text, lexical form, and pronunciation of the elements. The information used in Japanese grammars is specified in and by the <item> tag as follows:

- "display" attribute**
Display text, either Kanji, Hiragana, or Katakana, of the element. In this example, "?".
- "sapi:pron" attribute**
Pronunciation of the element, in Katakana. In this example, "??".
- Inner text of the <token> tag**
The lexical form of the element, in Hiragana. In this example, "??".

The following is an example of a complete item element (in this case, the Japanese word for *song*.)

```
<item>  
  <token display="?" sapi:pron="??"?></token>  
</item>
```

D/L/P for a Fantasy Name

If a game uses words from other languages, including invented languages in a fantasy or science fiction setting, the standard rules for parsing a string from a grammar rule into phonemes for the current NUI speech language setting might not properly handle names pronounced according to the linguistic rules of the game setting. This is another place where the D/L/P format can be used:

- "display" attribute**
Display text, in a suitable character set.
- "sapi:pron" attribute**
Pronunciation of the element, in phonemes. [A phoneme reference table](#).
- Inner text of the <token> tag**
The lexical form of the element.

The following is an example of a complete item element (in this case, the title character from the Old English story, *Beowulf*.)

```
<item>  
  <token display="Beowulf" sapi:pron="S1 B EH O S2 UH L F">Beowulf</token>  
</item>
```

sapi:alphabet Attribute for Japanese

The grammar compiler for Xbox 360 expects Japanese grammar files to specify "x-microsoft-sapi" for the *sapi:alphabet* attribute of the **grammar** element, in a .grxml file. This is a limitation specific to Xbox 360.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Glossary of Speech Recognition Terms

[See Also](#)



The following specialized terms are used when discussing speech recognition and grammar construction, including speech recognition in Natural User Interface (NUI).

- correctly accepted in grammar (CA/in)**
The fraction of user utterances covered by the grammar and correctly recognized and accepted by the recognizer.
- correctly accepted of all (CA/all)**
The fraction of all user utterances correctly recognized, and accepted, by the recognizer.
- correctly rejected out of grammar (CR/out)**
The fraction of user utterances not covered by the grammar and correctly rejected by the recognizer.
- falsely accepted in grammar (FA/in)**
The fraction of all user utterances covered by the grammar and incorrectly recognized by the recognizer.
- falsely accepted of all (FA/all)**
The fraction of all user utterances incorrectly recognized by the recognizer.
- falsely accepted out of grammar (FA/out)**
The fraction of all user utterances not covered by the grammar and incorrectly recognized by the recognizer.
- falsely rejected in grammar (FR/in)**
The fraction of all user utterances covered by the grammar and rejected by the recognizer.
- in-grammar (IG)**
The rate of user utterances covered by the grammar, measured as a fraction of all user utterances.
- out-of-grammar (OOG)**
The rate of user utterances not covered by the grammar, measured as a fraction of all user utterances.
- rule confidence**
The overall confidence for all the words in the phrase contained within a rule. Thus, for the top-level rule, the rule confidence gives an overall confidence for the whole phrase. Normally you use semantic property confidence rather than rule confidence; if semantic property confidence is not working out, that is, you cannot find a satisfactory set of phrases to use, try using rule confidence instead. The [FLOAT fSREngineConfidence](#) and [NUI_SPEECH_CONFIDENCE eConfidence](#) values in the [NUI_SPEECH_RULERESULT](#) struct represent rule confidence. See also *semantic property confidence*.

- semantic property confidence**
The confidence for a word. Also known as semantic confidence or property confidence. Normally you use semantic property confidence rather than rule confidence; if semantic property confidence is not working out, that is, you cannot find a satisfactory set of phrases to use, try using rule confidence instead. The [NUI_SPEECH_CONFIDENCE eConfidence](#) value in the [NUI_SPEECH_SEMANTICRESULT](#) struct represents the semantic property confidence. Also known as *semantic confidence* or *property confidence*. See also *rule confidence*.
- top-level rule**
The root rule in a grammar. Certain Speech Recognition behaviors are specific to top-level rules in your grammar. For example, there are [NUI_SPEECH_GRAMMARSTATE](#) values that enable or disable all top-level rules.

 [See Also](#)

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Wave Gesture Detection

[See Also](#)



The NUI Wave API enables your title to use the same wave gesture detection method used in the Xbox 360 Dashboard. This enables you to provide a uniform experience for your title's users who are already using Kinect with the Dashboard and other titles. You can see this API in action by running the [WaveGestureTracking](#) sample.

A wave gesture is defined as a number of consecutive of arm waves. You call the NUI wave recognition API periodically to determine whether a wave is in progress, and how much progress has been made. Use the progress value to provide feedback to users, such as the expanding-arc feedback used in the Dashboard.

Initializing

The NUI Wave API relies on skeleton tracking. To enable wave gesture detection, call [NuiInitialize](#) with the flag [NUI_INITIALIZE_FLAG_USES_SKELETON](#), and then call [NuiWaveSetEnabled](#), setting *bEnabled* to TRUE.

Monitoring

Periodically, call [NuiWaveGetGestureOwnerProgress](#). This function returns the skeleton tracking ID and progress of any wave gesture that is in progress. A progress value of 0.0 means that no wave gesture is in progress, while a value of 1.0 means that a gesture has completed. You can use intermediate progress values to give user feedback that a wave is in progress, and how close it is to completion. [NuiWaveGetGestureOwnerProgress](#) uses a simple, default policy to determine which player's progress to report. It simply reports the progress of the player whose wave is closest to completion.

Using a Custom Rule for Which Player Is Waving

If you want to track wave gesture progress for more than one player, or you want to make your own determination of which player's wave gesture to track, then periodically call [NuiWaveGetState](#) instead. This function returns a [NUI_WAVE_STATE](#) structure, which is an array of [NUI_WAVE_PLAYER_STATE](#) structures, each describing the wave gesture progress of one player. The [NUI_WAVE_PLAYER_STATE](#) structure contains detailed information about gesture progress, on a per-hand basis, for each player whose gesture is being tracked. You can decide how to use this information to provide feedback to users.

See Also

[NuiWave Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Coordinate System Transformations



Several coordinate systems are used with image frames and skeleton tracking. The NUI API supports transformations between these spaces.

Sections

- [Depth Image Space](#)
- [Skeleton Space](#)
- [Sensor Array and Tilt Compensation](#)
- [Floor Determination](#)
- [Skeletal Mirroring](#)

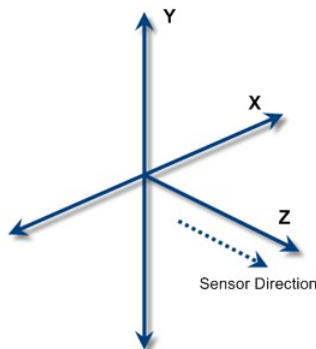
Depth Image Space

Image frames of the depth map are 320x240 pixels in size, with each pixel representing the distance, in millimeters, to the nearest object at that particular x and y coordinate. The x and y coordinates of the image frame do not represent physical units in the room, but rather pixels on the depth imaging sensor. The interpretation of the x and y coordinates depends on specifics of the optics and imaging sensor. For discussion purposes, this projected space is referred to as the *depth image space*.

Skeleton Space

Player skeleton positions are expressed in x, y, and z coordinates. Unlike the coordinate of depth-image space, these three coordinates are expressed in meters. The x, y, and z axes are the body axes of the depth sensor, using a right-handed coordinate system and placing the sensor array at the origin point with the positive z axis extending in the direction that the sensor array is pointing. The positive y axis extends upward and the positive x axis extends to the left (with respect to the sensor array). For discussion purposes, this is referred to as the *skeleton space*.

Figure 1. World-space coordinate system for the sensor array



Sensor Array and Tilt Compensation

Placement of the sensor array affects the images received from the camera. For example, the camera may be placed on a surface that is not level or the sensor array might be vertically pivoted to optimize the sensor's field of view. In these cases, the y-axis of the skeleton space is usually not perpendicular to the floor or parallel with gravity. From the perspective of the camera, people standing up straight could appear to be leaning. For these reasons, each skeleton frame contains a [vNormalToGravity](#) member. The value is a measurement from an internal 3-axis accelerometer that has been calibrated to align with the image sensor. In the absence of motion, the accelerometer measures a 1 g normal force to gravity. This can be treated as an upward vector. In addition, usage of [NuiTransformMatrixLevel](#) is recommended. This function constructs a transformation matrix, that rotates the y-axis of the skeleton space to point up. This matrix can be applied with any of the matrix transformation methods. The normal to gravity can be found in the [vNormalToGravity](#) member of the [NUI_SKELETON_FRAME](#) structure.

Floor Determination

Each skeleton frame contains [vFloorClipPlane](#), a member that contains the coefficients of an estimated floor plane equation. The skeleton tracking system updates this estimate for each frame and uses it as a clipping plane for removal of the background and segmentation of players. The general plane equation is:

$Ax + By + Cz + D = 0$

where:

- $A = vFloorClipPlane.x$
- $B = vFloorClipPlane.y$
- $C = vFloorClipPlane.z$
- $D = vFloorClipPlane.w$

The equation is normalized such that the physical interpretation of D is the height of the camera from the floor, in meters. The floor may not always be visible. In this case, [vFloorClipPlane](#) is a zero vector.

Skeletal Mirroring

By default, the skeleton system always mirrors the player being tracked. For titles that use an avatar to represent the player, mirroring could be desirable, if the avatar is shown facing into the screen. However, if the avatar is facing the player, mirroring would present the avatar as backwards. Depending on the title's needs, a transformation matrix to mirror the skeleton can be created by using **XMMatrixReflect** and then applied to the points in the *SkeletonPositions* array that represents the skeleton. Choosing the proper plane for reflection should be handled by the title.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Development Tools

[See Also](#)



The NUI Tools API enables you to build tools on the development computer to manipulate data captured on Xbox 360. The [NuiTools Reference](#) header file contains declarations for selected data types used by the Natural User Interface API. You can see this API in action by running and analyzing the following samples.

Sample	Description
NUI PC Conduit Sample	Shows how to use the Xbox Studio API from a (C++) program running on your development computer. Transfers NUI data between a Windows-based development computer and Xbox 360.
Managed NUI Conduit Sample	Shows how to use the Xbox Studio API from a managed (C#) program running on your development computer. Consumes NUI data (color, depth, player index and skeleton) and visualizing it to the user.
XedFileTagger Sample	Shows how to use the XedFile API from a (C#) program running on your development computer. Each depth event can be tagged with a string value that can be used for helping with development of gesture detection or joint filtering.

Examples

NUI PC Conduit Sample

The following code fragment shows how to use the [NUI_SKELETON_FRAME](#) and [NUI_SKELETON_DATA](#) data types defined in the [NuiTools Reference](#) header file.

For more information, see [NUI PC Conduit Sample](#).

```
C++

if( pEvent->StreamID == XSTUDIO_STREAM_ID_NUIAPI_SKELETON)
{
    assert( pEvent->cbDataBufferUsed == sizeof(NUI_SKELETON_FRAME) );

    NUI_SKELETON_FRAME* pFrame = (NUI_SKELETON_FRAME*)(pEvent->pvDataBuffer);
    memcpy(&g_skeletonFrame[(g_nCurSkeletonFrame+1)%2], pFrame, sizeof(NUI_SKELETON_FRAME) );
    g_nCurSkeletonFrame = (g_nCurSkeletonFrame+1)%2;
}
```

Managed NUI Conduit Sample

Managed NUI Conduit is a C# project that does not reference the C++ header NuiTools.h. Instead it defines its **NUI_SKELETON_FRAME** and **NUI_SKELETON_DATA**, as shown in the following code fragment.

For more information, see [Managed NUI Conduit Sample](#).

```
C#

[StructLayout(LayoutKind.Sequential, Pack = 4, Size = 448)]
internal struct NUI_SKELETON_DATA
{
    public NUI_SKELETON_TRACKING_STATE eTrackingState;
    public uint dwTrackingID;
    public uint dwEnrollmentIndex;
    public uint dwUserIndex;
    public XMVECTOR Position;

    private const int ArrayLength = (int)NUI_SKELETON_POSITION_INDEX.NUI_SKELETON_POSITION_COUNT;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = ArrayLength)]
    public XMVECTOR[] SkeletonPositions;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = ArrayLength)]
    public NUI_SKELETON_POSITION_TRACKING_STATE[] eSkeletonPositionTrackingState;

    public NUI_SKELETON_QUALITY_FLAGS dwQualityFlags;
}

[StructLayout(LayoutKind.Sequential, Pack = 4)]
internal struct NUI_SKELETON_FRAME
{
    public long liTimeStamp;
    public uint dwFrameNumber;
    public NUI_SKELETON_FRAME_FLAGS dwFlags;
    public XMVECTOR vFloorClipPlane;
}
```

```
public XMVECTOR vNormalToGravity;

private const int ArrayLength = NUI_SKELETON_COUNT;

[MarshalAs(UnmanagedType.ByValArray, SizeConst = ArrayLength)]
public NUI_SKELETON_DATA[] SkeletonData;
}
```

XedFileTagger Sample

XedFileTagger is a C# project that does not reference the C++ header NuiTools.h. Instead it defines its **NUI_SKELETON_FRAME**, as shown in the following code fragment. For more information, see [XedFileTagger Sample](#).

C#

```
[StructLayout(LayoutKind.Explicit, Size=2736)]
public struct NUI_SKELETON_FRAME
{
    [FieldOffset(0)]
    public Int64 liTimeStamp; // 8 Bytes

    [FieldOffset(8)]
    public UInt32 dwFrameNumber; // 4 Bytes

    [FieldOffset(12)]
    public UInt32 dwFlags; // 4 Bytes

    [FieldOffset(16)]
    public XMVector vFloorClipPlane; // 16 Bytes

    [FieldOffset(32)]
    public XMVector vNormalToGravity; // 16 Bytes

    [FieldOffset(48)]
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = SkeletonCount)]
    public NUI_SKELETON_DATA[] SkeletonData; // 2688 Bytes

    private const Int32 SkeletonCount = Constants.NUI_SKELETON_COUNT;

    // The default constructor does not know how to allocate the right size structure, therefore
    // we have to create this new constructor which will dynamically allocate the correct amount
    // of memory for the internal arrays
    public NUI_SKELETON_FRAME(Byte unused)
    {
        liTimeStamp = 0;
        dwFrameNumber = 0;
        dwFlags = 0;
        vFloorClipPlane = new XMVector();
        vNormalToGravity = new XMVector();
        SkeletonData = new NUI_SKELETON_DATA[SkeletonCount];

        for (UInt32 i = 0; i < SkeletonCount; i++)
        {
            SkeletonData[i] = new NUI_SKELETON_DATA(0);
        }
    }
}
```

See Also

[NuiTools Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface Reference



This section contains the API Reference pages for Natural User Interface (NUI), which is used for Kinect.

In This Section

[NuiAPI Reference](#)

NuiApi.h – Initializes Kinect with selected subsystems, and includes some other NUI headers.

[NuiAudio Reference](#)

NuiAudio.h - Performs multichannel echo cancellation for the microphone array in the Kinect Sensor, to support speech recognition. This processing is independent of the installed version of system software, unlike XHV.

[NuiDiagnostics Reference](#)

NuiDiagnostics.h – Displays the Kinect Troubleshooter in the Xbox Guide.

[NuiFitness Reference](#)

NuiFitnessApi.h – Tracks the amount of energy expended, based on the movement of a tracked skeleton or on a custom algorithm.

[NuiGesture Reference](#)

NuiGesture.h – Enables title developers to perform run-time gesture detection by using data generated with Visual Gesture Builder.

[NuiHandles Reference](#)

NuiHandles.h – Enables hand movements to manipulate UI controls such as buttons, scroll bars, and menus.

[NuiHeadOrientation Reference](#)

NuiHeadOrientation.h - Provides head orientation tracking, to detect the direction the player's head is facing.

[NuiHeadPosition Reference](#)

NuiHeadPosition.h - Provides lightweight head position tracking, as an alternative to using skeleton tracking and monitoring the position of the head joint.

[NuiIdentity Reference](#)

NuiIdentity.h – Performs face and skeleton recognition, to identify a detected skeleton as a particular player each time the skeleton comes into view.

[NuiImageCamera Reference](#)

NuiImageCamera.h – Processes color and depth image data from the Kinect Sensor.

[NuiSkeleton Reference](#)

NuiSkeleton.h – Tracks skeleton joint positions of players based on color and depth images from the Kinect Sensor.

- [NuiSpeech Reference](#)
NuiSpeech.h – Performs speech recognition using the Kinect microphone array.
- [NuiTools Reference](#)
NuiTools.h – For use in Win32-based tools that manipulate previously captured NUI data.
- [NuiWave Reference](#)
NuiWave.h – Detects the wave gesture, which is also used in the Xbox Dashboard and Guide.
- [XedFile Reference](#)
XedFile.h – Operates on the Xbox event data (XED) files created by the Xbox Studio tool.
- [XNui Reference](#)
Utility functions for NUI, such as suppressing the system gesture, OOBE mode, and Kinect connectivity status.
- [XStudio Reference](#)
XStudio.h – Records and plays back the XED files used by the Xbox Studio tool.

✚ **Remarks**

To add header files to a project, see [Adding NUI Header and Library Files](#).

To search the entire set of samples to find sample code that uses a particular API item, see [Using the XDK Samples](#).

- Related Topics**
- [XDK Unlocked - Videos](#)
- [NUI Overview](#)
- [NUI Samples](#)
- [NUI Tools](#)
- [NUI White Papers](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAPI Reference

[See Also](#)

☰

This section contains the API Reference pages for items in the header file NuiApi.h, which enables initializing Kinect with selected subsystems and includes most other NUI headers.

- In This Section**
- [NuiApi Functions](#)
- [NuiApi Constants](#)

✚ **Remarks**

NuiApi.h includes most of the NUI header files. Do not directly include the NUI header files that NuiApi.h already includes. For more information, see [Adding NUI Header and Library Files](#).

- ✚ **See Also**
- [Using Images](#)
- [Initializing NUI Image Camera](#)
- [Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiApi Functions

[See Also](#)

☰

- In This Section**
- [NuiInitialize](#)
Initializes Kinect to use the specified hardware thread and subsystems such as color camera, depth sensor, and skeleton tracking.
- [NuiShutdown](#)
Releases resources used by Kinect.

- ✚ **See Also**
- [Using Images](#)
- [Initializing NUI Image Camera](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiInitialize

[See Also](#) [Example](#)

☰

Initializes Kinect to use the specified hardware thread and subsystems such as color camera, depth sensor, and skeleton tracking.

✚ **Syntax**

C++

```
HRESULT NuiInitialize(  
    DWORD dwFlags,
```

```
        DWORD dwHardwareThreadSkeleton
    }
}
```

Parameters

dwFlags
[in] The subsystems the title will use, as a bitwise-OR'd combination of [NUI_INITIALIZE flags](#).

dwHardwareThreadSkeleton
[in] The hardware thread on which the skeleton tracking subsystem will run, as a hardware thread identifier (0-5).

To use the default hardware thread for skeleton tracking and depth data, which is thread 5, pass [NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD](#). If your title requires a particular hardware thread for skeleton tracking and depth data, pass the required value instead.

For performance reasons, you should perform skeleton tracking on a thread other than the Direct3D worker threads. When creating the D3D device using [CreateDevice](#), you can specify on which hardware threads Direct3D should create worker threads. These worker threads are used for patching the GPU command buffer when using [predicated tiling](#) or Xbox Procedural Synthesis (XPS). If no [D3DCREATE_CREATE_THREAD_ON_n](#) flags are specified in the *BehaviorFlags* parameter of **CreateDevice**, Direct3D creates the software threads on hardware thread 2 for predicate tiling and on hardware thread 3 for XPS.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	The <i>dwFlags</i> value is invalid. This can be because an invalid flag bit was specified, or because a mutually-incompatible set of flag bits was specified.
E_NUI_ALREADY_INITIALIZED	NuiInitialize has already been called.
E_NUI_DATABASE_NOT_FOUND	The exemplar or identity database files cannot be found.
E_NUI_DATABASE_VERSION_MISMATCH	The exemplar or identity database files are found, but do not match the version required by the title libraries.
E_OUTOFMEMORY	There is not enough memory to initialize the system.
E_NUI_DATABASE_CORRUPTED	The exemplar or identity database files are found, but are corrupted.

Remarks

When you call this function, the Xbox 360 system software allocates memory, loads data, and starts a thread that communicates with the sensor array. The time required to perform these operations varies depending on which Kinect capabilities have been requested in the *Flags* argument, and whether the game is loading data from the hard drive or from a game disc.

The resources consumed by the Natural User Interface (NUI) libraries vary according to which subsystems are specified in *dwFlags*. [NUI Resource Usage](#) describes the resource costs of the various subsystems.

After this function returns, the sensor array communication thread periodically checks to see if a sensor array is connected. When a sensor array is found and initialized, the thread pumps data from the sensor array for retrieval by the other NUI API functions. The sensor communication thread continues to operate until [NuiShutdown](#) is called.

Use the following sequence:

1. Initialize the D3D device.
2. Call **NuiInitialize**.
3. Perform gameplay.
4. Call [NuiShutdown](#).
5. Shut down the D3D device.

Calls to **NuiInitialize** should be paired with calls to [NuiShutdown](#). Calling **NuiInitialize** twice without an intervening call to [NuiShutdown](#) is an error. If you want to initialize and shut down the Natural User Interface system more than once during gameplay, call [NuiShutdown](#) in between calls to [NuiInitialize](#).

Add the header and library files to the project, corresponding to the selected subsystem flags. See [Adding NUI Header and Library Files](#).

Example

This function is used in all NUI samples, and is used in [Quick-Start Tutorial for Kinect Programming](#) as follows:

```
HRESULT Sample::Initialize()
{
    HRESULT hr;

    // Initialize NUI system
    hr = NuiInitialize(
        NUI_INITIALIZE_FLAG_USES_SKELETON |
        NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
        NUI_INITIALIZE_FLAG_USES_COLOR,
        NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD ); // Skeleton processing thread
    if( FAILED( hr ) )
    {
        ATG_PrintError( "Couldn't initialize NUI\n" );
        return hr;
    }

    return S_OK;
}
```

Requirements

Header: Declared in NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[Using Images](#)
[Initializing NUI Image Camera](#)
[Quick-Start Tutorial for Kinect Programming](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiShutdown

[See Also Example](#)



Releases resources used by Kinect.

Syntax

C++

```
void NuiShutdown()
```

Return Value

None.

Remarks

This function releases all resources associated with the Natural User Interface (NUI) system. Any outstanding operations are canceled, any open stream handles are closed, and any subsequent operations on open stream handles fail. Once this function has been called, NUI effectively is shut down, and all calls to it will fail.

When terminating the title, call this function before shutting down the D3D device currently in use.

If NUI has not been initialized, this function does nothing.

If you want to initialize and shut down NUI multiple times during gameplay, call this function between calls to [NuiInitialize](#).

Example

This function is used in many of the samples, often as shown below.

```
virtual ~Sample()
{
    if( m_pAvatarRenderer )
        delete m_pAvatarRenderer;
    if( m_pNuiJointConverter )
        delete m_pNuiJointConverter;

    // Shutdown XAvatar and release memory
    XAvatarShutdown();
    NuiSkeletonTrackingDisable();
    NuiShutdown();
}
```

Requirements

Header: Declared in NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[Using Images](#)
[Initializing NUI Image Camera](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiApi Constants

[See Also](#)

In This Section

- [NUI_INITIALIZE Flags](#)
Flags for initializing Kinect, including whether to use the color camera, the depth sensor, and skeleton tracking.
- [NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD](#)
Specifies using the default hardware thread for skeleton tracking and depth data, or the default hardware thread for speech recognition.
- [NUI_XALLOC Flags](#)
Recommends whether memory allocation should come from the upper or lower range of physical memory.

See Also

[Using Images](#)
[Initializing NUI Image Camera](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_INITIALIZE Flags

[See Also](#) [Example](#)

Flags for initializing Kinect, including whether to use the color camera, the depth sensor, and skeleton tracking.

Name	Description
NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX	Initializes the sensor array to provide depth information and player segmentation information. Specifying this flag starts the skeleton tracking engine.
NUI_INITIALIZE_FLAG_USES_COLOR	Initializes the sensor array to provide color information.
NUI_INITIALIZE_FLAG_USES_SKELETON	Initializes the sensor array to provide skeleton tracking information. Specifying this flag starts the skeleton tracking engine. This flag is mutually exclusive with NUI_INITIALIZE_FLAG_USES_HEAD_POSITION.
NUI_INITIALIZE_FLAG_USES_DEPTH	Initializes the sensor array to provide depth information. This depth information is formatted so it is identical to that returned when you use NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX. However, in this case, the player indices are always zero. If another NUI_INITIALIZE Flags flag starts the skeleton tracking engine, you can set player indices even when you specify this flag.
NUI_INITIALIZE_FLAG_USES_HIGH_QUALITY_COLOR	Initializes the sensor array to provide high-quality color information.

You can specify this flag or `NUI_INITIALIZE_FLAG_USES_COLOR`, but not both. To change from using high-quality color to using normal color, first call [NuiShutdown](#), and then call [NuiInitialize](#) specifying the flag `NUI_INITIALIZE_FLAGS_USES_COLOR`.

NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY	Enables Automatic Mode for Identity . When the Identity system is running in Automatic Mode, it will automatically run Identify operations on any new skeletons that appear in the sensor's field of view. To temporarily disable Automatic Mode during gameplay (such as to use the RGB stream for taking photos), call NuiIdentityPause , then later re-enable Automatic Mode by calling NuiIdentityResume .
NUI_INITIALIZE_FLAG_EXTRAPOLATE_FLOOR_PLANE	Allows the title to opt-in to the calculated floor plane. Without this flag, the reported floor plane will be zero. (Internally, auto-tilt always uses an extrapolated floor plane.) A floor search is performed the first time you invoke Kinect. You don't need to perform a floor search again while the console is turned on unless the camera is physically disturbed. Kinect can extrapolate the floor plane in case the floor was only visible during the initial floor search.
NUI_INITIALIZE_FLAG_USES_HEAD_POSITION	Initializes lightweight head position tracking. This flag is mutually exclusive with <code>NUI_INITIALIZE_FLAG_USES_SKELETON</code> . See NUI Head Tracking .
NUI_INITIALIZE_FLAG_USES_FITNESS	Enables use of the NUI Fitness API.
NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION	Initializes the color-based head detector and head orientation tracking. See NUI Head Tracking . If this flag is used, one of the following flags must also be used, so that one of these tracking systems is initialized; otherwise, NuiInitialize will fail: • <code>NUI_INITIALIZE_FLAG_USES_SKELETON</code> • <code>NUI_INITIALIZE_FLAG_USES_HEAD_POSITION</code> • <code>NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX</code>
NUI_INITIALIZE_FLAG_SUPPRESS_AUTOMATIC_UI	Suppresses the automatic user interface that prompts the player when there are camera-related errors. If you use this flag, you are responsible for monitoring and responding to Kinect sensor array disconnections and hardware errors. To monitor these disconnections and hardware errors, listen for the XN_SYS_NUIHARDWARESTATUSCHANGED notification, or use polling by calling XNuiGetHardwareStatus .
NUI_INITIALIZE_FLAG_NUI_GUIDE_DISABLED	Prevents Kinect from recognizing the gesture that invokes the Xbox Guide. By default, the gesture is recognized and brings up the Xbox Guide. Specifying this flag also disables the skeleton tracking engine and features that rely on that engine, including the following: • Identity features. • Recognition of the gesture that invokes the Xbox Guide. • Certain NUI API data streams. Using this flag is the only way to initialize NUI without paying the 56MB skeletal tracking cost.

Remarks

These flags are passed to [NuiInitialize](#). Combine these flags using bitwise-OR.
To initialize and enable NUI speech recognition, call [NuiSpeechEnable](#).

Example

Several of these flags are used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HRESULT Sample::Initialize()
{
    HRESULT hr;

    // Initialize NUI system
    hr = NuiInitialize(
        NUI_INITIALIZE_FLAG_USES_SKELETON |
        NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
        NUI_INITIALIZE_FLAG_USES_COLOR,
        NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD ); // Skeleton processing thread
    if (FAILED(hr))
    {
        ATG_PrintError( "Couldn't initialize NUI\n" );
        return hr;
    }

    return S_OK;
}
```

See Also

- [NUI Resource Usage](#)
- [Using Images](#)
- [Initializing NUI Image Camera](#)
- [NuiSpeechEnable](#)

NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD

[See Also Example](#)



Specifies using the default hardware thread for skeleton tracking and depth data, or the default hardware thread for speech recognition.

Name	Description
NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD	Specifies using the default hardware thread for skeleton tracking and depth data when calling NuiInitialize , or the default hardware thread for speech recognition when calling NuiSpeechEnable .

Example

This flag is used in the **NuiInitialize** call in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HRESULT Sample::Initialize()
{
    HRESULT hr;

    // Initialize NUI system
    hr = NuiInitialize(
        NUI_INITIALIZE_FLAG_USES_SKELETON |
        NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX |
        NUI_INITIALIZE_FLAG_USES_COLOR,
        NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD ); // Skeleton processing thread
    if (FAILED(hr))
    {
        ATG_PrintError( "Couldn't initialize NUI\n" );
        return hr;
    }

    return S_OK;
}
```

See Also

- [Using Images](#)
- [Initializing NUI Image Camera](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_XALLOC Flags

[See Also](#)

Recommends whether memory allocation should come from the upper or lower range of physical memory.

Name	Description
NUI_XALLOC_OBJECTTYPE_DEFAULT	Indicates that the allocation can come from either the upper or lower 256 MB of physical memory.
NUI_XALLOC_OBJECTTYPE_PREFER_UPPER_256MB	Indicates that the allocation should come from the upper 256 MB of physical memory.
NUI_XALLOC_OBJECTTYPE_PREFER_LOWER_256MB	Indicates that the allocation should come from the lower 256 MB of physical memory.

Remarks

You can use these flags to alternate allocating from upper and lower memory, to reduce stalls. Use these flags in this situation: when you override [XMemAlloc](#) and the **dwAllocatorId** field in the [XALLOC_ATTRIBUTES](#) structure is [eXALLOCAllocatorId_NuiApi](#).

For example, **XMemAlloc** has the parameter *dwAllocAttributes*, which is an XALLOC_ATTRIBUTES structure. If the **dwObjectType** field in the XALLOC_ATTRIBUTES structure has the value NUI_XALLOC_OBJECTTYPE_PREFER_UPPER_256 MB, that indicates that the allocation should come from the upper 256 MB of physical memory.

See Also

- [Using Images](#)
- [Initializing NUI Image Camera](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudio Reference

[See Also](#)

This section contains the API Reference pages for items in the header file NuiAudio.h, which performs multichannel echo cancellation for the microphone array in the Kinect Sensor, to support speech recognition. This processing is independent of the installed version of system software, unlike XHV.

In This Section

- [NuiAudio Functions](#)
- [NuiAudio Structures](#)
- [NuiAudio Callbacks](#)
- [NuiAudio Constants](#)

See Also

- [Multichannel Echo Cancellation with NuiAudio](#)
- [Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudio Functions

[See Also](#)

In This Section

- [NuiAudioCreate](#)
Initializes NuiAudio and starts streaming data through the microphone array pipeline.

[NuiAudioIsCalibrationValid](#)

Indicates whether the audio calibration file is present and valid.

[NuiAudioRegisterCallbacks](#)

Registers processing callbacks with NuiAudio.

[NuiAudioRelease](#)

Releases the NuiAudio handle and unregisters the error callback.

[NuiAudioSetMicArrayBeamAngle](#)

Overrides the beam angle of the microphone array, for the speech pipeline.

[NuiAudioUnregisterCallbacks](#)

Unregisters processing callbacks with NuiAudio.

Remarks

NuiAudioSetMicArrayBeamAngle can be called from a processing callback, because it doesn't block. The other NuiAudio functions cannot be called from NuiAudio callbacks, because they may block.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioCreate

[See Also](#)



Initializes NuiAudio and starts streaming data through the microphone array pipeline.

Syntax

C++

```
HRESULT NuiAudioCreate(  
    UINT32 uHardwareThreadRequested,  
    const PNUIAUDIO_ERROR_CALLBACK pfnErrorCallback,  
    DWORD Flags,  
    NUIAUDIO_HANDLE Handle,  
    UINT32 *pHardwareThreadUsed  
)
```

Parameters

- uHardwareThreadRequested*
[in] The Xbox hardware thread on which to run the microphone array pipeline. Must be within the range [0, 5]. If NuiAudio is already initialized, this parameter has no effect. The default value is [NUIAUDIO_DEFAULT_PROCESSOR](#).
- pfnErrorCallback*
[in] A pointer to a [PNUIAUDIO_ERROR_CALLBACK](#) function that is called when there is a critical system error which requires NuiAudio to be released and reinitialized. Must not be NULL.
- Flags*
[in] Which pipelines to initialize, as a bitwise-OR'd combination of [NuiAudio pipeline constants](#) flags.
- Handle*
[out] A handle to a singleton NuiAudio instance. The handle's value is unique to the caller.
- pHardwareThreadUsed*
[out, optional] The Xbox hardware thread on which the microphone array pipeline is running. The default is NULL.

Return Value

A COM error code. One of the following values:

Value	Description
S_OK	The function completed successfully.
E_INVALIDARG	A pointer parameter is NULL.

Remarks

Use the returned handle with subsequent calls to NuiAudio functions.
This function can be called multiple times.
NuiAudio must be initialized before callbacks can be registered. If NuiAudio is already initialized, calling this function has no effect until NuiAudio is released, other than registering the error callback.
NuiAudioCreate cannot be called from NuiAudio callbacks, because it may block.

Requirements

Header: Declared in NuiAudio.h.
Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioIsCalibrationValid

[See Also](#)



Indicates whether the audio calibration file is present and valid.

Syntax

C++

```
BOOL NuiAudioIsCalibrationValid(
    void
)
```

Parameters

Return Value

TRUE if the calibration file is present and valid. FALSE if the calibration file doesn't exist or is invalid.

Remarks

If the calibration file does not exist, or is invalid, multichannel echo cancellation will not work.
Consider prompting the user to run the [Kinect Tuner for Audio](#) when this function returns FALSE.

Requirements

Header: Declared in NuiAudio.h.
Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioRegisterCallbacks

[See Also](#)



Registers processing callbacks with NuiAudio.

Syntax

C++

```
void NuiAudioRegisterCallbacks(
    const NUIAUDIO_HANDLE Handle,
    DWORD Flags,
    const PNUIAUDIO_CALLBACK pfnProcessingCallback
)
```

Parameters

Handle
[in] A NuiAudio singleton handle, obtained from [NuiAudioCreate](#).

Flags
[in] A flag specifying which pipeline to register, as defined by [NuiAudio pipeline constants](#). You can only register a specific callback for a single version of the pipeline. This argument must be either NUIAUDIO_SPEECH_PIPELINE or NUIAUDIO_CHAT_PIPELINE; it cannot be "NUIAUDIO_SPEECH_PIPELINE | NUIAUDIO_CHAT_PIPELINE".

pfnProcessingCallback
[in] A pointer to a [PNUIAUDIO_CALLBACK](#) function to receive NuiAudio results (speech pipeline). Must not be NULL.

Return Value

None.

Remarks

Asserts on error.
This function cannot be called from NuiAudio callbacks, because it may block.

Requirements

Header: Declared in NuiAudio.h.
Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioRelease

[See Also](#)



Releases the NuiAudio handle and unregisters the error callback.

Syntax

C++

```
void NuiAudioRelease(
    const NUIAUDIO_HANDLE Handle
)
```

Parameters

Handle

[in] A NuiAudio singleton handle, obtained from [NuiAudioCreate](#).

Return Value

None.

Remarks

This function asserts on error.

When the last outstanding error callback is unregistered, the microphone array pipeline stops streaming, deallocates all its internal resources, and reverts to the uninitialized state.

The caller is responsible for ensuring all processing callbacks for the given handle are unregistered before calling this function. If NuiAudio is already uninitialized, calling this method has no effect. The caller can assume no more calls will occur to unregistered callbacks after this function returns.

This function cannot be called from NuiAudio callbacks, because it may block.

Requirements

Header: Declared in NuiAudio.h.

Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioSetMicArrayBeamAngle

[See Also](#)



Overrides the beam angle of the microphone array, for the speech pipeline.

Syntax

C++

```
void NuiAudioSetMicArrayBeamAngle(  
    const NUIAUDIO_HANDLE Handle,  
    DWORD Flags,  
    FLOAT BeamAngle  
)
```

Parameters

Handle
[in] A NuiAudio singleton handle, obtained from [NuiAudioCreate](#).

Flags
[in] Flags specifying which pipeline to lock or release, as a bitwise-OR'd combination of [NuiAudio pipeline constants](#). Currently only NUIAUDIO_SPEECH_PIPELINE is supported.

BeamAngle
[in] The beam angle of the microphone array, in radians.

An angle of zero is straight in front of the camera, positive values are to the left of the camera (from the camera's perspective) and negative values are to the right of the camera. In terms of degrees, the angle ranges from 50 degrees (to the left), to -50 degrees (to the right).

Must be within the range [NUIAUDIO_MICARRAY_BEAM_ANGLE_MIN, NUIAUDIO_MICARRAY_BEAM_ANGLE_MAX]. Or use the default setting, NUIAUDIO_MICARRAY_BEAM_ANGLE_AUTO, which causes the beam angle of the microphone array to be computed automatically. See [NUIAUDIO_MICARRAY_BEAM_ANGLE Constants](#).

Return Value

None.

Remarks

This function asserts on error.

This function can be called from a [NuiAudio callback](#), because it doesn't block.

Only the most recent call to **NuiAudioSetMicArrayBeamAngle** has an effect; the latest caller wins. This is because there's only ever one instance of speech MEC processing instantiated; NuiAudio is a singleton.

Requirements

Header: Declared in NuiAudio.h.

Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudioUnregisterCallbacks

[See Also](#)



Unregisters processing callbacks with NuiAudio.

Syntax

C++

```
void NuiAudioUnregisterCallbacks(  
    const NUIAUDIO_HANDLE Handle,  
    const PNUIAUDIO_CALLBACK pfnProcessingCallback  
)
```

Parameters

- Handle
- [in] A NuiAudio singleton handle, obtained from [NuiAudioCreate](#).
- pfnProcessingCallback
- [in] A pointer to a [PNUIAUDIO_CALLBACK](#) function to receive NuiAudio results (speech pipeline). Must not be NULL.

Return Value

None.

Remarks

This function asserts on error.

The caller can assume that no more callbacks will occur to unregistered callbacks after this function returns.

This function cannot be called from NuiAudio callbacks, because it may block.

Requirements

- Header: Declared in NuiAudio.h.
- Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudio Structures

[See Also](#)

In This Section

[NUIAUDIO_RESULTS](#)

Contains the output of a microphone array pipeline processing pass.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIAUDIO_RESULTS Structure

[See Also](#)

Contains the output of a microphone array pipeline processing pass.

Syntax

```
C++

typedef struct _NUIAUDIO_RESULTS {
    INT16 __restrict *pOutputMic;
    UINT32 Duration;
    FLOAT BeamAngle;
    FLOAT Confidence;
} NUIAUDIO_RESULTS;
```

Members

- pOutputMic**
- The NuiAudio output buffer, as a big-endian 16kHz 16-bit integer mono buffer, 16-byte aligned.
- Duration**
- The size of the NuiAudio output buffer, in frames. This value is always [NUIAUDIO_MAX_DURATION](#).
- A *frame* is a block of samples, one per channel, to be played simultaneously. For example, a mono stream has one sample per frame.
- BeamAngle**
- The most recent microphone array beam angle in radians, within the range [NUIAUDIO_MICARRAY_BEAM_ANGLE_MIN, NUIAUDIO_MICARRAY_BEAM_ANGLE_MAX] (see [NUIAUDIO_MICARRAY_BEAM_ANGLE Constants](#)).
- This member is demonstrated in the [Sound Location Tracking](#) sample.
- Confidence**
- The most recent microphone array beam angle confidence probability, within the range [0.0f, 1.0f].
- This member is only useful when [NUIAUDIO_MICARRAY_BEAM_ANGLE_AUTO](#) is in effect. When manually steering the beam, the confidence is always 100%.
- This member is demonstrated in the [Sound Location Tracking](#) sample.

Remarks

This structure is returned by a [PNUIAUDIO_CALLBACK](#) function.

Requirements

- Header: Declared in NuiAudio.h.
- Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudio Callbacks

[See Also](#)

In This Section

[PNUIAUDIO_CALLBACK](#)

Audio callback function for multichannel echo cancellation (MEC).

[PNUIAUDIO_ERROR_CALLBACK](#)

Called on a critical system error which requires NuiAudio to be released and reinitialized, such as the Kinect Sensor being unplugged.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

PNUIAUDIO_CALLBACK Callback

[See Also](#)

Audio callback function for multichannel echo cancellation (MEC).

Syntax

C++

```
typedef void (*PNUIAUDIO_CALLBACK) (
    NUIAUDIO_RESULTS *pNuiAudioResults
)
```

Parameters

pNuiAudioResults
[in] The results of multichannel echo cancellation, as a pointer to a [NUIAUDIO_RESULTS](#) structure.

Return Value

None.

Remarks

This method should not block, because it is called from the realtime processing thread.
This callback is called on the NuiAudio processing thread, which is specified in the initial call to [NuiAudioCreate](#), through the *uHardwareThreadRequested* parameter.
MEC processing and all processing callbacks must complete within 16ms. Use NuiAudio's associated PIX counters and events to verify this; see [PIX System Monitor: Predefined Counters and Events Reference](#). The most relevant PIX warning is "NuiSpeech | Glitch detected in multichannel echo cancelation."

Requirements

Header: Declared in NuiAudio.h.
Library: Use NuiAudio.lib.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

PNUIAUDIO_ERROR_CALLBACK Callback

[See Also](#) [Example](#)

Called on a critical system error which requires NuiAudio to be released and reinitialized, such as the Kinect Sensor being unplugged.

Syntax

C++

```
typedef void (*PNUIAUDIO_ERROR_CALLBACK) (
    HRESULT hr
)
```

Parameters

hr
[in] The system error code. See [System Error Codes](#).

Return Value

None.

Remarks

This callback function is called on a critical system error which requires NuiAudio to be released and reinitialized, such as the Kinect Sensor being unplugged.

Example

⊕ **Requirements**

Header: Declared in NuiAudio.h.
Library: Use NuiAudio.lib.

⊕ **See Also**

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiAudio Constants

[See Also](#)

In This Section

[NUIAUDIO_DEFAULT_PROCESSOR](#)

The default Xbox hardware thread on which to run the microphone array pipeline.

[NUIAUDIO_MAX_DURATION](#)

The maximum value of NUIAUDIO_RESULTS.Duration, which is the number of frames that can be held in the microphone array pipeline buffer.

[NUIAUDIO_MICARRAY_BEAM_ANGLE Constants](#)

The range of the microphone array beam angle.

[NUIAUDIO Pipeline Constants](#)

Specify the speech or chat pipeline.

⊕ **See Also**

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIAUDIO_DEFAULT_PROCESSOR

[See Also](#)

The default Xbox hardware thread on which to run the microphone array pipeline.

Name	Description
NUIAUDIO_DEFAULT_PROCESSOR	The default Xbox hardware thread on which to run the microphone array pipeline. This is thread 5. This overlaps the default NUI thread for skeleton tracking and depth data, which is also thread 5.

⊕ **See Also**

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIAUDIO_MAX_DURATION

[See Also](#)

The maximum value of [NUIAUDIO_RESULTS.Duration](#), which is the number of frames that can be held in the microphone array pipeline buffer.

Name	Description
NUIAUDIO_MAX_DURATION	The maximum value of the Duration member of the NUIAUDIO_RESULTS structure.

⊕ **See Also**

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIAUDIO_MICARRAY_BEAM_ANGLE Constants

[See Also](#)

The range of the microphone array beam angle.

The angle is in radians. An angle of zero is straight in front of the camera, positive values are to the left of the camera (from the camera's perspective), and negative values are to the right of the camera. In terms of degrees, the angle ranges from 50 degrees (to the left), to -50 degrees (to the right).

Name	Description
NUIAUDIO_MICARRAY_BEAM_ANGLE_AUTO	Automatically computes the beam angle, by focusing on the loudest sound detected in the playspace.
NUIAUDIO_MICARRAY_BEAM_ANGLE_MAX	The maximum angle of the range, to the left of the camera (from the camera's perspective).
NUIAUDIO_MICARRAY_BEAM_ANGLE_MIN	The minimum angle of the range, to the right of the camera (from the camera's perspective).

Remarks

These constants are used for the *BeamAngle* parameter of [NuiAudioSetMicArrayBeamAngle](#), and they define the range of the **BeamAngle** member of the [NUIAUDIO_RESULTS](#) structure.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIAUDIO Pipeline Constants

[See Also](#)

Specify the speech or chat pipeline.	
Name	Description
NUIAUDIO_SPEECH_PIPELINE	The speech pipeline for NuiAudio.
NUIAUDIO_CHAT_PIPELINE	The chat pipeline for NuiAudio.

Remarks

These flags are used for the *Flags* parameter of [NuiAudioCreate](#), [NuiAudioRegisterCallbacks](#), and [NuiAudioSetMicArrayBeamAngle](#).
See [Chat versus Speech Pipelines](#), in topic *Multichannel Echo Cancellation with NuiAudio*.

See Also

[Multichannel Echo Cancellation with NuiAudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiDiagnostics Reference

[See Also](#)

This section contains the API Reference pages for items in the header file NuiDiagnostics.h, which displays the Kinect Troubleshooter user interface in the Xbox Guide.

In This Section

[NuiDiagnostics Functions](#)

See Also

[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiDiagnostics Functions

In This Section

[XShowNuiTroubleshooterUI](#)

Displays the Kinect Troubleshooter user interface in the Xbox Guide.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XShowNuiTroubleshooterUI

[See Also](#) [Example](#)

Displays the Kinect Troubleshooter user interface in the Xbox Guide.

Syntax

C++

```
DWORD XShowNuiTroubleshooterUI ()
```

Return Value

One of the following values:

Value	Meaning
ERROR_SUCCESS	The function completed successfully.
ERROR_ACCESS_DENIED	The Xbox Guide is displayed already.
ERROR_FUNCTION_FAILED	The function failed.

Example

This function is used in the [Playspace Feedback](#) sample, as follows.

```
void Sample::UpdateInput()
{
    // Get current gamepad state
    ATG::GAMEPAD* pGamepad = ATG::Input::GetMergedInput();
    ...

    // Start shows the NUI Troubleshooter
    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_START )
    {
        // Ordinarily you'd want to check the return code here
        DWORD result = XShowNuiTroubleshooterUI();

        if ( result == ERROR_FUNCTION_FAILED )
        {
            ATG::DebugSpew( "XShowNuiTroubleshooterUI Failed" );
        }

        // ERROR_ACCESS_DENIED is unlikely in this scenario,
        // as we bring up the troubleshooter from a controller button press,
        // and the guide would have intercepted this.
    }
    ...
}
```

Requirements

Header: Declared in NuiDiagnostics.h. For include, use NuiApi.h.
Library: Use nuiapi.lib.

See Also

[Screen Displays Overview \(XShow Functions\)](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitness Reference

[See Also](#)

This section contains the API Reference pages for items in the header file NuiFitnessApi.h, which tracks the amount of energy expended, based on the movement of a tracked skeleton or on a custom algorithm.

In This Section

- [NuiFitness Functions](#)
- [NuiFitness Structures](#)
- [NuiFitness Enumerations](#)
- [NuiFitness Constants](#)

See Also

[NUI Fitness Tracking](#)
[Running Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitness Functions

[See Also](#)

In This Section

- [NuiFitnessGetBodyProfileRecord](#)
Gets the height, weight, age and gender (HWAG) data for a specified user.
- [NuiFitnessGetCurrentFitnessData](#)
Gets the specified user's accumulated fitness tracking data for the current session of using the current title.
- [NuiFitnessGetCurrentTitleSummary](#)
Get the fitness tracking data for the specified user across all the user's sessions of using the current title.
- [NuiFitnessGetOverallSummary](#)
Gets the fitness tracking data for the specified user across all the user's sessions of using all titles.
- [NuiFitnessGetPrivacySettings](#)
Returns the current privacy settings for fitness data for the specified player.
- [NuiFitnessPauseTracking](#)
Pauses automatic fitness tracking for a user.
- [NuiFitnessRecordCalorieValue](#)
For manual tracking mode, sets the current Calorie value for the specified duration.
- [NuiFitnessRecordMETValue](#)
Sets the average metabolic equivalent of task (MET) value for a specified user and duration, when using manual fitness tracking.
- [NuiFitnessResumeTracking](#)
Resumes automatic fitness tracking after [NuiFitnessPauseTracking](#) has been called.
- [NuiFitnessSetBodyProfileRecord](#)
Sets the height, weight, age and gender (HWAG) data for a specified user.
- [NuiFitnessStartTracking](#)

Starts fitness tracking for a tracked skeleton.

[NuiFitnessStopTracking](#)

Ends fitness tracking for a specified user and stores the calculated fitness tracking data into the user's profile.

[XFitnessQueryCreateEnumerator](#)

Creates an enumerator handle to get the fitness tracking data for the specified user across all the user's sessions of using the current title, optionally broken out by day, month, or year.

[XShowFitnessBodyProfileUI](#)

Displays a controller-based UI to enter the specified player's height, weight, age, and gender (HWAG) data.

[XShowFitnessPrivacySettingsUI](#)

Displays a controller-based UI for the player to change their privacy settings.

[XShowNuiFitnessBodyProfileUI](#)

Displays a Kinect-based UI to enter the specified player's height, weight, age, and gender (HWAG) data.

[XShowNuiFitnessPrivacySettingsUI](#)

Displays a Kinect-based UI for the player to change their privacy settings.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessGetBodyProfileRecord

[See Also](#)



Gets the height, weight, age and gender (HWAG) data for a specified user.

Syntax

C++

```
HRESULT NuiFitnessGetBodyProfileRecord(  
    DWORD UserIndex,  
    NUI_FITNESS_BODY_PROFILE_RECORD *pBodyProfileRecord  
)
```

Parameters

UserIndex

[in] The user to get body profile data for. Must be a valid value, 0 through XUSER_MAX_COUNT - 1.

pBodyProfileRecord

[out] The height, weight, age and gender (HWAG) data for the user, as a pointer to a [NUI_FITNESS_BODY_PROFILE_RECORD](#) structure that was allocated by the title. Must not be a null pointer, NULL, or 0. After the function call returns S_OK, the structure contains the HWAG values.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessGetBodyProfileRecord was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_ACCESSDENIED	The required privilege bit for using this function isn't set. See ID 42 in privilege .
E_INVALIDARG	The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1). Or, <i>pBodyProfileRecord</i> is a null pointer, NULL, or 0.
E_NUI_NOT_FOUND	HWAG data isn't found for the user.

Remarks

To use this function, you must obtain explicit permission from Microsoft. Contact your developer account manager (DAM) for more information. You should use this function if it makes sense for your title. Permission is required because this function is for personal information.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessGetCurrentFitnessData

[See Also](#)



Gets the specified user's accumulated fitness tracking data for the current session of using the current title.

Syntax

C++

```
HRESULT NuiFitnessGetCurrentFitnessData(  
    DWORD TrackingID,  
    NUI_FITNESS_DATA *pFitnessData  
)
```

Parameters

TrackingID

[in] The TrackingID for the tracked player. For automatic fitness tracking, this value is the ID of a currently tracked skeleton. For manual fitness tracking, this is a unique DWORD value defined by the title.

pFitnessData
[out] The accumulated fitness tracking data for the current session of using the title, as a pointer to a [NUI_FITNESS_DATA](#) structure.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessGetCurrentFitnessData was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0). Or, <i>pFitnessData</i> is a null pointer, NULL, or 0.
E_NUI_NOT_FOUND	The specified <i>TrackingID</i> is not currently being tracked.

Remarks

This function gets calculated fitness tracking data in real-time during a fitness tracking session. Call this function periodically while fitness tracking is active, to get real-time data for display.
This function must only be called between calls to [NuiFitnessStartTracking](#) and [NuiFitnessStopTracking](#).

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessGetCurrentTitleSummary

[See Also](#)



Get the fitness tracking data for the specified user across all the user's sessions of using the current title.

Syntax

C++

```
HRESULT NuiFitnessGetCurrentTitleSummary(  
    DWORD UserIndex,  
    NUI_FITNESS_TITLE_SUMMARY *pSummary  
)
```

Parameters

UserIndex
[in] The UserIndex for the user that you want to query. Must be valid.

pSummary
[out] The fitness tracking summary data, as a pointer to a title-allocated [NUI_FITNESS_TITLE_SUMMARY](#) structure. Must not be a null pointer, NULL, or 0. After the function call returns, the structure will contain the fitness tracking data.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessGetCurrentTitleSummary was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	Any of the following: - The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1). <i>pSummary</i> is a null pointer, NULL, or 0.
E_NUI_NOT_FOUND	No record is found.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessGetOverallSummary

[See Also](#)



Gets the fitness tracking data for the specified user across all the user's sessions of using all titles.

Syntax

C++

```
HRESULT NuiFitnessGetOverallSummary(  
    DWORD UserIndex,  
    NUI_FITNESS_TITLE_SUMMARY *pSummary  
)
```

Parameters

UserIndex
[in] The UserIndex for the user that you want to query. Must be valid.

pSummary
[out] The fitness summary data, as a pointer to a title-allocated [NUI_FITNESS_TITLE_SUMMARY](#) structure. Must not be a null pointer, NULL, or 0. After the function call returns, the structure will contain the summary data.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessGetOverallSummary was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	Any of the following: - The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1). <i>pSummary</i> is a null pointer, NULL, or 0.
E_NUI_NOT_FOUND	No record is found.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessGetPrivacySettings

[See Also](#)



Returns the current privacy settings for fitness data for the specified player.

Syntax

C++

```
HRESULT NuiFitnessGetPrivacySettings(  
    DWORD UserIndex,  
    NUI_FITNESS_PRIVACY_SETTINGS *pPrivacySettings  
)
```

Parameters

UserIndex
[in] The UserIndex of the player. Must be valid.

pPrivacySettings
[out] The returned privacy settings, as a [NUI_FITNESS_PRIVACY_SETTINGS](#) enumeration constant. Must not be a NULL pointer.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessGetPrivacySettings was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	<i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT-1), or <i>pPrivacySettings</i> is NULL.
E_NUI_NOT_LIVE_USER	The specified player doesn't have an Xbox LIVE account.
E_NUI_LIVE_USER_NOT_LOGGED_IN	The specified player is not logged in.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessPauseTracking

[See Also](#)



Pauses automatic fitness tracking for a user.

Syntax

C++

```
HRESULT NuiFitnessPauseTracking(  
    DWORD TrackingID  
)
```

Parameters

TrackingID
[in] The trackingID of the currently tracked skeleton to pause fitness tracking for.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessPauseTracking was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0).
E_UNEXPECTED	The specified <i>TrackingID</i> is used in manual mode.
E_NUI_NOT_FOUND	The specified <i>TrackingID</i> is not currently being tracked.

Remarks

This function is valid only in automatic tracking mode.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessRecordCalorieValue

[See Also](#)



For manual tracking mode, sets the current Calorie value for the specified duration.

Syntax

C++

```
HRESULT NuiFitnessRecordCalorieValue(  
    DWORD TrackingID,  
    FLOAT CalorieInJoules,  
    DWORD DurationInMS  
)
```

Parameters

TrackingID
[in] The tracking ID of the player.
CalorieInJoules
[in] Title calculated calorie value for the duration *DurationInMS*.
DurationInMS
[in] The duration in which the *CalorieInJoules* value is calculated, in milliseconds.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessRecordCalorieValue was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0).
E_UNEXPECTED	The specified <i>TrackingID</i> is used in automatic tracking mode.
E_NUI_NOT_FOUND	The specified <i>TrackingID</i> is not currently being tracked.

Remarks

This function is only valid in manual mode.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

NuiFitnessRecordMETValue

[See Also](#)



Sets the average metabolic equivalent of task (MET) value for a specified user and duration, when using manual fitness tracking.

Syntax

```
C++
HRESULT NuiFitnessRecordMETValue(
    DWORD TrackingID,
    FLOAT MET,
    DWORD DurationInMS
)
```

Parameters

TrackingID
[in] A unique ID representing the player. This value is title-defined, and must be in the range 0 through 7.

MET
[in] The average metabolic equivalent of task (MET) value, calculated by the title, for the specified activity interval *DurationInMS*.

DurationInMS
[in] The duration in which the *MET* value is calculated, in milliseconds. This is the duration of the interval backwards from the current time that the MET is being set for.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessRecordMETValue was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0). Or, the specified MET values is less than 0.5 or higher than 20.
E_UNEXPECTED	The specified <i>TrackingID</i> is used in automatic tracking mode.
E_NUI_NOT_FOUND	The specified <i>TrackingID</i> is not currently being tracked.

Remarks

NuiFitnessRecordMETValue must be called between [NuiFitnessStartTracking](#) and [NuiFitnessStopTracking](#), and *TrackingMode* must be NUI_FITNESS_TRACKING_MANUAL; otherwise, an error will be returned.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

NuiFitnessResumeTracking

[See Also](#)



Resumes automatic fitness tracking after [NuiFitnessPauseTracking](#) has been called.

Syntax

```
C++
HRESULT NuiFitnessResumeTracking(
    DWORD OldTrackingID,
    DWORD NewTrackingID
)
```

Parameters

OldTrackingID
[in] The tracking ID of the skeleton that was used when fitness tracking was paused.

NewTrackingID
[in] The current skeleton tracking ID for the user.

If the TrackingID didn't change, specify the same TrackingID as *OldTrackingID*. The *NewTrackingID* allows a game to handle the possibility of the tracking ID associated with a user changing while the game is paused. This can occur if the user moved out of the Kinect field of view while the game was paused and then came back into the field of view.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessResumeTracking was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>OldTrackingID</i> or <i>NewTrackingID</i> is invalid (0).
E_UNEXPECTED	The specified <i>OldTrackingID</i> or <i>NewTrackingID</i> is used in manual mode.

E_NUI_NOT_FOUND

The specified *OldTrackingID* or *NewTrackingID* is not currently being tracked.

Remarks

This function is valid only in automatic tracking mode.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessSetBodyProfileRecord

[See Also](#)

Sets the height, weight, age and gender (HWAG) data for a specified user.

Syntax

```
C++
HRESULT NuiFitnessSetBodyProfileRecord(
    DWORD UserIndex,
    NUI_FITNESS_BODY_PROFILE_RECORD BodyProfileRecord
)
```

Parameters

UserIndex
[in] The user to set data for. Must be a valid value, 0 through XUSER_MAX_COUNT - 1.
BodyProfileRecord
[in] A [NUI_FITNESS_BODY_PROFILE_RECORD](#) structure containing the height, weight, age and gender (HWAG) data of the user. The data must be within the documented range.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessSetBodyProfileRecord was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_ACCESSDENIED	The required privilege bit for using this function isn't set. See ID 42 in privilege .
E_INVALIDARG	The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1). Or, HWAG values are outside the documented range.

Remarks

To use this function, you must obtain explicit permission from Microsoft. Contact your developer account manager (DAM) for more information. You should use this function if it makes sense for your title. Permission is required because this function is for personal information.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessStartTracking

[See Also](#)

Starts fitness tracking for a tracked skeleton.

Syntax

```
C++
HRESULT NuiFitnessStartTracking(
    NUI_FITNESS_TRACKING_MODE TrackingMode,
    DWORD TrackingID,
    DWORD UserIndex
)
```

Parameters

TrackingMode
[in] A [NUI_FITNESS_TRACKING_MODE](#) value indicating whether automatic or manual fitness tracking should be performed.
TrackingID
[in] A unique ID representing the player to perform this operation on.
For automatic fitness tracking, this value must be the ID of a valid currently tracked skeleton.

For manual fitness tracking, this is a unique value defined by the title.

A maximum of 8 tracking IDs can be concurrently tracked. An error will be returned if **NuiFitnessStartTracking** is called and 8 IDs are already being tracked.

UserIndex

[in] The user index (0-3) of the player to start tracking. The player must be logged in.

The tracked Fitness data is stored into the specified user's profile. For in-memory-only Fitness tracking, specify XUSER_INDEX_NONE.

NuiFitnessStartTracking creates an internal mapping between *TrackingID* and *UserIndex*, and uses that mapping to decide which tracked user is indicated when other functions such as [NuiFitnessGetCurrentFitnessData](#) are called with a tracking ID.

Return Value

Returns S_OK on success; otherwise returns an error code:

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessStartTracking was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize . Or, the Skeleton Tracking engine is not running (was not set when calling NuiInitialize).
E_INVALIDARG	UserIndex is Invalid (greater than XUSER_MAX_COUNT and not XUSER_INDEX_NONE). Or, <i>TrackingID</i> is invalid (0).
E_NUI_LOST_TRACKING	In automatic tracking mode, the provided tracking ID doesn't exist.
E_NUI_TRACKING_ID_IN_USE	We are already tracking the provided TrackingID. That is, NuiFitnessStartTracking has already been called for the same <i>TrackingID</i> without calling NuiFitnessStopTracking .
E_NUI_TRACKED_PLAYER_LIMIT_EXCEEDED	We are already tracking 8 players.

Remarks

The title is responsible for associating the *TrackingID* with the *UserIndex* for storage and accurate height/weight/age/gender (HWAG) values. The title is also responsible for making sure that the manual trackingID doesn't conflict with the automatic trackingID.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

Remarks

NuiFitnessStartTracking must be called once for each skeleton that fitness tracking will be performed for.

To enable the NuiFitness API, the [NuiInitialize](#) function must be called with the NUI_INITIALIZE_FLAG_USES_FITNESS flag.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitnessStopTracking

[See Also](#)

Ends fitness tracking for a specified user and stores the calculated fitness tracking data into the user's profile.

Syntax

```
C++  
  
HRESULT NuiFitnessStopTracking(  
    DWORD TrackingID  
)
```

Parameters

TrackingID

[in] A unique ID representing the tracked player. For automatic fitness tracking, this value is the ID of a currently tracked skeleton. For manual fitness tracking, this is a unique DWORD value defined by the title.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before NuiFitnessStopTracking was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0).
E_NUI_NOT_FOUND	The specified <i>TrackingID</i> is not currently being tracked.

Remarks

This function stops fitness tracking after it has been started using [NuiFitnessStartTracking](#).

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

XFitnessQueryCreateEnumerator

[See Also](#) [Example](#)

Creates an enumerator handle to get the fitness tracking data for the specified user across all the user's sessions of using the current title, optionally broken out by day, month, or year.

Syntax

```
C++
HRESULT XFitnessQueryCreateEnumerator(
    DWORD UserIndex,
    const XFITNESS_QUERY_PARAMETERS *pQueryParams,
    DWORD cItems,
    PDWORD pcbBuffer,
    PHANDLE phEnum
)
```

Parameters

UserIndex
[in] The UserIndex for the user that you want to query. Must be valid.

pQueryParams
[in] The search criteria for the query, as a pointer to a [XFITNESS_QUERY_PARAMETERS](#) structure.

cItems
[in] The maximum number of items to retrieve.

pcbBuffer
[out, optional] The buffer size that is needed to contain the enumeration results, in bytes.

phEnum
[out] On successful return, a handle that can be passed to [XEnumerate](#) to iterate through fitness tracking records, which are [XFITNESS_RECORD](#) structures. Must not be a null pointer, NULL, or 0. After enumeration is complete, you should close this handle using [XCloseHandle](#).

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before XFitnessQueryCreateEnumerator was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	Any of the following: - The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1). <i>pQueryParams</i> is a null pointer, NULL, or 0. <i>phEnum</i> is a null pointer, NULL, or 0. - The date range in <i>pQueryParams</i> is not a valid range.

Remarks

To retrieve [XFITNESS_RECORD](#) structures, call **XFitnessQueryCreateEnumerator**, which returns a handle *phEnum*. Pass the handle to [XEnumerate](#) to iterate through fitness tracking records, as XFITNESS_RECORD structures.

Example

```
XFITNESS_QUERY_PARAMETERS query = {0};

// Specify the granularity of the search.
query.QueryRollupValues = XFITNESS_QUERY_ROLLUP_PER_MONTH;

// Set the user index that we want to query.
query.UserIndexToQuery = m_dwUserIndex;

HANDLE hEnum = 0;
DWORD dwBufferSize = 0;

// Create the Query handler.
hr = XFitnessQueryCreateEnumerator(m_dwUserIndex, // The user doing the search.
    &query,
    10, // Get 10 records per enumeration.
    &dwBufferSize,
    &hEnum);

if (SUCCEEDED(hr) && (nullptr != hEnum))
{
    if (dwBufferSize > sizeof(XFITNESS_RECORD))
    {
        DWORD dwError = 0;
        DWORD dwNumOfRecords = dwBufferSize/sizeof(XFITNESS_RECORD);
        XFITNESS_RECORD* pRecords = new XFITNESS_RECORD[dwNumOfRecords];

        // Get the data into *pRecords.
        dwError = XEnumerate(hEnum,
            (void*)pRecords, // Cast XFITNESS_RECORD* to void*.
            dwNumOfRecords * sizeof(XFITNESS_RECORD),
            &dwNumOfRecords,
            nullptr);

        if (0 == dwError)
        {
            m_dwRecordsCount = dwNumOfRecords;
        }

        delete[] pRecords;
    }

    XCloseHandle(hEnum);
}
```

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiFitnessApi.lib.

✚ See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XShowFitnessBodyProfileUI

[See Also](#)



Displays a controller-based UI to enter the specified player's height, weight, age, and gender (HWAG) data.

✚ Syntax

C++

```
HRESULT XShowFitnessBodyProfileUI(  
    DWORD UserIndex  
)
```

Parameters

UserIndex
[in] The user index (0-3) of the user whose HWAG data is being updated.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before XShowFitnessBodyProfileUI was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1).

✚ Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

✚ See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XShowFitnessPrivacySettingsUI

[See Also](#)



Displays a controller-based UI for the player to change their privacy settings.

✚ Syntax

C++

```
HRESULT XShowFitnessPrivacySettingsUI(  
    DWORD UserIndex  
)
```

Parameters

UserIndex
[in] The user index of the player. Must be valid.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before XShowFitnessPrivacySettingsUI was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1).

✚ Remarks

This function is meant to be called only when the user's profile is Xbox LIVE-enabled. The recommended usage is to call [NuiFitnessGetPrivacySettings](#) first, then check to see if the profile is Xbox LIVE-enabled and is signed in to Xbox LIVE, by checking whether that function call returns S_OK. If that function returns S_OK, it is appropriate to call **XShowFitnessPrivacySettingsUI**.

✚ Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiFitnessApi.lib.

✚ See Also

[NuiFitness Functions](#)

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XShowNuiFitnessBodyProfileUI

[See Also](#)



Displays a Kinect-based UI to enter the specified player's height, weight, age, and gender (HWAG) data.

Syntax

C++

```
HRESULT XShowNuiFitnessBodyProfileUI(  
    DWORD TrackingID,  
    DWORD UserIndex  
)
```

Parameters

TrackingID
[in] The skeleton TrackingID of the player who will use the UI. Must be valid.

UserIndex
[in] The user index (0-3) of the user whose HWAG data is being updated.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before XShowNuiFitnessBodyProfileUI was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The specified <i>TrackingID</i> is invalid (0). Or, the specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1).

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XShowNuiFitnessPrivacySettingsUI

[See Also](#)



Displays a Kinect-based UI for the player to change their privacy settings.

Syntax

C++

```
HRESULT XShowNuiFitnessPrivacySettingsUI(  
    DWORD TrackingID,  
    DWORD UserIndex  
)
```

Parameters

TrackingID
[in] The tracking ID of the player whose data will be updated.

UserIndex
[in] The TrackingID of the skeleton that will be in control of the UI. Must be valid.

Return Value

Returns S_OK on success; otherwise returns one of the following error codes.

Error Code	Meaning
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called before XShowNuiFitnessPrivacySettingsUI was called. Or, the flag NUI_INITIALIZE_FLAG_USES_FITNESS wasn't passed into NuiInitialize .
E_INVALIDARG	The trackingID is invalid (0), or the specified <i>UserIndex</i> is invalid (greater than XUSER_MAX_COUNT - 1).

Remarks

This function is meant to be called only when the user's profile is Xbox LIVE-enabled. The recommended usage is to call [NuiFitnessGetPrivacySettings](#) first, then check to see if the profile is Xbox LIVE-enabled and is signed in to Xbox LIVE, by checking whether that function call returns S_OK. If that function returns S_OK, it is appropriate to call **XShowNuiFitnessPrivacySettingsUI**.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiFitnessApi.lib.

See Also

[NuiFitness Functions](#)
[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitness Structures

[See Also](#)



In This Section

[NUI_FITNESS_BODY_PROFILE_RECORD](#)

Contains the body profile data for a user.

[NUI_FITNESS_DATA](#)

Contains fitness tracking data: the duration of activity, average metabolic equivalent of task (MET), and amount of energy expended. Used for a player in a session of using a title.

[NUI_FITNESS_TITLE_SUMMARY](#)

Holds the accumulated fitness tracking data.

[XFITNESS_QUERY_PARAMETERS](#)

Contains fitness query parameters.

[XFITNESS_RECORD](#)

Contains a fitness record returned from the Query API, indicating the amount of energy used during a duration by a player for a title.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_BODY_PROFILE_RECORD Structure

[See Also](#)



Contains the body profile data for a user.

Syntax

C++

```
typedef struct _NUI_FITNESS_BODY_PROFILE_RECORD {  
    FLOAT Height;  
    FLOAT Weight;  
    USHORT YearOfBirth;  
    NUI_GENDER_VALUE Gender;  
} NUI_FITNESS_BODY_PROFILE_RECORD;
```

Members

Height

The user's height in meters.

Weight

The user's weight in kilograms.

YearOfBirth

The year in which the user was born. Must be greater than 1900.

Gender

The gender of the user, as a [NUI_GENDER_VALUE](#) enumeration constant.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_DATA Structure

[See Also](#)



Contains fitness tracking data: the duration of activity, average metabolic equivalent of task (MET), and amount of energy expended. Used for a player in a session of using a title.

Syntax

C++

```
typedef struct _NUI_FITNESS_DATA {  
    DWORD DurationInMS;  
    FLOAT AverageMET;  
    DWORD Joules;  
} NUI_FITNESS_DATA;
```

Members

DurationInMS

The currently accumulated duration of the activity, in milliseconds.

AverageMET

The current average metabolic equivalent of task (MET) during the *DurationInMS*. This value indicates the physical intensity of the activity. When the title is using calorie-based manual input, this value is 0.

Joules

The accumulated energy expended during *DurationInMS*, in joules.

Energy expended is an alternative measure of physical exertion that is calculated without using height, weight, age and gender (HWAG). The energy expended can be used to compare the physical exertion of players without having to adjust for players' different HWAG values.

Remarks

This structure is returned by [NuiFitnessGetCurrentFitnessData](#) and can be returned by [NuiFitnessStopTracking](#).

Conversion between Joules and Calories

Use the following as the standard conversions for NUI Fitness, for food calories:

1 joule = 0.239005736 calories

1 calorie = 4.184 joules, or 4 23/125 joules

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_TITLE_SUMMARY Structure

[See Also](#)



Holds the accumulated fitness tracking data.

Syntax

C++

```
typedef struct _NUI_FITNESS_TITLE_SUMMARY {  
    SYSTEMTIME LatestTimeStamp;  
    DWORD TotalDuration;  
    DWORD TotalJoules;  
    FLOAT AverageMET;  
} NUI_FITNESS_TITLE_SUMMARY;
```

Members

LatestTimeStamp

The time at which the most recent fitness tracking data in this **Duration** was recorded.

TotalDuration

The total duration in which the fitness tracking data was collected, in seconds.

TotalJoules

The total energy burned, in joules, for the **Duration**. To convert this value to food calories, see the conversion formulas in [NUI_FITNESS_DATA](#).

AverageMET

The weighted average metabolic equivalent of task (MET), for the **Duration**.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.

Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XFITNESS_QUERY_PARAMETERS Structure

[See Also](#)



Contains fitness query parameters.

Syntax

C++

```
typedef struct _XFITNESS_QUERY_PARAMETERS {  
    DWORD UserIndexToQuery;  
    XUID OnlineUserIDToQuery;  
    SYSTEMTIME StartDate;  
    SYSTEMTIME EndDate;  
    XFITNESS_QUERY_ROLLUP_VALUES QueryRollupValues;  
} XFITNESS_QUERY_PARAMETERS;
```

Members

UserIndexToQuery

The user index of the player to query. Specifying a value for this member searches against the locally logged-in users. This member is mutually exclusive with **OnlineUserIDToQuery**.

OnlineUserIDToQuery
The online user ID of the user to query. Specifying a value for this member searches against the users in Xbox LIVE. This member is mutually exclusive with **UserIndexToQuery**.

StartDate
If this member is empty, the default start date is used, which is the date of the first record that is stored locally.

EndDate
If this member is empty, the default end date is used, which is the current date.

QueryRollupValues
The query rollup values, as a [XFITNESS_QUERY_ROLLUP_VALUES](#) enumeration constant.

✚ Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

✚ See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XFITNESS_RECORD Structure

[See Also](#)



Contains a fitness record returned from the Query API, indicating the amount of energy used during a duration by a player for a title.

✚ Syntax

C++

```
typedef struct _XFITNESS_RECORD {
    SYSTEMTIME TimeStamp;
    DWORD Duration;
    DWORD TitleID;
    FLOAT AverageMET;
    DWORD Joules;
    USHORT Reserved;
} XFITNESS_RECORD;
```

Members

TimeStamp
The time at which the record data started.

Duration
The duration covered by the record data, in seconds.

TitleID
The ID of the title for which the record is used.

AverageMET
The average metabolic equivalent of task (MET) for the **Duration**. For fitness records that were generated from calorie-based input, this field is 0.

Joules
The amount of energy burned during the **Duration**, in joules. To convert this value to food calories, see the conversion formulas in [NUI FITNESS DATA](#).

Reserved
Reserved for future use.

✚ Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

✚ See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitness Enumerations

[See Also](#)



In This Section

- [NUI_FITNESS_PRIVACY_SETTINGS](#)
Specifies privacy levels for fitness tracking data.
- [NUI_FITNESS_TRACKING_MODE](#)
Specifies automatic or manual mode for fitness tracking.
- [NUI_GENDER_VALUE](#)
Specifies a player's gender.
- [XFITNESS_QUERY_ROLLUP_VALUES](#)
Specifies the granularity of the query.

✚ See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.

Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_PRIVACY_SETTINGS Enumeration

[See Also](#)



Specifies privacy levels for fitness tracking data.

Syntax

C++

```
typedef enum _NUI_FITNESS_PRIVACY_SETTINGS
{
    NUI_FITNESS_PRIVACY_SETTINGS_UNKNOWN,
    NUI_FITNESS_PRIVACY_SETTINGS_NOTSET,
    NUI_FITNESS_PRIVACY_SETTINGS_LOCALONLY,
    NUI_FITNESS_PRIVACY_SETTINGS_BLOCKED,
    NUI_FITNESS_PRIVACY_SETTINGS_FRIENDS,
    NUI_FITNESS_PRIVACY_SETTINGS_EVERYONE
} NUI_FITNESS_PRIVACY_SETTINGS;
```

Constants

Constant	Description
NUI_FITNESS_PRIVACY_SETTINGS_UNKNOWN	The privacy level for fitness data is unknown.
NUI_FITNESS_PRIVACY_SETTINGS_NOTSET	The privacy level for fitness data is not set.
NUI_FITNESS_PRIVACY_SETTINGS_LOCALONLY	Fitness data is visible on the local Xbox console only.
NUI_FITNESS_PRIVACY_SETTINGS_BLOCKED	The visibility of fitness data is blocked.
NUI_FITNESS_PRIVACY_SETTINGS_FRIENDS	Fitness data is visible to friends.
NUI_FITNESS_PRIVACY_SETTINGS_EVERYONE	Fitness data is visible to everyone.

Remarks

This enumeration is returned by [NuiFitnessGetPrivacySettings](#).

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_TRACKING_MODE Enumeration

[See Also](#)



Specifies automatic or manual mode for fitness tracking.

Syntax

C++

```
typedef enum _NUI_FITNESS_TRACKING_MODE
{
    NUI_FITNESS_TRACKING_AUTO = 0,
    NUI_FITNESS_TRACKING_MANUAL
} NUI_FITNESS_TRACKING_MODE;
```

Constants

Constant	Description
NUI_FITNESS_TRACKING_AUTO	Automatic fitness tracking mode. In automatic mode, the NuiFitness library calculates the amount of energy expended and the metabolic equivalent of task (MET) value based on skeleton movement along with internal equations in the NuiFitness library.
NUI_FITNESS_TRACKING_MANUAL	Manual fitness tracking mode. In manual mode, the title calculates the amount of energy expended and the metabolic equivalent of task (MET) value, using an approach and equations defined by the title.

Remarks

This enumeration is passed into [NuiFitnessStartTracking](#).

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_GENDER_VALUE Enumeration

[See Also](#)



Specifies a player's gender.

Syntax

C++

```
typedef enum _NUI_GENDER_VALUE
{
    NUI_GENDER_MALE,
    NUI_GENDER_FEMALE,
    NUI_GENDER_UNKNOWN
} NUI_GENDER_VALUE;
```

Constants

Constant	Description
NUI_GENDER_MALE	Player is male.
NUI_GENDER_FEMALE	Player is female.
NUI_GENDER_UNKNOWN	Player's gender is unknown.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XFITNESS_QUERY_ROLLUP_VALUES Enumeration

[See Also](#)



Specifies the granularity of the query.

Syntax

C++

```
typedef enum _XFITNESS_QUERY_ROLLUP_VALUES
{
    XFITNESS_QUERY_ROLLUP_PER_RECORD,
    XFITNESS_QUERY_ROLLUP_PER_DAY,
    XFITNESS_QUERY_ROLLUP_PER_MONTH,
    XFITNESS_QUERY_ROLLUP_PER_YEAR,
    XFITNESS_QUERY_ROLLUP_TOTAL
} XFITNESS_QUERY_ROLLUP_VALUES;
```

Constants

Constant	Description
XFITNESS_QUERY_ROLLUP_PER_RECORD	Raw data, not valid for online search.
XFITNESS_QUERY_ROLLUP_PER_DAY	Accumulate data per day.
XFITNESS_QUERY_ROLLUP_PER_MONTH	Accumulate data per month.
XFITNESS_QUERY_ROLLUP_PER_YEAR	Accumulate data per year.
XFITNESS_QUERY_ROLLUP_TOTAL	Accumulate all data.

Requirements

Header: Declared in NuiFitnessApi.h. For include, use NuiApi.h.
Library: Use NuiApi.lib.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiFitness Constants

[See Also](#)



In This Section

[NUI_FITNESS_MAX_TRACKED_COUNT](#)

The maximum number of players that can be tracked for fitness calculation.

See Also

[NUI Fitness Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FITNESS_MAX_TRACKED_COUNT

[See Also](#)



The maximum number of players that can be tracked for fitness calculation.

Name	Description
NUI_FITNESS_MAX_TRACKED_COUNT	The maximum number of players that can be tracked for fitness calculation.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGesture Reference

[See Also](#)



This section contains the API reference pages for items in the header file NuiGesture.h.
This API enables title developers to perform run-time gesture detection by using the data generated with the Visual Gesture Builder tool.

In This Section

[NuiGesture Functions](#)

This section contains the functions reference pages.

[NUI_GESTURE_FRAME](#)

The output of the gesture detector for a single frame.

[NuiGesture Constants](#)

This section contains the constants reference pages.

The following are some examples of how to use the [NuiGesture](#) API.

```
// Storing results of NuiGesture
NUI_GESTURE_FRAME    m_GestureFrame;
KickBoxing::FRAME    m_KickBoxingGestureFrame;

// Load the gesture data file and enable gestures
RETURN_ON_FAIL( NuiGestureInitialize( "game:\\Media\\Gestures\\KickBoxing.gbd", KickBoxing::DIGEST, 0, NULL ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_PUNCH_LEFT, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_PUNCH_RIGHT, TRUE ) );

// Initialize the NuiGesture frame that will receive data from NuiGesture APIs
INITIALIZE_NUI_GESTURE_FRAME( &m_KickBoxingGestureFrame, &m_GestureFrame );

if ( SUCCEEDED( NuiGestureGetNextFrame( 0, &m_GestureFrame ) ) )
{
    // Do something interesting with the gesture detection results
}
```

Remarks

NuiApi.h includes most of the NUI header files. Do not directly include the NUI header files that NuiApi.h already includes. For more information, see [Adding NUI Header and Library Files](#).

See Also

[Visual Gesture Builder](#)

[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGesture Functions

[See Also](#)



In This Section

[NuiGestureEnableGesture](#)

Enables or disables a gesture.

[NuiGestureGetNextFrame](#)

Gets the next frame from a gesture detector stream.

[NuiGestureInitialize](#)

Loads database in memory and initializes the gesture detector.

[NuiGestureInitializeFromMemory](#)

Loads database from memory and initializes the gesture detector.

[NuiGestureSetActiveSkeletons](#)

Sets which skeletons are being actively recognized by the gesture detector.

[NuiGestureSetPreprocessFlags](#)

Sets the preprocess to input skeleton before detection.

[NuiGestureShutdown](#)

Unloads the database and shuts down the gesture detector.

See Also

[Visual Gesture Builder](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGestureEnableGesture

[See Also](#)



Enables or disables a gesture.

Syntax

C++

```
HRESULT NuiGestureEnableGesture(  
    DWORD GestureID,  
    BOOL Enable  
)
```

Parameters

GestureID

[in] The identifier of the gesture for which to set the status.

Note

This identifier is defined in the header file generated, along with the database, by the [Visual Gesture Builder](#).

Enable

[in] **true** to enable the gesture; otherwise, **false**.

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_FAIL	The status cannot be set because the GestureID is invalid.
E_NUI_GESTURE_NOT_INITIALIZED	The gesture detector is not initialized.

Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and St.lib.

See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGestureGetNextFrame

[See Also](#)



Gets the next frame from a gesture detector stream.

Syntax

C++

```
HRESULT NuiGestureGetNextFrame(  
    DWORD MillisecondsToWait,  
    NUI_GESTURE_FRAME *pFrame  
)
```

Parameters

MillisecondsToWait

[in] Sets the timeout in milliseconds to wait for a frame.

pFrame

[in, out] Pointer to the retrieved frame data.

Note

This is a pointer to the title allocated buffer which contains the per frame data. Before calling this function, in [NUI_GESTURE_FRAME](#), you must set the **Signature** and the **DataSize**, and allocate memory for **pData** with **DataSize**.

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_INVALIDARG	The Signature is not matching the gesture detector's output structure.
E_NUI_DEVICE_NOT_READY	The Kinect sensor array is not ready.
E_NUI_FRAME_NO_DATA	No new frame is available yet.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_FRAME_LIMIT_EXCEEDED	Too many frames have been requested and not released.
E_PENDING	An asynchronous operation is pending.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Stream data is not available to titles while the Guide is active.
E_NUI_GESTURE_NOT_INITIALIZED	The gesture detector is not initialized.

Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and St.lib.

See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGestureInitialize

[See Also](#)



Loads database in memory and initializes the gesture detector.

Syntax

C++

```
HRESULT NuiGestureInitialize(  
    const CHAR *GestureDatabasePath,  
    const DWORD GestureDatabaseDigest[NUI_GESTURE_DIGEST_DWORD_COUNT],  
    DWORD Flags,  
    HANDLE NextFrameEvent  
)
```

Parameters

GestureDatabasePath
Path to the gesture database.
GestureDatabaseDigest
[in] The database digest.
Flags
[in] Flags controlling the gesture detector's stream.
NextFrameEvent
[in, optional] The handle to a manual reset event for signaling the availability of the next frame.

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_INVALIDARG	Flags parameter is invalid.
E_FAIL	The operation failed to open the database.
E_OUTOFMEMORY	The operation failed because there is not enough memory.
E_NUI_FEATURE_NOT_INITIALIZED	The Skeleton stream is not enabled. Note This return value applies only to cases where the skeleton is not initialized in NuiInitialize .
E_NUI_GESTURE_ALREADY_INITIALIZED	The gesture detector is already initialized.

Remarks

The [NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA](#) is the only value allowed for the **Flags** parameter. The value of zero indicates that no value is defined for the **Flags**.
The [NUI_GESTURE_DIGEST_DWORD_COUNT](#) is the size of the array that contains the gesture database digest used for securty checks. The array is in the auto generated header file associated with the database.

Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and St.lib.

See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.

NuiGestureInitializeFromMemory

[See Also](#)



Loads database from memory and initializes the gesture detector.

Syntax

```
C++

HRESULT NuiGestureInitializeFromMemory(
    const VOID *pGestureDatabase,
    DWORD GestureDatabaseSize,
    const DWORD GestureDatabaseDigest[NUI_GESTURE_DIGEST_DWORD_COUNT],
    DWORD Flags,
    HANDLE NextFrameEvent
)
```

Parameters

- pGestureDatabase*
[in] Pointer to the database in memory.
- GestureDatabaseSize*
[in] Size of the database in bytes.
- GestureDatabaseDigest*
[in] The database digest.
- Flags*
[in] Flags controlling the gesture detector's stream.
- NextFrameEvent*
[in, optional] The handle to a manual reset event for signaling the availability of the next frame.

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_INVALIDARG	Flags parameter is invalid.
E_FAIL	The operation failed to open the database.
E_OUTOFMEMORY	The operation failed because there is not enough memory.
E_NUI_FEATURE_NOT_INITIALIZED	The Skeleton stream is not enabled. Note This return value applies only to cases where the skeleton is not initialized in NuiInitialize.
E_NUI_GESTURE_ALREADY_INITIALIZED	The gesture detector is already initialized.

Remarks

The [NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA](#) is the only value allowed for the **Flags** parameter. The value of zero indicates that no value is defined for the **Flags**. The [NUI_GESTURE_DIGEST_DWORD_COUNT](#) is the size of the array that contains the gesture database digest used for security checks. The array is in the auto generated header file associated with the database.

Requirements

- Header:** Declared in NuiGesture.h. For include, use NuiApi.h.
- Library:** Use NuiApi.lib and St.lib.

See Also

[NuiGesture Functions](#)

NuiGestureSetActiveSkeletons

[See Also](#)



Sets which skeletons are being actively recognized by the gesture detector.

Syntax

```
C++

HRESULT NuiGestureSetActiveSkeletons(
    DWORD TrackingIDs[NUI_SKELETON_MAX_TRACKED_COUNT]
)
```

Parameters

- TrackingIDs*
[in, optional] An array of the tracking identifiers of the skeletons to be recognized.
- Note

Set the element in the array to zero to disable the detector on that skeleton. Set this parameter to **null** to use the default tracking identifiers.

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_NUI_GESTURE_NOT_INITIALIZED	The gesture detector is not initialized.

⊕ Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and St.lib.

⊕ See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGestureSetPreprocessFlags

[See Also](#)



Sets the preprocess to input skeleton before detection.

⊕ Syntax

```
C++
HRESULT NuiGestureSetPreprocessFlags (
    DWORD Flags
)
```

Parameters

Flags
[in] The preprocess flags.
 **Note**
You can assign to this parameter the value obtained by performing an OR operation on the [gesture preprocessing constants](#).

Return Value

One of the following values:

Value	Meaning
S_OK	The operation completed successfully.
E_INVALIDARG	The flags parameter is invalid.
E_NUI_GESTURE_NOT_INITIALIZED	The gesture detector is not initialized.

⊕ Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and St.lib.

⊕ See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGestureShutdown

[See Also](#)



Unloads the database and shuts down the gesture detector.

⊕ Syntax

```
C++
void NuiGestureShutdown ()
```

Return Value

None.

⊕ Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and St.lib.

⊕ See Also

[NuiGesture Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_GESTURE_FRAME Structure

[See Also](#)



The output of the gesture detector for a single frame.

Syntax

C++

```
typedef struct _NUI_GESTURE_FRAME {
    LARGE_INTEGER TimeStamp;
    DWORD FrameNumber;
    DWORD Signature;
    PVOID pData;
    DWORD DataSize;
} NUI_GESTURE_FRAME;
```

Members

TimeStamp

The timestamp of the depth frame being recognized.

FrameNumber

The frame number of the depth frame that is used to generate this data.

Signature

The signature of pData.

pData

The buffer allocated by the title for the gesture detector's output.

DataSize

The size of pData in bytes.

The following code example shows how you can allocate memory for pData in your title.

```
// Allocate memory for a NuiGesture frame
NUI_GESTURE_FRAME m_GestureFrame;

// Allocate memory for pData inside the NuiGesture frame
KickBoxing::FRAME m_KickBoxingGestureFrame;

// Setup the pointer to pData inside the NuiGesture frame
INITIALIZE_NUI_GESTURE_FRAME( &m_KickBoxingGestureFrame, &m_GestureFrame );
```

Requirements

Header: Declared in NuiGesture.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and St.lib.

See Also

[Visual Gesture Builder](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiGesture Constants

[See Also](#)



In This Section

[NUI_GESTURE_DIGEST_DWORD_COUNT](#)

Represents the size of the array that contains the gesture database digest.

[NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA](#)

Controls the gesture detector's stream.

[Gesture Preprocessing Constants](#)

Defines the gesture preprocessing constants.

See Also

[Visual Gesture Builder](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_GESTURE_DIGEST_DWORD_COUNT

[See Also](#)



Name	Description
NUI_GESTURE_DIGEST_DWORD_COUNT	The size of the array that contains the gesture database digest.

Remarks

The gesture database digest is used for security checks. The array is in the auto generated header file associated with the database.

See Also

[NuiGesture Constants](#)
[NuiGestureInitialize](#)

[NuiGestureInitializeFromMemory](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA

[See Also](#)



Name	Description
NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA	Suppresses E_NUI_FRAME_NO_DATA errors.

Remarks

The NUI_GESTURE_FLAG_SUPPRESS_NO_FRAME_DATA is the only value allowed for the **Flags** parameter in the related function calls. The value of zero indicates that no value is defined for the **Flags**.

See Also

- [NuiGesture_Constants](#)
- [NuiGestureInitialize](#)
- [NuiGestureInitializeFromMemory](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_GESTURE_PREPROCESS Flags

[See Also](#)



Name	Description
NUI_GESTURE_PREPROCESS_NONE	Indicates that gesture preprocessing is not performed.
NUI_GESTURE_PREPROCESS_MIRROR_LEFT_RIGHT	Mirrors the skeleton left to right.  Note This flag indicates that NuiGesture APIs will mirror the input NuiSkeleton data before gesture detection performs detection. This is for scenarios where a title has a right-handed gesture database and needs to enable a left-handed gesture. Once this flag is specified, a left-handed skeleton will be mirrored to right-handed. Then it can be recognized by a right-handed gesture. This saves on training and memory cost that would be caused by generating data for both left-handed and right-handed gestures.
NUI_GESTURE_PREPROCESS_MAKE_SEATED	Converts the skeleton to seated skeleton.

See Also

- [NuiGesture_Constants](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandles Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiHandles.h, which enables using hand movements to manipulate user interface controls such as buttons, scroll bars, and menus.

In This Section

- [NuiHandles_Functions](#)
- [NuiHandles_Structures](#)
- [NuiHandles_Enumerations](#)
- [NuiHandles_Constants](#)

See Also

- [NUI_Handles Overview](#)
- [Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandles Functions

[See Also](#)



In This Section

[NuiHandlesArmGetEnabled](#)

Gets whether the specified arm is enabled.

[NuiHandlesArmGetPositionRay](#)

Gets the position of the specified arm in body space, as a vector.

[NuiHandlesArmGetScreenSpaceLocation](#)

Gets the location of the specified arm, in screen space coordinates.

[NuiHandlesArmGetSensorToPhizTransformationMatrix](#)

Gets a transformation matrix that transforms from sensor coordinates to PHIZ coordinates, for the specified arm.

[NuiHandlesArmGetTwoHandedMode](#)

Gets whether unified PHIZ is used for the specified arms.

[NuiHandlesArmSetEnabled](#)

Sets whether the specified arm is enabled.

[NuiHandlesArmSetPositionRay](#)

Sets the position of the specified arm in body space.

[NuiHandlesArmSetTwoHandedMode](#)

Sets whether unified PHIZ is used for the specified arms.

[NuiHandlesArmsGetApproachInfo](#)

Gets information about the specified arm's approach to the nearest grip.

[NuiHandlesArmsGetArmStatus](#)

Gets the level of confidence in the positions of the specified arms.

[NuiHandlesArmsGetBodySpace](#)

Gets the body space coordinate system for the specified arms, relative to camera space.

[NuiHandlesArmsGetDefaultTuningParams](#)

Gets the default tuning parameters for arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

[NuiHandlesArmsGetPlayerInfo](#)

Gets identity information about the player associated with the specified arms, including tracking ID and user index.

[NuiHandlesArmsGetTuningParams](#)

Gets the tuning parameters for the specified arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

[NuiHandlesArmsInit](#)

Initializes an arms structure.

[NuiHandlesArmsIsAttached](#)

Gets whether the specified arm is attached to a handle.

[NuiHandlesArmsSetTuningParams](#)

Sets the tuning parameters for the specified arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

[NuiHandlesArmsUpdate](#)

Updates the state of the specified arms, based on the passed-in data from the Kinect sensor array.

[NuiHandlesArmsUpdateRailProgress](#)

Updates the states of all handles currently attached to the specified arms, based on the most recently updated arm positions.

[NuiHandlesGlobalGetDefaultTuningParams](#)

Gets the default global tuning parameters for handles, to retrieve the width and height of the physical interaction zone (PHIZ).

[NuiHandlesGlobalGetTuningParams](#)

Gets the global tuning parameters for handles, to retrieve the width and height of the physical interaction zone (PHIZ).

[NuiHandlesGlobalSetTuningParams](#)

Sets the global tuning parameters for handles, to customize the width and height of the physical interaction zone (PHIZ).

[NuiHandlesGripAddRailAnalogStick](#)

Adds a new analog joystick rail to a grip.

[NuiHandlesGripAddRailArc](#)

Adds a new arc rail to a grip.

[NuiHandlesGripAddRailGearShift](#)

Adds a new gearshift rail to a grip.

[NuiHandlesGripAddRailHoverTimer](#)

Adds a hover timer to a grip.

[NuiHandlesGripForceDetach](#)

Detaches the specified grip from user input.

[NuiHandlesGripGetApproachInfo](#)

Gets the approach information about the nearest unattached arm.

[NuiHandlesGripGetCommittedRailId](#)

Gets the rail identifier of the rail to which the player has committed, if any.

[NuiHandlesGripGetDefaultTuningParams](#)

Gets the default tuning parameters for grips, which define how long an arm needs to be in an attach or detach zone before the arm attaches to or detaches from the specified grip.

[NuiHandlesGripGetEnabled](#)

Determines whether the specified grip is enabled, which determines whether the grip and its rails can be manipulated by the player.

[NuiHandlesGripGetHandedness](#)

Gets the handedness of the specified grip.

[NuiHandlesGripGetIsAttached](#)

Gets whether the specified grip is attached.

[NuiHandlesGripGetLastAttachPlayerInfo](#)

Gets identity information about the last player to attach to the specified grip.

[NuiHandlesGripGetNumRails](#)

Gets the number of rails that are on the specified grip.

[NuiHandlesGripGetPositionRay](#)

Gets the position ray for the specified grip, in body space coordinates.

[NuiHandlesGripGetRailEnabled](#)

Gets whether the specified rail is enabled.

[NuiHandlesGripGetRailState](#)

Gets whether the specified rail is enabled and committed, and gets the player's position along the rail.

[NuiHandlesGripGetScreenSpaceLocation](#)

Gets the persistent screen space location of the specified grip.

[NuiHandlesGripGetShape](#)

Gets the size of the attach, approach, and detach zones of the specified grip.

[NuiHandlesGripGetTuningParams](#)

Gets how long an arm needs to be in an attach or detach zone before the arm attaches to or detaches from the specified grip.

[NuiHandlesGripGetUserContext](#)

Gets the user context pointer of the specified grip, to store title-specific information associated with the grip.

[NuiHandlesGripInit](#)

Initializes the specified grip.

[NuiHandlesGripRailGetPersistentProgress](#)

Indicates the progress to which the specified arc rail or gearshift rail is set initially when the player attaches to the rail.

[NuiHandlesGripRailSetPersistentProgress](#)

Sets the progress that the specified arc rail or gearshift rail will have initially when the player attaches to the rail.

[NuiHandlesGripSetEnabled](#)

Enables or disables the specified grip, to control whether the grip and its rails can be manipulated by the player.

[NuiHandlesGripSetHandedness](#)

Sets whether the handedness of the specified grip is left, right, or ambidextrous.

[NuiHandlesGripSetPositionRay](#)

Sets the position of the specified grip in screen space.

[NuiHandlesGripSetRailEnabled](#)

Enables or disables the specified rail.

[NuiHandlesGripSetScreenSpaceLocation](#)

Sets the screen space location of the specified grip.

[NuiHandlesGripSetShape](#)

Sets the size of the attach, approach, and detach zones of the specified grip.

[NuiHandlesGripSetTuningParams](#)

Sets how long an arm needs to be in an attach zone or a detach zone before the arm attaches to or detaches from the specified grip.

[NuiHandlesGripSetUserContext](#)

Sets the user context pointer for the specified grip, to store title-specific information associated with the grip.

[NuiHandlesGripUninit](#)

Uninitializes the specified grip.

[NuiHandlesGripUpdate](#)

Updates the specified grip to reflect the current arm positions.

✚ See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmGetEnabled

[See Also](#)



Gets whether the specified arm is enabled.

✚ Syntax

C++

```
BOOL NuiHandlesArmGetEnabled(  
    PCNUI_HANDLES_ARMS pArms,  
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness  
)
```

Parameters

pArms

[in] The arms structure to query, as a pointer to the [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

eHandedness

[in] The arm to query (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.

Return Value

TRUE if the arm is enabled; otherwise, FALSE. Returns NUI_HANDLES_E_INVALID_ARG if *pArms* is NULL or *eHandedness* is not a valid enumeration constant.

✚ Remarks

To get the current value, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

✚ Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

✚ See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmGetPositionRay

[See Also](#)



Gets the position of the specified arm in body space, as a vector.

✚ Syntax

C++

```
XMFLOAT3 NuiHandlesArmGetPositionRay(  
    PCNUI_HANDLES_ARMS pArms,  
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness  
)
```

Parameters

pArms
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

eHandedness
[in] The arm to query (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.

Return Value
The normalized position ray of the arm, as an [XMVECTOR](#) in body space. The W value is always 0.
If *pArms* is NULL or *eHandedness* is too great, a zero vector is returned.

Remarks
To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmGetScreenSpaceLocation
[See Also](#) [Example](#)



Gets the location of the specified arm, in screen space coordinates.

Syntax

```
C++
XMVECTOR NuiHandlesArmGetScreenSpaceLocation(
    PCNUI_HANDLES_ARMS pArms,
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness
)
```

Parameters
pArms
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

eHandedness
[in] The arm to query (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) value.

Return Value
The location of the arm, as an [XMVECTOR](#), in screen coordinates. The Z value ranges from 0 through 1.0, and it measures how far away the arm is from the attach plane. The W value is always zero.

Remarks
You should use this function to locate a cursor onscreen that is tracking a player's hand position when the hand is not attached to a grip.
To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Example
This function is used in the [Menu Using Handles Sample](#) and [Menu Using Hover Sample](#) samples, called once for each arm, as follows.

```
NUI_HANDLES_ARMS g_HandlesArms;
...
// Get the current cursor position
XMFLOAT2 fLeftHandCursor = GetXYForVector( NuiHandlesArmGetScreenSpaceLocation(
    &g_HandlesArms, NUI_HANDLES_ARMS_HANDEDNESS_LEFT_ARM ) );
XMFLOAT2 fRightHandCursor = GetXYForVector( NuiHandlesArmGetScreenSpaceLocation(
    &g_HandlesArms, NUI_HANDLES_ARMS_HANDEDNESS_RIGHT_ARM ) );
```

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmGetSensorToPhizTransformationMatrix
[See Also](#)



Gets a transformation matrix that transforms from sensor coordinates to [Physical Interaction Zone \(PHIZ\)](#) coordinates, for the specified arm.

Syntax

```
C++
HRESULT NuiHandlesArmGetSensorToPhizTransformationMatrix(
    PCNUI_HANDLES_ARMS pArms,
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness,
```



```
        XMMATRIX *pMatrix  
    )
```

Parameters

- pArms*
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
- eHandedness*
[in] The arm (left or right) to query, as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.
- pMatrix*
[out] The transformation matrix, as a pointer to an [XMMATRIX](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the transformation matrix values. If this function does not return `NUI_HANDLES_S_OK`, this data is invalid.

Return Value

One of the following values:

Value	Meaning
<code>NUI_HANDLES_S_OK</code>	The function completed successfully.
<code>NUI_HANDLES_E_INVALID_ARG</code>	Either a required pointer parameter is NULL or <i>eHandedness</i> has an invalid value.

Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements

- Header:** Declared in `NuiHandles.h`.
- Library:** Use `NuiHandles.lib`.

See Also

- [The Physical Interaction Zone \(PHIZ\)](#)
- [NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmGetTwoHandedMode

[See Also](#)



Gets whether unified PHIZ is used for the specified arms.

Syntax

C++

```
BOOL NuiHandlesArmGetTwoHandedMode(  
    PCNUI_HANDLES_ARMS pArms  
)
```

Parameters

- pArms*
[in] The arms to query, as a pointer to the [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

Return Value

TRUE if the arms structure is using unified PHIZ; otherwise, FALSE. If *pArms* is NULL, it returns FALSE.

Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements

- Header:** Declared in `NuiHandles.h`.
- Library:** Use `NuiHandles.lib`.

See Also

- [The Physical Interaction Zone \(PHIZ\)](#)
- [NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmSetEnabled

[See Also](#)



Sets whether the specified arm is enabled.

Syntax

C++

```
HRESULT NuiHandlesArmSetEnabled(  
    PNUI_HANDLES_ARMS pArms,  
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness,  
    BOOL bEnabled  
)
```

Parameters

- pArms*
[in, out] The arms structure to set, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
- eHandedness*
[in] The arm to set (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.
- bEnabled*
[in] Indicates whether the arm is enabled.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> pointer is NULL, or the <i>eHandedness</i> value is invalid.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmSetPositionRay

[See Also](#)



Sets the position of the specified arm in body space.

Syntax

```
C++
HRESULT NuiHandlesArmSetPositionRay(
    PNUI_HANDLES_ARMS pArms,
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness,
    XMVECTOR vPositionRay
)
```

Parameters

- pArms*
[in, out] The arms structure to set, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
- eHandedness*
[in] The arm that is set (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.
- vPositionRay*
[in] The position to which to set the arm, as an [XMVECTOR](#). This vector is in body space coordinates, from the shoulder.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	Any of the following occurred: - The required <i>pArms</i> pointer is NULL. - The <i>eHandedness</i> value is too great. - The length of the ray vector, <i>vPositionRay</i>, is not between 0 and 1. (Ideally, it is 1.) - The ray vector, <i>vPositionRay</i>, is behind the player. The angle of the position ray should be less than 90 degrees, relative to the forward vector, when the rays start from the same origin. z is expected to be -1.0 to 0.0, with -1.0 pointing toward the camera.

Remarks

This function enables you to override the arm position that is determined by [NuiHandlesArmsUpdate](#).

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmSetTwoHandedMode

[See Also](#)



Sets whether unified PHIZ is used for the specified arms.

Syntax

C++

```
HRESULT NuiHandlesArmSetTwoHandedMode (
    PNUI_HANDLES_ARMS pArms,
    BOOL bIsTwoHanded
)
```

Parameters

- pArms*
[in] The arms to set, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
- bIsTwoHanded*
[in] Indicates whether the arm uses unified PHIZ.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> pointer is NULL.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

- [The Physical Interaction Zone \(PHIZ\)](#)
- [NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsGetApproachInfo

[See Also](#)



Gets information about the specified arm's approach to the nearest grip.

Syntax

C++

```
HRESULT NuiHandlesArmsGetApproachInfo (
    PCNUI_HANDLES_ARMS pArms,
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness,
    PNUI_HANDLES_APPROACH_INFO pApproachInfo
)
```

Parameters

- pArms*
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
- eHandedness*
[in] The arm to query (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.
- pApproachInfo*
[out] The approach information, as a pointer to a [NUI_HANDLES_APPROACH_INFO](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains approach information. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> or <i>pApproachInfo</i> pointer is NULL.

Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

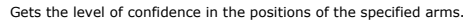
See Also

- [NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsGetArmStatus

[See Also](#) [Example](#)



C++

Parameters

[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

The quality of the arm position data, as a [NUI_HANDLES_ARM_STATUS](#) enumeration constant.

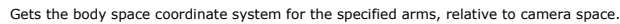
This function indicates three things: whether the arm is engaged and thus can attach to and manipulate handles; whether it is not tracked; or whether it is invalid. To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

This function is used in the [Menu Using Handles](#) sample.

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

NUI Handles Overview

[See Also](#)



C++

Parameters

[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

[out] The body space parameters for the specified arms structure, as a pointer to a [NUI_HANDLES_BODY_SPACE](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the body space parameters for the specified arms structure. If this function does not return `NUI_HANDLES_S_OK`, this data is invalid.

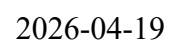
One of the following values:

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

NUI Handles Overview

See Also



Gets the default tuning parameters for arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

Syntax

C++

```
HRESULT NuiHandlesArmsGetDefaultTuningParams(  
    PNUI_HANDLES_TUNING_PARAMS_ARMS pArmsParams  
)
```

Parameters

pArmsParams
[out] The default tuning parameters for arms, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_ARMS](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the default tuning parameters for arms. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArmsParams</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsGetPlayerInfo

[See Also](#)



Gets identity information about the player associated with the specified arms, including tracking ID and user index.

Syntax

C++

```
HRESULT NuiHandlesArmsGetPlayerInfo(  
    PCNUI_HANDLES_ARMS pArms,  
    PNUI_HANDLES_PLAYER_INFO pPlayerInfo  
)
```

Parameters

pArms
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).
pPlayerInfo
[out] The identity information about the associated player, as a pointer to a [NUI_HANDLES_PLAYER_INFO](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the identity information. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> or <i>pPlayerInfo</i> pointer is NULL.

Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsGetTuningParams

[See Also](#)



Gets the tuning parameters for the specified arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

Syntax

C++

```
HRESULT NuiHandlesArmsGetTuningParams(  
    PCNUI_HANDLES_ARMS pArms,  
    PNUI_HANDLES_TUNING_PARAMS_ARMS pArmParams  
)
```

Parameters

pArms
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

pArmParams
[out] The tuning parameters for the arms structure, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_ARMS](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the tuning parameters for the arms. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> or <i>pArmParams</i> pointer is NULL.

Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsInit

[See Also](#) [Example](#)



Initializes an arms structure.

Syntax

C++

```
HRESULT NuiHandlesArmsInit(  
    PNUI_HANDLES_ARMS pArms  
)
```

Parameters

pArms
[out] The arms structure to initialize, as a pointer to a [NUI_HANDLES_ARMS](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the opaque [NUI_HANDLES_ARMS](#) structure is ready for use with the NuiHandles API. If this function does not return NUI_HANDLES_S_OK, this structure is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> pointer is NULL.

Remarks

Call this function before passing a [NUI_HANDLES_ARMS](#) to other NuiHandles functions.

Example

This function is used in the [Menu Using Handles](#) and [Menu Using Hover](#) samples.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsIsAttached

[See Also](#)



Gets whether the specified arm is attached to a handle.



Syntax

C++

```
BOOL NuiHandlesArmsIsAttached(
    PCNUI_HANDLES_ARMS pArms,
    NUI_HANDLES_ARMS_HANDEDNESS eHandedness
)
```

Parameters

pArms
[in] The arms structure to query, as a constant pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

eHandedness
[in] The hand to query (left or right), as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.

Return Value

TRUE if the specified arm is attached to a handle; otherwise, FALSE.



Remarks

To get current values, call this function promptly after calling [NuiHandlesArmsUpdate](#) on the [NUI_HANDLES_ARMS](#) structure specified in *pArms*.



Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.



See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsSetTuningParams

[See Also](#)



Sets the tuning parameters for the specified arms, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.



Syntax

C++

```
HRESULT NuiHandlesArmsSetTuningParams(
    PNUI_HANDLES_ARMS pArms,
    PCNUI_HANDLES_TUNING_PARAMS_ARMS pArmParams
)
```

Parameters

pArms
[in, out] The arms structure to set, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

pArmParams
[in] The tuning parameters to apply to the arms, as a constant pointer to a [NUI_HANDLES_TUNING_PARAMS_ARMS](#) structure.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> or <i>pArmParams</i> pointer is NULL, or the specified value of a tuning parameter is out of range.



Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.



See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsUpdate

[See Also](#) [Example](#)



Updates the state of the specified arms, based on the passed-in data from the Kinect sensor array.



Syntax

C++

```
HRESULT NuiHandlesArmsUpdate(  
    PNUI_HANDLES_ARMS pArms,  
    const INT iSkeletonDataIndex,  
    const NUI_SKELETON_FRAME *pUnfilteredSkeletonFrame,  
    const NUI_IMAGE_FRAME *pDepth320x240,  
    const D3DLOCKED_RECT *pDepthRect320x240,  
    const NUI_IMAGE_FRAME *pDepth80x60,  
    const D3DLOCKED_RECT *pDepthRect80x60  
)
```

Parameters

- pArms*
[in, out] The arms structure to update, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized using [NuiHandlesArmsInit](#).
- iSkeletonDataIndex*
[in] The skeleton whose arms to update, as an index that is used in the **pUnfilteredSkeletonFrame* structure.
- pUnfilteredSkeletonFrame*
[in] The skeleton frame to use to update the arm state, as a pointer to a [NUI_SKELETON_FRAME](#) structure. To get the structure, call [NuiSkeletonGetNextFrame](#).
You must not filter the skeleton data provided in this structure in any way, such as by applying tilt correction or joint smoothing. Such filtering interferes with the arm tracking process used in implementing arm tracking. Titles that use tilt correction or joint smoothing should apply such transformations to the skeleton data only after this function has returned.
- pDepth320x240*
[in] The image frame to use to update the arm state, as a pointer to a constant [NUI_IMAGE_FRAME](#) structure. To get the structure, call [NuiImageStreamGetNextFrame](#).
The image data must come from a depth image stream with 320×240 resolution. The image frame should have the same **dwFrameNumber** as the [NUI_SKELETON_FRAME](#) that contains the skeleton data in **pUnfilteredSkeletonFrame*.
For **eImageType** in this [NUI_IMAGE_FRAME](#) structure, this function needs the unregistered depth frame—that is, [NUI_IMAGE_TYPE_DEPTH](#). If you use a depth frame that has been registered to color space (that is, [NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE](#)), the quality of hand refinement will be severely degraded.
- pDepthRect320x240*
[in, optional] A locked rectangle on the 320×240 depth image, as a pointer to a [D3DLOCKED_RECT](#) structure.
If a locked rectangle is passed in, the locked rectangle is used, and the rectangle remains locked after the function returns. If NULL is passed in, this function locks a rectangle on the depth image, and frees the lock before the function returns.
If your code is already locking a rectangle, and you pass NULL for this parameter, your lock will be freed when this function returns. If you are using your own locked rectangle, you should set this parameter to point to the locked rectangle, to avoid having your lock freed unexpectedly.
- pDepth80x60*
[in] The image frame to use to update arm state, as a pointer to a constant [NUI_IMAGE_FRAME](#) structure. To get the structure, call [NuiImageStreamGetNextFrame](#).
The image data must come from a depth image stream with 80×60 resolution. The image frame should have the same **dwFrameNumber** as the [NUI_SKELETON_FRAME](#) that contains the skeleton data in **pUnfilteredSkeletonFrame*.
For **eImageType** in this [NUI_IMAGE_FRAME](#) structure, this function needs the unregistered depth frame—that is, [NUI_IMAGE_TYPE_DEPTH](#). If a depth frame that has been registered to color space is used (that is, [NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE](#)), the quality of hand refinement will be severely degraded.
- pDepthRect80x60*
[in, optional] A locked rectangle on the 80×60 depth image, as a pointer to a [D3DLOCKED_RECT](#) structure.
If a locked rectangle is passed in, the locked rectangle is used and the rectangle remains locked after the function returns. If NULL is passed in, this function locks a rectangle on the depth image, and frees the lock before the function returns.
If your code is already locking a rectangle, and you pass NULL for this parameter, your lock will be freed when this function returns. If you are using your own locked rectangle, you should set this parameter to point to the locked rectangle, to avoid having your lock freed unexpectedly.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	<i>pArms</i> , <i>pUnfilteredSkeletonFrame</i> , <i>pDepth80x60</i> , or <i>pDepth320x240</i> is NULL.
NUI_HANDLES_E_WRONG_IMAGE_FRAME_RESOLUTION	One or more of the passed-in depth images is in the wrong resolution.

Remarks

To ensure that grips and rails update properly, first call **NuiHandlesArmsUpdate**, then [NuiHandlesGripUpdate](#), and then [NuiHandlesArmsUpdateRailProgress](#).

Example

This function is used in the [Menu Using Handles](#) and [Menu Using Hover](#) samples.

Requirements

- Header:** Declared in [NuiHandles.h](#).
Library: Use [NuiHandles.lib](#).

See Also

[NUI_Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesArmsUpdateRailProgress

[See Also Example](#)



Updates the states of all handles currently attached to the specified arms, based on the most recently updated arm positions.

Syntax

```
C++  
  
HRESULT NuiHandlesArmsUpdateRailProgress(  
    PNUI_HANDLES_ARMS pArms,  
    DWORD dwMillisecondsPassedSinceLastSkeletonUpdate  
)
```

Parameters

pArms

[in, out] The arms structure to update, as a pointer to a [NUI_HANDLES_ARMS](#) structure that has been initialized by [NuiHandlesArmsInit](#).

dwMillisecondsPassedSinceLastSkeletonUpdate

[in] The number of milliseconds elapsed since the most recent call to [NuiSkeletonGetNextFrame](#) retrieved skeleton data. For some titles, skeleton data updates might occur at a slower rate than title code is rendering frames. The time value passed in this parameter must be from the latest skeleton data update, not from the latest render.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pArms</i> pointer is NULL.

Remarks

To make the grip state transition into the attached state, you must call this function. To ensure that grips and rails update properly, first call [NuiHandlesArmsUpdate](#), then [NuiHandlesGripUpdate](#), and then **NuiHandlesArmsUpdateRailProgress**.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGlobalGetDefaultTuningParams

[See Also](#)



Gets the default global tuning parameters for handles, to retrieve the width and height of the physical interaction zone (PHIZ).

Syntax

C++

```
HRESULT NuiHandlesGlobalGetDefaultTuningParams(  
    PNUI_HANDLES_TUNING_PARAMS_GLOBAL pGlobalParams  
)
```

Parameters

pGlobalParams

[out] The default global parameter values for tuning handles, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_GLOBAL](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the parameter values. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGlobalParams</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[NUI Handles Overview](#)
[NuiHandlesGlobalGetTuningParams](#)
[NuiHandlesGlobalSetTuningParams](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGlobalGetTuningParams

[See Also](#)



Gets the global tuning parameters for handles, to retrieve the width and height of the physical interaction zone (PHIZ).

Syntax

C++

```
HRESULT NuiHandlesGlobalGetTuningParams(  
    PNUI_HANDLES_TUNING_PARAMS_GLOBAL pGlobalParams  
)
```

Parameters

pGlobalParams
[out] The global parameter values for tuning handles, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_GLOBAL](#) structure.
It must be non-NULL when you call the function. After the function call returns, the structure contains the tuning parameters. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGlobalParams</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

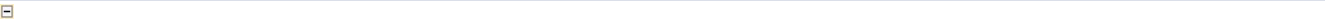
See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[NUI Handles Overview](#)
[NuiHandlesGlobalGetDefaultTuningParams](#)
[NuiHandlesGlobalSetTuningParams](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGlobalSetTuningParams

[See Also](#)



Sets the global tuning parameters for handles, to customize the width and height of the physical interaction zone (PHIZ).

Syntax

```
C++
HRESULT NuiHandlesGlobalSetTuningParams(
    PCNUI_HANDLES_TUNING_PARAMS_GLOBAL pGlobalParams
)
```

Parameters

pGlobalParams
[in] The global parameter values for tuning handles, as a constant pointer to a [NUI_HANDLES_TUNING_PARAMS_GLOBAL](#) structure.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGlobalParams</i> pointer is NULL, or the specified value of a tuning parameter is out of range.

Remarks

The physical interaction zone (PHIZ) is a region of space in front of the player where there is a spatial mapping relationship between the player's hand movements in physical space and cursor movement in virtual space (in the on-screen UI). A player's hand movement within the PHIZ translates to cursor control and movement on screen.
To customize the PHIZ, call this function, passing a structure that specifies:

- A horizontal offset
- The width of the PHIZ
- A vertical offset
- The height of the PHIZ

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[NUI Handles Overview](#)
[NuiHandlesGlobalGetDefaultTuningParams](#)
[NuiHandlesGlobalGetTuningParams](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripAddRailAnalogStick

[See Also](#)



Adds a new analog joystick rail to a grip.

Syntax

C++

```
HRESULT NuiHandlesGripAddRailAnalogStick(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID,  
    FLOAT fSizeInRadians,  
    FLOAT fDeadzoneSizeInRadians  
)
```

Parameters

- pGrip*
[in, out] The grip, as a [NUI_HANDLES_GRIP](#) structure.
- dwRailID*
[in] The title-defined identifier of the rail. If you use an identifier that has already been used in the specified grip, the properties of the existing rail are changed.
- fSizeInRadians*
[in] The angle in radians specifying the maximum radius of motion.
Valid values are 0 through pi/2. Must be >= *fDeadzoneSizeInRadians*.
- fDeadzoneSizeInRadians*
[in] The deadzone size in radians. The deadzone is a narrower circle around the central point where hand movement is ignored. The size of the deadzone can be tuned to regulate how sensitive the rail is to being engaged.
Valid values are 0 through pi/2. Must be <= *fSizeInRadians*.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_RAIL_ID_INVALID	The provided <i>dwRailID</i> is in use already.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.
NUI_HANDLES_E_MAX_RAILS_EXCEEDED	The grip already has the maximum number of rails (NUI_HANDLES_MAX_RAILS_PER_GRIP).

Remarks

By default, the added rail is enabled.
The angle and magnitude of the joystick position are stored in a [NUI_HANDLES_RAILSTATE_ANALOGSTICK](#) structure that is queried by calling [NuiHandlesGripGetRailState](#).
An analog joystick rail consists of a circular boundary around a central point, with progress measuring the deviation of the player's hand position from the central point, in polar coordinates, and motion confined within the circular boundary. The central point of the rail is determined by the position ray from the player when the grip is attached. The radius of the circle is defined by the angle the player must pivot his or her shoulder to point to the edge of the circle.

Requirements

- Header:** Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripAddRailArc

[See Also](#) [Example](#)



Adds a new arc rail to a grip.

Syntax

C++

```
HRESULT NuiHandlesGripAddRailArc(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID,  
    XMVECTOR vDirection,  
    FLOAT fLengthInRadians,  
    FLOAT fDeadzoneLengthInRadians,  
    FLOAT fToleranceConeAngleInRadians  
)
```

Parameters

- pGrip*
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.
- dwRailID*
[in] The title-defined identifier of the rail. If you use an identifier that has been used already in the specified grip, the properties of the existing rail are changed.
- vDirection*
[in] The direction of positive progress along the arc, in grip space coordinates, as an [XMVECTOR](#). To track linear progress from the grip in two different directions, define two arc rails, one for each direction.
The length of *vDirection* must be nonzero.
- fLengthInRadians*
[in] The length of the rail arc, expressed in radians. The angular distance begins from the position ray from the player when the grip is attached, and ends at the hand position that the player must pivot his or her shoulder to point to the far end of the rail.
Valid values are 0 through pi. It must be >= *fDeadzoneLengthInRadians*.
- fDeadzoneLengthInRadians*
[in] The angle in radians from the low limit of the deadzone around the grip to the high limit of the deadzone around the grip. The deadzone is a narrow area around the grip where hand movement is ignored. The size of the deadzone can be tuned to regulate how sensitive the rail is to being engaged.
Valid values are 0 through pi. It must not exceed *fLengthInRadians*.

ToleranceConeAngleInRadians

[in] Defines a tolerance cone that allows the attach area of the rail to expand as the handle progresses along the rail. This angle is measured from one side of the rail arc to the other. When a player's hand deviates from the track, the hand is still tracked, but the progress response to the hand's motion is damped to provide feedback.

Valid values are 0 through pi/2.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_RAIL_ID_INVALID	The provided <i>dwRailID</i> is in use already.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.
NUI_HANDLES_E_MAX_RAILS_EXCEEDED	The grip already has the maximum number of rails (NUI_HANDLES_MAX_RAILS_PER_GRIP).

Remarks

By default, the added rail is enabled.

The arc rail is the most commonly used kind of rail. It consists of a single track with progress measured from one point to another. The position along the rail is stored in a [NUI_HANDLES_RAILSTATE_ARC](#) structure that is queried by calling [NuiHandlesGripGetRailState](#).

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripAddRailGearShift

[See Also Example](#)

Adds a new gearshift rail to a grip.

Syntax

```
C++

HRESULT NuiHandlesGripAddRailGearShift (
    PNUI_HANDLES_GRIP pGrip,
    DWORD dwRailID,
    XMVECTOR vDirection,
    FLOAT fSpineLengthInRadians,
    FLOAT fToothLowAngleInRadians,
    FLOAT fToothHighAngleInRadians,
    FLOAT fToothDeadzoneInRadians,
    FLOAT fToothToleranceConeAngleInRadians,
    UINT nNumTeeth,
    FLOAT fScrollZoneTopLengthInRadians,
    FLOAT fScrollZoneBottomLengthInRadians
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the gearshift rail to add.
If you use an identifier that has been used already in the specified grip, the properties of the existing rail are changed.

vDirection
[in] The direction of positive motion along the spine, in grip space coordinates, as an [XMVECTOR](#).
The zero angle of the rail is determined by the position ray from the player when the grip is attached. The angles that define the rail's spine parameters are measured from that point. The angle that defines the position along a tooth is measured perpendicular to the spine.
The length of *vDirection* must be nonzero.

fSpineLengthInRadians
[in] The total length of the gearshift spine, including the scroll zones.
Valid values are >0, through pi.

fToothLowAngleInRadians
[in] The length of the portion of the tooth on one side of the spine.
If 0, there is no tooth on this side of the spine.
Valid values are 0 through -pi.

fToothHighAngleInRadians
[in] The length of the portion of the tooth on the other side of the spine.
If 0, there is no tooth on this side of the spine.
Valid values are 0 through pi/2. It must be > *fToothLowAngleInRadians*.

fToothDeadzoneInRadians
[in] The angle in radians defining the deadzone around the initial attach point.
The angle is from the low limit of the deadzone around the tooth to the high limit of the deadzone around the tooth. The deadzone is a narrow area around a tooth where hand movement is ignored. The size of the deadzone can be tuned to regulate how sensitive the rail is to being engaged.

Valid values are 0 or a positive number. It must be $\leq fToothHighAngleInRadians$; also, it must be $> fToothLowAngleInRadians$.

fToothToleranceConeAngleInRadians
[in] An angle that defines a tolerance cone.

This tolerance cone allows the attach area of the rail to expand as the handle progresses along the rail. The angle is measured from one side of the tooth to the other.

Valid values are 0 through $\pi/2$.

nNumTeeth
[in] The number of teeth coming off of the spine.

If there are scroll zones, the items that are controlled by the teeth can change as the player scrolls. The teeth themselves do not move as the player scrolls through items.

Valid values are 1 or greater.

fScrollZoneTopLengthInRadians
[in] The length of the scroll zone that has an index of -1.

If set to 0, there is no scroll zone beyond index 0 (that is, beyond tooth zone 0). The position of the "top" scroll zone is not necessarily at the top. Its placement is affected by *vDirection*, so this scroll zone can be considered scroll zone "low."

Valid values are 0 or greater. The combined size of the top and bottom scroll zones must not be greater than the spine length.

fScrollZoneBottomLengthInRadians
[in] The length of the scroll zone that has an index of *nNumTeeth*.

If set to 0, there is no scroll zone beyond index *nNumTeeth*-1 (that is, beyond the highest numbered tooth zone). The position of the "bottom" scroll zone is not necessarily at the bottom. Its placement is affected by *vDirection*, so this scroll zone can be considered scroll zone "high."

Valid values are 0 or greater. The combined size of the top and bottom scroll zones must not be greater than the spine length.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	Any of the following: - The required <i>pGrip</i> pointer is NULL. <i>vDirection</i> has length of 0. - The combined size of the scroll zones is greater than the spine length. - A parameter is out of range.
NUI_HANDLES_E_RAIL_ID_INVALID	NUI_HANDLES_INVALID_RAIL_ID was passed in as the <i>dwRailID</i> value.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.
NUI_HANDLES_E_MAX_RAILS_EXCEEDED	The grip already has the maximum number of rails (NUI_HANDLES_MAX_RAILS_PER_GRIP).

Remarks

By default, the added rail is enabled.

A gearshift rail consists of a spine and a number of teeth coming perpendicularly off one side of the spine, or crossing the spine. If the teeth are placed on one side of the spine, the shape of the gearshift rail resembles a comb. If the teeth lay across the spine symmetrically, the shape of the gearshift rail resembles a diagram on the gear shifter of a manual transmission of a car.

The position along the spine, the index number of the committed tooth, and the position along the committed tooth are stored in a [NUI_HANDLES_RAILSTATE_GEARSHIFT](#) structure that is queried by calling [NuiHandlesGripGetRailState](#).

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NuiHandlesGripAddRailHoverTimer

[See Also](#)



Adds a hover timer to a grip.

Syntax

C++

```
HRESULT NuiHandlesGripAddRailHoverTimer(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID,  
    DWORD dwTotalHoverTimeInMilliseconds,  
    DWORD dwHoverTimeToCommitInMilliseconds  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the hover timer.

If you use an identifier that has been used already in the specified grip, the properties of the existing hover timer are changed.

dwTotalHoverTimeInMilliseconds
[in] The time it takes for the progress to reach 1.0, starting from when the player's hand begins hovering over the grip.

It must be greater than *dwHoverTimeToCommitInMilliseconds*.

dwHoverTimeToCommitInMilliseconds
[in] The time, in milliseconds, that the player's hand must hover over the grip before the arm is committed or attached to the grip.

Return Value
One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required value <i>dwHoverTimeToCommitInMilliseconds</i> , <i>dwTotalHoverTimeInMilliseconds</i> , or <i>dwTotalHoverTimeInMilliseconds</i> is 0.

Remarks
The length of time the hand has been over the hover timer and the percent progress are stored in a [NUI_HANDLES_RAILSTATE_HOVERTIMER](#) structure that is queried by calling [NuiHandlesGripGetRailState](#).

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)
[Time-based Constraints with the Hover Timer](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripForceDetach

[See Also](#) [Example](#)



Detaches the specified grip from user input.

Syntax

```
C++  
  
HRESULT NuiHandlesGripForceDetach(  
    PNUI_HANDLES_GRIP pGrip  
)
```

Parameters
pGrip
[in, out] The grip to detach, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value
One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.

Remarks
After a forced detach, the time specified by *dwReattachTimeInMilliseconds* in [NUI_HANDLES_TUNING_PARAMS_ARMS](#) must pass before the arm can try to reattach to another grip. The NuiHandles API never detaches a grip automatically. You must always call this function when a grip is no longer in use. Call this function during the following conditions:

- When the player's hand has moved sufficiently far from the grip's rails to indicate that the player has abandoned the grip.
- When the player's position on a rail indicates that the player has made a positive choice using the grip.
- Before deleting a grip, to ensure that the grip is properly cleaned up.

Example
This function is used in the [Menu Using Handles](#) sample.

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetApproachInfo

[See Also](#)



Gets the approach information about the nearest unattached arm.

Syntax

```
C++
```

```
HRESULT NuiHandlesGripGetApproachInfo(  
    PCNUI_HANDLES_GRIP pGrip,  
    PNUI_HANDLES_APPROACH_INFO pApproachInfo  
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

pApproachInfo
[out] The approach information for the grip, as a pointer to a [NUI_HANDLES_APPROACH_INFO](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the approach information. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pApproachInfo</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetCommittedRailId

[See Also](#) [Example](#)



Gets the rail identifier of the rail to which the player has committed, if any.

Syntax

C++

```
HRESULT NuiHandlesGripGetCommittedRailId(  
    PCNUI_HANDLES_GRIP pGrip,  
    DWORD *pdwRailId  
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

pdwRailId
[out] The ID of the committed rail, as a pointer to a DWORD.

This pointer must be non-NULL when you call the function. After the function call returns, the DWORD contains the ID of the committed rail. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

If there is no committed rail, this parameter returns a pointer to [NUI_HANDLES_INVALID_RAIL_ID](#), and the return value of this function is NUI_HANDLES_S_OK.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pdwRailId</i> pointer is NULL.
NUI_HANDLES_INVALID_RAIL_ID	The grip is uncommitted.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetDefaultTuningParams

[See Also](#)



Gets the default tuning parameters for grips, which define how long an arm needs to be in an attach or detach zone before the arm attaches to or detaches from the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripGetDefaultTuningParams(  
    PNUI_HANDLES_TUNING_PARAMS_GRIP pGripParams  
)
```

Parameters

pGripParams
[out] The default tuning parameters for grips, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_GRIP](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the parameter values. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGripParams</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetEnabled

[See Also](#) [Example](#)



Determines whether the specified grip is enabled, which determines whether the grip and its rails can be manipulated by the player.

Syntax

C++

```
BOOL NuiHandlesGripGetEnabled(  
    PCNUI_HANDLES_GRIP pGrip  
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

TRUE if the grip is enabled; otherwise, FALSE.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetHandedness

[See Also](#)



Gets the handedness of the specified grip.

Syntax

C++

```
NUI_HANDLES_GRIP_HANDEDNESS NuiHandlesGripGetHandedness(  
    PCNUI_HANDLES_GRIP pGrip  
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

The [NUI_HANDLES_GRIP_HANDEDNESS](#) value of the grip.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetIsAttached

[See Also](#) [Example](#)



Gets whether the specified grip is attached.

Syntax

C++

```
BOOL NuiHandlesGripGetIsAttached(  
    PCNUI_HANDLES_GRIP pGrip  
)
```

Parameters

pGrip
[in] The grip to query, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

TRUE if the grip is attached; otherwise, FALSE.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetLastAttachPlayerInfo

[See Also](#)



Gets identity information about the last player to attach to the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripGetLastAttachPlayerInfo(  
    PCNUI_HANDLES_GRIP pGrip,  
    PNUI_HANDLES_PLAYER_INFO pPlayerInfo  
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

pPlayerInfo
[out] Identity information about the player, as a pointer to a [NUI_HANDLES_PLAYER_INFO](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the identity information. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pPlayerInfo</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NuiHandlesGripGetNumRails

[See Also](#)



Gets the number of rails that are on the specified grip.

Syntax

```
C++
HRESULT NuiHandlesGripGetNumRails(
    PCNUI_HANDLES_GRIP pGrip,
    DWORD *pdwNumRails
)
```

Parameters

pGrip
[in] The grip, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

pdwNumRails
[out] The number of rails that are on the grip, as a pointer to a DWORD.
This pointer must be non-NULL when you call the function. After the function call returns, the DWORD contains the number of rails that are on the grip. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pdwNumRails</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NuiHandlesGripGetPositionRay

[See Also](#)



Gets the position ray for the specified grip, in body space coordinates.

Syntax

```
C++
XMVECTOR NuiHandlesGripGetPositionRay(
    PCNUI_HANDLES_GRIP pGrip
)
```

Parameters

pGrip
[in] The grip to query, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

The position ray for the grip, as an [XMVECTOR](#). The vector is in body space coordinates, from the shoulder joint corresponding to the grip's handedness.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NuiHandlesGripGetRailEnabled

[See Also](#)



Gets whether the specified rail is enabled.

Syntax

C++

```
BOOL NuiHandlesGripGetRailEnabled(  
    PCNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID  
)
```

Parameters

pGrip
[in] The grip that has the rail, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the rail to query.

Return Value

TRUE if the rail is enabled; otherwise, FALSE if it is not enabled.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetRailState

[See Also Example](#)



Gets whether the specified rail is enabled and committed, and gets the player's position along the rail.

Syntax

C++

```
HRESULT NuiHandlesGripGetRailState(  
    PCNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailId,  
    PNUI_HANDLES_RAILSTATE pRailState  
)
```

Parameters

pGrip
[in] The grip that has the rail, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailId
[in] The title-defined identifier of the rail to query.

pRailState
[out] The state of the rail, as a pointer to a [NUI_HANDLES_RAILSTATE](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the rail state. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	One or more of the required <i>pGrip</i> or <i>pRailState</i> pointers is NULL.
NUI_HANDLES_E_RAIL_ID_INVALID	The <i>dwRailId</i> is invalid.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetScreenSpaceLocation

[See Also Example](#)



Gets the persistent screen space location of the specified grip.

Syntax

C++

```
XMVECTOR NuiHandlesGripGetScreenSpaceLocation(  
    PCNUI_HANDLES_GRIP pGrip  
)
```

Parameters

pGrip
[in] The grip, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

The location of the grip, as an [XMVECTOR](#) in screen space coordinates.

Remarks

If no persistent progress has been set on a grip or rail by calling [NuiHandlesGripRailSetPersistentProgress](#), **NuiHandlesGripGetScreenSpaceLocation** returns the original location of the grip. If persistence has been set, **NuiHandlesGripGetScreenSpaceLocation** returns the screen position of that persisted progress.

The screen location for a grip is not updated after an arm is attached to the grip. To get information about the progression of a grip across a rail, call [NuiHandlesGripGetRailState](#).

The return value is in screen space coordinates. However, the values might be outside the normal screen space range of -1 to +1 if the player's hand has moved in such a way that the grip is no longer on screen. You should verify that the returned location is valid before you use that location to draw to the screen.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetShape

[See Also](#)

Gets the size of the attach, approach, and detach zones of the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripGetShape(  
    PCNUI_HANDLES_GRIP pGrip,  
    PNUI_HANDLES_GRIPSHAPE pShape  
)
```

Parameters

pGrip
[in] The grip, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

pShape
[out] The grip zone sizes, as a pointer to a [NUI_HANDLES_GRIPSHAPE](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the grip zone sizes. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	A required pointer parameter is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetTuningParams

[See Also](#)

Gets how long an arm needs to be in an attach or detach zone before the arm attaches to or detaches from the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripGetTuningParams(  
    PCNUI_HANDLES_GRIP pGrip,
```

```
PNUI_HANDLES_TUNING_PARAMS_GRIP pGripParams
}
```

Parameters

pGrip
[in] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

pGripParams
[out] The parameter settings, as a pointer to a [NUI_HANDLES_TUNING_PARAMS_GRIP](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the parameter values. If this function does not return NUI_HANDLES_S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pGripParams</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripGetUserContext

[See Also](#)



Gets the user context pointer of the specified grip, to store title-specific information associated with the grip.

Syntax

C++

```
PVOID NuiHandlesGripGetUserContext(
    PCNUI_HANDLES_GRIP pGrip
)
```

Parameters

pGrip
[in] The grip, as a constant pointer to a [NUI_HANDLES_GRIP](#) structure.

Return Value

The user context pointer that was set by the title by calling [NuiHandlesGripSetUserContext](#). If no pointer has been set, it returns NULL.

Remarks

You use the user context pointer to point to a title-allocated structure or the buffer that holds title-specific information associated with the grip.

You are responsible for allocating and freeing the user context memory.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripInit

[See Also Example](#)



Initializes the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripInit(
    PNUI_HANDLES_GRIP pGrip
)
```

Parameters

pGrip
[out] The grip to initialize, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the opaque [NUI_HANDLES_GRIP](#) structure is ready for use with the NuiHandles API. If this function does not return NUI_HANDLES_S_OK, this structure is invalid.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.

⊕ **Remarks**

By default, the grip is enabled, and is oriented in the direction (0, 0, -1) in body space.

⊕ **Example**

This function is used in the [Menu Using Handles](#) sample.

⊕ **Requirements**

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

⊕ **See Also**

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripRailGetPersistentProgress

[See Also](#)



Indicates the progress to which the specified arc rail or gearshift rail is set initially when the player attaches to the rail.

⊕ **Syntax**

```
C++  
  
FLOAT NuiHandlesGripRailGetPersistentProgress(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID  
)
```

Parameters

pGrip
[in, out] The grip that has the rail, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the rail.

Return Value

The persistent progress that was set last, in the range 0 to 1.0.

If the rail is an arc rail, 0-1.0 is the entire range of the arc.

If the rail is a gearshift rail, 0-1.0 represents the progress along the spine, excluding the scroll zones.

If the required *pGrip* pointer is NULL, it returns NUI_HANDLES_E_INVALID_ARG.

⊕ **Remarks**

The persistent progress is the progress that the rail will assume upon attaching.

This function does not apply to analog joystick rails or hover timers. Also, this function returns the setting that was set by [NuiHandlesGripRailSetPersistentProgress](#).

⊕ **Requirements**

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

⊕ **See Also**

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripRailSetPersistentProgress

[See Also](#) [Example](#)



Sets the progress that the specified arc rail or gearshift rail will have initially when the player attaches to the rail.

⊕ **Syntax**

```
C++  
  
HRESULT NuiHandlesGripRailSetPersistentProgress(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID,  
    FLOAT fProgress  
)
```

Parameters

pGrip
[in, out] The grip that has the rail, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the rail.

fProgress
[in] The persistent progress along the rail. Valid values are 0 through 1.0.
If the rail is an arc rail, 0-1.0 represents the entire range of the arc.
If the rail is a gearshift rail, 0-1.0 represents the progress along the spine, excluding the scroll zones.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_RAIL_ID_INVALID	The specified rail ID is invalid.
NUI_HANDLES_E_UNEXPECTED_FAILURE	The rail is not an arc rail or a gearshift rail.

Remarks

The persistent progress is the progress that the rail will assume upon attaching.
This function does not apply to analog joystick rails or hover timers.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)
[NuiHandlesGripRailGetPersistentProgress](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetEnabled

[See Also Example](#)



Enables or disables the specified grip, to control whether the grip and its rails can be manipulated by the player.

Syntax

C++

```
HRESULT NuiHandlesGripSetEnabled(  
    PNUI_HANDLES_GRIP pGrip,  
    BOOL bEnabled  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

bEnabled
[in] Indicates whether the grip and associated rails are enabled. If TRUE, the grip and associated rails are interactive. If FALSE, the grip cannot be manipulated by the player.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.

Example

This function is used in the [Menu Using Handles](#) sample.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetHandedness

[See Also](#)



Sets whether the handedness of the specified grip is left, right, or ambidextrous.

Syntax

C++

```
HRESULT NuiHandlesGripSetHandedness(  
    PNUI_HANDLES_GRIP pGrip,  
    NUI_HANDLES_GRIP_HANDEDNESS eHandedness  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

eHandedness
[in] The [NUI_HANDLES_GRIP_HANDEDNESS](#) to set for the grip.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.

Remarks

This function can only set the handedness if the grip is not attached currently.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetPositionRay

[See Also](#)



Sets the position of the specified grip in screen space.

Syntax

C++

```
HRESULT NuiHandlesGripSetPositionRay(  
    PNUI_HANDLES_GRIP pGrip,  
    XMVECTOR vPositionRay  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

vPositionRay
[in] The position ray to be set, as an [XMVECTOR](#). This vector is in body space coordinates, measured from the shoulder associated with the grip.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.

Remarks

This function can only set the position ray if the grip is not attached currently.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetRailEnabled

[See Also](#)

Enables or disables the specified rail.

+

Syntax

C++

```
HRESULT NuiHandlesGripSetRailEnabled(  
    PNUI_HANDLES_GRIP pGrip,  
    DWORD dwRailID,  
    BOOL bEnabled  
)
```

Parameters

pGrip
[in] The grip that has the rail, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

dwRailID
[in] The title-defined identifier of the rail.

bEnabled
[in] Indicates whether the rail is enabled.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.
NUI_HANDLES_E_RAIL_ID_INVALID	The given grip has no rail with rail ID <i>dwRailID</i> .

+

Remarks

Enabling or disabling one rail does not enable or disable other rails on the same grip.
This function can only enable the rail if the grip is not attached currently.

+

Requirements

Header:

Declared in NuiHandles.h.

Library:

Use NuiHandles.lib.

+

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetScreenSpaceLocation

[See Also](#) [Example](#)

Sets the screen space location of the specified grip.

+

Syntax

C++

```
HRESULT NuiHandlesGripSetScreenSpaceLocation(  
    PNUI_HANDLES_GRIP pGrip,  
    XMVECTOR vScreenSpaceLoc  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

vScreenSpaceLoc
[in] The screen space location to be set, as an [XMVECTOR](#). Valid screen space coordinates are -1.0 through 1.0 in the X and Y directions.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL, or the <i>vScreenSpaceLoc</i> location is invalid.
NUI_HANDLES_E_ILLEGAL_WHILE_ATTACHED	The operation is not allowed while the grip is attached.

+

Remarks

This function can only set the screen space location if the grip is not attached currently.

+

Example

This function is used in the [Menu Using Handles](#) sample.

+

Requirements

Header:

Declared in NuiHandles.h.

Library:

Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetShape
[See Also](#)



Sets the size of the attach, approach, and detach zones of the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripSetShape(  
    PNUI_HANDLES_GRIP pGrip,  
    PCNUI_HANDLES_GRIPSHAPE pShape  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

pShape
[in] The zone sizes to apply to the grip, as a constant pointer to a [NUI_HANDLES_GRIPSHAPE](#) structure.

In this structure, the following must be true:

- fFastAttachRadius <= fSlowAttachRadius
- fSlowAttachRadius <= fApproachRadius
- fApproachRadius <= fSlowDetachRadius

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	A required pointer parameter is NULL, or <i>pShape</i> has invalid member values.

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetTuningParams
[See Also](#)



Sets how long an arm needs to be in an attach zone or a detach zone before the arm attaches to or detaches from the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripSetTuningParams(  
    PNUI_HANDLES_GRIP pGrip,  
    PCNUI_HANDLES_TUNING_PARAMS_GRIP pGripParams  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

pGripParams
[in] The tuning parameter settings, as a constant pointer to a [NUI_HANDLES_TUNING_PARAMS_GRIP](#) structure.

Return Value

One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pGripParams</i> pointer is NULL, or the specified value of a tuning parameter is out of range.

Requirements
Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripSetUserContext

[See Also](#)



Sets the user context pointer for the specified grip, to store title-specific information associated with the grip.

Syntax

C++

```
VOID NuiHandlesGripSetUserContext(  
    PNUI_HANDLES_GRIP pGrip,  
    PVOID pContext  
)
```

Parameters

pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

pContext
[in, out] The user context pointer to be set.

Return Value

Remarks

You use the user context pointer to point to a title-allocated structure or the buffer that holds title-specific information associated with the grip. You are responsible for allocating and freeing the user context memory.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripUninit

[See Also](#)



Uninitializes the specified grip.

Syntax

C++

```
HRESULT NuiHandlesGripUninit(  
    PNUI_HANDLES_GRIP pGrip,  
    PNUI_HANDLES_ARMS pArms  
)
```

Parameters

pGrip
[in, out] The grip to uninitialize, as a pointer to a [NUI_HANDLES_GRIP](#) structure.

pArms
[in, out] The arm that attached to the grip, as a pointer to a [NUI_HANDLES_ARMS](#) structure.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> or <i>pArms</i> pointer is NULL.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)
[NuiHandlesGripInit](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandlesGripUpdate

[See Also](#) [Example](#)



Updates the specified grip to reflect the current arm positions.

 **Syntax**

C++

```
HRESULT NuiHandlesGripUpdate(  
    PNUI_HANDLES_GRIP pGrip,  
    PNUI_HANDLES_ARMS pArms  
)
```

Parameters
pGrip
[in, out] The grip, as a pointer to a [NUI_HANDLES_GRIP](#) structure.
pArms
[in, out] The arms, as a pointer to a [NUI_HANDLES_ARMS](#) structure that was recently updated.

Return Value
One of the following values:

Value	Meaning
NUI_HANDLES_S_OK	The function completed successfully.
NUI_HANDLES_E_INVALID_ARG	The required <i>pGrip</i> pointer is NULL.

 **Remarks**

To ensure that grips and rails update properly, first call [NuiHandlesArmsUpdate](#), then **NuiHandlesGripUpdate**, and then [NuiHandlesArmsUpdateRailProgress](#).

 **Example**

This function is used in the [Menu Using Handles](#) sample.

 **Requirements**

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

 **See Also**

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandles Structures

[See Also](#)



In This Section
[NUI_HANDLES_APPROACH_INFO](#)
Describes a handle's approach, including the distance between grip and hand, the attach zone the arm is in, and whether the arm is attached to the grip.
[NUI_HANDLES_ARMS](#)
Represents a handle's arms.
[NUI_HANDLES_BODY_SPACE](#)
Describes the body space coordinate system relative to camera space.
[NUI_HANDLES_GRIP](#)
Represents a grip.
[NUI_HANDLES_GRIPSHAPE](#)
Specifies the size of the attach, approach, and detach zones of a grip.
[NUI_HANDLES_PLAYER_INFO](#)
Contains identity information about the associated player, including tracking ID and user index.
[NUI_HANDLES_RAILSTATE](#)
Describes whether the specified rail is enabled and committed, and the player's current position on the rail.
[NUI_HANDLES_RAILSTATE_ANALOGSTICK](#)
Indicates the current position of an analog joystick rail.
[NUI_HANDLES_RAILSTATE_ARC](#)
Indicates the player's progress along an arc rail.
[NUI_HANDLES_RAILSTATE_GEARSHIFT](#)
Indicates the status of a gearshift rail, including the progress along the spine, which tooth is committed, and the progress along the tooth.
[NUI_HANDLES_RAILSTATE_HOVERTIMER](#)
Indicates the length of time an arm has been attached to a hover timer, and the percent progress of the hover timer.
[NUI_HANDLES_TUNING_PARAMS_ARMS](#)
Contains the tuning parameters for an arm, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.
[NUI_HANDLES_TUNING_PARAMS_GLOBAL](#)
Defines global tuning parameters for handles, to customize the width and height of the physical interaction zone (PHIZ).
[NUI_HANDLES_TUNING_PARAMS_GRIP](#)
Specifies how long an arm needs to be in an attach zone or a detach zone before the arm attaches to or detaches from a grip.

 **See Also**

[NUI Handles Overview](#)

NUI_HANDLES_APPROACH_INFO Structure

[See Also](#)



Describes a handle's approach, including the distance between grip and hand, the attach zone the arm is in, and whether the arm is attached to the grip.

Syntax

```
C++
typedef struct _NUI_HANDLES_APPROACH_INFO {
    PNUI_HANDLES_GRIP pGrip;
    NUI_HANDLES_ARMS_HANDEDNESS eArmHandedness;
    XMVECTOR vFalloffVector;
    NUI_HANDLES_APPROACH_RANGE eApproachRange;
    FLOAT fAttachProgress;
    BOOL bIsAttached;
} NUI_HANDLES_APPROACH_INFO, *PNUI_HANDLES_APPROACH_INFO;
```

Members

- pGrip**
The grip that is being approached, as a pointer to a [NUI_HANDLES_GRIP](#) structure.
- eArmHandedness**
The hand (left or right) that is approaching the grip, as a [NUI_HANDLES_ARMS_HANDEDNESS](#) enumeration constant.
- vFalloffVector**
The distance between the grip and hand, as an [XMVECTOR](#), normalized to the size of the grip. A distance near 0 means the arm is nearly in the fast-attach zone. A distance near 1.0 means the arm is nearly in the slow-detach zone.
- eApproachRange**
The attach, approach, or detach zone that the arm is in, as a [NUI_HANDLES_APPROACH_RANGE](#) enumeration constant.
- fAttachProgress**
How soon the arm will attach to the grip or detach from the grip. Numbers near 0, when attached, mean the arm will soon detach from the grip. Numbers near 1.0, when detached, mean the arm will soon attach to the grip.
- bIsAttached**
Whether the arm is attached to the grip. If you are calling [NuiHandlesArmsGetApproachInfo](#), indicates whether the specified arm is attached to a grip. If you are calling [NuiHandlesGripGetApproachInfo](#), indicates whether the specified grip is attached to an arm.

Remarks

This structure is returned by [NuiHandlesArmsGetApproachInfo](#) and [NuiHandlesGripGetApproachInfo](#).

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NUI_HANDLES_ARMS Structure

[See Also](#)



Represents a handle's arms.

Syntax

```
C++
typedef struct _NUI_HANDLES_ARMS {
    XMVECTOR Reserved[NUI_HANDLES_ARMS_RESERVED_VECTOR_COUNT];
} NUI_HANDLES_ARMS, *PNUI_HANDLES_ARMS;
```

Members

- Reserved**
Reserved, implementation-specific data. Do not use this memory.

Remarks

This structure is an input parameter to almost all of the **NuiHandlesArm...** functions, such as [NuiHandlesArmGetScreenSpaceLocation](#).
This structure stores information about a handle's arms, using an opaque data structure.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

Version: 21256.3

NUI_HANDLES_BODY_SPACE Structure

[See Also](#)



Describes the body space coordinate system relative to camera space.

Syntax

C++

```
typedef struct _NUI_HANDLES_BODY_SPACE {
    XMVECTOR vOrigin;
    XMVECTOR vForward;
    XMVECTOR vRight;
    XMVECTOR vUp;
    XMVECTOR vRightShoulder;
    XMVECTOR vLeftShoulder;
    XMVECTOR vRefinedRightHand;
    XMVECTOR vRefinedLeftHand;
} NUI_HANDLES_BODY_SPACE, *PNUI_HANDLES_BODY_SPACE;
```

Members

- vOrigin**
The origin of the body space coordinate system as it is positioned in camera space, as an [XMVECTOR](#). This point is the origin of the body and, with unified PHIZ, this is also the origin of the PHIZ. This field only applies to unified PHIZ. See [The Physical Interaction Zone \(PHIZ\)](#).
- vForward**
The forward vector of the body space coordinate system as it is positioned in camera space.
- vRight**
The right vector of the body space coordinate system as it is positioned in camera space.
- vUp**
The up vector of the body space coordinate system as it is positioned in camera space.
- vRightShoulder**
The position of the right shoulder in camera space.
- vLeftShoulder**
The position of the left shoulder in camera space.
- vRefinedRightHand**
The position of the refined right-hand position in camera space.
- vRefinedLeftHand**
The position of the refined left-hand position in camera space.

Remarks

This structure is returned by [NuiHandlesArmsGetBodySpace](#).
vForward, *vRight*, and *vUp* don't change based on whether you are looking at unified, left-handed, or right-handed, because this structure describes the body space, not the PHIZ.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_GRIP Structure

[See Also](#)



Represents a grip.

Syntax

C++

```
typedef struct _NUI_HANDLES_GRIP {
    XMVECTOR Reserved[NUI_HANDLES_GRIP_RESERVED_VECTOR_COUNT];
} NUI_HANDLES_GRIP, *PNUI_HANDLES_GRIP;
```

Members

- Reserved**
Reserved, implementation-specific data. Do not use this memory.

Remarks

This structure is an input parameter to almost all of the **NuiHandlesGrip...** functions, such as [NuiHandlesGripGetScreenSpaceLocation](#).
This structure stores information about a grip, using an opaque data structure.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_GRIPSHAPE Structure

[See Also](#)



Specifies the size of the attach, approach, and detach zones of a grip.

Syntax

C++

```
typedef struct _NUI_HANDLES_GRIPSHAPE {
    FLOAT fFastAttachRadius;
    FLOAT fSlowAttachRadius;
    FLOAT fApproachRadius;
    FLOAT fSlowDetachRadius;
} NUI_HANDLES_GRIPSHAPE, *PNUI_HANDLES_GRIPSHAPE;
```

Members

fFastAttachRadius

The radius of the "fast attach" zone, in screen space units.

The "fast attach" zone is the size of the grip, as the radius of the grip. In the "fast attach" zone, the distance between the grip and the arm is very close, and the arm is quickly attached to the grip.

Valid values range from 0 through pi.

fSlowAttachRadius

The radius of the "slow attach" zone, in screen space units.

In the "slow attach" zone, the distance between the grip and arm is moderately close, and the arm is slowly attached to the grip.

Valid values range from 0 through pi.

fApproachRadius

The radius of the "approach" zone, in screen space units.

In the "approach" zone, the distance between the grip and arm is close enough so as to neither attach nor detach. If the arm is attached to the grip, it remains attached. If the arm is detached from the grip, it remains detached. While the arm is in the approach zone, the approach progress ([NUI_HANDLES_APPROACH_INFO.fAttachProgress](#)) decreases over the cooldown period of the arm ([NUI_HANDLES_TUNING_PARAMS.ARMS.dwCooldownTimeInMilliseconds](#)), but regardless of the attach progress, the arm will never transition between attached and detached.

Valid values range from 0 through pi.

fSlowDetachRadius

The radius of the "slow detach" zone, in screen space units.

In the "slow detach" zone, the distance between the grip and the arm is moderately far, and the arm is slowly detached from the grip.

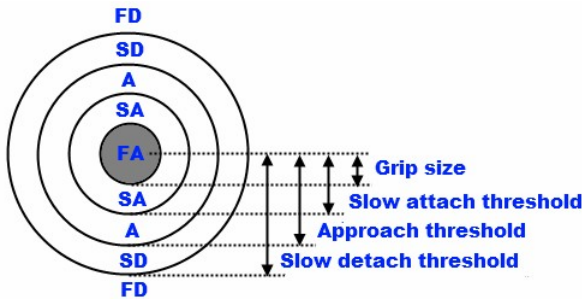
Valid values range from 0 through pi.

Remarks

This structure is returned by [NuiHandlesGripGetShape](#) and passed to [NuiHandlesGripSetShape](#).

The "fast detach" zone is the entire area outside the "slow detach" zone. In the "fast detach" zone, the distance between the grip and arm is very far, and the arm is quickly detached from the grip.

Figure 1. Parameters for handles



The attach zones are shown above.

- FA = fast attach zone
- SA = slow attach zone
- A = approach zone
- SD = slow detach zone
- FD = fast detach zone

Requirements

Header: Declared in [NuiHandles.h](#).

Library: Use [NuiHandles.lib](#).

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_PLAYER_INFO Structure

[See Also](#)



Contains identity information about the associated player, including tracking ID and user index.

Syntax

C++

```
typedef struct _NUI_HANDLES_PLAYER_INFO {
    DWORD dwTrackingID;
    DWORD dwUserIndex;
} NUI_HANDLES_PLAYER_INFO, *PNUI_HANDLES_PLAYER_INFO;
```

Members

- dwTrackingID**
The tracking ID assigned to this tracked player.
The ID is associated with a particular arm. You can query a grip to determine which player was last attached to the grip. **dwTrackingID** is a member of the [NUI_SKELETON_DATA](#) structure and is described in detail there.
- dwUserIndex**
If this arm or grip is associated with a user index, a value in the range 0 through 3. Otherwise, [XUSER_INDEX_NONE](#).

Remarks

This structure is returned by [NuiHandlesArmsGetPlayerInfo](#) and [NuiHandlesGripGetLastAttachPlayerInfo](#).

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_RAILSTATE Structure

[See Also](#)



Describes whether the specified rail is enabled and committed, and the player's current position on the rail.

Syntax

C++

```
typedef struct _NUI_HANDLES_RAILSTATE {
    BOOL bEnabled;
    BOOL bCommitted;
    DWORD dwRailID;
    union {
        NUI_HANDLES_RAILSTATE_ANALOGSTICK AnalogStick;
        NUI_HANDLES_RAILSTATE_ARC Arc;
        NUI_HANDLES_RAILSTATE_GEARSHIFT Gearshift;
        NUI_HANDLES_RAILSTATE_HOVERTIMER HoverTimer;
    };
} NUI_HANDLES_RAILSTATE, *PNUI_HANDLES_RAILSTATE;
```

Members

- bEnabled**
When TRUE, the rail is enabled.
- bCommitted**
When TRUE, the rail is committed, meaning that the player has begun moving on this rail.
- dwRailID**
The rail ID that was assigned when the rail was created.
- AnalogStick**
If this rail is an analog joystick rail, the [NUI_HANDLES_RAILSTATE_ANALOGSTICK](#) describing it.
- Arc**
If this rail is an arc rail, the [NUI_HANDLES_RAILSTATE_ARC](#) describing it.
- Gearshift**
If this rail is a gearshift rail, the [NUI_HANDLES_RAILSTATE_GEARSHIFT](#) describing it.
- HoverTimer**
If this rail is a hover timer, the [NUI_HANDLES_RAILSTATE_HOVERTIMER](#) describing it.

Remarks

This structure is returned by [NuiHandlesGripGetRailState](#).

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_RAILSTATE_ANALOGSTICK Structure

[See Also](#)



Indicates the current position of an analog joystick rail.

+

Syntax

C++

```
typedef struct _NUI_HANDLES_RAILSTATE_ANALOGSTICK {
    FLOAT fAngleInRadians;
    FLOAT fDistance;
} NUI_HANDLES_RAILSTATE_ANALOGSTICK, *PNUI_HANDLES_RAILSTATE_ANALOGSTICK;
```

Members

fAngleInRadians

The circular direction the player is pointing with respect to the center of the circle.
Pointing a little to the right of the center of the circle and pointing all the way to the right side of the circle yield the same value: 0 radians. Pointing a little above the center of the circle and pointing all the way to the top of the circle yield the same angle: pi/2 radians.
Valid values range from 0 to 2×pi.

fDistance

How far between the center of the circle and the radius of the circle the player is pointing, as a ratio.
When the player is pointing at the center of the circle, the distance is 0. When the player is pointing anywhere on the radius of the circle, the distance is 1.0.
Valid values range from 0 to 1.0.

+

Remarks

This structure is queried by calling [NuiHandlesGripGetRailState](#).
An analog joystick rail consists of a circular boundary around a central point. Position (progress) is measured as the deviation of the player's arm position from the central point, in polar coordinates, within the circular boundary.

+

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

+

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_RAILSTATE_ARC Structure

[See Also](#)



Indicates the player's progress along an arc rail.

+

Syntax

C++

```
typedef struct _NUI_HANDLES_RAILSTATE_ARC {
    FLOAT fProgress;
    FLOAT fDenormalizedProgress;
} NUI_HANDLES_RAILSTATE_ARC, *PNUI_HANDLES_RAILSTATE_ARC;
```

Members

fProgress

The player's progress along the rail, normalized so that 0 is the minimum angle and 1.0 is the maximum angle.

fDenormalizedProgress

The player's progress along the rail, in radians from the minimum angle.
Valid values range from 0 through pi.

+

Remarks

This structure is queried by calling [NuiHandlesGripGetRailState](#).
An arc rail consists of a single track, with progress measured from one end to the other. When a player's hand deviates from the track, the hand is still tracked, but the progress response to the hand's motion is damped to provide feedback.

+

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

+

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_RAILSTATE_GEARSHIFT Structure

[See Also](#)



Indicates the status of a gearshift rail, including the progress along the spine, which tooth is committed, and the progress along the tooth.

Syntax

C++

```
typedef struct _NUI_HANDLES_RAILSTATE_GEARSHIFT {
    INT nIndex;
    FLOAT fToothProgress;
    FLOAT fSpineProgress;
    FLOAT fSpineProgressAtToothCommit;
} NUI_HANDLES_RAILSTATE_GEARSHIFT, *PNUI_HANDLES_RAILSTATE_GEARSHIFT;
```

Members

- nIndex**
The index of the tooth that is committed.
- fToothProgress**
The progress along the tooth, normalized to the interval [0 .. 1.0].
- fSpineProgress**
The progress along the primary spine, normalized to the interval [0 .. 1.0].
- fSpineProgressAtToothCommit**
The progress along the primary spine when the tooth was initially committed, normalized to the interval [0 .. 1.0].

Remarks

This structure is queried by calling [NuiHandlesGripGetRailState](#).
A gearshift rail consists of a primary spine and a number of teeth perpendicular to that spine. The overall shape resembles a comb, where the back of the comb is the primary spine, with perpendicular teeth attached.

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_RAILSTATE_HOVERTIMER Structure

[See Also](#)



Indicates the length of time an arm has been attached to a hover timer, and the percent progress of the hover timer.

Syntax

C++

```
typedef struct _NUI_HANDLES_RAILSTATE_HOVERTIMER {
    DWORD dbTimePassedInMilliseconds;
    FLOAT fPercentageComplete;
} NUI_HANDLES_RAILSTATE_HOVERTIMER, *PNUI_HANDLES_RAILSTATE_HOVERTIMER;
```

Members

- dbTimePassedInMilliseconds**
The length of time that the arm has been attached to the hover timer, in milliseconds.
- fPercentageComplete**
How long the hand has been attached (committed) to the grip, compared to the set values for *dwTotalHoverTimeInMilliseconds* minus *dwHoverTimeToCommitInMilliseconds*.
- A value of 0 means that the commit time you set hasn't been reached, or has just been reached.
 - A value between 0 and 1 means that the commit time has been reached and exceeded, but that the specified total hover time you set hasn't been reached.
 - A value of 1 means that the *dwTotalHoverTimeInMilliseconds* you set has been reached or exceeded.
- The total time to reach the end of the hover timer is specified in the *dwTotalHoverTimeInMilliseconds* parameter of [NuiHandlesGripAddRailHoverTimer](#).

Remarks

This structure is queried by calling [NuiHandlesGripGetRailState](#).

Requirements

- Header:** Declared in NuiHandles.h.
- Library:** Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)
[Time-based Constraints with the Hover Timer](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_TUNING_PARAMS_ARMS Structure

[See Also](#)



Contains the tuning parameters for an arm, including how long to wait between detaching and attaching, and how far a hand is extended before the hand is attached to or detached from a grip.

Syntax

C++

```
typedef struct _NUI_HANDLES_TUNING_PARAMS_ARMS {
    DWORD dwCooldownTimeInMilliseconds;
    DWORD dwReattachTimeInMilliseconds;
    FLOAT fPreAttachExtentDistance;
    FLOAT fAttachExtentDistance;
    FLOAT fDetachExtentDistance;
} NUI_HANDLES_TUNING_PARAMS_ARMS, *PNUI_HANDLES_TUNING_PARAMS_ARMS;
```

Members**dwCooldownTimeInMilliseconds**

The time it takes for the arm's progress to go from 1.0 to 0, while the arm is in the approach zone.

While in the approach zone, the arm will not become detached, even if the arm's detach progress reaches 0.

dwReattachTimeInMilliseconds

The time after a forced detach before you can attach again.

fPreAttachExtentDistance

The extent distance that describes the threshold at which an arm can start making attach progress.

If the arm's extent distance is between *fAttachExtentDistance* and *fPreAttachExtentDistance*, over time, the corresponding arm's attach progress will increase towards 1.0. However, while in this "pre-attach" zone, the arm cannot become attached.

Valid values range from 0 through 1.0.

fAttachExtentDistance

The extent distance that describes the threshold at which an arm become attached.

If the arm's extent distance is smaller than *fAttachExtentDistance*, over time, the corresponding arm's attach progress will increase towards 1.0 and when it reaches 1.0, will become attached. If an arm has an attach progress of 1.0 and moves into this area, it will attach automatically.

Valid values range from 0 through 1.0.

fDetachExtentDistance

The extent distance that describes the threshold at which the arm will become detached.

If the extent distance is greater than *fDetachExtentDistance*, the arm will detach from any grip to which it is attached currently.

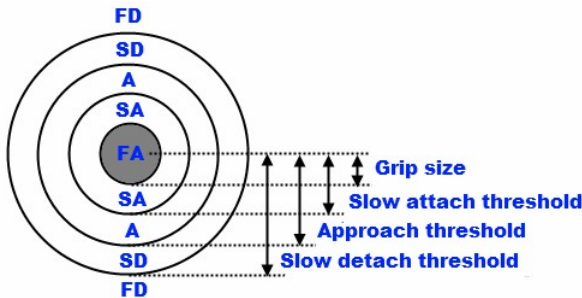
Valid values range from 0 through 1.0.

Remarks

This structure is returned by [NuiHandlesArmsGetDefaultTuningParams](#) and [NuiHandlesArmsGetTuningParams](#), and is passed to [NuiHandlesArmsSetTuningParams](#).

An *extent distance* is based on the proportion of the distance from the shoulder to the refined position of the hand. An extent distance of 0 corresponds to the hand being fully extended out from the shoulder. An extent distance of 1.0 corresponds to the hand being at the shoulder.

Figure 1. Parameters for handles



The attach zones are shown above.

- FA = fast attach zone
- SA = slow attach zone
- A = approach zone
- SD = slow detach zone
- FD = fast detach zone

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_TUNING_PARAMS_GLOBAL Structure

[See Also](#)



Defines global tuning parameters for handles, to customize the width and height of the physical interaction zone (PHIZ).

Syntax**C++**

```
typedef struct _NUI_HANDLES_TUNING_PARAMS_GLOBAL {
    FLOAT fPhizWidthInRadians;
    FLOAT fPhizHeightInRadians;
    FLOAT fPhizHorizontalAlignmentInRadians;
    FLOAT fPhizVerticalOffsetInRadians;
    FLOAT fRequiredUserAlignmentInRadians;
} NUI_HANDLES_TUNING_PARAMS_GLOBAL, *PNUI_HANDLES_TUNING_PARAMS_GLOBAL;
```

Members

fPhizWidthInRadians

The valid horizontal range through which the arm's position ray is allowed to move.
When the arm is outside of this range, the NuiHandles library ignores the arm.
Valid values range from 0 through pi.

fPhizHeightInRadians

The valid vertical range through which the arm's position ray is allowed to move.
When the arm is outside of this range, the NuiHandles library ignores the arm.
Valid values range from 0 through pi.

fPhizHorizontalAlignmentInRadians

A horizontal angle that defines the neutral position of the arm.
Horizontal movements in the PHIZ are measured from this neutral position.
Valid values range from 0 through pi.

fPhizVerticalOffsetInRadians

A vertical angle that defines the neutral position of the arm.
Vertical movements in the PHIZ are measured from this neutral position.
Valid values range from 0 through pi.

fRequiredUserAlignmentInRadians

The maximum angle the body can be turned, beyond which the player is no longer interpreted as signaling intent to engage a handle.
If the player's body is turned beyond this angle, the NuiHandles library will ignore the player's input.
Valid values range from 0 through pi.

Remarks

This structure is returned by [NuiHandlesGlobalGetDefaultTuningParams](#) and [NuiHandlesGlobalGetTuningParams](#), and passed to [NuiHandlesGlobalSetTuningParams](#).
To customize the PHIZ, pass this structure to [NuiHandlesGlobalSetTuningParams](#).

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_TUNING_PARAMS_GRIP Structure

[See Also](#)



Specifies how long an arm needs to be in an attach zone or a detach zone before the arm attaches to or detaches from a grip.

Syntax

C++

```
typedef struct _NUI_HANDLES_TUNING_PARAMS_GRIP {
    DWORD dwSlowAttachTimeInMilliseconds;
    DWORD dwFastAttachTimeInMilliseconds;
    DWORD dwSlowDetachTimeInMilliseconds;
    DWORD dwFastDetachTimeInMilliseconds;
} NUI_HANDLES_TUNING_PARAMS_GRIP, *PNUI_HANDLES_TUNING_PARAMS_GRIP;
```

Members

dwSlowAttachTimeInMilliseconds

The time the arm needs to be located in the slow attach zone before the arm is attached to the grip.
Valid values range from 50 to 1500.

dwFastAttachTimeInMilliseconds

The time the arm needs to be located in the fast attach zone before the arm is attached to the grip.
Valid values range from 50 to 1000.

dwSlowDetachTimeInMilliseconds

The time the arm needs to be located in the slow detach zone before the arm is detached from the grip.
Valid values range from 500 to 3000.

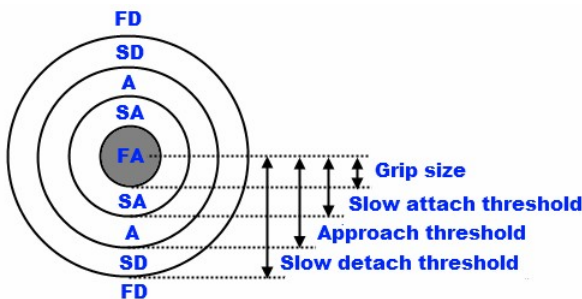
dwFastDetachTimeInMilliseconds

The time the arm needs to be located in the fast detach zone before the arm is detached from the grip.
Valid values range from five to 200.

Remarks

This structure is returned by [NuiHandlesGripGetTuningParams](#) and passed to [NuiHandlesGripSetTuningParams](#).
To get the default tuning parameters for a grip, call [NuiHandlesGripGetDefaultTuningParams](#).

Figure 1. Parameters for handles



The attach zones are shown above.

- FA = fast attach zone
- SA = slow attach zone
- A = approach zone
- SD = slow detach zone
- FD = fast detach zone

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandles Enumerations

[See Also](#)

In This Section

[NUI_HANDLES_APPROACH_RANGE](#)

Specifies the distance between a grip and an arm, in terms of attach, approach, or detach zones.

[NUI_HANDLES_ARM_STATUS](#)

Specifies the quality of the arm data, indicating the level of confidence in the reported arm position.

[NUI_HANDLES_ARMS_HANDEDNESS](#)

Specifies a left or right arm.

[NUI_HANDLES_GRIP_HANDEDNESS](#)

Specifies the handedness of a grip as left, right, or ambidextrous.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_APPROACH_RANGE Enumeration

[See Also](#)

Specifies the distance between a grip and an arm, in terms of attach, approach, or detach zones.

Syntax

C++

```
typedef enum _NUI_HANDLES_APPROACH_RANGE
{
    NUI_HANDLES_APPROACH_RANGE_FAST_ATTACH = 0,
    NUI_HANDLES_APPROACH_RANGE_SLOW_ATTACH,
    NUI_HANDLES_APPROACH_RANGE_APPROACH,
    NUI_HANDLES_APPROACH_RANGE_SLOW_DETACH,
    NUI_HANDLES_APPROACH_RANGE_FAST_DETACH,
    NUI_HANDLES_APPROACH_RANGE_NOT_EXTENDED
} NUI_HANDLES_APPROACH_RANGE;
```

Constants

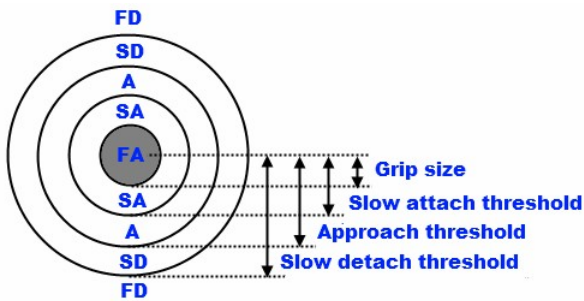
Constant	Description
NUI_HANDLES_APPROACH_RANGE_FAST_ATTACH	The "fast attach" range or zone. This range is the size of the grip, as the radius of the grip, in screen space units. Indicates that the distance between the grip and the arm is very close, so as to quickly attach the arm to the grip.
NUI_HANDLES_APPROACH_RANGE_SLOW_ATTACH	The "slow attach" range. Measured in screenspace units. Indicates that the distance between the grip and arm is moderately close, so as to slowly attach the arm to the grip.
NUI_HANDLES_APPROACH_RANGE_APPROACH	The "approach" range. Measured in screenspace units. Indicates that the distance between the grip and arm is close enough so as to neither attach nor detach. If attached, will remain attached; if detached, will remain detached.

NUI_HANDLES_APPROACH_RANGE_SLOW_DETACH	The "slow detach" range. Measured in screenspace units. Indicates that the distance between the grip and the arm is moderately far, so as to slowly detach the arm from the grip.
NUI_HANDLES_APPROACH_RANGE_FAST_DETACH	The "fast detach" range. Measured in screenspace units. Indicates that the distance between the grip and arm is very far, so as to quickly detach the arm from the grip.
NUI_HANDLES_APPROACH_RANGE_NOT_EXTENDED	The "not extended" state. Indicates that the player's hand is not extended away from the body far enough to reliably determine the distance from the hand to the grip. Behaves the same as NUI_HANDLES_APPROACH_RANGE_FAST_DETACH.

Remarks

A [NUI_HANDLES_APPROACH_INFO](#) structure contains one of these enumeration constants and is returned by [NuiHandlesArmsGetApproachInfo](#) and [NuiHandlesGripGetApproachInfo](#).

Figure 1. Parameters for handles



The attach zones are shown above.

- FA = fast attach zone
- SA = slow attach zone
- A = approach zone
- SD = slow detach zone
- FD = fast detach zone

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_ARM_STATUS Enumeration

[See Also](#)



Specifies the quality of the arm data, indicating the level of confidence in the reported arm position.

Syntax

```
C++
typedef enum _NUI_HANDLES_ARM_STATUS
{
    NUI_HANDLES_ARM_STATUS_ENGAGED = 0,
    NUI_HANDLES_ARM_STATUS_NOT_TRACKED,
    NUI_HANDLES_ARM_STATUS_INVALID_ARM_POS,
    NUI_HANDLES_ARM_STATUS_USER_NOT_FACING_DEVICE
} NUI_HANDLES_ARM_STATUS;
```

Constants

Constant	Description
NUI_HANDLES_ARM_STATUS_ENGAGED	The arm is engaged. This is the only condition that allows you to attach to or manipulate a handle.
NUI_HANDLES_ARM_STATUS_NOT_TRACKED	The arm is not tracked.
NUI_HANDLES_ARM_STATUS_INVALID_ARM_POS	The arm position is invalid. The system is not confident in the quality of the skeleton. This is a catch-all for various error states.
NUI_HANDLES_ARM_STATUS_USER_NOT_FACING_DEVICE	The player is not facing the Kinect sensor array.

Remarks

[NuiHandlesArmsGetArmStatus](#) returns one of these enumeration constants.

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NUI_HANDLES_ARMS_HANDEDNESS Enumeration

[See Also](#)



Specifies a left or right arm.

Syntax

```
C++
typedef enum _NUI_HANDLES_ARMS_HANDEDNESS
{
    NUI_HANDLES_ARMS_HANDEDNESS_DEFAULT = 0,
    NUI_HANDLES_ARMS_HANDEDNESS_LEFT_ARM = 0,
    NUI_HANDLES_ARMS_HANDEDNESS_RIGHT_ARM,
    NUI_HANDLES_ARMS_HANDEDNESS_COUNT
} NUI_HANDLES_ARMS_HANDEDNESS;
```

Constants

Constant	Description
NUI_HANDLES_ARMS_HANDEDNESS_DEFAULT	By default, the left arm is used.
NUI_HANDLES_ARMS_HANDEDNESS_LEFT_ARM	The left arm.
NUI_HANDLES_ARMS_HANDEDNESS_RIGHT_ARM	The right arm.
NUI_HANDLES_ARMS_HANDEDNESS_COUNT	You can use this automatically determined value as an upper bound for validating variables of this enum type. For example, if you are serializing data in and out of a file or buffer, and reading out each value as a DWORD value, you can validate the value against this member before you cast the value back to this enum type. This enables you to avoid an exception being thrown because an enum value is too high.

Remarks

The [NUI_HANDLES_APPROACH_INFO](#) structure contains one of these enumeration constants.

The following functions take one of these enumeration constants as an input parameter:

- [NuiHandlesArmGetEnabled](#)
- [NuiHandlesArmGetPositionRay](#)
- [NuiHandlesArmGetScreenSpaceLocation](#)
- [NuiHandlesArmGetSensorToPhizTransformationMatrix](#)
- [NuiHandlesArmSetEnabled](#)
- [NuiHandlesArmSetPositionRay](#)
- [NuiHandlesArmsGetApproachInfo](#)
- [NuiHandlesArmsIsAttached](#)

Requirements

Header: Declared in NuiHandles.h.

Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

NUI_HANDLES_GRIP_HANDEDNESS Enumeration

[See Also](#)



Specifies the handedness of a grip as left, right, or ambidextrous.

Syntax

```
C++
typedef enum _NUI_HANDLES_GRIP_HANDEDNESS
{
    NUI_HANDLES_GRIP_HANDEDNESS_DEFAULT = 0,
    NUI_HANDLES_GRIP_HANDEDNESS_AMBIDEXTROUS = 0,
    NUI_HANDLES_GRIP_HANDEDNESS_LEFT_ARM,
    NUI_HANDLES_GRIP_HANDEDNESS_RIGHT_ARM,
    NUI_HANDLES_GRIP_HANDEDNESS_COUNT
} NUI_HANDLES_GRIP_HANDEDNESS;
```

Constants

Constant	Description
NUI_HANDLES_GRIP_HANDEDNESS_DEFAULT	The default handedness of a grip, which is NUI_HANDLES_GRIP_HANDEDNESS_AMBIDEXTROUS.
NUI_HANDLES_GRIP_HANDEDNESS_AMBIDEXTROUS	The grip is ambidextrous.
NUI_HANDLES_GRIP_HANDEDNESS_LEFT_ARM	The grip is left-handed; only the left hand can attach to the grip.
NUI_HANDLES_GRIP_HANDEDNESS_RIGHT_ARM	The grip is right-handed; only the right hand can attach to the grip.

NUI_HANDLES_GRIP_HANDEDNESS_COUNT

You can use this automatically determined value as an upper bound for validating variables of this enum type. For example, if you are serializing data in and out of a file or buffer, and reading out each value as a DWORD value, you can validate the value against this member before you cast the value back to this enum type, to avoid an exception being thrown due to too high of an enum value.

Remarks

[NuiHandlesGripGetHandedness](#) returns one of these enumeration constants. [NuiHandlesGripSetHandedness](#) takes one of these enumeration constants.

Requirements

Header: Declared in NuiHandles.h.
Library: Use NuiHandles.lib.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHandles Constants

[See Also](#)

In This Section

[NUI_HANDLES_ARMS_RESERVED_VECTOR_COUNT](#)

The size of the NUI_HANDLES_ARMS structure.

[NUI_HANDLES_GRIP_RESERVED_VECTOR_COUNT](#)

The size of the NUI_HANDLES_GRIP structure.

[NUI_HANDLES_INVALID_RAIL_ID](#)

The ID to use for a rail ID that is invalid.

[NUI_HANDLES_MAX_RAILS_PER_GRIP](#)

The maximum number of rails that can be attached to one grip.

[NUIHANDLES_SEATED_GLOBAL_PARAMETERS](#)

Recommended size and position of the physical interaction zone (PHIZ) for use with the seated (upper body) skeleton tracking pipeline.

[NUIHANDLES_STANDING_GLOBAL_PARAMETERS](#)

Recommended size and position of the physical interaction zone (PHIZ) for use with the standing (full-body) skeleton tracking pipeline.

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_ARMS_RESERVED_VECTOR_COUNT

[See Also](#)

The size of the NUI_HANDLES_ARMS structure.

Name	Description
NUI_HANDLES_ARMS_RESERVED_VECTOR_COUNT	The size of the opaque NUI_HANDLES_ARMS structure data, in XMVECTOR .

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_GRIP_RESERVED_VECTOR_COUNT

[See Also](#)

The size of the NUI_HANDLES_GRIP structure.

Name	Description
NUI_HANDLES_GRIP_RESERVED_VECTOR_COUNT	The size of the opaque NUI_HANDLES_GRIP structure data, in XMVECTOR .

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_INVALID_RAIL_ID

[See Also](#)

The ID to use for a rail ID that is invalid.

Name	Description
NUI_HANDLES_INVALID_RAIL_ID	The ID to use for a rail ID that is invalid.

+

Remarks

[NuiHandlesGripGetCommittedRailId](#) can return this value.
[NuiHandlesGripAddRailGearShift](#) returns an error code if you pass in this value as the *dwRailID*.

+

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HANDLES_MAX_RAILS_PER_GRIP

[See Also](#)

The maximum number of rails that can be attached to one grip.

Name	Description
NUI_HANDLES_MAX_RAILS_PER_GRIP	The maximum number of rails that can be attached to one grip.

+

Remarks

The system checks against this value when you call the following functions:

- [NuiHandlesGripAddRailAnalogStick](#)
- [NuiHandlesGripAddRailArc](#)
- [NuiHandlesGripAddRailGearShift](#)

+

See Also

[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIHANDLES_SEATED_GLOBAL_PARAMETERS

[See Also](#)

Recommended size and position of the physical interaction zone (PHIZ) for use with the seated (upper body) skeleton tracking pipeline.
These are part of the default tuning parameters as get and set by [NuiHandlesGlobalGetTuningParams](#) and [NuiHandlesGlobalSetTuningParams](#). You can specify these values to populate a [NUI_HANDLES_TUNING_PARAMS_GLOBAL](#) structure.

Name	Description
NUIHANDLES_SEATED_GLOBAL_PARAMETERS_PHIZ_WIDTH	The width of the physical interaction zone.
NUIHANDLES_SEATED_GLOBAL_PARAMETERS_PHIZ_HEIGHT	The height of the physical interaction zone.
NUIHANDLES_SEATED_GLOBAL_PARAMETERS_HORZ_ALIGNMENT	The horizontal position of the physical interaction zone.
NUIHANDLES_SEATED_GLOBAL_PARAMETERS_VERT_OFFSET	The vertical position of the physical interaction zone.

+

See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[Seated Skeleton Tracking](#)
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIHANDLES_STANDING_GLOBAL_PARAMETERS

[See Also](#)

Recommended size and position of the physical interaction zone (PHIZ) for use with the standing (full-body) skeleton tracking pipeline.
These are part of the default tuning parameters as get and set by [NuiHandlesGlobalGetTuningParams](#) and [NuiHandlesGlobalSetTuningParams](#). You can specify these values to populate a [NUI_HANDLES_TUNING_PARAMS_GLOBAL](#) structure.

Name	Description
NUIHANDLES_STANDING_GLOBAL_PARAMETERS_PHIZ_WIDTH	The width of the physical interaction zone.
NUIHANDLES_STANDING_GLOBAL_PARAMETERS_PHIZ_HEIGHT	The height of the physical interaction zone.
NUIHANDLES_STANDING_GLOBAL_PARAMETERS_HORZ_ALIGNMENT	The horizontal position of the physical interaction zone.
NUIHANDLES_STANDING_GLOBAL_PARAMETERS_VERT_OFFSET	The vertical position of the physical interaction zone.

See Also

[The Physical Interaction Zone \(PHIZ\)](#)
[NUI Handles Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientation Reference

[See Also](#)

This section contains the API Reference pages for items in the header file NuiHeadOrientation.h, which provides head orientation tracking, to detect the direction the player's head is facing. This is useful for aiming and camera control mechanisms. Head orientation tracking is 3-axis tracking, including roll, pitch, and yaw.

In This Section

[NuiHeadOrientation Functions](#)
[NuiHeadOrientation Structures](#)
[NuiHeadOrientation Constants](#)

See Also

[Head Orientation Tracking](#)
[Simple Head Tracking Sample](#)
[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientation Functions

[See Also](#)

In This Section

[NuiHeadOrientationEnable](#)
Enables head orientation tracking.

[NuiHeadOrientationDisable](#)
Disables GPU and CPU processing for head orientation tracking (does not deallocate memory resources).

[NuiHeadOrientationSetTrackedHeads](#)
Specifies which heads are in an active tracking state.

[NuiHeadOrientationGetNextFrame](#)
Returns the current head orientation frame, which contains the orientation tracking data for all the heads being tracked.

[NuiHeadOrientationSmooth](#)
Filters head orientation to reduce jitter between frames.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientationEnable

[See Also](#)

Enables head orientation tracking.

Syntax

C++

```
HRESULT NuiHeadOrientationEnable(  
    HANDLE hNextFrameEvent,  
    DWORD Flags  
)
```

Parameters

hNextFrameEvent
[in, optional] A handle to a title-allocated, manual reset event that will be set whenever a new frame of skeleton data is available, and will be reset whenever the latest frame data is returned. This parameter enables the title to specify an object that will be signaled when a new frame is ready. This can be NULL.

Flags
[in] [NUI_HEAD_ORIENTATION_FLAGS](#) that control head orientation tracking, bitwise-OR'd together.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize wasn't called with the flag NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION .
E_INVALIDARG	<i>hNextFrameEvent</i> is not a valid event handle, or the <i>Flags</i> argument contains invalid flags.

If the title uses both head position tracking and head orientation tracking, you must specify the flag `NUI_HEAD_ORIENTATION_FLAG_TITLE_SETS_ACTIVE_HEADS` in **NuiHeadOrientationEnable**. Head orientation tracking doesn't have a policy to automatically pick up a tracking ID to work on. If the title uses skeleton tracking and head orientation tracking, if the title doesn't specify the active heads, head orientation tracking always computes the active tracked trackingID.

E_NUI_ALREADY_INITIALIZED	Head orientation tracking is already enabled.
---------------------------	---

⊕ **Remarks**

Once enabled, the system will start generating head orientation tracking data for each frame.

If skeleton tracking is not active, calling this function does not immediately enable head orientation tracking. In this case, the title must manually specify which heads to track by calling [NuiHeadOrientationSetTrackedHeads](#), because there is no default tracking policy in non-skeleton tracking mode.

⊕ **Requirements**

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and st.lib.

⊕ **See Also**

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientationDisable

[See Also](#)



Disables GPU and CPU processing for head orientation tracking (does not deallocate memory resources).

⊕ **Syntax**

C++

```
HRESULT NuiHeadOrientationDisable()
```

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.

⊕ **Remarks**

You can use this function to only use head orientation tracking in specific sections of the game and avoid using the GPU and CPU for head orientation tracking when in other sections of the game.

Memory for head orientation tracking remains allocated when head orientation tracking is disabled. To free the memory that was used for head orientation tracking, uninitialize NUI, using [NuiShutdown](#).

⊕ **Requirements**

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and st.lib.

⊕ **See Also**

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientationSetTrackedHeads

[See Also](#)



Specifies which heads are in an active tracking state.

⊕ **Syntax**

C++

```
HRESULT NuiHeadOrientationSetTrackedHeads(  
    DWORD TrackingIDs[NUI_HEAD_ORIENTATION_COUNT]  
)
```

Parameters

TrackingIDs
[in] A list of tracking IDs to make active, up to a maximum of [NUI_HEAD_ORIENTATION_COUNT](#). Specifying 0 for a slot turns off orientation processing for that slot.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize wasn't called with the flag NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION .

⊕ **Remarks**

In the context of head tracking, an active state means that orientation tracking is being performed on the head.

For this function to work, you must first pass the flag [NUI_HEAD_ORIENTATION_TITLE_SET_ACTIVE_HEADS](#) to [NuiHeadOrientationEnable](#).

The default policy for head orientation is tracking the two active skeletons. The default policy will be applied only when skeleton tracking is being used and the flag [NUI_HEAD_ORIENTATION_TITLE_SET_ACTIVE_HEADS](#) is not set on [NuiHeadOrientationEnable](#). **NuiHeadOrientationSetTrackedHeads** won't have any effect if the default policy is chosen, because the head orientation policy won't change after **NuiHeadOrientationEnable** is called.

✚ Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and st.lib.

✚ See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientationGetNextFrame

[See Also](#)

Returns the current head orientation frame, which contains the orientation tracking data for all the heads being tracked.

✚ Syntax

```
C++

HRESULT NuiHeadOrientationGetNextFrame(
    DWORD MillisecondsToWait,
    NUI_HEAD_ORIENTATION_FRAME *pHeadOrientationFrame
)
```

Parameters

MillisecondsToWait
[in] The time to wait before returning without the next frame of data. Identical to the skeletal tracking variant.

pHeadOrientationFrame
[out] The orientation tracking data for all currently tracked heads in the current frame, as a pointer to a title-allocated buffer holding a [NUI_HEAD_ORIENTATION_FRAME](#) structure. This pointer must be non-NULL when you call the function. After the function call returns, the buffer contains the head orientation tracking data. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize wasn't called with the flag NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION .
E_INVALIDARG	<i>pHeadOrientationFrame</i> is NULL.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor isn't connected.
E_NUI_STREAM_NOT_ENABLED	Head orientation tracking isn't enabled. See NuiHeadOrientationEnable .
E_NUI_SYSTEM_UI_PRESENT	The guide is being displayed.
E_NUI_FRAME_NO_DATA	This is a potentially fatal error condition. The frame isn't valid, and NUI_HEAD_ORIENTATION_FLAG_SUPPRESS_NO_FRAME_DATA isn't set.
E_PENDING	This function appears to be hung. This can happen when calling Present and NuiImageStreamGetNextFrame if the calls to those functions aren't paired.

✚ Remarks

This function is like [NuiSkeletonGetNextFrame](#).

✚ Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and st.lib.

✚ See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientationSmooth

[See Also](#)

Filters head orientation to reduce jitter between frames.

✚ Syntax

```
C++

HRESULT NuiHeadOrientationSmooth(
    NUI_HEAD_ORIENTATION_FRAME *pHeadOrientationFrame,
    CONST NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS *pSmoothingParams
)
```

Parameters

pHeadOrientationFrame
[in, out] On entry, points to a [NUI_HEAD_ORIENTATION_FRAME](#) structure that contains the head orientation data to be smoothed. On exit, the head orientation data in the structure has been replaced by smoothed data. If this function does not return S_OK, this data is unchanged.

pSmoothingParams
[in, optional] The parameters for the smoothing function, as a pointer to a [NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS](#) structure. To use the default values, pass NULL.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize wasn't called with the flag NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION .
E_INVALIDARG	<i>pHeadOrientationFrame</i> is NULL.

Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientation Structures

[See Also](#)

In This Section

- [NUI_HEAD_ORIENTATION_DATA](#)
Contains the head orientation tracking data for a head.
- [NUI_HEAD_ORIENTATION_FRAME](#)
Contains the orientation tracking data for all of the heads that are currently being tracked, for a single frame.
- [NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS](#)
Parameters for the smoothing function for head orientation across filter calls.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_DATA Structure

[See Also](#)

Contains the head orientation tracking data for a head.

Syntax

```
C++
typedef struct _NUI_HEAD_ORIENTATION_DATA {
    DWORD TrackingID;
    DWORD QualityFlags;
    DWORD Reserved0;
    DWORD Reserved1;
    XMVECTOR Orientation;
    RECT FaceBounds;
} NUI_HEAD_ORIENTATION_DATA;
```

Members

- TrackingID**
The tracking ID of the current head.
If skeleton tracking is running, this value matches the skeleton tracking ID of the corresponding skeleton that the head belongs to.
If skeleton tracking is not running, this ID is generated by the depth-based head detector, but has similar behavior to skeletal tracking IDs.
The tracking ID is a unique value that remains constant while a given head is visible to the camera.
Valid tracking IDs start at 1. 0 means there is no tracked head for this slot.
- QualityFlags**
[Head orientation quality flags](#) that report quality issues for head detection for the current frame, bitwise-OR'd together.
- Reserved0**
This member is reserved for system use.
- Reserved1**
This member is reserved for system use.
- Orientation**
The direction the head is facing in world space, as an [XMVECTOR Data Type](#) value with an (X,Y,Z) coordinate.
If skeleton tracking is running, this value comes from the head joint of the skeleton; otherwise, it is generated by the depth-based head detector using the center of mass. The W value is set to 1.0.
- FaceBounds**
The bounding rectangle (left, top, right, bottom) of the head in color image screen space based on 640x480.

Remarks

A [NUI_HEAD_ORIENTATION_FRAME](#) structure contains an array of **NUI_HEAD_ORIENTATION_DATA** structures.

Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[NUI_HEAD_POSITION_DATA](#)
[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_FRAME Structure

[See Also](#)

Contains the orientation tracking data for all of the heads that are currently being tracked, for a single frame.

Syntax

C++

```
typedef struct _NUI_HEAD_ORIENTATION_FRAME {
    UINT64 TimeStamp;
    DWORD FrameNumber;
    DWORD Reserved;
    NUI_HEAD_ORIENTATION_DATA HeadOrientationData[NUI_HEAD_ORIENTATION_COUNT];
} NUI_HEAD_ORIENTATION_FRAME;
```

Members

TimeStamp
The timestamp of the color and depth frames that were used to generate the orientation data.

FrameNumber
The frame number. This is the same as the frame number of the color and depth frames which were used to generate the orientation data.

Reserved
This member is reserved for system use.

HeadOrientationData
The head orientation tracking data for a single frame, as an array containing one [NUI_HEAD_ORIENTATION_DATA](#) structure for each head orientation tracking slot. There are [NUI_HEAD_ORIENTATION_COUNT](#) slots. Each NUI_HEAD_ORIENTATION_DATA structure contains the orientation tracking data for a single head.

Remarks

[NuiHeadOrientationGetNextFrame](#) returns a NUI_HEAD_POSITION_FRAME through the *pHeadOrientationFrame* output parameter.
One frame contains the head orientation for up to 2 players.

Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS Structure

[See Also](#)

Parameters for the smoothing function for head orientation across filter calls.

Syntax

C++

```
typedef struct _NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS {
    FLOAT fSmoothing;
    FLOAT fCorrection;
    FLOAT fPrediction;
    FLOAT fActiveZone;
    FLOAT fCenterDeadZone;
    FLOAT fEdgeDeadZone;
} NUI_HEAD_ORIENTATION_SMOOTH_PARAMETERS;
```

Members

fSmoothing
Smoothing parameter. Increasing the smoothing parameter value leads to more highly-smoothed head orientation values being returned.
As the smoothing value is increased, responsiveness to the raw data inherently decreases. Increased smoothing inherently leads to increased latency in the returned skeleton values.
Values must be in the range 0 through 1.0. Passing 0 causes the raw data to be returned.
Default: 0.5f

fCorrection
Correction parameter. Lower values are slower to correct towards the raw data, and appear smoother, while higher values will correct toward the raw data more quickly.
Values must be in the range 0 through 1.0.

Default: 0.3f

fPrediction

The number of frames to predict into the future.

Values must be greater than or equal to zero. Values greater than 0.5 will likely lead to overshooting when moving quickly.

Default: 0.3f

fActiveZone

The threshold at which change of head orientation is fully respected. Change of the head orientation across filter calls that's greater than this amount is fully respected (used) and is filtered for motion smoothing. Change that's less than this amount is ignored or partially respected. Unlike the dead zone values, this width amount doesn't vary depending on whether the head is centered or is angled.

Usually **fActiveZone** should be larger than **fCenterDeadZone** and larger than **fEdgeDeadZone**. If the active zone is smaller than the dead zone, the dead zone settings have no effect.

In radians. Default: XM_PI/45

fCenterDeadZone

The threshold at which change of head orientation is respected rather than ignored, when the head orientation is in the center (0) of its range of movement. Change of the head orientation across filter calls that's greater than this amount is partially respected or fully respected and is filtered for motion smoothing. Change that's less than this amount is ignored.

Usually **fCenterDeadZone** should be larger than **fEdgeDeadZone**, because more steadiness is desired when the head is centered.

In radians. Default: XM_PI/60

fEdgeDeadZone

The threshold at which change of head orientation is respected rather than ignored, when the head orientation is angled at a maximum. Change of the head orientation across filter calls that's greater than this amount is partially respected or fully respected and is filtered for motion smoothing. Change that's less than this amount is ignored.

In radians. Default: XM_PI/180

Remarks

What API Items Use This API Item

The [NuiHeadOrientationSmooth](#) function takes a pointer to this structure.

Components of Rotation Are Treated Independently

Each component of rotation - yaw, pitch, and roll - is independently analyzed across filter calls. For example, yaw in one filter call is compared against the yaw in the next filter call, and filtered for smooth motion, with no consideration of pitch or roll. Similarly, pitch and roll components are considered separately.

Active, Intermediate, and Dead Zones

[NuiHeadOrientationSmooth](#) uses a dead zone filter, forming 3 ranges:

- If the movement of the head between two filter calls is smaller than the dead zone, the movement is ignored.
- If the movement of the head between two filter calls is greater than the dead zone and less than the active zone, the movement is partially respected (used) and is sent through to be filtered using the specified smoothing parameters **fSmoothing**, **fCorrection**, and **fPrediction**. The amount of change is interpolated: If the change amount is slightly less than the active zone, the change will be almost fully respected. If the change amount is slightly more than the dead zone, the change will be almost ignored (the change will be slight).
- If the movement of the head between two filter calls is greater than the active zone, the movement is fully respected (used) and is sent through to be filtered using the specified smoothing parameters **fSmoothing**, **fCorrection**, and **fPrediction**.

Interpolating the Dead Zone Width between Center and Edge Angles

The width of the dead zone varies, depending on how far the head is turned. (Turning can be yaw, pitch, or roll.) The width of the dead zone at a given head angle is interpolated between the width of the dead zone at the center orientation (head not turned) and the dead zone value at the edge (head turned to the maximum one way or the other). For example, if the center dead zone is 3 degrees wide, and the edge dead zone is 1 degree wide, then when the head is half-turned, the dead zone is interpolated as 2 degrees wide.

The active zone doesn't vary depending on the head angle. Only the dead zone is a function of angle.

When the Default Values Are Used

If you pass NULL as the *pSmoothingParams* parameter when calling [NuiHeadOrientationSmooth](#), the default values are used.

Requirements

Header: Declared in NuiHeadOrientation.h. For include, use NuiApi.h.

Library: Use NuiApi.lib and st.lib.

See Also

[NUI_HEAD_POSITION_DATA](#)
[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadOrientation Constants

[See Also](#)

In This Section

[NUI_HEAD_ORIENTATION_COUNT](#)

The number of heads that can be tracked.

[NUI_HEAD_ORIENTATION_FLAGS](#)

Flags used for head orientation tracking.

[NUI_HEAD_ORIENTATION_QUALITY_FLAGS](#)

Flags that indicate problems preventing detection of the head orientation.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_COUNT

[See Also](#)

The number of heads that can be tracked.

Name	Description
NUI_HEAD_ORIENTATION_COUNT	The number of heads for which the head orientation can be tracked.

Remarks

The [NUI_HEAD_ORIENTATION_FRAME](#) structure contains an array of [NUI_HEAD_ORIENTATION_DATA](#) structures, one per head. The number of items in the array is `NUI_HEAD_ORIENTATION_COUNT`.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_FLAGS

[See Also](#)



Flags used for head orientation tracking.

These flags are used for head orientation tracking.

Name	Description
NUI_HEAD_ORIENTATION_FLAG_SUPPRESS_NO_FRAME_DATA	Suppresses NuiHeadOrientationGetNextFrame from returning an <code>E_NUI_FRAME_NO_DATA</code> error when a frame isn't available. Rather than returning the error <code>E_NUI_FRAME_NO_DATA</code>, calls to <code>NuiHeadOrientationGetNextFrame</code> block until data is available or the timeout period passes. If you pass this flag in the <i>dwFlags</i> argument to NuiHeadOrientationEnable, NuiHeadOrientationGetNextFrame will not return <code>E_NUI_FRAME_NO_DATA</code> errors. Instead, calls to NuiHeadOrientationGetNextFrame block until data is available or the timeout period passes. If you use this flag, you must never call NuiHeadOrientationGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap.
NUI_HEAD_ORIENTATION_FLAG_TITLE_SETS_ACTIVE_HEADS	Disables the default player selection mode and enables the title to manage which players have tracked head orientation. The default policy for head orientation is tracking the two active skeletons. This flag is used by NuiHeadOrientationEnable to disable the default player selection mode. After calling NuiHeadOrientationEnable with this flag, head orientation is not tracked, and the title must call NuiHeadOrientationSetTrackedHeads to manage which players will have head orientation tracked. NuiHeadOrientationDisable reverts to the default head orientation tracking policy. If this flag is not passed to NuiHeadOrientationEnable, the title cannot call NuiHeadOrientationSetTrackedHeads, and the default policy will remain in effect. If NuiHeadOrientationSetTrackedHeads is called without setting this flag, the call will fail with <code>E_NUI_FEATURE_NOT_INITIALIZED</code> and output a descriptive message in debug builds.

Remarks

These flags are passed to [NuiHeadOrientationEnable](#).

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_ORIENTATION_QUALITY Flags

[See Also](#)



Flags that indicate problems preventing detection of the head orientation.

Name	Description
NUI_HEAD_ORIENTATION_QUALITY_FACE_DETECT_FAILURE	The face detector was unable to lock onto a face. Either there is insufficient lighting, the user is looking too far away from the sensor, or something is occluding the face. If this flag is set, it means that the Orientation and FaceBounds members in the <code>NUI_HEAD_ORIENTATION_DATA</code> structure are invalid.
NUI_HEAD_ORIENTATION_QUALITY_HANDS_OCCLUDING_FACE	The player's hands are blocking the view of the player's face. Orientation data is not available. This flag only triggers when skeleton tracking is running.
NUI_HEAD_ORIENTATION_QUALITY_CLOSE_TO_TOP	Ignore; planned for deletion. The player's head is too close to the top of the field of view.
NUI_HEAD_ORIENTATION_QUALITY_CLOSE_TO_BOTTOM	Ignore; planned for deletion. The player's head is too close to the bottom of the field of view.
NUI_HEAD_ORIENTATION_QUALITY_CLOSE_TO_RIGHT	Ignore; planned for deletion. The player's head is too close to the right side of the field of view.
NUI_HEAD_ORIENTATION_QUALITY_CLOSE_TO_LEFT	Ignore; planned for deletion. The player's head is too close to the left side of the field of view.
NUI_HEAD_ORIENTATION_QUALITY_TOO_DARK	There is not enough lighting on the player's face for optimal operation of orientation tracking. This flag might also be combined with <code>NUI_HEAD_ORIENTATION_QUALITY_FACE_DETECT_FAILURE</code> if the lights were so low that no data is available.
NUI_HEAD_ORIENTATION_QUALITY_FAILURE	Indicates general failure in computing head orientation. This flag is set in all possible failure cases for head orientation detection. If this flag is set, it means that the Orientation member in the <code>NUI_HEAD_ORIENTATION_DATA</code> structure is invalid.

Remarks

One or more of these values may be reported in the **QualityFlags** member of the [NUI_HEAD_ORIENTATION_DATA](#) structure. The values are combined using bitwise-OR. Each value indicates a reason for loss of quality in the reported head orientation data.

See Also

[Head Orientation Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPosition Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiHeadPosition.h, which provides lightweight head position tracking, as an alternative to using skeleton tracking and monitoring the position of the head joint.

Head position tracking detects the location of the center of mass of the player's head. This is useful for detecting lean.

In This Section

[NuiHeadPosition Functions](#)

[NuiHeadPosition Structures](#)

[NuiHeadPosition Constants](#)

See Also

[Head Position Tracking](#)
[Simple Head Tracking Sample](#)
[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPosition Functions

[See Also](#)



In This Section

[NuiHeadPositionDisable](#)

Disables head position GPU and CPU processing (does not deallocate memory resources).

[NuiHeadPositionEnable](#)

Enables head position tracking.

[NuiHeadPositionGetNextFrame](#)

Returns the current head frame, which contains the position tracking data for all the heads being tracked.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPositionDisable

[See Also](#)



Disables head position GPU and CPU processing (does not deallocate memory resources).

Syntax

C++

```
HRESULT NuiHeadPositionDisable()
```

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.

Remarks

You can use this function to only use head position tracking in specific sections of the game and avoid using the GPU and CPU for head position tracking when in other sections of the game.

Memory for head position tracking stays allocated when head position tracking is disabled. To free the memory that was used for head orientation tracking, uninitialized NUI, using [NuiShutdown](#).

Requirements

Header: Declared in NuiHeadPosition.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

Head Position Tracking

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPositionEnable

See Also

Enables head position tracking.

Syntax

C++

```
HRESULT NuiHeadPositionEnable(  
    HANDLE hNextFrameEvent,  
    DWORD Flags  
)
```

Parameters

hNextFrameEvent
[in, optional] A handle to a title-allocated, manual reset event that will be set whenever a new frame of head position data is available, and will be reset whenever the latest frame data is returned. This parameter enables the title to specify an object that will be signaled when a new frame is ready. This can be NULL.

Flags
[in] A combination of flags that specify how head position tracking behaves, bitwise-OR'd together. The flags are as follows.

Flag Name	Description
NUI_HEAD_POSITION_FLAG_SUPPRESS_NO_FRAME_DATA	Same behavior as skeletal variant. Supresses NuiHeadPositionGetNextFrame from returning an error when a frame isn't available.

Return Value

One of the following values:

Value	Description
S_OK	Function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	The NUI_INITIALIZE_FLAG_USES_HEAD_POSITION flag wasn't set when calling NuiInitialize .
E_INVALIDARG	Any of the following: - A flag other than NUI_HEAD_POSITION_FLAG_SUPPRESS_NO_FRAME_DATA is set. - The pipeline that is linked in doesn't support the requested mode. <i>hNextFrameEvent</i> is not a valid event handle.
E_NUI_ALREADY_INITIALIZED	Head position tracking has already been enabled.

Remarks

Once enabled, the system will start generating head position tracking data for each frame. This function is similar to [NuiSkeletonTrackingEnable](#).

Requirements

Header: Declared in NuiHeadPosition.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

Head Position Tracking

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPositionGetNextFrame

See Also

Returns the current head frame, which contains the position tracking data for all the heads being tracked.

Syntax

C++

```
HRESULT NuiHeadPositionGetNextFrame(  
    DWORD MillisecondsToWait,  
    NUI_HEAD_POSITION_FRAME *pHeadPositionFrame  
)
```

Parameters

MillisecondsToWait
[in] The time to wait before returning without the next frame of data. Identical to the skeletal tracking variant.

pHeadPositionFrame
[out] The position tracking data for all currently tracked heads in the current frame, as a pointer to a title-allocated buffer holding a [NUI_HEAD_POSITION_FRAME](#) structure. This pointer must be non-NULL when you call the function. After the function call returns, the buffer contains the head position tracking data. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor is not connected.
E_NUI_SYSTEM_UI_PRESENT	The system UI is being shown.
E_NUI_FRAME_NO_DATA	No new head position frame is available yet. The most recent frame has been retrieved already by a previous call to this function. This error occurs when this function is being called more frequently than head position data is being updated. In that case, you should reuse the data from the previous successful call to this function. If you call NuiHeadPositionEnable passing the flag NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA , NuiHeadPositionGetNextFrame will not return E_NUI_FRAME_NO_DATA errors. Instead, calls to NuiHeadPositionGetNextFrame block until data is available or the timeout period passes. If you use that flag, you must never call NuiHeadPositionGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap .
E_PENDING	This function appears to be hung. This can happen if you call Present and NuiImageStreamGetNextFrame without strictly pairing calls to these functions.
E_NUI_STREAM_NOT_ENABLED	The stream is not enabled.

Remarks

This function is like [NuiSkeletonGetNextFrame](#).

Requirements

Header: Declared in NuiHeadPosition.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPosition Structures

[See Also](#)

In This Section

[NUI_HEAD_POSITION_DATA](#)

Contains the position tracking data for a head.

[NUI_HEAD_POSITION_FRAME](#)

Contains the position tracking data for all of the heads that are currently being tracked, for a single frame.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_POSITION_DATA Structure

[See Also](#)

Contains the position tracking data for a head.

Syntax

C++

```
typedef struct _NUI_HEAD_POSITION_DATA {  
    DWORD TrackingID;  
    DWORD QualityFlags;  
    DWORD Rank;  
    DWORD Reserved;  
    XMVECTOR Position;  
    RECT Bounds;  
} NUI_HEAD_POSITION_DATA;
```

Members

TrackingID

The tracking ID of the current head.

This ID is generated by the depth-based head detector, but has similar behavior to skeletal tracking IDs.

The tracking ID is a unique value that remains constant while a given head is visible to the camera.

Valid tracking IDs start at 1. 0 means there is no tracked head for this slot.

QualityFlags

Flags that report quality issues for head detection for the current frame, bitwise-OR'd together. The only supported flag is [NUI_HEAD_POSITION_QUALITY_LOST](#).

Rank

The relative confidence of the current proposals for which head goes with which controller. The value ranges from 1 to [NUI_HEAD_POSITION_COUNT](#), where 1 is the most probable head. For example, in a given frame, the heads could have rank 1, 2, 3, and 4. For non-tracked slots, this will be 0.

Reserved

Reserved for future use.

Position

The center of mass of the head in world space, as an [XMVECTOR Data Type](#) value with an (X,Y,Z) coordinate.

This value is generated by the depth-based head detector using the center of mass. The W value is set to 1.0.

Bounds

The bounding rectangle (left, top, right, bottom) of the head in screen space, based on the 320x240 depth map. This is the rectangle where the head was most recently detected. When a proposal becomes lost, this value remains unchanged.

Remarks

[NuiHeadPositionGetNextFrame](#) returns a [NUI_HEAD_POSITION_FRAME](#) through the *pHeadPositionFrame* output parameter. The NUI_HEAD_POSITION_FRAME structure contains an array of [NUI_HEAD_POSITION_DATA](#) structures.

Requirements

Header: Declared in NuiHeadPosition.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_POSITION_FRAME Structure

[See Also](#)



Contains the position tracking data for all of the heads that are currently being tracked, for a single frame.

Syntax

C++

```
typedef struct _NUI_HEAD_POSITION_FRAME {
    UINT64 TimeStamp;
    DWORD FrameNumber;
    DWORD Reserved;
    NUI_HEAD_POSITION_DATA HeadPositionData[NUI_HEAD_POSITION_COUNT];
} NUI_HEAD_POSITION_FRAME;
```

Members

TimeStamp

The timestamp of the depth frame that went into generating position data.

FrameNumber

The frame number of the depth frame that was used to generate this data.

Reserved

Reserved for future use.

HeadPositionData

The head position tracking data for a single frame, as an an array containing one [NUI_HEAD_POSITION_DATA](#) structure for each head position tracking slot. There are [NUI_HEAD_POSITION_COUNT](#) slots. Each NUI_HEAD_POSITION_DATA structure contains the position tracking data for a single head.

Remarks

[NuiHeadPositionGetNextFrame](#) returns a NUI_HEAD_POSITION_FRAME through the *pHeadPositionFrame* output parameter.

Requirements

Header: Declared in NuiHeadPosition.h. For include, use NuiApi.h.
Library: Use NuiApi.lib and st.lib.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiHeadPosition Constants

[See Also](#)



In This Section

[NUI_HEAD_POSITION_COUNT](#)

The number of heads that can be tracked.

[NUI_HEAD_POSITION_FLAG_SUPPRESS_NO_FRAME_DATA](#)

Supresses **NuiHeadPositionGetNextFrame** from returning an error when a frame isn't available.

[NUI_HEAD_POSITION_QUALITY_LOST](#)

Indicates that an actively tracked head proposal has been lost and might get deleted.

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_HEAD_POSITION_COUNT

[See Also](#)

The number of heads that can be tracked.

Name	Description
NUI_HEAD_POSITION_COUNT	The number of heads for which the head position can be tracked.

+

Remarks

The [NUI_HEAD_POSITION_FRAME](#) structure contains an array of [NUI_HEAD_POSITION_DATA](#) structures, one per head. The number of items in the array is NUI_HEAD_POSITION_COUNT. In the NUI_HEAD_POSITION_DATA structure, the **Rank** member ranges from 1 to **NUI_HEAD_POSITION_COUNT**.

+

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

[NUI_HEAD_POSITION_FLAG_SUPPRESS_NO_FRAME_DATA](#)

[See Also](#)

Suppresses **NuiHeadPositionGetNextFrame** from returning an error when a frame isn't available.

Name	Description
NUI_HEAD_POSITION_FLAG_SUPPRESS_NO_FRAME_DATA	Suppresses NuiHeadPositionGetNextFrame from returning an error when a frame isn't available. Like NUI_SKELETON_TRACKING_FLAG_SUPPRESS_NO_FRAME_DATA .

+

Remarks

To use this flag, pass this flag to [NuiHeadPositionEnable](#).

+

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

[NUI_HEAD_POSITION_QUALITY_LOST](#)

[See Also](#)

Indicates that an actively tracked head proposal has been lost and might get deleted.

Name	Description
NUI_HEAD_POSITION_QUALITY_LOST	A quality flag for when a tracked head proposal becomes lost. When an actively tracked proposal is not found for 30 frames, the proposal is tagged with this flag. After 30 more frames of not being found, the proposal is deleted. The number of frames in each case is subject to change.

+

Remarks

This constant is used by the **QualityFlags** member of the [NUI_HEAD_POSITION_DATA](#) structure.

+

See Also

[Head Position Tracking](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

[NuiIdentity Reference](#)

[See Also](#)

API Reference for NuiIdentity.h, which performs face and skeleton recognition, to identify a detected skeleton as a particular player each time the skeleton comes into view.

In This Section

[NuiIdentity Functions](#)
[NuiIdentity Structures](#)
[NuiIdentity Enumerations](#)
[NuiIdentity Callbacks](#)
[NuiIdentity Constants](#)

+

See Also

[Overview of NUI Identity](#)
[Nui Samples: Identity](#)
[NUI White Papers: Identity](#)
[Automatic Identity Handling](#)
[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentity Functions

[See Also](#)



In This Section

[NuiIdentityAbort](#)

Aborts all pending identity operations except those initiated by the Guide, and releases the color stream to enable it to be used by the title.

[NuiIdentityDetectedIntentToPlay](#)

If the specified skeleton matches a biometric-enabled gamer profile, enrolls the skeleton and automatically signs-in the skeleton to that gamer profile.

[NuiIdentityEnroll](#)

Enrolls the specified skeleton as a player. Also, it associates an identified skeleton with a specific enrollment index, unambiguously identifying an enrolled player.

[NuiIdentityGetEnrollmentInformation](#)

Retrieves information about a player enrolled by the current title, another title, or the Xbox Dashboard. This enables the player's enrollment status to be carried over between Kinect-enabled titles.

[NuiIdentityGetNextMessage](#)

Monitors the processing of each frame of skeleton data while [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) is processing, to report quality issues such as "too dark" to the title and player. Provides feedback about quality issues during processing, and then immediate notification of the face recognition results when processing is complete.

[NuiIdentityIdentify](#)

Performs face recognition on a tracked skeleton, to determine whether the player is already enrolled in the session identity store, and whether the detected face and skeleton matches a biometric-enabled gamer profile on the console. If the face and skeleton matches an enrolled player, this function tags the tracked skeleton with the correct enrollment index and user index.

[NuiIdentityPause](#)

Pauses all identity operations except those initiated by the Guide. This enables the title to regain the color stream temporarily, such as to take a photo of the player.

[NuiIdentityResume](#)

Resumes all previously paused and queued identity operations.

[NuiIdentityUnenroll](#)

Unenrolls a player, freeing a slot.

See Also

[NUI Identity Overview](#)

[Automatic Identity Handling](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentityAbort

[See Also](#)



Aborts all pending identity operations except those initiated by the Guide, and releases the color stream to enable it to be used by the title.

Syntax

C++

```
HRESULT NuiIdentityAbort()
```

Return Value

This function returns one of the following values:

Value	Description
S_OK	The operation is successful.
E_NUI_FEATURE_NOT_INITIALIZED	The identity feature is not initialized.
E_NUI_IDENTITY_BUSY	The Kinect Guide is using the identity engine. Any pending identity operations initiated by the title are aborted, but the color stream is not released to the title until the Guide is finished using the color stream. Also, you will get this error if any operation is in the process of aborting.

Remarks

If possible, use [NuiIdentityPause](#) instead of this function.

This function aborts all NUI identity operations that were initiated by the title, or by the NUI identity system when using [automatic identity handling](#). This function does not abort NUI identity operations that were initiated by the Kinect Guide. When the Kinect Guide initiates identity operations, all identity operations initiated by the title are aborted automatically.

You can call **NuiIdentityAbort** upon shutdown or when you want to quickly re-acquire access to the color stream. If you call this function while shutting down, you must call it in the thread that calls all destructors. This way, the function call will be blocked until the identity engine has cleaned up all of the title's callback functions.

You can call this function if you want to use the color stream data. The color stream is not guaranteed to be available immediately after this call. Your code must wait for the camera driver to switch the camera settings back to the title, and also wait for the hardware sensor to settle down.

You should not use **NuiIdentityAbort** as a timeout mechanism. If you do, it terminates all concurrent operations. Also, it has the potential to terminate an identity operation on another skeleton prematurely. To abort due to a timeout, return FALSE in the [PFN_NUI_IDENTITY_CALLBACK](#) function rather than calling **NuiIdentityAbort**. Returning FALSE in the identity callback aborts a single identity operation, while **NuiIdentityAbort** aborts all title-initiated identity operations.

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

[Automatic Identity Handling](#)

NuiIdentityDetectedIntentToPlay

[See Also](#)



Informs the NUI identity system of the skeleton tracking ID of the player who expressed intent to play the game. If the specified tracked skelelon has a face and skeleton dimensions that match a biometric-enabled gamer profile (and the player isn't already enrolled), the NUI identity system enrolls the skeleton in the session identity store and signs-in the player to the matching biometric-enabled gamer profile.

Syntax

C++

```
HRESULT NuiIdentityDetectedIntentToPlay(
    DWORD dwTrackingID
)
```

Parameters

dwTrackingID
 [in] The skeleton tracking ID of the player who indicated intent to play.
dwTrackingID is a member of the [NUI_SKELETON_DATA](#) structure, and is described in detail there.

Return Value

This function returns one of the following values:

Value	Meaning
S_OK	The function call succeeded.
E_NUI_FEATURE_NOT_INITIALIZED	Automatic identity handling is not enabled, and is required by this function.
E_NUI_IDENTITY_LOST_TRACK	The skeleton with the specified skeleton tracking ID is no longer being tracked.

Remarks

When the title detects that a skeleton has indicated intent to play, as defined by the title, call this function, passing the skeleton's tracking ID. In response to this call, if the NUI identity system previously failed to identify (recognize) the skeleton when the skeleton came into view, the NUI identity system will try to identify the skeleton again at this point. If the NUI identity system determines that the skeleton is not enrolled, and a matching biometric-enabled gamer profile is found, the NUI identity system enrolls the skeleton in the session identity store and signs-in the player to the matching gamer profile.
 If the NUI identity system determines that the skeleton is already enrolled, this function takes no additional action.
 If the NUI identity system determines that the skeleton is not enrolled, and a matching biometric-enabled gamer profile isn't found, this function doesn't enroll or sign-in the skeleton. The title can optionally call [NuiIdentityEnroll](#) to enroll the skeleton.
 This function returns immediately.

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)
[Automatic Identity Handling](#)

NuiIdentityEnroll

[See Also](#) [Example](#)



Enrolls the specified skeleton as a player. Also, it associates an identified skeleton with a specific enrollment index, unambiguously identifying an enrolled player.

Syntax

C++

```
HRESULT NuiIdentityEnroll(
    DWORD dwTrackingID,
    DWORD dwForceEnrollmentIndex,
    DWORD dwFlags,
    PFN_NUI_IDENTITY_CALLBACK pfnIdentityCallback,
    VOID *pvContext
)
```

Parameters

dwTrackingID
 [in] The tracking ID of the skeleton to enroll.
dwTrackingID is a member of the [NUI_SKELETON_DATA](#) structure, and is described in detail there.
dwForceEnrollmentIndex
 [in] When forcing enrollment, the target index to use to enroll the player.
 When reenrolling, this is the enrollment index to use, or [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#).
 This parameter is used if the [NUI_IDENTITY_FORCE_ENROLL](#) flag is passed in *dwFlags*. Otherwise, this parameter is ignored.
dwFlags
 [in] Enrollment options, as a bitwise-OR combination of [NUI_IDENTITY_enrollment_flags](#).
 Whenever you call this function, you must pass NUI_IDENTITY_ENROLL_SKELETON.
 If the NUI_IDENTITY_FORCE_ENROLL flag is passed, the enrollment operation will always succeed, regardless of quality issues. This enables basic player identification in a dark room.
pfnIdentityCallback

[in, optional] Your enrollment callback function that is used during the enrollment process, as a [PFN_NUI_IDENTITY_CALLBACK](#) function pointer. This function pointer can be NULL. If you are using automatic identity handling, pass NULL.

During the enrollment process, each time a frame has finished processing, the system invokes your enrollment callback function, passing it a [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) message. This message contains [NUI_IDENTITY_QUALITY_FLAGS](#). Thus, it indicates any quality issues that may be interfering with the process. You can use this quality flag information to prompt players to correct problems to enable enrollment.

When the entire enrollment process is complete, the system invokes your enrollment callback function again, passing it a [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure. This message contains the enrollment index, whether the profile matched the tracked skeleton, and an **HRESULT** code indicating the results of the enrollment operation.

If you provide an enrollment callback function, the callback function can cancel the enrollment operation.

pvContext

[in, optional] The context data that is passed to your enrollment callback function, as a VOID pointer. This can be NULL. If you are using automatic identity handling, pass NULL. The memory pointed to must remain valid for the duration of the asynchronous operation.

Return Value

This function initiates an asynchronous enrollment process that runs in the background for a number of frames. When the enrollment process starts successfully, **NuiIdentityEnroll** returns E_PENDING. If there are any problems initiating the enrollment process, an error code is returned.

If you provide a callback function in *pfIdentityCallback*, when the enrollment operation completes, the system calls your callback function, passing it a [NUI_IDENTITY_MESSAGE_COMPLETE](#) message that contains a result code for the operation.

If you do not provide a callback function, you will not get any specific indication that the enrollment operation has completed. However, you can infer that the enrollment operation completed successfully if the relevant skeleton's [NUI_SKELETON_DATA](#) starts arriving with a valid **dwEnrollmentIndex** value.

This function returns one of the following values:

Value	Meaning
S_OK	The specified skeleton has been enrolled already, and the NUI_IDENTITY_FORCE_ENROLL flag was not passed in <i>dwFlags</i> .
E_PENDING	The asynchronous enrollment process started successfully. When the enrollment operation is complete, the system calls your enrollment callback function, passing it a NUI_IDENTITY_MESSAGE_COMPLETE message.
E_INVALIDARG	One or more arguments were invalid. This could be caused by passing unsupported values, or by passing values in combinations that are not supported. This value is returned if <i>dwFlags</i> includes NUI_IDENTITY_FORCE_ENROLL and the maximum number of players are enrolled already.
E_NUI_IDENTITY_ENROLLMENT_LIMIT	The maximum number of players are enrolled already.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not connected.
E_NUI_IDENTITY_BUSY	The indicated skeleton is involved already in an identification or enrollment operation, or an operation is in the process of aborting.
E_NUI_IDENTITY_LOST_TRACK	The specified <i>dwTrackingId</i> isn't tracked currently.
E_NUI_FEATURE_NOT_INITIALIZED	NUI_INITIALIZE_FLAG_NUI_GUIDE_DISABLED was set.
E_NUI_MUST_CALL_IDENTIFY	NuiIdentityIdentify was not called before NuiIdentityEnroll .

Remarks

You should enroll players who have been identified, and have shown an intent to play the game. Enrollment is the process of associating an identified skeleton with a specific enrollment index, enabling an enrolled player to be identified unambiguously. Only call **NuiIdentityEnroll** after the user shows intent to play see ([NuiIdentityDetectedIntentToPlay](#)).

Before you call **NuiIdentityEnroll**, you must first call [NuiIdentityIdentify](#).

If the player matches a biometric-enabled profile on the console, the enrollment operation might sign in the player automatically. To detect whether enrollment initiated a sign-in operation, evaluate the **bProfileMatched** member in the [NUI_IDENTITY_MESSAGE_COMPLETE](#) message.

The color stream stops while the enrollment operation is running.

The timeout period of **NuiIdentityEnroll** is eight seconds. If the enrollment operation times out, the **dwEnrollmentIndex** field of the [NUI_SKELETON_DATA](#) structure is [NUI_IDENTITY_ENROLLMENT_INDEX_FAILURE](#).

Reenrollment

You should provide a user interface to indicate that a player was identified incorrectly, and allow the player to indicate who he or she is by choosing among the other enrolled players.

Following is a list of the misidentification cases and the correct value for *dwForceEnrollmentIndex*:

- If an existing player is misidentified as a new player, set *dwForceEnrollmentIndex* to the enrollment index of the existing player.
- If an existing player is misidentified as a different player, set *dwForceEnrollmentIndex* to the enrollment index of the correct player.
- If a new player is misidentified as an existing player, set *dwForceEnrollmentIndex* to [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#). This value forces the identity system to allocate a new enrollment index.

You can also use reenrollment to implement impersonation. This usage is acceptable.

Forcing an Enrollment

Force-enrollment causes the identity system to always allocate an enrollment index, even if current conditions would prevent it from successfully generating the data it needs to properly identify the player when the player returns. For example, it might indicate "too dark" or "player occluding face."

If you force an enrollment, the existing enrollment data for the given enrollment index is destroyed and replaced with fresh enrollment data for the reenrolling player.

To force an enrollment:

- In *dwTrackingID*, specify the tracking ID of the player's skeleton.
- In *dwForceEnrollmentIndex*, specify the player's chosen, correct enrollment index.
- In *dwFlags*, pass the [NUI_IDENTITY_FORCE_ENROLL](#) flag.

If the [NUI_IDENTITY_FORCE_ENROLL](#) flag is passed, *dwForceEnrollmentIndex* must be an index that is assigned already, or [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#). To force a new enrollment, use [NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN](#). You cannot specify an empty slot as the target index for force-enrollment. If you try to do so, you will get an E_INVALIDARG error.

Example

This function is used in the Identity sample. See [PFN_NUI_IDENTITY_CALLBACK](#).

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)
[Automatic Identity Handling](#)

NuiIdentityGetEnrollmentInformation

[See Also](#) [Example](#)

Retrieves information about a player enrolled by the current title, another title, or the Xbox Dashboard. This enables the player's enrollment status to be carried over between Kinect-enabled titles.

Syntax

```
C++
HRESULT NuiIdentityGetEnrollmentInformation(
    DWORD dwEnrollmentIndex,
    NUI_ENROLLMENT_INFORMATION *pEnrollmentInformation
)
```

Parameters

dwEnrollmentIndex
[in] The enrollment index of the player.

pEnrollmentInformation
[out] Enrollment information about the player, as a pointer to a [NUI_ENROLLMENT_INFORMATION](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the enrollment information. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully, and the skeleton has been enrolled.
E_INVALIDARG	The <i>dwEnrollmentIndex</i> value is out of range.

Remarks

When your Kinect-enabled title starts up, you should call this function to identify players who are already enrolled. Enrolled players who are signed in will then have their enrollment status carry over between Kinect-enabled titles.

When this function returns, the *dwEnrollmentFlags* field of the [NUI_ENROLLMENT_INFORMATION](#) structure might have the flag [NUI_IDENTITY_ENROLLED_QUALITY_ISSUE](#) set. This flag indicates that the enrollment index is assigned, but there is no valid print associated with the index. An enrollment index that has the [NUI_IDENTITY_ENROLLED_QUALITY_ISSUE](#) flag set will never match a skeleton automatically. Instead, to connect the skeleton with the enrollment index, call [NuiIdentityEnroll](#), passing the [NUI_IDENTITY_FORCE_ENROLL](#) flag.

The flag [NUI_IDENTITY_ENROLLED_QUALITY_ISSUE](#) will also be set in the following scenario: Suppose there are two enrolled players, A and B. If a skeleton is identified as A, but the player controlling the skeleton invokes a "Correct my identity" user interface and selects identity B, the enrollment A will remain assigned, but will set this flag.

Example

This function is used in the [Identity](#) sample, approximately as follows.

```
//-----
// Refreshes the GamerTag for each player based on the associated sign-in index
//-----
VOID Sample::UpdatePlayersGamerTag()
{
    for( DWORD i = 0 ; i < NUI_IDENTITY_MAX_ENROLLMENT_COUNT; ++ i )
    {
        NUI_ENROLLMENT_INFORMATION EnrollmentInformation;
        NuiIdentityGetEnrollmentInformation( i, &EnrollmentInformation );

        if( EnrollmentInformation.dwEnrollmentFlags == 0 )
        {
            // Player isn't enrolled
            ResetPlayerInfo( i );
        }
        else if( EnrollmentInformation.dwUserIndex == XUSER_INDEX_NONE )
        {
            m_PlayerInfo[ i ].wszGamerTag[ 0 ] = L'\0';
        }
        else
        {
            assert( EnrollmentInformation.dwUserIndex < XUSER_MAX_COUNT );

            CHAR szGamerTag[ XUSER_NAME_SIZE ];
            XUserGetName( EnrollmentInformation.dwUserIndex, szGamerTag, XUSER_NAME_SIZE );

            if( strlen( szGamerTag ) > 0 )
            {
                MultiByteToWideChar( CP_UTF8, 0, szGamerTag, strlen( szGamerTag ),
                    m_PlayerInfo[ i ].wszGamerTag, XUSER_NAME_SIZE );
            }
            else
            {
                m_PlayerInfo[ i ].wszGamerTag[ 0 ] = L'\0';
            }
        }
    }
}
```

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

NuiIdentityGetNextMessage

[See Also](#)



Monitors the processing of each frame of skeleton data while [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) is processing, to report quality issues such as "too dark" to the title and player. Provides feedback about quality issues during processing, and then immediate notification of the face recognition results when processing is complete.

Syntax

```
C++
HRESULT NuiIdentityGetNextMessage (
    NUI_IDENTITY_MESSAGE *pMessage
)
```

Parameters

pMessage
[out] The message from the identity engine, as a pointer to a [NUI_IDENTITY_MESSAGE](#) structure.

Return Value

This function returns one of the following values:

Value	Meaning
S_OK	A new message is available and copied to <i>*pMessage</i> .
E_INVALIDARG	A NULL pointer is passed in.
E_NUI_FEATURE_NOT_INITIALIZED	Automatic identity handling is not enabled, and is required by this function.
E_NUI_NO_MORE_ITEMS	No more messages are available.

Remarks

You can use this function to prompt and guide the player so you can identify the player.

This function is the preferred way to obtain identity messages, rather than using callbacks set on [NuiIdentityIdentify](#) and [NuiIdentityEnroll](#).

This routine returns immediately. The message includes quality and completion information. When returning the message, this function removes the message from the identity message queue.

The identity message queue only stores one message per tracking ID, and discards old messages on the same tracking ID if they have not yet been dequeued by the title. It is recommended that you dequeue all messages in the identity message queue. This enables you to stay up to date on operational status.

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

NuiIdentityIdentify

[See Also](#) [Example](#)



Performs face recognition on a tracked skeleton, to determine whether the player is already enrolled in the session identity store, and whether the detected face and skeleton matches a biometric-enabled gamer profile on the console. If the face and skeleton matches an enrolled player, this function tags the tracked skeleton with the correct enrollment index and user index.

Syntax

```
C++
HRESULT NuiIdentityIdentify(
    DWORD dwTrackingID,
    DWORD dwFlags,
    PFN_NUI_IDENTITY_CALLBACK pfnIdentityCallback,
    VOID *pvContext
)
```

Parameters

dwTrackingID
[in] The tracking ID of the skeleton data to be identified.
dwTrackingID is a member of the [NUI_SKELETON_DATA](#) structure, and is described in detail there.

dwFlags
[in] Reserved. Must be zero.

pfnIdentityCallback
[in, optional] If you have enabled automatic identity handling, you can pass NULL here instead of providing a callback function, and to monitor the status of the call to **NuiIdentityIdentify** (whether invoked automatically or explicitly) you can call [NuiIdentityGetNextMessage](#).
If you provide a callback function, the below information applies.
This parameter is your identification callback function that will be used during the identification process, as a [PFN_NUI_IDENTITY_CALLBACK](#) function pointer. This function pointer can be NULL.
During the identification process, each time a frame finishes processing, the system invokes your identification callback function, passing it a [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) message. This message contains [NUI_IDENTITY_QUALITY](#) flags indicating any quality issues that may be interfering with the process. You can use this quality flag information to prompt players to correct problems to enable identification.
When the entire identification process is complete, the system invokes your identification callback function again, passing it a [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure. This message contains the enrollment index, whether the profile matched the tracked skeleton, and an **hrResult** code indicating the results of the identification operation.

If you provide an identification callback function, the callback function can cancel the specific identification operation, by returning FALSE.

pvContext
[in, optional] The context data that will be passed to your identification callback function, as a VOID pointer. This can be NULL. If you have enabled automatic identity handling, you can pass NULL. The memory pointed to by *pvContext* must remain valid for the duration of the asynchronous operation.

Return Value

This function initiates an asynchronous process that runs in the background for a number of frames. When the identification process starts successfully, **NuiIdentityIdentify** returns E_PENDING. If there are any problems initiating the identification process, an error code is returned.

If you provide a callback function in *pfnIdentityCallback*, when the identification operation completes, the system calls your callback function, passing it a [NUI_IDENTITY_MESSAGE_COMPLETE](#) message that contains a result code for the operation.

If you don't call [NuiIdentityGetNextMessage](#) or provide a callback function, you will not get any specific indication that the identification operation has completed. However, you can infer that the identification operation completed successfully if the relevant skeleton's [NUI_SKELETON_DATA](#) starts arriving with a valid **dwEnrollmentIndex** value.

This function returns one of the following values:

Value	Meaning
S_OK	The operation completed successfully, even if the skeleton is unknown.
E_PENDING	The asynchronous identification process started successfully. When the identification operation is complete, the system calls your identification callback function, passing it a NUI_IDENTITY_MESSAGE_COMPLETE message.
E_INVALIDARG	The skeleton index is out of range, or flags are not supported.
E_ABORT	The callback routine aborted identification.
E_NUI_IDENTITY_LOST_TRACK	Tracking was lost on the skeleton before identification completed.
E_NUI_IDENTITY_QUALITY	Persistent quality issues caused identification to fail.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not connected.
E_NUI_IDENTITY_BUSY	The indicated skeleton is involved already in an identification or enrollment operation, or you called this function immediately after the camera became available while the identity engine was setting the white balance. Also, this error is returned if any operation is in the process of aborting.

Remarks

During the recognition (identification) processing, the NUI identity system checks whether the biometric-enabled gamer profile matches the detected face and skeleton. If there is a matching biometric-enabled gamer profile, this operation doesn't sign-in that gamer profile, but indicates that a matching biometric-enabled gamer profile was found. To determine whether the face and skeleton matched a biometric-enabled gamer profile, check **bProfileMatched** in the [NUI_IDENTITY_MESSAGE_COMPLETE](#) message.

The color stream stops while the identification operation is running.

Timeout

This function times out in 30 seconds, except that in [automatic identity handling](#), when the player shows intent to play (see [NuiIdentityDetectedIntentToPlay](#)), the timeout is eight seconds. The timeout time is counted only when the identity thread is active; the real-world time is about 10% longer. The timeout always applies, regardless of whether there is a callback function.

If the identification operation times out, the **dwEnrollmentIndex** field of the [NUI_SKELETON_DATA](#) structure will be [NUI_IDENTITY_ENROLLMENT_INDEX_FAILURE](#).

Timeout is not the actual operation time. Usually, this operation finishes quickly.

Example

This function is used in the Identity sample. See [PFN_NUI_IDENTITY_CALLBACK](#).

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)
[Automatic Identity Handling](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentityPause

[See Also](#)



Pauses all identity operations except those initiated by the Guide. This enables the title to regain the color stream temporarily, such as to take a photo of the player.

Syntax

C++

HRESULT NuiIdentityPause()

Return Value

This function returns one of the following values:

Value	Meaning
S_OK	The function call was successful.
E_NUI_FEATURE_NOT_INITIALIZED	The identity system has not been initialized.
E_NUI_IDENTITY_BUSY	There are pending operations from the Guide, so the identity engine cannot be paused immediately. The identity engine will pause after the Guide operation finishes.

Remarks

This function, together with [NuiIdentityResume](#), enables titles to temporarily regain the color stream, such as to take a photo of the player. The identity system will stay paused until the title calls **NuiIdentityResume**.

This function pauses all title-initiated identity operations that are pending or new. This function also pauses identity operations that were triggered by the [Automatic Mode for Identity](#), if

the automatic mode has been enabled (using [NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY](#)). If Guide-initiated identity operations are in progress, those operations continue without pausing. Then, when they finish, the identity engine pauses.

The title can issue new identity operations while the system is paused.

- If there is no operation in progress on the given tracking ID, the new operation immediately returns E_PENDING, and the new operation is added to the queue.
- If there is already an operation in progress on the given tracking ID, the new operation immediately returns E_BUSY, and the new operation is not added to the queue. There can only be one operation on a given skeleton at a time. The operation is either in progress or queued up.
- If tracking is lost on a tracking ID that had a paused identity operation in progress, the system will use [NuiIdentityGetNextMessage](#) to report TRACKING_ID_LOST even while the system is still paused.
- You can call **NuiIdentityPause** multiple times. The identity operations remain paused.

NuiIdentityPause is preferred over [NuiIdentityAbort](#).

If you haven't enabled automatic identity handling, you might not need to call **NuiIdentityPause**, because you are already manually managing when [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) runs.

✚ Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

✚ See Also

[NUI Identity Overview](#)
[Automatic Identity Handling](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentityResume

[See Also](#)



Resumes all previously paused and queued identity operations.

✚ Syntax

C++

```
HRESULT NuiIdentityResume()
```

Return Value

This function returns one of the following values:

Value	Meaning
S_OK	The function call was successful.
E_NUI_FEATURE_NOT_INITIALIZED	The identity system has not been initialized.

✚ Remarks

This function, together with [NuiIdentityPause](#), enables titles to temporarily regain the color stream, such as to take a photo of the player.

This function can be called multiple times. The identity operations remain active.

✚ Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

✚ See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentityUnenroll

[See Also](#) [Example](#)



Unenrolls a player, freeing a slot.

✚ Syntax

C++

```
HRESULT NuiIdentityUnenroll(  
    DWORD dwEnrollmentIndex  
)
```

Parameters

dwEnrollmentIndex
[in] The enrollment index to be unenrolled.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	The <i>dwEnrollmentIndex</i> value is invalid.

E_NUI_IDENTITY_NO_UNENROLL_SIGN_IN

The specified player is signed in. Signed-in players cannot be unenrolled.

Remarks

This function removes the specified enrollment from the identity system, freeing a slot for use by a new player. To support the maximum number of players, you can use this function. For example, older enrollments can be removed to make room for newer enrollments.

To unenroll a player, the player must be signed out.

Example

This function is used in the Identity sample. See [PFN_NUI_IDENTITY_CALLBACK](#).

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentity Structures

[See Also](#)

In This Section

[NUI_ENROLLMENT_INFORMATION](#)

Specifies the user index and enrollment status of a signed-in player.

[NUI_IDENTITY_MESSAGE](#)

A message container for "frame processed" information or "operation complete" information. Contains feedback about quality issues during processing, and then contains face recognition results when processing is complete.

[NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#)

Describes the result of a completed identification or enrollment operation, including whether the operation was generally successful, the enrollment ID or enrollment status of the detected skeleton, and whether a matching biometric-enabled gamer profile was found.

[NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#)

Indicates any quality issues that prevent identifying (recognizing) a skeleton in a frame of skeleton tracking data during skeleton identification or enrollment, such as "body turned" or "too bright".

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_ENROLLMENT_INFORMATION Structure

[See Also](#)

Specifies the user index and enrollment status of a signed-in player.

Syntax

C++

```
typedef struct _NUI_ENROLLMENT_INFORMATION {  
    DWORD dwUserIndex;  
    DWORD dwEnrollmentFlags;  
} NUI_ENROLLMENT_INFORMATION;
```

Members

dwUserIndex

The user index (0-3) of this enrolled, signed-in player, or XUSER_INDEX_NONE for a player who is non-enrolled or not signed in.

dwEnrollmentFlags

A bitwise-OR combination of [NUI_IDENTITY_query_flags](#) indicating enrollment status. To check for an enrollment index that is not in use, check that NUI_ENROLLMENT_INFORMATION.dwEnrollmentFlags == 0.

Remarks

This structure is returned by [NuiIdentityGetEnrollmentInformation](#).

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_MESSAGE Structure

[See Also](#)

A message container for "frame processed" information or "operation complete" information. Contains feedback about quality issues during processing, and then contains face recognition results when processing is complete.

Syntax

```
C++
typedef struct _NUI_IDENTITY_MESSAGE {
    NUI_IDENTITY_MESSAGE_ID MessageId;
    NUI_IDENTITY_OPERATION_ID OperationId;
    DWORD dwTrackingID;
    DWORD dwSkeletonFrameNumber;
    union {
        NUI_IDENTITY_MESSAGE_FRAME_PROCESSED FrameProcessed;
        NUI_IDENTITY_MESSAGE_COMPLETE Complete;
    }Data;
} NUI_IDENTITY_MESSAGE;
```

Members

MessageId
A [NUI_IDENTITY_MESSAGE_ID](#) enumeration value indicating what kind of message this is. The message is either *single frame processed* or *entire process complete*. The message indicates that a single frame has finished processing or that the entire identification or enrollment process has completed.

OperationId
The type of operation that generated the message, as a [NUI_IDENTITY_OPERATION_ID](#) enumeration value.

- [NUI_IDENTITY_OPERATION_ID_IDENTIFY](#) indicates a [NuiIdentityIdentify](#) operation.
- [NUI_IDENTITY_OPERATION_ID_ENROLL](#) indicates a [NuiIdentityEnroll](#) operation.

dwTrackingID
The tracking ID of the skeleton to which this message applies. **dwTrackingID** is a member of the [NUI_SKELETON_DATA](#) structure. See that topic for details.

dwSkeletonFrameNumber
The number of the frame of skeleton tracking data when the message is generated.

Data
Contains the appropriate type of structure describing either the processing of a single frame or the entire completed processing.

FrameProcessed
When **MessageId** is [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#), the **Data** member contains a populated [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure that contains any [NUI_IDENTITY_QUALITY](#) flags that describe quality issues encountered while processing a single frame.

Complete
When **MessageId** is [NUI_IDENTITY_MESSAGE_ID_COMPLETE](#), the **Data** member contains a populated [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure.

Remarks

[NuiIdentityGetNextMessage](#) or your [callback function](#) returns a [NUI_IDENTITY_MESSAGE](#) structure. The returned [NUI_IDENTITY_MESSAGE](#) structure at first contains a [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure, each time during processing multiple frames of skeleton data. The [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure contains [NUI_IDENTITY_QUALITY flags](#) to indicate any quality issues during face recognition processing.

When the NUI identity operation completes, the returned [NUI_IDENTITY_MESSAGE](#) structure contains a [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure. The [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure indicates whether the NUI identity operation was able to complete successfully, the [enrollment status](#) or a specific enrollment index (0-7) of the skeleton, and whether the detected face and skeleton matched a biometric-enabled gamer profile.

Requirements

Header: Declared in [NuiIdentity.h](#). For include, use [NuiApi.h](#).

Library: Use both [nuiapi.lib](#) and [st.lib](#).

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_MESSAGE_COMPLETE Structure

[See Also](#)

Describes the result of a completed identification or enrollment operation, including whether the operation was generally successful, the enrollment ID or enrollment status of the detected skeleton, and whether a matching biometric-enabled gamer profile was found.

Syntax

```
C++
typedef struct _NUI_IDENTITY_MESSAGE_COMPLETE {
    HRESULT hrResult;
    DWORD dwEnrollmentIndex;
    BOOL bProfileMatched;
} NUI_IDENTITY_MESSAGE_COMPLETE;
```

Members

hrResult
The result of the identification or enrollment operation. Possible values are shown below.

Value	Meaning
S_OK	The operation completed successfully. For the operation NuiIdentityIdentify , S_OK indicates that the operation identified the skeleton as a specific enrolled player (with enrollment ID 0-7), or determined that the player is not an enrolled player, or was unable to perform recognition processing on the skeleton. For the operation NuiIdentityEnroll , S_OK indicates that the skeleton was successfully enrolled.
E_NUI_IDENTITY_BUSY	During an identity operation, if the player brings up the Kinect Guide by using a gesture, the Kinect Guide might issue a new identity request that will override the title's request. In this case, you receive this error code. Your code should then wait until the enrollment index in the skeleton data is not busy. You should check that the enrollment index satisfies your needs. If the enrollment index is unsatisfactory, you can restart an identity

	operation.
E_NUI_IDENTITY_LOST_TRACK	The system lost tracking on the skeleton before the operation could be completed.
E_NUI_IDENTITY_QUALITY_ISSUE	Persistent quality issues caused the operation to fail.
E_ABORT	The identity operation was aborted by the title or player. To abort the identity operation, return FALSE when your callback function is called passing a NUI_IDENTITY_MESSAGE_FRAME_PROCESSED message.

dwEnrollmentIndex
When **hrResult** indicates success, this value is the result of the operation. Otherwise, this value is undefined. If the player is enrolled, this is an enrollment ID (0-7). If the player is not enrolled, one of the following constants is returned: [NUI_IDENTITY_ENROLLMENT_INDEX Constants](#).

bProfileMatched
Whether the detected face and skeleton matched a biometric-enabled gamer profile available on the console. When calling [NuiIdentityIdentify](#), this value is for informational purposes only. TRUE means the detected face and skeleton matches a biometric-enabled gamer profile on the console, and will be signed in if you enroll that player. FALSE means the detected face and skeleton does not match a biometric-enabled gamer profile on the console, so if the title enrolls the player, the player will not be automatically signed in, but will be enrolled as a guest player.

If you call [NuiIdentityEnroll](#) when this value is TRUE, a sign-in operation is initiated automatically due to the match. However, a value of TRUE doesn't guarantee that after **NuiIdentityEnroll** completes, the matched gamer profile is now signed in. The profile might still not be signed in, because the player may have cancelled the operation in the sign-in dialog box. To query the actual resulting sign-in status, call [XUserGetSigninState](#).

If this value is FALSE, there is not a biometric enabled gamer profile that matches the skeleton. A sign-in operation is not performed automatically.

Remarks
When the entire identification or enrollment operation completes, the NUI identity system returns this structure to the title. [NuiIdentityGetNextMessage](#) or your [callback function](#) (if used) returns a [NUI_IDENTITY_MESSAGE](#) structure, which eventually contains a NUI_IDENTITY_MESSAGE_COMPLETE structure.

Requirements
Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also
[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_MESSAGE_FRAME_PROCESSED Structure

[See Also](#)



Indicates any quality issues that prevent identifying (recognizing) a skeleton in a frame of skeleton tracking data during skeleton identification or enrollment, such as "body turned" or "too bright".

Syntax

```
C++
typedef struct _NUI_IDENTITY_MESSAGE_FRAME_PROCESSED {
    DWORD dwQualityFlags;
} NUI_IDENTITY_MESSAGE_FRAME_PROCESSED;
```

Members
dwQualityFlags
A bitwise-OR combination of [NUI_IDENTITY_QUALITY](#) flags indicating any issues that are interfering with the identification system.

Remarks
Each time a frame of skeleton tracking data finishes processing, an instance of this structure is passed to your [PFN_NUI_IDENTITY_CALLBACK](#) callback function that is monitoring the operation. Use the [NUI_IDENTITY_QUALITY](#) flags in this structure to guide the player to adjust their position or the environment to enable the identification or enrollment operation to succeed, such as "Stand farther away from the camera."

Requirements
Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also
[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentity Enumerations

[See Also](#)



In This Section

- [NUI_IDENTITY_MESSAGE_ID](#)
Identifies whether "frame processed" information or "operation complete" information is being passed to your callback function within the accompanying structure during the process of identifying or enrolling a player.
- [NUI_IDENTITY_OPERATION_ID](#)
Specifies what type of operation is involved, for a message passed from the identity engine.

See Also
[NUI Identity Overview](#)

NUI_IDENTITY_MESSAGE_ID Enumeration

[See Also](#)



Identifies whether "frame processed" information or "operation complete" information is being passed to your callback function within the accompanying structure during the process of identifying or enrolling a player.

Syntax

```
C++

typedef enum _NUI_IDENTITY_MESSAGE_ID
{
    NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED,
    NUI_IDENTITY_MESSAGE_ID_COMPLETE
} NUI_IDENTITY_MESSAGE_ID;
```

Constants

Constant	Description
NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED	This message-type identifier is sent every time the system has completed processing a single frame of skeleton tracking data, during the identification or enrollment process. It is accompanied by a NUI_IDENTITY_MESSAGE_FRAME_PROCESSED structure that provides feedback about conditions that might be impeding the process (if any), in the form of NUI_IDENTITY_QUALITY flags that are OR'd together.
NUI_IDENTITY_MESSAGE_ID_COMPLETE	This message-type identifier is sent when the entire identification or enrollment process is complete. It is accompanied by a NUI_IDENTITY_MESSAGE_COMPLETE structure that describes the results of the identification or enrollment operation, including the enrollment index and whether the profile matches.

Remarks

The NUI identity system uses these enumeration constants to indicate what kind of message is being sent from the NUI identity system to the title: "single frame processed" or "entire operation complete". The title calls [NuiIdentityGetNextMessage](#), or the NUI identity system calls your identification or enrollment [PFN_NUI_IDENTITY_CALLBACK](#) function, passing to the title a general-purpose [NUI_IDENTITY_MESSAGE](#) structure that contains one of these ID enumeration values to specify what kind of information is being passed.

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

NUI_IDENTITY_OPERATION_ID Enumeration

[See Also](#)



Specifies what type of operation is involved, for a message passed from the identity engine.

Syntax

```
C++

typedef enum _NUI_IDENTITY_OPERATION_ID
{
    NUI_IDENTITY_OPERATION_ID_NONE,
    NUI_IDENTITY_OPERATION_ID_IDENTIFY,
    NUI_IDENTITY_OPERATION_ID_ENROLL,
    NUI_IDENTITY_OPERATION_ID_TUNER
} NUI_IDENTITY_OPERATION_ID;
```

Constants

Constant	Description
NUI_IDENTITY_OPERATION_ID_NONE	A no-op or an invalid value. The identity engine should never generate a message with this value.
NUI_IDENTITY_OPERATION_ID_IDENTIFY	A pending NuiIdentityIdentify operation.
NUI_IDENTITY_OPERATION_ID_ENROLL	A pending NuiIdentityEnroll operation.
NUI_IDENTITY_OPERATION_ID_TUNER	This value is available only if the skeleton tracking part of the Kinect tuner is tested when you captured a XED file. It can be present in a NUI_IDENTITY_MESSAGE structure returned from XedReadNuiIdentityMessage . Use this value only if you use your own tools to handle XED files. It will not appear in the NUI_IDENTITY_MESSAGE that is returned from NuiIdentityGetNextMessage in the game runtime. The title code should ignore this value.

Remarks

The **OperationID** field of a [NUI_IDENTITY_MESSAGE](#) structure contains one of these enumeration values.

Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentity Callbacks

[See Also](#)

In This Section

[PFN_NUI_IDENTITY_CALLBACK](#)

Monitors the processing of each frame of skeleton data while [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) is processing, to report quality issues such as "too dark" to the title and player. Provides feedback about quality issues during processing, and then immediate notification of the face recognition results when processing is complete.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

PFN_NUI_IDENTITY_CALLBACK Callback

[See Also](#) [Example](#)

Monitors the processing of each frame of skeleton data while [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) is processing, to report quality issues such as "too dark" to the title and player. Provides feedback about quality issues during processing, and then immediate notification of the face recognition results when processing is complete.

Syntax

C++

```
typedef BOOL (*PFN_NUI_IDENTITY_CALLBACK) (
    PVOID pvContext,
    NUI_IDENTITY_MESSAGE *pMessage
)
```

Parameters

pvContext

[in] A pointer to context information passed as the *pvContext* parameter to the [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) function.

Create a structure that holds context information that is relevant to the title, pass that context information using the *pvContext* parameter to **NuiIdentityIdentify** or **NuiIdentityEnroll**, and then read that context information from the *pvContext* argument that is passed to the callback function.

pMessage

[in] A pointer to a general-purpose [NUI_IDENTITY_MESSAGE](#) structure that contains a message-type identifier to specify what kind of information is being passed, and the corresponding specific type of structure. The nested, specific structure contains either [NUI_IDENTITY_QUALITY](#) flags to indicate any problems when processing a single frame, or a result code indicating the results of the entire operation together with the enrollment index and whether the profile matches.

Return Value

Your callback function must return TRUE to continue the identification or enrollment process, or FALSE to terminate the process unsuccessfully. This return value applies only to the "frame completed" scenario, not the "operation completed" scenario. That is, this return value is used only when the **MessageID** in the [NUI_IDENTITY_MESSAGE](#) structure is [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#). This return value is ignored when the **MessageID** is [NUI_IDENTITY_MESSAGE_ID_COMPLETE](#).

Remarks

If you enable automatic identity handling, you should generally use [NuiIdentityGetNextMessage](#) instead of this callback function.

You can define an identification or enrollment callback function, and then specify that callback function when you call [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#).

To abort due to a timeout, return FALSE in the identity callback function rather than calling [NuiIdentityAbort](#). Returning FALSE in the identity callback aborts a single identity operation, while **NuiIdentityAbort** aborts all title-initiated identity operations. **NuiIdentityAbort** should not be used as a timeout mechanism because it terminates all concurrent operations. It could terminate an identity operation on another skeleton prematurely.

Usage Sequence

Your identification or enrollment callback function is invoked by the NUI identity system when the title or the NUI identity system calls **NuiIdentityIdentify** or **NuiIdentityEnroll**. The NUI identity system invokes your callback function each time a frame of skeleton data has finished processing, and then invokes your callback function again when the entire identification or enrollment process is complete. The callback function will fire only for the duration of the operation with which the title initiated.

Each time a frame of skeleton data has completed processing, the NUI identity system invokes the callback function, passing "frame processed" information. After a frame is processed, the structure pointed to by *pMessage* contains the message identifier [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#) (an enumeration constant), together with a nested [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure that contains flags to indicate any problems that were encountered while processing the frame. The [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) message provides feedback that your callback function can use to display what actions the player should do to facilitate identification or enrollment.

The identification or enrollment process continues as long as repeated calls to the callback function passing [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#) continue to return TRUE.

If the callback function returns FALSE to cancel the operation, the system calls the callback function again. It passes a [NUI_IDENTITY_MESSAGE](#) structure that contains the message-type identifier [NUI_IDENTITY_MESSAGE_ID_COMPLETE](#) and a [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure that contains an **hrResult** member with the value of E_ABORT.

When the entire identification or enrollment process has completed, the system again invokes the callback function, this time passing "operation completed" information. After the operation has completed, the structure pointed to by *pMessage* contains the message identifier [NUI_IDENTITY_MESSAGE_ID_COMPLETE](#) (an enumeration constant), together with a nested [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure that indicates whether the operation completed successfully and, if so, the identifier of the identified or enrolled player.

Completion Message Guaranteed

Titles are guaranteed to receive a completion message when the identification or enrollment operation completes, either successfully or unsuccessfully. For example, a title is guaranteed to receive a completion message when the title terminates the operation by returning FALSE from the callback function on a [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#) message. The following is an example of this scenario:

1. The title receives an identity callback with a [NUI_IDENTITY_MESSAGE_ID_FRAME_PROCESSED](#) message and decides to terminate due to a timeout, returning FALSE.
2. Soon afterwards, the title will receive another callback with a [NUI_IDENTITY_MESSAGE_COMPLETE](#) message notifying that identity has terminated the operation. **HRESULT** will be E_ABORT.

Thread Safety

You must implement this callback function in a thread-safe manner. If your callback function accesses shared data, your callback function must be thread-safe. This is necessary because it is called on the identity system thread.

You should write your callback function in such a way that it completes quickly because the identity thread is blocked by the callback function.

Example

Callback functions are used in the [Identity sample](#) as follows.

The **IdentityIdentifyCallback** sample function is used to obtain feedback from calls to [NuiIdentityIdentify](#) and forward the result to the main thread:

```

BOOL IdentityIdentifyCallback( PVOID pvContext, NUI_IDENTITY_MESSAGE* pMessage )
{
    assert( pvContext != NULL );
    assert( pMessage != NULL );

    IdentityMessagePipe* pMessagePipe = ( IdentityMessagePipe* ) pvContext;

    IDENTITY_CALLBACK_MESSAGE IdentityMessage = { IDENTITY_CALLBACK_IDENTIFY, *pMessage };
    #if defined(NDEBUG)
    pMessagePipe->Write( &IdentityMessage, sizeof( IDENTITY_CALLBACK_MESSAGE ) );
    #else
    bool bWasSent = pMessagePipe->Write( &IdentityMessage, sizeof( IDENTITY_CALLBACK_MESSAGE ) );
    assert( bWasSent );
    #endif

    return TRUE;
}

```

The **IdentityEnrollCallback** sample function is used to obtain feedback from calls to [NuiIdentityEnroll](#) and forward the result to the main thread:

```

BOOL IdentityEnrollCallback( PVOID pvContext, NUI_IDENTITY_MESSAGE* pMessage )
{
    assert( pvContext != NULL );
    assert( pMessage != NULL );

    IdentityMessagePipe* pMessagePipe = ( IdentityMessagePipe* ) pvContext;

    IDENTITY_CALLBACK_MESSAGE IdentityMessage = { IDENTITY_CALLBACK_ENROLL, *pMessage };
    #if defined(NDEBUG)
    pMessagePipe->Write( &IdentityMessage, sizeof( IDENTITY_CALLBACK_MESSAGE ) );
    #else
    bool bWasSent = pMessagePipe->Write( &IdentityMessage,
                                        sizeof( IDENTITY_CALLBACK_MESSAGE ) );
    assert( bWasSent );
    #endif

    return TRUE;
}

```

The callback functions are called in the sample as follows:

```

//-----
// Name: Update()
// Desc: Called once per frame to update the scene.
//-----
HRESULT Sample::Update()
{
    // ...
    HRESULT hr = NuiSkeletonGetNextFrame( NUI_CAMERA_TIMEOUT_DEFAULT, &m_SkeletonFrame );
    // ...

    // Identify and enroll / re-enroll the players as needed
    for( DWORD i = 0; i < NUI_SKELETON_COUNT; ++i )
    {
        DWORD dwTrackingID = m_SkeletonFrame.SkeletonData[ i ].dwTrackingID;

        switch( m_PlayerRequest[ i ].eRequest )
        {
            case PLAYER_NEEDIDENTIFY:
            {
                hr = NuiIdentityIdentify( dwTrackingID, // Identify for this specific skeleton
                                         0, // Flags, reserved - must be 0
                                         IdentityIdentifyCallback, // NuiIdentity will call this function to report progress
                                         &m_MessagePipe ); // Message pipe for our callback to use to forward messages

                assert( hr != E_INVALIDARG );
                assert( hr == E_PENDING || hr == E_NUI_IDENTITY_BUSY );
                m_adwAsyncOpTrackingID[ i ] = dwTrackingID;
                break;
            }

            case PLAYER_WANTENROLL:
            {
                hr = NuiIdentityEnroll( dwTrackingID, // Enroll this skeleton
                                       0, // Ignored, unless re-enrolling
                                       NUI_IDENTITY_ENROLL_SKELETON, // Enroll using skeleton data
                                       IdentityEnrollCallback, // NuiIdentity will call this function to report progress
                                       &m_MessagePipe ); // Message pipe for our callback to use to forward messages

                if( hr == E_NUI_IDENTITY_ENROLLMENT_LIMIT )
                {
                    // If we have reached the maximum number of players that can be enrolled at once,
                    // then remove the least recently active player, that isn't signed-in and re-enroll this player.
                    DWORD dwPlayerToUnenroll = DetermineLongestInactivePlayer( TRUE );
                    hr = NuiIdentityUnenroll( dwPlayerToUnenroll );
                    assert( SUCCEEDED( hr ) );
                    ResetPlayerInfo( dwPlayerToUnenroll );
                    hr = NuiIdentityEnroll( dwTrackingID, // Enroll this skeleton
                                           0, // Ignored, unless re-enrolling
                                           NUI_IDENTITY_ENROLL_SKELETON, // Enroll using skeleton data
                                           IdentityEnrollCallback, // NuiIdentity will call this function to report progress
                                           &m_MessagePipe ); // Message pipe for our callback to use to forward messages
                }

                assert( hr != E_INVALIDARG );
                assert( hr == E_PENDING || hr == E_NUI_IDENTITY_BUSY );
                m_adwAsyncOpTrackingID[ i ] = dwTrackingID;
                break;
            }

            case PLAYER_WANTREENROLL:
            {
                assert( m_PlayerRequest[ i ].dwData <= NUI_IDENTITY_MAX_ENROLLMENT_COUNT ||
                       m_PlayerRequest[ i ].dwData == NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN );

                hr = NuiIdentityEnroll(

```

```
dwTrackingID, // ID of the skeleton we want to re-enroll
m_PlayerRequest[ i ].dwData, // New enrollment index requested
NUI_IDENTITY_ENROLL_SKELETON | // Enroll using skeleton data
NUI_IDENTITY_FORCE_ENROLL, // Ask for re-enrollment
IdentityEnrollCallback, // Callback to process feedback from NuiIdentity API
&m_MessagePipe ); // Message pipe for our callback to use to forward messages

assert( hr == E_PENDING || hr == E_NUI_IDENTITY_BUSY );
m_adwAsyncOpTrackingID[ i ] = dwTrackingID;
m_SkeletonIdentityInfo[ i ].bIsEnrolling = TRUE;
break;
}

case PLAYER_NOREQUEST:
{
    break;
}

default:
{
    assert( false );
    break;
}
}
}

return S_OK;
}
```

⛶ Requirements

Header: Declared in NuiIdentity.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

⛶ See Also

[NUI Identity Overview](#)
[Identity Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiIdentity Constants

[See Also](#)

In This Section

[NUI_IDENTITY Enrollment Flags](#)

Flags used to specify enrollment options.

[NUI_IDENTITY Query Flags](#)

Flags used to indicate enrollment status when querying enrollment status.

[NUI_IDENTITY_ENROLLMENT_INDEX Constants](#)

Constants that report the enrollment status for a detected skeleton. If the skeleton is recognized as being an enrolled player, an enrollment index (0-7) is provided instead of one of these values. The NUI identity system reports these values to the title.

[NUI_IDENTITY_MAX_ENROLLMENT_COUNT](#)

The maximum number of player enrollment slots that can be allocated.

[NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK](#)

Groups the NUI_IDENTITY_QUALITY flags that are about environment issues.

[NUI_IDENTITY_QUALITY Flags](#)

Flags used to specify image quality issues that prevent the identification of players.

[NUI_IDENTITY_QUALITY_USER_MASK](#)

Groups the NUI_IDENTITY_QUALITY flags that are about user issues.

⛶ See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY Enrollment Flags

[See Also](#)

Flags used to specify enrollment options.

Name	Description
NUI_IDENTITY_ENROLL_SKELETON	Enrolls a skeleton.
NUI_IDENTITY_FORCE_ENROLL	Forces an enrollment or performs a re-enrollment. If this flag is specified, the NuiIdentityEnroll operation will always succeed, regardless of quality issues. This enables basic player identification in a dark room. To check for an enrollment index that is not in use, check that the <i>dwEnrollmentFlags</i> field of the NUI_ENROLLMENT_INFORMATION structure is 0. See also Enabling Players to Correct Their Identity and Forcing Enrollment Due to Quality Issues .

⛶ Remarks

These values are used in the *dwFlags* parameter of the [NuiIdentityEnroll](#) function, to specify enrollment options.

⛶ See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY Query Flags

[See Also](#)



Flags used to indicate enrollment status when querying enrollment status.

Name	Description
NUI_IDENTITY_ENROLL_SKELETON	Indicates that the specified skeleton is enrolled.
NUI_IDENTITY_ENROLLED_QUALITY_ISSUE	Indicates that the enrollment index is assigned, but there is no valid print associated with the index. An enrollment index with this flag will never match a skeleton automatically (by relying on face and skeleton recognition), when the skeleton comes into view. Instead, if you want to connect a specific skeleton to an enrollment index that has this flag, call NuiIdentityEnroll, passing the NUI_IDENTITY_FORCE_ENROLL flag. This flag can also be set in the following scenario. Suppose there are two enrolled players, A and B. If a skeleton is identified as A, but the player using the skeleton invokes a "Correct my identity" user interface and selects identity B, the enrollment A will remain assigned, but will have this flag set. If the title or the Guide force-enrolls a skeleton and there are persistent quality issues (such as bad lighting), the enrollment index will be linked to the tracked skeleton, but that linking won't be secured by the ability to recognize the face and skeleton, and this flag will be set. Thus, when the linked skeleton disappears, and then the skeleton comes into view again, the identity system will not be able to re-identify the newly visible skeleton as being associated with that enrollment index.

Remarks

When you call [NuiIdentityGetEnrollmentInformation](#), the system can set these flags in the *dwEnrollmentFlags* field of the [NUI_ENROLLMENT_INFORMATION](#) structure that is returned through *pEnrollmentInformation*.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_ENROLLMENT_INDEX Constants

[See Also](#)



Constants that report the enrollment status for a detected skeleton. If the skeleton is recognized as being an enrolled player, an enrollment index (0-7) is provided instead of one of these values. The NUI identity system reports these values to the title.

Name	Description
NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY	The skeleton data is being tracked consistently to one unidentified person, and identification has not been attempted yet. Call NuiIdentityIdentify to identify the person.
NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN	This means "not enrolled". Face recognition processing has completed, and concluded that the skeleton is not an enrolled player. The skeleton is unknown to the session identity store. However, a matching biometric-enabled gamer profile may have been found among the available gamer profiles. To see that recognition result, check the bProfileMatched member of the NUI_IDENTITY_MESSAGE_COMPLETE structure. When re-enrolling a player to correct a misidentification, this flag can be used by the title, as the <i>dwForceEnrollmentIndex</i> argument of NuiIdentityEnroll.
NUI_IDENTITY_ENROLLMENT_INDEX_BUSY	This value is returned in the following cases: - The skeleton <i>dwTrackingId</i> is currently involved in an identification or enrollment operation that hasn't completed. - The Identity library is initializing and isn't yet ready.
NUI_IDENTITY_ENROLLMENT_INDEX_FAILURE	NuiIdentityIdentify or NuiIdentityEnroll failed to complete. One of the following occurred: - The operation was cancelled by calling NuiIdentityAbort (which cancels all NUI identity operations that were initiated by the title). - The title didn't provide a callback to the NuiIdentityIdentify or NuiIdentityEnroll operation, and the operation timed-out due to quality issues. To have the system ignore quality issues and force NuiIdentityEnroll to succeed, you can pass the NUI_IDENTITY_FORCE_ENROLL flag. - The operation was cancelled by a callback function returning FALSE.

Remarks

These constants are used in the **dwEnrollmentIndex** member of a [NUI_SKELETON_DATA](#) structure when the skeleton data represents a player who has not been enrolled.

These are constants that report the enrollment status for a detected skeleton, including whether face recognition needs to be attempted, whether the skeleton is enrolled, whether identity processing is still ongoing, and whether the identification operation failed to complete.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.

Version: 21256.3

NUI_IDENTITY_MAX_ENROLLMENT_COUNT

[See Also](#)

The maximum number of player enrollment slots that can be allocated.

Name	Description
NUI_IDENTITY_MAX_ENROLLMENT_COUNT	The maximum number of enrolled players supported.

Remarks

This constant can be used to check whether an enrollment index is within the valid range when using API items that contain a *dwEnrollmentIndex* member or parameter, including the following:

- The [NUI_IDENTITY_MESSAGE_COMPLETE](#) structure
- The [NUI_SKELETON_DATA](#) structure
- [NuiIdentityGetEnrollmentInformation](#)
- [NuiIdentityEnroll](#)
- [NuiIdentityUnenroll](#)

For example, calling [NuiSkeletonGetNextFrame](#) returns a pointer to a NUI_SKELETON_FRAME. The NUI_SKELETON_FRAME contains an array of NUI_SKELETON_DATA structures. Each NUI_SKELETON_DATA structure contains a *dwEnrollmentIndex* member. The *dwEnrollmentIndex* member of each returned skeleton normally contains an enrollment index between zero and the maximum number of enrolled players supported, as specified by **NUI_IDENTITY_MAX_ENROLLMENT_COUNT**.

See Also

[NUI Identity Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK

[See Also](#)

Groups the NUI_IDENTITY_QUALITY flags that are about environment issues.

Name	Description
NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK	Groups the NUI_IDENTITY_QUALITY flags that are about environment issues.

Remarks

This #define value is a mask that groups the [NUI_IDENTITY_QUALITY](#) flags whose names begin with "NUI_IDENTITY_QUALITY_ENVIRONMENT."

You can use this mask to separate environment and user issues. For example:

```
if (dwQualityFlags & NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK)
{
    EnvironmentError();
}
else if (dwQualityFlags & NUI_IDENTITY_QUALITY_USER_MASK)
{
    UserError();
}
```

See Also

[NUI Identity Overview](#)
[NUI_IDENTITY_QUALITY_USER_MASK](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IDENTITY_QUALITY Flags

[See Also](#)

Flags used to specify image quality issues that prevent the identification of players.

Name	Description
NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE	Identity is unable to lock onto a player's face. If this flag is combined with another quality issue flag, the other flag takes precedence. If this flag is not combined with another quality issue flag, causes may include: • Lighting is uneven across the player's face, likely due to strong side lighting. • The player's eyes, nose, or mouth are occluded, likely by long hair or reflective glasses. • The player is not facing the sensor. Suggested user feedback: "Kinect can't see your face clearly. If the problem persists, go to the Kinect Tuner."
NUI_IDENTITY_QUALITY_USER_BODY_TURNED	The player's shoulders are not squared with the sensor. This flag may fire in combination with NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE. Suggested user feedback: "Face the sensor."
NUI_IDENTITY_QUALITY_USER_NOT_UPRIGHT	The player is not standing straight enough for the system to get an accurate height measurement. This quality issue does not block the identity operation, but it may lead to reduced accuracy rates.

	Suggested user feedback: "Stand up straight." Nonblocking.
NUI_IDENTITY_QUALITY_USER_OCCLUDED_FACE	The player's hand is too close to the player's face, fully or partially occluding the face. This flag may fire in combination with NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE. Suggested user feedback: "Don't cover your face."
NUI_IDENTITY_QUALITY_USER_OCCLUDED_BODY	The player's arms are in front of the player's torso, causing color recognition to fail. This quality issue does not block the identity operation, but it may lead to reduced accuracy rates. Suggested user feedback: "Put your hands down." Nonblocking.
NUI_IDENTITY_QUALITY_USER_FAR_AWAY	The player is outside of the recommended distance for optimal accuracy rates. This quality issue does not block the identity operation, but it may lead to reduced accuracy rates. Suggested user feedback: "Move closer to the sensor." You can provide visual indication of the direction to move. Nonblocking.
NUI_IDENTITY_QUALITY_USER_CLOSE	The player is too close to the sensor for accurate recognition. This quality issue does not block the identity operation, but it may lead to reduced accuracy rates. Suggested user feedback: "Move away from the sensor." You can provide visual indication of the direction to move. Nonblocking.
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_LEFT	The player is standing too far to the left in the sensor's field of view, and the player's image is partially clipped. Suggested user feedback: "Move to the right." You can provide visual indication of the direction to move.
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_RIGHT	The player is standing too far to the right in the sensor's field of view, and the player's image is partially clipped. Suggested user feedback: "Move to the left." You can provide visual indication of the direction to move.
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_TOP	The view of the player's face is cut off by the top of the sensor's field of view, and the player's image is partially clipped. This flag may fire in combination with NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE. There is no suggested user feedback in this case. This flag indicates a playspace issue, and should be addressed according to the title's plan for managing the playspace.
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_BOTTOM	The player's hip is below the bottom of the sensor's field of view. There is no suggested user feedback in this case. This flag indicates a playspace issue, and should be addressed according to the title's plan for managing the playspace.
NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK	The player's face is not lit well enough for the sensor to find a reasonable exposure level. This flag may fire in combination with NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE. Suggested user feedback: "Make sure your room is well lit."
NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_BRIGHT	The player's face is too bright for the camera to find a reasonable exposure level. In practice, this flag is only triggered when a bright light is pointing at the sensor. This flag may fire in combination with NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE. Suggested user feedback: "Avoid pointing lights directly at the sensor."

Remarks

When a skeleton comes into view, the NUI identity system calls [NuiIdentityIdentify](#) and sets these quality flags for the title, each time a frame of skeleton data finishes processing. Call [NuiIdentityGetNextMessage](#) to retrieve the quality flags. The values are combined using bitwise-OR. Use the quality flags to guide the player to adjust their stance and environment, so that the NUI identity system is able to perform face and skeleton recognition.

If automatic identity handling is not enabled, to obtain the quality issue flags, call [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#), passing a [PFN_NUI_IDENTITY_CALLBACK](#) function that you write. The callback function takes a [NUI_IDENTITY_MESSAGE](#) structure as a parameter. Each time [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#) processes a frame, the [NUI_IDENTITY_MESSAGE](#) structure contains a [NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#) structure, which contains a **dwQualityFlags** field that contains quality flags.

These flags indicate that there are some image quality issues interfering with the skeleton identification or enrollment process. Test for these values and then display a user interface to help the player resolve the problems.

These flag conditions block the identity operation, except where "nonblocking" is indicated.

Flags that begin with "NUI_IDENTITY_QUALITY_ENVIRONMENT" represent interference because of the environment. A named mask that groups these flags is [NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK](#).

Flags that begin with "NUI_IDENTITY_QUALITY_USER" represent interference because of something about the player. A named mask that groups these flags is [NUI_IDENTITY_QUALITY_USER_MASK](#).

Presenting Feedback to Players

Use your own user feedback text or approach rather than the suggested user feedback text, if appropriate for your title.

You should handle quality issues related to playspace by using your title's existing strategy for managing the playspace rather than by presenting an extra string that is directly related to identity.

See Also

[NUI Identity Overview](#)

NUI_IDENTITY_QUALITY_USER_MASK

[See Also](#)



Groups the NUI_IDENTITY_QUALITY flags that are about user issues.

Name	Description
NUI_IDENTITY_QUALITY_USER_MASK	Groups the NUI_IDENTITY_QUALITY flags that are about user issues.

Remarks

This #define value is a mask that groups the [NUI_IDENTITY_QUALITY](#) flags whose names begin with "NUI_IDENTITY_QUALITY_USER."
You can use this mask to separate environment and user issues. For example:

```
if (dwQualityFlags & NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK)
{
    EnvironmentError();
}
else if (dwQualityFlags & NUI_IDENTITY_QUALITY_USER_MASK)
{
    UserError();
}
```

See Also

- [NUI Identity Overview](#)
- [NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageCamera Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiImageCamera.h, which enables camera and image frame processing for Kinect.

In This Section

- [NuiImageCamera Functions](#)
- [NuiImageCamera Structures](#)
- [NuiImageCamera Enumerations](#)
- [NuiImageCamera Constants](#)

See Also

- [NUI Image Overview](#)
- [Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageCamera Functions

[See Also](#)



In This Section

- [NuiCameraAdjustTilt](#)
Gets information about detected player and object locations, and adjusts the camera angle before gameplay in order to track the relevant portion of the skeleton.
- [NuiCameraElevationGetAngle](#)
Gets the elevation angle of the sensor array, whether the elevation angle changed, and the cause of the movement.
- [NuiCameraElevationSetAngle](#)
Directly sets the angle of the Kinect sensor array. This requires a privilege bit.
- [NuiCameraElevationStopMovement](#)
Stops the tilt motor of the Kinect sensor array as quickly as possible.
- [NuiCameraGetColorImageViewArea](#)
Gets the zoom and pan settings that are set currently for the color image camera, controlling which portion of the field of view is captured by color streams.
- [NuiCameraGetExposureRegionOfInterest](#)
Gets the region that is being optimized for autoexposure.
- [NuiCameraGetNormalToGravity](#)
Gets the vector that points opposite to the direction of gravity.
- [NuiCameraGetProperty](#)
Gets an integer-valued camera property.
- [NuiCameraGetPropertyF](#)
Gets a float-valued camera property.
- [NuiCameraSetColorImageViewArea](#)
Sets the zoom and pan settings for the color image camera, thus controlling which portion of the field of view is captured by color streams.
- [NuiCameraSetExposureRegionOfInterest](#)
Specifies a region to optimize for autoexposure.
- [NuiCameraSetProperty](#)
Sets an integer-valued camera property.
- [NuiCameraSetPropertyF](#)
Sets a float-valued camera property.
- [NuiCameraSynchronizePropertyChangesBegin](#)
Begins a group of camera property changes so they take effect in the same frame.

[NuiCameraSynchronizePropertyChangesEnd](#)

Ends a group of camera property changes that take effect in the same frame.

[NuiImageGetColorPixelCoordinatesFromDepthPixel](#)

Returns the pixel coordinates in color image space that correspond to the specified pixel coordinates in depth image space.

[NuiImageStreamGetNextFrame](#)

Gets the next frame from an image stream from the Kinect sensor array.

[NuiImageStreamOpen](#)

Opens a color or depth image stream.

[NuiImageStreamReleaseFrame](#)

Releases an image frame buffer of an image stream so the buffer can be reused.

[NuiSetFrameEndEvent](#)

Sets an event to notify the title that NUI Image, Skeleton, and Gesture processing is finished on the CPU, and the title can now safely get the data.

✚ **See Also**

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraAdjustTilt

[See Also](#) [Example](#)



Gets information about detected player and object locations, and adjusts the camera angle before gameplay in order to track the relevant portion of the skeleton.

✚ **Syntax**

C++

```
DWORD NuiCameraAdjustTilt(  
    NUI_TILT_FLAGS TiltFlags,  
    FLOAT SpaceAboveHeadMeters,  
    FLOAT FarSpaceDistanceMeters,  
    FLOAT PreferredPlayspaceDistanceMeters,  
    NUI_TILT_OBJECTS *pTiltObjects,  
    XOVERLAPPED *pOverlapped  
)
```

Parameters

TiltFlags

[in] One or more [NUI_TILT_FLAGS](#) values that are used to set the camera angle.

NUI_TILT_FLAGS_NO_TILT enables you to get information about detected player and object locations. Also, it gives players the option to move to a better position before actually tilting the sensor array.

SpaceAboveHeadMeters

[in] The desired distance between the top of the player's head and the top of the frame, to provide control over the tilt clearance for near and far space.

This can be a number ranging from NUI_TILT_HEAD_SPACE_MIN to NUI_TILT_HEAD_SPACE_MAX, or a [NUI_TILT_HEAD_SPACE](#) named constant, such as NUI_TILT_HEAD_SPACE_HANDS_OVER_HEAD.

FarSpaceDistanceMeters

[in] Defines the beginning of the "far space" distance. This is the horizontal distance in world space.

If a player is found closer than this distance, the camera angle is adjusted automatically based on the player. If a player is not found closer than this distance, the camera angle is adjusted automatically based on the best view of the room rather than based on players. You can specify a custom value or [NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE](#).

For near-space tilting, only the first **Location** in the NUI_TILT_OBJECTS structure is taken into consideration for tilting. If the system reports that there are two visible objects, such as one object 6 ft away that is 3 ft tall and a second object 6.5 ft away that is 6 ft tall, the camera will tilt to best fit the closer of the two objects. If there are two players playing at the same time and you need to call this function, you should force far-space tilting.

PreferredPlayspaceDistanceMeters

[in] Tilts the camera for the specified near-space distance.

This enables defining a playspace that has consistent depth. If the value passed is 1.5 m and the player is seen at 2 m, the camera tilts as if the player is standing at 1.5 m. This means that the player can move around within the playspace without the player's head being cut off by the camera's vertical field of view if the player steps forward. This parameter only affects near-space tilting.

When this parameter is non-zero, the API calculates the tilt for the player at both *PreferredPlayspaceDistanceMeters* and *FarSpaceDistanceMeters*, and tilts to whichever is higher. In the normal case, the sensor array tilts to the player's height as if the player were standing at *PreferredPlayspaceDistanceMeters*. This prevents the view of the player's head from being cut off by the camera's vertical field of view when the player steps forward closer than this point. In the case where the camera is higher than the player's head, this would cause the head to be truncated, and this is prevented by tilting to the *FarSpaceDistanceMeters* distance.

When *PreferredPlayspaceDistanceMeters* is zero, near-space tilting is done exactly to the player's height at the player's current position without extrapolation.

pTiltObjects

[in, out, optional] An array of object locations and flags, as a pointer to a [NUI_TILT_OBJECTS](#) structure.

The first call to this function returns an array of detected object locations and associated flags. The flags indicate which **Location** is initially assumed to be the preferred target for tilting the sensor array, and whether the top of the object in each location is cut off from view. You can then move the "preferred" flag to a different **Location**, and flag other locations to ignore for tilting. You pass the modified array back into this function in a second call, which actually tilts the sensor array. The following flags are supported:

Flag	Description
NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP	Reports that the top of the player or object in the corresponding Location is not visible when using the current camera angle. This is set by the system. You can check whether the system set this flag and, if so, prompt the player to move.
NUI_TILT_OBJECT_INFLAG_IGNORE	Indicates that a player or object within 25 cm of the corresponding Location should be ignored as a target for tilting the sensor array. This is set by the title. If you set this flag on a Location , objects found at that location do not appear in the returned Objects array the next time you call this function, and are not used as targets for tilting. You can set this flag on multiple locations.
NUI_TILT_OBJECT_INFLAG_PREFERRED	Indicates that a player or object at the corresponding Location is the preferred target for tilting the sensor array. This is set by the system, and optionally set by the title. The first time you call this function, the function sets this flag on the nearest Location , which is the first entry in the Objects array. You can then move this flag to a different Location before passing the array back into this function. The function uses and preserves your setting. The next time the function returns the array, the object closest to your preferred location is the first in the returned array of objects. You can only specify a single NUI_TILT_OBJECT_INFLAG_PREFERRED location.

pOverlapped

[in, out, optional] A pointer to the [XOVERLAPPED](#) structure used to monitor the completion status of an asynchronous retrieval operation.

If you specify NULL for this parameter, this function behaves synchronously. This function will not return immediately, even if you specify NUI_TILT_FLAGS_NO_TILT in *TiltFlags*. It

does its processing on the next received depth map, so the fastest it can return when specifying NUI_TILT_FLAGS_NO_TILT is up to 33 ms later.

Return Value

One of the following values:

Value	Meaning
ERROR_SUCCESS	The function completed successfully.
ERROR_IO_PENDING	The overlapped operation has started, but has not yet completed. See Xbox Asynchronous Programming .
ERROR_RETRY	The function was called too soon after the previous call.
ERROR_TOO_MANY_CMDS	A player was not found after repeated tries, so a break was inserted.
ERROR_BUSY	The camera is busy and not responding to this call.

Remarks

You should tilt the camera as few times as possible, to minimize wear on the camera and to minimize tilting time. If tilting is required, the camera will need to tilt up farther before it tilts back down to the proper angle. It is possible for this function to identify and tilt on inanimate objects such as chairs, lamps, or plants. The object location tracking does not key off of skeleton tracking. An object could be a lamp, a chair, or a player that has not been identified yet such as a skeleton that is visible only from the waist down.

Intended Sequence of Calls

- To determine what objects the sensor array detects at which locations, call this function the first time without tilting the camera. In *TiltFlags*, specify [NUI_TILT_FLAGS_NO_TILT](#). In *pTiltObjects*, pass in a pointer to an empty [NUI_TILT_OBJECTS](#) structure.
- Examine the returned, populated NUI_TILT_OBJECTS structure to determine where the players are. Determine which **Location** has the [NUI_TILT_OBJECT_INFLAG_PREFERRED](#) flag set. You need to do this to see what the system assumes is the most likely target for tilting. The system usually sets this flag on the first member of the array.
- For a **Location** you care about, check whether the system set the flag [NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP](#). If it did, you know the top of the player is out of view. If the flag is set, prompt the player to move to a better position.
- Set the NUI_TILT_OBJECT_INFLAG_PREFERRED flag on the **Location** you want to prefer as the tilt target, and clear that flag on any other **Location** where it is set.
- Set the [NUI_TILT_OBJECT_INFLAG_IGNORE](#) flag on each **Location** you want to ignore as a tilt target.
- Call this function a second time, to actually tilt the camera, passing in the adjusted NUI_TILT_OBJECTS structure. Do not pass in the flag NUI_TILT_FLAGS_NO_TILT.
- In the returned structure, check for NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP again on the **Location** that is flagged as Preferred.
- If necessary, call this function again to tilt the sensor array, and then inspect the structure to confirm that the objects are now correctly in view.

Example

The following example tests whether the system set a flag for a location Objects[i].Location:
`if (pMyTiltObjects->Objects[i].Flags & NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP)`

Example

This function is used in the [Playspace Feedback](#) sample, as follows:

```
NUI_TILT_OBJECTS m_tiltObjects;

//-----
// Creates all graphics resources and initializes rendering and animation systems.
//-----
HRESULT Sample::Initialize()
{
    ZeroMemory( &m_tiltObjects, sizeof( NUI_TILT_OBJECTS) );
}

//-----
// Adjusts the camera for the new playspace configuration.
//-----
void Sample::ChangeTiltMode()
{
    FLOAT fHeadSpace = MapTiltModeToHeadSpace(m_currentTiltMode);
    DWORD dwResult = NuiCameraAdjustTilt( NUI_TILT_FLAGS_NONE,
                                          fHeadSpace,
                                          NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                          0.0f,
                                          &m_tiltObjects,
                                          &m_ovTiltComplete );

    if ( dwResult == ERROR_IO_PENDING )
    {
        ATG::DebugSpew( "Camera tilting to %s\n", g_pstrTiltModes[m_currentTiltMode] );
        m_tiltState = TILT_STATE_BUSY;
        return;
    }
    else if ( dwResult == ERROR_RETRY )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt called too soon after previous call\n" );
    }
    else if ( dwResult == ERROR_BUSY )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt failed because camera was busy\n" );
    }
    else if ( dwResult == ERROR_TOO_MANY_CMDS )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt failed to find player candidate; waiting\n" );
    }
    else if ( dwResult == ERROR_SUCCESS )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt completed immediately - camera tilt already optimal?\n" );
        m_tiltState = TILT_STATE_IDLE;
    }
    else
    {
        ATG::FatalError( "Unexpected result from NuiCameraAdjustTilt - %x\n", dwResult );
    }
}

//-----
// Polls to find out if the camera has stopped tilting yet.
//-----
void Sample::UpdateTiltState()
{
    if ( m_tiltState == TILT_STATE_IDLE )
        return;
}
```

```

if ( m_tiltState == TILT_STATE_NOINFO )
{
    // First time through, just grab the tilt object info.
    DWORD dwResult = NuiCameraAdjustTilt( NUI_TILT_FLAGS_NO_TILT,
                                          0.0f,
                                          NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                          NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                          &m_tiltObjects,
                                          &m_ovTiltComplete );

    if ( dwResult == ERROR_IO_PENDING )
    {
        m_tiltState = TILT_STATE_BUSY;
    }
    else if ( dwResult == ERROR_SUCCESS )
    {
        m_tiltState = TILT_STATE_IDLE;
    }

    // Leave in TILT_STATE_NOINFO for all these errors:

    else if ( dwResult == ERROR_RETRY )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt called too soon after previous call\n" );
    }
    else if ( dwResult == ERROR_BUSY )
    {
        // On startup, the camera may be busy for a few seconds, so if we get ERROR_BUSY, we just eat the message
        // and try next frame.
    }
    else if ( dwResult == ERROR_TOO_MANY_CMDS )
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt failed to find player candidate; waiting\n" );
    }
    else
    {
        ATG::DebugSpew( "NuiCameraAdjustTilt failed with NO_TILT passed in; this shouldn't happen\n" );
    }

    return;
}

if ( XHasOverlappedIoCompleted(&m_ovTiltComplete) )
{
    ATG::DebugSpew( "NuiCameraAdjustTilt - completed tilt adjustment\n" );

    m_tiltState = TILT_STATE_IDLE;
}
}

```

Example

The following example shows calling this function with no-tilt option, changing the flags in the returned structure, and then calling this function again to actually tilt the camera.

// Desc: Called once per frame, the call is the entry point for animating the scene.
HRESULT Sample::Update()
{

```

    // Update skeleton tracking
    UpdateSkeletonTracking( m_pd3dDevice );

    // Get input from all the gamepads
    ATG::GAMEPAD* pGamepad = ATG::Input::GetMergedInput();

    // Exit help screen if a player presses BACK
    if( pGamepad->wPressedButtons & XINPUT_GAMEPAD_BACK )
        m_bDrawHelp = !m_bDrawHelp;

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_DPAD_LEFT )
    {
        g_fHeadSpace -= 0.05;
        if ( g_fHeadSpace < NUI_TILT_HEAD_SPACE_MIN )
        {
            g_fHeadSpace = NUI_TILT_HEAD_SPACE_MAX;
        }
    }

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_DPAD_RIGHT )
    {
        g_fHeadSpace += 0.05;
        if ( g_fHeadSpace > NUI_TILT_HEAD_SPACE_MAX )
        {
            g_fHeadSpace = NUI_TILT_HEAD_SPACE_MIN;
        }
    }

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_DPAD_DOWN )
    {
        NuiCameraElevationStopMovement();
    }

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_DPAD_UP )
    {
        g_PreferredPlayspace = (g_PreferredPlayspace == 0.0f) ? 1.3716f : 0.0f;
    }

    if ( pGamepad->fX1 > 0 )
    {
        g_CursorX += 0.025f;
    }
    else if ( pGamepad->fX1 < 0 )
    {
        g_CursorX -= 0.025f;
    }

    if ( pGamepad->fY1 > 0 )
    {
        g_CursorZ -= 0.025f;
    }
    else if ( pGamepad->fY1 < 0 )
    {

```

```

        g_CursorZ += 0.025f;
    }

    if ( pGamepad->fY2 > 0 )
    {
        if ( g_TiltState != TILT_UP )
        {
            g_TiltState = TILT_UP;
            NuiCameraElevationSetAngle(NUI_CAMERA_ELEVATION_MAXIMUM);
        }
    }
    else if ( pGamepad->fY2 < 0 )
    {
        if ( g_TiltState != TILT_DOWN )
        {
            g_TiltState = TILT_DOWN;
            NuiCameraElevationSetAngle(NUI_CAMERA_ELEVATION_MINIMUM);
        }
    }
    else
    {
        if ( g_TiltState != TILT_IDLE )
        {
            g_TiltState = TILT_IDLE;
            NuiCameraElevationStopMovement();
        }
    }

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_LEFT_SHOULDER )
    {
        LONG angle;
        DWORD moving;
        XamNuiCameraElevationSetCallback(AutotiltCallback);
        NuiCameraElevationGetAngle(&angle, &moving);
        XamNuiCameraElevationReverseAutoTilt(angle);
    }

    if ( autotiltDone )
    {
        XamNuiCameraElevationSetCallback(NULL);
        autotiltDone = FALSE;
    }

    if ( overlapped.hEvent == NULL )
    {
        overlapped.hEvent = CreateEvent(NULL, FALSE, TRUE, NULL);
    }

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_A )
    {
        if ( WaitForSingleObject(overlapped.hEvent, 0) == WAIT_OBJECT_0 )
        {
            g_TiltObjects = g_TiltObjectsQuery;
            ResetEvent(overlapped.hEvent);
            g_TiltObjectsQuery = g_TiltIgnoredPreferred;
            DWORD result = NuiCameraAdjustTilt(NUI_TILT_FLAGS_FORCE_NEAR_SPACE,
                                                g_fHeadSpace,
                                                NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                                g_PreferredPlayspace,
                                                &g_TiltObjectsQuery,
                                                &overlapped);

            if ( result != ERROR_IO_PENDING )
            {
                SetEvent(overlapped.hEvent);
                printf("AdjustTilt Failed 0x%08x\n", result);
            }
        }
    }
    else if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_X )
    {
        if ( WaitForSingleObject(overlapped.hEvent, 0) == WAIT_OBJECT_0 )
        {
            ResetEvent(overlapped.hEvent);
            g_TiltObjects = g_TiltObjectsQuery;
            g_TiltObjectsQuery = g_TiltIgnoredPreferred;
            DWORD result = NuiCameraAdjustTilt(0,
                                                g_fHeadSpace,
                                                NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                                g_PreferredPlayspace,
                                                &g_TiltObjectsQuery,
                                                &overlapped);

            if ( result != ERROR_IO_PENDING )
            {
                SetEvent(overlapped.hEvent);
                printf("AdjustTilt Failed 0x%08x\n", result);
            }
        }
    }
    else if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_Y )
    {
        DWORD i, j;
        FLOAT x, z;
        BOOL found = FALSE;

        for ( i = 0; i < g_TiltIgnoredPreferred.Count; i++ )
        {
            x = g_TiltIgnoredPreferred.Objects[i].Location.x - g_CursorX;
            z = g_TiltIgnoredPreferred.Objects[i].Location.z - g_CursorZ;
            if ( x*x + z*z < 0.25f * 0.25f )
            {
                found = TRUE;
                g_TiltIgnoredPreferred.Objects[i].Location.x = g_CursorX;
                g_TiltIgnoredPreferred.Objects[i].Location.z = g_CursorZ;
                if ( g_TiltIgnoredPreferred.Objects[i].Flags & NUI_TILT_OBJECT_INFLAG_IGNORE )
                {
                    g_TiltIgnoredPreferred.Objects[i].Flags = NUI_TILT_OBJECT_INFLAG_PREFERRED;
                }
                else if ( g_TiltIgnoredPreferred.Objects[i].Flags & NUI_TILT_OBJECT_INFLAG_PREFERRED )
                {
                    g_TiltIgnoredPreferred.Objects[i].Flags = 0;
                    for ( j = i + 1; j < g_TiltIgnoredPreferred.Count; j++ )

```

```
        {
            g_TiltIgnoredPreferred.Objects[j - 1] = g_TiltIgnoredPreferred.Objects[j];
        }
        g_TiltIgnoredPreferred.Count--;
        i--;
    }
}

if ( !found && g_TiltIgnoredPreferred.Count < NUI_TILT_MAX_OBJECTS )
{
    g_TiltIgnoredPreferred.Objects[g_TiltIgnoredPreferred.Count].Location.x =
        g_CursorX;
    g_TiltIgnoredPreferred.Objects[g_TiltIgnoredPreferred.Count].Location.z =
        g_CursorZ;
    g_TiltIgnoredPreferred.Objects[g_TiltIgnoredPreferred.Count].Flags =
        (DWORD)NUI_TILT_OBJECT_INFLAG_IGNORE;
    g_TiltIgnoredPreferred.Count++;
}
}

else if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_B )
{
    XamNuiCameraElevationSetCallback(AutotiltCallback);
    XamNuiCameraElevationAutoTilt(0/*XNUITILT_DEFAULT_AUTOTILT_DISTANCE_MM*/);
}

else
{
    if ( WaitForSingleObject(overlapped.hEvent, 0) == WAIT_OBJECT_0 )
    {
        ResetEvent(overlapped.hEvent);
        g_TiltObjects = g_TiltObjectsQuery;
        g_TiltObjectsQuery = g_TiltIgnoredPreferred;
        DWORD result = NuiCameraAdjustTilt(NUI_TILT_FLAGS_NO_TILT,
                                           g_fHeadSpace,
                                           NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE,
                                           g_PreferredPlayspace,
                                           &g_TiltObjectsQuery,
                                           &overlapped);

        if ( result != ERROR_IO_PENDING )
        {
            SetEvent(overlapped.hEvent);
            //printf("AdjustTilt Failed 0x%08x\n", result);
        }
    }
}

return S_OK;
}
```

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraElevationGetAngle

[See Also](#)

Gets the elevation angle of the sensor array, whether the elevation angle changed, and the cause of the movement.

Syntax

C++

```
HRESULT NuiCameraElevationGetAngle(
    LONG *pAngleDegrees,
    DWORD *pMovingFlags
)
```

Parameters

pAngleDegrees

[out] The current elevation angle of the sensor array, as a pointer to a LONG value.

It must be non-NULL when you call the function. After the function call returns, the LONG value contains the elevation angle.

A tilt of 0 means the camera is pointing perpendicular to gravity. The tilt is relative to gravity rather than relative to the camera base.

Valid angles range from [NUI_CAMERA_ELEVATION_MINIMUM](#) to [NUI_CAMERA_ELEVATION_MAXIMUM](#), relative to level with respect to gravity. However, the angle also is subject to the physical limitations of the camera, and will be limited by whether the camera base is level. If the camera base is resting on a tilted surface, the middle of the camera's tilt range will not correspond to a tilt angle of zero, and the camera may not be physically capable of reaching the specified angle.

The angle returned by this function might be different from the angle specified in the previous call to [NuiCameraElevationSetAngle](#). If the camera is moved so that the base is at a different angle to level, or if the camera is tilted manually, the angle reported by **NuiCameraElevationGetAngle** will change, even if no call to [NuiCameraElevationSetAngle](#) has occurred.

pMovingFlags

[out] Flags that indicate whether the camera elevation angle changed, and the cause of the movement, as a pointer to a bitwise-OR'd combination of [NUI_CAMERA_ELEVATION](#) flags.

This parameter must be non-NULL when you call the function. After the function call returns, the DWORD value contains the flag settings.

If the NUI_CAMERA_ELEVATION_MOTOR_MOVEMENT flag is set, the sensor array was moved by the tilt motor. If the NUI_CAMERA_ELEVATION_DETECTED_MOVEMENT flag is set, the sensor array was bumped or moved by a person.

To use these flags, you must poll *pMovingFlags*. These flag settings persist until slightly after the motion stops.

Return Value

One of the following values:

Value	Meaning
-------	---------

S_OK	The function completed successfully.
E_INVALIDARG	<i>pAngleDegrees</i> or <i>pMovingFlags</i> is NULL.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI currently is active. Data is not available to titles while the Guide is active.

Remarks

This function does not require skeletal tracking to be running.

The camera infers the floor plane based on the depth map, rather than based on the accelerometer. In some cases, the floor plane may not be found, or may not be perpendicular to the gravity vector.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraElevationSetAngle

[See Also](#)

Directly sets the angle of the Kinect sensor array. This requires a privilege bit.

Syntax

C++

```
HRESULT NuiCameraElevationSetAngle(  
    LONG lAngleDegrees  
)
```

Parameters

lAngleDegrees
[in] The elevation angle, in degrees.

Zero indicates that the sensor array should point exactly horizontally. Positive values indicate that the sensor array should point above the horizon, and negative values indicate that the sensor array should point below the horizon. Valid angles range from [NUI_CAMERA_ELEVATION_MINIMUM](#) to NUI_CAMERA_ELEVATION_MAXIMUM, relative to level with respect to gravity.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully. Camera movement was successfully initiated.
E_INVALIDARG	If <i>lAngleDegrees</i> is outside the accepted range, E_INVALIDARG is returned, and the camera does not move.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Elevation may not be changed by titles while the Guide is active.

Remarks

Because this function requires a privilege bit, you should use [NuiCameraAdjustTilt](#) instead of this function whenever possible.

This function raises or lowers the sensor array's elevation to the specified value. To use this function, you must get permission from your developer account manager, and then set the Camera Angle privilege bit in the XEX header. Otherwise, calls to this routine will fail, and you will get an ERROR_ACCESS_DENIED error. For more information about how to set this privilege bit, see [ImageXEX](#) and [privilege](#).

This function initiates camera motion and then returns immediately. The actual motion may take up to two seconds to complete after this function has returned. To check whether the camera is in motion due to a previous call to **NuiCameraElevationSetAngle**, test whether the *pMovingFlags* output parameter of [NuiCameraElevationGetAngle](#) is set to [NUI_CAMERA_ELEVATION_MOTOR_MOVEMENT](#).

Valid angles range from [NUI_CAMERA_ELEVATION_MINIMUM](#) to NUI_CAMERA_ELEVATION_MAXIMUM, relative to level with respect to gravity. However, the angle is also subject to the physical limitations of the camera, and is affected by whether the camera base is level. If the camera base is resting on a tilted surface, the middle of the camera's tilt range will not correspond to a tilt angle of zero, and the camera may not be physically capable of reaching the specified angle relative to the ground plane. You should not assume that the angle that is returned by [NuiCameraElevationGetAngle](#) will be the same as the angle that was specified previously when you called **NuiCameraElevationSetAngle**.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraElevationStopMovement

[See Also](#)

Stops the tilt motor of the Kinect sensor array as quickly as possible.

⊕ Syntax

C++

```
HRESULT NuiCameraElevationStopMovement ()
```

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Elevation cannot be changed by titles while the Guide is active.

⊕ Remarks

This function aborts a [NuiCameraAdjustTilt](#), [NuiCameraElevationSetAngle](#), or system autotilt operation, if such an operation is in progress.

⊕ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

⊕ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraGetColorImageViewArea

[See Also](#)



Gets the zoom and pan settings that are set currently for the color image camera, controlling which portion of the field of view is captured by color streams.

⊕ Syntax

C++

```
HRESULT NuiCameraGetColorImageViewArea (
    NUI_IMAGE_VIEW_AREA *pViewArea
)
```

Parameters

pViewArea

[out] The zoom and pan settings of the color image, as a pointer to a [NUI_IMAGE_VIEW_AREA](#) structure. It must be non-NULL when you call the function. After the function call returns, the structure contains the settings.

This function returns the value that was set last, regardless of whether the value has taken effect yet.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	The pViewArea parameter is NULL.
E_NUI_DEVICE_NOT_CONNECTED	The device is not connected.
E_NUI_DEVICE_NOT_READY	The device is not ready.

⊕ Remarks

The color image view area controls which portion of the field of view is captured by all the color streams. This setting takes effect asynchronously, so several frames might be displayed before the setting takes effect. The *ViewArea* field in the [NUI_IMAGE_FRAME](#) structure will reflect the settings for each captured frame.

⊕ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

⊕ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraGetExposureRegionOfInterest

[See Also](#)



Gets the region that is being optimized for autoexposure.

⊕ Syntax

C++

```
HRESULT NuiCameraGetExposureRegionOfInterest(  
    NUI_CAMERA_TYPE CameraType,  
    NUI_CAMERA_AE_ROI *pRegionOfInterest  
)
```

Parameters

CameraType
[in] Specifies which camera to query, identified by [NUI_CAMERA_TYPE](#). This value must be NUI_CAMERA_TYPE_COLOR.

pRegionOfInterest
[out] The region of interest to optimize for autoexposure, as a pointer to a [NUI_CAMERA_AE_ROI](#) structure. It must be non-NULL when you call the function. After the function call returns, the structure contains the dimensions of the region.

Return Value

S_OK if successful, or an error code otherwise. Returns E_INVALIDARG if **pRegionOfInterest** is NULL.

Remarks

This function returns the values that were set most recently, although several frames might be displayed before autoexposure is applied to the new region of interest. Changing the region of interest frequently (for example, when tracking heads) is usually counterproductive, and may result in oscillation of exposure. The exposure region of interest is relative to the current view area, and not necessarily the full field of view.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraGetNormalToGravity

[See Also](#)



Gets the vector that points opposite to the direction of gravity.

Syntax

C++

```
HRESULT NuiCameraGetNormalToGravity(  
    XMVECTOR *pxmvNormal  
)
```

Parameters

pxmvNormal
[out] A unit vector pointing up, relative to gravity, as a pointer to an [XMVECTOR](#).
This pointer must be non-NULL when you call the function. After the function call returns, the XMVECTOR contains the vector. If this function does not return S_OK, this data is invalid.
This value is independent of the angle of the surface on which the camera is resting.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Data is not available to titles while the Guide is active.

Remarks

The Kinect sensor array measures the direction of gravity by using its accelerometer. This function returns the vector that is normal to the local gravity gradient. Also, this vector is available as the **vNormalToGravity** member of the [NUI_SKELETON_FRAME](#) structure.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraGetProperty

[See Also](#)



Gets an integer-valued camera property.

Syntax

C++

```
HRESULT NuiCameraGetProperty(  
    NUI_CAMERA_TYPE CameraType,  
    NUI_CAMERA_PROPERTY eCameraProperty,  
    LONG *lValue  
)
```

Parameters

CameraType
[in] Specifies which [NUI_CAMERA_TYPE](#) has the property of interest.

eCameraProperty
[in] Specifies which property value to retrieve, as a [NUI_CAMERA_PROPERTY](#) value.

lValue
[out] The property value, as a pointer to a LONG.
This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the property value. If this function does not return S_OK, this data is invalid.

Return Value
S_OK if successful, or an error code otherwise.

+ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

+ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraGetPropertyF

[See Also](#)



Gets a float-valued camera property.

+ Syntax

C++

```
HRESULT NuiCameraGetPropertyF(  
    NUI_CAMERA_TYPE CameraType,  
    NUI_CAMERA_PROPERTYF eCameraProperty,  
    FLOAT *pfValue  
)
```

Parameters

CameraType
[in] Specifies which [NUI_CAMERA_TYPE](#) has the property of interest.

eCameraProperty
[in] The [NUI_CAMERA_PROPERTYF](#) value that specifies which property value to retrieve.

pfValue
[out] The property value, as a pointer to a FLOAT.
This pointer must be non-NULL when you call the function. After the function call returns, the FLOAT contains the property value. If this function does not return S_OK, this data is invalid.

Return Value
S_OK if successful, or an error code otherwise.

+ Remarks

When autoexposure is on, you cannot get the color gain or exposure time settings. To see what values were set and used by autoexposure, turn off autoexposure, and then get the desired properties.

+ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

+ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSetColorImageViewArea

[See Also](#) [Example](#)



Sets the zoom and pan settings for the color image camera, thus controlling which portion of the field of view is captured by color streams.

+ Syntax

C++

```
HRESULT NuiCameraSetColorImageViewArea(  
    CONST NUI_IMAGE_VIEW_AREA *pCViewArea  
)
```


Parameters

pcViewArea
[in] The zoom and pan settings of the color still image, as a pointer to a [NUI_IMAGE_VIEW_AREA](#) structure.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	One of the following: - The required parameter <i>pcViewArea</i> is NULL. - NUI_IMAGE_DIGITAL_ZOOM_1X is not centered. - NUI_IMAGE_DIGITAL_ZOOM_2X values are out of range. - The specified zoom is not supported.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_DEVICE_NOT_READY	The Kinect sensor array is not ready.

Remarks

The color image view area controls which portion of the field of view is captured by all the color streams. The *ViewArea* field in the [NUI_IMAGE_FRAME](#) structure will reflect the settings for each captured frame.

This setting takes effect asynchronously, which means the function returns before the change takes effect. When you change the view area, typically one frame is dropped (but sometimes none are dropped). Then, the next one or two frames are marked with the flag [NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN](#), in the **dwFrameFlags** member of the [NUI_IMAGE_FRAME](#) structure. This flag indicates that the view area is unreliable. Unreliable means it is old when it should be new or it is new when it should be old, though the view area is still populated with a best guess.

If you use per-pixel registration to line up skeletons on a color frame ([NuiImageGetColorPixelCoordinatesFromDepthPixel](#)), and you change the view area, you must check the NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN flag. If you do not check this flag in that scenario, one or two frames will have alignment errors when you change the view area. **NuiImageGetColorPixelCoordinatesFromDepthPixel** might return the incorrect coordinates. If you rely on it, your title will have problems.

If you set the center X,Y values to odd values, the driver will truncate them automatically to even values because the device needs even values in order to have a properly zoomed frame. Image quality is not degraded when it is zoomed. The image from the color image camera is shown at its natural resolution. The unzoomed image is constructed by running a hardware binning algorithm from multiple sensor pixels. Zoomed images in low light may have lower quality because the sensor is collecting 1/4 of the light for each pixel that is displayed. Depending on the exposure settings, zoomed images in low light may have any of the following limitations:

- Motion blurring and reduced frame rate
- Grainy images, due to higher gain settings
- Dark images, due to underexposure

Example

This function is used in the [Simple Skeleton Tracking](#) sample, approximately as follows.

```
//-----
// Called once per frame to animate the scene.
//-----
HRESULT Sample::Update()
{
    // Update skeleton tracking
    UpdateSkeletonTracking( m_pd3dDevice );

    // Get input from all the gamepads
    ATG::GAMEPAD* pGamepad = ATG::Input::GetMergedInput();

    ...

    if ( pGamepad->wPressedButtons & XINPUT_GAMEPAD_Y )
    {
        g_bHeadTracking = !g_bHeadTracking;

        if ( g_bHeadTracking == FALSE )
        {
            NUI_IMAGE_VIEW_AREA ViewArea;
            ViewArea.lCenterX = 0;
            ViewArea.lCenterY = 0;
            ViewArea.eDigitalZoom = NUI_IMAGE_DIGITAL_ZOOM_1X;
            NuiCameraSetColorImageViewArea( &ViewArea );
        }
    }

    if ( g_bHeadTracking )
    {
        NUI_IMAGE_VIEW_AREA ViewArea;
        ViewArea.lCenterX = 0;
        ViewArea.lCenterY = 0;
        ViewArea.eDigitalZoom = NUI_IMAGE_DIGITAL_ZOOM_2X;

        for ( INT i = 0; i < NUI_SKELETON_COUNT; ++i )
        {
            if ( g_SkeletonFrame.SkeletonData[i].eTrackingState == NUI_SKELETON_TRACKED )
            {
                NUI_SKELETON_DATA *pSkeletonData = &g_SkeletonFrame.SkeletonData[i];
                if ( pSkeletonData->eSkeletonPositionTrackingState[NUI_SKELETON_POSITION_HEAD] != NUI_SKELETON_POSITION_NOT_TRACKED )
                {
                    LONG plDepthX, plDepthY, plColorX, plColorY;
                    USHORT usDepthValue;
                    NuiTransformSkeletonToDepthImage( pSkeletonData->SkeletonPositions[ NUI_SKELETON_POSITION_HEAD ],
                                                    &plDepthX, &plDepthY, &usDepthValue );

                    NuiImageGetColorPixelCoordinatesFromDepthPixel( NUI_IMAGE_RESOLUTION_640x480,
                                                                    NULL,
                                                                    plDepthX,
                                                                    plDepthY,
                                                                    usDepthValue,
                                                                    &plColorX,
                                                                    &plColorY );

                    ViewArea.lCenterX = plColorX - 320;
                    ViewArea.lCenterY = plColorY - 240;
                    if ( ViewArea.lCenterX <= -320 ) ViewArea.lCenterX = -319;
                    if ( ViewArea.lCenterX >= 320 ) ViewArea.lCenterX = 319;
                    if ( ViewArea.lCenterY <= -240 ) ViewArea.lCenterY = -239;
                    if ( ViewArea.lCenterY >= 240 ) ViewArea.lCenterY = 239;
                }
            }
        }
    }
}
```

```
        }

        break;
    }
}

// The if statement here can be safely omitted,
// however it is recommended because the NuiCameraSetColorImageViewArea function call fails
// when it is called more frequent than the successful arrival of color frame
if ( g_bColorFrameSucceeded )
    NuiCameraSetColorImageViewArea( &ViewArea );
}

return S_OK;
}
```

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSetExposureRegionOfInterest

[See Also](#)



Specifies a region to optimize for autoexposure.

Syntax

C++

```
HRESULT NuiCameraSetExposureRegionOfInterest(
    NUI_CAMERA_TYPE CameraType,
    CONST NUI_CAMERA_AE_ROI *pRegionOfInterest
)
```

Parameters

CameraType
[in] The camera to configure, as a [NUI_CAMERA_TYPE](#) enumeration constant. This value must be NUI_CAMERA_TYPE_COLOR.
pRegionOfInterest
[in] The region to optimize for autoexposure, as a pointer to a [NUI_CAMERA_AE_ROI](#) structure.

Return Value

S_OK if successful, or an error code otherwise. Returns E_INVALIDARG if *pRegionOfInterest* is NULL.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSetProperty

[See Also](#) [Example](#)



Sets an integer-valued camera property.

Syntax

C++

```
HRESULT NuiCameraSetProperty(
    NUI_CAMERA_TYPE CameraType,
    NUI_CAMERA_PROPERTY eCameraProperty,
    LONG lValue
)
```

Parameters

CameraType
[in] The camera to configure, as a [NUI_CAMERA_TYPE](#) enumeration constant.
eCameraProperty
[in] The property to set, as a [NUI_CAMERA_PROPERTY](#) enumeration constant.
lValue
[in] The new value of the property.

Return Value

S_OK if successful, or an error code otherwise.
When using face-based autoexposure:

- If [NuiInitialize](#) was called without starting the Skeleton Tracking engine, **NuiCameraSetProperty** returns E_NUI_FEATURE_NOT_INITIALIZED.

- If [NuiInitialize](#) was called starting the Skeleton Tracking engine, but [NuiSkeletonTrackingDisable](#) was then called, **NuiCameraSetProperty** returns S_OK with a warning. See NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED in [NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES](#).

Remarks

To set multiple properties simultaneously, first call [NuiCameraSynchronizePropertyChangesBegin](#), call **NuiCameraSetProperty** or [NuiCameraSetPropertyF](#) multiple times, and then call [NuiCameraSynchronizePropertyChangesEnd](#).

Example

This function is used in the [Simple Skeleton Tracking](#) sample.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSetPropertyF

[See Also](#) [Example](#)



Sets a float-valued camera property.

Syntax

C++

```
HRESULT NuiCameraSetPropertyF(
    NUI_CAMERA_TYPE CameraType,
    NUI_CAMERA_PROPERTY eCameraProperty,
    FLOAT fValue
)
```

Parameters

CameraType
[in] The camera to configure, as a [NUI_CAMERA_TYPE](#) enumeration constant.
eCameraProperty
[in] The property to set, as a [NUI_CAMERA_PROPERTY](#) enumeration constant.
fValue
[in] The new value of the property.

Return Value

S_OK if successful, or an error code otherwise.

Remarks

To set multiple properties simultaneously, first call [NuiCameraSynchronizePropertyChangesBegin](#), call **NuiCameraSetPropertyF** or [NuiCameraSetProperty](#) multiple times, and then call [NuiCameraSynchronizePropertyChangesEnd](#).

Example

This function is used in the [Simple Skeleton Tracking](#) sample.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSynchronizePropertyChangesBegin

[See Also](#)



Begins a group of camera property changes so they take effect in the same frame.

Syntax

C++

```
HRESULT NuiCameraSynchronizePropertyChangesBegin()
```

Return Value

S_OK if successful, or an error code otherwise.

Remarks

To set multiple camera properties simultaneously, first call **NuiCameraSynchronizePropertyChangesBegin**, call [NuiCameraGetProperty](#) or [NuiCameraGetPropertyF](#) multiple times, and then call [NuiCameraSynchronizePropertyChangesEnd](#). You must use **NuiCameraSynchronizePropertyChangesEnd** to end the block. Otherwise, no property changes will be applied. If you put camera property settings inside a Begin/End block, a best effort is made to apply all the grouped property settings at once. This is done so the settings are all applied

simultaneously in the same frame when the End statement is reached. If you do not put camera property settings inside a Begin/End block, each property setting is applied to the camera, one at a time, over the course of several frames.

Nesting Begin/End Blocks

You can nest a Begin/End block within another Begin/End block. Settings are applied when the outermost block ends. Be sure to end the outer block. Otherwise, no property changes will be applied.

Interaction with Identity Calls

The identity system changes camera settings for its own purposes. The title will not receive frames that are consumed by the identity system. Attempts to change camera settings while the identity system is in control will not be committed until the identity system releases camera control back to the title. When the identity function call returns, all of the title's settings are restored.

You should avoid calling identity functions in the middle of applying a batch of settings to the video camera because the return from the identity function will act as a commit for the settings that were changed before the identity function call.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiCameraSynchronizePropertyChangesEnd

[See Also](#)



Ends a group of camera property changes that take effect in the same frame.

Syntax

C++

```
HRESULT NuiCameraSynchronizePropertyChangesEnd()
```

Return Value

S_OK if successful, or an error code otherwise.

Remarks

To set multiple properties simultaneously, first call [NuiCameraSynchronizePropertyChangesBegin](#), call [NuiCameraGetProperty](#) or [NuiCameraGetPropertyF](#) multiple times, and then call **NuiCameraSynchronizePropertyChangesEnd**.

If you use **NuiCameraSynchronizePropertyChangesBegin**, you must also use **NuiCameraSynchronizePropertyChangesEnd**. Otherwise, no property changes will be applied.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageGetColorPixelCoordinatesFromDepthPixel

[See Also](#) [Example](#)



Returns the pixel coordinates in color image space that correspond to the specified pixel coordinates in depth image space.

Syntax

C++

```
HRESULT NuiImageGetColorPixelCoordinatesFromDepthPixel(  
    NUI_IMAGE_RESOLUTION eColorResolution,  
    CONST NUI_IMAGE_VIEW_AREA *pViewArea,  
    LONG lDepthX,  
    LONG lDepthY,  
    USHORT usDepthValue,  
    LONG *plColorX,  
    LONG *plColorY  
)
```

Parameters

eColorResolution
[in] The resolution of the color image, as a [NUI_IMAGE_RESOLUTION](#) enumeration constant.

pViewArea
[in, optional] The optional zoom and pan settings of the color image, as a [NUI_IMAGE_VIEW_AREA](#) structure.

If you provide this argument, you should pass in the view area from the image frame against which you are registering pixels, rather than manually instantiating and populating the structure. This helps ensure that your settings are valid.

Suppose you use per-pixel registration to line up skeletons on a color frame ([NuiImageGetColorPixelCoordinatesFromDepthPixel](#)). In that case, if you change the view area, you must check the flag [NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN](#). If you don't check this flag in that scenario, one or two frames will have alignment errors when you change the view area. This function might return the incorrect coordinates. Therefore, you need to check this flag before you rely on the output of this function.

If you do not provide this argument, pass NULL.

IDepthX
[in] The X coordinate in depth image space.

IDepthY
[in] The Y coordinate in depth image space.

usDepthValue
[in] The depth value in depth image space. The range of valid values is defined by the constants in [NUI_IMAGE_DEPTH_VALUES](#).

plColorX
[out] The X coordinate of the pixel in color image space.
This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the X coordinate. If this function does not return S_OK, this data is invalid.
This value may be outside the bounds of the color image.

plColorY
[out] The Y coordinate of the pixel in color image space.
This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the Y coordinate. If this function does not return S_OK, this data is invalid.
This value may be outside the bounds of the color image.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	One or more argument values are invalid.
E_NUI_DEVICE_NOT_READY	The Kinect sensor array is not yet ready.

Remarks

Because depth image data and color image data come from separate sensors, pixels in the two images may not always line up exactly. The two sensors may have different fields of view, or may not be aimed precisely in the same direction. This means that a point near the edge of the depth image may correspond to a pixel just beyond the edge of the color image, or vice versa.

This function accepts coordinates outside the bounds of the depth image. It may return pixels outside the color image. This means that you can use data from the two images in combination, even when the two images do not line up completely. You must verify that the coordinates that are returned lie within the color image before using the coordinates to reference pixels in that color image.

The depth image coordinates you specify are not required to be within the bounds of the depth frame image, but they should not be too far outside the depth frame image bounds. If the coordinates are far outside the depth frame image, they are unlikely to map to coordinates inside the bounds of the color image. This function will then return color image coordinates that are unlikely to be useful.

This function may be slow, and it is intended for mapping just a few points at a time. If you require that the entire color image be mapped to the same coordinate space as the depth image, you should open an image stream of type [NUI_IMAGE_TYPE_COLOR_IN_DEPTH_SPACE](#).

Resolution and zoom parameters are provided so you can apply this function against higher-resolution color still shots.

The implementation makes use of camera-specific calibration data from the factory.

Example

This function is used in the [Simple Skeleton Tracking](#) sample.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageStreamGetNextFrame

[See Also Example](#)

Gets the next frame from an image stream from the Kinect sensor array.

Syntax

```
C++
HRESULT NuiImageStreamGetNextFrame (
    HANDLE hStream,
    DWORD dwMillisecondsToWait,
    CONST NUI_IMAGE_FRAME **ppcImageFrame
)
```

Parameters

hStream
[in] The image stream from which to get the frame, as a handle to a stream that was opened by calling [NuiImageStreamOpen](#).

dwMillisecondsToWait
[in] The timeout before returning without a new frame of data, in milliseconds.

ppcImageFrame
[out] The retrieved frame data, as a pointer to a pointer to a [NUI_IMAGE_FRAME](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the frame data. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	<i>hStream</i> or <i>ppcImageFrame</i> is NULL.

E_NUI_DEVICE_NOT_READY	The Kinect sensor array is not ready.
E_NUI_FRAME_NO_DATA	No new image frame is available yet. The most recent frame has been retrieved already by a previous call to this function. This error occurs when this function is being called more frequently than image data is being updated. In that case, you should reuse the data from the previous successful call to this function. If you call NuiImageStreamOpen passing the flag NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA , NuiImageStreamGetNextFrame will not return E_NUI_FRAME_NO_DATA errors. If you use that flag, you must never call NuiImageStreamGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap .
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_FRAME_LIMIT_EXCEEDED	Too many frames have been requested and not released.
E_PENDING	An asynchronous operation is pending.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Stream data is not available to titles while the Guide is active.

✚ **Remarks**

The Kinect sensor array returns images in one of a limited number of buffers that are allocated internally. You must release the image buffer that contains the frame data to the system by calling [NuiImageStreamReleaseFrame](#) on **ppColorImageFrame* as soon as the frame data is no longer in use. If you do not release buffers to the system in this way, eventually calls to **NuiImageStreamGetNextFrame** will fail after all of the internally allocated buffers have been returned and none have been released. You set the number of buffers in the *dwFrameLimit* parameter of [NuiImageStreamOpen](#).

Calling this Function on the Render Thread

At most, one frame of data is generated per Direct3D swap call (for example, a call to [Present](#) or [Swap](#)). The render loop will deadlock if a game is calling **NuiImageStreamGetNextFrame** more than once per swap per open stream. For each stream, you can call **NuiImageStreamGetNextFrame** once per swap, and no more. If you call more than that, you will block until you time out, and then E_PENDING will be returned. This condition will persist until you call **Swap**.

To avoid deadlocking in instances where a frame is not generated, the API will insert a E_NUI_FRAME_NO_DATA error if a frame is not started during the swap. However, this only works if you make just one call per stream between **Swap** calls. Calling with *dwMillisecondsToWait* of zero eliminates the deadlock only if the title does not wait for success before making a swap call.

Since the API does not know which thread is being used for rendering, the API cannot protect titles from abuse other than providing a safety timeout and assertion. If **NuiImageStreamGetNextFrame** is never called on the render thread, E_NUI_FRAME_NO_DATA errors may be suppressed when the stream is opened (with [NuiImageStreamOpen](#)).

✚ **Example**

This function is used in almost all of the samples, including [Simple Skeleton Tracking](#).

This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HANDLE m_hImageStream;
HANDLE m_hDepthStream;

HRESULT Sample::Initialize()
{
    // Call NuiImageStreamOpen for m_hImageStream
    // Call NuiImageStreamOpen for m_hDepthStream
}

HRESULT Sample::Update()
{
    // Get next NUI color image frame
    HRESULT hr;
    const NUI_IMAGE_FRAME* pColorImageFrame;

    hr = NuiImageStreamGetNextFrame( m_hImageStream,
                                     10,
                                     &pColorImageFrame );

    if( SUCCEEDED( hr ) )
    {
        // Process data for current frame
        m_Visualizer.SetColorTexture( pColorImageFrame->pFrameTexture );

        NuiImageStreamReleaseFrame( m_hImageStream, pColorImageFrame );
    }

    // Get next NUI depth image frame
    const NUI_IMAGE_FRAME* pDepthImageFrame;

    hr = NuiImageStreamGetNextFrame( m_hDepthStream,
                                     10,
                                     &pDepthImageFrame );

    if( SUCCEEDED( hr ) )
    {
        // Process data for current frame
        m_Visualizer.SetDepthTexture( pDepthImageFrame->pFrameTexture );

        NuiImageStreamReleaseFrame( m_hDepthStream, pDepthImageFrame );
    }

    return S_OK;
}
```

✚ **Requirements**

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

✚ **See Also**

[NUI Image Overview](#)

NuiImageStreamOpen

[See Also Example](#)



Opens a color or depth image stream.

Syntax

C++

```
HRESULT NuiImageStreamOpen(
    NUI_IMAGE_TYPE eImageType,
    NUI_IMAGE_RESOLUTION eResolution,
    DWORD dwImageFrameFlags,
    DWORD dwFrameLimit,
    HANDLE hNextFrameEvent,
    HANDLE *phStreamHandle
)
```

Parameters

eImageType
[in] The data format of the images to stream, as a [NUI_IMAGE_TYPE](#) enumeration constant.

eResolution
[in] The resolution of the images to stream, as a [NUI_IMAGE_RESOLUTION](#) enumeration constant.
For color images, this must be NUI_IMAGE_RESOLUTION_640x480. For depth and player index images, this must be NUI_IMAGE_RESOLUTION_320x240.

dwImageFrameFlags
[in] Flags controlling the preprocessing of the image.
Only [NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA](#) is supported. Zero indicates no flags.
If you specify the flag NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA, [NuiImageStreamGetNextFrame](#) will never return E_NUI_FRAME_NO_DATA errors. If you use this flag, you must never call **NuiImageStreamGetNextFrame** from the same thread that calls **IDirect3DDevice9::Present** or **IDirect3DDevice9::Swap**.

dwFrameLimit
[in] The count of frames that the caller may hold concurrently.
The runtime allocates enough buffers to keep the stream running smoothly without the title, plus *dwFrameLimit* buffers for the title.

hNextFrameEvent
[in, optional] A handle to a manual reset event for signaling the availability of the next frame.
If this argument is provided, the signal will be set whenever a new frame is available, and will be reset when [NuiImageStreamGetNextFrame](#) is called.

phStreamHandle
[out] When the function call returns, points to a handle to the open stream.
You should close this handle, when you are finished using it, by calling **CloseHandle**. See [XCloseHandle](#), which has a link to **CloseHandle** documentation.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_NUI_FEATURE_NOT_INITIALIZED	NuiInitialize was not called with parameters that allow the requested stream type.
E_INVALIDARG	One or more of the <i>eImageType</i> , <i>eResolution</i> , or <i>dwImageFrameFlags</i> arguments are invalid, or the argument values are not mutually compatible.
E_NUI_IMAGE_STREAM_IN_USE	The request's stream type is already open.

Remarks

Use of [NUI_IMAGE_TYPE_DEPTH_80x60](#) requires that skeleton tracking be enabled in [NuiInitialize](#). For example, the NUI_IMAGE_TYPE_DEPTH_80x60 stream is not available when skeleton tracking is disabled by calling **NuiInitialize** with the NUI_INITIALIZE_FLAG_NUI_GUIDE_DISABLED flag. If skeleton tracking is not enabled and *eImageType* is set to NUI_IMAGE_TYPE_DEPTH_80x60, **NuiImageStreamOpen** returns E_INVALIDARG and a detailed message is sent to the debugger.

You can open one stream for each of the image types enabled through the flags passed to [NuiInitialize](#). The image types are listed in the [NUI_IMAGE_TYPE](#) enumeration.

The NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX image merges the player index and the depth data into a single buffer of 16 bits per pixel. This buffer is always at 320×240 pixel resolution. The three least-significant bits of each element in the buffer represent player indices. The remaining bits of each element represent the depth measurement from the Kinect sensor array to the nearest visible surface, in millimeters.

If the player index is 0, there is no player. Otherwise, player 1 corresponds to the first skeleton in the corresponding [NUI_SKELETON_FRAME](#), and player 2 corresponds to the second skeleton in the corresponding [NUI_SKELETON_FRAME](#).

The NUI_IMAGE_TYPE_DEPTH image has the same format as the NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX image, except the player index is always 0. This format is only available when NUI_INITIALIZE_FLAG_CONTROLLER_AUGMENTED was passed to [NuiInitialize](#).

The depth images, with or without player index, are available only at a resolution of 320×240 and a frame rate of 30 frames per second. Color image streams are available only at a resolution of 640×480 and a frame rate of 30 frames per second.

All image types are processed on the GPU during the system's reservation, which occurs during the swap. The camera's shutter will synchronize with the vblank signal such that images arrive just in time for processing during the system's GPU reservation time. Games must call **IDirect3DDevice9::Present** at least 30 frames per second, and use a presentation interval of D3DPRESENT_INTERVAL_ONE or D3DPRESENT_INTERVAL_TWO to maintain optimal streaming behavior. If the swap does not occur in time to process a frame, the frame is dropped. The frame number is still incremented, so you can detect that a frame was dropped.

The lower frame rates provide a frame every other or every fourth frame. At these frame rates, frame numbers are skipped so that the frame numbers correspond to the frame numbers of the closest depth and player segmentation images, which always run at 30 frames per second.

This function allocates memory for buffers. The runtime allocates enough buffers to keep the stream running smoothly without the title, plus *dwFrameLimit* buffers for the title. When you call [NuiImageStreamGetNextFrame](#), you gain exclusive access to the returned buffer. You must release exclusive access to the buffer by calling [NuiImageStreamReleaseFrame](#). If you do not release buffers when you finish using them, eventually you will acquire exclusive access to all the available buffers. When you erroneously hold all buffers, [NuiImageStreamGetNextFrame](#) fails because it cannot acquire any buffers in which to return new frames.

Example

This function is used in almost all of the samples, including [Simple Skeleton Tracking](#).
This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HANDLE m_hImageStream;
HANDLE m_hDepthStream;

HRESULT Sample::Initialize()
{
    HRESULT hr;

    // Initialize NUI system
    // ...

    // Open NUI color image stream
    hr = NuiImageStreamOpen( NUI_IMAGE_TYPE_COLOR,
                            NUI_IMAGE_RESOLUTION_640x480,
                            0,
                            1,
                            NULL,
                            &m_hImageStream );

    if (FAILED(hr))
    {
    }
```

```

        ATG_PrintError( "Couldn't initialize NUI color stream\n" );
        return hr;
    }

    // Open NUI depth image stream
    hr = NuiImageStreamOpen( NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX,
        NUI_IMAGE_RESOLUTION_320x240,
        0,
        1,
        NULL,
        &m_hDepthStream );

    if (FAILED(hr))
    {
        ATG_PrintError( "Couldn't initialize NUI depth stream\n" );
        return hr;
    }

    return S_OK;
}

```

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageStreamReleaseFrame

[See Also Example](#)



Releases an image frame buffer of an image stream so the buffer can be reused.

Syntax

C++

```

HRESULT NuiImageStreamReleaseFrame(
    HANDLE hStream,
    CONST NUI_IMAGE_FRAME *pImageFrame
)

```

Parameters

hStream

[in] A handle to a stream that was opened by calling [NuiImageStreamOpen](#), and that was used to get the image frame.

pImageFrame

A pointer to the image frame that is being released. It must be a valid pointer returned by [NuiImageStreamGetNextFrame](#) for the same stream. After calling **NuiImageStreamReleaseFrame**, you must not change the pointer.

Return Value

S_OK if successful, or an error code otherwise.

Example

This function is used in almost all of the samples, including [Simple Skeleton Tracking](#).

This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```

HANDLE m_hImageStream;
HANDLE m_hDepthStream;

HRESULT Sample::Initialize()
{
    // Call NuiImageStreamOpen for m_hImageStream
    // Call NuiImageStreamOpen for m_hDepthStream
}

HRESULT Sample::Update()
{
    // Get next NUI color image frame
    HRESULT hr;
    const NUI_IMAGE_FRAME* pColorImageFrame;

    hr = NuiImageStreamGetNextFrame( m_hImageStream,
        10,
        &pColorImageFrame );

    if( SUCCEEDED( hr ) )
    {
        // Process data for current frame
        m_Visualizer.SetColorTexture( pColorImageFrame->pFrameTexture );

        NuiImageStreamReleaseFrame( m_hImageStream, pColorImageFrame );
    }

    // Get next NUI depth image frame
    const NUI_IMAGE_FRAME* pDepthImageFrame;

    hr = NuiImageStreamGetNextFrame( m_hDepthStream,
        10,
        &pDepthImageFrame );

    if( SUCCEEDED( hr ) )
    {
        // Process data for current frame
        m_Visualizer.SetDepthTexture( pDepthImageFrame->pFrameTexture );

        NuiImageStreamReleaseFrame( m_hDepthStream, pDepthImageFrame );
    }
}

```



```
        return S_OK;
    }
```

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSetFrameEndEvent

[See Also](#) [Example](#)



Sets an event to notify the title that NUI Image, Skeleton, and Gesture processing is finished on the CPU, and the title can now safely get the data.

Syntax

C++

```
HRESULT NuiSetFrameEndEvent (
    HANDLE hEvent,
    DWORD dwFrameEventFlag
)
```

Parameters

hEvent
[in] The event to signal when a new frame is ready.

dwFrameEventFlag
[in] A frame event flag. Only [NUI_FRAME_EVENT_FLAG_SUPPRESS_NO_FRAME_DATA](#) is supported.
If the frame does not process any data, the event is still signaled unless you specify NUI_FRAME_EVENT_FLAG_SUPPRESS_NO_FRAME_DATA.

Return Value

S_OK if successful, or an error code otherwise.

Remarks

You can call this function once per [NuiInitialize](#) call. The *hEvent* provided is signaled after the NUI runtime is finished processing all data associated with the previous GPU frame. When you use [NuiImageStreamGetNextFrame](#), [NuiSkeletonGetNextFrame](#), or [NuiGestureGetNextFrame](#), all stream data generated in that frame is available before *hEvent* is signaled.

This event is never reset by the NUI runtime because there is not a well-defined time to do so. This is unlike the events provided to [NuiImageStreamOpen](#) and [NuiSkeletonTrackingEnable](#). Similarly, proper operation requires an auto-reset event, instead of a manual reset event.

When pumping multiple streams with a single thread, you should use a **NuiSetFrameEndEvent** event over separate per-stream events.

The flag [NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA](#) still applies on a per-stream basis when you call [NuiImageStreamGetNextFrame](#) provided you specify that flag for the stream in [NuiImageStreamOpen](#).

The flag [NUI_SKELETON_TRACKING_FLAG_SUPPRESS_NO_FRAME_DATA](#) still applies on a per-stream basis when you call [NuiSkeletonGetNextFrame](#) provided you specify that flag for the stream in [NuiSkeletonTrackingEnable](#).

For signaling when skeleton data is available, use an event in [NuiSkeletonTrackingEnable](#). For signaling when an image stream is available, use an event in [NuiImageStreamOpen](#).

To detect gestures by using Visual Gesture Builder data, see [NuiGesture Reference](#).

Creating Separate Events for Each Stream

When processing multiple streams on multiple threads, instead of using **NuiSetFrameEndEvent**, it is more efficient to create separate events for each stream. Creating separate events for each stream provides the following advantages:

- The title doesn't have to wait until processing for skeleton tracking is done before the title can get color and depth data. You can get the color and depth data 5 ms to 7 ms earlier than using this function.
- You can get skeleton data before gesture data has been processed. You don't have to wait until gesture processing finishes to get skeleton data, reducing potentially blocking time on the title thread.

Example

This function is used in most of the samples, including [Simple Skeleton Tracking](#).

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageCamera Structures

[See Also](#)



In This Section

- [NUI_CAMERA_AE_ROI](#)
Defines a region of interest to optimize for autoexposure.
- [NUI_IMAGE_FRAME](#)
Stores information about a depth image frame and color image frame, including frame number, image type, resolution, and view area.

[NUI_IMAGE_VIEW_AREA](#)
Specifies zoom and panning settings for color still images.

[NUI_TILT_OBJECTS](#)
Stores the position of detected objects or players, whether the object is the preferred target for tilting the sensor array, and whether the object should be ignored as a target for tilting.

See Also
[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_AE_ROI Structure

[See Also](#)

Defines a region of interest to optimize for autoexposure.

Syntax

C++

```
typedef struct _NUI_CAMERA_AE_ROI {
    FLOAT Left;
    FLOAT Top;
    FLOAT Width;
    FLOAT Height;
} NUI_CAMERA_AE_ROI;
```

Members

Left
The left edge of the region for which to optimize autoexposure. Valid values range from 0, which is the leftmost positioning, to 1.0, which is the rightmost positioning. *Left* + *Width* must be <= 1.0.

Top
The top edge of the region for which to optimize autoexposure. Valid values range from 0, which is the topmost positioning, to 1.0, which is the bottommost positioning. *Top* + *Height* must be <= 1.0.

Width
The width of the region for which to optimize autoexposure. Valid values range from [NUI_CAMERA_AE_ROI_MINIMUM_WIDTH](#) to 1.0. *Left* + *Width* must be <= 1.0.

Height
The height of the region for which to optimize autoexposure. Valid values range from [NUI_CAMERA_AE_ROI_MINIMUM_HEIGHT](#) to 1.0. *Top* + *Height* must be <= 1.0.

Remarks
This structure is passed to [NuiCameraSetExposureRegionOfInterest](#) and returned by [NuiCameraGetExposureRegionOfInterest](#).
These values are a fraction of the viewable image. The center is (0.5,0.5). [NUI_CAMERA_AE_ROI_MINIMUM_WIDTH](#) and [NUI_CAMERA_AE_ROI_MINIMUM_HEIGHT](#) are greater than zero, so that the rectangle is not allowed to be too small. The sum of *Left*+*Width* or *Top*+*Height* cannot be greater than 1.0; otherwise, the specified area would extend outside the image. Any floating point value is accepted.

Requirements
Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also
[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_FRAME Structure

[See Also Example](#)

Stores information about a depth image frame and color image frame, including frame number, image type, resolution, and view area.

Syntax

C++

```
typedef struct _NUI_IMAGE_FRAME {
    LARGE_INTEGER liTimeStamp;
    DWORD dwFrameNumber;
    NUI_IMAGE_TYPE eImageType;
    NUI_IMAGE_RESOLUTION eResolution;
    IDirect3DTexture9 *pFrameTexture;
    DWORD dwFrameFlags;
    NUI_IMAGE_VIEW_AREA ViewArea;
} NUI_IMAGE_FRAME;
```

Members

liTimeStamp
The timestamp of the most recent depth image frame. This value is in milliseconds since the Xbox 360 was last turned on. The following depth image types (specified in **eImageType**) are all derived from the depth image and they share the same *liTimeStamp*:

- NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX
- NUI_IMAGE_TYPE_DEPTH
- NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE
- NUI_IMAGE_TYPE_DEPTH_IN_COLOR_SPACE
- NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_80x60
- NUI_IMAGE_TYPE_DEPTH_80x60

The [NUI_SKELETON_FRAME](#) structure also shares this *liTimeStamp* for the same *dwFrameNumber*.

A color image and a depth image that have the same *dwFrameNumber* will generally have a different *liTimeStamp*. [NUI_IMAGE_TYPE_COLOR](#) and [NUI_IMAGE_TYPE_COLOR_IN_DEPTH_SPACE](#) are derived from the same color image and share the same *liTimeStamp*, but that *liTimeStamp* is different from the *liTimeStamp* of the [DEPTH](#) image types and [NUI_SKELETON_FRAME](#), for the same *dwFrameNumber*.

dwFrameNumber

The frame number of the most recent depth image frame. This value is a counter that increments every time a new depth frame is generated in the Kinect sensor array. Color and skeleton frame data always have the same frame number as the most recent depth frame.

The **dwFrameNumber** value increments by one for every depth frame generated by the Kinect sensor array. If a title is retrieving color or skeleton frames at a slower rate than the sensor array is generating depth frames, the title sees gaps in the frame number sequence. However, a later frame will always have a higher frame number than an earlier frame.

One reason that a title would get frames at a slower rate than the sensor is generating them is that color image resolution and exposure settings affect the frame rate for color images. The color image's frame number always corresponds to the frame number of the most recent depth image. This means that when color images are being taken less frequently than depth images due to long exposure time, color image frame numbers will appear to skip values.

A title may also see gaps in the frame number sequence if a system call such as [NuiIdentityEnroll](#) or [NuiIdentityIdentify](#) is intercepting frames in order to perform system functions.

Frame numbers reset to zero every time that [NuiInitialize](#) is called or the Kinect sensor array is plugged in. This means that a title can see the frame numbers reset during gameplay if the sensor array is unplugged and replugged, even if title code did not shut down and reinitialize the NUI API itself.

eImageType

The type of image captured, as a [NUI_IMAGE_TYPE](#) enumeration constant.

eResolution

The resolution of the image frame, as a [NUI_IMAGE_RESOLUTION](#) enumeration constant. Only 640×480 resolution is supported.

pFrameTexture

A pointer to a [IDirect3DTexture9](#) interface used to access or manipulate frame data as a texture resource.

dwFrameFlags

Flags indicating factors affecting the image, logically OR'd together. The following flags are supported:

Flag	Description
NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN	This flag indicates that the view area is unreliable (that is, it is old when it should be new or it is new when it should be old), though the view area is still populated with a best guess. When you change the view area, typically one frame is dropped (but sometimes none are dropped), and then the next one or two frames are marked with this flag.

ViewArea

The zoom and pan settings for the frame, as a [NUI_IMAGE_VIEW_AREA](#) structure.

✚ **Remarks**

This structure is passed to [NuiHandlesArmsUpdate](#) and [NuiImageStreamReleaseFrame](#), and is returned by [NuiImageStreamGetNextFrame](#).

✚ **Example**

This structure is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HANDLE m_hImageStream;
ATG::NuiVisualization m_Visualizer;

HRESULT Sample::Update()
{
    // Get next NUI color image frame
    HRESULT hr;
    const NUI_IMAGE_FRAME* pColorImageFrame;

    hr = NuiImageStreamGetNextFrame( m_hImageStream,
                                     10,
                                     &pColorImageFrame );

    if( SUCCEEDED( hr ) )
    {
        // Process data for current frame
        m_Visualizer.SetColorTexture( pColorImageFrame->pFrameTexture );

        NuiImageStreamReleaseFrame( m_hImageStream, pColorImageFrame );
    }

    return S_OK;
}
```

✚ **Requirements**

Header: Declared in [NuiImageCamera.h](#). For include, use [NuiApi.h](#).

Library: Use both [nuiapi.lib](#) and [st.lib](#).

✚ **See Also**

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_VIEW_AREA Structure

[See Also](#)



Specifies zoom and panning settings for color still images.

✚ **Syntax**

```
C++

typedef struct _NUI_IMAGE_VIEW_AREA {
    NUI_IMAGE_DIGITALZOOM eDigitalZoom;
    LONG lCenterX;
    LONG lCenterY;
} NUI_IMAGE_VIEW_AREA;
```

Members

eDigitalZoom

The zoom factor to use, as a [NUI_IMAGE_DIGITALZOOM](#) enumeration constant.

lCenterX

The horizontal offset from center for panning.

ICenterY

The vertical offset from center for panning.

Remarks

This structure is passed to [NuiCameraSetColorImageViewArea](#) and [NuiImageGetColorPixelCoordinatesFromDepthPixel](#), and is returned by [NuiCameraGetColorImageViewArea](#). A [NUI_IMAGE_FRAME](#) structure contains this structure.

With NUI_IMAGE_DIGITAL_ZOOM_1X, no panning is possible, and **ICenterX** and **ICenterY** must both be zero. If both **ICenterX** and **ICenterY** are zero, the camera is centered.

With NUI_IMAGE_DIGITAL_ZOOM_2X, valid values for **ICenterX** are even values in the range [-320, 320], and valid values for **ICenterY** are even values in the range [-240, 240]. (0,0) zooms centered on the image.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TILT_OBJECTS Structure

[See Also](#)



Stores the position of detected objects or players, whether the object is the preferred target for tilting the sensor array, and whether the object should be ignored as a target for tilting.

Syntax

```
C++

typedef struct _NUI_TILT_OBJECTS {
    FLOAT CameraHeightMeters;
    DWORD Count;
    struct {
        XMVECTOR Location;
        DWORD Flags;
    }Objects[NUI_TILT_MAX_OBJECTS];
} NUI_TILT_OBJECTS, *PNUI_TILT_OBJECTS;
```

Members

CameraHeightMeters

The height of the sensor array above the floor, in meters.

Count

The number of players or objects that are tracked in this structure, in the **Objects** array.

Objects[NUI_TILT_MAX_OBJECTS]

Stores the location of each tracked object and possible flags for each tracked object, using the unnamed structure. The maximum number of players or objects tracked by this array is [NUI_TILT_MAX_OBJECTS](#).

Location

Each **Location** vector specifies the position of a player or object, as an [XMVECTOR](#). The position is specified in terms of x (the left or right offset from the center) and z (the depth, or distance from the sensor array).

Flags

[NUI_TILT_OBJECT_FLAGS](#) controlling which **Location** to use as a target for tilting, or for the system to indicate that the top of an object is not visible, bitwise-OR'd together.

The following flags are supported:

Flag	Description
NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP	Reports that the top of the player or object in the corresponding Location is cut off in the field of view when using the current camera angle. This is set by the system. This flag indicates that a particular detected object is taller than the depth map can see at the current tilt. If the object is a player, this means that the depth map can only see the lower part of the player.
NUI_TILT_OBJECT_INFLAG_IGNORE	Indicates that a player or object within 25 cm of the corresponding Location should be ignored as a target for tilting the sensor array. This is set by the title.
NUI_TILT_OBJECT_INFLAG_PREFERRED	Indicates that a player or object at the corresponding Location is the preferred target for tilting the sensor array. This is set by the system, and optionally set by the title.

Remarks

This structure is returned by [NuiCameraAdjustTilt](#) and then passed back into that function.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageCamera Enumerations

[See Also](#)



In This Section

- [NUI_CAMERA_PROPERTY](#)
Specifies which LONG-valued property of the color camera is being retrieved or set.
- [NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES](#)
Specifies whether autoexposure and auto white balance mode is on or off.
- [NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES](#)
A get-only property to let titles know when face-based autoexposure has reached a steady state.
- [NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES](#)
Defines minimum frame rates for autoexposure.
- [NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_VALUES](#)
Defines maximum frame rates.
- [NUI_CAMERA_PROPERTY_QUALITY_PRESET_VALUES](#)
Defines the amount of noise reduction.
- [NUI_CAMERA_PROPERTY](#)
Specifies which floating-point property of the color camera is being retrieved or set.
- [NUI_CAMERA_TYPE](#)
Specifies the color or depth camera.
- [NUI_IMAGE_DIGITALZOOM](#)
Specifies a zoom value.
- [NUI_IMAGE_RESOLUTION](#)
Describes the resolution of color and depth images from the Kinect sensor array.
- [NUI_IMAGE_TYPE](#)
Describes the data format of an image, such as color, depth, or depth and player index.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY Enumeration

[See Also](#)



Specifies which LONG-valued property of the color camera is being retrieved or set.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTY
{
    NUI_CAMERA_PROPERTY_AE_AWB_MODE,
    NUI_CAMERA_PROPERTY_FRAME_RATE_MAX,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET,
    NUI_CAMERA_PROPERTY_AE_FACEBASED_TRACKING_ID,
    NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS,
    NUI_CAMERA_PROPERTY_MAXIMUM,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE
} NUI_CAMERA_PROPERTY;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTY_AE_AWB_MODE	The autoexposure and auto white balance mode. The following modes are available: NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES .
NUI_CAMERA_PROPERTY_FRAME_RATE_MAX	The maximum color frame rate. The following maximum frame rates are available: NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_VALUES .
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN	The minimum frame rate when autoexposure is on. The following minimum frame rates are available: NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES .
NUI_CAMERA_PROPERTY_QUALITY_PRESET	The amount of noise reduction. The following quality presets are available: NUI_CAMERA_PROPERTY_QUALITY_PRESET_VALUES .
NUI_CAMERA_PROPERTY_AE_FACEBASED_TRACKING_ID	The title can set this property to explicitly specify which player to apply face-based autoexposure to; this property indicates which face should be well-exposed in the scene. The title is responsible for updating the tracking ID if it changes. A tracking ID of 0 means that face-based autoexposure will adjust exposure against all tracked players. For more information, see Face-Based Autoexposure .
NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS	This is a get-only property to let the title know when face-based autoexposure has reached a steady state. This property returns a NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES enumeration value. For more information, see Face-Based Autoexposure .
NUI_CAMERA_PROPERTY_MAXIMUM	The maximum value in this enumeration. Use this value to range-check values before casting integers to NUI_CAMERA_PROPERTY .
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE	Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN instead. If you need to use this value and you want to disable the "deprecated" warning, add the following to your source code: <code>#pragma warning(disable:4995)</code>

Remarks

This enumeration specifies which LONG-valued property of the color camera is being retrieved by a call to [NuiCameraGetProperty](#) or is being set by a call to [NuiCameraSetProperty](#).
With autoexposure enabled, the frame rate may drop below the maximum for the image resolution because of low lighting conditions. Allowable frame rates are fixed at 30, 15, 10, 7.5, and 5 frames per second. When frame rates are lower than 30 frames per second, frame numbers are skipped such that frame numbers on color images match frame numbers on depth images, which always runs at 30 frames per second.

When using autoexposure, you can specify a fixed frame-rate or a range of frame-rates. To specify a fixed frame-rate for using autoexposure, make the min and max the same:

```
NuiSetProperty(NUI_CAMERA_TYPE_COLOR,
               NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN,
               NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_15FPS);
NuiSetProperty(NUI_CAMERA_TYPE_COLOR,
               NUI_CAMERA_PROPERTY_FRAME_RATE_MAX,
               NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_15FPS);
```

To specify a range of frame-rates for using autoexposure, make the min and max different:

```
NuiSetProperty(NUI_CAMERA_TYPE_COLOR,
               NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN,
               NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_10FPS);
NuiSetProperty(NUI_CAMERA_TYPE_COLOR,
               NUI_CAMERA_PROPERTY_FRAME_RATE_MAX,
               NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_15FPS);
```

You can specify a minimum autoexposure frame rate that is larger than the frame rate. When this happens, the actual frame rate (NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_xxxx) is respected, but the maximum autoexposure time is artificially constrained. For example, if using YUV with NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_15FPS and NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS, under very bright light, autoexposure will always choose the shorter exposure. In moderate to low light, autoexposure would generally choose to increase exposure time, but would be constrained to increase gain. You get less blur and more noise.

✚ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

✚ See Also

[NUI Image Overview](#)
[NuiCameraGetProperty](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES Enumeration

[See Also](#)



Specifies whether autoexposure and auto white balance mode is on or off.

✚ Syntax

```
C++
typedef enum _NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES
{
    NUI_CAMERA_PROPERTY_AE_AWB_MODE_STANDARD,
    NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED,
    NUI_CAMERA_PROPERTY_AE_AWB_MODE_OFF,
    NUI_CAMERA_PROPERTY_AE_AWB_MODE_DEFAULT
} NUI_CAMERA_PROPERTY_AE_AWB_MODE_VALUES;
```

✚ Constants

Constant	Description
NUI_CAMERA_PROPERTY_AE_AWB_MODE_STANDARD	Autoexposure and auto white balance are used to optimize exposure over the entire frame of camera data.
NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED	Autoexposure and auto white balance are used to optimize exposure for the faces of players whose skeletons are being tracked. To use face-based autoexposure, you must start the skeleton tracking engine, such as by passing NUI_INITIALIZE_FLAG_USES_DEPTH_AND_PLAYER_INDEX or NUI_INITIALIZE_FLAG_USES_SKELETON to NuiInitialize . If you try to set this property when the skeleton tracking engine is disabled, NuiCameraSetProperty fails, returning E_NUI_FEATURE_NOT_INITIALIZED. If you disable the skeleton tracking engine by calling NuiSkeletonTrackingDisable , NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED won't work, and will keep the existing camera settings. Calling NuiSkeletonTrackingDisable before setting the camera property to NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED will result in a warning, but NuiCameraSetProperty will return S_OK. In this case, NUI_CAMERA_PROPERTY_AE_AWB_MODE_FACEBASED will have no immediate effect; however, it will begin to work once the user starts the skeleton tracking engine again. For more information, see Face-Based Autoexposure .
NUI_CAMERA_PROPERTY_AE_AWB_MODE_OFF	Autoexposure and auto white balance are not performed.
NUI_CAMERA_PROPERTY_AE_AWB_MODE_DEFAULT	When setting autoexposure and auto white balance, the default mode is NUI_CAMERA_PROPERTY_AE_AWB_MODE_STANDARD.

✚ Remarks

These constants represent valid property values when getting or setting the [NUI_CAMERA_PROPERTY_AE_AWB_MODE](#) property.

✚ Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

✚ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES Enumeration

[See Also](#)



A get-only property to let titles know when face-based autoexposure has reached a steady state.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES
{
    NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_NOTSTARTED,
    NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_INPROGRESS,
    NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_READY,
    NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_INSUFFICIENT_LIGHT
} NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_VALUES;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_NOTSTARTED	Face-based autoexposure hasn't fully started yet, which likely means it is searching for faces.
NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_INPROGRESS	Face-based autoexposure is up and running, trying to find an optimal value for gain and exposure.
NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_READY	You have the best picture now, and face-based autoexposure is in a steady state.
NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS_INSUFFICIENT_LIGHT	The maximum gain set is too low, or the minimum frame rate is too high, for face-based autoexposure.

Remarks

The property [NUI_CAMERA_PROPERTY_AE_FACEBASED_STATUS](#) returns one of these enumeration constants.

Requirements

Header: Declared in NuiImageCamera.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

See Also

- [Face-Based Autoexposure](#)
- [NUI Image Overview](#)
- [NUI_IMAGE_VIEW_AREA](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES Enumeration

[See Also](#)



Defines minimum frame rates for autoexposure.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES
{
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_15FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_10FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_LOWEST,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_FIXED_30FPS = NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_NIGHT = NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_LOWEST,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_MIN_15FPS = NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_15FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_MIN_10FPS = NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_10FPS,
    NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_DEFAULT = NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS
} NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES, NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_VALUES;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS	Keeps the frame rate at 30 frames per second. When autoexposure is on, this setting prevents autoexposure from reducing the frame rate below 30 frames per second.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_15FPS	Specifies a minimum of 15 frames per second, when autoexposure is enabled. This setting is used to set the maximum exposure time that autoexposure will use. If that exposure time is insufficient to reach the desired exposure level, exposure will increase gain. In the normal quality color mode, this means the frame rate may vary between 15 to 30 frames per second, light permitting. In high quality, this mode delivers a fixed 15 frames per second.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_10FPS	Specifies a minimum of 10 frames per second, when autoexposure is enabled. This setting is used to set the maximum exposure time that autoexposure will use. If that exposure time is insufficient to reach the desired exposure level, exposure will increase gain. The frame rate may vary between 10 frames per second to the maximum of 30 frames per second (normal quality) or 15 frames per second (high quality), light permitting.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_LOWEST	Allows the frame rate to drop in low-light conditions.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_FIXED_30FPS	Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS instead.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_NIGHT	Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_LOWEST instead.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_MIN_15FPS	Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_15FPS instead.
NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_MIN_10FPS	Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_10FPS instead.

NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_DEFAULT

Deprecated. Use NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_30FPS instead.

Remarks

This enumeration specifies the possible values for the [NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN](#) property.

The enumeration name NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MODE_VALUES is deprecated. Use **NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN_VALUES** instead. If you need to use the deprecated enumeration members and you want to disable the "deprecated" warning, add the following to your source code:

```
#pragma warning(disable:4995)
```

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_VALUES Enumeration

[See Also](#)



Defines maximum frame rates.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_VALUES
{
    NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_30FPS,
    NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_15FPS,
    NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_10FPS,
} NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_VALUES;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_30FPS	A maximum of 30 frames per second.
NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_15FPS	A maximum of 15 frames per second.
NUI_CAMERA_PROPERTY_FRAME_RATE_MAX_10FPS	A maximum of 10 frames per second.

Remarks

This enumeration specifies the possible values for the [NUI_CAMERA_PROPERTY_FRAME_RATE_MAX](#) property.

If autoexposure is turned off, this rate still applies, as long as the manually set exposure time does not exceed the frame time at this rate. At longer exposure times, the frame rate is determined by the exposure time.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTY_QUALITY_PRESET_VALUES Enumeration

[See Also](#)



Defines the amount of noise reduction.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTY_QUALITY_PRESET_VALUES
{
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_DYNAMIC,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_MOST_AGGRESSIVE_NOISE_REDUCTION,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_AGGRESSIVE_NOISE_REDUCTION,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_MODERATE_NOISE_REDUCTION,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_CONSERVATIVE_NOISE_REDUCTION,
    NUI_CAMERA_PROPERTY_QUALITY_PRESET_NO_NOISE_REDUCTION
} NUI_CAMERA_PROPERTY_QUALITY_PRESET_VALUES;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTY_QUALITY_PRESET_DYNAMIC	Dynamically adjusts noise reduction, based on gain. This is the default.

NUI_CAMERA_PROPERTY_QUALITY_PRESET_MOST_AGGRESSIVE_NOISE_REDUCTION	The most aggressive degree of noise reduction.
NUI_CAMERA_PROPERTY_QUALITY_PRESET_AGGRESSIVE_NOISE_REDUCTION	A fairly aggressive degree of noise reduction. Useful if the title does analysis or its own filtering.
NUI_CAMERA_PROPERTY_QUALITY_PRESET_MODERATE_NOISE_REDUCTION	A moderate degree of noise reduction.
NUI_CAMERA_PROPERTY_QUALITY_PRESET_CONSERVATIVE_NOISE_REDUCTION	A small degree of noise reduction.
NUI_CAMERA_PROPERTY_QUALITY_PRESET_NO_NOISE_REDUCTION	No noise reduction.

Remarks

This enumeration specifies the possible values for the [NUI_CAMERA_PROPERTY_QUALITY_PRESET](#) property.

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTYF Enumeration

[See Also](#)

Specifies which floating-point property of the color camera is being retrieved or set.

Syntax

C++

```
typedef enum _NUI_CAMERA_PROPERTYF
{
    NUI_CAMERA_PROPERTYF_EXPOSURE_TIME,
    NUI_CAMERA_PROPERTYF_AE_EXPOSURE_COMPENSATION,
    NUI_CAMERA_PROPERTYF_COLOR_GAIN,
    NUI_CAMERA_PROPERTYF_AE_FACEBASED_MAX_GAIN,
    NUI_CAMERA_PROPERTYF_MAXIMUM
} NUI_CAMERA_PROPERTYF;
```

Constants

Constant	Description
NUI_CAMERA_PROPERTYF_EXPOSURE_TIME	The streaming exposure time in milliseconds. The default, minimum, and maximum values are defined by the constants in NUI_CAMERA_PROPERTYF_EXPOSURE_TIME Values . When autoexposure is enabled for streaming images, you cannot get or set this value. This value is ignored. When autoexposure is disabled, NuiCameraGetPropertyF returns the last value that was set (by autoexposure or by the title).
NUI_CAMERA_PROPERTYF_AE_EXPOSURE_COMPENSATION	An exposure stop value that enables you to change the target exposure level for autoexposure. Valid values are of type FLOAT. The default, minimum, and maximum values are defined by the constants in NUI_CAMERA_PROPERTYF_AE_COMPENSATION Values . This value is a logarithmic scale; adding 1.0 doubles the exposure, while subtracting 1.0 halves the exposure.
NUI_CAMERA_PROPERTYF_COLOR_GAIN	The color gain for the camera. Color channel gains are adjusted to the specified gain, or as close to the specified gain as possible while maintaining white balance. Valid values are of type FLOAT. The default value and the range of valid values are defined by NUI_CAMERA_PROPERTYF_GAIN Values . This value is a logarithmic scale; adding 1.0 doubles the gain, while subtracting 1.0 halves the gain. When autoexposure is enabled for streaming images, you cannot get or set this value. The value is ignored. When autoexposure is disabled, NuiCameraGetPropertyF returns the last value that was set (by autoexposure or by the title).
NUI_CAMERA_PROPERTYF_AE_FACEBASED_MAX_GAIN	The maximum gain value that face-based autoexposure is allowed to set. Values range from NUI_CAMERA_PROPERTYF_GAIN_MINIMUM to NUI_CAMERA_PROPERTYF_GAIN_MAXIMUM. The default value is NUI_CAMERA_PROPERTYF_GAIN_MAXIMUM. This value is a logarithmic scale; adding 1.0 doubles the gain. See NUI_CAMERA_PROPERTYF_GAIN Values . For more information, see Face-Based Autoexposure .
NUI_CAMERA_PROPERTYF_MAXIMUM	The maximum value in this enumeration. Use this value to range-check values before casting integers to NUI_CAMERA_PROPERTYF .

Remarks

This enumeration specifies which floating-point property of the color camera is being retrieved by a call to [NuiCameraGetPropertyF](#) or is being set by a call to [NuiCameraSetPropertyF](#). When autoexposure is not active, the frame rate is dependent on the exposure time, as follows.

Exposure Time (ms)	Frame Rate (fps)
≈33.3	30
>33.3 and = 66.6	15
> 66.6 and = 100.0	10
>100.0 and = 133.3	7.5
>133.3 and = 200	5
> 200	Still shots only

The nominal diagonal field of view of the depth sensor is 70 degrees. This corresponds approximately to a horizontal field of view of 58 degrees and a vertical field of view of 45 degrees. To duplicate the transformation used by the skeleton tracking system, use [NuiTransformSkeletonToDepthImage](#) and [NuiTransformDepthImageToSkeleton](#), or write your own transformation by using [NUI_CAMERA_DEPTH_IMAGE_TO_SKELETON_MULTIPLIER_320x240](#) and [NUI_CAMERA_SKELETON_TO_DEPTH_IMAGE_MULTIPLIER_320x240](#)

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)
[NuiCameraGetPropertyF](#)
[NuiCameraSetPropertyF](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_TYPE Enumeration

[See Also](#)



Specifies the color or depth camera.

Syntax

C++

```
typedef enum _NUI_CAMERA_TYPE
{
    NUI_CAMERA_TYPE_COLOR,
    NUI_CAMERA_TYPE_DEPTH
} NUI_CAMERA_TYPE;
```

Constants

Constant	Description
NUI_CAMERA_TYPE_COLOR	The color camera.
NUI_CAMERA_TYPE_DEPTH	The depth camera.

Remarks

The following functions take one of these enumeration constants as an input parameter:

- [NuiCameraGetProperty](#)
- [NuiCameraSetProperty](#)
- [NuiCameraGetPropertyF](#)
- [NuiCameraSetPropertyF](#)
- [NuiCameraGetExposureRegionOfInterest](#)
- [NuiCameraSetExposureRegionOfInterest](#)

Requirements

Header: Declared in NuiImageCamera.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_DIGITALZOOM Enumeration

[See Also](#)



Specifies a zoom value.

Syntax

C++

```
typedef enum _NUI_IMAGE_DIGITALZOOM
{
    NUI_IMAGE_DIGITAL_ZOOM_1X,
    NUI_IMAGE_DIGITAL_ZOOM_2X
} NUI_IMAGE_DIGITALZOOM;
```

Constants

Constant	Description
NUI_IMAGE_DIGITAL_ZOOM_1X	No zoom.
NUI_IMAGE_DIGITAL_ZOOM_2X	2× zoom in each linear dimension.

Remarks

A [NUI_IMAGE_VIEW_AREA](#) structure contains one of these enumeration constants.

⊕ Requirements

Header: Declared in NuiImageCamera.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

⊕ See Also

[NUI Image Overview](#)
[NUI_IMAGE_VIEW_AREA](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_RESOLUTION Enumeration

[See Also](#)



Describes the resolution of color and depth images from the Kinect sensor array.

⊕ Syntax

C++

```
typedef enum _NUI_IMAGE_RESOLUTION
{
    NUI_IMAGE_RESOLUTION_80x60,
    NUI_IMAGE_RESOLUTION_320x240,
    NUI_IMAGE_RESOLUTION_640x480
} NUI_IMAGE_RESOLUTION;
```

⊕ Constants

Constant	Description
NUI_IMAGE_RESOLUTION_80x60	80 pixels wide by 60 pixels high.
NUI_IMAGE_RESOLUTION_320x240	320 pixels wide by 240 pixels high.
NUI_IMAGE_RESOLUTION_640x480	640 pixels wide by 480 pixels high.

⊕ Remarks

These enumeration values are passed to [NuiImageGetColorPixelCoordinatesFromDepthPixel](#) and [NuiImageStreamOpen](#). A [NUI_IMAGE_FRAME](#) structure contains one of these enumeration constants.
Color images are only available in NUI_IMAGE_RESOLUTION_640x480.
Images that give depth information or combined depth and player index information are only available at the 80×60 and 320×240 pixel resolutions.

⊕ Requirements

Header: Declared in NuiImageCamera.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

⊕ See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_TYPE Enumeration

[See Also](#)



Describes the data format of an image, such as color, depth, or depth and player index.

⊕ Syntax

C++

```
typedef enum _NUI_IMAGE_TYPE
{
    NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX = 0,
    NUI_IMAGE_TYPE_COLOR,
    NUI_IMAGE_TYPE_COLOR_YUV,
    NUI_IMAGE_TYPE_DEPTH,
    NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE,
    NUI_IMAGE_TYPE_DEPTH_IN_COLOR_SPACE,
    NUI_IMAGE_TYPE_COLOR_IN_DEPTH_SPACE,
    NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_80x60,
    NUI_IMAGE_TYPE_DEPTH_80x60
} NUI_IMAGE_TYPE;
```

⊕ Constants

Constant	Description
NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX	Image from the depth camera, combined with an image generated by the skeleton tracking system that labels pixels in the depth map as belonging to specific players. The image format is a modified D3DFMT_LIN_D16 format, where the three least-significant bits of each pixel in this image give the index of this player's skeleton in the corresponding frame's NUI_SKELETON_FRAME data. (A value of zero indicates that no player is present at that location.) The rest of the bits represent distance from the depth sensor, in millimeters.

NUI_IMAGE_TYPE_COLOR	RGB image from the color camera. The image is linear X8R8G8B8 in sRGB color space, which means that gamma correction has already been applied. To correctly sample from the color image, you should use a gamma-correct D3D format. For an example of how to create such a format, see the function ATG::GetAs16SRGBFormat(D3DFMT_LIN_X8R8G8B8) in the ATG Sample Framework.
NUI_IMAGE_TYPE_COLOR_YUV	YUV image from the color camera. This stream type uses less memory than RGB streams, regardless of whether high-quality color is sent from the sensor array to the console. This stream type uses fewer GPU cycles than RGB streams, if high-quality color is sent from the sensor array to the console. U and V are subsampled. That is, you get one U and one V value for every two texels. This is why the YUV stream uses less memory than the RGB stream. The corresponding texture format is D3DFMT_LIN_UYVY. To open a YUV stream type, call NuiImageStreamOpen, passing NUI_IMAGE_TYPE_COLOR_YUV as the <i>eImageType</i> value.
NUI_IMAGE_TYPE_DEPTH	Image from the depth sensor. Pixels in this image are encoded the same as pixels from a NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX image, but with all player indices set to zero. However, if NuiInitialize is called with a flag that started the skeleton tracking engine, player indices might be set, even though they have not been explicitly requested.
NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE	Image with depth and player identification information, transformed into the spatial coordinates used in images from the color camera. The pixel at the specified coordinates in this image represents depth and player index information corresponding to the color image pixel at the same coordinates in a corresponding NUI_IMAGE_TYPE_COLOR image.
NUI_IMAGE_TYPE_DEPTH_IN_COLOR_SPACE	Image with depth information and all player indices set to zero, transformed into the spatial coordinates used in images from the color camera. The pixel at the specified coordinates in this image represents depth and player index information corresponding to the color image pixel at the same coordinates in a corresponding NUI_IMAGE_TYPE_COLOR image.
NUI_IMAGE_TYPE_COLOR_IN_DEPTH_SPACE	Image with color information, transformed into the spatial coordinates used in images from the depth sensor. The pixel at the specified coordinates in this image represents color image data corresponding to the depth image pixel at the same coordinates in a corresponding NUI_IMAGE_TYPE_DEPTH or NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX image. The image format is the same as NUI_IMAGE_TYPE_COLOR.
NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_80x60	Image with depth and player index information, in the same format as NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX, but at 80×60 pixel resolution.
NUI_IMAGE_TYPE_DEPTH_80x60	Image with depth information, in the same format as NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX, but at 80×60 pixel resolution. If NuiInitialize is called with a flag that started the skeleton tracking engine, player indices might be set, even though they have not been explicitly requested.

Remarks

This enumeration describes the data format of the image that is requested by calling [NuiImageStreamOpen](#). A [NUI_IMAGE_FRAME](#) structure contains one of these enumeration constants. [XedSetNuiColorBufferType](#) takes and returns one of these enumeration constants.

The depth and player segmentation data are combined in the pixel values of a single image. The three least significant bits of the pixel value contain the player index for the pixel. The remaining bits contain depth information, in millimeters.

Because depth image data and color image data come from separate sensors, pixels in the two images may not always line up exactly. The two sensors may have different fields of view, or may not be aimed precisely in the same direction. This means that a point near the edge of the depth image may correspond to a pixel just beyond the edge of the color image, or vice versa.

To do frame registration or green-screen effects, you can combine color and depth data in either of the following ways:

- Use frames of NUI_IMAGE_TYPE_COLOR with corresponding frames of either NUI_IMAGE_TYPE_DEPTH_IN_COLOR_SPACE or NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX_IN_COLOR_SPACE.
- Use frames of NUI_IMAGE_TYPE_COLOR_IN_DEPTH_SPACE with corresponding frames of one of the NUI_IMAGE_TYPE_DEPTH types.

Pixels near the edges of the transformed images might not have complete or valid pixel values if the relevant area is not visible to both sensors.

Requirements

Header: Declared in NuiImageCamera.h (for include, use NuiApi.h), and NuiTools.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Image Overview](#)

[NuiImageGetColorPixelCoordinatesFromDepthPixel](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiImageCamera Constants

[See Also](#)

In This Section

- [NUI_CAMERA_AE_ROI Values](#)
Values used for defining a region of interest for optimizing camera autoexposure settings.
- [NUI_CAMERA_COLOR_NOMINAL Values](#)
Values used for defining the color camera's nominal focal length and field of view.
- [NUI_CAMERA_DEPTH_NOMINAL Values](#)
Values used for defining the depth camera's nominal focal length and field of view.
- [NUI_CAMERA_ELEVATION Flags](#)
Indicates whether the NUI sensor array was moved by the tilt motor or by a person.
- [NUI_CAMERA_ELEVATION Values](#)
The highest and lowest angle that the Kinect sensor array can be tilted above and below the horizontal.
- [NUI_CAMERA_PROPERTYF_AE_COMPENSATION Values](#)
Exposure stop values used for autoexposure compensation settings for the color camera.

[NUI_CAMERA_PROPERTYF_EXPOSURE_TIME Values](#)
Exposure time values used for the color camera.

[NUI_CAMERA_PROPERTYF_GAIN Values](#)
Range of values for the color camera.

[NUI_FRAME_EVENT_FLAG_SUPPRESS_NO_FRAME_DATA](#)
Prevents signaling an event when no data was processed in the frame.

[NUI_IMAGE_COLOR_640x480_BUFFER_SIZE](#)
The size of the buffer that holds a 640×480 color image.

[NUI_IMAGE_COLOR_YUV_640x480_BUFFER_SIZE](#)
The size of the buffer that holds a 640×480 YUV color image.

[NUI_IMAGE_DEPTH_80x60_BUFFER_SIZE](#)
The size of the buffer that holds an 80×60 depth image.

[NUI_IMAGE_DEPTH_BUFFER_SIZE](#)
The size of the buffer that holds a depth image.

[NUI_IMAGE_DEPTH Values](#)
Range of values for the depth camera.

[NUI_IMAGE_FRAME Flags](#)
Flags the system might set when you call **NuiCameraSetColorImageViewArea**.

[NUI_IMAGE_PLAYER_INDEX_MASK](#)
A bit mask used to extract the player index from a depth image data pixel.

[NUI_IMAGE_PLAYER_INDEX_SHIFT](#)
The number of bits by which depth data is shifted left in a depth image pixel.

[NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA](#)
Suppresses E_NUI_FRAME_NO_DATA errors.

[NUI_IMAGE_STREAM_FRAME_LIMIT_MAXIMUM](#)
The number of frames that the caller can hold concurrently.

[NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE](#)
Used to tilt the sensor for the location where the title needs the player to stand, instead of tilting the sensor for the location where the player is standing currently.

[NUI_TILT_FLAGS](#)
Flags to control the tilting of the Kinect sensor array.

[NUI_TILT_HEAD_SPACE Values](#)
The distance between the top of the player's head and the top of the frame, in meters.

[NUI_TILT_MAX_OBJECTS](#)
Defines how many objects can be passed back to the title to evaluate as targets for tilting, and the number of locations that can be ignored or preferred as a target for tilting.

[NUI_TILT_OBJECT Flags](#)
Flags for object locations that are targets for tilting the sensor array.

See Also
[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_AE_ROI Values
[See Also](#)



Values used for defining a region of interest for optimizing camera autoexposure settings.

Name	Description
NUI_CAMERA_AE_ROI_MINIMUM_WIDTH	The minimum width of the region for which to optimize autoexposure.
NUI_CAMERA_AE_ROI_MINIMUM_HEIGHT	The minimum height of the region for which to optimize autoexposure.

Remarks
For more information about the meaning of these values, see [NUI_CAMERA_AE_ROI](#).

See Also
[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_COLOR_NOMINAL Values
[See Also](#)



Values used for defining the color camera's nominal focal length and field of view.

Name	Description
NUI_CAMERA_COLOR_NOMINAL_FOCAL_LENGTH_IN_PIXELS	The focal length of the color camera specified in pixels, assuming that the image resolution is 640x480.
NUI_CAMERA_COLOR_NOMINAL_INVERSE_FOCAL_LENGTH_IN_PIXELS	The inverse of the focal length of the color camera, specified in pixels. That is, 1/NUI_CAMERA_COLOR_NOMINAL_FOCAL_LENGTH_IN_PIXELS.
NUI_CAMERA_COLOR_NOMINAL_DIAGONAL_FOV	The nominal field of view of the color image in degrees when measuring diagonally. The actual value can vary from unit to unit.
NUI_CAMERA_COLOR_NOMINAL_HORIZONTAL_FOV	The nominal field of view of the color image in degrees when measuring horizontally.

	The actual value can vary from unit to unit.
NUI_CAMERA_COLOR_NOMINAL_VERTICAL_FOV	The nominal field of view of the color image in degrees when measuring vertically. The actual value can vary from unit to unit.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_DEPTH_NOMINAL Values

[See Also](#)



Values used for defining the depth camera's nominal focal length and field of view.

Name	Description
NUI_CAMERA_DEPTH_NOMINAL_FOCAL_LENGTH_IN_PIXELS	The focal length of the depth camera specified in pixels, assuming that the image is resolution 320x240. This constant can be used for projections and unprojections. However, NUI_CAMERA_DEPTH_IMAGE_TO_SKELETON_MULTIPLIER_320x240 and NUI_CAMERA_SKELETON_TO_DEPTH_IMAGE_MULTIPLIER_320x240 are preferred for this purpose. The use of these constants is demonstrated in NuiTransformSkeletonToDepthImage and NuiTransformDepthImageToSkeleton , which are inlined in NUIImageCamera.h.
NUI_CAMERA_DEPTH_NOMINAL_INVERSE_FOCAL_LENGTH_IN_PIXELS	The inverse of the focal length of the depth camera, specified in pixels. That is, 1/NUI_CAMERA_DEPTH_NOMINAL_FOCAL_LENGTH_IN_PIXELS.
NUI_CAMERA_DEPTH_NOMINAL_DIAGONAL_FOV	The nominal field of view of the depth image in degrees when measuring diagonally. The actual value can vary from unit to unit.
NUI_CAMERA_DEPTH_NOMINAL_HORIZONTAL_FOV	The nominal field of view of the depth image in degrees when measuring horizontally. The actual value can vary from unit to unit.
NUI_CAMERA_DEPTH_NOMINAL_VERTICAL_FOV	The nominal field of view of the depth image in degrees when measuring vertically. The actual value can vary from unit to unit.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_ELEVATION Flags

[See Also](#)



Indicates whether the NUI sensor array was moved by the tilt motor or by a person.

Name	Description
NUI_CAMERA_ELEVATION_DETECTED_MOVEMENT	Indicates that the NUI sensor array was bumped or moved by a person.
NUI_CAMERA_ELEVATION_MOTOR_MOVEMENT	Indicates that the NUI sensor array was moved by the tilt motor.

Remarks

These flags are returned by [NuiCameraElevationGetAngle](#).
These flag settings persist until slightly after the motion stops.

See Also

[NUI Image Overview](#)
[NUI_SKELETON_FRAME_FLAG_CAMERA_MOTION](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_ELEVATION Values

[See Also](#)



The highest and lowest angle that the Kinect sensor array can be tilted above and below the horizontal.

Name	Description
------	-------------

NUI_CAMERA_ELEVATION_MAXIMUM	The highest angle that the Kinect sensor can be tilted above the horizontal. The angle is in degrees.
NUI_CAMERA_ELEVATION_MINIMUM	The lowest angle that the Kinect sensor can be tilted below the horizontal. The angle is in degrees.

- +

Remarks
- These values constrain the values returned by [NuiCameraElevationGetAngle](#) and the values accepted by [NuiCameraElevationSetAngle](#).

- +

See Also
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTYF_AE_COMPENSATION Values

[See Also](#)



Exposure stop values used for autoexposure compensation settings for the color camera.	
Name	Description
NUI_CAMERA_PROPERTYF_AE_COMPENSATION_MINIMUM	The minimum exposure stop compensation setting.
NUI_CAMERA_PROPERTYF_AE_COMPENSATION_MAXIMUM	The maximum exposure stop compensation setting.
NUI_CAMERA_PROPERTYF_AE_COMPENSATION_DEFAULT	The default exposure stop compensation setting.

- +

Remarks
- These values are used for the [NUI_CAMERA_PROPERTYF_AE_EXPOSURE_COMPENSATION](#) property.
- This value is a logarithmic scale. Adding 1.0 doubles the exposure, while subtracting 1.0 halves the exposure.

- +

See Also
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTYF_EXPOSURE_TIME Values

[See Also](#)



Exposure time values used for the color camera.	
Name	Description
NUI_CAMERA_PROPERTYF_EXPOSURE_TIME_MINIMUM	The minimum exposure time of streaming images, in milliseconds.
NUI_CAMERA_PROPERTYF_EXPOSURE_TIME_MAXIMUM	The maximum exposure time of streaming images, in milliseconds. This value does not affect modes that have autoexposure enabled. For autoexposure modes, the maximum exposure time depends on NUI_CAMERA_PROPERTY_AE_FRAME_RATE_MIN and whether 50Hz or 60Hz lighting is detected.
NUI_CAMERA_PROPERTYF_EXPOSURE_TIME_DEFAULT	The default exposure time of streaming images, in milliseconds.

- +

Remarks
- These values are used for the [NUI_CAMERA_PROPERTYF_EXPOSURE_TIME](#) property.

- +

See Also
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_CAMERA_PROPERTYF_GAIN Values

[See Also](#)



Range of values for the color camera.	
Name	Description
NUI_CAMERA_PROPERTYF_GAIN_DEFAULT	The default gain property value.
NUI_CAMERA_PROPERTYF_GAIN_MAXIMUM	The maximum allowed gain property value.
NUI_CAMERA_PROPERTYF_GAIN_MINIMUM	The minimum allowed gain property value.

- +

Remarks
- These values are used for the NUI_CAMERA_PROPERTYF_COLOR_GAIN property in the [NUI_CAMERA_PROPERTYF](#) enumeration.

- +

See Also
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_FRAME_EVENT_FLAG_SUPPRESS_NO_FRAME_DATA

[See Also](#)



Prevents signaling an event when no data was processed in the frame.	
Name	Description
NUI_FRAME_EVENT_FLAG_SUPPRESS_NO_FRAME_DATA	Prevents signaling an event when no data was processed in the frame.

-  **Remarks**
- If no data was processed in the frame, the event will still be signaled, unless this constant is specified in the *dwFrameEventFlag* parameter of [NuiSetFrameEndEvent](#).

-  **See Also**
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_COLOR_640x480_BUFFER_SIZE

[See Also](#)



The size of the buffer that holds a 640×480 color image.	
Name	Description
NUI_IMAGE_COLOR_640x480_BUFFER_SIZE	The size, in bytes, of the buffer that holds a 640×480 color image.

-  **See Also**
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_COLOR_YUV_640x480_BUFFER_SIZE

[See Also](#)



The size of the buffer that holds a 640×480 YUV color image.	
Name	Description
NUI_IMAGE_COLOR_YUV_640x480_BUFFER_SIZE	The size, in bytes, of the buffer that holds a 640×480 YUV color image.

-  **See Also**
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_DEPTH_80x60_BUFFER_SIZE

[See Also](#)



The size of the buffer that holds an 80×60 depth image.	
Name	Description
NUI_IMAGE_DEPTH_80x60_BUFFER_SIZE	The size, in bytes, of the buffer that holds an 80×60 depth image.

-  **Remarks**
- The pitch for an 80×60 depth image is 256 bytes.

-  **See Also**
- [NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_DEPTH_BUFFER_SIZE

[See Also](#)



The size of the buffer that holds a depth image.

Name	Description
NUI_IMAGE_DEPTH_BUFFER_SIZE	The size, in bytes, of the buffer that holds a depth image.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_DEPTH Values

[See Also](#)



Range of values for the depth camera.

Name	Description
NUI_IMAGE_DEPTH_MAXIMUM	The largest supported depth value. (4000 << NUI_IMAGE_PLAYER_INDEX_SHIFT) NUI_IMAGE_PLAYER_INDEX_MASK The left shift and bitwise OR operation used in this definition represents a maximum depth value of 4,000 mm, appropriately shifted and masked so that it can be compared to an unshifted, but masked, value from a depth image.
NUI_IMAGE_DEPTH_MINIMUM	The smallest supported depth value. (800 << NUI_IMAGE_PLAYER_INDEX_SHIFT) The left shift and bitwise OR operation used in this definition represents a minimum depth value of 800 mm, appropriately shifted and masked so that it can be compared to an unshifted, but masked, value from a depth image.
NUI_IMAGE_DEPTH_NO_VALUE	Indicates there is no depth data.

Remarks

These values constrain the *usDepthValue* parameter of [NuiImageGetColorPixelCoordinatesFromDepthPixel](#).
The three low-order bits in depth image pixels encode player identification information. The remaining bits encode the depth, in millimeters.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_FRAME Flags

[See Also](#)



Flags the system might set when you call **NuiCameraSetColorImageViewArea**.

Name	Description
NUI_IMAGE_FRAME_FLAG_NONE	No flags were set.
NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN	Indicates that the view area is unreliable in the NUI_IMAGE_FRAME that has this flag. The zoom and pan settings might be the new settings specified in a call to NuiCameraSetColorImageViewArea , or might be the old zoom and pan settings.

Remarks

When you call [NuiCameraSetColorImageViewArea](#), the system might set the flag NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN. This setting takes effect asynchronously. The function returns before the change takes effect.

When you change the view area, typically one frame is dropped—but sometimes none are dropped. Then, the next one or two frames are marked with the flag NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN, in the **dwFrameFlags** member of the [NUI_IMAGE_FRAME](#) structure. This flag indicates that the view area is unreliable. That is, it is old when it should be new, or it is new when it should be old. In this case, the system populates the view area with a best guess.

Suppose you use per-pixel registration to line up skeletons on a color frame ([NuiImageGetColorPixelCoordinatesFromDepthPixel](#)). In that case, if you change the view area, you must check the NUI_IMAGE_FRAME_FLAG_VIEW_AREA_UNKNOWN flag. If you do not check this flag in that scenario, one or two frames will have alignment errors when you change the view area. **NuiImageGetColorPixelCoordinatesFromDepthPixel** might return the incorrect coordinates. Therefore, you should check this flag before you rely on the output of this function.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_IMAGE_PLAYER_INDEX_MASK

[See Also](#)



A bit mask used to extract the player index from a depth image data pixel.

Name	Description
------	-------------

NUI_IMAGE_PLAYER_INDEX_MASK	A bit mask used to extract the player index from a depth image data pixel.				
+ Remarks Extract the player index from a depth image pixel by combining the pixel value with NUI_IMAGE_PLAYER_INDEX_MASK using bitwise-AND. This bit mask is used by NUI_IMAGE_DEPTH_MAXIMUM. + See Also NUI Image Overview					
© 2013 Microsoft Corporation. All rights reserved. Send feedback to https://forums.xboxlive.com/. Version: 21256.3					
NUI_IMAGE_PLAYER_INDEX_SHIFT					
See Also					
The number of bits by which depth data is shifted left in a depth image pixel. <table><tr><th>Name</th><th>Description</th></tr><tr><td>NUI_IMAGE_PLAYER_INDEX_SHIFT</td><td>The number of bits by which depth data is shifted left in a depth image pixel.</td></tr></table>		Name	Description	NUI_IMAGE_PLAYER_INDEX_SHIFT	The number of bits by which depth data is shifted left in a depth image pixel.
Name	Description				
NUI_IMAGE_PLAYER_INDEX_SHIFT	The number of bits by which depth data is shifted left in a depth image pixel.				
+ Remarks This value is used by NUI_IMAGE_DEPTH_MAXIMUM and NUI_IMAGE_DEPTH_MINIMUM. The three low-order bits in depth image pixels encode player identification information. The remaining bits encode the depth, in millimeters. + See Also NUI Image Overview					
© 2013 Microsoft Corporation. All rights reserved. Send feedback to https://forums.xboxlive.com/. Version: 21256.3					
NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA					
See Also					
Suppresses E_NUI_FRAME_NO_DATA errors. <table><tr><th>Name</th><th>Description</th></tr><tr><td>NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA</td><td>Suppresses E_NUI_FRAME_NO_DATA errors.</td></tr></table>		Name	Description	NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA	Suppresses E_NUI_FRAME_NO_DATA errors.
Name	Description				
NUI_IMAGE_STREAM_FLAG_SUPPRESS_NO_FRAME_DATA	Suppresses E_NUI_FRAME_NO_DATA errors.				
+ Remarks If you pass this flag in the <i>dwImageFrameFlags</i> argument to NuiImageStreamOpen, NuiImageStreamGetNextFrame will not return E_NUI_FRAME_NO_DATA errors. If you use this flag, you must never call NuiImageStreamGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap. + See Also NUI Image Overview					
© 2013 Microsoft Corporation. All rights reserved. Send feedback to https://forums.xboxlive.com/. Version: 21256.3					
NUI_IMAGE_STREAM_FRAME_LIMIT_MAXIMUM					
See Also					
The number of frames that the caller can hold concurrently. <table><tr><th>Name</th><th>Description</th></tr><tr><td>NUI_IMAGE_STREAM_FRAME_LIMIT_MAXIMUM</td><td>The count of frames that the caller may hold concurrently.</td></tr></table>		Name	Description	NUI_IMAGE_STREAM_FRAME_LIMIT_MAXIMUM	The count of frames that the caller may hold concurrently.
Name	Description				
NUI_IMAGE_STREAM_FRAME_LIMIT_MAXIMUM	The count of frames that the caller may hold concurrently.				
+ Remarks This is the maximum value that may be passed to NuiImageStreamOpen in the <i>dwFrameLimit</i> argument. If you exceed this value, NuiImageStreamOpen returns E_INVALIDARG, and nuiapid.lib produces debug output explaining the error. + See Also NUI Image Overview					
© 2013 Microsoft Corporation. All rights reserved. Send feedback to https://forums.xboxlive.com/. Version: 21256.3					
NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE					
See Also					

Used to tilt the sensor for the location where the title needs the player to stand, instead of tilting the sensor for the location where the player is standing currently.

Name	Description
NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE	Specifies the standard distance where the title needs the player to stand.

Remarks

This distance value is passed in from the title. This value can be used as the *FarSpaceDistanceMeters* parameter of the [NuiCameraAdjustTilt](#) function.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TILT_FLAGS

[See Also](#)



Flags to control the tilting of the Kinect sensor array.	
Name	Description
NUI_TILT_FLAGS_NONE	Indicates no tilt flags.
NUI_TILT_FLAGS_FORCE_FAR_SPACE	Forces the sensor array to tilt for the typical optimal angle at a distance of 1.8 m. Used when a title wants to keep a player between 6 ft and 8 ft distance (if the room has enough space).
NUI_TILT_FLAGS_FORCE_NEAR_SPACE	Forces the sensor array to tilt for small playspaces. Used when a player doesn't have enough playspace, or to get players to stand closer to the sensor array. This flag puts the sensor array at an angle based on what the camera sees.
NUI_TILT_FLAGS_NO_TILT	Gets information about where the player is standing and populates the array of objects in the NUI_TILT_OBJECTS structure, but does not tilt the sensor array. You can set this flag and use the information that NuiCameraAdjustTilt returns in the structure to prompt the player to move to a better location, and then only tilt the sensor array once instead of twice.
NUI_TILT_FLAGS_OVERRIDE_TUNER	Uses the default best view of the room, rather than using the tuner settings.

Remarks

The *TiltFlags* input parameter of [NuiCameraAdjustTilt](#) takes a combination of these flags.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TILT_HEAD_SPACE Values

[See Also](#)



The distance between the top of the player's head and the top of the frame, in meters.	
Name	Description
NUI_TILT_HEAD_SPACE_MIN	The minimum space that is allowed between the top of the player's head and the top of the frame.
NUI_TILT_HEAD_SPACE_MAX	The maximum space that is allowed between the top of the player's head and the top of the frame.
NUI_TILT_HEAD_SPACE_FULL_SKELETON	Sets the top of the frame as close to the top of the player's head as possible, to get as much of the body as possible. Affects tilting for near space and far space.
NUI_TILT_HEAD_SPACE_UPPER_BODY	Sets the space above the player's head to 15 cm. Affects tilting for near space and far space.
NUI_TILT_HEAD_SPACE_HANDS_OVER_HEAD	Sets the space above the player's head to 30 cm. Affects tilting for near space and far space.

Remarks

The *SpaceAboveHeadMeters* input parameter of [NuiCameraAdjustTilt](#) takes one of these named constants.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.

Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TILT_MAX_OBJECTS

[See Also](#)



Defines how many objects can be passed back to the title to evaluate as targets for tilting, and the number of locations that can be ignored or preferred as a target for tilting.

Name	Description
NUI_TILT_MAX_OBJECTS	The number of objects the system can return to the title to evaluate as targets for tilting, or the number of locations that can be ignored or preferred as a target for tilting.

Remarks

When you call [NuiCameraAdjustTilt](#), you can receive and then pass back in a [NUI_TILT_OBJECTS](#) structure that contains an array of **Locations** to ignore or prefer as a target for tilting. The array can be as large as **NUI_TILT_MAX_OBJECTS**.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TILT_OBJECT Flags

[See Also](#)



Flags for object locations that are targets for tilting the sensor array.

Name	Description
NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP	Reports that the top of the player or object in the corresponding Location is cut off in the field of view when using the current camera angle.
NUI_TILT_OBJECT_INFLAG_IGNORE	Indicates that a player or object within 25 cm of the corresponding Location should be ignored as a target for tilting the sensor array.
NUI_TILT_OBJECT_INFLAG_PREFERRED	Indicates that a player or object at the corresponding Location is the preferred target for tilting the sensor array.

Remarks

These flags are used in the [NUI_TILT_OBJECTS](#) structure, which is returned by [NuiCameraAdjustTilt](#) and then passed back into that function.

See Also

[NUI Image Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeleton Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiSkeleton.h, which enables skeleton tracking for Kinect.

In This Section

[NuiSkeleton Functions](#)

[NuiSkeleton Structures](#)

[NuiSkeleton Enumerations](#)

[NuiSkeleton Constants](#)

See Also

[NUI Skeleton Tracking Overview](#)
[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeleton Functions

[See Also](#)



In This Section

[NuiSkeletonGetNextFrame](#)

Gets the next available frame of skeleton data, including data for each skeleton.

[NuiSkeletonSetTrackedSkeletons](#)

Sets which skeletons are being actively (fully) tracked.

[NuiSkeletonTrackingDisable](#)

Disables skeleton tracking by the title.

[NuiSkeletonTrackingEnable](#)

Enables skeleton tracking by the title.

[NuiTransformDepthImageToSkeleton](#)

Returns the skeleton space coordinates of a given pixel in the depth image.

[NuiTransformMatrixLevel](#)

Generates a transformation matrix that levels the skeleton space.

[NuiTransformSkeletonToDepthImage_Overloads](#)

Returns the pixel coordinates in the depth image for a given point in skeleton-space.

[NuiTransformSmooth](#)

Filters skeleton positions to reduce jitter between frames.

✚ See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeletonGetNextFrame

[See Also](#) [Example](#)



Gets the next available frame of skeleton data, including data for each skeleton.

✚ Syntax

C++

```
HRESULT NuiSkeletonGetNextFrame(  
    DWORD dwMillisecondsToWait,  
    NUI_SKELETON_FRAME *pSkeletonFrame  
)
```

Parameters

dwMillisecondsToWait

[in] The time to wait before returning without the next frame of data.

pSkeletonFrame

[out] A frame of skeleton data, as a pointer to a title-allocated buffer holding a [NUI_SKELETON_FRAME](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the buffer contains the skeleton data. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_NUI_FRAME_NO_DATA	No new skeleton frame is available yet. The most recent frame has already been retrieved by a previous call to this function. This error occurs when this function is being called more often than skeleton data is being updated. In that case, you should reuse the data from the previous successful call to this function. If you call NuiSkeletonTrackingEnable passing the flag NUI_SKELETON_TRACKING_FLAG_SUPPRESS_NO_FRAME_DATA , NuiSkeletonGetNextFrame will not return E_NUI_FRAME_NO_DATA errors. Instead, calls to NuiSkeletonGetNextFrame block until data is available or the timeout period passes. If you use that flag, you must never call NuiSkeletonGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap .
E_INVALIDARG	One or more arguments are invalid.
E_PENDING	NuiSkeletonGetNextFrame is returning because the specified timeout period has passed.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_STREAM_NOT_ENABLED	Skeleton tracking data is not available because NUI_INITIALIZE_FLAG_USES_SKELETON was not passed as a <i>Flag</i> value to NuiInitialize .
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI currently is active. Skeleton data is not available to titles while the Guide is active.

✚ Remarks

This function returns a [NUI_SKELETON_FRAME](#) structure that contains an array of [NUI_SKELETON_DATA](#) structures. Each NUI_SKELETON_DATA structure contains an array of skeleton joint positions, such as [SkeletonPositions\[NUI_SKELETON_POSITION_ELBOW_LEFT\]](#). Each skeleton joint position is an XMVECTOR that has 4 **float** values: w,x,y,z.

The skeleton tracking system runs in the background at a constant rate of 30 frames per second. When this function is called, it copies the most recent frame data to the caller's buffer.

A subsequent call will block until one of the following occurs:

- The next frame is available.
- The call times out after waiting for *dwMillisecondsToWait*.
- An error occurs in the pipeline.

Internally, this call wraps [WaitForSingleObject](#) by using an internal event signaled by the skeleton pipeline when a new frame is available. If you prefer to use your own event, primarily to use [WaitForMultipleObjects](#), you can specify an event handle (*hNextFrameEvent*) when calling [NuiSkeletonTrackingEnable](#). The next call to **NuiSkeletonGetNextFrame** after the event is signaled is guaranteed to return immediately with a frame of data or an error from the tracking pipeline, but not with a timeout.

Calling this Function on the Render Thread

At most one frame of data is generated per Direct3D swap call (for example, a call to [Present](#) or [Swap](#)). The render loop will deadlock if a game is calling **NuiSkeletonGetNextFrame** more than once per swap on the Direct3D render thread. For each stream, you can call **NuiSkeletonGetNextFrame** once per swap, and no more. If you call more than that, you will block until you timeout, and then E_PENDING will be returned. This condition will persist until you call **Swap**.

To avoid deadlocking in instances where a frame is not generated, the API will insert a E_NUI_FRAME_NO_DATA error if a frame is not started during the swap. However, this only works if you make just one call per stream between **Swap** calls. Calling with *dwMillisecondsToWait* of zero eliminates the deadlock only if the title does not wait for success before making a swap call.

Since the API does not know which thread is being used for rendering, the API cannot protect titles from abuse other than providing a safety timeout and assertion. If **NuiSkeletonGetNextFrame** is never called on the render thread, E_NUI_FRAME_NO_DATA errors may be suppressed when tracking is enabled (with [NuiSkeletonTrackingEnable](#)).

✚ Example

This function is used in most of the samples.

This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
NUI_SKELETON_FRAME m_SkeletonFrame;
```

```
HRESULT Sample::Update()
{
    HRESULT hr;

    // Get next NUI skeleton frame
    bool bSkeletonFound = FALSE;

    hr = NuiSkeletonGetNextFrame( NUI_CAMERA_TIMEOUT_DEFAULT, &m_SkeletonFrame );
    if( SUCCEEDED( hr ) )
    {
        UINT i = 0;

        hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );

        // Process the skeleton data
        while ( ( i < NUI_SKELETON_COUNT ) && bSkeletonFound != TRUE )
        {
            if (m_SkeletonFrame.SkeletonData[i].eTrackingState !=
                NUI_SKELETON_NOT_TRACKED )
            {
                m_Visualizer.SetSkeletons( &m_SkeletonFrame );
                iCurrentSkeletonIndex=i;
                bSkeletonFound= TRUE;
            }
            i++;
        }
    }

    return S_OK;
}
```

Requirements

Header: Declared in NUISkeleton.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeletonSetTrackedSkeletons

[See Also Example](#)

Sets which skeletons are being actively (fully) tracked.

Syntax

C++

```
HRESULT NuiSkeletonSetTrackedSkeletons (
    DWORD TrackingIDs[NUI_SKELETON_MAX_TRACKED_COUNT]
)
```

Parameters

TrackingIDs
[in] An array of the tracking IDs of the skeletons to be actively (fully) tracked.
A value of zero directs the skeleton tracking system to search for new skeletons and track a detected skeleton. Invalid tracking input values are treated as zeros. The size of the array is [NUI_SKELETON_MAX_TRACKED_COUNT](#).

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	The required <i>TrackingIDs</i> parameter is NULL.
E_NUI_SYSTEM_UI_PRESENT	The Xbox Guide UI is active currently. Skeleton tracking may not be changed by titles while the Guide is active.

Remarks

If this function is not called, the default behavior is to track up to two skeletons as they are found.
To disable the default player selection mode and directly manage which players have tracked skeletons, pass the flag [NUI_SKELETON_TRACKING_FLAG_TITLE_SETS_TRACKED_SKELETONS](#) in the *dwFlags* parameter of [NuiSkeletonTrackingEnable](#).

Example

This function is used in several of the samples, including [Active/Passive Skeletons](#) and [Turn Detection](#).

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeletonTrackingDisable

[See Also](#) [Example](#)

Disables skeleton tracking by the title.

+

Syntax

C++

```
HRESULT NuiSkeletonTrackingDisable()
```

Return Value

S_OK if successful, or an error code otherwise.

+

Remarks

This function releases additional resources allocated by [NuiSkeletonTrackingEnable](#).

+

Example

This function is used in several of the samples, including [AvatarMonkeySeeMonkeyDo](#).

+

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

+

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeletonTrackingEnable

[See Also](#) [Example](#)

Enables skeleton tracking by the title.

+

Syntax

C++

```
HRESULT NuiSkeletonTrackingEnable(  
    HANDLE hNextFrameEvent,  
    DWORD dwFlags  
)
```

Parameters

hNextFrameEvent
[in, optional] A handle to a title-allocated, manual reset event that will be set whenever a new frame of skeleton data is available, and will be reset whenever the latest frame data is returned. This can be NULL.

dwFlags
[in] [NUI_SKELETON_TRACKING_FLAGS](#) controlling skeleton tracking, bitwise-OR'd together.

To switch between full skeleton tracking mode and seated skeleton tracking mode, first call **NuiSkeletonTrackingDisable**, and then call **NuiSkeletonTrackingEnable** with the required flags. All active players are dropped, and then the system looks for players again. New players are assigned new tracking IDs. See [Seated Skeleton Tracking](#).

Return Value

S_OK if successful, or an error code otherwise.

+

Remarks

This function may allocate additional resources to return skeleton tracking data to titles.
This function duplicates the *hNextFrameEvent* handle. This duplicate handle is closed by [NuiSkeletonTrackingDisable](#). You can close the original handle passed to **NuiSkeletonTrackingEnable** at any time after calling **NuiSkeletonTrackingEnable**.

+

Example

This function is used in most of the samples, including [ActivePassiveSkeletons](#).
This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
HRESULT Sample::Initialize()  
{  
    HRESULT hr;  
  
    // Call NuiInitialize  
    // Call NuiImageStreamOpen to open NUI color image stream  
    // Call NuiImageStreamOpen to open NUI depth image stream  
  
    // Initialize NUI skeleton tracking  
    hr = NuiSkeletonTrackingEnable( NULL, 0 );  
  
    if (FAILED(hr))  
    {  
        ATG_PrintError( "Couldn't initialize NUI skeletal tracking stream\n" );  
        return hr;  
    }  
  
    return S_OK;  
}
```

+

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

file:///C:/Users/simon/AppData/Local/Temp/~hh15D.htm

2026-04-19

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformDepthImageToSkeleton

[See Also](#) [Example](#)



Returns the skeleton space coordinates of a given pixel in the depth image.

Syntax

C++

```
XMVECTOR NuiTransformDepthImageToSkeleton(  
    LONG lDepthX,  
    LONG lDepthY,  
    USHORT usDepthValue  
)
```

Parameters

- lDepthX*
[in] The X coordinate of the depth image pixel, in the range [0..319].
- lDepthY*
[in] The Y coordinate of the depth image pixel, in the range [0..239].
- usDepthValue*
[in] The depth value of the depth image pixel, in millimeters, shifted left by 3. The left-shift enables you to pass the value from the segmentation texture directly into this function.

Return Value

The skeleton space coordinates of the given depth image pixel, in meters.

Remarks

This function is defined as an inline function in NuiSkeleton.h.

Example

```
Example of using the usDepthValue parameter:  
USHORT usDepth = 1500; // 1.5 meters  
XMVECTOR vWorldSpace = NuiTransformDepthImageToSkeleton( x, y, usDepth << 3 );
```

Requirements

- Header:** Declared in NuiSkeleton.h. For include, use NuiApi.h.
- Library:** Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)
[NuiTransformSkeletonToDepthImage Overloads](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformMatrixLevel

[See Also](#) [Example](#)



Generates a transformation matrix that levels the skeleton space.

Syntax

C++

```
XMMATRIX NuiTransformMatrixLevel(  
    XMVECTOR vNormalToGravity  
)
```

Parameters

- vNormalToGravity*
[in] An [XMVECTOR](#) showing the direction of up in the source coordinate system.

Return Value

An [XMMATRIX](#) transformation matrix suitable for use with methods such as **XMVector4DTransform**. This matrix rotates the up vector to be equal to (0.0, 1.0, 0.0, 1.0).

Remarks

The transformation matrix is generated by rotating the up vector to be equal to (0.0, 1.0, 0.0, 1.0).

Example

This function is used in several of the samples, including [First Person](#).

Requirements

- Header:** Declared in NuiSkeleton.h. For include, use NuiApi.h.
- Library:** Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformSkeletonToDepthImage Overloads

[See Also](#)



The **NuiTransformSkeletonToDepthImage** functions are a set of overloaded functions that return the pixel coordinates in the depth image for a given point in skeleton-space.

In This Section

[NuiTransformSkeletonToDepthImage](#)

Returns the coordinates and value of the depth image pixel that corresponds to a point in skeleton space.

[NuiTransformSkeletonToDepthImage](#)

Returns the coordinates of the depth image pixel that corresponds to a point in skeleton space, enabling occlusion detection.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformSkeletonToDepthImage

[See Also](#) [Example](#)



Returns the coordinates and value of the depth image pixel that corresponds to a point in skeleton space.

Syntax

C++

```
void NuiTransformSkeletonToDepthImage(  
    XMVECTOR vPoint,  
    LONG *pDepthX,  
    LONG *pDepthY,  
    USHORT *pusDepthValue  
)
```

Parameters

vPoint

[in] The point of interest in skeleton space coordinates, as an [XMVECTOR](#). This point must have a valid depth value.

pDepthX

[out] The X coordinate of the pixel in the depth image corresponding to *vPoint*, as a pointer to a LONG.

This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the X coordinate. If this is a NULL pointer, this function returns without doing anything.

pDepthY

[out] The Y coordinate of the pixel in the depth image corresponding to *vPoint*, as a pointer to a LONG.

This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the Y coordinate. If this is a NULL pointer, this function returns without doing anything.

pusDepthValue

[out] The depth value of the pixel in the depth image corresponding to *vPoint*, as a pointer to USHORT.

This pointer must be non-NULL when you call the function. After the function call returns, the USHORT contains the depth value. If this is a NULL pointer, this function returns without doing anything.

The format of this value is the same as that for pixels in other depth images: the least significant three bits contain a player index that is always zero, and the remaining bits are the depth value in millimeters. This value is suitable for comparison with values from an image from any of the depth streams, provided player index information is masked appropriately from the other image.

Return Value

None.

Remarks

You can use this function to place virtual objects into the color image with depth-based occlusion. Call this function to determine which skeleton-space vertices of a virtual object are in front of, or behind, real objects visible in the depth map image. After occluded parts of the virtual objects have been culled, use [NuiImageGetColorPixelCoordinatesFromDepthPixel](#) to overlay the virtual object's vertices into color image pixel coordinates for rendering.

This function is defined as an inline function in NuiSkeleton.h.

The outputs are computed by applying a transform based on the optics of the depth camera. If the input point is not visible to the depth camera, the output coordinates will be outside the bounds of the depth image, and **pusDepthValue* will be invalid.

Example

This function overload is used in the following samples:

- [FrostyWindow](#)
- [SimpleSkeletonTracking](#)
- [SkeletalPostProcessing](#)
- [SoundLocationTracking](#)

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformSkeletonToDepthImage

[See Also](#) [Example](#)



Returns the coordinates of the depth image pixel that corresponds to a point in skeleton space, enabling occlusion detection.

Syntax

C++

```
void NuiTransformSkeletonToDepthImage(  
    XMVECTOR vPoint,  
    FLOAT *pfDepthX,  
    FLOAT *pfDepthY  
)
```

Parameters

vPoint
[in] The point of interest, in skeleton space coordinates, as an [XMVECTOR](#). This point must have a valid depth value.

pfDepthX
[out] The X coordinate of the pixel in the depth image corresponding to *vPoint*, as a pointer to a FLOAT.
This pointer must be non-NULL when you call the function. After the function call returns, the FLOAT contains the X coordinate. If this is a NULL pointer, this function returns without doing anything.

pfDepthY
[out] The Y coordinate of the pixel in the depth image corresponding to *vPoint*, as a pointer to a FLOAT.
This pointer must be non-NULL when you call the function. After the function call returns, the FLOAT contains the Y coordinate. If this is a NULL pointer, this function returns without doing anything.

Return Value

None.

Remarks

You can use this function to place virtual objects into the color image with depth-based occlusion. Call this function to determine which skeleton-space vertices of a virtual object are in front of, or behind, real objects visible in the depth map image. After occluded parts of the virtual objects have been culled, call [NuiImageGetColorPixelCoordinatesFromDepthPixel](#) to overlay the virtual object's vertices into color image pixel coordinates for rendering.

This function is defined as an inline function in NuiSkeleton.h.

The outputs are computed by applying a transform based on the optics of the depth camera. If the input point is not visible to the depth camera, the returned **pfDepthX* and **pfDepthY* coordinate values will be outside the bounds of the depth image.

Example

This function overload is used in the [Identity](#) and [Running](#) samples.

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTransformSmooth

[See Also](#) [Example](#)



Filters skeleton positions to reduce jitter between frames.

Syntax

C++

```
HRESULT NuiTransformSmooth(  
    NUI_SKELETON_FRAME *pSkeletonFrame,  
    CONST NUI_TRANSFORM_SMOOTH_PARAMETERS *pSmoothingParams  
)
```

Parameters

pSkeletonFrame
[in, out] On entry, points to a [NUI_SKELETON_FRAME](#) structure that contains the skeleton data to be smoothed. On exit, the skeleton data in the structure has been replaced by smoothed data. If this function does not return S_OK, this data is unchanged.

pSmoothingParams
[in, optional] The parameters for the smoothing function, as a [NUI_TRANSFORM_SMOOTH_PARAMETERS](#) structure. For descriptions of the parameters, see that structure.
If NULL, the following default values are used:

Name	Default Value
fSmoothing	0.5f

fCorrection	0.5f
fPrediction	0.5f
fJitterRadius	0.05f (5cm)
fMaxDeviationRadius	0.04f (4cm)

Return Value

S_OK if successful, or an error code otherwise. For example, E_UNEXPECTED or E_OUTOFMEMORY are error codes.

Remarks

The filter used is based on the Holt Double Exponential Smoothing method used for statistical analysis of economic data. This provides smoothing with less latency than other smoothing filter algorithms.

This function allocates 2 KB of memory the first time it is called. This memory is freed when the title terminates.

A single call of this function updates all currently-tracked skeletons, and takes 0.011 ms per skeleton to execute. For example, if two skeletons are being tracked, the total execution time is 0.022 ms.

Example

This function is used in several of the samples, including [SimpleSkeletonTracking](#).

This function is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
NUI_SKELETON_FRAME m_SkeletonFrame;

HRESULT Sample::Update()
{
    HRESULT hr;

    // Get next NUI skeleton frame
    bool bSkeletonFound = FALSE;

    hr = NuiSkeletonGetNextFrame( 10, &m_SkeletonFrame );
    if( SUCCEEDED( hr ) )
    {
        UINT i = 0;

        hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );

        // Process the skeleton data
        while ( ( i < NUI_SKELETON_COUNT ) && bSkeletonFound != TRUE )
        {
            if (m_SkeletonFrame.SkeletonData[i].eTrackingState !=
                NUI_SKELETON_NOT_TRACKED )
            {
                m_Visualizer.SetSkeletons( &m_SkeletonFrame );
                iCurrentSkeletonIndex=i;
                bSkeletonFound= TRUE;
            }
            i++;
        }
    }

    return S_OK;
}
```

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeleton Structures

[See Also](#)

In This Section

- [NUI_SKELETON_DATA](#)
Contains the data for one skeleton, including overall position, skeleton joint positions, and whether each skeleton joint is tracked successfully.
- [NUI_SKELETON_FRAME](#)
Stores a frame of data from the skeleton pipeline, including data for each skeleton.
- [NUI_TRANSFORM_SMOOTH_PARAMETERS](#)
Defines parameters for skeleton smoothing, including how much to smooth, how many frames to predict, and the amount of jitter reduction.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_DATA Structure

[See Also](#) [Example](#)

Contains the data for one skeleton, including overall position, skeleton joint positions, and whether each skeleton joint is tracked successfully.

Syntax

C++

```
typedef struct _NUI_SKELETON_DATA {
    NUI_SKELETON_TRACKING_STATE eTrackingState;
    DWORD dwTrackingID;
    DWORD dwEnrollmentIndex;
    DWORD dwUserIndex;
    XMVECTOR Position;
    XMVECTOR SkeletonPositions[NUI_SKELETON_POSITION_COUNT];
    NUI_SKELETON_POSITION_TRACKING_STATE eSkeletonPositionTrackingState[NUI_SKELETON_POSITION_COUNT];
    DWORD dwQualityFlags;
} NUI_SKELETON_DATA;
```

Members

eTrackingState

A [NUI_SKELETON_TRACKING_STATE](#) enumeration constant indicating which of the other members of this structure contains valid data. Specifies whether all skeleton joints are being successfully tracked, or only the player's overall position is being tracked.

dwTrackingID

The tracking ID currently assigned to this player. 0 indicates a non-tracked player. A tracking ID can be any DWORD value. A new player is assigned a greater ID than existing players, unless the value wraps, which is possible after a long time. A given player is not synonymous with a particular tracking ID. After running an identity operation, you might get the same **dwEnrollmentIndex** and **dwUserIndex** (that is, the same player) with a different tracking ID. Camera motion interrupts skeleton tracking, whether the motion is due to manually moving the sensor array or due to the motor. If the camera is moved, a new tracking ID is assigned to tracked skeletons, just as if the players left the field of view and returned. [XShowNui](#) functions have a *dwTrackingID* parameter related to this **dwTrackingID** member.

dwEnrollmentIndex

The enrollment index of the player, if this skeleton is associated with an enrolled player. If the player has not been identified (that is, the skeleton has not been associated with an enrollment slot), this value is a [NUI_IDENTITY_ENROLLMENT_INDEX](#) constant. The constant is either:

- NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY, indicating that [NuiIdentityIdentify](#) hasn't been called yet to identify the person, and needs to be called.
- An error flag.

dwUserIndex

The user index indicating which player's skeleton data is contained in this structure. If this skeleton is bound to a user index, this field will contain a value between 0 and 3. Otherwise, it will be [XUSER_INDEX_NONE](#).

Position

The position of the player, as an [XMVECTOR](#). When **eTrackingState** is NUI_SKELETON_POSITION_ONLY, this value is the center of mass of the tracked player. When **eTrackingState** is NUI_SKELETON_TRACKED, this value is the average of all valid skeleton positions.

SkeletonPositions

A fixed array of skeleton joint positions that represent the skeleton. Each skeleton joint position is stored in an [XMVECTOR](#). These skeleton joint positions are indexed with the values in the [NUI_SKELETON_POSITION_INDEX](#) enumeration. The x, y, and z components of each skeleton joint position in the array are the coordinates, in meters, from the camera. The w component is always unity.

eSkeletonPositionTrackingState

An array of indicators of whether each skeleton joint position is being successfully tracked, or is merely inferred. This is an array of [NUI_SKELETON_POSITION_TRACKING_STATE](#) enumeration constants corresponding to the array of skeleton joint positions. The first element in this array is the tracking state for the first skeleton joint in the *SkeletonPositions* array, the second element in this array is the tracking state for the second skeleton joint in the *SkeletonPositions* array, and so on.

dwQualityFlags

One or more [NUI_SKELETON_QUALITY](#) flags, combined with bitwise-OR, to indicate the quality of the skeleton data. You can use this information to prompt players to adjust conditions, to improve skeleton tracking.

Remarks

A [NUI_SKELETON_FRAME](#) structure contains an array of **NUI_SKELETON_DATA** structures. The set of skeleton joint positions may change between releases and is fixed at compile time. The provided static library will not link if the header version does not match. The static library will use a robust mapping scheme internally to maintain forward compatibility with future versions of the skeleton pipeline. The confidence values might not end on a 16-byte alignment value. Padding will be added either explicitly or by the compiler, to result in 16-byte alignment.

Example

See the example in [NUI_SKELETON_FRAME](#).

Requirements

Header: Declared in NuiSkeleton.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_FRAME Structure

[See Also](#) [Example](#)



Stores a frame of data from the skeleton pipeline, including data for each skeleton.

Syntax

C++

```
typedef struct _NUI_SKELETON_FRAME {
```

```

    LARGE_INTEGER liTimeStamp;
    DWORD dwFrameNumber;
    DWORD dwFlags;
    XMVECTOR vFloorClipPlane;
    XMVECTOR vNormalToGravity;
    NUI_SKELETON_DATA SkeletonData[NUI_SKELETON_COUNT];
} NUI_SKELETON_FRAME;

```

Members

liTimeStamp

The timestamp of the depth image that was processed to create this skeleton frame, as a [LARGE_INTEGER](#).

This value is in milliseconds since the Xbox 360 was last turned on. Skeleton frame data always has the same timestamp value as the most recent depth frame.

dwFrameNumber

The frame number of the depth image frame that was processed to create this skeleton frame.

This value is a counter that increments by one every time a new depth frame is generated by the Kinect sensor array. Color and skeleton frame data always has the same frame number as the most recent depth frame.

If you are retrieving color or skeleton frames at a slower rate than the sensor array is generating depth frames, you will see gaps in the frame number sequence. However, a later frame will always have a higher frame number than an earlier frame. You may also see gaps in the frame number sequence if a system call (such as [NuiIdentityEnroll](#) or [NuiIdentityIdentify](#)) is intercepting frames in order to perform system functions.

Frame numbers reset to zero every time that [NuiInitialize](#) is called or the Kinect sensor array is plugged in. This means that you can see the frame numbers reset during gameplay if the sensor array is unplugged and replugged, even if your code did not shut down and reinitialize the NUI API itself.

dwFlags

[NUI_SKELETON_FRAME](#) flags, such as [NUI_SKELETON_FRAME_FLAG_CAMERA_MOTION](#), bitwise-OR'd.

When the system passes this structure to your title, the system might set these flags.

When you pass this structure to a function, you can set the [NUI_SKELETON_FRAME_FLAG_EXTRAPOLATED_FLOOR](#) flag to opt-in to using an extrapolated floor as a fallback approach in case the floor is not detected. An extrapolated floor plane is not as accurate as actually seeing the floor. To opt-in to this feature, pass this flag. Otherwise, the floor plane will be populated with all zeros if the floor is not visible.

The system only sets [NUI_SKELETON_FRAME_FLAG_EXTRAPOLATED_FLOOR](#) flag if the floor is not currently visible or identified, and you opt-in to using an extrapolated floor.

A floor search is performed the first time Kinect is invoked. A floor search does not need to be performed again while the console is turned on unless the camera is disturbed physically. Kinect can extrapolate the floor plane in case the floor was only visible during the initial floor search.

If the floor has not been detected since the console was turned on or the camera was physically moved, you will not get an extrapolated floor. An extrapolated floor is seen across (and during) motor movements, although what is reported may lag slightly behind the actual camera angle (a little less than a second behind).

vFloorClipPlane

An estimate of the floor generated by the skeleton tracking system and used internally to clip below the floor. This is an [XMVECTOR](#).

If the floor is not visible to the Kinect sensor array, this vector is set to (0,0,0,0), unless [NUI_SKELETON_FRAME_FLAG_EXTRAPOLATED_FLOOR](#) is set in *dwFlags*. The units of measurement are meters.

vNormalToGravity

The measurement of gravity relative to the camera's body axes, as an [XMVECTOR](#). Gravity is measured at approximately the same time as the depth image for this frame.

This value is a unit normal vector, showing the direction of resultant forces only, not their magnitude. You do not need to normalize the **vNormalToGravity** vector when you obtain it.

SkeletonData

An array of [NUI_SKELETON_DATA](#) structures. Each [NUI_SKELETON_DATA](#) structure contains the data for one skeleton, including overall position, skeleton joint positions, and whether each skeleton joint is successfully tracked. The size of the array is given by the value of [NUI_SKELETON_COUNT](#).

A given tracking ID (**dwTrackingID** in [NUI_SKELETON_DATA](#)) will always appear at the same index in the **SkeletonData** array. A given player is not synonymous with a particular tracking ID. After running an identity operation, you might get the same **dwEnrollmentIndex** and **dwUserIndex** (that is, the same player) in a [NUI_SKELETON_DATA](#) structure, with a different tracking ID at a different skeleton index.

If the number of tracked skeletons is less than [NUI_SKELETON_COUNT](#), each array element that does not contain tracked skeleton data has its **eTrackingState** member set to [NUI_SKELETON_NOT_TRACKED](#).

The array is not guaranteed to have tracked skeleton starting from index 0. There can be gaps between tracked skeleton data.

To determine which player's skeleton data is contained in a given [NUI_SKELETON_DATA](#) structure, read the **dwUserIndex** member of that structure. The [NUI_SKELETON_DATA](#) structure corresponding to a given player will appear on same array index as long as the player remains tracked. The array index will stay the same when the **eTrackingState** changes between [NUI_SKELETON_TRACKED](#) and [NUI_SKELETON_POSITION_ONLY](#).

+ Remarks

This structure is returned by [NuiSkeletonGetNextFrame](#), and is passed to [NuiHandlesArmsUpdate](#) and [XedWriteNuiSkeletonFrame](#). This structure is modified by [NuiTransformSmooth](#), and is filled by [XedReadNuiSkeletonFrame](#).

The floor plane is an estimate based on image analysis. This estimate changes initially and then settles over time, unless a condition changes due to such factors as movement of the camera or furniture. The skeleton pipeline uses this plane as a lower clipping plane.

The general equation of a plane in homogenous coordinates is $Ax + By + Cz + D = 0$. The vector elements x , y , z , and w correspond to the coefficients A , B , C , and D , respectively. The camera is always at (0, 0, 0, 1), so the coefficient D is the negative of the height of the camera from the floor, in meters.

+ Example

[NUI_SKELETON_FRAME](#) and [NUI_SKELETON_DATA](#) are used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
NUI_SKELETON_FRAME m_SkeletonFrame;
```

```

HRESULT Sample::Update()
{
    // Get next NUI skeleton frame
    bool bSkeletonFound = FALSE;

    hr = NuiSkeletonGetNextFrame( 10, &m_SkeletonFrame );
    if( SUCCEEDED( hr ) )
    {
        UINT i = 0;

        hr = NuiTransformSmooth( &m_SkeletonFrame, NULL );

        // Process the skeleton data
        while ( ( i < NUI_SKELETON_COUNT ) && bSkeletonFound != TRUE )
        {
            if ( m_SkeletonFrame.SkeletonData[i].eTrackingState !=
                NUI_SKELETON_NOT_TRACKED )
            {
                m_Visualizer.SetSkeletons( &m_SkeletonFrame );
                iCurrentSkeletonIndex=i;
                bSkeletonFound= TRUE;
            }
            i++;
        }
    }
    ...

    reqSpeed = speed;
    static FLOAT backward = 0.0f;
    static FLOAT forward = 0.0f;

```

```
if( bSkeletonFound )
{
    // Find the right hand position

    // If the right hand is currently not tracked, pause the dolphin animation
    if( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
        eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_HAND_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED ||
        m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
        eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED )
    {
        reqSpeed = 0.0f;
    }
    else
    {
        halfSpeed =
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;

        FLOAT a = fabs(
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].x -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].x );

        FLOAT b = fabs(
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y );

        FLOAT radius = sqrt( a * a + b * b );
        backward = radius / 2.0f;
        forward = ( radius * -1.0f ) / 2.0f;
        reqSpeed =
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;
    }
}
```

▣ Requirements

Header: Declared in NuiSkeleton.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

▣ See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_TRANSFORM_SMOOTH_PARAMETERS Structure

[See Also](#)



Defines parameters for skeleton smoothing, including how much to smooth, how many frames to predict, and the amount of jitter reduction.

▣ Syntax

C++

```
typedef struct NUI_TRANSFORM_SMOOTH_PARAMETERS {
    FLOAT fSmoothing;
    FLOAT fCorrection;
    FLOAT fPrediction;
    FLOAT fJitterRadius;
    FLOAT fMaxDeviationRadius;
} NUI_TRANSFORM_SMOOTH_PARAMETERS;
```

Members

fSmoothing

Smoothing parameter. Increasing the smoothing parameter value leads to more highly-smoothed skeleton position values being returned.
It is the nature of smoothing that, as the smoothing value is increased, responsiveness to the raw data decreases. Thus, increased smoothing leads to increased latency in the returned skeleton values.
Values must be in the range 0 through 1.0. Passing 0 causes the raw data to be returned.

fCorrection

Correction parameter. Lower values are slower to correct towards the raw data, and appear smoother, while higher values will correct toward the raw data more quickly.
Values must be in the range 0 through 1.0.

fPrediction

The number of frames to predict into the future.
Values must be greater than or equal to zero. Values greater than 0.5 will likely lead to overshooting when moving quickly. This effect can be damped by using small values of **fMaxDeviationRadius**.

fJitterRadius

The radius in meters for jitter reduction.
Any jitter beyond this radius is clamped to the radius.

fMaxDeviationRadius

The maximum radius in meters that filtered positions are allowed to deviate from raw data.
Filtered values that would be more than this radius from the raw data are clamped at this distance, in the direction of the filtered value.

▣ Remarks

This structure contains input parameters for the [NuiTransformSmooth](#) function.

To see the effect of using different filter parameters, see [Filtering Sample](#).

The following examples show the use of various parameter values for different filter behaviors:

```
// Some smoothing with little latency (defaults).
// Only filters out small jitters.
// Good for gesture recognition in games.
const NUI_TRANSFORM_SMOOTH_PARAMETERS DefaultParams =
    {0.5f, 0.5f, 0.5f, 0.05f, 0.04f};

// Smoothed with some latency.
// Filters out medium jitters.
// Good for a menu system that needs to be smooth but
// doesn't need the reduced latency as much as gesture recognition does.
const NUI_TRANSFORM_SMOOTH_PARAMETERS SomewhatLatentParams =
    {0.5f, 0.1f, 0.5f, 0.1f, 0.1f};

// Very smooth, but with a lot of latency.
// Filters out large jitters.
// Good for situations where smooth data is absolutely required
// and latency is not an issue.
const NUI_TRANSFORM_SMOOTH_PARAMETERS VerySmoothParams =
    {0.7f, 0.3f, 1.0f, 1.0f, 1.0f};
```

Requirements

Header: Declared in NuiSkeleton.h. For include, use NuiApi.h.

Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSkeleton Enumerations

[See Also](#)

In This Section

[NUI_SKELETON_POSITION_INDEX](#)

Specifies skeleton joints such as spine, head, hands, and feet.

[NUI_SKELETON_POSITION_TRACKING_STATE](#)

Specifies whether a skeleton joint position is being successfully tracked or is merely inferred.

[NUI_SKELETON_TRACKING_STATE](#)

Specifies whether all skeleton joints are being successfully tracked, or only the player's overall position is being tracked.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_POSITION_INDEX Enumeration

[See Also Example](#)

Specifies skeleton joints such as spine, head, hands, and feet.

Syntax

C++

```
typedef enum _NUI_SKELETON_POSITION_INDEX
{
    NUI_SKELETON_POSITION_HIP_CENTER = 0,
    NUI_SKELETON_POSITION_SPINE,
    NUI_SKELETON_POSITION_SHOULDER_CENTER,
    NUI_SKELETON_POSITION_HEAD,
    NUI_SKELETON_POSITION_SHOULDER_LEFT,
    NUI_SKELETON_POSITION_ELBOU_LEFT,
    NUI_SKELETON_POSITION_WRIST_LEFT,
    NUI_SKELETON_POSITION_HAND_LEFT,
    NUI_SKELETON_POSITION_SHOULDER_RIGHT,
    NUI_SKELETON_POSITION_ELBOU_RIGHT,
    NUI_SKELETON_POSITION_WRIST_RIGHT,
    NUI_SKELETON_POSITION_HAND_RIGHT,
    NUI_SKELETON_POSITION_HIP_LEFT,
    NUI_SKELETON_POSITION_KNEE_LEFT,
    NUI_SKELETON_POSITION_ANKLE_LEFT,
    NUI_SKELETON_POSITION_FOOT_LEFT,
    NUI_SKELETON_POSITION_HIP_RIGHT,
    NUI_SKELETON_POSITION_KNEE_RIGHT,
    NUI_SKELETON_POSITION_ANKLE_RIGHT,
    NUI_SKELETON_POSITION_FOOT_RIGHT,
    NUI_SKELETON_POSITION_COUNT
} NUI_SKELETON_POSITION_INDEX;
```

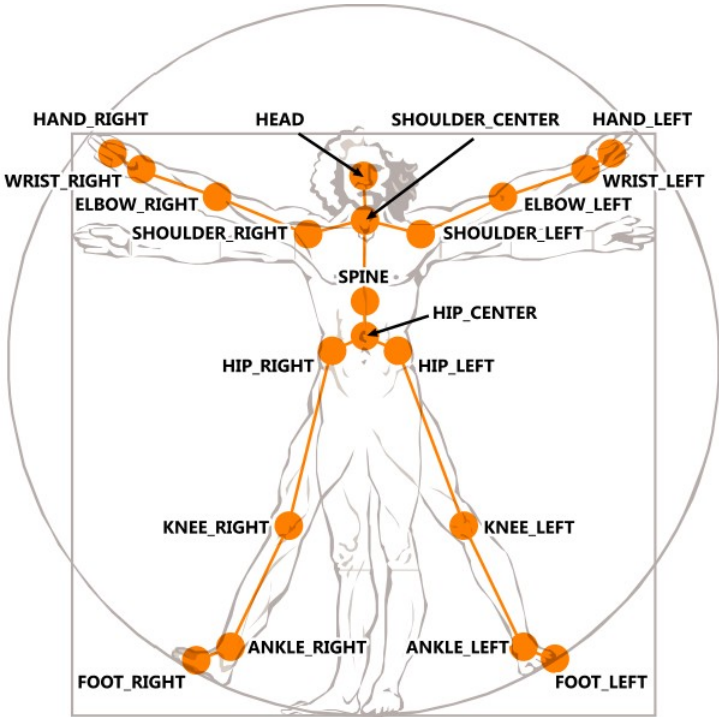
Constants

Constant	Description
NUI_SKELETON_POSITION_HIP_CENTER	Center position between hips.

NUI_SKELETON_POSITION_SPINE	Spine position.
NUI_SKELETON_POSITION_SHOULDER_CENTER	Center position between shoulders.
NUI_SKELETON_POSITION_HEAD	Head.
NUI_SKELETON_POSITION_SHOULDER_LEFT	Left shoulder.
NUI_SKELETON_POSITION_ELBOU_LEFT	Left elbow.
NUI_SKELETON_POSITION_WRIST_LEFT	Left wrist.
NUI_SKELETON_POSITION_HAND_LEFT	Left hand.
NUI_SKELETON_POSITION_SHOULDER_RIGHT	Right shoulder.
NUI_SKELETON_POSITION_ELBOU_RIGHT	Right elbow.
NUI_SKELETON_POSITION_WRIST_RIGHT	Right wrist.
NUI_SKELETON_POSITION_HAND_RIGHT	Right hand.
NUI_SKELETON_POSITION_HIP_LEFT	Left hip.
NUI_SKELETON_POSITION_KNEE_LEFT	Left knee.
NUI_SKELETON_POSITION_ANKLE_LEFT	Left ankle.
NUI_SKELETON_POSITION_FOOT_LEFT	Toe end of left foot
NUI_SKELETON_POSITION_HIP_RIGHT	Right hip.
NUI_SKELETON_POSITION_KNEE_RIGHT	Right knee.
NUI_SKELETON_POSITION_ANKLE_RIGHT	Right ankle.
NUI_SKELETON_POSITION_FOOT_RIGHT	Toe end of right foot.
NUI_SKELETON_POSITION_COUNT	Use as an index to terminate a loop. Not used as a position index.

Remarks

These values are used in the [NUI_SKELETON_DATA](#) structure.
Figure 1. Skeleton joint positions relative to the human body



You must always use these named enumeration constants, such as `NUI_SKELETON_POSITION_HAND_RIGHT`, rather than the corresponding enumeration index values, such as 0, 1, 2.... Otherwise, source compatibility with future releases may break silently.

[Seated skeleton tracking](#) does not track any joints for the lower body, including spine, hips, knees, ankles, and feet. Seated skeleton tracking only returns joint positions for head, neck, shoulders, elbows, wrists, and hands.

Example

The enumeration constants `NUI_SKELETON_POSITION_HAND_RIGHT` and `NUI_SKELETON_POSITION_SHOULDER_RIGHT` are used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
NUI_SKELETON_FRAME m_SkeletonFrame;
int iCurrentSkeletonIndex;
FLOAT halfSpeed;
FLOAT reqSpeed;
FLOAT speed;

HRESULT Sample::Update()
{
    ...

    bool bSkeletonFound = FALSE;
```



```
reqSpeed = speed;
static FLOAT backward = 0.0f;
static FLOAT forward = 0.0f;

if( bSkeletonFound )
{
    // Find the right hand position

    // If the right hand is currently not tracked, pause the dolphin animation
    if( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
        eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_HAND_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED ||
        m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
        eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ]
        != NUI_SKELETON_POSITION_TRACKED )
    {
        reqSpeed = 0.0f;
    }
    else
    {
        halfSpeed =
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;

        FLOAT a = fabs(
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].x -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].x );

        FLOAT b = fabs(
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y );

        FLOAT radius = sqrt( a * a + b * b );
        backward = radius / 2.0f;
        forward = ( radius * -1.0f ) / 2.0f;
        reqSpeed =
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_HAND_RIGHT ].y -
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            SkeletonPositions[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ].y;
    }
}
```

Requirements

Header: Declared in NuiSkeleton.h (for include, use NuiApi.h), and NuiTools.h.
Library: Use both nuiapi.lib and st.lib.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_POSITION_TRACKING_STATE Enumeration

[See Also Example](#)

Specifies whether a skeleton joint position is being successfully tracked or is merely inferred.

Syntax

```
C++

typedef enum _NUI_SKELETON_POSITION_TRACKING_STATE
{
    NUI_SKELETON_POSITION_NOT_TRACKED,
    NUI_SKELETON_POSITION_INFERRED,
    NUI_SKELETON_POSITION_TRACKED
} NUI_SKELETON_POSITION_TRACKING_STATE;
```

Constants

Constant	Description
NUI_SKELETON_POSITION_NOT_TRACKED	Indicates that the skeleton joint position is not being tracked, and its position cannot be inferred to any useful extent. You should not use such joints' positions as game input. If you are attempting to use a joint's position as a control input and the joint's position confidence is reported as NUI_SKELETON_POSITION_NOT_TRACKED, you should use a default value for that input. If you are attempting to use a joint's position to position a player avatar and the joint's position confidence is reported as NUI_SKELETON_POSITION_NOT_TRACKED, you should blend the avatar position with a default pose for the missing data.
NUI_SKELETON_POSITION_INFERRED	Indicates that the skeleton joint position is being inferred from previous frames' data, the positions of tracked joints, and assumptions about skeleton geometry. For example, if the elbow is not visible but the position of the shoulder joint is known, the elbow's position can be inferred by knowing that the elbow is attached to the shoulder. This is done by estimating the distance from the elbow to the shoulder, and measuring or assuming the angle of the upper arm. Wherever possible, the system attempts to infer a joint position rather than report NUI_SKELETON_POSITION_NOT_TRACKED. If you use inferred joint data as a control input, you should ensure that data values are reasonable, by using smoothing and boundary checking on the data. If you use inferred joint data to position a player avatar, you should ensure that the resulting avatar position is reasonable, and blend to a default pose when the position would not be reasonable.
NUI_SKELETON_POSITION_TRACKED	Indicates that the skeleton joint position is being successfully tracked. The reported joint position is based the current frame's data.

If you use such joints for input control or to position a player avatar, you should still consider using smoothing, to minimize the effect of noise and jitter in skeleton positions.

Remarks

A [NUI_SKELETON_DATA](#) structure contains an array of **NUI_SKELETON_POSITION_TRACKING_STATE** enumeration constants, one for each skeleton joint position.

Example

The NUI_SKELETON_POSITION_TRACKED enumeration constant is used in [Quick-Start Tutorial for Kinect Programming](#), as follows:

```
NUI_SKELETON_FRAME m_SkeletonFrame;
int iCurrentSkeletonIndex;
FLOAT reqSpeed;
FLOAT speed;

HRESULT Sample::Update()
{
    bool bSkeletonFound = FALSE;

    reqSpeed = speed;

    if( bSkeletonFound )
    {
        // Find the right hand position

        // If the right hand is currently not tracked, pause the dolphin animation
        if( m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_HAND_RIGHT ]
            != NUI_SKELETON_POSITION_TRACKED
            ||
            m_SkeletonFrame.SkeletonData[ iCurrentSkeletonIndex ].
            eSkeletonPositionTrackingState[ NUI_SKELETON_POSITION_SHOULDER_RIGHT ]
            != NUI_SKELETON_POSITION_TRACKED )
        {
            reqSpeed = 0.0f;
        }
        else
        {
            // Hand is tracked - use its position
        }
    }
}
```

Requirements

Header: Declared in NuiSkeleton.h (for include, use NuiApi.h), and NuiTools.h.

Library: Use both nuiapi.lib and st.lib.

See Also

- [NUI Skeleton Tracking Overview](#)
- [NuiTransformSmooth](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_TRACKING_STATE Enumeration

[See Also](#)



Specifies whether all skeleton joints are being successfully tracked, or only the player's overall position is being tracked.

Syntax

C++

```
typedef enum _NUI_SKELETON_TRACKING_STATE
{
    NUI_SKELETON_NOT_TRACKED,
    NUI_SKELETON_POSITION_ONLY,
    NUI_SKELETON_TRACKED
} NUI_SKELETON_TRACKING_STATE;
```

Constants

Constant	Description
NUI_SKELETON_NOT_TRACKED	Indicates that this skeleton is not being tracked. When the eTrackingState member of a NUI_SKELETON_DATA structure has this value, other members of that structure are not valid and should not be used.
NUI_SKELETON_POSITION_ONLY	Indicates that a player's overall position is being tracked, but their individual skeleton joints are not being tracked. The Position member of a NUI_SKELETON_DATA with this eTrackingState is valid, but SkeletonPositions and allied data are not valid and should not be used.
NUI_SKELETON_TRACKED	Indicates that all joints of a skeleton are being tracked, such as head, spine, hands, and feet. All members of the associated NUI_SKELETON_DATA contain usable data.

Remarks

These values are used in the **eTrackingState** member of a [NUI_SKELETON_DATA](#) structure to indicate which of the other members of that structure are valid.

Requirements

Header: Declared in NuiSkeleton.h (for include, use NuiApi.h), and NuiTools.h.

Library: Use both nuiapi.lib and st.lib.

See Also

- [NUI Skeleton Tracking Overview](#)
- [NuiTransformSmooth](#)

NuiSkeleton Constants

[See Also](#)



In This Section

- [NUI_CAMERA_DEPTH_IMAGE_TO_SKELETON_MULTIPLIER_320x240](#)
The scaling factor for converting depth image coordinates to skeleton coordinate space.
- [NUI_CAMERA_SKELETON_TO_DEPTH_IMAGE_MULTIPLIER_320x240](#)
The scaling factor for converting skeleton space coordinates to depth image space.
- [NUI_SKELETON_COUNT](#)
The number of skeletons in a frame. This is the number of skeletons that can be simultaneously tracked in any way, whether passively tracked or actively (fully) tracked.
- [NUI_SKELETON_FRAME_Flags](#)
Flags for the NUI_SKELETON_FRAME structure.
- [NUI_SKELETON_INVALID_TRACKING_ID](#)
The tracking ID to use as an invalid tracking ID for skeletons.
- [NUI_SKELETON_MAX_TRACKED_COUNT](#)
The maximum number of skeletons that can be actively (fully) tracked at one time.
- [NUI_SKELETON_QUALITY_Flags](#)
Flags that indicate which side of a skeleton is cut off in the camera's field of view.
- [NUI_SKELETON_TRACKING_Flags](#)
Flags used for skeleton tracking.

See Also
[NUI Skeleton Tracking Overview](#)

NUI_CAMERA_DEPTH_IMAGE_TO_SKELETON_MULTIPLIER_320x240

[See Also](#)



The scaling factor for converting depth image coordinates to skeleton coordinate space.

Name	Description
NUI_CAMERA_DEPTH_IMAGE_TO_SKELETON_MULTIPLIER_320x240	This value is based on the field of view of the Kinect sensor array. This value is used by the Skeleton Tracking implementation and NuiTransformDepthImageToSkeleton .

Remarks
The nominal diagonal field of view of the depth sensor is 70 degrees, which corresponds to a horizontal field of view of approximately 58 degrees, and a vertical field of view of approximately 45 degrees.
For an example using this value, see the inline definition of [NuiTransformDepthImageToSkeleton](#) in NuiSkeleton.h.

See Also
[NUI Skeleton Tracking Overview](#)

NUI_CAMERA_SKELETON_TO_DEPTH_IMAGE_MULTIPLIER_320x240

[See Also](#)



The scaling factor for converting skeleton space coordinates to depth image space.

Name	Description
NUI_CAMERA_SKELETON_TO_DEPTH_IMAGE_MULTIPLIER_320x240	This value is based on the field of view of the Kinect sensor array. This value is used by the Skeleton Tracking implementation and NuiTransformSkeletonToDepthImage .

Remarks
The nominal diagonal field of view of the depth sensor is 70 degrees, which corresponds to a horizontal field of view of approximately 58 degrees, and a vertical field of view of approximately 45 degrees.
For an example using this value, see the inline definition of [NuiTransformSkeletonToDepthImage](#) in NuiSkeleton.h.

See Also
[NUI Skeleton Tracking Overview](#)

NUI_SKELETON_COUNT

[See Also](#)



The number of skeletons in a frame. This is the number of skeletons that can be simultaneously tracked in any way, whether passively tracked or actively (fully) tracked.

Name	Description
NUI_SKELETON_COUNT	The number of NUI_SKELETON_DATA elements in a NUI_SKELETON_FRAME .

+

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_FRAME Flags

[See Also](#)



Flags for the NUI_SKELETON_FRAME structure.

Name	Description
NUI_SKELETON_FRAME_FLAG_CAMERA_MOTION	When this flag is set in the <i>dwFlags</i> member of a NUI_SKELETON_FRAME , the camera has been moved since the last reported frame. See also NUI_CAMERA_ELEVATION Flags, which indicates whether the sensor array was moved by the tilt motor or by a person.
NUI_SKELETON_FRAME_FLAG_EXTRAPOLATED_FLOOR	Indicates that the <i>vFloorClipPlane</i> that is returned with the NUI_SKELETON_FRAME structure is extrapolated rather than using the depth map to detect it directly. For more information, see <i>dwFlags</i> in NUI_SKELETON_FRAME .
NUI_SKELETON_FRAME_FLAG_UPPER_BODY_SKELETON	Indicates that this frame came from the upper body pipeline. This allows the XAM and the title components to treat this skeleton frame differently.
NUI_SKELETON_FRAME_FLAG_SEATED_SUPPORT_ENABLED	Indicates that support for seated skeleton tracking is enabled. When this flag is set, the skeleton returned will not have the lower body joints, and player segmentation mask data is not available.

+

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_INVALID_TRACKING_ID

[See Also](#)



The tracking ID to use as an invalid tracking ID for skeletons.

Name	Description
NUI_SKELETON_INVALID_TRACKING_ID	The tracking ID to use as an invalid tracking ID for skeletons.

+

Remarks

This constant is used internally by [NuiSkeletonSetTrackedSkeletons](#).

+

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_MAX_TRACKED_COUNT

[See Also](#)



The maximum number of skeletons that can be actively (fully) tracked at one time.

Name	Description
NUI_SKELETON_MAX_TRACKED_COUNT	The maximum number of skeletons that can be actively (fully) tracked at one time.

+

Remarks

This constant is used by [NuiSkeletonSetTrackedSkeletons](#).

+

See Also

[NUI Skeleton Tracking Overview](#)

file:///C:/Users/simon/AppData/Local/Temp/~hh15D.htm

2026-04-19

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_QUALITY Flags

[See Also](#)



Flags that indicate which side of a skeleton is cut off in the camera's field of view.

Name	Description
NUI_SKELETON_QUALITY_CLIPPED_RIGHT	Part of the player's body is out of frame to the camera's right.
NUI_SKELETON_QUALITY_CLIPPED_LEFT	Part of the player's body is out of frame to the camera's left.
NUI_SKELETON_QUALITY_CLIPPED_TOP	Part of the player's body is out of frame above the camera's field of view.
NUI_SKELETON_QUALITY_CLIPPED_BOTTOM	Part of the player's body is out of frame below the camera's field of view.

Remarks

One or more of these values may be reported in the **dwQualityFlags** member of the [NUI_SKELETON_DATA](#) structure. The values are combined using bitwise-OR. Each value indicates a reason for loss of quality in the reported skeleton data.

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SKELETON_TRACKING Flags

[See Also](#)



Flags used for skeleton tracking.

These flags are used for skeleton tracking.

Name	Description
NUI_SKELETON_TRACKING_FLAG_SUPPRESS_NO_FRAME_DATA	Prevents NuiSkeletonGetNextFrame from returning E_NUI_FRAME_NO_DATA errors. Rather than returning the error E_NUI_FRAME_NO_DATA, calls to NuiSkeletonGetNextFrame block until data is available or the timeout period passes. If you pass this flag in the <i>dwFlags</i> argument to NuiSkeletonTrackingEnable, NuiSkeletonGetNextFrame will not return E_NUI_FRAME_NO_DATA errors. Instead, calls to NuiSkeletonGetNextFrame block until data is available or the timeout period passes. If you use this flag, you must never call NuiSkeletonGetNextFrame from the same thread that calls IDirect3DDevice9::Present or IDirect3DDevice9::Swap.
NUI_SKELETON_TRACKING_FLAG_TITLE_SETS_TRACKED_SKELETONS	Disables the default player selection mode and enables the title to manage which players have tracked skeletons. This flag is used by NuiSkeletonTrackingEnable to disable the default player selection mode. After calling NuiSkeletonTrackingEnable with this flag, no skeletons will be tracked, and the title must call NuiSkeletonSetTrackedSkeletons to manage which players will have tracked skeletons. NuiSkeletonTrackingDisable reverts to the default tracked-skeleton selection policy. If this flag is not passed to NuiSkeletonTrackingEnable, the title cannot call NuiSkeletonSetTrackedSkeletons, and the default policy will remain in effect. If NuiSkeletonSetTrackedSkeletons is called without setting this flag, the call will fail with E_NUI_FEATURE_NOT_INITIALIZED and output a descriptive message in debug builds.
NUI_SKELETON_TRACKING_FLAG_ENABLE_SEATED_SUPPORT	Enables support for seated skeleton tracking. When this flag is set: • In the returned NUI_SKELETON_FRAME structure, in the dwFlags member, the system will set the flag NUI_SKELETON_FRAME_FLAG_SEATED_SUPPORT_ENABLED. • The skeleton returned will not have the lower body joints. • Player segmentation mask data is not available. To switch between full skeleton tracking mode and seated skeleton tracking mode, first call NuiSkeletonTrackingDisable, and then call NuiSkeletonTrackingEnable with the required flags. All active players are dropped, and then the system looks for players again. New players are assigned new tracking IDs.

Remarks

These flags are passed to [NuiSkeletonTrackingEnable](#).

See Also

[NUI Skeleton Tracking Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeech Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiSpeech.h, which enables speech recognition with the Kinect microphone array.

In This Section[NuiSpeech Functions](#)[NuiSpeech Structures](#)[NuiSpeech Enumerations](#)[NuiSpeech Constants](#)**See Also**[NUI Speech](#)[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

NuiSpeech Functions[See Also](#)**In This Section**[NuiSpeechAddRuleTransition](#)

Adds a rule reference transition from one grammar rule to another.

[NuiSpeechAddWordTransition](#)

Adds a word or phrase to a grammar.

[NuiSpeechClearRule](#)

Removes all grammar rule information except for the rule's initial state.

[NuiSpeechCommitGrammar](#)

Recompiles a grammar to incorporate rule modifications or additions, enabling dynamic grammar changes.

[NuiSpeechCreateGrammar](#)

Creates an empty grammar which can then be used for adding rules.

[NuiSpeechCreateRule](#)

Retrieves an existing top-level rule or creates a new top-level rule, which enables you to make changes at run time.

[NuiSpeechCreateState](#)

Creates a new state in the same grammar rule as an existing state.

[NuiSpeechDestroyEvent](#)

Destroys a speech engine event, and returns the memory to the speech engine.

[NuiSpeechDestroyPronunciationList](#)

Frees resources associated with word pronunciations.

[NuiSpeechDestroySerializedState](#)

Destroys the serialized state data from the speech engine—data that can be used to get stored speech data from the same source later.

[NuiSpeechDisable](#)

Uninitializes and unloads the speech engine.

[NuiSpeechEmulateRecognition](#)

Performs speech recognition on a text string as though the text were spoken.

[NuiSpeechEnable](#)

Initializes the speech engine with the specified language, sound source, and hardware thread.

[NuiSpeechGetDefaultPronunciations](#)

Gets the default pronunciations of a specified text string, enabling you to compare the player's pronunciation to the expected pronunciations.

[NuiSpeechGetEvents](#)

Polls for available events from the speech recognition engine, such as phrase started, phrase recognized, or bookmark reached.

[NuiSpeechGetGrammarState](#)

Gets whether the specified grammar is enabled and is the only grammar used.

[NuiSpeechGetProperty](#)

Retrieves the current value of a speech engine property.

[NuiSpeechGetSerializedState](#)

Retrieves the serialized state data from the speech engine, to obtain stored speech data from the same source later.

[NuiSpeechLoadGrammar](#)

Loads a compiled grammar from disc.

[NuiSpeechLoadGrammarFromMemory](#)

Loads a compiled grammar from a memory buffer supplied by the title.

[NuiSpeechProcessWaveInput](#)

Performs speech recognition processing on audio data from a .wav file, rather than using the Kinect microphone array.

[NuiSpeechProcessWaveInputFromMemory](#)

Performs speech recognition processing on audio data from memory, rather than using the Kinect microphone array.

[NuiSpeechSetEventInterest](#)

Tells the speech engine which events to send to the title, such as phrase started, phrase recognized, or bookmark reached.

[NuiSpeechSetGrammarState](#)

Sets whether the specified grammar is enabled and is the only grammar used.

[NuiSpeechSetProperty](#)

Sets a speech engine property, to customize the speech engine behavior.

[NuiSpeechSetRecognitionBookmark](#)

Sets a bookmark in the current speech recognition stream, so that when the recognizer reaches the bookmark, a bookmark event is raised.

[NuiSpeechSetRulePriority](#)

Sets the priority of the specified rule.

[NuiSpeechSetRuleState](#)

Activates or deactivates the passed-in top-level rule object.

[NuiSpeechSetRuleWeight](#)

Sets the weight on the specified rule.

[NuiSpeechSetSerializedState](#)

Stores serialized state data from the speech engine, to get this stored speech data from the same source later.

[NuiSpeechStartRecognition](#)

Begins the speech recognition process, using the enabled grammars.

[NuiSpeechStopRecognition](#)

Suspends speech recognition by disabling all grammars.

[NuiSpeechUnloadGrammar](#)

Unloads resources associated with the specified grammar.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechAddRuleTransition

[See Also](#)



Adds a rule reference transition from one grammar rule to another.

Syntax

C++

```
HRESULT NuiSpeechAddRuleTransition(  
    NUI_SPEECH_GRAMMAR *pGrammar,  
    NUI_SPEECH_STATEHANDLE hFromState,  
    NUI_SPEECH_STATEHANDLE hToState,  
    NUI_SPEECH_STATEHANDLE hRuleState,  
    FLOAT fWeight,  
    NUI_SPEECH_SEMANTIC *pSemantic  
)
```

Parameters

- pGrammar*
[in] The grammar whose rules are being modified, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.
- hFromState*
[in] A handle to the originating rule state.
The NUI_SPEECH_STATEHANDLE type is equivalent to LPVOID.
- hToState*
[in, optional] A handle to the destination state.
A NULL value specifies the terminating state for this rule.
- hRuleState*
[in] A handle to the initial rule state for the rule being added to the transition arc.
To match any spoken words, pass [NUI_SPEECH_RULE_TRANSITION_WILDCARD](#).
- fWeight*
[in] The relative weight of the transition, in case there are multiple transition arcs originating from *hFromState*.
Valid values are 0 through 1.0.
- pSemantic*
[in, optional] The semantic property associated with this transition, as a pointer to a [NUI_SPEECH_SEMANTIC](#) structure. This can be NULL.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
E_OUTOFMEMORY	Not enough memory to complete the request.
FAILED(hr)	If the engine cannot process the request.

Remarks

Attaching rule transitions to the rule states enables component rules to be built and shared with other dynamic grammars.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechAddWordTransition

[See Also](#)



Adds a word or phrase to a grammar.

Syntax

C++

```
HRESULT NuiSpeechAddWordTransition(  
    NUI_SPEECH_GRAMMAR *pGrammar,
```

```
    NUI_SPEECH_STATEHANDLE hFromState,  
    NUI_SPEECH_STATEHANDLE hToState,  
    LPCWSTR pcwszWords,  
    LPCWSTR pcwszSeparators,  
    NUI_SPEECH_WORDTYPE eWordType,  
    FLOAT fWeight,  
    NUI_SPEECH_SEMANTIC *pSemantic  
}
```

Parameters

pGrammar
[in] The grammar to which the word or words are to be added, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

hFromState
[in] The origin state, as a NUI_SPEECH_STATEHANDLE.
The type NUI_SPEECH_STATEHANDLE is equivalent to LPVOID.

hToState
[in, optional] The destination state, as a NUI_SPEECH_STATEHANDLE.
A NULL value specifies the terminating state for this rule. The type NUI_SPEECH_STATEHANDLE is equivalent to LPVOID.

pcwszWords
[in, optional] The word or words to add to this transition, as a null-terminated, delimited word list.
The word list is delimited by characters in *pcwszSeparators*.

pcwszSeparators
[in, optional] The delimiter characters for *pcwszWords*, as a null-terminated list.
The forward slash character ("/") is reserved and cannot be used as a separator. If this parameter is NULL, **pcwszWords* is treated as a single word.

eWordType
[in] The word type to be applied to the new word or words, as a [NUI_SPEECH_WORDTYPE](#) enumeration constant.

fWeight
[in] The relative weight of the transition, in case there are multiple transition arcs originating from *hFromState*.
Valid values are 0 through 1.0.

pSemantic
[in, optional] The property value associated with this transition, as a pointer to a [NUI_SPEECH_SEMANTIC](#) structure. This can be NULL.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
E_OUTOFMEMORY	There is not enough memory to complete the request.
SPERR_WORDFORMAT_ERROR	The word format is not recognized.
FAILED(hr)	The engine cannot process the request.

Remarks

Using this function is the most intuitive way to add new words or a series of words to a dynamic grammar. No efficiency is lost using this method, because the grammar builder may merge sub-phrases where they are the same.
You can further control the shape of the directed graph by building intermediate states. You can also export dynamic rules so that other grammars can take advantage of those dynamic rules.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechClearRule

[See Also](#)

Removes all grammar rule information except for the rule's initial state.

Syntax

```
C++  
  
HRESULT NuiSpeechClearRule(  
    NUI_SPEECH_GRAMMAR *pGrammar,  
    NUI_SPEECH_STATEHANDLE hRuleState  
)
```

Parameters

pGrammar
[in] The grammar containing the rule to be cleared, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

hRuleState
[in] A handle to a rule state, as returned by [NuiSpeechCreateRule](#) or [NuiSpeechCreateState](#). The NUI_SPEECH_STATEHANDLE type is equivalent to LPVOID.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

⊕ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechCommitGrammar

[See Also](#)



Recompiles a grammar to incorporate rule modifications or additions, enabling dynamic grammar changes.

⊕ Syntax

C++

```
HRESULT NuiSpeechCommitGrammar(  
    NUI_SPEECH_GRAMMAR *pGrammar  
)
```

Parameters

pGrammar
[in] The grammar to be recompiled, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
SPERR_NO_RULES	A grammar must have at least one rule with one word.
SPERR_NO_TERMINATING_RULE_PATH	At least one rule is not empty, but it has no terminating path from an initial state to a NULL state.
SPERR_CIRCULAR_RULE_REF	At least one rule has a direct or indirect reference to itself.
SPERR_STATE_WITH_NO_ARCS	At least one rule has a node without any outgoing transitions.
SPERR_EXPORT_DYNAMIC_RULE	Dynamic rules or rules referencing dynamic rules cannot be exported.

⊕ Remarks

For the speech recognizer to pick up dynamic grammar changes, you must call this method.

⊕ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechCreateGrammar

[See Also](#)



Creates an empty grammar which can then be used for adding rules.

⊕ Syntax

C++

```
HRESULT NuiSpeechCreateGrammar(  
    ULONG ulGrammarId,  
    NUI_SPEECH_GRAMMAR *pGrammar  
)
```

Parameters

ulGrammarId
[in] The unique identifier for this grammar. The identifier is set by title code.

pGrammar
[out] The new, empty grammar, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
E_INVALIDARG	<i>pGrammar</i> is not a valid pointer.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .
FAILED(hr)	Error message that can be found on MSDN if the engine couldn't create the grammar.

Remarks

This function enables you to build dynamic grammars without requiring that a "template" compiled grammar file exists first. Before shutdown, you must call [NuiSpeechUnloadGrammar](#) on a grammar.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechCreateRule

[See Also](#)



Retrieves an existing top-level rule or creates a new top-level rule, which enables you to make changes at run time.

Syntax

C++

```
HRESULT NuiSpeechCreateRule(  
    NUI_SPEECH_GRAMMAR *pGrammar,  
    LPCWSTR pwszRuleName,  
    DWORD dwOptions,  
    BOOL fCreateIfNotExist,  
    NUI_SPEECH_STATEHANDLE *phInitialState  
)
```

Parameters

pGrammar
[in] The grammar containing the rule, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

pwszRuleName
[in] The name of the rule to be retrieved or created.

dwOptions
[in] The rule options for a newly created rule, as a bitwise-OR combination of [NUI_SPEECH_RULEOPTIONS](#) enumeration constants. If the rule already exists, this parameter is ignored.

fCreateIfNotExist
[in] Indicates whether to create the rule if the rule does not already exist in the passed-in grammar.

- If TRUE and the rule does not exist in the passed-in grammar, this function creates the rule.
- If FALSE and the rule does not exist in the passed-in grammar, this function returns an error and does not create the rule.

phInitialState
[out] The initial state of the requested rule, as a pointer to a NUI_SPEECH_STATEHANDLE.

This pointer must be non-NULL when you call the function. After the function call returns, the NUI_SPEECH_STATEHANDLE contains the state information. If this function does not return NOERROR, this data is invalid.

The type NUI_SPEECH_STATEHANDLE is equivalent to LPVOID.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not started successfully with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
E_OUTOFMEMORY	Not enough memory is available to complete the request.
SPERR_RULE_NOT_FOUND	No rule matching the specified criteria could be found.
FAILED(hr)	The engine cannot process the request.

Remarks

To modify rules in a grammar or add rules to a grammar, the grammar must be created with the [NUI_SPEECH_LOADOPTIONS_DYNAMIC](#) enumeration constant when the grammar is loaded.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

NUI Speech

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechCreateState

See Also

Creates a new state in the same grammar rule as an existing state.

Syntax

C++

```
HRESULT NuiSpeechCreateState(  
    NUI_SPEECH_GRAMMAR *pGrammar,  
    NUI_SPEECH_STATEHANDLE hRule,  
    NUI_SPEECH_STATEHANDLE *phNewState  
)
```

Parameters

pGrammar
[in] The grammar containing the rule, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

hRule
[in] The handle of an existing rule state. The NUI_SPEECH_STATEHANDLE type is equivalent to LPVOID.

phNewState
[out] The new rule state, as a pointer to a NUI_SPEECH_STATEHANDLE.
This pointer must be non-NULL when you call the function. After the function call returns, the NUI_SPEECH_STATEHANDLE contains the new state. If this function does not return NOERROR, this data is invalid.
The type NUI_SPEECH_STATEHANDLE is equivalent to LPVOID.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not started successfully with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Remarks

Rule states can be constructed to allow fine-grained control over the rules when you add words dynamically to a grammar. It is possible to add word transitions with no intermediate states. This function may not be necessary for all titles.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

NUI Speech

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechDestroyEvent

See Also [Example](#)

Destroys a speech engine event, and returns the memory to the speech engine.

Syntax

C++

```
HRESULT NuiSpeechDestroyEvent(  
    NUI_SPEECH_EVENT *pEvent  
)
```

Parameters

pEvent
[in] The event to be destroyed, as a pointer to a [NUI_SPEECH_EVENT](#) structure. If NULL, this function does nothing.

Return Value

S_OK.

Remarks

The speech engine allocates internal resources when it creates some speech events. Call this function for each event after the event has been processed. Otherwise, you will have resource leaks.
You must also free the memory that you allocated to hold the event object.

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechDestroyPronunciationList

[See Also](#)



Frees resources associated with word pronunciations.

Syntax

C++

```
HRESULT NuiSpeechDestroyPronunciationList(  
    NUI_SPEECH_WORD_PRONUNCIATION_LIST *pWordPronunciationList  
)
```

Parameters

pWordPronunciationList
[in] A list of pronunciations whose resources are to be freed, as a pointer to a [NUI_SPEECH_WORD_PRONUNCIATION_LIST](#) structure. If NULL, this function does nothing.

Return Value

S_OK.

Remarks

When the specified pronunciations are no longer in use, call this function to free resources associated with those pronunciations. Otherwise, the title might leak memory.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechDestroySerializedState

[See Also](#) [Example](#)



Destroys the serialized state data from the speech engine—data that can be used to get stored speech data from the same source later.

Syntax

C++

```
HRESULT NuiSpeechDestroySerializedState(  
    BYTE *pbData  
)
```

Parameters

pbData
[in] Pointer to the buffer that holds the previously-stored serialized data. If NULL, this function does nothing.

Return Value

S_OK.

Remarks

This function destroys the serialized engine state that is returned by [NuiSpeechGetSerializedState](#).

Example

This function is used in the [SpeechRetainAudio](#) sample.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechDisable

[See Also](#) [Example](#)



Uninitializes and unloads the speech engine.

Syntax

C++

```
HRESULT NuiSpeechDisable()
```

Return Value

NOERROR.

Remarks

This function shuts down the speech engine entirely, and releases its resources for use by titles.

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechEmulateRecognition

[See Also](#)



Performs speech recognition on a text string as though the text were spoken.

Syntax

C++

```
HRESULT NuiSpeechEmulateRecognition(  
    LPCWSTR pcwszText,  
    ULONG Options  
)
```

Parameters

pcwszText

[in] The text to be recognized as though it had been spoken.

Options

[in] A bitwise-OR combination of [NUI_SPEECH_EMULATEOPTIONS](#) enumeration constants describing how the emulation should be performed.

Return Value

NOERROR.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechEnable

[See Also](#) [Example](#)



Initializes the speech engine with the specified language, sound source, and hardware thread.

Syntax

C++

```
HRESULT NuiSpeechEnable(  
    NUI_SPEECH_INIT_PROPERTIES *pInitProperties,  
    DWORD dwHardwareThread  
)
```

Parameters

pInitProperties
[in] A pointer to a [NUI_SPEECH_INIT_PROPERTIES](#) structure that specifies the language to be recognized and whether the sound source is the microphone array or wave data from memory.

dwHardwareThread
[in] The hardware thread on which to run the speech engine, as a thread identifier (0-5).

If you want to use the speech library's default hardware thread, which is thread 3, pass [NUI_INITIALIZE_DEFAULT_HARDWARE_THREAD](#). If your title requires a particular hardware thread for speech recognition, pass the required thread number instead.

If NuiSpeech is the first to initialize NuiAudio, NuiAudio will also use the same hardware thread as speech. That is, if somebody overrides NuiSpeech's default and NuiSpeech is still the first to initialize NuiAudio, NuiAudio will use whichever hardware thread was used to override speech. However, if something other than NuiSpeech initializes NuiAudio first with default parameters (by calling [NuiAudioCreate](#) with the *uHardwareThreadRequested* argument defaulting to [NUIAUDIO_DEFAULT_PROCESSOR](#)), NuiAudio will run on thread 5.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
E_INVALIDARG	One or more of the input arguments is invalid.
E_NUI_DEVICE_NOT_CONNECTED	The Kinect sensor array is not attached.
E_NUI_DATABASE_NOT_FOUND	The language model cannot be found.
E_NUI_DATABASE_VERSION_MISMATCH	The language model is found, but does not match the version required by the title libraries.

Remarks

Calling this function will succeed even if the device is not connected, has no power, or is not ready because it is still initializing after being plugged in. However, recognition won't occur until the device is connected and ready.

To check status on the recognition stream, you can watch for [NUI_SPEECH_EVENT_START_STREAM](#) and [NUI_SPEECH_EVENT_END_STREAM](#) events. To check for hardware status changes, you can use [XNotifyGetNext](#) or [XNuiGetHardwareStatus](#).

For the amount of resources used, see "NUI Speech" in [NUI Resource Usage](#).

Calling **NuiSpeechEnable** is likely to cause disc I/O activity as speech recognition files are loaded. You should not call this function while your title is performing I/O-intensive operations such as loading game resources or playing video.

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechGetDefaultPronunciations

[See Also](#)



Gets the default pronunciations of a specified text string, enabling you to compare the player's pronunciation to the expected pronunciations.

Syntax

```
C++
HRESULT NuiSpeechGetDefaultPronunciations(
    LPCWSTR pcwszText,
    NUI_SPEECH_WORD_PRONUNCIATION_LIST *pWordPronunciationList
)
```

Parameters

pcwszText
[in] A case-sensitive text string containing a word or phrase.

pWordPronunciationList
[out] The default pronunciations of the text string, as a pointer to a title-allocated [NUI_SPEECH_WORD_PRONUNCIATION_LIST](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the pronunciations. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
E_INVALIDARG	One or more of the input arguments is invalid.
SPERR_UNINITIALIZED	The speech system has not been initialized with NuiSpeechEnable .

Remarks

Use this function to retrieve all of the pronunciations of a given word that the recognition engine knows about.

This function retrieves only default pronunciations, not custom pronunciations specified in loaded grammars.

After processing the word pronunciation list, you must call [NuiSpeechDestroyPronunciationList](#) so the speech engine releases resources for the list. Otherwise, you will have memory leaks.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ **See Also**
[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechGetEvents
[See Also](#) [Example](#)



Polls for available events from the speech recognition engine, such as phrase started, phrase recognized, or bookmark reached.

✚ **Syntax**

C++

```
HRESULT NuiSpeechGetEvents(  
    ULONG ulCount,  
    NUI_SPEECH_EVENT *pEventArray,  
    ULONG *pulFetched  
)
```

Parameters

ulCount
[in] The number of [NUI_SPEECH_EVENT](#) structures that fit in the **pEventArray* buffer.

pEventArray
[out] The pending events, as a title-allocated array of [NUI_SPEECH_EVENT](#) structures.
This pointer must be non-NULL when you call the function. After the function call returns, the array contains the events. If this function does not return NOERROR, this data is invalid.

pulFetched
[out] The count of available events, as a pointer to a ULONG.
If no events were available, this is zero.
This pointer must be non-NULL when you call the function. After the function call returns, the ULONG contains the number of events. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not started successfully with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
FAILED(hr)	An error message from the recognizer, if the request cannot be satisfied.

✚ **Remarks**

This is a non-blocking call. If no events are available, the system returns an empty list. You should ensure that **pulFetched* is non-zero before accessing elements in **pEventArray*. Although the title owns the memory for *pEventArray*, you must still call [NuiSpeechDestroyEvent](#) for each event after processing it. This ensures that the speech engine releases any internal resources associated with the event.

✚ **Example**

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

✚ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ **See Also**
[NUI Speech](#)
[NuiSpeechSetEventInterest](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechGetGrammarState
[See Also](#)



Gets whether the specified grammar is enabled and is the only grammar used.

✚ **Syntax**

C++

```
HRESULT NuiSpeechGetGrammarState(  
    NUI_SPEECH_GRAMMAR *pGrammar,  
    NUI_SPEECH_GRAMMARSTATE *pState  
)
```

Parameters

pGrammar

[in] The grammar whose state is to be retrieved, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

This structure should have been returned by a previous call to [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#).

peState

[out] The current state of the grammar, as a pointer to a [NUI_SPEECH_GRAMMARSTATE](#) structure.

This pointer must be non-NULL when you call the function. After the function call returns, the structure contains the grammar state. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechGetProperty

[See Also](#)



Retrieves the current value of a speech engine property.

Syntax

C++

```
HRESULT NuiSpeechGetProperty(  
    NUI_SPEECH_PROPERTY eSpeechProperty,  
    PLONG pIValue  
)
```

Parameters

eSpeechProperty

[in] The property to be retrieved, as a [NUI_SPEECH_PROPERTY](#) enumeration constant.

pIValue

[out] The current value of the property, as a pointer to LONG.

This pointer must be non-NULL when you call the function. After the function call returns, the LONG contains the property value. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	The <i>eSpeechProperty</i> value is not a valid property identifier, or <i>pIValue</i> is an invalid pointer.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechGetSerializedState

[See Also Example](#)



Retrieves the serialized state data from the speech engine, to obtain stored speech data from the same source later.

Syntax

C++

```
HRESULT NuiSpeechGetSerializedState(  
    BYTE **ppbData,  
    ULONG *pulSize  
)
```

Parameters

ppbData
[out] The serialized state data, as a pointer to a buffer.
This pointer must be non-NULL when you call the function. After the function call returns, the buffer contains the data. If this function does not return NOERROR, this data is invalid.

puSize
[out] The size of the data in the buffer, in bytes, as a pointer to a ULONG value.
This pointer must be non-NULL when you call the function. After the function call returns, the ULONG contains the size of the data in the buffer. If this function does not return NOERROR, this value is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
FAILED(hr)	The engine cannot process the request.

Remarks

To get speech data from the same source, the current state of the engine can be stored by calling [NuiSpeechSetSerializedState](#) and then later restored by calling **NuiSpeechGetSerializedState**.
After calling this function, you can call [NuiSpeechDestroySerializedState](#).

Example

This function is used in the [Speech Retain Audio](#) sample.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechLoadGrammar

[See Also](#) [Example](#)



Loads a compiled grammar from disc.

Syntax

C++

```
HRESULT NuiSpeechLoadGrammar(  
    LPCWSTR pcwszGrammarName,  
    ULONG ulGrammarId,  
    NUI_SPEECH_LOADOPTIONS Options,  
    NUI_SPEECH_GRAMMAR *pGrammar  
)
```

Parameters

pcwszGrammarName
[in] The fully-qualified path to the grammar file on disc, as a null-terminated Unicode string. Although Xbox 360 filenames are normally ANSI strings, this parameter takes a Unicode string for the file path.

ulGrammarId
[in] The unique identifier for this grammar. The identifier is set by title code.

Options
[in] The loading options for the grammar, as a bitwise-OR combination of [NUI_SPEECH_LOADOPTIONS](#) enumeration constants.

pGrammar
[out] The loaded grammar, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains a handle to the grammar. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Remarks

It is not recommended that you call this function during the process of speech recognition. Instead, you should first call [NuiSpeechStopRecognition](#), then **NuiSpeechLoadGrammar**, and then [NuiSpeechStartRecognition](#).
Calling **NuiSpeechLoadGrammar** is likely to cause disc I/O activity as speech recognition files are loaded. You should not call this function while your title is performing I/O-intensive operations such as loading game resources or playing video.
When you are entirely finished with the grammar, you should call [NuiSpeechUnloadGrammar](#), to free resources associated with the grammar.
By default, the grammar is enabled and the top-level rule is activated.
Multiple grammars may be loaded and active at the same time.

The grammar format must be a compiled version of a valid World Wide Web Consortium (W3C) grammar. The W3C grammar specification is at <http://www.w3.org/TR/speech-grammar/>.

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechLoadGrammarFromMemory

[See Also](#)

Loads a compiled grammar from a memory buffer supplied by the title.

Syntax

```
C++
HRESULT NuiSpeechLoadGrammarFromMemory(
    const BYTE *pcbGrammar,
    ULONG ulGrammarSize,
    ULONG ulGrammarId,
    NUI_SPEECH_LOADOPTIONS Options,
    NUI_SPEECH_GRAMMAR *pGrammar
)
```

Parameters

- pcbGrammar*
[in] The compiled grammar, as a pointer to a memory buffer.
- ulGrammarSize*
[in] The size of the grammar, in bytes.
- ulGrammarId*
[in] The unique identifier for this grammar. The title establishes the identifier.
- Options*
[in] The loading options, which determine whether a grammar is static or can be modified at run time. This is a bitwise-OR combination of [NUI_SPEECH_LOADOPTIONS](#) enumeration constants.
- pGrammar*
[out] The loaded grammar, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.
This pointer must be non-NULL when you call the function. After the function call returns, the structure contains a handle to the grammar. If this function does not return NOERROR, this data is invalid.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Remarks

This function enables you to do your own I/O and memory management for grammars.
You should call [NuiSpeechUnloadGrammar](#) when you are done with the grammar. This ensures that the speech recognizer frees internal resources associated with this grammar.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechProcessWaveInput

[See Also](#)

Performs speech recognition processing on audio data from a .wav file, rather than using the Kinect microphone array.

Syntax

```
C++
HRESULT NuiSpeechProcessWaveInput(
    LPCWSTR pcwszWaveName
```

)

Parameters

pcwszWaveName
[in] The name of the wave (.wav) file to be processed.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
E_INVALIDARG	No file name was provided.
E_NUI_SPEECH_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_NUI_SPEECH_DEVICE_NOT_SUPPORTED	The microphone type was not set to NUI_SPEECH_WAVE when NuiSpeechEnable was called.
E_NUI_SPEECH_INVALID_AUDIO_STATE	Recognition has not been enabled by a call to NuiSpeechStartRecognition .

+ Remarks

The wave data must be in signed, 16-bit, 16-kHz, mono, little-endian PCM format. There are no special RIFF chunks.
Before calling this function, you must call [NuiSpeechEnable](#), with *MicrophoneType* in the [NUI_SPEECH_INIT_PROPERTIES](#) structure set to [NUI_SPEECH_WAVE](#).
When you don't have any more data to send, call [NuiSpeechStopRecognition](#).

+ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

+ See Also

[NUI_Speech](#)
[NuiSpeechProcessWaveInputFromMemory](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechProcessWaveInputFromMemory

[See Also](#)



Performs speech recognition processing on audio data from memory, rather than using the Kinect microphone array.

+ Syntax

C++

```
HRESULT NuiSpeechProcessWaveInputFromMemory(  
    const BYTE *pcbWave,  
    ULONG ulWaveSize  
)
```

Parameters

pcbWave
[in] A pointer to the wave data to be processed.
ulWaveSize
[in] The size of the wave data to be processed, in bytes.

Return Value

NOERROR if successful; otherwise, an error code.

+ Remarks

The wave data must be in signed, 16-bit, 16kHz, mono, little-endian PCM format. There are no special RIFF chunks.
Before you call this function, you must call [NuiSpeechEnable](#) with *MicrophoneType* in the [NUI_SPEECH_INIT_PROPERTIES](#) structure set to [NUI_SPEECH_WAVE](#).
When you don't have any more data to send, call [NuiSpeechStopRecognition](#).

+ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

+ See Also

[NUI_Speech](#)
[NuiSpeechProcessWaveInput](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetEventInterest

[See Also](#) [Example](#)



Tells the speech engine which events to send to the title, such as phrase started, phrase recognized, or bookmark reached.

+ Syntax

C++

```
HRESULT NuiSpeechSetEventInterest (
    ULONG ulEvents
)
```

Parameters

ulEvents
[in] The events of interest to the title, as a bitwise-OR combination of [NUI_SPEECH_EVENT](#) identifiers.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not started successfully with NuiSpeechEnable .
E_INVALIDARG	The <i>ulEvents</i> parameter includes invalid flags.

Remarks

Before you call [NuiSpeechStartRecognition](#), you should register an event interest by calling **NuiSpeechSetEventInterest**.

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetGrammarState

[See Also](#)



Sets whether the specified grammar is enabled and is the only grammar used.

Syntax

C++

```
HRESULT NuiSpeechSetGrammarState (
    NUI_SPEECH_GRAMMAR *pGrammar,
    NUI_SPEECH_GRAMMARSTATE eState
)
```

Parameters

pGrammar
[in] The grammar whose state to set, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure. This grammar should have been returned by a previous call to [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#).
eState
[in] The new state of the grammar, as a [NUI_SPEECH_GRAMMARSTATE](#) enumeration constant.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Remarks

This function activates or deactivates the top-level rule of the passed-in grammar object. Calling this function is equivalent to calling [NuiSpeechSetRuleState](#) on the top-level rule in the grammar.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetProperty

[See Also](#) [Example](#)



Sets a speech engine property, to customize the speech engine behavior.

Syntax

C++

```
HRESULT NuiSpeechSetProperty(  
    NUI_SPEECH_PROPERTY eSpeechProperty,  
    LONG lNewValue  
)
```

Parameters

eSpeechProperty
[in] The property to set, as a [NUI_SPEECH_PROPERTY](#) enumeration constant.

lNewValue
[in] The new value of the property.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	The <i>eSpeechProperty</i> value is not a valid property identifier, or <i>lValue</i> is not within the valid range for the specified property.
FAILED(hr)	An appropriate error message if the engine cannot apply the new value.

Example

This function is used in the [Speech Retain Audio](#) sample.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetRecognitionBookmark

[See Also](#)



Sets a bookmark in the current speech recognition stream, so that when the recognizer reaches the bookmark, a bookmark event is raised.

Syntax

C++

```
HRESULT NuiSpeechSetRecognitionBookmark(  
    ULONG Options,  
    ULONGLONG ullStreamPosition,  
    PVOID pvUserContext  
)
```

Parameters

Options
[in] Flags that describe how *ullStreamPosition* and *pvUserContext* are to be interpreted, as a bitwise-OR combination of [NUI_SPEECH_BOOKMARKOPTIONS](#) enumeration constants.

ullStreamPosition
[in] A stream position value. For a description of how *Options* settings specify the meaning of this value, see [NUI_SPEECH_BOOKMARKOPTIONS](#).

pvUserContext
[in, optional] The title-defined context data. This data will be returned to the title in the [NUI_SPEECH_EVENT_BOOKMARK](#) event that is raised when the bookmark is reached. This can be NULL.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

Remarks

When the speech recognizer reaches the specified stream position, a bookmark event ([NUI_SPEECH_EVENT_BOOKMARK](#)) is raised.

To receive this notification, you must call [NuiSpeechSetEventInterest](#) and register to receive NUI_SPEECH_EVENT_BOOKMARK events.

A bookmark correlates an audio stream position with an occurrence that is relevant to the title. The recognition engine has a latency period between the speech and the recognition. During this latency period, the player or title may continue operation such that the state of the title when the recognition returns is different from the state of the title when the recognition was initiated. This function enables the title to record a bookmark for the particular recognition attempt. An example is displaying a progress meter for speech recognition.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetRulePriority

[See Also](#)



Sets the priority of the specified rule.

Syntax

C++

```
HRESULT NuiSpeechSetRulePriority(
    NUI_SPEECH_GRAMMAR *pGrammar,
    LPCWSTR pcwszRuleName,
    int nPriority
)
```

Parameters

- pGrammar*
[in] The grammar that contains the rule, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.
- pcwszRuleName*
[in] The rule whose priority is to be set.
- nPriority*
[in] The new priority value for the rule. Valid values are from -128 to 127. Higher numbers are given priority.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine has not been successfully started with NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
FAILED(hr)	The engine cannot process the request.

Remarks

If several rules have the same priority, or if no priority has been assigned, the speech recognizer uses the most recently activated rules.

When an utterance matches multiple rules, all with comparable likelihood, then the rule with higher priority is reported as the first recognition result. Priority is only a factor in recognition results when more than one rule has been found that matches an utterance, and all the rules are recognized with roughly equal confidence.

One common grammar design is to have a single "system command" grammar that is always active, and multiple specific grammars that have commands that apply in specific areas of the game. The area-specific grammars are activated or deactivated according to where players are currently active. In such a design, the rules in the "system command" grammar are given higher-priority values, to ensure that the system commands always take precedence over similar-sounding commands from area-specific grammars.

Requirements

- Header:** Declared in NuiSpeech.h. For include, use NuiApi.h.
- Library:** Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetRuleState

[See Also](#)



Activates or deactivates the passed-in top-level rule object.

Syntax

C++

```
HRESULT NuiSpeechSetRuleState(
    NUI_SPEECH_GRAMMAR *pGrammar,
    LPCWSTR pcwszRuleName,
    NUI_SPEECH_RULESTATE eState
)
```

Parameters

- pGrammar*
[in] The grammar that contains the rule, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.
- pcwszRuleName*
[in] The name of the top-level rule whose state is to be set.
- eState*
[in] The new state of the rule, as a [NUI_SPEECH_RULESTATE](#) enumeration constant.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
SPERR_NOT_TOPLEVEL_RULE	The rule exists, but it is not a top-level rule. Only top-level rules can be activated or deactivated.
FAILED(hr)	The function fails if the engine cannot process the request.

Remarks

Only top-level rules can be activated or deactivated.

You should not activate rules until the title is ready to start receiving events. This is because if [NuiSpeechStartRecognition](#) has been called already, **NuiSpeechSetRuleState** immediately activates or deactivates the rule.

You can set or retrieve the rule state from within individual rules in a grammar. Manipulating the rules from within a grammar offers a finer level of control over which rules are used for recognition, compared to **NuiSpeechSetRuleState**.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetRuleWeight

[See Also](#)

Sets the weight on the specified rule.

Syntax

```
C++
HRESULT NuiSpeechSetRuleWeight(
    NUI_SPEECH_GRAMMAR *pGrammar,
    LPCWSTR pcwszRuleName,
    FLOAT fWeight
)
```

Parameters

pGrammar
[in] The grammar that contains the rule, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure.

pcwszRuleName
[in] The rule whose weight is to be set.

fWeight
[in] The new weight value for the rule. Valid values are 0 through 1.0.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.
FAILED(hr)	The function fails if the engine cannot process the request.

Remarks

The speech recognizer uses rule weight to decide whether an utterance is recognized, and to rank confidence values among alternatives. If an utterance is a potential match for more than one rule, the rule with highest weight is returned as the highest-confidence recognition result, and rules with lower weight are reported as lower-confidence recognition results.

Weights should be set as a result of user testing with the specific grammar being used by a title. Even a simple "yes"/"no" grammar will use very different weights in different situations, depending on how questions are posed and what responses players have been conditioned to use.

For example, play testing of one title might reveal that in a certain game state involving a yes/no grammar, players say variations on yes (yes, yup, OK) two thirds of the time, and variations on no (no, nope) one third of the time. Based on these findings, you would assign a weight to the yes rule that was twice the weight of the no rule. In another title, play testing might reveal different statistics for yes versus no. For the second title, you would set weights differently, based on the test results for that title.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechSetSerializedState

[See Also](#)



Stores serialized state data from the speech engine, to get this stored speech data from the same source later.

Syntax

C++

```
HRESULT NuiSpeechSetSerializedState(  
    const BYTE *pbData,  
    ULONG ulSize  
)
```

Parameters

pbData

[in] A pointer to the buffer that will hold the serialized data.

ulSize

[in] The size of the buffer, in bytes.

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments is invalid.
FAILED(hr)	The engine cannot process the request.

Remarks

To get speech data from the same source, you can call this function to store the current state of the engine, and then later call [NuiSpeechGetSerializedState](#) to restore it.

Requirements

Header:

Declared in NuiSpeech.h. For include, use NuiApi.h.

Library:

Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechStartRecognition

[See Also](#) [Example](#)



Begins the speech recognition process, using the enabled grammars.

Syntax

C++

```
HRESULT NuiSpeechStartRecognition()
```

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .

Remarks

This function enables all grammars that are loaded and set to Enabled, or enables the grammar that is loaded and set to Exclusive, and begins the speech recognition process.

If you are using [NuiSpeechGetEvents](#) to get speech events, you should call [NuiSpeechSetEventInterest](#) to register which events will be available before you call **NuiSpeechStartRecognition**.

It is recommended that you do not begin speech recognition until you set the event interest.

Recognition starts only if at least one top-level rule exists. By default, when a grammar is loaded, it is placed in an active (enabled) state so that recognition starts immediately after this call.

Use this function to restart recognition after you use **NuiSpeechStopRecognition** to suspend recognition.

If you call **NuiSpeechDisable** to unload the speech engine, you must call **NuiSpeechEnable** to reload it. Use the following sequence of function calls:

- [NuiSpeechStopRecognition](#)
- [NuiSpeechDisable](#)
- [NuiSpeechEnable](#)
- NuiSpeechStartRecognition**

Calling **NuiSpeechStartRecognition** will succeed even if the device is not connected, has no power, or is not ready because it is still initializing after being plugged in. However, recognition won't occur until the device is connected and ready.

To check status on the recognition stream, you can watch for [NUI_SPEECH_EVENT_START_STREAM](#) and [NUI_SPEECH_EVENT_END_STREAM](#) events. To check for hardware status changes, you can use [XNotifyGetNext](#) or [XNuiGetHardwareStatus](#).

Example

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechStopRecognition

[See Also](#) [Example](#)



Suspends speech recognition by disabling all grammars.

⊕ **Syntax**

C++

```
HRESULT NuiSpeechStopRecognition()
```

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .

⊕ **Remarks**

When a title changes a rule or a grammar state, you should stop speech recognition and then restart it. This ensures that the code that processes results only handles recognition events that are relevant for the latest grammar state.

After calling this function, the speech engine remains loaded in memory. To shut down the speech engine and release its resources, call [NuiSpeechDisable](#).

⊕ **Example**

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeechUnloadGrammar

[See Also](#) [Example](#)



Unloads resources associated with the specified grammar.

⊕ **Syntax**

C++

```
HRESULT NuiSpeechUnloadGrammar(  
    NUI_SPEECH_GRAMMAR *pGrammar  
)
```

Parameters

pGrammar

[in] The grammar to be unloaded, as a pointer to a [NUI_SPEECH_GRAMMAR](#) structure. This grammar should have been returned by a previous call to [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#).

Return Value

One of the following values:

Value	Meaning
NOERROR	The function completed successfully.
SPERR_UNINITIALIZED	The speech engine did not start when you called NuiSpeechEnable .
E_INVALIDARG	One or more arguments are invalid.

⊕ **Remarks**

Unloading a grammar closes all resources associated with the grammar. If the title might use a grammar again later, it is more efficient to disable the grammar using [NuiSpeechSetGrammarState](#) rather than to unload later reload the grammar.

⊕ **Example**

This function is used in the [Simple Speech Recognition](#) and [Speech Retain Audio](#) samples.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeech Structures

[See Also](#)

In This Section

[NUI_SPEECH_ELEMENT](#)

Describes a recognized word that is part of a successful recognition result, including the length and location of the word in the stream, the pronunciation of the word, and the confidence of the word.

[NUI_SPEECH_EVENT](#)

Describes a speech engine event, including the type of event, position in the stream, and recognition result.

[NUI_SPEECH_GRAMMAR](#)

Encapsulates a speech grammar, by associating a grammar ID with a handle to the grammar.

[NUI_SPEECH_INIT_PROPERTIES](#)

Specifies a language and whether the input is the microphone array or wave data.

[NUI_SPEECH_PHRASE](#)

Describes a phrase recognized by the speech engine, including length, number of elements, language, grammar, stream offset, and recognition rule.

[NUI_SPEECH_RECORESLT](#)

Describes a speech recognition result, including the recognized phrase and any alternate recognized phrases.

[NUI_SPEECH_RETAINEDAUDIO](#)

Describes buffered audio data, including size and location.

[NUI_SPEECH_RULERESULT](#)

Describes a rule matched in a successful recognition result, including rule name, index of the word in the phrase, number of words spanned, and confidence.

[NUI_SPEECH_SEMANTIC](#)

Defines a numeric ID for a semantic property when building a grammar, enabling handling recognition results through numeric identifiers rather than having to parse strings.

[NUI_SPEECH_SEMANTICRESULT](#)

Describes the semantic property of a successful recognition result, including value, first word spanned, number of words spanned, and confidence value.

[NUI_SPEECH_WORD_PRONUNCIATION](#)

Encapsulates a word pronunciation, including pronunciation, language, and whether a user or vendor provided the pronunciation.

[NUI_SPEECH_WORD_PRONUNCIATION_LIST](#)

Encapsulates an array of word pronunciations.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_ELEMENT Structure

[See Also](#)

Describes a recognized word that is part of a successful recognition result, including the length and location of the word in the stream, the pronunciation of the word, and the confidence of the word.

Syntax

C++

```
typedef struct _NUI_SPEECH_ELEMENT {
    ULONG ulAudioTimeOffset;
    ULONG ulAudioSizeTime;
    ULONG ulAudioStreamOffset;
    ULONG ulAudioSizeBytes;
    ULONG ulRetainedStreamOffset;
    ULONG ulRetainedSizeBytes;
    LPCWSTR pcwszDisplayText;
    LPCWSTR pcwszLexicalForm;
    LPCWSTR pcwszPronunciation;
    NUI_SPEECH_CONFIDENCE eRequiredConfidence;
    NUI_SPEECH_CONFIDENCE eActualConfidence;
    FLOAT fSREngineConfidence;
} NUI_SPEECH_ELEMENT;
```

Members

- ulAudioTimeOffset**
The starting offset of the element from the start of the phrase, in 100-nanosecond units.
- ulAudioSizeTime**
The length of the element, in 100-nanosecond units.
- ulAudioStreamOffset**
The starting offset of the element, relative to the start of the phrase in the input stream, in bytes.
- ulAudioSizeBytes**
The size of the element in the input stream, in bytes.
- ulRetainedStreamOffset**
The starting offset of the element, relative to the start of the phrase in the retained audio stream, in bytes.

ulRetainedSizeBytes
The size of the element in the retained audio stream, in bytes.

pcwszDisplayText
The null-terminated display text for this element.

pcwszLexicalForm
The null-terminated lexical form of this element.

pcwszPronunciation
The pronunciation of this element, as a null-terminated array of phone IDs. Each phone ID is a WCHAR.

eRequiredConfidence
The confidence value threshold required to recognize this element, as a [NUI_SPEECH_CONFIDENCE](#) enumeration constant.
If a word is prefixed with a minus sign ("-"), the required confidence is NUI_SPEECH_CONFIDENCE_LOW. If a word is prefixed with a plus sign ("+"), the required confidence is NUI_SPEECH_CONFIDENCE_HIGH. For example: "This -is a +test".

eActualConfidence
The actual confidence value when this element was recognized, as a [NUI_SPEECH_CONFIDENCE](#) enumeration constant. This value is always at least the value specified for *eRequiredConfidence*.

fsREngineConfidence
The confidence value for this element. Valid values are 0 through 1.0.

Remarks

A [NUI_SPEECH_PHRASE](#) structure includes a pointer to an array of **NUI_SPEECH_ELEMENT** structures, which are referred to by [NUI_SPEECH_RULERESULT](#) and [NUI_SPEECH_SEMANTICRESULT](#) structures.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_EVENT Structure

[See Also](#)

Describes a speech engine event, including the type of event, position in the stream, and recognition result.

Syntax

```
C++

typedef struct _NUI_SPEECH_EVENT {
    ULONG eventId;
    ULONG ulStreamNum;
    ULONGLONG ullAudioStreamOffset;
    union {
        NUI_SPEECH_RECORESLT *pResult;
        LPVOID pvUserContext;
        NUI_SPEECH_INTERFERENCE eInterference;
        NUI_SPEECH_RETAINEDAUDIO Audio;
    };
} NUI_SPEECH_EVENT;
```

Members

eventId
The type of event raised, as a [NUI_SPEECH_EVENT Identifiers](#) constant.

ulStreamNum
The stream number associated with the event. Every time that a new audio stream is opened, its stream number is one greater than the number of the previously opened stream.

ullAudioStreamOffset
The byte offset at which the event was raised, in the input audio stream associated with the event.

pResult
The recognition result, as a pointer to a [NUI_SPEECH_RECORESLT](#) structure, if **eventId** is NUI_SPEECH_EVENT_HYPOTHESIS, [NUI_SPEECH_EVENT RECOGNITION](#), or NUI_SPEECH_EVENT_FALSE_RECOGNITION.

pvUserContext
A pointer to the user context that was set when [NuiSpeechSetRecognitionBookmark](#) was called to create the bookmark, if **eventId** is [NUI_SPEECH_EVENT BOOKMARK](#).

eInterference
The kind of interference, as a [NUI_SPEECH_INTERFERENCE](#) enumeration constant, if **eventId** is [NUI_SPEECH_EVENT INTERFERENCE](#).

Audio
The retained audio data, as a [NUI_SPEECH_RETAINEDAUDIO](#) structure, if **eventId** is [NUI_SPEECH_EVENT RETAINEDAUDIO](#).

Remarks

This structure is passed to [NuiSpeechDestroyEvent](#). [NuiSpeechGetEvents](#) returns an array of **NUI_SPEECH_EVENT** structures.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_GRAMMAR Structure

[See Also](#)

Encapsulates a speech grammar, by associating a grammar ID with a handle to the grammar.

+

Syntax

C++

```
typedef struct _NUI_SPEECH_GRAMMAR {
    ULONG ulId;
    NUI_SPEECH_HANDLE hGrammar;
} NUI_SPEECH_GRAMMAR;
```

Members

ulId

The unique grammar ID that was set by the title when calling [NuiSpeechLoadGrammar](#).

hGrammar

A handle to implementation-specific information that is unique to this grammar. Do not use or modify this value.

+

Remarks

This structure is used by many of the NuiSpeech functions. For example, this structure is passed to [NuiSpeechCreateRule](#) and [NuiSpeechSetRulePriority](#), and is returned by [NuiSpeechLoadGrammar](#).

+

Requirements

Header:

Declared in NuiSpeech.h. For include, use NuiApi.h.

Library:

Use NuiSpeech.lib and NuiAudio.lib.

+

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_INIT_PROPERTIES Structure

[See Also](#)

Specifies a language and whether the input is the microphone array or wave data.

+

Syntax

C++

```
typedef struct _NUI_SPEECH_INIT_PROPERTIES {
    NUI_SPEECH_LANGUAGE Language;
    NUI_SPEECH_MICTYPE MicrophoneType;
} NUI_SPEECH_INIT_PROPERTIES;
```

Members

Language

The language to be recognized, as a [NUI_SPEECH_LANGUAGE](#) enumeration constant.

MicrophoneType

The type of audio source being used for input, as a [NUI_SPEECH_MICTYPE](#) enumeration constant.

+

Remarks

This structure is passed to [NuiSpeechEnable](#).

+

Requirements

Header:

Declared in NuiSpeech.h. For include, use NuiApi.h.

Library:

Use NuiSpeech.lib and NuiAudio.lib.

+

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_PHRASE Structure

[See Also](#)

Describes a phrase recognized by the speech engine, including length, number of elements, language, grammar, stream offset, and recognition rule.

+

Syntax

C++

```
typedef struct _NUI_SPEECH_PHRASE {
    ULONG cbSize;
    NUI_SPEECH_LANGUAGE eLanguage;
    ULONG ulGrammarID;
    ULONGLONG ullStartTime;
    ULONGLONG ullAudioStreamPosition;
    ULONG ulAudioSizeBytes;
```

```
        ULONG ulAudioSizeTime;
        ULONG ulRetainedSizeBytes;
        NUI_SPEECH_RULERESULT Rule;
        ULONG ulCountOfElements;
        NUI_SPEECH_ELEMENT *pElements;
        NUI_SPEECH_SEMANTICRESULT *pSemanticProperties;
    } NUI_SPEECH_PHRASE;
```

Members

- cbSize**
The size of this structure, in bytes.
- eLanguage**
The language in which the phrase is spoken, as a [NUI_SPEECH_LANGUAGE](#) enumeration constant.
- ulGrammarID**
The title-defined identifier of the grammar that contains the top-level rule used to recognize this phrase.
- ullStartTime**
The absolute time for the start of the phrase, as a 64-bit value.
If an application uses a wave file for input, this value is set to zero.
- ullAudioStreamPosition**
The starting offset of the phrase, in bytes, from the start of the audio stream. If downsampling an audio stream, this value is the byte position in the original audio stream.
If an application uses a wave file for input, this value is set to zero.
- ulAudioSizeBytes**
The size of the audio data for this phrase, in bytes.
- ulAudioSizeTime**
The length of the phrase audio, in units of 100 nanoseconds.
- ulRetainedSizeBytes**
The size of the retained audio data, in bytes.
- Rule**
The top-level rule used to recognize this phrase, as a [NUI_SPEECH_RULERESULT](#) structure. This rule is the root of the rule hierarchy.
- ulCountOfElements**
The number of elements that comprise the recognized phrase. This value is the same as the value of the **ulCountOfElements** member of the **Rule** member of this structure. It is provided here for convenience.
- pElements**
An array of the [NUI_SPEECH_ELEMENT](#) elements in this phrase. Each element includes position and text information.
- pSemanticProperties**
A pointer to the root of the semantic tag property tree. This is a tree of [NUI_SPEECH_SEMANTICRESULT](#) elements, not an array.

Remarks

A speech recognition result structure ([NUI_SPEECH_RECORESULT](#)) contains a **NUI_SPEECH_PHRASE** structure.

Requirements

- Header:** Declared in NuiSpeech.h. For include, use NuiApi.h.
- Library:** Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RECORESULT Structure

[See Also](#)



Describes a speech recognition result, including the recognized phrase and any alternate recognized phrases.

Syntax

C++

```
typedef struct _NUI_SPEECH_RECORESULT {
    NUI_SPEECH_PHRASE Phrase;
    ULONG cAlternates;
    NUI_SPEECH_PHRASE *pAlternates;
    NUI_SPEECH_RETAINEDAUDIO *pAudio;
} NUI_SPEECH_RECORESULT;
```

Members

- Phrase**
The recognized phrase with the highest confidence value, as a [NUI_SPEECH_PHRASE](#) structure.
- cAlternates**
The number of alternate phrases recognized by the speech engine.
- pAlternates**
Alternate phrases, as an array of [NUI_SPEECH_PHRASE](#) structures. Phrases are sorted in decreasing confidence order.
The recognition result doesn't automatically contain alternate phrases. To receive alternates, set [NUI_SPEECH_PROPERTY_MAXIMUM_ALTERNATES](#) to a non-zero, positive value.
- pAudio**
The retained audio used to generate this recognition result, as a pointer to a [NUI_SPEECH_RETAINEDAUDIO](#) structure, if the [NUI_SPEECH_PROPERTY_RETAIN_AUDIO](#) property has been set to TRUE. If the [NUI_SPEECH_PROPERTY_RETAIN_AUDIO](#) property has been set to FALSE, this value is NULL.

Remarks

This structure is included in [NUI_SPEECH_EVENT_HYPOTHESIS](#), [NUI_SPEECH_EVENT_RECOGNITION](#), and [NUI_SPEECH_EVENT_FALSE_RECOGNITION](#) events (described in [NUI_SPEECH_EVENT Identifiers](#)). That is, a [NUI_SPEECH_EVENT](#) structure contains a **NUI_SPEECH_RECORESULT** structure, if the **eventId** in the [NUI_SPEECH_EVENT](#) is one of those event types.

You should use the item counts in the returned phrases and verify that all data is valid before attempting to use it. [NUI_SPEECH_EVENT_FALSE_RECOGNITION](#) events will not necessarily provide complete element, rule, or semantic property information. The structures will accurately describe themselves, but they may not contain the expected number of entries.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ **See Also**
[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RETAINEDAUDIO Structure

[See Also](#)



Describes buffered audio data, including size and location.

✚ **Syntax**

C++

```
typedef struct _NUI_SPEECH_RETAINEDAUDIO {  
    ULONG ulAudioSize;  
    BYTE *pAudioData;  
} NUI_SPEECH_RETAINEDAUDIO;
```

Members

ulAudioSize
The count of bytes of audio data retained in the buffer.

pAudioData
A pointer to a buffer containing the retained audio. This audio data is formatted as a signed, 16-bit PCM of raw data with a 16-KHz sample rate; its byte order is Big-Endian.

✚ **Remarks**

This structure describes retained audio data returned by a [NUI_SPEECH_EVENT_RETAINEDAUDIO](#) event. A [NUI_SPEECH_EVENT](#) structure contains a **NUI_SPEECH_RETAINEDAUDIO** structure, if **eventId** in the [NUI_SPEECH_EVENT](#) is [NUI_SPEECH_EVENT_RETAINEDAUDIO](#).

A speech recognition result structure ([NUI_SPEECH_RECORESULT](#)) optionally contains a **NUI_SPEECH_RETAINEDAUDIO** structure.

✚ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ **See Also**
[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RULERESULT Structure

[See Also](#)



Describes a rule matched in a successful recognition result, including rule name, index of the word in the phrase, number of words spanned, and confidence.

✚ **Syntax**

C++

```
typedef struct _NUI_SPEECH_RULERESULT {  
    LPCWSTR pcwszName;  
    ULONG ulId;  
    ULONG ulFirstElement;  
    ULONG ulCountOfElements;  
    _NUI_SPEECH_RULERESULT *pNextSibling;  
    _NUI_SPEECH_RULERESULT *pFirstChild;  
    FLOAT fSREngineConfidence;  
    NUI_SPEECH_CONFIDENCE eConfidence;  
} NUI_SPEECH_RULERESULT;
```

Members

pcwszName
The null-terminated name of this rule.

ulId
The unique identifier of this rule.

ulFirstElement
The index of the first spoken word of this rule. The actual [NUI_SPEECH_ELEMENT](#) array indexed by this value is the **pElements** member of the corresponding [NUI_SPEECH_PHRASE](#) structure.

ulCountOfElements
The number of spoken words spanned by this rule.

pNextSibling
A pointer to this rule's next sibling in the parse tree.

pFirstChild
A pointer to this rule's first child in the parse tree.

fSREngineConfidence
The confidence value for this rule, as computed by the speech engine. Confidence for this rule is the overall confidence for the entire phrase. Valid values are 0 to 1.0.

eConfidence
A [NUI_SPEECH_CONFIDENCE](#) value derived from the raw confidence value according to threshold values set by the title.

✚ **Remarks**

A [NUI_SPEECH_PHRASE](#) structure contains a **NUI_SPEECH_RULERESULT** structure.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_SEMANTIC Structure

[See Also](#)



Defines a numeric ID for a semantic property when building a grammar, enabling handling recognition results through numeric identifiers rather than having to parse strings.

⊕ **Syntax**

C++

```
typedef struct _NUI_SPEECH_SEMANTIC {  
    LPCWSTR pcwszValue;  
} NUI_SPEECH_SEMANTIC;
```

Members

pcwszValue
Null-terminated value of this semantic property.

⊕ **Remarks**

This structure is passed to [NuiSpeechAddRuleTransition](#) and [NuiSpeechAddWordTransition](#). Titles that use these functions to build grammars dynamically use this structure to specify the semantic associated with the added transition. Titles that use static grammars in XML files specify the semantic using <tag>.
Semantic properties are optionally used in a grammar to assign a recognized word or phrase to a name-value pair. This enables title code to handle recognition results using numeric identifiers rather than having to parse strings from the recognition results.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_SEMANTICRESULT Structure

[See Also](#)



Describes the semantic property of a successful recognition result, including value, first word spanned, number of words spanned, and confidence value.

⊕ **Syntax**

C++

```
typedef struct _NUI_SPEECH_SEMANTICRESULT {  
    LPCWSTR pcwszValue;  
    ULONG ulFirstElement;  
    ULONG ulCountOfElements;  
    _NUI_SPEECH_SEMANTICRESULT *pFirstChild;  
    _NUI_SPEECH_SEMANTICRESULT *pNextSibling;  
    FLOAT fSREngineConfidence;  
    NUI_SPEECH_CONFIDENCE eConfidence;  
} NUI_SPEECH_SEMANTICRESULT;
```

Members

pcwszValue
The null-terminated value of this semantic property. This is the string specified using <tag> in the XML file describing a static grammar, or passed as a [NUI_SPEECH_SEMANTIC](#) to [NuiSpeechAddRuleTransition](#) or [NuiSpeechAddWordTransition](#) when building a dynamic grammar.

ulFirstElement
The first spoken word spanned by this semantic property.

ulCountOfElements
The number of spoken words spanned by this semantic property.

pFirstChild
A pointer to the first child of this semantic property.

pNextSibling
A pointer to the next child of this semantic property.

fSREngineConfidence
A confidence value for this semantic property, as computed by the speech engine. Valid values are 0 to 1.0.

eConfidence
A [NUI_SPEECH_CONFIDENCE](#) value derived from the raw confidence value, according to threshold values set by the title.

⊕ **Remarks**

A [NUI_SPEECH_PHRASE](#) structure contains a tree of **NUI_SPEECH_SEMANTICRESULT** structures.

Semantic properties are optionally used in a grammar to assign a recognized word or phrase to a name-value pair. This enables title code to handle recognition results using numeric identifiers rather than having to parse strings from the recognition results. **NUI_SPEECH_SEMANTICRESULT** contains semantic information about a recognition result.

The **pcwszValue** member duplicates the same-named member of the [NUI_SPEECH_SEMANTIC](#) structure that points to this **NUI_SPEECH_SEMANTICRESULT**.

If this semantic property is defined on a rule or rule reference, then **fsEngineConfidence** and **eConfidence** represent the confidence value for all words in the rule.

If this semantic property is defined on a word transition, then **fsEngineConfidence** and **eConfidence** represent the confidence value for the word or element.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_WORD_PRONUNCIATION Structure

[See Also](#)



Encapsulates a word pronunciation, including pronunciation, language, and whether a user or vendor provided the pronunciation.

⊕ **Syntax**

C++

```
typedef struct _NUI_SPEECH_WORD_PRONUNCIATION {  
    LPCWSTR pcwszPronunciation;  
    NUI_SPEECH_LANGUAGE eLanguage;  
    NUI_SPEECH_LEXICON_TYPE eLexiconType;  
} NUI_SPEECH_WORD_PRONUNCIATION;
```

Members

pcwszPronunciation
The pronunciation.

eLanguage
The language of the pronunciation, as a [NUI_SPEECH_LANGUAGE](#) enumeration constant.

eLexiconType
The lexicon type used to determine the pronunciation, as a [NUI_SPEECH_LEXICON_TYPE](#) enumeration constant.

⊕ **Remarks**

A [NUI_SPEECH_WORD_PRONUNCIATION_LIST](#) structure contains an array of **NUI_SPEECH_WORD_PRONUNCIATION** structures.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_WORD_PRONUNCIATION_LIST Structure

[See Also](#)



Encapsulates an array of word pronunciations.

⊕ **Syntax**

C++

```
typedef struct _NUI_SPEECH_WORD_PRONUNCIATION_LIST {  
    ULONG ulCountOfPronunciations;  
    NUI_SPEECH_WORD_PRONUNCIATION *pWordPronunciations;  
} NUI_SPEECH_WORD_PRONUNCIATION_LIST;
```

Members

ulCountOfPronunciations
The number of [NUI_SPEECH_WORD_PRONUNCIATION](#) structures in the array.

pWordPronunciations
A pointer to the first [NUI_SPEECH_WORD_PRONUNCIATION](#) structure in the array.

⊕ **Remarks**

This structure is returned by [NuiSpeechGetDefaultPronunciations](#), and is passed to [NuiSpeechDestroyPronunciationList](#).

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

NuiSpeech Enumerations

[See Also](#)



In This Section

- [NUI_SPEECH_BOOKMARKOPTIONS](#)
Specifies the units for the stream position, and whether the stream position is relative, when setting a bookmark in the speech recognition stream to raise an event.
- [NUI_SPEECH_CONFIDENCE](#)
Describes the level of confidence that speech matches a particular rule.
- [NUI_SPEECH_EMULATEOPTIONS](#)
Specifies options for emulating speech recognition on a text string, including whether to ignore case and whether to test the confusability of phrases.
- [NUI_SPEECH_GRAMMARSTATE](#)
Specifies whether a particular grammar is enabled for speech recognition and is the only grammar used.
- [NUI_SPEECH_INTERFERENCE](#)
Describes why sound was rejected by the speech engine as interference, such as too quiet or too fast.
- [NUI_SPEECH_LANGUAGE](#)
Specifies the language engine, grammar files, and lexicon used by the speech engine, such as United States English or Canadian French.
- [NUI_SPEECH_LEXICON_TYPE](#)
Specifies the type of lexicon from which pronunciations were retrieved, such as phonetic input from the player or vendor, or spelling-based pronunciation.
- [NUI_SPEECH_LOADOPTIONS](#)
Specifies whether a speech recognition grammar is loaded statically, or dynamically so that it can be modified at run time.
- [NUI_SPEECH_MICTYPE](#)
Specifies whether the speech recognition engine uses the microphone array or stored waveform data as input.
- [NUI_SPEECH_PROPERTY](#)
Specifies which property of the speech engine is being set or retrieved, such as a confidence threshold, the length of silence before completion, or whether to buffer the audio.
- [NUI_SPEECH_RULEOPTIONS](#)
Describes how a rule is used, such as whether it is a top-level rule, whether the rule is imported from another grammar, and whether the rule can be modified dynamically at run time.
- [NUI_SPEECH_RULESTATE](#)
Specifies whether a rule is active (enabled).
- [NUI_SPEECH_WORDTYPE](#)
Indicates whether a word is in lexical form and whether special characters are ignored for word transitions, when adding a word to a grammar.

✚ See Also

[NUI Speech](#)

NUI_SPEECH_BOOKMARKOPTIONS Enumeration

[See Also](#)



Specifies the units for the stream position, and whether the stream position is relative, when setting a bookmark in the speech recognition stream to raise an event.

✚ Syntax

```
C++
typedef enum _NUI_SPEECH_BOOKMARKOPTIONS
{
    NUI_SPEECH_BOOKMARKOPTIONS_NONE = 0x0,
    NUI_SPEECH_BOOKMARKOPTIONS_TIME_UNITS = 0x1,
    NUI_SPEECH_BOOKMARKOPTIONS_AHEAD = 0x2
} NUI_SPEECH_BOOKMARKOPTIONS;
```

✚ Constants

Constant	Description
NUI_SPEECH_BOOKMARKOPTIONS_NONE	When this value is specified, values passed to NuiSpeechSetRecognitionBookmark take the default options. Values are in bytes from the beginning of the given stream.
NUI_SPEECH_BOOKMARKOPTIONS_TIME_UNITS	When this value is specified, the value passed to NuiSpeechSetRecognitionBookmark is in 100-nanosecond units. When this value is not specified, the value passed is in bytes.
NUI_SPEECH_BOOKMARKOPTIONS_AHEAD	When this value is specified, the value passed to NuiSpeechSetRecognitionBookmark is relative, ahead of the current position.

✚ Remarks

These values are used to specify options when calling [NuiSpeechSetRecognitionBookmark](#).
To combine these values, use bitwise-OR.

✚ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ See Also

[NUI Speech](#)

NUI_SPEECH_CONFIDENCE Enumeration

[See Also](#)



Describes the level of confidence that speech matches a particular rule.

Syntax

```
C++

typedef enum _NUI_SPEECH_CONFIDENCE
{
    NUI_SPEECH_CONFIDENCE_LOW,
    NUI_SPEECH_CONFIDENCE_NORMAL,
    NUI_SPEECH_CONFIDENCE_HIGH
} NUI_SPEECH_CONFIDENCE;
```

Constants

Constant	Description
NUI_SPEECH_CONFIDENCE_LOW	Separates the ranges of low and rejected confidence.
NUI_SPEECH_CONFIDENCE_NORMAL	Separates the ranges of normal and low confidence.
NUI_SPEECH_CONFIDENCE_HIGH	Separates the ranges of high and normal confidence.

Remarks

A [NUI_SPEECH_ELEMENT](#) structure contains two **NUI_SPEECH_CONFIDENCE** enumeration constants, representing the required confidence and the actual confidence. A [NUI_SPEECH_RULERESULT](#) structure and a [NUI_SPEECH_SEMANTICRESULT](#) structure contain a **NUI_SPEECH_CONFIDENCE** enumeration constant.

This enumeration specifies valid values for the speech engine's confidence threshold properties. These properties serve to place the engine confidence into low, normal, and high confidence buckets. Their valid range is between 0 and 100.

When not otherwise specified, the default threshold values apply. These are shown in the following table.

Confidence range	Default value	Constant
Low	20	NUI_SPEECH_CONFIDENCE_LOW
Normal	50	NUI_SPEECH_CONFIDENCE_NORMAL
High	80	NUI_SPEECH_CONFIDENCE_HIGH

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI_Speech](#)

NUI_SPEECH_EMULATEOPTIONS Enumeration

[See Also](#)



Specifies options for emulating speech recognition on a text string, including whether to ignore case and whether to test the confusability of phrases.

Syntax

```
C++

typedef enum _NUI_SPEECH_EMULATEOPTIONS
{
    NUI_SPEECH_EMULATEOPTIONS_NONE = 0x00000000,
    NUI_SPEECH_EMULATEOPTIONS_IGNORECASE = 0x00000001,
    NUI_SPEECH_EMULATEOPTIONS_GRAMMAR_CONFUSION = 0x10000000,
    NUI_SPEECH_EMULATEOPTIONS_SIMULATE_ONCE = 0x20000000
} NUI_SPEECH_EMULATEOPTIONS;
```

Constants

Constant	Description
NUI_SPEECH_EMULATEOPTIONS_NONE	Indicates that no special treatment should be applied to the emulation.
NUI_SPEECH_EMULATEOPTIONS_IGNORECASE	Ignores case in the input string.
NUI_SPEECH_EMULATEOPTIONS_GRAMMAR_CONFUSION	Used to test grammar confusability. When this flag is specified, the speech engine generates multiple feature vectors based on the input text, and runs each vector through the recognizer. If there is confusability in the grammar, the speech engine will recognize multiple confused phrases, as alternate phrases within a single recognition event.
NUI_SPEECH_EMULATEOPTIONS_SIMULATE_ONCE	Used to test grammar confusability. This flag is similar to NUI_SPEECH_EMULATEOPTIONS_GRAMMAR_CONFUSION, which is described previously, except that it generates and tests only a single feature vector.

Remarks

These values are used to specify options when calling [NuiSpeechEmulateRecognition](#). That function treats an input text string as though it is spoken input, and emulates running the

recognition engine against the text string.
These values can be combined using bitwise-OR.

⊕ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_GRAMMARSTATE Enumeration

[See Also](#)



Specifies whether a particular grammar is enabled for speech recognition and is the only grammar used.

⊕ Syntax

C++

```
typedef enum _NUI_SPEECH_GRAMMARSTATE
{
    NUI_SPEECH_GRAMMARSTATE_DISABLED = 0,
    NUI_SPEECH_GRAMMARSTATE_ENABLED = 1,
    NUI_SPEECH_GRAMMARSTATE_EXCLUSIVE = 3
} NUI_SPEECH_GRAMMARSTATE;
```

⊕ Constants

Constant	Description
NUI_SPEECH_GRAMMARSTATE_DISABLED	Disables all top-level rules in this grammar for recognition.
NUI_SPEECH_GRAMMARSTATE_ENABLED	Enables all top-level rules in this grammar for recognition.
NUI_SPEECH_GRAMMARSTATE_EXCLUSIVE	Disables all top-level rules that are not part of this grammar. Setting this grammar state disables all other grammars other than this one.

⊕ Remarks

This enumeration is returned by [NuiSpeechGetGrammarState](#) and is passed to [NuiSpeechSetGrammarState](#).
This enumeration is used to specify which grammars are actively being used by the speech engine during recognition.
Since multiple grammars may be loaded simultaneously, this enumeration enables you to turn on and off specific grammars without unloading them and reloading them.

⊕ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_INTERFERENCE Enumeration

[See Also](#)



Describes why sound was rejected by the speech engine as interference, such as too quiet or too fast.

⊕ Syntax

C++

```
typedef enum _NUI_SPEECH_INTERFERENCE
{
    NUI_SPEECH_INTERFERENCE_NOISE = 1,
    NUI_SPEECH_INTERFERENCE_NOSIGNAL,
    NUI_SPEECH_INTERFERENCE_TOOLOUD,
    NUI_SPEECH_INTERFERENCE_TOOQUIET,
    NUI_SPEECH_INTERFERENCE_TOOFAST,
    NUI_SPEECH_INTERFERENCE_TOOSLOW
} NUI_SPEECH_INTERFERENCE;
```

⊕ Constants

Constant	Description
NUI_SPEECH_INTERFERENCE_NOISE	The sound received is interpreted by the engine as noise.
NUI_SPEECH_INTERFERENCE_NOSIGNAL	The sound received is of constant intensity.
NUI_SPEECH_INTERFERENCE_TOOLOUD	The sound's volume is too high for discrete recognition.
NUI_SPEECH_INTERFERENCE_TOOQUIET	The sound's volume is too low for discrete recognition.

NUI_SPEECH_INTERFERENCE_TOOFAST	The words were spoken too quickly for discrete recognition.
NUI_SPEECH_INTERFERENCE_TOOSLOW	The words were spoken too slowly for discrete recognition, or there was excessive time between words.

Remarks

The [NUI_SPEECH_EVENT](#) structure contains a **NUI_SPEECH_INTERFERENCE** enumeration constant, if the **eventId** of the NUI_SPEECH_EVENT is [NUI_SPEECH_EVENT_INTERFERENCE](#). NUI_SPEECH_EVENT_INTERFERENCE events are generated if there is a [NUI_SPEECH_EVENT_SOUND_START](#) followed by a NUI_SPEECH_EVENT_SOUND_END, without an intervening NUI_SPEECH_EVENT_PHRASE_START.

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI Speech](#)
[NUI_SPEECH_EVENT Identifiers](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_LANGUAGE Enumeration

[See Also](#)



Specifies the language engine, grammar files, and lexicon used by the speech engine, such as United States English or Canadian French.

Syntax

C++

```
typedef enum _NUI_SPEECH_LANGUAGE
{
    NUI_SPEECH_LANGUAGE_EN_US,
    NUI_SPEECH_LANGUAGE_FR_CA,
    NUI_SPEECH_LANGUAGE_EN_GB,
    NUI_SPEECH_LANGUAGE_ES_MX,
    NUI_SPEECH_LANGUAGE_JA_JP,
    NUI_SPEECH_LANGUAGE_FR_FR,
    NUI_SPEECH_LANGUAGE_ES_ES,
    NUI_SPEECH_LANGUAGE_DE_DE,
    NUI_SPEECH_LANGUAGE_IT_IT,
    NUI_SPEECH_LANGUAGE_EN_AU,
    NUI_SPEECH_LANGUAGE_PT_BR,
    NUI_SPEECH_LANGUAGE_DE_AT,
    NUI_SPEECH_LANGUAGE_DE_CH,
    NUI_SPEECH_LANGUAGE_FR_CH
} NUI_SPEECH_LANGUAGE;
```

Constants

Constant	Description
NUI_SPEECH_LANGUAGE_EN_US	en-US (English - United States). Also covers en-CA (English - Canada).
NUI_SPEECH_LANGUAGE_FR_CA	fr-CA (French - Canada).
NUI_SPEECH_LANGUAGE_EN_GB	en-GB (English - United Kingdom). Also covers en-IE (English - Ireland).
NUI_SPEECH_LANGUAGE_ES_MX	es-MX (Spanish - Mexico).
NUI_SPEECH_LANGUAGE_JA_JP	ja-JP (Japanese - Japan).
NUI_SPEECH_LANGUAGE_FR_FR	fr-FR (French - France).
NUI_SPEECH_LANGUAGE_ES_ES	es-ES (Spanish - Spain).
NUI_SPEECH_LANGUAGE_DE_DE	de-DE (German - Germany).
NUI_SPEECH_LANGUAGE_IT_IT	it-IT (Italian - Italy).
NUI_SPEECH_LANGUAGE_EN_AU	en-AU (English - Australia). Also covers en-NZ (English - New Zealand).
NUI_SPEECH_LANGUAGE_PT_BR	pt-BR (Portuguese - Brazil).
NUI_SPEECH_LANGUAGE_DE_AT	de-AT (German - Austria).
NUI_SPEECH_LANGUAGE_DE_CH	de-CH (German - Switzerland).
NUI_SPEECH_LANGUAGE_FR_CH	fr-CH (French - Switzerland).

Remarks

The [NUI_SPEECH_INIT_PROPERTIES](#), [NUI_SPEECH_PHRASE](#), and [NUI_SPEECH_WORD_PRONUNCIATION](#) structures each contain a **NUI_SPEECH_LANGUAGE** enumeration constant. Each enumeration member is the name of an acoustic model for speech recognition. For each acoustic model you specify, you must include the corresponding speech database file in your disc image. See [Kinect Database Files to Include on the Game Disc](#).
Once the language has been set by a call to [NuiSpeechEnable](#), you can't change it. To change the language, you must unload and reinitialize the speech engine. Call [NuiSpeechDisable](#), and then call [NuiSpeechEnable](#) with the new language specified.
For the list of acoustic model databases for language-country combinations, see [Kinect Database Files to Include on the Game Disc](#).
For confidence thresholds for each language-country combination, see [Developing NUI Speech Grammars](#).

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ See Also

[Local Language Support with NUI Speech](#)
[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_LEXICON_TYPE Enumeration

[See Also](#)



Specifies the type of lexicon from which pronunciations were retrieved, such as phonetic input from the player or vendor, or spelling-based pronunciation.

✚ Syntax

C++

```
typedef enum _NUI_SPEECH_LEXICON_TYPE
{
    NUI_SPEECH_LEXICON_TYPE_USER,
    NUI_SPEECH_LEXICON_TYPE_VENDOR,
    NUI_SPEECH_LEXICON_TYPE_LETTERTOSOUND
} NUI_SPEECH_LEXICON_TYPE;
```

✚ Constants

Constant	Description
NUI_SPEECH_LEXICON_TYPE_USER	Pronunciation was provided by the player, using some sort of phonetic input system. For example, a player speaking Japanese might have input the pronunciation using phonetic Katakana.
NUI_SPEECH_LEXICON_TYPE_VENDOR	Pronunciation was found directly from the lexicon included with the speech system.
NUI_SPEECH_LEXICON_TYPE_LETTERTOSOUND	Pronunciation has been synthesized by an algorithm that deduces pronunciation based on spelling.

✚ Remarks

The [NUI_SPEECH_WORD_PRONUNCIATION](#) structure contains a **NUI_SPEECH_LEXICON_TYPE** enumeration constant.

✚ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_LOADOPTIONS Enumeration

[See Also](#)



Specifies whether a speech recognition grammar is loaded statically, or dynamically so that it can be modified at run time.

✚ Syntax

C++

```
typedef enum _NUI_SPEECH_LOADOPTIONS
{
    NUI_SPEECH_LOADOPTIONS_STATIC = 0,
    NUI_SPEECH_LOADOPTIONS_DYNAMIC = 1
} NUI_SPEECH_LOADOPTIONS;
```

✚ Constants

Constant	Description
NUI_SPEECH_LOADOPTIONS_STATIC	The grammar is loaded statically and cannot be modified at run time.
NUI_SPEECH_LOADOPTIONS_DYNAMIC	The grammar is loaded dynamically, and rules can be modified and committed at run time.

✚ Remarks

Loading options determine whether a grammar is unmodifiable at run time (static) or can be modified at run time (dynamic). Loading options must be specified when the grammar is loaded by using [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#).

✚ Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

✚ See Also

[NUI Speech](#)

NUI_SPEECH_MICTYPE Enumeration

[See Also](#)



Specifies whether the speech recognition engine uses the microphone array or stored waveform data as input.

Syntax

```
C++
typedef enum NUI_SPEECH_MICTYPE
{
    NUI_SPEECH_KINECT,
    NUI_SPEECH_WAVE
} NUI_SPEECH_MICTYPE;
```

Constants

Constant	Description
NUI_SPEECH_KINECT	Specifies the Kinect microphone array.
NUI_SPEECH_WAVE	Specifies raw audio data that will be fed directly to the sound input buffers using NuiSpeechProcessWaveInput (for a .wav file) or NuiSpeechProcessWaveInputFromMemory .

Remarks

The [NUI_SPEECH_INIT_PROPERTIES](#) structure contains a **NUI_SPEECH_MICTYPE** enumeration constant.
For the specifications of the wave data, see [NuiSpeechProcessWaveInput](#) (for a .wav file) or [NuiSpeechProcessWaveInputFromMemory](#).

Requirements

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

See Also

[NUI_Speech](#)

NUI_SPEECH_PROPERTY Enumeration

[See Also](#)



Specifies which property of the speech engine is being set or retrieved, such as a confidence threshold, the length of silence before completion, or whether to buffer the audio.

Syntax

```
C++
typedef enum _NUI_SPEECH_PROPERTY
{
    NUI_SPEECH_PROPERTY_MAXIMUM_ALTERNATES,
    NUI_SPEECH_PROPERTY_HIGH_CONFIDENCE_THRESHOLD,
    NUI_SPEECH_PROPERTY_NORMAL_CONFIDENCE_THRESHOLD,
    NUI_SPEECH_PROPERTY_LOW_CONFIDENCE_THRESHOLD,
    NUI_SPEECH_PROPERTY_RESPONSE_SPEED,
    NUI_SPEECH_PROPERTY_COMPLEX_RESPONSE_SPEED,
    NUI_SPEECH_PROPERTY_RETAIN_AUDIO,
    NUI_SPEECH_PROPERTY_DISABLE_HYPOTHESES,
    NUI_SPEECH_PROPERTY_CFGCONFIDENCE_REJECTION_THRESHOLD
} _NUI_SPEECH_PROPERTY;
```

Constants

Constant	Description
NUI_SPEECH_PROPERTY_MAXIMUM_ALTERNATES	Specifies the maximum number of alternate phrases returned in the recognition result, in the pAlternates member of the NUI_SPEECH_RECORESULT structure. The default value is 0. To receive alternates, set this property to a non-zero, positive value.
NUI_SPEECH_PROPERTY_HIGH_CONFIDENCE_THRESHOLD	Specifies the value that separates the ranges of high and normal confidence. The default value is 80.
NUI_SPEECH_PROPERTY_NORMAL_CONFIDENCE_THRESHOLD	Specifies the value that separates the ranges of normal and low confidence. The default value is 50.
NUI_SPEECH_PROPERTY_LOW_CONFIDENCE_THRESHOLD	Specifies the value that separates the ranges of low and rejected confidence. The default value is 20.
NUI_SPEECH_PROPERTY_RESPONSE_SPEED	Indicates the amount of silence that the speech engine seeks before completing a recognition. This attribute is used when the recognition is <i>not</i> ambiguous. For example, in the case when a grammar has two sentences—"new game please" and "new game"—a <i>non</i> -ambiguous recognition would be "new game please". The valid range for this property is 100 - 10000 milliseconds. The default is 150 milliseconds. The engine automatically clamps to this value.
NUI_SPEECH_PROPERTY_COMPLEX_RESPONSE_SPEED	Indicates the amount of silence that the speech engine seeks before completing an <i>ambiguous</i> recognition. For example, in the case when the grammar has two sentences—"new game please" and "new game"—an ambiguous recognition would be "new game". The valid range for this property is NUI_SPEECH_PROPERTY_RESPONSE_SPEED - 15000 milliseconds. The default is 500 milliseconds. The engine automatically clamps to this value.
NUI_SPEECH_PROPERTY_RETAIN_AUDIO	Indicates that the engine should buffer the audio used in recognition. Valid values are TRUE or FALSE. The

	default value is FALSE.
NUI_SPEECH_PROPERTY_DISABLE_HYPOTHESES	Indicates that the engine should not send NUI_SPEECH_EVENT_HYPOTHESIS events. Valid values are TRUE or FALSE. The default value is TRUE.
NUI_SPEECH_PROPERTY_CFGCONFIDENCE_REJECTION_THRESHOLD	The engine will only send recognition events for recognitions with confidence values greater than this number. The default value is 0.

⊕ **Remarks**

These enumeration constants are used to designate which property to retrieve by calling [NuiSpeechGetProperty](#), or to set by calling [NuiSpeechSetProperty](#).

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RULEOPTIONS Enumeration

[See Also](#)



Describes how a rule is used, such as whether it is a top-level rule, whether the rule is imported from another grammar, and whether the rule can be modified dynamically at run time.

⊕ **Syntax**

C++

```
typedef enum _NUI_SPEECH_RULEOPTIONS
{
    NUI_SPEECH_RULEOPTIONS_TOPLEVEL = 0x1,
    NUI_SPEECH_RULEOPTIONS_ACTIVE = 0x2,
    NUI_SPEECH_RULEOPTIONS_IMPORT = 0x8,
    NUI_SPEECH_RULEOPTIONS_DYNAMIC = 0x20,
    NUI_SPEECH_RULEOPTIONS_ROOT = 0x40
} NUI_SPEECH_RULEOPTIONS;
```

⊕ **Constants**

Constant	Description
NUI_SPEECH_RULEOPTIONS_TOPLEVEL	Indicates this rule is a top-level rule, which means it is an entry point into the grammar.
NUI_SPEECH_RULEOPTIONS_ACTIVE	Specifies that this is a top-level rule that is active by default. When a rule is created in a dynamic grammar, and after committing the grammar, the top-level rules created as default active are not initially active. They must be explicitly set as active. The NUI_SPEECH_RULEOPTIONS_ACTIVE flag facilitates activating all default active top-level rules, rather than activating each rule individually, by calling NuiSpeechSetRuleState after committing the new rules, as follows: NuiSpeechSetRuleState(pGrammar, NULL, NUI_SPEECH_RULESTATE_ACTIVE)
NUI_SPEECH_RULEOPTIONS_IMPORT	This rule is imported from another grammar and, therefore, is not defined in this grammar.
NUI_SPEECH_RULEOPTIONS_DYNAMIC	This rule can be modified or has been created dynamically. The grammar must have been loaded dynamically (specified by NUI_SPEECH_LOADOPTIONS_DYNAMIC) to enable changes at run time.
NUI_SPEECH_RULEOPTIONS_ROOT	The root rule for the grammar.

⊕ **Remarks**

A combination of NUI_SPEECH_RULEOPTIONS enumeration constants is passed to [NuiSpeechCreateRule](#).
Use these values when dynamically building a grammar and when querying a grammar's existing rules.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.
Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RULESTATE Enumeration

[See Also](#)



Specifies whether a rule is active (enabled).

⊕ **Syntax**

C++

```
typedef enum _NUI_SPEECH_RULESTATE
{
    NUI_SPEECH_RULESTATE_INACTIVE = 0,
    NUI_SPEECH_RULESTATE_ACTIVE = 1
}
```

```
} NUI_SPEECH_RULESTATE;
```

⊕ **Constants**

Constant	Description
NUI_SPEECH_RULESTATE_INACTIVE	Disable this rule for recognition.
NUI_SPEECH_RULESTATE_ACTIVE	Enable this rule for recognition.

⊕ **Remarks**

[NuiSpeechSetRuleState](#) takes one of these enumeration constants as input.

You can set or retrieve the rule state from within individual rules in a grammar. Manipulating the rules from within a grammar offers a finer level of control over which rules are used for recognition, compared to **NuiSpeechSetRuleState**.

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_WORDTYPE Enumeration

[See Also](#)



Indicates whether a word is in lexical form and whether special characters are ignored for word transitions, when adding a word to a grammar.

⊕ **Syntax**

C++

```
typedef enum _NUI_SPEECH_WORDTYPE
{
    NUI_SPEECH_WORDTYPE_LEXICAL = 1,
    NUI_SPEECH_WORDTYPE_LEXICAL_NO_SPECIAL_CHARS = 3
} NUI_SPEECH_WORDTYPE;
```

⊕ **Constants**

Constant	Description
NUI_SPEECH_WORDTYPE_LEXICAL	Indicates the word is in lexical form and can be used to access the lexicon.
NUI_SPEECH_WORDTYPE_LEXICAL_NO_SPECIAL_CHARS	Deactivates handling of special characters in the grammar. For example, an asterisk ("*") is normally interpreted to mean dictation, but this flag causes "*" to be interpreted simply as an asterisk.

⊕ **Remarks**

This enumeration is used to describe word types when adding a word to a grammar by calling [NuiSpeechAddWordTransition](#).

⊕ **Requirements**

Header: Declared in NuiSpeech.h. For include, use NuiApi.h.

Library: Use NuiSpeech.lib and NuiAudio.lib.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiSpeech Constants

[See Also](#)



In This Section

[NUI_SPEECH_EVENT Identifiers](#)

Identifiers for speech engine events, such as phrase started, phrase recognized, or bookmark reached.

[NUI_SPEECH_RULE_TRANSITION_WILDCARD](#)

Specifies that the initial state for a new rule is to match any spoken words.

⊕ **See Also**

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_EVENT Identifiers

[See Also](#)

Identifiers for speech engine events, such as phrase started, phrase recognized, or bookmark reached.

Name	Description
NUI_SPEECH_EVENT_START_STREAM	Raised when the speech engine receives a new stream input with audio data, or when NuiSpeechStartRecognition is called.
NUI_SPEECH_EVENT_END_STREAM	Raised when the input stream is finished because there are no more audio bytes being supplied by the microphone, or after NuiSpeechStopRecognition is called.
NUI_SPEECH_EVENT_SOUND_START	Raised when the speech engine encounters the beginning of a sound.
NUI_SPEECH_EVENT_SOUND_END	Raised when audible sound is no longer available through the input stream, or the sound stream has been inactive.
NUI_SPEECH_EVENT_PHRASE_START	Raised when the engine begins recognizing a phrase. This event must be followed by a NUI_SPEECH_EVENT_RECOGNITION or NUI_SPEECH_EVENT_FALSE_RECOGNITION event.
NUI_SPEECH_EVENT_HYPOTHESIS	Raised when the engine has partially recognized the input stream. The event returned gives the engine's best guess of the phrase uttered up to that point in the stream.
NUI_SPEECH_EVENT_RECOGNITION	Raised when the engine has recognized the full phrase of the audio data.
NUI_SPEECH_EVENT_FALSE_RECOGNITION	Raised when the engine has no valid recognition.
NUI_SPEECH_EVENT_INTERFERENCE	Raised when the soundstream has a hindrance that prevents successful recognition. See also the NUI_SPEECH_INTERFERENCE enumeration.
NUI_SPEECH_EVENT_BOOKMARK	Raised when the engine reached the stream position specified by the client's bookmark.
NUI_SPEECH_EVENT_RETAINEDAUDIO	Returns the audio data that was sent to the recognizer. This event will only be raised if NuiSpeechSetProperty is called, setting the property NUI_SPEECH_PROPERTY_RETAIN_AUDIO to TRUE. See also the NUI_SPEECH_RETAINEDAUDIO structure.
NUI_SPEECH_ALL_RECOGNITION_EVENTS	A bitmask that identifies only recognition events, which are the following: • NUI_SPEECH_EVENT_HYPOTHESIS • NUI_SPEECH_EVENT_RECOGNITION • NUI_SPEECH_EVENT_FALSE_RECOGNITION
NUI_SPEECH_ALL_EVENTS	A bitmask that identifies all events.

+

Remarks

A [NUI_SPEECH_EVENT](#) structure contains one of these event identifiers. [NuiSpeechSetEventInterest](#) takes a combination of these event identifiers. When constructing event masks for multiple events, combine these event identifiers using bitwise-OR.

+

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_SPEECH_RULE_TRANSITION_WILDCARD

[See Also](#)

Specifies that the initial state for a new rule is to match any spoken words.

Name	Description
NUI_SPEECH_RULE_TRANSITION_WILDCARD	Specifies that the initial state for a new rule is to match any spoken words.

+

Remarks

This value can be used for dynamic grammars with the [NuiSpeechAddRuleTransition](#) function, to match any spoken words.

+

See Also

[NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiTools Reference

[See Also](#)

This section contains the API Reference pages for items in the header file NuiTools.h. This header file contains declarations for selected data types used by the Natural User Interface API. This header is suitable for use in Win32-based content management tools. Use NuiTools.h to build tools that manipulate data that was originally captured on a Xbox 360 console with an attached Kinect sensor array.

[NUI_CAMERA_COLOR_NOMINAL Values](#)
[NUI_CAMERA_DEPTH_NOMINAL Values](#)
[NUI_IDENTITY_MESSAGE](#)
[NUI_IDENTITY_MESSAGE_COMPLETE](#)
[NUI_IDENTITY_MESSAGE_FRAME_PROCESSED](#)
[NUI_IDENTITY_MESSAGE_ID](#)
[NUI_IDENTITY_OPERATION_ID](#)
[NUI_IDENTITY_QUALITY_ENVIRONMENT_MASK](#)
[NUI_IDENTITY_QUALITY_Flags](#)
[NUI_IDENTITY_QUALITY_USER_MASK](#)
[NUI_IMAGE_COLOR_640x480_BUFFER_SIZE](#)
[NUI_IMAGE_COLOR_YUV_640x480_BUFFER_SIZE](#)
[NUI_IMAGE_DEPTH_80x60_BUFFER_SIZE](#)
[NUI_IMAGE_DEPTH_BUFFER_SIZE](#)

[NUI_IMAGE_DEPTH Values](#)
[NUI_IMAGE_DIGITALZOOM](#)
[NUI_IMAGE_PLAYER_INDEX_MASK](#)
[NUI_IMAGE_PLAYER_INDEX_SHIFT](#)
[NUI_IMAGE_RESOLUTION](#)
[NUI_IMAGE_TYPE](#)
[NUI_IMAGE_VIEW_AREA](#)
[NUI_SKELETON_COUNT](#)
[NUI_SKELETON_DATA](#)
[NUI_SKELETON_FRAME](#)
[NUI_SKELETON_INVALID_TRACKING_ID](#)
[NUI_SKELETON_MAX_TRACKED_COUNT](#)
[NUI_SKELETON_POSITION_INDEX](#)
[NUI_SKELETON_POSITION_TRACKING_STATE](#)
[NUI_SKELETON_QUALITY Flags](#)
[NUI_SKELETON_TRACKING_STATE](#)

See Also

[NUI Overview](#)
[NUI Development Tools](#)
[NUI Samples](#)
[NUI Tools](#)
[Natural User Interface White Papers](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWave Reference

[See Also](#)



This section contains the API Reference pages for items in the header file NuiWave.h, which enables detecting the wave gesture, which is also used in the Xbox Dashboard and Guide.

In This Section

[NuiWave Functions](#)
[NuiWave Structures](#)

See Also

[NUI Wave Gesture Detection](#)
[Wave Gesture Tracking Sample](#)
[Adding NUI Header and Library Files](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWave Functions

[See Also](#)



In This Section

[NuiWaveGetGestureOwnerProgress](#)

Gets the ID of the player who NuiWave is tracking as the owner of the wave gesture, and, using the default policy for wave gesture recognition, gets the progress of that player's wave gesture recognition.

[NuiWaveGetState](#)

Gets the state of wave gesture recognition, which you can use when you define a custom policy for wave recognition.

[NuiWaveSetEnabled](#)

Enables or disables wave gesture recognition.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWaveGetGestureOwnerProgress

[See Also](#) [Example](#)



Gets the ID of the player who NuiWave is tracking as the owner of the wave gesture, and, using the default policy for wave gesture recognition, gets the progress of that player's wave gesture recognition.

Syntax

C++

```
HRESULT NuiWaveGetGestureOwnerProgress (
    PDWORD pdwTrackingID,
    PFLOAT pfWaveProgress
)
```

Parameters

pdwTrackingID

[out] The skeleton tracking ID of the player who waved or is waving, as a pointer to a DWORD.

This pointer must be non-NULL when you call the function. After the function call returns, the DWORD contains the skeleton tracking ID. If this function does not return S_OK, this data is invalid.

For more information about tracking IDs, refer to the **dwTrackingID** member of the [NUI_SKELETON_DATA](#) structure.

pfWaveProgress

[out] The progress of wave gesture recognition for the player who waved or is waving, as a pointer to a FLOAT. A value of 0.0 means no wave gesture is in progress currently. A value of 1.0 or greater means the entire wave gesture has completed.

A complete wave gesture is defined as a sequence of arm waves. When the first arm wave is completed, the progress returned will change from 0 to a number greater than 0 and less than 1.0. When the required number of arm waves are completed, the progress returned will be 1.0. In the next frame, the progress returned will be 0.0. This value takes on a series of fixed discrete values, not any continuous value between 0.0 and 1.0. *pfWaveProgress* is incremented to these fixed proportional values when each arm wave is completed.

This pointer must be non-NULL when you call the function. After the function call returns, the FLOAT contains the progress. If this function does not return S_OK, this data is invalid.

Return Value

One of the following values:

Value	Description
S_OK	The function completed successfully.
E_INVALIDARG	A pointer parameter is NULL.

Remarks

This function provides the simplest approach to detecting a wave gesture. It uses the default policy to determine who is returned as the waving player. The default policy is that if more than one player is waving, this method returns the ID of the player who is furthest along in wave gesture progress. If two players are tied in their wave gesture progress, one of the players will be returned, but it is undefined which player will be returned. To implement a different policy for deciding the owner of the wave gesture, use [NuiWaveGetState](#) instead.

You can use this method during a wave gesture, or to detect whether a wave gesture was completed.

Example

This function is used in the [WaveGestureTracking](#) sample.

Requirements

Header: Declared in NuiWave.h. For include, use NuiApi.h.

Library: Use nuiapi.lib.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWaveGetState

[See Also](#)



Gets the state of wave gesture recognition, which you can use when you define a custom policy for wave recognition.

Syntax

C++

```
HRESULT NuiWaveGetState(  
    PNUI_WAVE_STATE pState  
)
```

Parameters

pState

[out] The state of wave gesture recognition, as a pointer to a [NUI_WAVE_STATE](#) structure.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_INVALIDARG	A pointer parameter is NULL.
E_FAIL	Wave gesture recognition is not enabled (by NuiWaveSetEnabled).
E_NUI_FRAME_NO_DATA	Skeleton tracking is not in use.

Remarks

This function lets you define a custom policy for wave detection. If you want to use the default policy to determine who is returned as the waving player, use [NuiWaveGetGestureOwnerProgress](#) instead.

Requirements

Header: Declared in NuiWave.h. For include, use NuiApi.h.

Library: Use nuiapi.lib.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWaveSetEnabled

[See Also](#) [Example](#)



Enables or disables wave gesture recognition.

Syntax

C++

```
HRESULT NuiWaveSetEnabled(  
    BOOL bEnabled  
)
```

Parameters

bEnabled
[in] TRUE enables wave gesture recognition. FALSE disables wave gesture recognition.

Return Value

One of the following values:

Value	Meaning
S_OK	The function completed successfully.
E_FAIL	Skeleton tracking is not in use.

Remarks

You should use this function only after you call [NuiInitialize](#) with the flag [NUI_INITIALIZE_FLAG_USES_SKELETON](#). This enables skeleton tracking.

Example

This function is used in the following samples:

- [WaveGestureTracking](#)
- [SimpleIdentity](#)
- [SimpleIdentityUsingCallbacks](#)

Requirements

Header: Declared in NuiWave.h. For include, use NuiApi.h.

Library: Use nuiapi.lib.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiWave Structures

[See Also](#)

In This Section

[NUI_WAVE_PLAYER_STATE](#)

Stores the state of wave gesture recognition for a player.

[NUI_WAVE_STATE](#)

Stores the state of wave gesture recognition for all players.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_WAVE_PLAYER_STATE Structure

[See Also](#)

Stores the state of wave gesture recognition for a player.

Syntax

C++

```
typedef struct _NUI_WAVE_PLAYER_STATE {  
    FLOAT fLeftHandWaveProgress;  
    BOOL bLeftHandLastMotionToLeft;  
    FLOAT fRightHandWaveProgress;  
    BOOL bRightHandLastMotionToLeft;  
    DWORD dwTrackingID;  
} NUI_WAVE_PLAYER_STATE, *PNUI_WAVE_PLAYER_STATE;
```

Members

fLeftHandWaveProgress

The progress of wave gesture recognition using the left hand. A value of 0.0 means no wave gesture is in progress currently. A value of 1.0 or greater means the entire wave gesture has completed.

A complete wave gesture is defined as a sequence of arm waves. When the first arm wave is completed, the progress returned will change from 0 to a number between 0 and 1.0. When the required number of arm waves are completed, the progress returned will be 1.0 or greater. In the next frame, the progress returned will be 0.0. This value takes on a series of fixed discrete values, not any continuous value between 0.0 and 1.0. **fLeftHandWaveProgress** and **fRightHandWaveProgress** are incremented to these fixed proportional values when each arm wave is completed.

bLeftHandLastMotionToLeft

Whether the last motion of the left hand was a sweep to the left.

fRightHandWaveProgress
The progress of wave gesture recognition using the right hand. Similar to **fLeftHandWaveProgress**.

bRightHandLastMotionToLeft
Whether the last motion of the right hand was a sweep to the left.

dwTrackingID
The skeleton tracking ID of the player.

For more information about tracking IDs, refer to the **dwTrackingID** member of the [NUI_SKELETON_DATA](#) structure.

Remarks

A [NUI_WAVE_STATE](#) structure contains multiple **NUI_WAVE_PLAYER_STATE** structures.

Requirements

Header: Declared in NuiWave.h. For include, use NuiApi.h.
Library: Use nuiapi.lib.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI_WAVE_STATE Structure

[See Also](#)

Stores the state of wave gesture recognition for all players.

Syntax

C++

```
typedef struct _NUI_WAVE_STATE {  
    NUI_WAVE_PLAYER_STATE aPlayers[NUI_SKELETON_MAX_TRACKED_COUNT];  
} NUI_WAVE_STATE, *PNUI_WAVE_STATE;
```

Members

aPlayers
The state of wave gesture recognition for each player, as an array of [NUI_WAVE_PLAYER_STATE](#) structures.
The array contains up to [NUI_SKELETON_MAX_TRACKED_COUNT](#) items.

Remarks

[NuiWaveGetState](#) returns a **NUI_WAVE_STATE** structure.

Requirements

Header: Declared in NuiWave.h. For include, use NuiApi.h.
Library: Use nuiapi.lib.

See Also

[NUI Wave Gesture Detection](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedFile Reference

[See Also](#)

This section contains API Reference pages for the XedFile API, which operates on the Xbox event data (XED) files created by the [Xbox Studio](#) tool. This API can read color, depth, and skeletal frames from existing XED files. XedFile can also create new XED files and write color, depth, and skeletal frames to them.
For more information about using the XedFile API, see [Recorded Data Sample](#).

In This Section

- [XedFile Functions](#)
Contains reference pages for the functions that are defined in the header file XedFile.h.
- [XedFile Structures](#)
Contains the reference pages for the structures that are defined in the header file XedFile.h.
- [XedFile Constants](#)
Contains the reference pages for the constants that are defined in the header file XedFile.h.

See Also

[Natural User Interface Reference](#)
[Recorded Data Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedFile Functions

[See Also](#)

	
This topic contains the reference pages for the functions that are defined in the header file XedFile.h.	
In This Section	
XedCloseFile	Closes an open Xbox event data (XED) file.
XedCopyNuiColorFrame	Copies the specified color frame from an Xbox event data (XED) file to another XED file.
XedCopyNuiDepthFrame	Copies the specified depth frame from an Xbox event data (XED) file to another XED file.
XedCopyNuiIdentityMessage	Copies the specified identity message from an Xbox event data (XED) file to another XED file.
XedCopyNuiSkeletonFrame	Copies the specified skeletal frame from an Xbox event data (XED) file to another XED file.
XedCopyTitleData	Copies the specified title event from an Xbox event data (XED) file to another XED file.
XedCreateFile	Creates a new Xbox event data (XED) file for writing.
XedGetEventCount	Retrieves the number of events in the specified stream of the specified Xbox event data (XED) file.
XedGetEventIndexFromMicroseconds	Gets the index of the event with the specified timestamp in the specified stream of an Xbox event data (XED) file.
XedGetNuiColorEventCount	Retrieves the number of color events in the color stream of the specified Xbox event data (XED) file.
XedGetNuiColorEventIndexFromFrameNumber	Retrieves the index of the color event with the specified frame number in an Xbox event data (XED) file.
XedGetNuiColorEventIndexFromMicroseconds	Gets the index of the color event with the specified timestamp in an Xbox Event Data (XED) file.
XedGetNuiDepthEventCount	Retrieves the number of depth events in the depth stream of the specified Xbox event data (XED) file.
XedGetNuiDepthEventIndexFromFrameNumber	Retrieves the index of the depth event with the specified frame number in an Xbox event data (XED) file.
XedGetNuiDepthEventIndexFromMicroseconds	Gets the index of the depth event with the specified timestamp in an Xbox Event Data (XED) file.
XedGetNuiSkeletonEventCount	Retrieves the number of skeletal events in the skeletal stream of the specified Xbox event data (XED) file.
XedGetNuiSkeletonEventIndexFromFrameNumber	Retrieves the index of the skeletal event with the specified frame number in an Xbox event data (XED) file.
XedGetNuiSkeletonEventIndexFromMicroseconds	Gets the index of the skeletal event with the specified timestamp in an Xbox Event Data (XED) file.
XedGetTitleDataEventCount	Retrieves the number of title data events in the title data stream of the specified Xbox event data (XED) file.
XedGetTitleDataEventIndexFromMicroseconds	Gets the index of the title data event with the specified timestamp in an Xbox Event Data (XED) file.
XedOpenFile	Opens an Xbox event data (XED) file for reading.
XedReadNuiColorFrame	Reads the specified color frame from an Xbox event data (XED) file.
XedReadNuiDepthFrame	Reads the specified depth frame from an Xbox event data (XED) file.
XedReadNuiIdentityMessage	Reads the specified identity message from an Xbox event data (XED) file.
XedReadNuiSkeletonFrame	Reads the specified skeletal frame from an Xbox event data (XED) file.
XedReadTitleData	Reads the specified title event from an Xbox event data (XED) file.
XedSetFileNuiColorQuality	Sets the color quality to use when writing color frames to an Xbox event data (XED) file.
XedSetNuiColorBufferType	Sets whether to use the sRGB or YUV color space when writing color frames to an Xbox Event Data (XED) file, or when reading color frames from an XED file.
XedSetNuiDepthImageType	Sets whether to include the player segmentation index when reading or writing depth frames for an Xbox Event Data (XED) file.
XedWriteNuiColorFrame	Writes the specified color frame to an Xbox event data (XED) file.
XedWriteNuiDepthFrame	Writes the specified depth frame to an Xbox event data (XED) file.
XedWriteNuiIdentityMessage	Writes the specified identity message to an Xbox event data (XED) file.
XedWriteNuiSkeletonFrame	Writes the specified skeletal frame to an Xbox event data (XED) file.
XedWriteTitleData	Writes the specified title event to an Xbox event data (XED) file.
See Also	
XedFile Reference	
© 2013 Microsoft Corporation. All rights reserved. Send feedback to https://forums.xboxlive.com/. Version: 21256.3	
XedCloseFile	
See Also	
	

Closes an open Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedCloseFile(  
    XED_CONTEXT **ppContext  
)
```

Parameters

ppContext
[in] Pointer to the address of the [XED_CONTEXT](#) of the XED file to close. After the file has been closed, this pointer will be set to NULL to revoke the context from the caller.

Return Value

Returns S_OK if successful; otherwise, an error code.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedCopyNuiColorFrame

[See Also](#)

Copies the specified color frame from an Xbox event data (XED) file to another XED file.

Syntax

C++

```
HRESULT XedCopyNuiColorFrame(  
    XED_CONTEXT *pFromContext,  
    UINT eventIndex,  
    XED_CONTEXT *pToContext,  
    UINT frameNumber,  
    UINT64 frameMicroseconds  
)
```

Parameters

pFromContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to copy the frame. This file must have been opened by a call to the [XedOpenFile](#) function.
eventIndex
[in] Index of the event to copy.
pToContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to copy the frame. This file must have been opened by a call to the [XedCreateFile](#) function.
frameNumber
[in] Frame number to set for the frame in the destination XED file. Pass [XED_FRAMENUMBER_INVALID](#) to this parameter to copy the frame number from the source XED file.
frameMicroseconds
[in] Timestamp to set for the frame in the destination XED file. Pass [XED_TIMESTAMP_INVALID](#) to this parameter to copy the timestamp from the source XED file.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

This function is faster than reading a frame from the source XED file, and then writing it to the destination XED file.

Important

This function copies a color frame between two XED files without loss of fidelity. Use this function whenever you copy a color frame between two XED files. When you call the [XedReadNuiColorFrame](#) function to read a color frame from an XED file, its color space is converted to RGB. When you call the [XedWriteNuiColorFrame](#) function to write the color frame to a new file, the frame is converted back to its original color space. The color space conversion uses a lossy algorithm. Multiple read/write iterations will degrade the quality of the color frame.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedCopyNuiDepthFrame

[See Also](#)



Copies the specified depth frame from an Xbox event data (XED) file to another XED file.

+

Syntax

C++

```
HRESULT XedCopyNuiDepthFrame(  
    XED_CONTEXT *pFromContext,  
    UINT eventIndex,  
    XED_CONTEXT *pToContext,  
    UINT frameNumber,  
    UINT64 frameMicroseconds  
)
```

Parameters

pFromContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to copy the frame. This file must have been opened by a call to the [XedOpenFile](#) function.

eventIndex
[in] Index of the event to copy.

pToContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to copy the frame. This file must have been opened by a call to the [XedCreateFile](#) function.

frameNumber
[in] Frame number to set for the frame in the destination XED file. Pass [XED_FRAMENUMBER_INVALID](#) to this parameter to copy the frame number from the source XED file.

frameMicroseconds
[in] Timestamp to set for the frame in the destination XED file. Pass [XED_TIMESTAMP_INVALID](#) to this parameter to copy the timestamp from the source XED file.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

This function is faster than reading a frame from the source XED file, and then writing it to the destination XED file.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

[XedCopyNuiIdentityMessage](#)

[See Also](#)



Copies the specified identity message from an Xbox event data (XED) file to another XED file.

+

Syntax

C++

```
HRESULT XedCopyNuiIdentityMessage(  
    XED_CONTEXT *pSrcContext,  
    UINT srcEventIndex,  
    XED_CONTEXT *pDstContext,  
    DWORD dwDstTrackingID,  
    UINT64 dstEventMicroseconds  
)
```

Parameters

pSrcContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to copy the identity message. This file must have been opened by a call to the [XedOpenFile](#) function.

srcEventIndex
[in] Index of the event to copy.

pDstContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to copy the frame. This file must have been opened by a call to the [XedCreateFile](#) function.

dwDstTrackingID
[in] Skeleton tracking identifier to write to the destination XED file. Pass zero to this parameter to copy the tracking identifier from the source XED file.

dstEventMicroseconds
[in] Timestamp to set for the frame in the destination XED file. Pass [XED_TIMESTAMP_INVALID](#) to this parameter to copy the timestamp from the source XED file.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>dstEventMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

This function is faster than reading a frame from the source XED file, and then writing it to the destination XED file.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedCopyNuiSkeletonFrame

[See Also](#)

Copies the specified skeletal frame from an Xbox event data (XED) file to another XED file.

Syntax

```
C++

HRESULT XedCopyNuiSkeletonFrame (
    XED_CONTEXT *pFromContext,
    UINT eventIndex,
    XED_CONTEXT *pToContext,
    UINT frameNumber,
    UINT64 frameMicroseconds
)
```

Parameters

pFromContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to copy the frame. This file must have been opened by a call to the [XedOpenFile](#) function.

eventIndex
[in] Index of the event to copy.

pToContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to copy the frame. This file must have been opened by a call to the [XedCreateFile](#) function.

frameNumber
[in] Frame number to set for the frame in the destination XED file. Pass [XED_FRAMENUMBER_INVALID](#) to this parameter to copy the frame number from the source XED file.

frameMicroseconds
[in] Timestamp to set for the frame in the destination XED file. Pass [XED_TIMESTAMP_INVALID](#) to this parameter to copy the timestamp from the source XED file.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

This function is faster than reading a frame from the source XED file, and then writing it to the destination XED file.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedCopyTitleData

[See Also](#)

Copies the specified title event from an Xbox event data (XED) file to another XED file.

Syntax

```
C++

HRESULT XedCopyTitleData (
    XED_CONTEXT *pFromContext,
    UINT eventIndex,
    XED_CONTEXT *pToContext,
    UINT64 eventMicroseconds
)
```

Parameters

pFromContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to copy the event. This file must have been opened by a call to the [XedOpenFile](#) function.

eventIndex

[in] Index of the event to copy.

pToContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to copy the event. This file must have been opened by a call to the [XedCreateFile](#) function.

eventMicroseconds
[in] Timestamp to set for the event in the destination XED file. Pass [XED_TIMESTAMP_INVALID](#) to this parameter to copy the timestamp from the source XED file.

Return Value
Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>eventMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

- +

Remarks
- This function is faster than reading an event from the source XED file, and then writing it to the destination XED file.

- +

Requirements
- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

- +

See Also
- [XedFile Functions](#)
[Title Data \[Xbox Studio\]](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedCreateFile

[See Also](#)

Creates a new Xbox event data (XED) file for writing.

- +

Syntax

C++

```
HRESULT XedCreateFile(  
    const char *sFilePath,  
    const XED_CONTEXT *pTemplate,  
    DWORD flags,  
    XED_CONTEXT **ppContext  
)
```

Parameters

sFilePath
[in] Reference to the path and file name of the XED file to create.

pTemplate
[in, optional] Reference to the file context to use as a template. If specified, this must be an XED file that has been opened through a call to the [XedOpenFile](#) function. The newly created XED file will be configured for the same model of the Kinect sensor array as the template. If you pass NULL to this parameter, you cannot register the color and depth frames when you perform read operations on the file. This is because the color and depth data frames are expected to be already aligned. For more information about registration, see [XedReadNuiColorFrame](#).

flags
[in] Specifies the action to take when you create the specified XED file. This must be one of the values listed in the following table.

Value	Description
XED_FLAG_CREATE_NEW	Creates a new XED file. This function will fail if the specified XED file already exists.
XED_FLAG_CREATE_ALWAYS	Creates a new XED file. If the specified XED file already exists, this function will overwrite it.

ppContext
[out, optional] Pointer that will receive the address of the [XED_CONTEXT](#) of the newly created XED file.

Return Value
Returns S_OK if successful; otherwise, an error code.

- +

Remarks
- You can combine color and depth streams from files that were produced using different models of the Kinect sensor array. When you perform read operations on the streams, you must register the streams before you combine them. When you write the combined streams to a file, you must pass NULL to the *pTemplate* parameter because the streams have already been registered. For more information about registration, see [XedReadNuiColorFrame](#).

 **Note**

You cannot unregister a stream.

- +

Requirements
- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

- +

See Also
- [XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetEventCount

[See Also](#)



Retrieves the number of events in the specified stream of the specified Xbox event data (XED) file.

+

Syntax

C++

```
UINT XedGetEventCount (
    XED_CONTEXT *pContext,
    XSTUDIO_STREAM_ID streamId
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

streamId
[in] [XSTUDIO_STREAM_ID](#) value that specifies the stream to get the event count for.

Return Value

Number of events in the specified stream of the specified XED file.

+

Remarks

If a stream has *n* events, the event indices will always be incremented from zero to *n*–1. Event indices do not directly correspond to frame numbers, as some frames may have been dropped.

+

Requirements

Header:

Declared in XedFile.h.

Library:

Use XedFile.lib.

+

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetEventIndexFromMicroseconds

[See Also](#)



Gets the index of the event with the specified timestamp in the specified stream of an Xbox event data (XED) file.

+

Syntax

C++

```
UINT XedGetEventIndexFromMicroseconds (
    XED_CONTEXT *pContext,
    XSTUDIO_STREAM_ID streamId,
    UINT64 eventMicroseconds
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

streamId
[in] [XSTUDIO_STREAM_ID](#) value that specifies the stream to get the event index for.

eventMicroseconds
[in] Timestamp of the event to return the index of, in microseconds. The first event in the file is likely to have a timestamp substantially greater than zero.

Return Value

Returns the index of the event in the specified stream with the specified timestamp. If this function cannot find an event with the specified timestamp, it returns the index of the event that immediately precedes the timestamp. This function returns [XED_EVENTINDEX_INVALID](#) if the specified timestamp precedes the first event in the stream.

+

Requirements

Header:

Declared in XedFile.h.

Library:

Use XedFile.lib.

+

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiColorEventCount

[See Also](#)



Retrieves the number of color events in the color stream of the specified Xbox event data (XED) file.

+

Syntax

C++

```
UINT XedGetNuiColorEventCount(  
    XED_CONTEXT *pContext  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

Return Value

Number of color events in the color stream of the specified XED file.

Remarks

If a stream has *n* events, the event indices will always be incremented from zero to *n*−1. Event indices do not directly correspond to frame numbers, as some frames may have been dropped.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiColorEventIndexFromFrameNumber

[See Also](#)



Retrieves the index of the color event with the specified frame number in an Xbox event data (XED) file.

Syntax

C++

```
UINT XedGetNuiColorEventIndexFromFrameNumber(  
    XED_CONTEXT *pContext,  
    UINT frameNumber  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

frameNumber
[in] Frame number of the color event to return the index of.

Return Value

Returns the index of the color event, or [XED_EVENTINDEX_INVALID](#) if this function cannot find a color event with the specified frame number.

Remarks

Frame numbers of color events are independent of frame numbers of skeletal and depth events.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiColorEventIndexFromMicroseconds

[See Also](#)



Gets the index of the color event with the specified timestamp in an Xbox Event Data (XED) file.

Syntax

C++

```
UINT XedGetNuiColorEventIndexFromMicroseconds(  
    XED_CONTEXT *pContext,  
    UINT64 frameMicroseconds  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

frameMicroseconds
[in] Timestamp of the color event of which to return the index, in microseconds. The first event in the file is likely to have a timestamp substantially greater than zero.

Return Value

Returns the index of the color event with the specified timestamp. If this function cannot find a color event with the specified timestamp, it returns the index of the color event that

immediately precedes the timestamp. This function returns [XED_EVENTINDEX_INVALID](#) if the specified timestamp precedes the first event in the stream.

Remarks

This function does not enforce a gap constraint between events. If multiple events are dropped, the time between events can be greater than expected. Pass NULL to the *pBuffer* parameter of the [XedReadNuiColorFrame](#) function to get the timestamp of an event.

Note

It is possible for this function to return the same event index for different timestamps. When you repeatedly call the [XedCopyNuiColorFrame](#) function to copy multiple events from one XED file to another, the copy operation will fail if you specify a value for the *eventIndex* parameter that is already mapped to a different value of the *frameMicroseconds* parameter in the new file.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiDepthEventCount

[See Also](#)



Retrieves the number of depth events in the depth stream of the specified Xbox event data (XED) file.

Syntax

C++

```
UINT XedGetNuiDepthEventCount(  
    XED_CONTEXT *pContext  
)
```

Parameters

pContext

[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

Return Value

Number of depth events in the depth stream of the specified XED file.

Remarks

If a stream has *n* events, the event indices will always be incremented from zero to *n*−1. Event indices do not directly correspond to frame numbers, as some frames may have been dropped.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiDepthEventIndexFromFrameNumber

[See Also](#)



Retrieves the index of the depth event with the specified frame number in an Xbox event data (XED) file.

Syntax

C++

```
UINT XedGetNuiDepthEventIndexFromFrameNumber(  
    XED_CONTEXT *pContext,  
    UINT frameNumber  
)
```

Parameters

pContext

[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

frameNumber

[in] Frame number of the depth event to return the index of.

Return Value

Returns the index of the depth event, or [XED_EVENTINDEX_INVALID](#) if this function cannot find a depth event with the specified frame number.

Remarks

Skeletal events and depth events have corresponding frame numbers. If you have a skeletal event, you can pass its frame number to this function to retrieve the corresponding depth event.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

✚ **See Also**
[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiDepthEventIndexFromMicroseconds
[See Also](#)



Gets the index of the depth event with the specified timestamp in an Xbox Event Data (XED) file.

✚ **Syntax**

C++

```
UINT XedGetNuiDepthEventIndexFromMicroseconds(  
    XED_CONTEXT *pContext,  
    UINT64 frameMicroseconds  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.
frameMicroseconds
[in] Timestamp of the depth event of which to return the index, in microseconds. The first event in the file is likely to have a timestamp substantially greater than zero.

Return Value

Returns the index of the depth event with the specified timestamp. If this function cannot find a depth event with the specified timestamp, it returns the index of the depth event that immediately precedes the timestamp. This function returns [XED_EVENTINDEX_INVALID](#) if the specified timestamp precedes the first event in the stream.

✚ **Remarks**

This function does not enforce a gap constraint between events. If multiple events are dropped, the time between events can be greater than expected. Pass NULL to the *pBuffer* parameter of the [XedReadNuiDepthFrame](#) function to get the timestamp of an event.

✔ **Note**

It is possible for this function to return the same event index for different timestamps. When you repeatedly call the [XedCopyNuiDepthFrame](#) function to copy multiple events from one XED file to another, the copy operation will fail if you specify a value for the *eventIndex* parameter that is already mapped to a different value of the *frameMicroseconds* parameter in the new file.

✚ **Requirements**

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

✚ **See Also**
[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiSkeletonEventCount
[See Also](#)



Retrieves the number of skeletal events in the skeletal stream of the specified Xbox event data (XED) file.

✚ **Syntax**

C++

```
UINT XedGetNuiSkeletonEventCount(  
    XED_CONTEXT *pContext  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

Return Value

Number of skeletal events in the skeletal stream of the specified XED file.

✚ **Remarks**

If a stream has *n* events, the event indices will always be incremented from zero to *n*–1. Event indices do not directly correspond to frame numbers, as some frames may have been dropped.

✚ **Requirements**

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

✚ **See Also**
[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiSkeletonEventIndexFromFrameNumber

[See Also](#)



Retrieves the index of the skeletal event with the specified frame number in an Xbox event data (XED) file.

Syntax

C++

```
UINT XedGetNuiSkeletonEventIndexFromFrameNumber(  
    XED_CONTEXT *pContext,  
    UINT frameNumber  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.
frameNumber
[in] Frame number of the skeletal event to return the index of.

Return Value

Returns the index of the skeletal event, or [XED_EVENTINDEX_INVALID](#) if this function cannot find a skeletal event with the specified frame number.

Remarks

Skeletal events and depth events have corresponding frame numbers. If you have a depth event, you can pass its frame number to this function to retrieve the corresponding skeletal event.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedGetNuiSkeletonEventIndexFromMicroseconds

[See Also](#)



Gets the index of the skeletal event with the specified timestamp in an Xbox Event Data (XED) file.

Syntax

C++

```
UINT XedGetNuiSkeletonEventIndexFromMicroseconds(  
    XED_CONTEXT *pContext,  
    UINT64 frameMicroseconds  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.
frameMicroseconds
[in] Timestamp of the skeletal event of which to return the index, in microseconds. The first event in the file is likely to have a timestamp substantially greater than zero.

Return Value

Returns the index of the skeletal event with the specified timestamp. If this function cannot find a skeletal event with the specified timestamp, it returns the index of the skeletal event that immediately precedes the timestamp. This function returns [XED_EVENTINDEX_INVALID](#) if the specified timestamp precedes the first event in the stream.

Remarks

This function does not enforce a gap constraint between events. If multiple events are dropped, the time between events can be greater than expected. Pass NULL to the *pBuffer* parameter of the [XedReadNuiSkeletonFrame](#) function to get the timestamp of an event.

Note

It is possible for this function to return the same event index for different timestamps. When you repeatedly call the [XedCopyNuiSkeletonFrame](#) function to copy multiple events from one XED file to another, the copy operation will fail if you specify a value for the *eventIndex* parameter that is already mapped to a different value of the *frameMicroseconds* parameter in the new file.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

XedGetTitleDataEventCount

[See Also](#)



Retrieves the number of title data events in the title data stream of the specified Xbox event data (XED) file.

Syntax

C++

```
UINT XedGetTitleDataEventCount (
    XED_CONTEXT *pContext
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

Return Value

Number of title data events in the title data stream of the specified XED file.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[Title Data \[Xbox Studio\]](#)

XedGetTitleDataEventIndexFromMicroseconds

[See Also](#)



Gets the index of the title data event with the specified timestamp in an Xbox Event Data (XED) file.

Syntax

C++

```
UINT XedGetTitleDataEventIndexFromMicroseconds (
    XED_CONTEXT *pContext,
    UINT64 eventMicroseconds
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to search.

eventMicroseconds
[in] Timestamp of the title data event to which to return the index, in microseconds. The first event in the file is likely to have a timestamp substantially greater than zero.

Return Value

Returns the index of the title data event with the specified timestamp. If this function cannot find a title data event with the specified timestamp, it returns the index of the title data event that immediately precedes the timestamp. This function returns [XED_EVENTINDEX_INVALID](#) if the specified timestamp precedes the first event in the stream.

Remarks

This function does not enforce a gap constraint between events. If multiple events are dropped, the time between events can be greater than expected. Pass NULL to the *pBuffer* parameter of the [XedReadTitleData](#) function to get the timestamp of an event.

Note

It is possible for this function to return the same event index for different timestamps. When you repeatedly call the [XedCopyTitleData](#) function to copy multiple events from one XED file to another, the copy operation will fail if you specify a value for the *eventIndex* parameter that is already mapped to a different value of the *eventMicroseconds* parameter in the new file.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[Title Data \[Xbox Studio\]](#)

XedOpenFile

[See Also](#)



Opens an Xbox event data (XED) file for reading.

Syntax

C++

```
HRESULT XedOpenFile(
    const char *sFilePath,
    DWORD flags,
    XED_CONTEXT **ppContext
)
```

Parameters

- sFilePath*
[in] Pointer to the path and file name of the XED file to open.
- flags*
[in] Reserved; pass zero to this parameter.
- ppContext*
[out, optional] Pointer that will receive the address of the [XED_CONTEXT](#) of the opened XED file.

Return Value

Returns S_OK if successful; otherwise, an error code.

Requirements

- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedReadNuiColorFrame

[See Also](#)

Reads the specified color frame from an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedReadNuiColorFrame(
    XED_CONTEXT *pContext,
    UINT eventIndex,
    UINT *pFrameNumber,
    INT64 *pFrameMicroseconds,
    DWORD *pBuffer,
    UINT bufferSize,
    BOOL registerWithDepth
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to read. This file must have been opened by a call to the [XedOpenFile](#) function.
- eventIndex*
[in] Index of the event to read.
- pFrameNumber*
[out, optional] Pointer that will receive the frame number of the frame.
- pFrameMicroseconds*
[out, optional] Pointer that will receive the timestamp of the frame.
- pBuffer*
[out, optional] Pointer to the buffer that will receive the frame data. If you pass NULL to this parameter, the frame number and timestamp will be read from an in-memory cache, without causing file I/O.
- bufferSize*
[in] Size of the buffer pointed to by the *pBuffer* parameter, in bytes. The buffer size must be 640 × 480 × `sizeof(DWORD)`.
- registerWithDepth*
[in] Specify TRUE to register the color frame against the depth frame; otherwise, specify FALSE. If the XED file was created without specifying a template, you cannot use registration. For more information about registering frames, see the Remarks section.

Return Value

Returns S_OK if successful; otherwise, an error code.

Remarks

Registering a color frame corrects the linear offset between the depth and color cameras, rotational offsets, and differences in lens properties. When registered, pixels in the color frame line up with pixels in the depth frame.

The depth frame used to register the color frame is located by looking for the depth frame immediately preceding the color frame. If a depth frame cannot be found, an error is returned.

Important

When a color frame is read from an XED file, its color space is converted to RGB. When writing this frame to a new file, the frame is converted back to its original color space. The color space conversion uses a lossy algorithm. Multiple read/write iterations will degrade the quality of the color frame. Use the [XedCopyNuiColorFrame](#) function to copy a color frame between two XED files without loss of fidelity.

Requirements

- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedReadNuiDepthFrame

[See Also](#)



Reads the specified depth frame from an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedReadNuiDepthFrame(  
    XED_CONTEXT *pContext,  
    UINT eventIndex,  
    UINT *pFrameNumber,  
    UINT64 *pFrameMicroseconds,  
    WORD *pBuffer,  
    UINT bufferSize,  
    BOOL registerWithColor  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to read. This file must have been opened by a call to the [XedOpenFile](#) function.
- eventIndex*
[in] Index of the event to read.
- pFrameNumber*
[out, optional] Pointer that will receive the frame number of the frame.
- pFrameMicroseconds*
[out, optional] Pointer that will receive the timestamp of the frame.
- pBuffer*
[out, optional] Pointer to the buffer that will receive the frame data. The depth values in the frame reserve the lower three bits for the segmentation map, but the segmentation map is not recorded to the file. Do not use the segmentation map bits. If you pass NULL to this parameter, the frame number and timestamp will be read from an in-memory cache, without causing file I/O.
- bufferSize*
[in] Size of the buffer pointed to by the *pBuffer* parameter, in bytes. The buffer size must be $320 \times 240 \times \text{sizeof}(\text{WORD})$.
- registerWithColor*
[in] Specify TRUE to register the depth frame against the color frame; otherwise, specify FALSE. If the XED file was created without specifying a template, you cannot use registration. For more information about registering frames, see the Remarks section.

Return Value

Returns S_OK if successful; otherwise, an error code.

Remarks

Registering a depth frame corrects the linear offset between the depth and color cameras, rotational offsets, and differences in lens properties. When registered, pixels in the depth frame line up with pixels in the color frame. No color frame is needed to register the depth frame.

Requirements

- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedReadNuiIdentityMessage

[See Also](#)



Reads the specified identity message from an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedReadNuiIdentityMessage(  
    XED_CONTEXT *pContext,  
    UINT eventIndex,  
    UINT64 *pFrameMicroseconds,  
    NUI_IDENTITY_MESSAGE *pIdentityMessage,  
    UINT identityMessageSize  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to read. This file must have been opened by a call to the [XedOpenFile](#) function.
- eventIndex*
[in] Index of the event to read.
- pFrameMicroseconds*
[out, optional] Pointer that will receive the timestamp of the frame.
- pIdentityMessage*
[out, optional] Pointer to the [NUI_IDENTITY_MESSAGE](#) structure that will receive the identity message. If you pass NULL to this parameter, the timestamp will be read from an in-

memory cache, without causing file I/O.

identityMessageSize
[in] Size of the buffer pointed to by the *pIdentityMessage* parameter, in bytes.

Return Value
Returns S_OK if successful; otherwise, an error code.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedReadNuiSkeletonFrame

[See Also](#)

Reads the specified skeletal frame from an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedReadNuiSkeletonFrame(  
    XED_CONTEXT *pContext,  
    UINT eventIndex,  
    UINT *pFrameNumber,  
    UINT64 *pFrameMicroseconds,  
    NUI_SKELETON_FRAME *pSkeletonFrame,  
    UINT skeletonFrameSize  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to read. This file must have been opened by a call to the [XedOpenFile](#) function.

eventIndex
[in] Index of the event to read.

pFrameNumber
[out, optional] Pointer that will receive the frame number of the frame.

pFrameMicroseconds
[out, optional] Pointer that will receive the timestamp of the frame.

pSkeletonFrame
[out, optional] Pointer to the [NUI_SKELETON_FRAME](#) structure that will receive the frame data. If you pass NULL to this parameter, the frame number and timestamp will be read from an in-memory cache, without causing file I/O.

skeletonFrameSize
[in] Size of the buffer pointed to by the *pSkeletonFrame* parameter, in bytes.

Return Value
Returns S_OK if successful; otherwise, an error code.

Requirements

Header: Declared in XedFile.h.
Library: Use XedFile.lib.

See Also

[XedFile Functions](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedReadTitleData

[See Also](#)

Reads the specified title event from an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedReadTitleData(  
    XED_CONTEXT *pContext,  
    UINT eventIndex,  
    UINT64 *pEventMicroseconds,  
    XSTUDIO_TITLE_INDEX_DATA *pTitleIndexData,  
    void *pTitleDataBuffer,  
    UINT titleDataBufferSize,  
    UINT *pRequiredTitleDataBufferSize  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file from which to read. This file must have been opened by a call to the [XedOpenFile](#) function.

eventIndex
[in] Index of the event to read.

pEventMicroseconds
[out, optional] Pointer that will receive the timestamp of the event.

pTitleIndexData
[out, optional] Pointer to the [XSTUDIO_TITLE_INDEX_DATA](#) structure that will receive the custom title index data.

pTitleDataBuffer
[out, optional] Pointer to the buffer that will receive the event data. If you pass NULL to this parameter, the timestamp will be read from an in-memory cache, without causing file I/O.

titleDataBufferSize
[in] Size of the buffer pointed to by the *pTitleDataBuffer* parameter, in bytes.

pRequiredTitleDataBufferSize
[out] Pointer that will receive the total number of available bytes that were returned by the read operation.

Return Value
Returns S_OK if successful; otherwise, an error code.

- +

Requirements
- Header:** Declared in XedFile.h.

Library: Use XedFile.lib.
- +

See Also
- [XedFile Functions](#)

[Title Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedSetFileNuiColorQuality

[See Also](#)



Sets the color quality to use when writing color frames to an Xbox event data (XED) file.

+

Syntax

C++

```
HRESULT XedSetFileNuiColorQuality(  
    XED_CONTEXT *pContext,  
    DWORD newColorQuality,  
    DWORD *pPreviousColorQuality  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file for which to set the color quality. This file must have been opened by a call to the [XedCreateFile](#) function.

newColorQuality
[in] [NUI_INITIALIZE_FLAGS](#) flag that specifies the color quality to set. This must be either NUI_INITIALIZE_FLAG_USES_COLOR or NUI_INITIALIZE_FLAG_USES_HIGH_QUALITY_COLOR.

pPreviousColorQuality
[out, optional] Pointer that will receive the color quality that was set before this function was called.

Return Value
Returns S_OK if successful; otherwise, an error code.

- +

Remarks
- This function sets the color quality of the color frames that are passed to the [XedWriteNuiColorFrame](#) function.

The default color quality is NUI_INITIALIZE_FLAG_USES_COLOR.

- +

Requirements
- Header:** Declared in XedFile.h.

Library: Use XedFile.lib.
- +

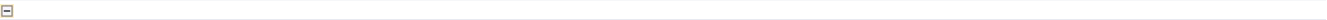
See Also
- [XedFile Functions](#)

[NUI_INITIALIZE_FLAGS](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedSetNuiColorBufferType

[See Also](#)



Sets whether to use the sRGB or YUV color space when writing color frames to an Xbox Event Data (XED) file, or when reading color frames from an XED file.

+

Syntax

C++

```
HRESULT XedSetNuiColorBufferType(  
    XED_CONTEXT *pContext,  
    NUI_IMAGE_TYPE newImageType,  
    NUI_IMAGE_TYPE *pPreviousImageType  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file for which to set the color space.

newImageType
[in] [NUI_IMAGE_TYPE](#) value that specifies the color space to set. This must be either NUI_IMAGE_TYPE_COLOR or NUI_IMAGE_TYPE_COLOR_YUV.

pPreviousImageType
[out, optional] Pointer that will receive the color space that was set before this function was called.

Return Value
Returns S_OK if successful; otherwise, an error code.

- Remarks
- This function sets the color space of the color frames that are returned by the [XedReadNuiColorFrame](#) function. It also sets the color space of the color frames that are passed to the [XedWriteNuiColorFrame](#) function.
- This function sets the color buffer type for the lifetime of the context that is referenced by the *pContext* parameter. The color buffer type is not stored in the XED file.
- The default color space is NUI_IMAGE_TYPE_COLOR.

- Requirements
- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

- See Also
- [XedFile Functions](#)
[NUI_IMAGE_TYPE](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedSetNuiDepthImageType

[See Also](#)



Sets whether to include the player segmentation index when reading or writing depth frames for an Xbox Event Data (XED) file.

Syntax

C++

```
HRESULT XedSetNuiDepthImageType(  
    XED_CONTEXT *pContext,  
    NUI_IMAGE_TYPE newImageType,  
    NUI_IMAGE_TYPE *pPreviousImageType  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file for which to set the depth image type.

newImageType
[in] [NUI_IMAGE_TYPE](#) value that specifies the new depth image type to set. This must be either NUI_IMAGE_TYPE_DEPTH or NUI_IMAGE_TYPE_DEPTH_AND_PLAYER_INDEX.

pPreviousImageType
Pointer that will receive the depth image type that was set before this function was called.

Return Value
Returns S_OK if successful; otherwise, an error code.

- Remarks
- This function sets the depth image type for the lifetime of the context that is referenced by the *pContext* parameter. The depth image type is not stored in the XED file.
- The default depth image type is NUI_IMAGE_TYPE_DEPTH.

- Requirements
- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

- See Also
- [XedFile Functions](#)
[NUI_IMAGE_TYPE](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedWriteNuiColorFrame

[See Also](#)



Writes the specified color frame to an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedWriteNuiColorFrame(  
    XED_CONTEXT *pContext,  
    UINT frameNumber,  
    UINT64 frameMicroseconds,
```

```
const DWORD *pBuffer,  
UINT bufferSize  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to write. This file must have been opened by a call to the [XedCreateFile](#) function.
- frameNumber*
[in] Frame number of the frame to write.
- frameMicroseconds*
[in] Timestamp of the frame to write.
- pBuffer*
[in] Pointer to a buffer that contains the frame data.
- bufferSize*
[in] Size of the buffer pointed to by the *pBuffer* parameter, in bytes. The buffer size must be $640 \times 480 \times \text{sizeof}(\text{DWORD})$.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

Frames are always written at the end of a file. When writing to a stream, all writes must be in chronological order—the frame numbers and timestamps must increment with subsequent writes. When combining streams from different files into a new file, the caller is responsible for generating new frame numbers and timestamps. Gaps are allowed between both frame numbers and timestamps to adjust for missing frames.

Playback performance is affected by the order in which streams and events are written to file. When you need to play back a file that contains multiple streams, you can improve playback performance by interleaving events across the streams in chronological order.

Requirements

- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

See Also

- [XedFile Functions](#)
- [XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedWriteNuiDepthFrame

[See Also](#)



Writes the specified depth frame to an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedWriteNuiDepthFrame(  
    XED_CONTEXT *pContext,  
    UINT frameNumber,  
    UINT64 frameMicroseconds,  
    const WORD *pBuffer,  
    UINT bufferSize  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to write. This file must have been opened by a call to the [XedCreateFile](#) function.
- frameNumber*
[in] Frame number of the frame to write.
- frameMicroseconds*
[in] Timestamp of the frame to write.
- pBuffer*
[in] Pointer to a buffer that contains the frame data. The depth values in the frame reserve the lower three bits for the segmentation map, but the segmentation map is not recorded to the file. Do not use the segmentation map bits.
- bufferSize*
[in] Size of the buffer pointed to by the *pBuffer* parameter, in bytes. The buffer size must be $320 \times 240 \times \text{sizeof}(\text{WORD})$.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

Frames are always written at the end of a file. When writing to a stream, all writes must be in chronological order—the frame numbers and timestamps must increment with subsequent writes. When combining streams from different files into a new file, the caller is responsible for generating new frame numbers and timestamps.

Playback performance is affected by the order in which streams and events are written to file. When you need to play back a file that contains multiple streams, you can improve playback performance by interleaving events across the streams in chronological order.

Requirements

- Header:** Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

- [XedFile Functions](#)
- [XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedWriteNuiIdentityMessage

See Also

Writes the specified identity message to an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedWriteNuiIdentityMessage(  
    XED_CONTEXT *pContext,  
    UINT64 frameMicroseconds,  
    const NUI_IDENTITY_MESSAGE *pIdentityMessage,  
    UINT identityMessageSize  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to write. This file must have been opened by a call to the [XedCreateFile](#) function.
- frameMicroseconds*
[in] Timestamp of the frame to write.
- pIdentityMessage*
[in] Pointer to a [NUI_IDENTITY_MESSAGE](#) structure that contains the identity message.
- identityMessageSize*
[in] Size of the buffer pointed to by the *pIdentityMessage* parameter, in bytes.

Return Value

Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has been written already to the stream.

Remarks

Frames are always written at the end of a file. When writing to a stream, all writes must be in chronological order—the timestamps must increment with subsequent writes. When combining streams from different files into a new file, the caller is responsible for generating new timestamps.

Playback performance is affected by the order in which streams and events are written to file. When you need to play back a file that contains multiple streams, you can improve playback performance by interleaving events across the streams in chronological order.

Important

When an identity operation is complete, you must write a special "completed" message to the XED file. Because this message includes internal identity data that is not exposed to users, you cannot use **XedWriteNuiIdentityMessage** to write the message. Instead, call the [XedCopyNuiIdentityMessage](#) function to copy a "completed" message from another XED file. This will also copy the internal identity data.

Requirements

- Header:** Declared in XedFile.h.
- Library:** Use XedFile.lib.

See Also

- [XedFile Functions](#)
- [XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedWriteNuiSkeletonFrame

See Also

Writes the specified skeletal frame to an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedWriteNuiSkeletonFrame(  
    XED_CONTEXT *pContext,  
    UINT frameNumber,  
    UINT64 frameMicroseconds,  
    const NUI_SKELETON_FRAME *pSkeletonFrame,  
    UINT skeletonFrameSize  
)
```

Parameters

- pContext*
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to write. This file must have been opened by a call to the [XedCreateFile](#) function.
- frameNumber*

[in] Frame number of the frame to write.

frameMicroseconds
[in] Timestamp of the frame to write.

pSkeletonFrame
[in] Pointer to a [NUI_SKELETON_FRAME](#) structure that contains the frame data.

skeletonFrameSize
[in] Size of the buffer pointed to by the *pSkeletonFrame* parameter, in bytes.

Return Value
Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>frameMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

Frames are always written at the end of a file. When writing to a stream, all writes must be in chronological order—the frame numbers and timestamps must increment with subsequent writes. When combining streams from different files into a new file, the caller is responsible for generating new frame numbers and timestamps.

Playback performance is affected by the order in which streams and events are written to file. When you need to play back a file that contains multiple streams, you can improve playback performance by interleaving events across the streams in chronological order.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedWriteTitleData

[See Also](#)



Writes the specified title event to an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XedWriteTitleData(  
    XED_CONTEXT *pContext,  
    UINT64 eventMicroseconds,  
    const XSTUDIO_TITLE_INDEX_DATA *pTitleIndexData,  
    const void *pTitleDataBuffer,  
    UINT titleDataBufferSize  
)
```

Parameters

pContext
[in] Pointer to the [XED_CONTEXT](#) of the XED file to which to write. This file must be opened by a call to the [XedCreateFile](#) function.

eventMicroseconds
[in] Timestamp of the event to write.

pTitleIndexData
[in, optional] Pointer to a [XSTUDIO_TITLE_INDEX_DATA](#) structure that contains custom title index data.

pTitleDataBuffer
[in] Pointer to a buffer that contains the title event data.

titleDataBufferSize
[in] Size of the buffer pointed to by the *pTitleDataBuffer* parameter, in bytes. The maximum size of the buffer is XSTUDIO_TITLE_DATA_MAX_BUFFER_SIZE.

Return Value
Returns S_OK if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The timestamp specified by the <i>eventMicroseconds</i> parameter is less than or equal to a timestamp that has already been written to the stream.

Remarks

Events are always written at the end of a file. When writing to a stream, all writes must be in chronological order—the timestamps must increment with subsequent writes. When combining streams from different files into a new file, the caller is responsible for generating new timestamps.

Playback performance is affected by the order in which streams and events are written to file. When you need to play back a file that contains multiple streams, you can improve playback performance by interleaving events across the streams in chronological order.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Functions](#)
[Title Data \[Xbox Studio\]](#)
[XStudio Return Codes](#)

XedFile Structures

[See Also](#)



This topic contains the reference pages for the structures that are defined in the header file XedFile.h.

In This Section

[XED_CONTEXT](#)

Opaque Xbox event data (XED) file context.

See Also

[XedFile Reference](#)

XED_CONTEXT Structure

[See Also](#)



Opaque Xbox event data (XED) file context.

Syntax

C++

```
typedef struct {  
} XED_CONTEXT;
```

Remarks

This is an opaque file context that is returned from the [XedCreateFile](#) and [XedOpenFile](#) functions. You must pass this context to the other XED file functions to perform operations on the XED file.

Requirements

Header: Declared in XedFile.h.

Library: Use XedFile.lib.

See Also

[XedFile Structures](#)

XedFile Constants

[See Also](#)



This topic contains the reference pages for the constants that are defined in the header file XedFile.h.

In This Section

[XED Constants](#)

Constant values that are used by the APIs defined in XedFile.h.

See Also

[XedFile Reference](#)

XED Constants

[See Also](#)



Constant values that are used by the APIs defined in XedFile.h.

Value	Description
XED_EVENTINDEX_INVALID	The specified event cannot be found in the Xbox event data (XED) file.
XED_FRAMENUMBER_INVALID	Pass this value to the Xbox event data (XED) copy APIs to copy the frame number of a frame from the source XED file to the destination XED file.
XED_TIMESTAMP_INVALID	Pass this value to the Xbox event data (XED) copy APIs to copy the timestamp of a frame from the source XED file to the destination XED file.

See Also

[XedFile Constants](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XNui Reference



This section contains the API Reference pages for XNui functions, which are utility functions for Natural User Interface (NUI).

In This Section

- [XNuiDelayUI](#)
Suppresses system gesture recognition for the specified duration, to give priority to the title for gesture recognition during critical times.
- [XNuiFirstUseCompleted](#)
Deprecated. Specifies whether the title should run in out-of-box experience (OOBE) mode.
- [XNuiGetHardwareStatus](#)
Determines the connectivity and hardware status of the Kinect Sensor.
- [XNuiIsChatMicrophoneEnabled](#)
Reports whether the Kinect microphone array is muted or enabled.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XNuiDelayUI

[See Also](#)



Suppresses system gesture recognition for the specified duration, to give priority to the title for gesture recognition during critical times.

Syntax

```
C++
HRESULT XNuiDelayUI(
    ULONG ulMilliseconds
)
```

Parameters

ulMilliseconds
[in] The delay, in milliseconds, before the system handle's UI and associated gestures are available to users. A value of 0 cancels a previous call to this function, and permits system gestures to be recognized immediately.

Return Value

S_OK.

Remarks

Use this function to suppress system gesture recognition during critical times, such as during potentially conflicting title gestures. System gesture recognition should only be suspended when the title requires Natural User Interface gestures to be detected, where those gestures might be similar to the system gesture. System gesture recognition should only be suspended temporarily.

If you call this function again before the previously-specified duration has passed, the new duration begins, overriding the previously-specified duration.

To use this function as a toggle switch, do the following:

- Call this function once with an adequate number of milliseconds to disable detecting the system gesture.
- Call the function again with zero to re-enable detecting the system gesture.

Always pair these calls together.

Requirements

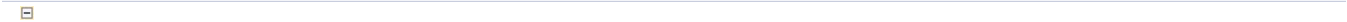
Header: Declared in xbox.h (include xtl.h instead).
Library: Use xapilib.lib.

See Also

[The System Gesture for Kinect for Xbox 360](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XNuiFirstUseCompleted



Deprecated. Specifies whether the title should run in out-of-box experience (OOBE) mode.

Note

This function is deprecated. It now always returns TRUE.

Syntax

```
C++
BOOL XNuiFirstUseCompleted()
```

Return Value

Always returns TRUE.

Remarks

This function formerly worked as follows: This function is designed for titles that ship boxed with the Kinect sensor array. When such titles are launched, they may call this function to determine whether the Kinect sensor array has already been configured. If the function returns FALSE, the title should run in first-use mode, also known as OOBE mode.

Requirements

Header: Declared in xbox.h (include xtl.h instead).
Library: Use xapilib.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XNuiGetHardwareStatus



Determines the connectivity and hardware status of the Kinect Sensor.

Syntax

C++

```
DWORD XNuiGetHardwareStatus()
```

Return Value

If the Kinect Sensor is physically connected, this function returns one or more of the following values, combined together by using the bitwise OR operation. If the Kinect Sensor is not physically connected, this function returns 0.

These values are defined in xam.h.

Value	Meaning
XNUI_HARDWARE_STATUS_CONNECTED	The Kinect Sensor is connected, but might not be ready for use. There may be a delay of up to 7 seconds from the time that the Kinect Sensor is connected to the time that it is ready to start giving data. This function never returns this value when Xbox Studio is running in headless mode.
XNUI_HARDWARE_STATUS_READY	The Kinect Sensor is connected, and the API can start consuming data.
XNUI_HARDWARE_STATUS_ERROR	A general hardware error from the Kinect Sensor. The system will automatically invoke the Troubleshooter UI. When the caller receives this value, the title should probably save any game state (progress), because the Troubleshooter might not return the user to the title.
XNUI_HARDWARE_STATUS_FORCED_OFF	Any of the following: - The Kinect Sensor has been automatically turned off because the user has set the console setting to automatically turn off the Kinect Sensor when the console or Kinect Sensor are not in use. - The console is in quiet mode, such as during DVD playback.

Remarks

You can use this function as an alternative to, or in addition to, listening for the [XN_SYS_NUIHARDWARESTATUSCHANGED](#) system notification.

Requirements

Header: Declared in xbox.h (include xtl.h instead).
Library: Use xapilib.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XNuiIsChatMicrophoneEnabled



Reports whether the Kinect microphone array is muted or enabled.

Syntax

C++

```
BOOL XNuiIsChatMicrophoneEnabled()
```

Return Value

TRUE if the Kinect chat microphone is enabled in the system settings; FALSE otherwise.

Requirements

Header: Declared in xbox.h (include xtl.h instead).
Library: Use xapilib.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Reference

[See Also](#)



This section contains API Reference pages for the APIs that record and play back the Xbox event data (XED) files used by the [Xbox Studio](#) tool. For more information about using the XStudio APIs, see [NUI PC Conduit Sample](#).

In This Section

[XStudio Functions](#)

Contains reference pages for the functions that are defined in the header file XStudio.h.

[XStudio Structures](#)

Contains the reference pages for the structures that are defined in the header file XStudio.h.

[XStudio Enumerations](#)

Contains the reference pages for the enumerations that are defined in the header file XStudio.h.

[XStudio Callbacks](#)

Contains reference pages for the callback functions that are defined in the header file XStudio.h.

[XStudio Constants](#)

Contains the reference pages for the constants that are defined in the header file XStudio.h.

[XStudio Types](#)

Describes the XStudio types.

[XStudio Return Codes](#)

Lists the XStudio return codes.

▣ See Also

[Xbox Studio](#)

[NUI Samples: Xbox Studio Samples](#)

[Natural User Interface Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Functions[See Also](#)

This topic contains the reference pages for the functions that are defined in the header file XStudio.h.

In This Section[XStudioAddSuspendPoint](#)

Adds a suspend point for playback of the specified stream.

[XStudioCloseFile](#)

Closes an Xbox event data (XED) file.

[XStudioCloseHandle](#)

Closes a handle to an XStudio callback function.

[XStudioConnect](#)

Establishes a connection from your computer to a development console.

[XStudioCreateFile](#)

Creates a new Xbox event data (XED) file to which you can record the specified streams, or opens an existing XED for playback.

[XStudioDisconnect](#)

Disconnects the connection from your computer to the development console.

[XStudioGetActiveStreams](#)

Retrieves a list of the streams that are active currently.

[XStudioGetConnectionStatus](#)

Determines whether your computer is connected to a development console.

[XStudioGetErrorStreams](#)

Gets a list of the streams that have encountered an error.

[XStudioGetFileStreams](#)

Gets a list of the streams that exist in the specified Xbox event data (XED) file.

[XStudioGetInputMode](#)

Gets the event source for the specified stream.

[XStudioGetLoopCount](#)

Gets the number of remaining loop iterations for playback of the specified streams.

[XStudioMapStreams](#)

Maps or unmaps the specified streams to the specified Xbox event data (XED) file or callback function.

[XStudioReadTitleData](#)

Reads a title data event from the title data stream.

[XStudioRegisterNotificationCallback](#)

Registers an XStudio callback function that receives notifications when the state of an event stream changes.

[XStudioRegisterStreamCallback](#)

Registers an XStudio callback function that can be used to produce or consume NUI events.

[XStudioResume](#)

Resumes playback of the specified suspended streams.

[XStudioSetInputMode](#)

Sets the event source for the specified streams.

[XStudioSetLoopCount](#)

Sets the number of loop iterations for playback of all mapped streams.

[XStudioSingleStep](#)

Advances playback of the specified streams by a single event.

[XStudioStart](#)

Starts recording or playback of the specified streams.

[XStudioStop](#)

Stops recording or playback of the specified streams.

[XStudioStopEx](#)

Stops an Xbox event data (XED) file or stream callback from producing or consuming events for the specified streams.

[XStudioSuspend](#)

Suspends playback of the specified streams.

[XStudioSwitchToProducers](#)

Sets the event producers for the specified streams to the currently mapped Xbox event data (XED) file.

[XStudioUnmapStreams](#)

Unmaps the specified streams from the specified Xbox event data (XED) file or callback function.

[XStudioWriteTitleData](#)

Writes a title data event to the title data stream.

See Also

[XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioAddSuspendPoint

[See Also](#)



Adds a suspend point for playback of the specified stream.

Syntax

C++

```
HRESULT XStudioAddSuspendPoint(  
    XSTUDIO_STREAM_ID dwStreamID,  
    DWORD dwEventIndex,  
    XSTUDIO_HANDLE *phandle  
)
```

Parameters

dwStreamID
[in] [XSTUDIO_STREAM_ID](#) value that specifies the stream for which to set the suspend point.

dwEventIndex
[in] Index of the event to set as the suspend point.

phandle
[out] Pointer to an [XSTUDIO_HANDLE](#) that receives a handle to the suspend point. To clear the suspend point, call the [XStudioCloseHandle](#) function to close the handle.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070055	ERROR_ALREADY_ASSIGNED	A suspend point is set already for the specified data stream.
0x80070057	E_XSTUDIO_INVALID_ARG	You passed NULL to the <i>phandle</i> parameter.
0x800704B1	E_XSTUDIO_NO_CONNECTION	XStudio is not connected to a development console.
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.

Remarks

This function is available only on the computer.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioResume](#)
[XStudioSingleStep](#)
[XStudioSuspend](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioCloseFile

[See Also](#)



Closes an Xbox event data (XED) file.

Syntax

C++

```
HRESULT XStudioCloseFile(  
    XSTUDIO_FILE_HANDLE hFile  
)
```

Parameters

hFile
[in] [XSTUDIO_FILE_HANDLE](#) to the XED file to close. This handle was returned from the [XStudioCreateFile](#) function when you created or opened the file. After this function returns, this handle is no longer valid.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the file was successfully closed; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>hFile</i> is invalid.

Remarks

When you call this function, all streams that are mapped to the file are unmapped automatically.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioCreateFile](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioCloseHandle

[See Also](#)

Closes a handle to an XStudio callback function.

Syntax

C++

```
HRESULT XStudioCloseHandle(  
    XSTUDIO_HANDLE Handle  
)
```

Parameters

Handle
[in] [XSTUDIO_HANDLE](#) to the callback function to close. This handle was returned from the [XStudioRegisterStreamCallback](#) function when you registered the callback function. After this function returns, this handle is no longer valid.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the handle was successfully closed; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>Handle</i> is invalid.

Remarks

This function is available only on the computer.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioRegisterStreamCallback](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioConnect

[See Also](#)

Establishes a connection from your computer to a development console.

Syntax

C++

```
HRESULT XStudioConnect(  
    const char *sConsoleName,  
    XSTUDIO_CONNECTION_TYPE ConnectionType  
)
```

Parameters

sConsoleName
[in] Name or IP address of the Xbox console to which you want to connect. If you are connecting over USB, specify NULL. If you are connecting over TCP/IP, specifying NULL connects to the default development console selected in Xbox 360 Neighborhood.
ConnectionType
[in] [XSTUDIO_CONNECTION_TYPE](#) value that specifies whether to connect over USB or TCP/IP. Unlike the behavior of the Xbox Studio tool, there is no fallback to TCP/IP if a USB connection attempt fails. If an easy transfer cable is connected to the console specified by *sConsoleName*, passing XSTUDIO_CONNECTION_TYPE_TCP to this parameter will result in an E_XSTUDIO_NO_CONNECTION error.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800700AA	E_XSTUDIO_BUSY	A connection already exists.
0x800704B1	E_XSTUDIO_NO_CONNECTION	A connection could not be established.

Remarks

This function is available only on the computer.

This function is synchronous, and will block until a connection is established or the function times out.

It is possible for an established connection to be lost. To get the current connection status, call the [XStudioGetConnectionStatus](#) function.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[Connecting to a Development Console \[Xbox Studio\]](#)
[XStudioDisconnect](#)
[XStudioGetConnectionStatus](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioCreateFile

[See Also](#)



Creates a new Xbox event data (XED) file to which you can record the specified streams, or opens an existing XED for playback.

Syntax

C++

```
HRESULT XStudioCreateFile(  
    const char *sFilePath,  
    DWORD dwDesiredAccess,  
    DWORD dwCreationDisposition,  
    XSTUDIO_STREAM_FLAGS dwStreams,  
    XSTUDIO_FILE_HANDLE *phFile  
)
```

Parameters

sFilePath
[in] Path and filename of the XED file to open or create. This file can be either on the computer or the development console. UNC paths are not supported. Console filenames have a maximum length of XCONTENT_MAX_FILENAME_LENGTH. For more information about this parameter, see the Remarks section.

dwDesiredAccess
[in] Specifies whether to request read or write access to the specified XED file. This must be one of values listed in the following table.

Value	Description
GENERIC_READ	Requests read access to the XED file. If you request read access, you must pass OPEN_EXISTING to the <i>dwCreationDisposition</i> parameter. You must also pass XSTUDIO_STREAM_FLAG_NONE to the <i>streamFlags</i> parameter.
GENERIC_WRITE	Requests write access to the XED file.

dwCreationDisposition
[in] Specifies the action to take when you create the specified XED file. This must be one of the values listed in the following table.

Value	Description
CREATE_NEW	Creates a new XED file. This function will fail if the specified XED file already exists.
CREATE_ALWAYS	Creates a new XED file. If the specified XED file already exists, this function will overwrite it.
OPEN_EXISTING	Opens an existing XED file. This function will fail if the specified XED file does not exist. If you pass GENERIC_READ to the <i>dwDesiredAccess</i> parameter, you must pass OPEN_EXISTING to this parameter.

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams that you want to be able to record to the specified XED file. Only streams that you specify when you create the file can be recorded to the file. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#). If you pass GENERIC_READ to the *dwDesiredAccess* parameter, you must pass XSTUDIO_STREAM_FLAG_NONE to this parameter.

phFile
[out] Pointer to an [XSTUDIO_FILE_HANDLE](#) that receives an opaque handle to the XED file. This handle can be used to perform operations on the file after it has been opened or created. When you no longer need the file, call the [XStudioCloseFile](#) function to close the handle.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the XED file was successfully opened or created; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070003	E_XSTUDIO_PATH_NOT_FOUND	XStudio cannot find the path specified by <i>sFilePath</i> .
0x800700B7	E_XSTUDIO_FILE_ALREADY_EXISTS	You passed GENERIC_WRITE to <i>dwDesiredAccess</i> and CREATE_NEW to <i>dwCreationDisposition</i> , but the file specified by <i>sFilePath</i> already exists.
0x83100002	E_XSTUDIO_PROTOCOL_MISMATCH	The XED file specified by <i>sFilePath</i> was created using an incompatible version of XStudio.
0x83100007	E_XSTUDIO_FILE_CONVERSION_REQUIRED	The XED file specified by <i>sFilePath</i> needs to be updated to the current version of XStudio.

Remarks

The value of the *sFileName* parameter determines whether the XED file is created on the computer or on the development console. If you specify a full path that begins with "GAME:\\" or "DEVKIT:\\", the file is created on the development console. If you specify a full path with any other drive specification, the file is created on the computer. If you specify a relative path, the file is created on the same platform from which this function was called.

Important

If you are creating a file on the computer, you must first establish a connection to a development console. The method to create a connection depends on the platform from which you call this function. If you call this function from the computer, you must call the [XStudioConnect](#) function to make the connection. If you call this function from the development console, you can connect either by using the [Xbox Studio](#) tool or by calling the [XStudioConnect](#) function from the computer. Development consoles can create a local

file without a connection.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioCloseFile](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioDisconnect

[See Also](#)



Disconnects the connection from your computer to the development console.

Syntax

C++

```
HRESULT XStudioDisconnect()
```

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800704B1	E_XSTUDIO_NO_CONNECTION	A connection does not exist.

Remarks

This function is available only on the computer.

When you call this function, all files, callback handles, and suspend points that are open currently on the computer are closed automatically. Also, any streams that are mapped to the files or callback handles are unmapped automatically.

Important

Files on the development console are not closed automatically. Before you call this function, close any open handles to files on the console by calling the [XStudioCloseFile](#) function.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioConnect](#)
[XStudioGetConnectionStatus](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioGetActiveStreams

[See Also](#)



Retrieves a list of the streams that are active currently.

Syntax

C++

```
HRESULT XStudioGetActiveStreams(  
    XSTUDIO_STREAM_FLAGS *pdwStreams  
)
```

Parameters

pdwStreams

[out] Pointer to an [XSTUDIO_STREAM_FLAGS](#) type that receives a list of the streams that are active currently. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAGS Constants](#). For more information about active streams, see the Remarks section.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the streams were successfully retrieved; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070057	E_XSTUDIO_INVALID_ARG	The pointer passed to the <i>pdwStreams</i> parameter is invalid.

Remarks

Streams are considered to be active if they are playing back or are suspended. Streams become inactive when the [XStudioStop](#) function is called, when playback reaches the end of the stream, or when an error occurs.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

- ✦ See Also
- [XStudio Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioGetConnectionStatus

[See Also](#)

Determines whether your computer is connected to a development console.

✦ Syntax

C++

HRESULT XStudioGetConnectionStatus ()

Return Value
Returns S_XSTUDIO_SUCCESS (0x00000000) if your computer is connected to a development console currently; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800704B1	E_XSTUDIO_NO_CONNECTION	Your computer is not connected to a development console.

- ✦ Remarks
- This function is available only on the computer.

- ✦ Requirements
- Header:** Declared in XStudioApi.h.
Library: Use XStudio.lib.

- ✦ See Also
- [XStudio Functions](#)
[XStudioConnect](#)
[XStudioDisconnect](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioGetErrorStreams

[See Also](#)

Gets a list of the streams that have encountered an error.

✦ Syntax

C++

HRESULT XStudioGetErrorStreams (XSTUDIO_STREAM_FLAGS *pdwStreams)

Parameters
pdwStreams
[out] Pointer to an [XSTUDIO_STREAM_FLAGS](#) type that receives a list of the streams that have encountered an error. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#). For more information about active streams, see the Remarks section.

Return Value
Returns S_XSTUDIO_SUCCESS (0x00000000) if the streams were retrieved successfully; otherwise, an error code.

- ✦ Remarks
- The following list contains examples of conditions that can trigger a stream error:
- A corrupt event was read from an Xbox event data (XED) file during playback.
 - The file system to which the XED file was being recorded ran out of disk space.

- ✦ Requirements
- Header:** Declared in XStudioApi.h.
Library: Use XStudio.lib.

- ✦ See Also
- [XStudio Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.

Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioGetFileStreams

[See Also](#)



Gets a list of the streams that exist in the specified Xbox event data (XED) file.

Syntax

C++

```
HRESULT XStudioGetFileStreams(  
    XSTUDIO_FILE_HANDLE hFile,  
    XSTUDIO_STREAM_FLAGS *pdwStreams  
)
```

Parameters

hFile

[in] [XSTUDIO_FILE_HANDLE](#) to the XED file from which to get the list of streams. This handle was returned from the [XStudioCreateFile](#) function when you created or opened the file.

pdwStreams

[out] Pointer to an [XSTUDIO_STREAM_FLAGS](#) type that receives a list of the streams that exist in the specified XED file. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the streams were retrieved successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>hFile</i> is invalid.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioGetInputMode

[See Also](#)



Gets the event source for the specified stream.

Syntax

C++

```
HRESULT XStudioGetInputMode(  
    XSTUDIO_STREAM_FLAGS streamFlags,  
    XSTUDIO_INPUT_MODE *pInputModeOut  
)
```

Parameters

streamFlags

[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.

pInputModeOut

[out] Pointer to an [XSTUDIO_INPUT_MODE](#) value that receives the input mode for the specified stream. This parameter will return XSTUDIO_INPUT_MODE_INVALID if the specified stream is not mapped.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the input mode is retrieved successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x83100008	E_XSTUDIO_INVALID_STREAM	One or more of the specified data streams is invalid.

Remarks

When the input mode is set to XSTUDIO_INPUT_MODE_SIMULATED, your title will receive color and depth data from XStudio, regardless of whether data is being played back. Live streams from the Kinect for Xbox 360 sensor array are intercepted.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioSetInputMode](#)
[XSTUDIO_INPUT_MODE](#)
[XStudio Return Codes](#)

XStudioGetLoopCount

[See Also](#)

Gets the number of remaining loop iterations for playback of the specified streams.

Syntax

C++

```
HRESULT XStudioGetLoopCount(  
    XSTUDIO_STREAM_FLAGS streamFlags,  
    UINT *pLoopCount  
)
```

Parameters

streamFlags
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.

pLoopCount
[out] Pointer that receives the maximum number of remaining loop iterations for the specified streams. When looping is complete, the counter resets to the value that was passed to the *loopCount* parameter of the [XStudioSetLoopCount](#) function. If **XStudioSetLoopCount** has not been called, this value is zero.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

- [XStudio Functions](#)
- [XStudioSetLoopCount](#)
- [XStudio Return Codes](#)

XStudioMapStreams

[See Also](#)

Maps or unmaps the specified streams to the specified Xbox event data (XED) file or callback function.

Syntax

C++

```
HRESULT XStudioMapStreams(  
    XSTUDIO_STREAM_FLAGS dwStreams,  
    XSTUDIO_HANDLE hHandle  
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to map or unmap to the XED file or callback function. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).

If you are mapping streams to an XED file that is open currently for write access, you can map only streams that you passed to the *streamFlags* parameter of the [XStudioCreateFile](#) function.

hHandle
[in] Handle to the XED file or callback function to which to map or unmap the streams. If you are mapping or unmapping the streams to a file, this is the [XSTUDIO_FILE_HANDLE](#) that was returned from the [XStudioCreateFile](#) function. If you are mapping or unmapping the streams to a callback function, this is the [XSTUDIO_HANDLE](#) that was returned from the [XStudioRegisterStreamCallback](#) function.

To unmap the specified streams, pass NULL to this parameter. Playback or recording of the specified streams is halted. All other mapped streams are unaffected.

 **Important**

Stream mapping is atomic. If any stream cannot be mapped, none of the specified streams are mapped.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the streams are mapped or unmapped successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>hHandle</i> is invalid.
0x800710DD	E_XSTUDIO_INVALID_OPERATION	At least one stream was mapped for recording and one stream for playback, and the combination was not valid. See Playing and Recording Data Simultaneously [Xbox Studio] .
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified stream was not valid for the current type of access for the specified file.
0x83100009	E_XSTUDIO_STREAM_NOT_MAPPED	You tried to unmap a stream that isn't mapped.
0x8310000A	E_XSTUDIO_STREAM_ALREADY_MAPPED	You tried to map a stream that is already mapped.

Remarks

Read or Write Access

If you are mapping or unmapping the streams to an XED file, the access type that you passed to the [XStudioCreateFile](#) function determines whether events are written to or read from the file. If the file specified by the *hHandle* parameter is open currently with read access, the streams specified by the *streamFlags* parameter will be read from the file. If the file is open currently with write access, the streams will be recorded to the file.

Note

The player index stream cannot be mapped to a file that is opened with read access. If the player index stream is specified by itself, this function will return `E_XSTUDIO_INVALID_STREAM`. If it is specified along with other streams, it is ignored.

Producer or Consumer Callbacks

If you are mapping or unmapping the streams to a callback function, the registration flag that you passed to the [XStudioRegisterStreamCallback](#) function determines whether events are produced or consumed by the callback function. If the callback function specified by the *hHandle* parameter is a producer callback, the callback is expected to create events for the streams specified by the *streamFlags* parameter. If the callback function is a consumer callback, it will receive the events for the specified streams.

Requirements

Header: Declared in `XStudioApi.h`.

Library: Use `XStudio.lib`.

See Also

- [XStudio Functions](#)
- [NUI PC Conduit Sample](#)
- [XStudioUnmapStreams](#)
- [XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioReadTitleData

[See Also](#)



Reads a title data event from the title data stream.

Syntax

C++

```
HRESULT XStudioReadTitleData(
    DWORD dwMillisecondsToWait,
    XSTUDIO_TITLE_INDEX_DATA *pTitleIndexData,
    void *pTitleDataBuffer,
    DWORD dwTitleDataBufferSize,
    DWORD *pdwRequiredTitleDataBufferSize
)
```

Parameters

- dwMillisecondsToWait*
[in] Time in milliseconds that this function should wait for the next title data event before the function returns.
- pTitleIndexData*
[out, optional] Pointer to an [XSTUDIO_TITLE_INDEX_DATA](#) structure that will receive the custom index data for the event.
- pTitleDataBuffer*
[out, optional] Pointer that will receive the title data for the event. If you specify `NULL`, no title data is returned to the title. However, the *pdwRequiredTitleDataBufferSize* parameter still receives the required buffer size.
- dwTitleDataBufferSize*
[in] Size in bytes of the buffer pointed to by the *pTitleDataBuffer* parameter. This value must be less than 32 KB. If you pass zero to this parameter, the value of the *pTitleDataBuffer* parameter is ignored.
- pdwRequiredTitleDataBufferSize*
[out] Pointer that will receive the size of the buffer required, in bytes.

Return Value

Returns `S_XSTUDIO_SUCCESS` (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070103	E_XSTUDIO_NO_MORE_ITEMS	Playback has passed the last title data event, but other streams are still running.
0x800705B4	E_XSTUDIO_TIMEOUT	The timeout period specified by the <i>dwMillisecondsToWait</i> parameter has elapsed without any title data event being received.
0x83100006	E_XSTUDIO_NO_DATA_AVAILABLE	No data is being played back, or there is no data to read.

Remarks

This function is available only on the development console.
You can call this function multiple times for each event. However, title data will be updated only if a new title event exists in the title data stream.

Important

There is no synchronization between the title data stream and the NUI streams. Playing the title data stream does not affect the color, depth, and skeletal streams. Also, if the title data stream is blocked while waiting for the next title data event, the NUI streams will continue to play or record.

Requirements

Header: Declared in `XStudioApi.h`.

Library: Use `XStudio.lib`.

See Also

- [XStudio Functions](#)
- [XStudioWriteTitleData](#)
- [Title Data \(Xbox Studio\)](#)

[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioRegisterNotificationCallback

[See Also](#)



Registers an XStudio callback function that receives notifications when the state of an event stream changes.

Syntax

C++

```
HRESULT XStudioRegisterNotificationCallback(  
    XSTUDIO_NOTIFICATION_CALLBACK pfnNotificationCallback,  
    XSTUDIO_STREAM_FLAGS dwStreams,  
    DWORD dwNotificationsFlags,  
    DWORD dwFlags,  
    void *pvContext,  
    XSTUDIO_HANDLE *phandle  
)
```

Parameters

- pfnNotificationCallback*
[in] Pointer to the [XSTUDIO_NOTIFICATION_CALLBACK](#) callback function to register for the specified streams.
- dwStreams*
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams for which to receive notifications. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).
- dwNotificationsFlags*
[in] Bitwise-OR combination of [XSTUDIO_NOTIF](#) values that specifies the notifications for which to register.
- dwFlags*
[in] Reserved. Pass zero to this parameter.
- pvContext*
[in] Pointer to custom context data that is passed to the callback.
- phandle*
[out] Pointer to an [XSTUDIO_HANDLE](#) that receives an opaque handle to the callback function. When you no longer need the callback function, call the [XStudioCloseHandle](#) function to close the handle.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the callback function was successfully registered; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070001	E_XSTUDIO_INVALID_FUNCTION	The callback function pointed to by the <i>pfnNotificationCallback</i> parameter is invalid.
0x800703EC	E_XSTUDIO_INVALID_FLAGS	You did not pass zero to the reserved <i>dwFlags</i> parameter.
0x80070057	E_XSTUDIO_INVALID_ARG	The handle pointed to by the <i>phandle</i> parameter is not valid.
0x8310000D	E_XSTUDIO_NO_STREAM_SPECIFIED	No data streams are passed to the <i>dwStreams</i> parameter.

Remarks

This function is available only on the computer.

Note

Callbacks must be re-mapped after title termination. When working with Xbox Studio if a callback is registered and started, title termination will cause the callback to stop transmitting data. After title termination, the application on the PC using the callback should call XStudioDisconnect and XStudioConnect to clear its state. Once the title is restarted, the callback can be registered and started again.

Requirements

- Header:** Declared in XStudioApi.h.
- Library:** Use XStudio.lib.

See Also

[XStudio Functions](#)
[XSTUDIO_NOTIFICATION_CALLBACK](#)
[XSTUDIO_NOTIF Constants](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioRegisterStreamCallback

[See Also](#)



Registers an XStudio callback function that can be used to produce or consume NUI events.

Syntax

C++

```
HRESULT XStudioRegisterStreamCallback(  
    XSTUDIO_STREAM_CALLBACK pfnStreamCallback,  
    DWORD dwStreamCallbackRegistrationFlags,  
    void *pvStreamCallbackContext,  
    XSTUDIO_HANDLE *phStreamCallback  
)
```

Parameters

pfnStreamCallback
[in] Pointer to the [XSTUDIO_STREAM_CALLBACK](#) callback function to register.

dwStreamCallbackRegistrationFlags
[in] [XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG](#) value that specifies the type of callback to register.

**pvStreamCallbackContext*
[in, optional] Pointer to custom context data that is passed to the callback.

phStreamCallback
[out] Pointer to an [XSTUDIO_HANDLE](#) that recieves an opaque handle to the callback function. When you no longer need the callback function, call the [XStudioCloseHandle](#) function to close the handle.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the callback function was successfully registered; otherwise, an error code.

Remarks

This function is available only on the computer.

Note

Callbacks must be re-mapped after title termination. When working with Xbox Studio if a callback is registered and started, title termination will cause the callback to stop transmitting data. After title termination, the application on the PC using the callback should call [XStudioDisconnect](#) and [XStudioConnect](#) to clear its state. Once the title is restarted, the callback can be registered and started again.

Creating Producer and Consumer Callbacks

You can use XStudio callbacks to create your own producers and consumers of NUI events. It is not possible to use the XStudio stream connections to register both a Producer and a Consumer connection to an Xbox 360 Console at the same time. The application will behave as expected if only one of the connections is registered.

Note

The title data stream is in big-endian format on the development console, but is in little-endian format on the computer. If your callback consumes or produces the title data stream, the callback must perform the required byte swapping.

Consumer Callbacks

If you register a *consumer callback*, the XStudio service will call the callback whenever it receives an event from the Kinect sensor array. Your callback can then use the data in the event structure. Consumer callbacks can be mapped for all streams, including the title data stream.

Producer Callbacks

If you register a *producer callback*, the XStudio service will continuously call your callback, sending empty events for each mapped stream. Your callback should populate the members of the event structure. When the producer callback exits, the populated events will be sent to the event consumer at a rate that is determined by the timestamp that you set for the events. If you register your producer callback with the XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_REALTIME flag, events will be delivered to your title without any timing from the system. Producer callbacks can be mapped for all streams except for the identity and player index streams.

If you register a *fill-in producer callback*, the XStudio service will not invoke your callback until playback of an Xbox event data (XED) file is completed. At that point, the service will continuously call your callback, sending empty events for each mapped stream. Your callback should populate the members of the event structure. When a fill-in producer is active, you can call the [XStudioSwitchToProducers](#) function to start another file playback without disrupting the currently active streams.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[NUI PC Conduit Sample](#)
[XSTUDIO_STREAM_CALLBACK](#)
[XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG Constants](#)
[XStudioSwitchToProducers](#)
[XStudioCloseHandle](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioResume

[See Also](#)

Resumes playback of the specified suspended streams.

Syntax

C++

```
HRESULT XStudioResume(  
    XSTUDIO_STREAM_FLAGS streamFlags  
)
```

Parameters

streamFlags
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the specified streams are resumed successfully, or if the streams are already playing back; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800710DD	E_XSTUDIO_INVALID_OPERATION	One or more of the specified streams currently is open for recording instead of for playback.
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.

Remarks

Playback is suspended by calling the [XStudioSuspend](#) function. When playback is suspended, the time cursor is paused, and no event data is played back.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioAddSuspendPoint](#)
[XStudioSingleStep](#)
[XStudioSuspend](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioSetInputMode

[See Also](#)

Sets the event source for the specified streams.

Syntax

C++

```
HRESULT XStudioSetInputMode(  
    XSTUDIO_STREAM_FLAGS streamFlags,  
    XSTUDIO_INPUT_MODE InputMode  
)
```

Parameters

streamFlags
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.
InputMode
[in] [XSTUDIO_INPUT_MODE](#) value that specifies the input mode to set for the specified streams.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the input mode is set successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070057	E_XSTUDIO_INVALID_ARG	The specified input mode is invalid.
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.

Remarks

When the input mode is set to XSTUDIO_INPUT_MODE_SIMULATED, your title will receive color and depth data from XStudio, regardless of whether data is being played back. Live streams from the Kinect for Xbox 360 sensor array are intercepted.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioGetInputMode](#)
[XSTUDIO_INPUT_MODE](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioSetLoopCount

[See Also](#)

Sets the number of loop iterations for playback of all mapped streams.

Syntax

C++

```
HRESULT XStudioSetLoopCount(  
    XSTUDIO_STREAM_FLAGS streamFlags,  
    UINT loopCount  
)
```

Parameters

streamFlags
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.
loopCount
[in] Number of times to loop the streams. Specify zero to disable looping (one playthrough and stop). Specify INFINITE to loop indefinitely.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
------	------	-------------

0x800710DD	E_XSTUDIO_INVALID_OPERATION	You called this function after playback is started.
0x83100008	E_XSTUDIO_INVALID_STREAM	One or more of the streams specified by the <i>streamFlags</i> parameter are invalid.

Remarks

To enable looping, you must call this function before starting playback of the mapped streams, and before any mapped streams become active.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioGetLoopCount](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioSingleStep

[See Also](#)



Advances playback of the specified streams by a single event.

Syntax

```
C++
HRESULT XStudioSingleStep(
    XSTUDIO_STREAM_FLAGS dwStreams
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to advance. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x83100008	E_XSTUDIO_INVALID_STREAM	One or more of the specified data streams are invalid.
0x83100012	E_XSTUDIO_NOT_SUSPENDED	You tried to single-step streams that are not suspended currently.

Remarks

This function is available only on the computer.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioAddSuspendPoint](#)
[XStudioResume](#)
[XStudioSuspend](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioStart

[See Also](#)



Starts recording or playback of the specified streams.

Syntax

```
C++
HRESULT XStudioStart(
    XSTUDIO_STREAM_FLAGS dwStreams
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to start. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#). You must specify at least one stream that has been mapped to the file through a call to the [XStudioMapStreams](#) function. You can specify XSTUDIO_STREAM_FLAG_ALL to start all mapped streams.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the specified streams start successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800700AA	E_XSTUDIO_BUSY	You tried to start a stream when a different stream was already started.
0x8310000D	E_XSTUDIO_NO_STREAM_SPECIFIED	No data stream was specified.
0x83100013	E_XSTUDIO_RECORD_WHILE_SUSPENDED	You tried to record while playback is suspended.

Remarks

If playback is stopped currently, this function resets the playback cursor to the beginning of the stream. If the specified streams currently are playing back, this function has no effect. You cannot start a stream if a different stream is running already. You can always start or stop the title data stream, regardless of the status of the other streams.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioStop](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioStop

[See Also](#)



Stops recording or playback of the specified streams.

Syntax

```
C++
HRESULT XStudioStop(
    XSTUDIO_STREAM_FLAGS dwStreams
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to stop. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#). You must specify at least one stream that has been mapped to the file through a call to the [XStudioMapStreams](#) function. You can specify XSTUDIO_STREAM_FLAG_ALL to stop all mapped streams. It is not an error to try to stop a mapped stream that is not being recorded currently or being played back.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the specified streams were successfully stopped; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x83100009	E_XSTUDIO_STREAM_NOT_MAPPED	None of the specified data streams are mapped.
0x8310000D	E_XSTUDIO_NO_STREAM_SPECIFIED	A data stream was not specified.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioStart](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioStopEx

[See Also](#)



Stops an Xbox event data (XED) file or stream callback from producing or consuming events for the specified streams.

Syntax

```
C++
HRESULT XStudioStopEx(
    XSTUDIO_STREAM_FLAGS dwStreams,
    XSTUDIO_HANDLE handle
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to stop. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#). You must specify at least one stream that has been mapped to the file through a call to the [XStudioMapStreams](#) function. You can specify XSTUDIO_STREAM_FLAG_ALL to stop all mapped streams. It is not an error to try to stop a mapped stream that is not being produced or consumed by a callback.

handle
[in] Handle to the XED file or callback function to stop producing or consuming streams. If you are stopping an XED file, this is the [XSTUDIO_FILE_HANDLE](#) that was returned from the

[XStudioCreateFile](#) function. If you are stopping a stream callback function, this is the [XSTUDIO_HANDLE](#) that was returned from the [XStudioRegisterStreamCallback](#) function.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the specified streams were successfully stopped; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>handle</i> is invalid.
0x83100009	E_XSTUDIO_STREAM_NOT_MAPPED	None of the specified data streams are mapped.
0x8310000D	E_XSTUDIO_NO_STREAM_SPECIFIED	A data stream was not specified.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioRegisterStreamCallback](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioSuspend

[See Also](#)

Suspends playback of the specified streams.

Syntax

```
C++
HRESULT XStudioSuspend(
    XSTUDIO_STREAM_FLAGS streamFlags
)
```

Parameters

streamFlags
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the specified streams are suspended successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800710DD	E_XSTUDIO_INVALID_OPERATION	One or more of the specified streams currently is not playing.
0x83100014	E_XSTUDIO_SUSPEND_WHILE_RECORDING	You tried to suspend while recording is in progress.
0x83100015	E_XSTUDIO_SUSPEND_WITHOUT_PLAYING	You tried to suspend streams that are not playing currently.

Remarks

When playback is suspended, the time cursor is paused, and no event data is played back. To resume playback, call the [XStudioResume](#) function.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioAddSuspendPoint](#)
[XStudioResume](#)
[XStudioSingleStep](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioSwitchToProducers

[See Also](#)

Sets the event producers for the specified streams to the currently mapped Xbox event data (XED) file.

Syntax

```
C++
HRESULT XStudioSwitchToProducers(
    XSTUDIO_STREAM_FLAGS dwStreams,
    XSTUDIO_PRODUCER producer
)
```

Parameters

dwStreams
[in] Must be XSTUDIO_STREAM_FLAG_DEFAULT.

producer
[in] Must be XSTUDIO_PRODUCER_FILE.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the producers for the specified streams are set successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x800710DD	E_XSTUDIO_INVALID_OPERATION	The XED file that is mapped currently does not have the same streams mapped that are active currently for the fill-in producer.
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.

Remarks

This function is available only on the computer.
You can call this function only if the fill-in producer is active currently. The XED file to which you are switching must have the same streams mapped that are active currently for the fill-in producer.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioRegisterStreamCallback](#)
[XSTUDIO_PRODUCER](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioUnmapStreams

[See Also](#)



Unmaps the specified streams from the specified Xbox event data (XED) file or callback function.

Syntax

```
C++
HRESULT XStudioUnmapStreams(
    XSTUDIO_STREAM_FLAGS dwStreams,
    XSTUDIO_HANDLE hHandle
)
```

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to unmap from the XED file or callback function. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).
hHandle
[in] Handle to the XED file or callback function from which to unmap the streams. If you are unmapping the streams from a file, this is the [XSTUDIO_FILE_HANDLE](#) that was returned from the [XStudioCreateFile](#) function. If you are unmapping the streams from a callback function, this is the [XSTUDIO_HANDLE](#) that was returned from the [XStudioRegisterStreamCallback](#) function.
Playback or recording of the specified streams is halted. All other mapped streams are unaffected.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if the streams are unmapped successfully; otherwise, an error code. The following table lists some of the possible error codes.

Code	Name	Description
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle specified by <i>hHandle</i> is invalid.
0x83100009	E_XSTUDIO_STREAM_NOT_MAPPED	You tried to unmap a stream that isn't mapped.

Remarks

This function unmaps the specified streams from the XED file or callback function, but leaves streams mapped to other handles intact.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Functions](#)
[XStudioMapStreams](#)
[XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudioWriteTitleData

[See Also](#)



Writes a title data event to the title data stream.

Syntax

C++

```
HRESULT XStudioWriteTitleData(  
    const XSTUDIO_TITLE_INDEX_DATA *pTitleIndexData,  
    const void *pTitleDataBuffer,  
    DWORD dwTitleDataBufferSize  
)
```

Parameters

- pTitleIndexData*
[in, optional] Pointer to an [XSTUDIO_TITLE_INDEX_DATA](#) structure that contains custom index data for the event.
- pTitleDataBuffer*
[in, optional] Pointer to a buffer that contains the title data to write for the event.
- dwTitleDataBufferSize*
[in] Size in bytes of the buffer pointed to by the *pTitleDataBuffer* parameter. This value must be less than 32 KB. If you pass NULL to the *pTitleDataBuffer* parameter, this value must be zero.

Return Value

Returns S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code.

Remarks

This function is available only on the development console.
You can call this function multiple times for each frame.
Every event automatically receives a timestamp when it is recorded. Applications on your development computer can read this timestamp by calling the [XedReadTitleData](#) function.

Requirements

- Header:** Declared in XStudioApi.h.
- Library:** Use XStudio.lib.

See Also

- [XStudio Functions](#)
- [XStudioReadTitleData](#)
- [Title Data | Xbox Studio](#)
- [XStudio Return Codes](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Structures

[See Also](#)



This topic contains the reference pages for the structures that are defined in the header file XStudio.h.

In This Section

- [XSTUDIO_NUICAM_IMAGE_INDEX_DATA](#)
Contains information about an event frame.
- [XSTUDIO_STREAM_EVENT](#)
Represents an XStudio event for a specific data stream.
- [XSTUDIO_TITLE_INDEX_DATA](#)
Custom title index data structure that can be used to provide custom search capabilities on the title data stream.

See Also

- [XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_NUICAM_IMAGE_INDEX_DATA Structure

[See Also](#)



Contains information about an event frame.

Syntax

C++

```
typedef struct _XSTUDIO_NUICAM_IMAGE_INDEX_DATA {  
    NUI_IMAGE_TYPE ImageType;  
    USHORT Width;  
    USHORT Height;  
    DWORD FrameNumber;  
    DWORD64 NuiTimeStamp;  
} XSTUDIO_NUICAM_IMAGE_INDEX_DATA;
```

Members

- ImageType**
[NUI_IMAGE_TYPE](#) value that specifies the type of the event.
- Width**
Width of the event frame.
- Height**
Height of the event frame.
- FrameNumber**
Frame number of the event frame.

NuiTimeStamp
Timestamp of the event frame, in milliseconds.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Structures](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_EVENT Structure

[See Also](#)

Represents an XStudio event for a specific data stream.

Syntax

```
C++

typedef struct _XSTUDIO_STREAM_EVENT {
    UINT cbSize;
    XSTUDIO_STREAM_ID StreamID;
    UINT uEventIndex;
    UINT64 uEventMicroseconds;
    void *pvIndexBuffer;
    UINT cbIndexBufferSize;
    UINT cbIndexBufferUsed;
    void *pvDataBuffer;
    UINT cbDataBufferSize;
    UINT cbDataBufferUsed;
} XSTUDIO_STREAM_EVENT;
```

Members

cbSize
Size of this structure, in bytes.

StreamID
[XSTUDIO_STREAM_ID](#) value that specifies the type of the stream for the event.

uEventIndex
Index of the event.

uEventMicroseconds
Timestamp of the event, in microseconds.

pvIndexBuffer
Pointer to an [XSTUDIO_NUICAM_IMAGE_INDEX_DATA](#) structure that contains information about the event frame.

cbIndexBufferSize
Size of the buffer pointed to by the **pvIndexBuffer** member, in bytes.

cbIndexBufferUsed
Number of bytes actually used in the buffer pointed to by the **pvIndexBuffer** member.

pvDataBuffer
Pointer to the data buffer for the event. The format of the data buffer depends on the type of the stream, as shown in the following table.

streamId value	Data format
XSTUDIO_STREAM_ID_NUICAM_COLOR	640×480 buffer of BGRX linear format (D3DFMT_LIN_B8G8R8X8)
XSTUDIO_STREAM_ID_NUICAM_DEPTH	320×240 buffer of USHORT (D3DFMT_LIN_D16)
XSTUDIO_STREAM_ID_NUIAPI_SKELETON	NUI_SKELETON_FRAME

cbDataBufferSize
Size of the buffer pointed to by the **pvDataBuffer** member, in bytes.

cbDataBufferUsed
Number of bytes actually used in the buffer pointed to by the **pvDataBuffer** member.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Structures](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_TITLE_INDEX_DATA Structure

[See Also](#)

Custom title index data structure that can be used to provide custom search capabilities on the title data stream.

Syntax

```
C++
```

```
typedef struct _XSTUDIO_TITLE_INDEX_DATA {
    UINT64 Reserved;
    DWORD IndexData[4];
} XSTUDIO_TITLE_INDEX_DATA;
```

Members

Reserved
Reserved; set to zero.

IndexData
Custom title index data.

Remarks

When you use the [XedFile Reference](#) APIs to access an Xbox event data (XED) file, you can use the index data to locate an event by mapping the index data to the event index.

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

See Also

[XStudio Structures](#)
[Title Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Enumerations

[See Also](#)

This topic contains the reference pages for the enumerations that are defined in the header file XStudio.h.

In This Section

[XSTUDIO_CONNECTION_TYPE](#)
Defines the types of connection that can be used between the computer and the development console.

[XSTUDIO_INPUT_MODE](#)
Defines the event sources that are used by XStudio.

[XSTUDIO_PRODUCER](#)
Defines the event producers for the XStudio APIs.

[XSTUDIO_STREAM_ID](#)
Defines the types of stream that can be used by the XStudio functions.

See Also

[XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_CONNECTION_TYPE Enumeration

[See Also](#)

Defines the types of connection that can be used between the computer and the development console.

Syntax

C++

```
typedef enum _XSTUDIO_CONNECTION_TYPE
{
    XSTUDIO_CONNECTION_TYPE_USB,
    XSTUDIO_CONNECTION_TYPE_TCP
} XSTUDIO_CONNECTION_TYPE;
```

Constants

Constant	Description
XSTUDIO_CONNECTION_TYPE_USB	Connect to the development console over a USB easy transfer cable.
XSTUDIO_CONNECTION_TYPE_TCP	Connect to the development console over TCP/IP.

Remarks

This enumeration is available only on the computer.

See Also

[XStudio Enumerations](#)
[Connecting to a Development Console \[Xbox Studio\]](#)
[XStudioConnect](#)

Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_INPUT_MODE Enumeration

[See Also](#)



Defines the event sources that are used by XStudio.

Syntax

C++

```
typedef enum _XSTUDIO_INPUT_MODE
{
    XSTUDIO_INPUT_MODE_DEFAULT,
    XSTUDIO_INPUT_MODE_SIMULATED,
    XSTUDIO_INPUT_MODE_INVALID
} XSTUDIO_INPUT_MODE;
```

Constants

Constant	Description
XSTUDIO_INPUT_MODE_DEFAULT	Stream input comes from the original source.
XSTUDIO_INPUT_MODE_SIMULATED	Stream input is redirected to come from XStudio. For more information, see XStudioSetInputMode .
XSTUDIO_INPUT_MODE_INVALID	When returned from the XStudioGetInputMode function, this value indicates that the specified stream is not mapped.

See Also

- [XStudio Enumerations](#)
- [XStudioGetInputMode](#)
- [XStudioSetInputMode](#)

Requirements

- Header:** Declared in XStudioApi.h.
- Library:** Use XStudio.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_PRODUCER Enumeration

[See Also](#)



Defines the event producers for the XStudio APIs.

Syntax

C++

```
typedef enum XSTUDIO_PRODUCER
{
    XSTUDIO_PRODUCER_FILE
} XSTUDIO_PRODUCER;
```

Constants

Constant	Description
XSTUDIO_PRODUCER_FILE	Events are produced from an Xbox event data (XED) file.

See Also

- [XStudio Enumerations](#)
- [XStudioSwitchToProducers](#)
- [XStudioUnmapStreams](#)

Requirements

- Header:** Declared in XStudioApi.h.
- Library:** Use XStudio.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_ID Enumeration

[See Also](#)



Defines the types of stream that can be used by the XStudio functions.

Syntax

C++

```
typedef enum _XSTUDIO_STREAM_ID
{
    XSTUDIO_STREAM_ID_NUICAM_BASE,
    XSTUDIO_STREAM_ID_NUICAM_DEPTH,
    XSTUDIO_STREAM_ID_NUICAM_COLOR,
    XSTUDIO_STREAM_ID_NUIAPI_BASE,
    XSTUDIO_STREAM_ID_NUIAPI_SKELETON,
    XSTUDIO_STREAM_ID_NUIAPI_IDENTITY,
    XSTUDIO_STREAM_ID_NUIAPI_PLAYER_INDEX,
    XSTUDIO_STREAM_ID_TITLE_BASE,
    XSTUDIO_STREAM_ID_TITLE_DATA,
    XSTUDIO_STREAM_ID_MISC_PRIVATE,
    XSTUDIO_STREAM_ID_COUNT,
    XSTUDIO_STREAM_ID_INVALID
} XSTUDIO_STREAM_ID;
```

Constants

Constant	Description
XSTUDIO_STREAM_ID_NUICAM_BASE	Utility member. Do not use.
XSTUDIO_STREAM_ID_NUICAM_DEPTH	Depth stream from the Kinect sensor array.
XSTUDIO_STREAM_ID_NUICAM_COLOR	Color stream from the Kinect sensor array.
XSTUDIO_STREAM_ID_NUIAPI_BASE	Utility member. Do not use.
XSTUDIO_STREAM_ID_NUIAPI_SKELETON	Skeletal stream from NuiAPI .
XSTUDIO_STREAM_ID_NUIAPI_IDENTITY	Identity stream from NuiAPI .
XSTUDIO_STREAM_ID_NUIAPI_PLAYER_INDEX	Depth stream from the Kinect sensor array, combined with an image generated by the skeleton tracking system that labels pixels in the depth map as belonging to specific players.
XSTUDIO_STREAM_ID_TITLE_BASE	Utility member. Do not use.
XSTUDIO_STREAM_ID_TITLE_DATA	Title data stream.
XSTUDIO_STREAM_ID_MISC_PRIVATE	Utility member. Do not use.
XSTUDIO_STREAM_ID_COUNT	Utility member indicating the current number of members in this enumeration.
XSTUDIO_STREAM_ID_INVALID	Invalid stream.

See Also

- [XStudio Enumerations](#)
- [XSTUDIO_STREAM_FLAGS](#)
- [Title Data \(Xbox Studio\)](#)

Requirements

- Header:** Declared in XStudioService.h.
- Library:** Use XStudio.lib.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Callbacks

[See Also](#)



This topic contains the reference pages for the callback functions that are defined in the header file XStudio.h.

In This Section

- [XSTUDIO_NOTIFICATION_CALLBACK](#)
Definition for a callback function that is invoked from the XStudio service whenever the state of a stream changes.
- [XSTUDIO_STREAM_CALLBACK](#)
Definition for a callback function that can be used to produce or consume NUI events.

See Also

- [XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_NOTIFICATION_CALLBACK Callback

[See Also](#)



Definition for a callback function that is invoked from the XStudio service whenever the state of a stream changes.

Syntax

C++

```
typedef HRESULT (*XSTUDIO_NOTIFICATION_CALLBACK) (
    XSTUDIO_STREAM_FLAGS dwStreams,
    DWORD dwNotification,
    void *pvContext,
    LPARAM lParam
```


)

Parameters

dwStreams
[in] [XSTUDIO_STREAM_FLAGS](#) type that specifies the streams to receive notifications for. This is a bitwise-OR combination of the flags that are described at [XSTUDIO_STREAM_FLAG Constants](#).

dwNotification
[in] [XSTUDIO_NOTIF](#) value that specifies the type of the notification.

pvContext
[in, optional] Pointer to custom context data. This is the pointer that was passed to the *pvContext* parameter of the [XStudioRegisterNotificationCallback](#) function.

lparam
[in] More information about the notification. The valid values depend on the type of the notification. For more information about possible values, see the descriptions of the [XSTUDIO_NOTIF](#) values.

Return Value

Return S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code.

Remarks

You can register a notification callback with XStudio by calling the [XStudioRegisterNotificationCallback](#) function.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Callbacks](#)
[XStudioRegisterNotificationCallback](#)
[XSTUDIO_NOTIF Constants](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_CALLBACK Callback

[See Also](#)



Definition for a callback function that can be used to produce or consume NUI events.

Syntax

C++

```
typedef HRESULT (*XSTUDIO_STREAM_CALLBACK) (  
    XSTUDIO_STREAM_EVENT *pStreamEvent,  
    void *pvStreamCallbackContext  
)
```

Parameters

pStreamEvent
[in, out] Pointer to an [XSTUDIO_STREAM_EVENT](#) structure that contains information about a NUI event.

pvStreamCallbackContext
[in, optional] Pointer to custom context data. This is the pointer that was passed to the *pvStreamCallbackContext* parameter of the [XStudioRegisterStreamCallback](#) function.

Return Value

Return S_XSTUDIO_SUCCESS (0x00000000) if successful; otherwise, an error code.

Remarks

You can register a stream callback with XStudio by calling the [XStudioRegisterStreamCallback](#) function.

Requirements

Header: Declared in XStudioApi.h.
Library: Use XStudio.lib.

See Also

[XStudio Callbacks](#)
[NUI PC Conduit Sample](#)
[XStudioRegisterStreamCallback](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Constants

[See Also](#)



This topic contains the reference pages for the constants that are defined in the header file XStudio.h.

In This Section

[XSTUDIO_NOTIF Constants](#)
Constant values that define the XStudio notification types.

[XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG Constants](#)
Constant values that define the type of stream callback being registered.

[XSTUDIO_STREAM_FLAG Constants](#)
Constant values that can be combined to specify the streams to record or play back.

✚ See Also

[XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_NOTIF Constants

[See Also](#)



Constant values that define the XStudio notification types.	
Value	Description
XSTUDIO_NOTIF_PLAYBACK_STARTING	Playback is starting. No events have been delivered yet.
XSTUDIO_NOTIF_PLAYBACK_STOPPED	Playback is stopping. No more events will be delivered.
XSTUDIO_NOTIF_RECORDING_STARTING	Recording is starting. No events have been delivered yet.
XSTUDIO_NOTIF_RECORDING_STOPPED	Recording is stopping. No more events will be delivered.
XSTUDIO_NOTIF_SUSPENDED	Playback is suspended.
XSTUDIO_NOTIF_RESUMING	Playback is resumed.
XSTUDIO_NOTIF_LOOPING	A loop iteration has completed.
XSTUDIO_NOTIF_SWITCHING_TO_PROD	The XStudio service is starting to switch to a new producer of NUI events. If this value is passed to the <i>dwNotification</i> parameter of an XSTUDIO_NOTIFICATION_CALLBACK callback function, the <i>lparam</i> parameter contains an XSTUDIO_PRODUCER value that specifies the new producer. This notification is only triggered after calling XStudioSwitchToProducers (to switch from a "fill-in" producer to a "file" producer). When this notification is triggered, it reports the event for all streams, not only the streams for which you are registered.
XSTUDIO_NOTIF_SWITCHED_TO_PROD	The XStudio service has finished switching to a new producer of NUI events. If this value is passed to the <i>dwNotification</i> parameter of an XSTUDIO_NOTIFICATION_CALLBACK callback function, the <i>lparam</i> parameter contains an XSTUDIO_PRODUCER value that specifies the new producer.

✚ See Also

[XStudio Constants](#)
[XStudioRegisterNotificationCallback](#)
[XSTUDIO_PRODUCER](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG Constants

[See Also](#)



Constant values that define the type of stream callback being registered.	
Value	Description
XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_CONSUMER	Registers a callback that can be used to consume NUI events.
XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_PRODUCER	Registers a callback that can be used to produce NUI events.
XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_PRODUCER_FILL_IN	Registers a fill-in callback that can be used to produce "idling" NUI events for streams. The fill-in producer is invoked when a played-back Xbox event data (XED) file reaches the end of all mapped streams.
XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_REALTIME	When combined with XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_PRODUCER by using bitwise-OR, delivers events from a producer callback as fast as possible, without any delayed timing between events.

✚ See Also

[XStudio Constants](#)
[XStudioRegisterStreamCallback](#)
[XStudioUnmapStreams](#)
[XSTUDIO_STREAM_CALLBACK](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_FLAG Constants

[See Also](#)



Constant values that can be combined to specify the streams to record or play back.	
Value	Description
XSTUDIO_STREAM_FLAG_NONE	No streams.
XSTUDIO_STREAM_FLAG_DEFAULT	All streams.

XSTUDIO_STREAM_FLAG_NUICAM_DEPTH	The depth image stream from the Kinect sensor array.
XSTUDIO_STREAM_FLAG_NUICAM_COLOR	The color image stream from the Kinect sensor array.
XSTUDIO_STREAM_FLAG_NUICAM_ALL	All streams from the Kinect sensor array. That is, the depth image stream and color image stream.
XSTUDIO_STREAM_FLAG_NUIAPI_SKELETON	The skeleton stream from NuiAPI .
XSTUDIO_STREAM_FLAG_NUIAPI_IDENTITY	The identity stream from NuiAPI .
XSTUDIO_STREAM_FLAG_NUIAPI_PLAYER_INDEX	The player index stream from NuiAPI .
XSTUDIO_STREAM_FLAG_NUIAPI_ALL	All streams from NuiAPI . That is, the skeleton stream, the identity stream, and the player index stream.
XSTUDIO_STREAM_FLAG_TITLE_DATA	The title data stream.
XSTUDIO_STREAM_FLAG_TITLE_ALL	All title streams. That is, the title data stream.
XSTUDIO_STREAM_FLAG_ALL	Equivalent to <code>(XSTUDIO_STREAM_FLAG_NUICAM_ALL XSTUDIO_STREAM_FLAG_NUIAPI_ALL XSTUDIO_STREAM_FLAG_TITLE_ALL)</code> .

▣ **Remarks**

These values should be combined using bitwise-OR.

▣ **See Also**

[XStudio Constants](#)
[Title Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Types

[See Also](#)



This section contains reference information for the XStudio types.

In This Section

[XSTUDIO_FILE_HANDLE](#)

Represents an opaque handle to an Xbox event data (XED) file.

[XSTUDIO_HANDLE](#)

Represents an opaque handle to an XStudio callback function.

[XSTUDIO_STREAM_FLAGS](#)

Represents a bitwise-OR combination of the constants described at [XSTUDIO_STREAM_FLAG Constants](#).

▣ **See Also**

[XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_FILE_HANDLE Field

[See Also](#)



Represents an opaque handle to an Xbox event data (XED) file.

▣ **Syntax**

C++

```
typedef unsigned int XSTUDIO_FILE_HANDLE
```

▣ **Remarks**

On the development console, the maximum number of open file handles is four.

▣ **Requirements**

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

▣ **See Also**

[XStudio Types](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_HANDLE Field

[See Also](#)



Represents an opaque handle to an XStudio callback function.

 Syntax

C++

```
typedef unsigned int XSTUDIO_HANDLE
```

 Remarks

On the computer, the maximum number of open handles (including file handles) is 32.

 Requirements

Header: Declared in XStudioService.h.

Library: Use XStudio.lib.

 See Also

[XStudio Types](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XSTUDIO_STREAM_FLAGS Field

[See Also](#)



Represents a bitwise-OR combination of the constants described at [XSTUDIO_STREAM_FLAG Constants](#).

 Syntax

C++

```
typedef DWORD XSTUDIO_STREAM_FLAGS
```

 Requirements

Header: Declared in XStudioApi.h.

Library: Use XStudio.lib.

 See Also

[XStudio Types](#)

[XSTUDIO_STREAM_FLAG Constants](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XStudio Return Codes

[See Also](#)



The following is a list of codes returned from the [XStudio](#) APIs.

Code	Name	Description
0x00000000	S_XSTUDIO_SUCCESS	The operation completed successfully.
0x03100001	S_XSTUDIO_STOPPED	Playback of XStudio data was stopped.
0x03100002	S_XSTUDIO_NO_PLAYER_INDEX	XStudio is using the XSTUDIO_STREAM_ID_NUIAPI_PLAYER_INDEX depth image type, but no player index is available for the depth event.
0x80004005	E_XSTUDIO_INTERNAL	XStudio encountered an internal error.
0x80070001	E_XSTUDIO_INVALID_FUNCTION	The specified function is invalid.
0x80070002	E_XSTUDIO_FILE_NOT_FOUND	XStudio cannot find the specified file.
0x80070003	E_XSTUDIO_PATH_NOT_FOUND	XStudio cannot find the specified path.
0x80070004	E_XSTUDIO_TOO_MANY_OPEN_FILES	XStudio cannot open the file.
0x80070005	E_XSTUDIO_ACCESS_DENIED	Network access is denied.
0x80070006	E_XSTUDIO_INVALID_HANDLE	The handle is invalid.
0x8007000D	E_XSTUDIO_INVALID_DATA	The data is invalid.
0x8007000E	E_XSTUDIO_OUT_OF_MEMORY	Not enough storage is available to complete the operation.
0x80070020	E_XSTUDIO_SHARING_VIOLATION	XStudio cannot access the file because it is being used by another process.
0x80070057	E_XSTUDIO_INVALID_ARG	An invalid parameter was passed to the function.
0x800700AA	E_XSTUDIO_BUSY	The requested resource is in use.
0x800700B7	E_XSTUDIO_FILE_ALREADY_EXISTS	The specified file already exists.
0x800700EA	E_XSTUDIO_MORE_DATA	More data is available.
0x80070103	E_XSTUDIO_NO_MORE_ITEMS	No more data is available.
0x800703EC	E_XSTUDIO_INVALID_FLAGS	The specified flags are invalid.
0x800704B1	E_XSTUDIO_NO_CONNECTION	XStudio is not connected.

0x800705B4	E_XSTUDIO_TIMEOUT	The operation returned because the timeout period expired.
0x800710DD	E_XSTUDIO_INVALID_OPERATION	The specified operation is invalid.
0x83100002	E_XSTUDIO_PROTOCOL_MISMATCH	The specified Xbox event data (XED) file was created using an incompatible version of XStudio.
0x83100003	E_XSTUDIO_TITLE_TERMINATE	XStudio was terminated by the title.
0x83100004	E_XSTUDIO_NUI_NOT_INITIALIZED	The Kinect sensor array has not been initialized.
0x83100005	E_XSTUDIO_HEADLESS_MODE	XStudio is running in headless mode. This means that no Kinect sensor array is attached to the development console.
0x83100006	E_XSTUDIO_NO_DATA_AVAILABLE	No data is available.
0x83100007	E_XSTUDIO_FILE_CONVERSION_REQUIRED	The specified XED file needs to be updated to the current version of XStudio.
0x83100008	E_XSTUDIO_INVALID_STREAM	The specified data stream is invalid.
0x83100009	E_XSTUDIO_STREAM_NOT_MAPPED	The specified data stream is not mapped.
0x8310000A	E_XSTUDIO_STREAM_ALREADY_MAPPED	The specified data stream is mapped already.
0x8310000D	E_XSTUDIO_NO_STREAM_SPECIFIED	A data stream was not specified.
0x8310000E	E_XSTUDIO_INVALID_TIMESTAMP	The specified timestamp is invalid.
0x8310000F	E_XSTUDIO_SKELETON_DATA_REQUIRED	The identity stream is mapped for playback, but the skeletal stream is not.
0x83100010	E_XSTUDIO_TRACKING_ID_NOT_FOUND	The tracking identifier cannot be found.
0x83100011	E_XSTUDIO_INVALID_EVENT_INDEX	The specified event index is invalid.
0x83100012	E_XSTUDIO_NOT_SUSPENDED	You tried to resume or single-step streams that are not suspended currently.
0x83100013	E_XSTUDIO_RECORD_WHILE_SUSPENDED	You tried to record while playback is suspended.
0x83100014	E_XSTUDIO_SUSPEND_WHILE_RECORDING	You tried to suspend while recording is in progress.
0x83100015	E_XSTUDIO_SUSPEND_WITHOUT_PLAYING	You tried to suspend streams that are not playing currently.

See Also

[XStudio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface Samples



The Natural User Interface ([NUI](#)) samples demonstrate the API and PC tools for Kinect titles. Each sample is shown in one category below, though each sample generally covers several areas.

Categories

- [Skeletal Tracking](#)
- [Head Tracking](#)
- [Data Processing and Optimization](#)
- [Gesture Inference and Detection](#)
- [Depth Data and Player Segmentation](#)
- [User Interface](#)
- [Audio and Speech](#)
- [Identity](#)
- [Xbox Studio](#)

Skeletal Tracking

[Active/Passive Skeletons Sample](#)

Uses **NuiSkeletonSetTrackedSkeletons** to specify which of the position-only tracked skeletons should become fully tracked.

[Advanced Skeletal Retargeting Sample](#)

Maps skeletal joint position data to skeletal rotations to make a character mirror the player's actions.

[Avatar Monkey-See-Monkey-Do Sample](#)

Supports control of an avatar.

[Avatar Retargeting Sample](#)

Maps the output from the Kinect Sensor to an avatar by using XDK libraries.

[Crumple Detection Sample](#)

Identifies and filters crumpled skeletons.

[Fitness Sample](#)

Demonstrates how to use the NuiFitness API to record fitness activities and retrieve records from a player profile.

[Frosty Window Sample](#)

Draws on a virtual canvas using skeletal tracking.

[Joint Constraints Sample](#)

Applies a simple constraint model to skeletal tracking data to give a more natural-looking result.

[Running Sample](#)

Measures a player's running speed using Kinect data.

[Shoulder Stabilization Sample](#)

Stabilizes the head and shoulders positions.

[Simple Skeleton Tracking Sample](#)

Reads data from the sensor array and feeds that data to the skeleton tracking API.

[Simple Seated Skeleton Tracking Sample](#)

Reads skeletal data from the seated skeleton tracking pipeline.

[Theremin Sample](#)

Compares skeleton positions to play a Theremin, an electronic musical instrument.

[Turn Detection Sample](#)

Detects when a player turns around, by implementing a filter.

Head Tracking

[Simple Head Tracking Sample](#)

Uses head position tracking without using skeleton tracking. Detects head shaking to associate a player and controller with a particular head.

[Head Tracker Sample](#)

Changes the perspective of the display based on the player's head movement, on a standard TV. Sets up view and projection matrices, based on the location of the head joint returned from the Skeleton Tracker.

[Stereoscopic 3D Head Tracking Sample](#)

Changes the perspective of the display based on the player's head movement, on a stereo 3D TV. Sets up projection matrices for a stereoscopic 3D scene, based on the location of the head joint returned from the Skeleton Tracker.

[Head Orientation Sample](#)

Retrieves information about the player's head orientation.

Data Processing and Optimization

[Filtering Sample](#)

Demonstrates the trade-off between smooth response and latency that several different filters (and parameterizations of those filters) offer.

[Hand Control Sample](#)

Uses second-order image moments to calculate the orientation of the segmented hand in image space.

[Hand Refinement Sample](#)

Checks that the hand joints are located on the hands by using a depth map, then smoothes the hand position by using the average pixels around this point in a high-resolution depth map.

[Reducing Latency Sample](#)

Uses a non-traditional game engine architecture and low-latency filtering to reduce latency in Kinect titles.

[Skeletal Post-Processing Sample](#)

Uses post-processing filters for skeleton joint data, keeping consistency of joint position data.

Gesture Inference and Detection

[Bowling Sample](#)

Infers a bowling gesture, even if the arm is partially hidden.

[Driving Gesture Filters Sample](#)

Uses gesture recognition for a driving game by using NUI imaging and skeleton tracking.

[First-Person Sample](#)

Uses a NUI-enabled control scheme to navigate around a 3D environment with a first-person view. An aiming gesture is supported.

[First-Person Natural Sample](#)

Uses a NUI-enabled control scheme to navigate around a 3D environment with a first-person view.

[Flight Sample](#)

Controls an airplane using full-body gestures.

[Forward Movement Sample](#)

Supports navigating forward through a 3D environment using a first person camera and simple body movements.

[Gesture Detection Sample](#)

Detects when a player jumps or ducks by implementing detection filters from several heuristics using skeleton joint data.

[Gesture Scoring Sample](#)

Performs gesture scoring based on pre-recorded animation.

[Grab Manipulator Sample](#)

Demonstrates a natural method for object interaction by combining hand open-closed detection with skeletal tracking.

[Hand Opened Closed Sample](#)

Demonstrates detection of whether a player's hand is opened or closed.

[Kick Boxing Sample](#)

Shows how to use machine learning to do gesture detection.

[Simple Gesture Detection Sample](#)

Shows how to use NuiGesture API together with the output of the Visual Gesture Builder tool.

[Wave Gesture Tracking Sample](#)

Detects the arm waving gesture using the NuiWave API, and displays the progress of the gesture.

Depth Data and Player Segmentation

[Advanced Segmentation Sample](#)

Separates an image or stream of images into foreground and background.

[Depth Map Particle Stream Sample](#)

Creates a visual effect that implements a particle system to interact with a player in real time.

[Depth Visualizer Sample](#)

Visually represents the depth map of the Kinect Sensor.

[Lean Detection Sample](#)

Uses raw depth data rather than skeletal tracking to determine whether a player is leaning left or right.

[Motion-Based Background Removal Sample](#)

Uses a raw depth map to visually separate players from the background using player movements, and uses the player mask to create a ghost blend and a heat distortion.

[NuiViewer Sample](#)

Demonstrates 3D depth-map and skeletal visualization similar to that used in NuiView, a tool that ships with the XDK.

[Segmentation Sample](#)

Loads and uses segmentation information from the Kinect sensor array by using the NUI Imaging API.

User Interface

[Asynchronous Cursors Sample](#)

Demonstrates how to use asynchronous precompiled command buffers and a rendering callback to update the rendering frame with the most recent cursor information.

[Lobby Chat Sample with Enhancements for Kinect](#)

Adds chat over Xbox LIVE, using the Kinect Sensor, to the LobbyChat sample.

[Menu Using Handles Sample](#)

Uses the Handles API for menu navigation.

[Menu Using Hover Sample](#)

Uses the Handles API to implement menu navigation based on the hover feature.

[Playspace Feedback Sample](#)

Provides feedback to guide a player into the optimal position in the playspace.

[Tank Simulator Sample](#)
Uses virtual controls to drive a virtual tank.

Audio and Speech

- [Dynamic Speech Sample](#)
Demonstrates how to create grammars at run time by calling the NuiSpeech API.
- [Simple Speech Recognition Sample](#)
Performs speech recognition using several example grammars.
- [Sound Location Tracking Sample](#)
Shows how to use the NuiAudio API to capture audio, and how to steer the beam-former to improve audio fidelity for specific gamers in the room.
- [Speech Retain Audio Sample](#)
Captures audio data used by the speech recognizer along with the associated metadata.

Identity

- [Identity Sample](#)
Uniquely recognizes up to eight enrolled players, with up to two active players at a time. Simulates a party game in which many players take part in short activities, competing against each other. Players join for a few rounds and then switch places with other participants.
- [Simple Identity Sample](#)
Manages checking if a skeleton is recognized as an enrolled player, responding to intent to play, signing in to a gamer profile, signing out of a gamer profile, and biometric sign-in. To see the use of the Identity API, see this sample first, because it uses automatic identity handling.
- [Simple Identity Using Callbacks Sample](#)
Manages player identity without using automatic identity handling.

Xbox Studio

- [NUI PC Conduit Sample](#)
Transfers NUI data between a Windows-based computer and an Xbox 360 development kit using the Xbox Studio API.
- [Managed NUI Conduit Sample](#)
Shows you how to use the Xbox Studio API from a managed (C#) program running on the PC.
- [Recorded Data Sample](#)
Uses the **XedFile** API to perform operations on files recorded from the Kinect sensor array using Xbox Studio and an easy-transfer USB cable.
- [XED To BVH Sample](#)
Uses the Kinect Sensor as a motion capture device by taking an Xbox Studio recorded data file (.xed) as input and converting it to a .bvh file.
- [XedFileTagger Sample](#)
Shows how to use the XedFile API from C# to add title-specific data to .xed files.

Related Topics

- [Using the XDK Samples](#)
- [XDK Unlocked - Videos](#)
- [NUI Overview](#)
- [NUI Reference](#)
- [NUI Tools](#)
- [NUI White Papers](#)

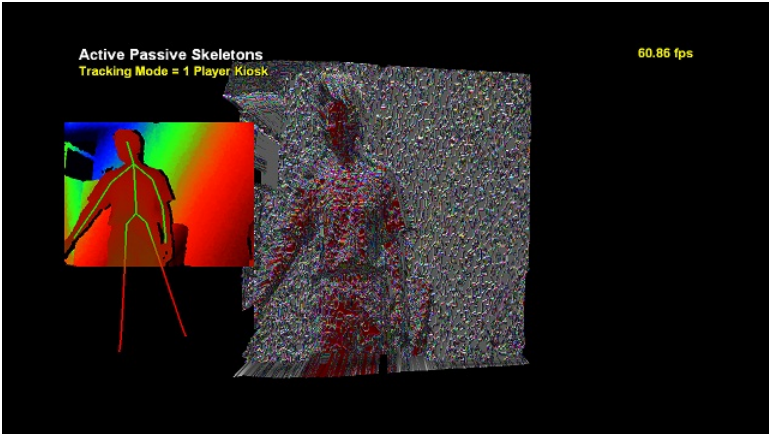
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Active/Passive Skeletons Sample

Active/Passive Skeletons uses the [NuiSkeletonSetTrackedSkeletons](#) method to specify which of the position-only tracked skeletons should become fully tracked.

Description

Figure 1. The Active/Passive Skeletons sample



As multiple users move around the play space, enter the play space, or exit the place space, the skeleton tracker tracks different position-only skeletons based on the current tracking policy.

Tracking Policy	Description
One-player Kiosk Mode	The closest position-only skeleton is tracked. If there is only one trackable skeleton, this skeleton is tracked. When a position-only skeleton comes more than .3 meters closer to the camera than the currently-tracked skeleton, the closest skeleton then is tracked.

Two-player Kiosk Mode	The two closest players are tracked. The threshold for switching players is .3 meters.
One-player Sticky Mode	The first available skeleton is tracked. When this skeleton leaves the play-area, the last-known-good location is stored. When a position-only skeleton comes within .3 meters of this last-known position and if no skeleton is being tracked, that skeleton is tracked.
Two-player Sticky Mode	The first two available skeletons are tracked. If a skeleton leaves the field of view, the position is recorded. Position-only skeletons can only be tracked if there is an available slot and if they are within .3 meters of an open position slot.

Controls

- Press the **A** button to select the next tracking policy.
- Press the **Back** button to display the help screen.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Advanced Segmentation Sample

Segmentation is the process of separating an image or stream of images into areas we recognize as foreground and background. Typically with Natural User Interface (NUI), this means visually separating players from the background or possibly from occluding objects in the foreground.

With the Skeletal Tracking (ST) pipeline running, the 320×240 depth map stream returned from the sensor has additional segmentation bits set, identifying pixels as either one of the players or the background. It's possible to use the mask to extract players' images from the color map. While simple, the results are relatively rough (see the [Segmentation](#) sample).

Description

Advanced Segmentation illustrates a variety of image processing techniques to refine the player cutouts at full resolution of the color map, 640×480. The general approach is to start from the segmentation mask and try to identify useful regions of the image, working to gradually classify every pixel as background or foreground (that is, part of the player). Since it uses the ST segmentation mask as a starting point, it requires the full ST pipeline.

Figure 1. The Advanced Segmentation sample



Controls

Advanced Segmentation classifies the pixels in the camera image in a variety of ways.

The main display shows two streams. On the left, it shows various versions of the color image or processing at 640×480, and on the right, various depth displays and results derived from depth. You can display various stages in the refinement pipeline for both depth and color. D-pad up and down select different color visualizations. Pressing the **A** and **Y** buttons step you backward and forward through the various depth displays.

Pressing the **B** button enlarges the color view, and pressing **B** returns to the main display. While the color display is enlarged, only the color display type can be changed by pressing the D-pad up and down.

Further Information

- The following sections provide greater detail of various technical aspects of the sample. There are several issues with segmentation using the ST mask alone:
- The depth image must be registered to the color image, or vice versa. Otherwise, they do not match closely at all. In registration, one map or another loses some fidelity or gains artifacts.
 - The segmentation map is 320×240, so each pixel matches to a 2×2 block in the color image.
 - The segmentation map can over- and under-estimate along the player silhouette.
 - The depth map contains noise. Although the eye tends to reduce the effect, the color map does this also.
 - The color map can undergo quite strong changes in intensity for a relatively static scene.
- This sample attempts to refine the pixel segmentation model at 640×480 by using several different mechanisms.

Region Classification

The sample works with bit masks, starting with the segmentation mask, to define areas that are definitely part of the foreground only, definitely part of the background only, and the area that contains a mixture of background and foreground—and by definition must contain the player's silhouette.

Background Model

The sample tries to keep a model of the background of the scene, updating its color information only when it is sure that it is seeing background, rather than player. You can use this model to decide if a color that is present at a particular *x,y* location matches the background or not. Note that this is not a definitive test—areas of a player can match background color, yet still be foreground. The background model updates continuously to account for brightness changes.

Edge Detection

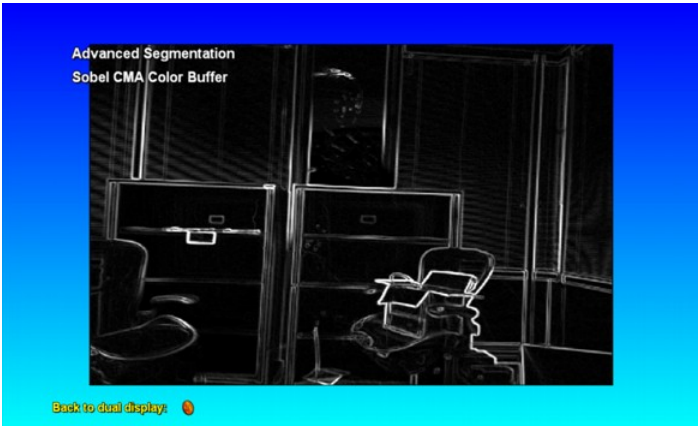
By comparing edges detected in the current color image against edges detected in the background model, pixels that differ strongly can be classified as definitely foreground.

Color Visualizations

The following are color visualizations:

- Original Color Buffer**
The 640×480 stream from the camera, unchanged.
- Select From Current Depth**
The color stream from the camera, but it is segmented using the mask in the right pane. This makes the most sense with the depth displays derived from the segmentation mask.
- Original Intensity**
The input color stream converted to gray scale.
- Sobel Original Image**
This is the difference between the Sobel edge detection for the color stream and the Sobel edge detection for the cumulative moving average buffer.
- CMA Color Buffer**
The color buffer representing the background. This may show a dark area in the center. This should correspond to areas of the background that the cumulative moving average buffer has never seen.
- Sobel CMA Color Buffer**
Edge detection on the cumulative moving average buffer.
- Difference of Sobels**
The difference between the intensity models.
- Final Segmentation**
The final results of the refined segmentation.

Figure 2. Edge Detection Pass on Background Buffer



Depth Visualizations

The following are depth visualizations:

- Depth Map**
The depth map supplied from the Kinect Sensor.
- ST Segmentation Mask**
The segmentation mask for player 1.
- ST Segmentation Mask Erode**
The eroded version of the mask—eroded such that it contains only foreground pixels.
- ST Segmentation Mask Dilate**
The dilated version of the mask—dilated so that it contains all of the foreground and all of the silhouette of the player.
- Definite Foreground Pixels**
Mask containing only foreground pixels.
- Definite Background Pixels**
Mask containing only background pixels.
- Pixels Containing Silhouette**
The set of pixels that contain the player's silhouette.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

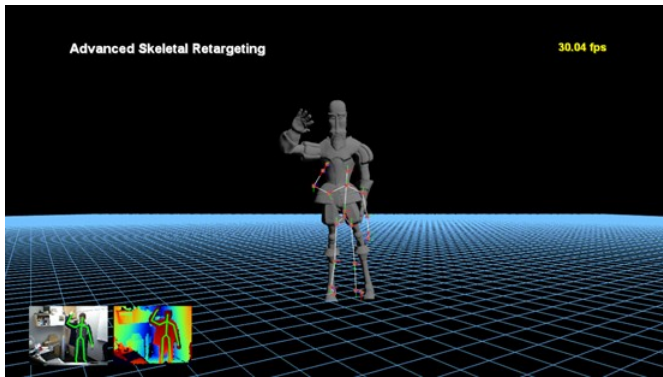
Advanced Skeletal Retargeting Sample



Advanced Skeletal Retargeting shows how to map the output from the Kinect sensor array to an arbitrary anthropomorphic skeleton. The skeletal positions received from the sensor array are remapped to skeletal rotations, and are used to mirror the player's actions with the character.

Description

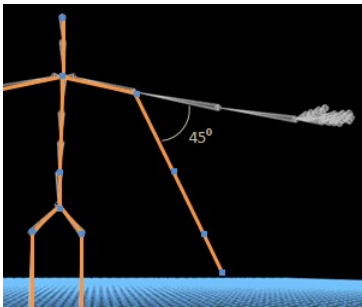
Figure 1. The Advanced Skeletal Retargeting sample



Advanced Skeletal Retargeting shows how to remap skeletal tracking data retrieved from the sensor array to a model that the player can control. Skeletal tracking data retrieved from the sensor array comes in the form of world-space skeleton joint positions. These positions are converted into the skeleton rotations necessary to control a skinned character.

You can convert Natural User Interface (NUI) skeleton joint positions to rotations by determining the shortest rotation from the bind pose direction of each bone to the direction from the NUI skeleton's joint position to the position of its child. For example, the shoulder rotation is taken as the shortest rotation from the bind pose direction of the shoulder joint to the direction of the skeleton from shoulder to elbow. The following figure shows this process. As this algorithm deals only with rotations and not translations, bone lengths are implicitly preserved.

Figure 2. The character skeleton bind pose is shown in white, and the NUI skeleton is shown in orange. This orientation of the NUI skeleton creates a 45 degree downward rotation of the shoulder from the bind pose.



Once you convert the NUI positions to rotations, you can map them to the model skeleton and use them to skin the mesh. Also, you may add procedural animation on top of these rotations to provide ambient animation. This can be especially useful for joints, such as fingers, that have no NUI counterpart.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Asynchronous Cursors Sample



Asynchronous Cursors demonstrates how to use asynchronous precompiled command buffers and a rendering callback to update the rendering frame with the most recent cursor information. This idea can be applied to the full NUI skeleton or to any resource, and it can help to reduce latency even in situations where you are rendering with [D3DCREATE_BUFFER_2_FRAMES](#).

Running the Sample

The left cursor uses the technique and is more responsive than the right cursor. For demonstration purposes, the hover progress uses the old technique. To see the difference in the techniques, move your hand across the button, and you will see the hover progress trailing the left-hand cursor.

How It Works

On the rendering thread, the sample has a callback that the GPU runs immediately before it needs the cursor information to render on the frame. The sample then calls the following asynchronous command buffer:

```
g_pAsyncCommandBufferCalls[g_iACBcallbackToUse]->Reset( 0, 0, 1, 0);
m_pd3dDevice->InsertCallback( D3DCALLBACK_IDLE, &DeferredProcedureCallbackFromGpu, g_iACBcallbackToUse );
m_pd3dDevice->InsertAsyncCommandBufferCall( g_pAsyncCommandBufferCalls[g_iACBcallbackToUse], D3DPRED_ALL_RENDER, 0 );
g_iACBcallbackToUse = 0x1 ^ g_iACBcallbackToUse;
```

When the GPU hits the callback, it runs code that determines the latest command buffer to use, and then calls fix up and signal:

```
void DeferredProcedureCallbackFromGpu(DWORD Context)
{
    LONG iIndexToUse = (g_iLastCursorReady) % 3;
    AcquireLockBarrier();
    g_pAsyncCommandBufferCalls[Context]->FixupAndSignal( g_pCursorCommandBuffers[iIndexToUse], 0, 0 );
}
```

On the title thread, which processes the NUI updates, is a precompiled command buffer that encapsulates the data and signals that the next command buffer is ready to use by calling [InterlockedIncrementRelease](#).

To ensure against any problems, the queue should have 3 command buffers. One is used on the GPU, one is written to, and a third is queued in case the update thread finishes updating before the GPU is done using the first command buffer. Since NUI updates occur at 30 fps in the game, there is no need to worry about NUI running 3 times before the GPU is done with the first command buffer. If you apply this method in another situation, ensure that the producer thread does not overwrite the current command buffer in use.

Since Direct3D can only buffer two frames, only two callbacks are required.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Avatar Monkey-See-Monkey-Do Sample



Monkey-See-Monkey-Do demonstrates how to map the output from the Xbox 360 Kinect sensor array to an avatar. The skeleton positions received from the sensor array are remapped to

avatar skeleton rotations and used to mirror the user's actions with the avatar.

Description

Figure 1. The Avatar Monkey-See-Monkey-Do sample



Monkey-See-Monkey-Do shows how to remap skeletal tracking data, retrieved from the Kinect Sensor, onto a model that can be controlled by the user. Skeletal tracking data retrieved from the sensor array comes in the form of world-space skeleton positions. These positions are converted into the skeleton rotations necessary to control an avatar. The skeleton positions returned from the sensor array can optionally be filtered through one of several positional filters.

Conversion from skeleton positions to joint rotations is accomplished by determining the shortest rotation between the bind pose and the vector from the current joint to its child. This is repeated for all camera joints, and the results are transferred to the corresponding avatar joints. These world-space rotations are then converted to local rotations by subtracting the rotation of the parent joints. Since no rotation about bone axes is provided (and can't be determined from skeleton positions), rotation about the axis is determined for each skeleton position. The neck and head, for example, are rotated to match the shoulders. The wrists are rotated to orient the hand forward when the arm is raised.

The [NUI Skeletal Tracking](#) API is used to capture and track skeletal movement read from the sensor array. For simplicity, the skeletal tracking data is captured in a polling fashion; if new data is not available when the API is polled, it is gathered on the next frame. This method can be optimized by using a separate thread to poll the API for new skeleton data, using an event handle that you provide. Note that you must poll for skeletal data at least once for every 30Hz frame to ensure that the pipeline is correctly pumped.

The sample contains options for tilt correction and data smoothing. If enabled, these improvements are applied to the skeletal data after capture. The sample also allows you to draw the skeleton positions for debugging purposes and to select the type of filtering that is applied by the sample to the skeleton data.

Filters

This sample contains a number of options for filtering the skeleton positions: blend, velocity-dampening, and a combination. Filters are useful for removing high-frequency noise from the skeleton positions. Each filter has different characteristics, and each may be optimal for different uses.

Blend Filter

The blend filter averages the previous frames to find the final skeleton positions. More noise can be removed by increasing the number of frames used, at the cost of additional latency and a loss of small-scale movements.

Velocity Dampening Filter

The velocity dampening filter attempts to remove noise by blending between the *expected* position (based on current velocity) and the *reported* position. This removes some latency by attempting to predict where the skeleton position will likely be in the next frame. This method also removes noise while keeping latency low for skeleton positions that don't change velocity. Skeleton positions that do change velocity (speed or direction) will still experience some latency.

Combination Filter

The combination filter is an example of a more complex filter. This filter begins by determining the amount of blending desired for each skeleton position, and that determination affects the amount of blending of previous skeleton positions to be done. This amount depends on how closely the reported velocity of the skeleton position coincides with the expected velocity. The blend rate of a skeleton position will be propagated to its neighboring skeleton positions if its rate is greater, by a certain threshold, than that of the neighboring skeleton positions. Once all blend rates have been determined, the final skeleton position is calculated as a weighted average of its reported location and its average location over the last few frames.

The combination filter is the highest quality filter of the three, but is also the most costly. The filter that can be turned on or off with the camera API is similar to the combination filter.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Avatar Retargeting Sample



Avatar Retargeting demonstrates how to map the output from the Kinect Sensor to an avatar using XDK libraries. The skeleton positions received from the sensor array are remapped to avatar skeleton rotations through the use of the **IXAvatarNuiMapper** interface, and used to mirror the user's actions with the avatar.

Description

Figure 1. The Avatar Retargeting sample



Avatar Retargeting illustrates how to remap skeletal tracking data, retrieved from the sensor array, onto a model that the user can control. Skeletal tracking data retrieved from the sensor array comes in the form of world-space skeleton positions. These positions are converted into the skeleton rotations necessary to control an avatar through the usage of the **IXAvatarNuiMapper** interface.

The **NUI Skeletal Tracking API** is used to capture and track skeletal movement read from the sensor array. For simplicity, the skeletal tracking data is captured in a polling fashion; if new data is not available when the API is polled, it is gathered on the next frame. This method can be optimized by using a separate thread to poll the API for new skeleton data, using an event handle that you provide. Note that you must poll for skeletal data at least once for every 30Hz frame to ensure that the pipeline is correctly pumped.

The **IXAvatarNuiMapper** interface provides functionality for retargeting the NUI skeleton to joint rotations for an avatar skeleton. This interface contains options for mirroring the skeleton, preventing self intersections, and rescaling the avatar to the proportions of the user.

The sample also demonstrates how a title can combine inputs from **NuiHeadOrientation** with the Avatar. It demonstrates how to compensate the head orientation data for sensor tilt and player location before applying the results to the avatar.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Bowling Sample



The Bowling sample demonstrates simple gesture inference using a bowling motion (swinging the arm back, down, then forward). *Gesture inference* is the act of interpreting a simple movement (or part of a movement) in a known context as a recognized gesture.

Figure 1. The Bowling sample



Description

The Bowling sample demonstrates using data from the Kinect sensor array.

For some gestures, full tracking that uses the Kinect sensor array is impossible. Usually, these gestures involve a body part being occluded from the camera by the player's own body, as would happen with swinging a baseball bat. This sample solves this problem by inferring certain gestures of a player as an attempt at a bowling swing, even if the player's arm disappears behind his or her body.

Using the Sample

The sample attempts to infer the part of a bowling gesture during which the player's arm is occluded from the camera. The sample works in two phases. First, a recording of the player executing the gesture is made. Second, the sample switches to a playback mode in which the player can replay the gesture at will and compare the differences between the raw skeleton returned by the skeletal pipeline and the one inferred by the bowling filter.

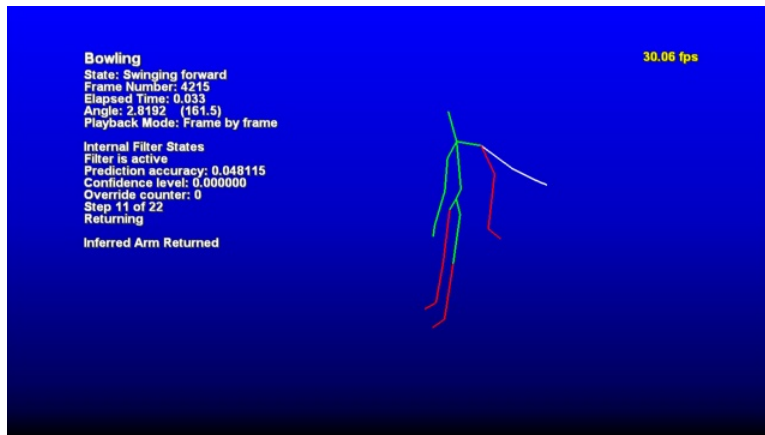
Recording Mode

The sample starts in recording mode. The player raises his or her right or left hand to indicate the hand to be used for the bowling gesture. The player then points both arms down to indicate that the gesture is about to be performed. The player then performs the gesture, and the sample records the gesture. The sample automatically switches to playback mode once it determines that the gesture has ended successfully.

Playback Mode

In this mode, the sample displays a simple skeleton that constantly replays the last few seconds of the recorded bowling gesture. The player has full control of the camera and can position it for the best possible view of any part of the skeleton. The player can alter the playback speed by pressing the **Y** button. Pressing the **A** button steps through the recorded data, frame-by-frame, and pressing the **B** button exits the frame-by-frame mode.

Figure 2. An inferred skeleton



Press the **X** button to exit playback mode and return to recording mode.

Onscreen Information

During playback mode, more information about the gesture and skeleton tracking data is displayed on screen.

Skeleton Representation

The original skeleton wireframe received from the skeletal pipeline is located in the center of the screen and drawn using green and red. Green indicates bones that were fully tracked while red indicates bones that have been inferred by the skeleton pipeline. When the bowling filter returns a skeleton that differs from the one provided by the skeletal pipeline, differing bones are shown in white.

Sample Data

The first data group provides information about the state of the sample and about the skeleton as received from the skeletal pipeline. The first line is the current state or phase of the gesture, followed by the frame number of the skeleton frame currently being processed, and the elapsed time (in seconds) since the previous frame.

The next line tracks the current angle (in radians, with degrees provided in parentheses) of the wrist to the plane of the player's floor during the bowling gesture. A value of 0 radians indicates that the player is standing upright and is facing the sensor array with an arm extended (either left or right). At this point, the wrist bone and the player's floor plane are parallel. As the player performs the gesture, the angle of the wrist relative to the floor changes. Therefore, when the player's arm is pointing down (midway through the gesture) the value is $\pi/2$ radians, and when the player's arm is pointing backwards the value is π radians.

The last piece of information shows the playback speed that is currently selected.

Internal Filter Data

This data group lists variables that are private to the bowling filter. To understand their use, examine the related sample code, located in BasicBowlingFilter.cpp.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

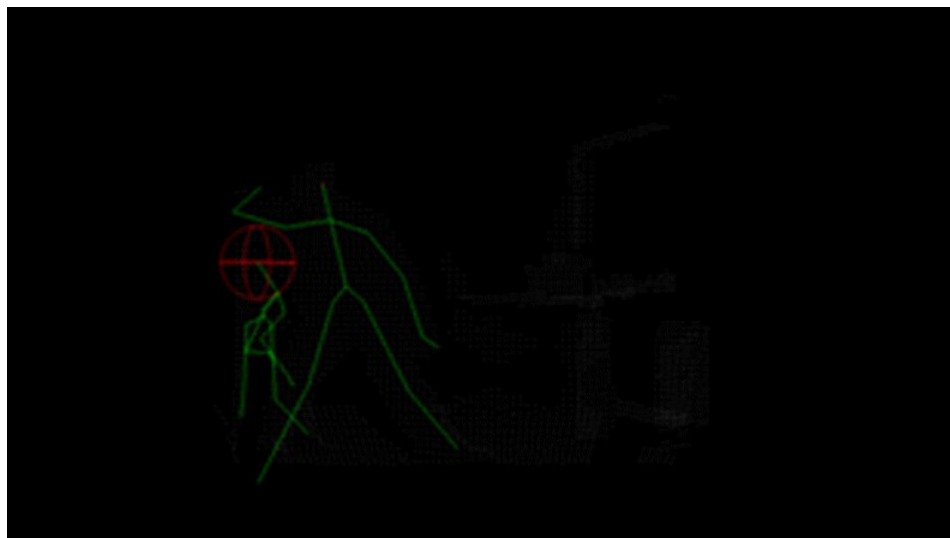
Crumple Detection Sample



Occasionally skeleton tracking can yield skeletons that don't match the underlying players in the depth map in expected ways. This can be a result of complex occlusions or boundary cases. You may want to be able to identify and filter these *crumpled* skeletons.

Crumple Detection demonstrates the use of additional measurements of the returned skeleton and corresponding depth map to evaluate the quality of a skeleton. In Crumple Detection, each tracked skeleton is drawn with a sphere attached to its head. The size of the sphere is proportional to the degree of crumple.

Figure 1. The size of the sphere attached to a skeleton is proportional to its degree of crumple.



Description

The overall quality of a skeleton and its corresponding depth map is scored by computing various metrics that account for the following:

- Occlusions
- Clipping by the boundaries of the field of view

- Unlikely skeleton geometry
- Head shape
- Motion
- Forearm appearance
- Foreground mask appearance

An overall score is generated by combining these terms.

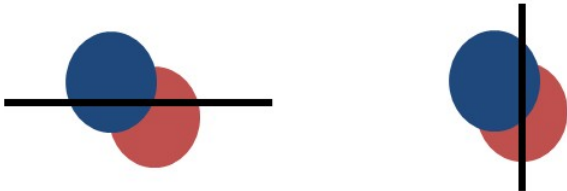
Occlusions

The skeletons can become increasingly unstable as their foreground masks are occluded by parts of the background or other players. Based on this observation, we derive a metric to predict when a skeleton becomes crumpled by measuring the occlusion of the skeleton's foreground mask.

You can do this by first marching through the rows of the depth image looking for boundaries of a player's mask. Once you find an edge, you can look to the left or right (as appropriate) and count the number of pixels in this margin that are in front. An overall score—the ratio of the number of potentially occluding pixels to the number of pixels of a player in the foreground mask—is taken as the likelihood of being occluded.

Similarly, you can measure occlusions by looking down the columns. The overall occlusion score is taken as the maximum of these two numbers. While you consider the potentially occluding pixels, you can only consider depth points that are above the estimated floor.

Figure 2. Potentially occluding depth pixels can be found by looking across rows or down columns.

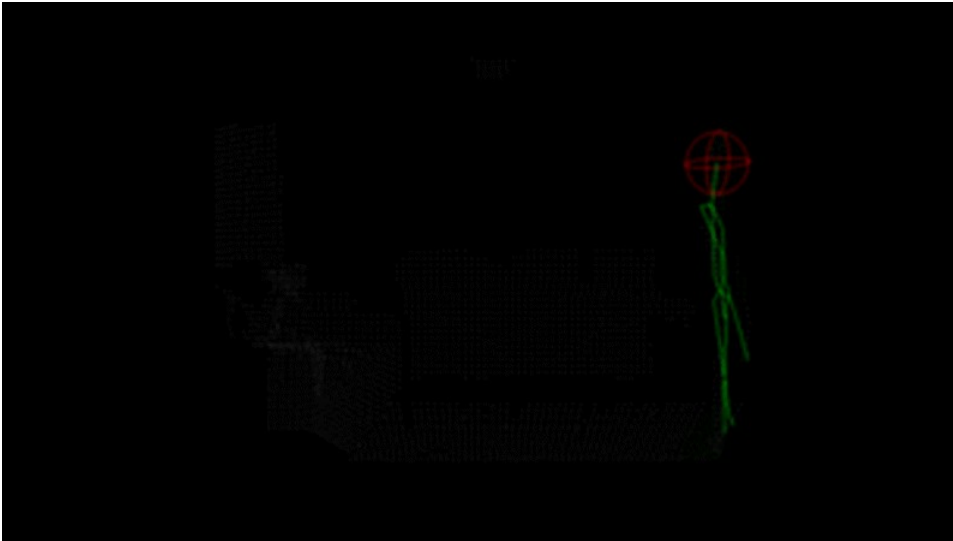


Clipping by Field of View

Skeletons often become unstable as they are clipped by the field of view. Based on this observation, you can derive a crumple score by computed how snug a skeleton projects into the margin of the depth image. This works well as a player walks out of the field of view to the left or right of the view frustum.

Clipping by the top edge of the field of view (FOV) is a bit trickier. Here we keep track of the height of the torso when a skeleton first appears. Clipping by the top of the field of view is then estimated by the discrepancy between the predicted and measured height of the player blob in the depth image.

Figure 3. Skeleton crumpling as the user walks out of the field of view.



Skeleton Geometry

A strong indicator of a poor skeleton is the asymmetry of limb lengths. You can also check for some highly unlikely configurations of joints, such as when the head falls below the hip.

Head Shape

An indicator of a skeleton latching onto a chair or door or other inanimate object is the shape of the depth pixels around the head. Here, you can project the skeleton's head and shoulders into the depth image, and look for nearby boundary points of the player's foreground mask. The closer these boundary points form a straight line, the more likely the skeleton is latched onto a chair or sofa. If the player's head becomes clipped (as measured by the FOV clipping, described earlier), you can disable this term.

Motion and Foreground Mask Shape

You can also measure the likelihood of a foreground blob motion by using frame differencing. As a player becomes stationary, you can measure the shape of the forearm in the depth image. This is done by making sure the forearm bones project onto nearby depth pixels, and that they line up with these depth pixels in 3D. Also, you can search for depth edges around the projected forearms.

In addition to looking at the forearm shape in the depth image, you can also look at the compactness of the overall player foreground mask. This is computed as the ratio of the number of boundary pixels to its area (in pixels). If a foreground mask becomes fragmented, its corresponding skeleton can become unstable, and its compactness becomes large. Then, large compactness then can be used to signify crumple.

Overall Score

These individual scores are mapped to values from zero and on, through normalization or through the use of a carefully tuned sigmoid functions. Here, lower scores correspond to more crumpled skeletons. You can take an overall score as a product of these individual terms.

Controls

By using the Xbox controller, you can toggle depth modes and drawing modes.

Control	Description
---------	-------------

A button	Toggles depth points
Y button	Toggles drawing modes

The drawing modes include the following:

- Debug-graphics**
Highlights depth pixels considered to be occluding as well as forearm pixels visited in examining the appearance of depth around the skeleton forearms.
- Score-details**
Show additional scoring information.
- Skeleton**
Draws skeletons with an overlay of scores associated with that skeleton.
The terms shown include the following:
 - Overall score**
Overall crumple score of the skeleton.
 - Occlusion score**
Component of the score due to occlusions.
 - Head-clip score**
Part of the score due to being clipped by the top of the viewing frustum.
 - Other-terms**

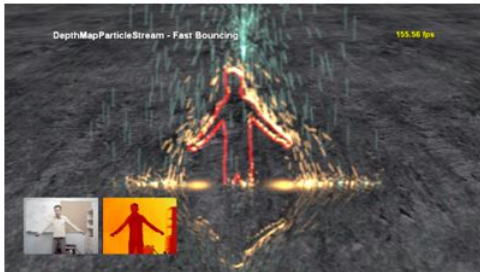
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Depth Map Particle Stream Sample

Creates a visual effect that implements a particle system to interact with a player in real time. Particles bounce off a player's silhouette and, based upon the speed at which a player moves, render trailing effects.

Description

Figure 1. The Depth Map Particle Stream sample



Depth Map Particle Stream creates a very cool and fun visual effect by implementing a particle system whose particles interact with the player and bounce off player's silhouette in real time. You can hold out your arm to collect some particles and then let them go, or you can just shake off the particles on your shoulder. The sample also renders a cool trails effect of the player. Because the trails of the player are only visible when the player is moving (the effect is even more interesting if the player moves faster), this actually motivates the player to move more. When this trails effect is turned off (left shoulder button), you tend to be more static, and with the trails effect on, you tend to move more as the graphics looks cooler when you move. This is an example of how a title can guide the player to do things that are the intentional gameplay experience.

How the Sample Works

In each frame, this sample performs sequential steps.

1. Retrieves the depth image by using the [NuiImageCamera API](#).
2. Extracts the lower 3 bits of each pixel of depth image as player index map(also known as the segmentation map).
3. Runs a simple image edge detection algorithm on the segmentation map to compute silhouette of the player. In the process of edge detection, the normal of each pixel that lies on the silhouette of the player also is calculated. The normal is necessary for those particles to bounce off the body into the right direction. Normal and silhouette information is stored to a texture called Edge Map.
4. Untiles Edge Map and uses it on the CPU to update the particle system.

For instructions about how the particle system is implemented, see the [Point Sprites sample](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Depth Visualizer Sample

Depth Visualizer demonstrates processing the depth map of the Kinect Sensor to produce an intuitive view of the captured scene. The sample intentionally does not retrieve or use any of the following:

- The color map
- The player segmentation data
- Skeletal tracking data

Therefore, the visualization is practical even for use with controller-augmented titles, which do not require the full Kinect system overhead (processor and RAM reservations).

Sections

- [Visualization Approach](#)
- [Rendering Phases](#)
- [User Interface Options](#)
- [Depth Fill Algorithm](#)

- [Conversions from NUI Space](#)
- [Estimation of Normals](#)
- [Filtering Depths and Normals](#)
- [Outlining and Contours](#)

Visualization Approach

The goal is to depict the depth map in a visually appealing fashion. A successful implementation should convey surface detail without too much noise. Also, the processing and memory requirements should be minimal.

The sample renders surface detail by computing and displaying an inferred normal map. A title might use this normal map to introduce a synthetic lighting environment into the scene. However, for an abstract visualization, it proves to be just as effective simply to render the normals directly as color. The normal map has reasonable quality on human players and other objects that are within the recommended play space. It has poorer quality on flat surfaces, such as walls, where noise tends to be more evident, and harder to eliminate. For this reason, the sample focuses attention on depths within the "sweet spot". Geometry outside the sweet spot is shown only dimly, and with lower resolution normals. (Since the sample will typically run in the constrained physical environment of an office or cubicle, it does not use the actual sweet spot. Instead, it uses a default that is nearer than the documented distance. Sweet spot position and size can be adjusted with run-time controls.)

To provide additional visual cues that a 3D scene is being shown, the sample renders a few types of synthetic geometry, which are correctly depth-sorted with the Kinect data:

- A vertical cylinder, representing the sweet spot
- Some moving planes, to give insight into the shape of objects elsewhere in the scene

It is valuable to emphasize the intersection contours between the synthetic geometry and the Kinect depth map—in much the same way that a topographic map is enhanced using height contours. The sample achieves this goal by artificially brightening near-intersection areas in the pixel shader, producing a "laser scan" effect. The sample also detects depth discontinuities and uses them to generate an outline effect.

Rendering Phases

The visualization is broken down into a series of render passes, most of which occur offscreen:

1. Optional downsample of depth (for performance reasons)
2. Optional filling-in of missing depths (for aesthetic reasons)
3. Optional blurring of depth map (for noise reduction)
4. Inferring of normal map from depth map
5. Optional blurring and downsampling of normal map (for noise reduction)
6. Rendering of normal map and synthetic geometry to the viewport, with correct depth testing

User Interface Options

The sample enables toggling the following parameters.

- **Low Res Depth:** The depth map can be downsampled from 320x240 to 160x120 before any other processing occurs. This operation reduces detail, but produces a four-fold speed-up during depth processing.
- **Low Res Normal:** The normal map can be generated at quarter resolution – 160x120 rather than 320x240. This operation reduces detail, but produces a four-fold speed-up during normal map processing.
- **Low Precision Normal:** The normal map can use a 16-bit format (5:5:6) rather than a 32-bit format (10:10:10:...). This operation slightly reduces precision, but uses only half the RAM for the normal map.
- **Small Effect Viewport:** The visualization renders at quarter resolution on screen – 320x240 rather than 640x480.
- **Hide Image Margin:** The lower-confidence depth regions around the perimeter can be cropped using a scissor rect. The extreme right of the texture sometimes has an irreconcilable IR shadow (see below). The other image margins are somewhat more likely than the image interior to have missing or fluctuating depths.
- **Thumbnail Depth and Normal:** The sample can render thumbnails of the depth map and normal map at the right side of the screen.
- **Color Code Depth:** Regions of interest in the depth map are color-coded.
- **Fill IR Shadow:** A GPU algorithm is applied to fill in gaps in the depth map attributable to infrared shadow. See below for details.
- **Fill Small Holes/Small Holes Scale (log):** A GPU algorithm is applied to fill in small areas of invalid depth surrounded by valid depths.
- **Erase Small Islands/Small Islands Scale (log):** A GPU algorithm is applied to erase in small areas of valid depth surrounded by invalid depths.
- **Depth Filter Iterations/Depth Filter Type/Depth Bilateral Edge Blur (log):** Parameters controlling the type, quality, and number of filter passes on the depth map.
- **Normal Filter Iterations/Normal Filter Type/Normal Bilateral Edge Blur (log):** Parameters controlling the type, quality, and number of filter passes on the normal map.
- **Fade Normal/Fade Normal Scale (log):** The normal map is downsampled to generate a mipmap chain. Pixels far from the play space sample from a lower resolution version of the normal map, and these areas therefore appear less noisy in the visualization.
- **Renormalize Normal:** Renormalizes the normal vector in the visualization shader and the thumbnail shader. Generally unnecessary, but slight difference is noticeable when using low precision or low resolution normal.
- **Draw Floor Plane:** The sample draws a plane meant to represent the floor of the play area. The computed NUI floor plane is accessible only when using skeletal tracking, so the sample instead assumes a fixed height.

Depth Fill Algorithm

Not all pixels in the Kinect depth map contain valid depths. Some contain a value of zero, indicating that no depth was retrieved at that location. These gaps occur for a number of reasons:

- **IR shadow:** The laser and the depth sensor work in conjunction to produce the depth image stream. For a depth to be measured, the surface in question must be visible to both the laser and the depth sensor. Since the laser and depth sensor are separated by several centimeters, some surfaces will be visible to the sensor but occluded from the laser.
- **Out of range:** Surfaces may be too close, or too far, from the camera for depth to be detected. In certain situations and certain directions (such as the sensor pointing out a window), there may be no surface present at any depth.
- **Difficult surface type:** The surface might be of a material which does not reflect the laser well. The black vinyl armrests of an office chair tend to not show up well. A glossy panel in an office wall is picked up at direct angles, but not at oblique angles.
- **Edges and thin surfaces:** Sometimes a single pixel may encompass a boundary between foreground and background surfaces. In that case, the depth is indeterminate, and it may also be difficult for the sensor to acquire a reading of either fragment. Hair can be an example of this situation.

In many situations, it can be advantageous to make an educated guess about the value of the missing pixels, even if that guess is incorrect. The most natural approach is to guess that the missing depths are nearly the same as some valid depths in their vicinity. For the specific case of IR shadow, we can also infer some additional information:

- Missing depths are background areas, occluded from the laser's point of view by a foreground surface.
- Missing depths are similar to valid depths located *further to the right* in the depth map (and likewise to the player's right, when facing the camera). The reason for the directional bias is that the depth sensor is offset on the Kinect console to the player's right from the laser. The sensor can therefore see objects behind the player's right side that are occluded from the laser's point of view.

Figure 1. Relative layout of laser and sensors



With these considerations in mind, a promising approach is to fill missing depths by “flood fill” of valid depths towards the left. For performance reasons, the sample implements the flood fill on the GPU, using a “reduction” method—creating successively smaller copies (mipmaps) of the depth map, each biasing leftwards, and then sampling the smaller maps to fill in missing values of larger maps. Reduction algorithms are particularly efficient on GPUs. CPU flood fill implementations might also be reasonable to pursue here.

An earlier, unreleased version of this sample applied the depth fill algorithm unconditionally to the entire scene. The sample worked well within the constrained environment of an office, with opaque surfaces nearby in all directions. In a more open environment, however, such as a showroom floor, the depth map contained large vacant regions. Filling these regions produced ugly and unappealing results. This experience provided a sobering lesson in the importance of broad testing with Kinect.

The following screen shots compare the screen output of a fairly complex depth scene before and after depth fill.

Figure 2. Without depth fill



Figure 3. With depth fill in IR shadow



A few features here are worth pointing out:

- As predicted, most of the missing depths in the original image occur to the right of foreground objects.
- Some of the invalid depths are not errors—the large white area is a glass pane. The sample correctly refrains from filling these areas.
- Some areas, such as the space to the right of the head, are certainly IR shadow, but cannot be filled, because there are no valid depths to their right.
- Some areas, such as the space underneath the right arm (vinyl armrest), are invalid due to bad materials.
- Some small gaps remain along edges between objects, such as in the lower left corner of the image (side of a table). These gaps are probably due to sensor uncertainty.

The sample implements a second, optional pass to fill in some of the remaining small holes. Following the small-holes pass, the depth map looks a bit cleaner:

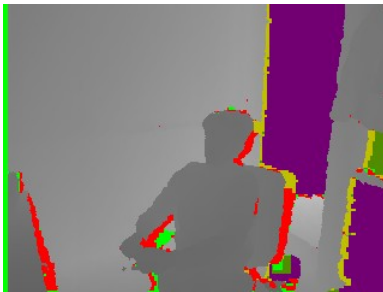
Figure 4. With depth fill in IR shadow and small holes



A color-coded version of the depth map (below) clarifies which pixels were valid to begin with, and which were affected by the clean-up passes. The colors have these meanings:

- Gray – valid depths of raw image (darker = nearer)
- Red – IR shadow, successfully filled
- Dull yellow – IR shadow, not successfully filled
- Bright green – small hole, successfully filled
- Dull green – small hole, not successfully filled
- Maroon – invalid depth, not IR shadow or small hole

Figure 5. Color-coded depth map

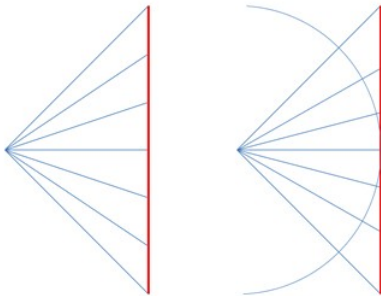


Conversions from NUI Space

In order to incorporate synthetic geometry or lights into the Kinect scene, it is necessary to convert depth pixels back into world space, or vice-versa. The transformation is implicit from the field of view of the camera, and from knowledge of what the depth values represent.

Kinect depths are roughly in units of mm, and they represent z-distance along the camera view axis (rather than radial distance). Depth pixels are equal-area in the image plane (rather than equal-angle). This implies the NUI-to-world transform should not need a perspective divide.

Figure 6. Correct (left) vs. incorrect (right) Nui depth projection



The conversion to world space therefore consists of a simple rescaling of x and y as in the following pseudo-code. Here X_{Nui} , Y_{Nui} are the texture coordinates of a depth pixel and $Depth_Map$ is the depth texture viewed as an array. World units have an x-axis pointing to player-right, a y-axis pointing up, and a z-axis pointing along the camera view direction.

```
Z_World_mm = Depth_Map[X_Nui + Y_Nui * Pitch]
X_World_mm = Z_World_mm * tan( Nui_Horizontal_FOV_Radians / 2.0f ) *
    2.0f * ( X_Nui - 0.5f ) / Depth_Map_Width_Texels
Y_World_mm = Z_World_mm * tan( Nui_Vertical_FOV_Radians / 2.0f ) *
    2.0f * ( Y_Nui - 0.5f ) / Depth_Map_Height_Texels
```

In order to use Kinect depths as depth buffer values, for GPU depth test purposes, the sample must convert them to projective space, using the same near and far plane as the rest of the scene. This conversion is a standard one:

```
Z_Proj = [ ( Z_Nui - Z_Near ) * Z_Far ] / ( Z_Far - Z_Near )
```

Estimation of Normals

The sample uses a simple method of estimating normals. Three Kinect depths are retrieved, at texture coordinates (x,y) , $(x+1,y)$, and $(x,y+1)$, and are converted to world space using the conversion described previously. These three points in world space are treated as a pivot and two offsets. The cross-product of the two offsets gives an approximate normal vector.

```
float3 vWorld00 = NuiToWorld( X_Nui, Y_Nui );
float3 vWorld10 = NuiToWorld( X_Nui + 1, Y_Nui );
float3 vWorld01 = NuiToWorld( X_Nui, Y_Nui + 1 );

float3 vTangent = vWorld10 - vWorld00;
float3 vBinormal = vWorld01 - vWorld00;
float3 vNormal = normalize( cross( vTangent, vBinormal ) );
```

The sample stores three components of the normal vector, rather than x and y only, since it is possible to see surfaces whose normal vector points slightly away from the camera. The visualization is not particularly sensitive to small errors in surface orientation, so a low-precision format is a reasonable choice for storing the normal data.

Filtering Depths and Normals

Without some filtering, any visualization will be unacceptably noisy. The raw data results in a normal map reminiscent of a Pointillist painting. The sample explores several filtering methods, recognizing that different titles may want different visual results (and may also have different performance constraints).

Figure 7. Unfiltered depth and normals



Gaussian Filters

Standard Gaussian filters will blur feature edges, giving a softer appearance.

Figure 8. Gaussian-filtered depth



A Gaussian filter applied to normals smooths out creases, and produces a sort of "Han Solo encased in carbonite" appearance.

Figure 9. Gaussian-filtered normal



Bilateral Filters

Bilateral filters provide a means to smooth away noise while preserving feature edges. Neighboring pixels are weighted not only by proximity to the central pixel, but also by closeness in value.

Figure 10. Bilateral-filtered depth



Figure 11. Bilateral-filtered normal



Outlining and Contours

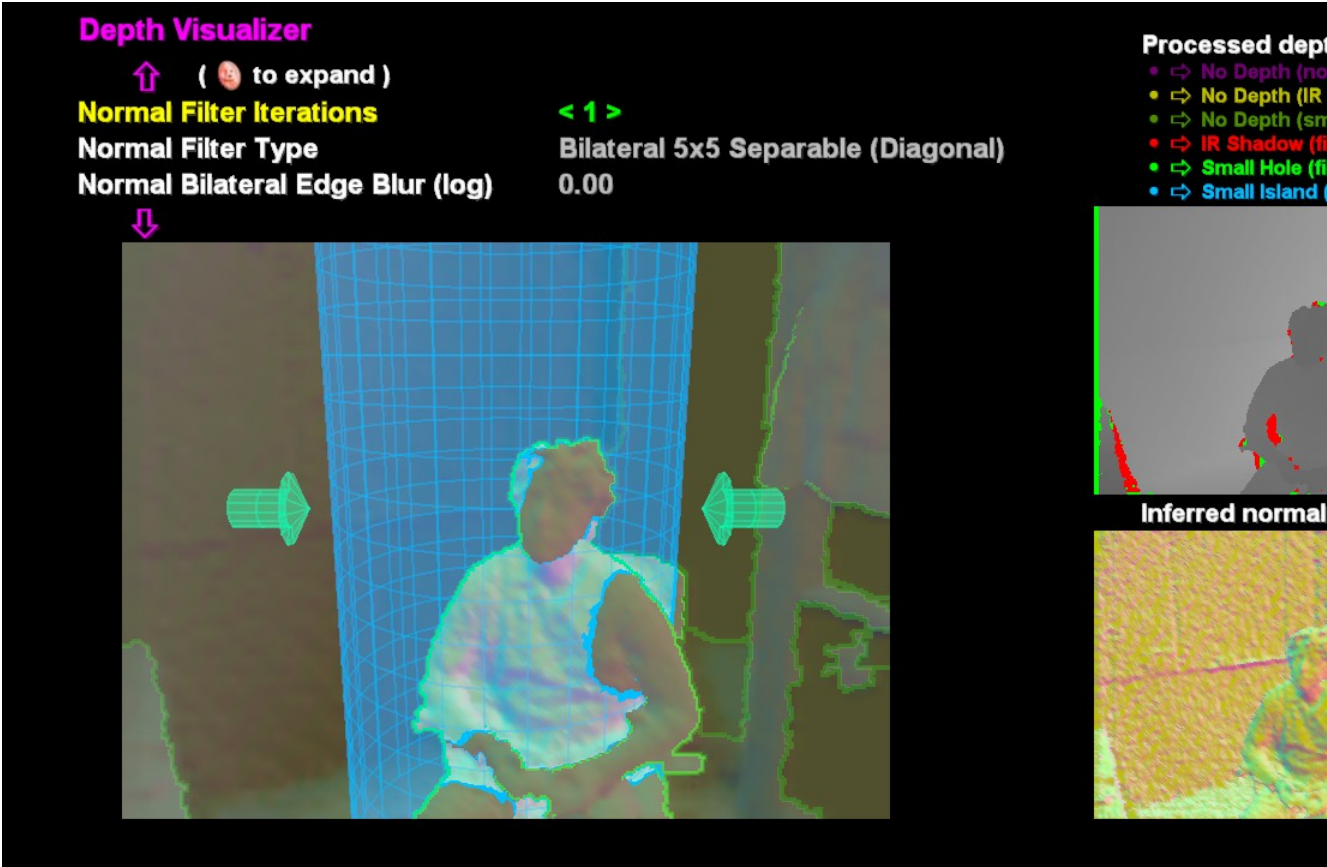
Outlines of image features, as well as intersection contours, help to produce the illusion of depth within the visualization. Calculation of outlines using the GPU's native gradient facility is a tried-and-true trick. The following HLSL code computes the length of the gradient of depth at a pixel. The ddx/ddy instructions all compile into a single instance of the texture instruction "getGradients", and this code reduces to only 3 operations in total.

```
float fDepthJump = sqrt( ddx( fDepth ) * ddx( fDepth ) +
                        ddy( fDepth ) * ddy( fDepth ) );
```

Intersection contours, such as the interpenetration of the shoulder and the blue cylinder in the screen shot below, are marked using collision detection code in the shader.

```
float fSweetSpotDist = DistanceFromLine( vPosWorld,
                                         g_vSweetSpotOriginAndRadius.xyz,
                                         g_vSweetSpotDirection );
float fNormalizedSweetSpotDist = fSweetSpotDist / g_vSweetSpotOriginAndRadius.w - 1.0f;
bool bNearSweetSpotBoundary = abs( fNormalizedSweetSpotDist ) < 0.01f;
```

Figure 12. Outlines and contours in the visualization



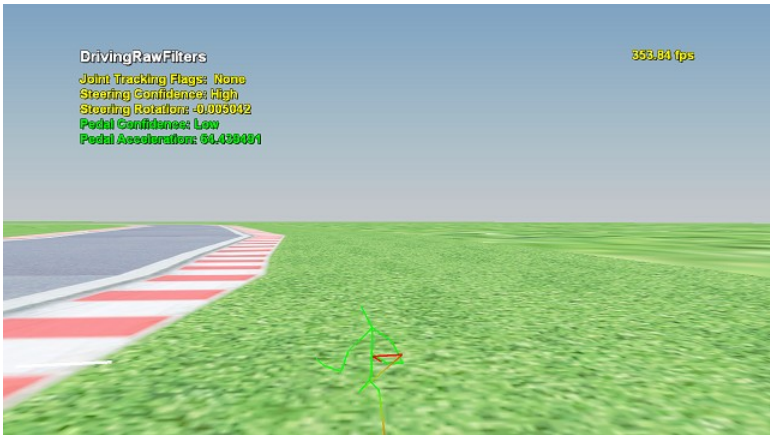
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Driving Gesture Filters Sample

The Natural User Interface (NUI) libraries can provide an intuitive control scheme in many different video game genres. This sample demonstrates controlling a vehicle on a track with several controls.

Description

Figure 1. The Driving Gesture Filters sample



Each control is implemented in its own class, which analyzes the position of various bones in the skeleton and derives control data, which is consumed by the game code. These control classes are also known as *gesture recognizers*. Each gesture recognizer also provides a confidence value that tells the code how sure it is that the player is actually performing the required motion. It is up to the game code to determine how to use this confidence value, typically through play testing.

In this sample, the gesture recognizers are implemented using the Natural User Interface (NUI) Imaging and Skeleton Tracking APIs. After initialization of the NUI components, the update loop polls to see if a new frame of skeleton data is available. When new data is available, it is processed by the gesture recognizers. If there is no new data, the update loop immediately exits.

Controls

To control the car, the player must use the following gestures:

- Steering is controlled by the player holding an imaginary steering wheel in front of his or her body, and rotating it. This is implemented by the gesture recognizer code contained in `SteeringWheelFilter.cpp`.

Tip

Hand tracking can degrade slightly when the player's hands occlude their respective elbow joints. This can be minimized by decreasing the sensitivity of the steering wheel gesture. This forces the player to make slightly larger movements, keeping the elbows visible and reducing the number of occluded elbow frames.

- Acceleration and braking are controlled by the player moving his or her foot up or forward. This is implemented by the gesture recognizer code contained in `AccelerationPedalFilter.cpp`.
- Turbo boost is controlled by the player moving either hand backward toward the player's body and then pushing it forward again. This is implemented by the gesture recognizer code contained in `TurboFilter.cpp`.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Dynamic Speech Sample

This sample, Dynamic Speech, demonstrates how to create grammars at run time by calling the [NuiSpeech API](#). It also shows the equivalent XML for grammars created at compile time. In this sample and documentation, *dynamic grammar* means a grammar created at run time through the use of the NuiSpeech API. It does not mean a grammar that changes its active rule set at run time; changes to the active rules can be made to grammars created both at run time and at compile time. When both are an option, we recommend a precompiled grammar with inline lexicon data (a .cfxp file) because of the memory savings.

For more information about grammars and NuiSpeech, see [NUI Speech Grammars](#).

Difference Between Compiled and Dynamic Grammars

There is confusion of the overloaded terms *dynamic grammar* and *compiled grammar*. For any content that is known at creation time of the disc or that is downloadable content (that is, all content that is not created or downloaded at run time), both a compiled and a dynamic grammar can be suitable. Both grammars can turn on and off rules and grammars to meet the changing needs of the game. A title that uses hundreds of words and phrases throughout the life span of a game can use either type of grammar.

Using a compiled grammar can allow developers to leverage more of the speech tools, the grammar is inline, it can lower the overall amount of memory used. Using a dynamic grammar allows a game to create new rules at run time, from live data or from user-generated content; however, this can have a high amount risk.

Risk of Untested Grammars

A dynamic grammar has the ability to take in new phrases and words at run time. It is possible—but not recommended—to use this ability to allow a user or a title to create new words and phrases for recognition on the fly. However, doing so is not recommended, because such a grammar is untested. It is possible that a new and untested phrase may cause confusability to the engine and may reduce recognition results, including false recognition and false acceptance of phrases.

Side by Side

Finally, remember that your title can have multiple, active grammars running simultaneously. This fact allows compiled and dynamic grammars to be run side by side. After loading a dynamic grammar or a grammar that is not inline, the memory savings of a compiled, inline grammar is lost.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Filtering Sample

Demonstrates the trade-off between smooth response and latency that several different filters (and parameterizations of those filters) offer.

Description

Figure 1. The Filtering sample



There are multiple filters: an average filter (Blend), a velocity dampening filter, a Taylor Series-based filter, and a Double Exponential filter.

Controls

Move your right hand in a circular motion, for example, and watch the smiley faces track your hand.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Fitness Sample

Fitness demonstrates how to use the NuiFitness API to record fitness activities in automatic mode and how to retrieve records from a player profile.

General Description

Fitness makes use of the automatic mode to record fitness data for the currently tracked player. If the tracked player is associated with a profile, the fitness data is saved to the player's profile; later, this data can be examined from the Stats screen.
In order to generate multiple records rapidly, the fitness activities last one minute. This duration isn't representative of real-world fitness activities, which can last much longer.
The entire sample is operated using the Kinect Sensor. Green menu items can be selected by hovering the cursor over them for a few seconds. White menu items are not selectable.

Main Screen

The scene as seen through the Kinect Sensor is displayed in the center of the screen, and any tracked skeleton is overlaid on top of the image.
On the right is a histogram that shows the amount a Joules spent per second.
On the left are a series of buttons with the following functions:

- Sign-in**
Opens the NUI Sign-in UI to allow a player to sign in to a profile.
- Attributes**
Opens the NUI Fitness UI to allow a player to sets body attributes. Accessible only when a profile is associated to the active skeleton.
- Stats**
Shows the Stats screen that demonstrates how to retrieve records from a player profile. Accessible only when a profile is associated to the active skeleton.

Stats Screen

The histogram displays the Metabolic Equivalent of Task (MET), an average stored in the player's profile for the Fitness sample. The data is presented by year, month, day, or individual records, as selected by the user. When more records exist than can be displayed, users can select **More** to advance to the next block of data.
Below the histogram is a table that shows the overall time, Joules, and MET values that the player has recorded in the sample.
Use the **Back** button to return to the main screen.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

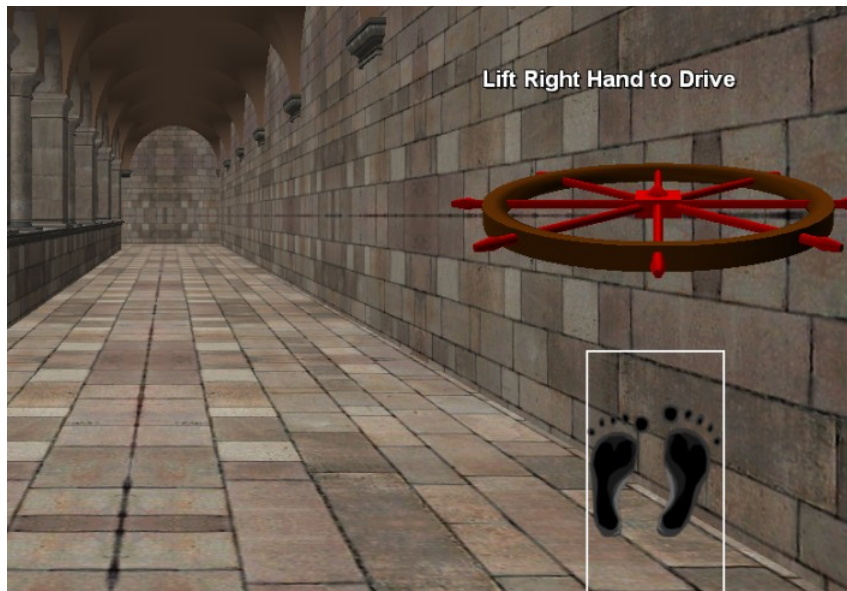
First-Person Sample

Description

First-Person uses a first-person view to simulate movement through a virtual 3D environment using a predefined set of gestures. First-Person implements support for the following actions:

- Moving forward and backward
- Strafing left and right
- Turning
- Rotating the view (also referred to as *aiming*)

Figure 1. The First-Person sample



Controls

Movement of the first-person view is accomplished by the following gestures:

- Moving closer or farther from the sensor array causes the first person view to move forward or backward.
- Leaning left or right causes the view to strafe to the left or right. An icon representing the feet of the user moves on the screen demonstrates the current amount the user is moving or strafing.
- Leaning forward or backward causes the first person view to pan down or up.
- Rotating the shoulders to the left or right causes the first person view to rotate to the left or right.

In addition, the height location of the perspective (that is, the Y component of the first person view) corresponds to the Y value of the head's skeleton position. This realistically simulates the action of the user jumping or ducking by correspondingly altering the height of the view's perspective.

Aiming within the sample is accomplished with the following gesture:

1. Lift your right hand until the turn wheel is engaged. The wheel fades from red to yellow as the engagement point is approached. The wheel changes to green when it is engaged.
2. Move your hand to the left or right to rotate the view of the character.

Using a specific hand gesture increases the responsiveness and accuracy of the system.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

First-Person Natural Sample

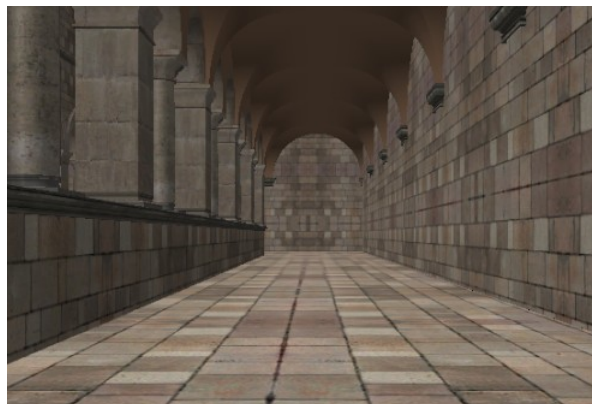


Description

First-Person Natural uses a first-person view to simulate movement through a virtual 3D environment using a predefined set of gestures. First-Person Natural implements support for the following actions:

- Moving forward and backward
- Strafing left and right
- Turning

Figure 1. The First-Person Natural sample



Controls

Movement of the first-person view is accomplished by the following gestures:

- Moving closer or farther from the sensor array causes the first person view to move forward or backward.
- Leaning left or right causes the view to strafe to the left or right. An icon representing the feet of the user moves on the screen, demonstrates the current amount the user is moving or

- strafing.
- Leaning forward or backward causes the first-person view to pan down or up.
 - Rotating the shoulders to the left or right causes the first-person view to rotate to the left or right.

In addition, the height location of the perspective (that is, the Y component of the first-person view) corresponds to the Y value of the head's skeleton position. This realistically simulates the action of the user jumping or ducking by correspondingly altering the height of the view's perspective.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Flight Sample

The Flight sample demonstrates control of an airplane using full-body gestures. In addition to being an exciting challenge, the sample shows how to derive airplane control from a skeleton and how to modify traditional airplane physics to make the airplane easier and more enjoyable to fly.

Description

The Flight sample features 3D graphics, with a chase camera following a red biplane. The biplane is always centered on the screen, and gestures control the direction of the biplane. In the sky, rings textured with a red and white checkerboard pattern provide the advancement goals for the game.

Figure 1. The Flight sample



Controls

To control the biplane, the user must use the following gestures:

- Pitch (nose up and down) is controlled by leaning your torso forwards and backwards.
- Yaw (nose left and right) is controlled by tilting your shoulders left and right.
- Throttle is increased by moving your right leg towards the camera, in front of your torso. To decrease throttle, move your right leg behind your torso.

All controls are cumulative—that is, pitching forward and then returning to rest will not return the biplane to level. You must pitch backward to level out.

Additional controls include:

- Press the BACK button to toggle the help screen visibility.
- Press the A button to reset the game to the initial conditions.
- Press the START button to pause the game.
- Press the Y button to change the current course. This automatically resets the game with the new playing field.

Gameplay and Scoring

To start the game, the user must increase throttle and then pitch upward to take off. Target the rings in order; the active ring will be solid and direction markers will show the correct way through the active ring. Once a ring is passed through or skipped, the following ring in the sequence will be activated. Once the last ring is flown through or skipped, the activity is complete. A ring is skipped when the biplane covers 60% of the distance between the active ring and the next ring.

For each ring successfully flown through, the user gets 100 points. If the ring is flown through sideways or backwards, the user only gets 50 points. For skipped rings, the user gains 0 points, but the next ring is activated. When the last ring is flown through or skipped, the user gets 1000 points.

In addition to scoring, ring completion is timed. The timer starts when the first ring is flown through or skipped, and the timer ends when the last ring is flown through or skipped.

If the user crashes the biplane, after a 5 second delay, the biplane resets to the starting position and the ring state & score resets. Press the A button to instantaneously reset the game.

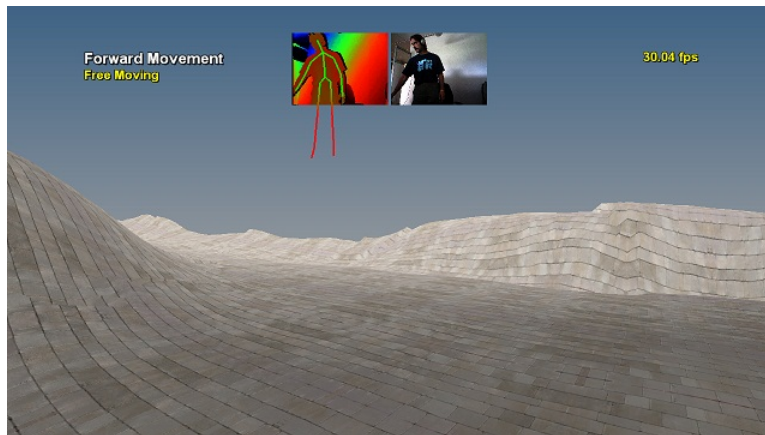
The current airspeed and altitude are displayed on the left and right of the screen. The current throttle setting is shown as a slider at the bottom left of the screen.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Forward Movement Sample

Description

Figure 1. The Forward Movement sample



Forward Movement demonstrates how to navigate forward through a 3D environment using a first person camera and simple body movements such as leaning to one side or turning their shoulders.

Controls

There are three methods to move forward through the landscape:

- Shifting his or her weight from side to side.
- Running in place.
- Jumping in place.

The player steers left or right by turning their shoulders to the left or right. There is also a mode where the player is on a rail, and can only progress along that rail.

Movement in the environment can be unconstrained or constrained to a single rail. Press the **A** button to toggle between these modes.

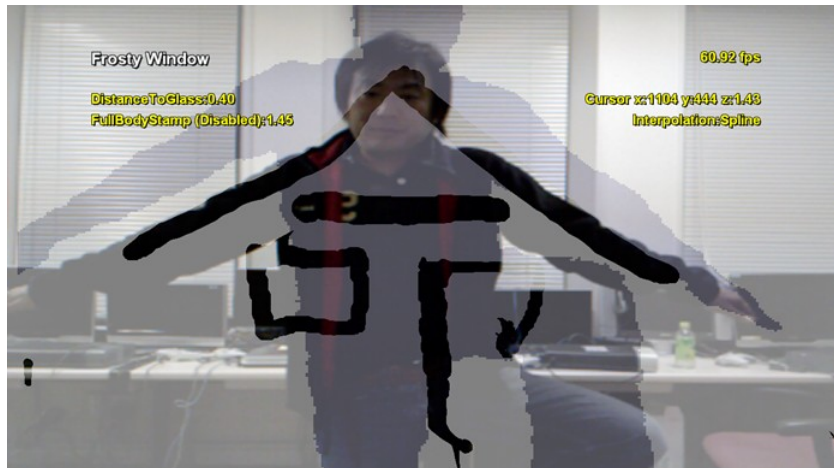
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Frosty Window Sample



Frosty Window is a simple miniature game that implements a pointer-based control by emulating a glass window with condensation accumulated on its surface. The player uses an arbitrary joint, recognized by the skeletal tracking system, to wipe off the condensation. This essentially equates to a simple monochrome paint-on-a-canvas application, where the arbitrary joint is the ink brush and the frosted window is the canvas.

Figure 1. The Frosty Window sample



Description

Frosty Window is based on the concept of drawing on a virtual canvas. To achieve this functionality, there were several problems that needed to be addressed. Note that there are several ways to go about solving these problems, and this sample demonstrates only one of many possible solutions.

Drawing Canvas

One of the first problems that needed to be addressed was to properly map the canvas from play space to rendered space. Space mapping is a common problem that all Kinect games share. In the case of this sample, the play space canvas is essentially limited to a finite plane anchored between the player and the sensor array. In screen space, this canvas is rendered as a transparent white layer superimposed on the color map captured from the sensor array. The white layer is the canvas in which the player can wipe off virtual condensation by controlling the pointer through Natural User Interface. Figures 2 and 3 illustrate the finite plane that maps to the canvas rendered on screen.

Figure 2. Side view of the drawable plane, highlighted in white

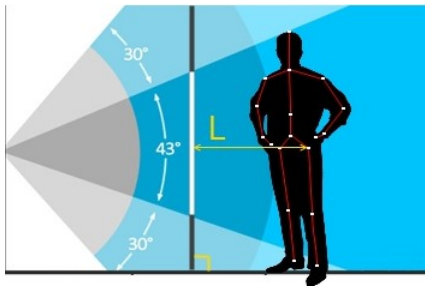
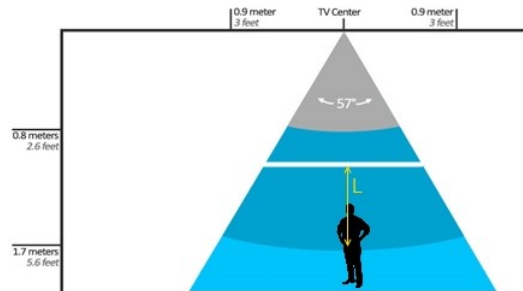


Figure 3. Overhead view of the drawable plane, highlighted in white



As the preceding figures show, the canvas for this sample is a finite plane:

- That is perpendicular to the floor plane.
- Where the distance from the player is an arbitrary length L from a predetermined point of reference.
- That inherently follows the player within the play space.

To alleviate occlusion issues, the sample changes the point of reference based on the X coordinate of the cursor. When the cursor is located on the right side of the spine, the left hip joint is used as the reference point. Also, when the cursor is located on the left side of the spine, the right hip joint is used as the reference point.

Polling the `NUI_SKELETON_POSITION_TRACKING_STATE` values for prospective reference joints may be a more compelling technique to determine the most appropriate reference point.

Pointing Indicator

This sample allows the player, at any given time, to use as the ink brush one of the following joints:

- `NUI_SKELETON_POSITION_HAND_RIGHT`
- `NUI_SKELETON_POSITION_HAND_LEFT`
- The average of `NUI_SKELETON_POSITION_HIP_RIGHT` and `NUI_SKELETON_POSITION_HIP_LEFT`

The active joint continues to be tracked, and its position is mapped to the cursor indicator, rendered on the screen. When the active joint is far away from the canvas, the pointer is invisible on the screen. As the active joint approaches the canvas, the superimposed pointer increases visibility until the alpha reaches a maximum: when in contact with the glass plane. This behavior allows the player to always be aware of the ink brush position, relative to the glass.

Implementing a curved surface map for the conversion of play space to screen space may be a compelling solution for a pointing indicator. This is because human arm swings trace an arc rather than straight lines. However, this sample constricts the player to drawing on a flat plane in order to simulate the construction of a glass window, and curved surface mapping is out of scope.

Full Body Stamp

In addition to using the hip or either hand as input to control the ink brush, the player can activate a full-body stamp mode. In this mode, the player uses his or her whole body like an ink stamp that can be pressed onto the canvas. The NUI Imaging API is leveraged to generate segmentation data from the stream captured by the sensor array and transposed onto the same glass canvas in screen space.

In play space, the full-body stamp canvas is a separate entity from the drawing canvas, described earlier in [Drawing Canvas](#). As mentioned earlier, the position of the drawing canvas is relative to the controlling player, and it moves in conjunction with that player. In contrast, the position of the full-body stamp canvas is relative to the sensor array, which means that it is essentially a stationary plane.

Like the drawing canvas, the full-body stamp canvas is positioned perpendicularly to the floor plane, and the Z coordinate is set to be an arbitrary distance from the sensor array.

Controls

The control pad can be used as follows:

- Press the left trigger and right trigger to cycle through the active drawing joint.
- Use the left thumbstick to adjust the distance between the player and the drawing canvas.
The default distance from the reference point is set to 0.4 meters.
- Use the right thumbstick to adjust the distance between the sensor array and the full-body stamp canvas.
The default distance from the sensor array is set to 1.4 meters.
- Press the X button to save the rendered image on the glass layer to the root directory of the DEVKIT volume.
- Press the Y button to activate and deactivate the full-body stamp.
- Press the A button to toggle between interpolation methods: linear or spline.
- Press the B button to reset the condensation on the glass to its initial state.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Grab Manipulator Sample



Grab Manipulator demonstrates how to combine hand open-closed detection with skeletal tracking to provide a natural method for object interaction in 3D.

Main Screen

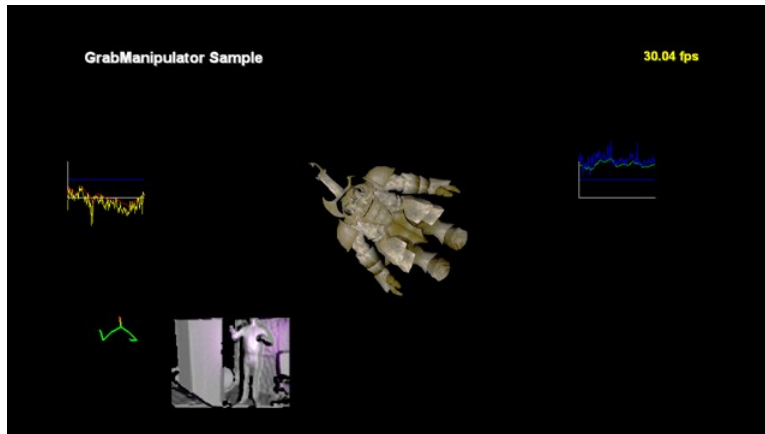
The main screen of Grab Manipulator displays the player skeleton and visualization, to ensure tracking is correct.

The *right* hand is used to rotate the model. When the right hand is closed, hand movement results in the model being rotated.

The *left* hand is used to move the model toward or away from the camera. When the left hand is closed, the model can be moved. When the hand is open, movement is ignored.

Graphs, on the left and right, show the history of the detection results for the left and right hands, respectively. Each graph displays two lines for filtered and unfiltered values. *Unfiltered* values are colored green for closed and yellow for open; *filtered* values are colored blue for closed and red for open.

Figure 1. Main screen



Details

The mesh rotation uses the vector in 3D space from the skeletally tracked shoulder joint to the hand. This vector is tracked for the current and previous frames. As the hand moves, the rotation angle is calculated using the [XMVector3AngleBetweenNormals](#) function, and the rotation axis is calculated by using the cross-product of this frame's vector and the previous frame's vector. No change is made if the angle between these vectors is less than FLT_EPSILON.

The mesh translation is even simpler, simply taking the delta in z between hand positions, frame to frame, and applying a scaling factor.

For more information on how hand open-closed detection works, see the documentation for the [Hand Opened-Closed](#) sample, as well as [Using Kinect for Hand-State Detection](#), a white paper.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

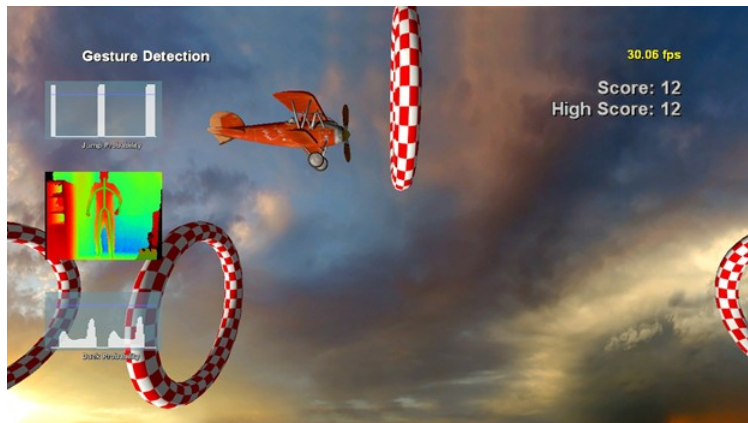
Gesture Detection Sample



Gesture Detection detects when a player jumps or ducks by implementing detection filters from several heuristics using skeleton joint data.

Description

Figure 1. The Gesture Detection sample



Gesture Detection implements detection filters using several heuristics, each of which calculates a probability value in the range [0..1]. For example, one heuristic might be calculating how probable it is that the player's spine is facing downwards, and another could be how probable it is that the player's hands are raised above his or her head.

A gesture detection filter simply combines the probabilities of these heuristics as a weighted average. The jump and duck filter probabilities can be seen in the debug overlays, with the horizontal blue line indicating the threshold that determines a Boolean decision for the detection. In order to get rid of false positives, the sample also implements heuristics marked as false detections. If any of these false detection heuristics show a 100 percent probability, the total probability will simply early out with a 0 percent probably for that specific detection filter. For example, a false detection heuristic for a ducking filter could be that at least one foot must be on the floor.

The **A** button can be pressed to reset the game. The sample shows a mini game where the goal is to jump and duck to fly the plane through the hoops.

How the Sample Works

The heuristics implemented can be separated into two groups, those that use angular space and those that use world space.

For any heuristics using world space coordinate calculations—head height—it is important to note that the skeleton data given as input to the detection filters should be tilt corrected. World space heuristics mainly examine joint heights, specifically the heights of the head, knees, and feet. In order to do any calculations based on height, first we need to establish a joint's baseline height, which is the normal height if the person is just standing in a neutral pose. If we knew the baseline height of the head joint, we can calculate the probability of a duck gesture by comparing the current head height to the baseline head height. If it is moving below the baseline by a certain amount, the player is probably ducking.

Determining a joint's baseline height is done by either taking the minimum or maximum, or the height if and only if the joint had the same height value within a small threshold over several frames. For example, for the head joint, we want to take the maximum height value. If the player jumps, the baseline will simply not update since the height values changes too much from frame to frame.

Test results show a 2.6 percent error for true positives in jumps, which are not detection jumps, and a 0 percent error rate for true positives for ducks, with very few false positives.

Reusing the Filters Code

The heuristics and filters are conveniently implemented in separate files to the sample so that developers can easily reuse the code. Simply include the GestureDetectionHeuristics.cpp and GestureDetectionFilters.cpp files in your project and declare the filters as follows:

```
// Declare detection filters
CDuckDetectionFilter    m_DuckGesture;
CJumpDetectionFilter    m_JumpGesture;
```

The filters then need to be initialized:

```
// Initialize detection filters
m_DuckGesture.Initialize();
m_JumpGesture.Initialize( pSkeletonFrame );
```

For every successful frame of skeleton data, update the filters:

```
// Low latency smoothing to mainly get rid of jitters
NuiTransformSmooth( pSkeletonFrame, NULL );

// Apply tilt correction since some heuristics operate in world space
ATG::ApplyTiltCorrectionInPlayerSpace( pSkeletonFrame, pSkeletonFrame );

// Update the filters with smoothed tilt corrected data
m_DuckGesture.Update( pSkeletonFrame );
m_JumpGesture.Update( pSkeletonFrame );
```

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

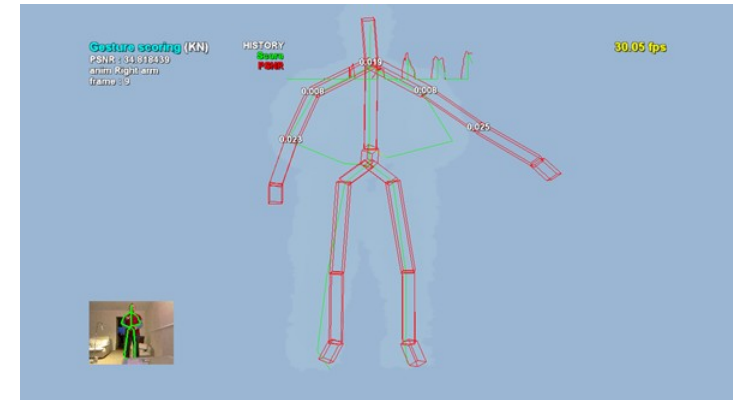
Gesture Scoring Sample

Gesture Scoring describes gesture scoring based on pre-recorded animation. Gesture Scoring demonstrates two variants of the animation scoring system, and implements a simple game mode that asks a player to match the animation exactly.

Introduction

Gesture Scoring starts in free mode where you can experiment with gesture scoring. The red skeleton plays the currently-selected animation repeatedly. You can use left and right buttons on the DPAD to change the animation. The green skeleton shows the current Natural User Interface (NUI) skeleton. The silhouette shows the current segmentation mask. The graph above the middle of the screen shows the history of score and peak signal to noise ratio (PSNR).

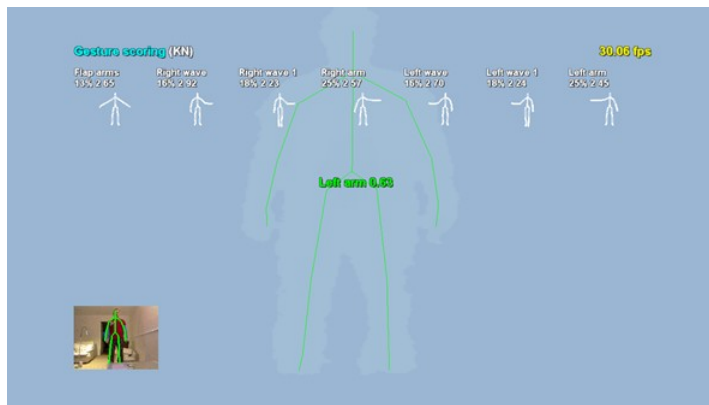
In this mode, Gesture Scoring is continuously scoring the NUI skeleton in green against the current frame of the animated skeleton in red. At every joint that is scored, there is a label that shows the angular distance between the two skeletons at that joint. You could also use Y to pause the animation, and try to match the static pose.



You can press the A button to start a *game* where the sample first shows the gesture it is expecting, and then scores the gesture. You have to try to match the timing and positions of the joints exactly to get the highest score. At the end of the sequence, you will see the score, and the sample will return to the default free mode.



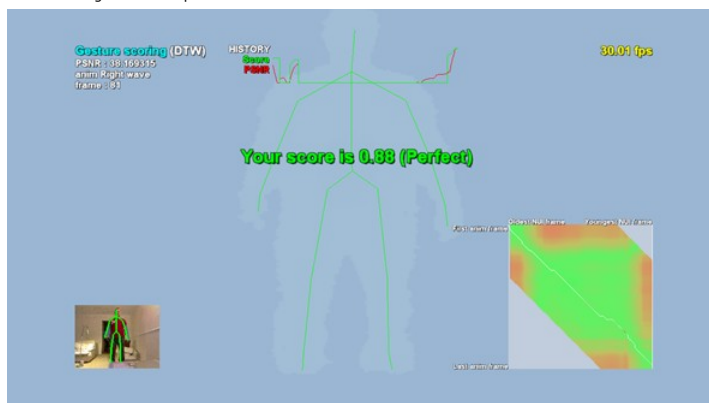
If you press the down button on DPAD in free mode, the sample enters the detection view. If you have a KN classifier selected, you will see something like this.



Gestures that are tracked currently are shown along the top here as little animated icons. They will turn red when there are no trackers associated with them. Otherwise, they will perform the animation repeatedly to the latest detected frame. In other words, if you stand still, they will move to about frame 10, and then reset back to the starting point. If you start doing the gesture, one of the little figures follows it through with you. If the gesture is complete, it is shown as detected in green text.

There are only two types of waving gestures. In a real application, you need to record several people repeating that gesture several times, and then try matching all of them.

You can toggle the classifier with the X button. If you choose the DTW classifier, you will see the DTW cost matrix in the bottom right corner of the screen. Green cells show low cost, and red cells show high cost. The path that minimizes the cost is shown in white.



Note that the DTW classifier is better at recognizing slower or quicker gestures, but it is more expensive.

Skeleton Representation

NUI skeleton data is returned as a set of joint positions in world space. The animations in the sample use the BVH data format. They are recorded using Xbox Studio, and then are converted to BVH using XED To BVH sample. Generally, BVH data can come from anywhere.

BVH skeleton is different from NUI skeleton, so at least we need to implement retargeting. This retargeting is done in the **AnimationAdapter** class in the sample. It is limited to supporting skeletons output by the XED To BVH sample.

Another important point about the animation is that it is going to have different bone sizes from the ones with which we're going to compare it. Thus, we need a way to be able to match two NUI skeletons with different bone lengths.

To accomplish that, the sample converts each joint from position representation into a normalized offset representation. It simply stores the normalized directions in world space to child joints instead of their positions. The NUI skeleton is processed in the same way in every frame, so we end up with two representations we can actually compare. Also, such representation is easier to rescale for different bone lengths for depth buffer querying.

Error Metric

The error metric in the sample is based on calculating the angular difference in the local space of the joints between the two skeletons. It returns a measure of how far the two skeletons are apart. It is simple, easy to understand, and works for the purposes of this sample. The titles can implement a per-joint scoring metric with weighting or with any other metric that fits it.

K-Nearest Classifier

K-Nearest neighbor is a method for classifying objects based on closest examples. In this sample the player is expected to repeat the animation exactly, so this allows for a brute force algorithm to match the current NUI skeleton with a frame we are expecting at that time. This works well in paused mode. However, if we try to match the moving animation, the player is always going to be doing it too fast or too slow.

The simplest solution for a moving scenario is to match the current frame with a range of animation frames around the expected frame. That allows for a certain amount of lag from the player. This is exactly what this classifier does.

This solution is inexpensive, and it handles some time variation in the game mode, but it won't detect a too-fast or too-slow gesture.

Dynamic Time Warping Classifier

To deal with the problem of matching two time sequences recorded with different and variable speed, you can use a dynamic time warping algorithm. It helps to find the correspondence between two time series by finding a *warping path* that makes one sequence look the most like the other.

There is one problem with the previous classifier. To detect a gesture started at an arbitrary point in time, we need to start a new classifier every several frames, and run them continuously.

We can accomplish the same goal by using just one DTW classifier per animation. DTW classifier works by building a cost matrix first, and then finding a cheapest path through that matrix. The cost matrix comprises N rows and M columns, where N is the number of frames in the animation and M is the length of NUI history array. The cheapest path gives us a correspondence of each animation frame to each NUI frame. This means we can expect a perfectly performed gesture to have a diagonal path.

We can track a gesture by constantly monitoring one DTW classifier per animation, looking for a path that is cheap and straight enough. When we find such a path, we can detect the gesture. Having the path information tells us whether the gesture is performed quickly, slowly, or in perfect timing.

Performance

Obviously, we can only match so many animations using the described methods before the CPU cost becomes too high. So if we recorded 32 animations of four different people doing a *right wave* gesture, we should be at the limit of what the previously-described matching algorithms can do.

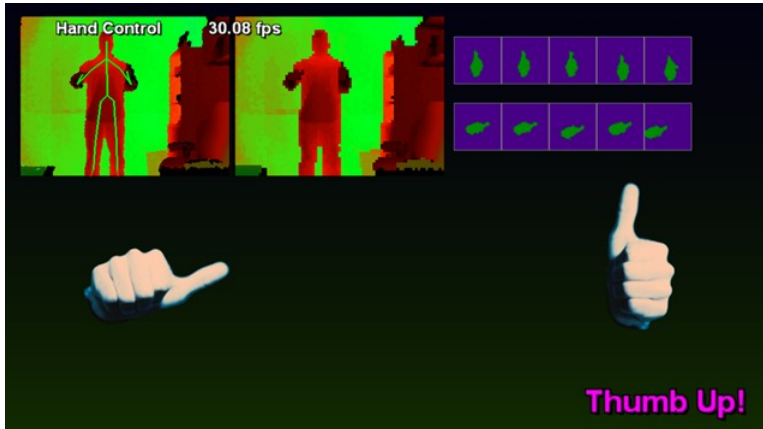
Machine learning methods can help with reducing the amount of data to be matched. Reducing the data is outside the scope of this sample. However, the methods described here are still important because they could help refine the results of the matching step. For example, if the fast algorithm that can check 1,000 gestures quickly finds 10 gestures that the player may be doing, DTW helps to pick the one with the highest score among them in a reasonable period of time.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Hand Control Sample

Description

Hand Control uses the second-order image moments to calculate the orientation of the segmented hand in image space. ATG Hand Refinement is used to center on the end of the hand and return a mask of hand pixels.



The mean is then calculated for the mask. Next, the central image moments needed for orientation are calculated. Finally, the skew along the axis of orientation is calculated to determine if the angle of orientation needs to be flipped 180 degrees.

Controls

To use this sample, point your fists forward with your thumbs extended. Rotate your hands, and the images of the thumbs on the screen will rotate based on the orientation of your hands.

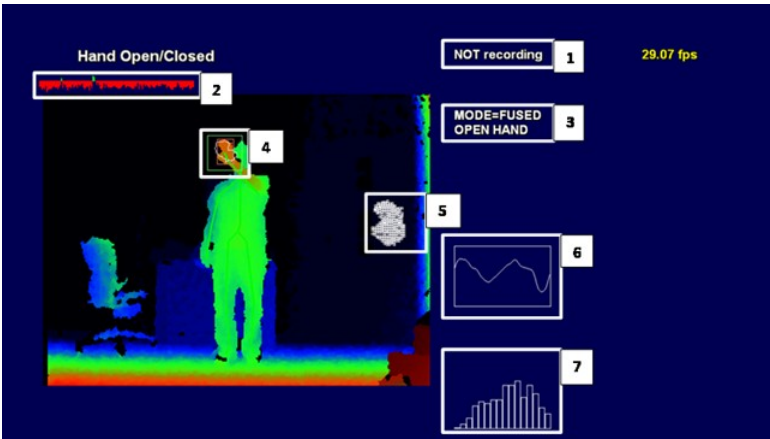


© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Hand Opened Closed Sample

This sample demonstrates a way to detect whether a player's hand is open or closed using machine learning. Machine learning algorithm called AdaBoost is used to train the detector.

Figure 1. Main screen



Screen element	Description
1	The sample can record the input data and write the .XED files onto the development kit's hard drive along with the detector confidence level per frame in the title data.
2	This graph shows history of the detection results. The red bars denote negative detector result and the green bars denote positive detector results. Note that "result" here just means the floating point value returned by the detection function. It's possible to attach "Hand Open" meaning to a positive or a negative value.
3	There 4 modes in this sample. Each mode uses its own detector. Fused <i>Fused</i> mode uses a detector that is trained on open hand data with a tag of 1 and closed hand data with a tag of -1. The training data contains only open and closed hand. Note that with this detector any hand state is going to be classified as either "open" or "closed". Open hand <i>Open hand</i> detector is trained so that open hand data has a tag of 1 and non-open hand data (including side views, partially closed hand and closed hand) has a tag of -1. Closed hand <i>Closed hand</i> detector is trained so that closed hand data has a tag of 1 and non-closed hand data (including side views, partially closed hand and open hand) has a tag of -1. Two experts <i>Two experts</i> mode uses both <i>Open hand</i> and <i>Closed hand</i> detectors and chooses the strongest response to return as a result. It returns 0 when the hand neither open nor closed.
4	The green square outline here shows the boundaries of the flood fill area, the red square outline shows the boundary of the found hand voxels, and the white outline shows the current hand contour.
5	These are the hand voxels.
6	This displays the contour of the hand. The contour starts at the wrist and goes in the counter clockwise direction around the hand.
7	This displays the histogram of distances from the voxels to the centroid.

Training Process

Before it can be used, the detector needs training. There are two tools in the sample to help with the training of the detector. Full source code is provided.

XEDToHOC

XEDToHOC takes an Xbox Studio recording (.xed file), and produces a file with the hand data that can be consumed by the trainer. Internally it essentially runs the hand data acquisition step of the detector, exactly like it would happen in run time.

HOCTrainer

HOCTrainer takes a list of files output by XEDToHOC and performs AdaBoost training of the detector followed by an optimisation pass using RBoost. The produces output file is loaded by the detector in runtime.

The training process starts with recording a training set of Xbox Studio captures. Each recorded .XED file has an associated tag – either "Hand Opened" or "Hand Closed". The recordings should contain data for either opened or closed hand, not the mixture of the two.

It's a good idea to record a variety of people at different distances during this step.

Once you have the training data ready, convert the .XED files to .OC files by using XEDToHOC. Then, create a text file lists your training set files with the tags, like the following (traininglistAll.txt):

```
open.oc,1
open4.oc,1
open3.oc,1
open2.oc,1
open1.oc,1
closed.oc,0
closed2.oc,0
closed3.oc,0
closed4.oc,0
```

Finally, run HOCTrainer on the list file to generate the detector file, using a command line like the following:

```
"HOCTrainer.exe" "traininglistFused.txt" opt.hoc > opt.hoc.txt
```

You can either use XStudio to save .XED recordings, or the sample itself. The sample will also write detector's confidence in the title data with each frame of .XED. That information can then be used to find which examples the detector failed on, and feed them back into the trainer.

Detector

Detection process starts with collecting voxels for the hand. It uses the hand position returned by skeletal tracking to start a flood fill operation on the texels of the depth map. The texels that get flood-filled during this step are converted into voxels and passed to the next stage.

Once the hand voxels are obtained, the data for the classifiers can be computed. This is a two-step process. In the first step the histogram of distances from voxels to the hand centroid is calculated, also the 2D contour of the hand, the ratio between the area of the hand in XY plane and the depth span of the hand and angle between the forearm and the hand joints at the wrist. In the second step the mean and variance of the histogram is computed, along with some moments of the contour graph. This data is now ready to be consumed by the classifiers.

The last stage feeds the computed data to the classifiers. Each classifier performs a simple threshold comparison on some piece of computed data. Discrete AdaBoost is used in the sample, so the output of each classifier is either -1 or 1. The outputs of the classifiers are then weighted, summed up, and the single value is returned to the caller.

Classifiers

There are several classifiers in the sample. There are a number of classifiers working on the histogram of distances from the voxels to the centroid of the hand. For opened and closed hand the shape of the histogram, and therefore its mean and variance, is different. A contour based classifier tries to find how symmetric the hand is because, again, closed and opened hands will have different symmetries. Even Skeletal Tracking could be used to build a classifier – the ratio between the length of the hand and the forearm will vary based on whether the user's hand is opened or closed.

References

"A short introduction to boosting", Freund and Schapire <http://www.site.uottawa.ca/~stan/csi5387/boost-tut-ppr.pdf>

"Gesture Detection using Machine Learning," a presentation at [Gamefest](#) 2011.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Hand Refinement Sample



Controls

Hand Refinement demonstrates how refinement of hand position works.

Figure 1. The Hand Refinement sample



As you move your hands around the screen, the red icon shows the raw data, and the yellow icon shows the refined position.

Overview

Hand Refinement works by using the 80×60 depth map to make sure that the hand joints are located on the hands. The average pixels around this point in the 320×240 depth map are then used to smooth the position.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Head Orientation Sample

[See Also](#)



The Head Orientation sample demonstrates how to use the [Head Orientation](#) APIs to retrieve information about the player's head orientation. The sample only tracks heads that correspond to a fully tracked skeleton—that is, the sample does not track more than two heads.

The sample shows the following Head Orientation components:

Yaw

As a gauge above the player's head

Pitch

As a gauge to the left of the player's head

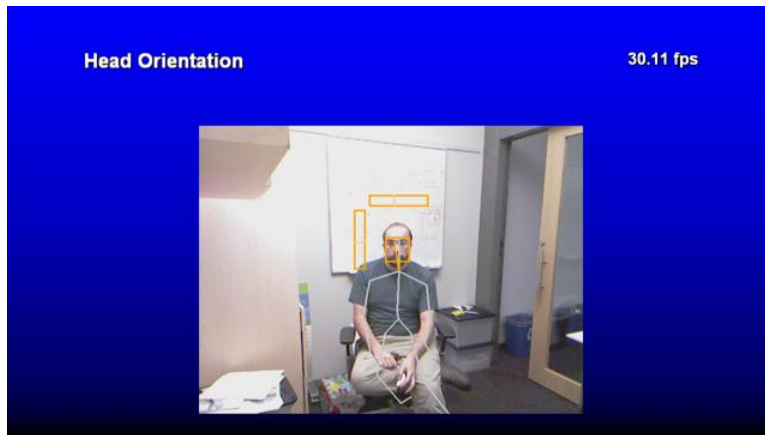
Roll

As a 2D vector originating at the player's neck

Facebounds

As a box around the player's face

Figure 1. The Head Orientation sample



To initialize Skeleton Tracking and Head Orientation, this sample calls [NuiInitialize](#) with the [flags](#) `NUI_INITIALIZE_FLAG_USES_SKELETON` and `NUI_INITIALIZE_FLAG_USES_HEAD_ORIENTATION`.

To enable Skeleton Tracking and Head Orientation, this sample calls [NuiSkeletonTrackingEnable](#) and [NuiHeadOrientationEnable](#).

To retrieve the head orientation data, the sample calls [NuiHeadOrientationGetNextFrame](#), passing a [NUI_HEAD_ORIENTATION_FRAME](#) structure which will contain a [NUI_HEAD_ORIENTATION_DATA](#) structure for each tracked head. [NuiHeadOrientationSmooth](#) then reduces jitter between frames.

See Also

[Head Tracking samples](#)
[Head Orientation Tracking](#)
[NuiHeadOrientation Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Head Tracker Sample

[See Also](#)

Head Tracker shows how to change perspective based on head location, on a standard TV. This sample shows how to set up view and projection matrices based on the location of the head joint returned from the Skeleton Tracker. It doesn't use the Head Position Tracking API. You can use these matrices to create the illusion that the player is looking into a 3D television.

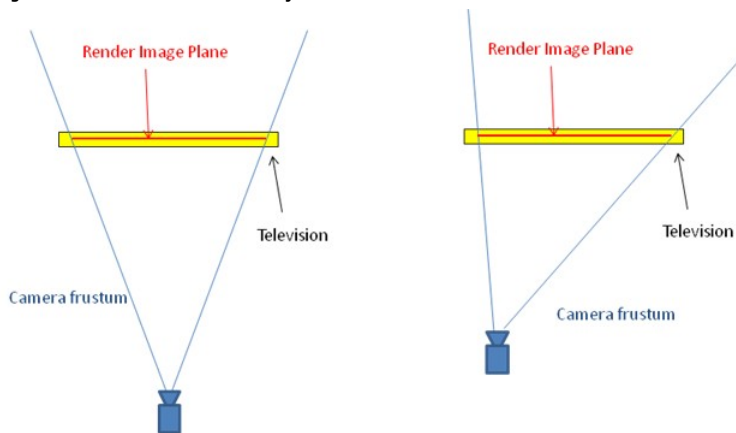
Controls

As the player's head moves, the camera moves back and forth.

View and Projection

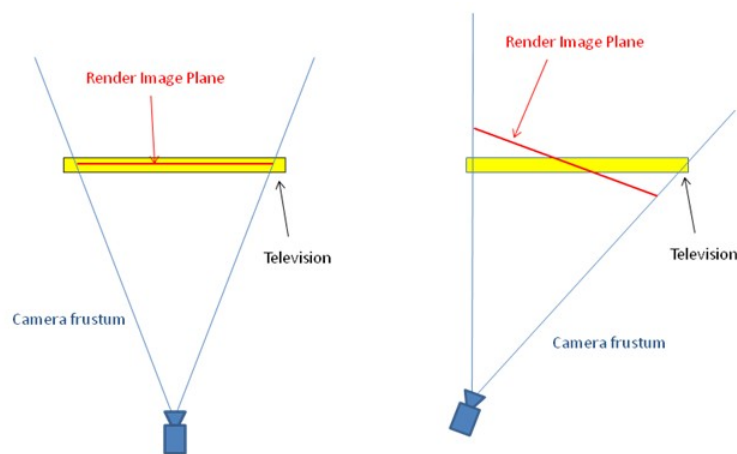
Head Tracker creates the illusion that the near plane is the same as the TV screen. When objects are in front of the near plane, they appear to come out of the television. To keep the near plane from clipping geometry, an off-center projection is used.

Figure 1. HeadTracker Off-center Projection



If you don't want to use an off-center projection, you can create a similar effect by strafing the camera back and forth when the head joint moves. A small amount of rotation can also be applied. This gives most of the illusion.

Figure 2. Head Tracker Projection—Strafing the Camera



See Also

[Head Tracking samples](#)

[NUI Skeleton Tracking](#)

[NuiSkeleton Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Identity Sample

[See Also](#)

Description

The Identity sample manages player identification, enrollment, and sign-in by using the Natural User Interface (NUI) [NuiIdentity](#) API, including [NuiIdentityIdentify](#), [NuiIdentityEnroll](#), and [XShowNuiSignInUI](#).

Figure 1. The Identity sample



The Identity sample simulates a party game in which many players take part in short activities, often competing against each other. Players usually join for a few rounds and then switch places with other participants. In this sample, the object of the game is to draw a circle in the air.

Run and interact with this sample to see the flow of Identity behavior that meets the TCR requirements. To fully exercise this sample, have one person create and use a biometric-enabled gamer profile. Have another person use a gamer profile that's not biometric-enabled. Have another person use a guest identity, such as Player 1. One person can partly exercise the sample, though having at least 3 people is best, to demonstrate keeping track of multiple players' identities.

Features of this sample:

- The sample tracks up to eight enrolled players.
- The sample supports up to two active players at a time. Active players are players who are fully tracked, linked to a player skeleton, and engaged with the sample.
- Any player who is signed-in (whether biometrically or manually) before launching the sample, is automatically enrolled and signed-in after identification is completed by the sample.
- A player is automatically identified when the player walks in front of the Kinect Sensor, but the player must signal intent to play by making the "intent to play" gesture (described in [Main Screen](#)). The player is enrolled when the gesture is complete.
- Initially, an enrolled player receives a unique player name and one of eight identification colors. If the player signs in, the unique name is replaced with the player's associated gamertag.
- If an enrolled player steps out of the field of view, that player is removed from the list of active players, but the player's data is kept if he or she is one of the eight most recent players.
- A returning player is automatically matched to his or her related player data.
- If the NUI identity system finds that the player is not enrolled, but finds a matching biometric-enabled gamer profile, the player is automatically enrolled and signed-in.

Gestures Used in This Sample

The sample is controlled by the specific motions described in the following table.

Motion	Description
Air Circle	Extend your right arm in front of you and draw a circle (clockwise or counterclockwise).

Flick Left	Start with your right forearm parallel to the floor (and slightly to the right of your elbow) and facing toward the Kinect sensor array. Complete the gesture by swiping your hand to the left as far as possible.
Flick Right	Start with your right forearm parallel to the floor (and slightly to the left of your elbow) and facing toward the Kinect sensor array. Complete the gesture by swiping your hand to the right as far as possible.
List Pick	Start with your right forearm parallel to the floor and facing the Kinect sensor array. Highlight different list elements by moving your hand up and down between your shoulder and elbow. Keep an item selected for two seconds to pick that item.
Stop	Raise both hands over your shoulders and hold the position for two seconds.
Wave	Raise your right hand over your head while keeping your left hand down, alongside your body.

Screens

- [Main screen](#)
- [History screen](#)
- [Correct Your Identity screen](#)
- [Pause screen](#)

Main screen

The main screen displays skeletons for each player currently tracked by the system. Each skeleton is displayed using the color assigned to the player when the player enrolled. It displays player-related information, in the player skeleton's color, such as the player's name (either assigned or the player's gamertag, depending on the situation); enrollment status; air circle count; and various messages from the [NuiIdentity](#) API to help guide the player during identification and enrollment.

Players will not be able to interact with the sample until they access either the sign-in or identity correction UI. Accessing either of these UI elements is equivalent to showing intent to play.

Gestures in the Main screen:

Gesture	Result
Stop	Displays the Correct Your Identity screen, allowing the initiating player to enroll as another player identity.
Flick right	Navigates to the History screen.
Wave	Displays the sign-in UI if the initiating player is enrolled.

History screen

The history screen displays statistics for the last eight enrolled players.

Gestures in the History screen:

Gesture	Result
Flick left	Navigates to the main screen.

Correct Your Identity screen

Allows a player to request a different enrollment ID. Only gestures from the initiating player are recognized. Gestures from other players are ignored.

To display the Correct Your Identity screen, in the main screen, perform the Stop gesture (raise both hands above your head).

Gestures in the Correct Your Identity screen:

Gesture	Result
List pick	Selects a new enrollment slot. Selecting New Player (the last slot) forces the sample to use the next available empty slot. If there are no empty slots, the player with the longest inactivity is bumped.
Stop	Cancels enrollment and returns to the main screen.

Pause screen

Allows a player to pause the sample and open the Guide using the system gesture. Only gestures from the initiating player are recognized. Gestures from other players are ignored.

To display the Pause screen, perform the System gesture.

Gestures in the Pause screen:

Gesture	Result
List pick	The player can open the Guide by selecting the Guide option or exit the menu by selecting Resume Sample .

See Also

[NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Joint Constraints Sample



Joint Constraints demonstrates a method of applying a simple constraint model to skeletal tracking data in order to give a more natural looking result. A set of cone constraints are placed on the joints of the skeleton. When a bone moves outside its assigned cone constraint, it is adjusted to lie on the closest edge of the cone. The constrained skeleton is then remapped to an Xbox Avatar.

Description

Joint Constraints produces a more human-like skeleton when skeletal tracking is degraded, such as during occlusion and field-of-view clipping.

Figure 1. Joint Constraints



It does this by using a simple biomechanical model for human skeletal joints. Each joint is assumed to be constrained to rotate in a conical direction. While this assumption is not true for all human joints, it is simple to calculate, and provides a good approximation for most joints. Even hinge joints, such as the elbows and knees map well to this approximation. While they can't move within a cone by themselves, with the rotation of the shoulder and hip, the lower arm and leg bones do in fact move in a roughly conical shape. Actual rotations on the elbow and knee joints can be corrected in a later skeletal retargeting step.

Cone constraints need rotation information, but skeletal tracking reports only joint positions. Therefore, the first step of constraining joints is to extract joint rotations. The world space direction for a bone is calculated from its start and end points, and then is converted into a local space rotation by finding the shortest rotation from the skeleton's bind pose orientation. At this point, the constraint cones have a local frame of reference for checking for constraint violations. Each joint is checked against its corresponding cone constraint to determine if it lies inside or outside the cone.

If a joint is detected to be in violation of its constraint, it can be corrected in a number of ways. Some possible options include using the last known good orientation, procedurally animating the joint, or blending into an idle animation. In the sample, the joint is adjusted to the nearest orientation that doesn't violate the constraint. In other words, joints that lie outside the constraint cone are adjusted to the nearest orientation on the cone.

Constrained joint rotations can be used directly in skinning a character, or joint positions can be reconstructed from the rotations for later use. Reconstructed positions can be used as a replacement for raw skeletal tracking positions. In this mode, the joint constraint code can be thought of as a joint filter. Once the positions have been filtered, they can be used directly, or used in other ways, such as by sending them to the avatar retargeting library.

The sample demonstrates joint constraints by displaying four avatars and a skeleton. The skeleton shows raw tracked joints in blue with wireframe cone constraints on the joints. When joints lie outside the joint constraint, they appear red. A white skeleton, which is reconstructed from constrained joints, is overlaid on top. To the left of the skeleton are two avatars remapped from skeletal positions using code from the Avatar Monkey-See-Monkey-Do sample. The leftmost avatar uses the constrained joints, while the other avatar uses the raw skeletal tracking joints. On the right side of the screen are two additional avatars that are retargeted with the **IXAvatarNuiMapper** interface. The rightmost avatar uses constrained joints, while the other avatar uses raw skeletal tracking data.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

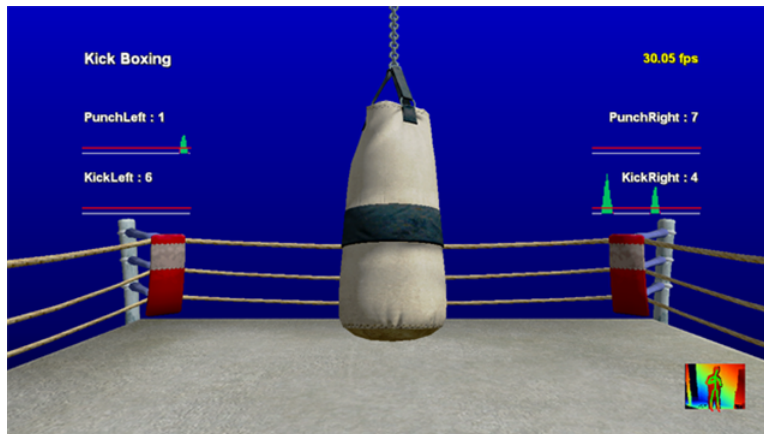
Kick Boxing Sample



Description

The Kick Boxing sample shows how the time consuming art of coding, maintaining and fine tuning input parameters and thresholds for gesture detection, can be replaced by an automatic process where a computer simply finds the best input parameters and thresholds. For this sample, punching and kicking gestures have been trained so that the player can punch and kick the punching bag.

Figure 1. Screen shot of Kick Boxing



Even though these are binary classifications, interesting analog signals can still be derived from the detection; for example, the sample shows a different behavior when hitting the bag softly versus hitting the bag with a lot of force.

Gesture counters are displayed together with a signal of the confidence value. The horizontal red line indicates the detection threshold. Whenever a confidence value is higher than the detection confidence value, the signal is shown as green, indicating that it will increase the gesture counter.

Button	Action
A	Toggle model of data collection.
B	Toggle joint smoothing.
X	Toggle debugging visualization.
Left shoulder	Decrease detection threshold.
Right shoulder	Increase detection threshold.

How the Sample Works

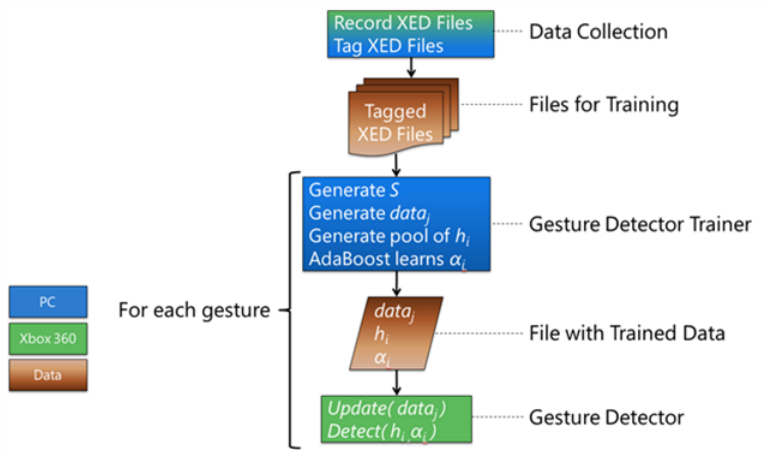
Since this is a data driven system, we need to first collect data. This is done by simply running the KickBoxing sample on the Xbox 360 and toggling it to data collection mode, using the **A** button. This will record XStudio files which can then be tagged by using [XedFileTagger](#), a sample. The tagging process is where we add meaning to the data by indicating which frames represents which gestures. For an example of how tagging can be done, [load the example data files of this sample into XedFileTagger](#). Note that tagging is essential to the results for accuracy and reducing latency.

Now that we have all the data, we need to separate the data into a training set and a testing set. For the training of this sample we used 66% of data for training and 33% for testing.

The gesture detector trainer uses the Adaptive Boosting (AdaBoost) machine-learning algorithm. It takes as input S , a set of labeled examples. These labeled examples are generated by GenerateLabeledExamples.exe, which simply compares the gesture you want to train for and the tags that were added with XedFileTagger. TrainGesture.exe now uses the ground truth labeled example data to do the training. It automatically generates $h_i(x)$, a pool of weak classifiers, and during AdaBoost finds α_i , the weights, so that the output of AdaBoost is $H(x)$, a strong classifier, which is simply the weighted sum of the weak classifiers.

For each gesture a data file is saved containing the data for the strong classifier. This data file is loaded at run time on the console and used in an update and detect function. Since the results of the detection are per-frame results and not per-gesture results, the results need to be filtered. The detector has a simple filter based on the sum of a sliding window of values which works well for all gestures tested, but you can choose to implement your own filter on the raw data.

Figure 2. System design diagram for training



For more information about using machine learning to tune gesture detection, see "Gesture Detection using Machine Learning," a presentation at [Gamefest 2011](#).

How to Use the Trainer

1. Record XED files of players doing positive and negative examples of gestures.
2. Use [XedFileTagger](#) to tag these XED files.
3. Place some of these tagged files in the \Training\ folder and some in the \Testing\ folder.
4. Customize TrainGestures.bat and run
5. Copy the .gesture files to the development kit.
6. Sample application on development kit loads .gesture files and do the detection.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Lean Detection Sample

If you simply need to track a player's movement, such as leaning left or right, it might be easier to use raw depth data rather than Natural User Interface (NUI) skeletal tracking.

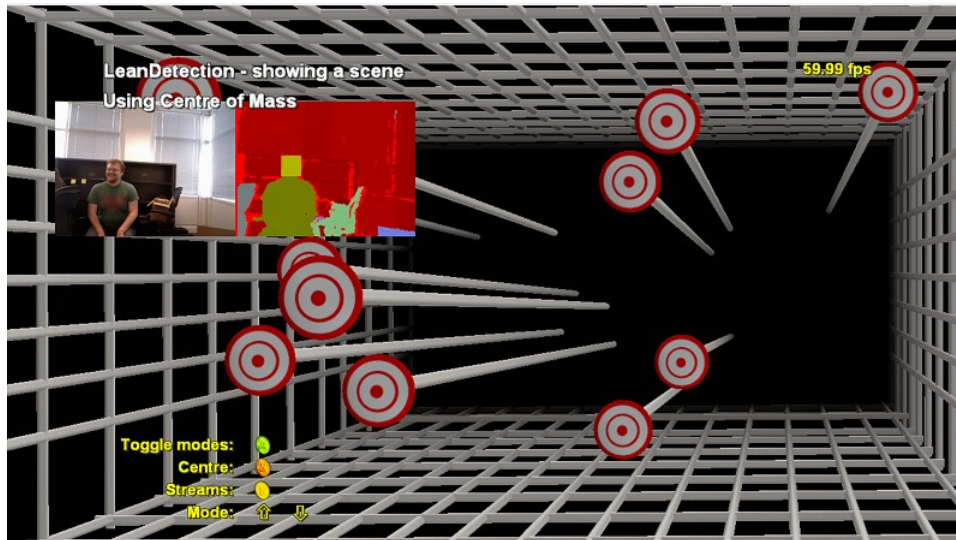
Description

Lean Detection shows how to use the raw depth data rather than skeletal tracking to determine whether a player is leaning left or right. The easiest way to do this is to remove background objects, and then detect the moving center of masses of foreground objects. Lean Detection expands on that idea by trying to identify the moving shapes as humans.

To do that, the depth buffer first is split into depth slices. Then, each slice is split into image islands. Finally, each island is classified against a signature silhouette of the human head. At this point, the sample looks at either the movement of the center of mass of the head or at the difference in depth around the head between the two frames.

Because the sample doesn't use skeletal tracking, the latency disappears, and the system reservation is greatly reduced, although you pay a price in robustness.

Figure 1. Lean Detection



Controls

To control the amount of lean, the user moves his or her head left and right. Pressing A toggles between showing the scene and the stream buffer. Pressing B centers the camera. Pressing Y toggles the stream display in the scene view. Pressing up and down on D-pad changes detection algorithms.

Further Information

The following sections provide greater detail of various technical aspects of the sample. You don't need this information to use Lean Detection.

Depth Slicing of the Depth Buffer

To detect slices in the depth buffer, the sample builds a histogram of depth and tries to detect spikes. If the sample detects a spike of a sufficient height and width, it generates a slice by storing near and far distance to the spike.

Islands Creation

Having a depth slice, the sample runs a flood fill algorithm to isolate disjoint islands in the image. The flood fill algorithm fills connected pixels with an island ID.

Signature Matching

Each island is taken through a signature detection algorithm that first builds the left and the right edge of the silhouette. The sample then builds a single ID profile by taking the difference between the left and right edge at each row of the silhouette. This profile is matched to a stored signature at different scales. If the island passes the signature tests, Lean Detection runs a neck-detection heuristic to find the head.

Lean Detection

Once it finds the head's position, Lean Detection is ready to detect leaning. The sample uses two methods to do that—delta depth checks and center-of-mass tracking.

When the player moves his or her head, the difference of this frame's depth and the previous frame's depth is positive on one side and negative on the other. The sample counts the number of positive and negative pixels to determine where the object is moving.

Center-of-mass tracking calculates the position of the center of mass of the player's head, and it tracks it over the frames to give the lean amount.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Lobby Chat Sample with Enhancements for Kinect



The [LobbyChat](#) sample was enhanced to provide basic support for vocal communication, using the Kinect Sensor, over Xbox LIVE. The new functionality of Lobby Chat Sample with Enhancements for Kinect demonstrates the simplest way to communicate using an Xbox 360 Headset or the shared microphone array in the Kinect Sensor over Xbox LIVE.

Kinect-Specific Enhancements

The enhancements in Lobby Chat Sample with Enhancements for Kinect distinguish a headset from the Kinect Sensor. This is helpful for titles that react differently based on the input device used for player chat. The functionality is implemented by the [IsHeadsetPresent](#) and [IsSharedMicPresent](#) methods.

While the sample is running, the color of player IDs in the lobby indicates the chat state of the players.

For local players:

- Red indicates the player is talking.
- White indicates the player is not talking, but has a voice chat headset connected.
- Green indicates the player is not talking, but is using the Kinect sensor array for vocal communication.
- Grey indicates the player is in the lobby and has no connected chat device.

For remote players:

- Red indicates the player is talking.
- White indicates the player is not talking.
- Black indicates the player is muted.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

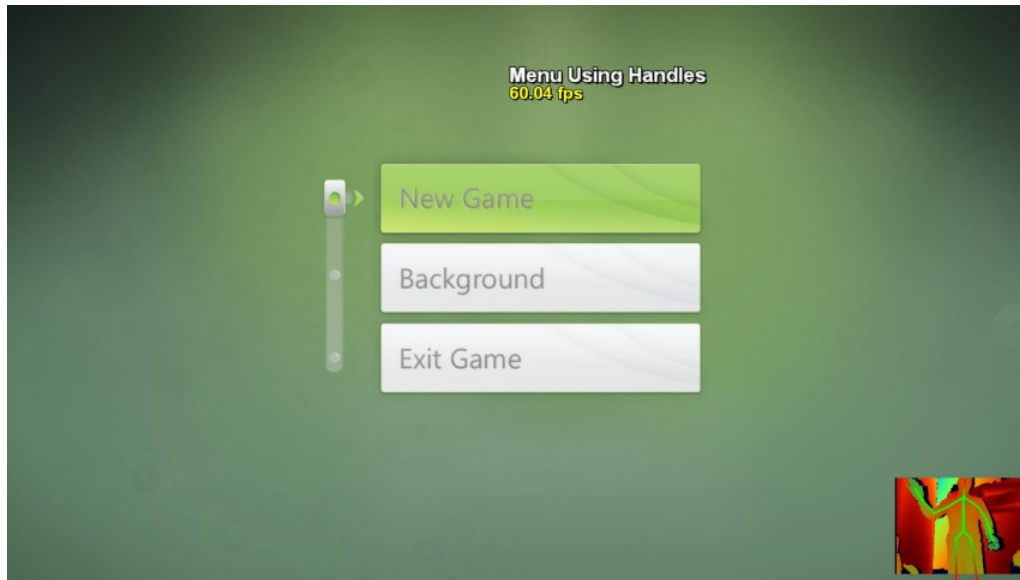
Menu Using Handles Sample



Description

Menu Using Handles uses the Handles API to do menu navigation. The menu buttons and handles are rendered as textured quads.

Figure 1. Menu navigation



Controls

Lift either hand over a grip to attach. Once attached, visual cues show how the handle can be moved along the rail.

Standard list

Attach to the white particle in the top left corner. Move the grip down to highlight the menu item. Move the handle right to select.

Back Button

Attach by moving the cursor over the button. Move the button left to activate it.

Standard T Pad

Attach by moving the cursor to the center of the control. Move left or right to scroll. Move up to select.

Single Element Standard list

Attach by moving the cursor over the button. Move the button right to activate it.

Game States

The sample has four game states:

GAME_STATE_MENU

From this screen you can go to the confirmation screen by selecting NEW GAME, the background screen by selecting BACKGROUND, or the game state by selecting RETURN.

GAME_STATE_GAME

You can go to the menu screen by pressing START.

GAME_STATE_BACKGROUND

You can select a color and go back to the menu, or simply go back to the menu by pressing BACK.

GAME_STATE_CONFIRM

You can confirm the new game by pressing the START button, or go back by using the BACK button.

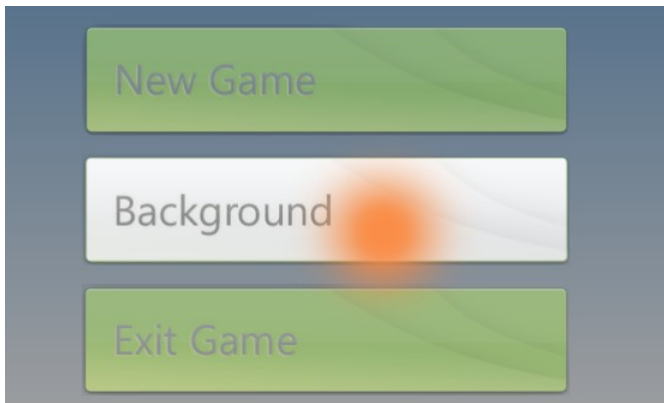
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Menu Using Hover Sample



Menu Using Hover uses the Handles API to implement menu navigation based on the hover feature.

Figure 1. Menu navigation using the hover technique.



Description

Menu Using Hover uses the Handles API to implement menu navigation based on the *hover* feature. This feature allows the player to hold his or her controlling hand over a specific user interface element and, after a specified amount of time (typically, a few seconds), select the item over which it is hovering.

Controls

Lift either hand over a user interface item to attach to the item. Once attached, hovering over an item for more than two seconds selects that item.

Game States

The sample has four game states:

GAME_STATE_MENU

From this screen you can go to the confirmation screen by selecting **NEW GAME**, the background screen by selecting **BACKGROUND**, or the game state by selecting **RETURN**.

GAME_STATE_GAME

You can go to the menu screen by pressing START.

GAME_STATE_BACKGROUND

You can select a color and go back to the menu, or simply go back to the menu by pressing BACK.

GAME_STATE_CONFIRM

You can confirm the new game by pressing the START button, or go back by using the BACK button.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Motion-Based Background Removal Sample



Description

Motion-Based Background Removal shows various things you can do with the raw depth map, including using player movements to visually separate the players from the background without using a segmentation map or skeletal tracking, and how to produce ghost blend and heat distortion effects by applying the obtained player mask to a color image.

Play with the Sample

Motion-Based Background Removal has six different demonstration modes.

Pressing the **A** button switches between *color mode* and *depth mode*.

Pressing the **B** button switches subdemo modes:

- Color mode has four subdemo modes: player removal, background removal, ghost blend, and heat distortion.
- Depth mode has two subdemo modes: player removal and background removal.

Pressing the **Y** button resets the background depth image and background color image. The sample then rebuilds them when the player moves again.

At any time, you can switch among all the demo modes.

How the Sample Works

The sample continuously looks at each depth image it receives and only keeps pixels with depths that become larger. By doing this, the sample gradually gets a background depth image without any players on it if the players move around.

Starting with the raw depth the sample receives an image like that shown in Figure 1.

Figure 1. Raw depth received



If the player moves around, the algorithm works by only keeping pixels for which depth become larger. So, as the player moves, the player gradually disappears (as in Figure 2) until disappearing completely and leaving only the background depth image. If the player moves enough to allow the entire background area that was originally occluded by the player to have a chance to be seen by the Kinect sensor array, then there are results like those in Figure 3.

Figure 2. The image of the player gradually disappears as the player moves

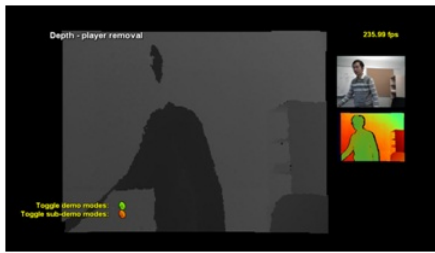
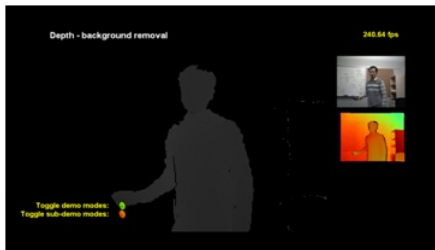


Figure 3. The image of the player ultimately disappears completely



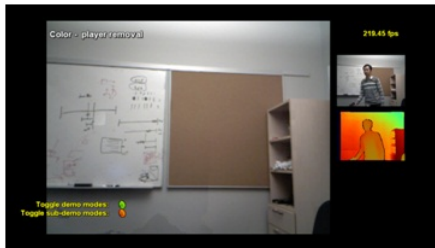
Then the sample subtracts this background depth image from the current depth image to obtain the player mask, as shown in Figure 4.

Figure 4. Player mask



The sample then applies this obtained player mask onto the color map and only keeps pixels that are on the background while the player moves. It gets a background color image without any player on it, as in Figure 5.

Figure 5. Player mask applied to color map

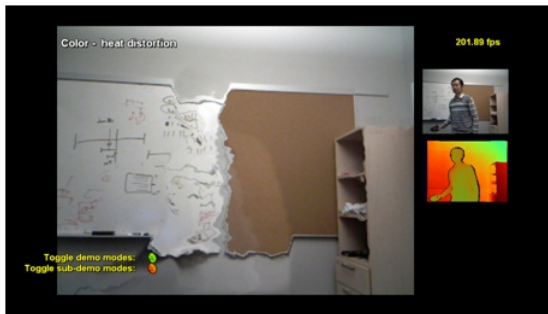


With this background-only color image and player mask, the sample implements two interesting effects. Ghost blend (textured transparent blend), shown in Figure 6, and heat distortion, shown in Figure 7:

Figure 6. Ghost blend



Figure 7. Heat distortion



This approach does have a limitation. If there are portions of the background that are far away and show up as black in the depth image, this algorithm will fail to separate any player present there.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NuiViewer Sample



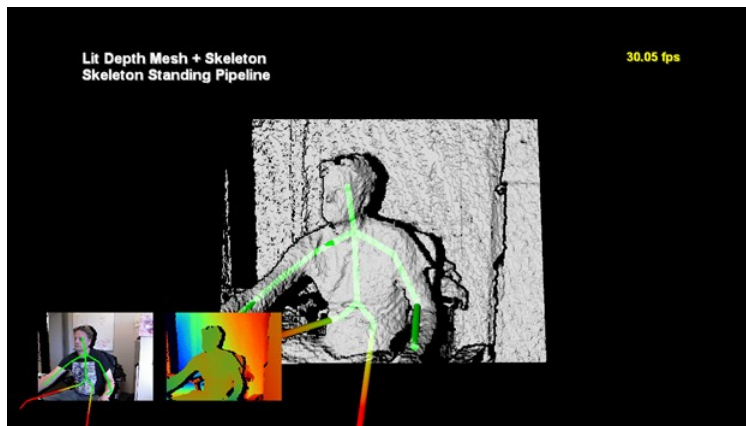
Description

The NuiViewer sample demonstrates how the 3D depth mesh from [NuiView](#), a tool that ships with the XDK, is rendered. It provides several different visualizations of the depth mesh; specifically, the following:

- Depth mesh with inferred normals, plain-shaded, and lighted
- Depth mesh with color map
- Depth mesh with segmentation visible (that is, only identified players outlined), rendered and lighted with inferred normals
- Depth mesh with segmentation and with color map
- 3D skeleton visualization
- Views 1-4 with skeleton included

The right and left bumpers navigate back and forth between visualizations. The **A** button will switch between using the seated and standing skeleton tracking pipeline. The left thumb-stick moves the camera, the right thumb-stick rotates.

Figure 1. NuiViewer



How the Sample Works

The depth mesh is rendered as a quad list, with the number of quads being equal to $(depth-width - 1) \times (depth-height - 1)$.

For each vertex, the vertex shader takes the vertex index as input, and from that infers the quad index and vertex within the quad. These values are used to calculate the u and v coordinates for each vertex in the quad, and depth values are sampled. The average depth is taken from the 4 quad depths, and if the vertex being worked on has a depth of a value that differs by greater than a specified delta, the quad is discarded (by outputting NaN as the z value from the shader).

If the vertex is not discarded, the world x , y , and z coordinates are calculated (in meters) using the formula described in the following pseudo-code:

```
World.x = ( depth * ( 2 * tan( Nui_Horizontal_FOV / 2 ) ) * ( u - 0.5 ) ) / 1000;
World.y = ( depth * ( 2 * tan( Nui_Vertical_FOV/2 ) ) * ( v - 0.5 ) ) / 1000;
World.z = depth / 1000;
```

Where u and v are the calculated u and v coordinates, $sampled_depth$ is the sampled depth value, and $Nui_Horizontal_FOV$ and $Nui_Vertical_FOV$ are the horizontal and vertical field of view (FOV) values for the Kinect Sensor.

These world coordinates are then transformed into clip-space in the usual manner.

Normals are generated by taking 3 neighboring vertex depth values, calculating the 3 world-space coordinates for these vertices using the formula above, and then using these 3 points to calculate the tangent and binormal. From these, the cross product derives the normal. The normal is then used in the lighted, untextured view for standard Lambert-style lighting.

The skeleton is rendered as a series of unit-cylinders (generated by the sample), which are transformed into the same world space as the depth mesh and oriented according to tracked bone values. Each cylinder has one end at the origin, and it is rotated and scaled according to the bone positions.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Playspace Feedback Sample



Playspace Feedback shows you how a title might provide feedback to a player in order to position the player in the optimal position in the playspace. It also shows you how to call the **XShowNuiTroubleshooter** function.

For more information about the playspace, best practices, and factors involved in designing around the Kinect playspace, see the latest version of the white paper [Natural Environments: Understanding and Developing for the Kinect Play Space](#).

Figure 1. The Playspace Feedback sample



Description

The player is shown in a playspace, controlling an avatar. As the player walks around the playspace, a number of feedback mechanisms are employed to encourage the player to stay in the "sweet spot" area. This area is marked by a green target on the floor of the game's playspace.

If the player is in the sweet spot, the player will pulse green.

If the player is outside the sweet spot, the player starts to fade out. The player's transparency depends on his or her distance from the sweet spot.

If the player hits an area 0.8 m around the edge of the sensor array depth camera frustum, on-screen hint icons will start to flash. This guides the player back towards the playspace. In this sample, these are 2D icons. In your title, you might consider a 3D arrow or other feedback mechanisms.

If tracking is lost entirely, a "lost player" icon will start to flash at the top of the display. When tracking is recovered, a "found player" icon appears briefly.

Slight camera-position feedback is employed to embed the player more viscerally into the game's playspace. As the player moves from the sweet spot, the camera jibs to the left or right to follow the player (the camera is always pointing at the origin).

Pressing X on any gamepad will toggle the sensor frustum display. Pressing START opens the NUI Troubleshooter display. Press BACK for on-screen help.

Techniques

Playspace Boundary Testing

All of the techniques described in the following sections may be found in `PlayspaceBounds.h`, `PlayspaceBounds.inl`, and `PlayspaceBounds.cpp`.

The player position used for all testing is the `vPosition` value found in `NUI_SKELETON_DATA`. This value is the center of mass of the player, which could be outside the depth-camera frustum in some circumstances.

Player vs. Sweet Spot Testing

The distance from the player to the sweet spot is a 2D distance check performed with the tilt-corrected player position. First, the player's location is tilt-corrected using a matrix returned from `NuiTransformMatrixLevel`. Then, the Y coordinate is zeroed out, and the resulting position is distance-checked against a pre-defined sweet-spot location.

Note

This may be subject to change in future revisions of the sample, and as more testing occurs.

The distance of the sweet spot from the camera and the radius of the sweet spot are both customizable constants.

Player vs. Depth Camera frustum Testing

Two frustums are built for the playspace depth camera. These are both built as spatial-partitioning planes. Testing to see if the player is on the same side of the plane in which the surface normal is pointing determines whether the player is inside the bounds of the playspace.

The outer frustum's planes are built from the physical parameters of the depth camera. The inner frustum's planes are built by translating the left and right planes of the outer frustum in along their surface normals. Front and back planes of both frustums are created directly from a normal and distance value, rather than building them from points, as these are aligned with the Z-axis.

Note that we do not test against the top and bottom planes of the frustum. We only test the vertical planes. While this works well for the default playspace parameters laid out in the white paper, more dynamic solutions might require taking these planes into account.

Rendering the Playspace Frustum

The playspace is rendered by the sample in worldspace. It uses the floor plane to determine the height of the camera, and then transforms the playspace frustum vertices to match the location and orientation of the Kinect sensor array in world-space.

If the floor plane cannot be seen (or for some reason, the floor plane's data is not available), a default height from the floor is used.

Use of XShowNuiTroubleshooter API

The `XShowNuiTroubleshooter` API is demonstrated in the `Sample::UpdateInput` function, found in `Sample.cpp`. This is the simplest case of bringing up the NUI Troubleshooter. Specifically, it is brought up entirely under user control. In a shipping title, this might be called from the **Pause** menu.

The `XShowNuiTroubleshooter` API is used to bring up a tuning screen. This allows the user to adjust the Kinect sensor array for best performance.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Recorded Data Sample

[See Also](#)



Description

Recorded Data demonstrates the use of the [XedFile](#) API to perform a variety of simple operations on files recorded from the Kinect Sensor using Xbox Studio and an easy-transfer USB cable. Provided is the full source code to RecordedData, a Win32 command-line tool, that will allow you to:

- Dump statistical information regarding an xed file to the command-line.
- Dump frame and timestamp information regarding all events in an .xed file to a CSV file.
- Dump skeleton joint position information to a CSV file that can be opened in a spreadsheet (for example, in Microsoft Excel) for further processing and analysis.
- Trim data from the beginning and end of an .xed file to produce a smaller file.
- Concatenate two .xed files together, reconciling timestamps to produce a single continuous file.

The provided source code also includes a C++ wrapper for **XedFile**, which includes forward and reverse iterators to make it easier to walk through the data in the file. You are encouraged to use this sample source code as a basis for writing your own tools.

Command-Line Options

```
RecordedData [ /I | /C | /T | /DTYPE ] parameters
```

Options	Description
/I	Outputs statistical information on the events in the file—for example: <pre>RecordedData /I inputFile.xed</pre>
/C	Concatenates two files to create a third file—for example: <pre>RecordedData /C fileA.xed fileB.xed outputFile.xed</pre>
/T	Trims data from the start, end or both ends of the file. The start and end trim times are specified in milliseconds—for example: <pre>RecordedData /T inputFile.xed outputFile.xed trimStartMilliseconds trimEndMilliseconds</pre>
/DEVENTS	Dumps a list of all of the events with their frame numbers, timestamps and types to the output, or to a .csv file. The output file name is optional. For example: <pre>RecordedData /DEVENTS inputFile.xed [outputfile.csv]</pre>
/DSKELETON	Dumps the contents of all the skeleton tracking events to the output, or to an optional .csv file—for example: <pre>RecordedData /DSKELETON inputFile.xed outputfile.csv</pre>

Example Operations

The source code to the Recorded Data tool shows how to perform a variety of common operations using the C++ wrapper library in conjunction with **XedFile**.

TxInfo.cpp

Demonstrates opening a file using the **CXEDFile** class, and then outputting the information returned by **CXEDFile::GetAllFrameInfo** to the command-line console.

TxDump.cpp

This source file demonstrates reading information from a file, and outputting it to a .csv file (or the command-line console). In particular, the **DumpSkeletonFile** function shows how to use a **CXEDFwdFrameIterator** to walk through all of the skeleton events in the input file, and output the joint information as CSV data. The **DumpEventFile** function shows how to use a **CXEDMuxFwdEventIterator** to output the frame number and timestamp of each of the events in the file in time-sequential order.

TxConcatenate.cpp

Demonstrates combining two input files and writing them to a third file. This file transform demonstrates:

- Reading frames from an .xed file.
- Writing frames to an .xed file.
- Allocating memory for different types of frame buffers using the wrapper library.
- Using a **CMuxFwdFrameIterator** to multiplex the different types of events together, so that they can be combined in sequential time order for the output.
- Adjusting timestamps and frame number for the second file so that they follow on logically from the first file.

TxTrimFile.cpp

This source file demonstrates how to read frames from an input file, and output frames to another file, while using timestamp information to trim events that are not needed. Specifically, it shows further use of the **CMuxFwdFrameIterator** and the **CMuxReverseFrameIterator**.

C++ Wrapper Library

The C++ wrapper library encapsulates the **XedFile** API in a way similar to that by which the ATL and WTL libraries wrap Win32 C-style functions. Full source code is provided for this wrapper library in the files CXEDFile.h, CXEDFile.inl, CXEDFile.cpp, XEDIterators.h, XEDIterators.inl, and XEDIterators.cpp. To use the library in your own tools, simply include these files in your project.

class CXEDFile

CXEDFile wraps an XED_CONTEXT pointer (typically returned by **XedCreateFile** or **XedOpenFile**). When the **CXEDFile** instance is destroyed, if it contains an active XED_CONTEXT pointer, the context will be closed by a call to **XedCloseFile**. You can use the CXEDFile class to:

- Create new .xed files.
- Open existing .xed files.
- Obtain statistical information on an open file.
- Read frames from the file into a buffer.
- Read information regarding frames in the file.
- Write new frames at the end of a file.

enum XEDFRAMETYPE

XEDFRAMETYPE enumerates the types of frames that can be read from the file.

struct XEDFrameInfo

XEDFrameInfo is a structure that contains information regarding the frame number, timestamp, event index, and type of a frame in the file.

struct XEDFrameRange

An **XEDFrameRange** contains **XEDFrameInfo** structures corresponding to the start and end of a range of frames, as well as the number of events in that range.

struct XEDMultiFrameInfo

`CXEDMultiFrameInfo` uses `CXEDFrameRange` structures to store range information for each of the different possible types of frame ranges in the file. It is returned by `CXEDFile::GetAllFrameInfo`.

class CXEDFrame

This class is the base class for all of the different frame types supported by `XedFile`, and carries frame number and timestamp information for the frame of data it contains.

class CXEDColorFrame

`CXEDColorFrame` is used to allocate memory for color frame buffers that are then populated by reading a color frame from a `CXEDFile` instance.

class CXEDDepthFrame

`CXEDDepthFrame` is used to allocate memory for depth frame buffers that are then populated by reading a depth frame from a `CXEDFile` instance.

class CXEDSkeletonFrame

`CXEDSkeletonFrame` is used to allocate memory for skeleton frame buffers that are then populated by reading a skeleton frame from a `CXEDFile` instance.

class CXEDEventIterator

`CXEDEventIterator` is the base class for both the forward (`CXEDFwdEventIterator`) and reverse (`CXEDReverseEventIterator`) iterators, which allow you to walk through all of the events of a specific type in sequence from the start or end of the file.

class CXEDFwdEventIterator

When constructed with an `CXEDFile` instance to attach to, and an `XEDFRAMETYPE` value specifying the type of events to iterate over, allows you to iterate sequentially over events of that type from the start of the file, in order of increasing event index.

class CXEDReverseEventIterator

When constructed with a `CXEDFile` instance to attach to, and an `XEDFRAMETYPE` value specifying the type of events to iterate over, allows you to iterate sequentially over events of that type from the end of the file, in order of decreasing event index.

template class TMuxEventIterator

This class uses iterators of all of the event types to walk through the file in order of increasing timestamp, returning each event of each type present in the file sequentially. The template parameter specifies which type of event iterator to use to walk through the file.

Its name comes from the fact that the iterator multiplexes (or "muxes") together three different streams of event information in the output.

Typedefs are provided for the `TMuxEventIterator` as follows:

- `CXEDMuxFwdFrameIterator` is a `TMuxEventIterator` that uses a `CXEDFwdEventIterator` to walk through the file from the beginning to the end.
- `CXEDMuxReverseFrameIterator` is a `TMuxEventIterator` that uses a `CXEDReverseEventIterator` to walk through the file backward from the end to the start.

See Also

- [Xbox Studio](#)
- [XStudio Reference](#)
- [XedFile Reference](#)
- [NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Reducing Latency Sample



Reducing Latency shows how to use a non-traditional game engine architecture and low-latency filtering to reduce latency in Kinect-enabled titles. This architecture uses a combination of double buffering and single buffering of resources. You may experience a difference of more than 66 ms of latency when you switch the sample to use the traditional game engine architecture with native average joint smoothing.

Reducing Latency also demonstrates a technique that improves the efficiency of color and/or depth image processing by using separate threads and separate events for data retrieval. In some cases, this can reduce the fetch time by up to 5 milliseconds.

Description

Reducing latency for Kinect titles is important. It could mean the difference between an awesome user experience and a frustrating user experience. The Reducing Latency sample implements two game engine architectures. Use the **A** button to switch between the two architectures to show the difference in latency. Use the **X** button to toggle joint filtering on and off.

Figure 1. The Reducing Latency sample



Traditional Game Engine Architecture

The traditional game engine architecture has a **GameUpdate** thread that updates resources for frame N, while a **Render** thread is running in parallel to submit draw calls to the GPU for frame N-1. This double buffering of resources introduces one frame of latency. That is, for a title running at 30 FPS, 33 ms of extra latency is introduced. Most titles also use predicated tiling in order to make use of the multisample antialiasing (MSAA) in hardware at 720p. When using predicated tiling, all draw calls within the **BeginTiling/EndTiling** block will be written into a command buffer so that it can be played back several times for the different tiles. This means that draw calls will only be submitted to the GPU once the **Render** thread gets to the **EndTiling**

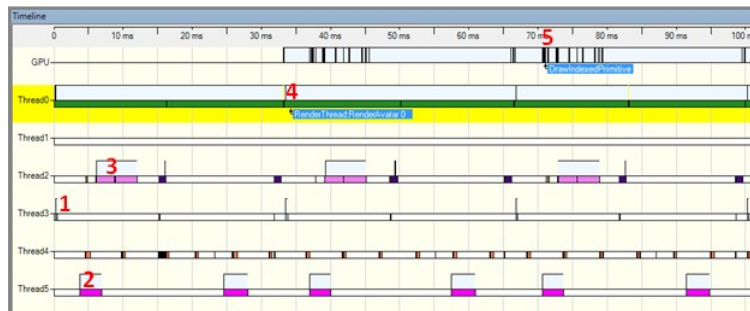
API. In order to get rid of the potential stall where the GPU must wait for the CPU to get to the **EndTiling** call, most titles make use of the flag [D3DCREATE_BUFFER_2_FRAMES](#), which allows the GPU to be one frame behind the CPU. This could introduce another frame of latency.

The **GameUpdate** thread and **SkeletonTrackingUpdate** thread for the high-latency mode simply update all resources using double buffering so that the render thread can safely render the data of the previous frame.

The **Render** thread of the high-latency mode looks like this:

- Shadow Pass (Frame N-1)
- BeginTiling
 - BeginZPas -> Opaque Pass (Frame N-1)
 - EndZPass
 - Transparent Pass (Frame N-1)
- EndTiling
- PostEffects
 - Water Distortion

Figure 2. Shows how a traditional game engine architecture can easily introduce 3 frames of latency



Legend:

- 1 GameUpdate (Frame N)
- 2 NUI Postprocessing (Frame N)
- 3 UpdateAvatar (Frame N)
- 4 RenderAvatar (Frame N), RenderScene (Frame N)
- 5 RenderAvatar (Frame N), RenderScene (Frame N)

Non-traditional Game Engine Architecture

To reduce latency, you can single buffer resources for objects that need to be rendered at low latency, and you can render without the use of predicated tiling and without GPU double buffering. A screen space anti-aliasing post effect is used in order to have the same visual quality as the high latency mode. For this sample, the avatar and objects receiving shadows from the avatar are dependent on skeleton tracking input.

The GameUpdate thread for the low-latency mode is the same as for the high-latency mode, but the "SkeletonTrackingUpdate" thread will always update into the same frame buffer as if it were single buffered. Using events, we can signal when the data for the avatar has been updated so that the Render thread can safely read the data to be rendered.

The Render thread of the low latency mode looks like this:

- Shadow Pass—Objects independent on skeleton tracking (Frame N-1)
- Manual Pre-Z Pass—Objects independent on skeleton tracking (Frame N-1)
- Opaque Pass—Objects independent on skeleton tracking (Frame N-1)
- Save color and depth buffers
- Wait until avatar data is updated from SkeletonTrackingUpdate thread
- Shadow Pass—Objects dependent on skeleton tracking (Frame N)
- Restore color and depth buffers
- Opaque Pass—Objects dependent on skeleton tracking (Frame N)
- Transparent pass
- Post effects
 - Screen space anti-aliasing
 - Water distortion

Figure 3. Shows a game engine architecture that introduces only one frame of latency



Legend:

- 1 GameUpdate (Frame N)

- 2
- NUI Postprocessing (Frame N+1)
- 3
- UpdateAvatar (Frame N+1)
- 4
- RenderAvatar (Frame N+1), RenderScene (Frame N)
- 5
- RenderAvatar (Frame N+1), RenderScene (Frame N)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Running Sample

[See Also](#)



The Running sample measures the user's running speed by using data retrieved from the Kinect Sensor. The sample shows a basic speed gauge (a vertical bar), which grows in length as the user runs faster. It also records the number of steps that the player has taken and, using the Fitness API, displays a value for approximate calories burned. The combination of automatic and manual fitness tracking modes makes the Running sample a great demonstration of how to integrate the [NuiFitness](#) library into your game.

Description

The Running sample illustrates how to determine whether the player has taken a step by using skeletal tracking data. The Y values of the player's knee skeleton positions are measured over time and if the player's knees have crossed sufficiently, it is recognized as a valid step by the title. The time of the current step is then recorded and a history of these steps is used to calculate an average number of steps per second.

Though a red color indicates low confidence about skeleton positions, this sample works well even with low confidence about skeleton position in the leg. Figure 1 shows the legs as red, and this level of confidence does not hinder the sample code from identifying that the player has taken a step; more confidence about skeleton positions in the leg is not necessary.

NuiFitness

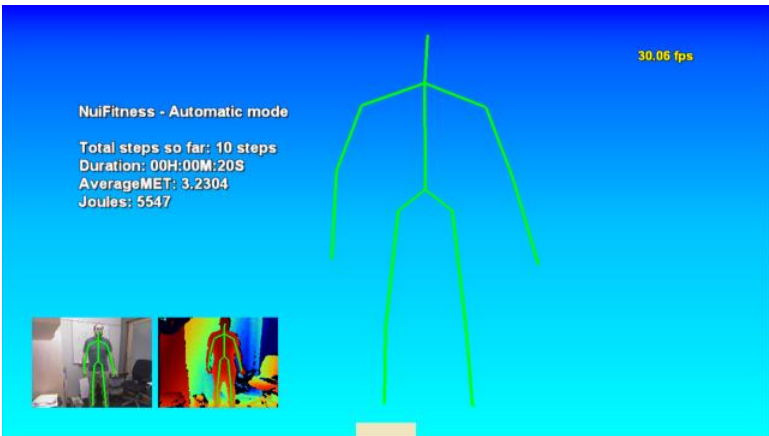
The sample demonstrates the two modes NuiFitness can operate in: automatic and manual. The sample begins in automatic mode and will start NuiFitness as soon as a player is tracked.

In automatic mode, the sample will pause NuiFitness whenever there is no player tracked. As soon as a new skeleton is acquired, the sample will resume NuiFitness so it continues collecting fitness data.

In manual mode, the sample will stop providing data to NuiFitness, providing the same effect as pause/resume in automatic mode.

Note that in this sample does not support gamer profiles and, therefore, does not demonstrate how NuiFitness can take advantage of the fitness data associated to a profile. Because there is no user profile associated with the players, fitness events are not logged. Titles will want to specify the player who is performing the activity to ensure that the fitness event is logged.

Figure 1. The Running sample



In this sample, the filters are implemented with the [NUI Imaging](#) and [Skeletal Tracking](#) APIs. After the NUI system has been initialized, the update loop polls the skeletal tracking data to see if a new frame is available. If new data is available, it is processed by the filters. If there is no new data, the update loop immediately exits.

Controls

Switch between automatic and manual mode by pressing the **Y** button. In automatic mode, NuiFitness automatically computes the values for MET and joules expenditure. In Manual mode, the sample computes its own the MET value by detecting steps and passes it to NuiFitness, which uses it to compute the expended joules.

Pressing the **B** button resets all metrics tracked by the sample (including NuiFitness tracking history).

See Also

- [NUI Fitness Tracking](#)
- [NuiFitness Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI PC Conduit Sample

[See Also](#)



NUI PC Conduit shows you how to use the Xbox Studio API to transfer Natural User Interface (NUI) data between a Windows-based computer and an Xbox 360 development kit.

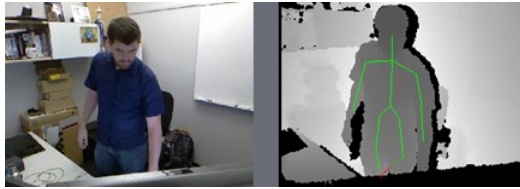
Before successfully building and running the sample, you must satisfy the following requirements:

- Installation of the DirectX SDK. The NUI PC Conduit sample links to several Direct3D 9 components. If these components are not present, the sample will not build.
- An application, initializing the Kinect system, running on the development console. This differs from other Kinect samples because the NUI PC Conduit sample runs on the development machine and not the console.

Description

You can use the Xbox Studio API to transfer NUI data between a Windows-based computer and an Xbox 360 development kit.

Figure 1. Transferring NUI data between a Windows-based computer and an Xbox 360 development kit



Xbox Studio can use either a USB connection (high bandwidth) or a TCP connection (connection over a standard LAN). For more information, see [Connecting to a Development Console \[Xbox Studio\]](#). Data can be either consumed from the Xbox development kit and used on the computer, or produced on the computer and sent to the development kit. When the computer produces data, it appears to the game as if it comes directly from the Kinect Sensor.

If you run NUI PC Conduit in *consumer* mode, it shows the color, depth, and skeleton data as it comes from the XStudio service. Consumer mode is the default mode. To run the sample in consumer mode, don't use any command line arguments.

In *producer* mode, the sample creates a regular pattern for the color, depth, and skeleton buffers, and sends it across to the development kit. To run the sample in producer mode, use the command line argument **-producer**.

See Also

[Xbox Studio](#)
[XStudio Reference](#)
[XedFile Reference](#)
[NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Managed NUI Conduit Sample

[See Also](#)

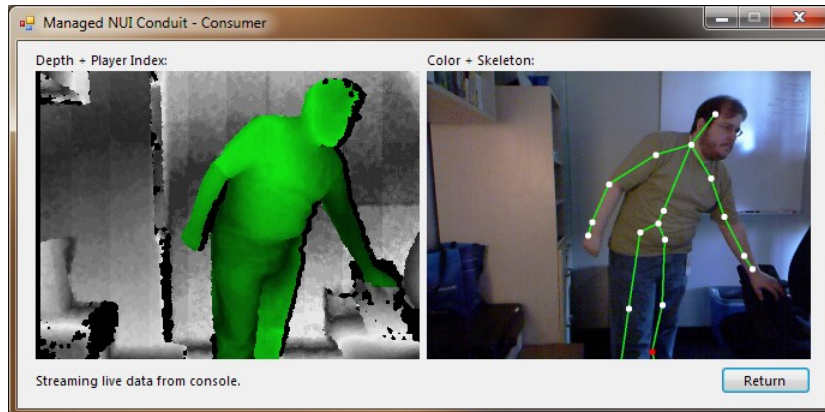
Managed NUI Conduit shows you how to use the Xbox Studio API from a managed (C#) program running on the PC.

Managed NUI Conduit requires an application running on the development console to initialize the Kinect system. This differs from other Kinect samples, because Managed NUI Conduit runs on the development computer.

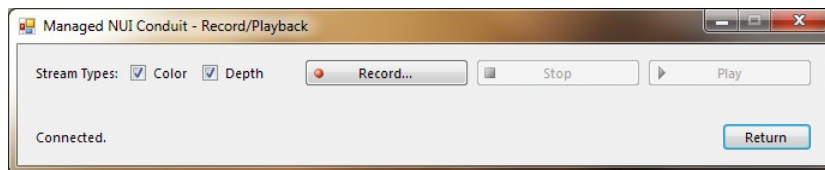
Description

The Xbox Studio API allows developers to write tools to transfer NUI data between the development console and the host computer. Managed NUI Conduit includes a wrapper around the native API set, allowing such tools to be written in a managed language (C#).

The sample demonstrates consuming NUI data (color, depth, player index and skeleton) and visualizing it to the user. To see this in action, select **Show Consumer** from the main window.



The sample also demonstrates recording and playing back NUI data (color and depth). Data is written in the XED file format. For this part of the sample, select **Show Record/Playback** from the main window.

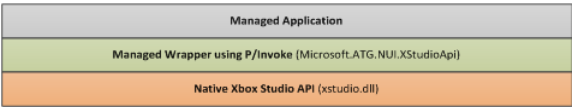


Managed P/Invoke Wrapper

To enable access to the native Xbox Studio API set, the sample includes a P/Invoke-based wrapper. All constants, enumerations, structures, delegates and methods are exposed as members of the **Microsoft.ATG.NUI.XStudioApi** static class.

Note

Naming conventions within this class do not follow typical .NET style. The intention is to expose, as closely as possible, the interface of the existing native API.



Marshaling Data

Most of the members in the P/Invoke wrapper are marked appropriately for automatic marshaling between managed and native code. However, some structures still contain opaque pointers and require custom marshaling code to resolve. The following code snippet extracts the underlying NUI_SKELETON_FRAME from the pvDataBuffer field in the XSTUDIO_STREAM_EVENT structure.

```
//using System.Runtime.InteropServices;
//using Microsoft.ATG.NUI;

XStudioApi.XSTUDIO_STREAM_EVENT streamEvent;

object o = Marshal.PtrToStructure(streamEvent.pvDataBuffer,
    typeof(XStudioApi.NUI_SKELETON_FRAME));
XStudioApi.NUI_SKELETON_FRAME frame = (XStudioApi.NUI_SKELETON_FRAME)o;
```

Delegates and Function Pointers

The Xbox Studio API defines two callback function types, both of which the wrapper has defined as .NET delegates. The methods that require function pointers are marked appropriately, so .NET will automatically convert a delegate to a function pointer. However, the garbage collector creates an additional challenge. The callback delegates needs to remain in memory for as long as the application uses the Xbox Studio API. The solution is to use **System.Runtime.InteropServices.GCHandle** to keep the garbage collector from collecting objects shared between native and managed code. The following code snippet demonstrates this technique while passing a delegate to the native Xbox Studio API.

```
//using System.Runtime.InteropServices;
//using Microsoft.ATG.NUI;

// Prevent delegate from being collected.
XStudioApi.XSTUDIO_STREAM_CALLBACK callbackDelegate = StreamCallback;
GCHandle callbackDelegateHandle = GCHandle.Alloc(callbackDelegate);

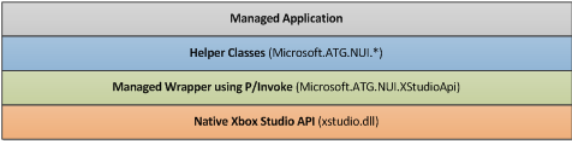
// Give delegate to native code.
IntPtr callbackHandle;
XStudioApi.HRESULT hr = XStudioApi.XStudioRegisterStreamCallback(
    callbackDelegate,
    XStudioApi.XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG.
        XSTUDIO_STREAM_CALLBACK_REGISTRATION_FLAG_CONSUMER,
    IntPtr.Zero,
    out callbackHandle);

// . . .

// Delegate no longer needed.
if (callbackDelegateHandle.IsAllocated)
    callbackDelegateHandle.Free();
```

Helper Classes

The Managed NUI Conduit sample includes a series of helper classes to aid in developing a managed application. These classes more closely follow typical .NET conventions and patterns, while addressing the issues of data marshaling and garbage collection of delegates.



The following code snippet provides an example of using the helper classes from the sample.

```
//using Microsoft.ATG.NUI;

StreamConsumerCallback consumer = null;
try
{
    XStudio.Connect(null, ConnectionType.Tcp);

    consumer = StreamConsumerCallback.Register(null);
    consumer.MappedStreams = StreamType.Color | StreamType.Depth;
    consumer.StreamEvent += new StreamEventHandler(consumer_StreamEvent);

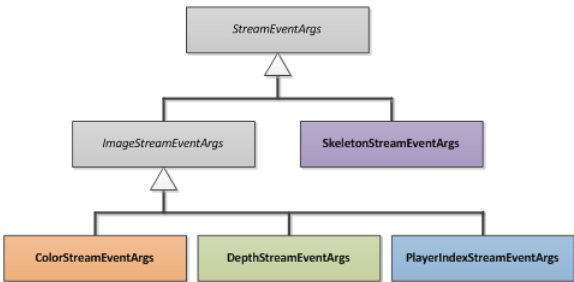
    XStudio.Start(consumer.MappedStreams);
}
catch (NUI.ApiException ex)
{
    // Handle error.
}
```

HRESULT codes and Exceptions

All methods from the native Xbox Studio API return HRESULT success and error codes. This is preserved in the P/Invoke wrapper and all essential codes may be found in the **Microsoft.ATG.NUI.XStudioApi.HRESULT** enumeration. The helper classes check for error codes and throw a **Microsoft.ATG.NUI.ApiException**, which encapsulates the name of the Xbox Studio API and the error code returned.

Consuming NUI Data

A number of helper classes provided in the Managed NUI Conduit sample are related to consuming NUI data from the development console. The **Microsoft.ATG.NUI.StreamConsumerCallback** class handles the marshaling of NUI data from the native Xbox Studio API and raises .NET events based on incoming data. All stream events derive from the **Microsoft.ATG.NUI.StreamEventArgs** class. Additionally, image-based events (color, depth and player index) derive from the **Microsoft.ATG.NUI.ImageStreamEventArgs** class.



Note
An application should perform minimal processing within the callback to avoid stalling the Xbox Studio API. Instead, store the stream event data and offload the work to another thread. Consider using the System.Threading.ThreadPool class to manage queuing work items.

The **Microsoft.ATG.NUI.NotificationCallback** class performs a similar job as **StreamConsumerCallback**, raising .NET events based on incoming notifications.

Recording/Playing Back NUI Data

The **Microsoft.ATG.NUI.XStudioFile** class facilitates recording NUI data to, and playing it back from Xbox Event Data (XED) files. The following code snippet records color and depth data from the development console.

```
//using Microsoft.ATG.NUI;

StreamType streamTypes = StreamType.Color | StreamType.Depth;
XStudioFile file = XStudioFile.Create("file.xed", true, streamTypes);

file.MappedStreams = streamTypes;
XStudio.Start(file.MappedStreams);

// . . .

XStudio.Stop(file.MappedStreams);
file.Dispose();
```

See Also
[Xbox Studio](#)
[XStudio Reference](#)
[XedFile Reference](#)
[NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Segmentation Sample

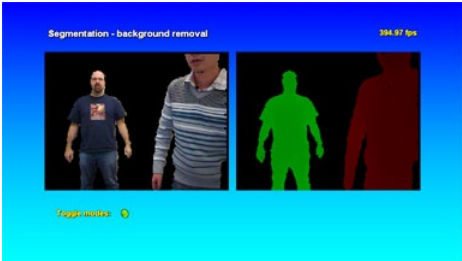
Description

The Segmentation sample illustrates basic usage of the player segmentation data generated by the Natural User Interface (NUI) [Imaging API](#). After receiving the player segmentation data, the sample uses this data to provide three display modes:

- background removal
- player index color overlay
- normal color and depth viewing

Press button **A** to cycle through the three modes.

Figure 1. The Segmentation sample



The sample first reads color and depth data from the Xbox 360 Kinect sensor array. The lower three bits of each pixel in the depth data contain the related player index, which indicates the player to whom that depth pixel belongs. If there is no index value present, then the pixel is part of the background. The title can use this information to perform various tasks related to the player's character in the game, such as background removal. Currently, the system supports segmentation of up to two players in the field of view (FOV) of the Kinect sensor array. In the sample, player 1 is marked by the color red and player 2 by green.

For simplicity, this sample processes the segmentation information, and the color and depth data, on the CPU. However, it is up to the developer whether a title processes the segmentation data on the GPU or CPU. If you choose to use the GPU for processing, you can use the following HLSL code to extract the lower three bits of each pixel (which contains the player index), if the value has already been read from the depth data.

```
float value;
...
//Fetch value from the texture containing the retrieved depth data
...
value /= 8;
value = frac(value);
value *= 8;
//The value variable now contains the player index
```

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Shoulder Stabilization Sample

The positions of the head and shoulder from skeleton tracking (ST) can become erratic when parts of the body occlude parts of the head and shoulder. Shoulder Stabilization stabilizes the head and shoulder joint positions. It detects when these joints are self-occluded and then it stabilizes them.

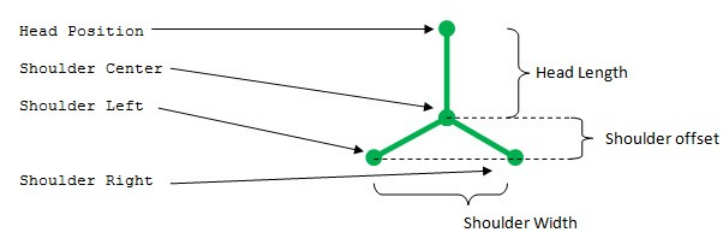
Figure 1. The Shoulder Stabilization sample



Description

The head and shoulders are stabilized by parameterizing by their overall position, orientation, shoulder width, head length, and an offset that explicitly lowers the shoulders from the base of the head.

Figure 2. Shoulder stabilization



The head and shoulder joints are stabilized by using a gradient-descent-based optimizer to align them with their corresponding joints from ST. This is done in such a way that the influence of an ST joint is inversely proportional to the degree that it is self-occluded. Damping is added to smooth the jitter when joints are occluded and body motion is small. This is how occluded ST joints are stabilized. A joint is considered self-occluded if its **eSkeletonPositionTrackingState** is not **NUI_SKELETON_POSITION_TRACKED**. We can also determine self-occlusion by projecting the head, shoulder, and arm joints into the image plane. By doing this, we can compute the distances of each joint to the projected joint and limb locations. The degree to which a joint is self-occluded is determined from its proximity to these projected structures. Joints are modified and their tracking state is set to **NUI_SKELETON_POSITION_INFERRED**.

Controls

Use the following controller buttons with this sample:

- A** Toggles depth points.
- Y** Toggles self-occlusions scores. The size of the spheres are proportional to the degree of self-occlusion.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simple Gesture Detection Sample

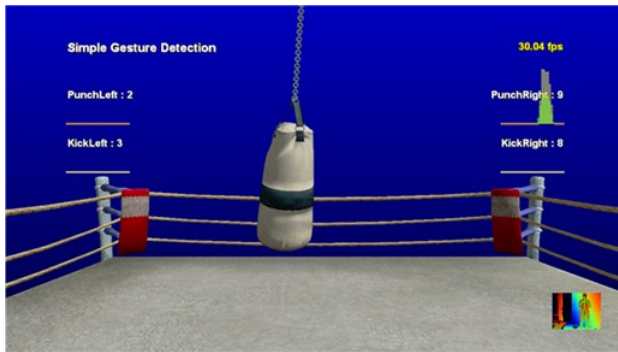
[See Also](#)

This sample shows how to use [NuiGestureReference](#) API together with the output of the [Visual Gesture Builder](#) tool to simplify gesture detection.

Note
You can download the source content for this sample at this location: [Simple Gesture Detection Content](#)

Description

Figure 1. Simple Gesture Detection



The Simple Gesture Detection sample shows how the time consuming art of coding, maintaining and fine tuning input parameters and thresholds for gesture detection, can be replaced by an automatic process that uses machine learning technology.

This process essentially turns gesture detection from an engineering to a content creation problem. In this sample, punching and kicking gestures have been trained so that the player can punch and kick the punching bag.

Gesture counters are displayed together with a signal of the confidence value (green) and the progress of how far along you are in the gesture (grey).

The following are the control buttons.

- A – toggle data collection mode.
- B – toggle joint smoothing.

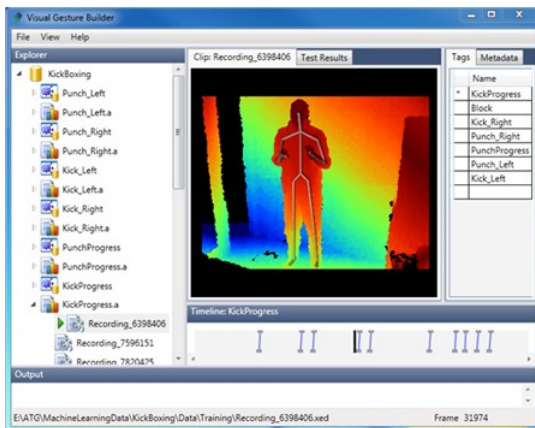
Note

This sample is very similar to the [Kick Boxing Sample](#), with the difference that it uses the NuiGesture APIs to perform the detection.

For more information about using machine learning to tune gesture detection, see "Gesture Detection using Machine Learning," a presentation at [Gamefest](#) 2011.

How the Sample Works

Figure 2. Visual Gesture Builder AdaBoostTrigger and PCAPProgress Projects



The following are the steps you must perform to run this example. Because the objective is to build a data driven application, first you need to collect data.

1. Collecting data. Run the sample on the Xbox 360. Use the A button to toggle its state to data collection mode. This will record [Xbox Studio](#) files.
2. Tagging data. Manage and tag the recorded files using [Visual Gesture Builder](#). The tagging process is where you add meaning to the data by indicating which frame represents which gesture.

Note

Tagging is essential to obtain accurate results and to reduce run-time latency in your title.

The Visual Gesture Builder tool produces two files, a gesture database file *KickBoxing.gbd* and a header file *KickBoxing.h*, both used in the title by the NuiGesture APIs. You need the following code to load and retrieve results for gesture detection.

```
// Storing results of NuiGesture
NUI_GESTURE_FRAME m_GestureFrame;
KickBoxing::FRAME m_KickBoxingGestureFrame;

// Load the gesture data file and enable all gestures
RETURN_ON_FAIL( NuiGestureInitialize( "game:\\Media\\Gestures\\KickBoxing.gbd", KickBoxing::DIGEST, 0, NULL ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_PUNCH_LEFT, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_PUNCH_RIGHT, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_KICK_LEFT, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_KICK_RIGHT, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_PUNCHPROGRESS, TRUE ) );
RETURN_ON_FAIL( NuiGestureEnableGesture( KickBoxing::GESTURE_KICKPROGRESS, TRUE ) );

// Initialize the NuiGesture frame that will receive data from NuiGesture APIs
INITIALIZE_NUI_GESTURE_FRAME( &m_KickBoxingGestureFrame, &m_GestureFrame );

if ( SUCCEEDED( NuiGestureGetNextFrame( 0, &m_GestureFrame ) ) )
{
    // Do something interesting with the gesture detection results
}
```

See Also

[NuiGesture Reference](#)
[Visual Gesture Builder](#)
[Xbox Studio](#)
[Kick Boxing Sample](#)
[XedFileTagger Sample](#)
[NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simple Head Tracking Sample

[See Also](#)



Simple Head Tracking demonstrates how to use the [NuiHeadPosition](#) API to track the player's head. For each player's face detected by NuiHeadPosition, the sample draws a rectangle around the player's face, along with the associated tracking and confidence ranking.

See Also

- [Head Tracking samples](#)
- [NUI Head Tracking](#)
- [NuiHeadPosition Reference](#)
- [Head Orientation Sample](#)
- [NuiHeadOrientation Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simple Identity Sample

[See Also](#)



This sample uses the NUI Identity system to manage player identification (that is, checking to see if a skeleton that has become visible is recognized as an enrolled player), responding to intent to play, signing in to a gamer profile, signing out of a gamer profile, and biometric sign-in. It also shows how to pause the NUI identity system before accessing the RGB color image stream.

Run and interact with this sample to see the flow of Identity behavior that meets the TCR requirements. To fully exercise this sample, create a biometric-enabled gamer profile and a gamer profile that's not biometric-enabled. One person can partly exercise the sample, though having more than one person is best, to demonstrate keeping track of multiple players' identities.

What the Sample Demonstrates

The Simple Identity sample demonstrates the following:

- The NUI identity system automatically calls [NuiIdentityIdentify](#) whenever a skeleton becomes visible, without the title having to call that function.
- Enrolling and signing-in a player who indicates intent to play, but only if the player is not enrolled yet and has a matching biometric-enabled gamer profile. To trigger this processing, the title only needs to detect and report intent to play (by any skeleton).
- Providing identity feedback about which gamer profile is associated with the player's face and skeleton, after the player signs in to a gamer profile.
- Providing identity feedback that the guest player identity will be used, after the player signs out of a gamer profile.
- Invoking the Kinect Guide and the sign-in UI correctly so the player stays engaged with the title, and doesn't have to wave to get control.
- Pausing the NUI identity system before accessing the RGB color image stream to take a snapshot, and then restoring the color stream to the identity system.
- Detecting intent to play in different forms: waving, and interacting with the **Pause** menu.

NuiIdentity Functions Used

This sample uses the following NuiIdentity functions:

Function	Purpose
NuiIdentityGetNextMessage	To monitor quality issues and guide the player in the skeleton recognition process.
NuiIdentityDetectedIntentToPlay	To conditionally enroll non-enrolled skeletons that match a biometric-enabled gamer profile.
NuiIdentityPause	To use the RGB color image stream.
NuiIdentityResume	To return the use of the RGB color image stream back to the NUI identity system.

This sample does not explicitly call [NuiIdentityIdentify](#) or [NuiIdentityEnroll](#).

Using the Sample

The sample has the following scenes (or screens):

- [Intent to Play scene](#)
- [Identification Feedback scene](#)
- [Lobby scene](#)
- [Game scene](#)
- [Pause menu](#)

Intent to Play scene



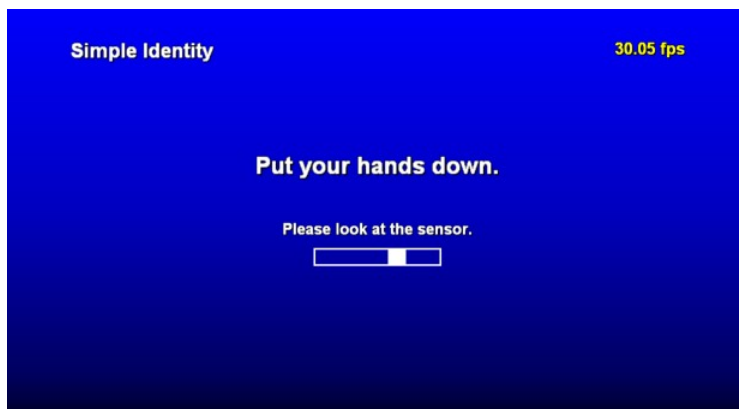
The **Intent to Play** scene is the first scene the player sees after launching the sample. The player must wave with the right hand in order to show intent to play, and be granted access to the rest of the sample.

The **Intent to Play** scene replaces the usual **Press Start to Begin** scene found at the beginning of controller-based titles.

When invoking the **Intent to Play** scene:

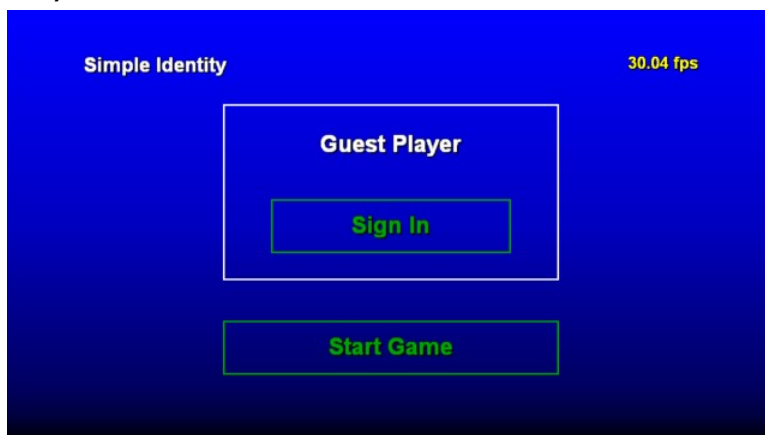
- The sample immediately starts identifying players as soon as they are tracked, though this processing isn't shown on the scene. Identifying players as soon as they are tracked speeds up the identification and enrollment process, and gets the player into the actual game faster.
- The sample uses the [NuiWave](#) API to detect wave gestures.
- If more than one player is trying to perform the wave gesture, only the first player to complete the gesture successfully will be acknowledged.

Identification Feedback scene



In the **Identification Feedback** scene, the NUI identity system performs recognition processing on a skeleton and face, to identify the skeleton as a particular player. This scene provides feedback about any quality issues that prevent recognition, and guides the player to enable identification. This is feedback about quality issues, not feedback about which identity the player is recognized as.

Lobby scene



The **Lobby** scene demonstrates one of the many ways a title can implement essential identity behaviors.

The **Lobby** scene does the following:

- Provides feedback about which identity the player is recognized as, and provides the opportunity to correct misidentification before actually playing the game. This behavior addresses TCR 158: NUI Identity Player Feedback.
- Lets the player manually select a gamer profile.
- This scene doesn't enroll any guest players or players who manually sign in to a gamer profile.

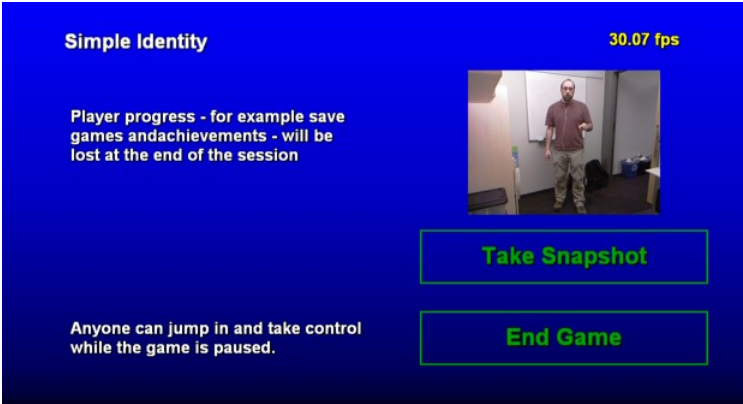
These behaviors are essential to providing a consistent and player-friendly experience. The Simple Identity sample uses the following mechanisms to implement them:

- If a player is signed-in to a gamer profile, the player's gamertag is shown. If a player is not signed-in to a gamer profile, the player is shown as *Guest Player*.
- The **Sign In** button invokes the NUI sign-in UI, which allows a player to manually sign in to a gamer profile. If you are signed in to a gamer profile, this button is labeled **Change Profile**.

- The **Start Game** button confirms that the player wants to launch the game using the indicated gamer profile or the guest player identity.

In the **Lobby** scene, the only player who can interact with the title is the player who waved in the **Intent to Play** scene. If that player’s skeleton tracking is lost, the sample immediately returns to the **Intent to Play** scene.

Game scene



The **Game** scene is the place where the gaming action would take place in a real title. At this point, the player has confirmed the gamer profile to use for this play session, or that the user is playing as a guest player. If the player is using a gamer profile, any achievements unlocked or progress made would be associated with this gamer profile.

Buttons in the Game Scene

In the **Game** scene, the active player can select any of these buttons, which are always visible:

- **Take Snapshot:** Pauses the NUI identity system, accesses the RGB color image stream, and then resumes the NUI identity system.
- **End Game:** Ends the game, and returns the player to the **Intent to Play** scene.

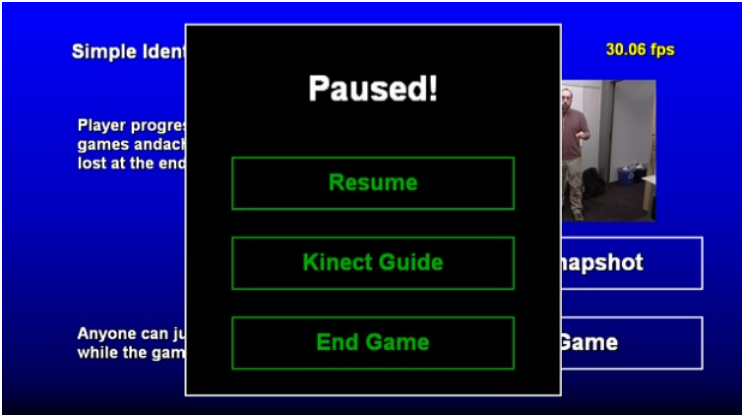
If Skeleton Tracking Is Lost

If skeleton tracking is lost on the active player in the **Game** scene, the sample stays in the **Game** scene, and the **Pause** menu is displayed. Whoever becomes visible and shows intent to play (by selecting **Resume** from the **Pause** menu) will be in control. The game continues to use the gamer profile or guest identity that was selected in the **Lobby** scene.

Signing Out During Gameplay

While in the **Game** scene, the player can access the Kinect Guide through the **Pause** menu and manage the gamer profiles present on the console. If the gamer profile being signed out is the gamer profile associated with this play session (the profile that was selected at the **Lobby** scene), the play session ends immediately, and the sample returns to the **Intent to Play** scene. If the gamer profile being signed out is not the gamer profile that is associated with this play session, gamer profile sign-out has no effect.

Pause menu



The **Pause** menu lets the controlling player perform certain actions.

Accessing the Pause Menu

To invoke the **Pause** menu:

- Any player standing in front of the sensor array can bring up the **Pause** menu by performing the system gesture.
- The **Pause** menu will be invoked automatically whenever no player is present in the play area while the **Game** scene is active.

Buttons in the Pause Menu

The **Pause** menu has the following buttons:

Button	Description
Resume	Resumes the sample at the point where the player invoked the Pause menu.
Kinect Guide	Invokes the Kinect Guide.
End Game	Ends the game, and returns to the Intent to Play scene.

Switching Players During the Pause Menu

If a different, new person selects **Resume**, the person is automatically enrolled and signed in, if a matching biometric-enabled gamer profile is found. Initially, the only person who is able to control the **Pause** menu is the player who invoked the **Pause** menu. If the player who opened the **Pause** menu leaves the play area while the **Pause** menu is still open, the first person to re-enter the play area takes control of the menu. Suppose a new person - that is, a person who didn’t go through the title’s initial identification/recognition process - clicks **Resume**. And suppose the **Pause** menu was opened from the **Game** scene. Selecting **Resume** is taken as indicating intent to play, and thus, if the person is not yet enrolled, and a matching biometric-enabled gamer profile is found, the NUI identity system enrolls the skeleton in the session identity store, and signs the player in to the matching gamer profile.

See Also

[Overview of NUI Identity](#)
[Automatic Identity Handling](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Simple Identity Using Callbacks Sample

[See Also](#)



This sample shows how to manage player identity without using automatic identity handling.

The "Simple Identity Using Callbacks" sample provides the same behavior as the "Simple Identity" sample, but to implement the behavior, instead of using automatic identity handling, it directly calls [NuiIdentityIdentify](#) and [NuiIdentityEnroll](#), and implements their associated callbacks.

Instead of using the entire pattern of API usage shown in this sample, it is recommended that titles use automatic identity handling, and then supplement that if needed by calling the Identity API, using only part of the approach shown in this sample.

See Also

[NUI Identity](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Simple Skeleton Tracking Sample



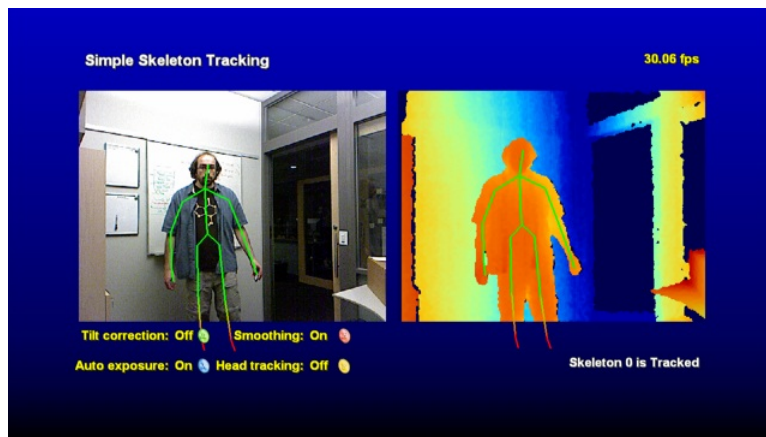
Simple Skeleton Tracking illustrates the simplest way to do skeleton tracking by synchronously reading data from the sensor array, and feeding that data into the skeleton tracking API.

Description

Simple Skeleton Tracking illustrates three essential tasks:

1. Reading color and depth data from the sensor array by using the [NUI Imaging API](#).
2. Generating skeleton tracking information from the depth data by using the [NUI Skeleton Tracking API](#).
3. Visualizing the results.

Figure 1. The Simple Skeleton Tracking sample



This sample shows a simple way to use the skeleton tracking APIs. A simple visualization is implemented to show how the data might be used, but the data can be used for other purposes (for example, to render an avatar with the player's pose).

Caution

The image data and depth data captured by the sensor array are not guaranteed to be aligned. Due to the current manufacturing process, the amount of misalignment between different sensor array units may vary. These variations will be reduced in future versions of the sensor array. You can use this sample to determine the amount of discrepancy from your specific sensor array.

The Natural User Interface (NUI) APIs are used to initialize and read data from the sensor array. After initialization of the NUI system, the update loop polls the skeletal data stream for new frame data. If available, the new data is processed by the filters. If there is no new data, the update loop immediately exits.

Controls

The following controls are supported:

- Press **A** to toggle tilt correction.
- Press **B** to toggle skeleton smoothing.
- Press **X** to toggle the exposure mode between automatic and manual. During manual exposure mode, adjust the exposure time using the direction pad; up increases the exposure time and down decreases the exposure time.
- Press the **Y** to toggle head tracking. If enabled, the sample zooms in (at twice the normal resolution) and continually adjusts the current view so that the head of the first player is always centered.

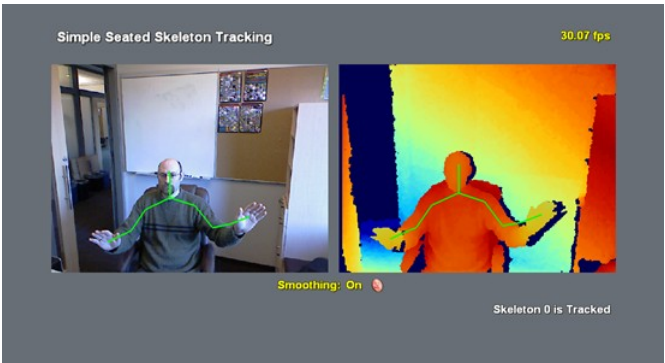
© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Simple Seated Skeleton Tracking Sample



Simple Seated Skeleton Tracking shows how to read skeletal data from the seated skeleton tracking pipeline.

Figure 1. The Simple Seated Skeleton Tracking sample



Description

Simple Seated Skeleton Tracking shows how to get data from the seated skeletal tracking pipeline and render it in a simple manner. The seated skeleton tracking pipeline is optimized for seated poses.

Using the seated skeleton tracking pipeline looks very similar to the normal skeletal tracking pipeline. To use the seated skeleton tracking pipeline, call [NuiSkeletonTrackingDisable](#), then call [NuiSkeletonTrackingEnable](#), passing the flag [NUI_SKELETON_TRACKING_FLAG_ENABLE_SEATED_SUPPORT](#).

The skeleton data is retrieved using the same APIs. However, the skeletal data is interpreted slightly differently. The seated skeleton tracking pipeline differs in how the joints are tracked. Titles shouldn't assume that code written for the normal skeletal tracking pipeline will behave the same when given joints from the seated skeleton tracking pipeline. The lower body joints are not tracked in a skeleton retrieved from the seated skeleton tracking pipeline. Also, the joint values reported by each of the pipelines probably will be different. You should take the differences between the two pipelines into consideration when you design code around skeletal tracking.

Here are some more restrictions with the seated skeleton tracking pipeline:

- No more than two players
- Player motion is required to start tracking
- No avatar retargeting support

Controls

The following controls are supported:

- Press **B** to toggle skeleton smoothing.
- Press **A** to switch between the seated skeleton tracking and full body pipelines.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simple Speech Recognition Sample



Description

Simple Speech Recognition illustrates basic functionality of the Natural User Interface (NUI) [Speech API](#), which provides speech recognition to Kinect for Xbox 360. This sample demonstrates the recommended startup and shutdown sequence for the NUI Speech API (which is necessary for changing grammars during gameplay) and command recognition using several example grammars. Recognized commands are displayed, along with their semantic property confidence and rule-based confidence values. Using property confidence instead of the rule confidence tends to give better results in regard to false acceptance and rejection.

The sample contains several different grammars, shown in the following table.

Grammar	Description
Phonetic alphabet	Contains elements from the NATO phonetic alphabet (for example, <i>foxtrot</i> , <i>whiskey</i> , and <i>charlie</i>).
Medieval	Contains commands that might commonly be used in an adventure or role-playing game. The grammar is provided in several localized forms, listed below.
Yes No	Contains a variety of different affirmation or negation phrases, all of which resolve to the single-word result <i>yes</i> or <i>no</i> . The grammar is provided in several localized forms, listed below.

Switch between available grammar files by pressing the left shoulder button and switch between available languages for a given grammar file (for example, Medieval or Yes No) by pressing the right shoulder button.

Note

The .grtxt files were generated manually to provide on-screen instructions in the sample. These files are not required by titles. However, it is recommended that the title provide feedback to the user when words and phrases are recognized.

Grammar Files

The following grammars are available when using the Simple Speech Recognition sample:

- [Phonetic Alphabet Grammar](#)
- [Medieval Grammar](#)
- [Yes No Grammar](#)

Phonetic Alphabet Grammar

The Phonetic Alphabet grammar recognizes words from the NATO phonetic alphabet (for example, *foxtrot*, *whiskey*, and *charlie*). It is provided in English-U.S. form only.

Grammar file name	Description
phonetic_alphabet_en_us.grxml	English-U.S.

Special recognition logic is used for recognizing the X element (Xray) of the NATO phonetic alphabet. Unlike the other phonetic alphabet words, recognition of the X element is handled using phonemes. The representative phonemes are: EH K S R EH I. For more information about phonemes recognized by the NUI speech engine, see [Phoneme Table for English \(United States\)](#) on

MSDN. For a table of all supported language phonemes, see [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Medieval Grammar

The Medieval grammar, `medieval_*.grxml`, provides a list of commands that might commonly be used in an adventure or role-playing game. It is provided in the following localized forms.

Grammar file name	Description
Medieval_de_de.grxml	German-Germany
Medieval_en_au.grxml	English-Australia
Medieval_en_gb.grxml	English-UK
Medieval_en_us.grxml	English-U.S.
Medieval_es_es.grxml	Spanish-Spain
Medieval_es_mx.grxml	Spanish-Mexico
Medieval_fr_ca.grxml	French-Canada
Medieval_fr_fr.grxml	French-France
Medieval_it_it.grxml	Italian-Italy
Medieval_ja_jp.grxml	Japanese-Japan
Medieval_pt_br.grxml	Portuguese-Brazil

These grammars demonstrate a variety of rules that perform slightly more complex matching behavior than usual to allow variations on matching behavior. For example, both "open spell book" and "spell book" match the "open spell book" command. There is also a default rule that attempts to match user speech that does not match the other rules in the grammar.

Yes No Grammar

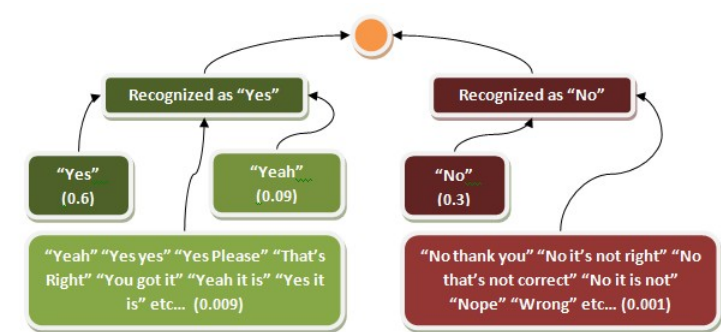
The Yes No grammar, `yes_no_*.grxml`, contains a variety of different affirmation or negation phrases, all of which resolve to the single-word result *yes* or *no*. It is provided in the following localized forms.

Grammar file name	Description
yes_no_en_gb.grxml	English-UK
yes_no_en_us.grxml	English-U.S.
yes_no_es_mx.grxml	Spanish-Mexico
yes_no_fr_ca.grxml	French-Canada
yes_no_ja_jp.grxml	Japanese-Japan
yes_no_pt_br.grxml	Portuguese-Brazil

The single words *yes* and *no* are weighted highly (greater than 1.0) to indicate that they're the most likely words a user might say in this situation. The other catch-all rules are weighted much lower (less than 1.0). This weighting allows the speech recognition engine to handle input that is not expected (such as *yes that's correct* or *that's it!* instead of "yes"), as requested by a title's user interface, while triggering most accurately on the explicitly requested forms.

The following graphic illustrates the rule branches for UK English using a weighted tree.

Figure 1. Rule branches for the UK English grammar file



The NUI Speech component supports the Kinect Sensor's built-in microphone array.

Note

This sample checks for [Multi-Channel Echo Cancellation \(MEC\)](#) calibration on startup by using the NuiAudio APIs, and notifies the player if they need to perform calibration. For the best results, before you run the sample or use speech recognition in your title, calibrate the microphone array in the Kinect Sensor, by using [Kinect Tuner for Audio](#). Calibration persists across reboots, but you should calibrate again if the positions of the sensor array, playback speakers, or room objects change.

For debugging issues with speech recognition, specify `NUI_SPEECH_EVENT_RETAINEDAUDIO` for the *eSpeechProperty* (used by [NuiSpeechGetProperty](#)) to save audio data from the utterances based on recognition events.

Grammar Creation

A *grammar*, a list of words and phrases recognized by the title, is compiled into a binary representation by passing an XML file, with word definitions, to the grammar compiler tool, [xbpg.exe](#). You can use the following command line to build the grammar for this sample:

`xbpg.exe /g compiledgrammarfile.cfg inputgrammarfile.grxml`

Grammar Loading and Unloading

Compiled grammar binaries can be loaded or removed by the title during gameplay with calls to [NuiSpeechLoadGrammar](#) or [NuiSpeechUnloadGrammar](#).

This sample takes an extra step—creating and destroying the speech recognition engine—depending on the language of the grammar file provided. The language model to load is determined by the text at the end of the compiled grammar's file name.

In normal practice, it is not necessary to shut down the speech engine unless you no longer need to perform speech recognition in your title or you need to switch languages. The support for any given language's speech recognition is loaded when the engine is initialized. For more information about support for a given language and NUI speech databases, see [Kinect Database Files to Include on the Game Disc](#).

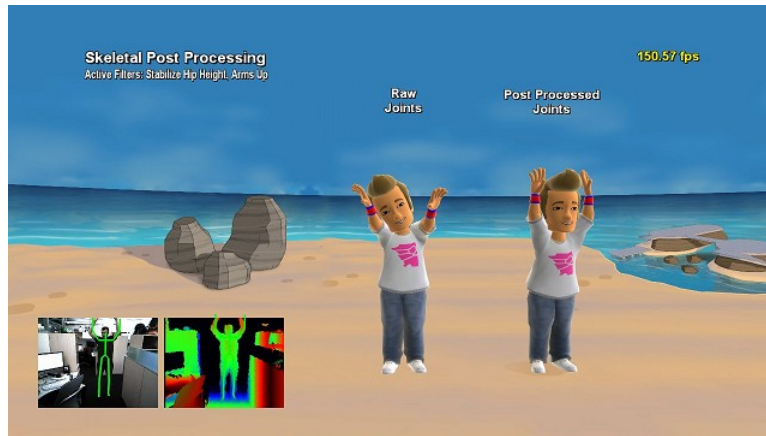
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Skeletal Post-Processing Sample



Skeletal Post-Processing demonstrates a variety of post-processing filters for skeleton joint data that are useful in game scenarios in which the consistency of joint position data is a high priority.

Figure 1. The Skeletal Post-Processing sample



Description

Skeletal Post-Processing demonstrates a variety of skeleton joint data post-processing filters that are useful in game scenarios where the consistency of joint position data is a high priority. Some filters are intended for general usage while others fulfill the needs of very specific situations. It is important to note that post-processing of skeletal data is always a balancing act between motion fidelity and consistency. These filters represent various points between these two goals. For example, the *arms up* filter provides more consistent arm positions when players raise their arms above their heads, but it also removes some expressiveness, such as a user waving her hands above her head.

Filter Descriptions

The following filters are demonstrated by the sample: *arms up*, *baseball swing*, *hip stabilization*, and *clipped legs*.

Arms Up

This filter detects when the player has one or both arms above his or her head. Procedurally, it rebuilds the arms, shoulders, and head into a stable pose.

For this pose, joint positions from the skeleton tracker can be seen in a variety of positions, depending on how close the arms are to the head as well as how much of the arms are cropped by the camera FOV. The filter detects five different patterns associated with an arm being held up. If one of these patterns is detected, a procedurally constructed arm is blended in, replacing the tracked joint positions. Each constructed arm does not move in any way—it is constructed with fixed angles and bone lengths derived from historical measurements and body posture. For this reason, expressive forearm movements (such as hand waving) are discarded.

Early research for this filter revealed that if the only action taken was downgrading the tracking quality values for the joints, this did not guarantee a visually appealing result when the output skeleton was used for avatar retargeting. In the case of avatar retargeting, it processes a joint that is inferred or not tracked. Typically, it uses the last tracked state of the joint to produce an avatar pose. In many cases, the last valid pose may not be visually appealing. Procedural generation of new arm, shoulder, and head joints solves this problem.

Baseball Swing

This filter repositions the player's arms as if the player is holding a baseball bat. The filter works for left- or right-handed batters.

The filter selects the arm closest to the camera as the leading arm, and repositions the hand of the leading arm such that the baseball bat points in the appropriate direction. In addition, the arm farthest from the camera (and often occluded) is positioned such that the player adopts a natural looking batter pose.

The filter attaches a baseball bat to the hands of the debug skeletons, but not to the avatars. Therefore, you should activate the debug skeletons display when you experiment with this filter.

Hip Stabilization

This filter attempts to stabilize the location of the hips relative to the legs and shoulders. In certain scenarios, such as the player raising arms high above the head, the hip positions returned from the skeleton tracker can drift upwards by a few centimeters. The filter recognizes this behavior by comparing current hip position with historical ratios between torso length, femur length, and leg length. When this comparison fails, the hip position is reconfigured, correcting the current drift.

Note

In certain situations, erroneous head positions produced by the skeleton tracker drag the shoulders slightly upwards. This affects the torso measurement and, consequently, the behavior of this filter.

Clipped Legs

This filter reduces position noise in legs that are clipped by the bottom of the camera FOV. In this scenario, the skeleton tracker uses limited amounts of depth map pixels from the user's upper thighs and hips to extrapolate inferred knee, ankle, and foot positions. This extrapolation magnifies small changes, leading to position noise. The clipped leg filter solves this problem with a double exponential filter and a high level of smoothing to remove most of the positional noise. These smoothed positions are blended in as more of the player's legs become clipped. Despite the high amount of smoothing, large leg movements (such as kicks and sidesteps) are reflected by the smoothed data and the filtered output.

Controls

Pressing the **Start** button while running the sample displays the available control options.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Sound Location Tracking Sample

[See Also](#)



Sound Location Tracking shows how to use the [NuiAudio API](#) to capture audio, and how to steer the beam-former to improve audio fidelity for recognizing speech from specific gamers in the room.

Description

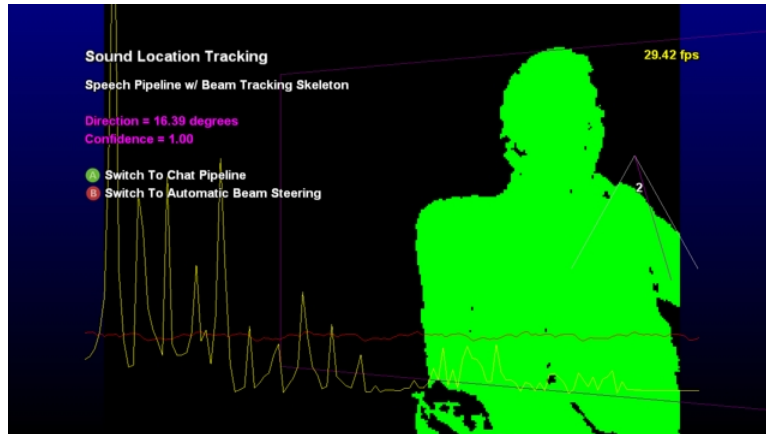
The NuiAudio pipeline performs extensive processing on incoming audio before it is passed on to a title, including multi-channel echo cancellation and automatic forming of audio beams to better isolate speaking gamers in noisy rooms by limiting sounds that do not arrive from a particular direction in a semicircle around the Kinect Sensor.

Sound Location Tracking demonstrates how to:

- Capture audio data from the Kinect Sensor through a low-latency callback.
- Switch between two available audio pipelines, speech and chat, that offer different features and performance characteristics.
- When using speech pipeline, it allows you to switch between manual beam steering, automatic beam steering (to the loudest audio source), and a mode in which the beam is automatically steered towards a person who is tracked by the skeletal tracking pipeline. This latter mode is ideal for gaming scenarios that use speech recognition in which you want to increase the fidelity of speech recognition for the active, currently engaged player.

Sound Location Tracking displays incoming audio from the Kinect Sensor as both an audio waveform (red) and a spectral chart (yellow), as seen in Figure 1. The sample also shows a diagnostic overhead view of the play space with the current beam direction highlighted.

Figure 1. Sound Location Tracking graphically displays incoming sound and its direction



Note

For best results, calibrate the microphone array of the Kinect Sensor by running the [Kinect Tuner for audio](#) before running this sample.

Details

- [Audio Pipeline Switching](#)
- [Automatic Tracking Mode \(Speech and Chat Pipelines\)](#)
- [Manual Steering Mode \(Speech Pipeline Only\)](#)
- [Beam-Steering To Tracked Skeletons \(Speech Pipeline Only\)](#)
- [Audio Visualization](#)

Audio Pipeline Switching

The chat pipeline uses fewer resources, but *only* provides automatic beam steering. The speech pipeline uses more resources and supports manual beam steering, as well as the same automatic mode as the chat pipeline. Control of the beam angle is available through [NuiAudioSetMicArrayBeamAngle](#). For more information, see [Chat Versus Speech Pipelines](#) in [Multichannel Echo Cancellation with NuiAudio](#).

Switching between pipelines requires shutdown and reinitialization of the NuiAudio subsystem.

Automatic Tracking Mode (Speech and Chat Pipelines)

In automatic tracking mode, the sensor array processes any sound, including speech or any environmental noise of substantial volume, and returns the angle of the sound source, in radians, relative to the Kinect Sensor.

To get the angle of the loudest sound from the Kinect Sensor, the **BeamAngle** member of the [NUIAUDIO_RESULTS](#) structure is inspected (rather than using the XHV2 function [NuiGetLocalChatData](#)). The returned angle is somewhat noisy and requires filtering.

Even if there are multiple audio sources in a room, only the angle of the loudest source is returned by the API.

This mode also uses the angle to set the tracking state of skeleton proposals through [NuiSkeletonSetTrackedSkeletons](#), engaging full skeleton tracking for any skeleton that appears to be an audio source.

Manual Steering Mode (Speech Pipeline Only)

In manual steering mode, the Kinect Sensor forms its beam along an angle set by calling [NuiAudioSetMicArrayBeamAngle](#). This angle can be modified by the user with the D-pad. Except for this key difference, the rest of the sample behaves the same way in this mode as when it Automatic Tracking mode.

Beam-Steering To Tracked Skeletons (Speech Pipeline Only)

In this mode, beam-steering to tracked skeletons, tracking behavior is flipped. The skeleton tracking system is told to calculate and select which of 6 potential skeletons is the best to accurately track. Skeleton tracking is directed to make such a selection by passing in 0 for both tracking IDs in a call to [NuiSkeletonSetTrackedSkeletons](#).

If no skeletons are currently tracked, the pipeline is set to automatic tracking, by passing in [NUIAUDIO_MICARRAY_BEAM_ANGLE_AUTO](#) for the angle in a call to [NuiAudioSetMicArrayBeamAngle](#). This allows speakers who are off-screen to still benefit from the beam-former's automatic behavior.

However, if any skeleton is currently tracked, the first one found in the [NUI_SKELETON_FRAME](#) returned by the skeleton tracking system will be used as a target for the beam-former. If available, the head joint will act as a target for the beam-former. (Otherwise, the center of mass of the skeleton will be used).

The X and Z coordinates of the target are combined using [atan2](#) to provide an angle from the center of the Kinect Sensor (at [0,0,0]) to the target, projected onto the XZ plane out of the sensor array (along the axis of the depth and color sensors).

Audio Visualization

Sound Location Tracking also demonstrates the use of fast Fourier transforms to create a frequency graph of the incoming audio. For more information, see [XDSP functions](#).

See Also

[Simple Speech Recognition Sample](#)

Version: 21256.3

Speech Retain Audio Sample



Speech Retain Audio shows how to capture audio data used by the speech recognizer along with the associated metadata.

Description

The sample captures the audio data for speech recognition in a three-step process. First, the raw audio data used by the speech recognition engine is exposed by setting the speech property [NUI_SPEECH_PROPERTY_RETAIN_AUDIO](#) by calling [NuiSetSpeechProperty](#). Second, the retained data is accessed by using the [NUI_SPEECH_RECORESULT](#) property of the [NUI_SPEECH_EVENT](#) structure. Finally, this data is post-processed, from a headerless 16-bit, mono, 16k PCM wave file in big-endian format, by adding a RIFF header and bit-swapping the data. The resultant audio is output to a .wav file in DEVKIT:\RETAIN_AUDIO\.

The metadata written in this example is written to a .txt file in DEVKIT:\RETAIN_AUDIO\ in the following format:

<PATH TO="" AUDIO="">;<RECOGNITION RESULT>;<SEMANTIC RESULT="">

The sample captures all audio received from the sensor array. This includes audio data identified by [NUI_SPEECH_EVENT_FALSE_RECOGNITION](#) and successful recognition events. Successful and unsuccessful events are equally important when tuning the grammar being tested. Using only successful events would influence the tuning efforts. For example, false recognitions provide insight into possible synonyms spoken by users that are not in the current grammar.

For more information about tools related to speech recognition, see [Speech Lab \(SpeechLab.xex\)](#) and [Speech Lab Analysis Tool \(xbSpeechLab.exe\)](#).

Controls

Pressing the **A** button starts or stops the speech recognition engine.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Stereoscopic 3D Head Tracking Sample

[See Also](#)



Stereoscopic 3D Head Tracking changes the perspective of the display based on the player's head movement, on a stereo 3D TV. It sets up projection matrices for a stereoscopic 3D (S3D) scene, based on the location of the head joint returned from the Skeleton Tracker. It doesn't use the Head Position Tracking API.

Two unique views are rendered (one for each eye), and the result is placed in a frame-packed front buffer for consumption by the display device. This sample also shows how to move the viewpoint of the scene based on the position of the gamer's head, provided by NUI skeletal tracking.

Description

Stereoscopic 3D Head Tracking generates projection matrices for the left and right stereoscopic 3D views using an algorithm adapted from the paper [Controlling Perceived Depth in Stereoscopic Images](#) by G. Jones, D. Lee, N. Holliman, and D. Ezra. In the algorithm, the physical display space is mapped to virtual cameras in a manner that preserves the depth effect for a moving head position. For more information about the technique, see [Considerations for High-Quality Stereoscopic 3D on Xbox 360](#), a white paper.

See Also

- [Head Tracking samples](#)
- [Considerations for High-Quality Stereoscopic 3D on Xbox 360](#)
- [NUI Skeleton Tracking](#)
- [NuiSkeleton Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

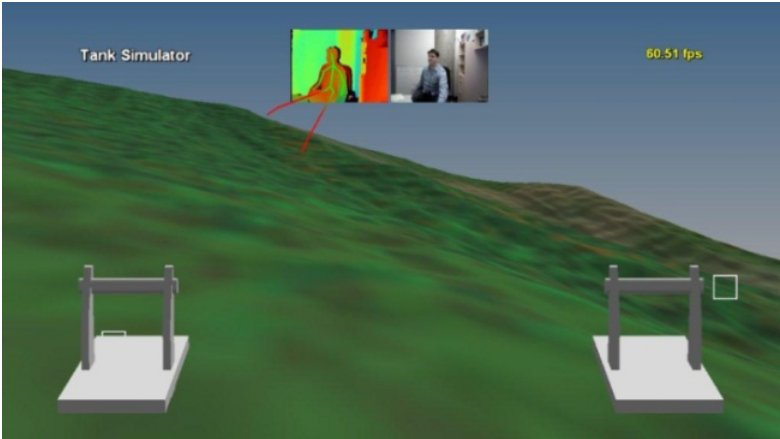
Tank Simulator Sample



Uses virtual controls to drive a virtual tank.

Description

Figure 1. The Tank Simulator sample



Tank Simulator uses a body-relative coordinate system so that the throttles are always out in front of the user.

Controls

Moving the left hand or the right hand forward snaps the throttle to the hand. The user then can control the correlated tank tread by moving a throttle forward to accelerate or backward to decelerate.

The user removes the hands from the controls by lifting them off. The controls snap back to their original position at the point where the user began removing the hands.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Theremin Sample



Description

A Theremin is a musical instrument that derives the pitch and volume for a tone based on the distance from the performer's hands to two proximity-sensing antennae on the instrument. This tone is then amplified and sent to a speaker. This sample simulates a virtual Theremin, allowing the user to produce tones by moving their hands.

Figure 1. The Theremin sample



The Theremin sample simplifies the operation of the Theremin by using similar control gestures as perceived by the [NUI Skeleton](#) API. These control gestures are then used to modify the output pitch and volume of a sine-wave sample; produced using the [XAudio2](#) API.

An interesting feature of the sample is the usage of a Picture in Picture feature (implemented by the ATGFramework project included in the sample) for drawing individual windows that display the raw data arriving from both the color and depth streams. In addition, an overlay of recognized player skeletons is also displayed.

Controls

The sample implements two different control schemes:

- The default control scheme locates a virtual Theremin in front of the user's current position. The antennae are positioned in space at roughly waist height in front of the player's shoulders.
- The second control scheme is a simpler version because there are no virtual Theremin antennae for manipulation. The pitch and tone are controlled by the current angle of the user's elbows to their arms.

The control schemes can be cycled by using the shoulder buttons of an attached gamepad.

Pressing **X** on the gamepad toggles between two different frequency modes: linear and octave. Linear frequency mode has a linear response curve. It is a quicker, but less accurate, type of frequency control. Octave frequency mode has a logarithmic response curve, which better fits the way that most musical instruments such as guitars and pianos behave. This mode also provides a more accurate control of the pitch.

Further Information

The following sections provide greater detail of various technical aspects of the sample. This information is not required for basic usage of the sample.

Natural Stance Detection

The sample attempts to detect when the player is no longer interested in playing the Theremin by determining if the player has turned away from the sensor array or dropped their arms to their sides. Both movements are detected using simple dot products. For speed considerations, the cosines of the relevant angles are stored rather than performing the calculation at runtime.

Elbow Angle Control Mode

The elbow angle control mode is a simple control scheme based on the current angle of the user's elbows. There is a dead zone at either end of the elbow's travel path, providing more tolerance for the differing mechanics of individual users.

This control scheme implements a body-local coordinate system without the overhead of actually generating a body-local coordinate system. A simple dot product between the elbow and hand skeleton positions and the elbow and shoulder skeleton position vectors for each arm is the only calculation required for determining the user's intent. The remaining calculations performed by this control scheme involve mapping the frequency and amplitude to linear ranges.

The mapping range for the elbows is from about 90 degrees (representing full amplitude and lowest frequency) to 165 degrees (representing zero amplitude and highest frequency). Note that this range is not a full 180 degrees (maximum angle for an open elbow). The range used was determined through iterative experimentation.

The control scheme implemented by this sample illustrates how simple gesture handling can provide usable results. It is not always necessary to implement an overly complex control scheme to achieve good control within your title.

Virtual Theremin Control Mode

This control scheme is more complicated than the elbow angle scheme:

- A virtual Theremin is positioned in front of the user at waist height.
- The control scheme is not based on a body-local coordinate system. Instead, it is a hybrid of the two control schemes. The user can make small movements without moving the virtual antennae. This is an attempt at making the Theremin seem locked in place in front of the user. This means that the Theremin follows the user as they walk around; settling in front of them when movement stops.
- The lengths of the user's arms are averaged by the sample and tracked in an effort to position the virtual Theremin within reach of individuals with varying body types.

These additional calculations all introduce complexity and the possibility for misinterpretation. When everything works, the control scheme is very natural and adaptable. However, there is a greater possibility for mistakes in skeletal tracking to occur, causing frustration for the user.

Waist Tracking

The sample tracks the position of the player's waist by performing a modified moving average (MMA) on the sample data. The window is 30 frames wide (or about 1 second long). The averaging equation is:

$$n' = \frac{n(w-1) + x}{w}$$

where w is the window length in frames, x is the new data, n is the last average, and n' is the newly calculated average.

This is an Exponential Moving Average (EMA), where $\alpha=1/w$.

If the user's waist moves out of a specific region (a sphere about 20cm in diameter), positioned at the last known good waist position, a recapture of the waist position is performed. After the recapture is finished, the averaged position is reset to the current position, and a settling time of one second is invoked. These calculations cause the virtual Theremin to automatically track the user as they move.

Arm Bone Length Tracking

Currently, arm bone lengths can suddenly change depending on occlusion of the skeleton. As a result, positioning objects in 3D-space becomes dependent on the user's body mechanics. With this in mind, bone lengths are averaged using an exponential windowed average, similar to the one used for the waist tracking calculation. The main difference is that the averaged value is not reset at any time because it is used to filter out noise, not to perform targeting or capture. The sample then uses the lengths of the individual bones to position the objects in space.

Virtual Antennae Positioning

In the sample, the virtual antennae are positioned in front of the user at waist height. Their positions are calculated as follows:

- 1. The vector across the shoulders of the player is calculated and crossed with the up vector. This calculation is performed after tilt correction; therefore, the up vector used is [0,1,0] and not the value returned by the hardware accelerometer. This returns an orthogonal vector (that is, forward vector) pointing outwards at right angles to the user's shoulders.
- 2. This forward vector is then normalized and multiplied by the length of the player's forearm to give a reasonable positioning (in front of the player) of the antennae.
- 3. The scaled forward vector is calculated relative to the left and right shoulder of the player. This provides a location in the local space of the sample (once again, in front of the player) for the left and right virtual antenna.
- 4. The y-component of the antenna positions is set to the waist height, earlier calculated by the waist tracking calculation.

Virtual Antennae Control

The virtual antennae have a control range between two spheres that are centered on the antennae positions. Default position values for the virtual antennae are used until tracking data comes in. This prevents any dependent position calculations from failing. The final values used by the antennae are generated from the user's arm bone lengths.

The inner sphere (of the control range mentioned earlier) represents the limits of the near-range dead zone for the controller. If the player's hand is within this sphere, the amplitude control is set to zero volume and the frequency control is set to its highest frequency. This matches the behavior of a real Theremin.

The outer sphere represents the limits of the far-range dead zone for the controller. If the player's hand is outside this sphere, the amplitude control is set to maximum volume and the frequency control is set to its lowest frequency. The outer sphere size is controlled by the player's total arm length.

The size of the inner dead zone spheres is different between the two antennae. The radius of the amplitude's inner dead zone is five times larger than the frequency's inner dead zone. This difference allows better staccato playing of the Theremin, making it easier to quiet the instrument without having to be totally accurate.

Frequency and Volume Control

The sample implements two mapping schemes for the audio frequency and also performs basic interpolation for the volume.

Frequency Mapping

The frequency range can be mapped in two ways (the selection of which is user-controlled). The first mapping is by linear frequency, where the controller range [0 ... 1] maps directly to the [min f - max f] range. Although human hearing responds to frequency logarithmically, this mapping method provides an adequate result. With this method, the frequency interval between two octaves doubles with the perceived increase by one octave of pitch.

As an alternate method, the second control scheme maps the controller range to the frequency range by mapping it from $c=[0 \dots 1]$ to $f_{log}=[\log_2 \min f \dots \log_2 \max f]$. The frequency is then retrieved by taking $2^{f_{log}}$.

Target Value Interpolation

The sample interpolates the frequency and volumes towards a target over time. They are limited to a change of approximately 10000Hz per second and a full-range traversal for the amplitude will take 0.1 seconds. Both of these values are represented as constants in the source code and can be adjusted by the developer.

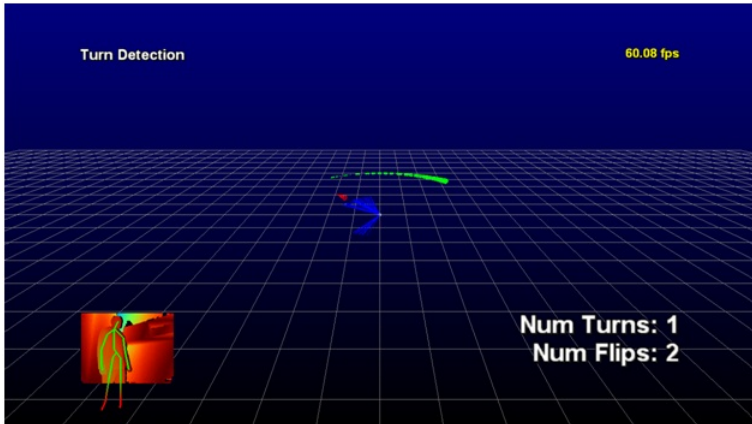
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Turn Detection Sample

Description

Turn Detection detects when a player turns around, by implementing a filter that operates on several body rotation angles that are calculated from shoulder and hip positions.

Figure 1. The Turn Detection sample



Turn Detection calculates a body rotation angle and indicates the rotation angle as a green sphere with a trail of the previous angles. It also shows the number of 180 degree flips the filter counted and calculates the number of full turns from that. Whenever a player's rotation angle is getting close to 90 degrees, the filter starts a search for an actual flip. Whenever the filter's search is active, all search angles will be indicated with blue direction vectors, and when a flip was found, that direction is indicated with a red cone.

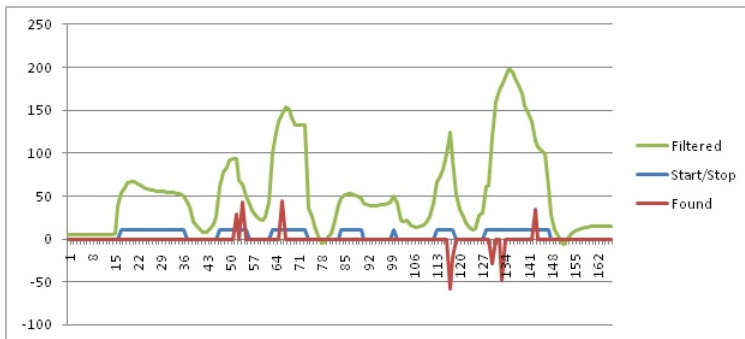
The **A** button can be pressed to reset the filter state.

How the Sample Works

The sample implements a filter that does the following:

1. Calculate four forward vectors using the shoulder and hip positions.
2. Calculate six body rotation angles from the vectors in step 1.
3. Calculate the minimum and maximum of the six angles in step 2.
4. Calculate the difference between the minimum and maximum angles.
5. When this difference is larger than a certain threshold, it indicates that we should search for a possible 180 degree flip.
6. Calculate the difference of the current shoulder angle and the previous shoulder angle.
7. While searching and the difference from step 6 is larger than a certain threshold, we found a flip.

Figure 2. How the filter works



The graph shows how two full turns completed by detecting two 180 degree flips counterclockwise, and then two 180 degree flips clockwise. The green graph shows the filtered values from step 4. The blue graph shows where the filter started to search for a flip, as in step 5. Lastly, the red graph indicates where we found 180 degree flips within a search, as in step 7.

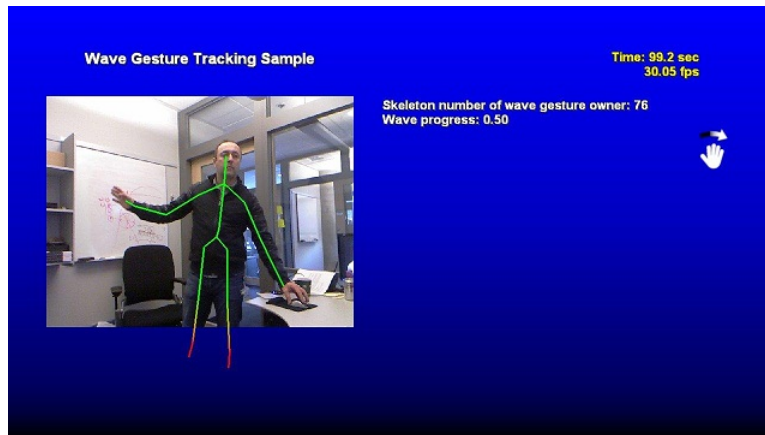
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Wave Gesture Tracking Sample

Description

Wave Gesture Tracking uses the NuiWave API to detect the arm waving gesture, and displays the gesture progress by using a progress bar and a hand icon.

Figure 1. The Wave Gesture Tracking sample



To enable wave gesture tracking, the sample calls [NuiSkeletonTrackingEnable](#) and then [NuiWaveSetEnabled](#).

To get the status of the wave gesture, the sample then calls another function.

- The function [NuiWaveGetGestureOwnerProgress](#), used in this sample, uses the default policy to decide which player owns the wave gesture if more than one player is waving. The default policy is that *the player whose wave gesture has the greatest progress* is considered the waving player. This default policy is sufficient for most use cases where a title just needs to detect whether any player has waved.
- To use a custom policy for deciding which player owns the wave gesture, you would instead call [NuiWaveGetState](#), which returns the wave gesture status for all tracked skeletons as the [NUI_WAVE_STATE](#) structure. You can use the data in this structure to implement a different policy for determining the gesture owner when two or more players are waving at the same time. For example, your custom policy could select the player who is closest to the Kinect sensor array, the player who started waving first, or the player who started waving last.

To display the progress of the wave gesture, this sample uses a progress bar that is shaped as a curved arrow, and a hand icon. The progress bar is filled within half a second after the progress returned by [NuiWaveGetGestureOwnerProgress](#) is incremented. A customized pixel shader draws the filling effect of the progress bar.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XedFileTagger Sample

[See Also](#)

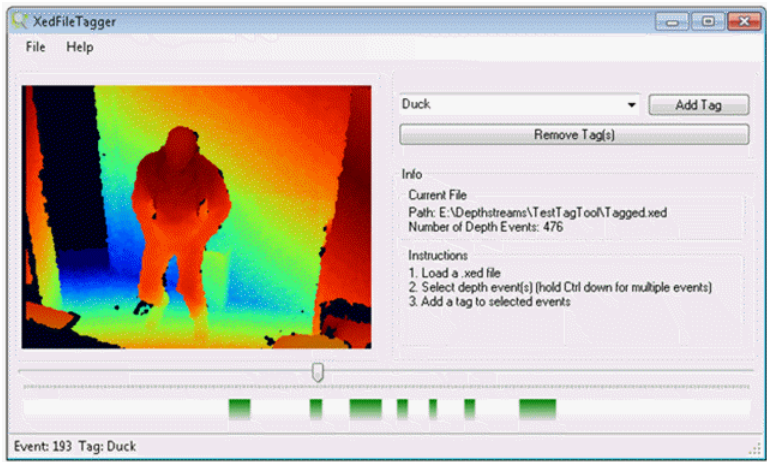
XED File Tagger shows how to use the [XedFile](#) API from within C# to add title-specific data to .xed files. In particular, each depth event can be tagged with a string value that can be used for helping with development of gesture detection or joint filtering.

Description

XedFile API can be used on the PC or Xbox 360 to read or write XStudio files. Since many software tools today are developed using C#, the sample shows how to use the XedFile API from the native XedFile.dll, in a C# application.

The sample also shows how to add new title-specific data to a .xed file. In this case, the user can select a depth event or a range of depth events and add tags in the form of text as title data. Any tags will be shown at the bottom of the window as a green bar with the status bar indicating the tag information of the currently selected depth event.

Figure 1. The XedFileTagger window



The XedFile API does not allow for reading and writing to the same stream simultaneously. Therefore, the sample first copies the file that will be read to %TEMP%\XedFileReader.tmp and then reads from that temporary file. When it saves a file, it writes to %TEMP%\XedFileWriter.tmp and then copies the temporary file to the original file. When saving a file, all events for color, depth, skeleton, identity, title data, and tag data are written in chronological order to ensure best performance when playing back the file.

Instructions

1.

Load a .xed file.

The .xed file should have recorded depth events. The file name and number of depth events are shown in the **Info** area.
2.

Use the scroll bar to view the depth events in the .xed file.

Then, select an event—or a range of events—by holding down the Ctrl key.
3.

Add a tag to the selected event(s) by typing new text or by selecting tags from the combo box.

Tags can also be removed from selected frames.

Note

If you use this sample to tag frames for machine learning gesture detection, you can use its batch conversion option to convert all tagged .xed files to the file format used by the Visual Gesture Builder tool.

See Also

- [Xbox Studio](#)
- [Visual Gesture Builder](#)
- [XStudio Reference](#)
- [XedFile Reference](#)
- [NuiGesture Reference](#)
- [Kick Boxing Sample](#)
- [Simple Gesture Detection Sample](#)
- [NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

XED To BVH Sample

[See Also](#)

Description

XED To BVH is a command-line tool that turns the Kinect Sensor into a motion-capture device. It takes the Xbox Studio recorded data file (.xed) as input, and converts it to a .bvh file. Then you can import the .bvh file into animation software (MotionBuilder for example) for further processing. Currently, the output .bvh is not compatible with Biped.

Usage

Make sure to include skeleton data when saving the .xed file from Xbox Studio. After that, run the following command, **XEDToBVH.exe /R n inputfile.xed outputfile.bvh**, where *n* specifies which frame in the input .xed file should be used as the reference frame for generating .bvh file. In the .bvh file, there is only one reference skeleton, where all bone lengths and offsets are specified, and all frames only include bone rotations to this single skeleton. So this reference frame number *n* should be chosen that the input frame doesn't have abnormal bone lengths.

See Also

- [Xbox Studio](#)
- [XStudio Reference](#)
- [XedFile Reference](#)
- [NUI Samples: Xbox Studio Samples](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface Tools

This section covers tools for developing Kinect titles.

In This Section

- [Kinect Tuner for Audio](#)
Profiles the acoustic environment of the play space, to optimize speech recognition.
- [NUIView Tool](#)
Displays wireframe and avatar representations of skeleton tracking data.
- [Speech Tools for NUI](#)
Tools for developing speech recognition features in Kinect titles.
- [Visual Gesture Builder](#)
Generates data used to perform gesture detection and tracking at run time.
- [Xbox Studio](#)
Displays, saves, and plays back the data streams from the sensor array.

Related Topics

- [XDK Unlocked - Videos](#)
- [NUI Overview](#)
- [NUI Reference](#)
- [NUI Samples](#)
- [NUI White Papers](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Kinect Tuner for Audio

[See Also](#)



Profiles the acoustic environment of the play space, to optimize speech recognition.
To use the microphone (mic) array in the Kinect sensor array, before you get data from the audio stream, you must first run the mic array calibration tool to profile the acoustic environment. This tool uses multichannel echo cancellation (MEC). This tool does not support voice chat headsets or USB microphones.

When to Run the Tool

You should run the Kinect Tuner for Audio whenever any one of the following occurs:

- Speaker locations are changed.
- The Kinect sensor array is moved.
- Large objects are moved that change the acoustics of the room.

Changing the tilt of the Kinect sensor array does not require re-profiling.

Consider prompting the user to run the Kinect Tuner for Audio when [NuiAudioIsCalibrationValid](#) returns FALSE.

Setting Up Your Acoustic Environment

Before running the Kinect Tuner for Audio, set up your acoustic environment as follows:

- Speech recognition works best when the player is 1 to 2 meters from the microphone.
- Any audio being played by the development console over main speakers should measure no more than 70 dB SPL loudness at 1 meter from the microphone.
- If you use an audio receiver, any additional signal processing (such as Loudness and Surround) should be turned off.
- Any background noise not generated by the development console should be kept to a minimum.
- Optimal sensor array placement is centered between the left and right speakers.
- Try to avoid indirect vibrations being transmitted to the sensor array.

Running the Tool

To run the Kinect Tuner for Audio, in the Guide, select **Home**, **Kinect Tuner**, and then **Audio**.

The Kinect Tuner for Audio profiles the acoustic environment in several steps:

1. The overall volume level from the speakers to the mic array is checked to ensure it is loud enough to run the calibration.
2. Test tones play in the speakers in the following sequence: Left Front, Right Front, Left Surround, Right Surround, Center.
3. The user speaks, such as "One, Two, Three, Four, Five."

The resulting acoustic environment profile is used automatically by the NUI Speech API.

See Also

[Audio Input with NUI Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUIView Tool



NUIView displays current information related to the skeleton tracking (ST) pipeline. This information includes a graphical representation of the player's skeleton and several different views of data related to the ST pipeline. NUIView also displays commands available by pressing buttons on the controller.

To run NUIView, in the Development Kit Launcher, scroll down to **nuiview.xex**, then press the **Launch** button.

Figure 1. Default view of NUIView Tool



The initial view is an overview mode that contains multiple windows: the main window, located in the left-hand region of the screen, and two secondary windows, located in the right-hand region of the screen. These windows can be configured to display different visualizations of the ST pipeline, processed from image data that is received from the Kinect Sensor.

Sections

- [Visualizations](#)
- [Data Visualizations](#)
- [Controls](#)

Visualizations

Skeleton and color visualizations appear by default in the window in the left-hand region of the screen, and they provide the following types of information:

- Points and Skeleton**
Displays pixels from the depth stream, transformed into 3D space, with an overlaid wireframe skeleton.
- Skeleton**
Displays the current position and orientation of the player's skeleton as a wireframe model.
- Points**
Displays pixels from the depth stream, transformed into 3D space.
- Color and Skeleton**
Displays pixels from the color stream, transformed into 3D space by the depth stream, with an overlaid wireframe skeleton.
- Color**
Displays pixels from the color stream, transformed into 3D space by the depth stream.
- Color (without background)**
Displays pixels from the color stream, transformed into 3D space by the depth stream, with the background removed.

Avatar visualizations appear by default in the window in the upper right of the screen, and they provide the following types of information:

- Avatar**
Displays an avatar that represents the current position and orientation of the player being tracked.
- Avatar and Skeleton**
Displays an avatar (with an overlaid wireframe skeleton) that represents the current position and orientation of the player being tracked.
- Avatar and Skeleton (no filtering)**
Displays an avatar (with an overlaid wireframe skeleton) without any filtering of skeleton positions.
- Retargeted Avatar and Skeleton**
Displays an avatar (with an overlaid wireframe skeleton) that represents the current player as naturally as possible, with many adjustments and refinements.

Data Visualizations

In addition to visualizations based on the lightly or heavily processed output of the different data streams, there is another set that illustrates how the pre-processing stage of the ST pipeline identifies the information in the frame. The following visualizations are currently available:

- BGR**
Displays the depth scene after the background has been removed.
- Player**
Displays the player mask that has been identified by the pre-processing stage.
- Color**
Displays the color stream.
- Depth**
The depth data as viewed by the Kinect Sensor.

By default, these visualizations are displayed in the lower right-hand window.

Controls

Below the current mode text, there is a green band that displays a set of buttons for common actions, the version number of the current firmware of the depth camera, and version number of NuiView.

Button Name	Action
BACK	Exits the application
Left/Right shoulder buttons	Cycles through available visualizations (previous with the left bumper and next with the right bumper) for the selected window.
X/Y buttons	Manipulates the tilt of the Kinect Sensor. Pressing X tilts the sensor up and pressing Y tilts the sensor down.
A	Pauses visualization of the data streams if running; otherwise, resumes visualization.

There are additional options that are not displayed on screen in the green band:

- START**
Displays available skeleton tracking modes. The default skeleton tracking mode is **Kiosk Mode**. The skeleton tracking modes are the following:
 - **Kiosk Mode** tracks a single player who is currently in the optimal location (front and center) of the playspace.
 - The **Default Policy** mode actively tracks the two players closest to the Kinect Sensor.

For more information about skeleton tracking modes, see [Active and Passive Skeleton Tracking](#) in [NUI Skeleton Tracking](#).

Direction pad (Left/Right/Up/Down)

Changes the selected window or, if the **Options** window is displayed, switches the current window.

Left stick

Controls the orientation and zoom of the current visualization. Pressing up zooms in, pressing down zooms out, and left or right strafes across the visualization respectively. Pressing the left stick cycles the positions of the different windows in a clockwise fashion.

Right stick

Controls the rotation and tilt of the current visualization. Pressing up tilts the visualization up, pressing down tilts it down, and left or right rotates the visualization accordingly. Pressing the right stick resets the orientation and position of the visualization.

Left/Right triggers

Raises (left trigger) and lowers (right trigger) the camera height of the visualization.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Speech Tools for NUI

[See Also](#)



These grammar development tools enable you to develop, test, and tune speech grammars for speech recognition features in Kinect titles. These tools are listed in the order in which they are typically used while developing speech functionality in a title.

In This Section

[Authoring Grammar Files in Visual Studio](#)

Add, edit, and validate speech grammar .grxml files in Visual Studio.

[Grammar Compiler \(xbpq.exe\)](#)

Compiles a grammar specification, in XML or CFG format, into a binary representation that is used by NUI speech recognition.

[Speech Lab \(SpeechLab.exe\)](#)

Tests the grammar of a NUI speech-enabled application.

[NUI Voice Collection \(NuiVoiceCollection.exe\)](#)

Gathers samples of users speaking, to use to test a grammar.

[Speech Prep \(xbSpeechPrep.exe\)](#)

Prepares formatted input files for the grammar tuning tools Simulator.exe and Speech Lab.

[Simulator \(Simulator.exe\)](#)

Performs speech recognition on an Xbox 360 development kit using pre-recorded speech files rather than input from the Kinect microphone array.

[Simulator Results Analyzer \(SimulatorResultsAnalyzer.exe\)](#)

Parses and analyzes the EMMA-format speech recognition results produced by Simulator.exe.

[Speech Lab Analysis Tool \(xbSpeechLab.exe\)](#)

Analyzes Speech Lab log files and reports confidence distribution, attempted phrase occurrences, and phrases with worst error rates.

[Analyzer Results Viewer \(AnalyzerResultsViewer.exe\)](#)

Graphically displays results from the Simulator Results Analyzer command-line tool, showing results of automated testing of speech recognition using pre-recorded voice files.

+ Remarks

The NUI Speech API is a direct sibling of the Microsoft Speech API (SAPI), and tools for SAPI grammar development can also be used for developing NUI speech grammars. The XDK includes a collection of useful tools for grammar development, including the help files that describe using those tools. These tools are run on your Windows-based PC, to analyze, debug, or optimize speech recognition grammars that conform to the grammar specification from the World Wide Web Consortium (W3C), described in the [W3C Speech Recognition Grammar Specification](#) on MSDN.

Some SAPI features that are supported by the NUI Speech tools (and mentioned in the tool help files) aren't supported by the NUI Speech API, which is a subset of SAPI. It is possible to use the NUI Speech tools to create a grammar that does not work as expected on the Xbox 360 because the grammar uses a feature that is not supported in the NUI Speech API.

The XDK installer installs the grammar tools to your development machine.

The speech grammar development tools are documented in the XDK and in detail in the Speech SDK Help file that is installed as part of the Microsoft Speech Platform SDK, such as (for version 11.0):

C:\Program Files (x86)\Microsoft SDKs\Speech\v11.0\Docs\MicrosoftSpeechPlatformSDK.chm

+ See Also

[NUI Speech Grammars](#)

[NUI Speech Overviews](#)

[NUI Speech Reference](#)

[NUI Samples: Audio and Speech](#)

[NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Authoring Grammar Files in Visual Studio



You can add, edit, and validate speech grammar .grxml files in Visual Studio. This schema-based parsing and validating uses speech extensions, which are Visual Studio plug-ins. The speech extensions are 32-bit only; they require the 32-bit version of Visual Studio. The speech extensions for Visual Studio are installed automatically with the XDK. This functionality supports grammar authoring for the Microsoft Speech Platform SDK from within Visual Studio 2010.

The speech extensions support Japanese (ja-jp) grammars, with display/lexical/pronunciation (DLP) format. However, the grammar file editing tools themselves are English-only, so English text appears in these tools in a JPN SKU of Visual Studio.

The tagging in the .grxml files conforms to the Speech Recognition Grammar Specification (SRGS). When you validate a grammar, it is also compiled against the xbpq.exe tool with errors displayed in the Output and Error windows, ensuring that the grammar will not fail when used against the Xbox development console environment.

This authoring is done in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Sections:

- [Input](#)
- [Output](#)
- [Adding an Existing Grammar File to a Visual Studio Project](#)
- [Creating a Pre-Populated .grxml Grammar File](#)
- [Creating an Empty .grxml Grammar File](#)
- [Validating Grammar Files](#)

Input

An existing .grxml file, or a pre-populated .grxml file provided by Visual Studio Speech Extensions, along with manual editing.

Output

A .grxml file.

Adding an Existing Grammar File to a Visual Studio Project

You can add an existing .grxml grammar file to a project. When you have added the grammar file, it will appear in the project list as part of the project.

To add a grammar to a Visual Studio project

- 1. In **Solution Explorer**, right-click the project to which you want to add an existing grammar, point to **Add**, and then click **Existing Item**. (Keyboard: Press **Shift+Alt+A**.)
- 2. In the **Add Existing Item** dialog, navigate to the .grxml file that you want to add and select it, and then click **Add**.
Note: You can also open the file from Windows Explorer: right click the file name, and then select **Open with**, and then **Visual Studio**. Also, you can add the file to an existing Solution instead of to a particular Project.

Creating a Pre-Populated .grxml Grammar File

To create and add a pre-populated, ready-to-use grammar file to your Xbox 360 Game project in Visual Studio 2010, do the following. This creates a GRXML grammar file for NUI speech recognition, conforming to the Speech Recognition Grammar Specification (SRGS).

To create a pre-populated .grxml grammar file

- 1. In Visual Studio 2010, in Solution Explorer, right-click your Xbox 360 game project, point to **Add**, then click **New Item**.
The **Add New Item** dialog box appears.
 - 2. In the tree on the left, click the **Data** group.
 - 3. Click **Xbox 360 Speech Grammar File (.grxml)**, enter a filename and location path, then click **Add**.
A .grxml file is created in your project directory and is added to your project. The namespace in the file points to an Xbox-specific SRGS schema.
- To compile the .grxml file into a .cfg file that can be used by the runtime NuiSpeech speech recognition engine, use the Xbox Prepare Grammar tool (xbpg.exe). .grxml files can't be consumed directly by NuiSpeech. The NuiSpeech engine uses .cfg or .cfpg files, which are compiled grammars in binary format. See [Grammar Compiler \(xbpg.exe\)](#).
- The .grxml file that is added is shown below.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<grammar mode="voice"
  root="MEDIA_PLAYBACK"
  tag-format="semantics/1.0-literals"
  version="1.0"
  xml:lang="en-US"
  xmlns="http://www.w3.org/2001/06/grammar"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions">
  <rule id="MEDIA_PLAYBACK" scope="public">
    <item>
      xbox
    </item>
    <one-of>
      <item>
        play<tag>play</tag>
      </item>
      <item>
        stop<tag>stop</tag>
      </item>
      <item>
        fast forward<tag>fast forward</tag>
      </item>
      <item>
        rewind<tag>rewind</tag>
      </item>
      <item>
        pause<tag>pause</tag>
      </item>
      <item>
        faster<tag>faster</tag>
      </item>
      <item>
        slower<tag>slower</tag>
      </item>
      <item>
        next<tag>next</tag>
      </item>
      <item>
        previous<tag>previous</tag>
      </item>
    </one-of>
  </rule>
</grammar>
```

For the grammar format for Japanese, see [Local Language Support with NUI Speech](#).

Creating an Empty .grxml Grammar File

In Visual Studio, you can create a new empty or pre-populated grammar file and author contents for the grammar.

To create a new, empty grammar file in Visual Studio

- 1. In **Solution Explorer**, right-click the project to which you want to add an existing grammar, point to **Add**, and then click **New Item**. (Keyboard: Press **Ctrl+Shift+A**.)
 - 2. In the **Add New Item** dialog, under **Visual Studio Templates**, select **XML File**.
 - 3. In the **Name** box, enter a name for the file, type **grxml** for the extension, and then choose **Add**.
- Visual Studio supports authoring of grammar files by validating entries against XML and grammar schemas as you type.

Validating Grammar Files

You can verify from within Visual Studio that one or all grammars associated with a project contain correct structure and syntax.

To validate a single grammar

- 1. Add or create a file within Visual Studio.
If the file has a ".grxml" extension, you will see real-time validation of the schema and semantic script as you type. **Errors**, **Warnings**, and **Info** will appear in the **Error** window.
- 2. In **Solution Explorer**, right-click the file that you want to validate, and then click **Validate Grammar**.

Information about the results of validation will appear in the **Error List** and the **Output** window.

To validate all grammars associated with a project

- On the Menu Bar, choose **Speech**, and then choose **Validate all grammars**.
Information about the results of validation will appear in the **Error List** and the **Output** window.
Visual Studio will only validate grammar files that have a ".grxml" extension.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Grammar Compiler (xbpg.exe)

[See Also](#)



The Xbox Prepare Grammar tool (xbpg.exe) compiles an existing grammar specification, in XML or CFG formats, into a binary representation that is used by NUI speech recognition. This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).
For an example of usage of NUI Speech, see [Simple Speech Recognition Sample](#).

Input File

The XML input file must conform to the grammar specification from the World Wide Web Consortium (W3C), described in [W3C Speech Recognition Grammar Specification](#) on MSDN. To create a ready-to-use XML file as a starting point, see [Authoring Grammar Files in Visual Studio](#).
For additional information on developing grammars for use in NUI Speech, see [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Output File

- Xbpg.exe can generate two types of compiled grammars: CFG (.cfg) files and CFG++ (.cfgp) files.
- CFG (.cfg) files are the same type of files generated by gc.exe, the grammar compiler that shipped with previous versions of the XDK. These files are *portable* in that they do not include engine-specific information inside of the binary that is generated. At run time on the Xbox 360 console, this means that NUI Speech will be doing extra compilation work to generate engine-specific pronunciation information, including the loading of the lexicon file that is specific to that language.
 - CFG++ (.cfgp) files are generated by folding in engine-specific pronunciation information at compilation time. This means that the model files used by NUI Speech need to be available and referenced at compilation time. When the engine loads a .cfgp file, it does not need to load the lexicon file at run time, unless the title is using a dynamic grammar and adding pronunciations at run time.

Command-line Syntax

```
xbpg [ /p Filename] [ /g Filename] [ /m ModelPath] input_filename
```

Option	Description
<i>/p <i>Filename</i></i>	Compiles the input file as a CFG++ file that has the specified filename. The CFG++ file folds in information that is specific to the engine against which it is compiled.
<i>/g <i>Filename</i></i>	Compiles the input file as a CFG file that has the specified filename.
<i>/m <i>ModelPath</i></i>	Specifies the path to the language models. Language models need to be laid out according to the following hierarchy: drive:\path\speech\<language>*. The /m switch allows you to specify where drive:\path is. NUI Speech will then load the models from speech\<language>*. If /m is not specified, then the directory is assumed to be %XEDK%\redist\xbox\NaturalInput (%XEDK% resolves to the XDK installation path on the development machine). This option is only valid if /p is already specified.
<i>input_filename</i>	Name of the file containing the grammar to compile. The input file can be either an XML file that uses the proper schema, or a CFG file generated by xbpg.exe or by the now-deprecated grammar compiler, gc.exe.

Usage Examples

Generate a CFG file

```
xbpg.exe /g grammar.cfg grammar.grxml
```

Generate a CFG++ file

```
xbpg.exe /p grammar.cfgp grammar.grxml
```

Generate both a CFG and CFG++ file and point to the models path

```
xbpg.exe /g grammar.cfg /p grammar.cfgp /m c:\models grammar.grxml
```

Japanese

For the grammar format for Japanese, see [Local Language Support with NUI Speech](#).

See Also

[Authoring Grammar Files in Visual Studio](#)
[Workflow for NUI Speech Grammar Development](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Speech Lab (SpeechLab.xex)

[See Also](#)



The Speech Lab tool enables you to do ad-hoc testing of the grammar of a speech-enabled Natural User Interface (NUI) application. You can load grammars, and then test them by saying words or phrases from the list of grammar elements shown by the tool. The recognition confidence and number of times the word or phrase was recognized is updated automatically. A log file, created on the console's hard drive, is also created containing this information. Later on, the title developer can use this log file to refine and improve the title's grammar recognition. This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Figure 1. Speech Lab in scripted mode

Speech Lab					
Phrase	Hits	Phrase Confidence	Avg Phrase Confidence	Property Confidence	Avg Property Confidence
cast fireball	2	1.00	0.99	0.98	0.98
drink health potion	2	0.99	0.99	0.98	0.98
open spellbook	1	0.83	0.83	0.97	0.97
pause game	1	0.98	0.98	0.98	0.98
view inventory	1	1.00	1.00	0.98	0.98

Y Switch to Scripted Mode
 X Clear Phrases

Menu

B Toggle Listen
 A

Sections

- [Getting Started](#)
- [Recognition Modes](#)
- [Main Menu Commands](#)
- [Miscellaneous Commands](#)
- [Command Line Options](#)
- [Analyzing Results](#)

Getting Started

Speech Lab (SpeechLab.xex) is provided with the XDK recovery, allowing grammars to be tested on an Xbox 360 console without integrating it into a game build. Grammars are compiled on the development computer using the [Xbox Prepare Grammar tool \(xbpg.exe\)](#).

To copy, load, and test a grammar file:

1. Copy the grammars (.cfg or .cfpg files) to be tested into the `e:\SpeechLab\cfg` directory on the console's hard drive. The speech lab tool can load any grammar found in the `devkit:\SpeechLab\cfg\` directory.

The following command example uses the **xbcp** command to copy the `mygrammar.cfg` file to the proper location on the console's hard drive.

```
xbcp -qtyd mygrammar.cfg devkit:\SpeechLab\cfg\
```

2. In the development kit launcher, start the **Speech Lab** application (SpeechLab.xex).
Speech Lab starts, and loads the first grammar file it finds in the `\cfg` directory.
3. If you want to load additional grammar files, press the controller's **Start** button, then select **Load Grammar** from the menu. Load multiple grammars simultaneously to test combinatorial scenarios.
Or, use the `/grammar=filename1;filename2` command line option to specify grammars to be loaded when speech lab starts.
4. Press **A** to activate listening mode.

To test, see the section "Testing and Tuning Process Flow", in the white paper [The User Says What? Testing and Tuning Guidelines for Speech Recognition with Kinect for Xbox 360](#).

Recognition Modes

Speech lab operates in one of three recognition modes.

- Free speech recognition mode processes input through the Kinect, reporting speech recognition events based on the sound picked up by the microphone array.
- Wave file recognition mode processes WAV files as input, and reports speech recognition events based on those input sound files.
- Text emulation mode processes a script file and reports recognition events as directed by the script. Confusability testing is a special case of text emulation mode. It performs a processor-intensive evaluation of a grammar and reports grammar rules that are similar enough that an utterance intended to be recognized by one rule, could be mistakenly reported as a match for another rule.

The recognition modes are mutually exclusive, so if you are using command line options to choose a mode, you may only specify one of the `/processScript`, `/playwav`, and `/checkConfusability` options.

Free speech recognition

In free speech mode, speech lab performs recognition on sound input from the Kinect microphone array, and maintains a phrase list on screen listing recognized grammar elements. For each element, it reports current hit count and an averaged confidence percentage value. This value represents the percentage, according to the speech engine, of how closely the input phrase matches the acoustic model for the given word. Subsequent recognitions of an existing word or phrase automatically increment the **Hits** column, and update the confidence percentage average for that grammar element.

By default, speech lab starts in free speech recognition mode, and you press the **A** button to initiate recognizing inputs. If you pass the `/autolisten` command line flag, then speech lab immediately begins recognizing inputs. This can be useful for some testing scenarios.

Wave input recognition

In wave input recognition mode, speech lab uses a set of WAV files as inputs. Each file is treated as a separate utterance to be recognized. You can enter wave recognition mode by choosing the **Process WAV Files** item from the main menu, or you can enter it by passing the `/playwav` command line flag.

There are two ways to specify the set of WAV files to be used. You can list the files in a script file, or you can choose to let speech lab operate on all of the WAV files in the game:\wav directory. To use a script file in wave input recognition mode while running speech lab, use **Load Script** from the main menu before you choose **Process WAV Files**. From the command line, pass a valid `/script=` value along with the `/playwav` flag. If you specify `/playwav` on the command line without specifying a script file, or you choose **Process WAV Files** from the main menu without loading a script first, then all of the WAV files in the game:\wav directory are used.

Text emulation

In text emulation mode, speech lab loads a script file and treats each line of the script file as a directive to return a particular recognition result. The WAV file part of the script line is ignored, and the Expected Phrase part of the script line is returned verbatim. To use text emulation mode from the main menu, choose **Process Script as Input** after loading a script. From the command line, pass `/processScript` and a valid `/script=` value. Note that in text emulation mode, the script is not optional.

The columns for each phrase in this mode differ from the columns shown in the default recognition mode. Instead of a single confidence percentage, there are three percentages (WER, SER, PER).

Confidence Percentage Type	Description
Word Error Rate (WER)	How many words in a phrase matched the expected phrase. 0.0 means no mismatches, ≥ 1.0 means no words matched.

Sentence Error Rate (SER)	Describes an exact phrase match (0.0 == match, 1.0 == no match).
Property Error Rate (PER)	Similar to WER, but the expected phrase's tag is compared against the input phrase's tag. This is only available if the grammar is created using the <tag> field to identify an alternate input.

 Note

In text emulation mode, the input phrase appears first at the top of the recognition list. After a successful recognition, it is moved to the next line.

Confusability checking is a special case of text emulation mode where the contents of the script are used as inputs to a process that identifies grammar rules that might be confused for the intended rule. Each script entry is checked against all rules, and when more than one match is possible, all possible matches are reported. Confusability checking requires extensive processing, and speech lab operates noticeably more slowly in this mode.

Confusability checking can only be initiated using command line arguments. Pass the "/checkConfusability" flag and a valid "/script=" value on the command line.

Confusability checking does not cause additional output to the screen. Instead, it inserts additional information about confusable grammar elements into the devkit:\speechLab\speechlab.xml log file for the session, detailing all of the possibly confusable recognition results for the utterance. The format of that output is:

```
<ALTERNATES count="1" confusion="on">
  <ALTERNATE>
    <RECO>open spellbook</RECO>
    <CONFIDENCE>1.000000</CONFIDENCE>
  </ALTERNATE>
</ALTERNATES>
```

Using script files

Speech lab can accept input based on the contents of a script file. The file consists of one or more lines, where each line specifies information about one test utterance. Script files lines have three parts, separated by the vertical bar character. The final part is optional. If you choose not to specify an expected tag, then the second vertical bar should be the last character on the line.

```
# Comment lines start with '#'
WAV file path|Expected phrase|Expected tag (optional)
```

Script files should be placed in the devkit:\SpeechLab\scripts\ folder on your development console. The following command example uses the xbcpc command to copy the myscript.txt file to the proper location on the console's hard drive.

```
xbcp -qtyd myscript.txt devkit:\speechLab\scripts\
```

When a script is used in live input mode, speech lab treats each non-comment line of the script as a required utterance. The expected phrase is displayed on the screen, and only when that phrase is recognized does the script advance. Even if the microphone picks up an utterance that appears elsewhere in the script, it will not be recognized. Only recognition of the current utterance will advance the script.

When a script file is used with either the "/processScript" option or the "/checkConfusability" option, the WAV file is not used, and the WAV file path is ignored.

Main Menu Commands

Press the **Start** button to access the main menu. You can use the main menu to access the following features:

- **Load Grammar**—Loads the specified grammar file from the cfg subdirectory of the SpeechLab tool.
- **Load Script**—Loads the specified grammar script from the scripts subdirectory of the SpeechLab tool. You can only use the scripts during the scripted recognition mode.
- **Save Phrases to a Script file**—Saves the current list of recognized scripts to the local script.txt file. If script.txt already exists, it is automatically overwritten.
- **Start Recording Audio Data**—Saves recorded voice as a WAV file in the devkit:\SpeechLab\wav directory. A **Save Script Name** prompt will appear. Each file contains a single word or phrase. A corresponding transcript file is saved in the devkit:\SpeechLab\scripts directory. Each script file will contain the phrases spoken during the session.
- **Process WAV Files**—Processes all existing WAV files found in the game:\wav directory as if they were spoken individually into the microphone array. The tool assumes that each file only contains a single word or phrase.
- **Process Script as Input**—Processes the phrases (updates their hit count and confidence percentage) in the currently-loaded script as if they had been spoken.
- **Enter Confidence Threshold**—Sets the confidence threshold to a value between 0 and 100. The default confidence threshold is 40.
- **Turn Adaptation On/Off**—Enables or disables speech engine adaptation.

 Note

Adaptation is always disabled during WAV processing.

Miscellaneous Commands

- Press **A** to activate the speech recognition engine that starts processing the microphone array input.
- Press **X** to clear the current phrase list.
- Press **Y** to toggle the current recognition mode.

Command Line Options

The following table lists the available command-line options:

Command	Description
/adaptation=on off	Turns engine adaptation on or off.
/autolisten	Starts SpeechLab in recognition mode—essentially pressing the "A" button for you. Use this option to implement automated testing.
/checkConfusability	Run in text emulation mode and perform grammar confusability checking. Note that this is a processor-intensive process. Response is sluggish when performing confusability checking. Requires a valid "/script=" value.
/clearLog	Clears the contents of the speech lab log file.
/complexResponseSpeed= <i>n</i>	Sets the complex engine response speed property.
/garbage=on off	Specifies whether SpeechLab will ignore garbage rules in the result.
/grammar= <i>filename1</i> ; <i>filename2</i>	Specifies the filename (or filenames, delimited by a semicolon) of a compiled grammar. It is assumed that the specified argument is in the devkit:\SpeechLab\cfg directory. You can use /grammar= <i>filename1::rulename1</i> ^ <i>rulename2</i> to specify the rules.
/infile= <i>filename</i>	Allows using a .ini file to pass in command-line options. This assumes the file is stored in the devkit:\SpeechLab directory.
/inputscript= <i>filename</i>	Specifies the filename of an input phrase script. It is assumed that the specified filename is local, and that it is located in the scripts subdirectory of the tool. Note that some input modes require this value, while others treat it as optional.
/maxAlts= <i>n</i>	Specifies the maximum number of alternate phrases for which to search. The default value is 5. Whenever this value is greater than zero, and an utterance might match more than one grammar rule, all of the potential matches are reported in the devkit:\speechLab\speechlab.xml log file. The format of the reported potential matches is the same as the format used by confusability reporting, as detailed in the Text emulation mode section of this topic.
/numLoops= <i>n</i>	The number of times to loop through specified wav files. If using a script, this is the number of times that the script is run. If not using a script, this is the number of iterations over the set of wav files in game:\wav.

/outputsript= <i>filename</i>	Specifies the filename of an output phrase script when using the /savewav command. This argument is optional.
/phrase.cutoff= <i>n</i>	Sets the phrase confidence threshold for rejected recognition events.
/playwav	Run in wave input mode. If a valid "/script=" value is given, use the script to specify which WAV files to use. If no script is specified, use all WAV files in the game:\wav directory.
/processScript	Run in text emulation mode. Requires a valid "/script=" value.
/reboot	Reboots the console after performing the specified checks: • If a script was specified, then at the end of the script. • If no script was specified, then when all files have been processed.
/responseSpeed= <i>n</i>	Sets the engine response speed property.
/savewav	Saves recorded voice as a WAV file in the devkit:\SpeechLab\wav directory and a corresponding script file in the devkit:\SpeechLab\scripts directory.
/script= <i>filename</i>	Specifies the filename of an input phrase script. (This is the same as /inputscript.)
/tag.cutoff= <i>n</i>	Sets the tag confidence threshold for rejected recognition events.
/thresh.low= <i>n</i>	The phrase confidence threshold for rejected phrases. <i>n</i> must be greater than zero and less than thresh.med. The default value is 1.
/thresh.med= <i>n</i>	The medium phrase confidence threshold. <i>n</i> must be greater than thresh.low and less than thresh.high. The default value is 50.
/thresh.high= <i>n</i>	The high phrase confidence threshold. <i>n</i> must be greater than thresh.med, and less than 100. The default value is 80.
/?	Display command line help in the debugger.

To run speech lab with command line options, use the [Remote Reboot \(XbReboot\)](#) command from the computer attached to your development console:

```
xbreboot xe:\Speechlab\Speechlab.xex /infile=test.ini
```

Analyzing Results

The speech lab tool generates an XML log file on the console, at devkit:\speechlab\speechlab.xml, for each session. The [Speech Lab Analysis Tool \(xbSpeechLab.exe\)](#), which is run on the development system, analyzes a log file and generates a report in HTML format. The report contains information about the confidence distribution, attempted phrase occurrences, and the phrases with the worst error rates.

See Also

[Workflow for NUI Speech Grammar Development](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Voice Collection (NuiVoiceCollection.xex)

[See Also](#)



Use NuiVoiceCollection.xex to gather samples of users speaking, for use in testing your title's grammar. This tool collects voice data that has already been processed for multichannel echo cancellation (MEC), and saves it so that it can be played back later for testing, and for analyzing and tuning your grammar. NUI Voice Collection (NuiVoiceCollection.xex) is a tool provided with the XDK recovery, allowing you to capture voice data from a targeted audience of end-users to create a source data set for NuiSpeech simulation and analysis.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Sections

- [Setup](#)
- [Gathering Voice Data](#)
- [Finding and Using Voice Data](#)
- [The AppConfig.xml File](#)
- [Script file](#)
- [Using the tool](#)

Setup

To use NuiVoiceCollection to gather user speech samples, you must first set up the developer console where it will run. NuiVoiceCollection and sample input and configuration files are copied to your XDK console during the [XDK Recovery](#) process. You can find NuiVoiceCollection.xex in the xe:\nuivoicerecollection directory, and sample input and configuration files in the xe:\nuivoicerecollection\media subdirectory.

1. Create an input file with the phrases that users will read.
The input file is a Unicode text file, listing each phrase on a separate line of the file. Note that this file must be saved as Unicode.
The input file should include all of the phrases that your grammar generates, as well as some phrases that are not in your grammar. A mixture of in- and out-of-grammar phrases is necessary for robust grammar tuning. In-grammar phrases must be recognized, of course. But it is just as important that out-of-grammar phrases are not recognized incorrectly. The grammar tuning process finds the optimal threshold values for distinguishing in- and out-of-grammar.
A sample input file named *example-script.txt* is in the xe:\nuivoicerecollection\media subdirectory.
2. In the *languages.xml* file, set the commandRes attribute for the appropriate language, to the name of the input file discussed in the previous step.
A sample languages.xml file is provided in the xe:\nuivoicerecollection\media subdirectory.
You can put multiple <language> tags in languages.xml if you want to gather voice data for multiple languages. Users will be prompted to indicate which language they are speaking.
3. In the *AppConfig.xml* file, set the BasicCommandSample value to be the number of phrases to be used for testing. This number should be less than or equal to the number of phrases in your input file.
A sample *AppConfig.xml* file is provided in the xe:\nuivoicerecollection\media subdirectory.

Gathering Voice Data

After the developer console is properly configured, run NuiVoiceCollection and perform one session for each user whose voice samples are being collected. The application first gathers some information about the user, such as gender, age, the language being collected, and where the user was raised. Next, it calibrates the Kinect sensor for ambient noise. Finally, it presents a

series of phrases on screen, asking the user to read each one out loud.

After each phrase is read, click the **Continue** button to proceed to the next phrase. If necessary, click the **Retry Phrase** button to repeat the phrase. Audio data is saved when the **Continue** button is pressed, and only for the most recent attempt.

If you are collecting data from users such as children, who might have trouble understanding what to say, start the collection process and have a researcher explain to the user what they should say. This explanation will be captured. At this point, press the **Retry Phrase** button, and then have the user speak. The explanation will be deleted, and only the user's speech will be saved.

Finding and Using Voice Data

The tool creates a new directory under the `xe:\NuiVoiceCollection` directory for each session. Directories are named SessionXXXX, where XXXX starts at 0001 and is incremented for each new session. The session directory contains a VoiceCollection.ini file that captures the user information gathered during session setup.

In the session directory, a new .wav file is created for each phrase the user spoke. These files are named UtteranceYYYY, where YYYY starts at 0001 and is incremented for each utterance. Note that the audio in these files is captured after MEC processing.

Given the same input file and configuration files, every users' Utterance0001.wav file is the recording of that user saying the first phrase in the input file, every Utterance0002.wav us the recording of the second phrase, and so on.

Copy these files to your Windows-based PC to use them as input for analysis and grammar tuning with the [Speech Tools for NUI](#).

Before starting data collection, modify or supply the following files:

Name	Purpose	Location
AppConfig.xml	Defines the number of phrases captured during voice collection, and specifies whether to play or not play background audio.	XE:\NuiVoiceCollection\media
Example-Script.txt	Lists the phrases to be captured for voice data collection. Localize into each supported language.	XE:\NuiVoiceCollection\media
Languages.xml	References all instances of Example-Script.txt, an input file for each language.	XE:\NuiVoiceCollection\media
Background.wav	Required audio file that is played during data capture. Up to 6 files can be referenced, and they are played sequentially. One file, silence.wav, is provided with the XDK.	XE:\NuiVoiceCollection\media

During each session of data collection, NUI Voice Collection generates the following files:

Name	Purpose	Location
VoiceCollection.ini	Captures the meta-data entered by the participant and a transcript of the phrases captured. One file is generated per session.	XE:\NuiVoiceCollection\Session000N
Utterance000N.wav	Audio file captured, one per utterance spoken by the participant.	XE:\NuiVoiceCollection\Session000N

The AppConfig.xml File

The AppConfig file contains detailed configuration information for the NuiVoiceCollection application. The AppConfig file should be placed in the `xe:\nuivoicerecollection\media` subdirectory.

The configuration information includes the following elements and attributes:

session element: **MaxFanOffTime** attribute

This integer is the number of minutes to turn off the cooling fan in the Kinect sensor while the session is in progress. Use this value to ensure that no fan noise is captured along with voice data. Set the number to allow reasonable time for a session to be completed.

BGMusic element: **Name** attribute

This is the name of a .wav file that will be played in the background while gathering samples. Music tracks are assumed to be under game:\media. You can have up to 6 different audio files by having multiple BGMusic elements. Use this value to provide game background noise that your testing indicates might be interfering with recognition or causing false positives. If you do not want to use specific background noise, use the provided silence.wav file.

track element

NuiVoiceCollection only supports a single track.

category element: **numPhrases** attribute

Ensure that the numPhrases <= number of utterances in the script. The full script name is composed of the prefix specified for each language in languages.xml concatenated with the name here. The actual filename for the category shown in this example is: media\example-script.txt

```
The following code is an example of AppConfig.xml.  
  
<xml version='1.0' encoding='UTF-16'>  
  <session MaxFanOffTime="35">  
    <BGMusic Name="silence.wav"/>  
    <track>  
      <category Name="example-script.txt" numPhrases="8"/>  
    </track>  
  </session>
```

Script file

Add the list of phrases to the script file; an example script, Example-Script.txt, is installed with the XDK. In your script file, be sure to reference both in-grammar (IG) phrases and out-of-grammar (OOG) phrases. IG phrases are the words that match the title's grammar(s). For example, if the grammar contains the phrase open spellbook, it would be logical to have open spellbook as an utterance to capture from users. OOG phrases, which do not match the intended grammar, should comprise 50% or more of the script's contents.

A script file used to capture US English phrases might contain phrases like the following (from Example-Script.txt):

```
#In Grammar Phrases  
open spellbook  
view inventory  
show quests  
pause game  
main menu  
drink health potion  
cast fireball  
  
#Out of Grammar Phrases  
fire weapon  
move forward  
take cover  
disarm bomb  
next  
back  
select
```

Provide one input script file for each language the application supports. Each input script file must be referenced in languages.xml. This file can contain, for example, references to scripts for multiple languages:

```
<xml version='1.0' encoding='UTF-16'>  
  <languages>  
    <language Name="Deutsch" scriptPrefix="media\"/>  
    <language Name="English" scriptPrefix="media\"/>  
    <language Name="Español" scriptPrefix="media\"/>  
    <language Name="Français" scriptPrefix="media\"/>  
    <language Name="Italiano" scriptPrefix="media\"/>  
    <language Name="???" scriptPrefix="media\"/>
```

```
</languages>
</xml>
```

Using the tool

Use: Invite each participant to use NUI Voice Collection. The Voice Collection tool collects demographic information and then prompts participants to speak the phrases in the script (or a subset of the phrases, depending on the value of **BasicCommandSample** in `AppConfig.xml`).

Once all of the participants have been through voice collection, copy the data from the directory `xe:\NuiVoiceCollection\Session000N\` on the development console to the development computer.

The following is a recommended organization of these files on the development computer:

```
C:\VoiceData\language\SessionNumber\UtteranceNumber.wav
```

For example:

```
C:\VoiceData\Japanese\Session0001\VoiceCollection.ini
C:\VoiceData\Japanese\Session0001\Utterance0001.wav
C:\VoiceData\Japanese\Session0001\Utterance0002.wav
```

See Also

- [Workflow for NUI Speech Grammar Development](#)
- [Using NUI Speech Grammars](#)
- [NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Speech Prep (xbSpeechPrep.exe)

[See Also](#)



The xbSpeechPrep command-line tool is used to prepare a collection of voice input samples that have been collected with the [NuiVoiceCollection](#) tool, for use by [Simulator.exe](#) or [Speech Lab](#), tools used to tune and validate speech recognition grammars.

xbSpeechPrep searches the specified directory for VoiceCollection.ini files and to create input files for use with Simulator.exe and/or Speech Lab. Specify a grammar, model language and output filename to create an EMMA XML input file for Simulator.exe. Or specify a script filename to create one Speech Lab script file for each VoiceCollection.ini file found in the data directory.

xbSpeechPrep enumerates VoiceCollection.ini and wave files that are collected from the NUI Voice Collection tool.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Input

Recorded voice samples from the Xbox console.

Output

An EMMA-formatted collection of the recorded voice input samples. xbSpeechPrep generates output files formatted in Extensible MultiModal Annotation (EMMA) markup language. Can also create Speech Lab script files.

Command-line Syntax

```
xbSpeechPrep [/data:inputdir] [/cfg:grammar.cfg /model:xx-yy /out:emmafile.xml] [/script:file] [/LimitOOG]
```

Option	Description
/data:inputdir	(Required) The top-level directory to be searched for voice data in the form of captured utterances and VoiceCollection.ini files.
/cfg:grammar.cfg /model:xx-yy /out:emmafile.xml	This group of switches is required for Simulator EMMA creation. (Optional) /cfg:grammar.cfg specifies the filename of the compiled grammar file for the grammar to be tuned by Simulator.exe. To tune multiple grammar files, use this switch multiple times, once for each .cfg file. (Optional) /model:xx-yy specifies the spoken languages to include in the generated grammar file. Use the standard xx-yy format for specifying languages, such as ja-jp for Japanese (Japan), en-us for English (United States), or es-mx for Spanish (Mexico). To specify that samples from multiple languages should be included, specify this flag multiple times, once per language. You can also specify wildcards for one or both parts of the language specifier, such as /model:* to specify all languages found, or /model:es-* to specify all Spanish files found, no matter which region. Utterances will only be included in the generated EMMA where the spoken language is covered by the speech model. (Optional) /out:emmafile.xml specifies the filename of the EMMA-formatted output file, to be used as input to Simulator.exe.
/script:file	(Optional) The filename of the Speech Lab script files to generate. This argument is required for Speech Lab script file creation. One Speech Lab script file will be created for each VoiceCollection.ini file found and will be written to the same directory as the VoiceCollection.ini file.
/LimitOOG	(Optional) Only applicable to Simulator EMMA creation. Limits the number of Out-Of-Grammar utterances in the Simulator EMMA input file to a maximum of half of the total utterances. If a large percentage of collected utterances are out-of-grammar, this can greatly speed up the test pass, while maintaining a sufficient number of out-of-grammar utterances to get useful results.

Remarks

If you specify both the **/cfg** option group and the **/script** option, then xbSpeechPrep creates both output files.

xbSpeechPrep is designed to work with voice data collected using the NuiVoiceCollection tool and stored using the following organization:

```
c:\VoiceData\<Language>\<Session000N>\<data>
```

For example:

```
c:\VoiceData\es-mx\Session0001\
VoiceCollection.ini
Utterance0001.wav
Utterance0002.wav
```

For more information about using collected voice data in your grammar development workflow, see [Using NUI Speech Grammars](#).

See Also

- [Workflow for NUI Speech Grammar Development](#)
- [NUI Whitepapers: Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simulator (Simulator.exe)

[See Also](#)



Simulator.exe is a command-line tool that performs simulated speech recognition on an Xbox 360 development kit using pre-recorded speech wave files rather than the live input stream from the Kinect Sensor's microphone array. The speech recognition engine runs on the Xbox 360 development kit, so you must have a powered-on development kit configured as the default console for the developer PC where you are running Simulator.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Additional documentation for this tool is available in the Help file that is installed as part of the Microsoft Speech Platform SDK, such as:

C:\Program Files (x86)\Microsoft SDKs\Speech\v11.0\Docs\MicrosoftSpeechPlatformSDK.chm

The version number in the path is subject to change.

Input

When tuning for NUI Speech, Simulator uses input files gathered using the [NuiVoiceCollection](#) tool. The input to Simulator is a list of captured voice samples, listed in the EMMA (Extensible Multi-Modal Annotation markup language) schema. Use the [xbSpeechPrep](#) tool to prepare the EMMA-formatted input file list for Simulator.

Simulator.exe requires audio files, transcript text for input, and one or more grammars for recognition matching. The tool uses a language-specific file, RecoConfig_<languagecode>.xml, to provide recognition results from the NUI speech engine running on the development console.

A RecoConfig_<languagecode>.xml file for each language/country combination is located in the directory:

%XEDK%\redist\speech\tools

Output

The output of Simulator tool is an XML-format file using the Extensible Multi-Modal Annotation (EMMA) schema, and indicates which phrase in a grammar was matched by the input. When used for tuning NUI speech, this tool uses a speech recognition engine running on the Xbox 360 development kit to provide recognition results. To analyze the output of this tool, use the [Simulator Results Analyzer](#).

Command-line Syntax

Simulator -in <FileName> [-Out <FileName>] [-RecoConfig <FileName>] [-MaxActiveRecognitions <integer>] [-Filter <FileName>]

[-HideInitialEngineState]

Parameter	Example Filename	Required?	Purpose	Origin of File
-in <FileName>	SimulatorInput.xml	Required	EMMA-annotated input file describing utterances to run through the simulator.	Created as output of xbSpeechPrep.exe.
-Out <FileName>	SimulatorOutput.xml	Optional	EMMA output document that reports simulated recognition results. If not specified, the filename defaults to the input filename and inserts a ".out" between the filename and its extension.	Output from Simulator.exe.
-RecoConfig <FileName>	RecoConfig_en_us.xml	Required	Configuration file to use for the recognizer. The <Provider> element specifies to use the Xbox engine, which is on the console. The <Properties> element specifies the language acoustic model to load.	Installed as part of the XDK.
-MaxActiveRecognitions <integer>	--	Optional	The maximum number of active recognitions; the maximum number of connections to make at one time to the recognition engine. Default is 20.	--
-Filter <FileName>	filterfile.xml	Optional	XML file used to specify whether the audio and/or transcripts are processed for each utterance in the EMMA input document. Specifies whether everything in the EMMA input file should be processed, or just a subset. If not specified, all audio and transcripts are processed for recognition.	User-provided.
-HideInitialEngineState <boolean>	--	Optional	Whether to suppress the initial engine state data from the output of the tool. Defaults to false , causing the initial engine state data to appear in the output. If set to true , the initial engine state data is omitted from the output, producing a smaller output file.	--

A full example of the input and output files for Simulator.exe are located in the directory %XEDK%\redist\speech\tools, such as the following:

C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools

Custom Pronunciations and Testing

While Speech Lab supports custom pronunciations, many of the other XDK speech testing tools do not, or they handle them differently than typical lexical pronunciations. In particular, for Simulator.exe, if the grammar is Japanese or contains custom pronunciations, the EMMA input to the tool must have the phrases specified in DLP format:

/D/L/P;

The terminating semicolon (;) is required. The other elements have the following meanings:

/**D** is the *display text* of the element, defined in the "sapi:display" attribute of the <token> tag. In Japanese, this is either Kanji, Hiragana, or Katakana, while in other languages this is either explicitly defined or is the same as the lexical form. In the following examples, /**D** is "/spellbook" or "/?".

/**L** is the *lexical form* of the element, which is defined in the inner text of the <token>. In Japanese, this is Hiragana. In the following examples, /**L** is "/spellbook" or "/???".

/**P** is the *pronunciation* of the element, defined by the "sapi:pron" attribute of the <token> tag. In Japanese, this is Katakana. For other languages, this is UPS pronunciation. In the following examples, /**P** is "/S P EH L B UH K" or "/???".

For example, a U.S. English grammar that contains the following item:

<item>
 open <token sapi:pron="S P EH L B UH K">spellbook</token>
 <tag>open spellbook</tag>
 </item>

must define the transcript of this phrase in the EMMA input to Simulator.exe as:

<ms:transcript>
 <ms:original>open /spellbook/spellbook/S P EH L B UH K;</ms:original>
 </ms:transcript>

Only the portion of the phrase that is defined in the grammar with a custom pronunciation needs to be in DLP format for the transcript.

For Japanese, all transcripts in the EMMA input files must be in DLP format. For example, a Japanese grammar containing the phrase song:

<item>
 <token sapi:display="?" sapi:pron="????"></token>
 </item>

requires the following format for the utterance transcript in the EMMA input to Simulator.exe:

<ms:transcript>
 <ms:original>/?????;</ms:original>
 </ms:transcript>

See Also

[Workflow for NUI Speech Grammar Development](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simulator Results Analyzer (SimulatorResultsAnalyzer.exe)

[See Also](#)



Use Simulator Results Analyzer to generate analyzed results of automated testing of speech recognition using pre-recorded voice files. SimulatorResultsAnalyzer.exe is a command-line tool that analyzes the EMMA-formatted speech recognition results produced by Simulator.exe.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Input

The input for Simulator Results Analyzer is a language-specific EMMA-formatted recognition results file, produced by Simulator.exe, that indicates which phrase in a grammar was matched by the input. This recognition results file is a language-specific file, RecoConfig_ *language_locale*.xml.

For example:

```
C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools\RecoConfig_en_us.xml
```

Output

This tool produces an XML-formatted file that contains the following:

- Information about each input utterance
- An analysis over all utterances
- An analysis over utterances in any specified grammar sets
- Whether utterances were correctly or incorrectly accepted or rejected, for a range of confidence score thresholds
- Whether or not utterance transcriptions are in-grammar
- Sums and percentages that illustrate the accuracy of recognition.

Full example output files for SimulatorResultsAnalyzer.exe are located in the directory:

```
%XEDK%\redist\speech\tools\
```

For example:

```
C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools\SimulatorResultsAnalyzerOutput.xml
```

Use

Additional documentation for this tool is available in the Help file that is installed as part of the Microsoft Speech Platform SDK, such as:

```
C:\Program Files (x86)\Microsoft SDKs\Speech\v11.0\Docs\MicrosoftSpeechPlatformSDK.chm
```

The version number in the path is subject to change.

See Also

[Workflow for NUI Speech Grammar Development](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Speech Lab Analysis Tool (xbSpeechLab.exe)

[See Also](#)



The Speech Lab Analysis tool, xbspeechlab.exe, analyzes Speech Lab log files and reports confidence distribution, attempted phrase occurrences, and phrases with worst error rates.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Input

The Speech Lab Analysis tool analyzes the log files generated by the [Speech Lab](#) tool and [SimulatorResultsAnalyzer.exe](#).

To get meaningful results when you analyze the log, you need to generate the log file while the Speech Lab tool is in Scripted mode for the Speech Lab Analysis tool.

Output

The Speech Lab Analysis tool generates a report in HTML format, showing the weakest performing phrases and guidelines for initial confidence values.

The report contains information about the following:

- The confidence distribution
- Attempted phrase occurrences
- The phrases with the worst error rates.

For an example of output, see [report.htm](#) in the directory:

```
%XEDK%\redist\speech\tools\
```

such as:

```
C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools\Report.htm
```

Command-line Syntax

```
xbspeechlab [ /x:name ][ /in:inputfile.ext ][ /file:batchlist.txt ] [ /out:file.htm ][ /usephraseconf ] [ /exportToExcel ]
```

Option	Description
/x:name	Specifies the name of the devkit console to query. This switch must come before the /in switch that references the devkit.
/in:inputfile.ext	Provides a source input file. Multiple switches are acceptable.

The following file types are allowed:

- filename.xml—SpeechLab or SimulatorResultsAnalyzer XML output file
- directory/speechlab*.xml—All files matching the pattern under the given directory.
- devkit:filename—Copies the file from an Xbox
Example: /in:devkit:\speechlab\speechlab.xml

/file:batchlist.txt	A source input file that lists the paths to individual input files. Use this to batch multiple input file specifications.
/out:file.htm	Filename for HTML report output. If no output file name is specified, then a file is created in the local directory, named "reportDATETIME.htm", where DATETIME is the date and time the report was generated, in a sortable format.
/usephraseconf	By default, output charts are based on semantic (property or tag) confidence values. When this option is specified, phrase confidence is used instead.
/exportToExcel	Exports report data to Excel for further analysis.

See Also

[Workflow for NUI Speech Grammar Development](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Analyzer Results Viewer (AnalyzerResultsViewer.exe)

Analyzer Results Viewer (AnalyzerResultsViewer.exe) is a graphical application that displays speech grammar recognition results produced by the command-line tool, [Simulator Results Analyzer \(SimulatorResultsAnalyzer.exe\)](#).

The Analyzer Results Viewer provides a visual representation of the output from the Simulator Results Analyzer tool (SimulatorResultsAnalyzer.exe), which analyzes recognition result sets produced by [Simulator \(Simulator.exe\)](#). Using the Analyzer Results Viewer, you can analyze utterance recognition results against one or more grammars, or compare results from multiple result sets. Analyzer Results Viewer makes it easy to see high-level summaries of recognition results and to drill down to individual utterances to troubleshoot issues.

You can analyze the results of a Simulator run against a batch of utterances and receive analysis results that will help you do the following:

- Accuracy - Evaluate the overall accuracy of the utterances when recognized against your set of one or more grammars.
- Out of grammar phrases - Examine out-of-grammar (OOG) phrases and consider whether to add them to your grammars as synonyms for related in-grammar phrases.
- Confidence Adjustment - Identify the confidence level setting for the speech recognition engine that balances the trade-off between correct accepts and false accepts and provides optimum recognition accuracy.

These activities are described in more detail below. For more information about analyzing recognition results, see [Interpreting the Results](#) at MSDN.

This tool is used in a sequence with other tools for NUI Speech Grammar tuning. See [Workflow for NUI Speech Grammar Development](#).

Sections:

- [Input](#)
- [Output](#)
- [Installing the Analyzer Results Viewer](#)
- [Launching the Analyzer Results Viewer](#)
- [Loading a File](#)
- [Analyzing Accuracy](#)
- [Analyzing Out-of-Grammar \(OOG\) Phrases](#)
- [Analyzing Confidence Adjustment](#)
- [Comparing Multiple Result Sets](#)
- [Viewing Definitions of Grammar Terms](#)

Input

The Analyzer Results Viewer takes the output from the Simulator Results Analyzer tool (SimulatorResultsAnalyzer.exe). Full example output files for SimulatorResultsAnalyzer.exe are located in the directory:

%XEDK%\redist\speech\tools\

For example:

C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools\SimulatorResultsAnalyzerOutput.xml

Output

This tool provides graphical displays; it doesn't produce output files.

Installing the Analyzer Results Viewer

The Analyzer Results Viewer tool may already be installed on your system if you have installed the latest version of the Xbox Developer Kit. If the Analyzer Results Viewer is not already installed:

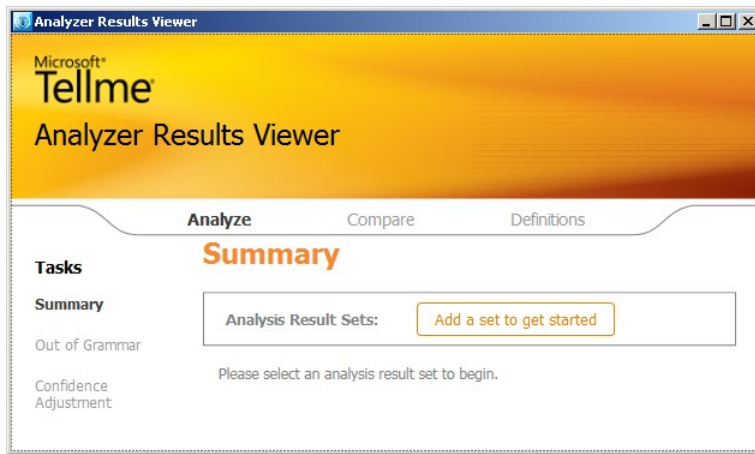
- Locate and run the installation file `AnalyzerResultsViewer.msi`.
- Alternatively, you may also install the tool by locating and running `MicrosoftSpeechPlatformSDKExtensionsForXbox.msi`.

Launching the Analyzer Results Viewer

1. Locate the folder where the Microsoft Speech Platform was installed during XDK installation. The default is C:\Program Files (x86)\Microsoft SDKs\Speech\v11.0.
2. In the `Tools\Analyzer Results Viewer` folder, open the file `AnalyzerResultsViewer.exe`.

The Analyzer Results Viewer appears. The **Summary** link is initially selected under **Tasks** in the upper left. The **Analyze** page is initially selected in the middle of the top.

Figure 1. Initial display on startup



Loading a File

1. Launch the Analyzer Results Viewer, as described above.
2. Under **Summary**, next to **Analysis Result Sets**, click the button **Add a set to get started**.
The Open dialog box appears, with the **XML documents (*.xml)** file filter selected.
3. Browse to an output file from Simulator Results Analyzer, such as the provided example file `C:\Program Files (x86)\Microsoft Xbox 360 SDK\redist\Speech\Tools\SimulatorResultsAnalyzerOutput.xml` and open it.

The counts and percentages are displayed for Semantic Accuracy, Phrase Accuracy, and Word Accuracy.

Figure 2. Accuracy summary

Semantic Accuracy	Phrase Accuracy	Word Accuracy	Copy Tables
Count : 934	Phrase Count : 934	Word Count : 1629	
Incorrect : 294	Incorrect : 765	Insertions : 61	
Correct : 640	Correct : 169	Deletions : 769	
Error Rate : 31.48%	Error Rate : 81.91%	Substitutions : 470	
Accuracy : 68.52%	Accuracy : 18.09%	Error Rate : 79.80%	
		Accuracy : 20.20%	

Details

No detail to display. Please select an overview item.

Utterances

No utterance to display. Please select a row in the Details section.

Recommendations

No recommendations currently.

Analyzing Accuracy

The **Summary** page identifies phrases that may require attention due to frequent incorrect recognitions. This evaluates the overall accuracy of the utterances when recognized against your set of one or more grammars. The following figures are particularly important:

- **Error Rate** indicates what percentage of the attempts to recognize individual words were unsuccessful.
- **Deletions** indicates the number of words for which there was no recognition result.
- **Substitutions** indicates the number of words that were incorrectly matched to a phrase in a grammar.

To analyze a result set for accuracy

1. Load a file, as described above.
The **Summary** page is initially selected under **Tasks** in the upper left. The **Analyze** page is initially selected in the middle of the top.
2. To view a breakout of the transcript text detail for any of the counts or percentages, click a number in one of the **Accuracy** tables.
The detail for the selected number displays in a table in the Details section under the Accuracy tables.

Figure 3. Accuracy Details

Semantic Accuracy	Phrase Accuracy	Word Accuracy	Copy Tables
Count : 934	Phrase Count : 934	Word Count : 1629	
Incorrect : 294	Incorrect : 765	Insertions : 61	
Correct : 640	Correct : 169	Deletions : 769	
Error Rate : 31.48%	Error Rate : 81.91%	Substitutions : 470	
Accuracy : 68.52%	Accuracy : 18.09%	Error Rate : 79.80%	
		Accuracy : 20.20%	

Details - Word Accuracy - Error Rate: 79.80%

Word	Count
Xbox	12
Previous	15
Fast	6
Forward	5
Stop	16
Play	52
Rewind	16
Slower	13
Next	13
Forward->slower	6
Faster	15
Dead	11
Avatar->xbox	17
Editor->slower	5
Cancel	14

- To see the list of the utterances associated with one of the Transcript entries, click a row in the table under **Details**. The Analyzer Results Viewer displays the list of utterances associated with the selected Transcript item.
- Click on any of the displayed utterances to audition the associated audio. The audio file plays if the file is in the location indicated.

Figure 4. Utterances

Next	13
Forward->slower	6
Faster	15
Dead	11
Avatar->xbox	17
Editor->slower	5
Cancel	14

Utterances

Click on a wave file name to listen to the utterance.

Utterance ID	Wave File Name	Wave Full Path	Transcript Text	Transcript Semantics	Reco Result Text	Reco
utterance_26	Utterance0013.i	file:///C:/Program%20Files%20(x	Xbox Fast Forward	fast forward	xbox slower	slow
utterance_363	Utterance0067.i	file:///C:/Program%20Files%20(x	Fast Forward		xbox slower	slow
utterance_451	Utterance0065.i	file:///C:/Program%20Files%20(x	Fast Forward		xbox slower	slow
utterance_507	Utterance0058.i	file:///C:/Program%20Files%20(x	Fast Forward		xbox slower	slow
utterance_541	Utterance0019.i	file:///C:/Program%20Files%20(x	Fast Forward		xbox slower	slow
utterance_797	Utterance0071.i	file:///C:/Program%20Files%20(x	Fast Forward		xbox slower	slow

To sort a table under **Details** or **Utterances**, select a column heading.

Analyzing Out-of-Grammar (OOG) Phrases

The **Out Of Grammar** page lists each out-of-grammar phrase that was mistakenly recognized for a phrase in the grammar and keeps a count of how many times each phrase was accepted in error. Words that were not in the grammar but that were accepted for an in-grammar phrase by mistake may indicate that you can improve a grammar's performance by adding one or more phrases. Examine out-of-grammar phrases and consider whether to add them to your grammars as synonyms for related in-grammar phrases.

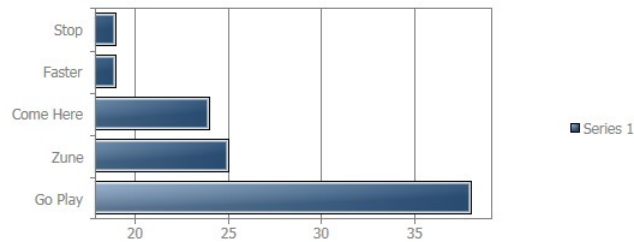
To analyze a result set for out-of-grammar phrases

- Load a file, as described above.
- Under **Tasks** in the upper left, click **Out of Grammar**.

The most frequently occurring out-of-grammar phrases are displayed.

Figure 5. Top Out of Grammar phrases

Top Out Of Grammar Phrases



Details

Transcript	Count
Go Play	38
Zune	25
Come Here	24
Faster	19
Stop	19

3. To see the list of the utterances associated with one of the Transcript entries, click a row in the table under **Details**.

The Analyzer Results Viewer displays the list of utterances associated with the selected Transcript item.

Figure 6. Out of Grammar phrases

Details

Transcript	Count
Go Play	38
Zune	25
Come Here	24
Faster	19
Stop	19
Cancel	18
Pause	18
Play Disc	18
Rewind	18
Avatar Editor	17
Kinect	17
No	17
Play	17
Shuffle	17
Stop In	17

Utterances

Click on a wave file name to listen to the utterance.

Utterance ID	Wave File Name	Wave Full Path	Transcript Text	Transcript Semantics	Reco Result Text	Rec
utterance_58	Utterance0038.i	file:///C:/Program%20Files%20(x	Come Here		xbox slower	slov
utterance_79	Utterance0065.i	file:///C:/Program%20Files%20(x	Come Here		xbox slower	slov
utterance_110	Utterance0023.i	file:///C:/Program%20Files%20(x	Come Here			
utterance_129	Utterance0049.i	file:///C:/Program%20Files%20(x	Come Here			
utterance_164	Utterance0012.i	file:///C:/Program%20Files%20(x	Come Here		xbox slower	slov

4. Click any of the displayed utterances to audition the audio for the out-of-grammar utterances in the result set. The audio file plays if the file is in the location indicated.

Analyzing Confidence Adjustment

The **Confidence Adjustment** page in Analyzer Results Viewer recommends the confidence threshold setting on the production speech recognition engine that provides optimum recognition accuracy for the input utterances. The recommended threshold setting identifies the best balance between correct accepts and false accepts to provide the best recognition experience for users.

- The speech recognition engine calculates a confidence score for each utterance and its semantics.
- An utterance is accepted if the confidence score assigned by the speech recognition engine to the utterance is greater than or equal to the value of its confidence threshold.

It is important to include utterances that are out of grammar (50% is recommended) in the input to the Simulator tool. Using only in-grammar utterances may lead you to select a confidence threshold that is too low, leading to a poor user experience.

To analyze a result set for optimum confidence threshold

- Load a file, as described above.
- Under **Tasks** in the upper left, click **Confidence Adjustment**.

The percentages for accepted and rejected phrase recognitions are displayed, at each of 21 confidence thresholds from 0.00 to 1.00 in increments of .05.

Figure 7. Confidence Adjustment page, summary

Confidence Type: Phrase**Optimal Threshold**

Optimal Threshold: 0.55

At this threshold, the projected accuracy is 97.22%.

CA_all: 17.24% CR_all: 79.98% FA_all: 0.54% FR_all: 2.25% CA_in: 88.46% FA_in: 0.00% FR_in: 11.54% CR_out: 99.34% FA_out: 0.66%

Details

Threshold	CA all	FA all	CR all	FR all	CA in	FA in	FR in	CR out	FA out
0.00	18.09%	30.19%	50.43%	1.28%	92.86%	0.55%	6.59%	62.63%	37.37%
0.05	18.09%	6.10%	74.52%	1.28%	92.86%	0.55%	6.59%	92.55%	7.45%
0.10	17.99%	4.50%	76.12%	1.39%	92.31%	0.55%	7.14%	94.55%	5.45%
0.15	17.99%	3.75%	76.87%	1.39%	92.31%	0.55%	7.14%	95.48%	4.52%
0.20	17.99%	3.53%	77.09%	1.39%	92.31%	0.55%	7.14%	95.74%	4.26%

To see a description of each of the column headings in the **Details** table, click the **Definitions** tab at the top.

- To see the percentages for accepted and rejected utterances (based on semantic comparison), in the **Confidence Type** drop-down list, click **Semantic**.
- To see the list of the utterances associated with one of the calculated percentages, click a row in the **Details** table.

The list of utterances associated with the selected percentage calculation are displayed. There are no utterances associated with the threshold levels 0.00 and 1.00.

Figure 8. Confidence Utterances

0.90	13.06%	0.00%	80.51%	6.42%	67.03%	0.00%	32.97%	100.00%	0.00%
0.95	12.21%	0.00%	80.51%	7.28%	62.64%	0.00%	37.36%	100.00%	0.00%
1.00	0.00%	0.00%	80.51%	19.49%	0.00%	0.00%	100.00%	100.00%	0.00%

Utterances

Click on a wave file name to listen to the utterance.

Utterance ID	Wave File Name	Wave Full Path	Transcript Text	Transcript Semantics	Reco Result Text	R
utterance_11	Utterance0020.u	file:///C:/Program%20Files%20(x	Xbox Rewind	rewind	xbox rewind	re
utterance_14	Utterance0039.u	file:///C:/Program%20Files%20(x	Xbox Play	play	xbox play	pl
utterance_15	Utterance0044.u	file:///C:/Program%20Files%20(x	Xbox Pause	pause	xbox pause	px
utterance_23	Utterance0002.u	file:///C:/Program%20Files%20(x	Xbox Slower	slower	xbox slower	sl
utterance_24	Utterance0003.u	file:///C:/Program%20Files%20(x	Xbox Next	next	xbox next	ni
utterance_25	Utterance0011.u	file:///C:/Program%20Files%20(x	Xbox Pause	pause	xbox pause	pi
utterance_27	Utterance0023.u	file:///C:/Program%20Files%20(x	Xbox Stop	stop	xbox stop	st
utterance_32	Utterance0001.u	file:///C:/Program%20Files%20(x	Play Dead			
utterance_34	Utterance0004.u	file:///C:/Program%20Files%20(x	Cancel			

You can load multiple files and view them individually. To load an additional file, click the **Add** button. Select the file name next to **Analysis Result Sets** to work with the selected result set.**Figure 9. Add button****Tasks**

Summary

Out of Grammar

Confidence AdjustmentTo sort the **Details** or **Utterances** table, select a column heading.**Confidence Adjustment**Analysis Result Sets: [...sAnalyzerOutput^x](#)

Add

Comparing Multiple Result Sets

Using Analyzer Results Viewer, you can compare the results of recognition from multiple result sets, for example to see which version of a grammar produces superior recognition accuracy.

To compare multiple result sets

- Load a file, as described above.
- Click the **Compare** tab, in the middle of the top.
- Under **Comparison**, next to **Analysis Result Sets**, click the button **Add another set to compare**.
- In the **Open** dialog, select one or more files and open them.

The Analyzer Results Viewer displays the counts and percentages for Semantic Accuracy, Phrase Accuracy, and Word Accuracy for all files opened.

Under the **Accuracy** tables, the Analyzer Results Viewer plots the accuracy results into separate correct accept/false accept (CA/FA) graphs that compare the accuracy of semantic recognition and phrase recognition among the loaded result sets.

- If you want to open additional files from another location, click the button **Add another set to compare**.
- To unload a result set, click the "x" next to the file name.
- To return to the Analysis page, click **Analyze** in the middle of the top.

Viewing Definitions of Grammar TermsThe **Definitions** page lists and describes general terms related to grammar analysis.

- Click the **Definitions** tab, in the middle of the top.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Visual Gesture Builder

[See Also](#)



Visual Gesture Builder (VGB) provides an Interactive Development Environment (IDE) to generate data that a title uses to perform gesture detection at run time. The [NuiGesture Reference](#) APIs enable you to use the data generated by VGB to perform run-time gesture detection.

Requirements

Operating System

A computer running Windows 7 or Windows Vista.

Note

This tool may require large amounts of RAM. For best results, use it on a computer running a 64-Bit edition of Windows.

Visual Studio 2010

Microsoft Visual C++ 2010 Redistributable and Microsoft .NET Framework 4 Client Profile. Each come with Visual Studio 2010 or can be downloaded separately. VGB requires Visual C++ 2010 Redistributable to support multithreading when creating a gesture database.

Files

Xbox Studio (.xed) files that at least contain skeleton and depth information.

Screen Resolution

The minimal screen resolution is 1024×768.

In This Section

[Overview](#)

Gives an overview of the Visual Gesture Builder tool.

[Quick Tour](#)

Gives a quick tour of the Visual Gesture Builder tool.

[Tutorial: Creating a Gesture Database](#)

Shows how to create a gesture database.

[Machine Learning Technologies](#)

Describes the technologies used to create the gesture data.

[Keyboard Shortcuts](#)

Describes the keyboard shortcuts used in Visual Gesture Builder.

[Xed Skeleton Updater](#)

Converts XED files for use in Visual Gesture Builder.

See Also

[NuiGesture Reference](#)

[Simple Gesture Detection Sample](#)

[Natural User Interface Tools](#)

[Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Overview

[See Also](#)



The Visual Gesture Builder (VGB) tool provides an Interactive Development Environment (IDE) to generate data that a title uses to perform gesture detection at runtime. Even for simple cases, gesture detection is a challenging task that requires many lines of code to obtain reliable results for different players and play spaces. By using a data-driven model, the VGB tool shifts the emphasis from code writing to content producing. This simplifies the developer's task for gesture detection.

Note

You can download a complete example at this location: [Simple Gesture Detection Content](#).

Machine learning technologies or algorithms, namely [AdaBoostTrigger Machine Learning](#) or [RFRProgress Machine Learning](#), are at the core of the VGB tool. The user selects the machine learning technology to use and tags frames in a clip related to a meaningful event, such as a punch, a kick and so on. At the end of the tagging process, the user builds a gesture database. This database contains data such as hit detection and swing progress that a title can use to perform gesture processing.

Note

Before using this tool for gesture tagging, you must record a set of Xbox Studio (.xed) files that contain at least depth and skeleton information of several users performing the gesture.

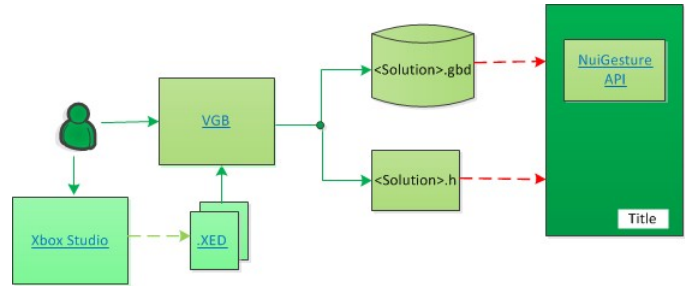
The following are some of the results of using the VGB tool:

- Mitigating the current code-intensive approach.
- Reusing gesture data in different versions of the same title.
- Sharing gesture data among titles.
- Creating a library of gesture data.
- Improving predictability and reliability of gesture data.
- Testing framework for gesture detection.
- Analyzing and visualizing gesture-detection results.
- Managing a large quantity of data in a user friendly IDE.

Using VGB

- The following are the basic steps for using the VGB tool:
1. Create a *solution*. A solution contains a group of gesture projects.
 2. Create one or more gesture *projects*. A gesture project has a gesture tag and a machine learning technology associated with it.
 3. Add a set of clip files to each project. A clip is a source file with example data that machine learning technologies can train on.
 4. Tag the frames in the clip files associated with each gesture.
 5. Build a training database.
- For more information, see [Tutorial: Creating a Gesture Database](#).
The following illustration shows the context for the VGB tool.

Figure 1. VGB Context



The [NuiGesture](#) APIs enable developers to use the data generated by the VGB tool to perform run-time gesture detection.

What's new in VGB Version 1.1

Feature	Description
AdaBoostTrigger Machine Learning	The AdaBoostTrigger Machine Learning is a binary technology type. It uses the Adaptive Boosting (AdaBoost) machine learning algorithm to determine when a player performs a certain gesture.
Color Skeleton	Allows selecting a color for the skeleton.
Command Line	Allows running the Visual Gesture Builder tool from the command line. This is useful when building or analyzing multiple gesture databases using a batch file.
Live Preview	Enables a developer to quickly view the results of a gesture database on the Xbox 360 in real-time, without having to integrate the database into a title.
RFRProgress Machine Learning	The RFRProgress Machine Learning is an analog technology type. It uses the Random Forest Regression (RFR) algorithm to determine the progress of a gesture performed by a player. Compared to PCAProgress machine learning, it generates more accurate results, runs faster and uses less memory at Xbox runtime.
PCAProgress Machine Learning	The PCAProgress Machine Learning is an analog technology type. It uses the Principle Component Analysis (PCA) algorithm to determine the progress of a gesture performed by a player.
Training Optimization	• Allows excluding frames from training clips which can reduce training times and reduce memory usage on the training computer. • Allows reusing data that has not changed. • AdaBoostTrigger Machine Learning is [1.5 .. 2] times faster on the training computer producing same results.
Viewing 2D	Allows rendering of 2D depth and color stream.

- See Also
- [NuiGesture Reference](#)
 - [Natural User Interface Tools](#)
 - [Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Quick Tour

[See Also](#)



The following topics provide an overview of the Visual Gesture Builder (VGB) Integrated Development Environment (IDE).

In This Section

- [Integrated Development Environment](#)
Describes the Visual Gesture Builder (VGB) Integrated Development Environment (IDE), the Menu Bar and the Explorer Pane.
- [Solution](#)
Describes the Solution Tab.
- [Training Project](#)
Describes the Training Project Tab.
- [Analysis Project](#)
Describes the Analysis Project Tab.
- [TimeLine](#)
Describes the TimeLine and Output Panes.
- [Batch Execution](#)
Describes how to run the VGB tool from the command line.

- See Also
- [NuiGesture Reference](#)
 - [Natural User Interface Tools](#)
 - [Xbox Studio](#)

Integrated Development Environment

[See Also](#)



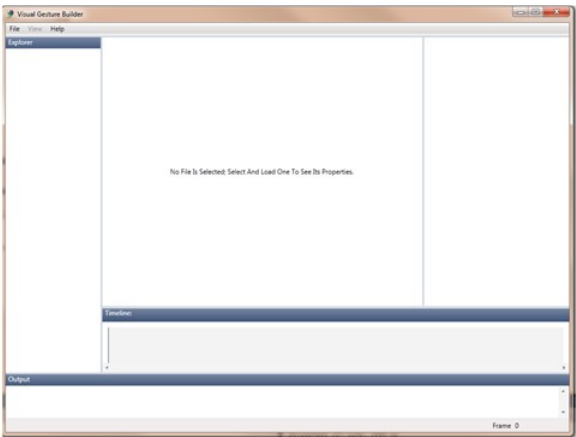
Describes the Visual Gesture Builder (VGB) Integrated Development Environment (IDE), the Menu Bar and the Explorer Pane.

- [Integrated Development Environment](#)
- [Menu Bar](#)
- [Explorer Pane](#)

Integrated Development Environment

The VGB IDE contains an intuitive and easy-to-use interface which has a menu bar and various panes that enable you to produce a gesture detection database.

Figure 1. Integrated Development Environment



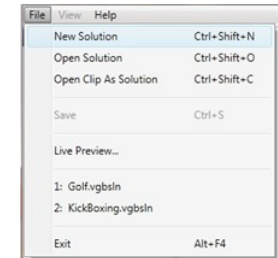
Menu Bar

The menu bar contains items that enable you to perform tasks such as opening, closing files and viewing data.

File Menu

Click the **File** menu item to perform the tasks described next.

Figure 2. Run VGB from this File Menu



Menu Item	Description	Short Cut
New Solution	Open a file where you can create a new solution.	CTRL+Shift+N
Open Solution	Open a previously created and saved solution.	CTRL+Shift+O
Open Clip as Solution	Open a file from another source that you have saved as a clip.	CTRL+Shift+C
Save	Save the selection solution or clip.	CTRL+S
Live Preview . . .	Displays the Live Preview results of a gesture database.	N/A
Exit	Close the VGB tool.	Alt+F4

Live Preview

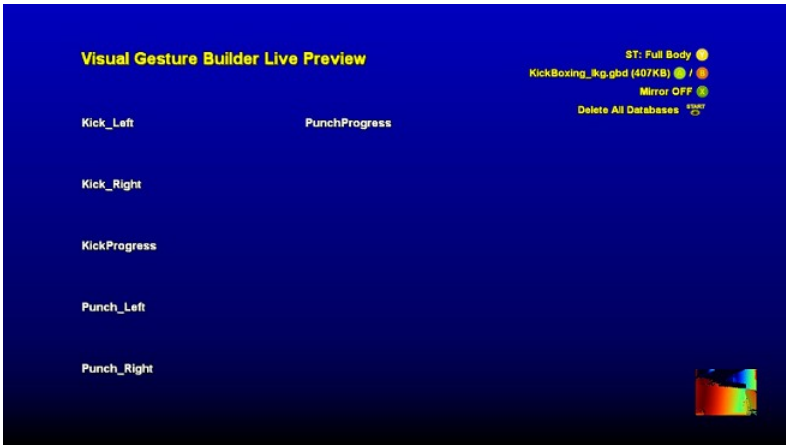
Live Preview enables you to quickly view the results of a gesture database on the Xbox 360 in real-time, without having to integrate the database into a title. The following are some of the advantages.

- You can quickly iterate and experiment with different project settings and compare the results in terms of accuracy, runtime memory and CPU requirements.
- Designers and technical artists can build the gesture database without having a developer integrate the database into a title.

The following are the commands for using **Live Preview**.

1. **Click** the **Live Preview** menu item. An **Open** window dialog is displayed.
2. **Select** the gesture database you want to preview. A script runs and prints information in the output console window, then it activates the viewer on Xbox 360.

Figure 3. Live Preview Viewer on Xbox 360



3. Use the controller **Y** button to switch the **Skeleton Tracking ST** pipeline between **full body ST** and **seated body ST**.
If a gesture was built for **full body ST**, then no detection will be seen when running in **seated body ST**.
4. Use the controller **A** and **B** buttons to cycle forward and backward through all gesture database files that have been deployed.
This can be very useful when wanting to quickly compare different database files, for example the same gestures built with different project settings.
5. Use the controller **X** button to toggle mirror left/right.
6. Use the controller **Start** button to delete all gesture database files on Xbox 360.

Live Preview displays the following information.

- Skeleton tracking mode.
- Currently loaded gesture database file.
- Memory consumed by the gesture database file.
- Name of each gesture.
- Detection graph for each gesture

Note

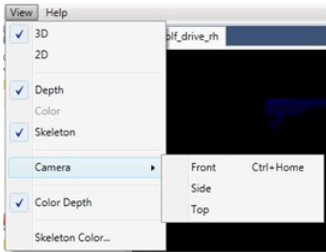
Performance Investigator for Xbox (PIX) can be used to determine CPU cost for gestures. Total CPU cost = Gesture Process End Time – Skeleton Process End Time

View Menu

This menu enables you to view data in 3D and 2D formats. In the 3D view only you can control the direction from which the data is viewed and whether the color and skeleton are rendered. You can choose from the following options.

3D View

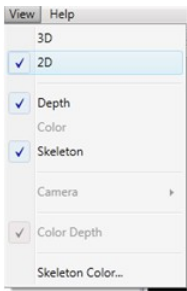
Figure 4. 3D View Menu



Menu Item	Description
3D	Render 3D data.
Depth	Turn on or off depth rendering.
Skeleton	Turn on or off skeleton rendering.
Camera	Select data view direction.
Front	View data from the +Z axis.
Side	View data from the +X axis.
Top	View data from the +Y axis.
Color Depth	Turn on or off color depth rendering.
Skeleton Color...	Select the color to use when rendering the skeleton.

2D View

Figure 5. 2D View Menu

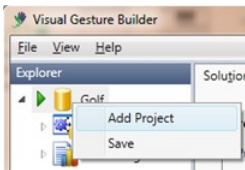


Menu Item	Description
2D	Render 2D data.
Depth	Turn on or off depth rendering.
Skeleton	Turn on or off skeleton rendering.
Skeleton Color...	Select the color to use when rendering the skeleton.

Explorer Pane

The Explorer Pane enables you to handle projects and clips. If you right-click the solution, in the context menu you can add a project or save the solution.

Figure 6. Explorer Pane



You can choose different types of projects based on the machine learning technology. You can select [AdaBoostTrigger Machine Learning](#) or [PCAPProgress Machine Learning](#) technology. For each project, you can create a training version and an analysis version. For more information, see related section later in this article. When you create a project, the VGB tool generates XML files for each clip and adds them to the project. These files are used to store tagging information while the clip files are unmodified.

See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Solution

See Also

--

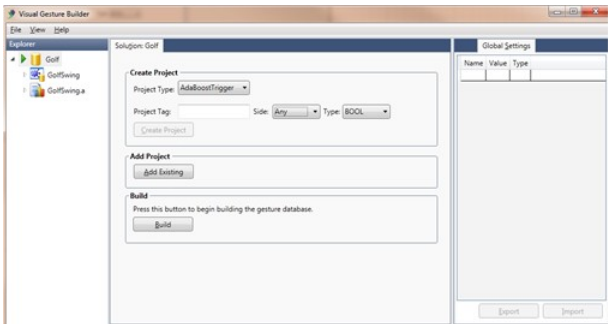
Describes the Solution Tab.

- [Solution](#)
- [Global Settings](#)

Solution

The Solution Tab displays the user interface for controlling the selected solution.

Figure 1. Solution



Solution User Interface

Control	Description
Create Project	
Project Type	Select the machine learning technology for generating gesture data.
Project Tag	Specify the intent of the gesture data. For example Punch, Swing and so on.
Side	Select left or right position. Tells the project on which side of a person the gesture is performed.
Type	Select type associated with the tag. The type to choose depends on the selected machine technology.

Add Project

Add Existing	Select the project to add to the solution.
Build	
Build	Create the database file (.gbd) and related header file.

Global Settings

The global settings enable you to override the specific settings for all the projects contained in the solution. The case sensitive settings must match the specific names and types used in the project. For more information, see [Machine Learning Technologies](#).

 **Note**
The output pane displays a message when the replacement is complete.

Parameter Properties	Description
Name	Any allowed name for the setting parameters. The name to choose depends on the selected machine technology.
Value	Assign the value to the setting parameter.
Type	Select the type associated with the setting parameter. The type to choose depends on the selected machine technology.

 See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Training Project

[See Also](#)

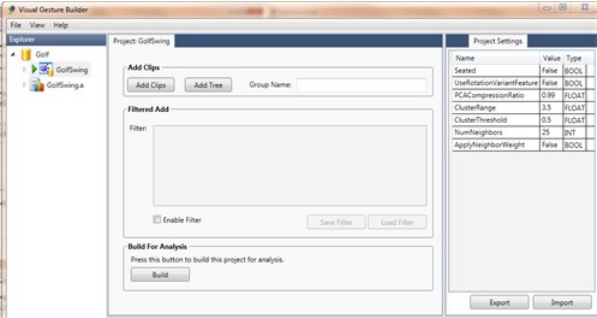
Describes the Training Project Tab.

- [Training Project Controls](#)
- [Training Project Settings](#)

Training Project Controls

For each project, you can create a training version and an analysis version. Each project has several different parameters for handling settings such as filtering and thresholds.

Figure 1. Training Project



The following are the controls you can use in the training project.

Control	Description
Add Clips	
Add Clips	Add one or more clips to the project.
Add Tree	Select the folder that contains the clips.
Group Name	Enter the group name for the clips. This gives you the ability to retrieve clips organized in separate folders.
Filtered Add	
Filter	Specify the filter parameters. The following example finds all clips with tag name "MyTag". If clips are found, they are added to the project. <pre>from c in doc.Element("ClipFile").Element("ClipAttributes").Elements("Items") where (string) c.Element("Items").Attribute("Name") == "MyTag" select c</pre>
 Note	This enables you to recursively add to the project all the clips contained in a specified folder. Optionally, you can enable a LINK filter which adds only the files that have properties you specify.
Enable Filter	Enable the filter.
Save Filter	Save the filter.
Load Filter	Load the filter.
Build for Analysis	
Build	Build the database file (.gba).
 Note	

This enables you to build a single project database (.gba). This database can only be used for analysis.

Training Project Settings

The training project settings enable you to modify project property values. These values affect training results and performance. For more information, see [Machine Learning Technologies](#).

Name	Description
Parameters	
Name	Any allowed name for the setting parameters. The name to choose depends on the selected machine technology.
Value	Assign the value to the setting parameter.
Type	Select the type associated with the setting parameter. The type to choose depends on the selected machine technology.
Controls	
Export	Export the project property values to a file. Use this control as follows: - Click the Export button. The Save As dialog window is displayed. - In the File name: box enter the name of the file where to save the property values. If the project name is <i>GolfSwing</i>, the default file name is <i>GolfSwing.fid</i>.
Import	Import the project property values from a file you select, such as <i>GolfSwing.fid</i> . Use this control as follows: - Click the Import button. The Open dialog window is displayed. - In the File name: box enter the name of the file that contains the property values to import.

See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Analysis Project

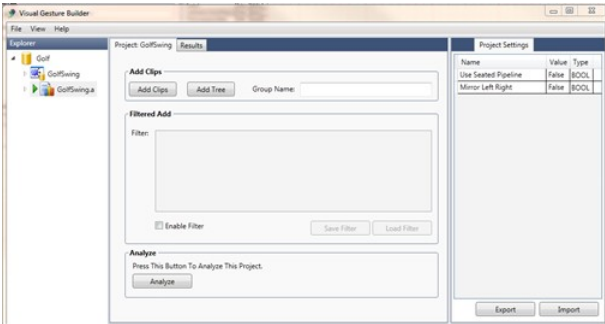
[See Also](#)

Describes the Analysis Project Tab.

- [Analysis Project Controls](#)
- [Analysis Project Settings](#)

Analysis Project Controls

Figure 1. Analysis Project



The following are the controls you can use in the analysis project.

Control	Description
Analysis Project	
Add Clips	
Add Clips	Add one or more clips to the project.
Add Tree	Select the folder that contains the clips.
Group Name	Enter the group name for the clips. This gives you the ability to retrieve clips organized in separate folders.
Filtered Add	
Filter	Specify the filter parameters.
Enable Filter	Enable the filter.
Save Filter	Save the filter.
Load Filter	Load the filter.
Analyze	
Analyze	Analyze how well the results compare to the tags you created in the project. You can see the results of the analysis clicking on the Results tag. The Analyze control enables you to use a database (.gba or .gbd) against all the clips in the project.
Note	
You can also use a data file (.adf) exported from previous analysis. Refer to the Batch Execution topic.	

By using a data file, you do not have to analyze each clip against the database that could be time consuming.

Analysis Results	
Results	Shows the analysis results. The results contain the number of frames compared, worst errors, average RMS, false positives, and false negatives.
Clear	Clears all the existing analysis results.
Export	Exports all the existing analysis results to .adf files, saved in a folder of your choice.

Analysis Project Settings

The analysis project settings enable you to modify the analysis settings such as mirroring, and to change between seated and full-body tracking. For more information, see [Machine Learning Technologies](#).

Name	Description
Parameters	
Name	Any allowed name for the setting parameters. The name to choose depends on the selected machine technology.
Value	Assign the value to the setting parameter.
Type	Select the type associated with the setting parameter. The type to choose depends on the selected machine technology.
Controls	
Export	Export the project property values to a file. Use this control as follows: - Click the Export button. The Save As dialog window is displayed. - In the File name: box enter the name of the file where you want to save the property values. If the project name is <i>GolfSwing.a</i>, the default file name is <i>GolfSwing.a.fid</i>.
Import	Import the project property values from a file you select, such as <i>GolfSwing.a.fid</i> . Use this control as follows: - Click the Import button. The Open dialog window is displayed. - In the File name: box enter the name of the file that contains the property values to import.

See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Timeline

[See Also](#)

Describes the TimeLine and Output Panes.

- [Timeline Pane](#)
- [Output Pane](#)

Timeline Pane

Enables you to perform time tagging of the clips.

Note

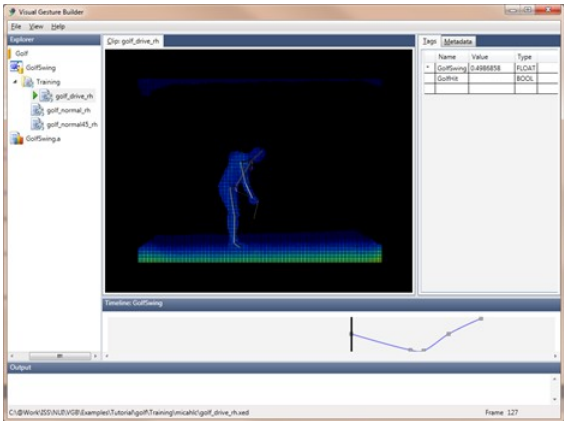
The **Timeline** pane title changes based on the tag selected by the user.

Tagging Clips

In order to train on clips, they must be tagged in some manner. Different machine language algorithms accept different types of tagging but, for a specific frame, some metadata representing the intended gesture is encoded. The following are the allowed tagging types.

- In [AdaBoostTrigger Machine Learning](#) projects, mark a specific part of the clip as a gesture.
- In [PCAProgress Machine Learning](#) projects, identify different parts of the gesture sequence with a value.
- In [RFRProgress Machine Learning](#) projects, identify different parts of the gesture sequence with a value.

Figure 1. Tagging Clips



The following is an example of steps you can perform to generate training data.

1. Click on one of the files. You are presented with the file view.
2. Scroll to some frame.
3. Add new metadata.
4. Repeat the previous steps as many times as needed.

Output Pane

This pane displays status and error messages. You can use the results of application operations that are displayed here for debugging purposes.

See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Batch Execution

[See Also](#)



You can run the VGB tool from the command line. Use this when you want to build multiple gesture databases using a batch file.

```
VisualGestureBuilder [ /build buildFile database] [ /analysis analysisFile.a.vgbproj database outputResultFile.adf] [ /?]
```

Option	Description
/build buildFile database	Builds the training solution specified by the buildFile argument and saves the database in the file specified by the database argument. The command creates the log file buildlog.log that contains the build results.
/analysis analysisFile.a.vgbproj database outputResultFile.adf	Analyzes the the project specified by the database argument (.gba or .gbd). It outputs the results in the outputResultFile.adf file. The command creates the log file analyzezlog.log that contains the analysis results. You can open the outputResultFile.adf using the Analyze button in the Analysis project.
/?	Displays usage information.

See Also

[Quick Tour](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Tutorial: Creating a Gesture Database

[See Also](#)



This tutorial shows you how to create a golf gesture database and use it in your title. An important feature of the Visual Gesture Builder (VGB) tool is the machine learning technology. This technology enables you to *train* clips by tagging them with gesture metadata. VGB uses this metadata to create the database. For more information, see [Machine Learning Technologies](#).

Note

You can download a complete example at this location: [Simple Gesture Detection Content](#).

- [Prerequisites](#)
- [Creating a Golf Solution](#)
- [Adding Clip Files to the Projects](#)
- [Adding Tags to the Clip Files](#)
- [Building the Gesture Database](#)
- [Analyzing the Gesture Database](#)
- [Using the Gesture Database](#)
- [Migrating to New Xbox 360 SDK Version](#)
- [CPU Time and Memory Requirements](#)

Prerequisites

1. Platform. For more information, see [Visual Gesture Builder](#).
2. Clip files. You can generate the clips using [Xbox Studio](#).

Creating a Golf Solution

This section describes how to create a solution and related projects. A solution may have several projects. Each project contains several clips that you must tag to generate gesture data. There are two kinds of projects:

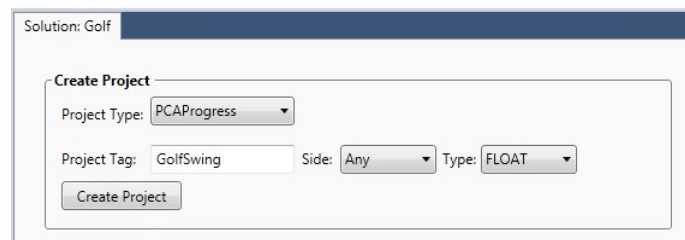
- Training Project. You use this project to tag the clips by locating the frames that hold meaningful gestures. You will use the tags to create the gesture database.
 - Analysis Project. You use this project to perform tagging of different clips than the ones used during the training phase. Later you will use these analysis tags to verify the accuracy of the gesture database.
1. In the desktop start menu, click **All Programs**.
 2. In the selection list, click **Microsoft Xbox 360 SDK**.
 3. In the selection list, click **Natural Input**.
 4. Finally, click **Visual Gesture Builder**. This starts the VGB tool.
 5. In the **File** menu click **New Solution**. The **Save As** dialog window is displayed.
 6. In the **File name** box enter *Golf*.
 7. Click **Save**. The solution window is displayed.
 8. From the **Project Type** drop-down list select **PCAPProgress**.

Note

The **Project Type** drop-down list provides you with two choices: *AdaBoostTrigger* and *PCAPProgress*. For more information, see [Machine Learning Technologies](#).

9. In the **Project Tag** input box enter GolfSwing.
10. In the combo box **Side** select **Any**.
11. Set **Type** to **FLOAT**.

Figure 1. Creating a Solution

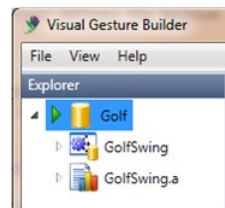


Note

It is good practice to use an unambiguous name for the **Project Tag** that is related to the gesture intent. This name is also used to generate the project name associated with the tag.

12. Click **Create Project**. The **Save As** dialog window is displayed.
 13. In the **File name** input box, leave the name as GolfSwing.
 14. Click **Save**. The GolfSwing and GolfSwing.a projects are created. They are shown in the **Explorer** pane.
- You use the GolfSwing project for training (tagging) the clips and GolfSwing.a for testing the gesture data.

Figure 2. Golf Solution



Adding Clip Files to the Projects

This section describes the clip-tagging process that you perform in the **Timeline** pane. This process represents the core mechanism to define and tune the temporal tags for the creation of the gesture database. The process comprises two main phases.

- Training Phase. During this phase you tag the clips by locating the frames that hold meaningful gestures. You use trigger tagging with [AdaBoostTrigger Machine Learning](#) or progress tagging with [PCAPProgress Machine Learning](#).
- Analysis Phase. During this phase you perform tagging of clips that are different than the ones used during the training phase. Later you will use these analysis tags to verify the accuracy of the gesture database.

Note

- The tagging information is stored in an auxiliary file named after the clip file using the extension **vgbclip**.
- You must create the clip files to add to the project. For more information, see [Xbox Studio](#).

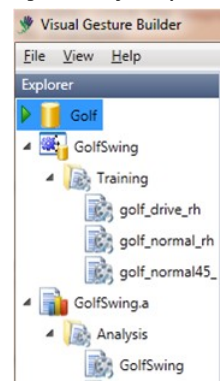
1. In the **Explorer** pane, click the GolfSwing project. The project window is displayed.
2. In the **Project GolfSwing** pane tab, in the **Group Name** box, enter the name of the folder where to store the clips for better organization and management.
3. Click **Add**. The **Open** dialog window is displayed.
4. Select one or more clips and click **Open**. The clips are added to the project and are shown in the **Explorer** pane.

Note

Use the **Add Tree** button if you want to add a folder that contains clips.

5. In the **Explorer** pane, click the GolfSwing.a project. The project window is displayed.
6. In the **Project: GolfSwing** pane tab, in the **Group Name** box enter the name of the folder where to store the clips for better organization and management.
7. Click **Add**. The **Open** dialog window is displayed.
8. Select one or more clips and click **Open**. The clips are added to the project and are shown in the **Explorer** pane.

Figure 3. Project Clip Files

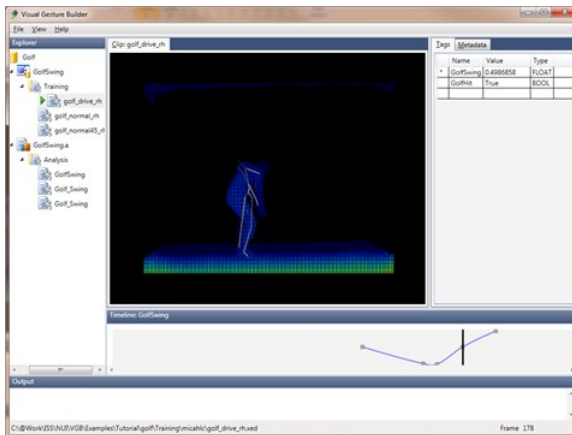


Adding Tags to the Clip Files

1. In the **Explorer** pane, select the clip you want to tag in the GolfSwing project. The project window is displayed.
2. Set the focus in the **Timeline** pane to identify the frames that contain meaningful gestures. You do this by using the skeleton image and moving the timeline cursor.
3. Scroll the clip using the cursor timeline (hold and drag) and seek for a meaningful gesture to tag. Select the frame and, in the **Tags** grid, set the value for the tag. For more information, see [Keyboard Shortcuts](#).

4. In the **Tags** grid for the GolfSwing tag, enter a value valid for your model. For example, if your model allows values in the range [0..1], assign a **FLOAT** value such as 0.1952061.
5. Repeat the previous steps as many times as needed to tag more clips or set of frames.
6. Once finished, in the **Explorer** pane right-click the clip and in the drop-down menu click **Save**.
7. Repeat the same steps for all the other clips.

Figure 4. Tagging Clip Files



Building the Gesture Database

1. In the **Explorer** pane click the *Golf* solution. The **Solution:Golf** tab pane is displayed.
2. Click the **Build** button. The **Save As** dialog window is displayed.
3. Select the folder location for saving the database. You can use the *Golf* solution folder.
4. In the **File name** input box, leave the default name *Golf.gbd*.
5. Click **Save**. The build process produces the database file *Golf.gbd* and the header file *Golf.h*. In your title, use the [NuiGesture Reference](#) API to load the gesture database. Also, include the header file to retrieve gesture information from the database.

Important

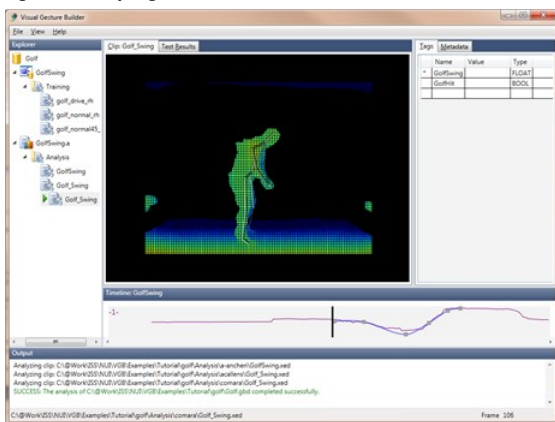
- If you select an existing database when building a solution or project, VGB checks to find out if it can reuse existing build results to accelerate the process.
- If you want to rebuild a solution or project without using existing results, remember to rename or delete the existing database.

Analyzing the Gesture Database

The following steps show how to verify that the gesture database has the accuracy that you require.

1. In the **Explorer** pane click the *GolfSwing.a* project. The **Project:GolfSwing** tab pane is displayed.
2. Click the **Analyze** button. The **Open** dialog window is displayed.
3. Navigate to the folder that contains the *Golf.gbd* database.
4. Select the *Golf.gbd* database and click the **Open** button. After the analysis is completed, the **Test Results** tab pane is displayed. The test results show the accuracy of the data produced for each clip.
5. In the **Explorer** pane, click any clip in the *GolfSwing.a* project. You can verify the accuracy in the **Timeline** pane where the data from the gesture database is plotted over the original data.

Figure 5. Analyzing the Gesture Database



Using the Gesture Database

The following steps show how to use the gesture database in your title.

1. Load the database in memory and initialize the gesture detector. At the time you initialize your gesture detection, you must make a call similar to the one shown next.

```
HRESULT result = NuiGestureInitialize("C:\\MyMedia\\Gestures\\Golf.gbd", Golf::DWORD DIGEST, 0, NULL);
```

The parameter *Golf::DIGEST* is defined in the auto generated *Golf.h* file. For more information, see [NuiGestureInitialize](#).

2. Enable the gestures that the title must handle. The following is an example of how to do it.

```
HRESULT result = NuiGestureEnableGesture(Golf::GESTURE_GOLFSWING, TRUE);
```

The parameter *Golf::GESTURE_GOLFSWING* is defined in the auto generated *Golf.h* file. For more information, see [NuiGestureEnableGesture](#).

3. Initialize the gesture frame structure. The following is an example of how to do it.

```
Golf::FRAME m_GolfGestureFrame;
NUI_GESTURE_FRAME m_GestureFrame;
Golf::INITIALIZE_NUI_GESTURE_FRAME(m_GolfGestureFrame, m_GestureFrame);
```

The structure `Golf::FRAME` and the function `Golf::INITIALIZE_NUI_GESTURE_FRAME` are defined in the auto generated `Golf.h` file. `NUI_GESTURE_FRAME` is defined in `NuiApi.h`. For more information, see [NUI_GESTURE_FRAME](#).

4. Use the gesture data in real-time. The following is an example of how to get the next frame from the gesture detector stream.

```
HRESULT result = NuiGestureGetNextFrame( 0, m_GestureFrame);
```

For more information, see [NuiGestureGetNextFrame](#).

Migrating to New Xbox 360 SDK Version

You can convert the .XED files to a new version of the Xbox 360 SDK by using the [Xed Skeleton Updater](#) tool.

CPU Time and Memory Requirements

Requirement	Description
CPU Time	NuiGesture uses a variable amount of CPU time depending on how many gestures must be processed. A set of PIX counters measures the CPU time used by NuiGesture . For more information, see the NUI section in PIX System Monitor: Predefined Counters and Events Reference and the PIX Counters and Events section in Reducing Latency in Kinect Titles .
Memory Size	The memory size required can be roughly estimated from the database size. You can see the exact memory size using Live Preview , which shows the memory needed for the loaded gesture database.

See Also

- [NuiGesture Reference](#)
- [Natural User Interface Tools](#)
- [Xbox Studio](#)
- [PIX System Monitor: Predefined Counters and Events Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Machine Learning Technologies

See Also



The Visual Gesture Builder (VGB) tool generates gesture data by using a set of machine learning technologies. You determine the kind of solution to build by selecting which technology to use.

In This Section

- [AdaBoostTrigger Machine Learning](#)
Detects when a player is performing a gesture.
- [PCAProgress Machine Learning](#)
Determines progress while a player is performing a gesture.
- [RFRProgress Machine Learning](#)
Determines progress while a player is performing a gesture.

See Also

- [NuiGesture Reference](#)
- [Natural User Interface Tools](#)
- [Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

AdaBoostTrigger Machine Learning

See Also



The AdaBoostTrigger is a *binary* technology type. It uses the Adaptive Boosting (AdaBoost) machine learning algorithm to determine when a player performs a certain gesture. During training time it accepts input tags, **boolean** values, which mark the occurrence of an event, such as a hit. This marking or tagging is used to evaluate whether or not a gesture has happened and determines the confidence value of the event.

Note

You can override the AdaBoostTrigger project settings in the solution global settings. To see how this is done, refer to the sample at this location: [Simple Gesture Detection Content](#).

- [Input Parameters](#)
- [Output Data Structure](#)

Input Parameters

The following table describes the input parameters that you can use when tagging a gesture. You enter these parameters in the **Project Settings** grid.

Parameter Name	Description
Accuracy Level	A floating point value in the range [0..1]. This value controls how accurate your results are, but also affects the training time. The higher the accuracy, the longer the training time. As a guideline you can use the following values: - Retail Build – 0.98. - Release Build – 0.95. - Quick Experiment – 0.8.
Number of Weak Classifiers at Runtime	The algorithm can potentially generate tens of thousands of weak classifiers. Using all of them will increase accuracy, but at the higher CPU cost on the Xbox 360. To use all the classifiers generated, use a value of 0. The CPU cost for 1000 weak classifiers is only 25 microseconds and the results have more than adequate accuracy.
Filter Results	The results of the algorithm are based on a frame, not on a gesture. A filter needs to be applied on the raw per frame results. The AdaBoostTrigger

provides a simple low latency filter, but you have the option of disabling filtering and applying your own filter at runtime on the Xbox 360. The filter used is a simple sliding window of N frames, summing up the results and comparing it against a threshold value. The number of frames can be seen as a frequency and the threshold can be seen as amplitude.

Auto Find Best Filtering Parameters	When filtering is switched on, you have the option to let the trainer automatically find best filtering parameters which will minimize the rate of false positives and false negatives
Weight Of False Positives During Auto Find	A value in the range of [0..1] which is used when automatically finding the best filtering parameters. If it is more important to reduce false positives, use a higher value. If it is more important to reduce false negatives, use a lower value.
Manual Filter Parameters: Number of Frames To Filter	A value in the range of [1..10]. You can choose the number of frames over which to filter if you do not choose to find the best filtering parameters automatically.
Manual Filter Parameter: Threshold	A value in the range of [0..1]. Lower values could increase true positives, at the risk of increasing false positives. Higher values could decrease false positives, at the risk of decreasing true positives.
Duplicate And Mirror Data During Training	Skeleton data can be duplicated and mirrored in order to have a larger set of training data.
% CPU For Training	A value in the range of [0..100] which indicates the % of CPU resources the trainer should use for training.
Seated	Set to true if you are training on the seated skeletal pipeline; otherwise, it is set to false .

Output Data Structure

The following data structure is contained in the auto-generated header file and is used by the NuiGesture APIs.

```
typedef struct _AdaBoostTrigger {
    BOOL Value; // TRUE if the gesture was detected
    FLOAT Confidence; // A confidence value in the range [0..1]
    BOOL FirstFrameDetected; // TRUE if this is the first frame at which we detected the gesture
    BOOL Valid; // TRUE if the data is valid
} AdaBoostTrigger;
```

Remarks

In training, if you only tag positive frames, AdaBoostTrigger uses all frames by default. The frames not tagged are used as negative frames. This generates a big training set, if the clips are very long.

In training, if you intend to use only some frame ranges, instead of all the frames of a clip, you can explicitly tag negative frames. During training, the tool detects these explicitly tagged negative frames and ignores the non-tagged frames.

The tool UI is designed to help you convert from implicit negative to explicit negative tagging.

When you select a range of frames and set its value to **FALSE**, if there are non-tagged frames (gap) in the range, only those gaps are set to **FALSE**. All existing tagged frames are not affected.

However, the range works like a bundle set to **FALSE** in its entirety, if there is no gap.

See Also

- [NuiGesture Reference](#)
- [Natural User Interface Tools](#)
- [Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

PCAPProgress Machine Learning

[See Also](#)

Choosing Which Tool to Use

Generally, you should use [RFRProgress Machine Learning](#) rather than PCAPProgress. Using RFRProgress instead of PCAPProgress provides the following advantages:

- The database size is independent of the number of input examples; the database size doesn't increase as you add more examples.
- Higher accuracy.
- Runtime cost is less, for both CPU and memory.

About PCAPProgress

The PCAPProgress is an *analog* technology type. It uses the Principle Component Analysis (PCA) algorithm to determine the progress of a gesture performed by a player. Analog PCAPProgress uses tagging models that are represented by analog signals that can occur during a gesture. Examples of these models could be an abstract concept such as the "progress" of a gesture or a quantifiable value such as the angle of an arm in a baseball pitch. Valid ranges for the PCAPProgress tags are based on the model that the analog signal represents.

The input is an analog signal that models an action as progress during a gesture. For example, the position of the arm in a golf swing.

The PCAPProgress is a *context*-based detector, meaning that the progress detection is only valid when a player is performing a gesture. The progress detection can be enabled as follows:

1. When the title is aware that the player is performing a gesture, it enables the detection and uses it to aid in rendering a simulation.
2. Progress detection is enabled when an AdaBoost trigger is active.

Note

You can override the PCAPProgress project settings in the solution's global settings. To see how this is done, refer to the sample at this location: [Simple Gesture Detection Content](#).

Input Parameters

The following table describes the input parameters that you can use when tagging a gesture. You enter these parameters in the **Project Settings** grid.

Parameter Name	Description
Seated	Set to true if you are training on the seated skeletal pipeline; otherwise, it is set to false .
Use Rotation Variant Feature	When set to true , the gesture detection will take into account the user's global rotation. In this case the detection produces a stronger signal that is less robust to orientation change. When set to false , gesture detection will try to ignore global rotation as much as possible. For example golf swing with the user head on and golf swing with the user at 45 degrees returns a similar signal.
PCA Compression	The higher the compression ratio, the less compressed the training database is and the bigger are the memory and disk space required. But the signal is more

Ratio	accurate. The compression ratio valid ranges are [0..1]. Setting the ratio to 1 disables the PCA compression.
Cluster Range	The lower this value, the less change can occur from frame to frame. This could produce better overall results, however, there will be more misdetections.
Number of Neighbors	The higher this number is, the more samples will be used. This results in smoother curves with more lag. This number should be small relative to the number of tagged frames.
Enable Uniform Clamping	Set this parameter to true , to produce databases of fixed size by clamping the total number of frames to train on. Use this mode to create small training databases with similar accuracy as the full database. The following are the input parameters used for clamping:
Uniform Clamping Number Of Buckets	When the uniform clamping is enabled, it defines the number of buckets. The higher the number, the more samples are guaranteed to be used in training.
Uniform Clamping Samples Per Bucket	When the uniform clamping is enabled, it defines the number of samples to include in every bucket.

Output Data Structure

The auto-generated header file contains the following data structure, which is used by the NuiGesture APIs.

```
typedef struct _PCAPProgress {
    FLOAT Value;           // Value depends on user's tagging range.
    BOOL Valid;            // TRUE if the data is valid.
} PCAPProgress;
```

See Also

- NuiGesture Reference
- Natural User Interface Tools
- Xbox Studio

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

RFRProgress Machine Learning

See Also



The RFRProgress is an *analog* technology type. It uses the Random Forest Regression (RFR) algorithm to determine the progress of a gesture performed by a player. Also, it uses tagging models that are represented by analog signals that occur during a gesture.

Examples of these models could be an abstract concept such as the "progress" of a gesture or a quantifiable value such as the angle of an arm in a baseball pitch. Valid ranges for the RFRProgress tags are based on the model that the analog signal represents.

The input is an analog signal that models an action as progress during a gesture. For example, the position of the arm in a golf swing.

The RFRProgress is a *context*-based detector, meaning that the progress detection is only valid when a player is performing a gesture. The progress detection can be enabled as follows:

- When the title is aware that the player is performing a gesture, it enables the detection and uses it to aid in rendering a simulation.
- Progress detection is enabled when an AdaBoost trigger is active.

Note

You can override the RFRProgress project settings in the solution's global settings.

Input Parameters

The following table describes the input parameters that you can use when tagging a gesture. You enter these parameters in the **Project Settings** grid.

Parameter Name	Description
Seated	Set to true if you are training on the seated skeletal pipeline; otherwise, it is set to false .
Use Rotation Variant Feature	When set to true , the gesture detection will take into account the user's global rotation. In this case the detection produces a stronger signal that is less robust to orientation change. When set to false , gesture detection will try to ignore global rotation as much as possible. For example golf swing with the user head on and golf swing with the user at 45 degrees returns a similar signal.
Number of Trees	This parameter affects the accuracy, memory and disk space. Larger value gives better results at the cost of more memory and disk space, and also increases runtime CPU use for searching through these trees.
Maximum Tree Depth	This parameter controls the maximal complexity of a decision tree. Larger value allows tree to grow deeper for complex scenarios, at the cost of more memory space and CPU use.
Cluster Threshold	The lower this value, the less change can occur from frame to frame. This could produce better overall results, however, there will be more misdetections. Note A smaller value means more localized search, which could be more accuracy if search is performed in the correct neighborhood. But it could also cause "stuck" in a neighborhood when no good matching could be found.
Weight Factor	The weight of the output from previous frame that will affect the current frame's output. 0 means no affect at all.

Output Data Structure

The auto-generated header file contains the following data structure, which is used by the NuiGesture APIs.

```
typedef struct _RFRProgress {
    FLOAT Value;           // Value depends on user's tagging range.
    BOOL Valid;            // TRUE if the data is valid
} RFRProgress;
```

Remarks

When compared to [PCAPProgress Machine Learning](#), [RFRProgress Machine Learning](#) provides better accuracy, uses less memory and CPU time.

See Also

- NuiGesture Reference
- Natural User Interface Tools
- Xbox Studio

Keyboard Shortcuts

[See Also](#)

The Visual Gesture Builder (VGB) integrated development environment (IDE) includes several keyboard shortcuts, described as follows.

Command	Shortcut	Description
File Menu Shortcuts		
New	CTRL + SHIFT + N	Creates a solution.
Open	CTRL + SHIFT + O	Opens a solution.
Open	CTRL + SHIFT + C	Opens a clip file as a solution.
Save	CTRL + S	Saves the selected solution or clip.
Exit	ALT + F4	Exit and close the tool.
Timeline Shortcuts		
Edit	ENTER	Sets the default maximum value. This value is true for BOOL , 1 for INT and 1.0 for FLOAT .
Edit	SPACE	Sets the default minimum value. This value is false for BOOL , 0 for INT and 0.0 for FLOAT .
Edit	DEL	Deletes the selected range or a single frame.
Edit	CTRL + DEL	Deletes all key frames in the selected range or a single key frame.
Edit	Insert	Reinserts the section of the curve that is missing.
Navigate	Home page	Moves the cursor to the first frame in the clip.
Navigate	End	Moves the cursor to the last frame in the clip.
Navigate	CTRL + LEFT	Moves the cursor to the previous range of tags or key frame.
Navigate	CTRL + RIGHT	Moves the cursor to the next end point or key frame.
Navigate	RIGHT	Moves the timeline cursor to the next frame.
Navigate	LEFT	Moves the timeline cursor to the previous frame.
Select	Page Up	Selects the previous attribute in the Tags grid as the active attribute.
Select	Page Down	Selects the next attribute in the Tags grid as the active attribute.
Select	SHIFT + LEFT	Selects time range.
Select	SHIFT + RIGHT	Selects time range.
3D Window Shortcuts		
Zoom	CTRL -	Zoom out.
Zoom	CTRL +	Zoom in.
Zoom	Mouse wheel	Zoom in and out.
Zoom	CTRL + Home	Restores the default view (render view).

[+ See Also](#)
[NuiGesture Reference](#)
[Natural User Interface Tools](#)
[Xbox Studio](#)

Xed Skeleton Updater

[See Also Example](#)

Xed Skeleton Updater (XedSkeletonUpdater), a command-line tool, converts XED files for use in [Visual Gesture Builder](#). To run Xed Skeleton Updater, in the **Xbox 360 Command Prompt** window, enter **XedSkeletonUpdaterClient** with the options described in this topic.

```
XedSkeletonUpdaterClient [ /x devkit] [ /seated] [ /start startframe#] [ /end endframe#] input file output file [ /?]
```

Option	Description
/x devkit	Specifies the name or IP address of the target console. Setting this value changes the default console for the system. All other commands will use this new value by default.
/seated	Generates seated skeleton tracking.
/start startframe#	Specifies the start frame in the input file.
/end endframe#	Specifies the end frame in the input file.
input file	The input file that contains the depth stream.
output file	The output file that contains the depth and skeleton stream.
/?	Displays usage information.

Remarks

When you migrate to a new version of the SDK, you can use Xed Skeleton Updater to convert XED files for use in Visual Gesture Builder. You can convert one file at the time. The Xed Skeleton Updater runs the depth stream from the input XED file to generate the skeleton stream. Then it saves the skeleton and depth stream in the output XED file.

Example

Converting *depth.xed* into *depthandskeleton.xed* with full body skeleton:

```
XedSkeletonUpdaterClient depth.xed depthandskeleton.xed
```

Converting *depth.xed* into *depthandskeleton.xed* with seated skeleton:

```
XedSkeletonUpdaterClient /seated depth.xed depthandskeleton.xed
```

Converting frames [100, 200] from *depth.xed* into *depthandskeleton.xed* with full body skeleton:

```
XedSkeletonUpdaterClient /start 100 /end 200 depth.xed depthandskeleton.xed
```

See Also

[NuiGesture Reference](#)
[Simple Seated Skeleton Tracking Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Xbox Studio

[See Also](#)



Xbox Studio is an application that retrieves and displays the depth and color streams provided by the Kinect for Xbox 360 sensor array. It tracks players' skeletons and displays stick figure visualizations of the skeletal tracking data that it receives. Xbox Studio can record this data for injection into a console title, or it can play it back for local viewing. You can also use Xbox Studio to graph the motion of skeleton joints over time.

You can use the APIs in [XedFile.h](#) to manipulate the Xbox event data (XED) files used by Xbox Studio. You can use these APIs to create new XED files from existing XED files. Also, you can use the APIs in [XStudio.h](#) to record and play back XED files programmatically.

Requirements

Operating System

To use all of the features of Xbox Studio, you must be running Windows 7 or Windows Vista.

Easy Transfer Cable

For best performance, use an easy transfer cable to transfer Xbox Studio data between your computer and development console. For more information about supported cables, see [Connecting to a Development Console \[Xbox Studio\]](#). You can transfer Xbox Studio data over TCP/IP, but performance will be limited by your network bandwidth.

In This Section

[Using Xbox Studio](#)

Performing routine tasks in Xbox Studio.

[Xbox Studio Application Windows](#)

Application windows in Xbox Studio provide visualization of color, depth, and skeleton data.

[Identity Data \[Xbox Studio\]](#)

The identity data stream records or plays back events that correlate skeletons and players.

[Title Data \[Xbox Studio\]](#)

The title data stream allows you to record and play back opaque title-defined data. You can record title data and play it back along with captured color, depth, and skeletal data.

[Xbox Studio Command-Line Utility](#)

Upgrades Xbox Studio Xbox event data (XED) files.

See Also

[XStudio Reference](#)
[NUI Samples: Xbox Studio Samples](#)
[Natural User Interface Tools](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Using Xbox Studio

[See Also](#)



This section contains information about performing routine tasks in Xbox Studio.

In This Section

[Connecting to a Development Console \[Xbox Studio\]](#)

Connecting Xbox Studio to a development console.

[Viewing Data \[Xbox Studio\]](#)

Viewing live Kinect data in Xbox Studio.

[Recording Data \[Xbox Studio\]](#)

Recording Kinect data in Xbox Studio.

[Playing Data \[Xbox Studio\]](#)

Playing recorded Kinect data in Xbox Studio.

[Playing and Recording Data Simultaneously \[Xbox Studio\]](#)

Simultaneously playing and recording Kinect data in Xbox Studio.

[Saving Data \[Xbox Studio\]](#)

Saving Kinect data in Xbox Studio.

[Loading Data \[Xbox Studio\]](#)

Loading Kinect data into Xbox Studio.

[Graphing Data \[Xbox Studio\]](#)

Graphing skeletal data in Xbox Studio.

See Also

[Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Connecting to a Development Console [Xbox Studio]

[See Also](#)



For best performance, transfer Xbox Studio data over a Windows-logo-certified easy transfer cable, such as those manufactured by Belkin or Bafo, using a Prolific chipset. If you do not have an easy transfer cable, Xbox Studio will try to connect to your development console over TCP/IP. However, performance will be limited by your network bandwidth.

The connection type is determined when you launch Xbox Studio. If an easy transfer cable is available, it will be used. Otherwise, Xbox Studio will try to use TCP/IP. If you connect or disconnect a cable after launching Xbox Studio, you must exit and relaunch the tool to use a different connection type.

Important

For Xbox Studio to receive data from the sensor array, your development console must be running an application, such as [Simple Skeleton Tracking](#), that initializes the sensor array. Xbox Studio will always receive the color and depth data streams. The skeletal data stream will be received only if the console application has called the [NuiSkeletonTrackingEnable](#) function. The application must then call either the [IDirect3DDevice9::Swap](#) or [IDirect3DDevice9::Present](#) method for every frame.

Sections

- [Connecting Using an Easy Transfer Cable](#)
- [Connecting Over TCP/IP](#)
- [Sharing a Development Console Over TCP/IP](#)
- [Improving Performance](#)

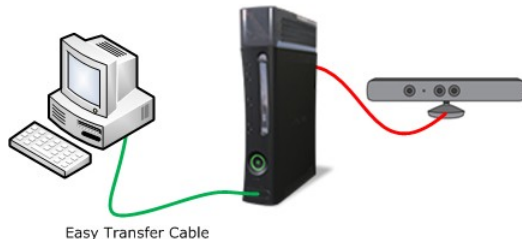
Connecting Using an Easy Transfer Cable

The easy transfer cable connects your development console to your computer. It cannot be connected to the same USB controller used by the Kinect sensor array.

To connect the Kinect and easy transfer cables

1. Connect the Kinect sensor array to the rear USB controller on the development console as follows:
 - For an XDK S console, connect the sensor array to the port on the rear of the console labeled "Kinect".
 - For an XDK console, connect the sensor array to the lone USB port on the rear of the console, next to the Ethernet port.
 2. Connect one end of the easy transfer cable to one of the USB ports on the front of the development console, in the hinged area next to the power button.
 3. Connect the other end of the easy transfer cable to a USB port on your computer.
- For best results, connect the cable to an unshared USB controller.

Figure 1. Connecting the cables for Xbox Studio



Operating System Versions

This section explains how to set up the easy transfer cable for each operating system.

Windows 7

You must plug in the easy transfer cable at least once before you run Xbox Studio for the first time. When the **Windows Easy Transfer** dialog box appears, close it. In Control Panel, click **Hardware and Sound** and then click **Autoplay**. In the drop-down list for **Easy Transfer Cable**, select **Take no action**.

Windows Vista

You must plug in the easy transfer cable at least once before you run Xbox Studio for the first time. When the AutoPlay dialog box appears, click **Set AutoPlay defaults in Control Panel**. In Control Panel, click the drop-down list for **Easy Transfer Cable**, and then select **Take no action**.

Connecting Over TCP/IP

To use Xbox Studio over a TCP/IP connection, you must close Xbox Studio, and then disconnect the easy transfer cable from both your PC and console. If an easy transfer cable is not connected, Xbox Studio will try to connect over TCP/IP to the development console that is selected in the drop-down list in the status bar. The first time that you start Xbox Studio, the tool will try to connect to the default console selected in Xbox 360 Neighborhood.

An attempt to connect to a development console over TCP/IP will fail in the following circumstances:

- The specified console cannot be found.
- The specified console is already connected to an instance of Xbox Studio that is running on another computer.
- An easy transfer cable is connected to the specified console, even if the cable is not connected to anything else.

To connect to a development console over TCP/IP

- In the status bar, click the drop-down list, and select the development console to which you want to connect.
The drop-down list is populated automatically from Xbox 360 Neighborhood. To add a new console without exiting Xbox Studio, click **Add Console** in the drop-down list to open the **Add Xbox 360 Development Kit** wizard.

Sharing a Development Console Over TCP/IP

Xbox Studio supports the sharing of a development console between developers on different computers over TCP/IP. Because only one instance of Xbox Studio can be connected to a development console at any given time, Xbox Studio allows you to explicitly connect to and disconnect from any development console on the network.

Xbox Studio automatically tries to connect whenever a connection is required—for example, when previewing live data, recording data, or playing data back to the console. By establishing a

dedicated connection to the console, you can prevent other developers from taking control of the development console during a critical moment. When you no longer need a dedicated connection to the console, you can disconnect to make the console available to other developers.

To connect to the development console

- On the toolbar, click **Connect**. 

To disconnect from the development console

- On the toolbar, click **Disconnect**. 

Automatic Connections

By default, Xbox Studio automatically tries to connect to a development console when the tool is launched. You can override this behavior to better control which developer is connected to the console.

To prevent Xbox Studio from connecting automatically to a development console

- In the **Tools** menu, click **Options**.
- In the **Options** dialog box, clear the **Connect to console at Xbox Studio startup** check box.

Improving Performance

Recording and playing back Kinect sensor data in Xbox Studio requires significant computer resources. If throughput of the data is not high enough, you can drop data frames. For best results, USB throughput on the computer side should be greater than 15 megabytes per second. Throughput is dependent on the model and device driver of your easy transfer cable, among other factors.

The following tips will help you get the best performance from Xbox Studio:

- Make sure that your computer has sufficient RAM to run your operating system. Xbox Studio runs best on Windows 7.
- Use a computer with a fast hard disk drive (HDD). The HDD should have plenty of empty space available, and should be reasonably unfragmented.
- Make system resources available to Xbox Studio—run as few other applications as possible.
- Close any Xbox Studio viewing windows that you aren't using.
- Plug the easy transfer cable into a USB controller on your computer that is not shared by any other devices.

See Also

[Using Xbox Studio](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Viewing Data [Xbox Studio]

[See Also](#)



You can use Xbox Studio to view live data from the Kinect for Xbox 360 sensor array. Data events are received approximately 30 times per second.

To preview live Kinect data

- On the toolbar, click **Live Preview**. 

If there is data opened currently, Xbox Studio will prompt you to save or discard the opened data. For more information about saving data, see [Saving Data \[Xbox Studio\]](#).
- In the stream selection grid to the left of the timeline, select the data streams that you want to view from the **Monitor** column. 

If Xbox Studio is connected to your development console using TCP/IP, the frame rate of the color and depth streams is limited by your network bandwidth. For more information about connection modes, see [Connecting to a Development Console \[Xbox Studio\]](#).

Color, depth, skeleton view, and skeleton data will each be shown in a separate viewing window, depending on which options are selected in the **View** menu. For information about how to record the live data from the sensor array, see [Recording Data \[Xbox Studio\]](#). Xbox Studio will remain in live preview mode until you perform one of the following actions:

- Stop recording data.
- Load data from an XED file. For more information, see [Loading Data \[Xbox Studio\]](#).
- Close the application.

See Also

[Using Xbox Studio](#)
[Playing Data \[Xbox Studio\]](#)
[Recording Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Recording Data [Xbox Studio]

[See Also](#)



One of the primary uses of Xbox Studio is to record data from the Kinect for Xbox 360 sensor array. Xbox Studio can record all visual data feeds from the sensor array, including color, depth, and skeletal data. Recording data automatically puts Xbox Studio in live preview mode.

To record data from the Kinect sensor array

- In the stream selection grid to the left of the timeline, select the data streams that you want to record from the **Record** column. 

If Xbox Studio is connected to your development console using TCP/IP, the frame rate of the color and depth streams is limited by your network bandwidth. For more information about connection modes, see [Connecting to a Development Console \[Xbox Studio\]](#).
- On the toolbar, click **Record**. 

If there is data opened currently, Xbox Studio will prompt you to save or discard the opened data. For more information about saving data, see [Saving Data \[Xbox Studio\]](#).
- When you are finished recording, click **Stop**. 

Xbox Studio will switch automatically from live preview mode to playback mode.

You can save recorded data to an Xbox event data (XED) file. For more information, see [Saving Data \[Xbox Studio\]](#).

See Also

[Using Xbox Studio](#)

[Viewing Data \[Xbox Studio\]](#)
[Playing Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
 Send feedback to <https://forums.xboxlive.com/>.
 Version: 21256.3

Playing Data [Xbox Studio]

[See Also](#)



You can play back Kinect for Xbox 360 data recorded in Xbox Studio. To enter the playback mode, you must have Kinect data opened in Xbox Studio first. You can open data using one of the following methods:

- Record new data from the sensor array. For more information, see [Recording Data \[Xbox Studio\]](#).
- Load data from an XED file. For more information, see [Loading Data \[Xbox Studio\]](#).

Sections

- [General Information](#)
- [Playing Data to a Console](#)
- [Suspending Playback](#)

General Information

You can choose to play back data either on your local computer or on a development console. Local playback is useful for simply viewing the data in the recording. You do not need to connect your computer to a development console for local playback.

Console playback is useful for debugging sensor data in your title. For more information about playing to a console, see [Playing Data to a Console](#).

To play opened Kinect data

- In the stream selection grid to the left of the timeline, select the data streams that you want to play back from the **Play** column. 
 - If Xbox Studio is connected to your development console using TCP/IP, the frame rate of the color and depth streams is limited by your network bandwidth. For more information about connection modes, see [Connecting to a Development Console \[Xbox Studio\]](#).
- On the toolbar, click **Play**. 
- When you are finished viewing the data, click **Stop**. 
 - Playback will also be halted when playback reaches the end of the playback range.

To set the current position of data playback

- Click on the desired location on the timeline.
 - For more information about working with the timeline, see [Main Window \[Xbox Studio\]](#).

To select a range of the timeline for playback

- Drag on the timeline.
 - or-
- Right-click on the timeline or graph, click **Select Range**, and then enter the start time and duration of the range.
 - or-
- In the **Timeline** menu, click **Select Range**, and then enter the start time and duration of the range.

To reset the playback range to the entire timeline

- In the **Timeline** menu, click **Clear Selection**.
 - or-
- Right-click on the timeline or graph, and then click **Clear Range**.
 - or-
- Press ESC.

Playing Data to a Console

Xbox Studio can play back Kinect data to a development console. This replaces the Kinect sensor array as the producer of NUI events, so that you can control exactly which data is sent to your title.

You can play back any Xbox event data file to a console without connecting a sensor array to the console. To do this, disconnect the sensor array from your development console before you launch Xbox Studio, open a prerecorded Xbox event data file, and click **Play on Console**.  When you play back the file, the data will be played back to the console in headless mode.

Note that if you choose to play back data on a console, you should play back both the color and depth streams. Title code running on simulated input data does not receive color data unless both color and depth data are being played back. This issue only affects playback. It does not affect the recording of color (you can record color alone,) and it does not indicate an invalid recorded file.

To toggle between local playback and playback to a development console

- On the toolbar, select or clear **Play on Console**. 

Looping Playback

When playing data to a development console, you can loop playback of the timeline or loop playback of the selected range of the timeline. You cannot loop local playback.

Looping is not supported with a mapped "fill-in" producer. Playback of an XED file does not loop correctly when a "fill-in" producer is mapped on the development workstation.

Looping de-synchronizes when playing streams from multiple files. When looping streams that originate from different XED files, the streams might get out of synchronization with each other.

Note

When looping is enabled, playback always starts from the beginning of the timeline or from the beginning of the selected range of the timeline. When looping is disabled, playback starts from the current position of the playback cursor.

To continuously loop playback

- On the toolbar, click **Loop**. 

Suspending Playback

You can suspend playback at any point. When playback is suspended, playback is still in an active state, but data is not sent.

To suspend playback

- On the toolbar, click **Pause**. 

To resume playback

- On the toolbar, click **Play**. 

Suspend Points

You can suspend playback at a predetermined event by setting a *suspend point* on a stream. Suspend points are activated only if the stream on which they are set is playing back.

To suspend playback at a specified timestamp

- Click above the timeline at the desired location.
- or-
- Do the following:
 - In the **Timeline** menu, click **Set Suspend Point**.
 - In the **Suspend Point** dialog box, select the stream for which to set the suspend point, and enter the timestamp for the suspend point.

To clear a suspend point

- In the **Timeline** menu, click **Clear Suspend Point**.

Single-stepping Playback

When playback to the console is suspended, you can single-step through selected streams. Single-stepping is different from advancing to the next event as follows:

- Next Event**  advances the timeline to the next event in each stream. This feature is not available during playback.
- Single Step**  sends the next event in each stream to the console. This feature is only available when playing data to the console.

To single-step playback to the console

- Suspend playback to the console.
- In the stream selection grid to the left of the timeline, select the data streams that you want to single-step from the **Single Step** column. 
- On the toolbar, click **Single Step**. 

See Also

- [Using Xbox Studio](#)
- [Viewing Data \[Xbox Studio\]](#)
- [Recording Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Playing and Recording Data Simultaneously [Xbox Studio]

[See Also](#)



The following streams are available:

- Depth stream
- Color stream
- Skeleton stream
- Identity stream
- Player Index stream (segmentation data)
- Title data stream

The streams are listed in [XSTUDIO_STREAM_FLAG Constants](#).

Recording a Stream While Playing Back a Stream

When playing back the skeleton, color, and/or depth stream, you can record the skeleton stream and/or the title data stream, but you cannot record any other stream. Recording the skeleton stream while playing back the depth stream enables you to determine the effects of skeletal tracking improvements.

To record the skeleton stream while playing back the color stream, you must also play back the depth stream.

[XStudioMapStreams](#) returns E_XSTUDIO_INVALID_OPERATION for unsupported combinations of streams.

Starting an Additional Stream

You can start the title data stream regardless of whether another stream is running.

You cannot start a stream (aside from the title data stream) if another stream (aside from the title data stream) is running; for supported combinations, you must start the streams (other than the title data stream) in the same call to [XStudioStart](#).

See Also

- [XSTUDIO_STREAM_FLAG Constants](#)
- [Using Xbox Studio](#)
- [Viewing Data \[Xbox Studio\]](#)
- [Recording Data \[Xbox Studio\]](#)
- [Playing Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Saving Data [Xbox Studio]

[See Also](#)



You can save Kinect for Xbox 360 sensor array data that has been recorded in Xbox Studio to an Xbox event data (XED) file. For information about recording data from the Kinect sensor array, see [Recording Data \[Xbox Studio\]](#).

To save opened data

- In the [File](#) menu, click **Save**.
- In the **Save As** dialog box, choose a file name and location for the Xbox event data file, and then click **Save**.

The default file name is "Xbox Studio Capture (YYYY MMM-DD HH.MM.SS.T).xed".

To save a range of the timeline

1. Select a range of the timeline by doing one of the following:
- Drag on the timeline.

-or-

• Right-click on the timeline or graph, click **Select Range**, and then enter values for the start time and duration of the range.

-or-

• In the [Timeline](#) menu, click **Select Range**, and then enter values for the start time and duration of the range.
2. Display the **Save As** dialog box by doing one of the following:
- Right-click on the timeline or graph, and then click **Save Range As**.

-or-

• In the **Timeline** menu, click **Save Range As**.
3. In the **Save As** dialog box, choose a file name and location for the Xbox event data file, and then click **Save**.

Recovering Unsaved Data

If you close Xbox Studio before you save a recording, you can recover the most recent recording from the temporary file directory. For information about setting the location of the temporary file directory, see [Tools](#).

To recover unsaved data

1. In Windows Explorer, navigate to your Xbox Studio temporary file directory.
2. Locate the .tmp file, and change the file extension to ".xed".

See Also

- [Using Xbox Studio](#)
- [Loading Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Loading Data [Xbox Studio]

[See Also](#)



You can use Xbox Studio to open data from the Kinect for Xbox 360 sensor array that has been saved to an Xbox event data (XED) file.

To load Kinect data

1. In the [File](#) menu, click **Open**.
2. In the **Open** dialog box, navigate to the XED file, and then click **Open**.

You can also open XED files by double-clicking them in Explorer or on the desktop.

Note

If the XED file was created with a previous version of Xbox Studio, the tool will perform a one-time conversion to the new file format. The converted file will be named "*previous-file-name.newN.xed*", where *N* is a zero-based index that prevents overwriting of any previous conversions of the old file.

See Also

- [Using Xbox Studio](#)
- [Saving Data \[Xbox Studio\]](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Graphing Data [Xbox Studio]

[See Also](#)



Xbox Studio can graph the position over time of skeleton joints in a recorded Xbox event data (XED) file. Each graph line represents the position of a single skeleton joint on a single axis. For example, a graph line can represent the position of the first gamer's left elbow on the y-axis.

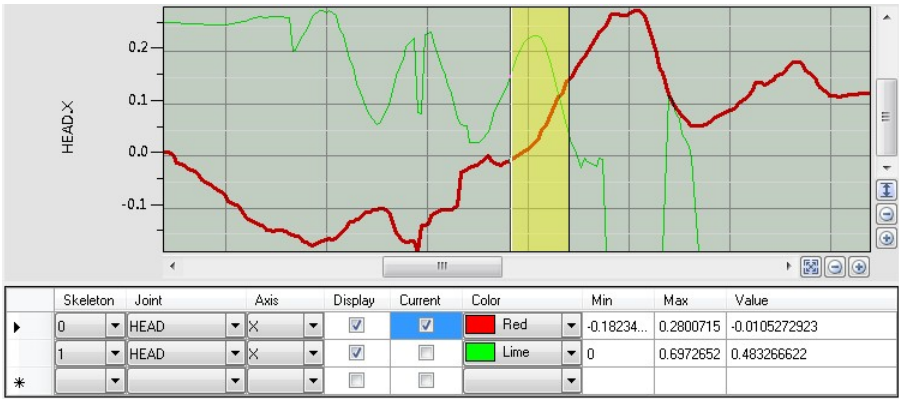
Sections

- [Skeleton Joint Graph](#)
- [Data Grid Columns](#)
- [Manipulating the Graph](#)

Skeleton Joint Graph

The skeleton joint graph is only available for data that has been recorded. It is not available during recording or live preview.

Figure 1. The Xbox Studio skeleton joint graph



To show or hide the skeletal data graph

- In the **View** menu, select or clear the **Graph** option.

Note

If your XED file contains a large amount of skeleton data, loading the graph might take some time. You can cancel loading the graph data at any time. If you enable the graph again later, the load will continue at the point where it left off.

To add a graph line

- In the empty row at the bottom of the data grid, select values for **Skeleton**, **Joint**, and **Axis**.
For a description of the columns in the data grid, see [Data Grid Columns](#).

To edit a graph line

- In the data grid row for the graph line, select a new value for at least one of the columns.

To delete a graph line

1. Click the row in the data grid for the graph line to delete.
2. Press DELETE.

Data Grid Columns

The following table contains information about the columns in the skeleton joint data grid.

Column	Description
Skeleton	Selects which tracked skeleton the graph line is for.
Joint	Selects which joint to graph.
Axis	Selects the axis along which to graph the joint.
Display	Toggles whether the graph line is displayed.
Current	Selects which graph line controls the active vertical range of the graph. The current graph line is drawn with a heavier line.
Color	Selects the color to use for the graph line. When a new graph line is added, it is assigned an unused color automatically. You can change this color later.
Min	Selects the minimum visible value of the graph line.
Max	Selects the maximum visible value of the graph line.
Value	<i>Not editable.</i> Displays the position of the joint along the selected axis, at the current location of the inspection cursor.

Manipulating the Graph

The horizontal axis of the skeleton joint graph zooms and pans along with the timeline. For information about using the zoom controls for the timeline, see [Timeline](#).

Each graph line can have a separate vertical range. The zoom controls for the vertical axis of the graph operate only on the *current graph line*. The current graph line also controls the vertical scale of the graph. For example, if the current graph line has a vertical range of [−1, 0], the vertical axis of the graph will be labeled from negative one to zero. If you then select a different graph line as the current line, the labeling of the tick marks on the vertical axis of the graph will be updated.

To set the current graph line

- In the data grid row for the graph line that you want to be the current line, select the **Current** check box.

The **Zoom to Selection** button located next to the horizontal scroll bar zooms the timeline so that the selected range always fills the window, even as you resize it. It also zooms every graph line—not just the current line—so that every value in the range is displayed in the graph. As mentioned previously, this is different from the vertical zoom controls that operate only on the current graph line.

Vertical Zoom Controls

The zoom controls for the vertical axis of the skeleton joint graph are located beneath the vertical scroll bar. These controls operate on the current graph line only. The following controls are provided:

Vertical Zoom to Fit

Zooms the vertical axis to fit the current graph line for the time range that is displayed currently in the timeline. The vertical axis will be zoomed so that the minimum and maximum values in the time range are both visible.

Vertical Zoom Out

Zooms out on the vertical axis for the current graph line.

Vertical Zoom In

Zooms in on the vertical axis for the current graph line.

See Also

[Using Xbox Studio](#)

Version: 21256.3

Xbox Studio Application Windows

[See Also](#)

This section contains information about the application windows in Xbox Studio, which provide visualization of color, depth, and skeleton data.

In This Section

[Main Window \[Xbox Studio\]](#)

Controls all Xbox Studio operations, including live preview, recording, and playback.

[Color Viewer Window \[Xbox Studio\]](#)

Provides visualization of live or recorded color data from the sensor array.

[Depth Viewer Window \[Xbox Studio\]](#)

Provides visualization of live or recorded depth data from the sensor array.

[Skeleton Viewer Window \[Xbox Studio\]](#)

Provides visualization of live or recorded skeleton data.

[Data Viewer Window \[Xbox Studio\]](#)

Provides visualization of live or recorded skeletal data from the sensor array.

See Also

[Xbox Studio](#)

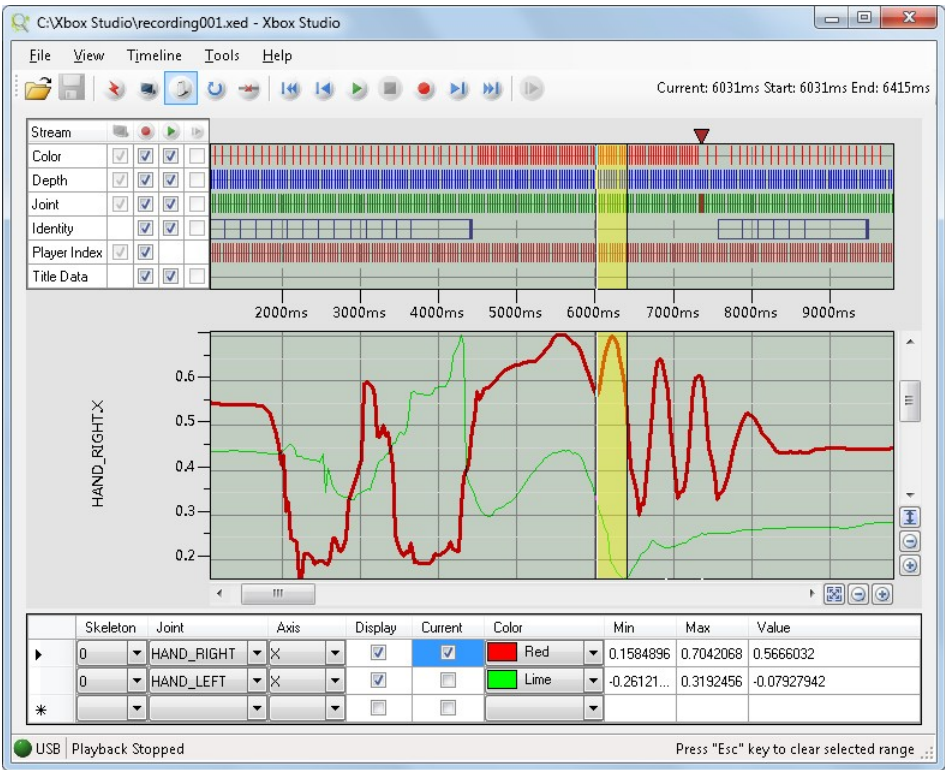
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Main Window [Xbox Studio]

[See Also](#)

The main window in Xbox Studio controls all operations, including live preview, recording, and playback.

Figure 1. The main window of Xbox Studio



Sections

- [Application Menus](#)
- [Toolbar](#)
- [Timeline](#)
- [Graph](#)
- [Status Bar](#)

Application Menus

Xbox Studio provides the following menus:

- [File](#)
- [Timeline](#)
- [View](#)

- [Tools](#)
- [Help](#)

File

The **File** menu provides the following commands:

Open (CTRL+O)

Opens an Xbox event data file (XED). For more information about opening an XED file, see [Loading Data \[Xbox Studio\]](#).

Close

Closes the current Xbox event data file.

Save (CTRL+S)

Saves the current recording to an Xbox event data file. For more information about saving data, see [Saving Data \[Xbox Studio\]](#).

Exit (ALT+F4)

Exits Xbox Studio.

Timeline

The **Timeline** menu provides the following commands:

Select Range

Allows you to set the start time and duration of the playback range, in milliseconds.

Clear Selection (ESC)

Resets the playback range to the entire timeline.

Save Range As

Saves the selected range of the timeline to an Xbox event data file. For more information about saving data, see [Saving Data \[Xbox Studio\]](#).

Set Suspend Point

Opens the **Suspend Point** dialog box, which allows you to add a suspend point to a stream. Playback will be suspended when it reaches the event with the specified timestamp. For more information about suspending playback, see [Playing Data \[Xbox Studio\]](#).

Clear Suspend Point

Clears the current suspend point.

View

The **View** menu provides the following commands:

Color

Toggles display of the **Color Viewer** window. For more information, see [Color Viewer Window \[Xbox Studio\]](#).

Depth

Toggles display of the **Depth Viewer** window. For more information, see [Depth Viewer Window \[Xbox Studio\]](#).

Skeleton

Toggles display of the **Skeleton Viewer** window. For more information, see [Skeleton Viewer Window \[Xbox Studio\]](#).

Data View

Toggles display of the **Data Viewer** window. For more information, see [Data Viewer Window \[Xbox Studio\]](#).

Graph

Toggles display of the skeleton joint graph. For more information, see [Graphing Data \[Xbox Studio\]](#).

Tools

The **Tools** menu provides the following commands:

Options

Opens the **Options** dialog box, which contains the following options:

- **Temporary File Location:** Allows you to set the directory where you can save the temporary Xbox event data files. If you close Xbox Studio before you save a recording, you can recover the most recent recording from this directory. For more information about recovering the most recent recording, see [Recovering Unsaved Data](#). You should set the temporary file location to a directory that is on the same disk drive in which you usually save Xbox event data files. The directory must exist already, and the user account under which you run Xbox Studio should have read/write access to the directory.
- **Connect to console at Xbox Studio startup:** Specifies whether to connect to the development console automatically when you start Xbox Studio. If you are sharing a development console with other developers over a network, preventing an automatic connection can help avoid conflicts with other users. For more information about sharing a development console, see [Sharing a Development Console Over TCP/IP](#).

Help

The **Help** menu provides the following commands:

Contents (F1)

Opens the XDK documentation for Xbox Studio.

About Xbox Studio

Shows version and copyright information for Xbox Studio.

Toolbar

The Xbox Studio toolbar contains the following tools:



Open (CTRL+O)

Opens an Xbox event data file (XED).



Save (CTRL+S)

Saves the current recording to an Xbox event data file. For more information about saving data, see [Saving Data \[Xbox Studio\]](#).



Connect/Disconnect (CTRL+C)

Toggles whether to connect Xbox Studio to the development console. If you are sharing a development console with other developers over a network, disconnecting frees the console for use by other users. For more information about sharing a development console, see [Sharing a Development Console Over TCP/IP](#).



Live Preview (CTRL+L)

Puts Xbox Studio into live preview mode. For more information about live preview mode and playback mode, see [Viewing Data \[Xbox Studio\]](#) and [Playing Data \[Xbox Studio\]](#).



Play on Console

Toggles whether to play back data on your development console or on your local computer. For more information about console playback versus local playback, see [Playing Data \[Xbox Studio\]](#).



Loop (CTRL+P)

Continuously loops playback to the console of the timeline or of the selected range of the timeline. This button is disabled if **Play on Console** is not selected because local playback does not loop.



Simulated Input Mode (CTRL+I)

Toggles whether to play back data in simulated input mode. In simulated input mode, live NUI events from the Kinect sensor array are not sent to the title. All streams are redirected to come only from Xbox Studio.



First Event (HOME)

Moves the current position of the timeline to the first event.



Previous Event (LEFT ARROW)

Moves the current position of the timeline to the previous event.

-  **Play/Pause (F5)**
Starts or pauses playback of recorded event data. For more information about playing back data, see [Playing Data \[Xbox Studio\]](#).
-  **Stop (SHIFT+F5)**
Stops recording or playback. Stopping playback resets the cursor to the beginning of the playback range.
-  **Record (CTRL+R)**
Starts recording the data from the sensor array. For more information about recording data, see [Recording Data \[Xbox Studio\]](#).
-  **Next Event (RIGHT ARROW)**
Moves the current position of the timeline to the next event.
-  **Last Event (END)**
Moves the current position of the timeline to the last event.
-  **Single Step (F10)**
Advances the current position of each stream by a single event.

The toolbar also shows time information about the file. While recording, it shows the elapsed time since recording began. During playback of data, it shows the elapsed time from the beginning of the data as well as the total length of the data. If you select a range of the timeline, it also shows the time of the start and end of the selected range.

Timeline

The timeline shows information about the XED file currently opened in Xbox Studio. It shows timing information for all events in the data streams received from the sensor array. It also shows and controls the current position of playback of the file, and allows you to specify a range of events for playback. Identity events that are part of the same identity operation are grouped together in the timeline.

The timeline contains a grid that allows you to specify which streams to view, record, play back, and single step. Xbox Studio saves the stream options for a USB connection separately from the options for a TCP/IP connection.

 **Note**

If you play back the identity stream, you must also play back the skeletal stream.

You can select a range for playback by dragging on the timeline. During playback, only this range will be played. For more information about selecting and deselecting a playback range, see [Playing Data \[Xbox Studio\]](#).

The timeline has two cursors:

- The *inspection cursor* follows the position of the mouse cursor. It controls which position in the timeline is displayed in the viewer windows when playback is stopped.
- You can move the *playback cursor* by clicking the timeline. It controls the location at which playback will begin when playing sensor array data. It also tracks the current position in the timeline during playback. If a range is selected in the timeline, clicking outside of the range will clear the selection.

 **Note**

When moving the inspection cursor, a viewer window can be empty if that part of the timeline does not contain an event for the related data. This is expected behavior.

By default, the timeline shows the entire file. To view more detailed timing information for the events received in each data stream from the sensor array, you can control the zoom factor of the timeline by using the horizontal zoom controls or by rotating the mouse wheel forward or backward.

Timeline Zoom Controls

The zoom controls for the timeline are located to the right of the horizontal scroll bar. The following controls are provided:

-  **Zoom to Selection**
Zooms the timeline so that the selected range always fills the window, even as you resize it. If there is not a selected range, the timeline zooms so that the entire timeline is displayed.
-  **Horizontal Zoom Out**
Zooms out on the timeline.
-  **Horizontal Zoom In**
Zooms in on the timeline.

Graph

The skeleton joint graph maps the position over time of the skeleton joints in a recorded Xbox event data file. For more information about graphing skeleton joints, see [Graphing Data \[Xbox Studio\]](#).

Status Bar

The status bar shows the information listed in the following table.

Information	Description
Connection status	Indicates whether Xbox Studio currently has a connection to the development console, as well as the type of the connection (USB or TCP/IP). If you are connecting over TCP/IP, the status bar also contains a drop-down list that allows you to specify which development console in Xbox 360 Neighborhood to which to connect. For more information about connecting over TCP/IP, see Connecting Over TCP/IP .
Current operation	Shows the operation that Xbox Studio is performing currently.

 **See Also**

[Xbox Studio Application Windows](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Color Viewer Window [Xbox Studio]

[See Also](#)



The Xbox Studio **Color Viewer** window shows recorded or live color data from the sensor array.

Figure 1. The Xbox Studio Color Viewer window



Sections

- [Zooming and Panning](#)
- [Selecting a Frame](#)
- [Saving Color Frames](#)
- [Player Index](#)
- [Status Bar](#)

Zooming and Panning

The **Color Viewer** window supports the following zoom factors:

- **Percentage:** Zooms relative to the original stream size.
- **Zoom to Fit:** Zooms to always fill the window, even as you resize it.

To control the zoom factor of the viewer

- Select the zoom factor from the combo box.
- or-
- Type a numerical value into the combo box, and then press ENTER.
- or-
- Use the zoom tools on the toolbar.
- or-
- When the viewer has the input focus, rotate the mouse wheel forward or backward.

To pan the displayed area of the viewer

- Drag within the viewer.
- or-
- Use the scroll bars (when displayed).

To reset zoom and pan for the viewer

- Select **Zoom to Fit** from the combo box.
- or-
- Click the **Zoom to Fit** button on the toolbar. 

Selecting a Frame

You can change which color frame is displayed in the viewer without using the timeline.

To display the previous frame in the color stream

- Press the LEFT ARROW key.

To display the next frame in the color stream

- Press the RIGHT ARROW key.

Saving Color Frames

You can save the currently displayed color frame to a bitmap file.

To save the current color frame as a bitmap

1. Right-click within the color viewer, and then click **Save Image As**.
2. In the **Save As** dialog box, choose a file name and location for the bitmap, and then click **Save**.

Player Index

Xbox Studio can display the player segmentation map that is exposed by the [NuiImageCamera](#) API. For more information about the segmentation map, see [NUI Image Camera Overview](#). To view the segmentation map, you must enable the player index stream for recording or live preview. You can do this by selecting the appropriate check boxes in the stream selection grid next to the [timeline](#).

When the player segmentation map is enabled, the color viewer displays a colored mask over each recognized player.

To toggle display of the player segmentation map

- Click **Toggle Player Index**. 

Status Bar

The status bar displays the following information.

Information	Description
Frame Number	Frame number of the frame that is displayed currently.
Color Quality	Displays "HiQ" if the color stream is in high quality mode.
Position	X and Y coordinates of the pixel at the current position of the cursor. The top left corner of the viewer is (0, 0); the bottom right corner is (640, 480).
Color	RGB and YUV values for the pixel at the current position of the cursor.

See Also

[Xbox Studio Application Windows](#)

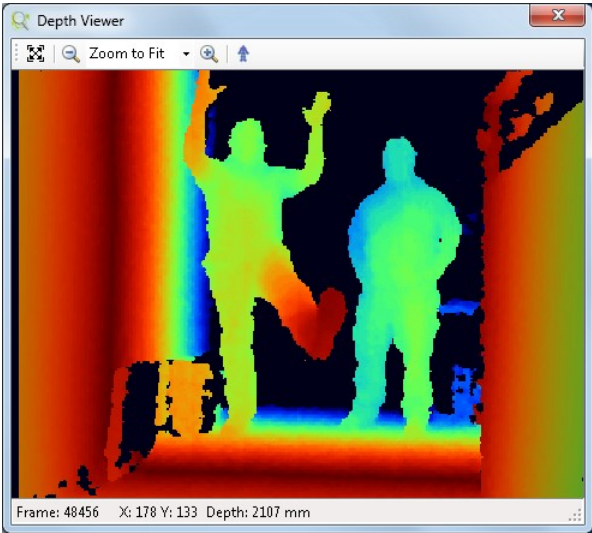
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Depth Viewer Window [Xbox Studio]

[See Also](#)

The Xbox Studio **Depth Viewer** window shows recorded or live depth data from the sensor array. Objects at medium range are displayed in red, transitioning through the spectrum to violet as an object gets closer to or farther away from the sensor array.

Figure 1. The Xbox Studio Depth Viewer window



Sections

- [Zooming and Panning](#)
- [Selecting a Frame](#)
- [Player Index](#)
- [Status Bar](#)

Zooming and Panning

The **Depth Viewer** window supports the following zoom factors:

- Percentage:** Zooms relative to the original stream size.
- Zoom to Fit:** Zooms to always fill the window, even as you resize it.

To control the zoom factor of the viewer

- Select the zoom factor from the combo box.
-or-
- Type a numerical value into the combo box, and then press ENTER.
-or-
- Use the zoom tools on the toolbar.
-or-
- When the viewer has the input focus, rotate the mouse wheel forward or backward.

To pan the displayed area of the viewer

- Drag within the viewer.
-or-
- Use the scroll bars (when displayed).

To reset zoom and pan for the viewer

- Select **Zoom to Fit** from the combo box.
-or-
- Click the **Zoom to Fit** button on the toolbar. 

Selecting a Frame

You can change which depth frame appears in the viewer without using the timeline.

To display the previous frame in the depth stream

- Press the LEFT ARROW key.

To display the next frame in the depth stream

- Press the RIGHT ARROW key.

Player Index

Xbox Studio can show the player segmentation map that is exposed by the [NuiImageCamera](#) API. For more information about the segmentation map, see [NUI Image Camera Overview](#). To view the segmentation map, you must enable the player index stream for recording or live preview. You can do this by selecting the appropriate check boxes in the stream selection grid next to the [timeline](#).

When the player segmentation map is enabled, the depth viewer displays a colored mask over each recognized player. Also, the typical red-to-violet depth map is converted to a grayscale spectrum.

To toggle display of the player segmentation map

- Click **Toggle Player Index**. 

Status Bar

The status bar displays the following information.

Information	Description
Frame Number	Frame number of the frame that is displayed currently.
Position	X and Y coordinates of the pixel at the current position of the cursor. The top left corner of the viewer is (0, 0); the bottom right corner is (320, 240).
Depth	Distance from the sensor array of the pixel at the current position of the cursor, in millimeters.

See Also

[Xbox Studio Application Windows](#)

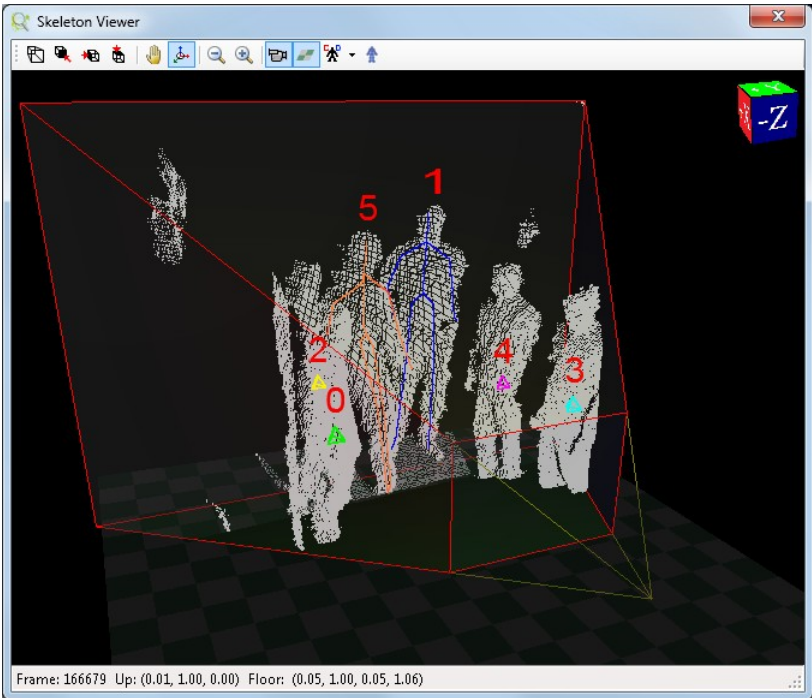
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Skeleton Viewer Window [Xbox Studio]

[See Also](#)

In Xbox Studio, the **Skeleton Viewer** window shows recorded or live skeletal data from the Kinect for Xbox 360 sensor array. You can track up to six skeletons in three-dimensional space. You can fully track two of the skeletons at a time. However, you can only track the general positions of the other four skeletons. The **Skeleton Viewer** window can also show color and depth data for tracked skeletons.

Figure 1. The Xbox Studio Skeleton Viewer window



If the skeletal stream is displayed, the two fully tracked skeletons are rendered in the viewer. The positions of the other four skeletons are displayed as small tetrahedrons. The numbers attached to the skeletons correspond to the skeleton identifiers in the [Data Viewer](#) window.

The *camera frustum* is a pyramidal volume that shows the physical space visible to the sensor array. The point of the pyramid represents the position of the sensor array. The rectangle near the sensor array represents the front clipping plane. The gray base of the pyramid represents the maximum distance of the depth sensor.

The cube in the top-right corner of the viewer shows the current direction of the coordinate axes in relation to the viewing angle, where positive X is to the left, positive Y is up, and positive Z is away from the sensor array. The cube will rotate as you rotate the view around the center point.

Sections

- [Selecting a Preset Viewing Angle](#)
- [Customizing the Viewing Angle](#)
- [Selecting the Streams to Display](#)
- [Selecting a Frame](#)
- [Player Index](#)
- [Status Bar](#)

Selecting a Preset Viewing Angle

The **Skeleton Viewer** window provides a number of preset viewing angles. The default view places the position of the sensor array in the lower-right corner of the viewer.

To set the viewing angle to the front view

- Click **Front View**. 

To set the viewing angle to the left side view

- Click **Left Side View**. 

To set the viewing angle to the top view

- Click **Top View**. 

To reset the viewing angle to the default view

- Click **Reset View**. 

Customizing the Viewing Angle

In addition to using the preset views, you can also move the viewing angle freely within three-dimensional space.

To move the viewing angle in or out

- Click **Camera In**  or **Camera Out**. 
- or-
- Hold down ALT and the right mouse button, and then drag forward or backward within the viewer.
- or-
- Rotate the mouse wheel forward or backward.

To pan the viewing angle

- Select **Pan**,  and then drag within the viewer.
- or-
- Hold down ALT and the middle mouse button, and then drag within the viewer.

To rotate the viewing angle in three-dimensional space

- Select **Rotate**,  and then drag within the viewer.
- or-
- Hold down ALT and the left mouse button, and then drag within the viewer.

To toggle display of the camera frustum

- Click **Toggle Camera Frustum**. 

To toggle display of the floor plane

- Click **Toggle Floor Plane**. 

Selecting the Streams to Display

The **Skeleton Viewer** window can show skeletal data from the sensor array, as well as depth and color information for the area immediately surrounding the skeletons. The appearance of the **Select View** button changes to reflect which streams are displayed currently, as shown in the following table.

Stream	Description
Skeleton	If selected for display, shows a black stick figure; otherwise, shows a gray stick figure.
Color	If selected for display, shows a red letter C.
Depth	If selected for display, shows a blue letter D.

To select the streams to display

- Click the arrow next to **Select View**,  and then select an option from the drop-down list.
- or-
- Click **Select View** to cycle through the available options.

Selecting a Frame

You can change which skeletal frame is displayed in the viewer without using the timeline.

To display the previous frame in the skeletal stream

- Press the LEFT ARROW key.

To display the next frame in the skeletal stream

- Press the RIGHT ARROW key.

Player Index

Xbox Studio can show the player segmentation map that is exposed by the [NuiImageCamera](#) API. For more information about the segmentation map, see [NUI Image Camera Overview](#). To view the segmentation map, you must enable the player index stream for recording or live preview. You can do this by selecting the appropriate check boxes in the stream selection grid next to the [timeline](#).

When the player segmentation map is enabled, the skeleton viewer displays a colored mask over each recognized player.

To toggle display of the player segmentation map

- Click **Toggle Player Index**.

Status Bar

The status bar displays the following information.

Information	Description
Frame Number	Frame number of the frame that is displayed currently.
Up Vector	Coordinates of the up vector.
Floor	Coordinates of the floor plane.

See Also

[Xbox Studio Application Windows](#)

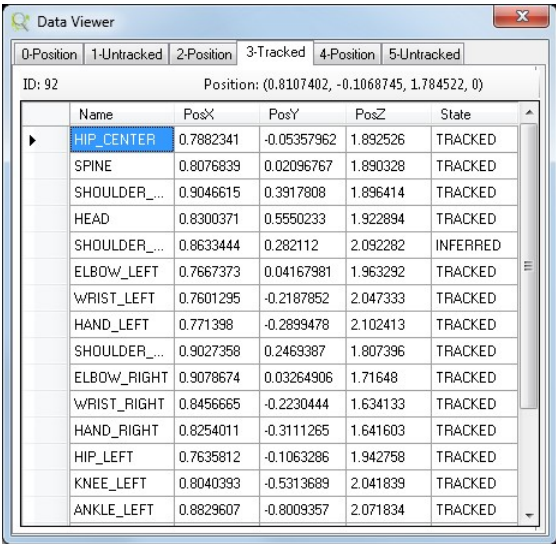
© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Data Viewer Window [Xbox Studio]

[See Also](#)

The Xbox Studio **Data Viewer** window shows recorded or live skeletal data from the sensor array.

Figure 1. The Xbox Studio Data Viewer window



The sensor array can track data for up to six skeletons. To view skeletal data for a skeleton, click the tab for that skeleton.

The **Data Viewer** window shows x, y, and z coordinates for the location of each of the skeleton positions defined in the [NUI_SKELETON_POSITION_INDEX](#) enumeration. It also shows the tracking state of each position. For more information about tracking states, see the Remarks section of the [NUI_SKELETON_POSITION_TRACKING_STATE](#) enumeration.

See Also

[Xbox Studio Application Windows](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Identity Data [Xbox Studio]

[See Also](#)

Xbox Studio, the [XedFile](#) APIs, and the [XStudio](#) APIs provide an identity data stream that allows you to record and play back identity events. For more information about identity data in general, see [Overview of NUI Identity](#).

Identity events are recorded to an Xbox event data (XED) file in the same way as color, depth, and skeletal events. Because the identity system relies on the skeletal tracking pipeline, if you record the identity stream you must also record either the skeletal or the depth stream.

Unlike other streams, the identity stream is stateful—whether a gamer is currently enrolled determines how an event is processed. Playing back the XED file when the identity system is in a different state than when the events were recorded can lead to unpredictable results.

There are two ways to ensure that your title correctly plays back recorded identity data. The easiest method is to ensure that the identity state during playback is exactly the same as it was when the identity events were recorded. For example, you can ensure that no gamers are enrolled, and then record a long clip that contains multiple identity operations. Before playing back the XED file, you must restore the identity system to a clean game state. This approach is useful for recreating a complex state, such as enrolling the maximum number of gamers.

Alternatively, you can record small clips that hold individual identity operations, and then play back those clips to manually reconstruct a particular state. In this approach, you must synchronize playback of the identity events with calls to the [NuiIdentityIdentify](#) and [NuiIdentityEnroll](#) functions. For example, to identify a gamer:

- Call [NuiIdentityIdentify](#) to begin the asynchronous identify operation.
- Play back a recorded identity event.
- When the operation is complete, handle the result in your [PFN_NUI_IDENTITY_CALLBACK](#) function.

If you need to record and play back identity events programmatically—for example, to develop test automation—use the [XedFile](#) identity APIs:

- [XedCopyNuiIdentityMessage](#) - Copies the specified identity message from a XED file to another XED file.
- [XedReadNuiIdentityMessage](#) - Reads the specified identity message from a XED file.

- [XedWriteNuiIdentityMessage](#) - Writes the specified identity message to a XED file.

✚ See Also

[Xbox Studio](#)
[Overview of NUI Identity](#)
[XedFile Reference](#)
[XStudio Reference](#)
[NuiIdentityEnroll](#)
[NuiIdentityIdentify](#)
[PFN_NUI_IDENTITY_CALLBACK](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Title Data [Xbox Studio]

[See Also](#)



Xbox Studio, the [XedFile](#) APIs, and the [XStudio](#) APIs provide a title data stream that allows you to record and play back opaque title-defined data. You can record title data and play it back along with the color, depth, and skeletal data captured by the Kinect sensor array. Here are two examples of the usefulness of title data:

- Recording a title's state or context information for use in debugging.
- Instrumenting a title so that button presses are recorded in the title data. During playback, you can use these button presses to start playback of the color and depth streams.

A small amount of title-defined index data is recorded along with each block of title data. This index data is cached separately to enable custom indexing functions on the title data. For example, you could index a title data block with a unique value that can be indexed. Once you do that, another function could return the event number of the title data when passed that value.

✚ See Also

[Xbox Studio](#)
[XedFile Reference](#)
[XStudio Reference](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Xbox Studio Command-Line Utility



The Xbox Studio command-line utility, XSCmd.exe, upgrades the Xbox event data (XED) files that are used by [Xbox Studio](#).

```
xscmd [ /u] [ /c] source destination [ /?]
```

Option	Description
/u	Upgrade XED files to conform to the most recent file format specification. The <i>source</i> argument specifies the files to convert. The <i>destination</i> argument specifies where the upgraded versions of the files will be placed.
/c	Continue to upgrade files if a conversion error occurs. If you do not specify this argument, conversion will stop immediately if a conversion error occurs, bypassing all of the remaining files specified by the <i>source</i> argument.
<i>source</i>	Filename of the XED file to upgrade, or a wildcarded filename pattern that specifies the files to convert (for example, c:\data*.xed).
<i>destination</i>	If the <i>source</i> argument specifies a single file, this is the filename that will be used for the upgraded version of the file. If the <i>source</i> argument is a wildcarded filename pattern, this is the path to the folder where the upgraded versions of the files will be placed. The folder must already exist. Existing files are not overwritten; the conversion will fail if a filename collision occurs.
/?	Displays usage information.

To convert existing skeleton tracking data to the new format, which can be used in Visual Gesture Builder, see [Xed Skeleton Updater](#).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural User Interface White Papers



This section contains white papers related to Natural User Interface (NUI) software and hardware.

Design

[Kinect Essentials and Showcases](#)

Provides guidance on features considered as showcase or essential features for Kinect titles on Xbox 360.

Development

[Natural Environments: Understanding and Developing for the Kinect Play Space](#)

Describes technical requirements of the sensor array, provides advice for configuring a development environment, and recommends conditions and details best practices for understanding the end-user's play space, establishing a workspace, and positioning and using the camera.

[Tilt-a-Whirl: Kinect Tilt, Play Space Management, and You](#)

Details pipeline and title implications for the voice chat codec updates occurring in the Fall 2010 System Update, and available for development in the June 2010 XDK.

[Visual Gesture Builder: A Data-Driven Solution to Gesture Detection](#)

Explains how a data-driven machine-learning solution can detect the most challenging gestures with very high accuracy, making developers more productive and raising the quality of gesture detection.

Handles

[The System Gesture for Kinect for Xbox 360](#)

Describes the *system gesture*, a gesture that players can use with Kinect to access the Xbox Guide and start a game.

Identity

[NUI Identity Basics: Implementing Essential Skeleton Identity Processing](#)

How to implement identity recognition behaviors to meet the minimum requirements for Kinect NUI Identity.

[Who Is Playing My Kinect Game Right Now?](#)

Provides a high-level view of how the identity system works in Kinect games and how it can be integrated into a title.

[Using NuiIdentity Effectively](#)

Describes how to handle player enrollment in a Kinect-enabled title using NUI Identity.

Skeleton

[NUI Joint Filtering: Best Practices](#)

Describes the filtering techniques and best practices for using skeleton joint data for Kinect-enabled titles. Also helps game developers to choose an appropriate filtering technique and fine-tune the filter parameters to match the specific needs of their Kinect-enabled titles.

[Kinect Image Segmentation: Challenges, Suggestions, and Solutions](#)

Discusses segmentation issues, including advantages and challenges in using the NUI depth map to augment traditional solutions.

[Know Thy User: Understanding Implications of Player Characteristics and Behaviors when using the Natural User Interface](#)

Describes the issues of the human body as input device: varied surfaces, multiple inputs, and giving helpful feedback.

[Reducing Latency in Kinect Titles](#)

Describes how latency is introduced and explains what can be done to reduce it.

[Simple Skeleton Normalization and Animation Comparison](#)

Explains how to implement a simple animation and pose scoring system for fitness and dancing games.

[Using Kinect for Hand-State Detection](#)

Describes ways of detecting whether a hand is closed or open for use in gameplay. Discusses the variations of the open hand, use of the HandOpenClosed sample, and explores the combination of machine learning and Kinect functionality.

Speech

[Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#)

Provides an introduction to developing grammars for the speech recognition system provided by Natural User Interface (NUI) technology.

[Grammar School 201: Grammar Construction Best Practices](#)

Describes the best practices for the speech recognition system provided by Natural User Interface (NUI) technology.

[Silver Tonque: Best Practices for Speech Development with Kinect for Xbox 360](#)

Describes best practices for grammar construction and game design, as well as potential dangers, when developing speech recognition for Kinect.

[Speech Design for Game Designers: An Overview](#)

Describes how the game designer can enhance the user experience by incorporating speech into a game.

[Talk To Me: Microphone and Speech Technology in XDK Kinect Hardware](#)

Addresses the fundamental software and hardware concepts of the Kinect audio input components and explains implementation paths prior to final hardware availability.

[The User Says What? Testing and Tuning Guidelines for Speech Recognition with Kinect for Xbox 360](#)

Defining, testing, and tuning a NUI speech recognition grammar.

[Vocal Clarity: Voice Chat Codec Updates for the Fall 2010 System Update](#)

Details changes to the audio pipeline of the voice chat codec in the Fall 2010 System Update and implications for voice chat.

Related Topics

[XDK Unlocked - Videos](#)

[NUI Overview](#)

[NUI Reference](#)

[NUI Samples](#)

[NUI Tools](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Kinect Essentials and Showcases



Compiled by Scott Selfon, Lead Program Manager

Advanced Technology Group (ATG)

Provides guidance on features considered as showcase or essential features for Kinect titles on Xbox 360.

The Five Pillars of Kinect Experiences

Our vision for Kinect is to enable groundbreaking, inventive, and unique gameplay experiences. The player controls the experience through his or her actions, and is not tethered to any device. As we considered the implications for game design, we identified five key pillars that drove our thinking around *showcases* and *essentials*.

Is It Fun?

Above all, Kinect experiences should be entertaining, giving the player rich rewards for successful achievement.

Is It Accessible?

Players should learn by exploration and discovery, with hints and clear design elements guiding the player to perform intuitive actions, rather than requiring mechanical training and calibration on the part of the player.

Is It Interactive?

Natural User Interface (NUI) gives the player more direct control over his or her experiences. Gameplay should be intuitive and immediate.

Is It Imaginative?

Experiences should give the player a feeling of augmenting what they can do in the real world, giving them almost supernatural control.

Is It Social?

Kinect experiences should involve everyone in the room and the games from which these experiences are derived should be as fun to watch as they are to play.

If you have any questions about these pillars, contact your Account Manager or Developer Account Manager.

Defining Showcases and Essentials

What are Essentials?

Essentials are platform features that must be supported by all Kinect games. They are seen by Microsoft as features that define the gaming experience on the platform, and represent the baseline of what gamers can consistently expect from experiences using NUI. Essentials are considered critical to the success of the platform and the titles that are part of it.

Note

Essentials are distinct from Technical Certification Requirements (TCRs), though TCRs enforcing *essentials* are called out in (italic) below. For the current Kinect TCRs, visit Xbox 360 Central or speak with your Account Manager or Developer Account Manager.

What are Showcases?

Showcases take your game beyond the mere *essentials*—they are platform features that are not explicitly required, but that can be supported to differentiate your game from your competitors' offerings. You can think of these as features you likely highlight through the game's marketing, or demonstrate onstage at a keynote. These are features that can help make your games stand out from the crowd, define the potential of Kinect, and push the platform to its limits.

Essentials

Controller-Free Gameplay

Kinect titles should be playable without picking up a controller from start to finish: gameplay, navigational interfaces, and biometric sign-in/sign-out should all be controllable by the player without needing to use the controller. An exception is if the player is having difficulty with a Kinect experience, the NUI Troubleshooter should be accessible via controller. (*TCR NUI Troubleshooter Access*)

References:

[The NUIHandles API](#)
[Technical Certification Requirements](#)

Resources:

[The System Gesture for Kinect](#)
[Menu Using Handles Sample](#)
[Menu Using Hover Sample](#)
[Hand Refinement Sample](#)

Stand Up and Play: Full Body Skeletal Tracking as Input

Using the player's full skeleton is a powerful and intuitive way to express natural user interface, and creates interactive and intuitive inputs for entertaining experiences.

References:

[NUI Architecture](#)
[Technical Certification Requirements](#)

Resources:

[Know Thy User](#)
[SimpleSkeletonTracking Sample](#)

Sweet Play in the Sweet Spot

(*TCR NUI Playable Distance*)

Kinect defines the sweet spot as a consistent distance (6 feet, or 1.8 meters) at which titles should work for the vast majority of sensor array and player characteristics. Gameplay should be robust at this distance without requiring the user to reposition the sensor array. (*TCR NUI Manual Sensor Adjustment*) Titles should also consider effective ways to guide the user back to the play space where they can most effectively interact with the title.

References:

[Technical Certification Requirements](#)

Resources:

[Natural Environments: Understanding and Developing for the Kinect Play Space](#)
[Playspace Feedback Sample](#)

Easy In/Easy Out: Jump In and Play

A key component of the social aspect of Kinect is multiplayer titles with low friction entry and exit. Players can jump in and out of a game on demand by using the identity system (*NUI Identity Enrollment*), and titles should intuitively validate the player's presence to indicate his or her awareness (*NUI Identity User Feedback*), as well as provide avenues for the player to adjust his or her logged-in identity (*NUI Identity Override*).

References:

[NUI Identity](#)
[Technical Certification Requirements](#)

Resources:

[Using NuiIdentity Effectively](#)
[Identity Sample](#)
[Active/Passive Skeletons Sample](#)

Ubiquitous Voice Chat

Voice communication is as intuitive as full body movement. The Kinect sensor array's microphone can be used as a consistent audio input channel, usable by titles in a variety of ways. At its most basic, all Kinect titles enable voice chat with online friends via the party chat, private chat, and in-game chat channels. Titles can also use voice input in more sophisticated ways (see the Showcases section).

Resources:

[Talk to Me](#)
[LobbyChat Sample](#)
[SimplePartyExperience Sample](#)

Showcases

Screen Presence Reflects Player Actions

Using the player's skeleton for input is an *essential*. Some of the most effective titles will go beyond this, providing innovative color/depth representations of the player, and allowing the player to feel in control of his or her avatar through animation blending or skeletal retargeting that map the player's actions to an Xbox 360 avatar or a title-provided character representation.

Resources:

[Avatar Retargeting Sample](#)
[Avatar Monkey-See-Monkey-Do Sample](#)
[Skeletal Post-Processing Sample](#)

Kinect Gameplay Over Xbox LIVE

Playing locally with friends is just one social aspect of Kinect; just as powerful is the impact of a higher level of interactivity with online friends and competitors. Ubiquitous voice chat is just one aspect of this: cooperative and competitive gameplay with other players who visually appear and sound just as human as the person controlling their actions. This brings another level of realism and immersion to real-time Xbox LIVE experiences.

Resources:

[LobbyChat Sample](#)
[Avatar Monkey-See-Monkey-Do Sample](#)

Augmented Reality

The Kinect sensor streams provide opportunities for significant immersive input beyond skeletal tracking. The depth image can inform the title about the player's environment, making the environment part of the game. Sound position tracking can inform the title as to where people are speaking from—even when not skeletally tracked.

Resources:
[HeadTracker Sample](#)
[Motion-based Background Removal Sample](#)
[Sound Location Tracking Sample](#)

Putting the Player in the Game

An RGB color image can be leveraged in a variety of ways. When used in conjunction with depth and segmentation, it can be used for "green-screen"-style compositing. Titles can also capture clips and snapshots of the player and their actions, correlating visual information with depth, skeletal, and audio data to create rich scenes for future playback and gameplay.

Resources:
[Active/Passive Skeletons Sample](#)
[Motion-based Background Removal Sample](#)
[Segmentation Sample](#)
[Frosty Window Sample](#)

Speech Recognition

Voice is as intuitive as body movement for communicating and interacting with a title. Speech recognition takes the *essential* of audio input to the realm of *showcase* by providing mechanisms where the title gives the perception of understanding the player's intent through his or her words. Speech can powerfully provide "hints" to a title, and often gives more context than a gesture can for communicating the player's intent. While speech recognition should never be required for titles delivered in regions where all languages are not supported (*NUI Speech Recognition Language Support*), it can be leveraged as an enhancement and unique gameplay aspect for players in language-supported regions.

References:
[NUI Speech Overviews](#)
[Technical Certification Requirements](#)

Resources:
[Silver Tongue: Best Practices for Kinect Speech Development](#)
[Talk to Me](#)
[Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#)
[Speech Design for Game Designers: An Overview](#)
[Grammar School 201: Grammar Construction Best Practices](#)
[Simple Speech Recognition Sample](#)

Everyone Plays

Kinect uniquely supports awareness and enrollment of up to eight players at a time, providing systems for identifying players even as they move in and out of the sensor array's field of view. Beyond the "easy in/easy out" *essential* discussed above, *showcase* titles will leverage this functionality to provide true party-style gameplay, giving everyone in the room the opportunity to quickly jump in and be a part of the experience.

References:
[NUI Identity](#)

Resources:
[Using NuiIdentity Effectively](#)
[ActivePassiveSkeletons Sample](#)

Multimodal Gameplay: Sensor Fusion

Some of the most magical experiences come from utilizing multiple inputs (and multiple outputs) simultaneously. The titles that leverage depth, color, skeletal, and audio input streams in cooperation, while providing direct and actionable visual and audible feedback to provide intuitive and reinforcing gameplay, will give the user the most complete, immersive, and satisfying control over the player's experience.

Resources:
[Menu Using Handles Sample](#)
[Menu Using Hover Sample](#)
[Depth Map Particle Stream Sample](#)
[Theremin Sample](#)

Explorative and Accessible Nature of Gameplay

Some of the best and most effective titles will have an intuitive and instruction-free control schema. Kinect provides an opportunity and expectation for a much broader range of skill levels and prior gaming experience among players. With Kinect, the title can learn from, and adapt to, the player's actions rather than training the player to conform to a rigid set of expectations. Training the game to respond to the user's intent rather than vice versa is one of the true game-changers of Kinect titles.

Resources:
[Driving Gesture Filters Sample](#)
[Flight Sample](#)
[Running Sample](#)
[First Person Sample](#)
[First Person Natural Sample](#)
[Forward Movement Sample](#)
[Tank Simulator Sample](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Natural Environs: Understanding and Developing for the Kinect Play Space



By Scott Selfon, Lead Program Manager
Advanced Technology Group (ATG)

Developing titles using the Xbox 360 Kinect sensor array requires rethinking the typical game development environment. Using cubicles, shared workspaces, and small offices to represent a typical gameplay environment can lead to additional challenges in acquiring and understanding incoming visual data, and they rarely represent the end-user's typical space. This document details recommended conditions and best practices for understanding the end user's play space, establishing a workspace, and positioning and utilizing the camera.

Kinect Sensor Array Specifications

The Kinect sensor array is expected to be placed in front of the user's play space, typically above, below, or on the television. This, of course, still represents a wide potential range of heights, and even distances since the camera could be on a shelf, a table, the television, or other surface. The goal for the Kinect sensor array is to work appropriately within the following constraints:

- Positioned heights between 1 foot (0.18 meters) and 6 feet (1.8 meters)
- Distances from flush (0 feet/meters) to 1 foot (0.3 meters) in front of the screen. (The expectation is that the sensor array would not typically be placed behind the screen since the screen itself or other surfaces could begin to obstruct the sensor array's view.)

Understanding the optical specifications can help mitigate these potential variations in position. Note that, where possible, these are practical (rather than theoretical) specifications, and may continue to change because of hardware or software revisions.

Table 1. Sensor array specifications and associated values and ranges

Sensor array specification	Values and ranges
Resolution, in pixels	Color: 640×480 pixels, Depth: 320×240 pixels
Field of view, horizontal	57.5 degrees
Field of view, vertical	43.5 degrees
Mechanized tilt range, vertical	± 27 degrees (Based on absolute horizontal, regardless of camera base tilt)
Manual pan range, horizontal	± 45 degrees
Depth sensor inner limit	2.62 feet (0.8 meters)
Depth sensor outer limit	13.12 feet (4.0 meters)
Practical depth inner limit	3.93 feet (1.2 meters)
Practical depth outer limit	11.48 feet (3.5 meters)
Sensor height from base of sensor array device	2.5 inches (0.0635 meters)

The sensor array's field of view (FOV) describes a frustum in 3D space. This leads to a play space where the user can be detected by the camera, with a more constrained rectangular area directly in front of the sensor array, and a larger rectangular area far from the sensor array. Additionally, based on the sensor array's position (its distance from the screen, behind or in front, as well as its angle of tilt and vertical position), the size and characteristics of the actual play space may vary significantly.

Figure 1. Side view of a lower camera position and its coverage of a 6-foot (1.8 meter) adult and a 3.3-foot (1 meter) child inside the practical play space distance of 3.9 feet (1.2 meters) to 11.5 feet (3.5 meters)

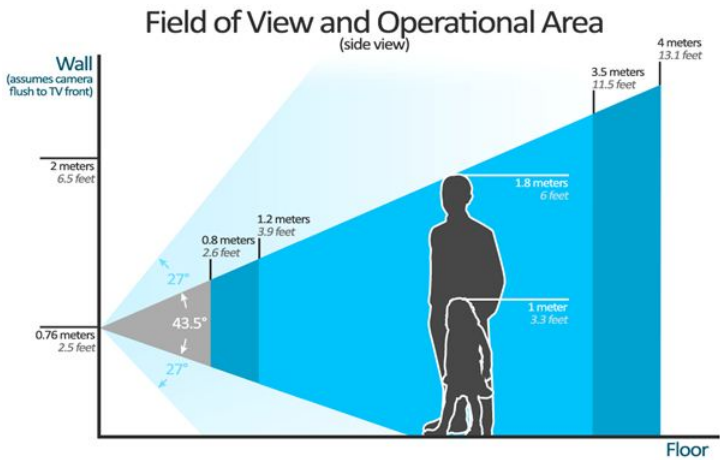
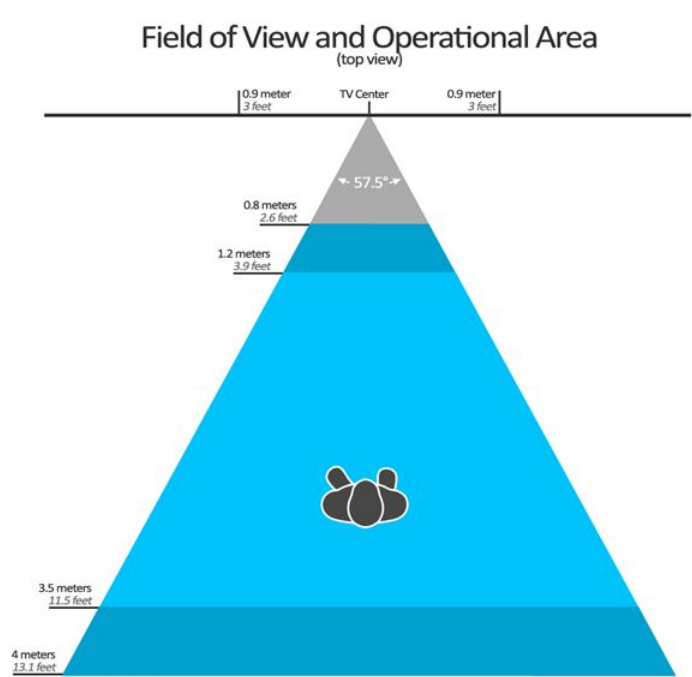


Figure 2. Top view of sensor array's field of view, with practical play space highlighted in light blue, a distance of 3.9 feet to 11.5 feet from the sensor array



Defining the "Sweet Spot"

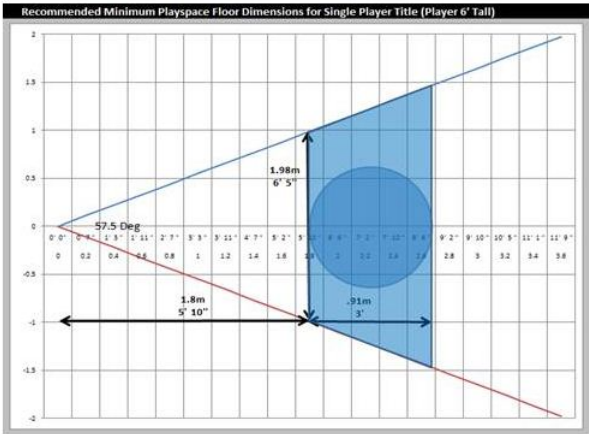
While the overall sensor array field of view could be considered the play space, its boundaries provide a variety of potential challenges:

- Decreasing ability to contain player in field of view, and increasing potential for self-occlusion at the near distance
- Increasing noisiness/reduced quality of depth data at the far distance
- Handling players who partially or wholly move out of the field of view (jumping, arms swinging out of the field of view, walking out of the field of view, and so on)

These challenges are magnified by the dual axes of player height and sensor array placement height, which can make the playable area hard to generalize.

It is desirable to define a space where the majority of worldwide players will have a great experience, regardless of vertical position of the sensor or their height. This space is called the *sweet spot*, a circle with a radius of 1.5 feet (0.46 meters), whose front edge is 5.8 feet (1.8 meters) in front of the sensor.

Figure 3. Illustration of location of sweet spot (circle) in relation to the field of view



The sweet spot is the position consumers are guided to through a variety of avenues, including the out-of-box experience (OOBE) of the Kinect sensor array, the troubleshooter, and the Dashboard.

The sweet spot is derived from several assumptions and goals:

- Users typically are between 3.25 feet (1 meter) and 6 feet (1.83 meters) in height
- Eighty percent (head relative) or more of the player should be within the vertical field of view to ensure a good gameplay experience
- The sensor array typically is placed horizontally centered with the screen
- The sensor array typically is positioned at a height of between 1 foot (0.18 meters) and 6 feet (1.8 meters)

While the sweet spot is designed specifically to enable great experiences regardless of sensor height, positioning the sensor array at the height of the player provides the best combination of perspective, skeletal tracking, and identity system performance.

Table 2. Suggested height guidance for sensor array placement

Description	Value
Maximum height from floor	6 feet 0 inches (1.83 meters)
Minimum height from floor 1 foot	(0.18 meters)

Figure 4. Measurement of sensor array height is relative to the base; actual sensors are approximately 2-1/2 inches higher

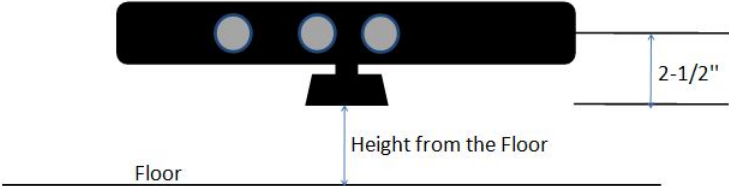


Figure 5. Sensor array perspective at height of 6 feet of player in sweet spot



Figure 6. Sensor array perspective at height of 2 feet 6 inches of player in sweet spot



Although the sweet spot provides the ideal player experience, players also can play at distances farther from the sensor—up to 11 feet 6 inches (3.5 meters). Players shorter than 6 feet (1.8 meters) in height may be able to be closer to the sensor; players taller than 6 feet (1.8 meters) may need to stand further back. The sweet spot presumes a one-player-at-a-time experience. If two players play simultaneously side by side, depending on actions they must perform, titles might wish to implement similar 1.5 foot (0.46 meter) radius circles for each, moving the two sweet spots backward to avoid leaving the field of view.

Sensor Array Tilt and the Sweet Spot

The Kinect sensor array possesses a vertical tilt motor, which allows for the current vertical field of view to be adjusted as needed. The tilt angle is configured by the consumer OOB, as well as the Troubleshooter. These experiences establish a tilt motor position that provides the optimal experience within the sweet spot, and takes advantage of both user feedback and sensor data (such as the floor plane, if present) to make these adjustments.

Titles can query the current tilt angle via the **NuiCameraElevationGetAngle** function. Note that retail titles cannot directly control the tilt by policy, as this is managed via the aforementioned user-system interfaces. During development, titles may use **NuiCameraElevationSetAngle**. (Unusual gameplay styles, such as requiring the player to lie on the floor, may be enabled by requesting an exception; title developers interested in implementing such scenarios should contact their Developer Account Manager.)

Understanding Dynamics of the Play Space

Once the sensor array is placed in a given position, it is often helpful to visualize the play space and sweet spot by taping off the environment's boundaries on the floor, and even potentially using string, pipes, or other materials to delineate the three-dimensional space. These can help inform the developer of key questions to answer when developing experiences that are enabled for natural input:

- To what extent can the player move about the play space?
- Is any vertical movement required (for example, jumping) that could require title attention and instruction at the closest portions of the play space?
- Will the title's gestures and poses support two players interacting at the same time?
- How does the title give feedback to users who may instinctively approach the screen as they play?

Seating Area

With the play space now visualized, an immediate challenge we might encounter is the fact that a living room's seating area often falls squarely within the play space. Couches are frequently positioned in what might horizontally be the widest portion of the play space. In some scenarios, players need to move their seating to enjoy the gameplay experience, but even typical seating locations will be within the sensor array's detectable play space. While this may restrict the player's ability to back up in standing experiences, this is often desirable for providing seated interactions.

It's also worth having a brief discussion here about the general background environment. The depth sensor depends on reflected light to read depth information. Therefore, objects that are highly reflective (mirrors and shiny metal) or highly absorptive (fluffy and/or dark materials) may not be registered by the depth sensor as successfully as other objects. Of course, if these items are background elements, the title would typically need to reject them, regardless.

Lighting Characteristics

Proper lighting—neither extremely dark nor extremely bright—is important for capturing visuals in the RGB camera, though less so for the depth camera. Incandescent, fluorescent, and natural lighting provide no special obstacles as indirect light sources. For direct light sources, developers and gamers need to take care not to point intense and constant light sources at the camera, because this can blind either or both visual sensors; a setting sun, for example, requires curtains or blinds to be drawn when the camera is facing west. The depth camera functions adequately in typical lighting, as well as in reduced lighting, although in near darkness both cameras have increased noise in their signals.

Development Environment

The sensor array's characteristics, described earlier, should provide the developer some degree of guidance regarding workable player position. If an office or cubicle is acting as the capture environment for natural input, developers should attempt to simulate typical gameplay as much as possible. Where this simulation is impractical, developers should be aware of potential differences in behavior based on the more physically restrictive play space.

Business environments can also provide challenges not typically seen in a home environment. Reflective whiteboards can scatter light and lead to reduced depth performance. Coworkers and objects passing nearby can cause a loss of confidence in the accuracy of incoming data. Desks and overhanging objects can create depth shadows, gaps where the sensor array cannot accurately determine depth data. Where possible, reduce these potential distractions.

Programming for a title that uses the Kinect sensor array can also be inherently awkward from a workflow perspective. The positional change from typing at a keyboard to standing and delivering poses and gestures as a player can become a gating factor to integration. Further, development environments are frequently not configured like living rooms, and may be unable to expose the full play space for testing. And, if any real-time monitoring, debugging, or video capture is required, screens and keyboards must be creatively positioned around (or just out of) the sensor array's field of view.

For these reasons, we recommend that you explore a variety of configuration changes to retain agility in designing a title.

- **Standing workstations.** In environments where a single programmer needs to program and act as the player, a non-seated workstation can be an effective way to ease the transition from coding to playing.
- **Mobile workstations.** Consider building workstations that can be physically moved between a more traditional office environment and a more typical gameplay environment. Wheeled desks that can hold a large television screen, along with a development computer and monitor, can make for more flexible development environments.
- **Record and playback functionality.** Using Xbox Studio or custom title-implemented tools, the sensor array data can be recorded for predictable playback. This allows for a single performance to be reused as needed to exercise changes in title code.
- **Remote controls.** Particularly if PC-triggered capture (video, PIX, and so on) or other toggle-style functionality is required during live gameplay, consider adapting a presentation mouse or other remotely activated device. Infrared data delivered by remote controls within the sensor array's field of view does not typically blind the sensor or otherwise adversely impact the incoming data stream.
- **Tag team.** Studios may explore having programmers work more closely with game designers or testers. The programmer remains at the computer while the designer or tester acts out and plays the title.
- **Mechanized players.** Several studios have successfully used posed mannequins or other simulacra to avoid the need for long-term human testing. More fully mechanized players allow for repeatable actions and gestures, though arguably the variation in even repeated human movement is an important and difficult challenge to address in natural input game development.

This is certainly an incomplete list, and we welcome feedback on workflow modifications that have worked for your titles at gamedev@microsoft.com.

Summary

With full-body game control comes a need to revise (in some cases quite dramatically) gameplay testing and development environments. Our hope is that this white paper provides you with some ideas for environment configuration that balance ease of development with practicality. We encourage you to contact Microsoft Game Developer Support Desk at gamedev@microsoft.com with any questions or comments about the topics presented here, and with challenges specific to your own title or engine, and to share your own experiences and best practices.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Tilt-a-Whirl: Kinect Tilt, Play Space Management, and You



Or "What do you mean you don't live in a Mansion?"

By Simon Cooke, Senior Software Engineer

Advanced Technology Group (ATG)

Introduction

The Kinect sensor array features a tilt mechanism that is under automated system and title control. This allows your title to tilt the play space, and to make tradeoffs depending on the player's height, the number of players, and the need for your title to be able to track different portions of that player.

The aim of this document is to teach you everything you need to know about how to effectively handle a variety of different player heights and play spaces so that you can maintain a high-quality end-user experience.

Why Tilt At All?

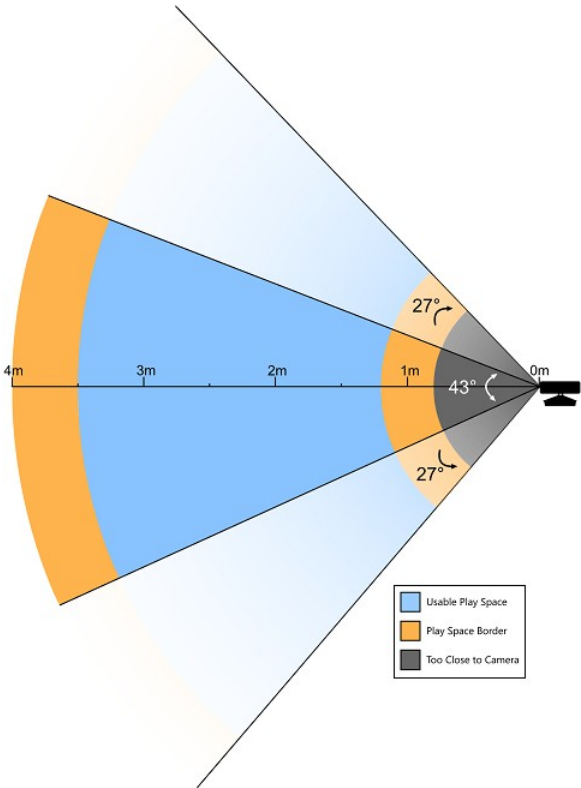
The Kinect depth sensor has a horizontal field of view (FOV) of about 57 degrees, and a vertical FOV of about 43 degrees (these are not exact figures; actual values can change from camera to camera). This represents the optimal tradeoff among cost, image quality, and FOV.

The effects of this are threefold:

- Players of reasonable height must stand at least 6 feet back from the depth sensor. Two players must stand back even farther.
- The sensor array should be placed between 2 feet (0.61 meters) and 6 feet (1.83 meters) from floor level; placing it at eye level provides the best results. The specified distances maximize the available play space and improve the quality of biometric sign-in results (Identity). If more than one person will be using the system, it is best to place the sensor array at the average eye level of the crowd.
- The sensor array axis needs to be adjustable to accommodate different sized players and play spaces.

To make the sensor array axis adjustable, a motor was added to the system. It can tilt the sensor array, giving a total *potential* FOV of 95 degrees.

Figure 1. Vertical FOV, including tilt of ±27 degrees



The blue (center) area of the FOV illustrated in Figure 1 indicates the effective, usable play space.

Predefined Constants

The FOV constraints and the tilt limits are defined as part of the NuiApi.h header file. Relevant constants are:

Constant	Description
NUI_CAMERA_ELEVATION_MAXIMUM	The maximum tilt of the sensor array in the direction of the ceiling (+27 degrees).
NUI_CAMERA_ELEVATION_MINIMUM	The maximum tilt of the sensor array in the direction of the floor (–27 degrees).
NUI_CAMERA_DEPTH_NOMINAL_HORIZONTAL_FOV	The horizontal FOV of the depth sensor (~57 degrees).
NUI_CAMERA_DEPTH_NOMINAL_VERTICAL_FOV	The vertical FOV of the depth sensor (~43 degrees).

You should use these constants in your code in preference to hard-coding raw numeric values. If these values change in later versions of the XDK, your code will not need to be modified.

Play Spaces – Far and Near

There are two types of play spaces defined for Kinect titles: the near play space and the far play space.

Near Play Space

The near play space is the area between the depth sensor and the beginning of the far play space. If the player stands in the near play space, it is unlikely the player's entire skeleton will fit in the FOV.

Because good skeletal tracking requires seeing the player's head over time, out of necessity you will have to clip the body of a player with the FOV if that player is within the near play space area.

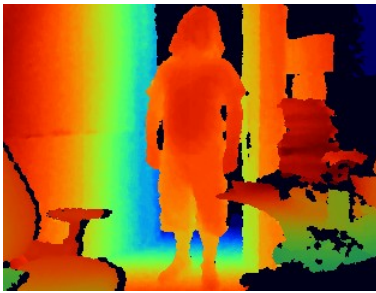
Tip

It is possible to stand too close to the sensor array; 0.8 meters is the absolute closest a player can be to the sensor array, and 1.2 meters is the theoretical "useful" minimum distance.

Far Play Space

The far play space begins with the area where the sensor array—if tilted so that the top of the FOV is at the top of the player's head—can detect the whole body of a player of a given height in the FOV.

Figure 2. A player standing at the beginning of the far play space



Note that the player in Figure 2 is standing on the edge of the far play space area when tuned for the player's height. By default, we assume the player's height is 5 feet 11 inches (180 centimeters) unless the Kinect tuner has been run. Also by default, tilting for far space *only* uses the tuner's values, and does not use active data from the depth buffer to tilt for potential players.

The far play space is the optimal area for single player gameplay that requires non-inferred, high-quality tracking of all joints. If the player moves back even farther, this allows for hands-over-head gameplay, or for two players to play a game simultaneously.

Note

According to a 2000 study by the CDC, 1 in 4 males are taller than this; it is a reasonable compromise, and is only an initial default.

http://www.cdc.gov/growthcharts/html_charts/statage.htm

What Play Space Should I Design For?

The play space you design for will depend on your title. Work with your game designers to determine the space that a player will need.

Things to consider:

- How much longitudinal motion (front/back) do you want or need to handle?
- How much lateral motion (side to side) do you need to cope with?
- How many players will be playing your title simultaneously?
- Can people jump in and out of your title at will?
- How does your title handle children? Adults?
- Many of your users may have a small play space in which to play their games. Can your title handle being played in a bedroom or in a small den?

This is covered in more detail later, but the basic advice is:

- Set up a test area.
- Tape off the edges of the area by using masking tape to determine where the limits are while your designers practice.

Note

In early testing—before you decide on a play space—you may want to fire up the Simple Skeleton Tracking sample or NUIView and experiment with a series of gameplay moves while keeping your head in the frame. While doing this, consider using salt or chalk to form a line on the floor around the play space so that you can see if designers cross the boundary. Using salt or chalk has the added advantage of keeping out most minor demons, but note that this does not work against zombies. Remember that zombies require removal of the head or destruction of the brain.

- Design your game and play space so that it can be played as *near as possible to the display* while still allowing the full range of motion for your title.
- Do not be afraid to crop the lower body. Most titles do not need full foot joint information. Titles can use the inferred bones, or can ignore feet in their gesture recognition by using the knee joints instead.
- Do not be afraid to crop above the head. Most titles can infer hand-over-head positions based on accurate elbow joint readings, and do not need precise hand or wrist joint position information.

TCR Requirement: NUI Playable Distance

The NUI Playable Distance requirement specifies that all titles must be playable by a single player up to 1.83 meters tall (6 feet or 183 centimeters), standing 1.8 meters (5 feet 11 inches or 180 centimeters) from the Xbox 360 Kinect sensor array.

This means that:

- Shorter players also should be able to play from that spot.
- You may support players standing *nearer* or *farther* from that spot, but at a minimum, you must support players standing there.
- You *must* have a single player mode for your game just in case players' play spaces are too small to support multiplayer gameplay.

In short, the TCR is the bare minimum level of play space management your title must support. If you cannot test and support a variety of different scenarios, you at *least* should aim to meet the requirements of the TCR.

Important

TCRs may be subject to change. Always refer to the latest version of the TCR documentation at <https://developer.xboxlive.com/en-us/xbox/certification/requirements/Pages/TCR.aspx>.

The default initial tilt that occurs when a title starts matches the TCR requirements described. If you are unwilling to do the extra work to manage your play space, you could do worse than avoiding calling the tilt APIs entirely.

Additional Reading

Scott Selson's white paper, *Natural Environments: Understanding and Developing for the Kinect Play Space*, is available at <https://developer.xboxlive.com/en-us/xbox/development/education/Pages/WhitePapers.aspx> under the Kinect & General Input section. The white paper gives details about how play spaces are chosen, the typical sizes of play areas in user homes, how to set up your office as a play space, dynamics of the play space "sweet spot," and more.

OBE and the Kinect Tuner's Roles

The Kinect tuner (which can be displayed by calling `XShowNuiTroubleshooterUI`) and the initial OBE (out-of-box experience) screen calibrate the system for a typical player.

OBE Tilt

The OBE performs a default tilt to far play space when it fires up. It tilts to optimize the depth sensor FOV for a player with a height of 5 feet 11 inches (180 centimeters) standing 2.13 meters away from the sensor array.

Kinect Tuner

The Kinect tuner is available at any time from the Guide (the title can also show it by calling `XShowNuiTroubleshooterUI`), and allows the player to tune the system mid-game.

Figure 3. The Kinect tuner option in the Guide

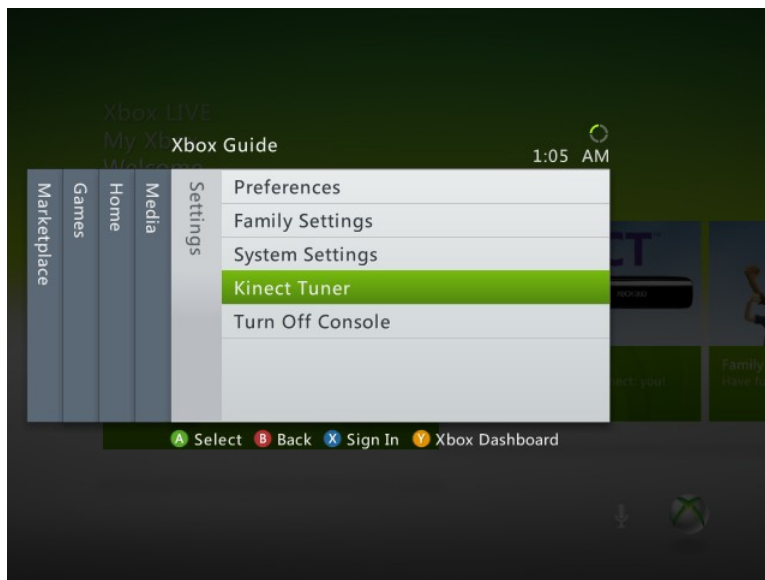


Figure 4. Different tuner calibration options

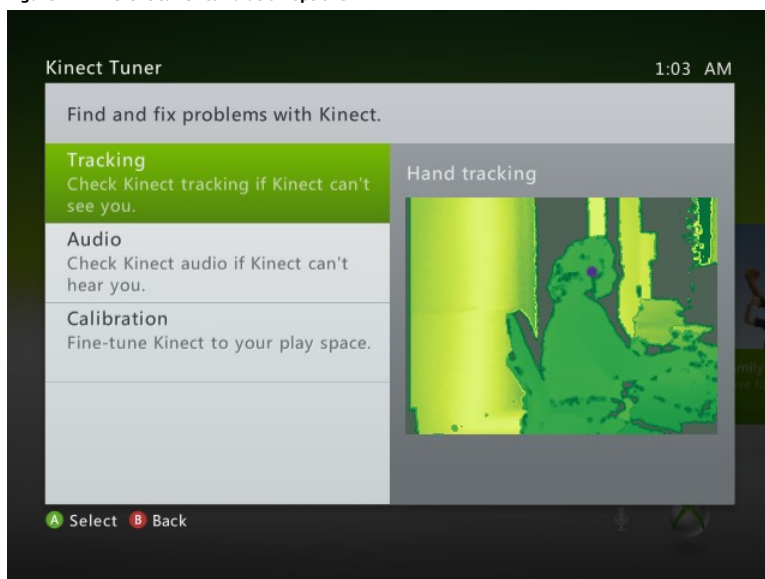
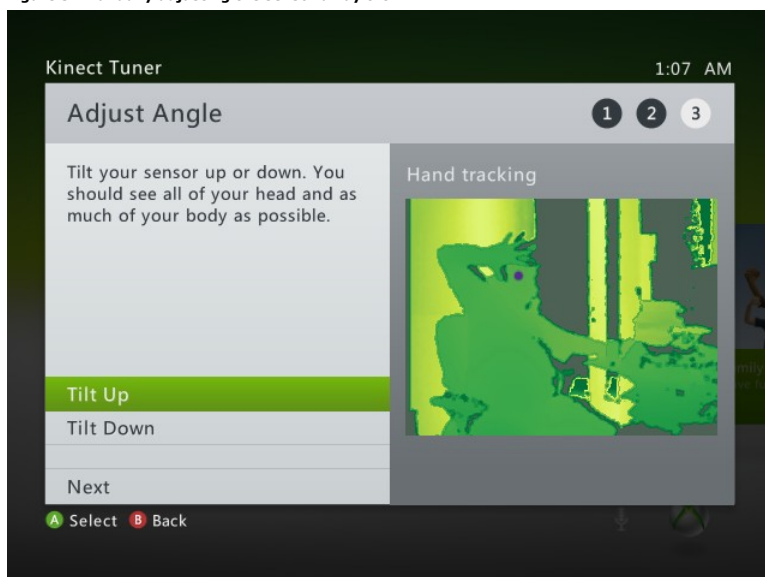


Figure 5. Manually adjusting the sensor array tilt



The tuner automatically tries to tilt for the player in front of the sensor array, and then allows the player to adjust the tilt manually until there is a good view on-screen. This configuration then is stored in the system as the new default far play space tilt setting.

The actual information stored is:

- The height of the player.
- The angle the sensor array was tilted to for the player.

This information is stored so that if the sensor array is moved and it cannot see the floor, there is still a reasonable set of default parameters on which to base tilts. The **NuiCameraAdjustTilt** API uses the height information exclusively to perform far play space tilts.

NuiCameraAdjustTilt API

```
NUIAPI DWORD NuiCameraAdjustTilt(
    NUI_TILT_FLAGS TiltFlags,
    FLOAT SpaceAboveHeadMeters,
    FLOAT FarSpaceDistanceMeters,
    FLOAT PreferredPlayspaceDistanceMeters,
    NUI_TILT_OBJECTS* pTiltObjects,
    XOVERLAPPED* pOverlapped
);
```

This is the primary API call by which tilt operations and a number of other utility functions can be performed. We'll go into more detail about how you can use it.

Important notes about the API

- This function can take up to 33 ms to return (it operates on the next depth frame to arrive).
- It requires that you enable the depth stream in your call to **NuiInitialize**.
- You should call **NuiCameraAdjustTilt** from its own thread, or call it asynchronously by using overlapped I/O.
- You should not call **NuiCameraAdjustTilt** on any thread that controls when frames are presented—for example, your Render thread—in a blocking fashion.

Failure to follow these guidelines can cause your title to hang because depth frames will not be pumping through the system.

Also, both the depth buffer data stream and the skeletal tracking data stream are *disabled* when the camera is in motion, and cause a reset of the skeletal tracking pipeline.

Identifying Potential Players

The **NuiCameraAdjustTilt** API provides the ability to find potential players in the depth map for little cost (2 ms of CPU per frame, and approximately 100 K of memory, which is released after the call is complete), and can be used both in depth only scenarios and with the full skeletal tracking pipeline.

The function evaluates the data available in the depth buffer without using skeletal tracking, and identifies potential player blobs that lie within the FOV of the depth sensor.

For each blob it finds, the following information is returned:

- The X and Z coordinates of the center of the blob relative to the sensor array.
- A flag indicating if the top of the player is cut off by the top of the FOV of the depth sensor (NUI_TILT_OBJECT_FLAG_INTERSECTS_TOP).

Anything that is roughly the same proportions as a human is reported as a player; this could include inanimate objects such as table lamps or chairs.

The function returns up to six potential objects, and always operates on the next available depth frame. Because it is a low-resource API, it does not cache any depth data, and has to wait for a new frame to arrive before it can do its work.

Preferring Locations and Rejecting Bad Matches

It is possible by using the API to reject bad matches and to prefer one specific location. If a location is marked with NUI_TILT_OBJECT_INFLAG_PREFERRED in the **NUI_TILT_OBJECTS** array, the potential player nearest to that location is used to control the tilt in preference to any other players who might be found in the scene. Only one location can be marked as preferred.

Similarly, if a possible player match is found at a location marked as bad (NUI_TILT_OBJECT_INFLAG_IGNORE) the match is ignored. This can be useful if the Tilt API repeatedly thinks that inanimate objects are acting as people, or to ignore players when tilting. By default, if no locations are flagged, the player nearest to the camera is preferred.

If a title has a specific, desired "sweet spot" to place the player in, it should pass this value in as a location in the **NUI_TILT_OBJECTS** array with the PREFERRED flag set. This becomes particularly important in two-player scenarios when switching between two players of different heights. To tilt for a specific player, tag the player as being preferred before initiating a tilt.

Finding the Floor

It is often useful to be able to determine the height of the sensor array above the floor plane. You can use this information to calculate the height of players, to calculate potential play spaces for your title, and possibly to engage players in augmented reality gameplay. There are two main ways to obtain this information:

- The most accurate way to obtain the floor plane from the system always is via the **NUI_SKELETON_FRAME** returned by the system. There are, however, some caveats:
 - If you want to use the floor plane—regardless if the sensor array is tilted so that it can see the floor—pass NUI_INITIALIZE_FLAG_EXTRAPOLATE_FLOOR_PLANE into **NuiInitialize** on startup (more on this later).
 - If the player moves the sensor array, the floor plane could be lost.
- The second option (probably worth using only if you're not using the full skeletal tracking pipeline in your title) is to use the sensor elevation information returned in the **NUI_TILT_OBJECTS** structure that is returned by the **NuiCameraAdjustTilt** API (passing in the NUI_TILT_FLAGS_NO_TILT flag). This at least tells you the height of the sensor array above the floor plane, if not the actual tilt of the floor.

Sweeping (for) the Floor

The sensor array automatically sweeps to find the floor plane the first time a title loads and calls **NuiInitialize**.

Name	Action
First call to NuiInitialize	• Sweeps to find the floor. • Automatically tilts to far play space.
Subsequent calls to NuiInitialize	• Automatically tilts to far play space.

In both cases, the far play space tilt uses settings stored by the Kinect tuner if the tuner has been run. Otherwise, it uses the default tilt for a player with a height of 5 feet 11 inches (180 centimeters) that is 2.13 meters away.

Losing the Floor Entirely

There are certain circumstances in which the floor plane might be invalid (all components of **vFloorClipPlane** are 0):

- Not finding the floor during a sweep of the room.
- The sensor array has been moved or shaken recently.

If the sensor array was moved or shaken, then the next time you call **NuiCameraAdjustTilt** or **NuiInitialize** the sensor array performs a floor sweep.

It is best to assume that:

- The floor plane value is available and valid most of the time (except for a short period after **NuiInitialize** is called).
- It could become unavailable at any time.
- If using the floor plane, situations may arise that require you to hold gameplay, but preferably in a more subtle fashion than showing an actual pause menu.

Automatic Tilt Operations

If the sensor array is not tilted appropriately for your title, or if you would like to fine-tune its FOV depending on your title's style of gameplay, you can call **NuiCameraAdjustTilt** to make adjustments. This API does not take an absolute pitch angle as an input. Instead, the API uses data taken from the Kinect tuner, the OOBE setup process, and the current depth buffer to accommodate the players standing in front of the sensor array.

The tilt operation, by default, acts on the best candidate player it can find nearest to the camera. This behavior can be modified by your title (see Identifying Potential Players). There are several ways to modify **NuiCameraAdjustTilt**'s behavior to specify the exact constraints the sensor array should use when tilting.

Max Headroom

It is possible to change how much space the system attempts to preserve above the top of the player's head in the depth buffer by passing in your own custom value or one of the following values as the *SpaceAboveHeadMeters* parameter.

Name	Action
NUI_TILT_HEAD_SPACE_FULL_SKELETON	Sets the top of the frame as near as possible to the top of the player's head to get as much of the body as possible (0.0 meters).
NUI_TILT_HEAD_SPACE_UPPER_BODY	Sets the space above the player's head to 0.15 meters.
NUI_TILT_HEAD_SPACE_HANDS_OVER_HEAD	Sets the space above the player's head to 0.3 meters.

The *SpaceAboveHeadMeters* parameter tunes sensor array tilting in *both* near play space and far play space scenarios. You should choose which you want to use depending on your gameplay scenario, or you can use your own custom value from **–0.15 meters to 0.5 meters** (approximately –6 inches to 19.5 inches).

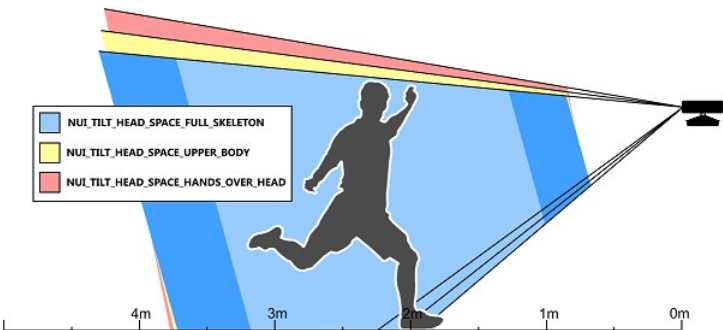
If your title never requires players to raise their arms above shoulder height, for example, you might specify **NUI_TILT_HEAD_SPACE_FULL_SKELETON**. However, if players need to hold up their arms and you need to track certain moves, consider using **NUI_TILT_HEAD_SPACE_HANDS_OVER_HEAD**. There is a compromise between the two settings. **NUI_TILT_HEAD_SPACE_UPPER_BODY** is midway between the two extremes. As ever, experiment with the different clearances, and see which matches the section, level, mini game, or entire title you are building.

Note

In reference to **NUI_TILT_HEAD_SPACE_HANDS_OVER_HEAD**, keep in mind that you do not necessarily need to track the player's entire arms during hands-over-head gameplay; you may be able to get away with tracking only the elbow joints.

You might notice that all of these options require that a player's head remain inside the FOV. This is broadly the case; the depth sensor has to see a player's head to identify it as human or not. Although we can handle scenarios where a player's head might pop out of the FOV for a few frames, skeletal tracking requires the head to be visible for both the initial lock and for longer term tracking.

Figure 6. The three different space over head constants in use

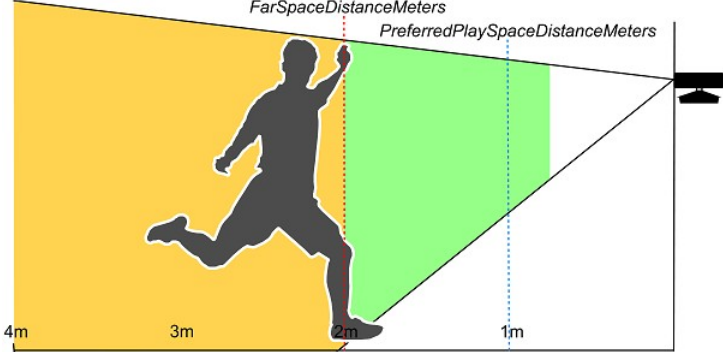


Near Play Space and Far Play Space Parameters

Name	Action
FarSpaceDistanceMeters	This value defines the starting point for "far space." If a player is nearer than this, the sensor array is tilted based on their height rather than the default far space settings. If you do not want to specify a value, use the default value NUI_TILT_DEFAULT_FAR_SPACE_DISTANCE , which is about 2.13 meters from the sensor array.
PreferredPlayspaceDistanceMeters	This value defines how near to the sensor array the player is expected to stand in the play space during normal gameplay.

If the player is nearer to the sensor array than the *FarSpaceDistanceMeters* value, then the sensor array tilts as if the player is standing *PreferredPlayspaceDistanceMeters* away. Figure 7 illustrates the effect.

Figure 7. Play space distance parameters in action



In Figure 7, *FarSpaceDistanceMeters* is set to 2.0 meters and *PreferredPlaySpaceDistanceMeters* is set to 1.0 meters. The player is shown standing in the far play space (shaded yellow). The sensor array tilts as if the player is standing at 2.0 meters. If the player were standing nearer to the sensor array than the *FarSpaceDistanceMeters* threshold value, the player would be standing in the near play space (shaded green).

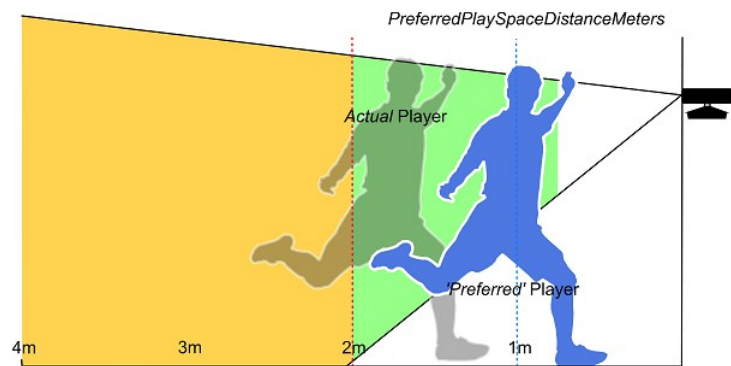
Figure 8 shows how tilt changes if the player is standing in the near play space. The system does not tilt based on the actual player position; instead it tilts as if the player is always standing at the *PreferredPlaySpaceDistanceMeters* threshold. This distance is treated as the front edge of the near play space. For example, if the camera is tilted with *SpaceOverHeadMeters* set to 0 meters (full skeleton mode) and the player moves nearer to the camera than the preferred distance, the top of the player's head disappears.

This behavior changes if the player is shorter than the camera height and instead of tilting as if a player is standing at the *PreferredPlaySpaceDistance*, the camera tilts as if the player is still standing at the *FarSpaceDistance*. This was a change in behavior in the July QFE release; without this, the players head disappears when the player is standing in the near play space in this scenario.

Tip

To make the calculation more robust, we extrapolate the player's location to both the far play space distance and the preferred play space distance, and tilt based on whichever of those two points would be higher relative to the camera.

Figure 8. Near play space player tilt operates on a virtual player standing at the preferred play space distance, provided the camera is below the player's head



Overriding Tilt Calculations

Forcing Near Play Space Tilts to Use the Player's Current Position

If you want to tilt explicitly based on where the player is standing in near play space when you call the tilt function, pass 0.0 as your *PreferredPlaySpaceDistanceMeters* value. This disables the extrapolation.

Forcing Far Play Space Tilts to Use the Player's Height

By default, tilts operating in the far play space region use the Kinect tuner's stored values to support as broad a range of players as possible without repeatedly tilting the sensor array. To force a far play space tilt to operate on the target player's actual height instead of the tuner values, pass the `NUI_TILT_FLAGS_OVERRIDE_TUNER` flag.

Forcing Tilts to Always Use the Player's Height

When the sensor array is tilting for a player in the near play space, it always uses the player's actual height as currently estimated by the depth buffer. By default, tilting for objects in the far play space uses the Kinect tuner settings.

To force the system to always tilt to the exact height of a player, pass the `NUI_TILT_FLAGS_FORCE_NEAR_SPACE` flag, and set the *PreferredPlaySpaceDistanceMeters* parameter to 0.

Play Space Override Flags

There are two other flags that can be used to override the API's decision about which space the player is standing in.

Name	Action
<code>NUI_TILT_FLAGS_FORCE_FAR_SPACE</code>	Forces the sensor to treat the player as if the player is standing in the far play space.
<code>NUI_TILT_FLAGS_FORCE_NEAR_SPACE</code>	Forces the sensor to treat the player as if the player is standing in the near play space.

Both flags are mutually exclusive, and force the sensor array to tilt as if the player is standing in the respective space, regardless of where the player actually is standing.

Whoa There Cowboy – This Is Not A Night at the Roxbury

You should not call `NuiCameraAdjustTilt` to actively tilt the motor too often. When this function is called more than 15 times without taking a break, the API returns `ERROR_RETRY` or `ERROR_TOO_MANY_CMDS` until you wait 20 seconds and try again.

Regardless of system-imposed limits, you should still avoid tilting unless absolutely necessary. Tilting interrupts the skeletal tracking pipeline and depth streams; without data your title will need to halt gameplay during the tilt operation.

You can call the API with the `NUI_TILT_FLAGS_NO_TILT` flag as often as you like. The call will not complete until the next depth frame arrives, and will block if made synchronously.

Common Gameplay Scenarios

In What Play Space Should I Design For? we covered some basic advice on how to think about tilting and how to use of your play space. Let's dive in a little deeper.

Control scheme	When to tilt	Recommended play space	Notes
Scenario: Single Player			
Fixed full skeleton	Start of level/session	Far	Consider using the defaults and not calling the Tilt APIs.
Fixed partial skeleton	Start of level/session	Near	Infer gestures from as few joints as necessary to allow greater freedom in player positioning/more cropping.
Varying	When change of control scheme requires it.	Depends on control specifics (try to enable near play space scenarios).	Football for example: Consider moving the player back into far play space to take a kick. Keep the player in near play space for normal gameplay.
Scenario: Single Player, Easy In/Easy Out			
Any	When new player shows intent, and is cropped, or control scheme changes.	Depends on control specifics (try to enable near play space scenarios).	Consider tracking the tallest player and tilting only when the height of a new player is radically different.
Scenario: Multiple players, Alternating, One at a Time			
Any	When control passes to a different player (or control scheme changes).	Depends on control specifics (try to enable near play space scenarios).	Consider tracking the tallest player and tilting only when the height of a new player is radically different.
Scenario: Multiple players, One at a Time, Easy In/Easy Out			
Any	When control passes to a different player, or a new player shows intent.	Most likely far play space to allow players to change places.	Consider tracking the tallest player and tilting only when the height of a new player is radically different.
Scenario: Simultaneous Multiplayer			
Any	At the start of the game session, or when control scheme changes.	Far	Use the default far play space tilt in most circumstances. If necessary, tilt for the tallest player. Ask the shortest player to move back if the player is cropped (lower extremities inferred). Consider pausing the title and asking players to move apart if they are too close together.
Scenario: Simultaneous Multiplayer			
Any	When control changes or a new player shows intent.	Far	Seriously consider using the default far play space tilt and <i>not</i> tilting based on different players. Moreover, players need to stand in the far play space zone to avoid being too close together.
Scenario: Crowd			
Any	When necessary; tilt for average	Far	Seriously consider using the default far play space tilt and <i>not</i> tilting

case if possible.

based on different players. Use a combination of NUI_TILT_OBJECTS data and NUI_SKELETON_FRAME data to select candidate players.

When and When Not to Tilt

You should avoid calling the Tilt API if the player is standing too near (closer than 1.2 meters) to the sensor array. Calling the Tilt API could cause you to tilt the sensor array too high for normal gameplay when the player returns to the appropriate play space. Instead, you can choose to do nothing or can suggest that the player stand back from the sensor array.

If you can identify the height of a player and know that the FOV tilt is insufficient, ask the player to move back *before* tilting. Before you try to tilt, identify the limitations with the sensor array's current setup, and use a combination of player shepherding followed by the tilt if possible. This usually produces better results.

A title that requires the player to stand in the far play space should tilt for the default far play space when title or gameplay session starts, and then show on-screen feedback to move the player into position. See the [PlaySpace Feedback sample](#) in the Xbox 360 Development Kit (XDK) for a demonstration of one way to do this.

When there are multiple players playing—not side by side—in a single session, consider tilting for the tallest player and making adjustments only when *absolutely* necessary. Each tilt operation takes time, and time spent tilting is time spent not entertaining players.

If you cannot identify potential player candidates for a period of time (30 seconds), consider performing a default far play space tilt and leaving the sensor array in that position. Also consider showing some UI elements asking new players to step up. After a period of inactivity (perhaps two minutes), you may want to open your pause menu with the Kinect tuner option highlighted.

How Often to Tilt

It is recommended that you tilt only at the beginning of your title or at the beginning of each logical gameplay session. Tilting the sensor array disrupts the skeletal tracking pipeline and the depth image stream, which most likely will interrupt most gameplay sessions. If you need to tilt, consider showing some form of on-screen UI during the operation.

In multiplayer scenarios, tilting between each player's turn is allowed, and is recommended for some types of gameplay. You may wish to tilt between turns in multiplayer modes that operate in round-robin fashion. Tracking each player's gross location (found in **NUI_SKELETON_FRAME**) and their skeletal tracking IDs should allow you to track the location of individual players during gameplay sessions. When a player's turn starts, pass the player's location to **NuiCameraAdjustTilt** as the preferred location, and tilt for that specific player.

Kinect Tilt Cookbook

Most tilt-related API calls are unavailable while the Guide is showing, and fail immediately with an error of E_NUI_SYSTEM_UI_PRESENT if you try to call them.

Identifying if the Sensor Array is Moving

Call **NuiCameraElevationGetAngle** to determine if the sensor array is moving if you are not using the skeletal tracking pipeline. This function writes flag values to a DWORD that indicate if the sensor array is being moved via motor control, or if it has been shaken, bumped, or moved by hand. These flags are mutually exclusive; you will never see both MOTOR and DETECTED at the same time.

Flag	Description
NUI_CAMERA_ELEVATION_MOTOR_MOVEMENT	The camera was moved by the motor.
NUI_CAMERA_ELEVATION_DETECTED_MOVEMENT	The camera was moved by a person.

If you are using the skeletal tracking pipeline, then you can examine the **NUI_SKELETON_FRAME** structure that contains a **dwFlags** member. This member indicates if the camera has been moved by setting the NUI_SKELETON_FRAME_FLAG_CAMERA_MOTION flag value. You then can call the **NuiCameraElevationGetAngle** API for more details if necessary.

Stopping the Tilt Motor

To stop the sensor array in mid-move, your title can call **NuiCameraElevationStopMovement**. It may take a few frames for the skeletal tracking pipeline to resynchronize after a move, so skeletal data may not be immediately available.

Setting a Specific Tilt Angle

Setting a specific tilt angle is discouraged except for titles with specific tilt needs—for example, yoga titles where the player would be lying on the floor. Such direct control is handled via the **NuiCameraElevationSetAngle** API, which requires approval for use (and is locked behind a privilege bit). Contact your developer account manager for advice about what steps to take if your title cannot use the **NuiCameraAdjustTilt** API call.

Getting the Current Sensor Array Pitch

There are several ways to obtain the degree of tilt for the X-axis of the sensor array. Perhaps the easiest way is to call **NuiCameraElevationGetAngle**, which also conveys other information about movement of the sensor array. **NuiCameraElevationGetAngle**, however, only returns an integer value rounded to the nearest degree. For more accurate results, you can calculate the tilt based on the readouts for the normal to gravity returned by the skeletal tracking system.

```
For example:

FLOAT CalculateCameraPitch( const NUI_SKELETON_FRAME& frame )
{
    XMVECTOR& vNormToGrav = frame.vNormalToGravity;
    FLOAT fDegrees = XMConvertToDegrees( atan2( vNormToGrav.z, vNormToGrav.y) );

    return fDegrees;
}
```

Because of limitations with consumer-grade accelerometers, the results always will be somewhat noisy. If a steady result is necessary, it is recommended that you filter the returned value further, although it is already averaged over a window of 900 ms and, as such, is relatively stable.

Handling Sensor Array Tilt Activity

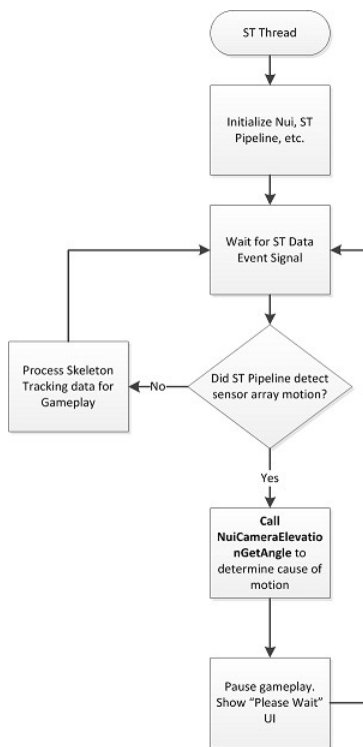
It is possible that the sensor array will be tilting under control of the Guide when your title starts up initially, or in response to a call made to **NuiCameraAdjustTilt**.

While the sensor array is tilting, skeletal tracking and the depth image stream are interrupted. For this reason, it is a good idea to provide feedback to the user when this happens. A robust way of handling this scenario is to build a state machine for the Kinect sensor. When you successfully call **NuiCameraAdjustTilt** and it returns ERROR_IO_PENDING—assuming you are calling the API asynchronously—to indicate that a sensor array tilt is in progress, you can enter a *Sensor Array in Motion* state. You also should enter this state if you call **NuiCameraElevationGetAngle** and its flags indicate sensor array motion, or if you obtain skeletal data from the system and the **dwFlags** member of **NUI_SKELETON_FRAME** indicates that the sensor array is moving.

When you detect sensor array motion, consider pausing gameplay for your title until the sensor array has settled and you are retrieving good skeletal data again.

The flowchart shown in Figure 9 shows one possible way in which your Skeletal Tracking thread might handle this scenario.

Figure 9. Handling sensor array motion in your skeletal tracking data capture thread



Using the Floor Plane as a Continuous Reference

Provided you pass the flag `NUI_INITIALIZE_FLAG_EXTRAPOLATE_FLOOR_PLANE` to **NuiInitialize**, the floor plane is available to you regardless if the floor is visible in the current depth sensor view. On title launch, the sensor array searches for the floor—tilting down if necessary until it can see it—and that value is cached by the system. This is useful, especially if you are performing skeletal retargeting operations and need to know the player's height. It also is helpful for tracking where the player is in relation to the play space and the sensor array FOV.

The API sets a **dwFlags** member (`NUI_SKELETON_FRAME_FLAG_EXTRAPOLATED_FLOOR`) in the **NUI_SKELETON_FRAME** structure. The flag indicates if the floor plane value is a synthesized value retained when the sensor array performed its initial sweep, or is a value calculated from a floor that is currently visible.

If you do not pass the extrapolated floor plane flag to **NuiInitialize**, an invalid (all components = 0) floor plane value always is returned if the depth sensor cannot see the floor.

Calculate the Horizontal Clearance at a Given Distance

If you need to calculate how much space a player has to move in (for example, to determine if two players can play), use the following formula:

$$c = d \tan\left(\frac{\theta_h}{2}\right)$$

where c is the horizontal width of the available play space at distance d from the sensor array along the sensor array's axis, and θ_h is **NUI_CAMERA_DEPTH_NOMINAL_HORIZONTAL_FOV**.

Estimate the Distance Required Between a Player and the Camera

If you want to make a conservative estimate of distance from the camera a player will need to stand to be fully visible, use the following formula:

$$\text{distance} \approx \text{height of player}$$

The vertical FOV of the camera is close to 45 degrees. From trigonometry, we know that

$$\tan(\theta) = \frac{\text{opposite}}{\text{adjacent}}$$

We also know that

$$\tan(45^\circ) = 1$$

In our case, *opposite* is the height of the player, and *adjacent* is the distance from the player. θ is the angle between the player's head, the camera, and the player's feet.

Find the Sensor Array's Height Above the Floor Plane

If the floor plane returned by **NUI_SKELETON_FRAME** is valid (x , y , and z are all non-zero), it can be used to calculate the height of the sensor array above the floor plane (the w component is the height in meters).

As an alternative, the **NuiCameraAdjustTilt** API returns the sensor array height as part of the **NUI_TILT_OBJECTS** data which it returns (the **CameraHeightMeters** member). This value is calculated during the sensor array's initial calibration sweep; the value returned in **NUI_SKELETON_FRAME.vFloorClipPlane** may be more accurate, as this value continues to be updated and settles over time.

Note that the height value returned in the **NUI_TILT_OBJECTS** structure always returns a valid value even if the floor plane is not visible, regardless whether you passed `NUI_INITIALIZE_FLAG_EXTRAPOLATE_FLOOR_PLANE` to **NuiInitialize**.

Get the Height of a Player

You can calculate an estimate of the player's height by calculating the distance from the **NUI_SKELETON_POSITION_HEAD** joint to the floor plane.

$$h = |P_{head} \cdot \hat{f} - f_w|$$

where $\hat{f}$ is the unit normal formed by the first three components of the floor clip plane vector, and f_w is the w component. This is effectively calculated thus:

```

FLOAT CalcPlayerHeight(XMVECTOR vHeadPosition, const XMVECTOR vFloorPlane)
{

```



```

XMVECTOR vPointNegW = XMVectorSetW( vHeadPosition, -1.0f );
return fabsf( XMVectorGetX( XMVector4Dot( vFloorPlane, vPointNegW ) ) );
}

```

You will most likely want to smooth this value over time, and throw out any outliers.

Tilting for a Specific Player (Using Untracked or Tracked Skeletons)

Assuming you have enabled skeletal tracking, this is simpler than the blob case. The recipe is as follows:

During a Gameplay Session

- Keep track of which skeletal tracking IDs match the active players in your game (Identity can help you here).

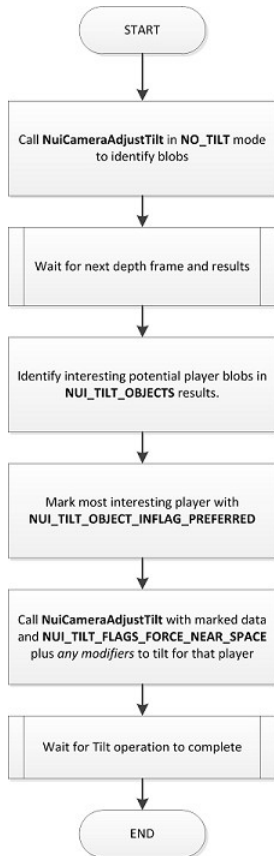
When Switching Players

- Show a prompt on the screen identifying the next player who is "up."
- Show a prompt on the screen identifying the next player who is "up."
- Build a **NUI_TILT_OBJECTS** structure, using the **NUI_SKELETON_DATA** *Position* value to specify a **NUI_TILT_FLAGS_PREFERRED** location for the player to stand at.
- Call **NuiCameraAdjustTilt**, passing in the structure, and wait for the camera to finish tilting before you hide the prompt. (You may wish to perform this operation on a separate thread, reserved solely for handling tilt operations.

Tilting for a Specific Player (Using Blobs)

It also is possible to tilt based solely on interesting blobs (especially if your title does not want to absorb the full cost of using the skeletal tracking pipeline). This scenario is shown below.

Figure 10. Selecting the player to focus tilt on by marking their location as "preferred"



Closing Arguments

Kinect development is a fun adventure to the cutting edge of technology and engineering, where what can be imagined and what is possible collide. And as ever in engineering, there are tradeoffs.

When developing a Kinect title, you will want to give extra care and attention to the Tilt APIs and how they affect your play space. The sensor array could tilt automatically during gameplay, without the title requesting it. The Tilt system disables the skeleton tracking pipeline and the depth image stream while the sensor array is in motion.

Note

Disabling the pipeline and image stream is similar to the way that your visual system stops reporting information if you move your eyes rapidly, known as [saccadic masking](#). We do it for exactly the same reason.

The user could recalibrate the sensor array in mid-game session from the Guide. You may want to tune for your specific gameplay scenario—possibly changing the tilt of the sensor array midlevel in some scenarios.

Recommendations

- Read the *Natural Environs* white paper by Scott Selfon; this will help familiarize you with tradeoffs in play space sizes for different scenarios.
- Start by figuring out what kind of play space your title needs for your specific kind of gameplay, and use the least restrictive you possibly can. For example, unless you are writing a soccer title, you probably do not need to see the player's feet. It might be *nice*, but it is unnecessary.
- Test your title in both near and far play space scenarios with a variety of players of different heights, especially combinations of adults and children.
- Do not be afraid to suspend gameplay if you must, but be sure to add some hysteresis to the system so that you do not have a hair trigger. In most circumstances, you can get away with gentle hints (color/lighting cues), followed by stronger hints (flashing arrows) before you stop everything and push back the player.
- Try to tilt the sensor array as infrequently as possible; the API stops you if you try to tilt too frequently.
- The Kinect tuner wizard (**XShowNuiTroubleshooterUI**) is your weapon of last resort, and you should treat it as such.

Our hope is that this white paper provides you with a framework you can use to improve the handling and maximize the usefulness of your customer's living rooms and other play spaces. We

encourage you to contact the Microsoft Game Developer Support Desk at gamedev@microsoft.com with any questions or comments about this white paper, the topics presented here, and challenges specific to your own title or engine.

Feel free to share your own experiences and best practices; developing for Kinect is a new field, and one which we're all learning about together.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Visual Gesture Builder: A Data-Driven Solution to Gesture Detection



by Claude Marais, Sr. Software Development Engineer
Advanced Technology Group (ATG)

Published: May 22, 2012

Many successful and innovative titles use gestures as input. These titles span a wide variety of genres, platforms and input technologies, from the touch screen of a smart phone to the full-motion, natural input of devices like the Kinect Sensor. This white paper explains how Visual Gesture Builder, a data-driven machine-learning solution for gesture detection, can be used efficiently to detect even the most challenging gestures with very high accuracy. This technology can make developers more productive and raise the quality of Kinect titles in terms of better gesture detection and reduced latency.

- [Introduction](#)
- [Visual Gesture Builder](#)
- [Collecting Data](#)
- [Tagging Data](#)
- [Building and Analyzing Gestures](#)
- [Tips and Tricks](#)
- [Pros and Cons](#)
- [Conclusion](#)
- [References](#)

Introduction

For Kinect titles, it is essential to successfully and robustly communicate a player's intent in a natural way, for this is the very heart of Kinect "you are the controller". This brings us to the importance of gesture detection.

Gestures

A *gesture* is an action or a motion that is intended to communicate feelings or intentions. For example, when your dog wags its tail at you, this is a gesture with which your dog communicates that he is happy to see you.

Gesture Detection

Gesture detection is the ability of a computer to understand human gestures as input. It has been around since 1963, when the first pen-based input device was designed. Gesture detection is still used in many technologies today, such as touch screens, computer mice, handwriting recognition, and Kinect.

Machine Learning

When we say *machine learning*, we refer to the ability of a computer to automatically learn to recognize complex patterns in data. A computer usually does this by learning from empirical data examples, and the result is that it can classify data that it has not yet observed. There are many different approaches to machine learning, such as weighted networks and neural networks, decision trees, support vector machines (SVM), clustering, Bayesian networks, and boosting. Examples of where machine learning is used are internet search engines, face detection on digital cameras, speech recognition software, stock market analysis, and artificial intelligence (AI) in games.

Traditional Gesture Detection with Kinect

Gesture detection using data from the Kinect Sensor is not a trivial problem to solve. As an example, look at the following code that could be used to implement the detection of a punch gesture.

```
if ( vHandPos.z - vShoulderPos.z > fThreshold1 &&
    fVelocityOfHand > fThreshold2 ||
    fVelocityOfElbow > fThreshold3 &&
    DotProduct( vUpperArm, vLowerArm) > fThreshold4 )
{
    bDetect = TRUE;
}
```

This is a very simple detection that can easily work in an optimal environment—but even this simple piece of code has four thresholds that need to be manually found, fine-tuned and maintained. For detections to work reliably for a wide variety of different players in a wide variety of different play areas, the code becomes much more complex very quickly as more code is added to handle additional complicating factors. Some of these factors are that different skeleton data is produced when wearing different clothes, or when the Kinect Sensor is at different heights and angles, and some joints might be occluded due to objects in the room (for example, by a coffee table). This is evident in the large number of lines of code that must be written to detect gestures in real world examples.

Table 1. The number of lines of code used for gesture detection in real-world examples.

Detector	Lines of source code
Wave	550
360 degree turn	500
Jump and duck	2000
Crumpling skeleton	2500
Kinect Sports: Boxing	950

Some of the challenges with the traditional approach of implementing gesture detection by examining data from the Kinect Sensor are:

- It's a time-consuming engineering task, requiring many lines of code.
- Kinect data is complex—for example, twenty 3D joint positions with low precision.
- Determining the best detection thresholds can be difficult.
- Latency is always added, since previous frames of data need to be examined.

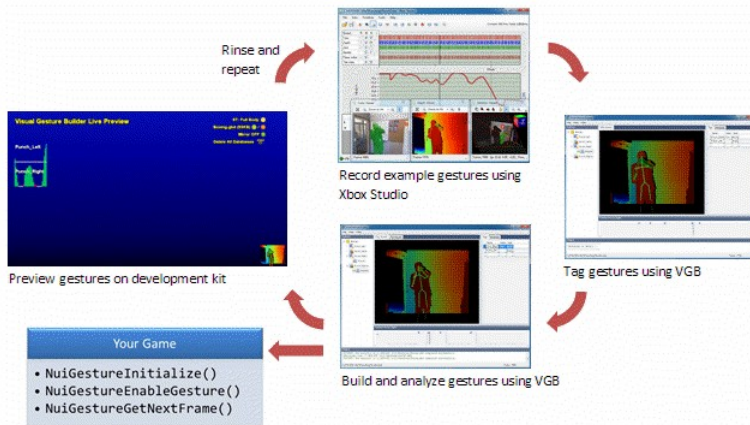
Visual Gesture Builder

Visual Gesture Builder (VGB) is a tool that provides a data-driven solution to gesture detection through machine learning. This essentially means that gesture detection is turned into a task of content creation, rather than code writing, and a task that non-engineers can perform (for example designers, animators, and technical artists). A small gesture database is built using VGB, and using the database has very low run-time costs in terms of memory overhead and CPU processing.

The process of creating a gesture detector using VGB is very simple. Firstly, because it is data-driven, you record players while they perform the gestures that you are interested in detecting. The recordings can be done using Xbox Studio or the Xbox Studio API. (For more information about Xbox Studio and the associated API, see [Xbox Studio](#).) Next, you give meaning to the data by tagging or labeling (using VGB) all the frames in the recordings that define a gesture.

Once that is done, you can simply build the gesture detector. VGB uses machine learning to build a database that can be used at run time in your title with the NuiGesture API. A live preview enables you to quickly iterate over gestures for fast prototyping. For more information about Live Preview in VGB, see [Integrated Development Environment](#).

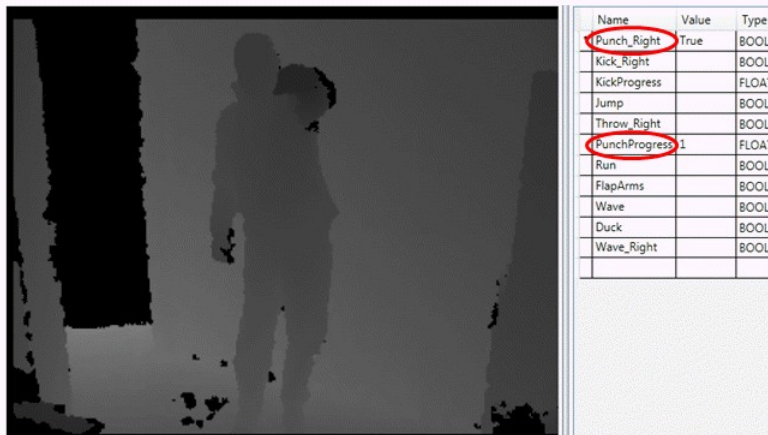
Figure 1. The data-driven process of creating a gesture detector using VGB.



Machine Learning Technologies

Various machine learning technologies are available in VGB. They are grouped into two categories: triggers (for example, AdaBoost Trigger) and progress indicators (for example, RFRProgress). A *trigger* is a binary detector that determines if a player is performing a gesture and the confidence of the system in that gesture. A *progress indicator* shows the progress of the player while he or she performs a gesture, showing, for example, 50% done with the gesture.

Figure 2. "Punch_Right" is a trigger gesture indicating that the player is in the process of performing the gesture. PunchProgress is a progress indicator, in this frame indicating that this is the last frame of the gesture (that is, 100% done with the gesture).



Triggers and progress indicators can be combined in creative ways. One example is to use a trigger gesture to determine context. For example a playable character in the game has the capabilities to walk, run, and jump. A simple approach to using gesture detection to control the character is to use trigger gestures to determine when a person is walking, running, or jumping. The playable character can therefore be controlled using gesture input, but only in a binary way—that is, the player is either walking or not.

Combining this with a progress indicator can add a whole new level of a person connecting with the playable character, making the person can feel that he *is* the playable character instead of feeling that he is controlling the playable character. Since the trigger gestures can determine the context, only the progress gesture for the currently active context needs to be run. The results of the progress indicator can finally be used to blend character animations appropriately so that when a person is walking slowly, the playable character is walking slowly, which provides a much more natural feel to the game.

Collecting Data

The recommended process of collecting data is to use Xbox Studio to record color, depth, and skeletal data of your target audience—individually—while they play a prototype of your game in different play spaces, under various tilt angles of the Kinect Sensor, and while wearing a variety of clothes.

Data to Record

Before recording a bunch of random players, we recommend that you first establish a few things. Firstly, you need to understand what data to record. Currently, the machine learning technologies of VGB operate only on skeletal data. Therefore, the minimum requirement for building a gesture database is to record skeletal data. However, we recommended also recording color and depth data.

Whenever improvements are made to skeletal tracking in the XDK, the same depth data will produce different skeletal data. When you have recorded depth, you can simply convert the old skeletal data to new skeletal data by using the new and improved skeletal tracking, which means that all previous recordings can be converted and used for training the system.

Color data adds a lot of size to recordings, but in many cases, simply looking at depth or skeletal data won't reveal the intentions of the players; however, by also looking at color data, players' intentions can be determined. Since this is a data-driven system, it is essential to tag frames correctly. So, make it easier for the people doing the tagging to interpret the recordings by including color data.

Target Audience

Although you need to include a diverse variety of people for generalization and criteria selection in training, it is important to define the target audience of your game. It could be teenagers, small children, adult males, the whole family, and so on. Make sure that this group of people is well represented when doing data collection—don't develop a game targeted at small children and mainly use gesture recordings of adults.

Record Players Individually

When recording players, record them individually, since people are easily influenced in the way they perform a gesture when they see how others perform the same gesture or when many other people are watching as they perform a gesture. It is best to have recordings of the widest possible range of ways that people perform relevant gestures in their own unique ways, since wide variety is what your title will encounter in the real world.

What Gestures to Record

Ideally, you record players while they are playing a prototype of your game, since people perform a gesture differently when asked to perform a *gesture* versus interacting with an actual game. For example, telling a person to "show me a punch gesture" has a different result than asking "please punch the boxing bag"—people perform differently in these contexts, though the

fundamental gesture may be the same.

Also, define the gestures that you want to detect during game play. For example, if it is a boxing game, you probably want to detect punches and blocks. These are defined as *positive* examples because they are the gestures you're aiming to detect. However, and most importantly, you also have to think of *negative* examples. Determine which motions cause problems for detection, such as false positives, and include them as negative examples for training. For example, a throw gesture might look very similar to a punch, since the hand and elbow move forward in both cases, but the two movements are actually slightly different.

In general, including negative examples in training data reduces false positives, and this leads to a better user experience. Therefore, we recommend that you record many more negative examples for your training data than positive examples.

Use a Variety of Play Spaces and Sensor Tilt Angles

Although the algorithms in VGB mostly operate on tilt-corrected skeletal data that is local to the player space, it is important to record players in different play spaces, and with different heights and tilt angles of the Kinect Sensor. The main reason for this variety is that skeletal tracking produces different skeletal data at different tilt angles and sensor heights, because body joints are differently observed by the Kinect Sensor; for example, a joint might be occluded from one angle but not from another. Furniture can also introduce occlusions that can change the result of skeletal tracking. Therefore, we recommend that, for each person recorded, the Kinect Sensor be slightly moved from its previous locations.

Variety of Clothing

Make sure to record people wearing a variety of clothing—for example, skirts, dresses, tank tops, and wool sweaters. Wearing different types of clothing can produce different skeletal data, even for the same person. Also, be sure to record people wearing the clothing that your customers wear, rather than only what you're accustomed to seeing and wearing. For example, if you're a developer in Florida, chances are you see—all year round—clothes that people in Belgium consider only for summer, but your game will be played by both Floridians and Europeans, wearing clothes typical for their climates.

Testing Data versus Training Data

It is important to understand that the accuracy of the machine-learning algorithm cannot be measured by testing it on the same data that it has been trained with. In other words, don't use *training* data for *testing* purposes. For this reason, whenever a gesture project is created in VGB, two projects are created: one for building and one for analyzing the gesture. An example of how to apportion data into different sets is to use 66% of data for training and 33% for testing.

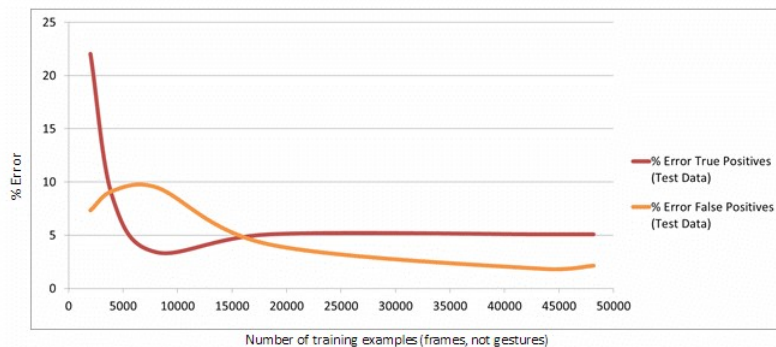
How Much Data Is Enough?

How much data is required is not a question that has the same answer for every gesture. Rather, the question should be, "How can I establish that I have accumulated enough data for a gesture?"

To answer this question requires counting false positives and false negatives. An example of a *false positive* is when a player performs gesture B, but gesture A is detected. An example of a *false negative* is when a player performs gesture A, but detection fails to identify that gesture A has been performed. The number of these two errors are two different values that need to be interpreted separately—the one is not the inverse of the other. "[Building and Analyzing Gestures](#)", later in this paper, explains in detail how to build a gesture, and then how to test or analyze it. The results of this analysis provide the count of errors from false positives and false negatives.

To understand whether you have enough data, you need to track how the values for false positives and false negatives change as you add new training data. In the graph in Figure 3, you can see that with very few training examples, the error rates are high, but as you add more training examples, the error rate goes down. The interesting part is that, at some point, the error rate simply stays the same, regardless of how many more training examples are added. When both values for false positives and false negatives start to flatten out at a low error rate, it is safe to assume that you have enough data in the training set. Other measurements can also be used—for example, precision, accuracy, and recall.

Figure 3. Error rate in false positives and false negatives, graphed against the number of training examples, is a good indication of when you have enough training data.



Tagging Data

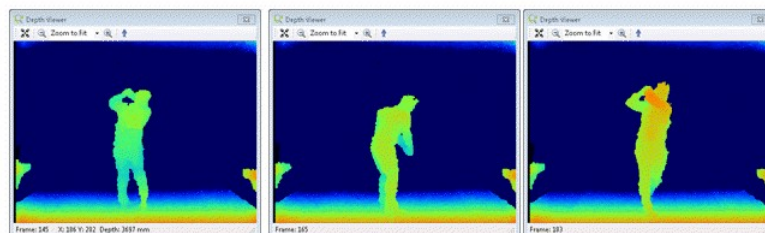
Tagging recorded data is the most time-consuming part of creating a gesture detector using VGB, but it is the most important step. Tagging plays a huge role in the results, since VGB uses a data-driven approach—in other words, *garbage in, garbage out*. The following are a number of best practices to following when tagging data:

- [Keep Gestures Simple](#)
- [Avoid Over-Labeling](#)
- [Separate Gestures from Poses](#)
- [Reduce Latency](#)
- [Be Consistent](#)
- [Specify Left or Right Gestures](#)
- [Verify Tagging](#)
- [Use Keyboard Shortcuts](#)

Keep Gestures Simple

Break complex gestures into smaller sub-gestures and build multiple detectors. For example, a golf swing can be broken up into three gestures: a back swing, a forward swing, and a follow through. Detection of cyclic gestures—like walking, rowing, and waving—are almost always more reliable when detection is divided into two simple gestures. For example, to detect a wave, have one detector for when the hand is moving leftward and another for when the hand is moving rightward.

Figure 4. Shows how a golf swing can be broken up into three simpler gestures. The first image shows a back swing; the middle image shows the forward swing where the golf club makes contact with the ball; and the last image shows the player's follow through.



Avoid Over-Labeling

Tag all the frames that make up a gesture, but only tag the core portion of the gesture. Try to find the canonical motions that uniquely represent an action. For example, when tagging for a jump gesture, it's best to not include the frames where the player is getting ready to jump (for example, moving downward), but rather, to tag only the frames where the player is starting to move upwards until he reaches the apex of the jump, where his body is no longer moving upward.

Separate Gestures from Poses

Don't mix and match frames from static poses with dynamic gestures. For example if you want to detect a jump gesture, tag the frames of the player in which he or she is moving upward; don't include frames where the player is getting ready to jump, where the player might be in a static pose.

Reduce Latency

Latency can be significantly reduced by tagging gestures earlier in a sequence of frames, rather than only tagging when the gesture finishes. Determine the point in the sequence when the player is committed to performing the gesture, and use that frame as the starting point for tagging the gesture. For example, the starting point for a punch gesture could be when the player is leaning forward to punch, but his hand is still close to his shoulder.

Figure 5. An example of how early a punch gesture can be tagged to reduce latency. All these frames include joint information of the player intending to do the punch, therefore all these frames are tagged as a punch.



Be Consistent

It is important to tag different examples of the same gesture in a consistent manner; that is, the starting points and the end points should be tagged in approximately the same places throughout the recordings. In some cases, it might be hard to understand the intention of a player by looking only at depth or skeletal data, and in such cases, examining the color data might make tagging much easier.

Specify Left or Right Gestures

When creating a project, an option called **Side** can be set to the value **Left** or **Right**. If the gesture to be detected can be performed on either side of the body or in either direction, then we recommended setting a value for **Side**; doing so means that your data and tags can be mirrored, doubling the size of your data set for training.

For example, most people are right-handed which means that when recording and tagging a punch gesture, you will have less training data for left-handed players than right-handed players. If your project has **Side** set to **Right**, then VGB can use all the training data from right-handed punches as training data for left-handed punches by mirroring the data.

Figure 6. Shows the "Side" option, which specifies on which side of the body, or in which direction, a gesture is performed.

Verify Tagging

It is crucial that you verify that data is tagged correctly. A gesture that is untagged is used as a negative example during training. This confuses machine learning, because the machine is shown examples of what gesture A looks like (positive examples), and then it is shown examples of what gesture A does *not* look like (negative examples)—but the untagged gesture is a positive example included in the data set of negative examples. It is also possible that a gesture could be marked with the wrong gesture tag. So, we recommended that a quality assurance (QA) team approve tagged data before it is added to a training set.

VGB can be used to automatically find errors in tagging. The following procedure can find errors where gestures were not tagged or where gestures were tagged with a wrong label.

1. Add all tagged recordings to the build project
2. Add all tagged recordings to the analysis project
3. Build the gesture
4. Analyze the gesture
5. Visually inspect the analysis for
 - a. Frames that were detected but not tagged
 - b. Frames that were not detected but tagged

Since training and testing is done using the same data (for this type of verification only), we expect the detector to be near 100% correct during analysis. Having any frames where the tags do not match the results of the detector is a strong indication of incorrect tagging. For example, analysis detects a Punch_Left gesture between frames 100-110, but these frames are tagged as a Punch_Right or not tagged at all. Visually inspecting the recording will reveal if the detector or the tag was wrong.

Figure 7. An example of using VGB for verifying correctness of tagging. The analysis indicates that the detector detected two gestures (indicated by the two purple peaks), but three gestures were tagged (indicated by the horizontal blue lines). Visual inspection of what the player actually did indicates that the detector is actually correct, and the tag in the middle is incorrect.

Use Keyboard Shortcuts

Since tagging is a time-consuming process, make sure that taggers know of the keyboard shortcuts. Using these could significantly reduce tagging time. For a list of the shortcuts, see [Keyboard Shortcuts](#).

Building and Analyzing Gestures

After tagging data, you can build either a solution or just one gesture project. During the building process, the appropriate machine-learning algorithm processes the tagged data. This creates a gesture database and a header file. The header file is compiled into your title code, which loads the gesture database at run time.

Some interesting information can be seen in the log files. For example, from the log file in Figure 8, you can see that the angle velocity between the spine, shoulder center, and left shoulder is actually a good indicator for a right punch. If you only looked at the top classifier, you can interpret this as:

```
if ( AngleVelocity( Spine, ShoulderCenter, ShoulderLeft ) > 0.5f )
{
    bRightPunchDetected = TRUE;
}
```

Therefore, in the case of the AdaBoost Trigger, you can actually do knowledge extraction from the log files to implement your own gesture detection.

Figure 8. A portion of the log file from an AdaBoost Trigger project that indicates the top 10 weak classifiers defining a particular gesture.

```
Build: Top 10 contributing weak classifiers:
Build: AngleVelocity( Spine, ShoulderCenter, ShoulderLeft ) rejecting inferred joints, fValue >= 0.500000, alpha = 1.613963
Build: Angle( Spine, ShoulderCenter, ElbowRight ) using inferred joints, fValue >= 88.000000, alpha = 0.993439
Build: VelocityY( ElbowLeft ) rejecting inferred joints, fValue >= 1.199998, alpha = 0.784924
Build: Angle( WristLeft, ShoulderCenter, WristRight ) using inferred joints, fValue >= 84.000000, alpha = 0.481454
Build: AngleVelocity( Spine, ShoulderCenter, ElbowRight ) rejecting inferred joints, fValue >= 2.780000, alpha = 0.586568
Build: Angle( Spine, ShoulderCenter, Head ) using inferred joints, fValue < 158.000000, alpha = 0.572036
Build: VelocityY( ElbowLeft ) rejecting inferred joints, fValue >= -0.000003, alpha = 0.571161
Build: Angle( Spine, ShoulderCenter, FootRight ) using inferred joints, fValue >= 14.000000, alpha = 0.498747
Build: Angle( HandLeft, ShoulderCenter, HandRight ) using inferred joints, fValue >= 80.000000, alpha = 0.455011
Build: AngleVelocity( ShoulderCenter, ShoulderRight, AnkleRight ) using inferred joints, fValue < -0.250000, alpha = 0.445416
```

To test how well the detector fares, you can test it at run time by using **Live Preview** in VGB, which runs the detector on the Xbox 360 development kit and is very useful for fast prototyping.

Another way of testing is to use the analysis project. This will run the detector on the PC. The biggest advantage of using the analysis project is that you can compare the results of multiple builds; for example, you can determine if adding more training data really improved the detector or not.

Figure 9. An example of the analysis of a gesture where five tagged gestures (blue lines) are correctly detected (purple peaks) and one not.



For more information about the analysis project in VGB, see [Analysis Project](#).

Tips and Tricks

Converting Old Data

It's great when skeletal tracking in the XDK improves, but that means that all XED file recordings now have skeletal data that is not representative of what you'll see with the new skeletal tracking at run time. Since VGB uses XED file recordings to represent gestures for the machine-learning algorithms, it is best to use XED file recordings that represent the skeletal data of the most up-to-date skeletal tracking. An easy way to convert XED files that contain old skeletal data to new skeletal data is to use Xed Skeleton Updater; for more information about this tool, see [Xed Skeleton Updater](#).

If, prior to the existence of VGB, you used XedFileTagger, a sample in the XDK, to tag gestures, then your existing XED files that are tagged can easily be updated to VGB tags by using the batch converter in the updated XedFileTagger sample.

Efficient Tagging

To tag efficiently, make sure you know all the keyboard shortcuts for VGB; at the same time, the shortcuts that you'll probably use most often are those in the following table.

Table 2. Commonly used keyboard shortcuts in Visual Gesture Builder.

Key stroke or combination	Action
Shift + Left Arrow Shift + Right Arrow	Selects a time range.
Enter	Sets the default maximum value.
Delete	Deletes the selected frame(s).
Ctrl + Left Arrow Ctrl + Right Arrow	Moves the cursor to the next/previous frame.
Page Up Page Down	Selects the previous/next attribute in the tags grid.

Some shortcut keys can also be combined. For example, by pressing Shift + Ctrl + Left Arrow/Right Arrow, you can quickly select multiple frames between tagged gestures, since Shift selects the frames and Ctrl jumps to the next/previous tagged gesture.

If you have a trigger and progress detector for the same gesture, the most efficient tagging process is the following:

1. Tag all the frames of the trigger. This will determine the start and end frames.
2. Press the Home key to move the cursor to the first frame in the clip.
3. For each instance of the gesture, do the following:
 - a. Press Ctrl + Right to quickly jump to the beginning of the gesture.
 - b. Press Page Up/Page Down to select the progress for this gesture.
 - c. Press Spacebar to assign a value of 0.0 progress to the beginning of the gesture.
 - d. Press Page Up/Page Down to select the trigger for this gesture.
 - e. Press Ctrl + Right Arrow to quickly jump to the end of the gesture.
 - f. Press Page Up/Page Down to select the progress for this gesture.
 - g. Press Enter to assign a value of 1.0 progress to the end of the gesture.

Since the tags are created in XML files, you could easily write your own tools to automate this or a similar process.

Reduce Training Time

Depending on how much training data you have and the type of hardware your training computer has, training times for AdaBoost Trigger can potentially be long. There are, however, a few easy things that can be done to reduce training times.

AdaBoost Trigger has a project setting, Accuracy Level, that you can specify when configuring a project. This project setting controls the accuracy threshold that must be reached as a result of machine learning. Therefore, it affects how long training will take by stopping the search for weak classifiers when a certain error metric has been reached. For more information about Accuracy Level, see [AdaBoostTrigger Machine Learning](#).

You can also exclude meaningless data from training, especially if the recordings used for training are from a person playing a prototype of your game and the recordings include many frames with the player standing idle. One way to exclude meaningless data (for example, where a player is not doing anything meaningful), is to use Xbox Studio to save selected frames into smaller XED files. This has the advantage of also compressing the XED file.

Another approach is to use VGB. Clip files that have trigger gestures only tagged with TRUE values assumes that all non-tagged frames are FALSE during training. If a clip file contains TRUE and FALSE values, then all frames not tagged will be excluded during training.

Another way to reduce training time and run-time cost is to mirror a gesture—for example, a right-handed golf swing could be used as a left-handed golf swing by mirroring the data. The [NuiGestureSetPreprocessFlags](#) function can be used to mirror the skeletal input data at run time. Essentially, you halve your training time by using this functionality, because you only have to train for one side, and at run time, you can use the same database for both sides or directions.

Measuring Run-Time Resource Costs

Live Preview in VGB shows how much run-time memory your title will consume after loading a gesture database.

Figure 10. An example of Live Preview that indicates that 71 KB of memory is used at run time on the Xbox 360 console.



The NuiGesture API runs on the NUI thread after skeletal processing has completed. To find out the run-time cost in CPU resources, you can use the PIX counters for NUI. The total CPU time consumed by the NuiGesture API can be calculated as:

$$\text{Total Gesture Time} = \text{Gesture Process Ending Time} - \text{Skeleton Process Ending Time}$$

Figure 11. Using PIX counters for NUI, we can calculate the total processing time for NuiGesture—in this example, as 10.64 ms - 10.29 ms = 0.35ms

Counters			
Color	Counter	Scale	Last Value
	Frames per Second	1.000	30.02
	Frame Time (ms)	1.000	33.32
	Depth Camera Arrival Time (ms)	1.000	26.47
	Skeleton Process End Time (ms)	1.000	10.29
	Gesture Process End Time (ms)	1.000	10.64

For more information about the PIX counters for NUI, see [PIX System Monitor: Predefined Counters and Events Reference](#).

Command-Line Options

Since Visual Gesture Builder turns the problem of writing gesture detectors into content building, you might want to build gestures overnight, and on a daily basis, with other content. Command-line options exist to build and analyze gestures so that you can do this from batch files or scripts; for more information see [Batch Execution](#).

Pros and Cons

Visual Gesture Builder provides benefits and also has some costs. Consider the following pros and cons in assessing VGB for your project.

Pros

- Gestures detection is created and handled as content, rather than being a time-consuming engineering task.
- Non-engineers can create gestures—for example, designers, animators, and technical artists. No knowledge of machine learning, skeletal tracking, or programming is required.
- High accuracy for detecting gestures can be achieved, even in cases where skeletal data is very noisy, such as sideways poses.
- By tagging data appropriately, perceived latency can be made very low.
- The run-time costs to CPU and memory are low—on magnitudes of microseconds and kilobytes.
- The database size is independent of the amount of training data.
- There are very few thresholds to tweak and maintain.
- Live Preview enables fast iteration times for easy prototyping.
- VGB provides a free test framework for gestures; that is, no extra engineering work has to be done to test gestures, as in the traditional implementation of gesture detectors.
- A very simple API is provided by the XDK.
- VGB provides an automatic test framework for gestures.
- If skeletal tracking in the XDK improves, you can simply retrain the gesture database.

Cons

- It's time-consuming to tag data—but at least it's not an engineering task, and therefore, you can outsource tagging.
- You require a powerful PC for training, but most studios already have a powerful PC for building other types of content.
- You require lots of disk space for storing XED file recordings, but luckily hard disk drives are becoming cheaper.

Conclusion

Using traditional methods of coding gesture detection using Kinect data is not a trivial task to do robustly. Visual Gesture Builder turns this into a trivial task, which can make developers more productive and raise the quality of Kinect games in terms of better gesture detection and reduced latency. Since tagging plays a huge role in obtaining good results, it is worth investing in a quality assurance team to approve data before adding the data to a training set.

References

Presentations

The following presentations are available from [Download Center](#):

- "[Innovative Solutions to Gesture Detection](#)." Claude Marais & David Quinn, Game Developer Conference 2012.
- "[Gesture Detection using Machine Learning](#)." Claude Marais, Gamefest 2011.
- "[Building Great Gesture Detection](#)." Claude Marais, Game Developer Conference 2011.
- "[XStudio Deep Dive: Designing, Debugging and Testing Your Kinect Title](#)." Claude Marais, Game Developer Conference 2011.

Xbox 360 SDK Samples

The following samples are included with the Xbox 360 Development Kit:

- [Crumple Detection](#)
- [Driving Raw Filters](#)
- [Gesture Detection](#)
- [Gesture Scoring](#)
- [Hand Control](#)
- [Hand Open Close](#)
- [Kick Boxing](#)
- [Lean Detection](#)
- [Simple Gesture Detection](#)
- [Turn Detection](#)

- [Wave Gesture Tracking](#)
- [Xed File Tagger](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

The System Gesture for Kinect for Xbox 360



by David Tuft, Software Development Engineer

Advanced Technology Group (ATG)

Published: February 24, 2010

Updated: October 8, 2010

The Xbox 360 system supports Natural User Interface (NUI), made possible by Kinect for Xbox 360. A Kinect-enabled Guide and Dashboard offer a subset of the Xbox Dashboard and Guide. These allow players to navigate without needing to hold a controller. Players who are playing titles that use natural input expect the ability to access the Guide from the title. To make this possible, a new *system gesture* has been established. (It is also known as the *Guide gesture*.) The system gesture enables an Xbox 360 player, playing a game that uses natural input, to activate the Guide, all without ever picking up a controller.

This white paper describes system gesture. All visual images in this white paper are mocked up and may change.

The System Gesture

To perform the system the player extends his or her *left* arm down and to the left, away from the body at a 45-degree angle. The *right* arm must be held stationary at the side. When the player stays in this pose for an allotted amount of time, a notification, [XN_SYS_NUIPAUSE](#), is sent to the title.

Handling the Pause Message

When the XN_SYS_NUIPAUSE message is received, the title is expected to load an in-game menu where the Kinect Guide is accessible. When the player selects the Kinect Guide from this interface, the game is expected to call [XShowNuiGuideUI](#).

It is important that the title uses the correct tracking ID to open the Guide. When processing XN_SYS_NUIPAUSE, the tracking ID of the player who performed the system gesture should be retrieved from the data parameter associated with the notification. The following code sample demonstrates doing this.

```
// Handle notification and retrieve the tracking ID of the player
// who performed the system gesture.

if ( XNotifyGetNext( m_hNotification, 0, &dwNotificationID, &ulParam ) )
{
    switch( dwNotificationID )
    {
        case XN_SYS_NUIPAUSE:
            dwTrackingID = (DWORD)( ulParam );
            break;
    }
}

// If we need to show the Guide, use the tracking ID of the player
// who performed the system gesture.

XShowNuiGuideUI( dwTrackingID );
```

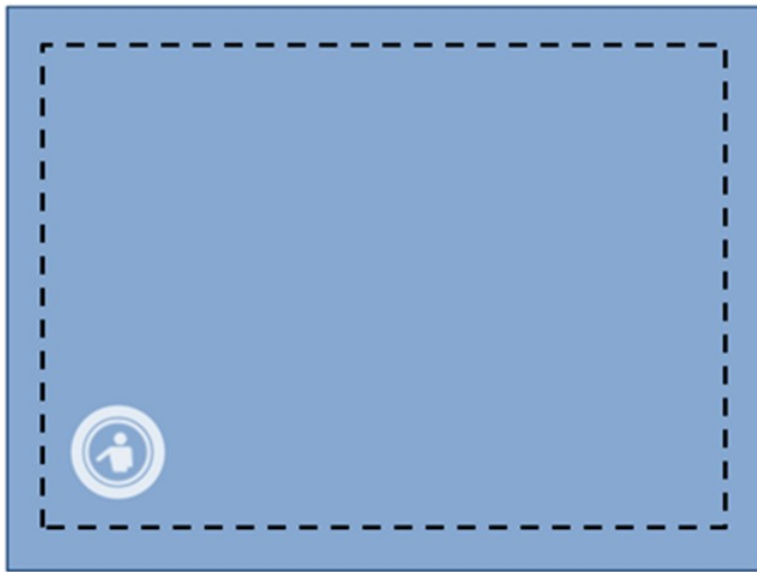
When the player performs the system gesture from somewhere within the game's front-end menu system, it is acceptable to launch the Guide directly. During non-interactive portions of the game where technical reasons prevent loading a menu interface, such as during loading screens or during game cut scenes, it is also acceptable to launch the Guide directly in response to XN_SYS_NUIPAUSE.

Figure 1. Performing the system gesture



A progress indicator is displayed in the lower left region of the screen, as shown in Figure 2. In the scenario where two players are being tracked and both are performing the system gesture, the progress indicator displays the progress of the player with the highest progress level.

Figure 2. Location of the progress indicator for the system gesture



Collisions Between the System Gesture & Gameplay

Gameplay scenarios may collide with the system gesture. For example, a sports game could have gameplay that encourages players to move their left arms in and out of the region of space that correlates to the system gesture, causing accidental activation of the Guide. Such collisions could result in a frustrating user experience. For those scenarios where the game can detect potential collisions, recognition of the system gesture can be temporarily deactivated.

Temporarily Ignoring of the System Gesture

There are some scenarios where the system gesture might interfere with gameplay gestures, such as full-body avatar retargeting. Titles will have the ability to temporarily disable the response of the system to the reserved gesture. To temporarily stop the system from recognizing the system gesture, call [XNuiDelayUI](#). To re-enable, call [XNuiDelayUI\(0\)](#).

Maintaining the Player's Access to the Guide

Titles are required to make the Kinect Guide available when the system gesture is performed. (For details, see [TCR 152](#), *NUI Kinect Guide Access*.) This requirement ensures consistency across titles and enables players to perform such in-game actions as inviting friends into the game and pausing the game. Players only need to learn once the mechanism that launches the guide. It is important to make sure that the system gesture is never ignored when the user is trying to launch the Guide.

It is up to the title to ignore recognition of the system gesture only when the player is not trying to perform the system gesture. Doing this is not always simple. However, the title has much more information about what the user is trying to do than does the system. This is because the title has game state.

Visual Clues to Use

Visual clues help players know when the system handle is about to recognize the system gesture. The system gesture progress indicator becomes more and more opaque as the player stays in the system gesture pose. Also, the players will see a progress animation.

Summary and Best Practices

Accidental recognition of the system gesture during game play can be a very frustrating experience for the player. It can also be an equally frustrating experience for the player to try unsuccessfully to launch the Kinect Guide. Avoid player frustration: Be sure to test your title's response to the system gesture across all areas of the title.

The system gesture enables players to launch the Kinect Guide and navigate in-game menus.

When the `XN_SYS_NUIPAUSE` message is received, the game can either launch a menu with a visible option to load the Guide, or it can launch the Kinect Guide directly.

[XNuiDelayUI](#) can be used as a toggle switch. Call this function once with a large number of milliseconds to disable the system gesture. Call the function again with zero to re-enable. Be sure to always pair these calls together. Failing to re-enable recognition of the system gesture can put your title in a state where it will fail [TCR 154](#), *NUI Title Interaction with System Gesture*.

The system gesture should only be ignored when the game is confident that the user is not attempting to perform the system gesture but is, instead, trying to play the game. The game makes this determination based on the player's actions in conjunction with the game's state and recent tracking of the player.

The *system gesture* is also known as the *Guide gesture*.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Identity Basics: Implementing Essential Skeleton Identity Processing



by Eric Langlois, Senior Software Development Engineer

Advanced Technology Group (ATG)

Published: May 9, 2012

Every Kinect title must provide support for identifying each skeleton as a particular player who is associated with a gamer profile. Although the level of sophistication of player identity tracking depends on the type of experience a title offers, every title must implement a set of specific behaviors designed to guarantee a uniform and satisfying player experience. This document explains how to correctly implement identity recognition behaviors to meet the minimum Xbox 360 requirements for Kinect Identity.

- [Introduction](#)
- [The Three Golden Rules \(or, the NUI Identity TCRs\)](#)
- [Flow of Skeleton Identity Processing](#)
- [How the SimpleIdentity Sample Implements Identity Processing](#)
- [Summary](#)

Introduction

In the NUI Identity system, to identify a skeleton is to perform biometric recognition on a skeleton that enters the field of view, to determine whether the skeleton belongs to an already enrolled player, or if not, determine whether the skeleton matches a biometric-enabled gamer profile and therefore should be enrolled and signed-in.

The NUI Identity system can automatically handle essential skeleton identity processing, such as performing skeleton recognition processing whenever a skeleton enters the field of view. We recommend that all titles enable automatic identity handling, which makes it easy to fulfill the TCR requirements. You can implement more involved experiences by also directly calling the low-level identity functions `NuiIdentityIdentify` and `NuiIdentityEnroll`. When automatic mode is enabled, the title doesn't need to listen for the message `NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY`, and doesn't need to call **`NuiIdentityIdentify`** or **`NuiIdentityEnroll`**.

The Three Golden Rules (or, the NUI Identity TCRs)

Of the many practices for implementing skeleton identity processing, three best practices for NUI Identity deserve special attention. They are the basis for building a consistent and enjoyable player experience for all Kinect titles. These identity best practices are so important that they have been made into Technical Certification Requirements (TCRs), to provide consistency and enjoyment for the players.

The Identity TCRs can be summarized as the following three golden rules.

- Enroll and sign-in a player, if the player has a matching biometric-enabled gamer profile.
- Indicate to the player which gamer profile or guest identity will be used, and allow the player to correct the identity before taking the first profile-related action.
- Limit identity processing to 15 seconds, before allowing gameplay.

The following descriptions expand the three golden rules with details about the Identity TCRs. You can see the latest version of the NUI Identity TCRs on GDN.

TCR 165: NUI Biometric Sign-In

This requirement ensures that all titles support Kinect ID enabled profiles (also called biometric-enabled gamer profiles or simply biometric profiles). Players who set up Kinect ID in their gamer profile expect biometric sign-in to work across all titles on the Xbox 360.

Consistency across all types of experiences on the Xbox 360 requires that every title attempts to enroll a newly detected skeleton that has shown intent to play and has a matching biometric profile. To this end, a title should attempt to perform recognition processing at least once on any skeleton that enters the field of view. If there are no image quality issues, this recognition processing can be completed silently, without requiring the player's direct involvement. If player identification fails (due to image quality issues), the title isn't required to make an additional attempt to identify the skeleton.

It is possible that a player with a biometric profile is already signed-in, such as through the Xbox Dashboard, before launching your title. But that is by no means certain, and your title must attempt to enroll and sign-in each non-enrolled player who has a matching biometric profile, at least until the maximum number of profiles supported by the title has been reached.

TCR 158: NUI Identity Player Feedback

This TCR protects gamer profiles from unwanted changes. According to this TCR, a Kinect title should not modify data for a profile unless the player has had the chance to correct or change which profile is used. Examples of profile-related operations subject to this rule:

- Awarding achievements
- Saving game progress
- Updating Rich Presence
- Writing to Xbox LIVE storage
- Writing to a leaderboard

It's OK to read from the gamer profile that's initially selected by the NUI Identity system, but information in the gamer profile must not be changed until the player confirms that identity. For example, a title can use information such as the gamertag or avatar to help the player clearly identify the matching profile. This feedback is essentially a protection mechanism to ensure data isn't written to the wrong profile, such as if an incorrect biometric-enabled profile was loaded due to mis-identifying a player, or if a player who enters the field of view is mistaken as a different, enrolled player.

Titles should provide an opportunity to the player to fix any mistaken identity without interrupting the flow. For simple scenarios, providing access to the system's sign-in UI is good enough.

If the identity system determines that a skeleton that enters the field of view is an already-enrolled player, you are not required to ask the player to confirm that identity.

TCR 166: NUI Identity Maximum Delay

This requirement ensures that players can always play the game, even if the player's skeleton cannot be identified. Identity operations may fail for diverse reasons, ranging from player positioning (such as not facing the sensor) to environment issues (such as insufficient lighting). These issues can result in a player not being identified, or an operation taking an excessive amount of time. Identity operations must never prevent a player from entering gameplay. A player should be able to access the game and shouldn't be put in a position where no progress can be made. A blocking operation should not last more than 15 seconds.

There is no time limit for non-blocking operations. For example, an identification operation that tries to identify a player who has just entered the play area can continue for as long as required, as long as gameplay is not prevented.

These golden rules, or areas of best practices, are key to providing a great player experience, and that is the reason they have been turned into TCRs. Beyond that, there is a lot of room for customization of the identity experience by the title. This document focuses on providing the experience defined by these three rules. A good understanding of the TCRs is essential before implementing the behavior discussed in this white paper.

Flow of Skeleton Identity Processing

A minimum implementation of skeleton identity recognition in a title involves the following sequence:

1. The NUI Identity system attempts to silently identify any skeleton that enters the field of view.
2. Detect that the skeleton is indicating intent to play, and report that intent to the NUI Identity system.
3. Prompt the player to resolve any image quality issues, while the NUI Identity system re-tries identifying the skeleton.
4. Enroll and sign-in the player, if not already enrolled and the skeleton matches a biometric-enabled gamer profile.
5. Display the gamer profile or guest identity that will be used for the player.
6. Allow the player to change the gamer profile or guest identity, and allow the player to manually sign-in to a gamer profile.
7. Start or resume gameplay, supporting identity actions during gameplay, such as pausing identity processing, and handling additional players who show intent to play.

This sequence is sufficient for single-player titles, or any title that only relies on a single player profile in multiplayer mode, and can be expanded for more complex scenarios. This sequence maximizes the likelihood that the principle player will be signed-in to a profile early on. This section examines these steps in detail.

1. Attempting to Silently Identify the Skeleton

During title startup, pass the flag `NUI_INITIALIZE_FLAG_AUTOMATIC_IDENTITY` in the call to **`NuiInitialize`**. This turns on automatic skeleton recognition processing, which will be working for you from behind the scenes at all times. Whenever a skeleton enters the field of view and starts to be tracked, the NUI Identity system silently attempts to perform skeleton recognition processing, to determine whether the person is an enrolled player and whether the skeleton matches a biometric-enabled gamer profile.

Image quality issues, such as insufficient lighting, might prevent this silent recognition processing from completing successfully. At this point, you must not prompt the detected person to resolve these issues, because the person hasn't indicated intent to play, which means the person might be a passerby or observer rather than a player.

The NUI Identity system attempts to identify skeletons immediately, because this can shave some precious seconds from the identification and enrollment process. At the start screen, players are likely to be already facing the screen and Kinect Sensor, so identification processing is likely to succeed without prompting the players to face the Kinect Sensor. This initial silent attempt to identify the skeleton is also likely to succeed because if the person does intend to play, and isn't playing their first Kinect title, they may expect to be identified and will naturally face the sensor and adopt a neutral pose.

2. Detecting and Reporting Intent to Play

The NUI Identity system relies on your title to differentiate between people who are actually engaged with the title and those who are not. In your title, you define the action that indicates intent to play. When you detect that the skeleton has indicated intent to play, call **`NuiIdentityDetectedIntentToPlay`** to report this to the NUI Identity system.

There are many different ways for a player to show intent to play, and they can vary greatly by title and context. Common ways of showing intent to play are waving, performing a title-specific gesture, or selecting an on-screen button at the introduction screen that follows intro videos and credits/disclaimers screens. Ideally, that gesture fits the theme of the title, but a title can choose to use the wave gesture used by the dashboard. A detector for the wave gesture is included in the XDK. (For an example of using the **`NuiWave`** API, see the `WaveTrackingGesture` sample.)

The requirement to detect intent to play before interacting with the player applies at all times, whether at an introduction screen or during gameplay. For example, selecting the **`Resume`** option from the title's Pause menu is a common way for a player to enter the game in mid-session, and that selection should constitute intent to play.

If more than one player is attempting to show intent to play, but only one profile is required, an effective simple approach is to show gesture feedback for one player only. That situation occurs in a single-player game, or at the Intent to Play screen when determining the primary profile. For example, usually it works well to show gesture feedback only for the player who is the closest to completing the intent-to-play gesture. Alternatively, you could show gesture feedback for the player who is closest to the Kinect Sensor, or closest to the center of the play area. In each title design, this sequence of gesture feedback priority should be made clear to the players.

3. Re-trying Identifying the Skeleton, Showing Quality Issues

When you call **NuiIdentityDetectedIntentToPlay**, identification processing for the specified skeleton is in one of the following states:

- Silent identification processing completed unsuccessfully; image quality issues prevented determining whether the skeleton is an already-enrolled player. In this case, when you call **NuiIdentityDetectedIntentToPlay**, the NUI Identity system retries identification processing. During that processing, call **NuiIdentityGetNextMessage** to retrieve image quality feedback from the NUI identity system. This retrieves a flag for each quality issue, such as `NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK`. Use this quality issue feedback to prompt the player to adjust their posture or environment so that skeleton recognition processing can succeed, such as "Face the sensor" or "Make sure your room is well lit."
- Identification processing is still in progress. Begin to provide feedback to the player about any image quality issues, to guide the player and enable identification processing to succeed.
- Silent identification processing completed successfully; the NUI Identity system has determined either that the player is not enrolled or is enrolled as a particular player. In this case, when you call **NuiIdentityDetectedIntentToPlay**, the NUI Identity system proceeds to the next step, which is conditionally enrolling and signing-in the player.

If identification processing hasn't completed within 15 seconds after the player indicated intent to play, treat the player as a new guest player. Display the player's identity and allow the player to correct identity, as described later. You can then optionally re-try skeleton recognition processing by calling the low-level function **NuiIdentityIdentify**.

In addition to **NuiIdentityGetNextMessage**, the NUI Identity system also provides feedback to you about the identity status of each skeleton at all times, through the `NUI_SKELETON_DATA` structure.

4. Enrolling and Signing-in a Biometric-Enabled Player

After you call **NuiIdentityDetectedIntentToPlay** and skeleton recognition processing has completed, the NUI Identity system enrolls any non-enrolled skeleton that has a matching biometric-enabled gamer profile. This enrolling also causes sign-in, because as soon as a player who has a biometric-enabled gamer profile is enrolled in the identity session store, the identity system also signs the player in to the player's gamer profile.

For a skeleton that doesn't match a biometric-enabled gamer profile, you can optionally enroll the skeleton as a guest player, by calling the low-level function **NuiIdentityEnroll**. A player might already be enrolled and signed-in before starting the title, or through the dashboard after the title started, such as by accessing the dashboard through a Pause menu.

It is a good idea to always call **NuiIdentityDetectedIntentToPlay** when a player indicates intent to play, even if the number of profiles supported simultaneously by the title has been reached. This allows the NUI Identity system to enroll a player who has a matching biometric profile, if an enrollment slot is available beyond the number of simultaneous profiles supported by the title.

5. Indicating Which Gamer Profile Will Be Used

When a skeleton enters the field of view and indicates intent to play, before entering gameplay, the title must display which player identity will be used. Until the player has confirmed the profile that the identity system selected, you can only read from the profile; you must not write to it yet.

For an enrolled, signed-in player, you can present the gamertag. If a profile is not available for the player, you can display **Guest** or a term that is adapted to the context of the game. You could also display the player's avatar.

6. Allowing Identity Correction and Manual Sign-in

Allow the player to change which identity is used. The player must be able to select an already-enrolled player identity, or manually sign-in to a gamer profile. If the player selects an option that advances toward gameplay, that action is considered as the player accepting the selected profile or accepting participation as a Guest player, a choice that means no gamer profile is selected.

Do not force a player to sign-in to a gamer profile. Your title must be ready to proceed with gameplay using a guest identity rather than a gamer profile, even if it means some functionality is unavailable, such as saving games and accessing Xbox LIVE.

To encourage a guest player to sign-in to a gamer profile, a good strategy is to present a "Correct My Identity" button that invokes the sign-in UI. The player can also access the Kinect Guide through the title's Pause menu, but placing a button on the screen is more user-friendly at this early stage in the game.

If your title doesn't enroll guest players, the sign-in UI is all you need to support profile correction. If your title enrolls guests, you will have to provide your own profile correction UI. For an in-depth discussion of providing a UI to let players correct their identity, see the white paper "Who Is Playing My Kinect Game Right Now?"

7. Supporting Identity Actions During Gameplay

NUI Identity processing occurs during gameplay as well as before gameplay.

Pausing Identity Processing During Gameplay

When you want to temporarily access the color image stream from the color camera, and any other system resources that are used by the NUI Identity system, pause identity processing by calling **NuiIdentityPause**. To re-enable identity processing, call **NuiIdentityResume**.

Although you are allowed to pause skeleton identity processing, sometimes for long periods of time (for example, once the game is in progress), it is preferable to let identity processing run in the background so players keep being identified whenever skeleton tracking is lost and regained.

The NUI Identity system uses the color image stream to capture a unique biometric print from a player. While capturing the biometric print, NUI Identity will change the camera settings, such as exposure and zoom, to get a clear view of the player's face, and therefore the color image stream won't be available to you. When you call **NuiIdentityPause**, if there is no identification operation in progress, the color image stream will immediately become available to you. If there is an identification operation in progress, NUI Identity will suspend the operation and return the color image stream settings to the last settings you selected.

Returning the color image stream to your settings takes up to a few frames. Ideally, you should call **NuiIdentityPause** in advance, so the color image stream can be restored to the application.

If your title is using the color image stream intensively, consider pausing NUI Identity processing for the duration of the gameplay session. However, while paused, identity processing will not be able to support your title and help you identify the players as they move around and enter or leave the play area.

Consider pausing NUI Identity during gameplay if your title is using every bit of CPU power available and cannot afford the occasional spike caused when NUI Identity tries to identify a newly tracked skeleton. However, this is a rare, extreme case.

Changing Profiles During Gameplay

Once the player has accepted the gamer profile or the absence of a profile, it is OK to use the selected profile (or the lack thereof) until the player explicitly expresses the need to change profiles. The player could change profiles by returning to the title's Lobby or signing out from the dashboard.

Additional Players Showing Intent to Play During Gameplay

As long as the profile remains signed-in, a title can keep using the profile even when other players (with or without a profile associated) come into play. To engage with the title, the players still must show intent to play. Accordingly, you should register the intent to play with the NUI Identity system so that any related biometric profile is loaded, even if you won't be using the profile yet.

There are many ways a player can show intent to play, once gameplay has started, and it is up to you to decide which ways are available at a specific time. For example, if the game is paused, entering the play area and selecting the **Resume** option should be considered indicating intent to play. After all, the player has clearly stated that he intends to continue where the previous player left off.

Even when skeleton identity recognition isn't essential to your title at certain times during gameplay, you should allow identity processing to run, but refrain from providing feedback about player identification, to not unduly disturb the players. In the end, players will benefit when identity processing is allowed to run in the background, even if only because their biometric-enabled profile is automatically enrolled and signed-in, as they would expect.

Sign Out During Gameplay

Remember that the player can sign out of your title at any time, such as by accessing the dashboard through the title's Pause menu, or by removing the Memory Unit (MU) on which the current gamer profile is stored.

This is all you need to know in order to implement skeleton identity processing at the minimum required level. The following section provides a detailed example of this minimal implementation of identity processing.

How the SimpleIdentity Sample Implements Identity Processing

In a typical single-player scenario, the player usually engages with the title for an extended period of time, and the engagements are often split over multiple gaming sessions. The title uses saved games to keep track of the player's progress and accomplishments across gaming sessions.

A gamer profile is the player's identity on the gaming console. Not only does the gamer profile hold the player's gamertag and avatar, but it also allows access to saved games and is required for Xbox LIVE play. A title should provide a clear flow of identification processing, to encourage and facilitate player sign-in, but a title should never force the player to sign in to a profile, and should offer at least some functionality to a player without a signed-in profile.

A title can use a strategy similar to the SimpleIdentity sample so the player can be sure the right profile is selected. Place a Lobby screen after the title's Intent to Play screen (described later), forming the following sequence of screens for titles in general:

1. Introduction (including legal notice and publisher and studio logos)

2. Intent to Play screen
3. Identification and Enrollment screen
4. Lobby screen
5. Game screens
6. Pause menu

Think of the Intent to Play screen as the replacement for the Press Start to Begin screen that you would normally find in a non-Kinect title.

Inserting a Lobby screen after the Intent to Play screen provides an important opportunity for the player to ensure that the player has been correctly identified by the identity system and is bound to the correct profile. The Lobby screen provides all services that players require to correct their identity and sign-in to a profile in one spot. This solution doesn't solve every potential problem, because the player can sign in or out at any time through the Kinect Guide, and players can leave or enter the play space at any time. Your title must still react to those events when they happen. However, providing a stage that is clearly for enrollment and sign-in will go a long way to providing players with a simple, straightforward experience for enrollment and sign-in.

The SimpleIdentity sample implements the strategy discussed above, though the sample doesn't have an Introduction screen. The following is a step-by-step explanation of the processing flow used by the sample. The vocabulary in the UI of the SimpleIdentity sample is only an example, and we encourage you to create your own naming conventions to suit your game's terminology.

The Intent to Play Screen

The Intent to Play screen usually comes after the title, credits, and other introduction screens and movies, and provides the first opportunity for the player to actively engage with the title. This sample uses automatic identity handling, so as soon as the skeleton is detected and tracked, the NUI Identity system is already attempting to silently perform recognition processing.

Figure 1. The Intent to Play screen.



The SimpleIdentity sample uses the **NuiWave** API to detect a wave of the arm from the player. The dashboard uses this same mechanism to detect intent to play, and so do many Kinect titles.

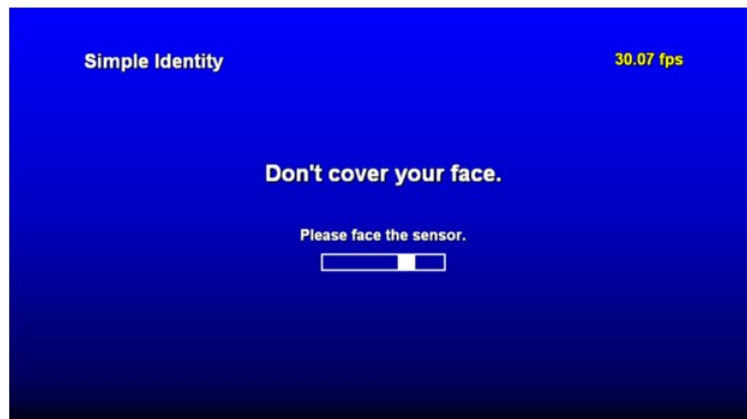
A title has much latitude when it comes to defining what constitutes intent to play. We encourage you to create a title-specific gesture to help immerse the player in the game. See the discussion of Intent to Play in the white paper "Who Is Playing My Kinect Game Right Now?"

The Identification and Enrollment Screen

Immediately after the player has shown intent to play, the title reports this intent, and the NUI Identity system then enrolls and signs-in the player if a matching biometric profile exists and the player isn't enrolled already.

The time to complete the initial identification cycle, prior to gameplay, can be drastically shortened if the player is already enrolled when the title is launched (from another title or the dashboard). In some cases, the steps will seem to happen all at once and complete as soon as the player shows intent to play.

Figure 2. The Identification and Enrollment screen.



During the identification and enrollment step, NUI Identity constantly provides the title with processing status and flags about any image quality issues, which are updated as the frames of skeleton tracking data come in. The title must relay this information to the player so the issues can be corrected. The title can use on-screen text, images, or audio cues to guide the player through the identification process. It is a good idea to remind the player to face the sensor during this process.

The Lobby Screen for Manual Enrollment and Sign-in

After the identification phase, the title must show which identity is used, along with providing an opportunity for the player to change the identity. The SimpleIdentity sample provides this correction opportunity through a Lobby screen.

The screenshot below shows the Lobby screen of the SimpleIdentity sample when the player isn't signed-in to a profile. In this case, the player doesn't have a matching biometric-enabled gamer profile, so was not automatically enrolled during the identification phase above. Our objective in this screen is to encourage the player to manually sign-in to a gamer profile, which will also result in the player being enrolled in the session identity store. The policy of this sample is to not enroll guest players; therefore, if the player starts playing the game as a guest player instead of signing in to a gamer profile, the player is not enrolled.

Figure 3. The Lobby screen.



So, what’s on the Lobby screen?

Guest Player: A title should always clearly indicate the player’s identity before gameplay, though the exact information conveyed will vary from title to title. In this example, the title refers to a player without a gamer profile as **Guest Player**. If the player signs in to a gamer profile, the generated name (**Guest Player** in this case) is replaced with the gamertag associated with the gamer profile. In addition to generated names and a player’s gamertag, a title has many options to uniquely indicate a player’s identity on screen, such as colors, items, and avatars.

Sign In: This button simply opens the Kinect Guide, provided by the system, including a **Sign In** button and screen. To encourage the player to sign-in before beginning to play, it makes sense to put a sign-in button on this screen. The explicit **Sign In** button is provided merely for the player’s convenience, since titles are mandated to always make the Kinect Guide available anyway, through the system gesture. If the player signs-in through the Kinect Guide, the NUI Identity system enrolls the player. For better clarity, when the player is signed-in to a gamer profile, the button’s text changes to **Change Profile**. This is only a cosmetic change, as the button still opens the sign-in UI, in this case to let the player sign-in to a different profile.

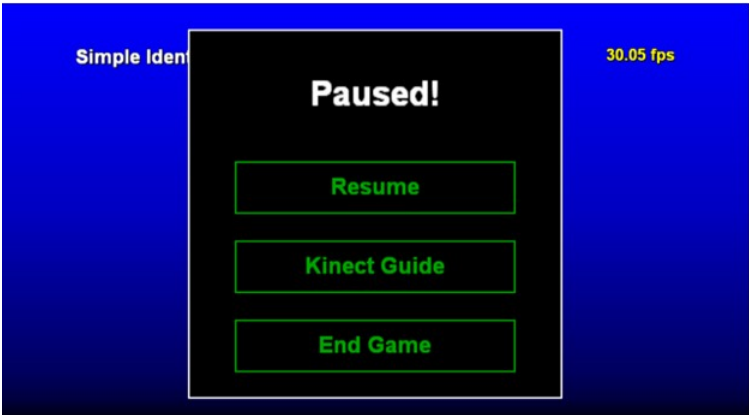
If being signed-in to a gamer profile is essential to having a great experience with the title, it is a good idea to bring up the sign-in UI just before entering the Lobby screen, if the player isn’t already enrolled and signed-in to a profile. The system code associated with the Kinect Sign-in UI will take care of the enrollment process at the same time as signing-in the player.

Start Game: This button launches the game. The game screen is not shown in this paper. A player should be allowed to proceed to the next screen regardless of whether the player has a gamer profile, after being given the chance to sign in to a profile or select a different profile. The next screen could be an additional menu or the game itself, depending on your title. If some features from the title aren’t available, or are limited for guest players, the title should make that clear to the player.

The Pause Menu for Mid-Game Intent to Play and Sign-in

Generally, your title should make a Pause menu available whenever a player performs the system gesture. One option of the Pause menu should send the player to the Kinect Guide; the rest of the menu composition is up to you. The SimpleIdentity sample provides the menu shown below.

Figure 4. The Pause menu.



- The Pause menu offers the following options:
- **Resume:** Returns the player to the game. This also constitutes indicating intent to play for whoever selects this option.
 - **Kinect Guide:** Opens the Kinect Guide, which includes a **Sign In** button and screen. Through system messages, the system will notify the title of any change in profile line-up. Titles should monitor system messages at all times, to be aware of the players’ profile sign-in and sign-out.
 - **End Game:** Ends the game in progress, and returns the player to the Intent to Play screen.

Summary

The following summarizes the basic flow of NUI Identity processing, along with best practices. These best practices help you fulfill the TCRs and provide the best player experience, while keeping the implementation of identity processing simple, yet flexible.

Step	Best Practices for this Step
Attempt to silently identify the skeleton	Call NuiInitialize , activating the NUI Identity system’s skeleton recognition processing, which occurs any time a skeleton becomes visible. Until the skeleton indicates intent to play, don’t interact with the skeleton, such as prompting to resolve image quality issues, or showing the person onscreen.
Detect and report intent to play	Any time a skeleton indicates intent to play, inform the NUI Identity system by calling NuiIdentityDetectedIntentToPlay . This enrolls and signs-in players who have a biometric-enabled gamer profile.
Re-try identifying the skeleton, prompting about quality issues	When the player indicates intent to play, prompt the player to resolve any quality issues reported by NuiIdentityGetNextMessage . If skeleton recognition processing is unable to complete by 15 seconds after indicating intent to play, treat the player as a guest player and allow gameplay to begin.
Enroll and sign-in a player who has a biometric-enabled gamer profile	The NUI Identity system completely handles this step. There’s no best practice defined regarding enrolling guest players. If you do want to enroll a player who doesn’t have a matching biometric-enabled gamer profile, call NuiIdentityEnroll .
Indicate which gamer profile will be used	Before gameplay begins, show which gamer profile or guest identity will be used. For a signed-in player, display the gamertag and avatar from the gamer profile.
Allow identity correction and manual sign-in	Before gameplay begins, provide easy ways for the player to manually sign-in to a gamer profile or correct the identity, such as a Sign In or Correct My Identity option on the Lobby and Pause screens. Allow game participation even if a player is not signed-in to a gamer profile. Don’t write to or

	modify a gamer profile until the player has confirmed that player identity by entering gameplay.
Support identity actions during gameplay	Avoid gameplay interruption that prompts the player to resolve image quality issues or confirm identity. Instead, make reasonable assumptions about player identity, together with providing ways to change the player identity.

These basics get you started with NUI Identity processing. For more advanced title design and development, continue with the other NUI Identity white papers and the Xbox Development Kit (XDK) documentation.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Who Is Playing My Kinect Game Right Now?



By Eric Langlois, Software Development Engineer
Advanced Technology Group (ATG)
Published: August 31, 2010

In a game that requires the Xbox 360 Kinect Sensor, you can no longer rely on a player to sign in with a specific controller, eliminating the controller as a way to identify the player. For Kinect, player identity is handled through the NUI Identity system, which lets you answer the question *who is playing my Kinect game right now?*

Intended Audience

This document is intended for game designers. It offers a logical, high-level view in plain English of how the identity system can be integrated into a title.

For technical details on implementing NuiIdentity in a title, see [Using NuiIdentity Effectively](#).

Understanding Identity

To understand NUI Identity, you need to be familiar with the terminology and some fundamental concepts. This section provides those before covering some advanced topics.

Terminology Used in this Document

The following terms are used throughout this white paper, and basic terms are presented first.

Skeleton

The data structure that holds the tracking data relative to a single player in the play space. (The system can track up to 6 players.) It includes things like tracking status, joint positions, enrollment status, sign-in status, and so on.

Print

The data that uniquely identifies a player and enables the identity system to recognize enrolled players. This data includes biometric information, such as facial attributes.

Session Identity

A *print* stored for the duration of the session. The system can keep up to 8 session identities in memory. Session identities only persist while the console is turned on.

Identification

The process through which the identity system attempts to recognize a player. The identity system collects a print of the player which it compares with the prints from the currently enrolled players. If a match is found, the identity system assigns that print as the session identity for the player's *skeleton*.

Enrollment

The process through which the print that identifies a particular player is added to the session identity store.

Profile

An Xbox profile as stored on the console or a memory card.

LIVE Profile

An Xbox LIVE-enabled profile, which allows players to connect to LIVE.

Kinect ID

A profile option that allows a *print* to be permanently bound to a *profile*. If a player with a matching Kinect ID-enabled profile is *enrolled*, the profile will automatically be *bound* to the player. The profile will also be signed-in, if it wasn't prior to the enrollment request.

Sign-in

The act of signing in a player. This operation is performed by the system, and the title has no control over it. The player can use the Xbox Guide to sign in or out at any time. A player who is signed out is automatically unenrolled by the identity system.

Bind

The act of associating a user profile and a print. The identity system manages the binding process; it is transparent to the title.

Carry Over Players

Players who are already signed in and enrolled when a title is launched. Players may have signed in using the Xbox Dashboard or another title. All signed-in players are retained when a Kinect title starts.

Intent to Play

The way players confirm that they want to engage with the title.

Unenrollment

The action of removing a specific print from the identity session store. A title can only unenroll prints that are not bound to a profile.

Fundamental Concepts

A player's interaction with a Kinect-enabled title begins with *enrollment*. If the player chooses, he or she can then *sign in* to a profile. Gameplay actually commences when the player shows *intent to play*.

Enrollment

The enrollment process enables the identity system to uniquely distinguish one player from others. The data collected for this can be used to maintain a strong link between a player and his or her skeletal representation for the duration of the session.

Enrolling a player is a two-step process that begins the moment a player is tracked as a skeleton and ends when an identity session has been created for the player. These steps always happen in the same order, though some are hidden from the title, depending on the situation. At other times, the title will play an active role in each step.

Step 1: Identification

As soon as a player starts being tracked as a skeleton, the identity system advises the title that it should start the identification process. The title is responsible for initiating the process, which lets the application control when identification happens and also do some pre-processing. In most cases, however, the title should simply comply when the system advises it to identify the player.

During the identification process, the title should relay the feedback from the identity system to the player. At the end of step 1, the title knows whether this player is known to the system or not.

Step 2: Enrollment

Once a player has been identified and no matching profile has been found, the title can enroll the player, which creates an identity session for this player. By the end of the enrollment step, the player's print has been added to the enrolled players (up to 8), a list maintained by the identity system.

All this data is kept for future reference while the console is turned on. So, if a player leaves the play area (for whatever reason) and comes back later, resolution will be faster. Barring any error, the player will be recognized seamlessly during the identification step, and no enrollment will be necessary.

Once your title has started the identification process, the system automatically enrolls the skeleton if the player's identity already has a matching identity session.

Sign-In

If a Kinect ID-enabled profile matches the print of the player a title is enrolling, the profile and the player will be automatically bound during the enrollment phase. Otherwise, players can call up the Guide at any time to manually bind a profile to their session identity. The title can also call the Kinect sign-in menu on the player's behalf to provide a quick path to signing in for the player. Either way, signing in is always controlled by the player, not by the title.

Signing in is optional, in that a player isn't required to sign-in to a profile. For some games, this is fine, while other games may need a profile to offer a more complete experience (for example, access to Xbox LIVE saved games, rich presence, automatic avatar assets). There are ways to encourage the player to sign in to a profile, described later in [Single-Player](#).

Intent to Play

Intent to play is a mechanism designed to prevent unintended interruptions in the interaction between a player and the title when individuals enter the play space, either intentionally or accidentally. A title should only accept commands from and engage with players who have shown intent to play.

There are as many ways for a player to show intent to play as there are title concepts, and each title must determine how players should show that they intend to play.

A few basic ways that a player can show intent to play would be:

Gesture and Voice Command

If someone who is in the play space performs a specific gesture, a title could consider this as showing intent to play. *Waving* is the gesture that a player performs to engage with the Guide. Waving is also general enough that any title can use it for intent to play, but a title can also use a gesture specific to its theme. Once the player has shown intent to play, the title enrolls him or her.

Sign-In

A title could make the sign-in option available immediately, before the player is identified or enrolled, since the Guide will take care of identification and enrollment as needed when the player signs in.

Correct My Identity

As for sign-in, accessing the **Correct my Identity** menu could be considered intent to play on the part of the player.

Once players have been enrolled, they should be allowed to engage with the title. In the most basic case, it means selecting **Play** on the menu.

Advanced Topics

The followings concepts are also important to understanding identity, and they build upon the concepts described earlier in [Fundamental Concepts](#).

Carry-Over Players

Enrolled players who are already signed in to a profile are automatically retained as enrolled players when a Kinect-enabled title launches. Your Kinect-enabled title should look for players who are already enrolled during its initialization phase, and your title should use that information to speed up the enrollment process, allowing the player to get in the game faster.

While a player who was already enrolled when the title launched is still enrolled after launch, each player should still show intent to play in order to engage in a meaningful manner with the title.

It is of the utmost importance to honor existing enrollments because they represent users who are already signed in, and they shouldn't need to choose their profile again. Enrolling those players as new would result in loss of connection to their profile.

Players Can Come and Go at Any Time

There are many reasons for players to leave and re-enter the play space. Some exits are involuntary or accidental, while others are deliberate. When a player exits involuntarily, it usually means that the Kinect Sensor's view of the player became occluded by an object or another player or that he or she disappeared from the camera's field of view (for example, by falling to the floor or stepping out of the play space). In these cases, after the player is no longer occluded or re-enters the sensor's field of view, the player will be reacquired rapidly by the system and immediately re-identified.

Most of the time everything will happen so quickly the player won't notice, but there is no guarantee, so the title must be ready to guide players at all times.

If tracking is lost, depending on the style of gameplay, the title could decide to pause the game until tracking is reacquired. From time to time, tracking may be lost for a few frames, and your title should not pause immediately, because the loss of tracking may be transient; for example, a player can accidentally step out of the play space or pass behind another player or object and then be immediately re-identified by the system.

The specific nature of your title will dictate exactly how long a player should disappear before the game is paused. As a rule of thumb, if tracking is lost for more than a few seconds, the game should enter its pause mode until the lost player or players are re-acquired.

Sometimes auto-enrollment will fail after the identity system attempts to re-identify a player. This usually happens because the identity system cannot single out the corresponding enrollment data amongst all the enrolled players. When this happens, the title must guide the player through the enrollment process and possibly ask the player to select a profile again.

Various factors may contribute to this situation: two or more player may be too similar, the lighting conditions may have changed since the original enrollment, or players may have changed clothing. These are all examples of aggravating factors.

In a Kinect-enabled title, a player can come and go at any time, and a title should always be ready to react to this situation. Though the exact reaction will vary depending on the type of game, the title should always acknowledge the change in player line-up and clearly indicate to the players what changes are taking place and how it will impact the game in progress. For some games, this will be an expected and normal behavior, while for others it will be exceptional and may have a profound impact, possibly even halting the game in progress. In all cases, though, it is important that a title steer the player through the process.

Players can also sign in or sign out through the Guide. The Guide automatically enrolls players before signing them in, and it will automatically un-enroll a player who signs out. The Guide also:

- Let's the player play as someone else.
- Always forces enrollment, so there can be profiles signed-in without skeletons attached.
- Uses a new enrollment index if a signed-in player signs into a different profile.
- Let's users complete the sign-in operation who are not the person who opened the Guide.

Profiles

You should be aware of *family profiles* and *manual sign-in*.

Family profiles may be associated with multiple players who share a single profile. While you should be aware of this possibility, family profiles should not affect your title, since profiles are managed by the Xbox 360 system.

Anyone can sign-in manually to any Kinect ID-enabled profile. If someone does this, and the true owner (identified biometrically) shows up later, sign-in for a Kinect ID-enabled profile won't occur for the owner.

Skeleton ID, Enrollment ID, and User Index

The numbers of IDs available in NUI can be confusing at first glance. A Kinect-enabled title has the following maximums:

- 8 maximum enrolled players
- 6 maximum simultaneously tracked players
- 4 maximum simultaneously signed-in players
- 2 maximum actively tracked skeletons

Some of these numbers (like the number of signed-in players) exists for legacy reasons while others (6 simultaneous tracked players) are due to technical requirements. It isn't really that complicated though:

- There is a direct link between the data in the depth mask and the skeleton data structures. The mask uses values from 1 to 7 to distinguish the player pixels from the rest of the environment, which uses 0. Subtracting 1 allows you to directly index one of the skeleton data structures.
- A skeleton can be *not tracked*, *tracked by position* (no joint data), or *fully tracked*.
- A fully tracked skeleton (up to 2 of the total 6) can be linked to an identity store in memory. It can also be linked to a profile if that identity is bound to one.

That's all there is to it.

Unenrolling a Player

A title can unenroll a player, as long as the player isn't signed-in to a profile. This can be used to allow new players to enroll when the limit of 8 enrolled players has been reached.

Forced Enrollment and its Consequences

It is sometimes necessary for a title to force enrollment of a player. If the identity systems fail to enroll the player in the normal way, for example, a title will have little choice but to forcibly enroll the player. Doing so will allow the title to treat the player in much the same way it would treat a player who has enrolled normally (whether the player is bound to a profile or not). However, forced enrollment won't allow automatic re-identification of the player when tracking is lost, because the system won't have a print of the player to match.

If tracking of a player who was forcibly enrolled is lost (either accidentally or intentionally on the part of the player), the player must go through normal identification and enrollment, or the

player will likely be forcibly enrolled again when he or she comes back. Furthermore, if the player was bound to a profile, he or she will have to identify the profile to bind to the skeleton again.

You must determine how important it is for your title to identify the player in any given situation. For example, if tracking is lost and re-acquired in the middle of a player's turn, the title is welcome to let the player continue to play as the player whose turn it is without re-identifying or forcibly enrolling the player.

Impact on your Title

Identity may have some technical impact on your title. During the identification and enrollment phases, the color stream will be redirected to the identity system, meaning that for a length of time, your title won't get the color feed. The color feed will be restored when the system is done with the identity operation.

If a title needs to access the color stream during identification, it can cancel the identity operation or it can wait to initiate identification at some other time that is more opportune. If a title makes heavy use of the color stream, it may have to display a special UI during identity operations.

The Scenarios

Keep in mind that many titles support more than one type of scenario, which could be single-player, multiplayer, or jump-in.

Single-Player

In a typical single-player scenario, the player usually engages with the title for an extended period of time, often split over multiple sessions. The title uses saved games to keep track of the player's progress and accomplishments.

Since a user profile is required to save the player's progress, the title should provide a clear flow to encourage and facilitate player sign-in. Nevertheless, the title should not force the user to sign in to a profile, and it should offer at least some functionality to a player without a signed-in profile. A player profile is also required in order to connect to Xbox LIVE.

A title can use a strategy similar to the one described here to encourage the player to sign-in to a profile. Simply place a Lobby screen right after the title's mandatory Intent to Play screen (described later in this paper); which should follow immediately after the title's introduction (including legal notice, publisher & studio logos, and so on). Think of the Intent to Play screen as the replacement for the Press Start to Begin screen that you would find in a non-Kinect title.

Inserting a Lobby screen at this point provides an important opportunity for the player to ensure that he or she has been correctly identified by the identity system and bound to the correct profile. By providing all the services that players need to correct their identity and sign-in to a profile in one spot, the title provides the player with an opportunity for the best experience. This solution doesn't solve every problem, since the player can sign in or out at any time through the Guide, and players can leave or enter the play space at any time. Your title still must react to those events, but providing a stage that is clearly for enrollment and sign-in will go a long way to providing players with a simple, straightforward experience for enrollment and sign-in.

Intent to Play Screen

Display of the Intent to Play screen is the first time, after the introduction, credits, and other initial screens, when the player actively engages with the title.

Lobby Screen

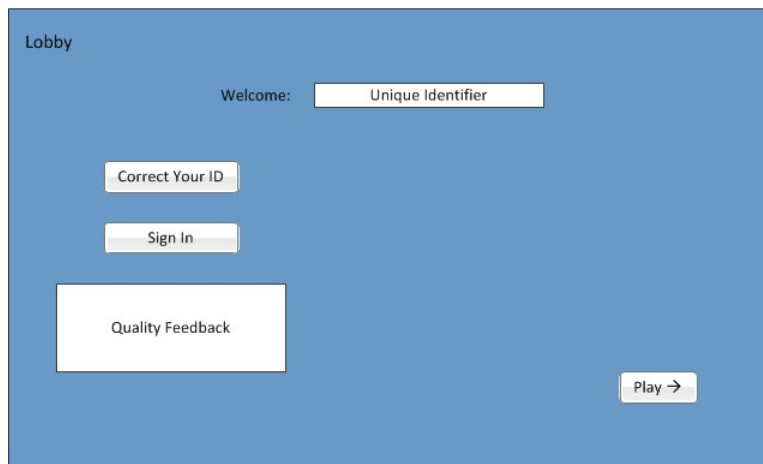
The following image gives a rough idea of what a Lobby screen could look like. Our objective is to have the player enrolled and signed-in before he or she leaves this screen.

The enrollment sign-in process happens in three phases for each player:

1. Identification
2. Enrollment
3. Sign-in

These steps always happen in this order, but the time required to complete this sequence can be drastically shortened if the player is already enrolled and signed-in (from another title or the Dashboard) or the player has biometric sign-in activated. In some case, these steps will seem to the title to happen simultaneously during the identification phase.

Figure 1. Lobby screen



So, what's on that screen?

Welcome

A title should always clearly communicate to the player his or her current identity. The exact information conveyed will vary from title to title. In this example, the title automatically generates a unique name for the player. If the player signs in to a profile, the generated name is replaced with the gamertag associated to the bound profile.

In addition to generated names and a player's gamer tag, a title has many options to uniquely identify a player on screen: it can use things like colors, items, avatars, and so on.

Correct Your ID

Whenever players are identified, there is a risk, albeit small, that they will be misidentified. Thus it is important that a title provides a means for players to correct their identity. A title will usually provide a link to a **Correct your ID** menu in the title's own pause menu. Since the Lobby screen is all about identifying, enrolling and signing-in the player, it makes sense to offer the option right away. This button becomes available after the player has been enrolled. (You can't correct your ID unless you've been identified, right?)

Sign In

Since we want to encourage the player to sign-in before beginning to play, it makes sense to put a sign-in button on this screen. This button simply opens the same system sign-in UI that the player can access from the Guide at any time; the location of the **Sign In** button here is just for the player's convenience. This button could be available at all time, or only after the player has been enrolled—it is up to you. The Guide will identify and enroll the player, if necessary.

As an alternative or in addition to the sign-in option on the Lobby screen, a title can also directly ask the player after he or she has shown intent to play, and before displaying the Lobby screen, if she or he would like to sign-in. If so, the title can immediately bring up the sign-in UI, which will take care of the enrollment process at the same time.

A title can also warn the player when he or she leaves the lobby without being signed in to a profile, and offer another chance to send the player to the sign-in UI.

Quality Feedback

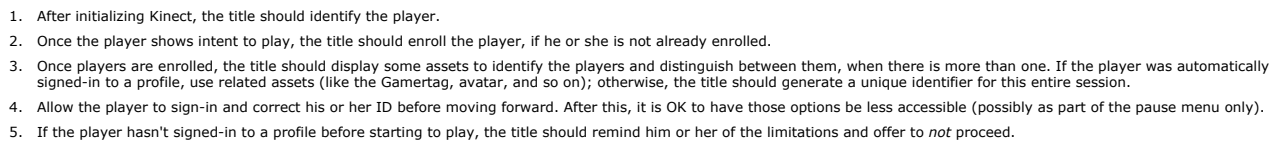
During the identification or enrollment process, NuIDentity constantly provides the title with status on its progress and the issues it is encountering while trying to identify the player. It is important that the title relay this information to the player by, for example, asking the player to move closer to the Kinect Sensor or to look directly at the Kinect Sensor. The title can use on-screen text, audio cues, and other techniques that guide the player through the identification process.

Play

Once a player has shown intent to play, he or she should be allowed to proceed to the next screen.

A diagram of the workflow associated with the Lobby screen is shown in Figure 2.

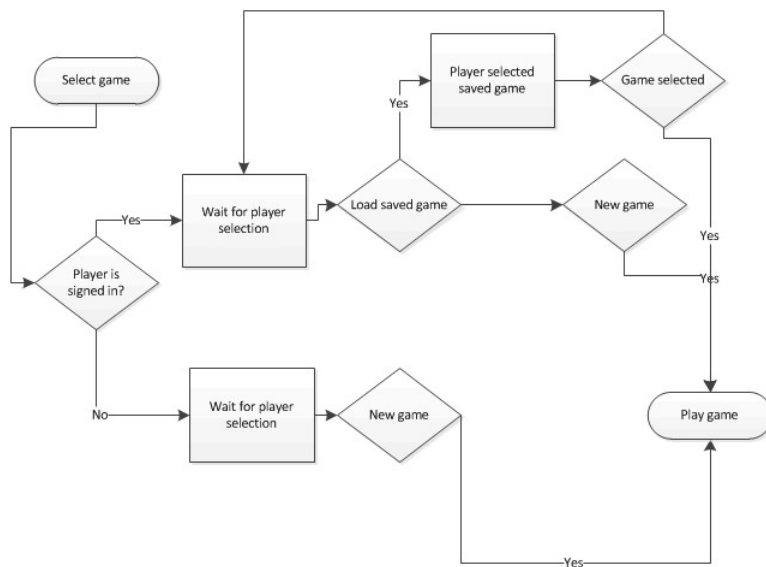
Figure 2. Workflow diagram for the Lobby screen



If the player isn't signed-in, the only option available is **New Game**. If the player is signed-in, he or she can start a new game or load a previously saved game associated to the profile. Then the game can begin.

A screenshot of a game selection menu. The background is a solid orange color. At the top left, the text "Select Game" is displayed in a white, sans-serif font. Below this text, there are two rectangular buttons with rounded corners and a light gray gradient. The first button, on the left, contains the text "New Game" in black. The second button, on the right, contains the text "Continue Game" in black. At the bottom left of the screen, there is a third button, also with rounded corners and a light gray gradient, containing a black left-pointing arrow followed by the text "Back" in black.

Figure 4. Workflow diagram for the Select Game screen



Correct Your ID Screen

The Correct Your ID screen allows a player to correct a faulty ID. This screen should always be available from the title's pause menu, and it is a good idea to make it available directly from the interface that players use when enrolling.

Multiplayer

In a multiplayer game, the flow of the title launch menu is very similar to one for a single-player title. The differences are essentially in the Lobby screen. While the Lobby screen of a single-player title deals with a single player, the Lobby screen for a multiplayer title has to cater to many players.

Intent to Play Screen

The Intent to Play screen is the same as the Press Start to Play screen seen on non-Kinect titles. As soon as a player, enrolled or not, performs the title's gesture for engaging, the title advances to the Lobby screen.

Lobby Screen

On the Lobby screen, the title provides an opportunity for the players to enroll and choose their profiles. The player who performs the gesture has already shown intent to play and shouldn't have to show it again. Like for a single-player game scenario, the Lobby screen is a great spot to let a player sign-in to a profile, select game-specific game assets, and correct their identities. When player selects **Play**, the title advances to the next menu screen.

Select Game Screen

On the Select Game screen, a signed-in player can load a previously saved game and, in the case of an Xbox LIVE title, connect to an Xbox LIVE gaming session. If the player has not signed-in to a profile the player will be able to start a new game but won't be able to save his or her progress.

Jump-In

For a jump-in title, players are expected to come and go during the game play. Special care must be given to designing the menu flow.

Jump-in games have some peculiarities when it comes to player identification and enrollment. Just like the name implies, players are expected to jump in and out frequently. Players will usually compete for short challenges and then the line-up will change.

Because they don't require a long-term commitment from the players, there is no need to save progress. Jump-in games are often played in party situations when many people won't have a profile on the console, so the whole title should be designed with these constraints in mind. The goal is to get players into the game as quickly as possible.

Bearing that in mind, for jump-in titles, you may want to allow guest players to jump in and play without being enrolled. Doing so can help to avoid disrupting the players' experience. Nevertheless, identity is especially useful in two situations for jump-in titles:

1. During game play, identity allows a title to distinguish between the players engaged with the title. In situations when tracking is lost, identity can assist in keeping track of players.
2. Though players might not be interested in tracking progress across sessions, they might be interested in an accumulated score for their entire play time, and identity can assist a title in providing this.

A jump-in title behaves differently than more traditional titles when a change in the player line-up occurs during a play session.

In a single-player game, player change isn't expected, and the player playing is usually signed in to a user profile to maintain progress between play sessions and publish his or her achievements. Player change is often highly disruptive to the game, and the title should notify the player immediately so that he or she can take appropriate actions.

At the other end of the spectrum lie the jump-in titles. In these titles, games are shorter and progress isn't usually kept between sessions, and many players could take part in a play session. For such titles, players don't usually attach the same importance to their achievements or progress as they do for single-player titles; they only want to have fun in the moment. Because of the nature of jump-in titles, a change in player line-up is more likely to happen.

Before we look at how a typical jump-in title can handle player changes, allow me to introduce Kinect Mimes. Kinect Mimes is a fictional Xbox 360 party game made up for the purpose of this discussion. In Kinect Mimes, or just *Mimes* for short, players attempt to reproduce and hold a specific pose for short period of time. A successful pose earns a point. The game is played in rounds during which one or two players face the sensor array and replicate the poses that the title character strikes. Two players may score a point at the same time, for the same pose.

Kinect Mimes features:

- A party mode where players can come and go at any time.
- Short rounds with a predetermined amount of poses to attempt.
- A two-player mode in which the players play simultaneously.
- A single player mode in which the player can track personal achievements over multiple sessions.
- Session leaderboards.

The title keeps scores during a session but not in between so that players can play multiple times during the evening and see their stats.

Let's see how we can use identity without delaying the players' enjoyment. We want to ease players into game play.

Menu Flow for Jump-In Game Scenarios

The first thing to do is determine which player is in charge of the menu. If there are many people gathering in front of the Kinect Sensor, it may be hard to determine who is in charge, especially if people are moving around.

The good news in this situation is that only two skeletons can be tracked simultaneously, and unless tracking is lost, they will stay linked to the same player. After the introductory screens and content, the players will face the Intent to Play screen. The first player with a tracked skeleton to wave becomes the controlling player in the UI, and no other player can control the screen unless he or she has shown intent to play. If more than one player qualifies, and the title can't determine which one should be the controlling player, Mimes prompts the user who wants to control to wave.

Now that we have a means of deciding which player controls the UI, let's see what the UI has to offer. Here, Mimes wants to get the players in the game as fast as possible.

The fastest way for the controlling player to get into the action is to select **PLAY**. Mimes lets the player select a profile, but will not enforce it. If a player is bound to a profile, Mimes uses the profile's Avatar. Otherwise, it assigns a random avatar. A title could also let a player customize his own player assets, but this should be optional to avoid slowing down the game flow.

The Guide takes care of enrolling players that sign-in using a user profile.

Each time that a new player is ready to be identified, Mimes tries to identify the player and provide on-screen feedback.

Players Leaving and Entering at Will

Imagine two players are engaged and attempting the poses demonstrated by the virtual coach. How should the title handle a change in the player line-up? What if a player disappears? What if both players disappear? What if a player is replaced by another person?

For a player who has disappeared, behavior depends on how long the player has been gone. If the player comes back right away, Mimes will try to ignore the fact that the player appeared to leave and will continue as if nothing happened. Maybe the player didn't mean to leave the game, or maybe someone passed in front of the player, effectively hiding the player from the sensor array for a short time. Maybe Mimes' challenge was too much for the player, and the player ended up falling on the floor!

There are many reasons a player may seem to leave the play area. In general, titles are advised to wait a little while for the player to come back before taking any disruptive action. The length of the wait will vary by title, but this kind of wait should be no longer than a few seconds (not minutes).

If it seems that the player has really left, then what? The answer depends on how disruptive it is to the player experience. In Mimes, we just continue. Mimes removes the player's avatar, so it is clear that the player isn't being tracked anymore. It is a good idea to design jump-in games so this isn't disruptive, because of the environment in which such games are often played. In most cases, the game should be able to just keep going. At the same time, the game should make it clear that tracking was lost.

Even though Mimes doesn't pause the game when a player goes away, a remaining player can always pause the game, if he or she wants to do so, by performing the system gesture to call the pause menu.

What if both players go away? In this case, pausing is probably the best idea, since no one is playing anymore. The title may want to wait a little before taking disruptive actions, because the absence of both players may just be a fluke, and one or both players will come back in no time.

The link to the identity session will have been severed, regardless of whether tracking was lost for just a few frames or for a long time. In this state, the title will be able to track the player's movements again, but it can't be sure who is controlling the skeleton until it tells the identity system to identify the player.

So the first questions to answer are: *How important is it for the title to know who is playing? Does it really matter if someone else stepped up and replaced a player?* In Mimes, we made the design decision that it didn't really matter.

If play has begun again after loss of an identity session, it is probably a good idea to re-identify the player, but depending on how you answered the previous question, your title will handle the feedback and results from the identity system differently. You may want to try to do this in the least disruptive manner possible, without interrupting the game in progress. You could choose not to present feedback to the player or limit feedback, depending on how important the link to identity is for your particular title.

If the operation fails and the player's identity isn't that important, a title can simply keep going without identity. A title can also try to re-identify the player from time to time, providing it doesn't disrupt the game. For its part, Mimes keeps the assets associated with the most recently identified player.

If the identity is important and identification fails, the title can consider halting the play in progress and guide the players through the steps that must be taken to return to play mode.

Depending on whether your game relies heavily on players having distinct identities, you may have to pause and offer options or restart the play, or you may want to just acknowledge the new player. Whatever you do, the choice is up to you, but it is good practice to avoid disrupting the players as much as possible.

So, how should a jump-in title react when one of the two actually engaged players disappears? We want to let players have fun and not disrupt their experiences.

Whenever skeleton tracking is lost and reacquired, the link to the identity session will be lost for that player. Your title should probably reacquire it, even if only silently, because the two players could still be the same but may have switched positions in the play space. Keeping them identified will allow your title to effectively track the players' movement in such situations. As an alternative, you could design your game in a way that players are not allowed to cross the play space and switch sides. How much you should insist on identifying players in the middle of play is up to you, and right choice depends on what provides the best play experience for the players.

Summary of Best Practices

To summarize, keep the following eight identity concepts in mind when designing a Kinect-enabled title:

- Guide the player. Provide an easy way of doing things, but avoid forcing the player to enroll or sign-in.
- Be ready for players enrolling and signing in to a profile at will through the Guide. The player line-up may change at any time.
- Focus on the player's enjoyment. If profiles aren't necessary, then don't require the player to select one.
- If it doesn't matter whether the player is the actually the enrolled person, don't force the player to enroll. Just let the new person impersonate the original player.
- Provide feedback during identity operations.
- Consider providing opportunities for players to enroll and sign-in before the beginning of the game.
- Honor players who are already enrolled when the title starts by not requiring them to enroll or show intent to play again.
- Players should be required to show intent to play only once.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Using NuiIdentity Effectively



by Eric Langlois, Software Development Engineer

Advanced Technology Group (ATG)

Published: May 4, 2010

Updated: January 7, 2011

Abstract

Knowing which skeleton corresponds to which player is crucial for Microsoft Kinect for Xbox 360 games. Although with a single player the problem is simple, things can quickly become complicated in a game supporting eight players rapidly moving in and out of the sensor's field of view. This challenge is managed by the identity subsystem (or NuiIdentity), an important part of Kinect and an integral part or the player experience. Moreover, consistent usage across titles is essential to ensure quick and easy enrollment of the players to minimize the time spent before entering play. This document discusses how to handle player enrollment in a Kinect-enabled title. It pays special attention to jump-in game scenarios providing guidance to implement clear and consistent control flow. It also discusses providing user interface (UI) text to let players change identity and process quality feedback messages.

This white paper includes the following topics:

- [New Challenges](#)
- [What Can NuiIdentity Do?](#)
- [Implementing NuiIdentity into Your Title](#)
- [Impact on Your Title](#)
- [Takeaways](#)

New Challenges

At its core, the skeletal pipeline does not recognize specific players—it only discerns the human body shape and uses it to produce a skeleton. This means that the link is weak between a skeleton and the player controlling it. The link can be broken at any time and can go unnoticed by the player or the skeletal pipeline (and, thus, the title).

It is true that once a link is established between a specific player and a skeleton it often stays that way for the remainder of the session. There are circumstances, however, that can break that fragile link. For example, if players step out of view of the camera and then step back in, tracking is lost for a short time and the players may or may not be assigned to the same skeletons. This also is the case if a tracked player steps out of view of the camera and his or her place is taken by another player. In both cases, the skeletal pipeline (and the title) would not know that the player linked to the skeleton changed.

This means that the skeleton and player association may change during the course of a game without the title recognizing it and sometimes without the player realizing it. It makes it almost impossible to associate players with things like game data (scores, assets, and so on), Xbox profiles, et cetera.

Until the advent of Kinect, a title would use the controller port number to identify the player and associate him or her with game assets and, when chosen, a player profile. Once a player selected a controller, the game could communicate with the player and assume he or she would be the same person. Players could exchange controllers if they wished to, but a controller port

would never be reassigned without the player and the title recognizing it.

There is an obvious need for establishing a strong link between a player and a skeleton in Kinect-enabled titles just as there is a need to establish a strong connection between a player and a port number in a controller-enabled game. The NuiIdentity library provides such a solution, offering a means for binding a skeleton and a player, including their profile, much like titles using controllers can match a player to a specific controller and their profile.

What Can NuiIdentity Do?

NuiIdentity provides services to help a title manage player identities and to track a player's association with the skeletons and gamer profiles. More specifically, NuiIdentity can link a player identity to a skeleton. To this end, it collects visual information to establish a player's identity and provides an enrollment index that allows the title to keep track of the player currently associated to a specific skeleton.

NuiIdentity also can maintain a link between an enrolled player and an Xbox 360 gamer profile. The sign-in process is managed by XAM, and the player (through the Guide) or the title (through a call to XShowNuiSigninUI) can initiate the sign-in process. Profiles enhance the player's experience and, when possible, a title should make it optional for the players to sign in to let the players enter the game as rapidly as possible. If the intent is to play on Xbox LIVE, then the players must be signed in. Signed-in players have the advantage that their enrollment index and identity data are carried over from title to title.

Distinguish Among up to Eight Players

NuiIdentity keeps identity information for up to eight players, allowing it to recognize and distinguish among those players and automatically bind and rebind them to a skeleton whenever they step in or step out of the field of view of the sensor array. This enables jump-in scenarios for games that want to allow more than the two players that can be actively tracked to participate in a game by exchanging positions so that each can play in turn. Four of the eight enrolled players can be signed in and linked to an Xbox 360 player profile at any time.

Special care was given to ensure that NuiIdentity would support jump-in scenarios, allowing players to join and leave game sessions quickly and easily with minimal impact on game flow. When a player is first identified by NuiIdentity as an unknown player, it is the responsibility of the title to request enrollment for this player, usually performed once the title has determined that the player has signaled intent to play. If the player leaves and returns, NuiIdentity notifies the title that the player needs to be identified again. This time when the title calls **NuiIdentityIdentify**, however, NuiIdentity identifies and recognizes the player as already enrolled and quickly makes the association. The title does not need to explicitly enroll the player.

NuiIdentity also provides services that allow a title to exchange player enrollment index and player identity data. A service is provided to allow titles to correct misrecognitions. A player also could use this to impersonate another player; there is no reason to prevent it.

Provide Feedback to the Title

Collecting visual data to uniquely identify a player and to distinguish between players using that data is a complex, and sometimes lengthy, process. Among other things, the process requires that multiple frames from the sensor be captured and analyzed (see [Impact on Your Title](#) for a detailed discussion of associated system costs). For this reason, NuiIdentity provides the title with as much control and feedback as possible in order to allow it to efficiently manage the impacts on the title's performance and game flow.

Identification and enrollment operations, for example, are generally initiated by the title, although there are some situations when identity operations can be initiated without the title's control; a player interacting with the Guide could cause such a situation. Note that in these situations, identity operations fit within the official skeletal pipeline budget. Identity operations are asynchronous, enabling the title to continue to do work in parallel while receiving regular updates. Identity progress is communicated through event callbacks for every skeleton frame processed by allowing the title to monitor the operation and relay the feedback to the player. The callback mechanism also allows the title to cancel the operation at any time.

These callbacks provide information about the quality issues (see [Real-Time Feedback](#) for a more detailed discussion of quality issues and quality flags) the identification or enrollment processes are facing. The title then can use that information to guide the player, so that the identification and enrollment processes quickly can gather the best possible data to make the process as accurate and as fast as possible. Some quality issues are easier than others to fix for the player. For example, a player should be able to correct their stance easily, but may have a harder time correcting the surrounding environment.

Cooperate with the Title

Providing a smooth experience to the players requires cooperation between the title and NuiIdentity. A breakdown of the responsibilities to the overall process is shown in the following table.

Table 1. Division of responsibilities in identification of players

NuiIdentity	Description
Assigns enrollment index to skeletons	Displays assets on screen to indicate enrollment status of the active players.
Provides feedback to title	Uses feedback from NuiIdentity to guide players (by providing on screen text messages, visual or audio cues, etc.) through the process of identification and enrollment.

Implementing NuiIdentity into Your Title

Typical Implementation Overview

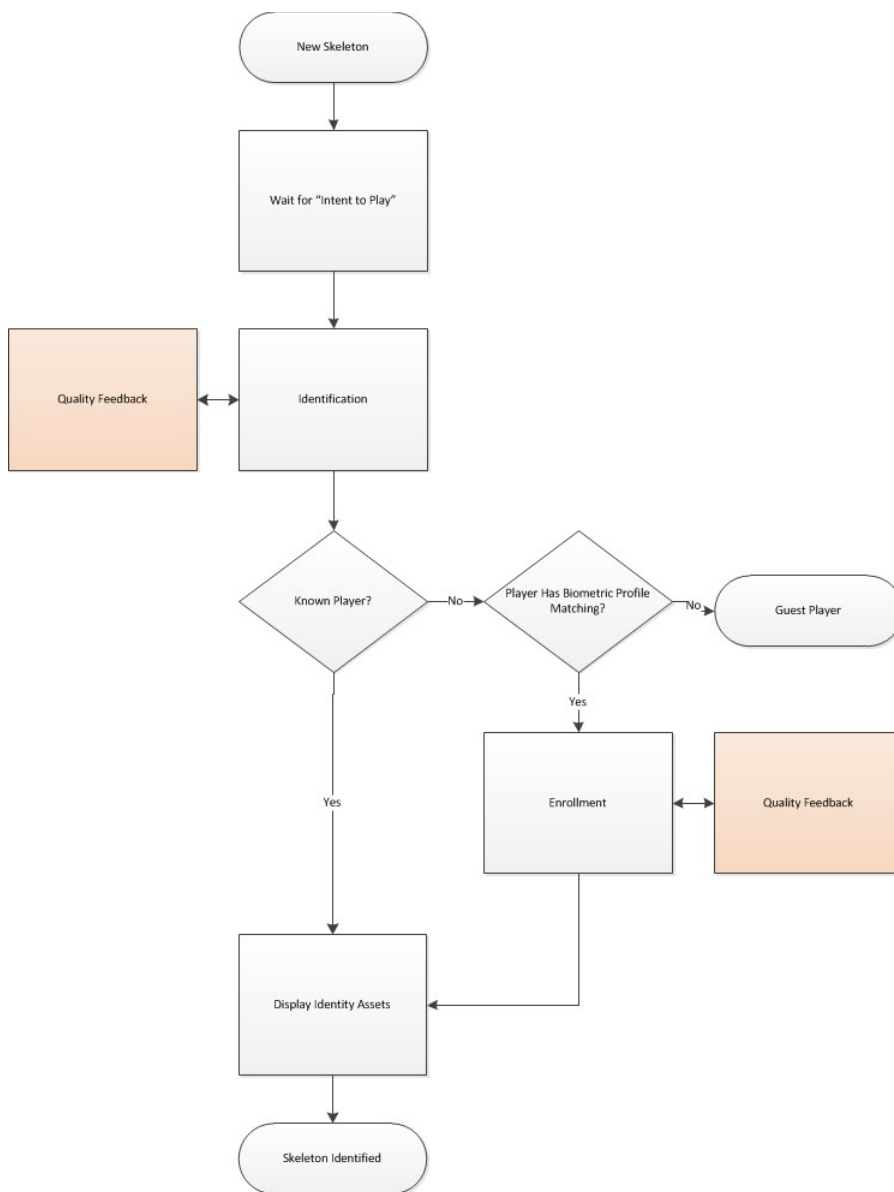
Technical Certification Requirements (TCRs) demand that titles enroll players that have shown intent to play and for which a matching biometric profile exists. For most titles, this is all the identity support needed—along with providing a means to bring the sign-in UI—to allow players to correct their identity or to sign in to a profile when a matching biometric profile is not found.

Titles that want to distinguish between players at all times need a more thorough implementation of NuiIdentity. For example, a title that does one or more of the following: assign different visual assets to identify the players, keep track of individual scores and track individual performance over multiple rounds, would need to manage the identity of all the players, regardless if they are signed in to a profile (from a biometric match or through the sign-in UI).

NuiIdentity uses the **dwEnrollmentIndex** field in the **NUI_SKELETON_DATA** structure to communicate the enrollment status of the player bound to a specific skeleton. A title should check this field for every tracked skeleton (remember that if a skeleton is not tracked, the rest of the data in the **NUI_SKELETON_DATA** structure is undefined and cannot be relied upon) every time the title obtains a new **NUI_SKELETON_FRAME** structure by calling **NuiSkeletonGetNextFrame**.

The following flow chart illustrates how a title could manage **dwEnrollmentIndex**.

Figure 1. Managing member of NUI_SKELETON_DATA structure



A Kinect-enabled title implementing player identification and enrollment must react appropriately and in a timely manner to changes in the **dwEnrollmentIndex** field of the **NUI_SKELETON_DATA** structure. After a successful call to **NuiSkeletonGetNextFrame**, **dwEnrollmentIndex** contains either a valid enrollment index between zero and the maximum number of enrolled players supported as specified by **NUI_IDENTITY_MAX_ENROLLMENT_COUNT**, or one of the values shown in the following table.

Table 2. Values of dwEnrollment when not a valid enrollment index

Value	Description
NUI_IDENTITY_ENROLLMENT_INDEX_BUSY	NuiIdentity is busy and cannot accept new requests for this skeleton.
NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY	NuiIdentity has not yet identified this skeleton.
NUI_IDENTITY_ENROLLMENT_INDEX_FAILURE	Previous operation on this skeleton has failed; for example, the operation was aborted.
NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN	NuiIdentity successfully identified this skeleton as unknown.

If the value of **dwEnrollmentIndex** is between zero and **NUI_IDENTITY_MAX_ENROLLMENT_COUNT**, then the skeleton is assigned to a player that is enrolled in the system and uniquely identified. Your title should be able to display special assets to communicate to the player that he or she is enrolled in the game. This is the state a title should be in most of the time.

If NuiIdentity sets **dwEnrollmentIndex** to **NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTIFY**, it means that this is a tracked skeleton assigned to an unidentified player and that the title should call **NuiIdentityIdentify** so that NuiIdentity can try to identify it. Deciding if NuiIdentityIdentify should be called at this point usually is simple. The reason NuiIdentity does not do this automatically is to allow titles to control the moment when the identification process runs and, thus, when resources are used. In a typical scenario, you will just want to call **NuiIdentityIdentify** immediately.

If **dwEnrollmentIndex** is set to **NUI_IDENTITY_ENROLLMENT_INDEX_UNKNOWN**, it means that a previous call to **NuiIdentityIdentify** succeeded and the player controlling this skeleton was identified, but not enrolled. This means that it is the first time the player has joined the session and, though we can distinguish him or her from other players, the player is not in the enrolled player database. The title will have to call **NuiIdentityEnroll** in order to add the player to the enrolled player database and to establish a link between this skeleton and the player identity data. In order to avoid enrolling people who are idle or just passing by, a title should only call NuiIdentityEnroll if the player demonstrated intent to play. Note that it is not necessary to enroll a player unless they have a matching biometric profile. It is the title's onus to determine what intent to play means. A player waving his or her hand, for example, could be considered showing intent to play. (Intent to play is discussed in [Best Practices](#).)

Note that if **NUI_IDENTITY_MAX_ENROLLMENT_COUNT** players are currently enrolled, a subsequent call to **NuiIdentityEnroll** will fail with **E_NUI_IDENTITY_ENROLLMENT_LIMIT**. There are many options available to the title when this situation arises (see [Ninth Player](#) for more detail).

Finally, if **dwEnrollmentIndex** is set to **NUI_IDENTITY_ENROLLMENT_INDEX_BUSY**, it means that the system is already identifying or enrolling the person associated with this skeleton (usually at the title's request, but the request also could have been initiated by the Guide). New calls made to NuiIdentityIdentify or **NuiIdentityEnroll** for this skeleton at this time will fail with **NUI_IDENTITY_ENROLLMENT_INDEX_BUSY**.

Real-Time Feedback

Feedback for asynchronous operations for the NuiIdentity family of APIs is provided through event callbacks instead of the more traditional [XOVERLAPPED](#) method. Experienced Xbox 360 developers will be familiar with the **XOverlapped** structure as the mechanism a title generally uses to obtain feedback from asynchronous operations. NuiIdentity uses a different approach where the title provides a callback function that NuiIdentity calls on a regular basis to provide feedback, removing the need for the title to actively query the state of the operation through an **XOverlapped** structure. The new callback system helps promote cleaner code that is easier to understand and implement.

Though specifying a callback function when calling **NuiIdentityIdentify** or **NuiIdentityEnroll** is not required at the API level, it is an essential part to facilitating the user enrollment experience and to implementing NuiIdentity correctly. This is the only way a title can know if a biometric profile match has been found for a specific player. Since titles are required to enroll players that have shown intent to play and possess a biometric profile (see [TCR 157](#)), specifying a callback function is needed in order to pass this requirement. In addition, the callback is the sole means for cancelling a specific identification or an enrollment operation in progress. **NuiIdentityAbort** will always abort all operations in progress.

Callbacks give the title an opportunity to monitor issues faced by NuiIdentity while trying to identify the player. The title should use this information to guide the player accordingly, improving the chances of success and reducing the time needed to identify the player. Without using the callback system, there is no way for a title to obtain feedback from **NuiIdentity** except through **dwEnrollmentIndex**, which is updated when the operation is complete. When not using the callbacks, a title will know that an asynchronous operation is completed successfully or failed by examining the **dwEnrollmentIndex** associated to a specific skeleton. It will be set to NUI_IDENTITY_ENROLLMENT_INDEX_FAILURE to indicate that the operation did not succeed, or, in case of success, to NUI_IDENTITY_ENROLLMENT_UNKNOWN or to a valid enrollment index.

NuiIdentity will call the callback function specified by the title in the call to **NuiIdentityIdentify** or **NuiIdentityEnroll** with the message NUI_IDENTITY_MESSAGE_FRAME_PROCESSED each time that a new skeleton frame is processed and with the message NUI_IDENTITY_MESSAGE_COMPLETE when the asynchronous operation ends. NUI_IDENTITY_MESSAGE_FRAME_PROCESSED is sent whenever a new frame of data is processed to let the title know that the operation is still in progress along with quality flags indicating any issues NuiIdentity has trying to identify the player. A title should use this data to provide accurate feedback to the player to expedite the identification process.

Note that these are flags, and many of them can be set at once indicating multiple issues. It is the title's obligation to determine how to provide feedback for multiple issues. For example, a title could focus on one issue at a time (so that a player is not overwhelmed) or provide feedback for multiple issues. After processing the message, the title has the option of canceling the operation by returning a value of false.

The following table shows a few examples of quality flags. For a complete list, see [NUI_IDENTITY_QUALITY_FLAGS](#) in the XDK documentation.

Table 3. Examples of quality flags

Quality flag	Description
NUI_IDENTITY_QUALITY_USER_CLOSE	The person is too close to the camera. The title should ask the person to move back.
NUI_IDENTITY_QUALITY_USER_CLIPPED_LEFT	The person is standing too far to the left; the title should ask the person to move toward the center of the play area.
NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK	Lighting conditions do not allow NuiIdentity to distinguish the player's features. The title, for example, could ask the player to turn on the light.

We recommend that titles handle one quality issue at a time, and that issues be processed in the order of importance listed in the following table.

Table 4. Quality flags: order of importance

NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_DARK
NUI_IDENTITY_QUALITY_ENVIRONMENT_TOO_BRIGHT
NUI_IDENTITY_QUALITY_USER_OCCLUDED_FACE
NUI_IDENTITY_QUALITY_USER_OCCLUDED_BODY
NUI_IDENTITY_QUALITY_USER_BODY_TURNED
NUI_IDENTITY_QUALITY_USER_NOT_UPRIGHT
NUI_IDENTITY_QUALITY_USER_FAR_AWAY
NUI_IDENTITY_QUALITY_USER_CLOSE
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_LEFT
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_RIGHT
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_TOP
NUI_IDENTITY_QUALITY_ENVIRONMENT_FACE_DETECT_FAILURE
NUI_IDENTITY_QUALITY_USER_CLIPPED_AT_BOTTOM

NUI_IDENTITY_MESSAGE_COMPLETE is sent to the callback when the operation terminates successfully or unsuccessfully. The message is sent before **dwEnrollmentIndex** for the corresponding skeleton is updated to avoid concurrency issues in multithreaded engines. Two parameters are embedded with the message. The first one is of type HRESULT to let the title know the exact outcome of the operation. The second one is a DWORD containing the enrollment index, and is valid only when the HRESULT parameter is set to S_OK. This is the same enrollment index as the one embedded in the skeleton structure and extensively discussed in the previous section. It is provided here so the title can start using it immediately, without having to wait for the next successful call to **NuiSkeletonGetNextFrame** for the updated skeleton data to be retrieved.

It is important to understand that the callback is called on the thread where the Kinect system is running. The title can designate the Kinect thread when it calls **NuiInitialize**. Be careful when accessing data from within the callback and be especially mindful if you used the data parameter to pass a pointer to a blob of title-specific data. The data it points to may also be accessed by the thread that set up the callback and called **NuiIdentityIdentify** or **NuiIdentityEnroll**.

Special Cases

This section discusses best practices that can help your title handle special situations that may arise and for which there is not a single solution that fits all situations. When reading this section, keep in mind that the best solution is the one best adapted to the specifics of your title and may be quite different from the ones proposed here.

Ninth Player

Given that the identity system supports up to eight enrolled players, the questions arises as to what to do if a ninth player is identified as unknown. When this situation arises, the title has a few options. It can silently *unenroll* the player absent the longest to make space for the new player. The title could also bring up a UI to let the player select what to do—for instance, ignore the new player or eject a specific one. A title could also decide to disallow any new player after the original eight are enrolled.

Changing a Player's Enrollment Index

It is possible that the Identity system will misidentify a player and confuse him or her with another player who is already enrolled. To deal with such a situation, the title must provide a means for a player to correct his or her identity (as required by [TCR 158](#), NUI Identity Player Feedback). Developers must determine how to handle such situations for their titles. This also can be used by the players to do impersonations, for example, enrolling as someone else.

Adapting to Different Game Types

There are three major scenarios or game types—classic, jump in, kiosk—when it comes to managing player identification and linking players to skeletons and other game-related assets, and NuiIdentity allows all three to be easily implemented by titles.

Classic

In this mode, one or two players will play the game for an extensive period of time. Players will go through the identification and enrollment process at the beginning of the game.

Jump-in

Here the goal is to have many players take part, by turns, in the game. It could be short mini games, for example, or other scenarios. The main idea is that there are more than two players exchanging positions so that each can participate in the game at some point. NuiIdentity helps by recognizing up to eight players at any time, making the process of exchanging players quick and easy. NuiIdentity will detect a change in player and notify the title, which can take the necessary action.

Players still must be identified and initially enrolled (unless previously enrolled by another title). After this initial step, the title is able to instantly detect a change in the player line up.

Kiosk

Kiosk mode is destined for in-store demos where players will be jumping into the game to try it out for a few minutes.

This mode has much in common with the jump-in mode except the pool of players is larger, which means that the system will likely be unable to remember players once they step out of the play area.

Dealing with Quality Issues

There are certain conditions—such as if the player is covering his or her face or if the lights are too low—that will prevent the identity system from working correctly. During an enrollment or identification operation, the identity system reports quality flags to the title by using a callback. These flags represent current conditions that impact the operation. The title needs to handle quality issues during the identity operation to ensure a successful result. It is recommended that the title do this in a game-specific way. In a ball game, for example, if the player is covering his or her face the title could throw a ball at the player's avatar to get them to uncover it.

Some quality issues represent problems with the play area. Low light is a good example. For these issues, the game has a few options. It can either attempt to help the user or invoke the Kinect troubleshooter. Invoking the troubleshooter will exit the title, so the game should first ask the user.

Be mindful that any modifications made to the environment in order to correctly identify the players will likely persist during gameplay. If the camera tilt, for example, changed during a troubleshooting session, it would be wise to leave it as is for the remainder of the session because NuiIdentity is continually identifying players in the background to detect any change in the player lineup. The same rule applies if the players are asked to turn on the lights.

Best Practices

Do Not Enroll Guest Players

...unless you really mean to! In many cases, there is no need to implement NuiIdentity beyond what is required to satisfy the TCRs. TCRs require titles to enroll only players that have a matching biometric profile, not every player.

Of course, if your title has a compelling reason to enroll guest players (for example, players that do not have a matching biometric profile), go ahead and do it. Titles that want to distinguish and recognize many players over the course of the game should consider enrolling guest players, especially if the title will assign assets to specific players and track individual scores or progress. Correctly implementing NuiIdentity in this context can be a challenging proposition.

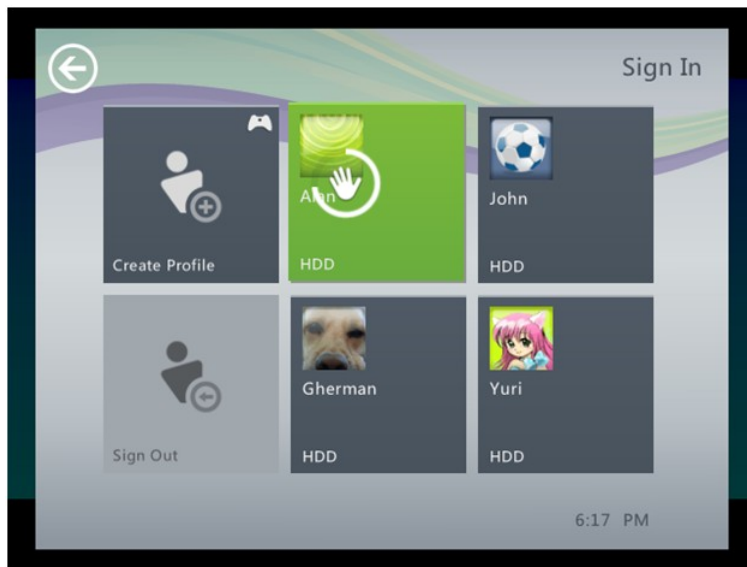
Sign-In Methods

There are two ways a user can be signed in via NUI: automatically using biometric identity data stored in a player's profile, and manually via a new sign-in API.

For automatic sign in, the player must have a profile on the Xbox that is enabled for biometric sign in. The option to enable biometric sign in is stored as a profile setting. It is presented to the user during out-of-box-experience, new profile creation, and a few other spots to catch any profiles that existed on the Xbox before Kinect. If the user chooses to enable this feature, a wizard guides the user through the necessary steps to ensure the best possible biometric data is captured. Biometric data is stored with the profile. The data travels with that profile if a different console (using a memory stick, for example) is used.

Once a player has a biometric sign in enabled profile on the Xbox, the player is signed in automatically when enrolled for the first time by a title. The Enroll API returns to the title regardless if the user was signed in as part of the enrollment process. Biometric sign in may fail because it does not find a match, or because it signs in the wrong person. To provide a better user experience, the system is tuned to favor not finding a match over signing in the wrong person even if it lowers the probability that an automatic sign in will occur. This makes the false positive case slightly better for the user.

Figure 2. A sample Kinect sign-in on Xbox 360



For users without a biometric profile, or in cases where the system fails, the title will have to determine how important it is to have the player signed in. The recommendation is that titles should require, if at all, a sign in only for the first player because manual sign in is a UI interruption that interferes with jump-in play. The exception is for online play via Xbox LIVE. Additional users then would have nonintrusive opportunities available for sign in. This could be as easy as a sign-in option in the multiplayer lobby, or a button that appears for a few seconds after a new jump-in player is enrolled. To manually sign in a player, the title calls **XShowNuiSigninUI**. This API opens a NUI-enabled dialog box that allows the user to choose their profile or to create a new one.

Intent to Play

An important concept related to identity is something we call *intent to play*. This is the idea that players have to do something to show the game that they want to engage—they have to show intent. The title should communicate to the player what the intent to play mechanism is. Before Kinect was introduced, this was as easy as **Press START to begin** at the title screen or **Press A to join** at the multiplayer lobby screen. In the NuiIdentity world, this could translate to a message as simple as **Wave to begin**. In some titles, interacting with a character in the game might be enough to show intent. Titles are free to choose the intent to play gesture (or gestures) that best fit their game, and developers are encouraged to be creative with this.

Handling Feedback

NuiIdentity uses different mechanisms to provide feedback about the state of the players bound to skeletons. The primary mechanism is the **dwEnrollmentIndex** field that is part of the **NUI_SKELETON_DATA** structure representing each skeleton. In the same structure, NuiIdentity also maintains the **dwUserIndex** that links an enrolled player to an Xbox 360 profile. Finally, NuiIdentity uses function callbacks to provide real-time feedback during lengthy asynchronous operations.

dwEnrollmentIndex

A member of the **NUI_SKELETON_DATA** structure, **dwEnrollmentIndex** is the most important means that NuiIdentity uses to keep the title updated about changes in the player line up. A title should check the value of this field every time it obtains a new skeleton frame through **NuiSkeletonGetNextFrame** and should take appropriate action whenever a change is detected. Its value is defined only when **eTrackingState** equals **NUI_SKELETON_TRACKED** and should be ignored otherwise. For a detailed discussion about how to handle changes to **dwEnrollmentIndex**, see [Implementing NuiIdentity in Your Title](#).

dwUserIndex

Just like **dwEnrollmentIndex**, **dwUserIndex** (which also is a member of **NUI_SKELETON_DATA**) can change at any time. A title should always check this value to detect any change in player sign-ins. This value should not be trusted unless **dwEnrollmentIndex** contains a valid index.

- Between 0 and **XUSER_MAX_COUNT**: The player is signed in and the title should be taking advantage of it to establish a strong link between the game and the player.
- XUSER_NONE**: The player is not signed in.

Message Callbacks

NuiIdentity uses title-provided function callbacks in order to provide real-time feedback on asynchronous operations executing at the request of the title. Even if the use of function callbacks is optional, it is strongly advised to use them as they provide the title with much more detailed feedback than **dwEnrollmentIndex**. For a complete discussion about the callback mechanism, see [Real-Time Feedback](#).

Unenrolling

If a title needs to dismiss an enrolled player, it can do so by calling **NuiIdentityUnenroll**. The function unenrolls the requested player as long as the player is not signed in to a profile. A signed-in player must be signed out in order to be unenrolled. When a player is signed out, he or she is unenrolled automatically at the same time. A title can use **NuiIdentityUnenroll** to dismiss players who have been inactive for some time in order to allow new players to enroll and join the game.

Calling **NuiIdentityEnroll** with the NUI_IDENTITY_ENROLL_REENROLL flag to enroll a player has the same effect as calling **NuiIdentityUnenroll** first to dismiss the print for the corresponding **dwEnrollmentIndex**.

Impact on Your Title

Resources

The identity system exposes a relatively small set of APIs. It consists primarily of a few APIs for moving players between the player types, and an API for identifying a skeleton. The system reports identity information directly through the existing NuiIdentity APIs. This information is directly attached to the skeleton frame.

The challenging aspect of implementing skeletal identity in a title is that the NuiIdentityIdentify and NuiIdentityEnroll APIs have non-trivial cost, and require a varying number of data frames in order to succeed. As of the November 2010 XDK, the system takes anywhere from 10 milliseconds to 20 milliseconds on a single core to generate one frame of identity data for a single skeleton. Up to 20 "good quality" frames of identity data are required to identify or enroll a skeleton. More frames may be required if there are quality issues—the player, for example, is not facing the sensor. Developers control when to process identity and can specify (through the call to **NuiInitialize**) on which thread the NuiIdentity operations will run.

The good news is that identity processing is only needed in two cases: when the game wants to add a player to the system (enrollment), and when the skeleton tracking (ST) system experiences a break in continuity. Such a break could be as simple as a player exiting the camera's field of view and a different player returning, or a player becoming occluded during gameplay and then reappearing. Typically, as long as continuous tracking is maintained, the system will not need to re-identify a skeleton. When a break in continuity occurs, the identity system tags skeletons with NUI_IDENTITY_ENROLLMENT_INDEX_CALL_IDENTITY as the enrollment index. Depending on the situation, a title may be able to delay handling this message until a favorable point, or may completely ignore it as long as [TCR 157](#) is satisfied.

NuiIdentity and the Color Stream

When requesting that a player be identified or enrolled (by calling **NuiIdentityIdentify** or **NuiIdentityEnroll**), a title should expect to stop receiving color stream frames for the duration of the operation. The skeleton and depth image streams are unaffected.

This happens because NuiIdentity changes the color camera parameters (like exposure and zoom) as needed. A title can always call **NuiIdentityAbort** to reclaim immediate ownership of the color stream. After **NuiIdentityAbort** is called, the color stream is returned quickly, but not instantaneously. It takes time for the original settings to be restored (up to 100 ms in certain cases). The title should look at the result from **NuiImageStreamGetNextFrame** to determine if the color stream has been returned to the title. If the identity operation was initiated by the Guide, calling **NuiIdentityAbort** has no effect. The title has to wait until the operation terminates, which can take several seconds. In lieu of calling **NuiIdentityAbort**, a title can anticipate its need for the color stream and for a short time can delay calling **NuiIdentityIdentify** or **NuiIdentityEnroll**.

Asynchronous Operations and the Guide

The skeletal pipeline usually provides a new skeleton frame to your title every 33 milliseconds, but there are times when the delay between two skeletal frames may be longer. Sometimes the delay can be seconds and even minutes, and your title should be prepared to see a long break in the usually regular stream of skeleton frames. A long delay between two skeleton frames generally is the result of another part of the Xbox 360 system temporarily taking control of the streams provided by the sensor for a specific task. For example, the new Kinect sign-in UI allows players to use gestures to sign in to accounts; to achieve this, the Guide needs to reroute the skeleton frames for its own needs. This will happen after the user or the title has brought up the Guide. If the Guide needs access to skeleton data, it may temporarily take control of the color and depth streams in the same way.

When the skeletal pipeline is again rerouted to the Guide, any NuiIdentity asynchronous operation in progress is paused. Title-provided callback functions are not called during that time. The operation resumes as soon as the data starts flowing back to the title.

Takeaways

In most cases, adding enrollment support to a Kinect-enabled title should be straightforward, but be mindful of the specific needs of your title and do not hesitate to adapt the process to fit your own needs.

Take advantage of the event system to monitor asynchronous identity operations and to provide useful feedback to the players to make the identification and enrollment process as quick and easy as possible.

NuiIdentity uses the **dwEnrollmentIndex** field in **NUI_SKELETON_DATA** to communicate the enrollment status of a specific player. Check it every time you obtain a new skeleton frame from **NuiSkeletonGetNextFrame**.

Take into consideration your specific title needs when making enrollment decisions and be aware of your engine architecture to ensure correctness and performance. If you use a single-threaded game engine, understand that callbacks will run on a different thread and will introduce some multithreading considerations into your engine.

- Identity is a key feature of Kinect; make room for it in your development schedule.
- Identity and enrollment are managed by the title.
- Sign in is managed by the Guide.
- Feedback to the player at all levels is important for a quality experience.
- Ensure misidentification is handled in your title.
- Honor a signed-in player at launch.
- Enroll players that have shown intent to play and have a biometric profile match.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

NUI Joint Filtering: Best Practices



by Mehran Azimi, Software Development Engineer
Advanced Technology Group (ATG)

- [Introduction](#)
- [Why We Need Joint Filtering](#)
- [Joint Filtering Basics](#)
- [Smoothing Filters](#)
- [Practical Tips and Takeaways](#)
- [References](#)

1. Introduction

The skeletal tracking (ST) system of the Natural User Interface (NUI) provides joint positions of tracked players' skeletons to a title. These joint positions are the key data consumed by game code as the player position and pose, and they are used for many purposes, such as gesture detection, avatar retargeting, navigating user interfaces, and so on.

In practice, there is some noise present in the joint positions returned by the ST system. An important step before consuming ST data is to use a noise reduction filter to remove as much noise as possible from the joint data. Such filters are called *smoothing filters* because they result in smoother positions over time.

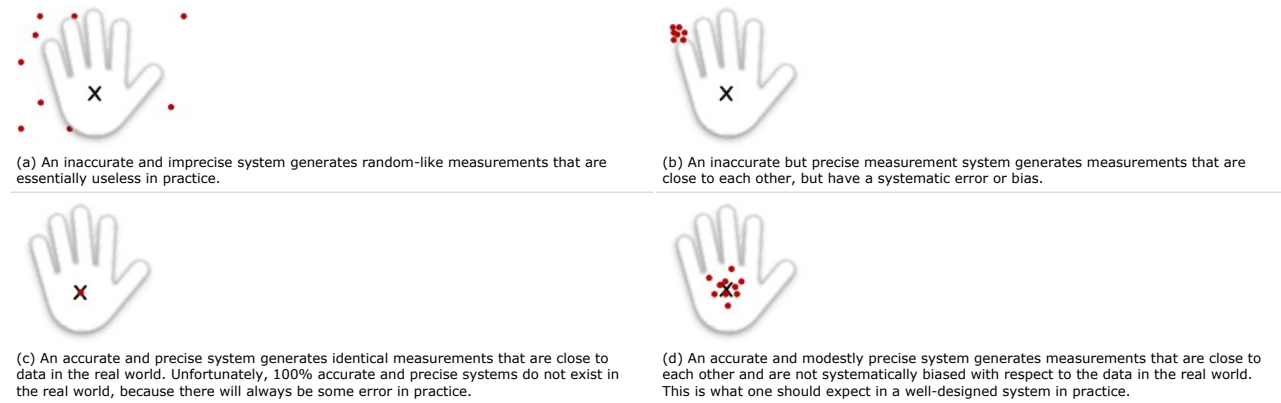
This white paper describes the filtering techniques and best practices for using skeleton joint data for Kinect-enabled titles, and its goal is to help game developers to choose an appropriate filtering technique and fine-tune the filter parameters to match the specific needs of their Kinect-enabled title. The paper covers different areas related to joint filtering, such as the type of noise one should expect in ST data; how filtering affects the latency and how forecasting can be used to reduce latency; the characteristics of an ideal joint filter in terms of responsiveness, latency, and smoothing effect; and how tracking state data returned by ST can be used to improve filtering. Then it describes the specific characteristics of a few useful filtering techniques in detail. The paper concludes with a summary of best practices and practical tips for filtering.

This white paper is accompanied by a Microsoft Excel spreadsheet that creates useful graphs for each filtering technique discussed in this paper. The spreadsheet allows you to compare two filtering techniques, or to select the parameters of each filter, to experiment with how the parameters affect the characteristics of the filter output. You can download the spreadsheet from GDNP: [NUI joint filtering forecaster](#).

2. Why We Need Joint Filtering

Measurement errors and noise are by-products of almost any system that measures a physical quantity via a sensor. The characteristics of this error are usually described by accuracy and precision of the system, where *accuracy* is defined as the degree of closeness of measured quantity to its actual value, and *precision* is defined as the degree to which repeated measurements are close to each other. An accurate system does not have any systematic error in measurements and therefore does not add a systematic bias. A precise system results in measurements close to each other when the measurement is repeated [1,4]. The accuracy and precision concepts are illustrated in Table 1 for a system that is measuring a hand position in the real world.

Table 1. Accuracy vs. Precision: The black X represents the hand location in the real world, and red dots represent a few measurements of hand position by a measurement system.



Just like any measurement system, the joint positions data returned by the NUI ST system has some noise. There are many parameters that affect the characteristics and level of noise, which include room lighting; the player's body size; the player's distance from the sensor array; the player's pose (for example, for hand data, if the player's hand is open or fisted); location of the sensor array; quantization noise; rounding effects introduced by computations; and so on. Note that the joint positions returned by ST are *accurate*, meaning that there is no bias in the joint position data to the actual positions in the real world. This means that if a player is standing still, then the average of the joint positions data, over time, is close to the positions in the real world. However, the joint positions data are not necessarily perfectly precise, meaning that they are scattered around the correct positions in each frame. In practice, the joint positions are accurate within a centimeter range, not millimeters.

There are cases that the ST system does not have enough information in a captured frame to determine a specific joint position. Examples of these cases include occlusion by furniture or other players, self-occlusion of a joint by other body parts of the player, and moving a joint out of the sensor array's field of view. In most of these cases, the ST system is still able to infer the joint position, and the NUI_SKELETON_POSITION_TRACKING_STATE parameter, returned as part of a NUI_SKELETON_DATA structure, is set to NUI_SKELETON_POSITION_INFERRED for that joint. This parameter can be treated as the confidence level of the ST system regarding the joint position. Though the inferred joint positions are a very refined estimate of joint position, they may become inaccurate in some cases, depending on a player's pose. Therefore, one should expect that inferred joints have higher noise values, along with a possibility of a bias. This bias is usually observed as temporary spikes in the joint position data, which goes away as the joint tracking state level goes back to NUI_SKELETON_POSITION_TRACKED.

Therefore, two types of noise are present in joint positions. One is the relatively small white noise that is always present for all joints and caused by imprecision; the other is temporary spikes caused by inaccuracy, which happen when the joint has an inferred tracking state. Since these noises have different characteristics, different filtering techniques should be used for each. That is, developers need to use a combination of two or three filtering techniques to achieve good results in a Kinect-enabled game.

3. Joint Filtering Basics

Before describing any specific filtering technique, there are a few important concepts to cover that are related to filtering; they include latency and how it relates to filtering delays, how forecasting can improve latency, the tradeoff between latency and smoothing effects in filter design, and what makes an ideal filter.

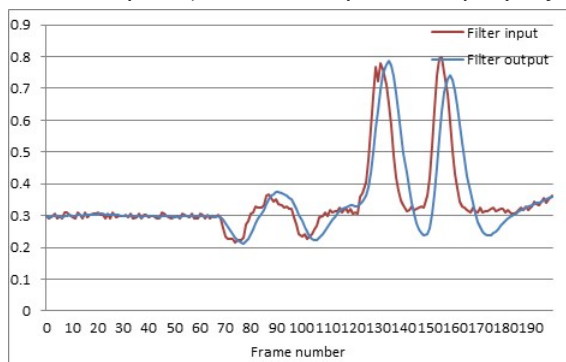
- [Latency and How it Relates to Filtering](#)
- [Filter Smoothing Effect vs. Filtering Delay](#)
- [Error Propagation to Variables Calculated from Skeletal Tracking Data](#)
- [Filter Notations](#)
- [Filter Response to Step Function and Sinusoidal Waveform Inputs](#)
- [Using Joint Tracking State in Filtering](#)
- [Using Body Kinematics to Correct the Filter Output Data](#)

3.1. Latency and How it Relates to Filtering

Gameplay latency for a Kinect title can be defined as the time it takes from when a player makes a move, till the time at which the player sees the game response to his or her body movement on the screen. Latency degrades the game experience as soon as players start to notice there is a delay in game response to their movements. User research shows that 72% of players start noticing this delay when latency is more than 100 msec, and therefore, it is suggested that developers aim for an overall latency of 100 msec [15]. For a detailed discussion on latency in Kinect games, refer to [14] in [References](#).

The *joint filtering latency* is how much time it takes for filter output to catch up to the actual joint position when there is a movement in a joint. This is shown in Figure 1, which shows that filter output is lagging behind input when there are changes in input. It is important to note that the latency introduced by joint filtering is not the CPU time it takes for the filtering routine to execute.

Figure 1. Output of a typical joint filter in response to a NUI joint movement. Note that latency added by joint filtering is the lag between output and input when there is movement in input data, and the amount depends on how quickly the joint is moving.



In general, the filtering delay depends on how quickly the input is changing, and hence, one cannot attribute a specific delay value to a given filter for all cases. This is referred to as *phase distortion* in signal processing [6,7]. A special class of filters called *linear phase* filters have the same delay for all input frequencies, and such a specific delay time can be attributed to the filter for all inputs. Reducing phase distortion is important in some signal processing applications, specifically in audio processing; however, it is not necessarily as important in NUI joint filtering, so having a linear phase filter is not a design criterion in NUI joint filtering.

A useful technique to reduce latency is to tweak the joint filter to predict the future joint positions. That is, the filter output would be a smoothed estimate of joint position in subsequent frames. If forecasting is used, then joint filtering would reduce the overall latency. However, since the forecasted outputs are estimated from previous data, the forecasted data may not

always be accurate, especially when a movement is suddenly started or stopped by the player. Forecasting may propagate and magnify the noise in previous data to future data, and hence, may increase the noise. Almost all joint filtering techniques can forecast or can be tweaked to forecast future outputs. The accuracy of predicted outputs depends on the underlying data model that the filter is using and how the filter parameters are selected. Note that it is usually practical to forecast a joint position for about two frames in a Kinect game, which could reduce the filtering latency by about 66 msec in ideal cases. However, in practice, the smoothing effect of the filter, along with the prediction errors in forecasted data, would result in smaller latency reductions.

3.2. Filter Smoothing Effect vs. Filtering Delay

An ideal joint filter would remove all unwanted noise and jitters from the joint data resulting in smooth joint position data over time. It would also follow the movements of the joint without any lag or delay. Unfortunately, there is a tradeoff between these two objectives in practice, and choosing a filtering technique that aggressively smoothes out the data would result in higher filtering delay, which would increase the perceived latency. As an intuitive explanation for this concept, consider a case where a player is standing still and therefore the input to the joint filter is mostly a constant position along with some noise. In order to produce a smooth output, the filter should not be sensitive to the changes in input due to noise. Now suppose the player starts moving his/her hand. In order to be responsive to these movements, the filter should be designed to be sensitive to changes due to player movement, which is an opposite of the requirement for noise removal. In practice, most filters take some time to see enough movement before they start following these changes in output, and therefore their output lags behind the changes in input.

Accordingly, one should understand how *latency* and *smoothness* affect the user experience or gameplay, and identify which one is more important to create a good experience for the player. Then, carefully choose a filtering method and fine-tune its parameters to match the specific needs of the game. In most Kinect titles, data output from the ST system is used for a variety of purposes—such as gesture detection, avatar retargeting, interacting with UI items and virtual objects in the game, and so on—where all are different in terms of how smoothness and latency affect them. Similarly, joints have different characteristics from one another in terms of how fast they can move, or how they are used to create the overall player experience. For example, in a Kinect boxing game, a player's hands can move much faster than the spine joint, and therefore, one needs to use different filtering techniques for hands than the spine and other joints. Alternately, consider a fitness game that uses hand movements to score how well the player is doing a particular exercise (that is, gesture scoring), and to animate the player's avatar on screen at the same time. Latency is usually less important in gesture scoring, since the whole gesture should be done before the game can compute the score; consequently, the game should use a high-latency filtering technique with more aggressive smoothing for gesture scoring, while also using a different low-latency filtering technique for animating the player's avatar.

So, there is no filtering solution that fits the needs of all use cases in Kinect games for all joints. Depending on which joint data is to be used and how the filter output is consumed, one should apply different filtering techniques or fine-tune the parameters *per joint* and *per application*.

3.3. Error Propagation to Variables Calculated from Skeletal Tracking Data

One can apply the filtering techniques discussed in this paper to the variables calculated from joint positions. For example, a game may need to use the elbow angle formed between the player's hand, elbow, and shoulder. Then the same filtering techniques discussed here can be applied to the calculated angle to smooth it out over time. Another example is using spherical coordinate systems local to a joint's parents, where each point in space is represented by a (r, θ) triplet, instead of a Cartesian system of (x, y, z) coordinates. For example, one can use the right shoulder as the origin and represent the elbow position by using this local spherical coordinate system. In this case, the Cartesian coordinates returned by ST are transformed to local spherical coordinates (r, θ) , and filtering is done on each component separately. The radius r , which represents the bone length, can be filtered more aggressively than the angles (θ) , which represent the joint movements.

It is important to note that applying mathematical operations to noisy joint data would propagate noise and may amplify the noise level. The underlying concept of noise propagation is similar in essence to propagation of rounding or precision errors of floating-point variables in math operations, though rounding errors on floating-point values are small and are ignored in most practical cases, while measurement noise is relatively much larger. Operations to calculate body part sizes, such as addition, subtraction, and multiplication, amplify the noise [1-4]. For example, calculating bone length or calculating relative joint coordinates, such as elbow position relative to shoulder or hand position relative to head, all require subtracting two joint positions. In all these cases, the noise in resulting data is amplified. Trigonometric functions (such as sine, cosine, or inverse tangent), which are typically used for calculating or manipulating joint angles, affect the noise in different ways. The effect of any function on noise depends on the local slope of that function, that is, the local derivative of the function around the data point used. For example, suppose θ is a joint angle calculated from noisy ST data, and hence θ is noisy as well. Now consider function $\tan\theta$, which is sensitive around $\theta=90^\circ$, because $(d/d\theta)\tan\theta$ converges to ∞ around $\theta=90^\circ$. Hence, calculating the $\tan\theta$ function would amplify the noise drastically if θ is close to 90° . However, since $(d/d\theta)\tan\theta=1$ around $\theta=0^\circ$, then noise is neither amplified or decreased if θ is close to zero.

A more thorough discussion of this topic can be found in [References](#) under "Error analysis and error propagation in noisy data"; see [1-4] for more details.

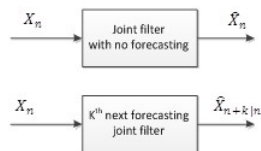
3.4. Filter Notations

The joint filter implementation in a typical Kinect game receives joint positions from ST as input in each frame, and returns the filtered joint positions as output. The filter treats each joint's x , y , and z coordinates independently from other joints or other dimensions. That is, a filter is independently applied to the x , y , and z coordinate of each joint separately—and potentially each with different filtering parameters. Note that, though it is typical to directly filter the Cartesian position data returned by ST, one can apply the same filtering techniques to any data calculated from joint positions.

This means the input of each joint filter is a one-dimensional time series that represents a single joint position in a given dimension at frame n (for example, y coordinates of the right hand position). The filter input at frame n is denoted by X_n and the filter output generated from applying the filter by $\hat{X}_n$.

The k next forecasted output calculated at time n is denoted by $\hat{X}_{(n+k|n)}$, where the " $|n$ " notation denotes that this forecast is done at time n , and hence, by definition, only previous samples up to X_n are available for calculating $\hat{X}_{(n+k|n)}$.

Figure 2. Filter notations used in this white paper



3.5. Filter Response to Step Function and Sinusoidal Waveform Inputs

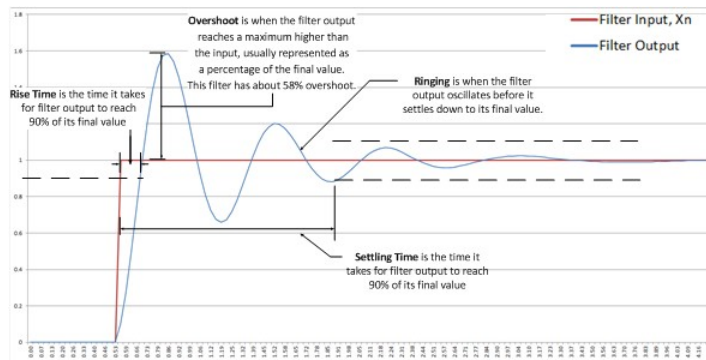
In order to understand a filtering technique and how filtering parameters affect the filter characteristics, it is useful to study the filter output in response to some predefined inputs, specifically step function and sine wave inputs.

Step function input models a sudden jump in input data, and is defined as:

$$X_n = \begin{cases} 0, & n < N \\ 1, & n \geq N \end{cases}$$

That is, input X_n is zero for up to a given time and then jumps and stays at 1 at time N . Though one does not expect to see such input in practice, a step function is helpful because it shows how quickly and how accurately the filter tracks sudden changes in input. Figure 4 shows the typical output of a filter to a step function input, as well as definitions for some of the filter characteristics of our interest.

Figure 3. Typical response of a filter to step function input



- **Rise time** is the time required for the filter output to reach 90% of its asymptotic final value, which is 0.90 in case of a unit step function input. In some fields, a similar parameter called *time constant* is used, which is the time required for the filter output to reach $(1 - e^{-1})\% = 63\%$ of its final asymptotic value. A small rise time is an indication of a filter with low latency.
- **Overshoot** is when the output of the filter reaches a higher maximum value than input, and this is usually represented as a percentage of the final asymptotic value. Note that not all filters have overshoot in their output. Overshoot is undesirable, and it is usually present in low-latency filters that are sensitive to changes in input.
- **Ringing** is the effect where filter output oscillates before it settles down to its final value.
- **Settling time** is applicable to filters that have overshoot or ringing, and it is the time it takes for the filter output to reach and remain within a given error margin of its final asymptotic value. The range of the error margin is usually 10 percent.

Rise time shows how quickly the filter catches up with sudden changes in input, while overshoot, ringing, and settling time are indications of how well a filter can settle down after it has responded to a sudden change in input.

A filter's response to a step function does not reveal all of the useful characteristics of a filtering technique, because it only shows the filter's response to sudden changes. It is also useful to study a filter's response to sine waveform input, both in time and frequency domains. As shown in Figure 4, the response in the time domain can show the lag in filter output, which depends on input frequency in most cases (that is, in nonlinear phase filters). Note that the output may not reach the maximum or minimum level of the input due to the filter attenuating that frequency. This is sometimes referred to as *dampening the peaks and valleys* in input by aggressive smoothing. As an example, Figure 5 shows the hand position x of a player who has opened and closed his arm quickly two times, resulting in a sinusoidal peak in input. As noted, aggressive smoothing of data has resulted in filter output not reaching the same maximum and minimum level of the input. Also, it is interesting to note in Figures 4 and 5 that, in some frames, the input is increasing while the output is decreasing; this can be attributed to the filtering latency. This filter would not be a good choice for drawing a cursor on the screen based on the player's current hand position, because it would produce an undesirable effect when the player's hand changes direction; the cursor would catch up and change direction after a while, which would create an awkward experience for the player.

Figure 4. Typical response of a filter to sine waveform input

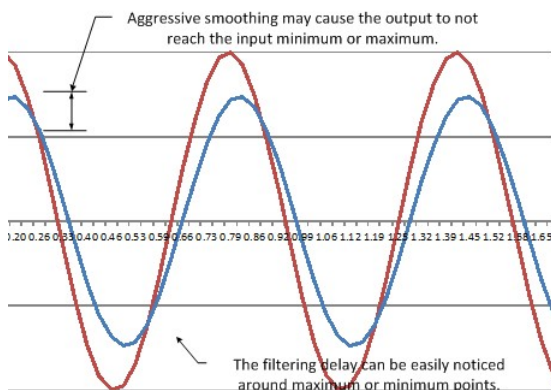
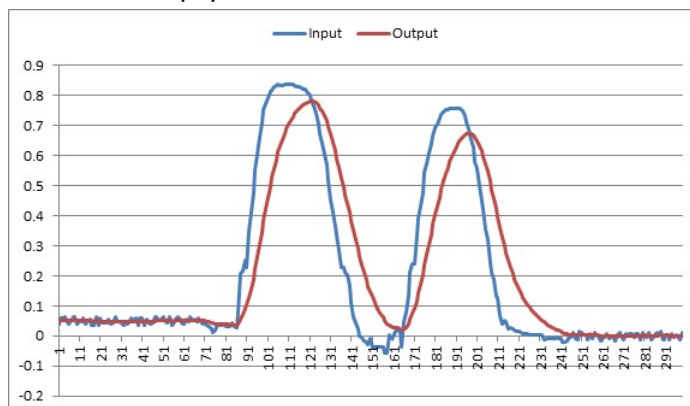


Figure 5. Aggressive smoothing would reduce the minimum and maximum in sinusoidal input data. The data is from actual hand movement where a player has opened and closed his arms rapidly two times.



The filter's response in the frequency domain shows how a filter responds to all ranges of frequency inputs. All smoothing filters used for NUI joint filtering are low pass filters, where an ideal low pass filter would let through the input frequency components that are lower than a cut-off frequency, but remove the frequency components that are higher than the cut-off frequency. The low-pass characteristics of NUI joint filters is based on the assumption that joint movements have relatively lower frequency than the noise, though this is not necessarily a correct assumption for all cases—specifically, when a player makes rapid movements, such as quick hand movements or sudden jumps. Note there are powerful methods of filter design available in signal processing textbooks that realize a desired frequency response, such as a low pass filter of given order N and with a given cutoff frequency [6]. Since the frequency range of NUI joint data overlaps with the noise, low-pass filters with a design based only on the frequency response criteria do not necessarily provide good results in Kinect games. Design methods of frequency domain filters are not within the scope of this white paper. However, you can still become familiar with the underlying concepts used in filter design in the frequency domain to better understand filtering characteristics in general, and items [6, 7] in [References](#) are good starting points.

3.6. Using Joint Tracking State in Filtering

Joints that are inferred are more likely to have temporary spike noises due to being less accurate. Also, the random noise levels are usually higher for inferred joints. In practice, for Kinect

games, developers should consider a joint's tracking state to be valuable information about the quality of the joint data, and apply a more aggressive smoothing filter when a joint's state is inferred. This can easily be done by checking the joint's tracking state in the filter's implementation and adaptively update the filter parameters based on the joint's tracking state. Also, filters that are specifically more powerful in removing spike noise should be applied when a joint's state is inferred.

3.7. Using Body Kinematics to Correct the Filter Output Data

The anatomy and kinematics of a player provide valuable information that can be used to enhance the ST joint data. For example, some joints move or bend only in a certain direction, or some joints, such as hands, can move faster than other joints. How this data can be applied depends on the joint and the contexts the joint data are used. As a practical example, suppose that a player's right hand joint is filtered with a low latency filter that forecast one frame in future. This type of filter usually results in overshoots in response to sudden movements. Therefore, due to overshoot, the filter may calculate the hand position to be too far from the player's body. This overshoot can be corrected by using an estimate of hand bone length, and correcting the filtered hand position such that the hand is positioned an acceptable distance from the player's body.

Another useful kinematic property is the limitations of hinge joint. A hinge joint is a joint that can bend along only one axis (that is, it has only one degree of freedom). For example, elbows and knees are hinge joints, because they can bend in only one direction and within a limited range. These kinematic limitations of elbow or knee can be used to correct the filtered joint positions.

4. Smoothing Filters

This section discusses details of a few smoothing filtering techniques that are useful for NUI joint filters. First are the Auto Regressive Moving Average (ARMA) filters, which are a general class of filters. Specific smoothing filters for noise removal are also covered, including Moving Average, Double Moving Average, Exponential Filter, Double Exponential Filter, and Savitzky-Golay filters. All these smoothing filters are special cases of ARMA filters. Finally, the section discusses a filter based on the Taylor series that is useful for forecasting.

Also covered are Median and Jitter Removal filters, which although they have some smoothing effect, are specifically useful for removing spike noise from data.

Note that this white paper is accompanied by an Excel spreadsheet ([NUI joint filtering forecaster](#)) that allows you to choose a specific filtering technique, select an input type (step function, sinusoidal waveform or actual NUI hand data returned from ST), choose to add noise to input or not, and select a desired filtering parameter. Then Excel plots the filter output and calculates some key parameters of the filter, such as rise time and overshoot percentage in response to a step function. The reader is encouraged to use the spreadsheet for each filter discussed here to get more familiar with filter properties and experiment to see how the parameters of each filter affect the filter output.

- [Auto Regressive Moving Average \(ARMA\) Filters](#)
- [Simple Averaging Filter](#)
- [Double Moving Averaging Filter](#)
- [Savitzky-Golay Smoothing Filter](#)
- [Exponential Smoothing Filter](#)
- [Double Exponential Smoothing Filter](#)
- [Adaptive Double Exponential Smoothing Filter](#)
- [Taylor Series Filter](#)
- [Median Filter](#)
- [Jitter Removal Filter](#)

4.1. Auto Regressive Moving Average (ARMA) Filters

Auto regressive moving average (ARMA) filters are a general class of linear filters. All the smoothing filters we discuss in this paper are a special case of ARMA filters. The output of an ARMA filter is a weighted average of current and N previous inputs, and M previous filter outputs:

$$\hat{X}_n = \sum_{i=0}^N a_i X_{n-i} + \sum_{i=1}^M b_i \hat{X}_{n-i}$$

Where the a_i and b_i coefficients are the filtering parameters. The first term is known as the moving average (MA) term, and the second term is known as the auto-regressive (AR) term.

Moving average (MA) filters are a special case of ARMA filters where all b_i parameters are zero:

$$\hat{X}_n = \sum_{i=0}^N a_i X_{n-i}$$

The coefficients a_i are the weight factors and are selected such that $\sum_{i=0}^N a_i = 1$ in all NUI joint filter applications. This property is a result of using low-pass filtering and allowing DC components through without attenuation. As an intuitive explanation, suppose the input to be filtered is the constant 1 for all frames n . Then, one intuitively expects the filter output to be 1 as well for all n , which would result in $\sum_{i=0}^N a_i = 1$. This can be used as a quick sanity check that the derived a_i coefficients are correct.

The moving average filter can be extended to a central moving average filter, where filter output is the weighted average of N past and M future inputs:

$$\hat{X}_n = \sum_{i=-M}^N a_i X_{n-i}$$

Since the output of this filter depends on M future inputs after X_n , then any implementation of this filter will add a latency of at least M frames. Therefore, this filter is only practical in offline cases in which all data is available in advance, or in cases in which increasing the latency by an order of a few frames is tolerable. For example, in some Kinect-enabled fitness games, it may make sense to score how well the player has done an exercise after the whole performance is finished. The central MA filters usually perform better in terms of noise removal than simple MA filters.

ARMA filters are usually designed by assuming an underlying data model for data and using this model to calculate the filter orders N and M , and the a_i and b_i coefficients. The data model should be chosen based on the actual data characteristics. In some approaches the underlying data models are chosen based on a statistical approach that would preserve the higher order moments of data. The details of these methods are outside the scope of this paper; see [5] for a detailed discussion. This paper mentions the order of statistical moments up to which input data are preserved by the filter. In general, filters that preserve higher-order moments perform better.

4.2. Simple Averaging Filter

The simple averaging filter is the simplest joint filter, where the filter output is the average of N recent inputs, which is an MA filter of order N with $a_i = 1/(N+1)$ for all i :

$$\hat{X}_n = \frac{1}{N+1} \sum_{i=0}^N X_{n-i}$$

From a statistical point of view, the averaging filter is a naive filter that fits a horizontal line (that is, a constant) to N recent inputs and uses it as the filter output. Therefore, an averaging filter is not taking advantage of joint data characteristics or noise statistical distribution, and it preserves only the first-order moment of data, which is the average. A simple averaging filter doesn't provide satisfactory results in most cases of filtering NUI joints.

An averaging filter using a large N would result in more smoothing than a smaller N , but it would introduce more filtering delay. The filtering delay can be noted in the output from an averaging filter in response to inputs from step functions and sinusoidal waveforms in the accompanying Excel spreadsheet, where filtering delay is directly proportional to N . For example, the step function rise time for $N=5$ and $N=10$ are about 4.5 frames (148 msec) and 9 frames (297 msec), respectively. The simple averaging is a linear phase filter, which means that all frequency components in input are delayed by the same amount [6]. To experience this, try different frequencies for sinusoidal input in the spreadsheet, and notice that the output delay is the same for all filters.

The smoothing effect of the filter can be more easily noticed when noise is added to step function or sinusoidal inputs in the Excel spreadsheet. Also note that output from the averaging filter cannot reach the peaks and valleys of most sinusoidal waveform inputs.

Since an averaging filter fits a horizontal line of data, it forecasts the future outputs as a constant as well, which is the average of last N inputs up to time n :

$$\hat{X}_{n+k|n} = \hat{X}_n = \frac{1}{N+1} \sum_{i=0}^N X_{n-i} \quad \text{for all } k$$

The simple averaging filter performs well in terms of forecasting only if the input data is stationary and there is no trend in data. In NUI joint filtering, stationary input data means that a joint has little movement. If there is movement in the joint, then the filter's input data will have a slope, and hence, averaging filters performs poorly, especially in terms of forecasting. For NUI joint data with movement at a constant speed, it can be shown that the averaging filter has a constant bias in its output [8,9]. A method to eliminate this error is to use a filtering method called *double moving average*, which is discussed in the next section.

4.3. Double Moving Averaging Filter

Double moving averages are used in many applications, such as stock market forecasting, and they are useful when data has a linear *trend*. The trend in NUI joint data is equivalent to a joint movement at a constant velocity. The underlying data model used by double moving averaging filter is to fit a linear line to local input data, and hence it is more adaptable to track changes

in input data than simple averaging filter [8,9]. Let $MA_n^{(1)}$ and $MA_n^{(2)}$ be the first and second order moving averages of input data at time n :

$$MA_n^{(1)} = \frac{1}{N+1} \sum_{i=0}^N X_{n-i}$$

$$MA_n^{(2)} = \frac{1}{N+1} \sum_{i=0}^N MA_n^{(1)}$$

Therefore $MA_n^{(2)}$ is a moving average of a moving average of input data (hence the name double moving average). If we assume that the underlying data follow a linear model, it can be shown that $MA_n^{(1)}$ systematically lags behind the actual data, and it can also be shown that a second moving average $MA_n^{(2)}$ lags $MA_n^{(1)}$ average by approximately the same amount. To account for the systematic lag, the difference of second and first order averages ($MA_n^{(1)} - MA_n^{(2)}$) is added to the filter output. Then filter output is given as the first order of moving average plus the trend adjustment term:

$$\hat{X}_n = MA_n^{(1)} + (MA_n^{(1)} - MA_n^{(2)}) = 2MA_n^{(1)} - MA_n^{(2)}$$

A similar approach is used for adjusting trends in forecasting, and future data are estimated as:

$$\hat{X}_{n+k|n} = \hat{X}_n + \frac{2k}{N-1} (MA_n^{(1)} - MA_n^{(2)})$$

The double moving averaging filter has the advantage of being more responsive to changes in input data as compared to a moving averaging filter. Note that for a given window N , the filtering equations can be combined and the filter output equation can be rewritten in terms of a weighted moving average of past inputs, which would result in easier implementation. For example, for $N=2$, the second moving average and filter output are given by:

$$MA_n^{(2)} = \frac{1}{9} (X_n + 2X_{n-1} + 3X_{n-2} + 2X_{n-3} + X_{n-4})$$

which would result in:

$$\hat{X}_n = \frac{5}{9} X_n + \frac{4}{9} X_{n-1} + \frac{1}{3} X_{n-2} - \frac{2}{9} X_{n-3} - \frac{1}{9} X_{n-4}$$

It is interesting to note that this filter uses larger weights for recent inputs, which is generally true for any filtering window N .

4.4. Savitzky-Golay Smoothing Filter

A Savitzky-Golay smoothing filter (also known as a *smoothing polynomial filter* or a *least squares smoothing filter*) fits a polynomial to neighbor input data for each input X_n in a least-square sense and uses the value of the polynomial at time n as the filter output. A polynomial of order K is defined as:

$$f_K(x) = \sum_{i=0}^K c_i x^i$$

If we use N previous and M future samples as the neighbor samples, then a Savitzky-Golay filter finds c_i coefficients such that minimize the term:

$$\sum_{i=-M}^N (X_{n-i} - f_K(n-i))^2$$

and uses the polynomial value at time n as the filter output—that is, $\hat{X}_n = f_K(n)$. Though it may first seem that this filter results in a complicated implementation, it turns out to be easy. It has been shown that the output of a Savitzky-Golay filter can be expressed as a weighted moving average filter; that is:

$$\hat{X}_n = \sum_{i=-M}^N a_i X_{n-i}$$

where the filtering coefficients a_i are all constant for all X_n values and do not change for different n , or even different inputs. Thus, to implement a Savitzky-Golay filter requires only choosing the appropriate filter order K , and determining how many samples before and after should be used (that is, choose N and M). Then, the coefficients a_i can be calculated offline by using available off-the-shelf algorithms, and filter output is easily calculated using these coefficients [11,12].

Savitzky-Golay filters are optimal in two different senses; firstly, they minimize the least-squares error in fitting a polynomial to each windowed frame of input data, and secondly, in preserving the K first statistical moments of the input signal. In general, Savitzky-Golay filters are useful for cases in which the frequency span of input data without noise is large, and therefore, Savitzky-Golay filters are good candidates for NUI joint filtering.

A Savitzky-Golay filter of order K preserves the first $K+1$ moments of the data [11]. For $K=0$, the Savitzky-Golay filter fits $f_K(x)=c_0$, or a constant value, to neighbors of each input, which would turn to a simple averaging filter with equal weight coefficients a_i . For $K=1$, a straight line is fitted to local input data, which is usually referred to as linear regression in statistics text books. For $K=2$ and $K=3$, a parabola and a cubic curve are fitted, respectively. The choice of n affects the filter smoothing effect, where a cubic curve using $K=3$ seems to be a good choice for NUI joint filtering, since it is the lowest-degree polynomial that supports inflection and is still smooth. Using a higher-order polynomial yields jumpy curves with too many local minima and maxima, which would result in a lessened smoothing effect.

The Savitzky-Golay filter can also be used for estimating the derivatives of input [11], which is easily calculated from the coefficients c_i . Thus, a Savitzky-Golay filter can produce the joint speed and acceleration along the smoothed position for NUI joint data.

4.5. Exponential Smoothing Filter

An exponential smoothing filter, also known as an exponentially weighted moving average (EWMA), is a popular filter in many different fields. The exponential filter output is given by:

$$\hat{X}_n = \alpha X_n + (1-\alpha)\hat{X}_{n-1}$$

Where α is called the dampening factor and $0 < \alpha < 1$. By substituting $\hat{X}_{(n-1)}$ this can be expanded to obtain:

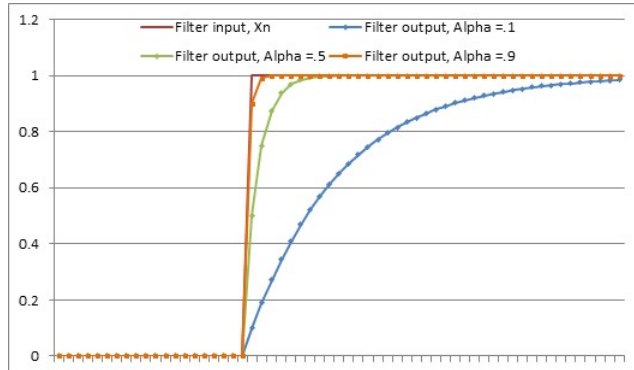
$$\hat{X}_n = \alpha \sum_{i=0}^n (1-\alpha)^i X_{n-i}$$

Therefore, filter output at time n is a weighted average of all past inputs, where weights $a_i = \alpha(1-\alpha)^i$ decrease exponentially with time (more precisely, with geometric progression, which is the discrete version of an exponential function). Also, all previous inputs contribute to the smoothed filter output, but their contribution is dampened by increasing power of parameter $1-\alpha$. Since $\hat{X}_n$ depends on all past inputs, an exponential filter is said to have an infinite memory of all past inputs.

Similar to a simple averaging filter, the exponential filter fits a straight horizontal line to the input data. The difference is that an exponential filter places relatively more weight on recent input data, and is correspondingly more responsive to recent changes in input than a simple averaging filter.

The dampening factor α affects the filtering delay and smoothing effect. A larger α corresponds to larger weight on recent inputs, and results in faster dampening of older inputs. This results in less latency and less smoothing of the data. A small α gives larger weight to older input samples, and hence, the effect of older inputs is larger. This results in a more aggressive smoothing effect and more latency, as shown in Figure 6.

Figure 6. Effect of α on filtering delay of an exponential smoothing filter



Since an exponential filter fits a horizontal line to data, it forecasts the future data as a constant value similar to an averaging filter:

$$\hat{X}_{n+k|n} = \hat{X}_n$$

A variant of exponential filter, called a double exponential filter, addresses this limitation of exponential filters and is described in the next section.

One can incorporate state data for a joint into the implementation of an exponential filter by adaptively using a smaller α for joints for which tracking state indicates inferred positions. This results in more aggressive filtering of inferred joints.

4.6. Double Exponential Smoothing Filter

The double exponential smoothing filter is a popular smoothing filter used in many applications. Similar to a double moving averaging filter, the double exponential smoothing filter smooths the smoothed output by applying a second exponential filter (hence the name double exponential), and it uses this to account for trends in input data. There are various formulations of double exponential smoothing filters, with minor differences between them, but the most popular formulation is defined by the following set of equations:

$$\text{Trend: } b_n = \gamma(\hat{X}_n - \hat{X}_{n-1}) + (1-\gamma)b_{n-1}$$

$$\text{Filter output: } \hat{X}_n = \alpha X_n + (1-\alpha)(\hat{X}_{n-1} + b_{n-1})$$

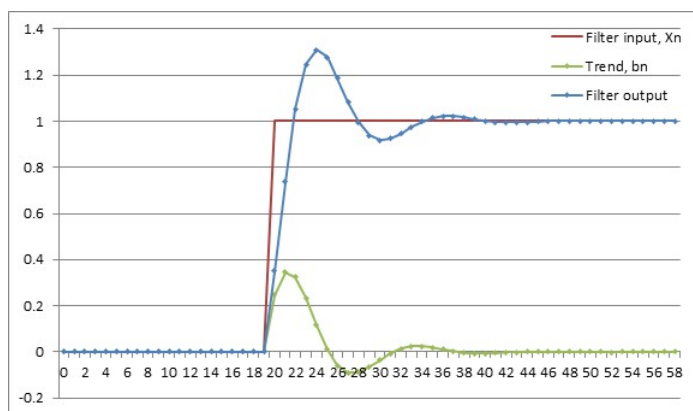
As can be noted, the trend b_n is calculated as an exponential filter of the difference between a filter's last two outputs. Then the sum of the current trend and the previous filter output—that is, $\hat{X}_{(n-1)} + b_{(n-1)}$ —are used in calculating the filter output. Including the trend helps to reduce the delay as the filter fits a line to local input data, where the trend b_n is the slope of this fitted line. The γ parameter controls the weights on the input data that was used for calculating the trend, and hence, γ controls how sensitive the trends is to recent changes in input. A large γ results in less latency in trend; that is, the trend element follows the recent changes in input faster, while a small γ gives larger weight to older input samples and, hence, results in longer delay in trend elements catching up with changes in input.

Note that a trend is the smoothed difference between the last two estimated joint positions (that is, a trend is the smoothed value of $\hat{X}_n - \hat{X}_{(n-1)}$); so a trend can be thought of as the estimated velocity of the joint in the case of NUI joint filtering. Therefore, we can think of γ as the dampening factor used in exponential filtering of joint velocity; and that the smoothed joint velocity is accounted for when joint position is calculated as the filter output.

The trend factor b_n can easily result in overshooting in filter outputs when there are sudden joint movements or stops. For example, suppose a player suddenly moves his or her hand and then stops, which results in a filter input similar to a step function (that is, a sudden jump in filter input). This is shown in Figure 7. In this case, the trend term b_n helps the filter output to catch up more quickly with this change in input; however, since b_n itself is smoothed out, b_n will need some time to settle back to zero, which will result in overshoot and ringing in output.

Note that there is a delay between b_n maximum and overshoot.

Figure 7. Output and trend of a double exponential smoothing filter in response to step function input; $\alpha=0.35$ and $\gamma=0.70$

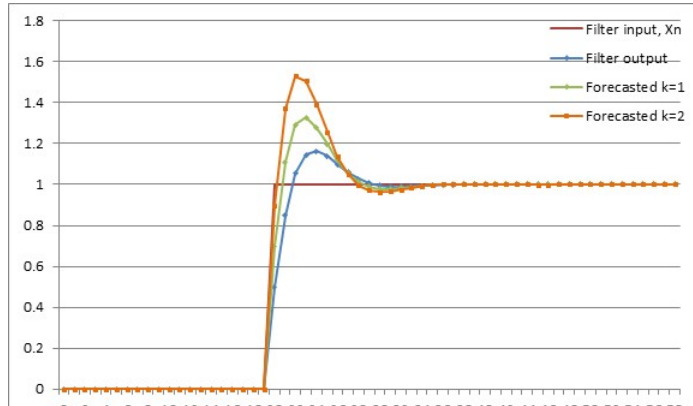


The double exponential smoothing filter fits a line to data and, therefore, forecasts the future data as a straight line with a slope equal to trend term b_n :

$$\hat{X}_{n+k|n} = \hat{X}_n + k b_n$$

In general, this filter performs better than a single smoothing filter in terms of forecasting. However, as shown in Figure 8, the filter overshoot is larger in forecasted outputs.

Figure 8. Output and trend of a double exponential smoothing filter in response to step function input; $\alpha=0.50$ and $\gamma=0.40$



There are numerical techniques that adaptively update the α and γ parameters such that the error in filter predictions for a given k are minimized in a least-square sense. For example, when predicting one sample ahead (that is, $k=1$), the prediction error at time n considering the past N forecasts is given by:

$$\text{Prediction Error} = \sum_{i=n-N+1}^{n-1} (\hat{X}_{i+1|i} - X_{i+1})^2$$

Then α and γ are updated at time n in a direction that this prediction error is minimized (for details, see the Levenberg-Marquardt algorithm in [10,12]). This criterion is useful if precise forecasting is the only concern; however, it does not take into account the smoothing effect of the filter, and so the α and γ parameters calculated by this approach do not necessarily result in smooth output.

4.7. Adaptive Double Exponential Smoothing Filter

A simple but useful improvement to a double exponential smoothing filter for NUI joints is to adjust the α and γ parameters adaptively based on the joint velocity, such that when the joint is not moving quickly more aggressive filtering is applied by using smaller α and γ parameters. This adaptation results in smoothed output when a joint is not moving quickly. Alternately, larger α and γ parameters are used when the joint is moving quickly, which results in better responsiveness to input changes and, hence, a lower latency.

This idea can be implemented in different ways. For example, the adaptive double exponential smoothing filter that is implemented in XDK samples implements this idea by using two preset α and γ parameters: one for low-velocity cases, say α_{low} and γ_{low} , and one for high-velocity cases, say α_{high} and γ_{high} . There are also two velocity thresholds used, say v_{low} and v_{high} . Then for each input X_n , the velocity is estimated as $v_n = |X_n - X_{n-1}|$, and filtering parameters α and γ are set as a linear interpolation between their low and high values based on the current velocity. For example, the α parameter used at time n , denoted by α_n , is set to be:

$$\alpha_n = \begin{cases} \alpha_{low} & v_n \leq v_{low} \\ \alpha_{high} + \frac{v_n - v_{low}}{v_{high} - v_{low}} (\alpha_{low} - \alpha_{high}), & v_{low} \leq v_n \leq v_{high} \\ \alpha_{high} & v_n \geq v_{high} \end{cases}$$

4.8. Taylor Series Filter

The Taylor series expansion is a well-known representation of a function in mathematics, where a continuous function $f(x)$ is expressed as an infinite sum of terms, calculated from its derivatives at a given point a [13]:

$$f(x) = \sum_{i=0}^{\infty} \frac{f^{(i)}(a)}{i!} (x-a)^i$$

$f^{(i)}(a)$ is the i^{th} derivative of function $f(x)$ at point a . This series is used in many applications for approximating a function as a polynomial of order N at points close to expansion point a , where N is the number of terms in the expansion that are included in this approximation.

The Taylor series can be used for forecasting NUI joint data. The underlying assumption is that joint movement is approximated with a polynomial of order N over time. In other terms, we fit a polynomial of size N to past N inputs, and then use this polynomial approximation to forecast next joint data. Note that the NUI joint data are in discrete time; therefore, the derivatives used in Taylor series coefficients are approximated numerically as higher order backward differences of input data. For example, the first, second and third degree derivatives are approximated as:

$$f^{(1)}(n) = X_n - X_{n-1}$$

$$f^{(2)}(n) = f^{(1)}(n) - f^{(1)}(n-1) = X_n - 2X_{n-1} + X_{n-2}$$

$$f^{(3)}(n) = f^{(2)}(n) - f^{(2)}(n-1) = X_n - 3X_{n-1} + 3X_{n-2} - X_{n-3}$$

As said, the next input is estimated by using the fitted polynomial—that is, $X_{(n+1)|n} = f(n+1)$. Substituting the preceding terms for $f^{(i)}(n)$ in a Taylor series polynomial expansion of size N and choosing $a=n$ results in an estimate for next inputs in term of past inputs:

$$X_{n+1|n} = \sum_{i=0}^N \frac{f^{(i)}(n)}{i!}$$

For example, for $N=3$, by using the preceding equations, one obtains:

$$X_{n+1|n} = \frac{8}{3} X_n - \frac{5}{2} X_{n-1} + X_{n-2} - \frac{1}{6} X_{n-3}$$

which is, again, a weighted moving average of past N inputs.

The Taylor series expansion is helpful, because it can forecast the future positions of joints. Note that the Taylor series does not smooth the data or attempt to remove any noise by itself, though this can be compensated for by applying a smoothing filter, such as exponential smoothing filter, to the output of a Taylor series filter. One approach for tweaking the Taylor series filter is to smooth the data that the Taylor series uses to forecast the current input from previous inputs (that is, $X_{(n|n-1)}$) and then calculate the smoothed output as a linear interpolation between X_n and $X_{(n|n-1)}$. That is, the filter output is given by:

$$\hat{X}_n = \alpha X_n + (1-\alpha)X_{n|n-1}$$

The Taylor series filter that is implemented in XDK samples uses this approach.

Also note that the approximation of derivatives using only the backward differences is a naive approximation of the derivatives [13]. This is imposed on us, because we do not have any future input data to use in difference equations to estimate the derivatives. The Savitzky-Golay filter is known to create good estimates for the derivatives. So, instead of the difference

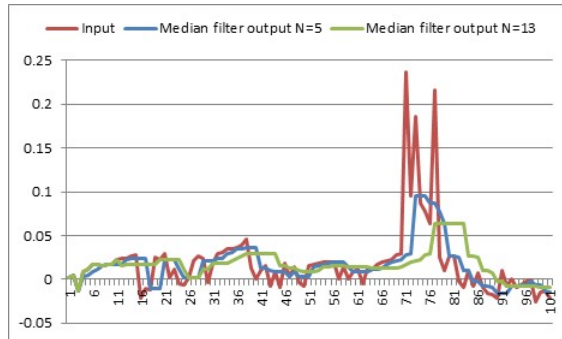
equations presented in the preceding text, one can use the Savitzky-Golay filter to calculate the derivatives and use them in a Taylor series filter to forecast future outputs. Also remember that the approximation by a Taylor series is accurate when the expansion is around a near point (that is, when $x-a$ is small), which means it is not practical to forecast more than one input into the future by using the Taylor series.

Note that the Taylor series filter may, at first, seem identical to the Savitzky-Golay filter, because both are fitting a local polynomial to input data. However, they are different, because a Savitzky-Golay filter is *over-fitting* a local polynomial to input samples, which means that the number of input data used to calculate the model parameters is greater than the model parameters. Since there is more than one potential solution, the least-square error approach is used to find the model parameters that minimize the error in a least-square sense. This allows the Savitzky-Golay filter to handle noise in the input data better, and the results are much smoother. However, in a Taylor series filter, there is no over-fitting of data, and the underlying polynomial is simply approximated by approximating the derivatives of input. This can be noted as exactly $N+1$ samples (N previous samples along current sample) are used in calculating the $N+1$ polynomial coefficients in a Taylor series approach.

4.9. Median Filter

In a *median filter* (also known as *moving median filter*), the filter's output is the median of the last N inputs. Median filters are useful in removing impulsive spike noises, as shown in Figure 9. Ideally, the filter size N should be selected to be larger than the duration of the spike noise peaks. However, the filter's latency directly depends on N , and hence, a larger N adds more latency.

Figure 9. Median filter applied to actual NUI data. Ideally, the filter order N should be larger than the duration of the spike noises.



Median filters do not take advantage of the statistical distribution of data or noise, and though they have some smoothing effect, they are not suitable for removing random noise from joint data.

4.10. Jitter Removal Filter

A jitter removal filter attempts to dampen the spikes in input by limiting the changes allowed in output in each frame. That is, filter output is the same as input if the difference between the current input data and the previous filter output is less than a threshold. Otherwise, the filter limits the changes in output, which can be done by using different methods. For example, the following variant of a jitter removal filter uses an exponential filter to dampen large changes seen in input:

$$\hat{X}_n = \begin{cases} X_n, & \text{if } |X_n - \hat{X}_{n-1}| < \text{threshold} \\ \alpha X_n + (1 - \alpha) \hat{X}_{n-1}, & \text{otherwise} \end{cases}$$

Alternately, one can use a simple averaging filter instead of the exponential filter. Since median filters usually perform better in terms of removing spikes, changes in input can be limited by the median:

$$\hat{X}_n = \begin{cases} X_n, & \text{if } |X_n - X_{med}| < \text{threshold} \\ X_{med}, & \text{otherwise} \end{cases}$$

Where X_{med} denotes the median of the last N inputs.

Jitter removal filters basically bypass the filtering for cases in which a jump has not been detected that is as large as the threshold in input data. Ideally, the threshold should be selected to be less than the impulsive jumps in the input that are due to spike noise, but larger than normal changes that are due to actual joint movements; in practice, these two criteria may overlap with each other. In a Kinect-enabled game, the joint tracking state can be used to adaptively select this threshold—that is, to use a smaller threshold when the joint data is inferred.

5. Practical Tips and Takeaways

Following is a summary of the tips described in this white paper:

- **No filtering solution fits all cases:** There is no filtering technique that can be universally used in all Kinect-enabled games. You must choose and fine-tune the filtering techniques that are right for your game.
- **Latency vs. Smoothing Tradeoff:** Be aware of the tradeoff between latency and smoothing in joint filtering. Understand how latency affects your game, and become familiar with different filtering techniques so you can choose and fine-tune the right filter for your game.
- **Filter per-joint and per-application:** Joints have a variety of characteristics, and depending on how the filter output is to be used, different filtering techniques should be used per joint.
- **Remember you can filter any data:** We usually apply filtering to the Cartesian coordinates (x,y,z) of joints; however, filtering can be applied to any data calculated from joint positions. For example, one can directly filter the bone length, relative coordinates of a joint with reference to another joint, spherical coordinates of a joint, and so on. Applying mathematical calculations to noisy data or calculating relative joints usually amplifies the noise, so be careful.
- **It may take more than one filter to get good results:** A good filtering solution is usually a combination of various filtering techniques, which may include applying a jitter removal filter to remove spike noise, a smoothing filter, and a forecasting filter to reduce latency, and then adjusting the outputs based on player kinematics and anatomy to avoid awkward cases caused by overshoot.
- **Use the joint tracking state:** Take advantage of the joint tracking state data, and use it in your filter implementations to apply more aggressive smoothing and jitter removal when a joint position is inferred.
- **Include future frame data in filtering, if possible:** In offline cases, or in cases when it's acceptable to increase latency by a few frames, include the future joint data in filtering (for example, use ARMA filters, such as central moving average or Savitzky-Golay with $M = 0$), which will result in better noise removal.
- **Account for the actual time elapsed between two NUI frames:** In some cases, the call to get the joint positions from the ST system may fail, which would result in a dropped frame. Therefore, the input data to the filter would be missing for that dropped frame, which means that the joint positions are no longer 33 msec apart from each other. Make sure that your filtering implementation uses the time stamp of the NUI frames and that it estimates the missing data points with an interpolation of data before and after the missing data.
- **Reset the filter when a skeleton is lost:** When a player moves outside of the camera's field of view, its skeleton is lost and is no longer tracked. The ST system may use a different tracking ID for the same player later, or it may reuse that tracking ID for other players. So make sure you reset the filter after a player skeleton is lost.
- **Use debug visualization to see how filter parameters affect your game:** Visualizing the filtered joint positions on top of the depth map is useful to see how your filtered data is different from the actual depth data. It is useful to visualize only one joint, like the right hand, and render a history of previous positions on screen. This joint path will show you how smooth the filter output was over the past frames.
- **Use Xbox Studio to export joint data and analyze them offline:** You can use Xbox Studio to export joint data and analyze them offline. Use the Excel spreadsheet that accompanies this white paper to quickly try different filtering techniques and change the filter parameters.

Note that the XDK includes the `NuiTransformSmooth` method, which is an implementation of a joint filter that is based on double exponential smoothing combined with jitter removal and some overshoot control. The XDK samples include implementations of some of the filters discussed; see the samples [Filtering](#) and [MonkeySeeMonkeyDo](#).

6. References

Error Analysis and Error Propagation in Noisy Data

1. John Robert Taylor, *An Introduction to Error Analysis: The Study of Uncertainties in Physical Measurements*. University Science Books, 1999.

2. Manfred Drosig, *Dealing with Uncertainties: A Guide to Error Analysis*. Springer, 2009.
3. Bevington, Philip, and D. Keith Robinson, *Data Reduction and Error Analysis for the Physical Sciences*. McGraw-Hill Science, 3rd edition, 2003.
4. Hughes, Ifan, and Thomas Hase, *Measurements and their Uncertainties: A practical guide to modern error analysis*. Oxford University Press, 2010.

ARMA Filters

5. Brockwell, Peter J., and Richard A. Davis, *Time Series: Theory and Methods*, 2nd edition. Springer, 2009.

Digital Filter Design in Frequency Domain

6. Oppenheim, Alan V., Ronald W. Schaffer, and John R. Buck, *Discrete-Time Signal Processing*. Prentice Hall, 1999.
7. A. Antoniou, *Digital Filters: Analysis, Design, and Applications*. McGraw-Hill, 2000.

Moving Average and Exponential Filters

8. Robert Goodell Brown, *Smoothing, Forecasting and Prediction of Discrete Time Series*. Dover Publications, 2004.
9. Hoang Pham, *Springer handbook of engineering statistics*. Springer, 2006.
10. Nocedal, Jorge, and Stephen J. Wright, *Numerical Optimization*, 2nd edition. Springer, 2006.

Savitzky-Golay Filter

11. Vijay Madisetti, *The digital signal processing handbook*. CRC Press, 2009.
12. William H. Press, *Numerical Recipes: the art of scientific computing*. Cambridge University Press, 2007.

Taylor Series and Derivative Estimation Using Difference Equations

13. Joe D. Hoffman, *Numerical Methods for Engineers and Scientists*, 2nd edition. CRC Press, 2001.

ST Joint Filtering and Latency

14. Claude Marais, "[Reducing Latency in Kinect Titles](#)."
15. Claude Marais, "Reducing Latency and Effectively Dealing with Noisy Data," presented at Gamefest Conference 2010 as part of the "Project Natal" presentations, available from <https://developer.xboxlive.com/en-us/xbox/documentlibrary/events/Documents/Gamefest2010.aspx>.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Kinect Image Segmentation: Challenges, Suggestions, and Solutions



By Allan Murphy, Software Development Engineer
Advanced Technology Group (ATG)

- [Problem Statement and Background](#)
- [Segmentation from Color Stream Alone](#)
- [Concise Problem Statement](#)
- [Adding Depth](#)
- [Applying the Depth Map in Practice](#)
- [Advanced Segmentation Sample Algorithm](#)
- [Algorithm Problems and Potential Improvements](#)
- [Note on Performance](#)
- [References](#)

Problem Statement and Background

Separating components of an image or a stream of camera images into background and foreground regions has many applications, not the least of which is separating specific parts: typically people from the foreground, and their surrounding environment from the background. The process is called segmentation. In game titles, segmentation often is used to place images of players in other settings for various entertaining effects, or to snapshot the player only for use in the user interface (UI) as an icon for example. This is similar to the green screen technique used in the television and motion picture industries.

On first examination, segmenting an image seems like it might be a relatively simple problem. Perhaps a simple subtraction of some kind of gathered background from the current image in color space will pick out the player only? Sadly, while occasionally giving reasonable results and seeming "almost there," this is not robust. Segmentation—from color streams alone at least—has been the topic of research and development for many years ([1](#)), and is still not robustly solved at the interactive rates (for example, 30 frames per second (fps)) expected in games.

The Kinect sensor provides a unique advantage in the segmentation problem. Depth information, as well as color, is available. It is tempting to think that adding depth will robustly solve the problem. While adding depth does provide a big advantage, it is still not quite as straightforward as it might seem.

This white paper offers:

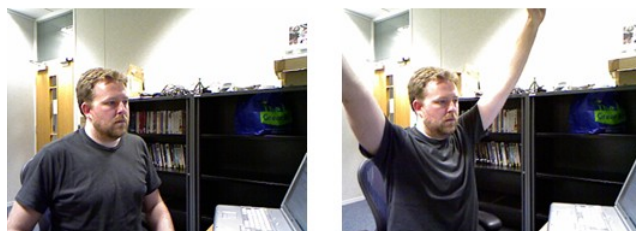
1. a discussion about challenges that make this problem difficult in color space alone
2. identification of advantages and challenges in using the NUI depth map to augment traditional solutions
3. a sample approach to improved segmentation

Segmentation from Color Stream Alone

Usually, color-stream-only-segmentation algorithms use a mechanism where the player initially is asked to move outside the camera view. This is done so that the system can snapshot the entire background area and build some kind of model of it for differentiating foreground at a later time.

There are several issues that make the color-only version of this problem difficult in practice, even with a pre-gathered background.

Figure 1. Side-by-side comparison



The image at left is the original; segmentation has been applied to the one at right. Although lighting, camera settings, and background have not changed, the image on the right is brighter.

Note that in the image at left, there is significantly more shadow at the top of the wall to the left of the door than there is in the image at right.

- [Noise and Intensity Variation in Color Signal from Consumer-level Cameras](#)
- [Intensity Variation and Motion in Target Environment](#)
- [Meaning of a Match Between Foreground Color and Background Color](#)
- [Shadows](#)
- [Holes](#)
- [Where to Start](#)
- [Circular Nature of Model](#)

Noise and Intensity Variation in Color Signal from Consumer-level Cameras

Like all cameras, the color signal from the Kinect sensor contains an element of noise. This means any background and foreground pixel comparison model must account for these variations—for example, via tracking deviation from the mean. Exact comparison of RGB components marks practically all pixels as not matching the background. This situation is the same in other color spaces such as HSV and YUV. Worse still, the color stream itself can vary markedly in overall brightness depending on the scene, without a change in the target environment (for example, via the camera auto-varying exposure time). Figure 1 gives an example.

Either this kind of change completely invalidates the pre-built background model because it was built some time ago without the player in view and at a different overall brightness, or the background model must cope with this change. Coping with the change effectively is difficult because at the time of intensity change, it is likely the camera is unable to see all of the background because the player will be in view. Pixels near the player boundary are the most important pixels to segmentation.

Intensity Variation and Motion in Target Environment

Typically, the environment the sensor sees is not completely fixed. The level of illumination may change markedly because of actual lighting variations—for example, the sun goes behind a cloud, or someone turns off a light. Illumination also may be affected by the background environment, which may move or alter—for example, curtains moving, objects outside curtains blowing in the window, doors opening, people moving in and out of view, or people occluding lights. Kinect games require players to move, and to swap in and out of the camera view in order to take turns, which makes the identified changes in lighting likely.

Meaning of a Match Between Foreground Color and Background Color

Even if a good background model of some kind is available, consider the situation where the color of an area from the current color frame matches the color of the area in the background model.

Imagine a player wearing a dark blue shirt and standing against a blue wall. Including lighting and shadowing effects, there will be pixels in the player's shirt that are close in color to the blue of the background. A straight comparison would be "this area here is blue, and that's the same as the background blue in the same position," but the player's shirt is definitely part of the foreground. A background model approach leaves the segmentation algorithm with no further way to classify an area of the current image at this point. Unfortunately, in typical scenes encountered in gaming scenarios—for example, living rooms or people in everyday clothing—this situation is common and leaves holes in the resultant segmentation where foreground is incorrectly classified as background. In Figure 1, areas of the T-shirt around the armpit are close in color to the dark areas in the shelving in the background. There may be no edge to detect in areas like this, and no difference between the foreground and background colors.

In fact, many approaches to segmentation adopt both high-frequency and low-frequency background models to deal with the situation of foreground color matching background color. Keeping a background model per pixel represents a high-frequency model. Other low-frequency models—something representing macro-level information about larger areas of the image—are used to describe the background in an attempt to resolve the matching problem. Fundamentally though, most algorithms struggle when foreground color matches background color in significant areas. Segmentation techniques that rely on finding silhouette edges often fail badly here because they *leak* where edge pixels match the background. In Figure 1, for example, the skin on the upper side of the raised right arm is illuminated enough to be similar to the bright wall behind, even though they are different colors. Changing camera exposure may fix this edge, but it also may force problems elsewhere—for example, edges in darker areas.

Shadows

Depending on lighting in the environment, the player's shadow may well be cast on visible objects. In the "color stream only" situation, background areas in the player's shadow will register as being different from the background model, and may be erroneously marked as foreground. A simple "area that is same color as background but darker is shadow" approach does not work well because the illumination in the room may make shadow regions of differing colors depending on light color, position, and occlusion. Shadows can overlap to make deeper areas of shadow, and so on. Marking pixels that are the "same color as the background model color but darker" as background can also incorrectly mark parts of the player as background. This incorrect marking can occur, for example, if the player's shirt happens to be the same color as the background but darker, and is classified as shadowed background.

Holes

Approaches often are derived that work well for closed foreground areas, but do not for holes in the foreground. Holes in the foreground region can happen. Imagine, for example, a player with his hand on his hip, which is an area of an image inside the player bounds that is a completely legitimate background.

Valid holes containing background pixels can cause significant problems. Recall that areas of the player may happen to be the same color as the background, and, thus, may be marked erroneously as background and leave small holes. It is tempting to think that filling any interior holes in the player shape will fix that problem.

Any approach that proposes automatically filling accidental holes in foreground areas, however, must avoid filling holes that are actually background. Simply because a hole is inside the player silhouette does not mean it is not background; automatically filling these holes is an incorrect action. Unfortunately, deciding if pixels in a hole are background or foreground is effectively a subset of the global segmentation problem. To make matters worse, holes often contain areas of background that are shadowed.

Where to Start

Given just a single color frame, there is no set of pixels that can be determined as definitely foreground or definitely background *a priori*. Even pixels that vary quite markedly from their background image equivalents are not necessarily definitely foreground (for reasons given above). The lack of a definite starting point hinders segmentation algorithms, and makes it difficult to disregard small, unconnected areas or islands of pixels identified as definitely background.

Circular Nature of Model

Using temporal coherence to build on a previous frame's segmentation results is sensible only if the segmentation is completely robust. Naturally, this is a circular argument. Non-robust implementations will result in a negative feedback loop and incorrect results.

Concise Problem Statement

The single biggest problem with segmentation can be described in one sentence: How do you classify regions when foreground colors match background colors?

Adding Depth

The Kinect APIs return a color stream in one of two formats. 640×480 32-bit RGB at 30 fps (frame rate dependent on lighting conditions) is the mode most games use; the alternative is high quality mode, which is 640×480 16-bit YUV at 15 fps (note that in the 640×480 30-fps image, RGB is reconstructed from the camera data; so it is less accurate than the 16-bpp YUV data). The sensor also returns a 320×240 16-bit depth stream—13 bits of depth value, 3 bits of segmentation mask. In segmentation, the addition of a depth stream gives a big advantage over the color stream only case.

- [Segmentation Mask Characteristics](#)
- [Challenges Using the Depth Map Effectively](#)

Segmentation Mask Characteristics

If you run the full NUI skeletal pipeline, the depth map returned each frame includes a segmentation mask value in the bottom 3 bits of each depth value in the depth bitmap, classifying each pixel as belonging to background (0), or part of one player (index 1–7).

The segmentation mask has several excellent characteristics that provide improvements over color stream only algorithms:

- Few false positives—the area marked in the mask is almost always just the player.
- Robust under a wide-range of lighting conditions.
- Able to deal with more than one player well.
- Copes well with background environment movement.
- Does not care about visible shadows (see Figure 3)

Voilà. Segmentation job done? Unfortunately, no. The last three points effectively remove some problems as mentioned above, but challenges remain.

Challenges Using the Depth Map Effectively

The Segmentation Sample included with the Xbox 360 Development Kit (XDK) shows the results of using the segmentation mask from the depth map stream alone to segment the player from the color map. While this gives results many 2D segmentation algorithms would kill for—for example, it never *leaks* into large areas of the background—there are some areas for improvement. If segmenting the color frame by using the segmentation mask alone, you will find:

- At player edges, some parts of the background are included in the segmentation mask.
- At player edges, some parts of the foreground are excluded from the segmentation mask.
- Hair in particular causes problems, as it tends not to give consistent depth reading.

Working with the segmentation mask has several challenges in practice:

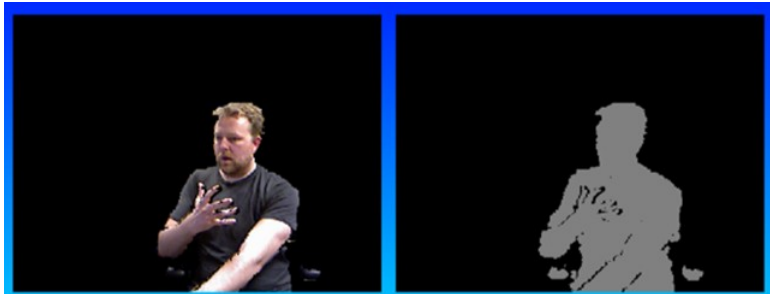
- Depth and color frames are not recorded from same world position.

The depth and color sensors are not in the same physical position on the Kinect sensor, and so they return slightly different views of the scene, just as each eye returns a different view of the world. To do a good job of segmentation, these streams must be matched up—the difference is too great otherwise. Using **NuiImageStreamOpen**, one stream can be registered to the other (that is, transform one stream to match the other), but the transformation necessarily results in some artifacts. Which stream is transformed makes a difference to the artifacts encountered.

The depth buffer can contain values labeled as **NUI_IMAGE_DEPTH_NO_VALUE** (corresponding to zero), which means the camera is looking at something inside or outside its range (0.8 m $\rightarrow$ 4.0 m). An example would be the camera pointing at a window or through a door into another room; most likely the nearest object is farther than 4 m away.

If the color frame is registered to the depth frame (color frame transformed to the space of the depth sensor), color values are lost where the depth map is **NUI_IMAGE_DEPTH_NO_VALUE**. This is because there is no sensible way to manipulate the color map to fit depth at points where the depth is unknown. The registration algorithm in this case cannot put any sensible values in the resultant color map—it inserts black pixels that look like cracks (see Figure 2).

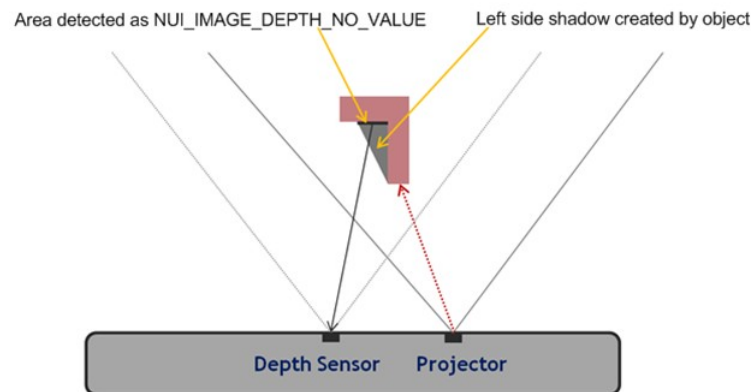
Figure 2. Segmentation example



In the left image, the foreground is segmented from the color frame by using the segmentation mask from skeletal tracking (ST) in the right image. The depth stream has been registered from the color frame, so cracks appear.

Additionally, the depth map contains shadows. If the current scene is viewed from the point of view of the Kinect sensor, then the sensor reading the depth values is to the left of the infrared projector that marks the depth. This means some areas of objects visible to the sensor do not have their depth marked, especially along the left side of vertical edges in depth.

Figure 3. Shadow in depth along vertical edge caused by differing sensor and projector positions



As shown in Figure 3, areas of the scene not marked by the projector will be marked as **NUI_IMAGE_DEPTH_NO_VALUE**, so some objects in an unregistered depth map will have an outline of **NUI_IMAGE_DEPTH_NO_VALUE** along depth discontinuities—essentially a gap. When the color map is registered to the depth map, discontinuities will appear as a black area exactly where segmentation algorithms aim to get a neat outline. Those color values need to be unchanged for the best results. Worse, these black areas in the RGB map caused by discontinuities in the depth map are indistinguishable from areas of the scene that are too close or too far from the camera.

Conversely, if the depth frame is registered to the color frame, depth values are transformed to the space of the color image, and small holes and cracks appear in the segmentation mask where there are depth edges. In the right image shown in Figure 2, you can see these holes—particularly around the hand and forearm.

Depending on the image processing you are doing, one of these two registration directions will have more advantages than disadvantages. For segmentation, we cannot afford to damage the original color stream frame, so we are confined to registering depth to color and coping with artifacts in the depth map.

- Segmentation mask is half the resolution of the color buffer

By nature, using the 320×240 depth frame means 2×2 blocks in the 640×480 color frame are associated with a single depth. Any segmentation decision based on the 320×240 segmentation mask classifies four color frame pixels at once. This can never give us the smooth outline we would like at 640×480 . Errors will appear around the very edge of the player where the player boundary runs through the 2×2 blocks. In general, the segmentation mask is conservative—most are 2×2 blocks marked in the color image, and are in the foreground. In some cases, however, especially with regard to hair, the segmentation mask can be too conservative, leaving larger areas of the player marked as background; this also happens intermittently along edges.

- Noise in depth map

As with all cameras, there is some noise in the depth image returned from the sensor. Along edges of objects, this noise will cause pixels in the segmentation map to flip occasionally between background and foreground, even with immobile objects. This noise means the edges of the segmentation mask are unreliable.

Applying the Depth Map in Practice

Since the task at hand is refining the segmentation of the player in the 640×480 color image, it seems wisest to register depth to color. Registering color to depth results in unacceptable information loss just where it is needed most for this algorithm (see [Challenges Using the Depth Map Effectively](#)).

The first advantage over a color stream only algorithm is that with depth there is a concrete starting point. The segmentation mask marks an area that is almost entirely foreground, with errors along the edges only. Moving forward from this state, however, is not simple for several reasons.

1. The final segmentation result has to be at 640×480 , and the information from the depth map is half that resolution. Interpolating depth or some other mechanism of smoothing or filtering depth values will only tend to create values that do not necessarily represent anything in the world and make life more difficult, not less.
2. Upscaling the segmentation mask to 640×480 , and then blurring and cutting out the player with alpha-blended edges in the blurred border regions will work, but edges will be vague and the player area may be missing significant parts—like hair.

3. The area that needs the best information possible—along the silhouette of the player—is exactly the area where depth is most susceptible to noise and not erring consistently one way or another. That means not relying upon expanding the segmentation mask until the edge is found, or similarly contracting the segmentation mask to find the correct foreground pixels.
4. While the segmentation mask marks an area that is entirely in the foreground, it has cracks and holes because of registration. It is not simple to determine if a pixel in an area inside the player area is genuinely background, or is part of a small, but valid, gap—for example, between arm and chest. To use this map in practice, the holes and cracks need to be filled without filling valid holes.

The big problem is that once the depth map is unable to help classify pixels, which is the case along the silhouette of the player, there is no other information available to assist in pixel classification than the color map. With the color map only, the problem rapidly degenerates to the color stream only segmentation case, which has its own set of associated problems and dangers.

The following section describes one method of trying to refine this situation and improve the ability to find the player's silhouette at full color map resolution.

Advanced Segmentation Sample Algorithm

The algorithm used by the Advanced Segmentation sample tries to improve the situation described in the Applying the Depth Map in Practice section, and deal with the issues from the color stream only case in four ways:

1. Breaks pixels into three sets: background, foreground, and boundary. Boundary pixels are pixels from the area around the edge of the player. These pixels, which may be part of the player or part of the background, still need to be refined.
2. Builds a model of the background color at 640×480 , and uses it to classify boundary pixels.
3. Finds player and environment edges, and uses the edges to further classify boundary pixels.
4. Classifies remaining pixels via fallback.

It should be noted that for each stage listed above, there are multiple ways to implement the classification provided. For example classifying a boundary area can be done many different ways.

Figure 4. Example scene (left) and player segmentation mask (right) from the ST pipeline



- [Stage 1 – Break Pixels into Three Clear Sets](#)
- [Stage 2 – Model Background Color](#)
- [Stage 3 – Find Player Edges](#)
- [Stage 4 – Classify Remaining Pixels](#)

Stage 1 – Break Pixels into Three Clear Sets

As noted in the [Challenges Using the Depth Map Effectively](#) section, the segmentation mask (as overlaid on the color frame) contains mainly foreground pixels and some background pixels around the edges. It also contains some cracks and holes.

Figure 5. Player segmented (left) using the ST segmentation mask (right)



The cracks and holes can be dealt with via the morphological operation of closing, which effectively is a morphological dilate followed by an erode. This removes small holes and cracks without bloating the silhouette.

The closed segmentation mask now provides an excellent starting point (see Figure 4), as well as a best worst case that is identical to the Advanced Segmentation sample, but without pixel level holes:

- The final result should never contain *less* than the closed segmentation mask pixel set. Otherwise, it is likely valid foreground pixels were thrown away.
- The final result should never be considerably larger than this set of pixels, because the segmentation mask leaves out large areas like arms or heads only in extreme conditions. Extreme cases include a high IR level in the scene, or parts of player outside depth range of sensor. In these cases it is fair to assume color stream only segmentation would also be in deep trouble.

If the color map is segmented using this mask, portions of background are still included for reasons detailed in the [Applying the Depth Map in Practice](#) section. In Figure 6, areas across the right shoulder and the top right of the head are missing.

Figure 6. Player segmented (left) using the closed segmentation mask (right)



It is now helpful to classify as many pixels as possible as background or as foreground, not only to improve the situation, but also for performance reasons. The majority of the color map usually is background. It would be helpful to disregard this in further processing, especially when working with 640×480 color maps, which are a significant size in memory.

A further observation is that, at worst, the segmentation mask only penetrates the actual player foreground area by a limited amount. Therefore, it is possible to erode the closed segmentation mask by enough that the remainder consists of entirely valid foreground pixels (although missing some valid foreground, naturally). Figure 7 illustrates segmentation erosion.

Figure 7. Player segmented (left) using the closed, eroded segmentation mask (right)



Similarly, the segmentation mask tends only to "leak" into background pixels around the player edges to a short distance. The mask can be dilated enough that the dilated version always contains the entirety of the player (along with some background, naturally), as shown in Figure 8.

Figure 8. Player segmented (left) using the closed and dilated segmentation mask (right)



The mask selects the entire player, including the outline. It also selects some extra pixels of background.

From the two masks, two other useful masks can be derived: pixels that are definitely background for this frame—the complement of the dilated mask—and the set of boundary pixels that contain a mixture of background and foreground pixels as shown in Figure 9. The boundary area is the intersection of the complement of the eroded mask with the dilated mask, or, in other words, the dilated mask with the eroded mask removed as illustrated in Figure 10.

Figure 9. Scene segmented (left) using the complement of the closed and dilated segmentation mask (right)



The mask selects pixels that are definitely background only.

The process above breaks the pixels in the scene into:

- A set that is definitely foreground.
- A set that is definitely background.
- A set that definitely contains player edges, which are still to be classified.

Mask construction can be done entirely with morphological operators on a 320×240 1-bpp mask, which is quick to work with on the CPU. 9600 bytes is fast to load and using VMX many operators can be done on 16 pixels in one instruction, which drops instruction count considerably.

Figure 10. Scene segmented (left) using the intersection of the closed and dilated segmentation mask and the complement of the closed and eroded segmentation mask (right)



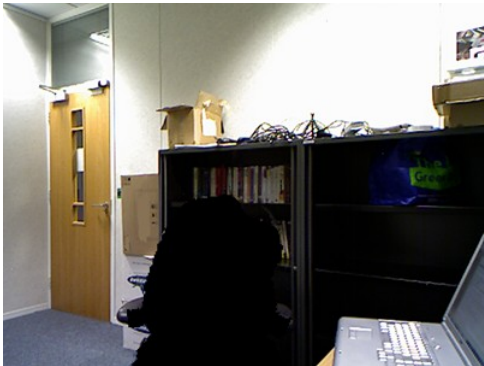
Additionally, the count of pixels has been reduced markedly to do more complex classification. In Figure 10, ~75 percent of the image has been identified as background, and ~15 percent as foreground; neither group requires further work. Only ~10 percent of pixels need more investigation; dropping ~90 percent of 640 × 480 RGB values is a good optimization (especially for CPU).

Stage 2 – Model Background Color

As per the color stream only algorithms, segmentation is enhanced by using a background color model. There are, however, several advances on that model. Stage 1, which is shown in Figure 9, generates a mask containing pixels that *definitely* are background. A model of the background color can be maintained by using only pixels from the color image that are definitely background. This can be done while the application is running rather than requiring an initial background snapshot. This provides a secondary benefit. Because the background model can be updated continually, it can adapt to intensity changes on the fly. This can be done by keeping a 640 × 480 color map, which contains the cumulative moving average of the colors encountered for that pixel in the background. The background color model contains a color map *and* a bitmask that indicates if each pixel has ever had a valid color recorded for it. The bitmask, however, is separate from the color map.

As the player moves around, more background is exposed to the background model (this is similar to the Motion-Based Background Removal Sample in the XDK), so the model gradually builds a picture of most of the environment. The cumulative moving average buffer model can be imagined as starting as a picture of the scene, but with a black shape where the player is (area where no background color has been grabbed because the player occludes it). As the player moves around and exposes more background to the model, the black area gradually reduces in size until (ideally) most of the background is seen. Figure 11 illustrates this condition.

Figure 11. Background color model



The lower central black area is the region where foreground pixels have always occluded the background. In this case, this represents the player's torso.

The border pixel set can be further refined by comparing pixel color from the current color frame from the sensor with the background color model. Colors that are a long distance from the background model color in RGB space are definitely foreground, and can be marked as such. This tends to provide a speckled improvement in the border pixels—given a suitable distance threshold—but it can leave large areas of foreground in the silhouette area still unclassified. Remember that a pixel color that is close to the background color is not the same as the pixel being background (see [Stage 3](#)).

Table 1. Classification logic for background model comparison

Background model comparison result	Meaning
Current frame and background color are very different	Pixel is foreground in current frame
Current frame and background color are very similar	Cannot classify pixel with this information

Stage 3 – Find Player Edges

Border pixels can be further classified with edge detection. Sobel edge detection is considered, which returns an intensity value according to the strength of the change along an edge, and is relatively inexpensive compared to more advanced (and expensive) edge detectors like Canny-Shen. Just detecting edges in the current color frame from the sensor does not help a great deal, though some initially interesting, but ultimately fruitless ideas are listed here:

- Find the player silhouette in the depth map via edge detection, and use to segment the color frame.
The depth map is half resolution, so it will not give a refinement at the resolution required. It also contains areas of NUI_IMAGE_DEPTH_NO_VALUE, which will register as strong edges that are irrelevant to the scene. Repairing the result of NUI_IMAGE_DEPTH_NO_VALUE means synthesizing depth values in those areas. Those synthesized values will not represent actual objects, and will give incorrect edge results in many cases.
- Use edge detection on the color map to follow player silhouette edges.
The player's silhouette is not guaranteed to be a strong intensity edge at all points (see Figure 1, right image), though large parts of it are. Remember that parts of the player may be similar to the background color, and, thus, give a low-intensity or zero-intensity edge detection response. Tuning an edge-following algorithm to work with lower-intensity edges results in the algorithm following edges that are not along the edge of the player, but are off into the background or into the interior of the player image.
- For edge detection on the color map, use a depth map to attenuate edges not in the foreground.
This seems promising, but the very reasons that make the ST segmentation mask imperfect cause this approach to have noise exactly where it is least welcome—at edges. Also, it is worth remembering that the player may be near the camera's depth range, so attenuating by depth might actually reduce the intensity of the player edges that should be retained.
There is, however, one possible useful relationship here—areas where the current color frame differs from the background model are of interest. Edges in the current color frame can be found, but edges in the background model maintained in stage 2 also can be found (see [Stage 2 - Model Background Color](#) for details).
Edges in the current color frame can be found from the sensor, but edges also can be found in the background model as described in [Stage 2](#). The algorithm maintains a picture of the background seen so far. There is a useful relationship between the current color frame edge response and the background model edge response at any particular pixel. Table 2 identifies two situations where the pixel under consideration is definitely foreground. Figure 13 shows the result of the operation.

Table 2. Classification logic for edge response combinations

	Current color frame response	Background model response	Meaning
1	Low	Low	Cannot classify pixel with this information
2	High	Low	Pixel is foreground in this frame

3	Low	High	Pixel is foreground in this frame
4	High	High	Cannot classify pixel with this information

Let's consider each of the four situations.

In situation 1, the current color frame does not contain an edge at this pixel, nor does the background model. This could mean the current frame contains a flat area in the foreground that happens to be over a flat area in the background. Or, the current frame could contain the same flat background area as the background. No conclusions can be drawn from this situation.

Situation 4 is the reverse of situation 1. The current color frame contains an edge, and so does the background. Again, no conclusion can be drawn here. Perhaps the edge in the foreground is the same edge as in the background, perhaps not.

In situation 2, there is a high edge-detection response from the current color frame, but not from the background model image. This situation means something in the current color frame has a strong edge where the background does not. This must be an edge in a foreground object obscuring a flatter area in the background.

Situation 3 is less intuitive, but still identifies valid foreground. There is a high edge-detection response in the background color image, but not in the current color image. An object with *no* edge in this position is occluding an edge in the background color image, so this pixel, too, is definitely part of foreground.

Table 2 shows two input values mapping to an output value. The relationship between the two values is the same as the Boolean operator XOR. In practice, however, the useful value is the absolute difference between the intensities in the two-edge detection responses. This comparison—with an appropriate threshold—gives a good full resolution outline of the player. The outline is not necessarily complete, and for that reason, is would be incorrect to flood fill any internal areas, for example, to get a complete model. Furthermore, the edge cannot be followed. Combined with the background model described in [Stage 2](#), the algorithm starts to give better segmentation results at full resolution.

Figure 12. Sobel edge detector response on current color frame (left) and background model (right)

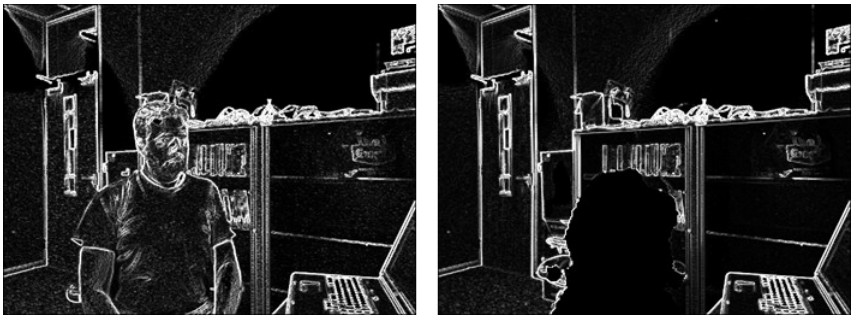


Figure 13. Difference of Sobel edge response



Note the improved outline, but speckled noise in the background.

Stage 4 – Classify Remaining Pixels

Stages 1 through 3 can leave some pixels in the boundary area that are still unclassified. These pixels will typically be of a color close to the background model (otherwise stage 2 would catch them) and are not part of a strong edge (otherwise stage 3 would catch them). Classification now collapses back to the world of the color stream only problem—colors close to background color are not easily classified. A suitable fallback is hard to determine here, but a reasonable one is just to fall back to the segmentation map. Most of these pixels tend to fall inside the foreground and not along edges, so falling back to a segmentation mask check here is not unreasonable.

Algorithm Problems and Potential Improvements

This algorithm still has several issues and areas for improvement, and is far from perfect.

- Valid holes showing background
The operation of closing on the original segmentation mask will tend to fill some small areas representing valid small holes. Additionally, the erode and dilate operations will tend to exacerbate this problem on very small holes. The result is not disastrous except on single pixel sized holes, which tend to be filled.
- Background model initially is less useful, or less useful when there is less motion
If the player does not move much, the background model will not get a chance to gather much background information around the player silhouette, unfortunately exactly where it is needed most. This effect can be reduced by dilating the original segmentation mask less. This increases the area of image labeled as definitely background, but at the same time risks foreground colors contaminating the background color model, which is undesirable.
- Close up objects break background model
If the player moves too close to the camera, areas of the player then are marked as NUI_DEPTH_IMAGE_DEPTH_NO_VALUE; in turn, those areas appear as background rather than as part of the segmentation mask. Those pixels then are marked as background, which means the background color model is updated with colors from the current color map in those areas, and are likely to be parts of the player. Contaminating the background model with foreground pixels degrades its performance enormously, but because it is continuously updated, it will recover eventually if the player returns wholly into valid depth camera range.
- RGB not best space for color comparison
Color spaces other than RGB do a better job of comparison, often with less data, which means better performance, too. The YUV color space is one suggestion, and there are other color spaces that cope with lighting differences more robustly and may give better results.
- Threshold derivation
The algorithm uses two thresholds—one for "high response" in the Sobel results comparison, and one for "far enough away" in the color comparison with the background model. Both are derived from experimentation, but may perform badly in varying lighting conditions. It should be possible to make these comparisons more robust by using an overall unclamped intensity (for Sobel) and standard deviation (for background model comparison).

Note on Performance

In a piece of bitter irony, an implication of some of the issues defined here is that the slower this algorithm runs, the more work it has to do—not unlike a physics simulation having to run more steps the slower it goes since objects can move farther between updates. The longer the time between frames with this algorithm, the more erosion and dilation is required to be certain that the player silhouette is captured in the classification phase. The more the silhouette is dilated, the more likely it is that the background model never gathers good color data around the player edge, the more likely that part of the algorithm will give poor results. In other words, this algorithm will work best when fed new color and depth frames at 30 fps (the

maximum update rate of the camera).

References

(1) A paper reviewing image segmentation techniques by Dr. Maneesha Singh, a Research Fellow at the Research School of Informatics, Loughborough University, Loughborough, United Kingdom. <http://www-staff.lboro.ac.uk/~coms/Segmentation-review.htm>

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Know Thy User: Understanding Implications of Player Characteristics and Behaviors when using the Natural User Interface



by Scott Selfon, Lead Program Manager
Advanced Technology Group (ATG)

Note

This documentation is an early release of the final documentation, which may be changed substantially prior to final commercial release and is confidential and proprietary information of Microsoft Corporation. It is disclosed pursuant to a nondisclosure agreement between the recipient and Microsoft.

Today's game input devices are relatively standardized: keyboards, mouse buttons, thumb sticks, and triggers allow for flexible layout and implementation, yet they provide fairly determinate, robust, and predictable input. Modifying this behavior is left to the user of the input device—for example, understanding a new keyboard layout, adjusting to the shape and size of a mouse, or learning the resistance of various buttons and triggers.

When the player's body is the input device, however, many assumptions that developers can make about input must be questioned, revisited, and modified. This white paper discusses some of the potential challenges about understanding and responding to people, with all of their unique attributes, as the source of input for gameplay.

Note

Because the sensor arrays that are currently shipping are pre-release hardware, their specifications, fidelity, optics, and form factor may be subject to change. The guidelines presented here aim to minimize divergence with any future hardware updates, but the guidelines will be updated if future hardware revisions offer different performance characteristics or when best practices can be recommended.

For details on the sensor array's field of view, guidance on expected placement positions, and discussions about the playing environment, see [Natural Environments: Understanding and Developing for the Kinect Play Space](#).

End-User Goals and Performance for the Natural Input Sensor Array

The sensor array is designed to allow for interaction with children and adults alike; principal height targets support those between approximately 3.3 feet (1 meter) and 6 feet (1.8 meters) tall, with support for taller or shorter players being enabled through adjustment by users to the tilt and position of the device. Note that performance and accuracy may degrade as the vertical angle between the sensor array and player increases, because the system will be working hard to compensate for both distorted perceptual characteristics (for example, a visibly larger head and visibly smaller feet for a high-mounted sensor array) and occluded skeleton positions.

The sensor array is not adversely affected by most typical user characteristics, such as:

- Height
- Weight
- Body shape or type
- Skin tone
- Clothing color
- Gender

The depth sensor is designed to recognize people in a wide variety of environments, even when the sensor array is fully tilted (+/- 28 vertical degrees from its center position) or doesn't have full view of the player's feet. However, information about depth and skeleton positions may become less reliable in some player scenarios, like the following:

- Clothing or accessories are strongly reflective or refractive (for example, jewelry).
- Clothing is highly light-absorptive (for instance, dark fluffy fabrics).
- Clothing masks the player's general shape or obscures the locations of skeleton positions.
- Objects obstruct the sensor array's view of the player (whether environmental or being held by the player).
- Hair obscures the location of skeleton positions. As with fluffy materials that tend to absorb or disperse light rather than reflecting it back to the sensor array, hair can act as a mask for recognition of skeleton positions. If the hair fully obscures shoulders, then detection of the entire skeleton may fail.
- Skeleton positions in direct contact with each other, with minimal difference in position or depth (for example, arms or legs crossed, arms and hands flat at the player's side, and so on).

Certain user positions may create issues with the stability of skeleton positions, leading to jittery or otherwise unpredictable positioning. Examples of body positions that can cause problems include:

- Having skeleton positions that are outside of the sensor array's field of view (that is, they are off screen, hidden behind objects or other skeleton positions, and so on).
- Sideways and backward orientation of the player's body (where multiple skeleton positions are obfuscated).
- Some seated poses.

Intermittent noise in skeleton positions can be corrected by using the smoothing functionality of the APIs, or functions implemented by the title, and more severe confusion of skeleton positions may be handled by future software improvements. Titles can also use the confidence values returned with skeleton positions (NUI_SKELETON_POSITION_CONFIDENCE_* constants) to determine the quality of the data that is received from the sensor array.

Playing with Friends

Kinect for Xbox 360 supports multiplayer gaming with the sensor array (currently, with tracking for up to two skeletons). In general, ordering will remain consistent, so long as neither player is fully obstructed by the other player or by an object, and presuming neither player leaves the playspace. However, as players cross fully in front of or behind each other, ordering cannot be guaranteed by current releases.

When two players are interacting with the sensor array simultaneously, the chances for a skeleton position to become momentarily obfuscated can increase. Despite this, the sensor array is designed to support and successfully distinguish between two active players within its field of view; the accuracy of and confidence in the location of a skeleton position will reflect whether it is visible to the sensor array.

Developing User Feedback Mechanisms

What should a title do when skeleton positions become unreliable or the user is otherwise becoming difficult to detect? User feedback is a key mechanism for maintaining the magic of natural input experiences, and titles should provide positive and intuitive mechanisms to assist the player. Note that, while it is expected that the most common scenarios (lost skeletons and exiting the field of view) will have solutions provided by the system UI, developers may still wish to focus on how to handle game mechanics in cases like the following:

Behavior when skeleton is lost

Developers may wish to adopt mechanisms similar to those for disconnected controllers, with titles automatically pausing the game and waiting for the player to be reacquired before accepting further input. Visual feedback may further assist the player.

Behavior at edges of the playspace

Developers may wish to provide visual and/or audible hints to the player that they are approaching boundaries, so they can respond appropriately and move back into the optimal playspace without feeling overly restricted.

Responding to Player Movement

A title's response to the player physically moving around the playspace can vary significantly. Some ideas include noting only movements relative to the player's core location, mapping the

playspace to the game space, and considering only interactions with virtual objects.

Strictly Player-Relative Actions

For some titles, player movement around the playspace may be effectively ignored; playing in the left back edge of the playspace is intended to be identical to playing in the right front edge. In this case, the title will typically track all gestures and poses relative to the player's core location as he or she moves around the playspace.

Mapping the Playspace to the Game Space

For some environments with naturally existing boundaries (for instance, the batter's box in a baseball game), it is natural to map the playspace to the game's virtual space; if the player moves to the right, their virtual on-screen representation reflects this real-world movement. If the virtual space is larger than the available playspace, player's movements can often be scaled to at least twice their real values.

Interacting with Virtual Objects

Directly mapping the player's actions to the game space may create a less than satisfying experience when virtual objects don't appear where the player perceives them to be, or when it is non-trivial for the player to reach the object's corresponding location in real-world space. In many situations, titles might need to allow for more forgiving interactions, with on-screen elements gravitating toward the player or responding to a larger area of interaction than their visual size might indicate.

In addition, visual input alone isn't always enough for the player to understand and successfully manipulate objects based on location. Developers should consider other forms of feedback, such as proximity-based acoustic cues and other navigational aids.

Summary

The paradigm of player as input device creates a new set of challenges for handling incomplete or noisy data, and for deriving intent from potentially infinite variation in input and from a player attempting to interact with virtual objects. This paper has discussed some potential sources of noise and mitigation strategies for translating this data into a compelling and engaging gameplay experience. The underlying XDK libraries will continue to offer improvements in data robustness over time, but considering the player experience at all times will lead to the most transparent and magical interactions that truly turn the player into the controller.

We encourage you to contact the Microsoft Game Developer Support Desk at gamedev@microsoft.com with any questions or comments about the topics presented here, and with challenges specific to your own title or engine, and to share your own experiences and best practices.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Reducing Latency in Kinect Titles

By Claude Marais, Sr. Software Development Engineer

Advanced Technology Group (ATG)

Published: June 15, 2010

Updated: October 8, 2010

The focus of this white paper is on visual latency, including skeleton tracking (ST), in Kinect titles.

Latency in a Kinect title can be defined as the difference in time from the instant a player starts a motion to the time that the player sees something changing on the TV based on that input motion. Managing latency in Kinect titles is crucial, and can easily mean the difference between an enjoyable player experience and a frustrating player experience.

The goal of this white paper is to explain how latency is introduced and what developers can do to reduce latency. It is aimed at software engineers who already have a basic knowledge of Kinect development.

Latency in Kinect Titles

To understand latency in Kinect titles, we need to look at all the components that introduce latency. Figures 1, 2, and 3 show all the components that introduce latency. Each figure shows a different view of the same data so that you can understand exactly where latency gets introduced, what is specific to Kinect, and where you can reduce latency.

Figure 1. Shows where latency gets introduced in Kinect titles

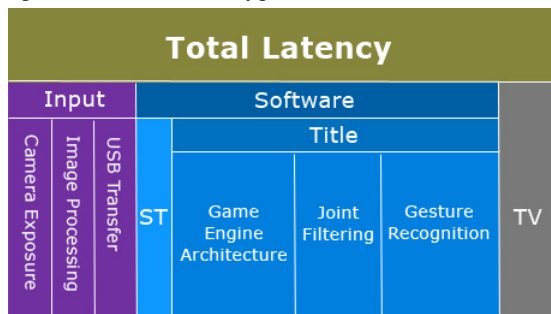


Figure 2. Shows which latency components are specific to Kinect

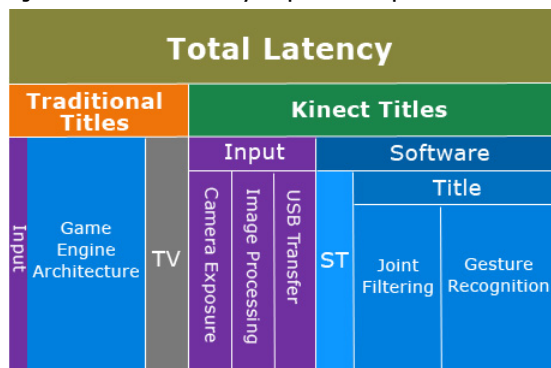
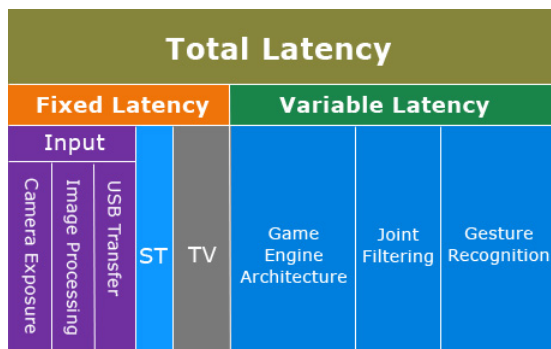


Figure 3. Shows which components have fixed latency and which have variable latency



Input

Xbox 360 games normally have an input latency of around 8 milliseconds (ms) from the Xbox 360 controller. With Kinect, input latency is on average 66 ms. This input latency is due to camera exposure, image processing, and USB transfer from the sensor array to the Xbox 360. Transferring image data from the sensor array also needs to be in sync with every second VBlank of the Xbox 360—that is, 30 Hertz (Hz)—so that data is received at the Xbox 360 in a consistent and predictable manner.

Software

Skeleton Tracking (ST)

ST introduces a small amount of latency—on the order of 10 ms to 11 ms—since the ST system first has to process the raw skeleton data from the sensor array before the title can use it.

Figure 4. Shows where and when ST is processed on the Xbox



Game Engine Architecture

Game Processing

At a minimum, it takes one frame of game processing to produce an image. All games have this one frame of latency. For a Kinect title running at 30 Hz, latency from this regular game processing is 33 ms.

Buffering Between Title CPU Threads

Game state and resources (for example, vertex buffers and dynamic textures) are normally double buffered so that different CPU threads can safely, and concurrently, read and write to the resources. A game update thread, for example, normally writes to and updates resources for the current frame, while a render thread is reading and rendering data from the previous frame. This resource double buffering comes at the cost of an extra frame of latency—33 ms for a 30 Hz title.

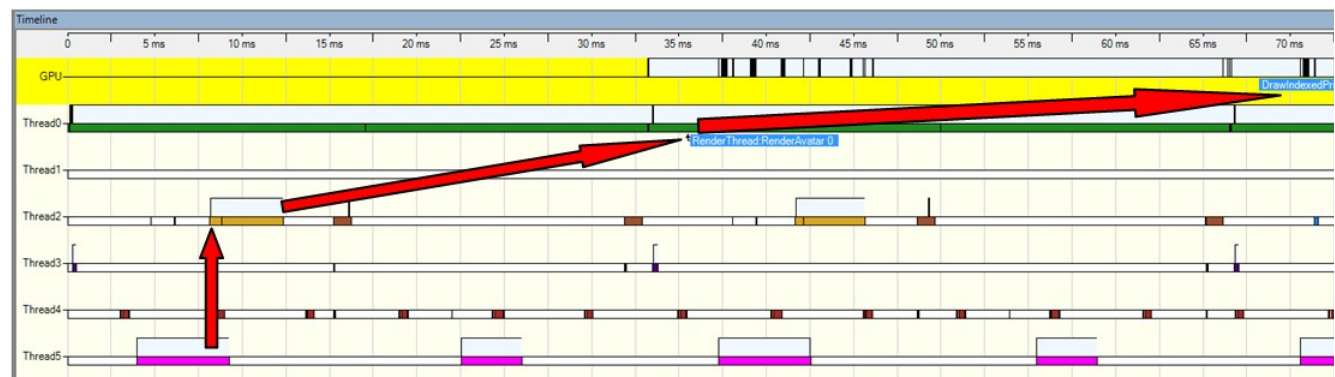
Buffering D3D Commands Between CPU and GPU

In order to make use of Xbox 360 hardware multisample antialiasing (MSAA) while rendering at high definition 720p, titles use predicated tiling. All render commands within a **BeginTiling/EndTiling** block are recorded into a command buffer so that they can be played back efficiently on the GPU for different tiles. This effectively means that render commands will be submitted only to the GPU once the title calls **EndTiling**. This could mean that the GPU can be waiting a significant time for the CPU to submit render calls before receiving commands to process. Using GPU double buffering, enabled with `D3DCREATE_BUFFER_2_FRAMES`, Direct3D allows the title to queue rendering commands in the command buffer for two complete frames. Double buffering permits the CPU to build a complete frame while the GPU simultaneously renders the previous frame. It also could introduce one frame of latency: 33 ms for a 30 Hz title.

Figure 5. How the traditional game engine architecture could introduce 99 ms of latency



Figure 6. PIX timing capture from the Reducing Latency sample's high latency mode



You can clearly see how this traditional game engine architecture could introduce 99 ms of latency. The first arrow shows that when ST is done, an event is set and the avatar can be updated with the new data. The second arrow shows that the data from the updated avatar is used on the render thread only one frame later. The third arrow shows how the rendered avatar gets to the GPU only one frame later.

Joint Filtering

The sensor array returns noisy data, because it is precise within the centimeter range, not the millimeter range. In many cases, body parts also might be occluded or clipped. Joint positions returned by the ST system, therefore, have to be filtered, which can add latency. For example, a naive average smoothing filter can add 66 ms to 165 ms of latency because it uses data from previous frames.

Gesture Recognition

Latency for gesture recognition can be defined as the difference in time between the instant that a skeleton with filtered joint positions starts a motion and the time that a gesture recognition system recognizes that motion. For example, determining that a skeleton has punched forward takes several frames to analyze before the gesture recognition system can verify that it is indeed a forward punch and an animation can start to play. The latency for gesture recognition is in the range of 99 ms to 264 ms.

TV

All games have to deal with TV latency, which is not something that a game developer can control. It is, however, useful to be aware of the range of TV latency players have to deal with. There are many different types of TVs (CRT, LCD, LED, DLP, Plasma, and so on), and some have game modes with reduced latency. TV latency is in the range 8 ms to 120 ms. On average, a good TV has 30 ms to 45 ms of latency.

Reducing Latency

Skeleton Tracking

Titles can retrieve skeleton data in one of two ways: event-driven or using a timeout. An event-driven system is recommended. Refer to the sample code in the XDK on how to use the following function:

```
HRESULT NuiSetFrameEndEvent (
    HANDLE hEvent,
    DWORD dwFrameEventFlag)
```

Using a title-defined event will ensure that the title is notified immediately whenever NUI processing is finished and new data is available, ensuring that the title processes the most up-to-date skeleton data.

If the title needs to also process depth and/or color data (for example, for visualization purposes), it is more efficient to create separate events for retrieving color, depth, and skeleton data on separate threads. Using separate events for color and depth data ensures that the title doesn't need to wait (5-7 ms) until skeleton tracking is done processing before being able to retrieve color and depth data.

For signaling when skeleton data is available, use an event in the following function:

```
HRESULT NuiSkeletonTrackingEnable (
    HANDLE hNextFrameEvent,
    DWORD dwFlags)
```

For signaling when an image stream is available, use an event in the following function (note HANDLE hNextFrameEvent):

```
HRESULT NuiImageStreamOpen (
    NUI_IMAGE_TYPE eImageType,
    NUI_IMAGE_RESOLUTION eResolution,
    DWORD dwImageFrameFlags,
    DWORD dwFrameLimit,
    HANDLE hNextFrameEvent,
    HANDLE *phStreamHandle)
```

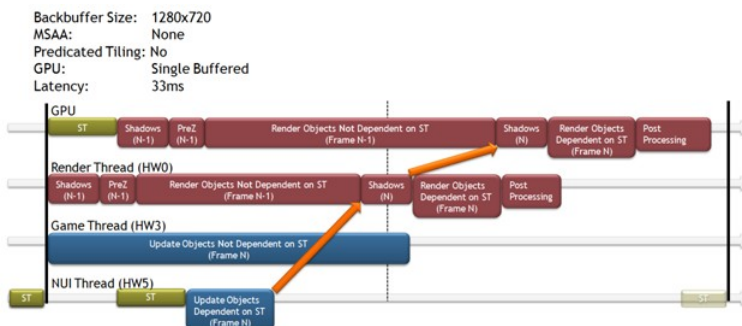
Latency for ST can easily be hidden by doing work in parallel on other threads that do not depend on ST input. For an example of doing work in parallel while waiting for new skeleton data, and of using an event-driven system to acquire data from ST, see [Reducing Latency](#), a sample in the XDK.

Game Engine Architecture

In Figure 3, we see that game engine architecture, joint filtering, and gesture recognition are really the only components where a title can reduce latency. Latency for the remaining components is fixed.

Compare Figure 7 with Figure 5. Figure 7 shows an architecture latency of only 33 ms, whereas Figure 5 shows 99 ms.

Figure 7. Proposed game engine architecture that has only 33 ms of latency



Shadow maps can be generated for all objects not dependent on ST input using resources updated from the previous frame.

If needed, a manual pre-Z pass can be rendered before rendering the main pass. Note that we normally recommend using the **BeginZPass/EndZPass** functions for an automatic pre-Z pass, but these functions work the same as **BeginTiling/EndTiling** in that all draw calls between the **BeginZPass** and **EndZPass** block will be rendered into a command buffer and will only be submitted to the GPU once the render thread reaches the **EndZPass** function, which could introduce latency.

The main render pass is next, and should not include any objects that receive shadows from objects (for example, a floor or terrain) dependent on ST since we first need to generate the shadow maps for the shadow casters dependent on ST input. It is, of course, possible to render the shadows of these objects from the previous frame, which will have the visual artifact of the character's shadow being rendered with one frame of latency.

Depending on how much time it takes to process the shadow, manual pre-Z, and main passes on the CPU, it is most likely that the skeleton data for the current frame has now been processed and is ready to be used for rendering. If not, you would have to wait until the data is ready by making use of a title event, for example:

```
WaitForSingleObject ( hAvatarUpdatedEvent, 16 );
```

Next, we can use this processed skeleton data to generate shadow maps for objects dependent on ST input, but first the back buffer and depth buffer have to be saved so that they can be restored after the shadow pass. For the most efficient way to restore the back buffer and depth buffer, including hierarchical Z (Hi-Z), see ["Xbox 360 GPU Performance Update,"](#) a presentation from [Gamefest 2007](#) by Matt Lee. Also see three samples in the XDK: [FastStencilFill](#), [PredicatedTilingFullDepth](#), and [Reducing Latency](#).

Once the buffers are restored, all objects dependent on ST input can now be rendered, as well as any other objects not yet rendered—for example, shadow receivers and transparent objects, as well as post-processing and UI render passes.

Using this mixture of resource double buffering and single buffering, we can reduce the latency for game engine architecture by one frame.

Figure 8. Proposed game engine architecture that has only 33 ms of latency

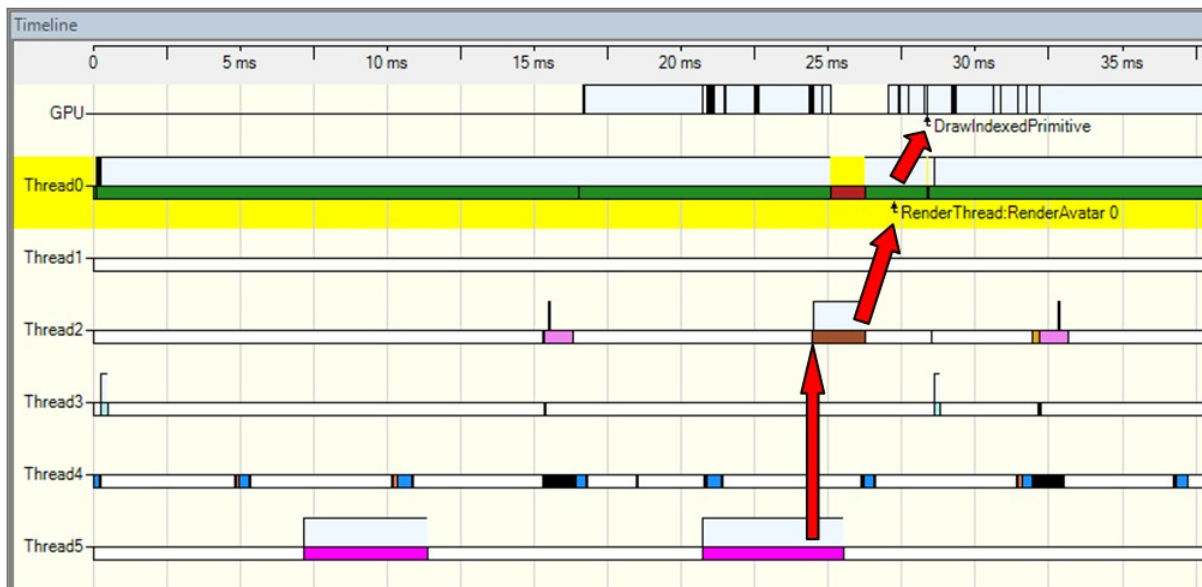


Figure 8 illustrates how this proposed game engine architecture introduces only 33 ms of latency. The first arrow shows that when ST is done, an event is set and the avatar can be updated with the new data. The second arrow shows that the data from the updated avatar can be used on the render thread during the same frame. The third arrow shows how the rendered avatar gets to the GPU during the same frame.

Buffering Between Title CPU Threads

It is possible to group rendering work into objects that are dependent on ST input and those that are not. Resources for objects that are not dependent on ST input can still be double buffered between CPU threads in order to use hardware efficiently. These objects could include crowd rendering, the background environment, and nonplayable characters (NPCs).

Resources for objects such as the main playable character that do depend on ST input can be single buffered in order not to introduce a frame's worth of latency. This means that the ordering in which objects and render passes are rendered on the render thread will be slightly different than the traditional game engine architecture. For an implementation of this proposed game engine, see [Reducing Latency](#), a sample in the XDK.

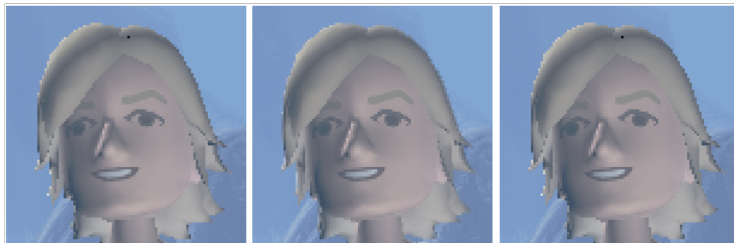
Using this mixture of resource double buffering and single buffering, we can reduce the latency for game engine architecture by one frame.

Buffering D3D Commands Between CPU and GPU

In order to reduce latency introduced by buffering D3D commands between the CPU and GPU, a title should not use predicated tiling or GPU double buffering.

As mentioned, the main reason for using predicated tiling is to make use of MSAA at 720p. Several alternative antialiasing techniques exist, including screen space antialiasing, edge-based antialiasing, and antialiasing introduced by post effects such as bloom and motion blur. Refer to the XDK samples [Reducing Latency](#) and [Edge-Based Antialiasing](#).

Figure 9. The image at left has no antialiasing, the middle image shows a screen space antialiasing technique, and the right image shows 2x MSAA

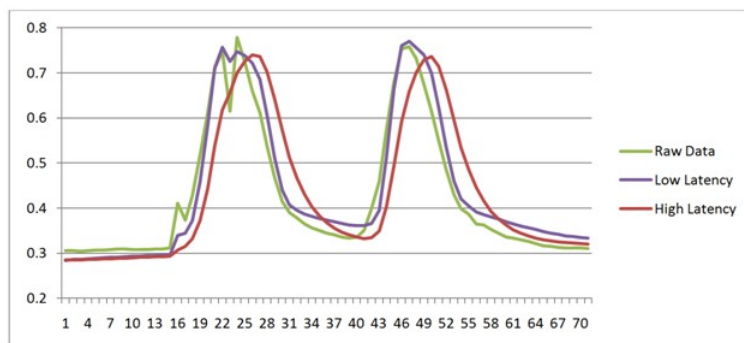


Joint Filtering

We recommend using a low-latency smoothing filter such as the Double Exponential smoothing filter or Taylor series. The **NuiTransformSmooth** function uses an implementation of the Double Exponential smoothing filter. Implementations of several filters, including the Double Exponential and Taylor series, can be found in the ATG framework; specifically look at the XDK samples [Filtering](#) and [Avatar Monkey-See-Monkey-Do](#).

We recommend doing the prediction in spherical space, and then applying bone length restrictions and finally doing smoothing in Cartesian space.

Figure 10. Plotting the Z coordinate of a hand throwing a forward punch



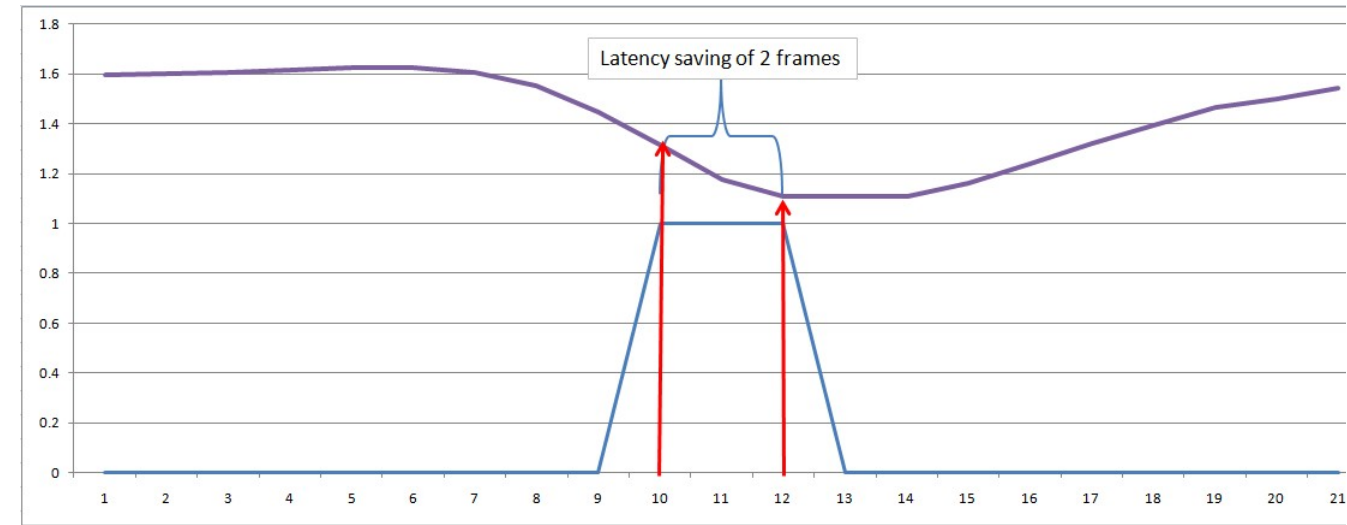
The Double Exponential smoothing filter in purple smoothes the data, introducing minimal latency and leaving a much cleaner signal to work with. A naive temporal smoothing filter also smoothes the data, but introduces much more latency.

Gesture Recognition

Reducing latency for gesture recognition is one of the most challenging problems with regard to latency because the gesture recognition system has to rely on a sliding window of joint history

information to recognize different gestures. One way to reduce latency by about two frames is to use a predictive filter. A predictive filter, such as the Double Exponential or Alpha-Beta-Gamma filter, can use the current trend of a graph to extrapolate data. By doing this, we can recognize gestures earlier than just looking at smoothed or raw data. This works particularly well for fast movements like punches.

Figure 11. Plotting Z coordinates for a hand throwing a forward punch and showing how predictive filtering can be used to recognize a gesture faster, effectively reducing latency



The first red arrow shows that by predicting the hand position, you can detect the forward punch two frames earlier than where the second arrow detects the punch.

Dealing with Synchronization

To ensure it is always using the most up-to-date ST data, a title needs to make sure that the skeleton update thread and render thread are synchronized; otherwise, the render thread might be using ST data from a previous frame. The title must also consider that frames might be dropped from ST. In order to deal with the synchronization effectively, a title event and GPU fence could be used. For an implementation of the proposed solution for synchronization, see [Reducing Latency](#), a sample in the XDK.

A title event, such as *hAvatarUpdatedEvent*, can be set whenever new ST data is acquired and processing finishes for an avatar. This update should occur on a different thread than the render thread so that the render thread can continue to render commands to the GPU without having to wait for ST data.

```
// Once update is done on skeleton update thread, signal the
// render thread that avatar has been updated
SetEvent( hAvatarUpdatedEvent );
```

Once the render thread needs to render objects depending on ST data, it should use the following to wait until the avatar data has been processed and then reset the event.

```
// Wait for avatar to be updated with latest skeleton data
// before rendering
PIXBeginNamedEvent( 0, "Wait For Avatar Update" );
WaitForSingleObject( hAvatarUpdatedEvent, 16 );
ResetEvent( hAvatarUpdatedEvent );
PIXEndNamedEvent();
```

This in itself is not enough to keep things synchronized. After the previous frame's **SynchronizeToPresentationInterval**, Direct3D allows the render thread to continue running until enough drawing commands are submitted to fill a segment of the command buffer, typically 32 KB in size. This means that the render thread can potentially run ahead of the skeleton update thread by enough time to still wait on the previous frame's *hAvatarUpdatedEvent* if a frame were dropped for some reason. Blocking on a GPU fence ensures that this will not happen. The fence can be inserted in several places during the frame, but for the Reducing Latency sample it worked best to insert it just before waiting on the *hAvatarUpdatedEvent*.

```
PIXBeginNamedEvent( 0, "Sync RenderThread to SkeletonUpdateThread" );
DWORD dwFence = m_pd3dDevice->InsertFence();
m_pd3dDevice->BlockOnFence( dwFence );
PIXEndNamedEvent();
```

For more information about using fences to synchronize the CPU and GPU to reduce latency when the title is using GPU double buffering, see the following presentations (in .zip files) from Gamefest:

- ["Advanced Xbox 360 Graphics Techniques using Command Buffers and Predicated Tiling,"](#) by Matt Lee
- ["From the Trenches – Xbox 360 Development War Stories,"](#) by Kutta Srinivasan
- ["Reducing Latency and Effectively Dealing with Noisy Data,"](#) by Claude Marais

Table 1. Summary of how to reduce latency

Use	Do not use
Event to signal when NUI data is ready	D3DCREATE_BUFFER_2_FRAMES
Low latency smoothing filter	BeginTiling/EndTiling
InsertFence to manually sync CPU and GPU	BeginZPass/EndZPass
Alternative antialiasing—for example, screen space AA	MSAA
Resource single buffering	Resource double buffering

Measuring Latency

Measuring latency is an interesting challenge. Different methods like video or sound recordings can be used to measure latency. Since we are interested only in title latency, not fixed latency, we'll discuss how to easily determine title latency.

Preparation

Ensure your game has a debug mode where it can render both the unfiltered and filtered skeleton data. This will be used to determine latency for filtering and gesture recognition. The next step is to use Xbox Studio to record different gestures, multiple times, while playing the game. It's always good to have a large set of data to reduce any possible error. We suggest that at least three different gestures be used and that 10 samples for each gesture be recorded.

Measuring

Game Engine Architecture

Measuring latency for game engine architecture is simple. In most cases, you can use the following table to establish latency due to game engine architecture.

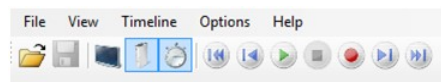
Table 2. Latency measurements for game engine architecture

Title resources	GPU single buffered	GPU double buffered, title CPU bound	GPU double buffered, title not CPU bound
Single buffered	33 ms	33 ms	66 ms
Double buffered	66 ms	66 ms	99 ms
Triple buffered	99 ms	99 ms	132 ms

Joint Filtering

Run the game and enable the debug skeleton rendering as mentioned. Use Xbox Studio to play back the recordings made, as described in [Playing Data \(Xbox Studio\)](#), using Playback Synchronized Skeleton as shown in Figure 12. This enables you to set breakpoints in the code and retrieve skeleton data from the video recording one frame at a time so that you can count the difference in frames between the filtered and unfiltered data. An easy way to measure this difference is to count the number of frames it takes from the time a gesture starts on the unfiltered debug skeleton to the time that same gesture starts on the filtered debug skeleton. Sampling data for three gestures, with 10 samples for each, results in 30 samples. Taking the mean value of the 30 samples should give a good indication of what latency occurs because of filtering.

Figure 12. Xbox Studio toolbar with the console icon and clock icon enabled to ensure Synchronized Skeleton playback mode



Gesture Recognition

Determining latency for gesture recognition can be done in the same way as it is for joint filtering. Count the number of frames between the filtered skeleton and when an action or animation starts in the actual game, based on the filtered input skeleton data.

PIX Counters and Events

It is a good idea to always know when you have data and where it is. Use **PIXSetMarker** with the NUI timestamps or NUI frame numbers so that it will show up in PIX captures.

```
PIXSetMarker(0, "%i - GetNextFrame", nuiFrame.dwFrameNumber);
PIXSetMarker(0, "%i - UpdateAvatar", nuiFrame.dwFrameNumber);
PIXSetMarker(0, "%i - SubmitForRendering", renderData.dwFrameNumber);
PIXSetMarker(0, "%i - Rendered", renderData.dwFrameNumber);
```

A number of NUI PIX counters and events are also available. You can use them to debug and investigate latency issues in your title.

Figure 13. PIX counters and events for NUI

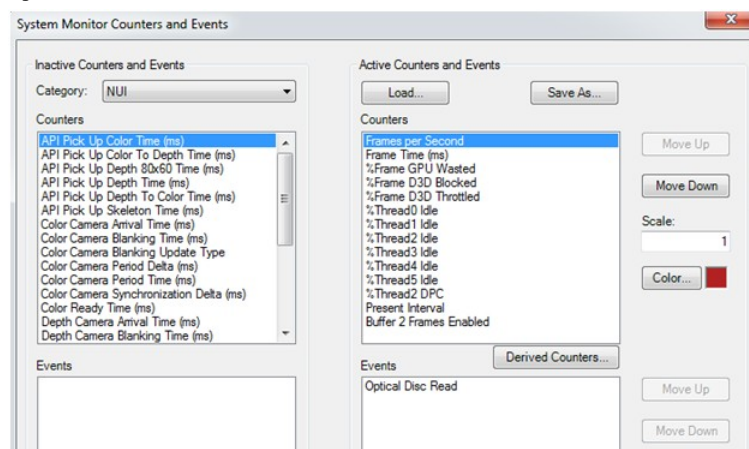


Figure 14. NUI counters you can use to investigate the data flow for ST

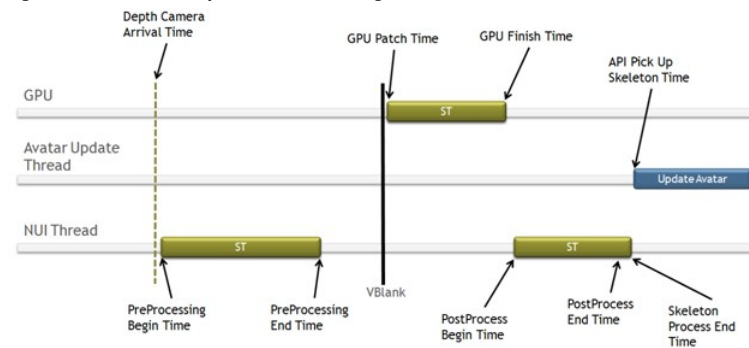


Figure 15. Example of time values for NUI counters used for ST

Counters			
Color	Counter	Scale	Last Value
	Depth Camera Arrival Time (ms)	1.000	24.55
	PreProcess Begin Time (ms)	1.000	0.01
	PreProcess End Time (ms)	1.000	3.42
	GPU Patch Time (ms)	1.000	9.18
	GPU Finish Time (ms)	1.000	12.62
	PostProcess Begin Time (ms)	1.000	12.63
	PostProcess End Time (ms)	1.000	15.69
	Skeleton Process End Time (ms)	1.000	15.72
	API Pick Up Skeleton Time (ms)	1.000	15.77

Depth Camera Arrival Time is the time from the previous synched VBlank. All other counters are relative to this time. For example, a value for PreProcess End Time of 3.42 ms means that preprocessing has finished 3.42 ms after raw depth data has arrived at the Xbox 360 console.

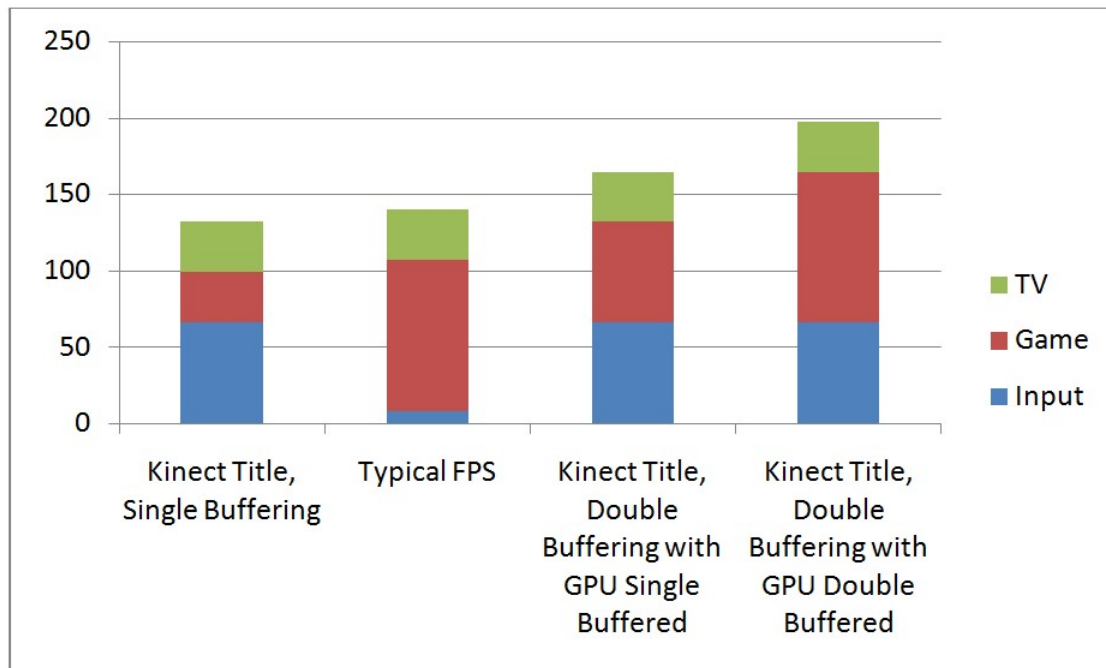
Some interesting symptoms to look for when debugging latency:

- Big time difference between PreProcess End Time and GPU Patch Time
The camera is not syncing with VBlank, or the title has an irregular frame rate causing **Present** to drift around.
- Big time difference between GPU Finish Time and Post Process Begin
The CPU is busy so the NUI thread could not wake up in a timely manner.
- Big time difference between Skeleton Process End Time and API Pick Up Skeleton Time
The title did not pick up data fast enough, so try using an event-driven system.

Comparison with First-Person Shooters

Since fast reaction time in first-person shooter (FPS) games is important, it would be an interesting exercise to compare the end-to-end latency of a typical FPS game to the latency of Kinect titles. Specifically, we'll do the comparison for Kinect titles that make use of full body avatar retargeting, since these titles can rely on a low-latency smoothing filter and do not necessarily have to make use of any gesture recognition system.

Figure 16. Comparing the end-to-end latency of a typical FPS to a Kinect title using full body avatar retargeting



Conclusion

Managing latency will be crucial to the success of many Kinect titles. For Kinect titles to have the same latency as traditional games, developers need to reduce latency in game engine architecture, and to minimize extra latency that can be introduced by joint filtering and by gesture recognition.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Simple Skeleton Normalization and Animation Comparison

By Ivan Nevraev, Software Development Engineer

Advanced Technology Group (ATG)

Many Kinect games need a gesture detection system. Exemplar-based gesture detection systems, among others, are suitable for detecting the complex poses and movements associated with fitness exercise or dancing. This white paper explains how to implement a simple animation and pose scoring system for fitness and dancing games. How to choose a suitable NUI skeleton representation is among the topics addressed. So is calculating the difference between two NUI skeletons, and performing static pose matching. Two methods used to compare animation are also presented.

This white paper contains the following sections:

- [Introduction](#)
- [Skeleton Representation](#)
- [Distance Metric](#)
- [Static Pose Comparison](#)

- [Brute Force Matching of Animation](#)
- [Dynamic Time Warping](#)
- [Putting It All Together: Building a Gesture Detector](#)
- [References](#)

Introduction

This paper, which complements the [Gesture Scoring Sample](#), presents a design for a simple animation comparison system that can be used in a dancing game or a fitness exercise game. Many Kinect developers may need to solve the problem of matching a player's skeleton with a prerecorded animation or with a static pose. These same developers might seek an answer to "How different is the player's current pose from the pose expected?"

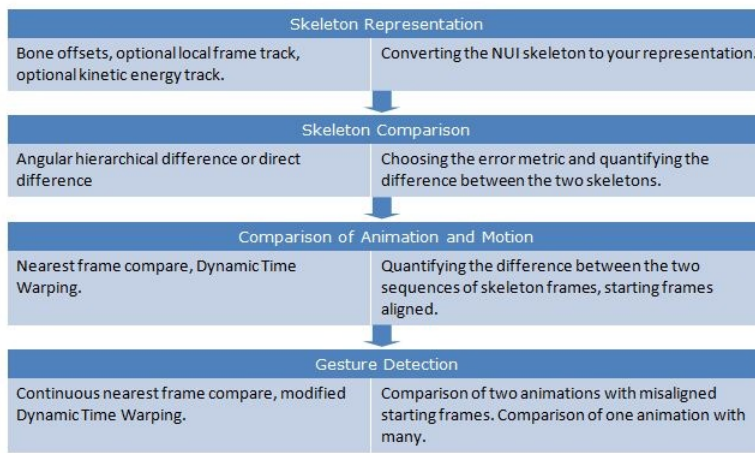
This white paper starts with selecting a skeleton representation, discusses how to normalize NUI skeleton and which extra animation tracks to generate, and then explains how to compare two normalized skeletons using either direct difference or hierarchical difference. This comparison is followed by the explanation of how to quantify the amount of difference between an animation and a real-time sequence of NUI skeleton frames by using nearest frame compare and classic dynamic time warping (DTW). Finally, there is a description about how to add handling of the player's latency to the nearest frame compare method and improve DTW.

Gesture Detection System

Sometimes a title may need to match an animation with a known starting time, as in a dancing game where there is a music beat. At other times, a title may simply need to look at a history of recorded NUI frames, and to ask, "Did that sequence of frames look like this animation here?" Any system that can answer that question reasonably quickly can be used for gesture detection.

Figure 1 shows the components of a gesture detection system. Any given layer uses all layers in the diagram. Such a view of the detector's architecture is somewhat arbitrary, but reflects the organization of the rest of the document.

Figure 1. Components of the gesture detector



Content in the left column identifies what the component does, and in the right column, why it does it.

Naming Conventions

Three terms used consistently through this document are defined next:

- **Gesture**
A prerecorded full-body or partial-body animation that is expected to be compared with a given stream of NUI frames without knowing the starting time of that animation.
- **Scoring**
The process of quantifying the skeleton or the result of the animation comparison.
- **Matching**
The process of detecting which skeleton or animation looks most like a specific skeleton or animation.

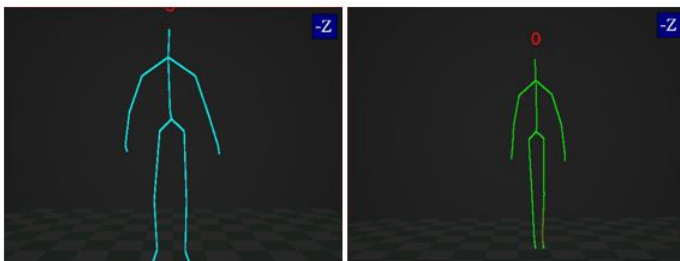
Skeleton Representation

NUI APIs return a skeleton as a set of joint positions in camera space. Faced with the problem of comparing two skeletons, the difficulty in using plain skeleton data for comparison is quickly discovered. For example, computing Euclidean distance between the corresponding joints of two skeletons is easy. The difficulty arises in interpreting that distance as a sensible comparison metric even when ensuring the skeletons are rooted at the same position in space. The difficulty in making computational comparisons using plain skeleton data increases if the two skeletons are for players with different body frames. Knowing the positions of the joints for these two players becomes even less useful because the position of each joint will be different.

Every person has a different skeletal frame. Even if you record skeleton data for the same player a second time, the two NUI skeletons will not be the same. There are many reasons for the disparity, including some—such as differences in clothing and environment, and noise in the skeletal tracking (ST) pipeline—beyond our control. As a result, comparing two skeletons by relying on a direct comparison of joint positions is difficult. A better skeleton representation is needed.

Let's assume your interest is comparing two skeletons and not their position in space. Ignoring the position of the skeleton's center of mass in camera space, the main differences in the NUI skeletons are in bone lengths, body proportions, and postures. NUI bone lengths—as expected—are different in people of varying heights and builds. ST may introduce variations in bone lengths, even for the same person moving in different ways, with joints occluded or outside the field of view of the camera, et cetera. Differences in body proportions and postures affect angles that the bones form with the joints. A person with a larger skeletal frame, for example, will look different from a person with a smaller frame, even if the bone lengths are purposely set to be the same between the two skeletons.

Figure 2. Two players of identical height, but different body frames



To be able to compare people of different sizes, consider ignoring the bone length. It's possible to do without drastically affecting the gesture scoring result. This can be achieved by storing normalized offsets (directions) from the parent joints to the children joints. Effectively, the bone becomes a direction, thus removing one degree of freedom on the skeleton enables you to compare people of different sizes, but with similar frames. Following is the code snippet showing how to calculate the bone offset from two joint positions.

```

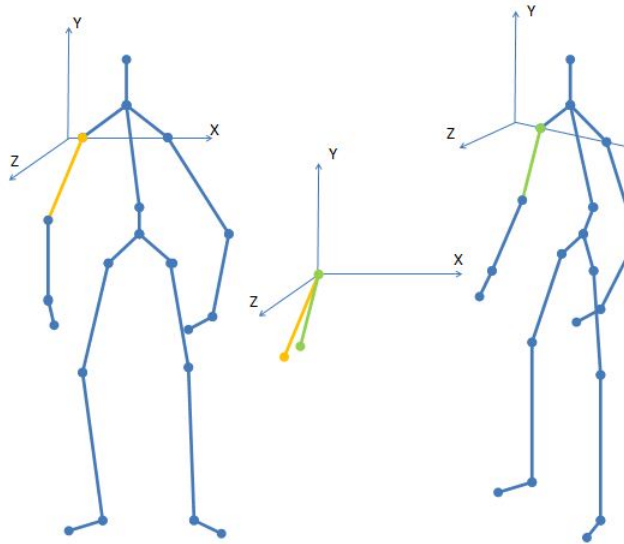
vBoneOffset = VectorNormalize( m_nuiSkeletonData.SkeletonPositions[ uThisIndex ] -
                               m_nuiSkeletonData.SkeletonPositions[ uParentIndex ] );

```

Another important point is the significance of the local coordinate frame of the player. Generally, the player is expected to be reasonably aligned with the near plane in camera space, but there always will be some rotation of the local player's frame. To illustrate why this might be a problem, look at Figure 3.

Imagine a scenario in which there are two identical skeletons, with one skeleton slightly rotated in the camera space around the Y axis. In Figure 3, the skeleton on the left is a prerecorded animation; an attempt is being made to match it to the skeleton on the right. Despite the two images being the same skeleton, the direct comparison in camera space yields a nonzero distance.

Figure 3. Local frame vs. world frame comparison



The skeletons are identical. On the left is the recorded version, on the right is the player. The yellow and green bones will not coincide in the camera space. Depending on the game, this may or may not be desirable.

The discrepancy happens in the camera space because although the local transforms of the joints are the same, the local frame to camera frame transform is different. To compare skeletons, it is necessary to choose either the local coordinate frame or the camera frame. Depending on the application, either may be relevant.

If the intent is to detect a gesture such as a hand wave in the local coordinate frame of the player, the player's body orientation could be ignored. On the other hand, if the goal is for the player to strike a pose that exactly matches a specific skeleton position in camera space, then body orientation must be taken into account. For this reason, the comparison needs to be performed in the camera frame.

If both local and camera frame comparison are needed, then another step could be finding the orientation of the player in the camera frame, creating a local to world player's rotation matrix, and transforming the bone directions into the local frame of the player. The local frame could be added as extra entries in the skeleton representation to be compared separately if needed. To define this coordinate frame, the X axis could be set to the vector between the shoulders, and Y axis could be set to the vector formed by the difference of neck joint and ankles, although it's just one way of doing it. Note that some applications may be able to work by using offsets directly in camera space without taking the player's world transform into account.

Also note that both local and camera frame comparisons ignore the camera space position of the skeleton and bone lengths.

To better differentiate movements, kinetic energy per joint can be stored. This can be added as extra data per joint, and could be helpful when comparing animations. Consider a case in which you are trying to decide if a player is performing an animation like a small hand wave. Inevitably, thresholds are added to the skeleton comparison. If the player stands with a hand matching the middle point of the wave, for example, the whole animation may match because the leftmost and rightmost positions of the hand are not too far away from the central position of the hand. Remember, it is a small wave. Storing kinetic energy enables the amount of movement of each joint per frame to be compared, and makes it possible to differentiate a moving hand from a stationary hand that—by coincidence—matches some part of the animation.

Distance Metric

Now that a skeleton representation exists, the next step is to decide how to compare two skeletons. One way to compare the skeletons is to calculate a hierarchical transform between two skeletons represented as a set of normalized joint offsets. Following is the pseudocode showing how to calculate the distance per joint.

```

FrameBoneDir = FrameBoneOffsets[ i ] * MatrixInverse( ForwardXform[ Parent ] )
RotationAxis = Cross( ReferenceBoneOffsets[ i ], FrameBoneDir )
RotationAngle = Acos( Dot( ReferenceBoneOffsets[ i ], FrameBoneDir ) )
Forwards[ i ] = RotationAxisAngle( RotationAxis, RotationAngle ) *
                ForwardXform[ Parent ]
Distance[ i ] = RotationAngle / PI

```

It works by rotating the hierarchy being matched at every joint so that the parent bones coincide. This provides a degree of rotation from one skeleton to another in the local coordinate frame of each joint. The aggregate value of *Distance_i* gives an amount of variation from one skeleton to another.

If gestures such as hand waves are expected to happen most often in a plane parallel to the sensor, there is the option of flattening the skeleton in Z before calculating the hierarchical error. Exercising this option has the effect of amplifying the X and Y components of the motion, and attenuating the depth component as the Z component of the skeleton's joints' position is partially ignored. The flattening effect is achieved by multiplying Z coordinate of each joint by a small number such as 0.25. This method can be used with some gestures to make the comparison more robust. It is not always good for generic gestures, however, and the method's measure of usefulness is determined by the application.

Depending on the application, you may want to track the player's local frame here in order to rotate the skeleton's bone offsets into their local space before calculating a distance metric. If the same actions are performed for the input skeleton, the end results are a directly comparable player's coordinate frame, and a number of normalized bone offsets in the same coordinate frame. These results make it possible to omit the hierarchical angular difference in favor of a direct comparison of bone directions. It is worth experimenting with both representations to find out which one best suits your game.

Regardless of the method chosen, there is a *Distance_i* value per joint that represents the amount of difference from one skeleton to another at that joint. Combining these difference values establishes the amount of difference between two skeletons. It is possible to define error metric as the difference values directly, but for a more tangible metric, Peak Signal to Noise Ratio (PSNR) can be used. Note that PSNR is just one way to define the error metric.

In engineering, PSNR is used when a ratio between the maximum power of the signal and the corrupting noise is required. PSNR also is commonly used to define the quality of lossy image compression codecs, in which case PSNR defines the ratio between the original image and the decompressed image. PSNR has a logarithmic scale, and is measured in decibels. For the foregoing scenario, think of the reference skeleton as the original signal, and the NUI skeleton as a signal with noise.

By using per joint differences from the reference skeleton (*Distance_i*), the Mean Square Error (MSE) is computed first, and then PSNR by using this equation:

$$MSE = \frac{1}{N} \sum Distance_i^2$$

$$PSNR = 10 * \log_{10} \left(\frac{MAX^2}{MSE} \right)$$

MAX value is 1, defined as 180 degrees of angular difference for the hierarchical metric method. In other words, a maximum amount of error occurs when the two bones face in the opposite direction. Note that higher PSNR means lower error. Two identical skeletons will yield a PSNR of infinity.

Static Pose Comparison

Now that there is the capability to compare two skeletons and quantify their differences, skeleton pose scoring can be implemented. Begin with a static pose comparison—a scoring between a single-recorded reference skeleton versus a player's NUI skeleton.

Use the distance metric to obtain the amount of difference between two skeletons. The metric is independent not only upon the player's skeleton dimensions, but also the player's frame. In order to compare two skeletons, a scoring system needs to be developed. That system needs to have a tolerance for slight differences in the rotational component of the bones. There is a need to be able to ignore small variations in bone directions, but not large variations that mean the bone is moving.

The primary interest here is in comparing some joints, not the whole body. Reducing the comparison scope by calculating differences only on specific joints makes the comparison less susceptible to noise. One way to reduce the scope is to implement joints weighting. For performance reasons, computations for the bones that are not part of the gesture can be avoided.

Each skeleton comparison yields a PSNR metric. To calculate the score that makes sense to the game, you must understand what PSNRs occur when the players try to match the pose. Finding the limits of values of PSNR is best done by experimenting with a starting value such as 25 decibels (dB) as a good match. In the [Gesture Scoring Sample](#), a good match usually returns PSNRs of 30 dB and higher. You must recalibrate the scoring system if Z flattening or joints weighting is used to set your PSNR values, and then the amount of flattening or the joints' weights is changed. Here is an example of the scoring function used in the Gesture Scoring Sample:

$$\text{Score}(\text{PSNR}) = \text{clamp}\left(\frac{\text{CurrentPSNR} - \text{BadPSNR}}{\text{GoodPSNR} - \text{BadPSNR}}, 0, 1\right)$$

The equation returns 0 score if PSNR is lower than the minimum, 1 if PSNR is larger than the maximum, and a linearly interpolated value in between. In effect, this statement means a range of poses close enough to the static pose used for a match is accepted. Good and bad PSNR values vary per gesture type, and are best set per animation.

Brute Force Matching of Animation

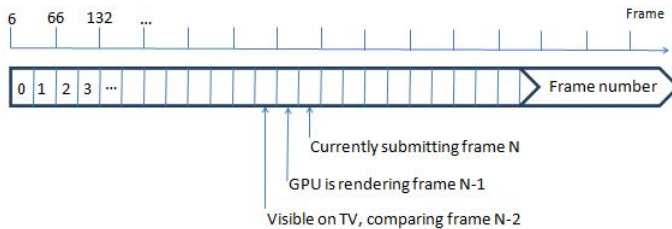
The static pose comparison can be expanded by trying to match a whole animation sequence with just one single-frame scoring function.

Consider a simple case of matching the NUI skeleton to an animation that starts at a given time; how to score animations continuously is discussed later in this white paper. Imagine a 3-2-1 countdown with an avatar on the screen that shows which animation the player is expected to perform and to match as precisely as possible. You will want to know how well the player performed the exercise.

On the code side, there is a previously recorded reference animation and it is just a sequence of skeleton frames at 30 frames per second (fps). Each frame is a normalized snapshot of an NUI skeleton as recorded at the frame's time. There also is a continuous sequence of NUI frames—also converted to the normalized representation—coming from the sensor at 30 fps.

Because there is an assumption that the player is attempting to repeat the reference animation, the start of both streams is synchronized in the experiment. The first approach that comes to mind is to compare the current NUI frame to the frame animation at corresponding time. This approach is simplistic, but it helps to illustrate a few important concepts. Figure 4 shows how it works.

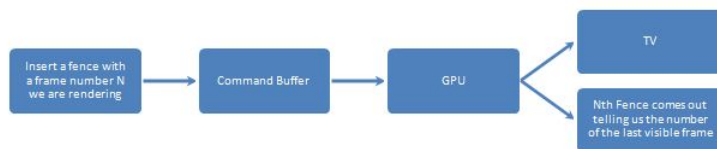
Figure 4. One frame comparison



Note that system latency must be taken into account. For additional information, see [Reducing Latency in Kinect Titles](#), a white paper by Claude Marais.

Due to the time required for a rendered animation frame to appear on the TV screen, you need to compensate for the rendering latency here, or you will be trying to compare the player's position with the frame that has not been displayed yet. The player tries to repeat the rendered frame, and it takes time for the player's skeleton to be captured by the Kinect sensor and processed by the game. Compensate for the rendering latency by inserting a fence into the GPU stream, and associating the fence with the current animation frame number. The Gesture Scoring Sample compensates for latency by calling **InsertFence** after a call to **Present**, noting the animation frame for the returned fence ID, and later checking which fence is crossed by the GPU by calling **IsFencePending**. Because the GPU crosses the fence only after **Present** is called, if the frame is displayed on the TV, then the fence is no longer pending.

Figure 5. A fence associated with a frame number is put into the command buffer



Immediately after the GPU displays the frame, it crosses the fence, and the game checks the number of the animation frame currently displayed.

With each frame, a visible animation frame is scored as indicated by the last crossed fence against a current NUI frame. People struggle to match a recording of themselves frame for frame in terms of timing. This struggle occurs because when looking at the skeleton that needs to be matched, a player tends to speed up or slow down a little bit all the time. Most people cannot reliably or consistently repeat the same gesture. Inevitably there is a variance in speed throughout the gesture, so comparing just one frame at a time is not going to work.

One way to cope with varying speed is to search around the currently expected frame to allow the player to catch up if they are too slow or too fast. Add some scoring falloff around the expected crest to ensure there is a penalty for motions that are too slow or too fast. That approach works better, and is quite capable of scoring an animation.

Figure 6. Adjusting a frame



Instead of scoring a single frame, the best matching one in a range around the expected frame is selected.

The approach that allows for variable player's response time is able to score a single animation if the start of the animation is synchronized with the start of player's motion. By starting a new detector with each frame, animations can be scored continuously, regardless of when the player starts the motion. The capability to score animations continuously allows a gesture to be detected just by checking if any detector reached its final frame without dropping its score to zero too many times.

When moderately optimized, this method of detection can continuously detect 16 full-body animations at a CPU cost of 2 milliseconds (ms). The method, however, has a major drawback: detection itself cannot cope very well with differences in speed throughout the motion.

Dynamic Time Warping

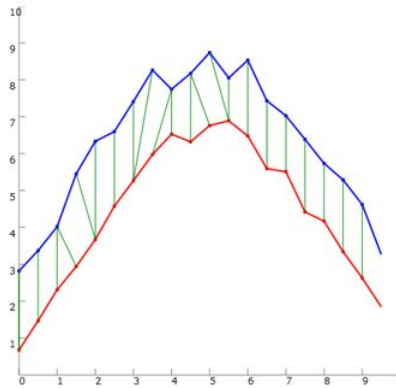
The brute force approach described in the previous section does not adapt well to the gesture being done at different speeds throughout the sequence. If the crest in the detection wave of detectors is lost, you have to start detecting the gesture again. A method that inherently can cope with different gesture speeds would be ideal.

Luckily, a similar problem was encountered 30 years ago in the field of speech recognition, so there is a way to handle varying gesture speeds. The resolution is method is called dynamic time warping or DTW, and it is used to optimally align two time series. A time series is a sequence of data points. A data point can be a single number, or a frame of animation. DTW can align two time series as long as a cost function is defined for all pairs of data points. For two numbers, a cost function can be an absolute value of the arithmetic difference. For two 3D vectors, a cost function can be Euclidean distance. For our animation frames, it is the PSNR-based distance metric defined in the [Static Pose Comparison](#) section. Figure 7 shows an example of DTW at work.

Note

For more information about dynamic time warping, refer to [Speaker recognition using hidden Markov models, dynamic time warping and vector quantization](#) by K. Yu, J. Mason, and J. Oglesby, and to a doctorate dissertation by Zhihua Wang, [Time Series Matching: a Multi-filter Approach](#).

Figure 7. Two DTW-aligned time series with lines connecting the feature points



Green lines show the alignment of two sequences.

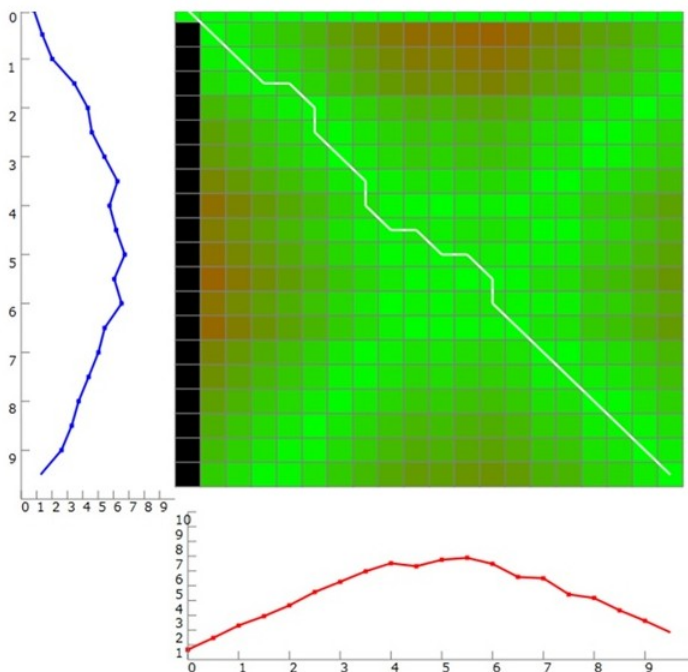
An understanding of how DTW works is useful before describing how DTW is solved. For example, assume a prerecorded sequence of floating-point values must be aligned with a different sequence of floating-point values being recorded continuously from some device. The cost function is just an absolute value of the arithmetic difference between two floating-point values and the scanning window, or the duration of our continuous recording is equal to the reference sequence's duration. New floating-point samples are added on the right, and old samples are pushed to the left. The intent is to find the best alignment—one that minimizes cost—between the prerecorded sequence and the current sequence.

The continuously scanned sequence is plotted along the X axis, and the prerecorded reference sequence is plotted along the Y axis. The matrix inside is called a cost matrix, and each cell in that matrix stores a distance metric from one sequence's frame to another. For example, values in each column are distances between a single current frame and each reference frame, and the values in each row are distances between a single reference frame and each current frame.

The cost matrix is calculated by computing pairwise costs for each frame of the prerecorded sequence, and each frame of the current sequence.

When DTW is finished, there is a *warping path* that requires aligning the two sequences. Each step in the warping path stores two indices of the aligned points in the sequences. The white line in Figure 8 represents the warping path. The minimum cost is a cumulative distance along the warping path.

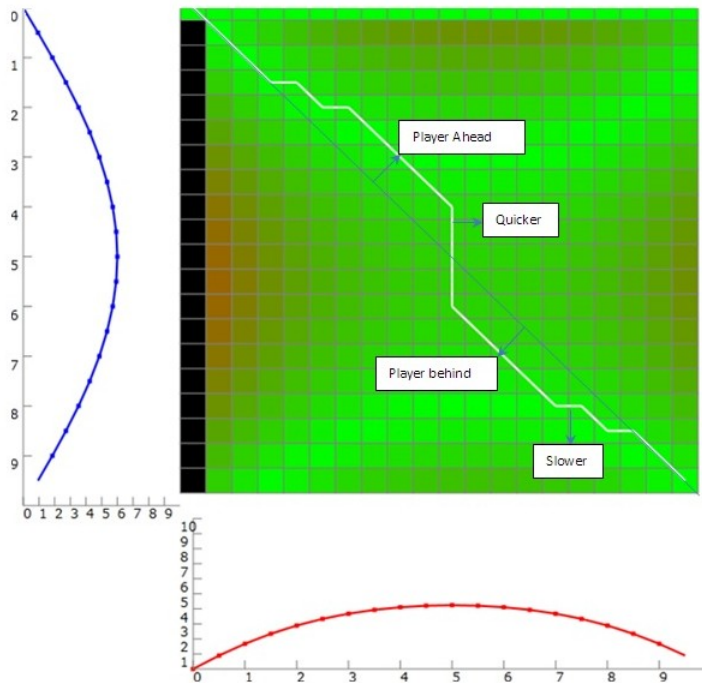
Figure 8. Two time series with the cost matrix and the warping path



The time series are the same as shown in Figure 7.

Information about the speed of the gesture throughout the sequence can be extracted by determining if the warping path ends up above or below the diagonal. The speed of the sequence relative to the reference can be determined by looking at the inclination of the warping path. If the recorded sequence is running at the same speed as a prerecorded sequence, then the warping path lies on the diagonal. If the warping path goes below the diagonal, then the reference sequence was faster for that stretch of time. If the warping path goes above the diagonal, it means that the reference sequence was slower. In Figure 9, one sequence is slower than the other near the beginning, becomes faster toward the end, and then finishes at a matching speed.

Figure 9. DTW warping path of two sequences



The two sequences show different speeds plotted on the heatmap of the cost matrix. The path constraint is shown in red.

From the graph in Figure 9, it is clear a restriction can be imposed on where the warping path lies. For example, if one sequence is two times slower than the other, there is no interest in it; but there is an interest in clamping it to the half-speed sequence. In DTW terms, this is known as a global constraint, and a global constraint of that shape is called a Sakoe-Chiba band (refer to Zhihua Wang's dissertation, *Time Series Matching: a Multi-filter Approach*). Enforce the constraint by looking only for a warping path inside the Sakoe-Chiba band. This allows you to save work by not evaluating cost for the cells that lie outside the band.

The actual scoring can be done by tracing the found path from the bottom to the top. A path will always exist through the DTW cost matrix, so before concluding there is an acceptable match between the two sequences, look at its cost. If a path with a satisfactory cost exists, then there is a match between the two sequences. In the absence of a match, make a decision to accept or reject the path.

Until this point, square DTW matrices have been examined. The meaning of a square DTW matrix is that the lengths of the two time series you are trying to align are identical. To match slow animations (longer sequence), extend the matrix so it is wider than it is taller; the length of the reference time series, in other words, stays the same, but the currently examined time series—the player's animation—is longer. In terms of animation comparison, the reference animation goes along the Y axis of the matrix, and the currently compared animation goes along the X axis of the matrix. Allow the X axis to have more frames than the reference animation in order to not lose track of the slowest expected animation.

Consider skeleton matching with DTW now. There are two sequences of skeleton frames: one is the reference, the other one is from NUI. A distance function also was defined earlier. Now you have the data needed to use DTW to find how well the two sequences match.

Because brute force path searching through the cost matrix is computationally expensive, dynamic programming is used instead. Each cell in the matrix is tagged with the minimum cost to reach that cell from the upper-left corner. First expand the cost matrix by adding one row at the top, and one column on the left. The upper-left corner should have a local cost of 0. Start iterating from the (1,1) cell of the matrix, filling it strictly from left to right and top to bottom. Calculate a local cost at each cell by using the cost function, and then look back at the three cells in the global cost matrix already filled—one to the left, one above, and one diagonally to the top left. Select the cell with a minimum cost out of the three, and then store a link to that cell in the current cell of the path matrix. In the global cost matrix of the current cell, store the sum of the current cell's cost and the minimum of the three cells cost. After scanning the cost matrix, use the path matrix to track the path from the bottom right corner, which gives us the optimal warping path through the cost matrix. The warping path is all that is needed to establish the alignment between the two sequences. Although warping path tracing is not shown, following is pseudocode of the main algorithm (refer to the Gesture Scoring sample for more details).

```
for( i=1; i <= NumReferenceAnimFrames; ++i )
{
    for( j=1; j <= NumHistoryFrames; ++j )
    {
        Cost = CostFunction( i, j );
        Diag = MatCost[ j - 1 ][ i - 1 ];
        Left = MatCost[ j - 1 ][ i ];
        Up = MatCost[ j ][ i - 1 ];
        // choose the minimum of 3
        Minimum = fDiag;
        StepDirection = STEP_DIAG;
        if( Left < Minimum )
        {
            Minimum = Left;
            StepDirection = STEP_HORZ;
        }
        if( Up < Minimum )
        {
            Minimum = Up;
            StepDirection = STEP_VERT;
        }
        // store cost for this cell and also store the path, note the path matrix
        // is smaller by 1 in both dimensions
        MatCostLocal[ j ][ i ] = Cost;
        MatCost[ j ][ i ] = Cost + Minimum;
        MatPath[ j - 1 ][ i - 1 ] = StepDirection;
    }
}
```

Note that the Gesture Scoring Sample fills the entire top row with zeros, instead of just the top corner as is done in the DTW implementation. Otherwise, it is possible to get warping paths running horizontally along the top edge for a distance that can complicate the scoring. By doing this, DTW is being allowed to correctly recognize sequences that start at the current frame, and end at some arbitrary frame in the past. If the top row of the matrix is filled with zeros, the DTW can terminate the warping path at an arbitrary column instead of forcing it to run the warping path into the top-left corner.

It is possible to search the bottom row for a minimum cost value from right to left and start tracking from there. This subsequence may start in the past because its starting column may not be the rightmost column, so it is useful if you need to find the shortest or cheapest subsequence that realized in the past.

Clearly, calculating the entire cost matrix for each frame is prohibitively expensive because it requires you to calculate several thousand cost values. Because a continuous time sequence is being scored, you can reuse most of the values in the cost matrix for each frame, and recalculate only the rightmost column of the cost matrix. Recall that each cell with coordinates (N, M) of the cost matrix stores a distance metric between the Nth frame of the NUI frames sequence and the Mth frame of the reference animation. For each frame, a new NUI frame to the sequence is added, which shifts the columns in the cost matrix left by one. Moreover, the shifted columns never change, so they can be cached. Caching means only a handful of distance metrics are cached each frame, which provides a massive CPU saving.

DTW is a step up from a brute force detector because it allows you to track a gesture with varying speed throughout the sequence. Dynamic time warping also allows building a reasonably sophisticated animation scoring system. An unoptimized DTW detector will cost about 5 ms for 16 full-body animations being continuously scored.

Putting It All Together: Building a Gesture Detector

Now that you have a description as to how to score and detect a single animation, the next logical step is to build a robust gesture scoring system. Regardless of the leeway built into the system, having both correct detection and large enough error thresholds to detect all the wave gestures is not always possible. If thresholds are relaxed too much, you soon will start hitting false positives.

Storing kinetic energy per joint animation track could be useful, but having it just postpones the inevitable loss of detection precision. Instead of relying on a single recording of the animation, record several instances of the same animation made by different people. Everyone should be trying to repeat the same gesture in as close fashion to the reference animation as possible. This way, we can bundle those instances into a single definition of "animation," and run a DTW detector per instance, calling the whole animation finished when one of the instances is completed.

Of course, there are other ways to build a gesture detection system, not only based on exemplars. For exemplar-based systems, however, advanced users can expand upon the methods of this paper by using techniques such as hidden Markov models with continuous state variables (see [A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition](#) by Lawrence R. Rabiner). Coverage of Machine Learning algorithms is outside the scope of this white paper, but this is a great starting point for more advanced techniques than described here.

References

Dasgupta, Sanjoy, Christos H. Papadimitriou, and Umesh Vazirani. 2006. Algorithms. New York: The McGraw-Hill Companies. <http://www.cs.berkeley.edu/~vazirani/algorithms/chap6.pdf> (Penultimate)

Lawrence R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition." Proceedings of the IEEE 77 (2): 257–286. doi: 10.1109/5.18626. <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=18626>

Sakoe, Hiroaki, and Seibi Chiba. "Dynamic programming algorithm optimization for spoken word recognition." 1978. IEEE Transactions on Acoustics, Speech and Signal Processing 26 (1):43–49. doi: 10.1109/TASSP.1978.1163055. <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=1163055>

Zhihua Wang. "Time Series Matching: a Multi-filter Approach." PhD diss., New York University, 2006. http://cs.nyu.edu/web/Research/Theses/wang_zhihua.pdf

Yu, K., J. Mason, and J. Oglesby. "Speaker recognition using hidden Markov models, dynamic time warping and vector quantization." 1995. *IEE Proceedings – Vision, Image and Signal Processing* 8 (5):313–318. http://ieeexplore.ieee.org/xpls/abs_all.jsp?arnumber=487791&tag=1

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>
Version: 21256.3

Using Kinect for Hand-State Detection



by Ivan Navraev, Senior Software Development Engineer
Advanced Technology Group (ATG)

Published: January 6, 2012

This white paper and the accompanying [Hand Open Closed](#), a sample included in the December 2011 Xbox 360 Development Kit (XDK) address Kinect Sensor detection of hand states. A combination of machine learning and Kinect functionality focus on whether one or both hands are open or closed.

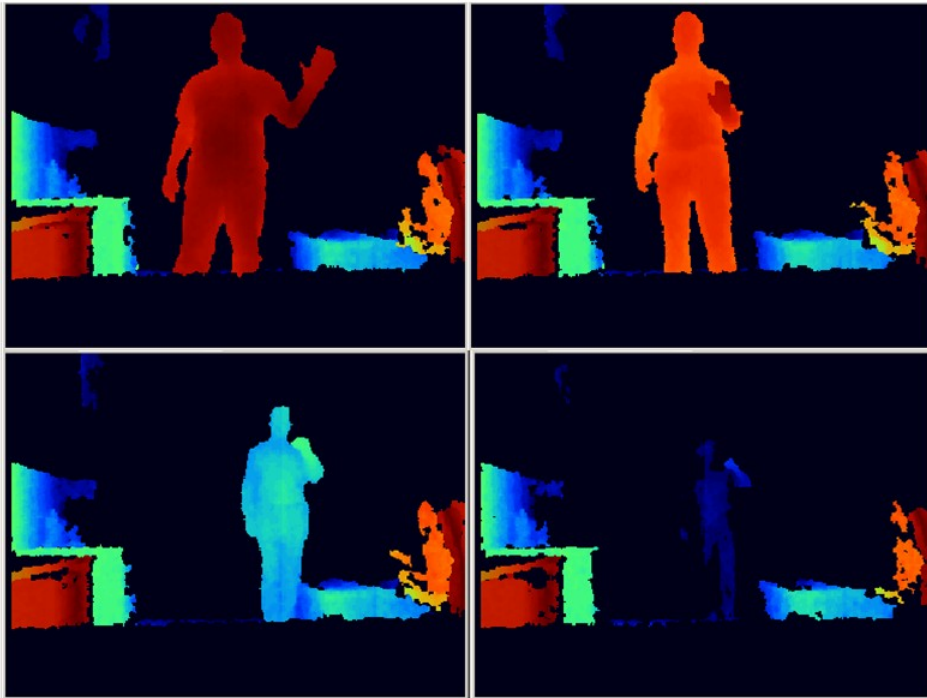
The final result of the approach described in this white paper attains an average detection rate of 93 percent accuracy for perceiving whether a hand is open or closed. This high level of detection can be used in a game to control the UI or perhaps as part of gameplay. Other applications of this detection technology are possible, such as detecting the orientation of the thumb, detecting whether a hand is reaching, or whether a finger is pointing at a UI control. The HandOpenClosed sample includes the source code for the tool chain and runtime.

- [Acquiring Hand Data](#)
- [Assessing Individual Weak Solutions](#)
- [Classifying with AdaBoost](#)
- [Implementing Distance-Based Classifiers](#)
- [Surveying Other Classifiers](#)
- [Acquiring Machine Learning Data](#)
- [Implementing Ensemble Trainer](#)
- [Improving Run-time Performance](#)
- [Conclusion](#)
- [Appendix A. A rigorous explanation of AdaBoost](#)
- [Appendix B. Optimizing with RBoost](#)
- [For Further Reading](#)

Acquiring Hand Data

As an experiment, observe Figure 1, which shows the depth map alone. Assuming skeletal tracking (ST) data is available, is it possible to figure out whether a person is holding a hand open or closed? At first glance, it looks like there is enough information in those two data sources to determine whether the hand is open or closed.

Figure 1. Examples of Natural User Interface (NUI) frames on which the detection algorithm is expected to work

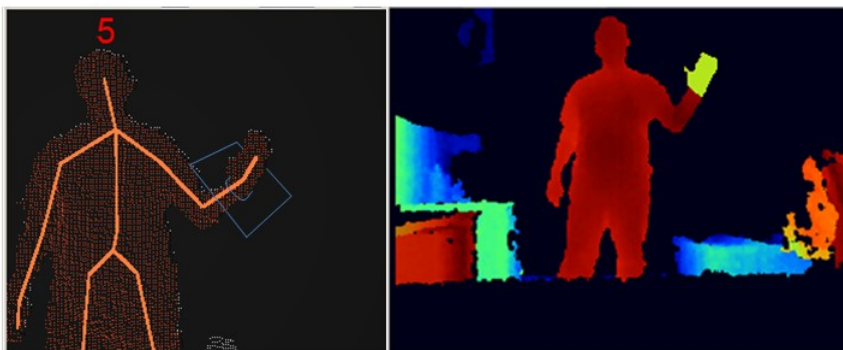


Clockwise from top left, examples of depth buffers containing open and closed hands at different distances, color-coded dark red through to dark blue as the distance increases.

Looking at the screenshots, it is possible to describe some of the features that are immediately noticeable. For example, does the hand look round or are the fingers visible? Looking at the depth data, does the hand look flat or spherical? It might be possible to write detection algorithms based on these observations, although it is not clear yet how the empirical features are best quantified.

It's crucial to first identify the data required for detection. A good starting point is to isolate just the hand pixels. This simplifies the problem because only a small number of pixels will end up being used for the detection; in fact, the rest of the depth data is likely irrelevant as it is difficult to imagine how the depth of a head pixel could be influenced by the status of the hand. The simplest way of isolating the hand is to perform a 3D flood fill on depth pixels starting from the hand joint position. Choosing the joint position as a starting point is fast and easy, but any other way of finding the hand can be used. The goal of the flood fill is to mark the depth pixels belonging to the hand, so it should be bounded, first, by per-pixel differences in depth such that it doesn't flood onto the background and, second, by a clipping plane placed at the wrist to stop filling up the arm. This is a simple and fast approach that doesn't require expensive hand position refinements. Figure 2 shows the result.

Figure 2. Identifying and isolating hand data



The image on the left shows the voxels and the plane of the hand; the image on the right shows a flood-filled hand.

After isolating the hand pixels, construct voxels for each valid hand pixel from the 2D position of the pixel and its depth value. You obtain these depth values from the Kinect depth frame.

In addition to image-based depth data, there is also skeletal-tracking data and color data. Skeletal-tracking data provides the original hand position and the wrist position, but it can also be used for hand-status detection. The ratio of lengths between the elbow-to-wrist and wrist-to-hand joint vectors varies strongly according to whether the hand is open or closed. Currently the color data isn't used, although it can be expected to perform well; implementing it is left for future work.

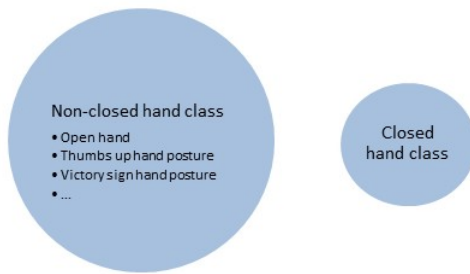
Assessing Individual Weak Solutions

Consider what can be done with the hand data. It is still not obvious how to formulate a robust detection algorithm from the simple features of the hand data such as "is it flat?" or "does it look symmetrical?" A good starting point is to try expressing each feature programmatically. In this context, *feature* means a quantity describing an instance of the data.

Individually, each feature's code is relatively easy to write. Let's assume for the moment there are such functions already. These functions take the input data and return a floating-point value based on analyzing a data feature. When there is some code that returns a value given the input hand data, the next step would be to take the output values for all recorded hand frames and analyze them to find what's different between open and closed hands. This creates a mapping from the floating-point output value to closed or open classes. It is easy enough to plot that data and look for patterns. The next step in this heuristic approach is to perhaps apply a threshold to the run-time data, to detect whether the hand is open or closed just by using a single comparison.

A *classifier* is the function that takes the input hand data and outputs the status of the hand as closed or open. The name suggests that the function provides mapping from the hand data into two discrete classes – here, closed or open. To be more precise, the two classes are closed and not closed as open hand strictly means that it's not closed, but non-closed hand doesn't always mean it's open, as the next diagram illustrates.

Figure 3. Open (non-closed) and closed classes relationship



This diagram illustrates the relationship of the open hand class as a part of a larger non-closed hand class.

The next example shows what happens if classifiers built around one or two features are combined using this method.

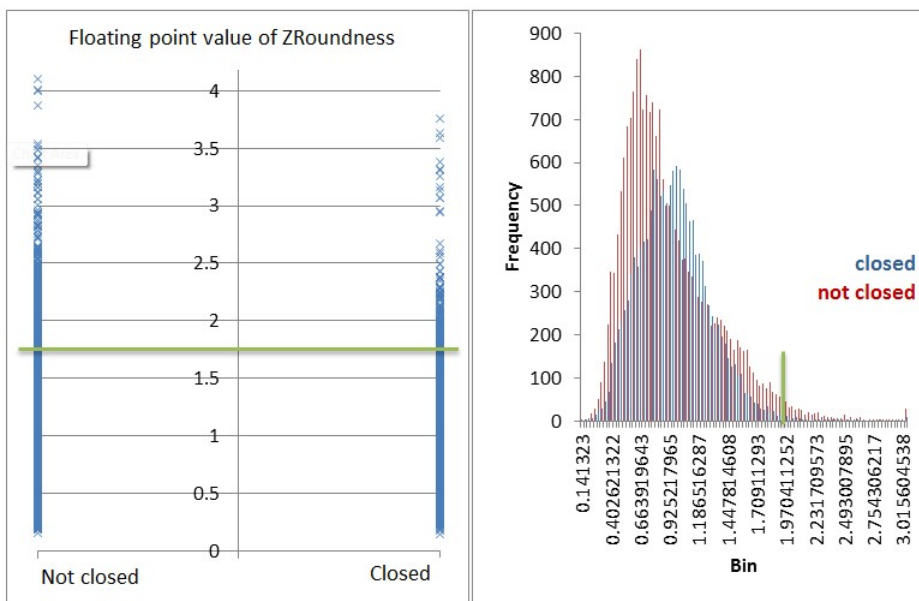
We'll start with the first question: Is the hand flat? The flatness of the hand is defined as a ratio of the area in the plane of the sensor to the depth span of the voxels. The flatness detector is called ZRoundness in the XDK HandOpenClosed sample.

$$\begin{aligned} \text{centerToVoxel}_i &= \text{centerPosition} - \text{voxelPosition}_i \\ \text{Size}_{xy} &= \max \left(\sqrt{\text{dot}(\text{centerToVoxel}_i, xy, \text{centerToVoxel}_i, xy)} \right) \\ \text{Size}_z &= \max(|\text{centerToVoxel}_i, z|) \\ \text{ZRoundness} &= \frac{\text{Size}_z}{\text{Size}_{xy}} \end{aligned}$$

There could be many similar formulations. For instance, the size in depth can be taken as a radial distance to the voxel from the centroid (geometric center) instead of the distance to the plane where the centroid lies. The HandOpenClosed sample provides an environment for trying out classifiers.

To understand how the above detector behaves, a few Xbox Studio clips of an open hand and a closed hand can be recorded, each data frame passed to the **ZRoundness** function to obtain a floating-point roundness value, and those values plotted on the graph on the Y axis against the X axis, which has non-closed hand samples at X=-1 and closed hand samples at X=1. The HandOpenClosed sample can output such data for analysis directly into a .CSV file. Most math packages accept .CSV as input.

Figure 4. ZRoundness shown in a graph and a histogram



The ZRoundness feature of 38,000 samples is shown here as a graph and as a histogram. The green line shows the point at which the closed class can be separated from the not closed class.

The closed hand column (X=1) has fewer points above a certain value of **ZRoundness**, and this happens earlier for that column than for the non-closed hand column (X=-1). Intuitively, this makes sense because closed hands look more spherical and the ratio of extents in the sensor plane (XY) to extents in depth (Z) gets closer to 1 as it gets more spherical. Even visually, the two classes of data can be separated slightly by a line approximately at **ZRoundness**=2. The fact that there is some overlap between the two classes above **ZRoundness**=2 in the data doesn't make much difference for this example because 99 percent of the data in the closed hand class sits below **ZRoundness**=2. This cutoff value is called a threshold. It is possible to write a detector where at run time, when a value of **ZRoundness** is greater than 2, the hand is considered not closed, and otherwise it might be closed. It is possible to write something like the following for detection.

```
enum Posture
{
    CLOSED,
    NOT_CLOSED
};

Posture IsClosed( const Data& data )
{
    float v = ComputeZRoundness( data );
    return ( v > 2 ) ? NOT_CLOSED : CLOSED;
```

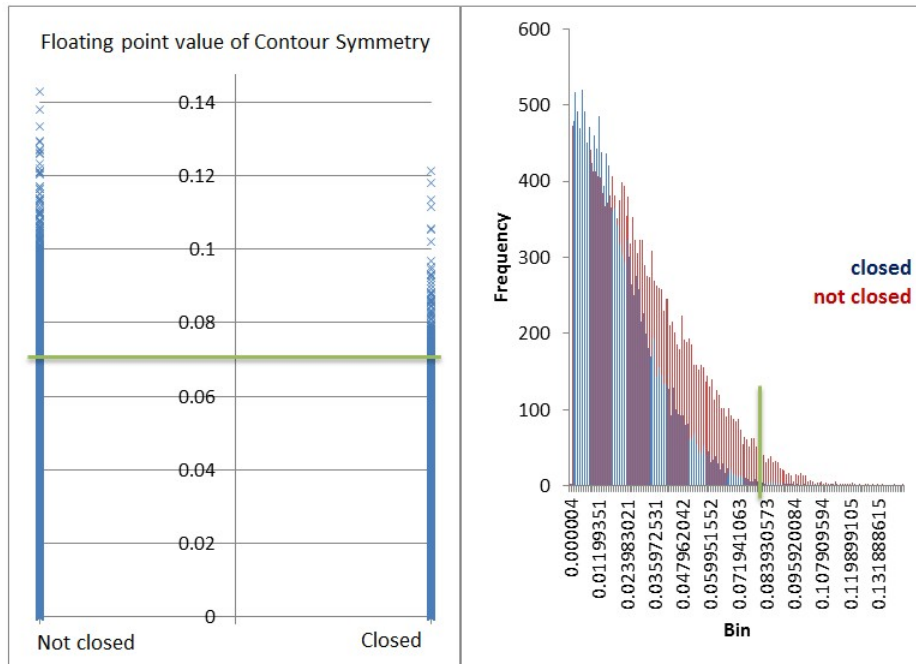
Applying this single rule gives a chance only slightly greater than 50 percent that the classifier makes the right classification. This is not accurate. Also, finding the separation line manually is difficult in all cases when the two classes don't overlap. It is possible to add extra thresholds, increasing the accuracy of the classifier, but the accuracy can only improve to a certain point by using just one classifier.

The data in the above graphs is not linearly separable, resulting in the absence of a single line that separates class not closed from class closed for either of the classifiers. Methods exist to improve that. Kernel methods as used in Support Vector Machines deal with recasting the data into a higher dimensional space to possibly enable linear separation. It is also possible to seek class boundaries using nonlinear functions. These methods are outside the scope of this white paper—refer to references listed at the end of this paper for more information.

Consider what happens when another classifier is added to the mix. Let it be a *Contour Symmetry* classifier, which is defined later in this paper. This classifier returns a floating-point value showing how symmetrical the contour of the image of the hand is in the sensor plane.

Figure 5 shows distribution of two classes of data when input samples are processed with the Contour Symmetry classifier.

Figure 5. Contour Symmetry feature of 38,000 samples



These graphs show the distribution of two classes of data when input samples are processed with the Contour Symmetry classifier.

Thresholds can be created for this classifier to distinguish between the open and closed cases. For example, by visually examining the graph, a threshold of 0.08 looks reasonable.

With two classifiers, answers have to be weighted in some way because their contributions may not be equal. Finding those two weights manually is not a trivial process as the significance of the classifier's "answer," for a given input sample, is difficult to find. In fact, many gesture detection algorithms that use heuristics are plagued by a significant number of such weight factors and there is no clear way to find them in all but the most trivial cases. Consider the final following detector.

```
enum Posture
{
    CLOSED = 1, // arbitrary choice between +1/-1 here
    NOT_CLOSED = -1
};

Posture IsClosed1( const Data& data )
{
    float v = ComputeZRoundness( data );
    return ( v > 2 ) ? NOT_CLOSED : CLOSED;
}

Posture IsClosed2( const Data& data )
{
    float v = ComputeContourSymmetry( data );
    return ( v > 0.08 ) ? NOT_CLOSED : CLOSED;
}

Posture IsClosed( const Data& data )
{
    float v = w0 * IsClosed1( data ) + w1*IsClosed2( data )
    return ( v > threshold ) ? CLOSED : NOT_CLOSED
}
```

A weak classifier is defined as one that is only slightly correlated with the true classification, like the functions **IsClosed1** and **IsClosed2** above – their accuracy is only slightly better than random guessing. It has been shown that by combining several weak classifiers it is possible to create a stronger classifier. A seemingly unsolvable problem here is how to find the right thresholds and right weights for the weak classifiers. Understanding and balancing all of the threshold values of all weak classifiers for all instances of the example data by hand is an impossible task. A heuristic approach would have to be limited to analyzing a reasonably small number of well-understood features, and manual tweaking of the threshold parameters and weights. For a complex problem without a well-defined, well-separated feature set, this is not a viable solution.

Considering the above, it might look impossible to employ such methods to classify the hand state. By using machine learning, it is possible to achieve a 70+ percent accuracy rate just by using the above two weak classifiers.

Classifying with AdaBoost

Despite the seemingly impossible task at hand, the idea of combining the weak classifiers similar to the ones discussed so far to form a stronger classifier is sound. The field of machine learning offers a solution. Machine learning can be defined as the study of computer algorithms that improve automatically through experience. In machine learning such a combination of weak classifiers is called an ensemble; one of the methods to generate a classifier that is stronger than an individual weak classifier in an ensemble is called boosting. One of the best known implementations of boosting is called Adaptive Boosting (AdaBoost) of which there are many variants. This example uses Discrete AdaBoost. Discrete here means the output values from the weak classifiers can be only +1 or -1.

AdaBoost is an algorithm for constructing a strong classifier as a linear combination of weak classifiers. Here is how the AdaBoost training process goes.

1. Start with a set of training-hand data samples, obtained as described earlier. Assign a label to each sample, either +1 or -1. For instance, closed hand data can have a label of +1 and non-closed hand data can have a label of -1, although it's arbitrary. Initially, assign an equal weight to all data samples. This is the only manual part of the process. Some of the training data should be retained for testing. For more information, see *Acquiring Machine Learning Data* on page 17.
2. Implement a set of weak classifiers in your favorite programming language. These should take the hand data as input to output a single floating-point value as a result of analyzing a certain feature of the hand data. The trainer creates instances of those weak classifiers, each with a unique threshold. An instance of a weak classifier is a function that takes the floating-point output from the weak classifier function, compares it with the threshold, and returns either +1 or -1, therefore assigning a class to the hand data sample. Such functions are called decision stumps, alluding to one-level decision trees.
3. Build a strong classifier by repeating the following steps.

- Find the most successful instance of a weak classifier, and add it to the strong classifier with some weight. The most successful instance is defined as the one that makes the smallest classification error on the weighted training data samples.
- Scale the weights of the training samples so that the samples on which the best classifier chosen in step 3a made a correct classification, receive a smaller weight and the samples on which it made a mistake receive a greater weight. This ensures that in the next iteration a different classifier is chosen that correctly classifies the samples that were incorrectly classified in this iteration.

The selection of weights in 3a and 3b forms the core of AdaBoost. Refer to the Appendix for further explanation of the algorithm.

The above algorithm can be really effective. It can be used in combination with any other machine learning algorithms as well. It's short and simple to understand and debug. Although in some recent publications it has been suggested that Discrete AdaBoost doesn't perform well in the presence of random noise, we found it more than satisfactory for our goals.

Another useful property of AdaBoost is its ability to quickly reject unimportant classifiers and give more weight to more important ones. On a basic level, this means it's possible to add as many different classifiers as you want and AdaBoost will automatically find only the most important ones. This is subject to a problem known as overfitting, which will be covered later in this paper.

From the above, it follows that with AdaBoost it is easy to do knowledge extraction; in other words, given a data set it's possible to find what classifiers (features) best describe the data. It may give an important insight into the nature of the training data.

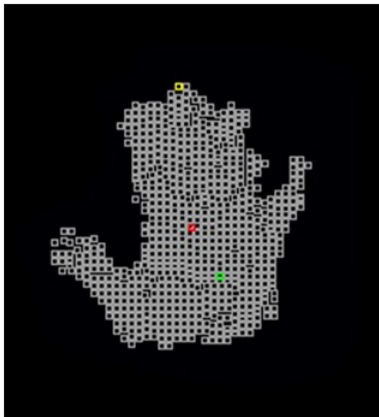
Finally, Discrete AdaBoost has a high run-time performance, sufficient to be used on Xbox 360.

Implementing Distance-Based Classifiers

These implementation details start with different types of classifiers. A classifier looks at features in the data and outputs a value of some sort that is subsequently mapped onto +1 and -1 results through the use of multiple instances of the classifier that have different thresholds. The sample uses weak classifiers. Looking at some of them should be of interest in the context of this paper, to see examples of how a weak classifier can be designed.

One of the most powerful weak classifiers for hand-status detection is based on analyzing features of the histogram of distances from voxels forming the hand to the centroid of the palm or the wrist. Figure 6 shows hand voxels and a few important points on them.

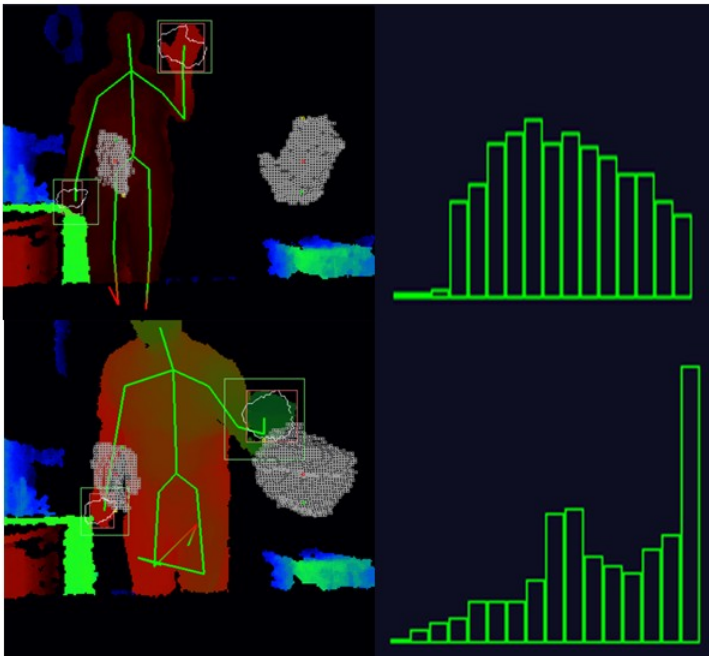
Figure 6. Hand voxels and some important points



The red point is the centroid, the green point is the wrist position, and the yellow point is the farthest point from the wrist.

The shape of the graph changes when the hand is closed.

Figure 7. Distance histogram differences for open and closed hand



The right column shows distance histogram differences for the open and closed hand. The grey clouds of points in the left column are the corresponding hand voxels scaled to uniform size.

The distances are normalized to the extent of the hand voxels cloud. When the hand is closed, the hand voxels move noticeably to the farther distances, shifting the graph to the right. When the hand is open, the voxels clump together, moving the peak of the distribution towards the left edge. By learning the shape of the histogram, it is possible to construct a good classifier.

The histogram graph consists of vertical bars representing the number of pixels having a distance in the corresponding range.

Learning the height of each bar is trivial, but how is it possible to learn the shape of the graph? The simplest way is to have a number of classifiers taking the difference between the heights

of two bars. This means that for a 16-bucket histogram we have to create 240 classifiers. Each such classifier will require about 100 instances, so this method should be able to learn the shape of the histogram by using 24,000 instances of classifiers. For a 16-bucket histogram, it is also a good idea to create 16 classifiers that would learn the height of each individual bar as well, with about 100 instances each.

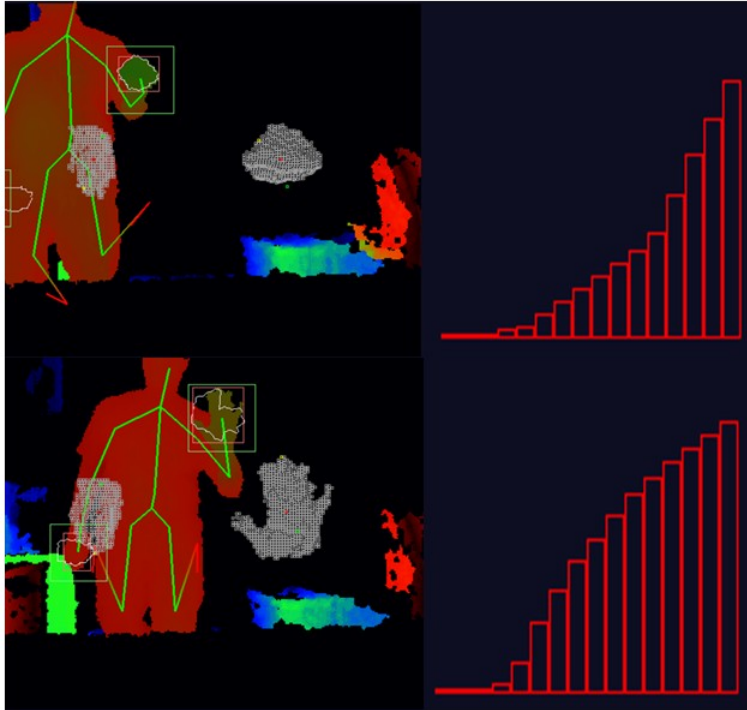
The histogram is normalized, which means that hands of different sizes will produce similar shapes, and the classifiers that learn the values of each of the 16 buckets will have a defined range—[0...1] in this case.

Additionally, the shape of the graph of the histogram can be described to some extent by statistical parameters such as mean, median, and variance. All these parameters are simple enough to learn by creating a classifier that simply compares the value of the mean, median, or variance with a given threshold.

Another important observation is that by taking distances from different points and building several histograms, it is possible to combine several such histograms for increased accuracy. The HandOpenClosed sample creates two sets of data. One set of data is based on the centroid of the hand and another on the position of the wrist. Examining the weights the AdaBoost trainer assigns to the data sets, it looks like the histogram built with the origin being at the position of the wrist has greater weight, or is more important. However, the combination of the two makes the final classifier more robust.

It is possible to sum up the heights of the bars going from left to right (from the nearest distance to the farthest distance) and create an integral, or cumulative, histogram. The shape of this histogram also changes markedly depending on whether the hand is open or closed, as can be seen in Figure 8.

Figure 8. Integral histogram shapes for the open and closed hand



Note the markedly different shapes of the graphs on the right.

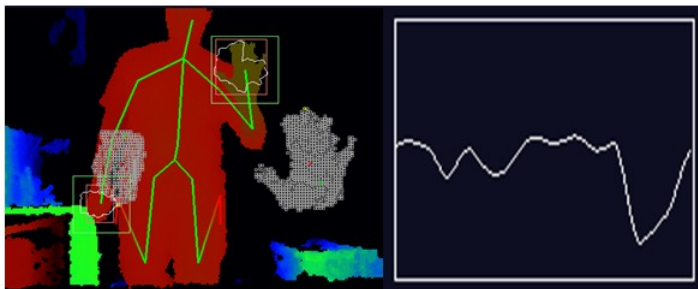
Visually, it's clear that the histograms are different. One is concave, the other is convex. Considering the quantity being plotted, it is easier to capture the shape of this type of histogram. As each bar is the sum of all previous bars, the interdependency between the bars is implicit. Also, as this is a normalized integral (cumulative) histogram, each bar represents the percentile point in the distribution of distances. To clarify this, consider the middle bar—its height tells us the percentage of all distances that are shorter than the half of the maximum distance. The classifier that learns the shape of that histogram simply takes the height of each bar as input; no comparison of the pairs is required here.

Surveying Other Classifiers

The detector in the HandOpenClosed sample mainly relies on the histogram of distance classifiers because when the shape of the image of the hand is noisy or too small or both, that classifier produces a reasonably strong detection signal. Given an adult hand, this means that the detection rate remains reasonable up to the far clipping plane of the sensor. The shape of the image of the hand is well-defined for adults up to the distance of about 2.8 meters from the sensor, so it makes sense to add classifiers using the shape of the hand for classification. The idea here is not to replace the histogram classifier, but to augment it. AdaBoost makes possible the addition of many classifiers to the algorithm. AdaBoost chooses the best ones for the training data set, subject to the overfitting problem.

A contour is the difference between the distances from a central point to the outline of the image of the hand to a unit circle.

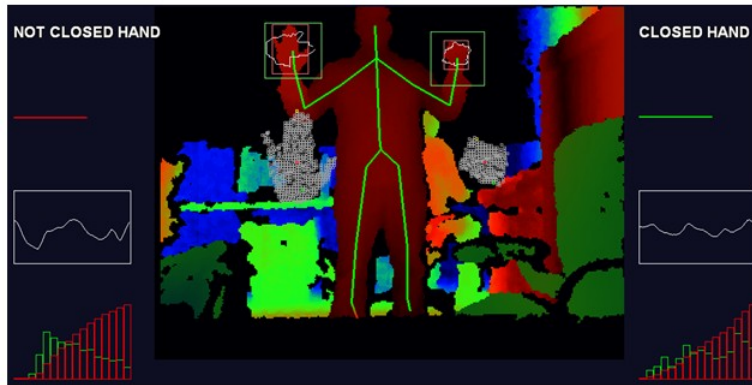
Figure 9. Example of an open hand contour



The white graph on the right denotes the distances from the center to the outline of the hand image.

In the sample, the contour is built by tracing 2D lines from the points on the bounding box of the hand towards the center point, and storing how many pixels it traveled before it hit a hand voxel. Other implementations of contour search exist—for example, see the recommended reading list. In the sample, the contour is also smoothed by applying a low pass filter on it. This is to suppress high frequency noise that arises from the overly simplistic, contour-finding implementation and the instability of the depth image in general.

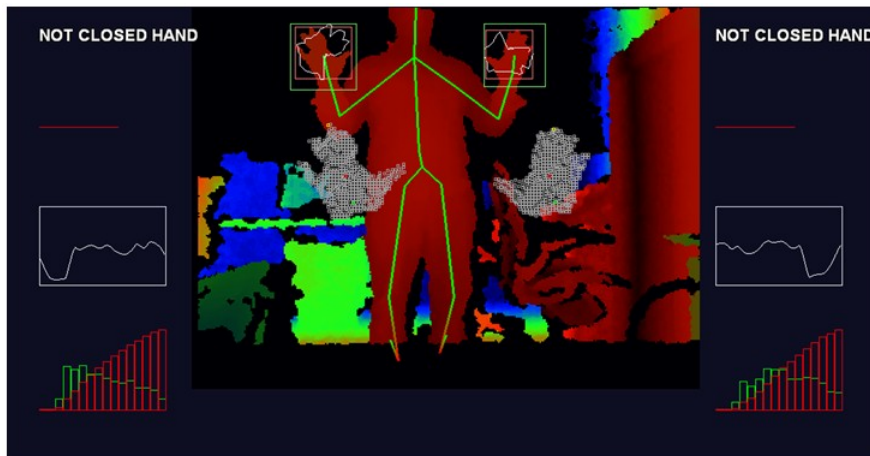
After inspecting contours of open and closed hands, it becomes obvious that there are several features that distinguish an open hand from a closed hand. For example, contours of closed hands look flatter and more symmetrical, as shown in Figure 10.

Figure 10. Contours and amplitude of open and closed hands

The contour of the open hand, on the left, results in a higher amplitude than the closed hand.

Graphs of contours of open hands, however, contain more variation with higher peaks and deeper valleys.

If line tracing of the contour starts from the point on the hand boundary where the wrist connects to the hand, then an interesting feature of this representation of the data becomes apparent. It reduces variability of the contour between the frames due to hand movement and noise and also gives the ability to detect whether a specific finger is being held up, as the peak will always appear around the same area in the graph. The sample doesn't make use of this feature. See Figure 11 for the prominent valleys formed by the thumbs in the contour images.

Figure 11. Graphs showing separate thumb position

The contour of the open hand creates valleys in the amplitude that correspond to the thumb. The two hand images result in amplitude variations for an open hand with a separate thumb.

There are a number of classifiers that capture the shape of the contour. The spread of the contour graph, the difference from the mean to the median, the amount of symmetry are some examples. In the final detector, contour-based classifiers improve accuracy for the cases where the outline of the hand has a reasonable number of voxels.

The detector in the HandOpenClosed sample uses a few more types of classifiers, including one based on skeletal tracking instead of the depth image. For example, a ratio of hand to forearm bone lengths yields a remarkably strong detection signal. When the hand is open, the hand joint moves up away from the wrist, and when the hand is closed, it moves back down towards the wrist. It is possible that this behavior may change in the future. The classifier of the form $v = \text{playerSize} * \text{numVoxels} / \text{centroid.z}$ produces a fairly good detection signal. Intuitively, it counts the number of voxels in the hand independent of the size of the player and the distance to the hand. It is possible to find such classifiers by plotting the training data set in a math package and trying to find different combinations of parameters that provide the best separation visually.

Acquiring Machine Learning Data

The machine learning process revolves around the idea that the detector is trained on a set of data and then it is expected to classify previously unseen data in a reliable way.

How many training samples is enough? The boosting process can be thought of as separating the multidimensional problem space into abstract volumes each of which contains one class of data. At the beginning, an instance of a classifier is selected, and that adds a separation perpendicular to its axis at the point of the instance's threshold in that space. The samples are reweighted and the next classifier is chosen, which adds another separation, and the process continues until there are enough instances of weak classifiers to describe the training set.

So having just one sample of one class doesn't make sense as it means there is one single point in the whole space. This means the whole space is of that class. There is no need for a detector if that's the case—the hand is either always open or closed.

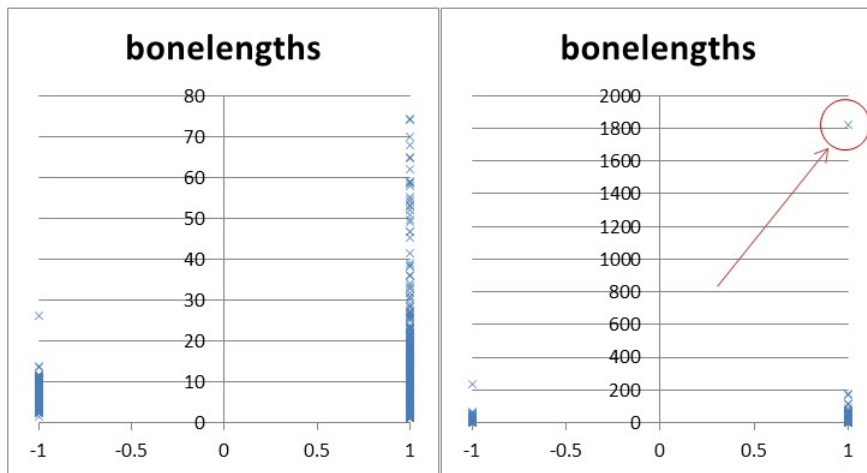
Having two samples of opposite class doesn't make much sense either as boosting will quickly find a separation plane that perfectly splits the space into two subspaces. In this case, it is possible to write code with an *if()* statement that will classify the sample. There would be no need for machine learning if the data could be classified like this.

Practice shows that for robust learning of the hand status a lot more than two training samples are needed. The training samples should be taken at different distances, different angles to the camera, by people with different hand sizes. To cover a range of the numbers of voxels in the training data, there are many examples with few voxels and examples with many voxels in the database. For example, the database used for the HandOpenClosed sample contains about 38,000 training frames recorded by four people. The detector trained on that data attains 92.3 percent accuracy in detection on the test data set and feels robust in action.

Some training samples in the dataset will be outliers. As is the case with all cameras and measuring instruments, the data coming from Kinect generally contains some noise. Sometimes, the data sample will contain invalid or random data due to errors in the floodfill process, noisy depth data that distorts the outline of the hand, or direct sunlight causing invalid depth pixels across the hand image, among other reasons. When included in the training set, outliers force AdaBoost to try to match them, unnecessarily and wrongly boosting classifiers that can classify those invalid samples. The outliers' incorrect boosting reduces the effectiveness of training. Clean recordings before training are a necessity. There are methods for removing outliers, but the everpresent danger of removing valid samples that look like outliers requires vigilance and domain knowledge.

One example of accidentally removing valid samples that look like outliers is the classifier based on the ratio of two bones. Figure 12 includes the two graphs of the ratio.

Figure 12. Graph of apparent outliers next to the clean data



The left graph shows well-behaved, easily separable data, and the right graph shows real data with some samples completely off the chart. The two graphs have very different scales.

In adults, a reasonable ratio of forearm length to hand length is about three to four. With skeletal tracking, it varies in a wider range, but what's useful is that in most cases this ratio is a good indicator of whether the hand is open or closed as skeletal tracking tends to move the hand joint closer to the wrist when the hand is closed. Here we have training samples with ratios of 200 and even 1800. It turns out that when the hand occludes the shoulder joint, ST starts to be unsure about the wrist and hand joint positions and therefore what used to be a good signal becomes noise. This was found to be a bug in the HandOpenClosed sample, when in certain hand positions the output from the detector was completely wrong. This was happening because the trainer hadn't seen the noisy data during training, and without the noisy data the bone ratio weak classifier attains the highest weight in the mix.

After adding to the training set about 3,000 training samples of postures where the hand was occluding the shoulder, and then retraining, this problem went away. If the data were considered to be outliers and were thrown away, the original bug would return, and the graph would look like the one on the left in Figure 12.

At the very least, the number of training samples should exceed the number of instances of classifiers; otherwise, *overfitting* may occur. Overfitting is an undesired condition that happens when the trainer manages to tightly fit classifiers to match the training samples. As the training samples represent only a small subset of the real data, some margins should be added to the model to avoid incorrect classification of the real data. The best way to avoid the incorrect classification is to give the model a lot of training samples of both classes.

How many instances of classifiers is enough? Recall that in this implementation the weak classifiers are decision stumps and use different thresholds on the output of the data classifier to decide which class the given data sample belongs to. It might be counter-intuitive, but it helps to keep the number of thresholds (instances of the classifiers) low. This prevents overfitting and also increases training speed. Accuracy of the final classifier will grow slowly as more instances are added. The difference in accuracy between having 60 and 128 thresholds per classifier is about three percent, and the difference between having 128 and 256 thresholds is less than one percent, but the increase in training time is threefold.

How can the resulting accuracy be measured? It is good to know whether the changes that are made to the detector and the trainer improved accuracy or not.

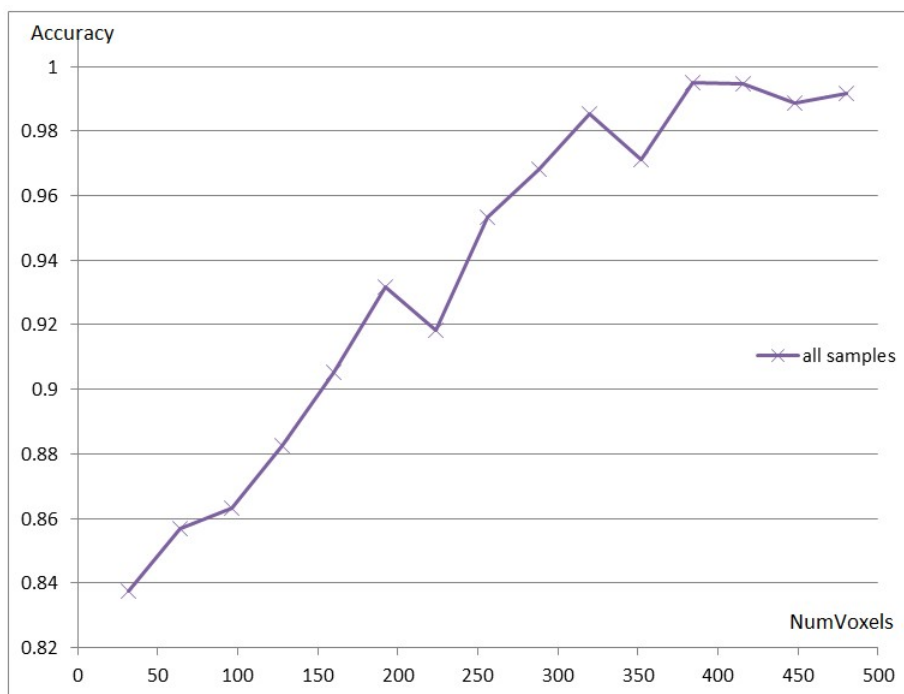
There are several good methods for measuring classifier performance, and there are plenty of references about, as shown, for example in references. The method used in the HandOpenClosed sample is called *holdout*—by setting aside (or "holding out") a portion of training samples into the test set, the original set is separated into two and the classifier is trained on one and tested on another. This gives a much better idea about the performance of the classifier than testing on the training set.

Here are the reasons for this—first, if a classifier saw an example during training, it will perform much better on it, and second, the classifier will have to perform in the wild and no one is going to be using the training set to play your game. There are other methods for testing performance, such as N-fold cross validation and bootstrapping. In the HandOpenClosed sample, holdout is used simply for training performance reasons.

Implementing Ensemble Trainer

The results of the final classifier make clear that the accuracy of the classifier isn't constant with the number of voxels available. Figure 13 illustrates the accuracy of the classifier.

Figure 13. Range of classifier accuracy



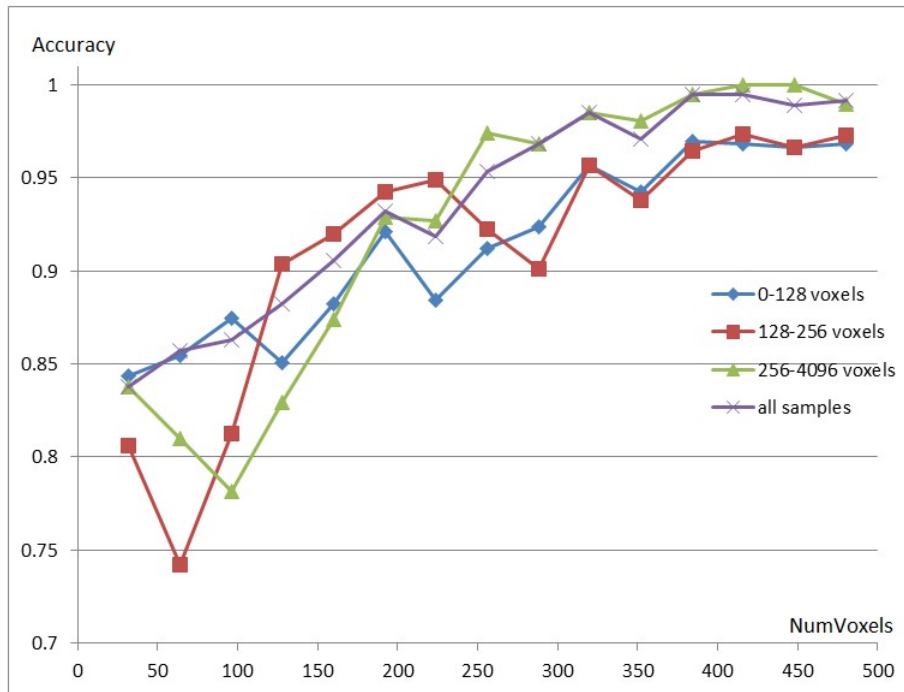
Accuracy of the classifier trained on 38,000 samples from different distances against the number of voxels per data sample.

The accuracy reduction can be explained by increasing proportion of noise in the distance, when the number of hand voxels decrease. Close to the sensor, when there are plenty of voxels,

image-based features like the contour are more pronounced. In the distance, where only a handful of voxels is available, averaging features, such as ZRoundness and histogram analysis, are more telling.

It is possible to train classifiers for different ranges of input data so that their individual accuracy is higher than the accuracy of a classifier trained on the entire data set. The results in Figure 14 show exactly what was anticipated, that different weak classifiers have higher accuracy for different data ranges. The strong classifiers become experts on their volume of training data. From the graph in Figure 14, you can see that a classifier trained on a small number of voxels outperforms all other classifiers in that range.

Figure 14. Graph of classifier accuracy



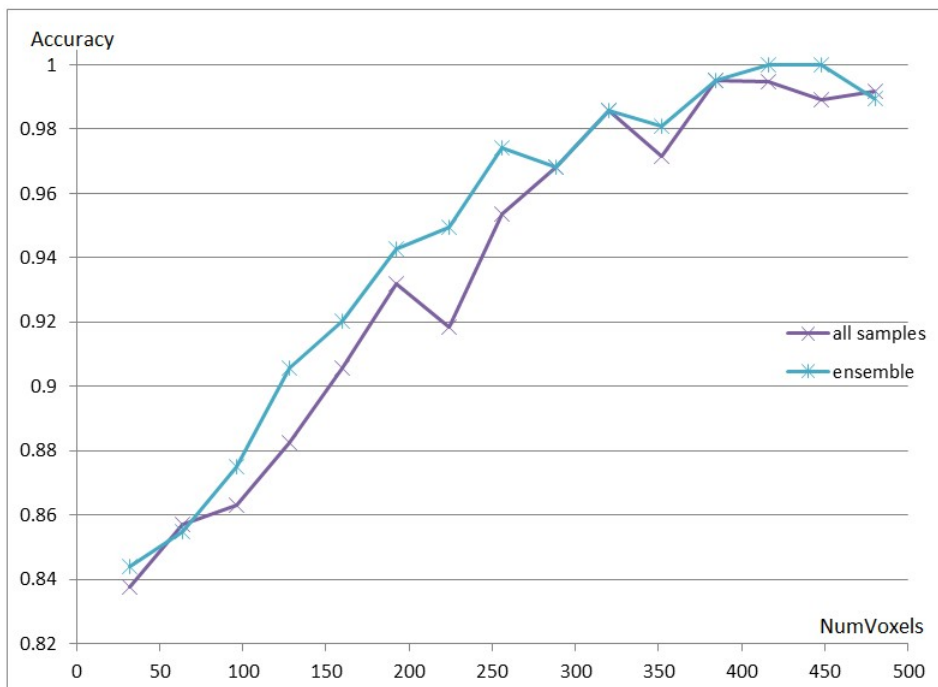
Accuracy of classifiers trained on different ranges of number of voxels superimposed on top of the classifier trained on all samples.

The process of combining multiple classifiers is called ensemble learning or ensemble methods. Ensemble methods combine multiple classifiers to obtain better performance than could be obtained from any of the constituent models.

As mentioned earlier, AdaBoost itself is an ensemble method. However, trying to use AdaBoost on the ensemble of strong classifiers won't do much good because the weights assigned after boosting are constant.

What is desired here is for the weight to be a function of the input sample, so that the right classifier is selected for the input sample. The simplest method to achieve that is to split the input data into ranges and test each of the strong classifiers in the ensemble on that range, and select the winner for that range. This is called *gating* because a data sample is routed to a specific constituent classifier. As the process is repeated during run time, an expert strong classifier for each input is found and queried. Figure 15 illustrates the comparison of the ensemble classifier to the original classifier.

Figure 15. Ensemble classifier compared to the single classifier



This graph shows the ensemble classifier, consisting of three strong classifiers, compared to the single classifier.

Clearly, the ensemble classifier stays above the convex hull of other classifiers, which is exactly the result needed. The HandOpenClosed sample uses this method to achieve an extra few percent of accuracy.

One unsolved problem in the sample is automating the choice of data split boundaries. In the current version of the sample the splits are found empirically.

Improving Run-time Performance

Run-time performance of the detector depends on the number of classifiers selected by AdaBoost and on how expensive it is to update each classifier with new data. Each classifier has to provide a response to the input data. It has been found that it's possible to simply limit the number of classifier instances at a slight decrease in detection accuracy. To do that, it's enough to sort the instances by decreasing weight α_t and then throw away the tail. It's possible to predict and change run-time performance by selecting the number of classifiers that are left in the array.

If a 2000-instances classifier is reduced to a 500-instances classifier, a five to seven percent loss in accuracy may be expected, although it varies depending on the input data.

As the detector works on the depth image, the size of the hand in screen space also affects the run-time performance due to the variable costs of the flood fill process.

Conclusion

The described method was originally used to detect open and closed hands in a game title, but it can be applied to many scenarios. Reasonably good results have been obtained when training it on a "thumbs up" gesture for example. Also, once the hand status is determined, the finger position can be found as well, which enables even more scenarios, like finger swipes, pointing, or detecting a thumbs up or down hand posture.

Discrete AdaBoost is a simple method that gives surprisingly robust results in hand status detection and also ST gesture detection. It allows writing simple classifiers and tuning their parameters automatically. There are still plenty of other approaches to explore—for example, Real AdaBoost or Support Vector Machines. This work represents just the first step toward hand status detection with Kinect.

Appendix A. A rigorous explanation of AdaBoost

Let's first have a look at pseudocode of Discrete Adaboost.

1. Given a training set (x_i, y_i) of size N , where $x_i \in X$ (X is the domain of training data samples) and $y_i = \{-1; 1\}$ and the number of iterations T and the set of weak classifiers $H[T]$.
2. Set $D_1(i) = \frac{1}{N}, i = 1 \dots N$
3. for $t=1$ to T
4. find the classifier h_t from set H that minimizes the error with respect to D_t , or $h_t = \text{argmin} (W_-(h_t))$ where $W_-(h_t) = \sum_{i=1}^N D_t(i) [y_i \neq h_t(x_i)]$ (h here stands for "hypothesis").
5. if $W_-(h_t) \geq 0.5$ then stop
6. let $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - W_-(h_t)}{W_-(h_t)} \right)$
7. update $D_{t+1}(i) = D_t(i) \exp(-\alpha_t y_i h_t(x_i))$ then renormalize D_{t+1} to one.
8. The final classifier is $C(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$

Following is the explanation of the algorithm.

In step 1, we're given a training set of data x_i with class labels y_i . The data here is the voxels for the hand for the i -th frame of recording, obtained as described earlier. The class label y_i can be either -1 or 1, so a number -1 or 1 is attached to each sample in the training set. This is simple to achieve with hand open closed detection—the frames of an Xbox Studio clip of open hands and non-opened hands needs to be marked with $y=1$ for closed hands and with $y=-1$ for open hands. Note that +1 and -1 is just a convention and it's possible to attach +1 to open hand and -1 to closed hand; the interpretation of y_i is entirely up to the user. Each .XED clip will contain K_i frames of data, so after labeling them up and putting them into one array we get the following frames of labeled data in the training set.

$$N = \sum_{i=1}^{\text{numFiles}} K_i$$

In step 1, we also introduce a set H of weak classifiers. Weak classifiers are sometimes called *weak learners*. Each classifier is simply a function that takes the input data frame and outputs a floating-point value $v \in \mathbb{R}$ as a result of analyzing some "feature" of the data frame. For AdaBoost, a value of -1 or 1 is needed to be output from a classifier. This is clear after looking at the equation in step 4 where y_i is compared to the output of the weak classifier h_t . This problem can be solved by adding multiple instances of the classifier that would threshold the output value with a constant threshold and output 1 or -1 accordingly. Such functions are called "decision stumps", referring to single-level decision trees. The number of instances per classifier has to be enough to cover the whole range of v with a reasonable granularity. Also, each instance must be duplicated to check for "greater than" or "less than" condition. So the instances of such classifiers form the set H of weak classifiers.

In step 2, the vector D_1 represents a distribution of weights of training samples; the weight here means how important a training sample is. As it is possible to have N greater than 100,000, using a double-precision, floating-point type is preferred for D_t and any calculations related to it for accuracy. It is initialized with a flat distribution, where each training sample is weighted the same. The vector has a subscript because it is modified on each step of the algorithm.

In step 3, the iterations start. The maximum number of iterations AdaBoost can make is the number of weak classifiers T in the set H . On each iteration, one weak classifier h_t is chosen from H that has a minimum error over the distribution D_t . We find this error by summing the elements of D_t that correspond to the training samples on which the classifier makes a mistake. This is what the $[y_i \neq h_t(x_i)]$ part of the equation denotes. So basically, the result of this step is a classifier that makes the smallest mistake on the training samples with the current distribution D_t .

In Step 5, if the smallest error is greater than 0.5, the algorithm must be stopped. Improvement of the strong classifier is only guaranteed mathematically if the error is less than 0.5.

Step 6 calculates the weight α_t for the chosen classifier that will be used at run time for classification to weight the output of h_t . The error here is denoted as W_- . It is simply the sum of weights from D where $y_i h_t(x_i) < 0$. This sum may require a little explanation. Considering the label $y_i \in \{-1; 1\}$ and the output of weak classifier $h_t(x_i) \in \{-1; 1\}$, their product can only be $y_i h_t(x_i) \in \{-1; 1\}$. When an incorrect classification is made, this product is negative. W_+ denotes the sum of the weight where $y_i h_t(x_i) > 0$, which denotes the sum of weights of correctly classified samples. By the definition of distribution D , $W_+ + W_- = 1$, the written equation for alpha is more compact:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{W_+(h_t)}{W_-(h_t)} \right).$$

Alternative definitions of α_t exist.

Step 7 recalculates the weights for the distribution D so that the training samples on which the best classifier made a mistake are given a higher weight, and the samples on which it did a correct classification receive a reduced weight. This means that on the next iteration the algorithm is more likely to select a classifier that succeeds on the samples on which the currently best classifier failed.

Finally, step 8 shows what happens at run time. At run time, the classifiers are applied to a previously unseen data point x and their responses are weighted with corresponding α_t and summed together to form a strong classifier response.

Appendix B. Optimizing with RBoost

The research focus on AdaBoost during the past decade has resulted in proposed modifications of the algorithm. One of the mathematical features that has been sought is to optimize the sparsity of the strong classifier, or the ability to achieve a desired result with as few weak classifiers as possible. Sparsity in this context means to maximize the number of weak classifiers with a zero weight. AdaBoost finds a solution, but it's not necessarily a sparse solution.

RBoost is an easy-to-implement method for increasing sparsity of the AdaBoost solution. Like AdaBoost, RBoost iterates on all of the training samples. But instead of just giving more weight to the best classifier, RBoost redistributes weight between the worst and the best classifiers.

This process ends with a new, reweighted set of weak classifiers that contains fewer instances than the original. The resulting detector has a slightly increased accuracy. It is a much desired property because the run-time speed directly depends on how many instances of weak classifiers must be queried, especially if accuracy doesn't have to be sacrificed.

The HandOpenClosed sample uses an implementation of Discrete AdaBoost followed by a round of RBoost.

For Further Reading

Ian H. Witten, Eibe Frank, and Mark A. Hall, Data Mining: Practical Machine Learning Tools and Techniques, (Morgan Kaufmann Publishers, 2011).

Ethem Alpaydin, *Introduction to Machine Learning*, (MIT Press, 2010).

David J. Hand, Heikki Mannila, and Padhraic Smyth, *Principles of Data Mining*, (MIT Press, 2001).

Yoav Freund and Robert E. Schapire, [A Short Introduction to Boosting](#), translated by Naoki Abe (Journal of Japanese Society for Artificial Intelligence, September, 1999).

Philip M. Long and Rocco A. Servedio, [Random Classification Noise Defeats All Convex Potential Boosters.](#)

Zhen James Xiang and Peter J. Ramadge, [Sparse Boosting](#), Dept. of Electrical Engineering, Princeton University, Princeton NJ, ICASSP 2009 Final Submission 9/29/08.

Robert E. Schapire, [The Strength of Weak Learnability](#), Machine Learning, Vol. 5, Issue 2, Kluwer Academic Publishers, Boston 1990.

Tony Ambrus and Shammi Mohamed, [Kinect Hands: Finger Tracking and Voxel UI](#), (presented at Gamefest 2011, Seattle, Washington, August 22 and 23, 2011).

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles

by Scott Selfon, Development Lead
Advanced Technology Group (ATG)

Published: February 9, 2012
Updated: May 8, 2012

In addition to voice chat and sound position tracking, developers can use the microphone array in the Xbox 360 Kinect Sensor to enable speech recognition. When developers design robust grammars, speech can be a powerful, intuitive, and an almost magical part of gameplay, adding another dimension of interactivity to gestures and visual response.

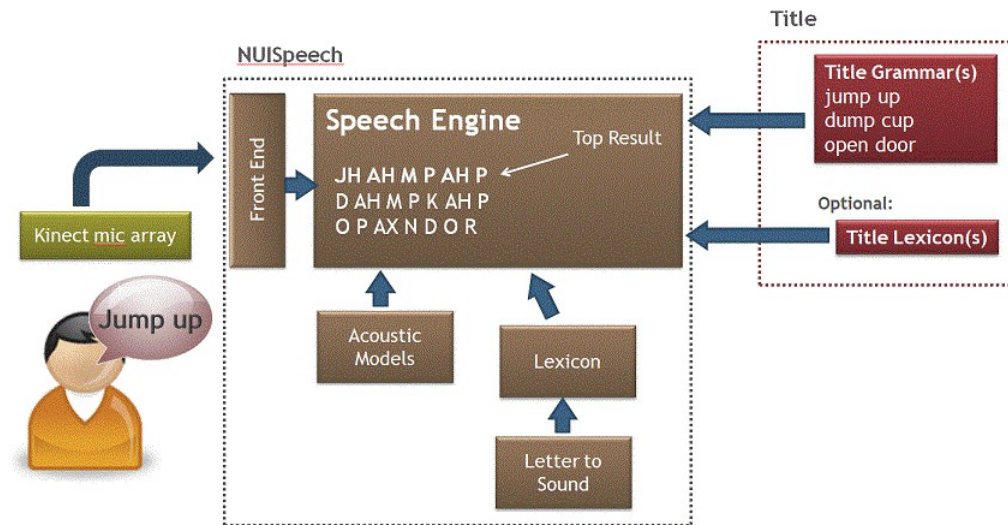
The functionality of speech recognition is exposed on the Xbox 360 console by the NuiSpeech library. The title provides one or more *grammars*, which are lists of rules for the words and phrases that the NuiSpeech engine should attempt to recognize. This paper focuses on the development of grammars for typical game scenarios.

NuiSpeech is distinct from the Kinect Sensor, which possesses hardware-based audio processing to reject background noise from the title and environment. NuiSpeech is a software library that runs on the Xbox 360 console itself and uses CPU processing time to analyze the clean voice data stream from the microphone array. It can be used concurrently with other software libraries, so that NuiSpeech speech recognition can be run on incoming data while the game developer simultaneously routes that data to voice chat, performs emotive analysis on the voice data, and so on. Note that NuiSpeech cannot provide simultaneous multiplayer speech recognition—that is, two people speaking at the same time—though turn-based multiplayer speech recognition is supported.

The Basics of Speech Recognition

Speech recognition consists of multiple aspects, some of which are delivered by the title and others, such as NuiSpeech and its acoustic models, by the Xbox 360 Development Kit (XDK).

Figure 1. An outline of speech recognition modules and processes



The title provides one or more grammars, as well as an optional lexicon, or lexicons, with any custom pronunciations. (In the example in Figure 1, a single grammar supplies three short phrases.) NuiSpeech uses an *acoustic model* for a specified language or regional dialect, which models how people sound when they talk.

The XDK contains most of the primary speech recognition functionality. Developers need to build grammars and, optionally, lexicons for custom pronunciations. The XDK delivers:

NuiSpeech engine

This is based on Microsoft Speech Technologies—including both the Microsoft Speech Recognition Engine and the Microsoft Speech API (SAPI). It is a mature and robust speech recognition technology used on Windows, mobile devices, and automotive applications. For more information, see [Microsoft Speech Technologies](#) on MSDN.

Acoustic models

These are statistical representations of the different speech sounds that make up a particular language. (United States English, for example, contains roughly 40 unique speech sounds.) Based on the results of NuiSpeech sampling, the currently-spoken text, the acoustic model returns the likelihood that a given utterance was produced by a particular phone string. Acoustic models are trained from multiple recordings of multiple speakers for a given language.

A Lexicon

This defines the speech sounds that make up words in a given language. The XDK provides a generalized lexicon for each language, which handles a large, representative selection of words in that language. Developers can expand the supplied lexicon by creating custom lexicons that specify custom pronunciations for specific words or phrases.

Developers are responsible for creating grammars. A *grammar* is a list of rules for the words and phrases that the NuiSpeech engine may recognize in a given game situation. NuiSpeech grammars originate as XML files that implement a subset of the [Speech Recognition Grammar Specification \(SRGS\), Version 1.0](#) from the World Wide Web Consortium (W3C). Supported elements and attributes are described later in this white paper. Grammars are compiled using the grammar compiler tool, xbp.exe, on Windows for consumption on Xbox 360.

The lexicon provided by the XDK is a good, generalized lexicon that handles many of the more common words in a language. Recognition of non-standard and custom pronunciations—such as names, different pronunciations from the norm, foreign words, title-specific vocabulary, and so on—can be enabled within a grammar using the <token> tag, or delivered as separate lexicon files that comply with the pronunciation lexicon specification (PLS) of the W3C, which can be found at [Pronunciation Lexicon Specification \(PLS\) Version 1.0](#). Both are as described in more detail later in this paper.

Grammars can be loaded (by calling **NuiSpeechLoadGrammar** or **NuiSpeechLoadGrammarFromMemory**) and removed (by calling **NuiSpeechUnloadGrammar**) as needed, and individual rules can be enabled or disabled dynamically.

Grammar Building Blocks

NuiSpeech does not fully implement the W3C speech recognition grammar specification. The following is a partial list of restrictions or other aspects that titles should not use:

- ABNF form grammars—the grammar compiler requires XML form grammars only.
- References to external grammars—each grammar should be self-contained.
- Multi-language grammars—a single grammar that actively references words from multiple NuiSpeech lexicons.
- A subset of lexicon support.
- Support for semantics other than string literals.

While based on the same technology, NuiSpeech does not implement the full spectrum of functionality present in Microsoft SAPI. In particular, functional differences include:

- No support for text to speech (TTS).
- Support only for W3C specifications (not including VoiceXML 2.0).
- Language support.
- No dictation APIs are present.
- No support for semantic script.
- No text normalization.

That is, if you want to recognize when the user says *one*, rather than using the string "1", you need to spell out the defined utterance as "one" for English, "uno" for Spanish, and so on.

Specifying a Grammar with <grammar>

Each grammar XML file must be encapsulated within the *grammar* element. A basic example is:

```
<grammar version="1.0" xml:lang="en-US" tag-format="semantics/1.0-literals">
...
</grammar>
```

For *grammar*, *version* is a required attribute, and for NuiSpeech should be 1.0.

Another required attribute is *xml:lang*, which specifies the language (lexicon) used by this grammar. Note that the language must match the language that NuiSpeech used for initialization. For the current list, see [Local Language Support with NUI Speech](#).

📌 Note

Languages designated as *preview* are intended for development only, and they may not represent an available subset or superset of languages approved for use by titles that use NuiSpeech. For all information about approved languages, consult the XDK documentation and release notes.

If a grammar uses even basic semantic operations, such as <tag>, *tag-format* is required, though it is otherwise optional. Because most grammars use some semantics, *tag-format* should be specified. NuiSpeech solely supports *tag-format="semantics/1.0-literals"*.

There are a variety of optional modifying attributes that can be applied:

mode

The default setting is *mode="voice"*; therefore, this attribute is not needed. Dual tone multi-frequency (DTMF) recognition is not supported on Xbox 360.

encoding

This is optional. Generally, it is needed only for non-ASCII character sets.

root

This is optional, but recommended. Root specifies a root rule that is automatically activated when this grammar is loaded. Usage: *root="rulename"*

Defining Rules with <rule>

Grammars are broken down into one or more rules. Public rules that are not the root can be enabled or disabled dynamically at run time by the title by using **NuiSpeechSetRuleState**. By default, only an optional root rule specified in the <grammar> element is enabled; all others are disabled. (Note that new rules can also be created dynamically at run time by using **NuiSpeechCreateRule**.)

```
<rule id="YesOrNo">
...
</rule>
```

Optional modifying attributes of interest include *scope*. Its default is "private", which means the rule is not visible outside of this grammar. Since NuiSpeech grammars frequently are fully self-contained, public rules, it may not be necessary to use this attribute for Xbox applications. Usage: *scope="public"*

Identifying Alternatives with <one-of> and <item>

Rules frequently involve comparing selection from a list. The <one-of> element encapsulates multiple alternate potential utterances as <item> elements. The <item> element encapsulates a specific rule expansion—typically the core utterance to be recognized, though it can also refer to other rules using the <ruleref> element.

A basic example is:

```
<rule id="YesOrNo">
  <one-of>
    <item>yes</item>
    <item>no</item>
  </one-of>
</rule>
```

<one-of> elements can also be embedded within items to allow for multiple variations on an accepted utterance. For example:

```
<rule id="YesOrNo">
  <one-of>
    <item>
      <one-of>
        <item>yes</item>
        <item>yeah</item>
        <item>yup</item>
      </one-of>
    </item>
    <item>
      <one-of>
        <item>no</item>
        <item>nope</item>
        <item>nuh uh</item>
      </one-of>
    </item>
  </one-of>
</rule>
```

Individual <item> elements can be biased by using the weight attribute. Weight defaults to 1.0, and allows for some utterances to become more or less preferred in recognition. Typically, values greater than 1.0 indicate more bias—though the value is arbitrary and relative to the weight of other items—and values less than 1.0 indicate less bias. This attribute can become useful during testing and grammar iteration to adjust for words typically spoken by an end user.

```
<rule id="yes">
  <one-of>
    <item>
      <one-of>
        <item weight="6.0">yes</item> <!-- weighted 6/9 -->
        <item weight="2.0">yeah</item> <!-- weighted 2/9 -->
        <item>yup</item> <!-- defaults to "1.0", so weighted 1/9 -->
      </one-of>
    </item>
  </one-of>
</rule>
```

Consolidating Responses with <tag>

The <tag> element is used by NuiSpeech to map multiple recognized utterances to a single shared meaning, in the form of a string. The string does not need to match any of the individual utterances. The tag is reported back in NUI_SPEECH_SEMANTICRESULT by the member **pcwszValue**.

A yes/no rule that uses tags to abstract multiple affirmative and negative responses might be implemented as follows:

```
<rule id="yesOrNo">
  <one-of>
    <item>
      <one-of>
        <item>yes</item>
        <item>yeah</item>
        <item>yup</item>
      </one-of>
      <tag>yes</tag>
    </item>
    <item>
      <one-of>
        <item>no</item>
        <item>nope</item>
        <item>nah</item>
      </one-of>
      <tag>no</tag>
    </item>
  </one-of>
</rule>
```

Note that using the <tag> element requires the <grammar> tag to include the following attribute:

```
tag-format="semantics/1.0-literals"
```

Reusing Rules with <ruleref>

Rules can contain references to other rules by use of <ruleref>. A single rule could list global menu navigation words, for example, and other rules could reuse that same rule. Thus, you don't have to generate every possible permutation for phrases in the grammar file.

```
<rule id="Navigation">
  <ruleref uri="#action"/>
  <ruleref uri="#direction"/>
</rule>
<rule id="action">
  <one-of>
    <item>point</item>
    <item>jump</item>
    <item>go</item>
  </one-of>
</rule>
<rule id="direction">
  <one-of>
    <item>up</item>
    <item>down</item>
    <item>left</item>
    <item>right</item>
  </one-of>
</rule>
```

If your title uses rule references, you should be aware of potential divergences in phrase structures for other languages. In the preceding example, the grammar might need to be differently constructed when localizing in a language that uses different word order, such as *noun + verb*.

Discarding Garbage with <ruleref special="GARBAGE"/>

The active grammar rule tries, as much as possible, to match utterances to words or phrases that comprise the rule. While false positives may in some cases be screened by evaluating the reported confidence of a speech recognition match, the reported confidence may in some cases still be high. You can choose to invoke the garbage rule as part of any other rule, through the context <ruleref special="GARBAGE"/>. When speech is recognized by using this special rule, it will return the string "..." (three periods).

As an example, if we want our yes/no grammar to avoid false positives for remarks like "why not" (arguably a yes, but will generally be interpreted as *no*), we can invoke the garbage rule to collect utterances that aren't clearly a yes or no:

```
<rule id="yesOrNo">
  <one-of>
    <item>yes</item>
    <item>no</item>
    <item>
      <ruleref special="GARBAGE" />
    </item>
  </one-of>
</rule>
```

Of course, this can lead to more frequent false negatives. Also, while the garbage rule attempts to consume and ignore all invalid utterances, NuiSpeech performance may be somewhat reduced.

Custom Pronunciations

When a grammar uses words whose pronunciation is not in the XDK lexicon—for example, proper names of characters or places—the lexicon provided by NuiSpeech might not render the desired pronunciation. In this scenario, the title can provide custom pronunciations. These can reside in a separate lexicon file (which a grammar can reference using the lexicon element), or they can be used directly in place of the spelled word in grammar rules. Typically, custom pronunciations use the SAPI Universal Phone Set (UPS), which is based on the International Phonetic Alphabet (IPA).

NuiSpeech Language	UPS Listing
English – United States	Phoneme Table for English (United States)
French – Canada	Phoneme Table for French (Canada)
German – Germany	Phoneme Table for German (Germany)
English – United Kingdom	Phoneme Table for English (United Kingdom)
Spanish – Mexico	Phoneme Table for Spanish (United States)
Japanese – Japan	Japanese Phonemes (SAPI 5.4)
	(Note: Japanese uses Katakana rather than UPS.)

When you use phoneme labels, note that they are case-sensitive and must be space-delimited.

Custom pronunciations can be specified inline in a grammar as a replacement for a word within a rule, or they can be contained within a custom lexicon supplied by the title. A grammar refers to a lexicon by using the <lexicon> element.

Defining Pronunciation In-line with <token>

If a custom pronunciation is isolated to a single grammar, it can be embedded within the grammar directly by wrapping words in the <token> element using the *sapi:pron* attribute for that

word. When you do that, however, <grammar> needs several more attributes.

```
<?xml version="1.0" encoding="utf-8"?>
<grammar version="1.0" sapi:alphabet="x-microsoft-ups"
  xml:lang="en-us" tag-format="semantics/1.0-literals"
  xmlns="http://www.w3.org/2001/06/grammar" root="Names"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions">
  <rule id="Names" scope="public">
    <one-of>
      <item>
        <token sapi:display="URDNOT1" sapi:pron="ERR D N AO T">Urdnot</token>
        <tag>Alias1</tag>
      </item>
      <item>
        <token sapi:display="URDNOT2" sapi:pron="ER rho D N AA T">Urdnot</token>
        <tag>Alias1</tag>
      </item>
      <item>
        <token sapi:display="WREX" sapi:pron="RA EH K S">Wrex</token>
        <tag>Alias2</tag>
      </item>
    </one-of>
  </rule>
</grammar>
```

Referencing an External Lexicon with <lexicon>

You can use the optional lexicon element to direct NuiSpeech to use custom pronunciations residing in a separate title-provided lexicon for certain words or phrases. If used, the lexicon element should occur in the grammar XML file before any rule declarations. For example:

```
<lexicon uri="Assets/GameLocations.pls"/>
```

Lexicon Files

Lexicons are XML files, as are the grammars. You can use the generic file extension .xml, though a common alternative is .pls, named for the Pronunciation Lexicon Specification from the W3C. When you use custom pronunciations that reside in a separate lexicon file, that file should be wrapped in a <lexicon> element in the referencing grammar. Lexicons should consistently use an <xml> prologue to specify the encoding style. This allows for more consistent support of non-Western alphabets, such as Japanese.

Note that even separate lexicon files are embedded into a grammar as part of its compilation. Lexicons are not compiled separately.

Following is an example of a lexicon that allows NuiSpeech to interpret the pronunciations given in the preceding grammar-embedded custom pronunciation.

```
<?xml version="1.0" encoding="utf-8"?>
<lexicon version="1.0"
  xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.w3.org/2005/01/pronunciation-lexicon
http://www.w3.org/TR/2007/CR-pronunciation-lexicon-20071212/pls.xsd"
  alphabet="x-microsoft-ups" xml:lang="en-US">
  <lexeme>
    <grapheme>Urdnot</grapheme>
    <phoneme>ERR D N AO T</phoneme>
    <phoneme>ER rho D N AA T</phoneme>
    <!-- illustration of multiple acceptable pronunciations -->
  </lexeme>
  <lexeme>
    <grapheme>Wrex</grapheme>
    <phoneme>RA EH K S</phoneme>
  </lexeme>
</lexicon>
```

In lexicon files, several attributes should be used with the <lexicon> element:

- version**
This is required and should be set to 1.0.
- xml-lang**
Required. This attribute specifies the language of the lexicon, and it should match the initialization parameters of NuiSpeech. For more information, see the table in [Specifying a Grammar with <grammar>](#).
- xmlns="http://www.w3.org/2005/01/pronunciation-lexicon"**
Required. This attribute specifies the namespace.
- alphabet**
Required. This attribute should be set to "x-microsoft-ups".

Specifying a Lexicon with <lexeme>

The <lexeme> element wraps the lexicon equivalent to a grammar rule. It contains one or more <grapheme> elements (spelled words) corresponding to a single <phoneme> element (pronunciation for the spelled word).

Spelling Words with <grapheme>

The <grapheme> element provides the literal spelling of the word to use a custom pronunciation. A title can also use multiple <grapheme> elements with a single <phoneme> element to allow for spelling variations, though this is less typical for games, where the title is controlling which spellings are used in its grammars.

Providing Custom Pronunciation with <phoneme>

The <phoneme> element provides the custom pronunciation for the preceding <grapheme> element or elements. Your titles can use multiple <phoneme> elements within a <lexeme> element to provide alternative pronunciations, as in the preceding sample lexicon. Phonemes in NuiSpeech should use the SAPI Universal Phone Set (UPS), which is based on the IPA. Listings of the Universal Phone Sets for NuiSpeech-supported languages can be found in [Custom Pronunciations](#).

Wrap Up

Speech is a powerful and potentially magical part of the gameplay experience, allowing for the opportunity of intuitive hands-free interaction with the characters and environments of a title. This document attempts to describe the basic XML elements required to develop a grammar file; please see the full specification from W3C for more information on advanced functionality and detailed rules for each element not fully covered here.

With an understanding of basic grammar construction, developers can turn their focus to an equal challenge of speech: designing for ambiguous, incomplete, or uncertain results, and responding to the user's intent, even if specific meaning can't always be gleaned. We look forward to the solutions that you implement in this area, and we encourage you to participate in the forums on Game Developer Network ([forums.xboxlive.com](#)) to share any comments or questions about this white paper or the speech recognition technology of the NuiSpeech API.



and David Trainor, Software Development Engineer

Speech at Microsoft

Published: May 6, 2010

Updated: November 10, 2011

Introduction

Grammars contain a list of words and phrases that the NuSpeech engine should recognize in a particular context. When developing a speech-enabled game, developers can use *grammars* to elicit speech input from the user, and then use the textual interpretation from the recognizer to affect the game environment or the user.

Predicting what a user will say at any instance is a challenge. Speech input is unlike traditional controller-based input in that the input may contain extraneous phrases, be a synonym of the word that is expected, or have enough noise to be unintelligible. Fortunately, the [W3C grammar specification](#) has a number of constructs and features that help alleviate some of these issues. Many of the features most specifically applicable to speech recognition in games are documented in [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Grammar development is still an emerging art, and some tricks speech application experts use to achieve high recognition rates in their applications are not self-evident. Also, the design of the game can heavily influence what users will say. Close collaboration between game designers and grammar developers can help create a successful speech-enabled game. The goal of this paper is to describe how a developer would speech-enable a familiar gaming scenario and solve some common issues that arise by using techniques used in existing speech applications.

What're Ya Buying?

The NuSpeech recognition engine expects two inputs: a spoken phrase and a textual grammar. In many games, a user may happen upon a merchant selling various weapons or armor, and the merchant may ask the user, "Would you like to see my wares?" This interaction could be easily speech-enabled by passing a recording of the user's voice and a grammar to the recognition engine to find out if the user answered *yes* or *no*.

Grammar Sample 1: A simple confirmation grammar

```
<?xml version="1.0" encoding="utf-8"?>
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="yesOrNo"
  version="1.0">
  <rule id="yesOrNo">
    <one-of>
      <item>yes</item>
      <item>no</item>
    </one-of>
  </rule>
</grammar>
```

The recognition engine will return either "yes" or "no" as the text result, and the game engine may either initiate a transaction UI, or close out the conversation accordingly.

Best Practice

Note the use of the specific prompt from the merchant. Developers should use prompts to coax the user to choose from a constrained set of options. Typically, the smaller the choice of grammar, then the better the overall recognition performance. "What would you like to buy?" is an open-ended prompt and requires a much bigger grammar to support the options that the user might say. Compare that question to "Would you like to buy weapons or armor?" which would require a much simpler, and, therefore, a better performing grammar.

Handling Yup

After play testing the merchant interaction, a developer may find that the recognition performance of their grammar is inadequate, and an analysis of what users actually may say reveal many could say "yeah."

The grammar tag `<one-of>` can be used to specify synonyms for the same thing—"yes" in this case. Similarly, `<tag>` is used to specify which single result to return for a given set of `<one-of>` tags.

Grammar Sample 2: Using both together, a grammar that specifies a number of different options for "yes" and "no"

```
<?xml version="1.0" encoding="utf-8"?>
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="yesOrNo"
  version="1.0">
  <rule id="yesOrNo">
    <one-of>
      <item>
        <one-of>
          <item>yes</item>
          <item>yeah</item>
          <item>yup</item>
        </one-of>
        <tag>yes</tag>
      </item>
      <item>
        <one-of>
          <item>no</item>
          <item>nope</item>
          <item>nah</item>
        </one-of>
        <tag>no</tag>
      </item>
    </one-of>
  </rule>
</grammar>
```

Best Practice

Developers may feel the need to overload the grammars with as many synonyms as they can think of for a particular word. However, one should only add phrases to the grammar that are discovered through play testing, and even then, should add only the most common synonyms encountered. The trade-off, generally speaking, is that adding more synonyms can reduce the recognition performance of the existing entries, so adding a rarely uttered synonym for "yes" (like "affirmative" or "aye") doesn't buy you much coverage at the expense of overall recognition performance of the entire grammar.

It Depends on the Context

Once the user has bypassed the initial confirmation and started shopping, there will be a number of different interactions that the user can have with the merchant. At this point, detecting if the user said *yes* or *no* is unnecessary; instead, we want to know if the user wants to buy or sell items. Switching active grammars by context is a very important technique to improve recognition performance, because each grammar is smaller and you're not attempting to recognize phrases that don't apply to the current interaction.

After unloading the confirmation grammar (sample 2), the next stage in the shopping process is to find out what the user wants to do.

Grammar Sample 3: A typical grammar if the shopkeeper allows both buying and selling items, and deals in weapons, armor, and spells

```
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
  <rule id="Main" scope="private">
    <one-of>
      <item>sell armor</item>
      <item>sell weapons</item>
      <item>sell spells</item>
      <item>buy armor</item>
      <item>buy weapons</item>
      <item>buy spells</item>
    </one-of>
  </rule>
</grammar>
```

Grammar Maintenance

As seen in Grammar Sample 3, things start to get complicated as you add additional actions and categories of items. As the merchant adds more actions (repair) or starts selling more categories of items (potions), the number of phrase permutations grows in size.

Grammar Sample 4: The solution is to use control flow via XML to separate the action (verb) from the category (noun)

```
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
  <rule id="Main" scope="private">
    <one-of>
      <item>sell</item>
      <item>buy</item>
    </one-of>
    <one-of>
      <item>armor</item>
      <item>weapons</item>
      <item>spells</item>
    </one-of>
  </rule>
</grammar>
```

This solution helps to avoid needing to define every permutation in the grammar. As grammars become more complex, developers may see the need to further separate the control flow from the data. Grammars can be separated into subgrammars by using `<ruleref>` tags that refer to other rules.

Grammar Sample 5: Using sub-grammars to make grammar maintenance easier

```
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
  <rule id="Main" scope="private">
    <ruleref uri="#transaction_type" />
    <ruleref uri="#transaction_category" />
  </rule>
  <rule id="transaction_type">
    <one-of>
      <item>sell</item>
      <item>buy</item>
    </one-of>
  </rule>
  <rule id="transaction_category">
    <one-of>
      <item>armor</item>
      <item>weapons</item>
      <item>spells</item>
    </one-of>
  </rule>
</grammar>
```

Global Grammars

The NuiSpeech recognition engine supports multiple grammars being loaded and active simultaneously. For example, when a player enters a town in a game, the title may load all of the townsfolk's grammars into the engine, but activate only the grammar for a particular person when the player hears them. For best recognition performance, a game should try to have only one active grammar when performing a recognition, but there is often a need to always have a global grammar active. For example, "pause game," "save game," and "options" could be globally speech-enabled by putting them in a grammar that is active at all times.

Best Practice

Be wary of adding too many commands to a global grammar, particularly if the phrases can be confused with other contextual grammars. For example, adding a global command such as "exit game" should be weighed against the possibility of accidentally misrecognizing it.

Giving the User Options

Now that the categories of merchandise are separated from the actions the user can take, it becomes more difficult to include both full phrases and subphrases for recognition. For example, the user might say "buy" and expect to hear a verbal reply listing all available merchandise categories or to see a list of everything for sale. That particular grammar would omit the "transaction_category" rule from the "Main" rule. The current grammar, however, does have the "transaction_category" rule. Instead, developers can make grammar components optional to convey to the recognizer that a user may omit a particular item.

Grammar Sample 6: The "transaction_category" (containing weapons, armor, and potions) is optional

```
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US">
```

```

        root="Main"
        version="1.0">
<rule id="Main" scope="private">
  <ruleref uri="#transaction_type" />
  <item repeat="0-1">
    <ruleref uri="#transaction_category" />
  </item>
</rule>
<rule id="transaction_type">
  <one-of>
    <item>sell</item>
    <item>buy</item>
  </one-of>
</rule>
<rule id="transaction_category">
  <one-of>
    <item>armor</item>
    <item>weapons</item>
    <item>spells</item>
  </one-of>
</rule>
</grammar>

```

Optional rules open the door for many other grammar improvements, including dealing with synonyms where extra words are added or omitted. For example, Grammar Sample 6 will not properly recognize a phrase like "buy some weapons" even though the game would probably consider it the same as "buy weapons." The word "some" will cause a false recognition. One solution would be to add "some" to the end of the "transaction_type" rule or to the beginning of the "transaction_category" rule and create every possible permutation. A simpler and more elegant solution, however, would be to add an optional item between the two rules that holds the word "some." These kinds of words may or may not provide useful information to the game about the intent of the user, but they help cover the ranges of what people might say.

Grammar Sample 7: Adding the optional word "some" makes both "buy weapons" and "buy some weapons" available speech options for the user

```

<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SGSEExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
<rule id="Main" scope="private">
  <ruleref uri="#transaction_type" />
  <item repeat="0-1">some</item>
  <item repeat="0-1">
    <ruleref uri="#transaction_category" />
  </item>
</rule>
<rule id="transaction_type">
  <one-of>
    <item>sell</item>
    <item>buy</item>
  </one-of>
</rule>
<rule id="transaction_category">
  <one-of>
    <item>armor</item>
    <item>weapons</item>
    <item>spells</item>
  </one-of>
</rule>
</grammar>

```

Best Practice

Use caution when adding filler words and add only ones commonly seen during play testing. Adding optional branches, such as shown in Grammar Sample 7, complicates the recognition process and should be used only to help very common use cases.

Dealing with Infinite Possibilities

While adding optional phrases helps capture all valid utterances, there are an infinite number of things a user can actually say while playing a game. Whether the user is talking to their friend or has music playing in the back room, unintended noise may make its way to the recognition engine, despite noise suppression within the Kinect sensor array. The NuiSpeech recognizer will take these extraneous sounds and try hard to make them match what is in the currently-active grammars. Indeed, if a developer were to use the grammar listed in Grammar Sample 7, the recognizer would occasionally pick up extraneous speech or noise and return successful recognitions.

It is impossible to completely remove all recognitions from noise. However, developers can mitigate the problem by adding a special rule called "garbage." The garbage rule is a catch-all where the recognizer completely ignores a section of audio and returns "... " as the recognized element. This rule, like any other grammar rule, can be placed between two grammar items or in parallel with grammar items. In Grammar Sample 8, putting a garbage rule in parallel with the rest of the grammar catches any arbitrary phrases, completely bypassing the rest of the grammar. Now when a user says something that is not in the grammar, the garbage rule will catch it and return "..." to the system. Developers can ignore "..." if they choose and reduce the number of noise recognitions.

Grammar Sample 8: Adding a garbage rule in conjunction with other grammar helps catch arbitrary phrases

```

<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SGSEExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
<rule id="Main" scope="private">
  <one-of>
    <item>
      <ruleref uri="#transaction_type" />
      <item repeat="0-1">some</item>
      <ruleref uri="#transaction_category" />
    </item>
    <item>
      <ruleref special="GARBAGE" />
    </item>
  </one-of>
</rule>
<rule id="transaction_type">
  <one-of>
    <item>sell</item>
    <item>buy</item>
  </one-of>
</rule>
<rule id="transaction_category">
  <item repeat="0-1">
    <one-of>

```

```
<item>armor</item>
<item>weapons</item>
<item>spells</item>
</one-of>
</item>
</rule>
</grammar>
```

Making Up New Words

Interacting with the merchant is now speech-enabled! Or is it? Now the merchant decides to sell "Acheriablacn," a category of ore only found in the mythical world of some amazing Kinect-enabled title. The NuiSpeech recognizer contains a lexicon file, which includes pronunciation information for many words (as well as helps the recognizer guess the pronunciation for other words). However, the recognizer does not always get everything right. In this case, the way that users and game characters pronounce *Acheriablacn* does not always match up with the way that the recognizer interprets by default. To solve this problem, a custom pronunciation for the word *Acheriablacn* needs to be provided.

To provide the recognizer with custom pronunciations, a symbolic representation of the word needs to be created. These symbols are *phonemes*, which are small distinct sounds people make. By putting together phonemes, developers can create custom pronunciations for words. It is important to note that this paper does not cover how to create phonemic representations of words, but rather how to use phonemes to solve a development problem. For example, a simple phoneme representation of "hello" is "h eh l ow." For a more detailed discussion of phoneme representations of words, see [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Grammar Sample 9: Including a phoneme

```
<grammar xmlns="http://www.w3.org/2001/06/grammar"
  tag-format="semantics/1.0-literals"
  sapi:alphabet="x-microsoft-ups"
  xmlns:sapi="http://schemas.microsoft.com/Speech/2002/06/SRGSExtensions"
  xml:lang="en-US"
  root="Main"
  version="1.0">
  <rule id="Main" scope="private">
    <one-of>
      <item>
        <ruleref uri="#transaction_type" />
        <item repeat="0-1">some</item>
        <item repeat="0-1">
          <ruleref uri="#transaction_category" />
        </item>
      </item>
      <item>
        <ruleref special="GARBAGE" />
      </item>
    </one-of>
  </rule>
  <rule id="transaction_type">
    <one-of>
      <item>sell</item>
      <item>buy</item>
    </one-of>
  </rule>
  <rule id="transaction_category">
    <one-of>
      <item>armor</item>
      <item>weapons</item>
      <item>spells</item>
      <item>
        <token sapi:pron="AA SH EH R AH B L AA K AH N">Acheriablacn</token>
      </item>
    </one-of>
  </rule>
</grammar>
```

Wrap Up

Based on initial inspection, grammar construction can appear to be relatively simple, but even minor modifications in flow or content can have significant effects on the final recognition performance. This white paper outlined a number of recipes that can be used to solve common scenarios found in gameplay. For more details, see the [W3C Speech Recognition Grammar Specification](#) and [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).

Resources

- W3C Speech Recognition Grammar Specification: <http://www.w3.org/TR/speech-grammar/>
- W3C Pronunciation Lexicon Specification: <http://www.w3.org/TR/pronunciation-lexicon/>
- [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#)
- [Speech Design for Game Designers: An Overview](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360



by Scott Selfon, Lead Program Manager
Advanced Technology Group (ATG)

Grammar construction	Impact	Comments
Target 10 or fewer active phrases for any given game context	Having fewer active phrases leads to better recognition accuracy and clearer constraints for the user.	Change context based upon game state, player's past speech, gestures, and so on. Titles can enable & disable individual rules dynamically.
Typically avoid the <i>garbage</i> rule	Doing so decreases false acceptances, but it increases false rejections and CPU utilization.	It is better to understand and tune for confidence thresholds and to use selective engagement models (gating keywords or gameplay activation).
Use phrases with two or more syllables	Longer phrases lead to more robust recognition and reduce false acceptance of something outside of the grammar.	Balance length with ease of utterance. Avoid unnaturally long phrases.
Provide custom pronunciations where appropriate	Increase recognition for uncommon or invented words and phrases.	NuiSpeech makes an educated guess, but to maximize success, consider providing pronunciation for uncommon words.
Consider localization during design	Use of semantic tags rather than specific words can reduce code changes, because the recognition code path remains	Leverage SRGS tags for branching rules (see Grammar School 101). However, be aware of potential differences in phrase construction across languages, and when

	consistent across languages.	localizing, use spoken (versus written) constructions.
Game design	Comments	
Target a 90 percent rate of <i>user-perceived</i> successful recognition	This is often accomplished as much through game design as grammar construction: use context-sensitive recognition, act intelligently for false rejections or low-confidence values, incorporate feedback from usability and play testing.	
User education & training	Tutorials, training, and visual/auditory feedback when the title is actively listening for the player to say something.	
Engagement model	There are a variety of potential options for <i>when</i> —key player actions, key player spoken phrases, context-based, and so on—and <i>how</i> —as a listening character, a menu system, and so on—the system listens for and accepts speech.	
Provide appropriate direction	For constrained grammars, consider supporting synonyms. For broader grammars, keep commands intuitive and consider ways to communicate options to the player.	
Provide feedback to the player	Communicate both successes and failures in intuitive ways. Consider using context to maximize perceived successes. Consider confirmations in critical situations or when confidence is low.	
Don't compete with the player	In scenarios where speech is expected, consider reducing game-generated sounds (particularly in the speech frequencies).	
Combine speech with gesture	Give the player alternate avenues for performing actions when speech or gesture is not recognized.	
Potential pitfalls	Examples	Mitigations
Very short phrases	Phrases comprised of individual digits, letters, or other single-syllable words (<i>pause</i> , <i>start</i> , and so on).	Consider longer phrases. Use garbage rule in grammars to reduce some false positives.
Broad or large active rules	The alphabet; multiple synonyms for multiple enabled actions.	Look for ways to reduce active scope based on context.
Ambiguous or confusable phrases	Phrases that differ by few sounds, or are wholly or partially contained within each other.	Evaluate available words (across all localizations) in any given game context.
Word spotting	Surrounding interested action words with the garbage rule.	Train players to speak longer commands; rather than depend on key words that could be missed.
Speech as an afterthought	"Speech-ifying" menus without evaluating their grammars, flow, or error handling.	Consider and thoughtfully plan functionality by using these best practices.

For more information about NuiSpeech and creation of grammars, or for specific assistance for your title's scenarios, send e-mail to gameds@microsoft.com, see the white papers on [Game Developer Network](#), or consult the XDK documentation.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Speech Design for Game Designers: An Overview



by Meg Niman, User Experience Designer
Speech at Microsoft

Introduction

Speech is a natural way for people to interact. As speech is an integral part of our everyday interactions, adding speech to gameplay creates a more immersive and realistic experience. It brings the game experiences to life. Paired with gesture, speech creates incredible opportunities for game development. The benefits of this lifelike experience also create design challenges. This paper has several objectives. It presents ideas that should help you, as a game designer, determine where speech will work well. It explains how you can think about speech design. Finally, it recommends some methods for working with speech constraints. Throughout, it offers principles, examples, tips, and a couple of small case studies to help a game designer incorporate speech. At the end, there is a summary reference sheet of the principles and tips.

Key Speech Terms

The following terms are important for a basic understanding of how speech applications work. By understanding these key elements, a game designer can maximize speech's potential while working within its inherent constraints.

- *Speech commands/phrases* are the words the speech recognition engine, also known as the recognizer, listens for, which are each mapped to an action. For example, a command could be "come here," which would cause one character to approach another.
- *Synonyms* are alternate ways to say a command. Supporting multiple synonyms may improve overall speech success, since alternate ways of saying a command will be recognized.
- *Context* is the environment, state, and circumstances of the game that affect what a player can say and do.
- A *Grammar* is a list of rules for the words and phrases that the NuiSpeech engine may recognize in a given game situation. NuiSpeech grammars originate as XML files that implement a subset of version 1.0 of the speech recognition grammar specification of the World Wide Web Consortium (W3C). For more details, see [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#).
 - In other words, grammars are speech commands and their synonyms. For the speech recognizer to recognize any word or phrase, the word or phrase needs to be in a grammar. Multiple grammars can be active at once and you can activate different grammars based on context.
- *Speech errors* occur when the recognizer does not recognize correctly what was said. The different types of speech errors are discussed in greater detail in the "Design error handling into the game" section.
- *Side speech* is when the recognizer hears speech not intended for recognition (the player and/or other people are talking, but not to the Xbox).
- *Background noise* is from other sounds in the room like a fan whirring, a dog barking, and so on. Kinect for Xbox 360 will compensate for many noises, but may interpret some of these as speech and try to recognize the sounds.

Technical Considerations

The following technical details should be considered when designing for the initial release of Kinect for Xbox 360.

- One speaker can be active (recognized) at a time.
- For more robust recognition, the best practice is to limit the active grammar to 10 commands at once, including reasonable synonyms.
- One language can be recognized by NuiSpeech at a time.
- Best recognition if the player is speaking normally (not yelling, whispering, and so on).
- Player's emotion can affect recognition (may be more difficult to recognize phrases if player is excited or disappointed).
- Background noise can affect recognition. For more information, see [Talk To Me: Microphone and Speech Technology in XDK Kinect Hardware](#). The player will be able to talk and be recognized while game audio is playing at reasonable levels.
- Recognizer will have end-pointing. This means it will be able to determine when the player has stopped speaking. In other words, the player will not have to say something like "stop listening" or "I'm done" after each command.

Identify Speech Opportunities

Creating great speech experiences requires game designers to spend some time thinking about how to incorporate speech into their games. Speech has many benefits and good speech

design can capitalize on these positive aspects.

When designing speech into your game, maintain a focus on the player and the speech experience you create for that player. Rather than trying to use speech everywhere, think specifically about how speech will work in the game and how players will feel when using speech. This will help you find the right places to enable speech recognition.

Principle: Use speech for what it's good at

In order to create a player-focused speech experience, think about *which actions are good candidates for speech*.

Speech works well for things that are difficult to do otherwise (it may be easier to say "show map" than to create a gesture to bring up the map). Speech also can work well in conversations: it's easier to say a phrase than picking it by way of a gesture from a phrase list. Speech can also be magical when it empowers the player or provides a sense of delight. Look for places that will play to speech's strengths and work with its limitations.

Plan for the time it takes to say and recognize a command. Using speech for aspects of game play that need to be fast (especially when related to whether a character will live or die) or for irreversible actions may be problematic. Speech errors are also possible and the player will need time to correct them.

Using speech to access non-critical information may work well. For example, speech could be used to see the contents of a backpack. These actions enhance the gameplay, but are not critical. More exploratory games, or exploratory parts of gameplay where time is less critical, can be great places to incorporate speech.

When determining where to use speech, consider:

- What might feel natural for the player to say?
- When might it feel natural to speak?
- When will it be easier for the player to use speech compared to other inputs?
- Are there contexts (like when there's background noise like other people cheering) that are not good candidates for speech?
- When does the player have time to say a phrase, including possibly correcting speech errors?
- What actions are better suited for gesture?

Another way to determine if speech makes sense for a given situation is to ask: Does misrecognition fit into the storyline and interaction in the game?

Examples where this is true can be good candidates for speech recognition:

- Radio communication, which can break up or have static
- Talking to an entity that doesn't always understand, including alien races or animals
- Talking to another person

Principle: Consider the interaction model

When using speech in your game, it can be helpful to think about who the player is talking to. By allowing the player to talk to another character in the game, you're approximating a real-life conversation. Other in-game entities that may be natural to speak to include non-human characters, a command center, or even high-tech computers in the game world using speech recognition.

The types of interactions where speech works well can be broken into:

- *Conversations*, where the questions and interactions have a constrained set of responses, such that all responses fit within the number of supported concurrent commands. For example:
 - "Would you like to see my wares?" Responses would be "yes," "no," and their synonyms.
 - "Do you want to buy or sell?" Responses would be "buy" or "sell."
 - "I have armor, weapons, and spells. Which do you want?" Responses would be the examples given and possibly "none of them."
- *Command and control*, where the player is issuing orders in a constrained language. For example:
 - "Unit Alpha, move to station Bravo." The unit, action, and position are all a limited set of words.
 - "Sarah, can you lead me to your house?" A conversational request is actually a command directed at another character.
- *Verbalized actions*, where saying a magic word/command causes something to happen that in reality you would physically do. These commands can also be characters' thoughts, which for the gameplay would need to be verbalized. Since these aren't phrases that someone would actually say, you may need to carefully word the commands so they sound cool/feel good to say out loud.
 - "Switch weapon" or "sword." If not playing a game, you would reach for the weapon you want instead of saying it out loud.
 - "Lucky rabbit's foot." The desired object is named instead of reaching into a backpack and grabbing it.
- *Meta-commands*, where the player is addressing the game. For example:
 - "Pause game."
 - "What do I do now?" and "Help" could fall into this category. These Help commands could also be handled conversationally, where the player says, "Sarah, can you help me?"

Case Study: "Open Door"

"Open door" may seem like a great speech command. It's easy to say and someone would say it in real life. Or would they? I might say, "Noah, open the door." Or I'd just grab the handle and open it. Perhaps this action would be better handled with gesture.

Let's say you enable a gesture to open the door. You could also enable a command to accompany the gesture, like saying "Anybody home?" or "Hello?" as the player opens the door. Alternately, you could have a few options: say "Knock, knock," make knocking gesture, or both say the command and make the gesture. If the speech recognition engine hears the player say "knock knock" and recognizes the gesture of knocking on the door, the game may have more confidence that the player really is trying to do the action Knock.

For speech, "Open sesame" might be a good command, since it is fun to say and, when you say it, the door magically opens.

Design for Success

When designing for speech, the success of the experience can be measured in part by how often the game does what it should in response to speech (and other noises). Ideally, it will recognize correctly what the player says when the player speaks a command and will subsequently perform the desired action. It will also ignore sounds that aren't speech. Hopefully, the player won't frequently try to say something that the game doesn't have in its active grammar.

The phrasing of the command—what you expect and possibly teach the player to say—can affect the speech recognition. If the phrases in grammar are not the way players will actually try to say the commands, then you will get speech errors instead of recognition success. Since the recognizer can only match the player's speech to items in the active grammar, it's important that the grammar contains the right words.

Principle: Write the way a player would talk

Speech commands often work best when *written in the way a player would say them, not in the language of game designers/developers*. For example, while designing and developing a game, an action may be called "exit vehicle." However, a player probably thinks of this as getting out of a car. So instead of making "exit vehicle" the command, you could choose the phrases "get out of the car" or "ditch the car." By using commands that are intuitive for players, you reduce their need to think about *how* to say the commands. This is also a benefit compared to a controller, where actions are arbitrarily assigned to buttons. Instead, you can now use a more natural phrase to verbalize the action.

Tip

When writing a speech command, try saying the phrase out loud. Also say it to someone else or when other people are nearby. If it feels strange or silly to say the command aloud, think of rephrasing or possibly not enabling that command with speech.

Principle: Minimize confusion (for the recognizer and the player)

When writing speech commands, it's important to consider the recognition accuracy for the phrases you create.

To create good speech commands, follow these guidelines:

- *Multi-syllabic phrases generally perform better than single syllable commands.*
- *Good speech commands are phonetically distinct (don't sound alike—not confusable).*

Phonetically distinct phrases contain different speech sounds, which minimizes confusability. This can be hard to discern by simply looking at the words, since you're also concerned with sounds that are similar (like the "r" and "v" in the words "fast" and "vast"). Creating a grammar and testing it to see if any commands frequently result in speech errors, especially false accepts, can help you know which phrases to tweak.

Principle: Include reasonable synonyms (heard during play testing)

When thinking about the phrases to support, you should also consider reasonable synonyms. These are not *all* the different ways that someone could express a command, but instead you are trying to identify the handful (or fewer) ways that most people would express the command. User research/play testing is the best way to determine the most natural way of speaking a command.

Tip

Testing your game with real people can help unearth synonyms to add to the grammar.

Prompting the player to say a specific command will not guarantee that the player will actually say that command. Often players will vary the wording slightly without realizing it. For example, for the command "buy supplies," you may also want to include synonyms like "buy some supplies" and "buy more supplies." Each of the variations of "buy supplies" can be mapped to the Buy Supplies action.

Tip

You may have more success by adding a few synonyms to the grammar than by trying to change what the player says.

When considering synonyms, you should consider if the synonyms are confusable with other phrases active in grammar. However, if multiple phonetically similar phrases all perform the same action, then the phonetic confusion between those synonyms isn't an issue. For example, if the player wants to sell weapons, the grammar could have the phrases "sell weapons," "sell my weapons," and "sell a weapon." Even though all three phrases contain the words "sell" and "weapon," there won't be a confusability problem since they all trigger the Sell Weapon action. As a counter example, if you have the phrase "jump" and add the synonym "jump up" in the same grammar where you also have the command "dump cup," you may have confusability issues between "jump up" and "dump cup."

There's also a balance between supporting appropriate synonyms and overloading the grammar. If you include too many items in grammar, you increase your chances of speech errors. The goal is to cover the majority of players with the minimum number of synonyms. The general rule is you can have up to 10 active phrases in grammar, including synonyms.

Tip

Often a few of the most frequently used synonyms cover the majority of players. The frequency of synonyms has a long tail.

Principle: Help players know what to say

While speech is a natural interaction, players will still need to know what they can say. Depending on the game play, you can introduce speech commands in different ways. Some places where speech commands can be exposed include:

- *Using another entity* to help the player. This can be as instruction about what can be said or can be woven into conversation.
- *Using a narrator* to help the player know what to say.
- *Displaying relevant commands* on screen in text.
- *During game setup*.

When teaching the available commands, often a "just in time" strategy works well, where the relevant and useful command is exposed to the player at a time when the player can actually use that command or immediately after using an alternate method. For example, if a player has just chosen a phrase for the character to say from a phrase list with gesture, the game could give the player a tip or hint that the player can also say that phrase using his or her own voice.

Since it is recommended that your active grammar have 10 or fewer phrases in it, you will likely need to guide the player to say the commands you're expecting (the ones in grammar). Otherwise, it may take a lot of trial and error (and player frustration) to figure out the right command to trigger the desired action.

Principle: Use gameplay to indicate speech recognition

Giving players feedback about whether their speech is recognized is key to creating a successful speech experience. The following are some techniques to indicate speech recognition to the player:

- Take action (like displaying a map if the player says "show map")
- Another character responds appropriately
- Audio output (tones/sounds associated with the action)
- Confirmation (display text on the screen or play audio confirming the recognition)

Taking action is usually the quickest and most effective way to indicate recognition. Often a change on screen (switch to a new screen, process of buying supplies begins, and so on) is enough for a player to know the command was recognized. Similarly, the lack of change is quickly interpreted by the player as a speech error.

Principle: Provide feedback to help the player improve recognition

Just as it's important to let players know that a command was recognized, players need feedback to help them troubleshoot speech errors.

Feedback that shows:

- Speech is too loud helps the player know to lower the volume of his or her voice.
- Background noise is interfering with recognition indicates to the player that he or she needs to make the room quieter.
- Feedback that shows when:
 - Speech is too loud helps the player know to lower the volume
 - Background noise is interfering with recognition
 - Speech is not detected (which may be a lack of usual feedback for speech sounds)

Feedback mechanisms include:

- Showing an equalizer or wave forms that are mapped to the volume of the player's speech (which could be implemented by looking at the amplitude of the signal coming from the Kinect sensor array).
- Changing an on-screen character's expression, like tilting the head, raising the eyebrows, and so on.
- Audio output (short tones and other sounds).

Ideally, any feedback is subtle, so it doesn't distract from the game play.

Principle: Use confirmation sparingly

In speech, there are generally two ways to confirm recognition: *implicit confirmation* and *explicit confirmation*. Both forms of confirmation should enable the player to correct the recognition if it is incorrect.

Confirmations, especially explicit confirmation, can feel like a "road bump" to the player. Rather than moving forward based on a command, the gameplay is stalled in order to confirm the player's desired action. Because they can negatively affect the perception of forward progress, confirmations that add more time or create more steps should be used only when absolutely necessary.

Implicit confirmation is when the recognition is implied by what the game does. Implicit confirmation can be woven into the gameplay, where the action resulting from the speech command is clear to the player and that action serves as a confirmation that the command was understood. A character (or text on screen) paraphrasing the recognized action can also be an implicit confirmation. For example, if a character says, "Sarah, can you lead me to your house?" Sarah could say, "Yes, my place is just around the corner."

Explicit confirmation is when the player is explicitly asked if he or she wants a certain action to be taken. These types of confirmations can feel like an extra step, slowing the interaction down, so they should be used carefully. Actions that are irreversible, like exiting a game, may be good candidates for explicit confirmation. Many games already use explicit confirmations for these types of actions by asking the player to verify that he or she wants to take an action before completing it. Explicit confirmations can also be woven into gameplay.

Other Questions to Ask

- What are the goals for the game? How does speech fit into those goals?
- How will you introduce the player to voice functionality?
- When can the player speak?
 - Are the speech commands available throughout the game or just in certain contexts? Are some speech commands global (available everywhere) while others are based on the situation (contextual)?

- Is the game always listening for speech or just at certain moments? Does the player need to do a gesture to begin speech recognition? Does the player need to say a keyword, like a character's name, before speaking a command?
- What can the player say?
- What feedback will you provide?
- How do voice commands interplay with gesture? Do some gestures naturally have a speech pairing? Can a combination of speech and gesture be used together to better understand what the player is trying to do?

Work with Constraints

Just as there may be good candidates for speech, there are also times when using speech will have low recognition success. Such a time may result in a bad player experience. Use speech for what it's good at, but also recognize there are limitations.

Principle: Acknowledge inherent limitations of speech

Speech commands take time to say and time to recognize and process. Actions that need really fast responses should not be done with speech. Any time you design a speech command, you have to plan for error, which can also take time to correct. As the grammar becomes more complex, there may be more errors as well.

Case Study: "Cast Iceball"	
"Cast iceball" seems like a fun thing to say. It might make the player feel powerful. The player could say it when the player is fighting a fire-breathing dragon and use his or her voice to defeat it.	
For the gameplay, the player has the following items: an iceball, a fireball, and a spellbook.	
When the player comes upon the dragon, the player would say "Cast iceball," and the dragon's fire would go out.	
But what if there was a speech error?	
If there was a no match (defined below), the player's command wouldn't be recognized. The player may be burned by the fire during the time it takes to say the command again.	
If the recognizer didn't hear the player, the player also probably would be burned.	
If there was a false accept, the speech recognition engine might think the player said "Cast fireball." The dragon's fire would be enhanced and the player could die.	
Given the need for quick action, the negative impact of misrecognition, and the catastrophic result of a false accept, casting iceballs or fireballs with speech may not be a good idea.	

You also may want to think about consecutive commands and how the player would feel when saying them. Prototyping or having other ways to try the commands out yourself may help you decide which ones to enable.

Principle: Design error handling into the game

Plan for recognition errors to occur. There will always be recognition errors, but error frequency may go up as the grammars get larger and more complex. The following are the main types of speech errors and their typical causes.

Error type	Causes
No match	Player spoke something in grammar ("cast fireball"), but the recognizer did not recognize it (<i>false reject/false negative</i>)
	Player spoke something out of grammar ("throw fire torch" which is not a recognized command) (correct reject)
	Side speech or background noise (a dog barking)
No input	Player spoke too quietly ("cast fireball" but nothing happens) (<i>false no input/false negative</i>)
	Player did not speak due to confusion or uncertainty (silence)
False Accept/False Positive	Player spoke something in grammar ("cast fireball"), but the speech recognition engine recognized it as something else ("cast iceball")
	Player spoke something out of grammar ("throw fire torch"), but the speech recognition engine recognized it as something in grammar ("cast fireball")
	Side speech (conversation with a friend) or background noise triggered the recognizer and the speech recognition engine recognized it as something in grammar ("cast fireball")

It may be possible to mask the error by integrating it into the game play. This can be done in a *natural way, like by having another character raise his or her eyebrows*, or by having a character tilt the head if the command is not understood. The game also could simply ignore errors. This is especially desirable if you expect a lot of side speech—doing nothing after an error may be the best strategy.

Alternately, a *more explicit response* to a no match could be a character saying, "What's that?" and waiting for the player to repeat a response.

You may want to reserve the mechanism used to show errors for just speech errors. If you use that same mechanism to impart other non-speech information to the player, then players will not know if the error is because they spoke and were not understood or because something else is going on in the game. In some games this may be okay, but in others it may be difficult for the player to understand what's going on if the same action is used for speech and non-speech events.

Principle: Allow for correction of errors

When speech errors happen, it's important that the player can correct them quickly. *Often players respond to a speech error by repeating themselves*. To handle many of the errors, the game just needs to be prepared for the player to try again.

In the case of a *no input*, the player may have said something, but spoke too softly (*false no input*). The player perceives an error, because he or she was not heard, but the recognizer will not know that the player said something and will not return any speech information. Here too the player will likely just repeat the command. Feedback that nothing was heard can help a player know to speak louder.

In the case of a *false accept, where the game performs the wrong action, the game should allow the player to correct that action*. The tricky part is that the recognizer and therefore the game will not know that the wrong action was taken. In some situations, the correction may be that the player says "go back" or "cancel." In other situations, the player may need to say a different command to get back to the previous state. To support this type of speech correction, the speech recognizer should continue to listen for speech commands for a short while after the recognition has processed.

Optimize the Speech Experience

As you design for speech, you'll need to keep the speech constraints and performance in mind. A speech design that doesn't actually recognize the player is not a successful design.

Generally speaking, you should design to have up to 10 phrases active at one time. The speech recognition works best if those phrases are phonetically distinct (phrases don't sound too much alike). As you increase the number of phrases or increase the phonetic confusion between phrases, the speech recognition performance may drop off quickly. How fast it drops off depends on the specific design. In all cases, you're going to need to have people play the game and speak while playing to observe the performance in order to properly tune the grammars.

Principle: Context, context, context

When incorporating speech into a game, think about the contexts in which you want to use speech. Then think about what commands are relevant to that context. Ideally, you will have 10 or fewer command active at any one time.

However, 10 phrases at one time does not mean you can only have 10 phrases total for the entire game. By thinking about grammars in terms of different contexts, you can identify the key phrases that need to be active at that time. These phrases may be different across contexts, which is how you get from 10 phrases to hundreds of total commands.

For example, in a role-playing game, there may be many different contexts. One context might be that the character is in a store. Another context could be that the player is walking on a path with a companion. While in the store, a person might want to say types of items to buy, like "armor," "spells," and "weapons." When walking on a path with a companion, the player might want to say the companion's name, or ask for directions. The player might also want to say "open spellbook," which would perform a different action from when the player said "spell" in the store. You also would not need to worry about conflicts between the commands "spell" and "open spellbook" because they wouldn't be active at the same time. "Spell" is in grammar while the player is in the store; "open spellbook" is only in grammar when the character is walking around.

You can also use context to add or take away items from the active grammar. For example, if the player had a handgun, but dropped it, you could remove "handgun" from grammar. Since the player no longer has the gun, the player probably won't ask for it. If the player goes to a store and buys a new handgun, you could add "handgun" back into the grammar after the purchase.

Principle: Conduct research to improve the speech experience

Research can take many forms, including:

- *Lightweight user research* by asking other designers and developers about wording choices. For example, to get a new weapon, you could ask people how they would name that action. If most people say "switch gun" instead of "switch weapon," you may want "switch gun" to be the command you expose to players, but support "switch weapon" as a synonym.
- *Formal play testing* where you can bring people into a lab and see *what they try to say* when playing the game to determine where speech is working and where it needs to be improved. You can also test your *methods for exposing the speech options* to see if players know what to say while playing the game. You can also use play testing to gather data for tuning the grammars. More information on this can be found in the grammar and tuning white paper.

Preferably, to uncover any localization issues, this research would be *conducted in each language the game supports*.

Play to the Magical Quality of Speech

Principle: Design experiences that delight the player

As you create your games, there may be ways to make the speech experience whimsical and fun. The specifics will be very game dependent, but sometimes it can just be fun to say certain words or phrases. Delight can also come from an unexpected command or phrase being recognized. Sprinkling these phrases into the gameplay can help create a magical experience.

Tip

Delight comes from the unexpected.

Wrap Up

Designing for speech requires thoughtful planning and careful implementation. There are constant tradeoffs when using speech, but good design can overcome these challenges. In the end, seeing players effortlessly using speech in the game is the biggest reward. Please contact gameds@microsoft.com with any questions or comments about this white paper.

Speech Design for Games Summary Reference Sheet

Design Principles

- Identify speech opportunities
 - Use speech where speech works well
 - Consider the interaction model
- Design for success
 - Write the way a player would talk
 - Minimize confusion (for the recognizer and the player)
 - Include reasonable synonyms (heard during play testing)
 - Help players to know what to say
 - Use game play to indicate speech recognition
 - Provide feedback to help the player improve recognition
 - Use confirmation sparingly
- Work with constraints
 - Acknowledge inherent limitations of speech
 - Design error handling into the game
 - Allow for correction of errors
- Optimize the speech experience
 - Remember: Context, context, context
 - Conduct research to improve the speech experience
- Play to the magical quality of speech
 - Design experiences that delight the player

Tips

- When writing a speech command, try saying the phrase out loud. Also, say it to someone else or when other people are nearby. If it feels strange or silly to say the command aloud, think of rephrasing or possibly not enabling that command with speech.
- Testing your game with real people can help unearth synonyms to add to the grammar.
- You may have more success by adding a few synonyms to the grammar than by trying to change what the player says.
- Often a few of the most frequently used synonyms cover the majority of players. The frequency of synonyms has a long tail.
- Delight comes from the unexpected.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Talk To Me: Microphone and Speech Technology in XDK Kinect Hardware



by Scott Selfon, Lead Program Manager

Advanced Technology Group (ATG)

Published: February 5, 2010

Updated: September 8, 2010

Voice is a key component of play experiences with Kinect for Xbox 360, where it becomes a powerful player control mechanism for navigation and gameplay, and enables voice chat for a wide array of scenarios. As a gameplay control mechanism, voice can be as powerful and dynamic as gestures, and can be used in conjunction with body movement or independently. This paper addresses the fundamental software and hardware concepts of the audio input components of Kinect: a microphone array, signal processing to reduce background noise, a speech recognition title library, and use of audio input data for other title applications.

Note

- Xbox 360 Kinect sensor arrays contain a microphone array. Audio input characteristics and specifications may be subject to change.
- Current libraries may not enable all of the functionality described in this white paper. Some functionality is supported at a system level as system services, which are serviced through Xbox 360 Fall System Updates.

For details on the sensor array's field of view, guidance on expected placement positions, and discussions about the playing environment, see [Natural Environments: Understanding and Developing for the Kinect Play Space](#).

This paper covers the following topics:

- [Targeted End-User Scenarios around Voice Input](#)

- [Configuring an Audio Input](#)
- [The Audio Input Pipeline of Kinect](#)
- [Developing for Speech Recognition](#)
- [Wrap-up](#)

Targeted End-User Scenarios around Voice Input

Audio input is as valid and intuitive as movement input. Ideally suited for many activities, audio can be used in conjunction with or independently from gestures. Also, microphone use doesn't have to be limited to speech recognition. For example, evaluating the volume level can inform the game whether the player is whispering or yelling, and music-oriented titles have demonstrated the value of real-time pitch evaluation for judging a singer's performance.

The microphone array in the Kinect sensor is intended to enable multiple game use scenarios, including:

Voice chat

Like an Xbox 360 Headset or Xbox 360 Wireless Headset, the microphone array can act as a voice chat input device. The array utilizes multichannel echo cancellation (MEC) and other processing to reduce and filter game-generated audio and non-spoken ambient environmental noise from the microphone's audio stream.

Speech recognition

Players can speak words and phrases that are recognized by the title, Guide, or Dashboard. NuiSpeech is the library that enables this functionality for titles.

Raw voice data access

A title can listen to and evaluate voice data after MEC processing has been completed, outside of (or in parallel to) the realm of speech recognition. Volume monitoring and voice emotional analysis are great examples of this, as is detecting non-vocal noises that come from a room—for example, clapping hands in a game show scenario.

Sound position tracking

The microphone array can relate positional information back to the title in terms of the relative angle of incoming audio source(s).

The microphone array does *not* support these features for initial Kinect-enabled titles:

Simultaneous voice chat channels

The microphone array cannot fully separate incoming dialogue from multiple concurrently talking players. Voice chat will therefore be on a single channel, shared by all players.

Simultaneous multiplayer speech recognition

Again, the microphone array cannot separate multiple simultaneous dialogue streams, so two players issuing voice commands at the same time is not a supported scenario.

Noise reduction/cancellation and speech recognition technologies are robust in typical gameplay scenarios. Specifically, intelligible voice chat and speech recognition should be possible even during moderately loud gameplay.

If your title requires high-fidelity voice capture, you may want to adopt conservative practices for game audio mixing. Strategies for this, as well as discussion of the Fall 2010 System Update feature that implements system-wide ducking during party and private chat via Kinect, are discussed in [Talking It Over: Strategies for Making Voice Chat More Intelligible in Dynamic Sound Playback Situations](#).

If your title uses local loopback of voice data, particularly during playback of loud game audio (such as karaoke), you may want to require a headset microphone to avoid potential feedback; the Kinect microphone array can be distinguished from a generic headset for voice chat by using [IsSharedMicPresent](#).

For optimum performance of the microphone array and its noise cancellation features, the user is expected to be positioned in the typical play space (approximately 1 to 3 meters from the microphone array). Noise cancellation and suppression (discussed in the next section) are expected to provide 30 decibels or more of noise reduction.

Configuring an Audio Input

Without Microphone Array Hardware

Titles can programmatically deliver arbitrary audio data to the NuiSpeech API (see [NuiSpeechProcessWaveInput](#)), allowing scenarios such as testing speech recognition by using pre-recorded files rather than a live microphone. Note that aspects of implementation and design that are specific to the microphone array (such as tracking speaker position and multichannel echo cancellation) will not be available in this development configuration.

With Microphone Array Hardware

With the Kinect sensor array, audio quality and speech recognition can be validated against retail behaviors, and functionality that requires an array, including sound position tracking, becomes available.

Multichannel Echo Cancellation (MEC) Calibration

The Xbox 360 Kinect Out of Box Experience (OOBE) and Kinect Tuner both provide an audio calibration tool for the hardware-based *multichannel echo cancellation* (MEC) of the sensor array. MEC is a process by which the known output of the Xbox 360 console is recognized in the microphone's input and is cancelled/suppressed from the recorded bitstream. Calibration should be performed each time that players reposition the speakers or the sensor array, adjust the DSP settings of their amplifier or receiver, or significantly alter the general configuration of their rooms.

The Audio Input Pipeline of Kinect

The Kinect sensor array provides a microphone array. This array is a configuration of four microphones that allows for effective noise reduction, as well as MEC. The sensor array contains the entire 5-channel MEC and noise reduction/cancellation pipeline in the hardware, making it available for titles to use post-MEC audio data at negligible CPU cost.

Figure 1. Audio pipeline for the Kinect microphone array.

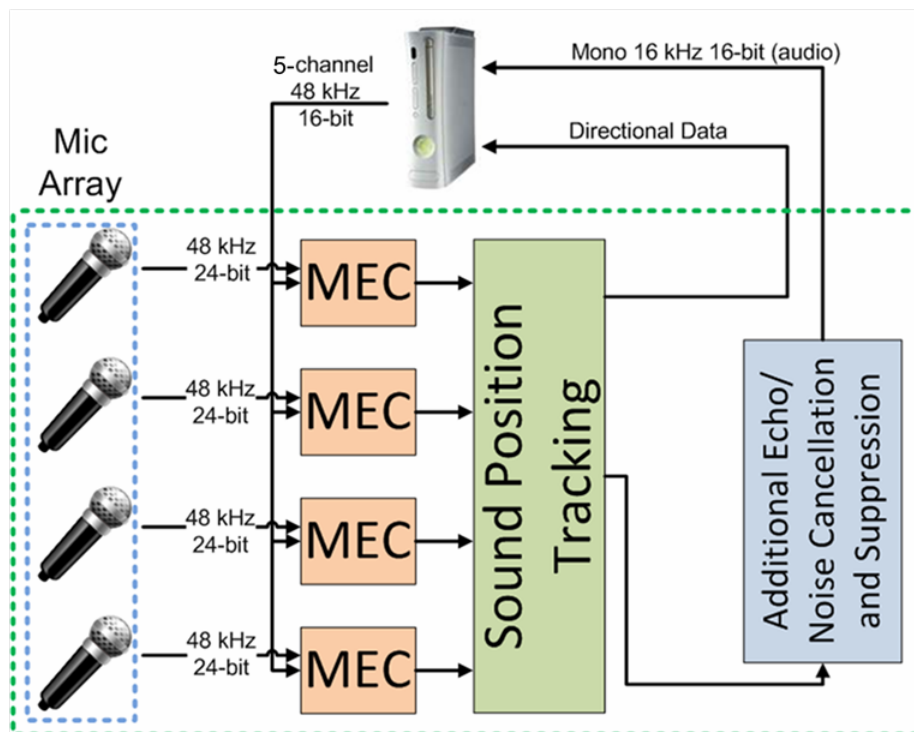


Figure 1 shows the audio pipeline for Kinect hardware (dotted green line) that includes the microphone array (blue dotted line). Each of the four microphones captures audio data at 24-bit resolution. Multichannel echo cancellation (MEC) is applied to each microphone's input. Additional processing allows for sound position tracking functionality, where the general direction of sounds can be reported back to the title. This information also allows for off-axis noise to be further reduced from the voice audio bitstream that is presented back to the Xbox 360 for consumption.

The Kinect sensor array captures audio data at 24-bit resolution. This is in part to allow for accuracy across a wide dynamic range of voice data, from normal speech at three or more meters to the player yelling over a loud game mix. Each microphone has its own MEC applied to further reduce game audio in the recorded channels. The four channels are then coalesced into a single voice bitstream through a process that also exposes sound position tracking.

Sound position tracking evaluates the four audio input streams and determines probable sound sources. Based on differences in the characteristics of the four recordings, the Kinect hardware can report to the title probable directional data for the sounds that the microphones have detected; for more information, see [NuiGetLocalChatData](#). This data is also used to further reduce off-axis background noise. Additional noise reduction is performed as before, and the noise-reduced voice chat data stream is returned to the Xbox 360 in sync with sound position tracking data. Sound position tracking is most accurate when the number of active non-game audio sound sources is restricted.

Developing for Speech Recognition

Xbox 360 speech recognition is implemented using the [NuiSpeech API](#). NuiSpeech is based on [Microsoft Speech Technologies](#)—including both the Microsoft Speech Recognition Engine and the Microsoft Speech API (SAPI)—a mature and robust speech recognition technology used on Windows, mobile devices, and automotive applications.

Once MEC calibration is run by the Kinect OOB or Kinect Tuner for a given environment, the Xbox 360 console is ready for speech-related title development. Speech recognition involves several processes:

1. Development of one or more *grammars*, a set of rules defining which words, phrases, and sentences will be recognized. (This is different from a *lexicon*, which defines the phonetic rules for pronouncing words in a given language or locale.) Grammars are XML files that comply with [the W3C specification for speech recognition grammars](#). [Grammar School](#), a white paper, discusses some typical game-utilized tags and attributes.
2. Grammars are then compiled with the [grammar compiler tool \(xbpg.exe\)](#) into a binary format.
3. A title loads one or more grammars by using [NuiSpeechLoadGrammar](#) or [NuiSpeechLoadGrammarFromMemory](#).
4. NuiSpeech reports confidence values for detected speech tokens (individual grammar elements) as well as complete rules.

For a broader overview of NuiSpeech and speech recognition, see [Grammar School 101: An Introduction to Speech Recognition Grammar Development for Xbox 360 Titles](#) and other speech-related white papers.

Grammar assembly can strongly influence the success of speech recognition, as well as the latency and processing time applied to determine whether spoken text matches a particular rule in a grammar. It is also recommended that you consider designing experiences that give the perception of accuracy even when actual accuracy may not be possible.

Perception depends on the player's context, too. For example, when the screen places focus on a non-player character, that character might respond to any verbalization made by the player, rather than only responding to a recognized pronunciation of the character's name.

Similarly, for a situation where the player needs to evade an incoming attack, consider designing an appropriate response regardless of the specific words the player speaks.

For a set of best practices for speech design and grammar assembly, see [Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360](#) here in the XDK documentation, or download it as a single page for printing from Game Developer Network: [Silver Tongue \(Word 2007 document\)](#).

Wrap-up

The human voice is as natural an input interface as full body interactions are. Speech is a compelling avenue for character interaction with a title, whether for command and control, navigation, or raw audio data evaluation. The presence of an open microphone in the Kinect sensor array provides unique opportunities for voice-driven gameplay.

We look forward to the solutions that you implement for your titles in this area, and we encourage you to visit the forums on Game Developer Network (GDN) with any comments or questions about this white paper or the audio input technology of Kinect.

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

The User Says What? Testing and Tuning Guidelines for Speech Recognition with Kinect for Xbox 360

[See Also](#)



Compiled by Scott Selfon, Advanced Technology Group, and Dennis Mooney, Program Manager – Xbox Speech

While programmatic implementation of speech recognition using NuiSpeech in titles is a relatively lightweight process, the work of validating and tuning the selected phrases represents a potentially broadly scoped project. Thankfully, there are a number of tools provided in the XDK to assist with everything from collecting voice data to simulation, analysis, and recommendations, allowing title developers to feel confident about their use of NuiSpeech confidence values to create robust speech-recognition experiences. This white paper covers the

tools available for testing and tuning speech recognition, preceded with a brief reference guide that provides an overview of strategies for preparing for and conducting testing.

Sections

- [Speech Recognition Terminology](#)
- [Speech Data Collection Goals and Methodology](#)
- [Testing and Tuning Process Flow](#)
- [Strategies for Responding to Poor Speech Recognition Performance](#)
- [Conclusion](#)

Speech Recognition Terminology

This paper uses the following terms and abbreviations.

Term	Abbreviation	Definition
in-grammar	IG	Word or phrase within the currently active set of NuiSpeech rules
out-of-grammar	OOG	Word or phrase not within the currently active set of NuiSpeech rules
correct accept	CA	In-grammar (IG) utterance, successfully signaled by NuiSpeech
correct reject	CR	Out-of-grammar (OOG) utterance, successfully signaled as out-of-grammar
false accept	FA	In-grammar (IG) or out of grammar (OOG) utterance incorrectly signaled as the wrong in-grammar phrase
false reject	FR	In-grammar (IG) utterance, incorrectly signaled as out-of-grammar

Speech Data Collection Goals and Methodology

Creating a consistent and high-quality corpus of collected voice data (often called *utterances*) is key to validating the robustness of grammars. The following table provides some initial guidance for selecting and recording speakers.

Best practices for user selection and data gathering	Best practices for recording environment
1. Native speakers (even for <i>ad hoc</i>) for matched region/locale. 2. Ideally collect voice data from 10 men, 10 women, 10 children per region. (You can skew the survey based on expected player demographic.) 3. Gather >1 utterance of each supported phrase per user. 4. Capture >50% sampling of OOG phrases. (Data collected for one project can supplement OOG phrases for other projects.)	1. Isolated from other audio sources (especially other development kits and talkers). 2. Simulate player environment: living room rather than cubicle or lab; Kinect Sensor in typical location (above or below TV, possibly near center channel speaker). 3. MEC Calibration performed prior to capture and test (available through the Kinect Tuner).

Testing and Tuning Process Flow

Grammar creation is fairly trivial, but how do you know that a grammar functions as expected? How do you know that the unusual words that are specific to your title are correctly matched to the intended pronunciations by users and that phrases aren't confusable? The following table walks through the basic steps of testing and tuning a grammar for use by a production title.

Step	Goals	Tools, samples, and resources	Process, actions, and tips
Ad-hoc testing (IG and OOG)	1. Sanity check (1 native speaker) 2. Flag pronunciation issues 3. Flag confusability issues	• Speech Lab (SpeechLab.exe) (included with XDK recovery) • Custom pronunciations • Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360	In grammar: Speak all phrases per active rule in grammar. Note weaker confidence values (<85%, and in particular <75%) for CAs, as well as FRs—consider alternates, reducing size of grammar, and so on. Out-of-grammar: • Speak expected OOG phrases (what user might say that isn't supported, or is contained in other rules). • Sidetalk testing: Leave NuiSpeech active with grammar and engage in an unrelated conversation (local, via phone, and so on). For both of these out-of-grammar approaches, note FAs, particularly of high confidence; consider adjusting such phrases through alternates, gating keywords, scenario-based rule enabling, and so on.
Ad-hoc testing (cross-grammar)	Flag conflicts and confusability	• Speech Lab (SpeechLab.exe) (included with XDK recovery) • Silver Tongue: Best Practices for Speech Development with Kinect for Xbox 360	Test appropriate combinatorials of grammars expected to be simultaneously active. Note collisions, FAs, and FRs.
Formal data collection	Create corpus of voice utterances that can be tested as grammars are tuned	• NUI Voice Collection (NuiVoiceCollection.exe) (included with XDK recovery) • See "Speech Data Collection Goals and Methodology", above.	1. Create a script of phrases (50% IG, 50% OOG) 2. Bring through users to record phrases, optionally over representative gameplay audio Tip: It's often useful to coordinate voice data collection with gesture/skeletal data collection.
Simulation, analysis, and recommendations	1. Further highlight difficult phrases 2. Determine confidence threshold for a given grammar, using collected data	1. Speech Prep (xbSpeechPrep.exe) ↓ 2. Simulator (Simulator.exe) / Speech Lab (SpeechLab.exe) (XDK) ↓ 3. Simulator Results Analyzer (SimulatorResultsAnalyzer.exe) ↓ 4. Speech Lab Analysis Tool (xbSpeechLab.exe)	The outcome report includes: • Weakest phrase(s) and strategies to strengthen • Recommended confidence threshold (typically allowing >85% successful recognition, <2% FAs for OOG based on collected voice data)
Title-implemented testing of logged & collected data (<i>optional</i>)	Precisely logged capture of each utterance: flag and revisit	• Title, instrumented with <i>retain audio</i> NuiSpeech engine • Speech Retain Audio, XDK sample	1. Present debug feedback on speech recognition status to testers. 2. Document the <i>how</i> of failures: FR, FA, or returned confidence outside of threshold.

Strategies for Responding to Poor Speech Recognition Performance

What should you do when you find words or phrases in a grammar that perform poorly? This table provides a list of the most common and effective strategies for debugging and adjusting either the environment or the grammar to allow for better performance.

Observed behavior	Steps to mitigate
False rejects (FRs) or low-confidence accepts	• Validate listening environment: avoid spaces w/ many speech sources, calibrate sensor, and so on.

- Replace a poorly performing phrase with an alternate term (for example, change "buy" to "purchase")
- Combine the existing phrase with an additive term (for example, change "no" to "select no")
- Lower the global confidence threshold. Risk: May increase FAs.
- Lower the confidence threshold for specific phrase. Risk: May increase FAs if another phrase is similar.
- Use custom pronunciations. Risk: May reduce accuracy for overall audience, so we recommend working with a speech expert.

False accepts (FAs)	• For <i>sidetalk</i>, conversational FAs, consider engagement model and reducing active listening scenarios: activation keyword, scenario-based speech recognition, and so on. • Evaluate robustness of phrases in grammar: too short? Too ambiguous? • Note recorded phrase that triggered and evaluate likelihood of occurrence in title; if frequent, consider replacing accepted phrase with alternate or additive term. • Raise the global confidence threshold. Risk: May increase FRs. • Raise the confidence threshold for a specific phrase. Risk: May increase FAs if another phrase is similar.
---------------------	---

Conclusion

Testing and tuning for speech recognition is a critical aspect of creating successful speech-recognition scenarios that provide a magical experience for a wide range of users. The mature pipeline of tools offered in the Xbox 360 Development Kit allows for many aspects of this process to be automated and measured, but the title developer is responsible for acquiring data from a set of speakers and utterances that is as broad as possible to validate the title's specific needs.

With a robust corpus of data at hand, developers can tune with increasing accuracy and provide robust and consistent behavior to the broad range of users who will interact with the title using their voices. Developers who are interested in maximizing their speech investment should reuse gathered speech data, much as they might reuse other training data. Speech intended for one in-grammar scenario can form perfectly valid test data for false accepts and false rejects in other out-of-grammar scenarios. Developers should also consider collecting audio data while collecting other forms of data (such as gesture recording), so that when users are brought in they provide a broad set of data that informs multiple aspects of natural user input.

- For strategies for setting confidence thresholds, see [Developing NUI Speech Grammars](#).
- For the sequence of using the NUI speech grammar development tools, see [Workflow for NUI Speech Grammar Development](#).
- For more information about NuSpeech and creation of grammars, see [NUI Whitepapers: Speech](#).
- For specific assistance for your title's scenarios, visit the forums.

✦ See Also

[NUI Speech Overviews](#)
[NUI Speech Reference](#)
[NUI Samples: Audio and Speech](#)

© 2013 Microsoft Corporation. All rights reserved.
Send feedback to <https://forums.xboxlive.com/>.
Version: 21256.3

Vocal Clarity: Voice Chat Codec Updates for the Fall 2010 System Update

By Scott Selfon, Lead Program Manager
Advanced Technology Group (ATG)
Published: June 15, 2010
Updated: October 6, 2010

Voice is a powerful input mechanism for Xbox 360 titles, allowing for visceral real-time communication with other players in cooperative and competitive settings. The quality of voice data previously available has been constrained because of challenges with low latency and low bandwidth requirements for the Xbox 360 system reservation. With technology improvements and increasing prevalence of higher bandwidth player connectivity, updates to the voice chat codec become both practical and compelling. Additionally, the microphone array in the Xbox 360 Kinect Sensor offers an opportunity for ubiquitous higher-quality voice chat data. This white paper details pipeline and title implications for the voice chat codec updates occurring in the Fall 2010 System Update, and available for development in the June 2010 XDK. The paper also provides more detail on voice chat acoustic characteristics for titles to use when mixing what is perhaps the most dynamic audio of all—speech coming from other players in a game session.

For more discussion about typical player bandwidth characteristics, see [The Real World: Console Latency and Bandwidth on Xbox LIVE](#) white paper.

Basics of Voice Chat on Xbox 360

From a title perspective, voice chat audio data management is implemented via the XHV2 (Xbox High-level Voice) library, or in a more abstracted manner via QNet (which in turn uses XHV2). For XHV2 use, the title retains responsibility for managing and prioritizing the transmission and reception of voice chat data from the Xbox LIVE service.

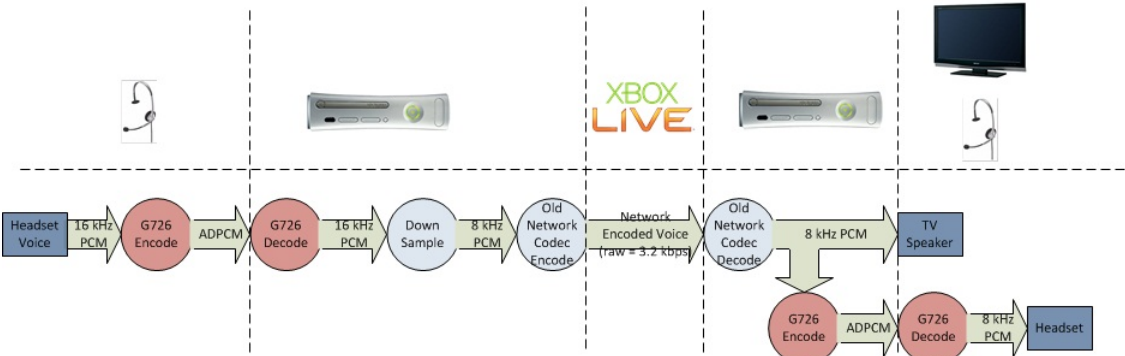
In addition to this *in-game chat* functionality, there are several other consumers of Xbox 360 voice chat technology, both of which use existing XAM resources (in addition to network bandwidth during chat) to enable consistent cross-title system features.

- **Party Chat** is a system feature allowing players to chat with a group of friends, even across titles. Party Chat is managed through the Xbox Guide.
- **Private Chat** is a system feature allowing players to chat in one-on-one conversations with other Xbox LIVE users in up to four separate, isolated conversations.

For more information about these features, see <http://support.xbox.com/support/en/us/nxe/xboxlive/xboxlivecommunity/chat/chat.aspx>.

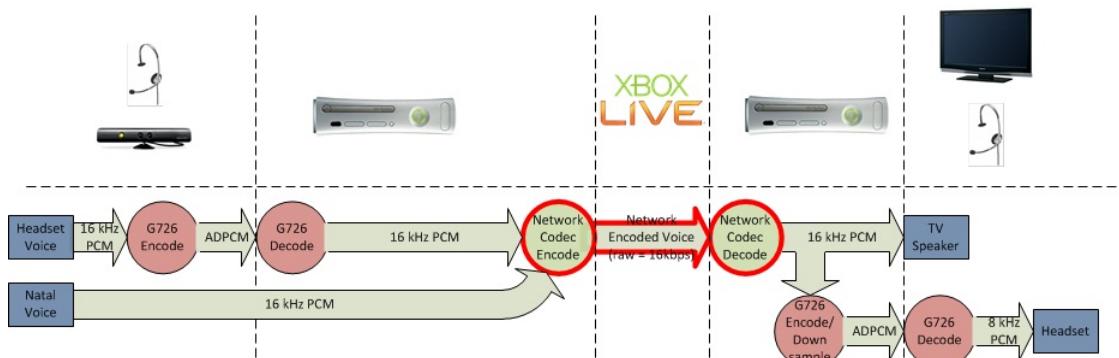
The voice chat pipeline involves several data format conversions, many of which are lossy and color the quality of voice chat data. This is one of the core reasons for the voice chat codec upgrade, which allows for higher-fidelity audio data to be maintained through transmission and reception for final playback to another player's headset or speakers.

Figure 1. Pre-Fall 2010 System Update voice chat pipeline



Headset voice is downsampled and transmitted at 8 kilohertz (kHz).

Figure 2. Post-Fall 2010 System Update voice chat pipeline



Audio input from both headset and Kinect sensor array retain their 16-kHz sampling rates throughout the pipeline, and use a higher quality network codec.

Below are three .wav files, examples of how the voice chat codecs affect voice data. Example 1, at left, is the "clean" source data as spoken by a player into his or her local headset. Example 2, in the center, is the voice data as it might sound when transmitted over Xbox LIVE using the old network codec and pipeline, and rendered through a remote player's speakers. Example 3, at right, is the voice data as it might sound when transmitted over Xbox LIVE and rendered through a remote player's speakers in the Fall 2010 System Update pipeline.

Example 1. Source data as spoken into a local headset



[Click to listen to the example.](#)

Example 2. Voice data transmitted over Xbox LIVE and rendered using a remote player's speakers



[Click to listen to the example.](#)

Example 3. Voice data transmitted over Xbox LIVE and rendered using a remote player's speakers in the Fall 2010 System Update pipeline



[Click to listen to the example.](#)

Voice Chat Codec Update and Title Implications

The changes related to the voice chat codec focus on the network codec. The new codec runs at a 16-kHz sampling rate, which alone provides for substantial improvements over the multiple layers of downsampling and upsampling offered by the prior codec.

Table 1. Comparison of voice chats before and after the Fall 2010 System Update voice chat pipeline

	Prior to June 2010 XDK (development) and Fall 2010 System Update (retail)	Post-Fall 2010 System Update	Comments
Sampling rate	8 kHz	16 kHz	
Bit rate	3.2 kilobits per second (Kbps)	16 Kbps	Encoded voice data bit rate
Private/In-Game chat bit rate (per channel)	7.5 Kbps	20.5 Kbps	Reflects VDP packet overhead (4.3 Kbps for 100 milliseconds (ms) of voice)
Party chat bit rate (per channel)	12 Kbps	24.8 Kbps	Reflects VDP overhead plus Party management (4.5 Kbps for 100 ms of voice)
Party chat bit rate (cap)	36 Kbps	74.4 Kbps	Three voice cap; mesh ensures all participants (even more than three) receive voice
CPU % for Encode	0.716 %	0.167 %	
CPU % for Decode	0.336 %	0.170 %	
CPU % Total	1.060 %	0.336 %	

The most significant change shown in this table is the bandwidth requirement increase. While the updated voice chat codec does require increased bandwidth, as a system-available feature (party chat and private chat), voice chat has not been counted against certification requirements relating to bandwidth restrictions. Bandwidth-related TCRs (TCR 096 for peer to peer, and TCR 097 for LIVE) remain unchanged and unaffected by bandwidth differences between the old and new voice chat bit rates.

Table 2. Upstream bandwidth comparisons for in-game chat between previous (7.5 Kbps) and Fall 2010 System Update voice chat pipelines (20.5 Kbps)

	Before Fall 2010 System Update		Fall 2010 System Update	
	Talker(s)		Talker(s)	
Listener(s)	1	2+	1	2+
1 (2-way)	7.5 Kbps	15 Kbps	20.5 Kbps	41 Kbps
2 (3-way)	15 Kbps	30 Kbps	41.0 Kbps	82 Kbps
3 (4-way)	22.5 Kbps	45 Kbps	61.5 Kbps	123 Kbps

As a feature of the system update, private and party chat transparently use the new codec, even when chat occurs while playing legacy titles built and released before the system update. In-game chat will continue to use the codec that accompanied a given XDK's version of the XHV2 voice chat library; versions of XHV2 prior to the June 2010 XDK will continue to use the old codec.

Wrap Up

Voice chat is a key connection between players on Xbox LIVE, allowing a user to directly communicate with friends and adversaries alike. Providing enhancements to this channel will allow for immersive uses of chat in both cooperative and competitive gameplay settings. Fidelity improvements also allow titles to consider blending additional elements into the headset mix without sacrificing intelligibility of the voice channel. We look forward to the unique solutions that you implement in this area, and we encourage you to contact gameds@microsoft.com with any comments or questions about this white paper or the voice chat technology in the XHV2 library and the XDK.

Version: 21256.3